

This International Student Edition is for use outside of the U.S.

# Applied Numerical Methods *with* PYTHON

---

*for Engineers and Scientists*



Mc  
Graw  
Hill

Steven C. Chapra  
David E. Clough

---

# Applied Numerical Methods

*with Python for Engineers and Scientists*

***Steven C. Chapra***

Tufts University, Professor Emeritus

***David E. Clough***

University of Colorado, Boulder, Professor Emeritus





## APPLIED NUMERICAL METHODS WITH PYTHON FOR ENGINEERS AND SCIENTISTS

Published by McGraw Hill LLC, 1325 Avenue of the Americas, New York, NY 10121. Copyright ©2022 by McGraw Hill LLC. All rights reserved. Printed in the United States of America. No part of this publication may be reproduced or distributed in any form or by any means, or stored in a database or retrieval system, without the prior written consent of McGraw Hill LLC, including, but not limited to, in any network or other electronic storage or transmission, or broadcast for distance learning.

Some ancillaries, including electronic and print components, may not be available to customers outside the United States.

This book is printed on acid-free paper.

1 2 3 4 5 6 7 8 9 LWI 26 25 24 23 22 21

ISBN 978-1-265-01796-5

MHID 1-265-01796-4

Cover Image: *VisualCommunications/E+/Getty Images*

All credits appearing on page or at the end of the book are considered to be an extension of the copyright page.

The Internet addresses listed in the text were accurate at the time of publication. The inclusion of a website does not indicate an endorsement by the authors or McGraw Hill LLC, and McGraw Hill LLC does not guarantee the accuracy of the information presented at these sites.

 [mheducation.com/highered](https://mheducation.com/highered)

## *Dedication:*

Steven C. Chapra

To Cynthia K. Chapra

in gratitude for her love, support,

and for allowing this nerd the time and space to play with his computer.

David E. Clough

To my parents, John and Eunice Clough, for providing a loving, supportive home and a launching pad for my career as an educator.



## ABOUT THE AUTHORS

**Steve Chapra** is the Emeritus Professor and Louis Berger Chair in the Civil and Environmental Engineering Department at Tufts University. His other books include *Surface Water-Quality Modeling*, *Numerical Methods for Engineers*, and *Applied Numerical Methods with MATLAB*.

Dr. Chapra received engineering degrees from Manhattan College and the University of Michigan. Before joining Tufts, he worked for the U.S. Environmental Protection Agency and the National Oceanic and Atmospheric Administration and taught at Texas A&M University, the University of Colorado, and Imperial College London. His general research interests focus on surface water-quality modeling and advanced computer applications in environmental engineering.

He is a Fellow and Life Member of the American Society of Civil Engineering (ASCE) and has received several awards for his scholarly and academic contributions, including the Rudolph Hering Medal (ASCE) for his research, and the Meriam-Wiley Distinguished Author Award (American Society for Engineering Education). He has also been recognized as an outstanding teacher and advisor among the engineering faculties at Texas A&M University, the University of Colorado, and Tufts University. As a strong proponent of continuing education, he has taught over 90 workshops for professionals on numerical methods, computer programming, and environmental modeling.

**David Clough** joined the faculty of the Department of Chemical and Biological Engineering at the University of Colorado in 1975 after a brief career with DuPont in Wilmington, Delaware. He retired from Colorado in 2017 and holds the position of Professor Emeritus. He remains active at Colorado by assisting faculty and students in teaching and research. Notably, he teaches a series of workshops on process modeling and computer simulation as part of the senior design course sequence.

Dr. Clough received degrees in chemical engineering from Case Western Reserve University and the University of Colorado. He has extensive experience in applied computing, process automation, and the modeling of various processes with emphasis on dynamic behavior, including polymerization, high-temperature catalytic reactors, fluidized beds, open-channel flow, biomedical instrumentation, and solar-thermal reactors.

Clough first learned to program in the original Fortran language while in high school in the early 1960s. Since then, he has experience in a wide array of programming languages and computing tools and has applied his expertise through his teaching, research, and industrial applications.

Over his career, Clough has taught hundreds of short courses to practicing professionals on applied computing and problem-solving. Spanning three decades, his courses on spreadsheet problem-solving, offered through the American Institute of Chemical Engineers, have been among the most popular offered by AIChE. During Steve Chapra's years at Colorado, they collaborated to develop the introduction to engineering computing course that has now been taught to thousands of first-year engineering students.

# CONTENTS

About the Authors  iv

Preface  xi

## PART ONE Modeling, Computers, and Error Analysis 1

1.1 Motivation  1

1.2 Part Organization  2

## CHAPTER 1

### Mathematical Modeling, Numerical Methods, and Problem Solving 3

1.1 A Simple Mathematical Model  4

1.2 Conservation Laws in Engineering and Science  9

1.3 Numerical Methods Covered in This Book  10

1.4 **Case Study:** It's a Real Drag  13

Problems  15

## CHAPTER 2

### Python Fundamentals 20

2.1 The Spyder/IPython Environment  21

2.2 Assignment  22

2.3 Mathematical Operations  28

2.4 Use of Built-In Functions  32

2.5 Graphics  35

2.6 Other Resources  40

2.7 **Case Study:** Exploratory Data Analysis  41

Problems  44

## CHAPTER 3

### Programming in Python 48

3.1 Python Script Files  49

- 3.2 Input–Output  **53**
- 3.3 Structured Programming  **56**
- 3.4 Nesting and Indentation  **66**
- 3.5 Python Functions with Function Name Arguments  
 **68**
- 3.6 **Case Study:** Bungee Jumper Velocity  **74**  
Problems  **77**

---

## CHAPTER 4

### Roundoff and Truncation Errors **84**

- 4.1 Errors  **84**
- 4.2 Roundoff Errors  **90**
- 4.3 Truncation Errors  **97**
- 4.4 Total Numerical Error  **107**
- 4.5 Blunders, Model Errors, and Data Uncertainty  **111**  
Problems  **112**

---

Page vi

---

## PART TWO      Roots and Optimization **115**

- 2.1      Overview**  **115**
- 2.2      Part Organization**  **116**

---

## CHAPTER 5

### Roots: Bracketing Methods **117**

- 5.1 Roots in Engineering and Science  **118**
- 5.2 Graphical and Trial-And-Error Methods  **118**
- 5.3 Bracketing Methods and Initial Guesses  **122**
- 5.4 Bisection  **125**
- 5.5 False Position  **131**
- 5.6 **Case Study:** Greenhouse Gases and Rainwater  
 **133**  
Problems  **136**

---

## CHAPTER 6

### Roots: Open Methods **141**

- 6.1 Fixed-Point Iteration  **142**

- 6.2 The Wegstein Method  146
- 6.3 Newton-Raphson  149
- 6.4 Secant Methods  155
- 6.5 Brent's Method  156
- 6.6 Python SciPy Function: `brentq`  160
- 6.7 Polynomials  161
- 6.8 **Case Study:** Pipe Friction  164
- Problems  168

---

## CHAPTER 7

### Optimization 175

- 7.1 Introduction and Background  176
- 7.2 One-Dimensional Optimization  178
- 7.3 Multidimensional Optimization  187
- 7.4 **Case Study:** Equilibrium and Minimum Potential Energy  
 189
- Problems  191

---

## PART THREE Linear Systems 197

- 3.1 Overview  197
- 3.2 Part Organization  199

---

## CHAPTER 8

### Linear Algebraic Equations and Matrices 200

- 8.1 Matrix Algebra Overview  201
- 8.2 Solving Linear Algebraic Equations with Python  211
- 8.3 **Case Study:** Currents and Voltages in Circuits  212
- Problems  216

---

## CHAPTER 9

### Gauss Elimination 219

- 9.1 Solving Small Numbers of Equations  219
- 9.2 Naive Gauss Elimination  224
- 9.3 Pivoting  230
- 9.4 Tridiagonal Systems  233

- 9.5 **Case Study:** Model of a Heated Rod  **235**  
Problems  **238**

## CHAPTER 10

---

### **LU Factorization** **242**

- 10.1 Overview of *LU* Factorization  **242**  
10.2 Gauss Elimination as *LU* Factorization  **244**  
10.3 Cholesky Factorization  **249**  
10.4 The Python `np.linalg.solve` Function  **252**  
Problems  **252**

## CHAPTER 11

---

### **Matrix Inverse and Condition** **253**

- 11.1 The Matrix Inverse  **253**  
11.2 Error Analysis and System Condition  **256**  
11.3 **Case Study:** Indoor Air Pollution  **260**  
Problems  **263**

## CHAPTER 12

---

### **Iterative Methods** **266**

- 12.1 Linear Systems: Gauss-Seidel  **266**  
12.2 Nonlinear Systems  **271**  
12.3 **Case Study:** Chemical Reactions  **278**  
Problems  **280**

## CHAPTER 13

---

### **Eigenvalues** **283**

- 13.1 Eigenvalues and Eigenvectors—The Basics  **284**  
13.2 Applications of Eigenvalues and Eigenvectors  **286**  
13.3 Physical Settings—Mass-Spring Systems  **291**  
13.4 The Power Method  **293**  
13.5 Python Numpy Functions: `eig` and `eigvals`  **295**  
13.6 **Case Study:** Eigenvalues and Earthquakes  **296**  
Problems  **300**



**PART FOUR**    Curve Fitting  **303**

**4.1 Overview**  **303**

**4.2 Part Organization**  **304**

---

**CHAPTER 14**

**Straight-Line Linear Regression**  **306**

14.1 Statistics Review  **307**

14.2 Random Numbers and Simulation  **316**

14.3 Straight-Line Least Squares Regression  **320**

14.4 Linearization of Nonlinear Relationships  **330**

14.5 Computer Applications  **335**

14.6 **Case Study:** Enzyme Kinetics  **339**

Problems  **342**

---

Page viii

---

---

**CHAPTER 15**

**General Linear and Nonlinear Regression**  **349**

15.1 Polynomial Regression  **349**

15.2 Multiple Linear Regression  **353**

15.3 General Linear Least-Squares Regression  **355**

15.4 Model Building and Selection in Regression  **359**

15.5 Nonlinear Regression  **364**

15.6 **Case Study:** Fitting Experimental Data  **368**

Problems  **371**

---

**CHAPTER 16**

**Fourier Analysis**  **375**

16.1 Curve Fitting with Sinusoidal Functions  **376**

16.2 Continuous Fourier Series  **381**

16.3 Frequency and Time Domains  **383**

16.4 Fourier Integral and Transform  **384**

16.5 Discrete Fourier Transform (DFT)  **386**

16.6 The Power Spectrum  **390**

16.7 **Case Study:** Sunspots  **391**

Problems  **393**

## CHAPTER 17

---

### Polynomial Interpolation 396

- 17.1 Introduction to Interpolation  397
- 17.2 Newton Interpolating Polynomial  399
- 17.3 Lagrange Interpolating Polynomial  405
- 17.4 Inverse Interpolation  408
- 17.5 Extrapolation and Oscillations  409
- Problems  414

## CHAPTER 18

---

### Splines and Piecewise Interpolation 418

- 18.1 Introduction to Splines  418
- 18.2 Linear Splines  420
- 18.3 Quadratic Splines  425
- 18.4 Cubic Splines  428
- 18.5 Piecewise Interpolation in Python  432
- 18.6 Multidimensional Interpolation  437
- 18.7 Smoothing of Data Series  439
- 18.8 **Case Study:** Heat Transfer in a Lake  447
- Problems  450

---

Page ix

## PART FIVE Integration and Differentiation 455

- 5.1 Overview  455
- 5.2 Part Organization  456

## CHAPTER 19

---

### Numerical Integration Formulas 457

- 19.1 Introduction and Background  458
- 19.2 Newton-Cotes Formulas  461
- 19.3 The Trapezoidal Rule  462
- 19.4 Simpson's Rules  467
- 19.5 Higher-Order Newton-Cotes Formulas  472
- 19.6 Integration with Unequal Segments  473
- 19.7 Open Methods  477
- 19.8 Multiple Integrals  477

19.9 **Case Study:** Computing Work with Numerical Integration

 **479**

Problems  **482**

---

## CHAPTER 20

### Numerical Integration of Functions **486**

20.1 Introduction  **486**

20.2 Romberg Integration  **487**

20.3 Gauss Quadrature  **491**

20.4 Adaptive Quadrature  **496**

20.5 **Case Study:** Root-Mean-Square Current  **499**

Problems  **502**

---

## CHAPTER 21

### Numerical Differentiation **506**

21.1 Introduction and Background  **507**

21.2 High-Accuracy Differentiation Formulas  **509**

21.3 Richardson Extrapolation  **512**

21.4 Derivatives of Unequally Spaced Data  **513**

21.5 Derivatives and Integrals for Data with Errors  **514**

21.6 Partial Derivatives  **515**

21.7 Numerical Differentiation with Python  **516**

21.8 **Case Study:** Visualizing Fields  **520**

Problems  **522**

---

## PART SIX

### Ordinary Differential Equations **527**

**6.1 Overview**  **527**

**6.2 Part Organization**  **530**

---

## CHAPTER 22

### Initial-Value Problems **532**

22.1 Overview  **533**

22.2 Euler's Method  **534**

22.3 Improvements of Euler's Method  **539**

|      |                                                                                       |                                                                                       |
|------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| 22.4 | Runge-Kutta Methods                                                                   |  543 |
| 22.5 | Systems of Equations                                                                  |  547 |
| 22.6 | <b>Case Study:</b> Predator-Prey Models and Chaos                                     |                                                                                       |
|      |  552 |                                                                                       |
|      | Problems                                                                              |  556 |

## CHAPTER 23

---

|                                           |                                                                                       |                                                                                         |
|-------------------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>Adaptive Methods and Stiff Systems</b> |                                                                                       |  561 |
| 23.1                                      | Adaptive Runge-Kutta Methods                                                          |  561 |
| 23.2                                      | Multistep Methods                                                                     |  572   |
| 23.3                                      | Stiffness                                                                             |  575   |
| 23.4                                      | Python Application: Bungee Jumper with Cord                                           |                                                                                         |
|                                           |  580 |                                                                                         |
| 23.5                                      | <b>Case Study:</b> Pliny's Intermittent Fountain                                      |  582 |
|                                           | Problems                                                                              |  585   |

## CHAPTER 24

---

|                                |                                         |                                                                                           |
|--------------------------------|-----------------------------------------|-------------------------------------------------------------------------------------------|
| <b>Boundary-Value Problems</b> |                                         |  591   |
| 24.1                           | Introduction and Background             |  592 |
| 24.2                           | The Shooting Method                     |  595   |
| 24.3                           | Finite-Difference Methods               |  601 |
| 24.4                           | Python Function: <code>solve_bvp</code> |  607 |
|                                | Problems                                |  609   |

|                     |                                                                                         |                                                                                           |
|---------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>APPENDIX A:</b>  | <b>MATPLOTLIB</b>                                                                       |  614  |
| <b>APPENDIX B:</b>  | <b>CUBIC SPLINE SMOOTHING</b>                                                           |  635 |
| <b>APPENDIX C:</b>  | <b>PYTHON BUILT-IN KEYWORDS:<br/>FUNCTIONS, METHODS, OPERATORS,<br/>TYPES</b>           |  639   |
| <b>APPENDIX D:</b>  | <b>PYTHON FUNCTIONS AND SCRIPTS<br/>PRESENTED IN THE TEXT</b>                           |  641 |
| <b>BIBLIOGRAPHY</b> |  643 |                                                                                           |
| <b>INDEX</b>        |  645 |                                                                                           |



# PREFACE

When we first learned to use computers as students in the 1960s, Fortran was the language of choice for engineering and scientific computations. Over the ensuing half century, numerous other languages have proven useful for implementing the numerical calculations that are so valuable to our research and teaching. Along with a succession of improved Fortran versions, other languages such as Algol, Basic, Pascal, and C/C++ have all found their way into our computational toolbox.

One disadvantage of this evolution has been that most of these languages lacked fully integrated numerical libraries to allow programmers to tap the vast arsenal of “industrial strength” algorithms necessary for most engineering and scientific applications.

In 1984, MathWorks, Inc., remedied this shortcoming by introducing a fully integrated, multi-paradigm numerical computing environment and high-level programming language called MATLAB<sup>®</sup> (or MATrix LABoratory). Along with procedural programming, MATLAB allows plotting, creation of user interfaces, and interfacing with other languages. But most importantly, MATLAB and its supplementary toolboxes have a vast library of built-in functions and toolboxes that empower the programmer to apply state-of-the-art numerical methods without having to develop code from the ground up.

Although MATLAB provides a high-quality and powerful computing environment, it has the disadvantage of being relatively expensive. Although this is usually a negligible consideration for large organizations such as many universities, we have observed that smaller consulting firms, local governments, individuals, and even some larger entities like corporations can consider the expense prohibitive. Hence, a need exists for a lower-cost alternative.

Enter Python<sup>®</sup>. Created by Guido van Rossum and first released in 1991, Python is a multi-paradigm, open-source computing environment that has ready access to powerful numerical routines yet is free to use by any individual or organization. In addition, it is well managed and maintained, as is not always the case with “free,” open-source software. Hence, it is becoming an increasingly popular alternative to MATLAB.

Because of its growing use in engineering and science education, we decided to write this textbook to support a one-semester course in numerical methods. It has been written for students who want to learn and apply numerical methods in order to solve problems in engineering and science. As such, the methods are motivated by problems rather than by mathematics. That said, enough theory is provided so that students come away with insight into the techniques and their shortcomings.

Python provides a great environment for such a course as it affords a nice combination of a high-level programming language with access to powerful built-in numerical capabilities. Consequently, it allows students to implement moderately complicated algorithms in a structured and coherent style while its numerical capabilities empower students to solve more difficult problems without having to “reinvent the wheel.”

The basic content, organization, and pedagogy of this book are like our other numerical methods textbooks. In particular, a conversational writing style is intentionally maintained in order to make the book easier to read. This book tries to speak directly to the reader and is designed in part to be a tool

for self-teaching. As such, we also believe it will have value outside the classroom for professionals desiring to gain proficiency in both numerical methods and Python.

We have endeavored to maintain features contributing to its pedagogical effectiveness including extensive use of worked examples and engineering and scientific applications. Most importantly, we have made a concerted effort to make this book as “student-friendly” as possible. Thus, we’ve tried to keep our explanations straightforward and practical.

We clarify here that this is not a text whose primary purpose is to teach the Python programming language in depth, and yet no prior experience with Python is required. We provide enough background in Python programming to facilitate the implementation of numerical methods. We have endeavored to use an inductive approach where students will learn aspects of Python “just in time” through application and gradually be able to generalize those experiences to a broader-based familiarity with the language. There are many examples of Python code provided that will bring the students along and provide patterns for their own code development. Purposefully, we chose the Spyder integrated development environment, as it provides students with a relatively friendly interface, complete with many features like MATLAB. These include a command window, editor, variable explorer, debugging tools, and a useful help interface. Programmers who are expert in Python may be disappointed at some features we have left out, but we claim to keep in focus the primary purpose of this text, empowering STEM students by learning numerical methods.

Although our primary intent is to empower students by providing them with a sound introduction to numerical problem-solving, we have the ancillary objective of making this introduction exciting and pleasurable. We believe that motivated students who enjoy engineering and science, problem-solving, mathematics—and yes—programming will ultimately make better professionals. If our book fosters enthusiasm and appreciation for these subjects, we will consider the effort a success.

## ADDITIONAL FEATURES

### Proctorio

### Remote Proctoring & Browser-Locking Capabilities



Remote proctoring and browser-locking capabilities, hosted by Proctorio within Connect, provide control of the assessment environment by enabling security options and verifying the identity of the student.

Seamlessly integrated within Connect, these services allow instructors to control students’ assessment experience by restricting browser activity, recording students’ activity, and verifying students are doing their own work.

Instant and detailed reporting gives instructors an at-a-glance view of potential academic integrity concerns, thereby avoiding personal bias and supporting evidence-based claims.



## ReadAnywhere



Read or study when it's convenient for you with McGraw Hill's free ReadAnywhere app. Available for iOS or Android smartphones or tablets, ReadAnywhere gives users access to McGraw Hill tools including the eBook and SmartBook 2.0 or Adaptive Learning Assignments in Connect. Take notes, highlight, and complete assignments offline—all of your work will sync when you open the app with WiFi access. Log in with your McGraw Hill Connect username and password to start learning—anytime, anywhere!

## Tegrity: Lectures 24/7

Tegrity in Connect is a tool that makes class time available 24/7 by automatically capturing every lecture. With a simple one-click start-and-stop process, you capture all computer screens and corresponding audio in a format that is easy to search, frame by frame. Students can replay any part of any class with easy-to-use, browser-based viewing on a PC, Mac, iPod, or other mobile device.

Educators know that the more students can see, hear, and experience class resources, the better they learn. In fact, studies prove it. Tegrity's unique search feature helps students efficiently find what they need, when they need it, across an entire semester of class recordings. Help turn your students' study time into learning moments immediately supported by your lecture. With Tegrity, you also increase intent listening and class participation by easing students' concerns about note-taking. Using Tegrity in Connect will make it more likely you will see students' faces, not the tops of their heads.

## Test Builder in Connect

Available within Connect, Test Builder is a cloud-based tool that enables instructors to format tests that can be printed, administered within a Learning Management System, or exported as a Word document of the test bank. Test Builder offers a modern, streamlined interface for easy content configuration that matches course needs, without requiring a download.

Test Builder allows you to:

- access all test bank content from a particular title.
- easily pinpoint the most relevant content through robust filtering options.
- manipulate the order of questions or scramble questions and/or answers.
- pin questions to a specific location within a test.
- determine your preferred treatment of algorithmic questions.
- choose the layout and spacing.
- add instructions and configure default settings.

---

Page xiii

Test Builder provides a secure interface for better protection of content and allows for just-in-time updates to flow directly into assessments.

## Writing Assignment

Available within Connect and Connect Master, the Writing Assignment tool delivers a learning experience to help students improve their written communication skills and conceptual understanding. As an instructor you can assign, monitor, grade, and provide feedback on writing more efficiently and effectively.

## Create Your Book, Your Way

McGraw Hill's Content Collections Powered by Create<sup>®</sup> is a self-service website that enables instructors to create custom course materials—print and eBooks—by drawing upon McGraw Hill's comprehensive, cross-disciplinary content. Choose what you want from our high-quality textbooks, articles, and cases. Combine it with your own content quickly and easily, and tap into other rights-secured, third-party content such as readings, cases, and articles. Content can be arranged in a way that makes the most sense for your course and you can include the course name and information as well. Choose the best format for your course: color print, black-and-white print, or eBook. The eBook can be included in your Connect course and is available on the free ReadAnywhere app for smartphone or tablet access as well. When you are finished customizing, you will receive a free digital copy to review in just minutes! Visit McGraw Hill Create<sup>®</sup>—[www.mcgrawhillcreate.com](http://www.mcgrawhillcreate.com)—today and begin building!

---

Page xv

---

## ACKNOWLEDGMENTS

Several members of the McGraw Hill team have contributed to this project. Special thanks are due to Heather Ervolino (Product Developer), Beth Bettcher (Portfolio Manager), Lisa Granger (Marketing Manager), and Jeni McAtee (Content Project Manager) for their encouragement, support, and direction.

Several of our colleagues, notably Greg Coyle and Jon Lamontagne of Tufts, and Scott Rowe of the University of Colorado, have been very supportive and made helpful suggestions. Several years ago, Christine Shoemaker of Cornell University first suggested we develop a Python version of our numerical methods text and now here it is.

It should be stressed that although we received useful advice from the aforementioned individuals, we are responsible for any inaccuracies or mistakes you may find in this book. Please contact us via e-mail if you should detect any errors.

Finally, we want to thank our families for the love, patience, and support they have provided through the time we've spent on this project.

Steven C. Chapra  
Tufts University  
Medford, Massachusetts  
[steven.chapra@tufts.edu](mailto:steven.chapra@tufts.edu)

David E. Clough  
University of Colorado  
Boulder, Colorado  
[david.clough@colorado.edu](mailto:david.clough@colorado.edu)



## Instructors: Student Success Starts with You

### Tools to enhance your unique voice

Want to build your own course? No problem. Prefer to use an OLC-aligned, prebuilt course? Easy. Want to make changes throughout the semester? Sure. And you'll save time with Connect's auto-grading too.

**65%**  
Less Time  
Grading

---



Laptop: McGraw Hill; Woman/child: George Doyle/Getty Images

### Study made personal

Incorporate adaptive study resources like SmartBook® 2.0 into your course and help your students be better prepared in less time. Learn more about the powerful personalized learning experience available in SmartBook 2.0 at [www.mheducation.com/highered/connect/smartbook](http://www.mheducation.com/highered/connect/smartbook)

### Affordable solutions, added value



Make technology work for you with LMS integration for single sign-on access, mobile access to the digital textbook, and reports to quickly show you how each of your students is doing. And with our Inclusive Access program you can provide all these tools at a discount to your students. Ask your McGraw Hill representative for more information.

Feedback: Johnna/Getty Images

### Solutions for your challenges



A product isn't a solution. Real solutions are affordable, reliable, and come with training and ongoing support when you need it and how you want it. Visit [www.supportateverystep.com](http://www.supportateverystep.com) for videos and resources both you and your students can use throughout the semester.

Classroom: Johnna/Getty Images



## Students: Get Learning that Fits You

### Effective tools for efficient studying

Connect is designed to help you be more productive with simple, flexible, intuitive tools that maximize your study time and meet your individual learning needs. Get learning that works for you with Connect.

### Study anytime, anywhere

Download the free ReadAnywhere app and access your online eBook, SmartBook 2.0, or Adaptive Learning Assignments when it's convenient, even if you're offline. And since the app automatically syncs with your Connect account, all of your work is available every time you open it. Find out more at [www.mheducation.com/readanywhere](http://www.mheducation.com/readanywhere)

*"I really liked this app—it made it easy to study when you don't have your textbook in front of you."*

- Jordan Cunningham,  
Eastern Washington University



Calendar: owestphotos/Getty Images

### Everything you need in one place

Your Connect course has everything you need—whether reading on your digital eBook or completing assignments for class, Connect makes it easy to get your work done.

### Learning for everyone

McGraw Hill works directly with Accessibility Services Departments and faculty to meet the learning needs of all students. Please contact your Accessibility Services Office and ask them to email [accessibility@mheducation.com](mailto:accessibility@mheducation.com), or visit [www.mheducation.com/about/accessibility](http://www.mheducation.com/about/accessibility) for more information.

Top: Jenner Images/Getty Images, Left: Hero Images/Getty Images, Right: Hero Images/Getty Images



## Modeling, Computers, and Error Analysis

### 1.1 MOTIVATION

What are numerical methods and why should you study them?

*Numerical methods* are techniques by which mathematical problems are formulated so that they can be solved with arithmetic and logical operations. Because digital computers excel at performing such operations, numerical methods are sometimes referred to as *computer mathematics*.

In the pre-computer era, the time and drudgery of implementing such calculations seriously limited their practical use. However, with the advent of fast, inexpensive digital computers, the role of numerical methods in engineering and scientific problem solving has exploded. Because they figure so prominently in much of our work, we believe that numerical methods should be a part of every engineer's and scientist's basic education. Just as we all must have solid foundations in the other areas of mathematics and science, we should also have a fundamental understanding of numerical methods. In particular, we should have a solid appreciation of both their capabilities and their limitations.

Beyond contributing to your overall education, there are several additional reasons why you should study numerical methods:

1. Numerical methods greatly expand the types of problems you can address. They are capable of handling large systems of equations, nonlinearities, and complicated geometries that are not uncommon in engineering and science and that are often impossible to solve analytically with standard calculus. As such, they greatly enhance your problem-solving skills.
2. Numerical methods allow you to use "canned" software with insight. During your career, you will invariably have occasion to use commercially available prepackaged computer programs that involve numerical methods. The intelligent use of these programs is greatly enhanced by an understanding of the basic theory underlying the methods. In the absence of such understanding, you will be left to treat such packages as "black boxes" with little critical insight into their inner workings or the validity of the results they produce.
3. Many problems cannot be approached using canned programs. If you are conversant with numerical methods and are adept at computer programming, you can design your own programs to solve problems without having to buy or commission expensive software.
4. Numerical methods are an efficient vehicle for learning to use computers. Because numerical methods are expressly designed for computer implementation, they are ideal for illustrating the computer's powers and limitations. When you successfully implement numerical methods on a computer and then

apply them to solve otherwise intractable problems, you will be provided with a dramatic demonstration of how computers can serve your professional development. At the same time, you will also learn to acknowledge and control the errors of approximation that are part and parcel of large-scale numerical calculations.

5. Numerical methods provide a vehicle for you to reinforce your understanding of mathematics. Because one function of numerical methods is to reduce higher mathematics to basic arithmetic operations, they get at the “nuts and bolts” of some otherwise obscure topics. Enhanced understanding and insight can result from this alternative perspective.



With these reasons as motivation, we can now set out to understand how numerical methods and digital computers work in tandem to generate reliable solutions to mathematical problems. The remainder of this book is devoted to this task.

## 1.2 PART ORGANIZATION

---

This book is divided into six parts. The latter five parts focus on the major areas of numerical methods. Although it might be tempting to jump right into this material, *Part One* consists of four chapters dealing with essential background material.

 *Chapter 1* provides a concrete example of how a numerical method can be employed to solve a real problem. To do this, we develop a *mathematical model* of a free-falling bungee jumper. The model, which is based on Newton's second law, results in an ordinary differential equation. After first using calculus to develop a closed-form solution, we then show how a comparable solution can be generated with a simple numerical method. We end the chapter with an overview of the major areas of numerical methods that we cover in  *Parts Two* through  *Six*.

 *Chapters 2* and  *3* introduce the Python® computer language in the Spyder/Python development environment.  *Chapter 2* introduces the Python language via the IPython command console and the Spyder editor. The *IPython command console* provides a straightforward means to orient you to the environment and illustrates how it is used for common operations such as performing calculations and creating plots. The *Spyder editor* is the interface you will use later to create longer program scripts that are easy to modify and save.

 *Chapter 3* focuses on the use of the Spyder editor and introduces the structural elements of the Python language that are essential to implementing the algorithms of numerical methods.

 *Chapter 4* deals with the important topic of error analysis, which must be understood for the effective use of numerical methods. The first part of the chapter focuses on the *roundoff errors* that result because digital computers cannot represent some quantities exactly. The latter part addresses *truncation errors* that arise from using an approximation in place of an exact mathematical procedure.

# Mathematical Modeling, Numerical Methods, and Problem Solving

## CHAPTER OBJECTIVES

The primary objective of this chapter is to provide you with a concrete idea of what numerical methods are and how they relate to engineering and scientific problem solving. Specific objectives and topics covered are

- Learning how mathematical models can be formulated on the basis of scientific principles to simulate the behavior of a simple physical system.
- Understanding how numerical methods afford a means to generate solutions in a manner that can be implemented on a digital computer.
- Understanding the different types of conservation laws that lie beneath the models used in the various engineering disciplines and appreciating the difference between steady-state and dynamic solutions of these models.
- Learning about the different types of numerical methods we will cover in this book.

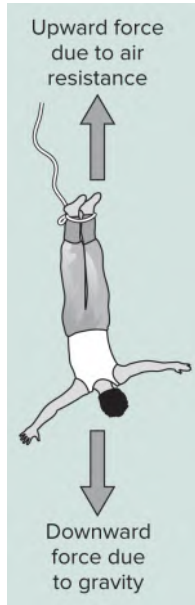
## YOU'VE GOT A PROBLEM

Suppose that a bungee-jumping company hires you. You're given the task of predicting the velocity of a jumper (📎 Fig. 1.1) as a function of time during the free-fall part of the jump. This information will be used as part of a larger analysis to determine the length and required strength of the bungee cord for jumpers of different mass.

---

**FIGURE 1.1** Forces acting on a free-falling bungee jumper.





You know from your studies of physics that the acceleration should be equal to the ratio of the force to the mass (Newton's second law). Based on this insight and your knowledge of physics and fluid mechanics, you develop the following mathematical model for the rate of change of velocity with respect to time,

$$\frac{dv}{dt} = g - \frac{c_d}{m}v^2$$

where  $v$  = downward vertical velocity (m/s),  $t$  = time(s),  $g$  = the acceleration due to gravity ( $\cong 9.81 \text{ m/s}^2$ ),  $c_d$  = a lumped drag coefficient (kg/m), and  $m$  = the jumper's mass (kg). The drag coefficient is called "lumped" because its magnitude depends on factors such as the jumper's area and the fluid density (see [🔗 Sec. 1.4](#)).

Because this is a differential equation, you know that calculus might be used to obtain an analytical or exact solution for  $v$  as a function of  $t$ . However, in the following pages, we will illustrate an alternative solution approach. This will involve developing a computer-oriented numerical or approximate solution.

Aside from showing you how the computer can be used to solve this particular problem, our more general objective will be to illustrate (a) what numerical methods are and (b) how they figure in engineering and scientific problem solving. In so doing, we will also show how mathematical models figure prominently in the way engineers and scientists use numerical methods in their work.

# 1.1 A SIMPLE MATHEMATICAL MODEL

A *mathematical model* can be broadly defined as a formulation or an equation that expresses the essential features of a physical system or process in mathematical terms. In a very general sense, it can be represented as a functional relationship of the form

$$\text{Dependent variable} = f\left(\begin{matrix} \text{independent} \\ \text{variables} \end{matrix}, \begin{matrix} \text{parameters,} \\ \text{forcing} \\ \text{functions} \end{matrix}\right) \quad (1.1)$$

where the *dependent variable* is a characteristic that typically reflects the behavior or state of the system; the *independent variables* are usually dimensions, such as time and space, along which the system's behavior is being determined; the *parameters* are reflective of the system's properties or composition; and the *forcing functions* are external influences acting upon it.

The actual mathematical expression of  **Eq. (1.1)** can range from a simple algebraic relationship to large complicated sets of differential equations. For example, on the basis of his observations, Newton formulated his second law of motion, which states that the time rate of change of momentum of a body is equal to the resultant force acting on it. The mathematical expression, or model, of the second law is the well-known equation

$$F = ma \quad (1.2)$$

where  $F$  is the net force acting on the body (N, or  $\text{kg}\cdot\text{m}/\text{s}^2$ ),  $m$  is the mass of the object (kg), and  $a$  is its acceleration ( $\text{m}/\text{s}^2$ ).

The second law can be recast in the format of  **Eq. (1.1)** by merely dividing both sides by  $m$  to give

$$a = \frac{F}{m} \quad (1.3)$$

where  $a$  is the dependent variable reflecting the system's behavior,  $F$  is the forcing function, and  $m$  is a parameter. Note that for this simple case there is no independent variable because we are not yet predicting how acceleration varies in time or space.

 **Equation (1.3)** has a number of characteristics that are typical of mathematical models of the physical world.

- It describes a natural process or system in mathematical terms.
- It represents an idealization and simplification of reality. That is, the model ignores negligible details of the natural process and focuses on its essential manifestations. Thus, the second law does not

include the effects of relativity that are of minimal importance when applied to objects and forces that interact on or about the earth's surface at velocities and on scales visible to humans.

- Finally, it yields reproducible results and, consequently, can be used for predictive purposes. For example, if the force on an object and its mass are known, [Eq. \(1.3\)](#) can be used to compute acceleration.

Because of its simple algebraic form, the solution of [Eq. \(1.2\)](#) was obtained easily. However, other mathematical models of physical phenomena may be much more complex, and either cannot be solved exactly or require more sophisticated mathematical techniques than simple algebra for their solution. To illustrate a more complex model of this kind, Newton's second law can be used to determine the terminal velocity of a free-falling body near the earth's surface. Our falling body will be a bungee jumper ( [Fig. 1.1](#)). For this case, a model can be derived by expressing the acceleration as the time rate of change of the velocity ( $dv/dt$ ) and substituting it into [Eq. \(1.3\)](#) to yield

$$\frac{dv}{dt} = \frac{F}{m} \quad (1.4)$$

where  $v$  is velocity (in meters per second). Thus, the rate of change of the velocity is equal to the net force acting on the body normalized to its mass. If the net force is positive, the object will accelerate. If it is negative, the object will decelerate. If the net force is zero, the object's velocity will remain at a constant level.

Next, we will express the net force in terms of measurable variables and parameters. For a body falling within the vicinity of the earth, the net force is composed of two opposing forces: the downward pull of gravity  $F_D$  and the upward force of air resistance  $F_U$  ([Fig. 1.1](#)):

---

Page 5

$$F = F_D + F_U \quad (1.5)$$

If force in the downward direction is assigned a positive sign, the second law can be used to formulate the force due to gravity as

$$F_D = mg \quad (1.6)$$

where  $g$  is the acceleration due to gravity ( $9.81 \text{ m/s}^2$ ).

Air resistance can be formulated in a variety of ways. Knowledge from the science of fluid mechanics suggests that a good first approximation would be to assume that it is proportional to the square of the velocity,

$$F_U = -c_d v^2 \quad (1.7)$$

where  $c_d$  is a proportionality constant called the *lumped drag coefficient* (kg/m). Thus, the greater the fall velocity, the greater the upward force due to air resistance. The parameter  $c_d$  accounts for properties of the falling object, such as shape or surface roughness, that affect air resistance. For the present case,  $c_d$  might be a function of the type of clothing or the orientation used by the jumper during free fall.

The net force is the difference between the downward and upward force. Therefore, Eqs. (1.4) through (1.7) can be combined to yield

$$\frac{dv}{dt} = g - \frac{c_d}{m} v^2 \quad (1.8)$$

Equation (1.8) is a model that relates the acceleration of a falling object to the forces acting on it. It is a *differential equation* because it is written in terms of the differential rate of change ( $dv/dt$ ) of the variable that we are interested in predicting. However, in contrast to the solution of Newton's second law in Eq. (1.3), the exact solution of Eq. (1.8) for the velocity of the jumper cannot be obtained using simple algebraic manipulation. Rather, more advanced techniques such as those of calculus must be applied to obtain an exact or analytical solution. For example, if the jumper is initially at rest ( $v = 0$  at  $t = 0$ ), calculus can be used to solve Eq. (1.8) for

$$v(t) = \sqrt{\frac{gm}{c_d}} \tanh\left(\sqrt{\frac{gc_d}{m}} t\right) \quad (1.9)$$

where  $\tanh$  is the hyperbolic tangent that can be computed either directly<sup>1</sup> or via the more elementary exponential function as in

$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (1.10)$$

Note that Eq. (1.9) is cast in the general form of Eq. (1.1), where  $v(t)$  is the dependent variable,  $t$  is the independent variable,  $c_d$  and  $m$  are parameters, and  $g$  is the forcing function.

## EXAMPLE 1.1

### Analytical Solution to the Bungee Jumper Problem

#### Problem Statement

A bungee jumper with a mass of 68.1 kg leaps from a stationary hot air balloon. Use Eq. (1.9) to compute velocity for the first 12 s of free fall. Also determine the terminal velocity that will be

attained for an infinitely long cord (or alternatively, the jumpmaster is having a particularly bad day!). Use a drag coefficient of 0.25 kg/m.

## Solution

Inserting the parameters into [Eq. \(1.9\)](#) yields

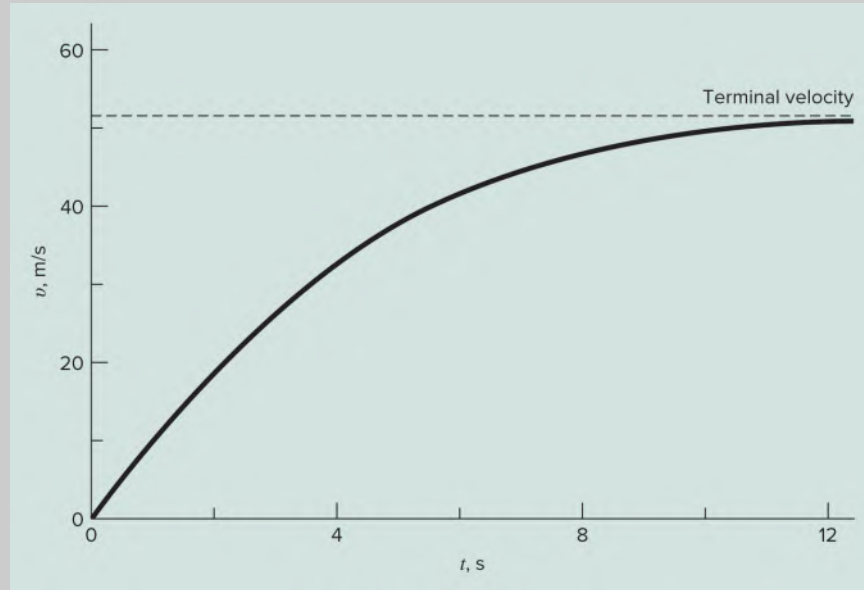
$$v(t) = \sqrt{\frac{9.81(68.1)}{0.25}} \tanh\left(\sqrt{\frac{9.81(0.25)}{68.1}} t\right) = 51.6938 \tanh(0.18977t)$$

which can be used to compute

| $t, \text{ s}$ | $v, \text{ m/s}$ |
|----------------|------------------|
| 0              | 0                |
| 2              | 18.7292          |
| 4              | 33.1118          |
| 6              | 42.0762          |
| 8              | 46.9575          |
| 10             | 49.4214          |
| 12             | 50.6175          |
| $\infty$       | 51.6938          |

According to the model, the jumper accelerates rapidly ([Fig. 1.2](#)). A velocity of 49.4214 m/s (about 110 mi/hr) is attained after 10 s. Note also that after a sufficiently long time, a constant velocity, called the *terminal velocity*, of 51.6983 m/s (115.6 mi/hr) is reached. This velocity is constant because, eventually, the force of gravity will be in balance with the air resistance. Thus, the net force is zero and acceleration has ceased.

**FIGURE 1.2** The analytical solution for the bungee jumper problem as computed in [Example 1.1](#). Velocity increases with time and asymptotically approaches a terminal velocity.



Equation (1.9) is called an *analytical* or *closed form solution* because it exactly satisfies the original differential equation. Unfortunately, there are many mathematical models that cannot be solved exactly. In many of these cases, the only alternative is to develop a numerical solution that approximates the exact solution.

*Numerical methods* are those in which the mathematical problem is reformulated so it can be solved by arithmetic operations. This can be illustrated for Eq. (1.8) by realizing that the time rate of change of velocity can be approximated by (Fig. 1.3):

Page 7

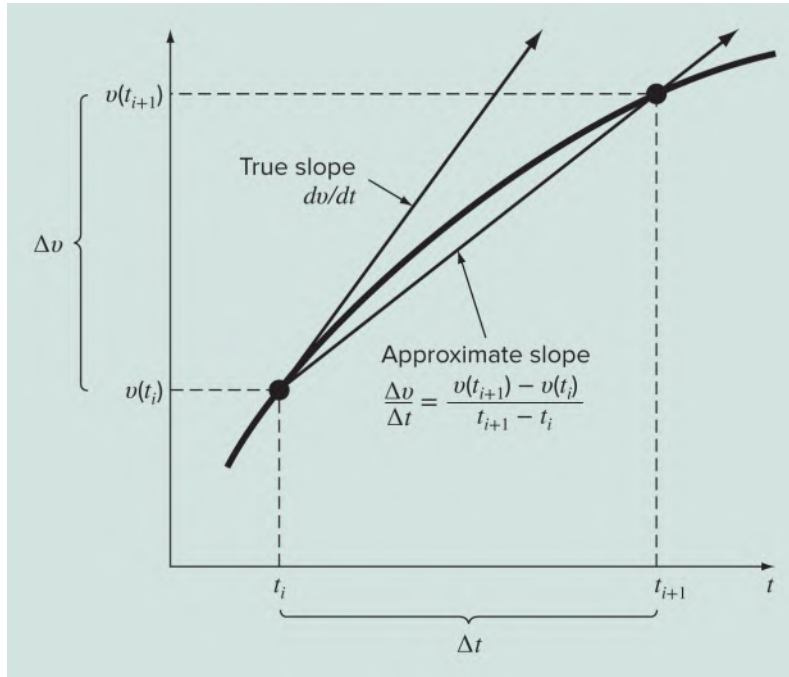
$$\frac{dv}{dt} \cong \frac{\Delta v}{\Delta t} = \frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i} \quad (1.11)$$

where  $\Delta v$  and  $\Delta t$  are differences in velocity and time computed over finite intervals,  $v(t_i)$  is velocity at an initial time  $t_i$ , and  $v(t_{i+1})$  is velocity at some later time  $t_{i+1}$ . Note that  $dv/dt \cong \Delta v/\Delta t$  is approximate because  $\Delta t$  is finite. Remember from calculus that

$$\frac{dv}{dt} = \lim_{\Delta t \rightarrow 0} \frac{\Delta v}{\Delta t}$$

Equation (1.11) represents the reverse process.

**FIGURE 1.3** The use of a finite difference to approximate the first derivative of  $v$  with respect to  $t$ .



 **Equation (1.11)** is called a *finite-difference approximation* of the derivative at time  $t_i$ . It can be substituted into  **Eq. (1.8)** to give

$$\frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i} = g - \frac{c_d}{m} v(t_i)^2$$

This equation can then be rearranged to yield

$$v(t_{i+1}) = v(t_i) + \left[ g - \frac{c_d}{m} v(t_i)^2 \right] (t_{i+1} - t_i) \quad (1.12)$$

Notice that the term in brackets is the right-hand side of the differential equation itself [ **Eq. (1.8)**]. That is, it provides a means to compute the rate of change or slope of  $v$ . Thus, the equation can be rewritten more concisely as

$$v_{i+1} = v_i + \frac{dv_i}{dt} \Delta t \quad (1.13)$$

where the nomenclature  $v_i$  designates velocity at time  $t_i$ , and  $\Delta t = t_{i+1} - t_i$ .

We can now see that the differential equation has been transformed into an equation that can be used to determine the velocity algebraically at  $t_{i+1}$  using the slope and previous values of  $v$  and  $t$ . If you are given an initial value for velocity at some time  $t_i$ , you can easily compute velocity at a later time  $t_{i+1}$ . This new

value of velocity at  $t_{i+1}$  can in turn be employed to extend the computation to velocity at  $t_{i+2}$  and so on. Thus, at any time along the way,

$$\text{New value} = \text{old value} + \text{slope} \times \text{step size}$$

This approach is formally called *Euler's method*. We'll discuss it in more detail when we turn to differential equations later in this book.

## EXAMPLE 1.2

### Numerical Solution to the Bungee Jumper Problem

#### Problem Statement

Perform the same computation as in [Example 1.1](#) but use [Eq. \(1.12\)](#) to compute velocity with Euler's method. Employ a step size of 2 s for the calculation.

#### Solution

At the start of the computation ( $t_0 = 0$ ), the velocity of the jumper is zero. Using this information and the parameter values from [Example 1.1](#), [Eq. \(1.12\)](#) can be used to compute velocity at  $t_1 = 2$  s:

$$v = 0 + \left[ 9.81 - \frac{0.25}{68.1} (0)^2 \right] \times 2 = 19.62 \text{ m/s}$$

For the next interval (from  $t = 2$  to 4 s), the computation is repeated, with the result

$$v = 19.62 + \left[ 9.81 - \frac{0.25}{68.1} (19.62)^2 \right] \times 2 = 36.4137 \text{ m/s}$$

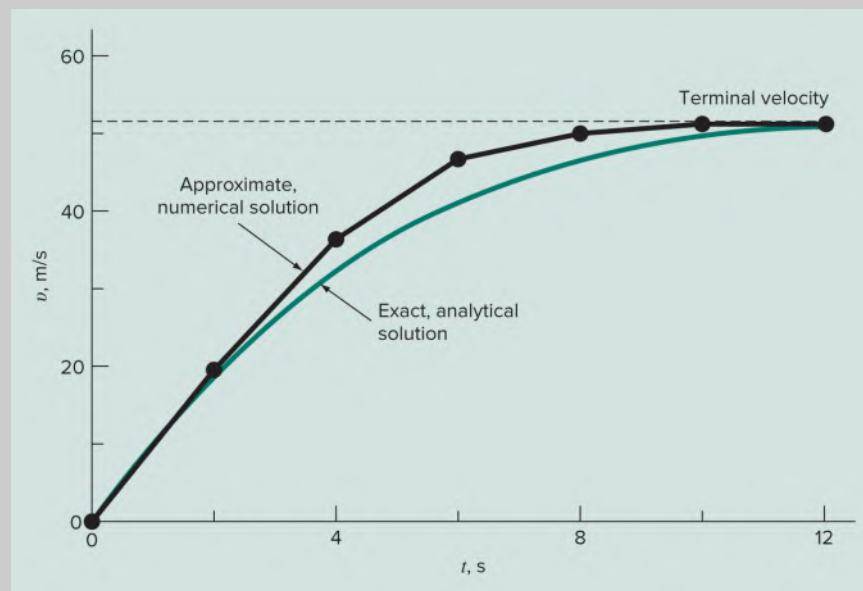
The calculation is continued in a similar fashion to obtain additional values:

| $t, \text{ s}$ | $v, \text{ m/s}$ |
|----------------|------------------|
| 0              | 0                |
| 2              | 19.6200          |
| 4              | 36.4137          |
| 6              | 46.2983          |
| 8              | 50.1802          |
| 10             | 51.3123          |
| 12             | 51.6008          |
| $\infty$       | 51.6938          |



The results are plotted in [Fig. 1.4](#) along with the exact solution. We can see that the numerical method captures the essential features of the exact solution. However, because we have employed straight-line segments to approximate a continuously curving function, there is some discrepancy between the two results. One way to minimize such discrepancies is to use a smaller step size. For example, applying [Eq. \(1.12\)](#) at 1-s intervals results in a smaller error, as the straight-line segments track closer to the true solution. Using hand calculations, the effort associated with using smaller and smaller step sizes would make such numerical solutions impractical. However, with the aid of the computer, large numbers of calculations can be performed easily. Thus, you can accurately model the velocity of the jumper without having to solve the differential equation exactly.

**FIGURE 1.4** Comparison of the numerical and analytical solutions for the bungee jumper problem.



As in [Example 1.2](#), a computational price must be paid for a more accurate numerical result. Each halving of the step size to attain more accuracy leads to a doubling of the number of computations. Thus, we see that there is a trade-off between accuracy and computational effort. Such trade-offs figure prominently in numerical methods and constitute an important theme of this book.

## 1.2 CONSERVATION LAWS IN ENGINEERING AND SCIENCE

---

Aside from Newton's second law, there are other major organizing principles in science and engineering. Among the most important of these are the *conservation laws*. Although they form the basis for a variety of complicated and powerful mathematical models, the great conservation laws of science and engineering are conceptually easy to understand. They all boil down to

$$\text{Change} = \text{increases} - \text{decreases} \quad (1.14)$$

This is precisely the format that we employed when using Newton's law to develop a force balance for the bungee jumper [[Eq. \(1.8\)](#)].

Although simple, [Eq. \(1.14\)](#) embodies one of the most fundamental ways in which conservation laws are used in engineering and science—that is, to predict changes with respect to time. We will give it a special name—the *time-variable* (or *transient*) computation.

Aside from predicting changes, another way in which conservation laws are applied is for cases where change is nonexistent. If change is zero, [Eq. \(1.14\)](#) becomes

$$\text{Change} = 0 = \text{increases} - \text{decreases}$$

or

$$\text{Increases} = \text{decreases} \quad (1.15)$$

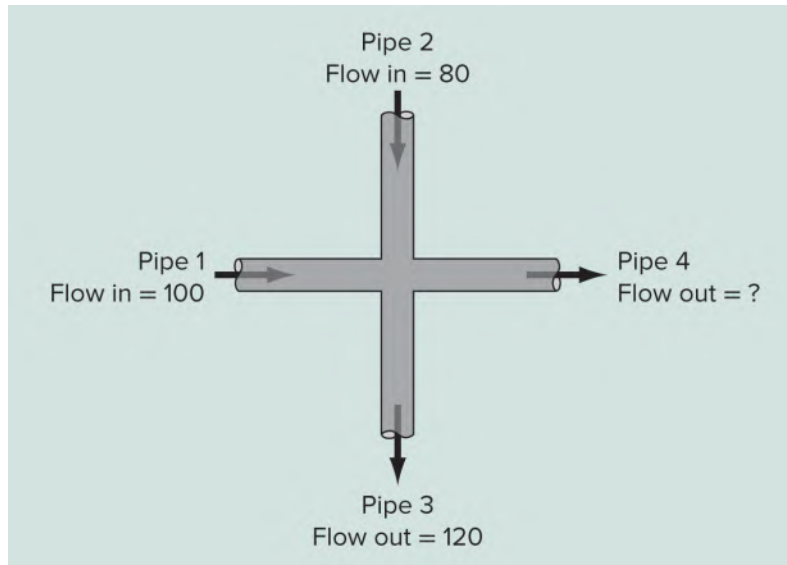
Thus, if no change occurs, the increases and decreases must be in balance. This case, which is also given a special name—the *steadystate* calculation—has many applications in engineering and science. For example, for steady-state incompressible fluid flow in pipes, the flow into a junction must be balanced by flow going out, as in

$$\text{Flow in} = \text{flow out}$$

For the junction in [Fig. 1.5](#), the balance that can be used to compute that the flow out of the fourth pipe must be 60.

---

**FIGURE 1.5** A flow balance for steady incompressible fluid flow at the junction of pipes.



For the bungee jumper, the steady-state condition would correspond to the case where the net force was zero or [Eq. (1.8) with  $dv/dt = 0$ ]

$$mg = c_d v^2 \quad (1.16)$$

Thus, at steady state, the downward and upward forces are in balance and Eq. (1.16) can be solved for the terminal velocity

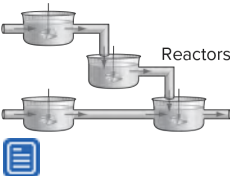
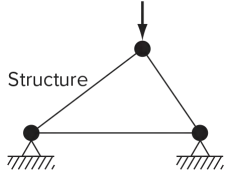
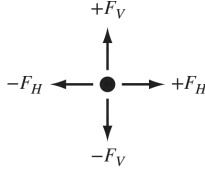
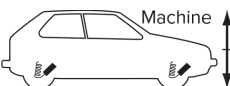
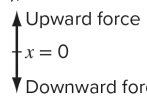
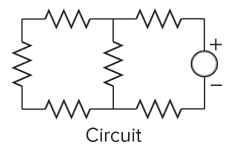
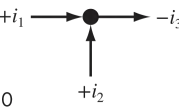
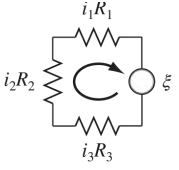
$$v = \sqrt{\frac{gm}{c_d}}$$

Although Eqs. (1.14) and (1.15) might appear trivially simple, they embody the two fundamental ways that conservation laws are employed in engineering and science. As such, they will form an important part of our efforts in subsequent chapters to illustrate the connection between numerical methods and engineering and science.

Table 1.1 summarizes some models and associated conservation laws that figure prominently in engineering. Many chemical engineering problems involve mass balances for reactors. The mass balance is derived from the conservation of mass. It specifies that the change of mass of a chemical in the reactor depends on the amount of mass flowing in minus the mass flowing out.

**TABLE 1.1** Devices and types of balances that are commonly used in the four major areas of engineering. For each case, the conservation law on which the balance is based is specified.

| Field | Device | Organizing Principle | Mathematical Expression |
|-------|--------|----------------------|-------------------------|
|-------|--------|----------------------|-------------------------|

| Field                  | Device                                                                            | Organizing Principle     | Mathematical Expression                                                                                                                                                                                                                 |
|------------------------|-----------------------------------------------------------------------------------|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Chemical engineering   |  | Conservation of mass     | Mass balance:<br><br>Over a unit of time period<br>$\Delta \text{mass} = \text{inputs} - \text{outputs}$                                             |
| Civil engineering      |  | Conservation of momentum | Force balance:<br><br>At each node<br>$\Sigma \text{ horizontal forces } (F_H) = 0$<br>$\Sigma \text{ vertical forces } (F_V) = 0$                   |
| Mechanical engineering |  | Conservation of momentum | Force balance:<br><br>$m \frac{d^2 x}{dt^2} = \text{downward force} - \text{upward force}$                                                           |
| Electrical engineering |  | Conservation of charge   | Current balance:<br><br>For each node<br>$\Sigma \text{ current } (i) = 0$                                                                           |
|                        |                                                                                   | Conservation of energy   | Voltage balance:<br><br>Around each loop<br>$\Sigma \text{ emf's} - \Sigma \text{ voltage drops for resistors} = 0$<br>$\Sigma \xi - \Sigma iR = 0$ |

Civil and mechanical engineers often focus on models developed from the conservation of momentum. For civil engineering, force balances are utilized to analyze structures such as the simple truss in [Table 1.1](#). The same principles are employed for the mechanical engineering case studies to analyze the transient up-and-down motion or vibrations of an automobile.

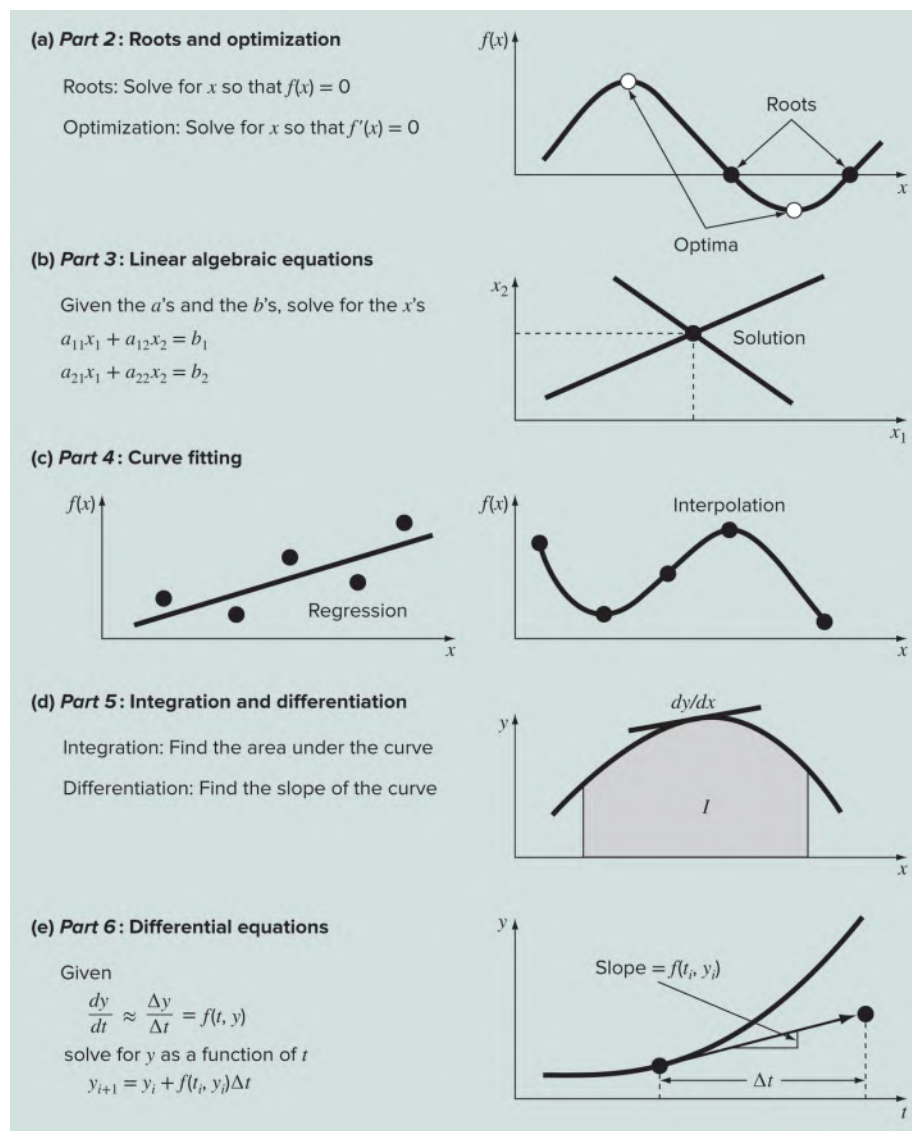
Finally, electrical engineering studies employ both current and energy balances to model electric circuits. The current balance, which results from the conservation of charge, is similar in spirit to the flow balance depicted in [Fig. 1.5](#). Just as flow must balance at the junction of pipes, electric current must balance at the junction of electric wires. The energy balance specifies that the changes of voltage around any loop of the circuit must add up to zero.

It should be noted that there are many other branches of engineering beyond chemical, civil, electrical, and mechanical. Many of these are related to the Big Four. For example, chemical engineering skills are used extensively in areas such as environmental, petroleum, and biomedical engineering. Similarly, aerospace engineering has much in common with mechanical engineering. We will endeavor to include examples from these areas in the coming pages.

# 1.3 NUMERICAL METHODS COVERED IN THIS BOOK

Euler's method was chosen for this introductory chapter because it is typical of many other classes of numerical methods. In essence, most consist of recasting mathematical operations into the simple kind of algebraic and logical operations compatible with digital computers.  **Figure 1.6** summarizes the major areas covered in this text.

**FIGURE 1.6** Summary of the numerical methods covered in this book.



 **Part Two** deals with two related topics: root finding and optimization. As depicted in  **Fig. 1.6a**, *root location* involves searching for the zeros of a function. In contrast, *optimization* involves determining a value or values of an independent variable that correspond to a “best” or optimal value of a function. Thus, as in  **Fig. 1.6a**, optimization involves identifying maxima and minima. Although somewhat different approaches are used, root location and optimization both typically arise in design contexts.

 **Part Three** is devoted to solving systems of simultaneous linear algebraic equations ( **Fig. 1.6b**). Such systems are similar in spirit to roots of equations in the sense that they are concerned with values that satisfy equations. However, in contrast to satisfying a single equation, a set of values is sought that simultaneously satisfies a set of linear algebraic equations. Such equations arise in a variety of problem contexts and in all disciplines of engineering and science. In particular, they originate in the mathematical modeling of large systems of interconnected elements such as structures, electric circuits, and fluid networks. However, they are also encountered in other areas of numerical methods such as curve fitting and differential equations.

As an engineer or scientist, you will often have occasion to fit curves to data points. The techniques developed for this purpose can be divided into two general categories: regression and interpolation. As described in  **Part Four** ( **Fig. 1.6c**), *regression* is employed where there is a significant degree of error associated with the data. Experimental results are often of this kind. For these situations, the strategy is to derive a single curve that represents the general trend of the data without necessarily matching any individual points.

---

Page 11

In contrast, *interpolation* is used where the objective is to determine intermediate values between relatively error-free data points. Such is usually the case for tabulated information. The strategy in such cases is to fit a curve directly through the data points and use the curve to predict the intermediate values.

As depicted in  **Fig. 1.6d**,  **Part Five** is devoted to integration and differentiation. A physical interpretation of *numerical integration* is the determination of the area under a curve. Integration has many applications in engineering and science, ranging from the determination of the centroids of oddly shaped objects to the calculation of total quantities based on sets of discrete measurements. In addition, numerical integration formulas play an important role in the solution of differential equations.

---

Page 12

 **Part Five** also covers methods for *numerical differentiation*. As you know from your study of calculus, this involves the determination of a function’s slope or its rate of change.

Finally,  **Part Six** focuses on the solution of *ordinary differential equations* ( **Fig. 1.6e**). Such equations are of great significance in all areas of engineering and science. This is because many physical laws are couched in terms of the rate of change of a quantity rather than the magnitude of the quantity itself. Examples range from population-forecasting models (rate of change of population) to the acceleration of a falling body (rate of change of velocity). Two types of problems are addressed: initial-value and boundary-value problems.

## 1.4 CASE STUDY

### IT'S A REAL DRAG

#### Background

In our model of the free-falling bungee jumper, we assumed that drag depends on the square of velocity [\[Eq. \(1.7\)\]](#). A more detailed representation, which was originally formulated by Lord Rayleigh, can be written as

$$F_d = -\frac{1}{2} \rho v^2 A C_d \vec{v} \quad (1.17)$$

where  $F_d$  = the drag force (N),  $\rho$  = fluid density ( $\text{kg}/\text{m}^3$ ),  $A$  = the frontal area of the object on a plane perpendicular to the direction of motion ( $\text{m}^2$ ),  $C_d$  = a dimensionless drag coefficient, and  $\vec{v}$  = a unit vector indicating the direction of velocity.

This relationship, which assumes turbulent conditions (i.e., a high *Reynolds number*), allows us to express the lumped drag coefficient from [Eq. \(1.7\)](#) in more fundamental terms as

$$c_d = \frac{1}{2} \rho A C_d \quad (1.18)$$

Thus, the lumped drag coefficient depends on the object's area, the fluid's density, and a dimensionless drag coefficient. The latter accounts for all the other factors that contribute to air resistance such as the object's "roughness." For example, a jumper wearing a baggy outfit will have a higher  $C_d$  than one wearing a sleek jumpsuit.

Note that for cases where velocity is very low, the flow regime around the object will be laminar and the relationship between the drag force and velocity becomes linear. This is referred to as *Stokes drag*.

In developing our bungee jumper model, we assumed that the downward direction was positive. Thus, [Eq. \(1.7\)](#) is an accurate representation of [Eq. \(1.17\)](#), because  $\vec{v} = +1$  and the drag force is negative. Hence, drag reduces velocity.

But what happens if the jumper has an upward (i.e., negative) velocity? In this case,  $\vec{v} = -1$  and [Eq. \(1.17\)](#) yields a positive drag force. Again, this is physically correct as the positive drag force acts downward against the upward negative velocity.

Unfortunately, for this case, [Eq. \(1.7\)](#) yields a negative drag force because it does not include the unit directional vector. In other words, by squaring the velocity, its sign and hence its direction is lost. Consequently, the model yields the physically unrealistic result that air resistance acts to accelerate an upward velocity!

In this case study, we will modify our model so that it works properly for both downward and upward velocities. We will test the modified model for the same case as [Example 1.2](#), but with an initial value of  $v(0) = -40$  m/s. In addition, we will also illustrate how we can extend the numerical analysis to determine the jumper's position.

## Solution

The following simple modification allows the sign to be incorporated into the drag force:

$$F_d = -\frac{1}{2}\rho v|v|AC_d \quad (1.19)$$

or in terms of the lumped drag:

$$F_d = -c_d v|v| \quad (1.20)$$

Thus, the differential equation to be solved is

$$\frac{dv}{dt} = g - \frac{c_d}{m} v|v| \quad (1.21)$$

In order to determine the jumper's position, we recognize that distance traveled,  $x$  (m), is related to velocity by

$$\frac{dx}{dt} = v \quad (1.22)$$

In contrast to velocity, this formulation assumes that upward distance is positive. In the same fashion as [Eq. \(1.12\)](#), this equation can be integrated numerically with Euler's method:

$$x_{i+1} = x_i + v(t_i)\Delta t \quad (1.23)$$

Assuming that the jumper's initial position is defined as  $x(0) = 0$ , and using the parameter values from [Examples 1.1](#) and [1.2](#), the velocity and distance at  $t = 2$  s can be computed as

$$\begin{aligned} v(2) &= -40 + \left[ 9.81 - \frac{0.25}{68.1}(-40)(40) \right] 2 = -8.6326 \text{ m/s} \\ x(2) &= 0 - (-40)2 = 80 \text{ m} \end{aligned}$$

Note that if we had used the incorrect drag formulation, the results would be  $-32.1274$  m/s and 80 m.

The computation can be repeated for the next interval ( $t = 2$  to 4 s):



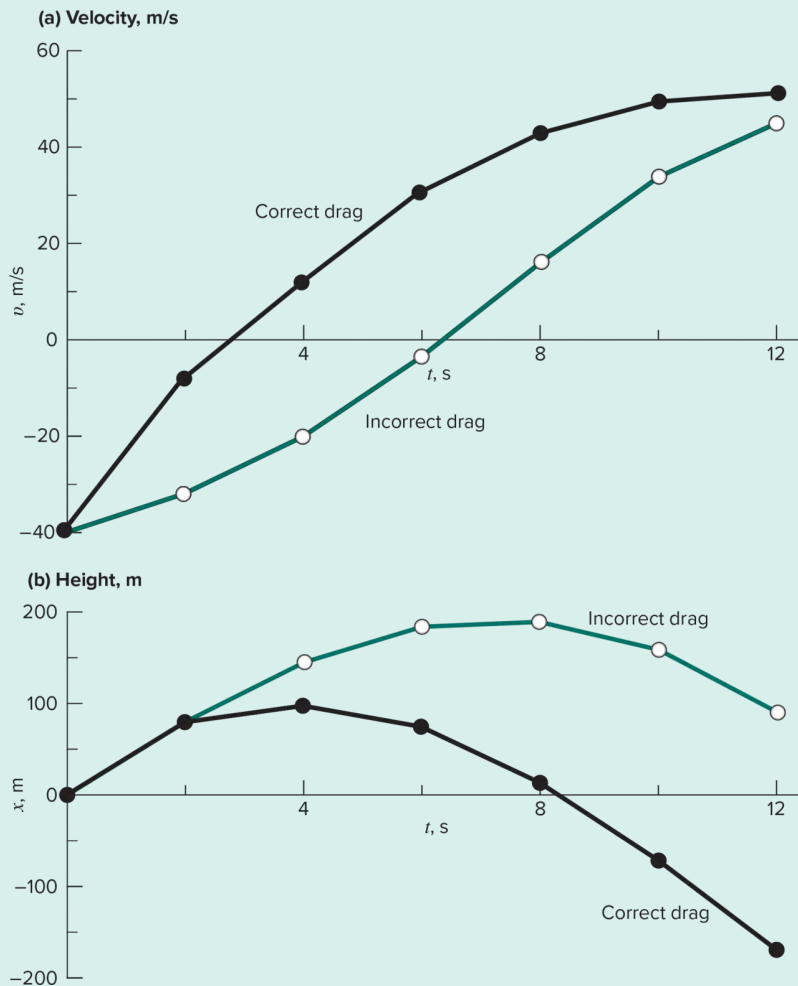
$$v(4) = -8.6326 + \left[ 9.81 - \frac{0.25}{68.1}(-8.6326)(8.6326) \right] 2 = 11.5346 \text{ m/s}$$

$$x(4) = 80 - (-8.6326)2 = 97.2651 \text{ m}$$

The incorrect drag formulation gives  $-20.0858 \text{ m/s}$  and  $144.2549 \text{ m}$ .

The calculation is continued and the results are shown in [Fig. 1.7](#) along with those obtained with the incorrect drag model. Notice that the correct formulation decelerates more rapidly because drag always diminishes the velocity.

**FIGURE 1.7** Plots of (a) velocity and (b) height for the free-falling bungee jumper with an upward (negative) initial velocity generated with Euler's method. Results for both the correct [\[Eq. \(1.20\)\]](#) and incorrect [\[Eq. \(1.7\)\]](#) drag formulations are displayed.



With time, both velocity solutions converge on the same terminal velocity because eventually both are directed downward in which case, [Eq. \(1.7\)](#) is correct. However, the impact on the height prediction is quite dramatic with the incorrect drag case resulting in a much higher trajectory.

This case study demonstrates how important it is to have the correct physical model. In some cases, the solution will yield results that are clearly unrealistic. The current example is more insidious as there is no visual evidence

that the incorrect solution is wrong. That is, the incorrect solution “looks” reasonable.

# PROBLEMS

**1.1** Use calculus to verify that [Eq. \(1.9\)](#) is a solution of [Eq. \(1.8\)](#) for the initial condition  $v(0) = 0$ .

**1.2** Use calculus to solve [Eq. \(1.21\)](#) for the case where the initial velocity is (a) positive and (b) negative. (c) Based on your results for (a) and (b), perform the same computation as in [Example 1.1](#) but with an initial velocity of  $-40$  m/s. Compute values of the velocity from  $t = 0$  to  $12$  s at intervals of  $2$  s. Note that for this case, the zero velocity occurs at  $t = 3.470239$  s.

**1.3** The following information is available for a bank account:

| Date | Deposits | Withdrawals | Balance |
|------|----------|-------------|---------|
| 5/1  |          |             | 1512.33 |
|      | 220.13   | 327.26      |         |
| 6/1  |          |             |         |
|      | 216.80   | 378.61      |         |
| 7/1  |          |             |         |
|      | 450.25   | 106.80      |         |
| 8/1  |          |             |         |
|      | 127.31   | 350.61      |         |
| 9/1  |          |             |         |

Note that the money earns interest which is computed as

$$\text{Interest} = iB_i$$

where  $i$  = the interest rate expressed as a fraction per month, and  $B_i$  the initial balance at the beginning of the month.

- (a) Use the conservation of cash to compute the balance on 6/1, 7/1, 8/1, and 9/1 if the interest rate is 1% per month ( $i = 0.01/\text{month}$ ). Show each step in the computation.
- (b) Write a differential equation for the cash balance in the form

$$\frac{dB}{dt} = f[D(t), W(t), i]$$

where  $t$  = time (months),  $D(t)$  = deposits as a function of time (\$/month),  $W(t)$  = withdrawals as a function of time (\$/month). For this case, assume that interest is compounded continuously; that is, interest =  $iB$ .

- (c) Use Euler's method with a time step of 0.5 month to simulate the balance. Assume that the deposits and withdrawals are applied uniformly over the month.
- (d) Develop a plot of balance versus time for (a) and (c).

**1.4** Repeat [Example 1.2](#). Compute the velocity to  $t = 12$  s, with a step size of (a) 1 and (b) 0.5 s. Can you make any statement regarding the errors of the calculation based on the results?

**1.5** Rather than the nonlinear relationship of [Eq. \(1.7\)](#), you might choose to model the upward force on the bungee jumper as a linear relationship:

$$F_U = -c'v$$

where  $c' =$  a first-order drag coefficient (kg/s).

- (a) Using calculus, obtain the closed-form solution for the case where the jumper is initially at rest ( $v = 0$  at  $t = 0$ ).
- (b) Repeat the numerical calculation in [Example 1.2](#) with the same initial condition and parameter values. Use a value of 11.5 kg/s for  $c'$ .

**1.6** For the free-falling bungee jumper with linear drag ([Prob. 1.5](#)), assume a first jumper is 70 kg and has a drag coefficient of 12 kg/s. If a second jumper has a drag coefficient of 15 kg/s and a mass of 80 kg, how long will it take her to reach the same velocity jumper 1 reached in 9 s?

**1.7** For the second-order drag model [\[Eq. \(1.8\)\]](#), compute the velocity of a free-falling parachutist using Euler's method for the case where  $m = 80$  kg and  $c_d = 0.25$  kg/m. Perform the calculation from  $t = 0$  to 20 s with a step size of 1 s. Use an initial condition that the parachutist has an upward velocity of 20 m/s at  $t = 0$ . At  $t = 10$  s, assume that the chute is instantaneously deployed so that the drag coefficient jumps to 1.5 kg/m.

**1.8** The amount of a uniformly distributed radioactive contaminant contained in a closed reactor is measured by its concentration  $c$  (becquerel/liter or Bq/L). The contaminant decreases at a decay rate proportional to its concentration; that is

$$\text{Decay rate} = -kc$$

where  $k$  is a constant with units of  $\text{day}^{-1}$ . Therefore, according to [Eq. \(1.14\)](#), a mass balance for the reactor can be written as

$$\frac{dc}{dt} = -kc$$

$$\left( \begin{array}{c} \text{change} \\ \text{in mass} \end{array} \right) = \left( \begin{array}{c} \text{decrease} \\ \text{by decay} \end{array} \right)$$

- (a) Use Euler's method to solve this equation from  $t = 0$  to 1 d with  $k = 0.175 \text{ d}^{-1}$ . Employ a step size of  $\Delta t = 0.1$  d. The concentration at  $t = 0$  is 100 Bq/L.
- (b) Plot the solution on a semilog graph (i.e.,  $\ln c$  versus  $t$ ) and determine the slope. Interpret your results.

**1.9** A storage tank (Fig. P1.9) contains a liquid at depth  $y$  where  $y = 0$  when the tank is half full. Liquid is withdrawn at a constant flow rate  $Q$  to meet demands. The contents are resupplied at a sinusoidal rate  $3Q \sin^2(t)$ . Equation (1.14) can be written for this system as

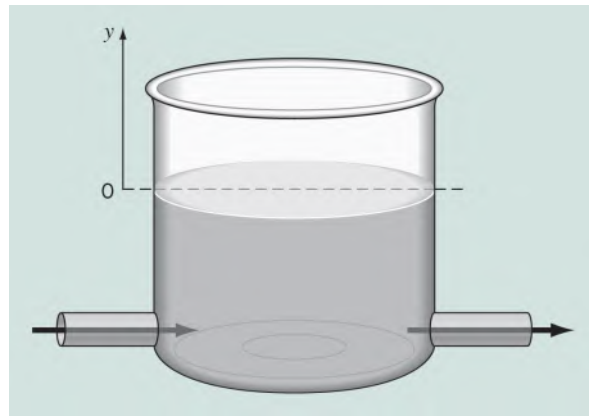
$$\begin{aligned} \frac{d(Ay)}{dt} &= 3Q \sin^2(t) - Q \\ \left( \begin{array}{c} \text{change in} \\ \text{volume} \end{array} \right) &= (\text{inflow}) - (\text{outflow}) \end{aligned}$$

or, since the surface area  $A$  is constant

$$\frac{dy}{dt} = 3 \frac{Q}{A} \sin^2(t) - \frac{Q}{A}$$

Use Euler's method to solve for the depth  $y$  from  $t = 0$  to 10 d with a step size of 0.5 d. The parameter values are  $A = 1250 \text{ m}^2$  and  $Q = 450 \text{ m}^3/\text{d}$ . Assume that the initial condition is  $y = 0$ .

**FIGURE P1.9**



**1.10** For the same storage tank described in Prob. 1.9, suppose that the outflow is not constant but rather depends on the depth. For this case, the differential equation for depth can be written as

$$\frac{dy}{dt} = 3 \frac{Q}{A} \sin^2(t) - \frac{\alpha(1+y)^{1.5}}{A}$$

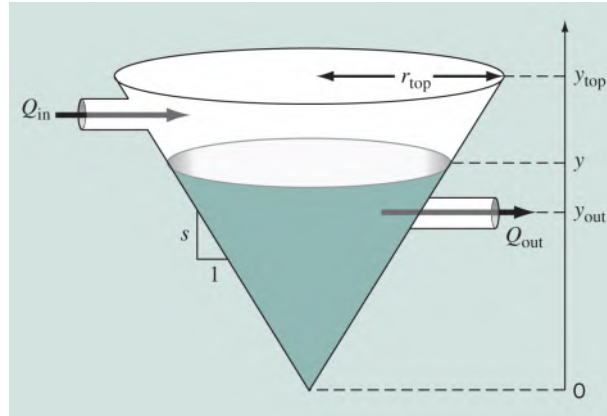
Use Euler's method to solve for the depth  $y$  from  $t = 0$  to 10 d with a step size of 0.5 d. The parameter values are  $A = 1250 \text{ m}^2$ ,  $Q = 450 \text{ m}^3/\text{d}$ , and  $\alpha = 150$ . Assume that the initial condition is  $y = 0$ .

**1.11** Apply the conservation of volume (see Prob. 1.9) to simulate the level of liquid in a conical storage tank (Fig. P1.11). The liquid flows in at a sinusoidal rate of  $Q_{\text{in}} = 3 \sin^2(t)$  and flows out according to

$$\begin{aligned} Q_{\text{out}} &= 3(y - y_{\text{out}})^{1.5} & y > y_{\text{out}} \\ Q_{\text{out}} &= 0 & y \leq y_{\text{out}} \end{aligned}$$

where flow has units of  $\text{m}^3/\text{d}$  and  $y$  = the elevation of the water surface above the bottom of the tank (m). Use Euler's method to solve for the depth  $y$  from  $t = 0$  to 10 d with a step size of 0.5 d. The parameter values are  $r_{\text{top}} = 2.5$  m,  $y_{\text{top}} = 4$  m, and  $y_{\text{out}} = 1$  m. Assume that the level is initially below the outlet pipe with  $y(0) = 0.8$  m.

**FIGURE P1.11**



**1.12** A group of 35 students attends a class in an insulated room which measures 11 by 8 by 3 m. Each student takes up about  $0.075 \text{ m}^3$  and gives out about 80 W of heat ( $1 \text{ W} = 1 \text{ J/s}$ ). Calculate the air temperature rise during the first 20 minutes of the class if the room is completely sealed and insulated. Assume the heat capacity  $C_v$  for air is  $0.718 \text{ kJ}/(\text{kg} \cdot \text{K})$ . Assume air is an ideal gas at  $20^\circ \text{C}$  and  $101.325 \text{ kPa}$ . Note that the heat absorbed by the air,  $Q$  is related to the mass of the air,  $m$ , the heat capacity, and the change in temperature by the following relationship:

$$Q = m \int_{T_1}^{T_2} C_v dT = m C_v (T_2 - T_1)$$

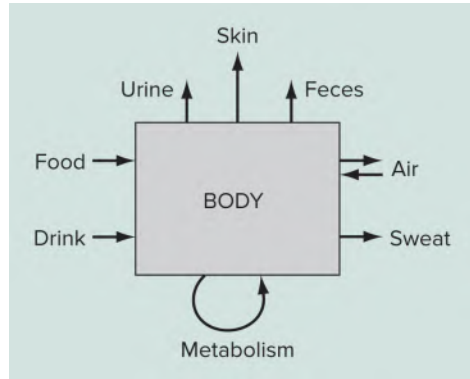
The mass of air can be obtained from the ideal gas law:

$$PV = \frac{m}{\text{Mwt}} RT$$

where  $P$  is the gas pressure,  $V$  is the volume of the gas,  $\text{Mwt}$  is the molecular weight of the gas (for air,  $28.97 \text{ kg/kmol}$ ), and  $R$  is the ideal gas constant  $[8.314 \text{ kPa} \cdot \text{m}^3/(\text{kmol} \cdot \text{K})]$ .

**1.13**  **Figure P1.13** depicts the various ways in which an average person gains and loses water in one day. One liter is ingested as food, and the body metabolically produces 0.3 liters. In breathing air, the exchange is 0.05 liters while inhaling, and 0.4 liters while exhaling over a one-day period. The body will also lose 0.3, 1.4, 0.2, and 0.35 liters through sweat, urine, feces, and the skin, respectively. To maintain steady state, how much water must be drunk per day?

**FIGURE P1.13**



**1.14** In our example of the free-falling bungee jumper, we assumed that the acceleration due to gravity was a constant value of  $9.81 \text{ m/s}^2$ . Although this is a decent approximation when we are examining falling objects near the surface of the earth, the gravitational force decreases as we move above sea level. A more general representation based on Newton's inverse square law of gravitational attraction can be written as

$$g(x) = g(0) \frac{R^2}{(R+x)^2}$$

where  $g(x)$  = gravitational acceleration at altitude  $x$  (in m) measured upward from the earth's surface ( $\text{m/s}^2$ ),  $g(0)$  = gravitational acceleration at the earth's surface ( $\cong 9.81 \text{ m/s}^2$ ), and  $R$  = the earth's radius ( $\cong 6.37 \times 10^6 \text{ m}$ ).

- (a) In a fashion similar to the derivation of [Eq. \(1.8\)](#), use a force balance to derive a differential equation for velocity as a function of time that utilizes this more complete representation of gravitation. However, for this derivation, assume that upward velocity is positive.
- (b) For the case where drag is negligible, use the chain rule to express the differential equation as a function of altitude rather than time. Recall that the chain rule is

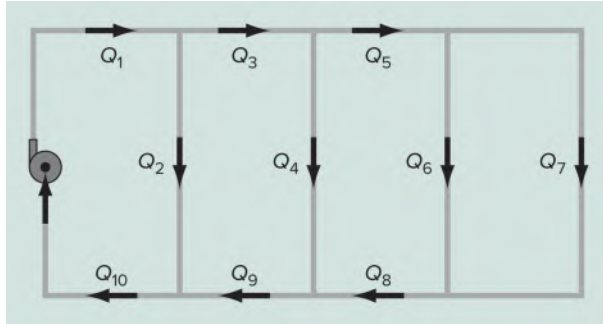
$$\frac{dv}{dt} = \frac{dv}{dx} \frac{dx}{dt}$$

- (c) Use calculus to obtain the closed form solution where  $v = v_0$  at  $x = 0$ .
- (d) Use Euler's method to obtain a numerical solution from  $x = 0$  to 100,000 m using a step of 10,000 m where the initial velocity is 1500 m/s upward. Compare your result with the analytical solution.

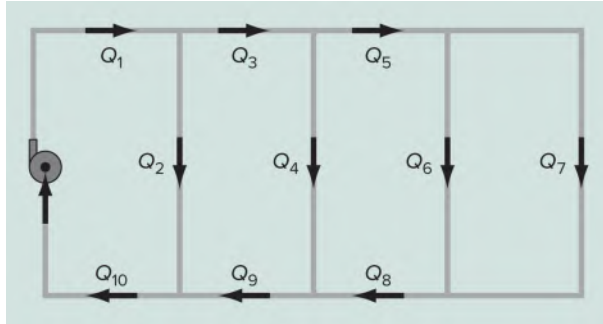
**1.15** Suppose that a spherical droplet of liquid evaporates at a rate that is proportional to its surface area.

$$\frac{dV}{dt} = -kA$$

where  $V = \text{volume}(\text{mm}^3)$ ,  $t = \text{time}(\text{min})$ ,  $k = \text{the evaporation rate}(\text{mm}/\text{min})$ , and  $A = \text{surface area}(\text{mm}^2)$ . Use Euler's method to compute the volume of the droplet from  $t = 0$  to 10 min using a step size of 0.25 min. Assume that  $k = 0.08 \text{ mm}/\text{min}$  and that the droplet initially has a radius of 2.5 mm. Assess the validity of your results by determining the radius of your final computed volume and verifying that it is consistent with the evaporation rate.

**1.16** A fluid is pumped into the network shown in  Fig. P1.16. If  $Q_2 = 0.7$ ,  $Q_3 = 0.5$ ,  $Q_7 = 0.1$ , and  $Q_8 = 0.3 \text{ m}^3/\text{s}$ , determine the other flows.

**FIGURE P1.16**



**1.17** *Newton's law of cooling* says that the temperature of a body changes at a rate proportional to the difference between its temperature and that of the surrounding medium (the ambient temperature),

$$\frac{dT}{dt} = -k(T - T_a)$$

where  $T = \text{the temperature of the body } (^{\circ}\text{C})$ ,  $t = \text{time } (\text{min})$ ,  $k = \text{the proportionality constant (per minute)}$ , and  $T_a = \text{the ambient temperature } (^{\circ}\text{C})$ . Suppose that a cup of coffee originally has a temperature of  $70^{\circ}\text{C}$ . Use Euler's method to compute the temperature from  $t = 0$  to 20 min using a step size of 2 min if  $T_a = 20^{\circ}\text{C}$  and  $k = 0.019/\text{min}$ .

**1.18** You are working as a crime scene investigator and must predict the temperature of a homicide victim over a 5-hr period. You know that the room where the victim was found was at  $10^{\circ}\text{C}$  when the body was discovered.

- Use Newton's law of cooling ( **Prob. 1.17**) and Euler's method to compute the victim's body temperature for the 5-hr period using values of  $k = 0.12/\text{hr}$  and  $\Delta t = 0.5 \text{ hr}$ . Assume that the victim's body temperature at the time of death was  $37^{\circ}\text{C}$ , and that the room temperature was at a constant value of  $10^{\circ}\text{C}$  over the 5-hr period.
- Further investigation reveals that the room temperature had actually dropped linearly from  $20$  to  $10^{\circ}\text{C}$  over the 5-hr period. Repeat the same calculation as in (a) but incorporate this new information.
- Compare the results from (a) and (b) by plotting them on the same graph.

**1.19** The velocity is equal to the rate of change of distance,  $x \text{ (m)}$ :



$$\frac{dx}{dt} = v(t) \quad (\text{P1.19})$$

Use Euler's method to numerically integrate [Eqs. \(P1.19\)](#) and [Eq. \(1.8\)](#) in order to determine both the velocity and distance fallen as a function of time for the first 10 seconds of freefall using the same parameters and conditions as in [Example 1.2](#). Develop a plot of your results.

**1.20** In addition to the downward force of gravity (weight) and drag, an object falling through a fluid is also subject to a buoyancy force which is proportional to the displaced volume (*Archimedes' principle*). For example, for a sphere with diameter  $d$  (m), the sphere's volume is  $V = \pi d^3/6$ , and its projected area is  $A = \pi d^2/4$ . The buoyancy force can then be computed as  $F_b = -\rho V g$ . We neglected buoyancy in our derivation of [Eq. \(1.8\)](#) because it is relatively small for an object like a bungee jumper moving through air. However, for a more dense fluid like water, it becomes more prominent.

- Derive a differential equation in the same fashion as [Eq. \(1.8\)](#), but include the buoyancy force and represent the drag force as described in [Sec. 1.4](#).
- Rewrite the differential equation from (a) for the special case of a sphere.
- Use the equation developed in (b) to compute the terminal velocity (i.e., for the steady-state case). Use the following parameter values for a sphere falling through water: sphere diameter = 1 cm, sphere density =  $2700 \text{ kg/m}^3$ , water density =  $1000 \text{ kg/m}^3$ , and  $C_d = 0.47$ .
- Use Euler's method with a step size of  $\Delta t = 0.03125 \text{ s}$  to numerically solve for the velocity from  $t = 0$  to  $0.25 \text{ s}$  with an initial velocity of zero.

**1.21** As noted in [Sec. 1.4](#), a fundamental representation of the drag force, which assumes turbulent conditions, can be formulated as

$$F_d = -\frac{1}{2} \rho A C_d v |v|$$

where  $F_d$  = the drag force (N),  $\rho$  = fluid density ( $\text{kg/m}^3$ ),  $A$  = the frontal area of the object on a plane perpendicular to the direction of motion ( $\text{m}^2$ ),  $v$  = velocity (m/s), and  $C_d$  = a dimensionless drag coefficient.

- Write the pair of differential equations for velocity and position (see [Prob. 1.19](#)) to describe the vertical motion of a sphere with diameter,  $d$  (m), and a density of  $\rho_s$  ( $\text{kg/m}^3$ ). The differential equation for velocity should be written as a function of the sphere's diameter.
- Use Euler's method with a step size of  $\Delta t = 2 \text{ s}$  to compute the position and velocity of a sphere over the first 14 seconds. Employ the following parameters in your calculation:  $d = 120 \text{ cm}$ ,  $\rho = 1.3 \text{ kg/m}^3$ ,  $\rho_s = 2700 \text{ kg/m}^3$ , and  $C_d = 0.47$ . Assume that the sphere has the initial conditions:  $x(0) = 100 \text{ m}$  and  $v(0) = -40 \text{ m/s}$ .
- Develop a plot of your results (i.e.,  $y$  and  $v$  versus  $t$ ) and use it to graphically estimate when the sphere would hit the ground.
- Compute the value for the bulk second-order drag coefficient,  $c_d'$  ( $\text{kg/m}$ ). Note that the bulk second-order drag coefficient is the term in the final differential equation for velocity that multiplies

the term  $v|v|$ .

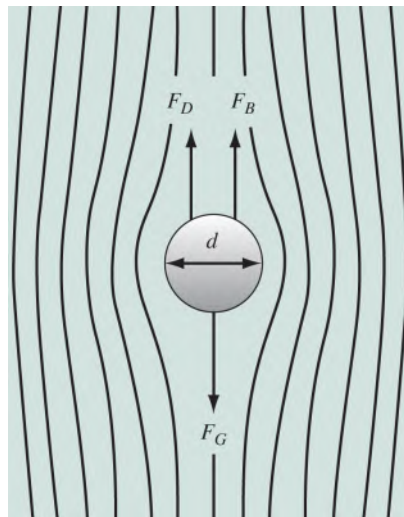
**1.22** As depicted in  **Fig. P1.22**, a spherical particle settling through a quiescent fluid is subject to three forces: the downward force of gravity ( $F_G$ ), and the upward forces of buoyancy ( $F_B$ ) and drag ( $F_D$ ). Both the gravity and buoyancy forces can be computed with Newton's second law with the latter equal to the weight of the displaced fluid. For laminar flow, the drag force can be computed with *Stoke's law*,

$$F_D = 3\pi\mu dv$$

where  $\mu$  = the dynamic viscosity of the fluid (Pa·s),  $d$  = the particle diameter (m), and  $v$  = the particle's settling velocity (m/s). The mass of the particle can be expressed as the product of the particle's volume and density,  $\rho_s$  (kg/m<sup>3</sup>), and the mass of the displaced fluid can be computed as the product of the particle's volume and the fluid's density,  $\rho$  (kg/m<sup>3</sup>). The volume of a sphere is  $\pi d^3/6$ . In addition, laminar flow corresponds to the case where the dimensionless Reynolds number,  $Re$ , is less than 1, where  $Re = \rho dv/\mu$ .

- (a) Use a force balance for the particle to develop the differential equation for  $dv/dt$  as a function of  $d$ ,  $\rho$ ,  $\rho_s$ , and  $\mu$ .
- (b) At steady-state, use this equation to solve for the particle's terminal velocity.
- (c) Employ the result of (b) to compute the particle's terminal velocity in m/s for a spherical silt particle settling in water:  $d = 10\ \mu\text{m}$ ,  $\rho = 1000\ \text{kg/m}^3$ ,  $\rho_s = 2650\ \text{kg/m}^3$ , and  $\mu = 0.0014\ \text{Pa}\cdot\text{s}$ .
- (d) Check whether flow is laminar.
- (e) Use Euler's method to compute the velocity from  $t = 0$  to  $2^{-15}\ \text{s}$  with  $\Delta t = 2^{-18}\ \text{s}$  given the initial condition:  $v(0) = 0$ . Use a tool like a spreadsheet to perform the calculation and generate the plot.

**FIGURE P1.22**



**1.23** As depicted in  **Fig. P1.23**, the downward deflection,  $y$  (m), of a cantilever beam with a uniform load,  $w = 10,000$  kg/m, can be computed as

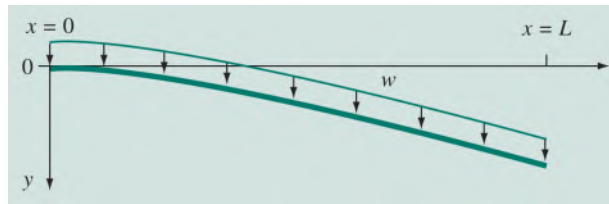
$$y = \frac{w}{24EI}(x^4 - 4Lx^3 + 6L^2x^2)$$

where  $x$  = distance(m),  $E$  = the modulus of elasticity  $= 2 \times 10^{11}$  Pa,  
 $I$  = moment of inertia  $= 3.25 \times 10^{-4}$  m<sup>4</sup>, and  $L$  = length = 4 m. This equation can be differentiated to yield the slope of the downward deflection as a function of  $x$

$$\frac{dy}{dx} = \frac{w}{24EI}(4x^3 - 12Lx^2 + 12L^2x)$$

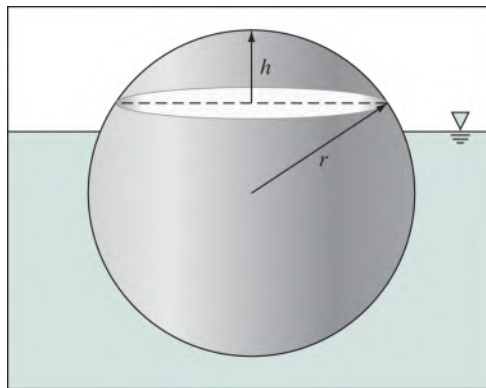
If  $y = 0$  at  $x = 0$ , use this equation with Euler's method ( $\Delta x = 0.125$  m) to compute the deflection from  $x = 0$  to  $L$ . Develop a plot of your results along with the analytical solution computed with the first equation.

**FIGURE P1.23** A cantilever beam.



**1.24** Use *Archimedes' principle* to develop a steady-state force balance for a spherical ball of ice floating in seawater. The force balance should be expressed as a third-order polynomial (cubic) in terms of height of the cap above the water line ( $h$ ), and the seawater's density ( $\rho_f$ ), the ball's density ( $\rho_s$ ) and radius ( $r$ ).

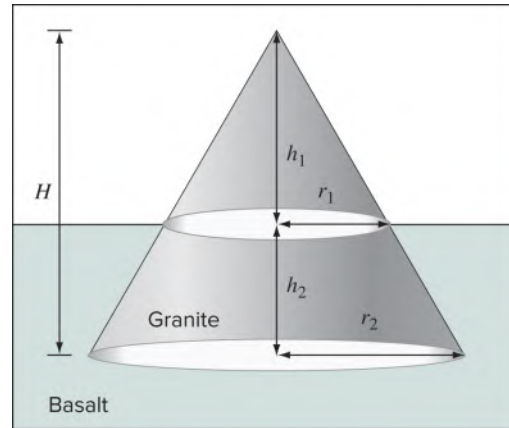
**FIGURE P1.24**



**1.25** Beyond fluids, *Archimedes' principle* has proven useful in geology when applied to solids on the earth's crust.  **Figure P1.25** depicts one such case where a lighter conical granite mountain “floats on”

a denser basalt layer at the earth's surface. Note that the part of the cone below the surface is formally referred to as a *frustum*. Develop a steady-state force balance for this case in terms of the following parameters: basalt's density ( $\rho_b$ ), granite's density ( $\rho_g$ ), the cone's bottom radius ( $r$ ), and the height above ( $h_1$ ) and below ( $h_2$ ) the earth's surface.

**FIGURE P1.25**



**1.26** As depicted in [Fig. P1.26](#), an *RLC circuit* consists of three elements: a resistor ( $R$ ), an inductor ( $L$ ), and a capacitor ( $C$ ). The flow of current across each element induces a voltage drop. Kirchhoff's second voltage law states that the algebraic sum of these voltage drops around a closed circuit is zero,

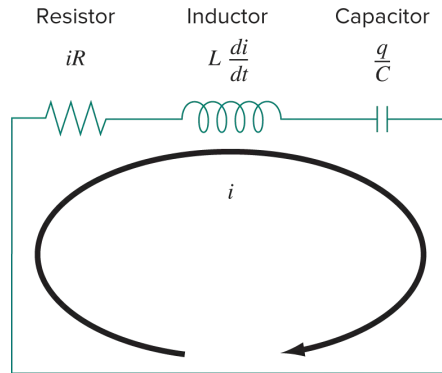
$$iR + L \frac{di}{dt} + \frac{q}{C} = 0$$

where  $i$  = current,  $R$  = resistance,  $L$  = inductance,  $t$  = time,  $q$  = charge, and  $C$  = capacitance. In addition, the current is related to charge as in

$$\frac{dq}{dt} = i$$

- If the initial values are  $i(0) = 0$  and  $q(0) = 1$  C, use Euler's method to solve this pair of differential equations from  $t = 0$  to  $0.1$  s using a step size of  $\Delta t = 0.01$  s. Employ the following parameters for your calculation:  $R = 200 \, \Omega$ ,  $L = 5$  H, and  $C = 10^{-4}$  F.
- Develop a plot of  $i$  and  $q$  versus  $t$ .

**FIGURE P1.26**

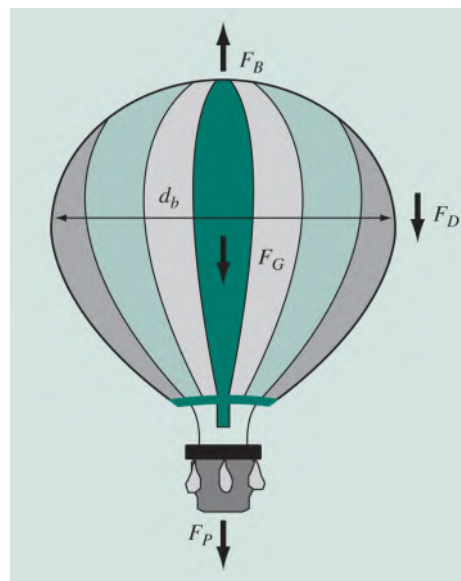


**1.27** Suppose that a parachutist with linear drag ( $m = 70$  kg,  $c = 12.5$  kg/s) jumps from an airplane flying at an altitude of 200 m with a horizontal velocity of 180 m/s relative to the ground.

- Write a system of four differential equations for  $x$ ,  $y$ ,  $v_x = dx/dt$  and  $v_y = dy/dt$ .
- If the initial horizontal position is defined as  $x = 0$ , use Euler's methods with  $\Delta t = 1$  s to compute the jumper's position over the first 10 seconds.
- Develop plots of  $y$  versus  $t$  and  $y$  versus  $x$ . Use the plot to graphically estimate when and where the jumper would hit the ground if the chute failed to open.

**1.28**  **Figure P1.28** shows the forces exerted on a hot air balloon system.

**FIGURE P1.28** Forces on a hot air balloon:  $F_B$  = buoyancy,  $F_G$  = weight of gas,  $F_P$  = weight of payload (including the balloon envelope), and  $F_D$  = drag. Note that the direction of the drag is downward when the balloon is rising.



Formulate the drag force as

$$F_D = \frac{1}{2} \rho_a v^2 A C_d$$

where  $\rho_a$  = air density ( $\text{kg}/\text{m}^3$ ),  $v$  = velocity ( $\text{m}/\text{s}$ ),  $A$  = projected frontal area ( $\text{m}^2$ ), and  $C_d$  = the dimensionless drag coefficient ( $\cong 0.47$  for a sphere). Note also that the total mass of the balloon consists of two components:

$$m = m_G + m_P$$

where  $m_G$  = the mass of the gas inside the expanded balloon (kg) and  $m_P$  = the mass of the payload (basket, passengers, and the unexpanded balloon = 265 kg). Assume that the ideal gas law holds ( $P = \rho RT$ ), that the balloon is a perfect sphere with a diameter of 17.3 m, and that the heated air inside the envelope is at roughly the same pressure as the outside air.

Other necessary parameters are:

Normal atmospheric pressure,  $P = 101,300 \text{ Pa}$

The gas constant for dry air,  $R = 287 \text{ Joules}/(\text{kg}\cdot\text{K})$

The air inside the balloon is heated to an average temperature,  $T = 100 \text{ }^\circ\text{C}$

The normal (ambient) air density,  $\rho = 1.2 \text{ kg}/\text{m}^3$ .

- (a) Use a force balance to develop the differential equation for  $dv/dt$  as a function of the model's fundamental parameters.
- (b) At steady-state, calculate the particle's terminal velocity.
- (c) Use Euler's method and Excel to compute the velocity from  $t = 0$  to 60 s with  $\Delta t = 2 \text{ s}$  given the previous parameters along with the initial condition:  $v(0) = 0$ . Develop a plot of your results.

# Python Fundamentals

## CHAPTER OBJECTIVES

The primary objective of this chapter is to introduce you to the Python programming language with a focus on numerical calculations. Further, we will illustrate here, via the IPython Console, the use of Python in calculator mode to implement interactive computations. We will also show how to use the Spyder Editor window to enter and run blocks of Python code. Specific objectives and topics covered are

- Including key modules associated with numerical/scientific computations in the Python program development environment.
- Learning how real and complex numbers are assigned to variables.
- Learning how vector and matrix arrays are assigned values, including numerical sequences using the `arange`, `linspace`, and `logspace` functions from the NumPy module.
- Understanding how to construct mathematical expressions, including priority rules and operator precedence.
- Gaining a general understanding of Python's built-in functions, including those accessed through modules commonly imported for numerical calculations, `math` and `numpy`.
- Finding information about built-in functions via the Help facility.
- Learning how to create a simple line plot based on an equation using the `pylab` module.

## YOU'VE GOT A PROBLEM

In  **Chap. 1**, we used a force balance to determine the terminal velocity of a free-falling object, like a bungee jumper:

$$v_t = \sqrt{\frac{mg}{c_d}}$$

where  $v_t$  = terminal velocity (m/s),  $g$  = gravitational acceleration (approximately = 9.81 m/s<sup>2</sup>),  $m$  = mass (kg), and  $c_d$  = drag coefficient (kg/m).

Aside from predicting the terminal velocity, this equation can also be rearranged to compute the drag coefficient:

(2.1)

$$c_d = \frac{m g}{v_t^2}$$

Thus, if we measure the terminal velocity of several jumpers of known mass, this equation provides a means to estimate the drag coefficient. The data in [Table 2.1](#) were collected for this purpose.

**TABLE 2.1** Data for the mass and associated terminal velocities of several jumpers.

|                                  |      |      |      |      |      |      |      |
|----------------------------------|------|------|------|------|------|------|------|
| <b><i>m</i>, kg</b>              | 83.6 | 60.2 | 72.1 | 91.1 | 92.9 | 65.3 | 80.9 |
| <b><i>v<sub>t</sub></i>, m/s</b> | 53.4 | 48.5 | 50.9 | 55.7 | 54.0 | 47.7 | 51.1 |

In this chapter, we will learn how Python can be used to analyze such data. Beyond showing how Python can be employed to compute quantities like drag coefficients, we will also illustrate how the graphical capabilities of the `pylab` module provide additional insight into such analyses.

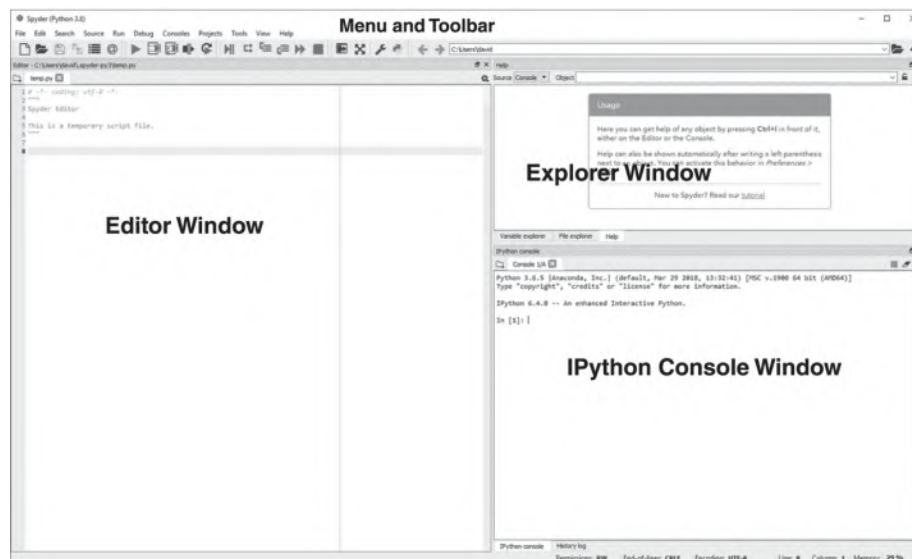


## 2.1 THE SPYDER/IPYTHON ENVIRONMENT<sup>1</sup>

Python is an open source programming language, available at no cost. Python programs can be written and developed in numerous environments. They can be written in a simple text editor, like Notepad in the Windows operating system, and executed in a primitive command window. Most programmers, however, prefer to use an Integrated Development Environment (IDE). To do that here, we must make a choice of such an IDE and will do so via one of the most popular environments Spyder as an element of the Anaconda software package, which uses the IPython Console. Information on acquiring this IDE can be obtained via the Internet at the URL <https://www.anaconda.com/distribution/>.

The Spyder IDE appears as shown in [Fig. 2.1](#). For our purposes in this chapter, we will be typing commands into the IPython Console window. Further orientation for the IDE is obtained via the tutorial referenced in the Explorer window. Later in the chapter, we will enter code into the Editor window and run it from there.

**FIGURE 2.1** Spyder integrated development environment.



The calculator mode for Python operates in a sequential fashion as you type in commands line by line. For each command, you get a result. Thus, you can think of it as a fancy calculator. For example, if you type in

```
In [1]: 55-16
```

the Console will display the result<sup>2</sup>

```
Out[1]: 39
```

## 2.2 ASSIGNMENT

Assignment refers to allocating values to variable names. This results in the storage of the value(s) in the memory location corresponding to the variable name.

### 2.2.1 Scalars

The assignment of values to scalar variables is similar to that of other computer languages. Try typing

```
a = 4
```

The Console does not echo a result but rather proceeds to the next command line. If you open the Variable Explorer tab in the Explorer window, you should see

| Name | Type | Size | Value |
|------|------|------|-------|
| a    | int  | 1    | 4     |

Try entering the command

```
A = 6
```

and you will note in the Variable Explorer window

| Name | Type | Size | Value |
|------|------|------|-------|
| A    | int  | 1    | 6     |
| a    | int  | 1    | 4     |

So, you can see that variable names are case sensitive. The variable A is distinct from a. You can display a variable's value in the Console by entering only its name

```
A
```

and its value will be displayed in the Out line.

Multiple assignments can be made on one line by separating them with semicolons.

```
y = -3; z = 25
```

We can assign complex values to variables, since Python is able to manage complex calculations automatically. The unit imaginary number,  $\sqrt{-1}$ , is used to identify the imaginary part of a complex number with the letter label j. Consequently, a complex value can be assigned simply as in

```
c = 2 + 4j
```

Then, entering

```
c
```

will produce an output

```
(2+4j)
```

There are several predefined variables available in Python by importing the `math` module with the command

```
import math
```

Two examples are for  $\pi$  and Euler's number  $e$ .<sup>3</sup>

```
math.pi
3.141592653589793

math.e
2.718281828459045
```

Notice that the internal precision is 16 figures.<sup>4</sup>

Python will generally display quantities to maximum precision. We can adjust this display using the `format` method. We will not present all the details of this method at this point but will introduce them along the way in subsequent sections and chapters. Here are two examples:

Format the display of  $\pi$  to four decimal places:

```
{0:7.4f}'.format(math.pi)
' 3.1416'
```

Interpretation

```
{0:7.4f}'
0 : first item formatted
7 : minimum field width of 7 characters
4 : number of decimal places
f : fixed decimal display
```

Note that the result is displayed within `'` as a string type, not a numerical type.

Format Planck's constant,  $h$ , in exponential form with four decimal places:

First, we can access the value via the `SciPy` module with

```
import scipy.constants as pc
```

and then

```
{0:12.4e}'.format(pc.h)
' 6.6261e-34'
```

Above `pc` is defined as shorthand for `scipy.constants` to save keystrokes. The syntax of the format string is like the previous example.

## 2.2.2 Arrays, Vectors, and Matrices

Python has different means for representing collections of information, including numbers. It is not our purpose here to introduce all those in detail but rather to focus on collections of numerical data. For now, here is a summary of Python's collection types:

| Type | Description | Example |
|------|-------------|---------|
|------|-------------|---------|

| Type       | Description                                                                                                                                              | Example                                                          |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| list       | a collection of various data types; e.g., using brackets                                                                                                 | [2 , False , 'oats' , 0.618034]                                  |
| tuple      | an immutable list (cannot be extended, shrunk, have elements removed or reassigned); e.g., using parentheses                                             | (2 , False , 'oats' , 0.618034)                                  |
| set        | an unordered collection of unique objects; e.g., using braces                                                                                            | {2 , False , 'oats' , 0.618034}                                  |
| dictionary | a collection of objects, each identified by a key, not a numerical index or subscript; e.g., value pairs within braces                                   | Fourteeners={'Elbert':4401.2, 'Massive':4398., 'Harvard':4395.6} |
| array      | a collection of a single data type indexed by integer subscripts for use in numerical methods and statistical calculations, provided by the NumPy module | see examples below                                               |

The latter array is of primary interest in this text, although all are integral to general applications of Python. An array, also called `ndarray` as the NumPy class name, can be created from a list or tuple using the `np.array` function. Here is an example:

```
import numpy as np
x = np.array( [ 12.2, 10.9, 13.6, 8.4, 11.1 ] )
```

The variable `x` represents a one-dimensional array. We can display it as

```
x
array([12.2, 10.9, 13.6, 8.4, 11.1])
```

A two-dimensional array can be created as a list of lists follows

```
A = np.array( [ [ 2, 4 ], [ 1, 3 ] ] )
```

and displayed as

```
A
array([[2, 4],
       [1, 3]])
```

Python and the NumPy module also provide a *matrix* object that facilitates linear algebra manipulations and calculations. A matrix can be created similar to an array and, alternately, using a style similar to that of the MATLAB software. It also allows us to distinguish row and column vectors. Here are examples:

```
A = np.matrix(' 2 4 ; 1 3 ')
A
matrix([[2, 4],
        [1, 3]])
```

Notice that the argument to the `np.matrix` function is a string (enclosed in ').

In creating arrays with the `np.array` function, there is no distinction as to a row vector or column vector. With the `np.matrix` function, there is such a distinction.

```
x = np.matrix( ' 12.2 10.9 13.6 8.4 11.1 ' )
```

```

x
matrix([[12.2, 10.9, 13.6, 8.4, 11.1]])

w = np.matrix( ' 12.2 ; 10.9 ; 13.6 ; 8.4 ; 11.1 ' )

w
matrix([[12.2],
        [10.9],
        [13.6],
        [ 8.4],
        [11.1]])

```

You can see that Python stores these as lists within a list. As we have mentioned, the array and matrix objects will facilitate our calculations.

The Variable Explorer presents the variables we have created.

|   |         |        |                             |
|---|---------|--------|-----------------------------|
| A | int32   | (2, 2) | [[2 4]<br>[1 3]]            |
| w | float64 | (5, 1) | [[12.2]<br>[10.9]           |
| x | float64 | (1, 5) | [[12.2 10.9 13.6 8.4 11.1]] |

Note that the size of the arrays is given in (rows, columns) format, and that w is shown as 5 rows and 1 column. Also, the type of data stored is shown. The content of w is not displayed completely, but, if you double-click that entry, you will see

| w - NumPy array |      |
|-----------------|------|
|                 | 0    |
| 0               | 12.2 |
| 1               | 10.9 |
| 2               | 13.6 |
| 3               | 8.4  |
| 4               | 11.1 |

The rows are indexed 0 to 4 and the single column is indexed 0. This is an important observation: the indices or subscripts of arrays in Python start with zero, not 1 as you might expect from mathematical description. This is similar to the C language and other software such as Excel's VBA.<sup>5</sup> The 1 origin is found in the MATLAB software and Fortran language.

Illustrations of subscripting (or indexing):

```

w[2,0]    (3rd row, 1st column)
13.6

A[0,1]    (1st row, 2nd column)
4

```

Note that the subscripts are enclosed in brackets [ ].

The NumPy module includes functions to create special arrays, and these can be imbedded in the `matrix` function to create similar matrices. One example is to fill an array with zeros and then convert it to a matrix.

```
Z = np.zeros((5,3))
Z
array([[0., 0., 0.],
       [0., 0., 0.],
       [0., 0., 0.],
       [0., 0., 0.],
       [0., 0., 0.]])
Zm = np.matrix(Z)
Zm
matrix([[0., 0., 0.],
        [0., 0., 0.],
        [0., 0., 0.],
        [0., 0., 0.],
        [0., 0., 0.]])
```

Similar functions are `ones` and `eye`, the latter to create an identity array or matrix.

---

Page 26

---

```
O = np.ones((2,3))
O
array([[1., 1., 1.],
       [1., 1., 1.]])
I = np.eye(3,3)
I
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

## 2.2.3 The : Subscript and Subscript Ranges

There are occasions where we want to reference all rows or all columns of a matrix. We can use the `colon ( : )` to do this.

```
A[0,:]          (1st row, all columns)
matrix([[2, 4]])

A[:,1]          (all rows, 2nd column)
matrix([[4],
        [3]])
```

Note that we always must include a row and column index, even for a row or column vector.

We can also use the colon to select ranges of subscripts.

```
x[0,0:2]        (first row, first 2 elements)
matrix([[12.2, 10.9]])
```

**NOTE WELL:** This illustrates an important idiosyncrasy of Python lists. When you specify a range of an index, the result does not include the upper limit. That is, `0 : 2` only includes 0 and 1.

## 2.2.4 The arange, linspace, and logspace Functions

These three functions, part of the NumPy module, are used frequently to create arrays (or matrices) filled with sequences of numbers. The syntax for `arange` is

```
np.arange(x1,x2,dx)
```

and it is understood as, “start a sequence at `x1` and proceed in steps of `dx` to or exceeding `x2`, but do not include the last number of the sequence.” Here is an example:

```
x = np.arange(0,1,0.1)
```

and `x` will contain `[0. 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9]`, excluding the upper limit 1. A way to include the upper limit is to use

```
np.arange(x1,x2+dx,dx)
```

The general syntax for `linspace` is

```
np.linspace(x1,x2,n)      (n equally-spaced points from x1 to x2)
```

Here is an example:

```
d = np.linspace(1,100,8)
d
array([ 1.         , 15.14285714, 29.28571429, 43.42857143,
       57.57142857, 71.71428571, 85.85714286, 100.        ])
```

If the last argument, `n`, is left out, the default number is 50.

Alternately, we might want an array to be filled with the numbers evenly spaced by their logarithm. We can use the `logspace` function with the syntax

```
np.logspace(logx1,logx2,n)
```

and an example is

```
np.logspace(-1,2,4)      (4 points from 10-1 to 102 at log10 values -1,0,1,2)
array([ 0.1,   1. , 10. , 100. ])
```

The default log base is 10, and it is the most common, but you can use another, like 2.

```
np.logspace(1,6,6,base=2.0)
array([ 2.,  4.,  8., 16., 32., 64.])
```

Again, the default number of points is 50.

## 2.2.5 Character Strings

Text data play an important role in general applications with Python. Alphanumeric or *character strings*, as they are called, are represented by enclosing them in single or double quotation marks ( ' or " ).

```
f = 'Alison'
s = 'Krauss'
```

A *string* is a character array, and we can reference elements using an index or subscript.

```
s[2]
```

'a' again with 0 as the origin index, 2 indicates the 3<sup>rd</sup> character)

We can concatenate (paste together) strings using the + sign.

```
f+s
' AlisonKrauss'
```

or, perhaps better,

```
f+' '+s
'Alison Krauss'
```

If you have a long line of code in Python, you can split it over two or more lines by enclosing it in a set of parentheses or by using the backslash ( \ ) character. For example,

```
a = [ 1, 2, 3, 4, 5 \
, 6, 7, 8 ]

a
[1, 2, 3, 4, 5, 6, 7, 8]
```

When it comes to dividing a long string among two or more lines, adding a \ within the string will not get the job done. Here are a couple alternatives:

Splitting into substrings, using \

```
LincGetty = 'Four score and seven years ago, ' \
'our fathers brought forth, upon this continent, ' \
'a new nation ... '
```

Splitting into substrings, enclosing in ( )

```
LincGetty = ('Four score and seven years ago, '
'our fathers brought forth, upon this continent, '
'a new nation ... ')
```

With the result in both cases

```
LincGetty
'Four score and seven years ago, our fathers brought forth, upon this
continent, a new nation ... '
```

A number of built-in Python functions and methods are available to operate on strings.

 **Table 2.2** lists a few of the more commonly used ones.

**TABLE 2.2** Some useful string functions and methods.

| Functions          | Description                                                 |
|--------------------|-------------------------------------------------------------|
| len(s)             | function, number of characters in the string s              |
| str(x)             | function, convert the number x to a string                  |
| int(s)             | convert the string s to integer number                      |
| float(s)           | convert the string s to floating point number               |
| <b>Methods</b>     |                                                             |
| endswith(suffix)   | returns True if the string ends with suffix                 |
| startswith(prefix) | returns True if the string starts with prefix               |
| index(substring)   | returns the lowest index in the string containing substring |



| Functions              | Description                                                     |
|------------------------|-----------------------------------------------------------------|
| <code>upper()</code>   | converts letters in the string to all uppercase characters      |
| <code>lower()</code>   | converts letters in the string to all lowercase characters      |
| <code>isdigit()</code> | returns True if all the characters in the string are digits     |
| <code>isalpha()</code> | returns True if all the characters in the string are alphabetic |

The syntax for a method is `s.methodname(•)`. Here are a few examples:

```
len('How long is this string?')
24

str(3.14159)
'3.14159'

float('3.14159')
3.14159

s = 'my string'
s.startswith('my')
True

'abcde'.upper()
'ABCDE'

s.isalpha()      (the space in s is not an alphabetic character)
False
```

You can display string values (and other output) using the `print` command.

```
Pres = 'Washington Adams Jefferson Madison'
print(Pres)
Washington Adams Jefferson Madison
```

Note that the `'` characters are not included in the output. If you wish to display the names above on separate lines, use the *escape sequence* `\n`.

```
Pres1 = 'Washington\nAdams\nJefferson\nMadison'
print(Pres1)
Washington
Adams
Jefferson
Madison
```

## 2.3 MATHEMATICAL OPERATIONS

Operations with scalar numerical quantities are handled in a straightforward manner, similar to other computer languages. Python's arithmetic operators are

|    |                                  |
|----|----------------------------------|
| +  | addition                         |
| −  | subtraction and negation (unary) |
| *  | multiplication                   |
| /  | division (floating point)        |
| // | division (integer)               |
| %  | modulus (remainder)              |
| ** | exponentiation                   |

Arithmetic expressions are generally evaluated left to right and within parentheses. It is important also to recognize which of the operations above are completed before others, that is, the order of precedence from highest to lowest. This is

|             |         |
|-------------|---------|
| **          | highest |
| − (unary)   | •       |
| *, /, //, % | •       |
| +, −        | lowest  |

These operators will work in the IPython Console in calculator fashion. Try

```
import numpy as np
np.pi * 0.5**2 / 4
0.19634954084936207    (computes the area of a circle of diameter 0.5)
```

The order of operations, following the table above, is

```
0.5**2 ⇒ result1
np.pi * result1 ⇒ result2
result2 / 4 ⇒ final result
```

It is convenient in many calculations to use variables to store key quantities, like using memory registers in a hand calculator.

```
x = 3 ; y = 2 ; z = 1.5    (you can have multiple statements on one line using ; to separate)
− x ** y
−9                        (** comes first, followed by −)
```

If you want to force the negation to occur first, enclose that in parentheses.

```
( − x ) ** y
9
```

Given the algebraic expression,  $x^{y^z}$  we might expect the  $y^z$  to be computed first. In other words, repeated exponentiation would be evaluated right-to-left. That is the case in Python.

```

x ** y ** z
22.361590938430393

x ** ( y ** z )      (you can see that y**z comes first, right-to-left, same as the last)
22.361590938430393

( x ** y ) ** z      (use parentheses to enforce the left-to-right order)
27.0

```

Calculations can also involve complex quantities. If we define

```

a = 2 + 4j
b = 0.5 - 0.3j

```

Then,

```

1/a
(0.1-0.2j)

```

(to confirm:  $\frac{1}{2+4j} \frac{2-4j}{2-4j} = \frac{2-4j}{20} = 0.1 - 0.2j$ )

Also,

```

a * b
(2.2+1.4j)

```

and

```

a ** 2
(-12+16j)

```

(notice that complex results are displayed in parentheses)

We can also carry out calculations with matrix objects.

```

a = np.matrix(' 1 2 3 4 5 ')
a
matrix([[1, 2, 3, 4, 5]])

b = np.matrix(' 2 ; 4 ; 6 ; 8 ; 10 ')
b
matrix([[ 2],
        [ 4],
        [ 6],
        [ 8],
        [10]])

a*b
matrix([[110]])

```

(this is the inner product of a and b)  
(notice how the "matrix" name is included in the result display)

Alternately, using the print function,

```

print(a*b)
[[110]]

```

$(1 \times 5) * (5 \times 1) \Rightarrow (1 \times 1, \text{a scalar})$

Can we create the outer product of a and b?

```

b*a
matrix([[ 2, 4, 6, 8, 10],
        [ 4, 8, 12, 16, 20],
        [ 6, 12, 18, 24, 30],
        [ 8, 16, 24, 32, 40],
        [10, 20, 30, 40, 50]])

```

$(5 \times 1) * (1 \times 5) \Rightarrow (5 \times 5, \text{a square matrix})$

Yes, we can!

Let's consider calculations with a matrix like

```

A
matrix([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])

```

and redefine a and b as vectors

```
a = np.matrix(' 1 2 3 ')
b = np.matrix(' 4 ; 5 ; 6 ')
```

Here are some more examples:

```
a*A
matrix([[30, 36, 42]])

A*b
matrix([[ 32],
        [ 77],
        [122]])
```

and

```
A*a
ValueError: shapes (3,3) and (1,3) not aligned: 3 (dim 1) != 1 (dim 0)
```

The last line is only part of the error message, but it is the most meaningful part.  $(3 \times 3) \times (1 \times 3)$  does not align in the inner dimensions (1 and 3) here.

We can multiply matrices.

---

Page 31

---

```
A*A
matrix([[ 30,  36,  42],
        [ 66,  81,  96],
        [102, 126, 150]])
```

What about this?

```
A**2
matrix([[ 30,  36,  42],
        [ 66,  81,  96],
        [102, 126, 150]])      (same result as A*A)
```

The *transpose* of a matrix can be computed as follows:

```
A.transpose()
matrix([[1, 4, 7],
        [2, 5, 8],
        [3, 6, 9]])      (rows and columns are switched)
```

At times, we want to carry out an *array operation* rather than a matrix multiplication; in other words, we want to multiply each element of A with its corresponding element and provide the result. This is not that easily done in Python, but we can use the `multiply` function from the NumPy module.

```
np.multiply(A,A)
matrix([[ 1,  4,  9],
        [16, 25, 36],
        [49, 64, 81]])
```

A question that arises is whether these calculations can be made with array objects, in addition to matrix objects.

```
a = np.array([ 1, 2, 3 ])
b = np.array([ 4, 5, 6 ])

a*b
array([ 4, 10, 18])      (Using * with arrays performs element-wise multiplication.)
```

The *inner* or *dot product* is computed as

```
a.dot(b)      (This uses the dot method.)
32
```

The vector *cross product* is provided by the `CROSS` function.

```
np.cross(a,b)
array([-3,  6, -3])
```

As you may recall, the cross product is defined as

$$\mathbf{a} \times \mathbf{b} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix} = (a_2b_3 - a_3b_2)\mathbf{i} + (a_3b_1 - a_1b_3)\mathbf{j} + (a_1b_2 - b_1a_2)\mathbf{k}$$

There are two features of the IPython Console we should highlight. First, you can use the  $\uparrow$  key to recall previous commands. This is particularly convenient when you enter a command with an error. With the  $\uparrow$  key, you will see your previous command. You can correct it, press the Enter key, and proceed. Multiply  $\uparrow$  keystrokes take you back several commands.

Second, if you know the name of a Python function or method, you can get information on it by typing the name in the `help(•)` command. For example,

```
help(int)
Help on class int in module builtins:

class int(object)
| int(x=0) -> integer
| int(x, base=10) -> integer
|
| Convert a number or string to an integer, or return 0 if no arguments
| are given. If x is a number, return x.__int__(). For floating point
| numbers, this truncates towards zero.
```

---

Page 32

---

There is much more displayed. The above is only the first part.

There are more general help facilities available from the *Spyder Help menu*. Also, it is worth mentioning that, if you have a specific question regarding Python, entering it in an Internet browser search engine, like Google or Bing, will often locate an answer.

## 2.4 USE OF BUILT-IN FUNCTIONS

---

Python and its many modules have a rich collection of built-in functions. You can use the `help(•)` command, as we illustrated above, to find out more about them, if you know their names. Another example is

```
import math
help(math.log)
Help on built-in function log in module math:

log(...)
    log(x[, base])

    Return the logarithm of x to the given base.
    If the base not specified, returns the natural logarithm (base e) of x.
```

The base Python language does not have an extensive library of built-in functions for numerical calculations. Two that we use frequently are `abs(•)` and `round(•)`. For real number arguments, `abs(•)` returns the absolute value. For complex numbers, it returns the magnitude, that is,

$$\text{abs}(a + bj) = \sqrt{a^2 + b^2}$$

Here are two examples:

```
abs(-2.45)
2.45

abs(-6+4j)
7.211102550927978
```

The `round(•)` function converts a floating point number to the nearest integer. For a number midway between two integers, it rounds to the even integer. Examples:

```
round(-7.6)
-8

round(4.5)
4
```

Two other useful built-in functions that operate on multiple arguments are `max(•)` and `min(•)`.

```
x = 14 ; y = 22
max(x,y)
22

a = np.array([2, 7, 3, -4, 3.5])
min(a)
-4.0

max(a)
7.0
```

There are two modules, `math` and `cmath`, that provide many of the functions we require for scientific numerical calculations. The latter provides functions for complex number arguments. With the command,

```
import math
```

the following functions become available, amongst many.<sup>6</sup> Note: Each must be prefaced by `math.`, for example, `math.exp(x)`.

|                       |                |
|-----------------------|----------------|
| <code>sqrt(x)</code>  | $\sqrt{x}$     |
| <code>log(x)</code>   | $\ln(x)$       |
| <code>log10(x)</code> | $\log_{10}(x)$ |
| <code>log(x,b)</code> | $\log_b(x)$    |
| <code>exp(x)</code>   | $e^x$          |

Trigonometric functions: (where appropriate, arguments/results are in radians, not degrees)

`sin(x)`, `cos(x)`, `tan(x)`  
`asin(x)`, `acos(x)`, `atan(x)`, `atan2(x,y)`

The `atan(x)` function returns a result in quadrants I and IV, i.e.,  $-\pi/2$  to  $+\pi/2$ ; the `atan2(x,y)` function returns a four-quadrant result for  $\tan^{-1}(y/x)$ , i.e.,  $-\pi$  to  $+\pi$ .

Hyperbolic functions:

`sinh(x)`, `cosh(x)`, `tanh(x)`, `asinh(x)`, `acosh(x)`

Conversion functions:

`degrees(x)`      converts radians to degrees, that is,  $x \frac{180}{\pi}$   
`radians(x)`      converts degrees to radians

Here are several examples to try with the `math` module functions:

```
math.log10(100)
2.0

math.log(100)
4.605170185988092

math.log(100)/math.log(10)
2.0

math.log(100,10)
2.0

math.exp(math.log(1))
1.0

math.sin(math.radians(45))
0.7071067811865476

math.atan(3/(-2))      (in quadrant IV)
-0.982793723247329

math.atan2(3,-2)      (in quadrant II)
2.158798930342464
```

A common need in numerical calculations is to implement an engineering or scientific formula. Recall that the velocity of a free-falling bungee jumper can be computed with [Eq. \(1.9\)](#)

$$v = \sqrt{\frac{mg}{c_d}} \tanh\left(\sqrt{\frac{c_d g}{m}} t\right)$$

where  $v$  is the velocity in m/s,  $g$  is the acceleration due to gravity,  $9.81 \text{ m/s}^2$ ,  $m$  is the mass (kg),  $c_d$  is the drag coefficient (kg/m), and  $t$  is time (s).

If we just want to calculate a result once, a single Python statement will do, assuming the values of the parameters have all been set. For example,

```
g = 9.81 ; m = 68.1 ; cd = 0.25
t = 10
v = math.sqrt(m*g/cd)*math.tanh(math.sqrt(cd*g/m)*t)
print('{0:5.2f}'.format(v))
49.42
```

However, if we want to use the formula repeatedly, for example for different values of  $t$ , then it is advantageous to package the formula in a new function, a user-defined function. We will deal with user-defined functions in detail in later chapters, but it would be good to see an example at this point.

To do this, we must make use of the Editor window in Spyder:



```
temp.py
1 #-*- coding: utf-8 -*-
2 """
3 Spyder Editor
4
5 This is a temporary script file.
6 """
7
8
```



For now, we can replace the text already there by default with the following:

```
import math
def bungee(t):
    g = 9.81 ; m = 68.1 ; cd = 0.25
    v = math.sqrt(m*g/cd)*math.tanh(math.sqrt(cd*g/m)*t)
    return v
```

This code needs to be executed by clicking the Run button ► or pressing the  $F5$  key. The Console should display a runfile output. Then, this user-defined function `bungee(•)` can be used in the Console to generate results.

```
bungee(10)
49.42136691869133
```

Next, we will illustrate *vectorization* in Python with the aid of the NumPy module. We can create an array that contains values from 0 to 20 in steps of 2.

```
import numpy as np
tm = np.linspace(0,20,11)      ( Alternately, tm = np.arange(0,22,2) )
tm
Out[6]: array([ 0.,  2.,  4.,  6.,  8., 10., 12., 14., 16., 18., 20.]
```

Set the values of  $g$ ,  $m$  and  $cd$  in the Console.

```
g = 9.81 ; m = 68.1 ; cd = 0.25
```



Create a statement that uses the `np.tanh` function instead of the `math.tanh` function used in the user-defined function.<sup>7</sup>

```
v = math.sqrt(m*g/cd)*np.tanh(math.sqrt(cd*g/m)*tm)
```

This *vectorizes* the formula and transforms the `tm` array into the `v` array.

```
v
array([ 0.          , 18.72918885, 33.11182504, 42.07622706, 46.95749513,
       49.42136692, 50.61747935, 51.18714999, 51.45599493, 51.58232304,
       51.64156286])
```

Vectorization is also supported for matrix objects. We can convert the array `tm` to a matrix.

```
tmm = np.matrix(tm)
tmm
matrix([[ 0.,  2.,  4.,  6.,  8., 10., 12., 14., 16., 18., 20.]])
```

And then use it in the formula.

```
v = math.sqrt(m*g/cd)*np.tanh(math.sqrt(cd*g/m)*tmm)
```

with a matrix result.

Also, we can transpose `tmm` into a column vector and carry out the calculation to produce a column result.

```
tmmt = tmm.transpose()
tmmt
matrix([[ 0.],
       [ 2.],
       [ 4.],
       [ 6.],
       [ 8.],
       [10.],
       [12.],
       [14.],
       [16.],
       [18.],
       [20.]])

v = math.sqrt(m*g/cd)*np.tanh(math.sqrt(cd*g/m)*tmmt)
v
matrix([[ 0.          ],
       [18.72918885],
       [33.11182504],
       [42.07622706],
       [46.95749513],
       [49.42136692],
       [50.61747935],
       [51.18714999],
       [51.45599493],
       [51.58232304],
       [51.64156286]])
```

As already mentioned, these results can be presented in a more compact format.

## 2.5 GRAPHICS

---

Python allows graphs to be created quickly and conveniently using the `pylab` interface to the Matplotlib module. We will introduce more features of Matplotlib in later chapters.

For example, to create a plot of  $v$  versus  $tmmt$  from the previous section, enter in the Console:

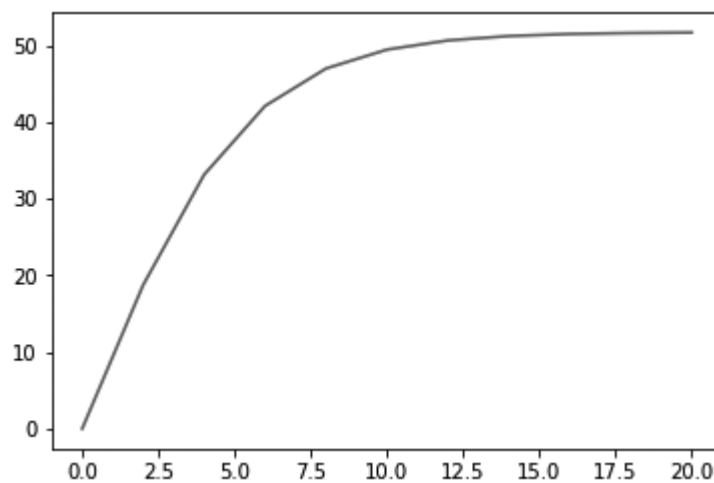
```
import pylab
pylab.plot(tmmt,v)
[<matplotlib.lines.Line2D at 0x18404051fd0>]
```

The plot can be seen under the Plots tab in the Explorer Window.

---

Page 36

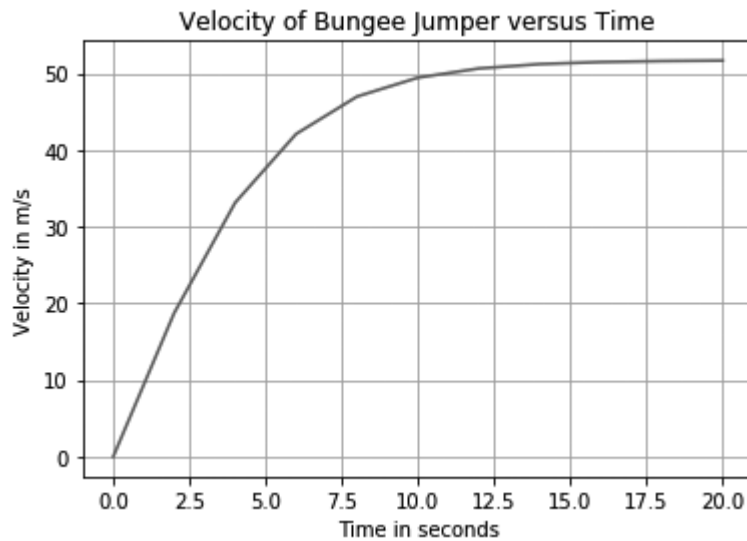
---



You can customize the graph by creating a set of statements in the Editor as shown below.

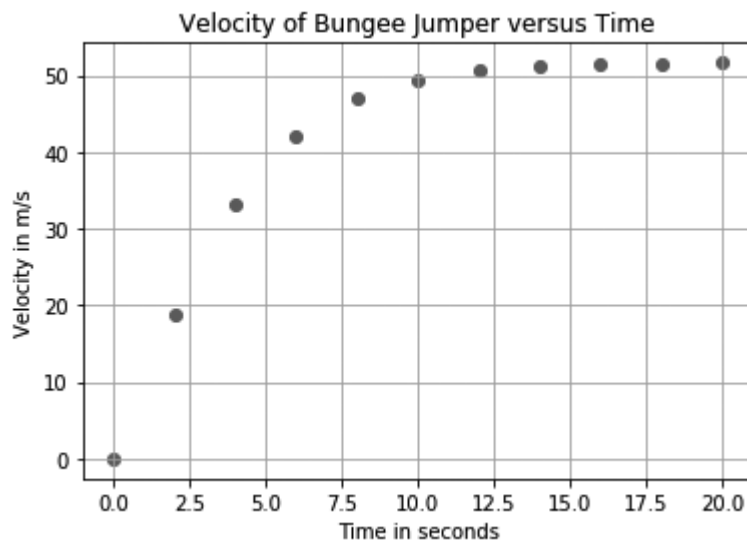
```
import math
import numpy as np
import pylab
g = 9.81 ; m = 68.1 ; cd = 0.25
tm = np.linspace(0,20,11)
v = math.sqrt(m*g/cd)*np.tanh(math.sqrt(cd*g/m)*tm)
pylab.plot(tm,v)
pylab.title('Velocity of Bungee Jumper versus Time')
pylab.xlabel('Time in seconds')
pylab.ylabel('Velocity in m/s')
pylab.grid()
```

When you run this program, the plot is displayed in the Explorer window above the Console by selecting the Plot tab.



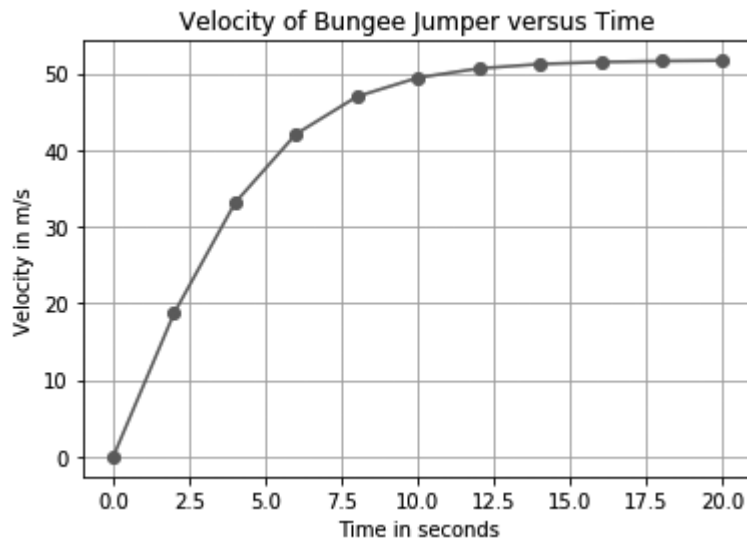
This plot displays a solid blue line by default. If you want to plot each point with a marker symbol, you can use the `scatter` function instead. However, markers are generally reserved for plotting data and not the result of an analytical formula.

```
pylab.scatter(tm,v)
```



You can add markers to the plot with the line by adding the `marker` argument.

```
pylab.plot(tm,v,marker='o')
```



The codes for different markers are shown in the table below.

| Character | Symbol | Description |
|-----------|--------|-------------|
| .         | •      | point       |
| o         | ○      | circle      |
| +         | +      | plus sign   |
| x         | ×      |             |
| D         | ◇      | diamond     |
| v         | ▽      | del         |
| ^         | △      | triangle    |
| s         | □      | square      |

Line and marker colors can be adjusted by the `color` argument. The table here shows the color choices.

| Character | Color   |
|-----------|---------|
| k         | black   |
| b         | blue    |
| g         | green   |
| r         | red     |
| c         | cyan    |
| m         | magenta |
| y         | yellow  |
| w         | white   |

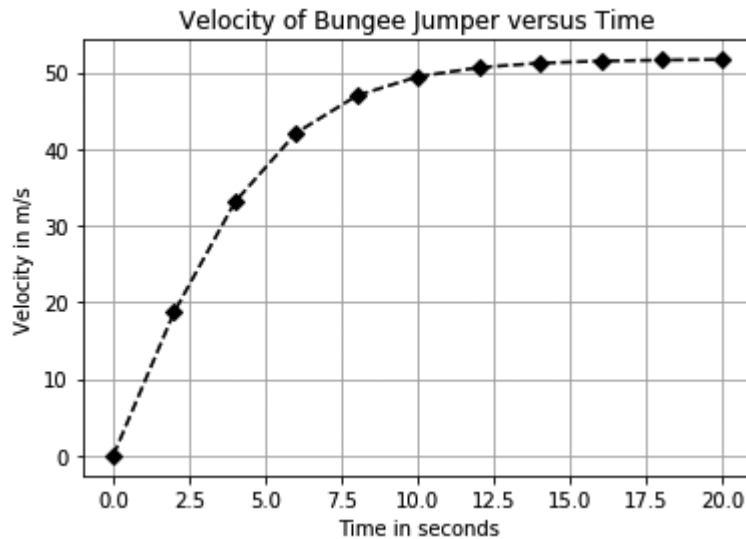
The style of lines can be set by the `linestyle` argument. This table shows the choices.

| Character | Style    |
|-----------|----------|
| -         | solid    |
| --        | dashed   |
| :         | dotted   |
| -.        | dash-dot |

By modifying the `pylab.plot` command to

```
pylab.plot(tm,v,ls='--',c='k',marker='D')
```

we get the result



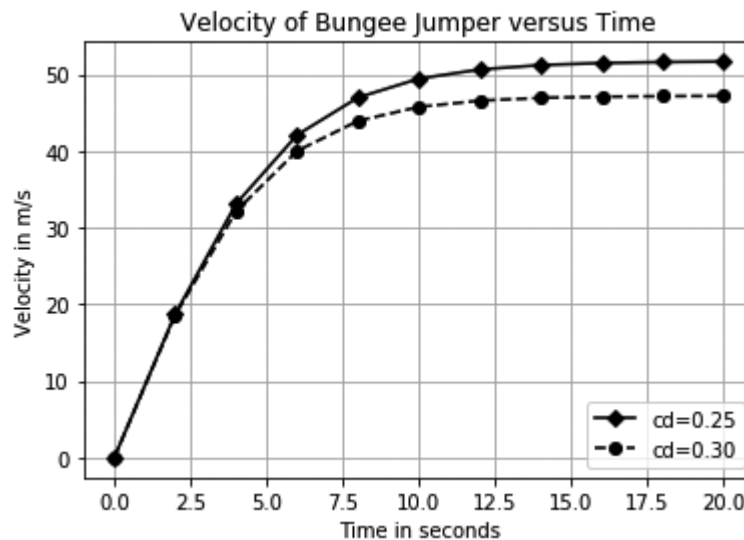
You will note that we used the accepted abbreviations for `linestyle` (`ls`) and `color` (`c`) above. In addition, the more general Matplotlib module allows greater control over the plot format. Later on that.

We can plot two curves on a graph and provide a legend. Perhaps, we would want to study what happens when the drag coefficient,  $c_d$ , increases by 20% to 0.30.

```
import math
import numpy as np
import pylab
g = 9.81 ; m = 68.1 ; cd = 0.25
tm = np.linspace(0,20,11)
v = math.sqrt(m*g/cd)*np.tanh(math.sqrt(cd*g/m)*tm)
cd = 0.30
v1 = math.sqrt(m*g/cd)*np.tanh(math.sqrt(cd*g/m)*tm)
pylab.plot(tm,v,ls='--',c='k',marker='D',label='cd=0.25')
pylab.plot(tm,v1,ls='--',c='k',marker='o',label='cd=0.30')
pylab.title('Velocity of Bungee Jumper versus Time')
pylab.xlabel('Time in seconds')
pylab.ylabel('Velocity in m/s')
```

```
pylab.legend(loc='lower right')
pylab.grid()
```

You can see below that the terminal velocity drops by about 8 m/s.



You can right-click the plot and select Copy Image, then paste the plot elsewhere, for example, into a document or presentation file. You can also use the shortcut Ctrl-C to copy the plot.

It is good practice to save your program code frequently in the Editor window, using  on the toolbar or File ⇒ Save on the menu or the Ctrl-S shortcut key combination. When the current code is saved, the toolbar button will appear faded out, . When you have multiple lines of code, you will find it more convenient to use the Editor window and not the Console.

Apart from line plots, there are other styles available via the `pylab` interface and many more in the Matplotlib module.

We will illustrate another example here that is common for engineers and scientists to use, the *histogram plot*. A histogram is typically based on a fairly large set of experimental data. In Python, we can generate such a simulated set internally or read data from an external source. We will illustrate the former here.

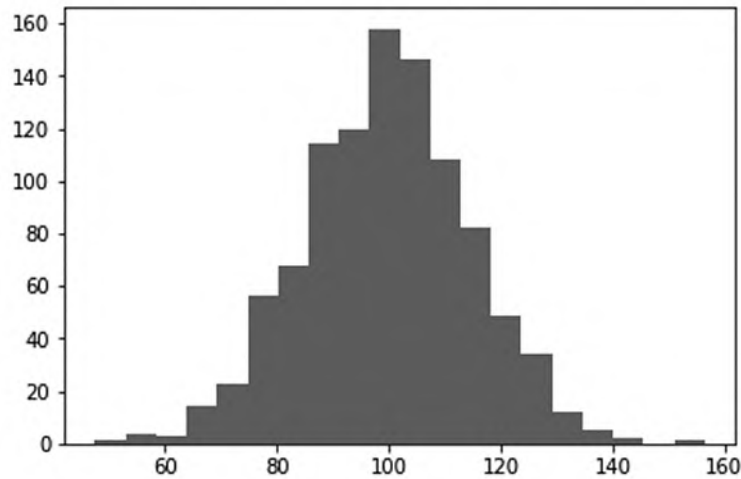
Open another file window in the Editor [ File ⇒ New file..., Ctrl-N, or  on the toolbar. ].

We will start by creating a set of data using the `random` module.

```
import pylab
import random
testdata = [] # start with an empty list
for i in range(1000): # repeat this loop 1000 times
    # add a random number to testdata
    testdata.append(random.normalvariate(100,15))
pylab.hist(testdata,bins=20)
```

Notice that we have added three comments, each starting with the `#` character. This is good practice and helps explain the code. Also, we have introduced a “count-controlled” *for loop* using the `range` type. We

will see much more about for loops, along with other program structures, in later chapters. The `range` type here provides 1000 integers from 0 to 999. This code also shows how to build a list, starting with an empty one, and using the `append` method to add an entry to the end of the list. Finally, we use `pylab`'s `hist` function to create the histogram plot, as a typical bar chart.



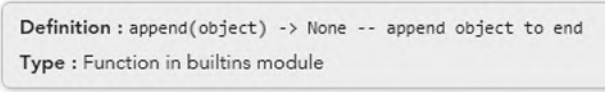
Along the way in the text, we will discover many more plotting capabilities of the `pylab` interface and its parent module, `Matplotlib`. These will include plotting contours on the complex plane, log-log and semilog plots, three-dimensional plots and contour plots. As described next, a variety of resources are available to learn about these as well as other Python capabilities.

## 2.6 OTHER RESOURCES

---

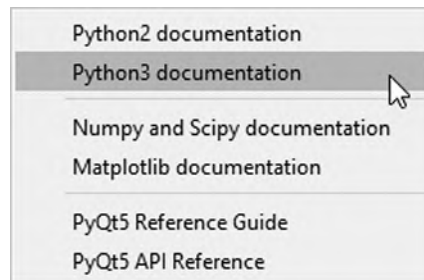
The earlier sections of this chapter are focused on those features of Python that we will be using in the remainder of the book. As such, this chapter is far from a comprehensive introduction to Python and its broad capabilities. If you are interested in learning Python in depth, especially from the software development perspective, you should consult one of the excellent texts devoted entirely to Python, for example, Hill, 2015; Nagar, 2018; Ramalho, 2015; VanderPlas, 2017. Also, for beginners, consider Tale, 2016 and 2017.

There are various levels of “help” available on Python and its supporting modules. We have already introduced the `help(•)` function used directly in the IPython Console. This requires that you know the command for which you want more information. In the Spyder interface, you can also place your cursor on or just before a keyword and press the Ctrl-I combination. For example, in the program above, placing the cursor on the `append` method and keying Ctrl-I yields in the window in the upper right:



```
Definition : append(object) -> None -- append object to end
Type : Function in builtins module
```

More detailed information is available via the Help menu and Online Documentation choice.



For the Python3 choice, you will get



# Python 3.7.3 documentation

Welcome! This is the documentation for Python 3.7.3.

## Parts of the documentation:

### What's new in Python 3.7?

*or all "What's new" documents since 2.0*

### Tutorial

*start here*

### Library Reference

*keep this under your pillow*

### Language Reference

*describes syntax and language elements*

### Python Setup and Usage

*how to use Python on different platforms*

### Python HOWTOs

*in-depth documents on specific topics*

### Installing Python Modules

*installing from the Python Package Index & other sources*

### Distributing Python Modules

*publishing modules for installation by others*

### Extending and Embedding

*tutorial for C/C++ programmers*

### Python/C API

*reference for C/C++ programmers*

### FAQs

*frequently asked questions (with answers!)*

and you can go on from there.

As we have mentioned before, using your Internet browser and search engine, a posed question on Python will often direct you to a source for the answer. Here is an example:

how do I make a log-log plot in Python?

producing this item among many:

### Python Log Plots | plotly

<https://plot.ly/python/log-plot/> ▼

How to make Log plots in Python with Plotly. ... Get started by downloading the client and reading the primer. You can set up Plotly to work in online or offline ...

and you can go on from there. In many instances, a good approach is to look for Python code that carries out a calculation similar to what you are attempting. And, of course, as you gain more experience in time, your own knowledge base will be adequate for you to tackle new tasks and problems.

## EXPLORATORY DATA ANALYSIS

### Background

Your textbooks are filled with formulas developed in the past by renowned engineers and scientists. Although these are of great utility, we must often supplement these relationships by collecting and analyzing data. Sometimes this leads to a new formula; however, prior to arriving at a final predictive equation, we usually “play with” or explore the data by performing calculations and developing plots. In most cases, our intent is to gain insight into the patterns and mechanisms hidden in the data.

In this case study, we will illustrate how Python facilitates such exploratory data analysis. We will do this by estimating the drag coefficient of a free-falling human based on [Eq. \(2.1\)](#) and the data from [Table 2.1](#). However, beyond merely computing the drag coefficient, we will use Python’s graphical capabilities via pylab to discern patterns in the data.

### Solution

The data from [Table 2.1](#) along with gravitational acceleration can be entered in the Editor window as

```
"""
Case Study
Exploratory Data Analysis
D. E. Clough
4/26/2019
"""
import numpy as np
import pylab
m = np.matrix('83.6,60.2,72.1,91.1,92.9,65.3,80.9')
vt = np.matrix('53.4,48.5,50.9,55.7,54.0,47.7,51.1')
g = 9.81
cd = g*m/np.power(vt,2)
print(cd)
```

When you run (F5) the code, the Console displays

```
[[0.28760258 0.2510626 0.27300381 0.28805605 0.31253395 0.28154345 0.30393151]]
```

Comments regarding the code above:

We have included a header delimited by the symbols `"""`. This is a good practice for documentation’s sake. We cannot use the exponentiation operator `**` to square the elements of a matrix/vector. That operator will attempt a matrix multiplication. Instead, we use the `power` function from the NumPy module. The divide operator `/` handles the elementwise division with no problem.

We can add a few statements to the code to compute and display a few sample statistics for the `cd` vector.

```
cdavg = np.average(cd)
cdmin = np.min(cd)
cdmax = np.max(cd)
print('cd statistics')
print('average =', '{0:5.3f}'.format(cdavg))
print('minimum = ', '{0:5.3f}'.format(cdmin))
print('maximum = ', '{0:5.3f}'.format(cdmax))
```

And the results are

```
cd statistics
```

```
average = 0.285
minimum = 0.251
maximum = 0.313
```

Thus, the average drag coefficient is 0.285, and there is a range from 0.251 to 0.313 kg/m.

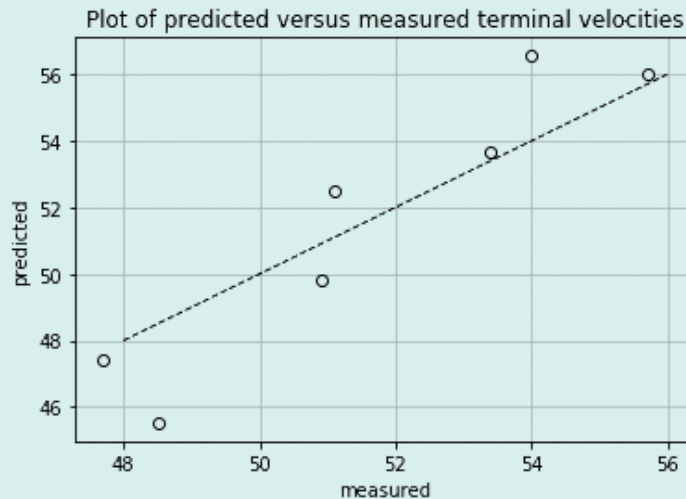
Now, let's start to play with these data by using [Eq. \(2.1\)](#) to make a prediction of the terminal velocity based on the average drag coefficient above. Below, we illustrate another method to format the print result.

```
vpred = np.sqrt(m*g/cdavg)
np.set_printoptions(precision=3)
print(vpred)

[[53.607 45.49 49.783 55.96 56.51 47.377 52.734]]
```

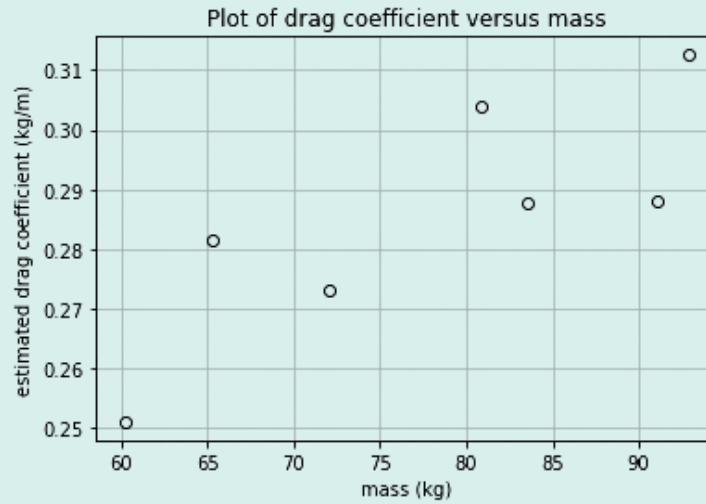
We can plot these values versus the actual measured terminal velocities. We will also superimpose a dashed line indicating perfect agreement between predicted and actual (the 1:1 or 45° line) to help assess the results.

```
pylab.plot(vt,vpred,c='k',marker='o',mfc='w',mec='k')
pylab.plot([48,56],[48,56],ls='--',c='k',lw=1.)
pylab.grid()
pylab.title('Plot of predicted versus measured terminal velocities')
pylab.xlabel('measured')
pylab.ylabel('predicted')
```



Above, we add the `mfc` (marker fill color) and `mec` (marker edge color) specifications to the first `plot` command. We also add the `lw` (linewidth) argument to the second `plot` command. The plot indicates some agreement but considerable variability between predicted and measured terminal velocities. Also, it appears that our predictions are low at lower velocities and high at the high end. This raises a suspicion that the drag coefficients may have a trend. We can investigate that with a plot of estimated drag coefficient versus mass.

```
pylab.figure()
pylab.plot(m,cd,marker='o',mfc='w',mec='k')
pylab.xlabel('mass (kg)')
pylab.ylabel('estimated drag coefficient (kg/m)')
pylab.title('Plot of drag coefficient versus mass')
pylab.grid()
```



The plot above suggests that rather than being constant, the drag coefficient appears to increase with the mass of the jumper. Based on this result, you would likely conclude that your model needs to be improved. At the least, it might motivate you to conduct further experiments with a larger number of jumpers to confirm your preliminary finding.

In addition, the result might also stimulate you to go to the fluid mechanics literature and learn more about the science of drag. As previously described in [Sec. 1.4](#), you would discover that the parameter  $c_d$  is actually a lumped drag coefficient that along with the true drag includes other factors such as the jumper's frontal area and the air density:

$$c_d = \frac{C_D \rho A}{2} \quad (2.2)$$

where  $C_D$  = a dimensionless drag coefficient,  $\rho$  = air density ( $\text{kg/m}^3$ ), and  $A$  = frontal area ( $\text{m}^2$ ), which is the area projected on a plane normal to the direction of the velocity.

Assuming that the densities were relatively constant during the data collection (a reasonable assumption if the jumpers took off from the same elevation on the same day), [Eq. \(2.2\)](#) suggests that heavier jumpers might have larger frontal areas. This hypothesis could be investigated and substantiated by measuring the frontal areas of individuals of varying masses.

# PROBLEMS

---

**2.1** Predict what will be displayed when the following Python commands are executed. Then try them. Explain how they work. Use help resources as needed.

```
import numpy as np
A = np.matrix(' 1,2,3 ; 2,4,6 ; 3,2,1 ')
print(A)

At = A.transpose()
print(At)

A1 = A[:,2]
print(A1)

nm = np.sum(np.diag(A))
print(nm)

Ad = np.delete(A,1,0)
print(Ad)
```

**2.2** Create program statements in Python that compute a vector of  $y$  values based on the following formulas:

(a)  $y = \frac{6t^3 - 3t - 4}{8 \sin(5t)} \quad 0.1 \leq t \leq 0.25$

(b)  $y = \frac{3t - 2}{4t} - \frac{\pi}{2}t \quad 1 \leq t \leq 5$

where  $t$  is a vector of 10 values in the range prescribed. Create a plot of  $y$  versus  $t$  for each of the above.

**2.3** Write and test Python statements to compute and display a vector of  $x$  values using the following equation:

$$x = \frac{y(a + bz)^{1.8}}{z(1 - y)}$$

where  $y$  and  $z$  are vectors of same length as  $x$ . Experiment with different values of  $a$  and  $b$  and different ranges of  $y$  and  $z$ . Comment on any issues you discover.

**2.4** What will be displayed when the following Python statements are run?

```
(a) import numpy as np
    A = np.matrix([[1,2],[3,4],[5,6]])
    print(A)
    A2 = A[2,:].transpose()
    print(A2)

(b) y = np.array(np.arange(0,7,1.5)).transpose()
    print(y)

(c) a = 2 ; b = 8 ; c = 4
    print(a/b*c)
    print(a/b/c)
```

```
print(a/(b*c))
```

**2.5** The “humps” function,  $f(x)$ , defines a curve that has two maxima (peaks) of unequal height over an interval  $a \leq x \leq b$ . An example is

$$f(x) = \frac{1}{(x - 0.3)^2 + 0.01} + \frac{1}{(x - 0.9)^2 + 0.04} - 6 \quad 0 \leq x \leq 2$$

Write Python code to compute and plot  $f(x)$  versus  $x$  for 100 values of  $x$  in the range specified.

**2.6** Use the NumPy `linspace` function to create vectors equivalent to the following Python statements:

(a) `np.arange(4,35,6)`

(b) `np.arange(-4,2)`

**2.7** Use NumPy’s `arange` function to create vectors identical to the following created with the `linspace` function:

(a) `np.linspace(-2,1.5,8)`

(b) `np.linspace(8,4.5,8)`

**2.8** The NumPy function, `linspace(a,b,n)`, generates an array of  $n$  equally spaced points between  $a$  and  $b$ . Describe the `arange` function that will generate the same result. Test your formulation for  $a = -3$ ,  $b = 5$ , and  $n = 6$ .

**2.9** The following Python statements create a matrix  $A$ :

```
import numpy as np
a1 = np.matrix(' 3 2 1 ')
a2 = np.matrix(np.arange(0,1.1,.5))
a3 = np.matrix(np.linspace(6,8,3))
A = np.vstack((a1,a2,a3))
```

(a) Write out the resulting matrix.

(b) Write two distinct Python commands to matrix multiply the second row by the third column of  $A$ . Confirm that these yield the same result, a scalar quantity.

**2.10** The following equation can be used to compute values of  $y$  as a function of  $x$ :

$$y = b e^{-ax} \sin(bx) (0.0012x^4 - 0.15x^3 + 0.075x^2 + 2.5x)$$

where  $a$  and  $b$  are parameters.

- (a) Write Python code to compute a vector  $y$  for  $a = 2$ ,  $b = 5$ , and  $x$  as a vector with values from 0 to  $\pi/2$  in increments  $\Delta x = \pi/40$ .
- (b) Compute the vector  $z = y^2$  with element-by-element-wise operations.
- (c) Combine  $x$ ,  $y$ , and  $z$  into a 3-column matrix  $W$ . Display  $W$  using a “precision=3” format.
- (d) Generate a plot of  $y$  and  $z$  versus  $x$ . Include a title, axis labels, and a legend. For the  $y$  versus  $x$  curve, format a solid blue line. For the  $z$  versus  $x$  curve, choose a magenta dashed line. Include

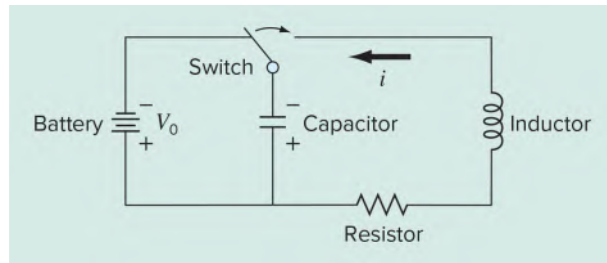
grid lines.

**2.11** A simple electrical circuit consisting of a resistor, a capacitor, and an inductor (coil) is depicted in  **Fig. P2.11**. When the switch is closed, the voltage potential across the capacitor as a function of time is given by

$$V(t) = V_0 e^{-\left(\frac{R}{2L}\right)t} \cos\left(\sqrt{\frac{1}{LC} - \left(\frac{R}{2L}\right)^2} t\right)$$

where  $t$  = time (s),  $V_0$  = initial voltage (volts),  $R$  = resistance (ohms),  $L$  = inductance (henrys),  $C$  = capacitance (farads).<sup>8</sup> Use Python to generate a plot of this function from  $t = 0$  to 0.8 s, given that  $V_0 = 10$  volts,  $R = 60$  ohms,  $L = 9$  henrys, and  $C = 50$  microfarads.

**FIGURE P2.11**



**2.12** The standard normal probability density function is a bell-shaped curve that is described by

$$f(z) = \frac{1}{\sqrt{2\pi}} e^{-\frac{z^2}{2}}$$

Develop Python commands to generate a plot of this function from  $z = -4$  to 4. Use enough points to provide a smooth curve plot. Label the ordinate as frequency and the abscissa as  $z$ . Compute and display the maximum value of  $f(z)$ . Where does it occur?

**2.13** If a force  $F$  (N) is applied to compress a spring, the spring's displacement  $x$  (m) can often be modeled simply by Hooke's Law:

$$F = kx$$

where  $k$  is the spring constant (N/m). The potential energy stored in the spring,  $U$  (J), can then be computed as

$$U = \frac{1}{2} kx^2$$

Five springs are tested and the following data compiled:

|                    |       |       |       |       |       |
|--------------------|-------|-------|-------|-------|-------|
| <b><i>F</i>, N</b> | 14    | 18    | 8     | 9     | 13    |
| <b><i>x</i>, m</b> | 0.013 | 0.020 | 0.009 | 0.010 | 0.012 |

Use Python to store the  $F$  and  $x$  values in vectors. The compute vectors of the spring constants,  $k$ , and the potential energies,  $U$ . Use the `np.max` function to determine the maximum potential energy.

**2.14** The density of pure water ( $\text{kg/m}^3$ ) at atmospheric pressure varies with temperature between the freezing point ( $0^\circ\text{C}$ ) and the boiling point ( $100^\circ\text{C}$ ). This relationship can be accurately described by a 4<sup>th</sup>-order polynomial.

$$\rho = 999.904 + 0.0508007T - 0.00748423T^2 + 4.11472 \times 10^{-5}T^3 - 1.2329 \times 10^{-7}T^4$$

where  $\rho$  is the density in  $\text{kg/m}^3$  and  $T$  is the temperature in  $^\circ\text{C}$ . Use Python to generate a vector  $TF$  of Fahrenheit temperatures from 32 to 212  $^\circ\text{F}$  in 3.6  $^\circ\text{F}$  increments. Convert this vector to another vector  $TC$  of Celsius temperatures. Then, compute a vector of densities based on the formula above. Create a plot of  $\rho$  versus  $TC$ . Determine at what temperature the maximum density occurs. Recall that  $T[^\circ\text{C}] = (T[^\circ\text{F}] - 32)/1.8$ .

**2.15** Manning's equation can be used to compute the velocity of water in a rectangular open channel:

$$U = \frac{\sqrt{S}}{n} \left( \frac{BH}{B + 2H} \right)^{2/3}$$

where  $U$  = velocity (m/s),  $S$  = channel slope,  $n$  = roughness coefficient,  $B$  = channel width (m), and  $H$  = water depth (m). The following data are available for five channels:

| <b><i>n</i></b> | <b><i>S</i></b> | <b><i>B</i></b> | <b><i>H</i></b> |
|-----------------|-----------------|-----------------|-----------------|
| 0.035           | 0.0001          | 10              | 2.0             |
| 0.020           | 0.0002          | 8               | 1.0             |
| 0.015           | 0.0010          | 20              | 1.5             |
| 0.030           | 0.0007          | 24              | 3.0             |
| 0.022           | 0.0003          | 15              | 2.5             |

Create Python statements to store these values in a matrix  $P$  where each row represents one of the channels and each column represents one of the parameters. Add one or more statements to compute the velocities in a column vector  $U$  based on the values in the parameter matrix. Display the resulting vector.

**2.16** It is accepted general practice in engineering and science to plot analytical mathematical equations using line segments and experimental data as markers. Occasionally, interconnecting lines are added to data plots for the purpose of pattern recognition. Here are a few data for concentration,  $c$ , versus time,  $t$ , for the photodegradation of aqueous bromine.



|                      |     |     |     |     |     |     |
|----------------------|-----|-----|-----|-----|-----|-----|
| <b><i>t</i>, min</b> | 10  | 20  | 30  | 40  | 50  | 60  |
| <b><i>c</i>, ppm</b> | 3.4 | 2.6 | 1.6 | 1.3 | 1.0 | 0.5 |

These data can be modeled by the following function:

$$c = 4.84e^{-0.034t}$$

Use Python to create a plot displaying both the data (using diamond-shaped, green-filled markers with black edges) and the function (using a green, dashed line). Plot the function for the range  $0 \leq t \leq 70$  min. Include grid lines.

**2.17** The `pylab` interface to the Matplotlib module allows for semilog and log-log plots. Instead of using the `pylab.plot` function, you can use, for example, the `pylab.semilogy` function to apply a log scale to the ordinate. Modify your Python code from [Prob. 2.16](#) to create a `semilogy` plot. Observe the resulting plot and explain its features.

**2.18** Here are some wind tunnel data for force,  $F$ , versus velocity,  $v$ .

Page 46

|                      |    |    |     |     |     |      |     |      |
|----------------------|----|----|-----|-----|-----|------|-----|------|
| <b><i>v</i>, m/s</b> | 10 | 20 | 30  | 40  | 50  | 60   | 70  | 80   |
| <b><i>F</i>, N</b>   | 25 | 75 | 380 | 550 | 610 | 1220 | 830 | 1450 |

These data can be described by the following function:

$$F = 0.274 v^{1.984}$$

Use Python to create a plot displaying both the data (using circular magenta markers) and the function (using a thin black dotted line). Plot the function over the range  $0 \leq v \leq 100$  s, and label the axes. Include grid lines.

**2.19** The `pylab.loglog` function is similar to the `pylab.plot` function except that logarithmic scales are used for both the  $x$  and  $y$  axes. Create a log-log plot of the data and function from

[Prob. 2.18](#). Comment on the resulting plot.

**2.20** The Maclaurin series expansion for the cosine is

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \frac{x^8}{8!} - \dots$$

Use Python to create a plot of the cosine using the `np.cos` function (solid black line) along with a curve of the series expansion (black dashed line) up to and including the  $x^8/8!$  term. Add a legend to your plot. Use the `math.factorial` function for the various  $n!$  evaluations. Make the range of the function  $0 \leq x \leq 3\pi/2$ .

**2.21** You contact the jumpers used to generate the data in [Table 2.1](#) and measure their frontal areas. The resulting values, which are ordered in the same sequence as the corresponding values in

 **Table 2.1**, are

| $A, m^2$ | 0.455 | 0.402 | 0.452 | 0.486 | 0.531 | 0.475 | 0.487 |
|----------|-------|-------|-------|-------|-------|-------|-------|
|----------|-------|-------|-------|-------|-------|-------|-------|

- If the air density is  $\rho = 1.223 \text{ kg/m}^3$ , use Python to compute values of the dimensionless drag coefficient,  $C_D$ .
- Determine the average, minimum, and maximum of the resulting  $C_D$  values.
- Create two plots. The first is  $A$  versus  $m$ , and the second is  $C_D$  versus  $m$ . Include descriptive axis labels and titles on the plots. Include grid lines.

**2.22** The *Weibull probability distribution* is widely used to represent reliability in manufacturing processes. The density function for this distribution is given by

$$f(x) = \frac{\alpha}{\beta^\alpha} x^{\alpha-1} e^{-\left(\frac{x}{\beta}\right)^\alpha} \quad x \geq 0$$

where  $x$  is a random variable,  $\alpha$  is a shape parameter, and  $\beta$  is a scale parameter for the distribution. For a scale factor  $\beta = 1$ , the density becomes

$$f(x) = \alpha x^{\alpha-1} e^{-x^\alpha} \quad x \geq 0$$

- Create a plot of  $f(x)$  for the range of  $0 \leq x \leq 3$  with curves for  $\alpha = 5$  and  $\alpha = 2$ . Use different colors for the curves and provide a legend. Include grid lines.
- Python's NumPy module has a sub-module `random` that provides a `weibull` function for generating random numbers according to the distribution with  $\beta = 1$  and a given  $\alpha$  value. You can generate  $n$  random numbers with the statements

```
np.random.weibull(alpha,n)
```

Generate an array of 1000 random numbers for  $\alpha = 5$ . Plot a histogram chart for the results. Repeat the process for  $\alpha = 2$ . Comment on any differences you observe between the two histogram charts.

**2.23** Predict what will be displayed after the following commands are entered in the Console:

- `np.arange(6)`
- `np.arange(6,1,-1)`
- `np.linspace(1,6,6)`
- `np.linspace(6,1,6)`
- `np.logspace(-2,2,5)`

After you have made your predictions, test the commands. Note any differences from your predictions and explain what you have learned.

**2.24** The trajectory of an object can be modeled as

$$y = \tan(\theta_0)x - \frac{g}{2v_0^2 \cos^2(\theta_0)} x^2 + y_0$$

where  $y$  = elevation (m),  $\theta_0$  = initial or launch angle (rad),  $x$  = horizontal distance or range (m),  $g$  = gravitational acceleration ( $9.81 \text{ m/s}^2$ ),  $v_0$  = initial velocity (m/s), and  $y_0$  = initial elevation (m).

Program Python to find the trajectories for  $y_0 = 0 \text{ m}$  and  $v_0 = 28 \text{ m/s}$  for launch angles ranging from  $15^\circ$  to  $75^\circ$  in increments of  $15^\circ$ . Employ a range of horizontal distances from 0 to 80 m in increments of 5 m.

The results should be assembled in a matrix where the first dimension, the rows, correspond to the distances and the second dimension, the columns, correspond to the launch angles. Use this matrix to generate a plot of a family of curves of  $y$  versus  $x$  for the launch angles. Style each curve differently and include a legend describing the launch angles. As required, adjust the  $y$ -axis scale so the minimum is zero meters. Include grid lines.

**2.25** The temperature dependence of the rate of a first-order chemical reaction is described by the *Arrhenius equation*:

$$k = k_0 e^{-\frac{E}{RT}}$$

where  $k$  = reaction rate (1/s),  $k_0$  = pre-exponential (or frequency) factor (1/s),  $E$  = activation energy (J/mol),  $R$  = universal gas law constant ( $8.314 \text{ J/(mol} \cdot \text{K)}$ ), and  $T$  = absolute temperature (K).

A reaction is modeled with  $k_0 = 7 \times 10^{16} \text{ 1/s}$  and  $E = 1 \times 10^5 \text{ J/mol}$ . Write Python code to generate reaction rates for  $253 \leq T \leq 325 \text{ K}$ . Create a plot for  $k$  versus  $T$ . Using the `pylab.semilogy` function, create a second plot of  $\log_{10} k$  versus  $1/T$ . Comment on your results.

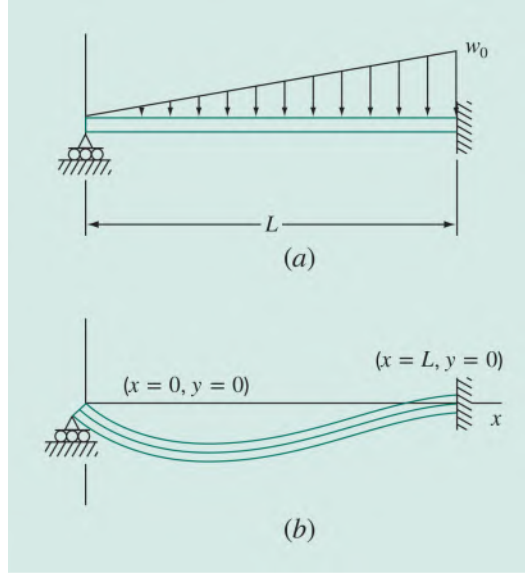
**2.26**  **Figure P2.26a** shows a uniform beam subjected to a linearly increasing load. The deflection of the beam,  $y(\text{m})$ , is depicted in  **Fig. 2.26b** and can be computed from the equation

$$y = \frac{w_0}{120EIL} (-x^5 + 2L^2x^3 - L^4x)$$

where  $E$  = modulus of elasticity ( $\text{N/m}^2$ ) and  $I$  = moment of inertia ( $\text{m}^4$ ).

---

**FIGURE P2.26**



Employ this equation and calculus to generate Python pylab plots of the following quantities versus distance along the beam,  $x$ .

(a) deflection,  $y$ ,

(b) slope,  $\theta(x) = \frac{dy}{dx}$

(c) moment,  $M(x) = EI \frac{d^2y}{dx^2}$

(d) shear,  $V(x) = EI \frac{d^3y}{dx^3}$

(e) loading,  $w(x) = -EI \frac{d^4y}{dx^4}$

Use the following parameter settings for your computations:  $L = 600$  cm,  $E = 50,000$  kN/cm<sup>2</sup>,  $I = 30,000$  cm<sup>2</sup>,  $w_0 = 2.5$  kN/cm, and  $\Delta x = 10$  cm. Handle units of measurement with care, converting to the SI standard. Include conversions in your code. Include grid lines, axis labels, and titles on your plots.

**2.27** The *butterfly curve* is given by the following parametric equations:

$$x = \sin(t) \left( e^{\cos(t)} - 2 \cos(4t) - \sin^5\left(\frac{t}{12}\right) \right)$$

$$y = \cos(t) \left( e^{\cos(t)} - 2 \cos(4t) - \sin^5\left(\frac{t}{12}\right) \right)$$

Generate values of  $x$  and  $y$  for  $0 \leq t \leq 100$  in steps of  $\Delta t = 1/16$ . Use Python's Matplotlib pylab interface to construct the following plots:

(a)  $x$  and  $y$  versus  $t$

(b)  $y$  versus  $x$

For the plot in (a), use different black line styles and include a legend. Make the plot in (b) square, for example, make the  $x$ - and  $y$ -axes equal. Create axes titles and a plot title for both (a) and (b). Include grid lines on your plots.

**2.28** The butterfly curve from  **Prob. 2.27** can also be represented in polar coordinates as

$$r = e^{\sin(\theta)} - 2 \cos(4\theta) - \sin^5\left(\frac{2\theta - \pi}{24}\right)$$

Using Python, generate values of  $r$  for  $0 \leq \theta \leq 8\pi$  with  $\Delta\theta = \pi/32$ . Use the `pylab.polar` plotting function to generate the polar plot of the butterfly function with a dashed blue line. Include grid lines. Explore the polar plot command using the help facilities.

# Programming in Python

## CHAPTER OBJECTIVES

The primary objective of this chapter is to learn how to write Python scripts to implement numerical methods. Specific objectives and topics covered are

- Learning how to create well-documented Python scripts in the Spyder Editor window and invoke them from the Console or run them from the Spyder menu.
- Learning how to construct simple functions in the Editor window.
- Understanding how functions and scripts differ.
- Developing good practices in documentation, including script headers, comments between lines of code, and explanatory notes at the ends of lines of code.
- Learning how to write Python code in the Editor window to prompt the user for input and to display results in the Console window.
- Gaining an understanding of how to read and write data files with Python.
- Understanding how to implement the structured programming elements of selection and iteration in Python.
- Gaining a better understanding of vectorization and why it is beneficial in writing concise and reliable Python programs.
- Learning how to pass, in addition to variable names and constant values, the names of other functions to a function to create more versatile Python programs.

## YOU'VE GOT A PROBLEM

In  **Chap. 1**, we used a force balance to develop a mathematical model to predict the fall velocity of a bungee jumper. This model took the form of the following differential equation:

$$\frac{dv}{dt} = g - \frac{c_d}{m} v|v|$$

where  $v$  = terminal velocity (m/s),  $g$  = gravitational acceleration (approximately = 9.81 m/s<sup>2</sup>),  $m$  = mass (kg), and  $c_d$  = drag coefficient (kg/m).

We also learned that an approximate numerical solution of this equation could be obtained employing Euler's method:

$$v_{i+1} = v_i + \frac{dv_i}{dt} \Delta t$$

This formula can be implemented repeatedly to compute velocity as a function of time. However, to obtain good accuracy, many small  $\Delta t$  steps must be taken. This would be extremely laborious and time consuming to implement by hand. However, with the aid of Python, such calculations can be performed easily.

So, our problem now is to figure out how to do this. This chapter will provide a thorough introduction to how Python script files can be used to obtain such solutions.

## 3.1 PYTHON SCRIPT FILES

As we have seen in [Chap. 2](#), a common way to operate in the Spyder/IPython integrated development environment (IDE) is to type single commands into the IPython Console. Creating Python scripts in the Editor window provides an alternate, advantageous way of performing operations that greatly expand Python's problem-solving capabilities. A Python script consists of a series of statements that can be run with a single command. Python scripts are saved as files with a `.py` extension. Scripts typically contain statements and program structures called functions, which also package statements.

### 3.1.1 Python Scripts

A script is merely a collection or series of Python statements that are saved as a file. Scripts are useful for retaining a series of commands that you want to execute on more than one occasion. They are also helpful when you make an error because you can edit the script, save it, and re-run it without having to type in all the commands again in the Console. The script can be executed by clicking the  button at the top of the Editor window, pressing the `F5` key, or, the long way, selecting Run from the menu and clicking Run.

#### EXAMPLE 3.1

##### Script File

##### Problem Statement

Develop a script file to compute the velocity of the free-falling bungee jumper for the case where the initial velocity is zero.

##### Solution

Create a new window in the Editor for a script by clicking the New File button, , or pressing the `Ctrl-N` key combination. The Spyder Editor will provide a new window with default entries in the first few lines.

```
# -*- coding: utf-8 -*-
"""
```

```
Created on Sat May 11 15:43:22 2019
```

```
@author: yourname
"""
```

Typically, you can remove the first line, and it is recommended to add more information in the header between the `"""` to describe the purpose of the script. For example,

```
"""
```

```
This script computes the velocity of a free-falling bungee jumper at a specific time from launch based on  E
```

```
< |>
```

```
Sat May 11 15:43:22 2019
```



Author: David Clough  
"""

Enter the following Python statements after the header:

```
import math
g = 9.81 # m/s^2
m = 68.1 # kg
t = 12 # s
cd = 0.25 # kg/m
v = math.sqrt(m*g/cd)*math.tanh(math.sqrt(cd*g/m)*t)
print('velocity = ',v, ' m/s')
```

You will notice above that we have added comments to the ends of the lines setting the basic data values to indicate the units of those quantities. That is a good practice for clarity.

Save the file using the  button or the *Ctrl-S* key combination as `scriptdemo.py`. Run the script by clicking  or pressing the *F5* key. In the Console, you should see

```
velocity = 50.61747935192882 m/s
```

The script executes just as if you had typed each of its lines in the Console window.

As a final step, type `g` in the Console window and press Enter. You should see

Page 50

```
g
Out[n]: 9.81 (n: will be a counting integer)
```

So, you can see that, even though `g` was defined within the script, it is visible in the Console workspace and can be seen in the Variable Explorer above the Console.

|   |       |   |      |
|---|-------|---|------|
| g | float | 1 | 9.81 |
|---|-------|---|------|

We will explore this concept with functions, introduced next.

## 3.1.2 Python Functions

Python functions are scripts that accept input arguments and return results. They are invoked or “called” from a Python statement outside the function definition. Code for functions can be interspersed amongst other Python script statements or can be located in a separate `.py` file. For the function to be recognized, it must be executed. Python functions are analogous to user-defined functions in programming languages such as MATLAB, C/C++, Visual Basic and VBA,<sup>1</sup> and Fortran.

The general syntax for a Python function is

```
def function_name(arguments):
    """ docstring """
    statements
    return output_results/statement
```

where `def` stands for definition, `function_name` is how the function is identified, `arguments` is a list of one or more names, usually variable names, but occasionally the name of another function. A function may have no arguments, just an empty `()`. The *docstring* is not required, but it is recommended. It provides information about the function, its input arguments and return results. The

*docstring* can be more than one line between the two sets of `"""`. The *statements* compute the *output\_results*, although the `return` statement may have a Python statement.

The *output\_results* may have one or more variable names computed in the statements above. Note that the colon (`:`) is required at the end of the *def* statement, and subsequent statements must be indented (“tabbed” or four individual spaces). The Editor and the Console will indent automatically as the function is entered. It is good practice to include comments to clarify the statements in the function, apart from the *docstring*. The *docstring* becomes a property of the function and can be retrieved as

```
function_name.__doc__
```

 Note: Two underscores before and after doc.

Although we can enter a function definition in the Console, we typically place it in the Editor window and save it as a `.py` file or part of a `.py` file that has additional script statements. It is common to name functions with lowercase letters and possibly other characters such as numerals and a joining symbol like the underscore (`_`).

## EXAMPLE 3.2

### A Python Function

#### Problem Statement

As in [Example 3.1](#), compute the velocity of the free-falling bungee jumper, but now use a Python function to implement the calculation.

#### Solution

Create a new window in the Editor and type the following statements:

```
import math
import scipy.constants as pc

def freefall(t,m,cd):
    """
        function freefall: computes bungee jumper velocity
                        with second-order drag
        input arguments:
        t = time (s), m = mass (kg), cd = drag coefficient (kg/m)
        returns velocity in m/s
    """
    g = pc.g # retrieve gravitational acceleration from SciPy
    return math.sqrt(m*g/cd)*math.tanh(math.sqrt(g*cd/m)*t)
```

---

Page 51

---

Save the file (From the menu, *File* ⇒ *Save as* or *Ctrl-Shift-S*) as `freefall.py`. Note: You can give it any reasonable name, but `freefall` will remind us of the function contained. Execute the program,  or F5. There will be no response. To test the function, enter the following in the Console window:

```
freefall(12,68.1,0.25)
```

and you should see the response:

```
50.608007473466444
```

An advantage of a user-defined function like `freefall` is that you can invoke it repeatedly for different argument values. Suppose that you wanted to compute the velocity of a 100-kg jumper after 8 seconds with the same drag coefficient.

```
'velocity = {0:g} m/s '.format(freefall(8,100,0.25))  
'velocity = 53.1749 m/s '
```

Here, we use the `g` format. This is a general format specifier with a default of 6 significant figures. It uses `f` format between  $10^{-4}$  and  $10^6$  and `e` format outside of that range.

To see the `docstring` help text, enter

```
print(freefall.__doc__)
```

and the following should be displayed:

```
function freefall: computes bungee jumper velocity  
                    with second-order drag  
input arguments:  
t = time (s), m = mass (kg), cd = drag coefficient (kg/m)  
returns velocity in m/s
```

Note that, at the end of this example, if we had typed in the Console window

```
g
```

the following error message would have been displayed, along with additional error text,

```
NameError: name 'g' is not defined
```

Even though `g` had a value (9.80665 from SciPy) within the `freefall` function, it does not have a value in the Console workspace. As noted at the end of [Example 3.1](#), this is an important distinction between functions and general script code. The variables within a function are said to be *local* and are erased after the function is executed. In contrast, the variables in a script retain their existence after the script is executed.

Python functions can return more than one result. The results are returned in a *tuple* displayed within parentheses. They can then be extracted into their individual components.

```
import numpy as np  
def stats(x):  
    n = len(x)  
    avg = np.average(x)  
    s = np.std(x)  
    return n,avg,s
```

Here is an example of its application in the IPython Console.

```
y = [8,5,10,12,6,7,5,4]  
stats(y)  
(8, 7.125, 2.5708704751503917)
```

---

Page 52

---

For the most part, we will make use of script and function files created in the Editor window for the remainder of this book.

## 3.1.3 Variable Scope

Python variables have a property known as *scope* that refers to the context of the computing environment in which the variable has a unique identity and value. Typically, a variable's scope is limited either to the workspace associated with a Python script/IPython Console or within a function. This feature prevents confusion and possible errors when a programmer unintentionally gives the same name to variables in different contexts.

Any variables defined with direct commands through the IPython Console are in the general Python workspace. You can readily inspect the value of the variable by entering its name in the Console. You can also view information in the Variable Explorer window above the Console. For example, in our last example with the `stats` function, `y` is defined in the Console and can be seen in the Variable explorer as

| Name | Type | Size |                            |
|------|------|------|----------------------------|
| y    | list | 8    | [8, 5, 10, 12, 6, 7, 5, 4] |

We need to understand better the relationship between local and workspace variables. As a naive example, consider this function and script that adds two numbers,

```
def adder(a,b):
    print('x = ',x)
    s = a+b
    return s
x = 88
c = 1
d = 5
print('sum = ',adder(c,d))
print('s = ',s)
```

The Editor immediately flags the last line with a warning symbol, , and the error message, “undefined name s.” The variable `s` is local to the function `adder` and cannot be “seen” from the general script. If we remove that statement and run the script with function, the Console displays

```
x = 88
sum = 6
```

It is then clear that, from within the function, the workspace variable `x` can be seen. Another way to understand this is that it is possible to look upward into the workspace from a function, but not downward into a function from the workspace. Here is another example that deals with multiple functions.

```
import numpy as np
def sgnsqr(x):
    x1 = x*abs(x)
    return x1
def sgnsqrt(x):
    print('x1 = ',x1)
    x2 = np.sqrt(abs(x))*np.sign(x)
    return x2
x = -2.0
print('x squared with sign = ',sgnsqr(x))
print('square root of x with sign = ',sgnsqrt(x))
print('x2 = ',x2)
```

The Spyder Editor immediately flags the two highlighted statements as errors. This is because, from the second `sgnsqrt` function, the `x1` variable in the first `sgnsqr` function cannot be seen. Also, as before, the `x2` variable local to `sgnsqr` cannot be seen from the general workspace. Removing these two statements and running the code, we get

```
x squared with sign = -4.0
```

square root of x with sign = -1.4142135623730951

We conclude that workspace variables in Python are *global* in scope. Based on our discussion in the last section, the question arises whether we can change the scope properties of local variables in Python. It turns out that we can.

If we want to change the scope of a variable in a function from local to global, we can declare it as global. See this modification of our adder function example.

```
def adder(a,b):
    global x
    x = 88
    s = a+b
    return s
c = 1
d = 5
print('sum = ',adder(c,d))
print('x = ',x)
```

The result is

```
sum = 6
x = 88
```

We see here that the value of x is assigned within the function, but it is available in the script following the function.

As a last situation, we consider functions imbedded in other functions. This is an example.

```
def fun1(x):
    b = -1
    def fun2(a,x):
        b = a*x**5
        return b
    return fun2(b,x)
print('fun1 result = ',fun1(2))
print('fun2 result = ',fun2(2,-1))
```

Here, the fun2 function is imbedded within the fun1 function. The last line of code displays an error, undefined name fun2. The fun2 function cannot be invoked from outside the fun1 function. If we remove the last statement, the result is

```
fun1 result = -32
```

The sequence here is

1. fun1 is invoked with the argument 2
2. fun1 returns the result of fun2(-1,2) fun2 calculates  $-1 \times 2^5 \Rightarrow -32$
3. -32 is displayed

Although the examples used here may not be realistic, they illustrate the key concepts of scope. We always must keep scope in mind as we create all but the simplest programs in Python.

## 3.2 INPUT–OUTPUT

---

Up to this point, we have seen many examples of using the `print` function to display results in the Console. In this section, we will expand our capabilities to include user input, file input/output and a workspace save/load feature for arrays in Python.

### The input Function

This function allows you to prompt the user for entries to be assigned to a variable. The function returns a string result, so, if you are entering a numerical quantity, you must convert it using the `float` or `int` function. The general syntax is

```
s = input('prompt string')
```

---

Page 54

---

The result `s` is a string. Here is a specific example:

```
Ts = input('enter a value for the temperature in degC: ')
Tin = float(Ts)
print('Temperature in degF is ', '{0:6.1f}'.format(Tin*1.8+32))
```

with the Console displaying

```
enter a value for the temperature in degC: 37
Temperature in degF is      98.6
```

Above, the user entered the 37 value. For more compact code, the `float` function can be “wrapped around” the `input` function as

```
Tin = float(input('enter a value for the temperature in degC: '))
```

### Reading Data Files

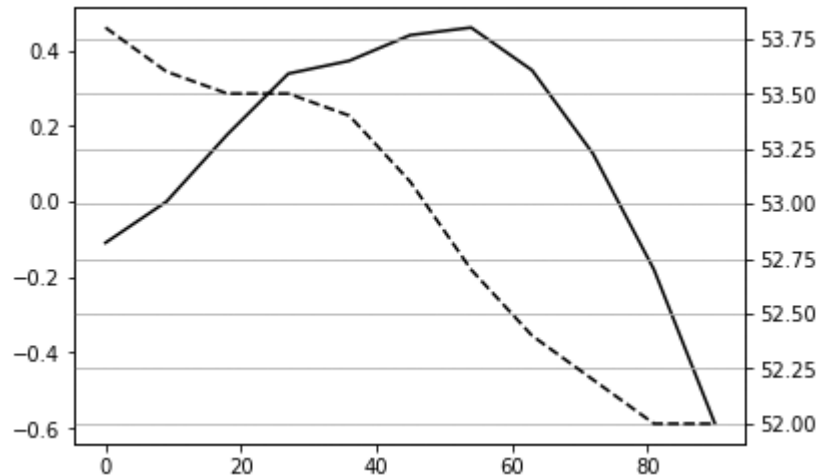
In engineering and science, it is a typical need to read in a collection of data from an external file, often a text file. These files could contain thousands of records with tens of data in each record. We will illustrate reading in a smaller set of data from a file here, but the principles extend to much larger files. Consider the following abbreviated set of data stored in a tab-delimited text file named `testdata.txt`:

|    |        |      |
|----|--------|------|
| 0  | -0.109 | 53.8 |
| 9  | 0      | 53.6 |
| 18 | 0.178  | 53.5 |
| 27 | 0.339  | 53.5 |
| 36 | 0.373  | 53.4 |
| 45 | 0.441  | 53.1 |
| 54 | 0.461  | 52.7 |
| 63 | 0.348  | 52.4 |
| 72 | 0.127  | 52.2 |
| 81 | -0.18  | 52   |
| 90 | -0.588 | 52   |

We would like to read this file into a Python program and assign its values to three arrays, `time`, `x` and `y`. You can create this data file with a text editor program like Notepad or WordPad in Windows. The file commands in the NumPy module are typically used for our read/write tasks. Here is code to acquire the data from the file above and create a plot.

```
import numpy as np
import pylab
time,x,y = np.loadtxt('testdata.txt',unpack=True)
pylab.plot(time,x,c='k')
pylab.twinx()
pylab.plot(time,y,c='k',ls='--')
pylab.grid()
```

The result is



Setting `unpack=True` causes the data records to be separated by columns into our variables, `time`, `x` and `y`. The `pylab.twinx()` command plots the `y` curve with the scale on the right axis. This is because `x` and `y` are of such differing magnitudes. There is some extra work to do to get axis labels and a legend, but we skip that here.

Frequently, data files are “comma delimited” and have a `.csv` file extension. The `np.loadtxt` function separates data in the same record by recognizing spaces or tabs between the data. Consider a modified version of the previous data file, `testdata.csv`.

```
0,-0.109,53.8
9,0,53.6
18,0.178,53.5
27,0.339,53.5
36,0.373,53.4
45,0.441,53.1
54,0.461,52.7
63,0.348,52.4
72,0.127,52.2
81,-0.18,52
90,-0.588,52
```

We can modify the `np.loadtxt` to specify the delimiter as a comma.

```
import numpy as np
time,x,y = np.loadtxt('testdata.csv',unpack=True,delimiter=',')
print(time[1],x[1],y[1])
```

with the result

```
9.0 0.0 53.6
```

There are further specifications for the `np.loadtxt` function that you can find in Spyder Help. It is worth mentioning that there is another file-reading function in the NumPy module called `genfromtxt()` that provides more options and can manage such situations as missing data in file records.

## Writing Data Files

It is a common practice to save the results of calculations in Python to an external file, often a text file. Text files can then be read in by another Python program or by other software, for example, statistical packages, spreadsheets, data base packages. In the script below, the `testdata.csv` file is read and then written out as `testdata1.txt`. So that the data appear in the same record form as the original, it is necessary to create and transpose an array of `time`, `x` and `y`.

```
import numpy as np
time,x,y = np.loadtxt('testdata.csv',unpack=True,delimiter=',')
np.savetxt('testdata1.txt',np.array([time,x,y]).transpose(),
          fmt=['%5d','%7.3f','%5.1f'])
```

The resulting `testdata1.txt` file looks like this

```
0      -0.109    53.8
9       0.000    53.6
18      0.178    53.5
27      0.339    53.5
36      0.373    53.4
45      0.441    53.1
54      0.461    52.7
63      0.348    52.4
72      0.127    52.2
81     -0.180    52.0
90     -0.588    52.0
```

You see above how we applied a specific formatting to the three columns using the `fmt` specifier.

## Save and Load Arrays

The NumPy module provides `save` and `load` functions for storing and retrieving specific arrays. The arrays are stored in an external `.npy` file in binary format. Here is an example of the `save` and `load` functions.

```
import numpy as np
time,x,y = np.loadtxt('testdata.csv',unpack=True,delimiter=',')
X = np.array([time,x,y]).transpose()
np.save('Xdata.npy',X)
X2=np.load('Xdata.npy')
print(X2)
```

And the `print` statement generates this output in the Console:

```
[[ 0.    -0.109  53.8 ]
 [ 9.     0.     53.6 ]
 [18.    0.178  53.5 ]
 [27.    0.339  53.5 ]
 [36.    0.373  53.4 ]
 [45.    0.441  53.1 ]
 [54.    0.461  52.7 ]
 [63.    0.348  52.4 ]
 [72.    0.127  52.2 ]
```



```
[81.    -0.18  52.   ]  
[90.    -0.588 52.  ]]
```

Notice that the result is an array. It may be convenient to use `save` and `load` if you have an array as a result of one Python program and want to load it into another program.

## 3.3 STRUCTURED PROGRAMMING

---

The simplest Python scripts perform a set of instructions sequentially. These include most of the examples you have seen so far in this text. That is, the script statements are executed line by line starting at the top of the code and moving down to the end. Because a strict sequence is highly limiting, all computer languages provide statements allowing programs to take nonsequential paths. These can be classified as

- *Decisions* (or Selection). The branching of flow based on a decision.
- *Loops* (or Repetition). The looping of flow to allow statements to be repeated.

### 3.3.1 Decisions

#### The if Structure

This structure allows you to execute a set of statements if a logical condition is true. Its general syntax is

```
if condition :  
    statements
```

The statements must be indented, just like the function definition. Note the use of the colon ( : ). The end of the structure is when the indentation ends. As an example, here is a simple Python function to evaluate whether a grade is passing. This is entered in the Editor window and saved as `SimpleIfStructureExample.py`.

```
def grader(grade):  
    """  
    determines if a grade is passing  
    input:  
        grade = numerical value of grade (0 to 100)  
    output:  
        displayed message if grade is passing  
    """  
    if grade >= 60:  
        print('passing grade')
```

Here is a test of the `grader` function in the Console:

```
grader(85.6)  
passing grade
```

Note: When the grade is below 60, there is no response. We can make the code a bit more compact by combining the `if` and `print` lines.

```
if grade >= 60: print('passing grade')
```

You can add statements to that line separated by semicolons ( ; ); however, we recommend using the compact *one-line if* style for only one statement. Otherwise, use the indentation style. The latter makes your code more readable.

### Logical Conditions

The simplest form of a logical condition is a **Boolean** variable that only stores the values **True** or **False**. Then, the **if** statement would be, for example,

```
if reset :
```

where `reset` is a **Boolean** type variable.

A more common form of the logical condition is a relational expression that compares two values as in

```
value1 relational_operator value2
```

where *value<sub>1</sub>* and *value<sub>2</sub>* can be constants, variables, or expressions. The *relational\_operator* is one of those listed in [Table 3.1](#).

**TABLE 3.1** Summary of relational operators in Python.

| Example                    | Operator           | Relationship             |
|----------------------------|--------------------|--------------------------|
| <code>x == 0</code>        | <code>==</code>    | equal to                 |
| <code>unit != 'm'</code>   | <code>!=</code>    | not equal to             |
| <code>a &gt; 0</code>      | <code>&gt;</code>  | greater than             |
| <code>s &lt; t</code>      | <code>&lt;</code>  | less than                |
| <code>3.9 &gt;= a/3</code> | <code>&gt;=</code> | greater than or equal to |
| <code>r &lt;= 0</code>     | <code>&lt;=</code> | less than or equal to    |

Python also allows testing of more than one logical condition by employing logical operators. We will emphasize the following:

- not** Used to perform logical negation of an expression.  
`not expression`  
If the expression is true, the result is false. Conversely, if the expression is false, the result is true.
- and** Used to perform a logical conjunction on two expressions.  
`expression1 and expression2`  
If both expressions evaluate to true, the result is true. If either or both expressions evaluate to false, the result is false.
- or** Used to perform a logical disjunction on two expressions.  
`expression1 or expression2`  
If either or both expressions evaluate to true, the result is true. If both expressions evaluate to false, the result is false.

[Table 3.2](#) summarizes all possible outcomes for each of these operators. Just as for arithmetic operations, there is a priority order for evaluation logical operations. These are from highest to

lowest, not, and, or. In choosing between operators of equal priority, Python evaluates them from left to right. Finally, as with arithmetic operators, parentheses can be used to enforce a priority order.

**TABLE 3.2** A truth table summarizing the possible outcomes for logical operators employed in Python. The order of priority of the operators is shown at the top of the table.

|   |   | Highest | →       | Lowest |
|---|---|---------|---------|--------|
| x | y | not x   | x and y | x or y |
| T | T | F       | T       | T      |
| T | F | F       | F       | T      |
| F | T | T       | F       | T      |
| F | F | T       | F       | F      |

Let’s investigate how the computer employs the priorities to evaluate a logical expression. If  $a = -1$ ,  $b = 2$ ,  $x = 1$ , and  $y = 'b'$ , evaluate whether the following is true or false:

```
a * b > 0 and b == 2 and x > 7 or not y > 'd'
```

To make it easier to evaluate, substitute the values for the variables.

```
-1 * 2 > 0 and 2 == 2 and 1 > 7 or not 'b' > 'd'
```

The first priority for Python is to evaluate any arithmetic expressions. Here, there is only one,  $-1 * 2$ .

```
-2 > 0 and 2 == 2 and 1 > 7 or not 'b' > 'd'
```

Next, all the relational expressions are evaluated.

```
-2 > 0 and 2 == 2 and 1 > 7 or not 'b' > 'd'
F      and    T      and    F      or not    F
```

Note:  $'b' > 'd'$  is false because  $'b'$  is less than  $'d'$ ; that is, it is before  $'d'$  alphabetically.

At this point, the logical operators are evaluated in priority order. The highest is the not, so  $\text{not } F = T$ .

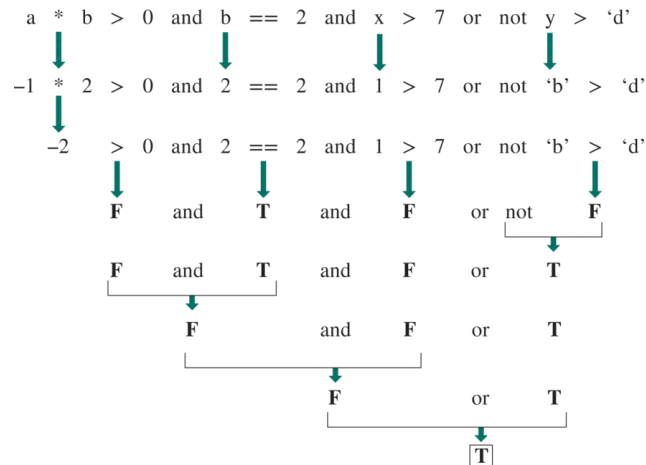
```
F      and    T      and    F      or    T
```

The and operators are evaluated next left to right.

```
F      and    F      or    T
      F          or    T
```

And the final evaluation is the OR operator with the result being true. The entire process is illustrated in  Fig. 3.1.

**FIGURE 3.1** A step-by-step evaluation of a complex logical expression.



Substitute constants

Evaluate arithmetic expressions

Evaluate relational expressions

Evaluate compound logical expressions



## The if..else Structure

This structure allows you to execute one set of statements if a logical condition is true and to execute an alternate set if the condition is false. Its general syntax is

```

if condition :
    statements1
else:
    statements2

```

As before with Python, indentation (tab or four spaces) is required. The structure is terminated when the next Python statement is not indented.

## The if...elif...else Structure

It often happens that, when the condition of the if structure evaluates as false, it is required to test another condition. This type of structure occurs when we have more than two options for a particular problem setting. For such cases, a special form of the decision structure, the *if...elif...else*, has been developed. It has general syntax

```

if condition1 :
    statements1
elif condition2:
    statements2
elif condition3:
    statements3
.
.
.
else:
    statements_else

```

Page 59

The number of elif statements depends on the number of alternative tests required. The tests are in priority order; that is, as soon as a condition tests true, its associated statements are executed, and the structure is exited without checking the remaining conditions. The else clause is not required unless there is code that must execute when all conditions tested are false.

## EXAMPLE 3.3

### if Structures

#### Problem Statement

For a scalar, the built-in NumPy `sign` function returns the sign of its argument as  $-1$  for negative,  $0$  for zero, or  $1$  for positive. Here is a set of Console commands that illustrates how it works:

```
import numpy as np
np.sign(25.6)
1.0
np.sign(-1.776)
-1.0
np.sign(0)
0
```

Develop a Python function that performs the same calculation.

---

Page 60

---

#### Solution

First, an if structure can be used to return  $1$  if the argument is positive:

```
def sgn(x):
    if x > 0 :
        return 1
```

After executing the function in the Editor, we can test it in the Console.

```
sgn(25.6)
1
```

and there is no result with

```
sgn(-1)
```

Although the function handles positive numbers correctly, as we see above, when it is run with a negative argument, nothing is displayed. To remedy this shortcoming partially, an if...else structure can be used to display  $-1$  if the condition is false.

```
def sgn(x):
    if x > 0 :
        return 1
    else:
        return -1
```

If we try the last test again, it now provides the correct result.

```
sgn(-1)
-1
```

Although the positive and negative cases are now handled properly, when we test the function with a zero argument,  $-1$  is returned.

```
sgn(0)
-1
```

This is incorrect, as we need a result of zero in this case. Our if...else structure can be expanded to an if...elif...else structure to handle all cases correctly.

```
def sgn(x):
    if x > 0:
        return 1
    elif x < 0:
        return -1
    else:
        return 0
```

Now, we can test with the three cases to ensure that the function works correctly.

```
sgn(25.6)
1

sgn(-1.776)
-1

sgn(0)
0
```

We note here that other languages have a *Select Case* or *switch* structure that allows for multiple selections without regard to order. Python does not have such an option. So, when using `if...elif...else` structures, you must pay attention to the order of the tests.

## 3.3.2 More on Arguments

Page 61

For the examples of functions we have seen above, the arguments to the function, when it is invoked, must be specified in the order in which they are listed in the `def` statement. These are *positional* arguments. It is also possible to employ *keyword* arguments that can be specified in any order. One can mix positional and keyword arguments; however, positional arguments must appear first in the list in the `def` statement.

We will illustrate this with an example of a temperature conversion function.

```
def tempconvert(tempin, units='degF'):
    if units == 'degF':
        return (tempin-32)/1.8
    else:
        return tempin*1.8+32
```

Here, we have specified a default value for the `units` argument. If this argument is left out when the function is called, by default the conversion will be from Fahrenheit to Celsius. However, if we designate a different string for units, such as `'degC'`, the conversion will be the reverse. Here are examples of using the function in the Console.

```
tempconvert(98.6)
Out[86]: 36.999999999999999

tempconvert(37, units='degC')
Out[87]: 98.600000000000001
```

By specifying an argument as its name in the function definition, we can enter the arguments in any order as in

```
tempconvert(units='degF', tempin=-40)
-40.0
```

## 3.3.3 Loops

As the name implies, loops perform operations repetitively. There are two types of loops, depending on how the repetitions are terminated. The count-controlled loop, called a *for loop* in Python, ends after a specified number of repetitions. The general loop, called a *while loop* in Python, repeats only as long as a condition is satisfied.

## The for Structure

A *for loop* repeats statements a specific number of times. There is a very general syntax in Python for this structure:

```
for item in iterable_object:
    statements
```

An example of this would be

```
for choice in ['blue','green','yellow','magenta','cyan']:
    statements
```

The statements would be repeated five times with the variable `choice` equal to each color in the list in order.

In most science and engineering numerical applications, we are more interested in for loops that repeat based on an index that has a starting value, an ending value, and possibly a step size. Here are a few examples:

- from 1 to 25 in steps of 1
- from 10 to 1 in steps of -1
- from 0 to 100 in steps of 10

We could use the general syntax and have

```
for i in [1,2,3,4,5]:
    statements
```

Python provides a *range* type that facilitates these index specifications. The *range* type is not a list or an array but rather a mechanism for generating a sequence of integer index values. It is a bit tricky to use, so it is important to understand it well because we will be using it frequently with for loops. The syntax will be

```
for index in range(start,end,step):
    statements
```

---

Page 62

If we leave out *start* and *step*, an example would be

```
range(4)
```

which would generate four indices: 0,1,2,3.

You will note that, by default, *start* = 0. Second, you will note that the generated indices stop just before *end* is reached. Let's explore this with some additional examples.

```
range(1,10,3)
```

Here, the sequence would be 1, 4, 7, and next 10. But since *end* = 10, it is not generated. The result would be 1, 4, 7.

We often use loop index variables as subscripts in arrays and matrices. Since these subscripts have an origin of 0, the simple *range* type works well. For example, an array with 25 elements can be indexed by



`range(25)` (that will generate the sequence 0, 1, 2, ..., 24 or 25 items.)

To illustrate these concepts, consider a Python function that computes the sum of integers from 1 to  $n$ .

```
def sumint(n):
    sum = 0
    for i in range(n):
        sum = sum + (i+1)
    return sum
print(sumint(10))
print(10*11/2)
```

When this code is executed, the result is

```
55
55.0
```

The second `print` function takes advantage of the shortcut formula,

$$\sum_{i=1}^n i = \frac{n(n+1)}{2},$$

and confirms the first result. Since the sequence generated by the *range* type is 0, 1, ...,  $n-1$ , we must use  $i+1$  in our `sum` statement. Note that `i` and `sum` will not have values outside the function. Another curiosity is to check the value taken by `i` after the loop has terminated. We add a statement to do that.

```
def sumint(n):
    sum = 0
    for i in range(n):
        sum = sum + (i+1)
    print('i at end of for loop = ',i)
    return sum
print(sumint(10))
print(10*11/2)
```

With the result

```
i at end of for loop = 9
55
55.0
```

You can see that `i` has its last value after ending the loop. We note this because it is not the case in several other programming languages where the loop index is one step beyond the end value after exit.

## EXAMPLE 3.4

### Using a for Loop to Compute the Factorial

#### Problem Statement

Develop a Python function to compute the factorial.<sup>2</sup>

$$\begin{aligned}
 0! &= 1 \\
 1! &= 1 \\
 2! &= 1 \times 2 = 2 \\
 3! &= 1 \times 2 \times 3 = 6 \\
 4! &= 1 \times 2 \times 3 \times 4 = 24 \\
 5! &= 1 \times 2 \times 3 \times 4 \times 5 = 120
 \end{aligned}$$

## Solution

A simple Python function to implement this calculation can be developed as

```
def factor(n):
    """
    computes the product of all the integers from 1 to n
    special case: factor(0)=1
    """
    x = 1
    for i in range(n):
        x = x * (i+1)
    return x
```

Here are some tests of the function in the Console.

```
factor(0)
1

factor(1)
1

factor(5)
120

factor(20)
2432902008176640000
```

For `factor(5)`, this loop will execute five times (`i` from 0 to 4), each time multiplying the previous `x` by `i+1`. The result is 120 or 5!. Note what happens for `n = 0`. With `range(0)`, does the for loop even execute once? We can test that with the Python script

```
for i in range(0):
    print(i)
```

When this script is executed, there is no output in the Console; consequently, the loop never executed. The `range(0)` does not return a value for `i`.<sup>3</sup> Because we set the value of `x = 1` before the for loop, this value is retained for `factor(0)`.

## Vectorization

With some practice, the for loop is easy to implement and understand. However, in Python, it is not necessarily the most efficient means to repeat statements a specific number of times. With the NumPy module, Python can operate directly on arrays and matrices, and *vectorization* can provide a more efficient option. For example, the following loop structure:

```
import numpy as np
y = np.zeros(251)
for i in range(251):
    t = i * 0.02
```

```
y[i] = np.cos(t)
```

The for loop fills the empty *y* array with 251 evenly spaced values of  $\cos(t)$  from  $t = 0$  to 50.

---

Page 64

---

Alternately, we can accomplish the same with vectorized code:

```
import numpy as np
y = np.zeros(251)
t = np.arange(0.0, 50.02, 0.02)
y = np.cos(t)
```

It should be noted that, for more complex code, it may not be obvious how to vectorize the code. That said, where for loops are used with arrays and matrices, vectorization should always be considered.

## Preallocation of Memory

Python will not automatically resize arrays as new elements are created. For example, in the previous code, this modification will not work.

```
import numpy as np
for i in range(251):
    t = i * 0.02
    y[i] = np.cos(t)
```

This code will not run and displays an error in the Editor window, “undefined variable *y*.” You will notice in the previous code, we created the *y* array first with the `np.zeros` function. We had to prejudge the required size of this array.

An alternate approach available to us is to extend the size of *y* element-by-element by using the `np.append` function. See below.

```
import numpy as np
y = np.array(0)
for i in range(250):
    t = i * 0.02
    y = np.append(y, np.cos(t))
```

There is computational efficiency in pre-sizing arrays; however, there are situations where the eventual size of an array is not known until the loop executes. In the latter, the `np.append` method is recommended.

## The while Structure

A *while loop* executes as long as a logical condition is true. Its general syntax is

```
while condition:
    statements
```

As long as `condition` is true, the statements are repeated. A simple example is

```
x = 8
while x > 0:
    x = x - 3
    print(x)
```

When this code is run, the result is

```
5
2
-1
```

Notice that `-1` is displayed before the next `while` tests true and the loop is exited.

## Additional Loop Features

There are several modifications we can make to for and while loops to provide flexibility and modifications to the code structure. The most prominent is the *break* command.

---

Page 65

Other programming languages, such as Fortran and Visual Basic, allow a loop exit anywhere in the loop code. This is not a built-in feature of Python, but the functionality can be mimicked by adding an if statement and break command. We could call this the *while...break* structure. We also note that the break command can be used to exit a for loop prematurely. Here is the general syntax:

```
while True:
    statements
    if condition: break
    statements
```

Here, the loop will never exit on the while test, which is always true. It will exit on the break command when the condition tests true. When the if statement is just below the while statement, this is called a *pretest loop*. When it is interspersed in the statements, it is a *midtest loop*, and when it is the last statement, it is called a *posttest loop*. It is also possible to have more than one if...break test in the while loop. This provides a very flexible, general looping structure.

Here is an example of the *pretest loop*<sup>4</sup>:

```
x = 100
while True:
    if x < 0: break
    x = x - 5
    print(x)
```

Notice that the loop modifies x so that the if test eventually is true. The result displayed in the Console when this script is run is

```
-5
```

Every while loop must have a condition for exit that will, at some point, test true; otherwise, it would become a so-called *infinite loop* that would never stop. Alternately, we could place the if test at the end of the loop code and create a *posttest loop*.

```
x = 100
while True:
    x = x - 5
    if x < 0: break
    print(x)
```

with the same result as above. However, this loop will repeat one time less than its predecessor.

Recognize also that because the `print` statement in these two examples is not indented, it means that it is outside the loop. If you desired to have the print within the loop, you would have typed

```
x = 100
while True:
    if x < 0: break
    x = x - 5
    print(x)
```

in which case the output would be

```
100
95
90
.
```

```
.  
.   
5  
0  
-5
```

It should be clear that, in fact, all three loop structures, pre-, mid-, or posttest, are really the same. That is, depending on where we place the if...break statement, we will have a pre-, mid-, or posttest loop. It is this simplicity that led the designers of computer languages, such as Fortran and Visual Basic to favor this structure over other forms of the general loop, such as the while structure.<sup>5</sup>

Python provides a *continue* command that allows the current iteration of a loop to be terminated prematurely but continues the loop repetition. Here is an example.

---

Page 66

---

```
import random  
i = 0  
sum = 0  
while True:  
    x = random.normalvariate(100,20)  
    if x < 100: continue  
    i = i + 1  
    sum = sum + x  
    if i >= 10: break  
avg = sum/i  
print(avg)
```

The result from one execution of this script is

```
111.92167734130621
```

The `random.normalvariate` function generates random numbers from a simulated normal distribution with mean 100 and standard deviation 20. If the random number generated is less than 100, the `continue` command causes the code to skip the subsequent calculation and repeats the loop. If the `x` is greater than or equal to ( `>=` ) 100, `i` is incremented and the running `sum` accumulates the value of `x`.<sup>6</sup> After ten increments/accumulations, the loop exits, and the average of the ten is computed and displayed in the Console.

It is possible, inadvertently, to create an infinite loop in Python. If this occurs in the Spyder environment, you can abort execution by clicking the red square button in the upper right hand corner of the Console window.

Both the `for` loop and the `while` loop provide for an `else` clause. We will not often use it in this book, but it is worth mentioning for the sake of completeness. The general syntax for each loop type with `else` is

```
for item in iterable_object:  
    statements1  
    if condition: break  
    statements2  
else  
    statements3  
  
while True:  
    statements1  
    if condition: break  
    statements2  
else  
    statements3
```

If either loop terminates normally, the `else` clause is executed. If the `break` command causes loop exit, the `else` clause is not executed. This may seem counterintuitive, so it is important to pay note.

## 3.4 NESTING AND INDENTATION

We need to understand that structures can be nested within each other. Nesting refers to placing structures within other structures, whether they are decisions or loops. The following example illustrates the concept:

Page 67

### EXAMPLE 3.5

#### Nesting Structures

##### Problem Statement

The roots of a quadratic equation

$$f(x) = ax^2 + bx + c$$

can be determined with the quadratic formula

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Develop a Python function to implement this formula given values of the coefficients.

##### Solution

Top-down design provides a nice approach for debugging an algorithm to compute the roots. This involves developing the general structure without details and then refining the algorithm. To start, we recognize that, if the parameter  $a$  is zero, we will have “special” cases; for example, single roots or trivial values; otherwise, we will have conventional cases using the quadratic formula. The “big picture” version can be programmed as

```
def quadroots(a,b,c):
    """
    quadroots: roots of the quadratic equation
               quadroots(a,b,c): real and complex roots
               of a quadratic polynomial
    input:
        a = second-order coefficient
        b = first-order coefficient
        c = zero-order coefficient
    output:
        r1: real part of the first root
        i1: imaginary part of first root
        r2: real part of the second root
        i2: imaginary part of second root
    """
    if a == 0:
        # special cases
    else:
        # quadratic formula
    return r1,i1,r2,i2
```

Next, we develop refined code to handle the special cases:

```
# special cases
if b != 0:
    # single root
    r1 = -c/b
    return r1
else:
    # trivial solution
    print('Trivial solution. Try again.')
```

And we can develop refined code to handle the quadratic formula cases:

```
# quadratic formula
d = b**2 - 4*a*c
if d >= 0:
    # real roots
    r1 = (-b+math.sqrt(d))/(2*a)
    r2 = (-b-math.sqrt(d))/(2*a)
    i1 = 0
    i2 = 0
else:
    # complex roots
    r1 = -b/(2*a)
    i1 = math.sqrt(abs(d))/(2*a)
    r2 = r1
    i2 = -i1
return r1,i1,r2,i2
```

Then, we substitute these blocks back into the simple “big picture” framework to give the final result.

Page 68

```
def quadroots(a,b,c):
    """
    quadroots: roots of the quadratic equation
    quadroots(a,b,c): real and complex roots
    of a quadratic polynomial

    input:
    a = second-order coefficient
    b = first-order coefficient
    c = zero-order coefficient

    output:
    r1: real part of the first root
    i1: imaginary part of first root
    r2: real part of the second root
    i2: imaginary part of second root
    """
    import math
    if a == 0:
```

```
        # special cases
        if b != 0:
            # single root
            r1 = -c/b
            return r1
        else:
            # trivial solution
            print('Trivial solution. Try again.')
```

```
    else:
```

```
        # quadratic formula
        d = b**2 - 4*a*c
        if d >= 0:
            # real roots
            r1 = (-b+math.sqrt(d))/(2*a)
            r2 = (-b-math.sqrt(d))/(2*a)
```

```
        i1 = 0
        i2 = 0
    else:
        # complex roots
        r1 = -b/(2*a)
        i1 = math.sqrt(abs(d))/(2*a)
        r2 = r1
        i2 = -i1
    return r1,i1,r2,i2
```

Notice the placement of the return statements to provide the appropriate results, depending on the case.

As highlighted by the shading, notice how indentation helps to make the underlying logical structure clear. Of course, this indentation is enforced by Python. Here are results from the Console illustrating how the function performs.

```
quadroots(1,1,1)
(-0.5, 0.8660254037844386, -0.5, -0.8660254037844386)

quadroots(1,5,1)
(-0.20871215252208009, 0, -4.79128784779195, 0)

quadroots(0,5,1)
-0.2

quadroots(0,0,0)
Trivial solution. Try again.
```



## 3.5 PYTHON FUNCTIONS WITH FUNCTION NAME ARGUMENTS

---

Much of the remainder of the book involves developing functions to evaluate other functions numerically. Although a customized function could be developed for every new equation analyzed, a better alternative is to design a generic function and pass the name of the particular equation we wish to analyze as an argument. We will call this a function-function. Before we describe in detail how to accomplish this in Python, we will first introduce lambda functions, which provide a handy means to define simple user-defined functions without developing a full-blown Python `def` function.

---

Page 69

---

### 3.5.1 Lambda Functions

*Lambda* functions allow you to create a simple function without creating a Python `def` function. They can be defined with the following syntax:

```
function_name = lambda arglist: expression
```

Here is a specific example:

```
f1 = lambda x,y: x**2 + y**2
```

Once this function is defined by executing it, either in a script or in the Console, it can be used just as other functions.

```
f1(3,4)
25
```

The lambda function can include parameters from the workspace. For example,

```
a = 4
b = 2
f2 = lambda x: a*x**b
```

The `a` and `b` variables have global scope. After this script is executed, the `f2` function can be used in the Console.

```
f2(3)
36
```

Then, the value of `a` can be modified in the Console, and this will affect the definition of the `f2` function.

```
a=3
f2(3)
27
```

Lambda functions are of a type generally called *anonymous functions*.

### 3.5.2 Function Functions

Function functions are functions that operate on other functions which are passed to them as input arguments. Python naturally accommodates arguments that are function names. For a simple example,

consider that we want to create a function that computes the value of another function,  $f(x)$ , at the midpoint of two values of  $x$ , called  $x_1$  and  $x_2$ . The Python code to accomplish this is

```
import numpy as np
def midpoint(f,x1,x2):
    """
    this function evaluates f(x) at the midpoint
    between x1 and x2
    """
    xmid = (x1+x2)/2
    return (f(xmid))

def f(w):
    return np.sin(w)*np.cosh(w)-5

fmid = midpoint(f,0,3)
print('function value at midpoint =',fmid)
```

Page 70

You can see that we have included the argument  $f$  in the `def` statement for the `midpoint` function. This implies that our function `midpoint` will evaluate for different functions  $f$ . Here, we illustrate that by using the function

$$f(x) = \sin(x)\cosh(x) - 5$$

and we invoke the midpoint function for  $x_1 = 0$  and  $x_2 = 3$ , with the result

```
function value at midpoint = -2.6534832023556882
```

The versatility here is that we can define a new function

$$g(x) = \frac{\pi r^2}{4}$$

and use the midpoint function to evaluate it for  $x_1 = 1.5$  and  $x_2 = 2.78$ . Here, we use a lambda function for  $g(x)$ , defined in the Console.

```
g = lambda x: np.pi*x**2/4
midpoint(g,1.5,2.78)
3.596809429094953
```

In the remainder of the book, we will find use of Python function functions to carry out sophisticated numerical methods. We will create these and use others from established modules, such as NumPy.

## EXAMPLE 3.6

### Building and Implementing a Function Function

#### Problem Statement

Develop a Python function function to determine the average value of a function over a range. Illustrate its use for the bungee jumper velocity over the range  $t = 0$  to 12 seconds.

$$v(t) = \sqrt{\frac{mg}{c_d}} \tanh\left(\sqrt{\frac{c_d g}{m}} t\right)$$

where  $g = 9.81$ ,  $m = 68.1$ , and  $c_d = 0.25$ .

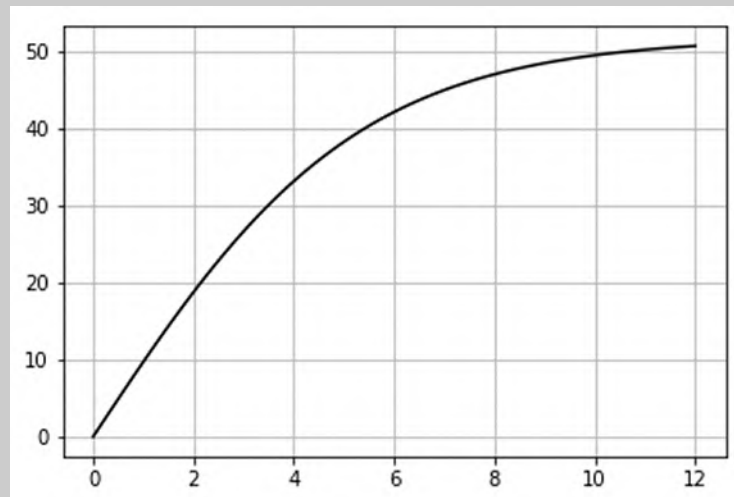
## Solution

The average value of the function for a set of independent variable values can be computed with built-in Python commands as

```
import numpy as np
import pylab
g = 9.81
m = 68.1
cd = 0.25
t = np.linspace(0,12,100)
v = np.sqrt(m*g/cd)*np.tanh(np.sqrt(cd*g/m)*t)
avgv = np.average(v)
print(' average velocity = ',avgv)
pylab.plot(t,v,'k')
pylab.grid()
```

with the result

```
average velocity = 36.08702728414769
```



Inspection of the plot shows that this result is a reasonable estimate of the curve's average height. With this background, we can write a Python function to perform the same computation.

```
import numpy as np
def funcavg(a,b,n):
    """
    computes the average value of a function
    over a range
    requires import of numpy module
    input:
        a = lower bound of range
        b = upper bound of range
        n = number of intervals
    output:
        favg = average value of function
    """
    x = np.linspace(a,b,n)
    y = func(x)
    favg = np.average(y)
    return favg
```

```
def func(t):
    g = 9.81
    m = 68.1
    cd = 0.25
    f = np.sqrt(m*g/cd)*np.tanh(np.sqrt(cd*g/m)*t)
    return f
```

The main function, `funcavg`, used `np.linspace` to generate equally spaced  $x$  values across the range. These values are then passed to a subfunction `func` in order to generate the corresponding  $y$  values. Finally, the average value is computed. Once executed in the Editor window, the function can be invoked from the Console as

```
funcavg(0,12,60)
36.01273277768904
```

Now, let's rewrite the `funcavg` function so that, rather than being specific to `func`, it evaluates a nonspecific function name `f` that is passed as an argument.

```
def funcavg(f,a,b,n):
    """
    computes the average value of a function
    over a range
    requires import of numpy module
    input:
        f = function to be evaluated
        a = lower bound of range
        b = upper bound of range
        n = number of intervals
    output:
        favg = average value of function
    """
    x = np.linspace(a,b,n)
    y = f(x)
    favg = np.average(y)
    return favg
```

Because we have removed function `func` from the code and replaced it with the argument `f`, this version is truly generic. It can be run from the Console by defining a lambda function.

```
import numpy as np
g = 9.81 ; m = 68.1 ; cd = 0.25
vel = lambda t: np.sqrt(m*g/cd)*np.tanh(np.sqrt(cd*g/m)*t)
funcavg(vel,0,12,60)
36.01273277768904
```

We could also add another function to the Editor window and apply `funcavg` to that function.

```
def f2(x):
    return np.sin(x)*np.cosh(x)-5
```

In the Console,

```
funcavg(f2,0,3,50)
-2.9645972090434975
```

As a final example, we can include a lambda function definition directly in the argument list of `funcavg`.

```
funcavg(lambda x:np.sin(x),0,2*np.pi,180)
0.0
```

Does this result make sense to you?

We can now see that `funcavg` is designed to evaluate any valid Python expression in function form. We will do this frequently throughout the remainder of this text in several contexts from nonlinear equation solving to the solution of differential equations.

## 3.5.3 Passing Parameters

Recall from [Chap. 1](#) that the terms in mathematical models can be divided into the categories of dependent and independent variables, parameters, and forcing functions. For the bungee jumper model, the velocity ( $v$ ) is the dependent variable, time ( $t$ ) is the independent variable, the mass ( $m$ ) and drag coefficient ( $c_d$ ) are parameters, and the gravitational acceleration constant ( $g$ ) is the forcing function. It is commonplace to investigate the behavior of such models by performing a sensitivity analysis or case study. This involves observing how the dependent variable changes as the parameters and forcing functions are varied.

In [Example 3.6](#), we developed a function `funcavg`, and used it to determine the average value of the bungee jumper velocity for the case where the parameters were set at  $m = 68.1$  kg and  $c_d = 0.25$  m/kg. Suppose that we wanted to analyze the same function but with different parameter values. Of course, we could retype the function with new values for each case, but it would be preferable just to change the parameters.

As we learned in [Sec. 3.5.1](#), it is possible to incorporate parameters as arguments in functions, and, when we include default values in the function `def` statement, we can specify these by name, and in any order after the unspecified arguments. These are called keyword arguments. For example, in our `func` function in the previous section, instead of specifying the parameters within the function, we could write it as

```
import numpy as np
def func(t,m=70,cd=0.25):
    g = 9.81
    f = np.sqrt(m*g/cd)*np.tanh(np.sqrt(cd*g/m)*t)
    return f
```

So, if the function is invoked without specifying  $m$  or  $c_d$ , the default values will be used. Alternately, we can specify one or both and in any order after  $t$ . Here is an example:

```
func(12,cd=0.3)
47.1508430811294
```

The default value of  $m = 70$  is used in this calculation.

Python provides a mechanism for passing a variable number of arguments to a function. If we want to use the function `func` above as the input argument to `funcavg` from the previous section, we would want to be able to pass the parameters  $m$  and  $cd$  through `funcavg` to `func`. It would be awkward to modify the `funcavg` `def` statement as follows

```
def funcavg(f,a,b,n,m,cd):
```

because that would customize `funcavg` to the particular function (`func`) we are investigating. There is a better way to handle this by including the argument `*args` at the end of the `funcavg`

argument list to indicate that additional arguments may be supplied. Then, we would include `*args` in the `f(x)` statement to pass the extra arguments through to the evaluation of `f`. Alternately, if we use keyword arguments in the function being evaluated, such as in `func` above, we use `**kwargs` instead of `*args`. The code below illustrates both methods.

```
import numpy as np
def funcavg(f,a,b,n,*args):
    """
    computes the average value of a function
    over a range
    requires import of numpy module
    input:
        f = function to be evaluated
        a = lower bound of range
        b = upper bound of range
        n = number of intervals
    output:
        favg = average value of function
    """
    x = np.linspace(a,b,n)
    y = f(x,*args)
    favg = np.average(y)
    return favg

def func(t,m,cd):
    g = 9.81
    f = np.sqrt(m*g/cd)*np.tanh(np.sqrt(cd*g/m)*t)
    return f

print('{0:7.3f}'.format(funcavg(func,0,12,60,68.1,0.25)))
```

with the result when run

```
36.013
```

When the `funcavg` function is invoked in the `print` command, extra arguments required by `func` are specified (68.1 and 0.25). These are passed through via the use of `*args` to `func` as  $m$  and  $c_d$  via the `*args` specification in the `f` function. Now, `funcavg` is not restricted only to the `func` example.

Below, we implement variable keyword arguments using `**kwargs`.

```
import numpy as np
def funcavg(f,a,b,n,**kwargs):
    """
    computes the average value of a function
    over a range
    requires import of numpy module
    input:
        f = function to be evaluated
        a = lower bound of range
        b = upper bound of range
        n = number of intervals
    output:
        favg = average value of function
    """
    x = np.linspace(a,b,n)
    y = f(x,**kwargs)
    favg = np.average(y)
    return favg

def func(t,m=70.0,cd=0.25):
    g = 9.81
    f = np.sqrt(m*g/cd)*np.tanh(np.sqrt(cd*g/m)*t)
    return f
```

```
print('{0:7.3f}'.format(funcavg(func,0,12,60,cd=0.3)))
```

When this is executed, the Console displays

```
34.322
```

The ability to pass parameters of a specific case through a generic function function to be evaluated in the specific function will prove useful in our implementation of numerical methods.

## 3.6 CASE STUDY

### BUNGEE JUMPER VELOCITY

#### Background

In this section, we will use Python to solve the free-falling bungee jumper problem we posed at the beginning of this chapter. This involves obtaining a solution of

$$\frac{dv}{dt} = g - \frac{c_d}{m} v|v|$$

Recall that, given an initial condition for time and velocity, the problem involved iteratively solving the formula,

$$v_{i+1} = v_i + \frac{dv_i}{dt} \Delta t$$

Now also remember that to attain good accuracy, we would employ small steps. Therefore, we would probably want to apply the formula repeatedly to step out from our initial time to attain the value at the final time. Consequently, an algorithm to solve the problem would be based on a loop.

#### Solution

Suppose that we started the computation at  $t = 0$  and wanted to predict the velocity at  $t = 12$  s using a time step of  $\Delta t = 0.5$  s. We would therefore need to apply the problem would be based on a loop. We would therefore need to apply the iterative equation 24 times, that is

$$n = \frac{12}{0.5} = 24$$

where  $n$  = the number of iterations of the loop. Because this result is exact, that is, the ratio is an integer, we can use a for loop as the basis of the algorithm. Here is a Python function to do this including a subfunction defining the differential equation:

```
def velocity1(dt,ti,tf,vi):
    """
    Solution of bungee jumper velocity
    by Euler's method
    Input:
        dt = time step (s)
        ti = initial time (s)
        tf = final time (s)
        vi = initial value of velocity (m/s)
    output:
        vf = velocity at tf (m/s)
    """
    t = ti
    v = vi
    n = int((tf-ti)/dt)
    for i in range(n):
        dvdt = deriv(v)
        v = v + dvdt*dt
```



```

        t = t + dt
    vf = v
    return vf

def deriv(v):
    g = 9.81
    m = 68.1
    cd = 0.25
    dv = g - cd/m * v * abs(v)
    return dv

```

The `velocity1` function can be invoked from the Console with the result

```

velocity1(0.5,0,12,0)
50.92590783030664

```

You will note in the code above that we use the `int` function to convert the calculation into an integer via truncation before assigning the result to `n`. This is because the `range` type in the next statement requires an integer argument.

We note that the true value obtained from the analytical solution is approximately 50.6175 (see [Example 3.1](#)). We can try a much smaller value of `dt` to obtain a more accurate numerical result.

```

velocity1(0.001,0,12,0)
50.61812389135902

```

Although this function is certainly simple to program, it is not foolproof. It is possible that the  $(t_f - t_i) / dt$  ratio is not close to an integral number of intervals. We can replace the `for` loop with a `while` loop that adjusts the final step to “hit” the end time exactly. Replace this code

```

n = int((tf-ti)/dt)
for i in range(n):
    dvdt = deriv(v)
    v = v + dvdt*dt
    t = t + dt
vf = v

```

with

```

h = dt
while True:
    if t + dt > tf: h = tf - t
    dvdt = deriv(v)
    v = v + dvdt*h
    t = t + h
    if t >= tf: break

```

and rename `velocity1` to `velocity2` in the `def` statement.

As the `while` loop is entered, a one-line `if` structure is used to determine if adding `dt` to the current time will take us past the end time, `tf`. If so, the step size, `h`, is reduced so that `tf` will be met exactly. If not, we continue with the original `h`. After the final adjustment of `t`, the final `if` statement will test true, and the `while` loop will be exited via the `break` command.

Notice that before entering the loop, we assign the value of the time step, `dt`, to another variable, `h`, so that our routine does not change the given value of `dt` if and when we shorten the last time step. We do this in anticipation that we might need to use the original value of `dt` somewhere else in the event that this code was integrated into a larger program.

If we run this new version, the result should be the same as for the version based on the `for` loop structure, using the same test. And it is

```

velocity2(0.5,0,12,0)
50.92590783030664

```

Further, we can use a step size that is not evenly divisible into  $t_f - t_i$ :

```
velocity2(0.35,0,12,0)
50.83478844076012
```

The result is slightly different because the approximation of the solution changes with step size. We should note that the algorithm is still not foolproof. For example, the user could have mistakenly entered a step size greater than the calculation interval, for example,  $t_f - t_i = 5$  and  $dt = 20$ . Thus, you might want to include error checking in your code to “trap” such errors and then allow the user to correct the mistake.

As a final note, we should recognize that the foregoing code is not generic. That is, we have designed it to solve the specific problem of the velocity of the bungee jumper. A more generic version can be developed as

```
def odesimp(dydt,dt,ti,tf,yi):
    t = ti ; y = yi ; h = dt
    while True:
        if t + dt > tf: h = tf - t
        y = y + dydt(y)*h
        t = t + h
        if t >= tf: break
    yend = y
    return yend
```

---

Page 76

---

Notice how we have stripped out the parts of the algorithm that were specific to the bungee jumper example (including the specific subfunction defining the differential equation) while keeping the essential feature of the solution technique. We can then use this routine to solve the bungee jumper example by specifying the differential equation with a function, perhaps a lambda function, and passing the function name to `odesimp` as the first argument in order to generate the solution.

```
dvd_t = lambda v: g - (cd/m)*v*abs(v)
odesimp(dvd_t,0.5,0,12,0)
50.92590783030664
```

We could then analyze a different function without having to go in and modify the `odesimp` function. For example, if  $y = 10$  at  $t = 0$ , the differential equation

$$\frac{dy}{dt} = -0.1y$$

has the analytical solution  $y = 10e^{-0.1t}$ . Therefore, the solution at  $t = 5$  would be  $y(5) = 10e^{-0.1(5)} = 6.0653$ . We can use `odesimp` to obtain a similar result numerically as in

```
odesimp(lambda y: -0.1*y,0.005,0,5,10)
6.064548228400564
```

Finally, we can use `*args` to allow passing parameters and develop a final and superior version. To do this, we create the `odesimp2` function from the `odesimp` function as shown below.

```
def odesimp2(dydt,dt,ti,tf,yi,*args):
    t = ti ; y = yi ; h = dt
    while True:
        if t + dt > tf: h = tf - t
        y = y + dydt(y,*args)*h
        t = t + h
        if t >= tf: break
    yend = y
    return yend
```

Then, instead of a lambda function, we have the option to create a function for `dvd_t` in the Editor:

```
def dvd_t(v,cd,m):
    g=9.81
    return g - (cd/m)*v*abs(v)
```

and add a statement to the script to solve for a specific case.

```
print('{0:7.3f}'.format(odesimp2(dvdt,0.5,0,12,0,0.25,68.1)))
```

The result in the Console appears as

```
50.926
```

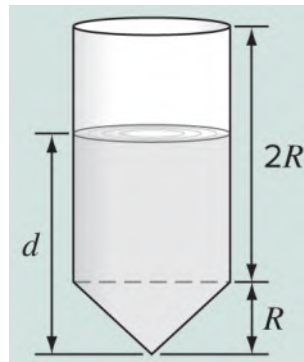
You can see that we took the time to add a more compact format to the display of the result. Since we used `*args`, we must supply values of `cd` and `m` as the last two arguments in the `odesimp2` function for this particular function evaluation. We could have used keyword arguments with default values. Then, `**kwargs` would replace `*args` in both places in the `odesimp2` function.

## PROBLEMS

**3.1**  **Figure P3.1** shows a cylindrical tank with a conical base. If the liquid level is quite low, in the conical part, the volume is simply the conical volume of liquid. If the liquid level is midrange, in the cylindrical part, the total volume of liquid includes the filled conical part and the partially filled cylindrical part. Use decisional structures to write a Python function to compute the tank's liquid volume given the values of tank radius,  $R$ , and liquid depth,  $d$ . The tank is designed so that the conical part has height  $R$  and the cylindrical part has height  $2R$ , as shown in the figure. Design the function so that it returns the liquid volume for all cases where the depth is less than  $3R$ . Return an error message, "Overtop," if  $d > 3R$ . Test your function with the following input data:

|     |     |      |     |     |
|-----|-----|------|-----|-----|
| $R$ | 0.9 | 1.5  | 1.3 | 1.3 |
| $d$ | 1.0 | 1.25 | 3.8 | 4.0 |

**FIGURE P3.1**



**3.2** An amount of money  $P$  is invested in an account where interest is compounded at the end of the period. The future worth  $F$  yielded at an interest rate  $i$  after  $n$  periods may be determined by the following formula:

$$F = P(1 + i)^n$$

Develop a Python function that calculates the future worth of an investment for each year from 1 to  $n$ . The input to the function should include the initial investment,  $P$ , the interest rate,  $i$  (as a decimal fraction, not a percentage), and the number of periods,  $n$ , over which the future worth is to be calculated. Note: If the periods are years, then the interest rate must be per year, not month. The output should consist of a table with headings and columns for periods and future worth. Test the program with  $P = 100,000$ ,  $i = 0.05$  per year, and  $n = 10$  years.

**3.3** Economic formulas are available to compute monthly payments for loans. Suppose that you borrow an amount of money,  $P$ , and agree to repay it in  $n$  monthly payments at a monthly interest rate,  $i$ , a decimal fraction, not a percentage. The formula to compute the monthly payment  $A$  is

$$A = P \frac{i(1+i)^n}{(1+i)^n - 1}$$

Note: Interest rates can be confusing. Loans are often quoted with an annual percentage rate or *APR*. The relationship between *APR* and monthly interest rate is

$$i = \frac{APR/100}{12}$$

There is a difference between the *APR* and annual single-payment percentage rate equivalent to 12 monthly payments at rate *i*. The latter would be

$$(1+i)^n - 1$$

As an example, an *APR* of 12% is a monthly *i* of 0.01. The annual equivalent turns out to be about 12.68%. Write a Python function to determine a monthly payment, given the amount (*P*), term (years), and *APR* in %. Consider you are obtaining an auto loan of the amount \$40,000 at an *APR* of 5.3%. Use your function to write a script that produces a table of term in years and monthly payment for 3-, 4-, 5- and 6-year loan.

**3.4** The average daily temperature, *T*, for an area can be approximated by the following formula:

$$T = T_{mean} + (T_{peak} - T_{mean}) \cos(\omega(t - t_{peak}))$$

where *t* = day of the year from 0 to 364, *T*<sub>mean</sub> = the average annual temperature, *T*<sub>peak</sub> = the peak temperature, *ω* = the frequency of the annual variation ( $2\pi/365$ ), and *t*<sub>peak</sub> = the day of the peak temperature ( $\cong 205$  d). Parameters for some U.S. cities are listed here.

| City     | <i>T</i> <sub>mean</sub> (°C) | <i>T</i> <sub>peak</sub> (°C) |
|----------|-------------------------------|-------------------------------|
| Miami    | 24.9                          | 37.0                          |
| Yuma     | 31.7                          | 42.0                          |
| Bismarck | 12.8                          | 29.4                          |
| Seattle  | 11.3                          | 22.8                          |
| Boston   | 10.9                          | 27.2                          |

Develop a Python function that computes the average temperature between two days of the year for a particular city. Test it for

- (a) January–February in Yuma (*t* = 0 to 59)
- (b) July–August in Seattle (*t* = 180 to 242)

**3.5** The sine function is equivalent to the following infinite series:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$$

Create a Python script that implements this formula so that it computes and displays the approximation to  $\sin(x)$  as each term in the series is added. In other words, compute the sequence of values for

$$\sin(x) = x$$

$$\sin(x) = x - \frac{x^3}{3!}$$

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!}$$

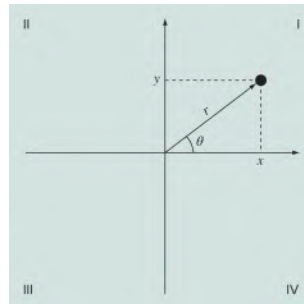
up to the order term of your choosing. For each of the preceding, compute and display the percent relative error as

$$\%error = \frac{\text{true value} - \text{series approximation}}{\text{true value}} \times 100$$

As a test case, employ the program to compute  $\sin(0.9)$  for up to and including eight terms, that is, up to the term  $x^{15}/15!$ .

**3.6** Two conditions are required to specify the location of a point relative to an origin in two-dimensional space ( **Fig. P3.6**).

**FIGURE P3.6**



- The horizontal and vertical distances  $(x, y)$  in Cartesian coordinates.
- The radius and angle  $(r, \theta)$  in polar coordinates.

It is relatively straightforward to compute Cartesian coordinates based on polar coordinates.

Page 78

The reverse process is not so simple. The radius can be computed by the following formula based on the Pythagorean Theorem,  $x^2 + y^2 = r^2$ ,

$$r = \sqrt{x^2 + y^2}$$

If the coordinates are within quadrants I and IV, that is,  $x > 0$ ,

$$\theta = \tan^{-1}\left(\frac{y}{x}\right) \quad \text{Note: The Math module implements this with the } \text{atan}(y/x) \text{ function.}$$

The angle here,  $\theta$ , is measured positively in quadrant I and negatively in quadrant IV. The difficulty arises for the other cases. The following table summarizes the possibilities:

| $x$   | $y$   | $\theta$               |
|-------|-------|------------------------|
| $< 0$ | $> 0$ | $\tan^{-1}(y/x) + \pi$ |
| $< 0$ | $< 0$ | $\tan^{-1}(y/x) - \pi$ |
| $< 0$ | $= 0$ | $\pi$                  |
| $= 0$ | $> 0$ | $\pi/2$                |
| $= 0$ | $< 0$ | $-\pi/2$               |
| $= 0$ | $= 0$ | $0$                    |

Write a Python function using *if...elif* structures to calculate  $r$  and  $\theta$  given values of  $x$  and  $y$  on the basis of all four quadrants. Test your program by evaluating the following cases:

| $x$ | $y$ | $r$ | $\theta$ |
|-----|-----|-----|----------|
| 2   | 0   |     |          |
| 2   | 1   |     |          |
| 0   | 3   |     |          |
| -3  | 1   |     |          |
| -2  | 0   |     |          |
| -1  | -2  |     |          |
| 0   | 0   |     |          |
| 0   | -2  |     |          |
| 2   | 2   |     |          |

The Math module has a built-in function  $\text{atan2}(y, x)$ . Test this function and compare it with yours. Note any differences.

Additional notes:

- (a) The value of  $\pi$  is available from the Math module as *math.pi*.
- (b) The NumPy module has functions  $\text{arctan}(y/x)$  and  $\text{arctan2}(y, x)$  that are equivalent to the Math module functions except that the NumPy functions allow vectorization.

**3.7** Develop a Python function to determine polar coordinates as described in [Prob. 3.6](#). However, rather than designing the function to evaluate a single case, pass vectors of  $x$  and  $y$ . Have the function display a table with labeled columns for  $x$ ,  $y$ ,  $r$ , and  $\theta$ . Test the program for the cases outlined in

[Prob. 3.6](#).

**3.8** Develop a Python function that is passed a numeric grade from 0 to 100 and returns a letter grade according to the scheme:

| Letter | Criterion                               |
|--------|-----------------------------------------|
| A      | $90 \leq \text{numeric grade} \leq 100$ |
| B      | $80 \leq \text{numeric grade} < 90$     |
| C      | $70 \leq \text{numeric grade} < 80$     |
| D      | $60 \leq \text{numeric grade} < 70$     |
| F      | $0 \leq \text{numeric grade} < 60$      |

The first line of the function should be

```
def lettergrade(score):
```

Design the function so that it displays an error message in the event that the user enters a score that is less than zero or greater than 100. Test your function with numerical scores of 89.9999, 90, 45, and 120.

**3.9** Manning's equation can be used to predict the velocity of water in a rectangular open channel:

$$U = \frac{\sqrt{S}}{n} \left( \frac{BH}{B + 2H} \right)^{2/3}$$

where  $U$  = velocity (m/s),  $S$  = channel slope,  $n$  = roughness coefficient,  $B$  = channel width (m), and  $H$  = depth (m). The following data are available for five channels:

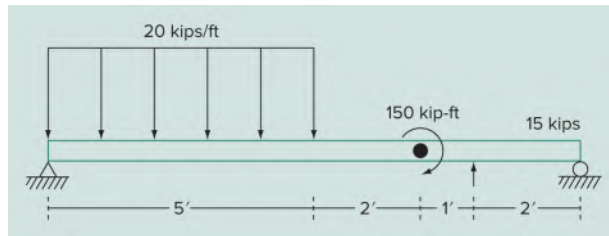
| $n$   | $S$    | $B$ | $H$ |
|-------|--------|-----|-----|
| 0.036 | 0.0001 | 10  | 2.0 |
| 0.020 | 0.0002 | 8   | 1.0 |
| 0.015 | 0.0012 | 20  | 1.5 |
| 0.030 | 0.0007 | 25  | 3.0 |
| 0.022 | 0.0003 | 15  | 2.6 |

Write a Python script that computes the velocity for each of these channels. Enter the table values into arrays for each parameter. Compute the velocities in vectorized fashion with one Python statement. Have the script display the input data along with the computed velocity in tabular form where velocity is the fifth column. Include headings on the table to label the columns.

**3.10** A simply supported beam is loaded as shown in  **Fig. P3.10**. Using singularity functions, the displacement along the beam can be expressed by the equation:

$$u_y(x) = -\frac{5}{6} \left[ \langle x - 0 \rangle^4 - \langle x - 5 \rangle^4 \right] + \frac{15}{6} \langle x - 8 \rangle^3 + 75 \langle x - 7 \rangle^2 + \frac{57}{6} x^3 - 238.25x$$



**FIGURE P3.10**

By definition, the singularity function can be expressed as follows:

$$\langle x - a \rangle^n = \begin{cases} (x - a)^n & \text{when } x > a \\ 0 & \text{when } x \leq a \end{cases}$$

Develop a Python script that creates a plot of displacement, as a dashed black line, versus distance along the beam. Note that  $x = 0$  at the left end of the beam.

Note in the figure that the kip is a non-SI unit representing 1,000 lb-force.

**3.11** The volume of liquid,  $V$ , in a partially filled, horizontal, cylindrical tank of radius  $r$  and length  $L$  is related to the depth of the liquid,  $h$ , at the center line of the cylinder by the formula

$$V = \left[ r^2 \cos^{-1} \left( \frac{r-h}{r} \right) - (r-h) \sqrt{2rh - h^2} \right] L$$

Develop a Python function to create a plot of liquid volume versus depth. Here are the first few lines:

```
import numpy as np
def cyltank(r,L,plot_title):
    """
    create a plot of the volume of liquid
    in a horizontal, cylindrical tank
    from empty to full tank
    inputs:
        r = inside radius of tank
        L = length
        plot_title = string for title of plot
    """
```

Test your program with

```
cyltank(3,5,'Volume vs.Depth for Horizontal Cylindrical Tank')
```

**3.12** Develop a vectorized version of the following Python script:

```
import numpy as np
tstart = 0
tend = 20
ni = 8
t = np.zeros(ni+1)
y = np.zeros(ni+1)
t[0] = tstart
y[0] = 12 + 6*np.cos(2*np.pi*t[0]/(tend-tstart))
```

```
for i in range(1,ni+1):
    t[i]=t[i-1]+(tend-tstart)/ni
    y[i]=12+6*np.cos(2*np.pi*t[i]/(tend-tstart))
```

**3.13** The “*divide and average*” method, an old-time iterative method for approximating the square root of any positive number,<sup>7</sup>  $a$ , can be formulated as

$$x_{i+1} = \frac{x_i + a/x_i}{2}$$

where the subscripts indicate iteration  $i$  to  $i + 1$ . The relative change term from one iteration to the next can be described as

$$\varepsilon_{i+1} = \left| \frac{x_{i+1} - x_i}{x_{i+1}} \right| \times 100\%$$

Write a well-structured Python function based on a while...break loop. Repeat the loop until the relative change is less than a specified tolerance. Test your program for a tolerance of 0.0001% and a values of 2, 10 and -3. For negative values of  $a$ , your function should return the imaginary result, that is, -4 returns a value close to  $2j$ .

Hint: You can create an imaginary value from a real value by multiplying it by  $1j$ .

**3.14** *Piecewise functions* are sometimes useful when the relationship between a dependent and independent variable cannot be adequately represented by a single equation. For example, the velocity of a rocket might be described by

$$v(t) = \begin{cases} 10t^2 - 5t & 0 \leq t \leq 8 \\ 624 - 3t & 8 < t \leq 16 \\ 36t + 12(t - 16)^2 & 16 < t \leq 26 \\ 2136e^{-0.1(t-26)} & t > 26 \\ 0 & \text{otherwise} \end{cases}$$

Develop a Python function to compute  $v$  with an input of  $t$ . Then, add a script that uses the function to generate a plot a  $v$  versus  $t$  for  $t = -5$  to 50.

**3.15** Develop a Python function called `rounder` to round a number  $x$  to a specified number of decimal digits,  $n$ . The first line of the function should be

```
def rounder(x,n):
```

When  $n$  is negative, the rounding should be to left of the decimal point, for example,  $n = -1$ , round to the tens. Test your function with the following:

| $x$       | $n$ |
|-----------|-----|
| 477.9587  | 2   |
| -477.9587 | 2   |
| 0.125     | 2   |

| $x$      | $n$ |
|----------|-----|
| 0.362945 | 4   |
| 8192     | -1  |
| -135782  | -3  |

**3.16** Develop a Python function to determine the elapsed days in a year. The function definition statement should be

Page 80

```
def days(mo,da,leap):
```

where `mo` = numeral for the month, 1-12, `day` = numeral for the day, 1-31, and `leap` = `True` for leap year and `False` for not.

**3.17** Determination of leap year in the current Western calendar is interesting. Leap year is every year divisible by four, except at the turn of the century, where, if the year is divisible by 400, there is no leap year. So, 1900 was a leap year, but 2000 was not. Develop a Python function that has an input of the year and returns a result of `True` if it is a leap year and `False` if it is not a leap year. Test it with various years to validate the function.

**3.18** Write a Python function with a function argument, a “function function,” that returns the difference between the passed function’s values for a minimum and maximum range of the function’s independent variable. Also, your function should generate a plot of the passed function over the specified range. Test your function for the following cases:

- (a)  $f(t) = 8e^{-0.25t}$   $0 \leq t \leq 6\pi$
- (b)  $f(x) = e^{4x} \sin\left(\frac{1}{x}\right)$   $0.01 \leq x \leq 0.2$
- (c)  $f(x) = \frac{1}{(x-0.3)^2 + 0.01} + \frac{1}{(x-0.9)^2 + 0.04} - 6$   $0 \leq x \leq 2$

**3.19** Modify the function `odesimp` developed at the end of [Sec. 3.6](#) so that it can be passed the arguments of the passed function. Test it for the following case:

```
dvdv = lambda v,m,cd: g - (cd/m)*v*abs(v)
odesimp(dvdv,0.5,0.12,-10,70,0.23)
```

**3.20** Develop a Python script to compute the velocity,  $v$ , and position,  $z$ , of a hot air balloon as described in [Prob. 1.28](#). Perform the calculation from  $t = 0$  to 60 s with a step size of 1.6 s. At  $z = 200$  m, assume that part of the payload (100 kg) is dropped out of the balloon. Your script should be structured as follows:

```
# hot air balloon script
import numpy as np
import pylab

g = 9.81 # m/s**2

# set parameters
r = 8.65 # balloon radius, m
cd = 0.47 # dimensionless drag coefficient
```

```

mp = 265 # initial mass of balloon and payload, kg
p = 101325 # atmospheric pressure at seal level, Pa
rgas = 287 # gas law constant for dry air. Pa*m**3/(kg*K)
tg = 100 # gas temperature, degC
tamb = 10 # ambient air temperature, degC
zd = 200 # elevation at which mass is jettisoned, m
md = 100 # mass jettisoned, kg
ti = 0 # initial time, s
tf = 60 # final time, s
vi = 0 # initial velocity, m/s
zi = 0 # initial elevation, m
dt = 1.6 # integration step size, s

# preliminary computations
d = 2*r # balloon diameter, m
tgabs = tg + 273.15 # absolute temperature of gas, K
tambabs = tamb + 273.15 # absolute temperature of ambient air, K
ab = np.pi * d**2 / 4 # cross-sectional area of balloon, m**2
vb = 4/3*np.pi*r**3 # balloon volume, m**3
rhog = p/rgas/tgabs # density of gas, kg/m**3
rhoa = p/rgas/tambabs # density of ambient air, kg/m**3
mg = rhog*vb # mass of gas in balloon, kg
cdd = rhoa*ab*cd/2 # dimensional drag coefficient, m/kg

# compute number of integration steps required
# create vectors for time, velocity, and elevation
# initialize the first elements of these vectors

# while loop to carry out Euler integration
# integrate dvdt to get v, v to get z, and advance t by dt
# adjust final step to match tf

# print out a well-formatted table of time, elevation, and velocity
# create a plot of elevation and velocity versus time with twin axes

```

### 3.21 A general equation for a sinusoid can be written as

$$y(t) = \bar{y} + \Delta y \sin(2\pi ft - \phi)$$

where  $y$  = dependent variable,  $\bar{y}$  = mean value,  $\Delta y$  = amplitude,  $f$  = frequency in cycles/time (when time in seconds, hertz or Hz),  $t$  = time, the independent variable, and  $\phi$  = phase shift.

Using Python, carry out a graphical study on how  $y(t)$  changes with changes in the parameters  $\bar{y}$ ,  $\Delta y$ ,  $f$ , and  $\phi$ . Also, relate the sine function to the cosine function. In particular, create plots over the range  $0 \leq t \leq 2\pi$  with two curves each for these cases:

$$\begin{array}{ll}
 y(t) = \sin(3\pi t) & y(t) = \sin(2\pi t) \\
 y(t) = \sin(2\pi t - \pi/4) & y(t) = \sin(2\pi t) \\
 y(t) = 0.5 + 1.2 \sin(2\pi t) & y(t) = \sin(2\pi t) \\
 y(t) = \cos(2\pi t - \pi/2) & y(t) = \sin(2\pi t)
 \end{array}$$

Additionally, create a polar plot of  $y(t)$  versus  $t$  for the base case above.

**3.22** A *fractal* is a curve or geometric figure, each part of which has the same statistical character as the whole. Fractals are useful in modeling structures, such as eroded coastlines or snowflakes, in which patterns recur at progressively smaller scales. They can also describe partly random or chaotic phenomena, such as crystal growth, turbulence in fluid flow, and galaxy formation. Devaney (1990) has

written a nice little book that includes a simple algorithm to create an interesting fractal pattern. Here is a step-by-step description of this algorithm:

1. Assign a value to variables  $m$  and  $n$ .
2. Start a for loop to iterate  $i$  10,000 times.
3. Compute a random number,  $q$ , from a uniform distribution between 0 and 3.
4. If the value of  $q$  is less than 1 go to Step 5; otherwise, go to Step 6.
5. Compute new values  $m = m/2$  and  $n = n/2$  and go to Step 9.
6. If the value of  $q$  is less than 2, go to Step 7; otherwise, go to Step 8.
7. Compute new values  $m = m/2$  and  $n = (300 + n)/2$ . Go to Step 9.
8. Compute new values  $m = (300 + m)/2$  and  $n = (300 + n)/2$ .
9. If  $i$  is less than 1000, go to Step 10; otherwise, go to Step 11.
10. Plot a point at the coordinates  $(m, n)$ .
11. Terminate  $i^{\text{th}}$  iteration.

Develop a script for this algorithm using for loop and if structures. Run it for the following two cases:

- (a)  $m = 2, n = 1$
- (b)  $m = 100, n = 200$

**3.23** Write a well-structured Python function called `fnorm` to calculate the Frobenius norm of an  $m \times n$  matrix.

$$\|A\|_f = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2}$$

Here is an example script that uses the function.

```
A = np.matrix(' 5 7 9 ; 1 8 4 ; 7 6 2')
fn = fnorm(A)
print(fn)
```

**3.24** The pressure and temperature of the earth's atmosphere are constantly changing depending on several factors including altitude, latitude/longitude, time of day, and season. To take all these variations into account when designing flight vehicles is impractical. Therefore, a standard atmosphere is frequently used to provide engineers and scientists with a common reference for their research and development efforts. The *International Standard Atmosphere* is one such model of how conditions of the earth's atmosphere change over a wide range of altitudes or elevations. The following table shows the values of temperature and pressure at selected altitudes.

|                        |               | Base<br>Geopotential               |                          |                                 |                              |
|------------------------|---------------|------------------------------------|--------------------------|---------------------------------|------------------------------|
| Layer<br>Index,<br>$i$ | Layer<br>Name | Altitude<br>above MSL,<br>$h$ (km) | Lapse<br>Rate<br>(°C/km) | Base<br>Temperature<br>$T$ (°C) | Base<br>Pressure<br>$P$ (Pa) |

| Layer Index, <i>i</i> | Layer Name   | Base Geopotential                 |  | Lapse Rate (°C/km) | Base Temperature <i>T</i> (°C) | Base Pressure <i>P</i> (Pa) |
|-----------------------|--------------|-----------------------------------|--|--------------------|--------------------------------|-----------------------------|
|                       |              | Altitude above MSL, <i>h</i> (km) |  |                    |                                |                             |
| 1                     | Troposphere  | 0                                 |  | −6.5               | 15.0                           | 101325                      |
| 2                     | Tropopause   | 11                                |  | 0                  | −56.5                          | 22632                       |
| 3                     | Stratosphere | 20                                |  | 1                  | −56.5                          | 5474                        |
| 4                     | Stratosphere | 32                                |  | 2.8                | −44.5                          | 8680                        |
| 5                     | Stratopause  | 47                                |  | 0                  | −2.5                           | 11090                       |
| 6                     | Mesosphere   | 51                                |  | −2.8               | −2.5                           | 66.93                       |
| 7                     | Mesosphere   | 71                                |  | −2.0               | −58.5                          | 3.956                       |
| 8                     | Mesopause    | 84.852                            |  | —                  | −86.28                         | 0.375                       |

The temperature at each altitude can then be computed as

$$T(h) = T_i + \gamma_i(h - h_i) \quad h_i \leq h \leq h_{i+1}$$

where  $T(h)$  = temperature at altitude  $h$  (°C),  $T_i$  = the base temperature for layer  $i$  (°C),  $\gamma_i$  = the lapse rate or the rate at which the temperature changes linearly with altitude for layer  $i$  (°C/km), and  $h_i$  = the base geopotential altitude above mean sea level (MSL) for layer  $i$  (km).

The pressure at each altitude can then be computed as

$$p(h) = p_i + \frac{P_{i+1} - P_i}{h_{i+1} - h_i}(h - h_i)$$

where  $p(h)$  = the pressure at altitude  $h$  (Pa),  $p_i$  = the base pressure for layer  $i$  (Pa). The density,  $\rho$  (kg/m<sup>3</sup>), can then be calculated from the ideal gas law

$$\rho = \frac{MWp}{RT_a}$$

where  $MW$  = the average molecular weight of air (28.96 kg/kg mol),  $R$  = the gas law constant (8314.5 Pa·m<sup>3</sup>/(kgmol·K), and  $T_a$  = the absolute temperature (K).

Develop a Python function, `stdatm`, to determine values of the three properties for a given altitude. If the user requests a value for an altitude outside the range of the table, the function should return an error message. Use the following script to create plots of temperature, pressure and density versus altitude. For each plot, have altitude on the vertical axis.

```
"""
Script to generate a plot of
temperature, pressure and density
for the International Standard Atmosphere
```

```

"""
import numpy as np
import pylab

h = np.array([0,11,20,32,47,51,71,84.852])
gamma = np.array([-6.5,0,1,2.8,0,-2.8,-2])
t = np.array([15,-56.5,-56.5,-44.5,-2.5,-2.5,-58.5,-86.28])
p = np.array([101325,22632,5474.9,868.02,110.91,66.939,3.9564,0.3734])
hint = np.arange(0,84.852,0.1)
n = len(hint)
tint = np.zeros(n)
pint = np.zeros(n)
rhohint = np.zeros(n)
for i in range(n):
    tint[i],pint[i],rhohint[i]=stdatm(h,t,p,gamma,hint[i])

# add code to create plots

# check error trapping with this command
ttest,ptest,rhotest = stdatm(h,t,p,gamma,85)

```

**3.25** Develop a Python function to convert a vector of temperatures from Celsius to Fahrenheit and vice versa. Test it with the following data for the average monthly temperatures at Death Valley, CA, and at the South Pole.

| Death Valley |    |     | South Pole |     |     |
|--------------|----|-----|------------|-----|-----|
| Day          | °F | °C  | Day        | °F  | °C  |
| 15           | 54 | -27 | 195        | 102 | -59 |
| 45           | 60 | -40 | 225        | 101 | -59 |
| 75           | 69 | -53 | 255        | 92  | -59 |
| 105          | 77 | -56 | 285        | 78  | -50 |
| 135          | 87 | -57 | 315        | 63  | -38 |
| 165          | 96 | -57 | 345        | 52  | -27 |

Use the following script as a starting point to generate two plots of temperatures versus day number, one displaying °F units and the second displaying °C units. Show curves for both locations on each plot. If the user requests a unit other than 'C' or 'F', provide them with the opportunity to correct that input.

```

"""
Script to generate plots of temperatures versus day of the year
for Death Valley and the South Pole.
Two plots generated: 1) using degC 2) using degF.
Both locations included on each plot.
Function tempconv accepts arrays of temperature input.
"""

import numpy as np
import pylab

# Add code to create function tempconv(tempin,scalecode) where scalecode is
# the temperature scale to which tempin will be converted.
day = np.array([15,45,75,105,135,165,195,225,255,285,315,345])
tfdv = np.array([54,60,69,77,87,96,102,101,92,78,63,52])

```

```

tcsp = np.array([-27,-40,-53,-56,-57,-57,-59,-59,-59,-50,-38,-27])
n = len(day)
tcdv = np.zeros(n)
tfsp = np.zeros(n)
tcdv = tempconv(tfdv,'C')
tfsp = tempconv(tcsp,'F')

# add code to create plots

# test your error check with
testtemp = tempconv(1200,'K')
print('{0:6.1f}'.format(testtemp))

```

**3.26** In  **Prob. 3.25**, there were only two choices for the temperature units, °C and °F. We could have included degrees Rankine (°R) and Kelvins (K) for absolute temperature, and that would have added some complexity to our tempconv function. When dealing with pressure, the situation is far more confusing because of the number of different units in common use. Here are some of the possibilities along with their SI equivalent in pascals (Pa).

| Index,<br><i>i</i> | Unit, <i>U<sub>i</sub></i> | Common Usage                              | Conversion to<br>Pa, <i>C<sub>i</sub></i> |
|--------------------|----------------------------|-------------------------------------------|-------------------------------------------|
| 1                  | psi                        | wide use in the U.S., tire pressure       | 6894.76                                   |
| 2                  | atm                        | high pressures, science                   | 101325                                    |
| 3                  | inHg                       | meteorology, barometric pressure          | 3376.85                                   |
| 4                  | kg/cm <sup>2</sup>         | wide use in Europe                        | 98066.5                                   |
| 5                  | inH <sub>2</sub> O         | heating/ventilating/AC in the U.S.        | 248.843                                   |
| 6                  | Pa                         | SI standard, N/m <sup>2</sup>             | 1                                         |
| 7                  | bar                        | wide use in science                       | 100000                                    |
| 8                  | dyne/cm <sup>2</sup>       | CGS unit, former use in science           | 0.1                                       |
| 9                  | ftH <sub>2</sub> O         | hydraulics, “head” measurement            | 2988.98                                   |
| 10                 | mmHg                       | barometric, former use in science         | 133.322                                   |
| 11                 | torr                       | vacuum measurement, same as mmHg          | 133.322                                   |
| 12                 | ksi                        | structural eng’g in the U.S., same as psi | 6894.76                                   |

There is an additional distinction when it comes to actual, measured pressures. The table above describes how to convert one of the units to Pa, but not necessarily a measured pressure in one of the units to Pa. The concept of gauge pressure versus absolute pressure comes into play. Gauge pressure is the pressure above (or below) ambient pressure. Absolute pressure is gauge pressure plus ambient pressure. For example, psig indicates gauge pressure; whereas psia stands for absolute pressure. At nominal sea-level conditions, psia = psig + 14.696. In this problem, we will consider all pressures



without consideration of gauge or absolute. The information in the table above can be used to implement a unit conversion calculation. One way to do this is to store both the units code and the corresponding number of pascals in individual arrays with matching subscripts. As examples,

```
U[1] = 'psi'      C[1] = 6894.76
U[2] = 'atm'     C[2] = 101325
```

The conversion of a given unit in the table to another can then pass through conversion to Pa and back to the desired unit. The general formula for this would be

$$P_d = \frac{C_i}{C_j} P_g$$

where  $P_g$  = pressure in given units,  $P_d$  = pressure in desired units,  $j$  = index of desired unit,  $i$  = index of given unit. As an example, to convert a tire pressure of 28.6 psi to atm, we would use

$$P_d = \frac{C_i}{C_j} P_g = \frac{6894.76 \text{ Pa/psi}}{101325 \text{ Pa/atm}} 28.6 \text{ psi} = 1.95 \text{ atm}$$

So, we see that the conversion from a pressure in one unit to another involves first determining the indices corresponding to the given and desired units, and then implementing the conversion formula. Here is a step-by-step algorithm to do this:

1. Assign the values of the unit,  $U$ , and conversion,  $C$ , arrays.
2. Have the user select the input units by entering the value of  $i$ . If the user enters a correct value within the range 1-12, continue to Step 3. If the user enters a value outside the range, display an error message and repeat this step.
3. Have the user enter the given pressure value,  $P_g$ .
4. Have the user select the desired units by entering the value of  $j$ . If the user enters a correct value within the range 1-12, continue to Step 5. If the user enters a value outside the range, display an error message and repeat this step.
5. Use the formula and the  $i$  and  $j$  indices to convert the quantity in given input units to its value in the desired output units.
6. Display the pressure and given units and the pressure and desired units.
7. Ask the user if another output result with different desired units for the same input pressure and units is desired. If yes, go back to Step 4. If no, go on to Step 8.
8. Ask if another conversion with new pressure and given units is desired. If yes, go back to Step 2. If no, end the algorithm.

Develop a well-structured Python script using decision and repetition structures to implement this algorithm. Test your script for the following cases:

- (a) Duplicate the example above to ensure that you get a value of 1.95 atm from an input of 28.6 psi.
- (b) Try to enter a choice code for  $i$  of 13. Does the program trap this error and allow it to be corrected? Then, try a choice code of  $Q$ . What happens then?
- (c) For the yes/no questions in the program requiring an input of  $y$  or  $n$ , try a different letter. Does the program scold you and allow you to correct your response? What if you answer  $Y$  and not  $y$ ?

Should it be forgiving? What happens if you enter a number like 1.3 instead of  $y$  or  $n$ ? What happens if you press the Enter key without entering  $y$  or  $n$ ? If your script has problems with these tests, can you modify it to resolve those problems?

(d) Illustrate the following conversions:

20 bar to psi

75 ftH<sub>2</sub>O to atm

$1 \times 10^{-4}$  torr to Pa

# Roundoff and Truncation Errors

## CHAPTER OBJECTIVES

The primary objective of this chapter is to acquaint you with the major sources of errors involved in numerical methods. Specific objectives and topics covered are

- Understanding the distinction between accuracy and precision.
- Learning how to quantify error.
- Learning how error estimates can be used to decide when to terminate an iterative calculation.
- Understanding how roundoff errors occur because digital computers have a limited ability to represent numbers.
- Understanding why floating-point numbers have limits on their range and precision.
- Recognizing that truncation errors occur when exact mathematical formulations are represented by approximations.
- Knowing how to use the Taylor series to estimate truncation errors.
- Understanding how to write forward, backward, and centered finite-difference approximations to first and second derivatives.
- Recognizing that efforts to minimize truncation errors can sometimes increase roundoff errors.

## YOU'VE GOT A PROBLEM

In  **Chap. 1**, you developed a numerical model for the velocity of a bungee jumper. To solve the problem with a computer, you had to approximate the derivative of velocity with a finite difference:

$$\frac{dv}{dt} \cong \frac{\Delta v}{\Delta t} = \frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i}$$

Thus, the resulting solution is not exact—that is, it has error.

In addition, the computer you use to obtain the solution is also an imperfect tool. Because it is a digital device, the computer is limited in its ability to represent the magnitudes and precision of numbers. Consequently, the machine itself yields results that contain error.

So, both your mathematical approximation and your digital computer cause your resulting model prediction to be uncertain. Your problem then is: How do you deal with such uncertainty? In particular, is it possible to understand, quantify, and control such errors in order to obtain acceptable results? This

chapter introduces you to some approaches and concepts that engineers and scientists use to deal with this dilemma.

## 4.1 ERRORS

Engineers and scientists constantly find themselves having to accomplish objectives and make decisions based on uncertain information. Although perfection is a laudable goal, it is rarely if ever attained. For example, even though the model developed from Newton's second law is an excellent approximation, it would never in practice exactly predict the bungee jumper's fall. A variety of factors, such as winds and slight variations in air resistance, would result in deviations from the prediction. Additionally, there are errors associated with making measurements of the jumper's fall. If these deviations are systematically high or low, then we might need to develop a new model. However, if they are randomly distributed and tightly grouped around the prediction, then the deviations might be considered negligible and the modeled deemed adequate. Numerical approximations also introduce similar discrepancies into the analysis.

This chapter covers basic topics related to the identification, quantification, and minimization of these errors. General information concerned with the quantification of error is reviewed in this section. This is followed by [Secs. 4.2](#) and [4.3](#), dealing with the two major forms of numerical error: roundoff error (due to computer approximations) and truncation error (due to mathematical approximations). We also describe how strategies to reduce truncation error sometimes increase roundoff. Finally, we briefly discuss errors not directly connected with the numerical methods themselves. These include experimental blunders, model errors, and data-related errors.

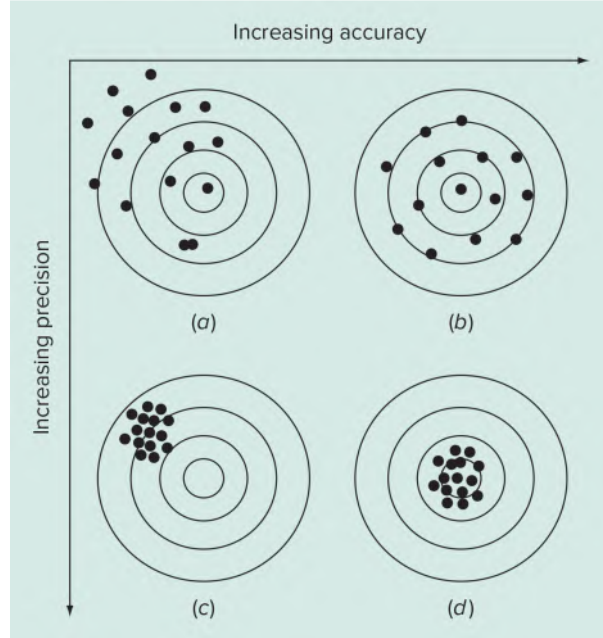
### 4.1.1 Accuracy and Precision

The errors associated with both numerical calculations and experimental measurements can be characterized with regard to their accuracy and precision. *Accuracy* refers to how closely a computed or measured value agrees with the true value. *Precision* refers to how closely individual computed or measured values agree with each other.

These concepts can be illustrated graphically using an analogy from target practice. The bullet holes on each target in [Fig. 4.1](#) can be thought of as the predictions of a numerical technique; whereas, the bull's eye represents the truth. *Inaccuracy* (also called *bias*) is defined as systematic deviation from the truth. Thus, although the shots in [Fig. 4.1c](#) are more tightly grouped than in [Fig. 4.1a](#), the two cases are equally biased because they are both centered in the upper left quadrant of the target. *Imprecision* (also called *uncertainty*), on the other hand, refers to the magnitude of the scatter. Therefore, although [Figs. 4.1b](#) and [4.1d](#) are equally accurate, that is, centered on the bull's eye, the latter is more precise because the shots are tightly grouped.

---

**FIGURE 4.1** An example from marksmanship illustrating the concepts of accuracy and precision: (a) inaccurate and imprecise, (b) accurate and imprecise, (c) inaccurate and precise, and (d) accurate and precise.



Numerical methods should be sufficiently accurate or unbiased to meet the requirements of a particular problem. They also should be precise enough for adequate design. In this book, we will use the collective term *error* to represent both the inaccuracy and imprecision of our predictions.

## 4.1.2 Error Definitions

Numerical errors arise from the use of approximations to represent exact mathematical descriptions and operations. For such errors, the relationship between the exact, or true, result and the approximation can be formulated as

$$\text{true value} = \text{approximation} + \text{error} \quad (4.1)$$

By rearranging  Eq. (4.1), we find that the numerical error is equal to the discrepancy between the truth and the approximation, as in

$$E_t = \text{true value} - \text{approximation} \quad (4.2)$$

where  $E_t$  is used to designate the exact value of the error. The subscript “*t*” is included to designate that this is the “true” error. This contrasts with other cases, as described shortly, where an “approximate” estimate of the error must be employed. Note that the true error is commonly expressed as an absolute value and referred to as the *absolute error*.

A shortcoming of this definition is that it takes no account of the order of magnitude of the value under examination. For example, an error of a centimeter is much more significant if we are measuring a rivet than a bridge. One way to account for the magnitudes of the quantities being evaluated is to normalize the error to the true value, as in

$$\text{true fractional relative error} = \frac{\text{true value} - \text{approximation}}{\text{true value}}$$

Although this measure is useful, it is typically modified in two ways. First, the sign of this equation may be either positive or negative. If the approximation is greater than the true value, the error will be negative; whereas, if the approximation is less than the true value, the error will be positive. The denominator may be less than zero, which can also lead to a negative error. When performing computations, we are often unconcerned with the sign of the error but are more interested in its magnitude. Second, the relative fractional error is typically multiplied by 100% to express it as a percent error. The result of both modifications yields

$$\varepsilon_t = \left| \frac{\text{true value} - \text{approximation}}{\text{true value}} \right| \times 100\% \quad (4.3)$$

where  $\varepsilon_t$  is the absolute true percent relative error.

For example, suppose that you have the task of measuring the lengths of a bridge and a rivet and come up with 9999 and 9 cm, respectively. If the true values are 10,000 and 10 cm, respectively, the error in both cases is 1 cm. However, their absolute percent relative errors can be computed using [Eq. \(4.3\)](#) as 0.01% and 10%, respectively. Thus, although both measurements have an absolute error of 1 cm, the relative error of the rivet is much greater. We would probably conclude that we have done an adequate job of measuring the bridge, whereas our estimate for the rivet leaves something to be desired.

Notice that for [Eqs. \(4.2\)](#) and [Eq. \(4.3\)](#),  $E$  and  $\varepsilon$  are subscripted with a  $t$  to signify that the error is based on the true value. For the example of the rivet and the bridge, we were provided with this value. However, in actual situations such information is rarely available. For numerical methods, the true value will only be known when we deal with functions that can be solved analytically. Such will typically be the case when we investigate the theoretical behavior of a particular technique for simple systems. However, in real-world applications, we will obviously not know the true answer *a priori*. For these situations, an alternative is to normalize the error using the best available estimate of the true value, that is, to the approximation itself, as in

$$\varepsilon_a = \left| \frac{\text{approximation error}}{\text{approximation}} \right| \times 100\% \quad (4.4)$$

where the subscript  $a$  signifies that the error is normalized to an approximate value. Note also that, for real-world applications, [Eq. \(4.2\)](#) cannot be used to calculate the error term in the numerator

of  **Eq. (4.4)**. One of the challenges of numerical methods is to determine error estimates in the absence of knowledge regarding the true value. For example, certain numerical methods use *iteration* to compute answers. In such cases, a present approximation is made based on a previous approximation. This process is performed repeatedly, or iteratively, to compute successively better and better (hopefully) approximations. For such cases, the error is often estimated as the difference between the previous and present approximations. Thus, the percent relative error is determined according to

$$\varepsilon_a = \left| \frac{\text{present approximation} - \text{previous approximation}}{\text{present approximation}} \right| \times 100\% \quad (4.5)$$

This and other approaches for expressing errors are elaborated in subsequent chapters.

When performing an iterative numerical method, we must consider when to stop iterating. To do this, we must specify an acceptable value of the percent relative error,  $\varepsilon_s$ . In such cases, the computation is repeated until

$$\varepsilon_a < \varepsilon_s \quad (4.6)$$

This relationship is referred to as a *stopping criterion*. If it is satisfied, our result is assumed to be within the prespecified acceptable level  $\varepsilon_s$ .

It is also convenient to relate these errors to the number of significant figures in the approximation. It can be shown (Scarborough, 1966) that, if the following criterion is met, we can be assured that the result is correct to at least  $n$  significant figures.

$$\varepsilon_s = (0.5 \times 10^{2-n})\% \quad (4.7)$$

## EXAMPLE 4.1

### Error Estimates in Iterative Methods

#### Problem Statement

In mathematics, functions can often be represented by infinite series. For example, the exponential function can be computed, approximately, using

$$e^x \cong 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!} \quad (\text{E4.1.1})$$



Thus, as more terms are added to the sequence, the approximation becomes a better and better estimate of the true value of  $e^x$ . [Eq. \(E4.1.1\)](#) is called a *Maclaurin series expansion*.

Starting with the simplest version,  $e^x \cong 1$ , add terms one by one in order to estimate  $e^{0.5}$ . After each new term is added, compute the true and approximate percent relative errors with [Eqs. \(4.3\)](#) and [\(4.5\)](#), respectively. Note that the true value is closely approximated to 15 significant figures as  $e^{0.5} \cong 1.64872127070013$ . Add terms until the absolute value of the approximate error estimate  $\varepsilon_a$  falls below a prespecified error criterion,  $\varepsilon_s$ , corresponding to three significant figures.

## Solution

First, [Eq. \(4.7\)](#) can be employed to determine the error criterion that ensures a result that is correct to at least three significant figures:

$$\varepsilon_s = (0.5 \times 10^{2-3})\% = 0.05\%$$

Thus, we will add terms to the series until  $\varepsilon_a$  falls below this level.

The first estimate is simply to [Eq. \(E4.1.1\)](#) with a single term. Thus, the first estimate is equal to 1. The second estimate is then generated by adding the second term as in

$$e^x = 1 + x$$

or for  $x = 0.5$

$$e^{0.5} = 1 + 0.5 = 1.5$$

This represents a true percent relative error of [Eq. \(4.3\)](#)

$$\varepsilon_t = \left| \frac{1.64872127070013 - 1.5}{1.64872127070013} \right| \times 100\% \cong 9.02\%$$

[Equation \(4.5\)](#) can be used to determine an approximate estimate of the error, as in

$$\varepsilon_a = \left| \frac{1.5 - 1}{1.5} \right| \times 100\% \cong 33.3\%$$

Because  $\varepsilon_a$  is not less than the required value of  $\varepsilon_s$ , we would continue the computation by adding another term,  $x^2/2!$ , and repeating the error calculations. The process is continued until

$\varepsilon_a < \varepsilon_s$ . The entire computation can be summarized as

| Terms | Result      | $\varepsilon_t, \%$ | $\varepsilon_a, \%$ |
|-------|-------------|---------------------|---------------------|
| 1     | 1           | 39.3                |                     |
| 2     | 1.5         | 9.02                | 33.3                |
| 3     | 1.625       | 1.44                | 7.69                |
| 4     | 1.645833333 | 0.175               | 1.27                |
| 5     | 1.648437500 | 0.0172              | 0.158               |
| 6     | 1.648697917 | 0.00142             | 0.0158              |

Thus, after six terms are included, the approximate error falls below  $\varepsilon_s = 0.05\%$ , and the computation is terminated. However, notice that, rather than three significant figures, the result is accurate to five, rounding the term 6 result back to four decimal places. This is because, for this case, both [Eqs. \(4.5\)](#) and [\(4.7\)](#) are conservative. That is, they ensure that the result is at least as good as they specify. Although, this is not always the case for [Eq. \(4.5\)](#), it is true most of the time.

## 4.1.3 Computer Algorithm for Iterative Calculations

Many of the numerical methods described in the remainder of this text involve iterative calculations of the sort illustrated in [Example 4.1](#). These all entail solving a mathematical problem by computing successive approximations to the solution starting from an initial guess.

The computer implementation of such iterative solutions involves loops. As we saw in [Sec. 3.3.3](#), these come in two basic flavors: *count-controlled* and *general decision loops*. Most iterative solutions use decision loops. Thus, rather than employing a prespecified number of iterations, the process typically is repeated until an approximate error estimate falls below a stopping criterion, as in [Example 4.1](#).

To do this for the same problem as [Example 4.1](#), the series expansion can be expressed as

$$e^x \cong \sum_{i=0}^n \frac{x^i}{i!}$$

A Python function to implement this formula is shown in [Fig. 4.2](#). The function is passed the value to be evaluated, `x`, along with a stopping error criterion, `es`, and a maximum allowable number of iterations, `maxit`. If the function is invoked without the last two arguments, the default values specified in the `def` statement will be used. If `maxit` is to be specified with the default `es` accepted, the keyword argument, `maxit=...`, must be included in the function call.

**FIGURE 4.2** A Python function to solve an iterative calculation. This example is set up to evaluate the Maclaurin series expansion for  $e^x$  as described in [Example 4.1](#).

```
import math
def termeth(x,es=1e-4,maxit=50):
    """
    Maclaurin series expansion of the exponential function requires
    math module
    input:
        x = value at which the series is evaluated
        es = stopping criterion (default = 1e-4)
        maxit = maximum number of iterations (default=10)
    output:
        fx = estimated function value
        ea = approximate relative error (%)
        iter = number of iterations
    """
    # initialization
    iter = 1 ; sol = 1 ; ea = 100
    # iterative calculation
    while True:
        solold = sol
        sol = sol + x**iter / math.factorial(iter)
        iter = iter + 1
        if sol != 0: ea = abs((sol-solold)/sol)*100
        if ea < es or iter == maxit: break
    fx = sol
    return fx,ea,iter
```

The function initializes three variables: (a) `iter`, set to 1, which keeps track of the number of iterations, (b) `sol`, set to the first term of the expansion, 1, which holds the current value of the solution, and (c) a variable `ea`, set initially to 100%, which holds the approximate percent relative error. Note that the initial setting of `ea` ensures that the loop will execute at least once.

These initializations are followed by a decision loop that implements the iterative technique. Prior to generating a new solution, the previous value of `sol` is assigned to `solold`. Then, a new value of `sol` is computed by adding the appropriate series term for iteration `iter`, and the iteration counter is incremented. If the new value of `sol` is nonzero, the percent relative error, `ea`, is determined. The stopping criterion and iteration limit are then tested. If both are false, the loop repeats. If either is true, the loop terminates, and the solution is returned.

When the `termeth` function is executed, it can then be invoked from the Console (or from other code in the Editor window). It will generate a tuple of the estimate for the exponential function along with the approximate error and the number of iterations required. For example,

```
termeth(1,1e-6,100)
(2.718281826198493, 9.216155641522974e-07, 12)
```

If we want to access the elements of the returned solution instead of the tuple, we can use this code in the Console:

```
approxval,ea,iter = termeth(1,1e-6,100)
approxval
2.718281826198493
ea
9.216155641522974e-07
iter
12
```

We can see that after 12 iterations, we obtain a result of 2.718281826198493 with a small approximate error  $\cong 9.2162 \times 10^{-7}\%$ . This result can be verified by using the `exp` function from the `Math` module. If we accept this result as the true value (It is computed in the background by a similar iterative technique with a very small stopping criterion.), we can compare our result and compute the “true” percent relative error by entering these commands in the Console:

```
import math
truval = math.exp(1)

truval
2.718281828459045

et = abs((truval-approxval)/truval)*100

et
8.316106763523326e-08
```

By inspection, you can see that our result deviates from the “true” result in the ninth decimal place, and, as was the case in [Example 4.1](#), we obtain the desirable outcome that the true error is less than the approximate error.

It is good practice to test our function with various inputs, including input values that will cause the iteration limit to be reached. See the examples below. Page 90

```
itermeth(0.5,1e-7)
(1.6487212706873655, 1.632262003529942e-08, 11)

itermeth(0.5,1e-7,10)
(1.648721270418251, 3.264523199531576e-07, 10)
```

Notice in the first case, that the error criterion is met and the number of iterations required is 11. However, in the second case, the iteration limit is set to 10. Of course then, the iteration limit is reached and the error criterion is not met. Consequently, it is necessary to test the returned iteration value against the specified (or default) limit to determine whether the limit was reached without meeting the error criterion.

Another example is to accept the default error criterion but specify the iteration limit.

```
itermeth(0.5,maxit=10)
(1.6487211681547618, 9.40182752709793e-05, 8)
```

We have had to use a *keyword* with the iteration limit argument because we left the stopping error criterion argument out.

Another question that arises here is when might `sol` evaluate to zero. Consider the case of  $x = -1$ . On the first iteration,

```
sol = 1 + (-1)**1/1! = 0
```

Subsequently, however, nonzero terms would be added. So, the “divide by zero” protection is needed here.

```
itermeth(-1,1e-6,100)
(0.3678794413212817, 5.674890913943406e-07, 13)
```

We observe that one more iteration was required here compared to the  $x = 1$  case. This would have to do with the alternating signs of the terms added to the approximation.

## 4.2 ROUND OFF ERRORS

---

Roundoff errors arise because digital computers cannot represent some numerical quantities exactly. They are important to engineering and scientific problem solving because they can lead to erroneous results. In certain cases, they can cause a calculation to go unstable and yield obviously spurious results. Such calculations are said to be *ill-conditioned*. Worse still, they can lead to subtle discrepancies that are difficult to detect.

There are two major facets of roundoff errors involved in numerical calculations:

1. Digital computers have magnitude and precision limits on their ability to represent numbers.
2. Certain numerical manipulations are highly sensitive to roundoff errors. This can result from both mathematical considerations and the way in which computations perform arithmetic operations.

### 4.2.1 Computer Number Representation

Numerical roundoff errors are directly related to the way numbers are stored in a computer. The fundamental unit whereby information is represented is called a *word*. This is an entity that consists of a string of *binary digits*, or *bits*. In modern computers, the common word contains eight bits and is called a *byte*. Numbers are typically stored in several bytes. To understand how this is accomplished, we must first review some material related to number systems.

A number system is merely a convention for representing quantities. Because we have 10 fingers and 10 toes, the number system we are most familiar with is the *decimal*, or *base-10*, system. A base is the number used as the reference for constructing the system. The base-10 system uses the 10 digits<sup>1</sup>—0,1,2,3,4,5,6,7,8,9—to represent numbers. By themselves, these digits are satisfactory for counting 0 to 9.

For larger quantities, combinations of these basic digits are used, with the positions or *place value* specifying the magnitude. The rightmost digit in a whole number represents a number from 0 to 9. The second digit from the right represents the numbers 0, 10, 20, . . . 90 or a multiple of 10 of the base digits. The third digit from the right represents a multiple of 100 and so on. For example, if we have the number 8642.9, then we have 8 groups of 1000, 6 groups of 100, 4 groups of 10, two groups of 1, and nine groups of 0.1 or

$$(8 \times 10^3) + (6 \times 10^2) + (4 \times 10^1) + (2 \times 10^0) + (9 \times 10^{-1}) = 8642.9$$

This type of representation is called *positional notation*.

Now, because the decimal system is so familiar, it is not commonly realized that there are alternatives. For example, if we excluded thumbs or big toes, or if humans only had 8 fingers and toes, we would undoubtedly have developed a *base-8*, or *octal*, representation. In the same sense, our computer is like a two-fingered animal who is limited to two states—either 0 or 1 (alternately, off or on, 0 volts or 3 volts). This relates to the fact that the primary logic circuits of digital computers are on/off electrical components. Hence, numbers in the computer are represented with a *binary*, or *base-2*, system. Just as

with the decimal system, quantities can be represented using positional notation. For example, the binary number  $101.1_2$  is equivalent to

$$(1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) + (1 \times 2^{-1}) = 101.1_2$$

or, in the decimal equivalent,

$$4 + 0 + 1 + 0.5 = 5.5_{10}$$

We have used the subscripts 2 and 10 above to clarify the base of the number and avoid any ambiguity.

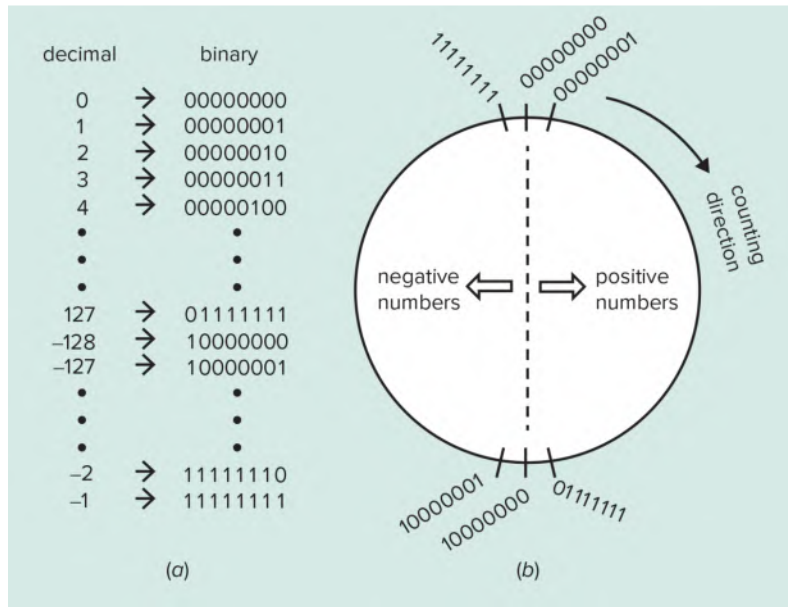
## Integer Representation

Now that we have reviewed how base-2 numbers can be related to base 10 numbers (The reverse is not obvious at this point.), it is simple to conceive how integers are represented on a computer. We do have to deal with a limitation and a dilemma. The limitation is the number of bytes used to represent the integer. If that is one byte, we have a limit of eight binary digits. If it is four bytes, the limit is 32 bits. The dilemma is how to represent negative numbers.

Let's explore that by considering counting binary numbers for a word-length of one byte. This allows us to represent  $2^8$  or 256 numbers. No other numbers can be represented. Starting at zero (all zeros), we count up. Recall that, in the binary system,  $1 + 1 = 0$  with a carry of 1. Consider what happens when we add 1 to the last number, 1111111. The result is 00000000, since the last carry is discarded.<sup>2</sup> Oh then, we added 1 to the last number and the result is the first number. There is some sense of representing this number system in the form of a circle, rather than a column ( Fig. 4.3). This suggests how to represent negative numbers by those on the left side of the circle. In fact, when we add the numbers either side of 00000000, we get 00000001 + 11111111 = 00000000. So, the second number has the sense of -1, when added to +1 yields 0. The distinguishing characteristic of the numbers on the left side of the circle is that the first bit is 1. This is then typically called the *sign bit*—when it is 0, the number is positive, and when it is 1, the number is negative. The only remaining issue is what to do with the number at the bottom of the circle, 10000000. Notice that it is self-negative—add it to itself and you get 0. By convention, since it has the sign bit = 1, we consider this to be a negative number. This common scheme is called *2's complement* representation. Converting our numbers to decimal, on the right side including the top, we have the numbers 0 through 127. And on the left side, including the bottom, we have the numbers -1 to -128. The negative range extends one number beyond the positive range.

---

**FIGURE 4.3** One-byte integer representation.



The typical ranges used by computer programming systems are the last two. The 4-byte or “long integer” type has an extensive range, perhaps far wider than we need for counting numbers. The advantage of the 2-byte representation is that it only requires half the memory, but its range is restricted.

Python has an interesting approach to representing integers. As numbers become large, either positive or negative, Python automatically extends the number of bytes required to represent the number. Consequently, using Python, there is no limit to the range of integers. However, as we now consider floating point numbers, the concepts we have introduced here will be important.

## Floating-Point Representation

Numerical quantities with decimal fractions, digits to the right of the decimal point, are typically represented in computers using a *floating-point format*. In this approach, which is very much like scientific notation, the number is expressed as

$$\pm s \times b^e$$

where  $s$  = the *significand* or *mantissa*,  $b$  = base of the number system being used, and  $e$  = the exponent.

Prior to being expressed in this form, the number is normalized by moving the point, decimal, binary, or otherwise, over so that only one digit is to the left of the point. This is done so that computer memory is not wasted storing useless non-significant zeros. For example, a value like 0.005678 could be represented in a wasteful manner as  $0.005678 \times 10^0$ . However, normalization would yield  $5.678 \times 10^{-3}$ , which means the two leading zeros do not need to be stored. The useless zeros are eliminated. When we consider normalizing a base-2 number, the digit to the left of the binary point will always be a 1. Consequently,

when the computer stores the number, it doesn't need to store that leading 1. It can be programmed to "know" that the 1 exists but isn't stored. This saves one bit.

Before describing the base-2 implementation used on computers in detail, we will first explore the fundamental implications of such floating-point representation. In particular, what are the ramifications of the fact that, in order to be stored in the computer, both the mantissa and exponent are limited to a finite number of bits? As in the next example, a nice way to do this is within the context of our more familiar base-10 decimal world.

## EXAMPLE 4.2

### Implication of Floating-Point Representation

#### Problem Statement

Suppose that we had a hypothetical base-10 computer with a 5-digit word size. Assume that one digit is used for the sign, two for the exponent, and two for the mantissa. For simplicity, assume that one of the exponent digits is used for its sign, leaving a single digit for its magnitude. Characterize this representation scheme.

#### Solution

A general representation of the number following normalization would be

$$s_1 d_1 d_2 \times 10^{s_0 d_0}$$

where  $s_0$  and  $s_1$  are the signs,  $d_0$  is the magnitude of the exponent, and  $d_1 d_2$  are the magnitude of the mantissa digits.

Page 93

Now, let's play with this system. First, what is the largest possible positive quantity that can be represented? Clearly, it would correspond to both signs being positive and all magnitude digits set to the largest possible value in base-10, that is, 9.

$$\text{Largest value} = +9.9 \times 10^{+9}$$

So, the largest possible number would be a little less than 10 billion. Although this might seem like a big number, it really isn't. For example, in this system, it would not be possible to represent a commonly used constant like Avogadro's number,  $6.022 \times 10^{23}$ .

In the same sense, the smallest possible positive number would be

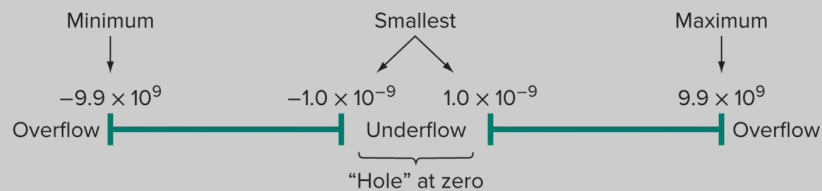
$$\text{Smallest value} = +0.1 \times 10^{-9}$$



Again, although this value might seem small, you could not use it to represent a quantity like Planck's constant,  $6.626 \times 10^{-34}$  J·s.

Similar negative values could be described. The resulting ranges are described in [Fig. 4.4](#). Large positive and negative numbers that fall outside the range would cause an *overflow error*. In a similar fashion, for very small positive and negative quantities, there is a “hole” at zero, and such small numbers would be converted to zero.

**FIGURE 4.4** The number line showing the possible ranges corresponding to the hypothetical base-10 floating-point scheme described in [Example 4.2](#).



Recognize that the exponent overwhelmingly determines these range limitations. For example, if we increase the mantissa field by one digit, the maximum value increases slightly to  $+9.99 \times 10^{+9}$ . In contrast, a one-digit increase in the exponent field raises the maximum by 90 orders of magnitude to  $+9.9 \times 10^{+99}$ !

When it comes to precision, however, the situation is reversed. Whereas the significand plays a minor role in defining the range, it has a profound effect on specifying the precision. This is dramatically illustrated for this example where we have limited the significand to only two digits. As shown in [Fig. 4.5](#), just as there is a “hole” at zero, there are also holes or gaps between values.

**FIGURE 4.5** A small portion of the number line corresponding to the hypothetical base-10 floating-point scheme described in [Example 4.2](#). The numbers indicate values that can be represented exactly. All other quantities falling in the “holes” between these values would exhibit some roundoff error.



For example, a simple rational number with a finite number of digits, like  $2^{-5} = 0.03125$ , would have to be stored as  $3.1 \times 10^{-2}$  or 0.031. Thus, a roundoff error is introduced. For this case, it represents a percent relative error of

$$\frac{0.03125 - 0.031}{0.03125} 100 = 0.8\%$$

While we could store a number like 0.03125 exactly by expanding the digits of the mantissa, quantities with an infinite number of digits must always be approximated. For example, a commonly used constant such as  $\pi$ , 3.14159..., would have to be represented as  $3.1 \times 10^0$ . For this case, the percent relative error is

$$\frac{3.14159 \dots - 3.1}{3.14159 \dots} 100 \cong 1.32\%$$

Although adding digits to the significand can improve the approximation, such quantities will always have some roundoff error when stored in a computer.

Another more subtle effect of floating-point representation is illustrated by  Fig. 4.5. Notice how the gaps between the numbers increase as we move between the orders of magnitude. For numbers with an exponent of  $-1$ , that is, between 0.1 and 1, the spacing is 0.01. Once we cross over the range from 1 to 10, the gap width increases to 0.1. This means that the roundoff error of a number will be proportional to its magnitude. In addition, it means that the relative error will have an upper bound. The maximum relative error for our example would be 0.05. This value is called the *machine epsilon* or machine precision.

As illustrated in  Example 4.2, the fact that both the exponent and mantissa have a finite number of digits means that there are both range and precision limits on floating-point representation. Now, let us examine how floating-point quantities are actually represented in a real computer using base-2 or binary numbers.

First, let's look at normalization. As we described earlier, since binary numbers consist exclusively of 0s and 1s, a bonus occurs when they are normalized. That is, the bit to the left of the binary point will always be one! This means that the leading bit does not have to be stored. Hence, non-zero binary floating-point numbers can be expressed as

$$\pm(1 + f) \times 2^e$$

where  $f$  is the mantissa, that is, the fractional part of the normalized significand. For example, if we normalized the binary number 1101.1, the result would be  $1.1011 \times 2^3$  or  $(1 + 0.1011) \times 2^3$ . Thus, although the original number has five significant bits, we only have to store the four fractional part bits, 1011.

Python stores floating-point numbers according to the *IEEE 754 double-precision standard*. This is the scheme adopted by many software programs. Eight bytes (64 bits) are used to represent a floating-point number. As shown in  Fig. 4.6, the first bit on the left is reserved for the number's sign, 0 for positive and 1 for negative. In a similar spirit to the way in which integers are stored, the exponent and its sign are stored in the next 11 bits. Finally, 52 bits are set aside for the mantissa. However, because of normalization, 53 bits are actually represented with the first bit always a 1.

Diagram illustrating the IEEE 754 Single Precision floating-point format structure:

- Sign (1 bit):** The leftmost bit, used to indicate the sign of the number.
- Signed exponent (11 bits):** The next 11 bits, used to represent the exponent of the number.
- Mantissa (52 bits):** The remaining 52 bits, used to represent the fractional part of the number.

| Binary Value | Decimal Value |
|--------------|---------------|
| 1111111110   | 1023          |
|              | .             |
|              | .             |
| 10000000000  | 1             |
| 0111111111   | 0             |
| 0111111011   | -1            |
|              | .             |
|              | .             |
| 00000000001  | -1022         |

Page 95

As we have shown above, by convention, the 11 bits used for the exponent translates into a numerical range from  $-1,023$  to  $1,023$ . This range represents  $2^{11} - 1 = 2,047$  unique numbers including and symmetric about zero. The largest positive number that can be stored, shown bit-by-bit is

[illegible]
$$\text{Largest number} = +1.111 \dots 111 \times 2^{+1023}$$

The significand above has 52 ones to the right of the binary point. It is just below a decimal value of 2 (or binary 10). It is actually  $2 - 2^{-52} \cong 2$ . We can translate the largest number to base-10 as

$$+2^{+1023} \cong 1.7977 \times 10^{308}$$

In a similar fashion, the smallest positive number can be represented as

$$\textit{Smallest number} = +1.000 \dots 000 \times 2^{-1022}$$

In base-10, this number is  $2^{-1022} \cong 2.2251 \times 10^{-308}$ . As you can see, the range available with the IEEE 64-bit standard is more than ample to handle engineering and scientific computations.

## Precision

The 52 bits used for the mantissa correspond to about 15-to-16 base-10 digits. Since  $2^{52} \cong 4.5036 \times 10^{15}$ , the precision is one part in about  $4.5 \times 10^{15}$ . Python expresses 16 significant figures. For example,  $\pi$  is

```
import math
print(math.pi)
3.141592653589793
```

Note that, for the IEEE 64-bit standard, the machine epsilon is  $2^{-52} \cong 2.2204 \times 10^{-16}$ .

Using the NumPy module, we can determine the largest and smallest positive numbers and the machine epsilon directly.

```
import numpy as np
print(np.finfo(float).max)
1.7976931348623157e+308
print(np.finfo(float).min)
-1.7976931348623157e+308
print(np.finfo(float).eps)
2.220446049250313e-16
```

As required, these values can be assigned to variables.

## 4.2.2 Arithmetic Manipulations of Computer Numbers

Aside from the limitations of a computer's number system, the actual arithmetic manipulations involving these numbers can also result in roundoff error. To understand how this occurs, let's look at how the computer performs simple addition and subtraction.

Because of their familiarity, normalized base-10 numbers will be employed to illustrate the effect of roundoff errors on simple addition and subtraction. Other number bases, such as binary, would behave in

a similar fashion. To simplify the discussion, we will employ a hypothetical decimal computer with a 4-digit mantissa and a 1-digit exponent.

When two floating-point numbers are added, the numbers are first expressed so that they have the same exponents. For example, if we want to add  $1.557$  and  $4.341 \times 10^{-2}$ , the computer would express the numbers as  $1.557 \times 10^0$  and  $0.04341 \times 10^0$ , aligning the exponents. If we then add the mantissas, we get  $1.640041 \times 10^0$ . Now, because this hypothetical computer only carries a 4-digit mantissa, in aligning  $4.341 \times 10^{-2}$ , the computer gets  $0.043 \times 10^0$ , and the sum is  $1.600 \times 10^0$ . The computer lost the latter “41” digits of  $4.341 \times 10^{-2}$ , and there is a roundoff error in the sum.

Subtraction is performed identically to addition except that the sign of the subtrahend is reversed. In other words, the computer adds the negative. For example, suppose that we are subtracting  $26.86$  from  $36.41$ . That is,

$$\begin{array}{r} 3.641 \times 10^1 \\ -2.686 \times 10^1 \\ \hline 0.955 \times 10^1 \end{array}$$

You will note that, if we normalize the result, it is  $9.550 \times 10^0$ , which only has three significant figures. The zero appended after the 5 merely fills in the missing digit for our number system. A more dramatic example is

$$\begin{array}{r} 7.642 \times 10^2 \\ -7.641 \times 10^2 \\ \hline 0.001 \times 10^2 \end{array}$$

This result would normalize to  $1.000 \times 10^{-1}$  with only one significant figure. The subtraction of two nearly equal numbers, and concurrent loss of precision, is called *subtractive cancellation*. It is a classic example of how the way computers handle mathematics can lead to numerical problems. In simplistic terms, we have to be concerned about subtracting a very large number from another large number to obtain a small result. Other calculations that cause problems include the following.

## Large Computations

Certain methods require arithmetic manipulations of extremely large numbers to arrive at their final results. In addition, these computations are often interdependent. That is, the later calculations are dependent on the results of the earlier ones. Consequently, even though an individual roundoff error could be small, the cumulative effect over the course of a lengthy computation can be significant. A very simple case involves summing a base-10 number that is round, but not round in the equivalent base-2. Suppose that the following Python function is constructed:

```
def sumdemo():
    s = 0
    for i in range(10000):
```

```
s = s + 0.0001
return s
```

When this function is executed and invoked, the result is

```
print(sumdemo())
0.9999999999999999062
```

You would expect the sum to be equal to one, since  $10000 \times 0.0001 = 1$ . However, although 0.0001 is a nice round number in base-10, it cannot be expressed exactly in base-2. By including binary 1 digits to sum to an approximation of 0.0001, but not exceed that number, we get

```
0.0000000000000000110100011 ...2
```

and this binary fraction continues without equaling  $0.0001_{10}$  exactly. This last binary fraction equals

```
0.0000998973846435547...10
```

We should note that Python has features that are designed to minimize such errors. For example, suppose that you form an array as in

```
import numpy as np
s = np.arange(0,1.0001,0.0001)
```

Rather than encountering the approximation, as above with the sumdemo function, in this case,

```
print(s[10000])
1.0
```

## Adding a Large and a Small Number

Suppose we add a small number, 0.0010, to a large number, 4000, using a hypothetical computer with the 4-digit mantissa and the 1-digit exponent. After aligning the exponents, we have

$$\begin{array}{rcl} 4.000 & \times & 10^3 \\ 0.000001 & \times & 10^3 \\ \hline 4.000001 & \times & 10^3 \end{array}$$

which, in our limited number system, results in  $4.000 \times 10^3$ . Thus, we might as well have not performed the addition! This kind of error can occur in the computation of an approximation to an infinite series. The initial terms in such series are often relatively large in comparison with the later terms. After a few terms have been added to the sum, we are in the situation of adding a small quantity to a large quantity. One way to mitigate this type of error is to sum the series in reverse order. In this way, each new term will be of comparable magnitude to the accumulated sum. Of course, this raises the question of how many terms to consider, that is, where to start the computation.

## Smearing

Smearing occurs whenever the individual terms in a summation are larger than the summation itself. One case where this occurs is in a series of mixed signs. As mentioned before, it can also occur in a

subtraction of two very large terms to yield a very small, but important, difference.

## Inner Products

As should be clear from the last sections, approximations to some infinite series are particularly prone to roundoff error. Fortunately, the calculation of series is not one of the more common operations in numerical methods. A far more ubiquitous manipulation is the calculation of the inner product of two vectors, as in

$$\mathbf{x}'\mathbf{y} = \sum_{i=1}^n x_i y_i = x_1 y_1 + x_2 y_2 + \cdots x_n y_n$$

This operation is very common, particularly in the solution of simultaneous linear algebraic equations. Such summations are prone to roundoff error, especially when  $n$  is large. Consequently, we find that the IEEE standard used by Python to represent floating-point numbers, with its high precision, is advantageous.

## 4.3 TRUNCATION ERRORS

*Truncation errors* are those that result from using an approximation in place of an exact mathematical procedure. For example, in [Chap. 1](#), we approximated the derivative of velocity of a bungee jumper by a finite-difference equation of the form [from [Eq. \(1.11\)](#)]

$$\frac{dv}{dt} \cong \frac{\Delta v}{\Delta t} = \frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i} \quad (4.8)$$

A truncation error was introduced into the numerical solution because the difference equation only approximates the true value of the derivative (recall [Fig. 1.3](#)). To gain insight into the properties of such errors, we now turn to a mathematical formulation that is used widely in numerical methods to express functions in an approximate fashion—the Taylor series.

Page 98

### 4.3.1 The Taylor Series

*Taylor's theorem*<sup>4</sup> and its associated formula, the *Taylor series*, are of great value in the study of numerical methods. It can be stated concisely for a function of a single independent variable  $x$  as:

**Taylor's theorem:** If a function  $f$  and its first  $n + 1$  derivatives are continuous on an interval containing  $x_i$  and  $x_{i+1} = x_i + h$ , then the value of the function at  $x_{i+1}$  is given by the Taylor series,

$$f(x_{i+1}) = f(x_i) + f'(x_i)h + \frac{f''(x_i)}{2}h^2 + \frac{f^{(3)}(x_i)}{3!}h^3 + \cdots + \frac{f^{(n)}(x_i)}{n!}h^n + R_n \quad (4.9)$$

with the remainder defined as

$$R_n = \frac{f^{(n+1)}(\xi)}{(n+1)!}h^{n+1} \quad (4.10)$$

where the subscript  $n$  connotes that this is the remainder for the  $n^{\text{th}}$ -order approximation and  $\xi$  is a value of  $x$  that lies somewhere between  $x_i$  and  $x_{i+1}$ .

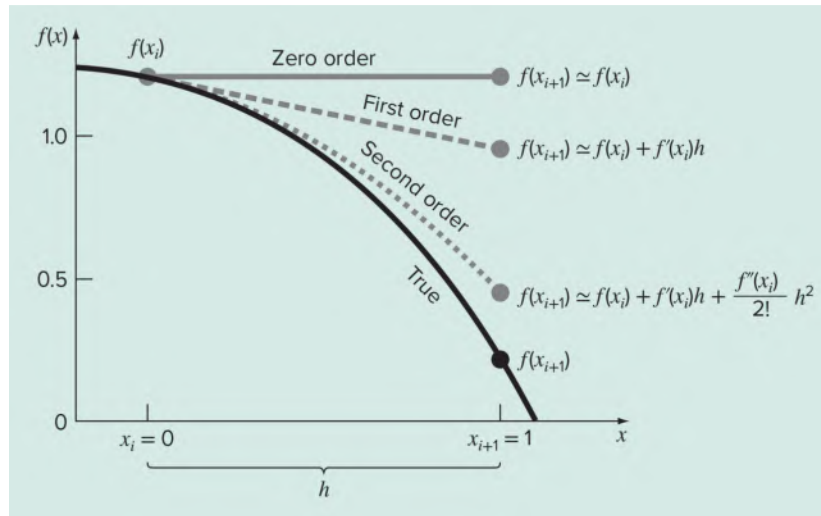
Recognizing that the function and its derivatives evaluated at the base point,  $x_i$ , will be constants, the Taylor theorem states that any smooth function can be approximated by a polynomial. The Taylor series then provides a means to express this idea mathematically in a form that can be used to generate practical results.



A useful way to gain insight into the Taylor series is to build it term by term. A good problem context for this exercise is to predict a function value at one point in terms of the function value and its derivatives at another point.

Suppose that you are blindfolded and taken to a location on the side of a hill facing downslope (  **Fig. 4.7**). We'll call your horizontal location  $x_i$  and your vertical distance with respect to the base of the hill  $f(x_i)$ . You are given the task of predicting the height at a position  $x_{i+1}$ , which is a horizontal distance  $h$  away from you.

**FIGURE 4.7** The approximation of  $f(x) = -0.1x^4 - 0.15x^3 - 0.5x^2 - 0.25x + 1.2$  at  $x = 1$  by zero-order, first-order, and second-order Taylor series expansions.



At first, you are placed on a platform that is completely horizontal so that you have no idea that the hill is sloping down away from you. At this point, what would be your best guess at the height at  $x_{i+1}$ ? If you think about it (remember, you have no idea whatsoever what's in front of you.), the best guess would be the same height as where you're standing now! You could express this prediction mathematically as

$$f(x_{i+1}) \cong f(x_i) \quad (4.11)$$

This relationship, which is called the *zero-order approximation*, indicates that the value of  $f$  at the new point is the same as the value at the old point. This result makes intuitive sense because, if  $x_i$  and  $x_{i+1}$  are close together, it is likely that the new value is probably similar to the old value.

 **Equation (4.11)** provides a perfect estimate if the function being approximated is, in fact, a constant. For our problem, you would be correct only if you happened to be standing on a perfectly flat plateau. However, if the function changes at all over the interval, additional terms of the Taylor series are required to provide a better estimate.

So, now you step off the platform and stand on the hill surface itself with one leg positioned in front of you and the other behind. You immediately sense that the front foot is lower than the back foot. In fact, you're allowed a quantitative estimate of the slope by measuring the difference in elevation,  $\Delta f$ , and divide it by the distance between your feet,  $\Delta x$ . With this additional information, you're clearly in a better position to predict the height at  $x_{i+1}$ . In essence, you use the slope estimate to project a straight line out to  $x_{i+1}$ .

$$f(x_{i+1}) \cong f(x_i) + \frac{\Delta f}{\Delta x} h \quad (4.12)$$

where  $\Delta f / \Delta x$  is an estimate of the first derivative of  $f$  with respect to  $x$  at  $x_i$ . If you have a perfect estimate of the slope,  $\Delta f / \Delta x \rightarrow df/dx$  or  $f'(x_i)$ , [Eq. \(4.12\)](#) can be expressed more concisely as

$$f(x_{i+1}) \cong f(x_i) + f'(x_i)h \quad (4.13)$$

This is called a *first-order approximation* because the additional first-order term consists of a slope,  $f'(x_i)$ , multiplied by  $h$ , the distance between  $x_i$  and  $x_{i+1}$ . Thus, the expression is now in the form of a straight line that can predict an increase or decrease of the function between  $x_i$  and  $x_{i+1}$ .

Although [Eq. \(4.13\)](#) can predict a change, it is only exact for a straight-line function or *linear* trend. To get a better prediction, we need to add more terms to our equation. So, now you stand on the hill and take two measurements. First, you measure the slope behind you by keeping one foot planted at  $x_i$  and moving the other foot back a distance  $\Delta x$  and use that to estimate the slope behind you:

$$f'_b(x - \Delta x) = \frac{f(x_i) - f(x_{i-1})}{\Delta x}$$

Then, you measure the slope in front of you by keeping one foot planted at  $x_i$  and moving the other foot forward  $\Delta x$  and use that to estimate the slope in the forward direction:

$$f'_f(x_i + \Delta x) = \frac{f(x_{i+1}) - f(x_i)}{\Delta x}$$

You immediately recognize that the slope behind you is milder than the one in front. Clearly, the drop in height is “accelerating” downward in front of you and the odds are that  $f(x_{i+1})$  is even lower than your previous linear prediction. To quantify the magnitude of this prediction, we can use our backward and forward slopes to estimate the second derivative as the derivative of derivatives:

$$(4.14)$$

$$f''(x_i) \cong \frac{\frac{f(x_{i+1}) - f(x_i)}{\Delta x} - \frac{f(x_i) - f(x_{i-1}))}{\Delta x}}{\Delta x} = \frac{f(x_{i+1}) - 2f(x_i) + f(x_{i-1}))}{(\Delta x)^2}$$

Following [Eq. \(4.9\)](#), we can now add a second-order term to your predictive equation,

$$f(x_{i+1}) \cong f(x_i) + f'(x_i)h + \frac{f''(x_i)}{2}h^2 \quad (4.15)$$

Thus, we have used the Taylor series to formulate a parabola that yields an improved prediction by capturing some of the hill slope's downward curvature. We can also see why the Taylor theorem states that any smooth function can be approximated as a polynomial and that the Taylor series provides a means to express this idea mathematically.

Clearly, we could keep adding more derivatives to capture more of the function's curvature as in the complete Taylor series expansion

Page 100

$$f(x_{i+1}) = f(x_i) + f'(x_i)h + \frac{f''(x_i)}{2}h^2 + \frac{f^{(3)}(x_i)}{3!}h^3 + \frac{f^{(4)}(x_i)}{4!}h^4 + \dots \quad (4.16)$$

Note that, because this equation represents the entire infinite series, an equal sign (=) has replaced the approximately equals sign ( $\cong$ ) used in the lower-order approximations [[Eqs. \(4.11\)](#), [\(4.13\)](#), and [\(4.15\)](#)].

In general, the  $n^{\text{th}}$ -order Taylor series expansion will be exact for an  $n^{\text{th}}$ -order polynomial. For other, differentiable and continuous functions, such as exponentials and sinusoids, a finite number of series terms will not yield an exact estimate. Each additional term will contribute some improvement, however slight, to the approximation. This behavior will be demonstrated in [Example 4.3](#). Only if an infinite number of terms are added will the series yield an exact result.

Although the foregoing is true, the value of Taylor series expansions is that, in most cases, the inclusion of only a few terms will result in an approximation that is close enough to the true value for practical purposes. The assessment of how many terms are required to get “close enough” is based on the remainder term of the expansion,

$$R_n = \frac{f^{(n+1)}(\xi)}{(n+1)!}h^{n+1} \quad (4.17)$$

This relationship has two major drawbacks. First,  $\xi$  is not known exactly but merely lies between  $x_i$  and  $x_{i+1}$ . Second, to evaluate [Eq. \(4.17\)](#), we need to determine the  $(n+1)^{\text{th}}$  derivative of  $f(x)$ . To do

this, we need to know  $f(x)$ . However, if we knew  $f(x)$ , we would not have to perform the Taylor series expansion in the first place.

Despite these apparent shortcomings, [Eq. \(4.17\)](#) is still useful for gaining insight into truncation errors. This is because we do have control over the term  $h$  in the equation. In other words, we can choose how far away from  $x$  we want to evaluate  $f(x)$ , and we can control the number of terms we include in the expansion. Consequently, [Eq. \(4.17\)](#) is often expressed as

$$R_n = O(h^{n+1})$$

where the term  $O(h^{n+1})$  means that the truncation error is of the order of  $h^{n+1}$ . That is, the error is proportional to the step size  $h$  raised to the  $(n + 1)^{\text{th}}$  power. Although this approximation implies nothing regarding the magnitude of the derivatives that multiply  $h_{n+1}$ , it is extremely useful in judging the comparative error of numerical methods based on Taylor series expansions. For example, if the error is  $O(h)$ , halving the step size will halve the error. On the other hand, if the error is  $O(h^2)$ , halving the step size will quarter the error.

In general, we can usually assume that the truncation error is decreased by the addition of terms to the Taylor series. In many cases, if  $h$  is sufficiently small, the first- and other lower-order terms usually account for a disproportionately high percent of the error. Thus, only a few terms are required to obtain an adequate approximation. This property is illustrated by the following example.

## EXAMPLE 4.3

### Approximation of a Function with a Taylor Series Expansion

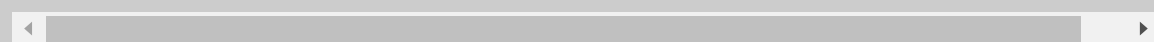
#### Problem Statement

Use Taylor series expansions with  $n = 0$  to 6 to approximate  $f(x) = \cos(x)$  at  $x_{i+1} = \pi/3$  based on the value of  $f(x)$  and its derivatives at  $x_{i+1} = \pi/4$ . Note that this means  $h = \pi/3 - \pi/4 = \pi/12$ .

#### Solution

Our knowledge of the true function allows us to determine the correct value,  $f(\pi/3) = 0.5$ . The zeroth-order approximation, from [Eq. \(4.11\)](#), is

$$f\left(\frac{\pi}{3}\right) \cong \cos\left(\frac{\pi}{4}\right) = \frac{1}{\sqrt{2}} \cong 0.7071067811865476 \quad (\text{full precision from Python})$$



This represents a true percent relative error

$$\varepsilon_t = \left| \frac{0.5 - 0.7071067811865476}{0.5} \right| \times 100\% \cong 41.4\%$$

For the first-order approximation ([Eq. 4.13](#)), we add the first derivative term, where  $f'(x) = -\sin(x)$ :

$$f\left(\frac{\pi}{3}\right) \cong \cos\left(\frac{\pi}{4}\right) - \sin\left(\frac{\pi}{4}\right)\left(\frac{\pi}{12}\right) \cong 0.5219866587632823$$

which has  $\varepsilon_t \cong 4.40\%$ . For the second-order approximation, we add the second derivative term, where  $f''(x) = -\cos(x)$ :

$$f\left(\frac{\pi}{3}\right) \cong \cos\left(\frac{\pi}{4}\right) - \sin\left(\frac{\pi}{4}\right)\left(\frac{\pi}{12}\right) - \frac{\cos(\pi/4)}{2}\left(\frac{\pi}{12}\right)^2 \cong 0.4977544914034251$$

with  $\varepsilon_t \cong 0.449\%$ . Thus, the inclusion of additional terms results in an improved estimate. This process can be continued, and the results listed in the table below.

| Order<br>(n) | $f^{(n)}(x)$ | $f(\pi/3)$<br>Taylor | $ \varepsilon_t $ |
|--------------|--------------|----------------------|-------------------|
| 0            | $\cos(x)$    | 0.707106781          | -41.4%            |
| 1            | $-\sin(x)$   | 0.521986659          | -4.40%            |
| 2            | $-\cos(x)$   | 0.497754491          | 0.449%            |
| 3            | $\sin(x)$    | 0.499869147          | 0.0262%           |
| 4            | $\cos(x)$    | 0.500007551          | -0.00151%         |
| 5            | $-\sin(x)$   | 0.500000304          | -0.0000608%       |
| 6            | $-\cos(x)$   | 0.499999988          | 0.00000244%       |

Notice that the derivatives never approach zero as would be the case for a polynomial. Observe that each additional term results in some improvement in the estimate. However, also notice how most of the improvement comes with the first few terms. For this case, by the time we have added the third-order term, the error is reduced to 0.0262%, which means that we have attained 99.974% of the true value. Consequently, although the addition of more terms reduces the error further, the improvement becomes negligible.

## 4.3.2 The Remainder for the Taylor Series Expansion

Before documenting how the Taylor series is used to estimate numerical errors, we must explain why we included the argument  $\xi$  in [Eq. \(4.17\)](#). To do this, we will use a simple, visually based explanation.

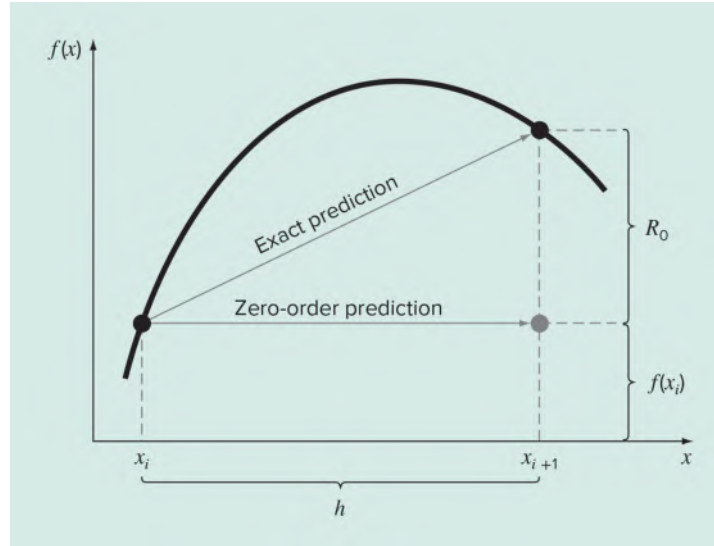
Suppose that we truncated the Taylor series expansion after the zeroth-order term as in [Eq. \(4.11\)](#)

$$f(x_{i+1}) \cong f(x_i)$$

A visual depiction of this zeroth-order prediction is shown in [Fig. 4.8](#). The remainder, or error, of this prediction, which is also shown in the illustration as  $R_0$ , consists of the infinite series of terms that were truncated:

$$R_0 = f'(x_i)h + \frac{f''(x_i)}{2!}h^2 + \frac{f'''(x_i)}{3!}h^3 + \dots$$

**FIGURE 4.8** Graphical depiction of a zero-order Taylor series prediction and remainder.



It is obviously inconvenient to deal with the remainder in this infinite series format. One simplification might be to truncate the remainder estimate itself, as in

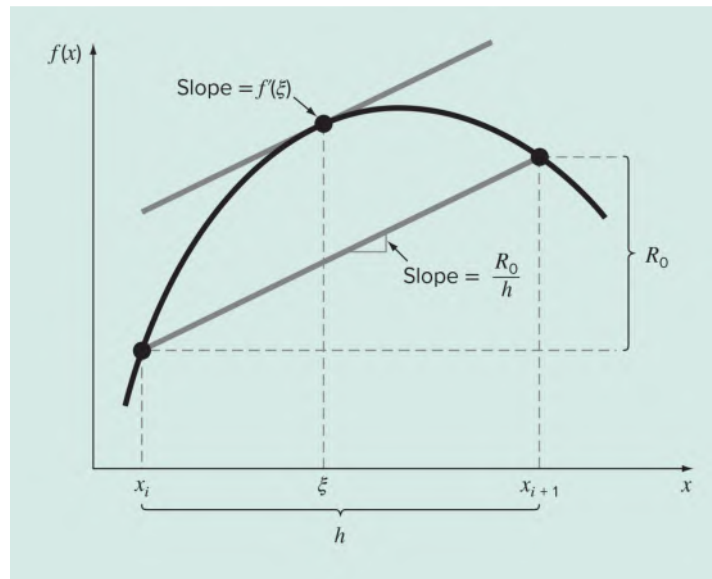
$$R_0 \cong f'(x_i)h \quad (4.18)$$

Although, as stated in the previous section, lower-order derivative terms usually account for a greater share of the remainder than the higher-order terms, this result is still inexact because of the neglected

second- and higher-order terms. This “inexactness” is implied by the approximate equality symbol ( $\cong$ ) employed in [Eq. \(4.18\)](#).

As an alternative simplification, which transforms the approximation into an equivalence, is based on graphical insight. As shown in [Fig. 4.9](#), the *derivative mean-value theorem* states that if a function  $f(x)$  and its first derivative are continuous over an interval from  $x_i$  to  $x_{i+1}$ , then there exists at least one point on the function that has a slope, designated by  $f'(\xi)$ , that is parallel to the line joining  $\{x_i, f(x_i)\}$  and  $\{x_{i+1}, f(x_{i+1})\}$ . The parameter  $\xi$  marks the  $x$  value where this slope occurs ([Fig. 4.9](#)). A physical illustration of this theorem is that, if you travel between two points with an average velocity, there will be at least one moment during the trip when you will be moving at that average velocity. To be clear,  $\xi$  may not be unique—there may be more than one point where this occurs. But the point is that there is at least one  $x$  value within the interval where the error estimate is exact.

**FIGURE 4.9** Graphical depiction of the derivative mean-value theorem.



By invoking this theorem, it is simple to realize that, as illustrated in [Fig. 4.9](#), the slope  $f'(\xi)$  is equal to the rise  $R_0$  divided by the run  $h$ , or

$$f'(\xi) = \frac{R_0}{h}$$

which can be rearranged to give

$$R_0 = f'(\xi)h \quad (4.19)$$

Thus, we have derived the zeroth-order version of [Eq. \(4.17\)](#). The higher-order versions are a logical extension of the reasoning used to derive [Eq. \(4.19\)](#). The first-order version is

$$R_1 = \frac{f''(\xi)}{2!} h^2 \quad (4.20)$$

For this case, the value of  $\xi$  conforms to the  $x$  value corresponding to the second derivative that makes [Eq. \(4.20\)](#) exact. Similar higher-order versions can be developed from [Eq. \(4.17\)](#).

### 4.3.3 Using the Taylor Series to Estimate Truncation Errors

Although the Taylor series will be extremely useful in estimating truncation errors throughout this book, it may not be clear to you how the expansion can be applied to numerical methods. In fact, we have already done so in our example of the bungee jumper from [Chap. 1](#). Recall that the objective of both [Examples 1.1](#) and [1.2](#) was to predict velocity as a function of time. That is, we were interested in determining  $v(t)$ . As specified by [Eq. \(4.9\)](#),  $v(t)$  can be expanded in a Taylor series:

$$v(t_{i+1}) = v(t_i) + v'(t_i)(t_{i+1} - t_i) + \frac{v''(t_i)}{2!}(t_{i+1} - t_i)^2 + \cdots + R_n$$

Now let us truncate the series after the first derivative term:

$$v(t_{i+1}) = v(t_i) + v'(t_i)(t_{i+1} - t_i) + R_1 \quad (4.20)$$

[Equation \(4.20\)](#) can be solved for  $v'(t)$ :

$$v'(t_i) = \frac{\overbrace{v(t_{i+1}) - v(t_i)}^{\text{First-order approximation}}}{\overbrace{t_{i+1} - t_i}^{\text{Truncation error}}} - \frac{R_1}{\overbrace{t_{i+1} - t_i}^{\text{Truncation error}}} \quad (4.21)$$

The first part of [Eq. \(4.21\)](#) is the same relationship that was used to approximate the derivative in [Example 1.2](#) [[Eq. \(1.11\)](#)]. However, because of the Taylor series approach, we have now obtained an estimate of the truncation error associated with this approximation of the derivative. Using [Eqs. \(4.17\)](#) and [\(4.21\)](#) yields



$$\frac{R_1}{t_{i+1} - t_i} = \frac{v''(\xi)}{2!} (t_{i+1} - t_i)$$

or

$$\frac{R_1}{t_{i+1} - t_i} = O(t_{i+1} - t_i)$$

Thus, the estimate of the derivative [[Eq. \(1.11\)](#) or the first part of [Eq. \(4.21\)](#)] has a truncation error of order  $t_{i+1} - t_i$ . In other words, the error of our derivative approximation should be proportional to the step size. Consequently, if we halve the step size, we would expect to halve the error of the derivative.

## 4.3.4 Numerical Differentiation

[Equation \(4.21\)](#) is given a formal label in numerical methods—it is called a *finite difference*. It can be represented generally as

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i} + O(x_{i+1} - x_i) \quad (4.22)$$

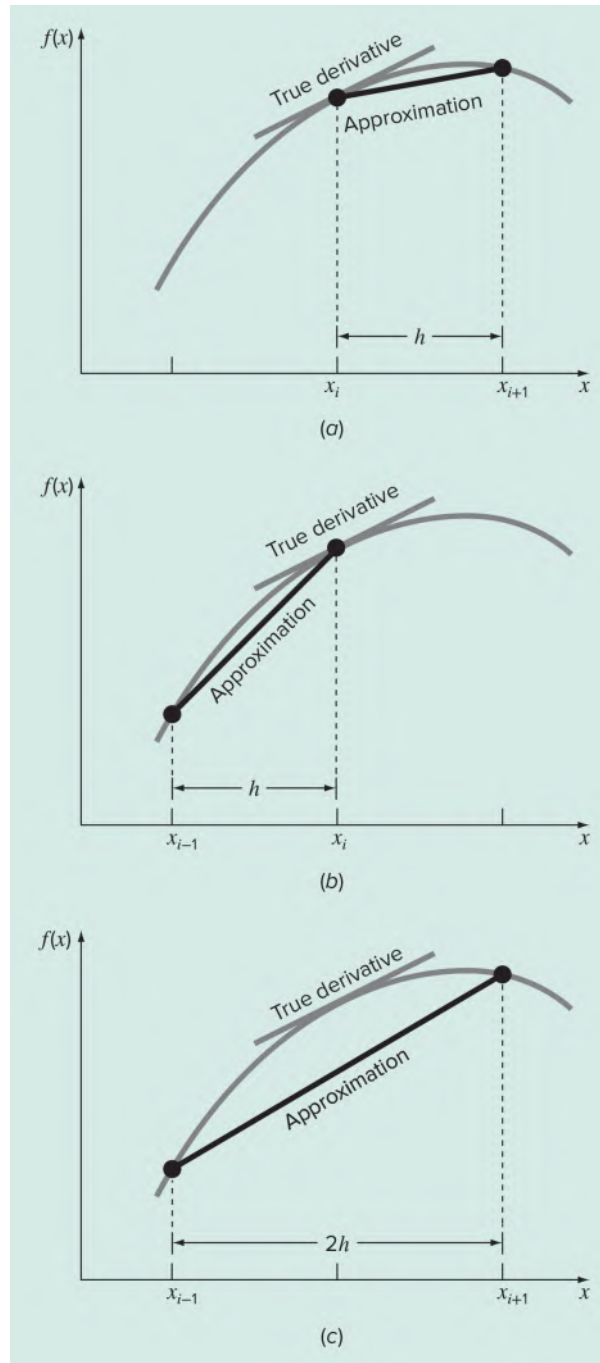
or

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{h} + O(h) \quad (4.23)$$

where  $h$  is called the step size—that is, the length of the interval over which the approximation is made,  $x_{i+1} - x_i$ . It is termed a “forward” difference because it utilizes data at  $i$  and  $i + 1$  to estimate the derivative ([Fig. 4.10a](#)).

---

**FIGURE 4.10** Graphical depiction of (a) forward, (b) backward, and (c) centered finite-difference approximations of the first derivative.



This forward difference is but one of many that can be developed from the Taylor series to approximate derivatives numerically. For example, backward and centered difference approximations of the first derivative can be developed in a fashion similar to the derivation of [Eq. \(4.21\)](#). The former utilizes values at  $x_{i-1}$  and  $x_i$  ([Fig. 4.10b](#)), whereas the latter uses values that are equally spaced about the point at which the derivative is estimated ([Fig. 4.10c](#)). More accurate approximations of the first derivative can be developed by including higher-order terms of the Taylor series. Finally, all the foregoing

versions can also be developed for second, third, and higher derivatives. The following sections provide brief summaries illustrating how some of these cases are derived.

## Backward Difference Approximation of the First Derivative

The Taylor series can be expanded backward to calculate a previous value based on a present value, as in

$$f(x_{i-1}) = f(x_i) - f'(x_i)h + \frac{f''(x_i)}{2!}h^2 - \dots \quad (4.24)$$

Truncating this equation after the first derivative and rearranging yields

$$f'(x_i) \cong \frac{f(x_i) - f(x_{i-1})}{h} \quad (4.25)$$

where the error is  $O(h)$ .

## Centered Difference Approximation of the First Derivative

A third way to approximate the first derivative is to subtract  Eq. (4.24) from the forward Taylor series expansion,

$$f(x_{i+1}) = f(x_i) + f'(x_i)h + \frac{f''(x_i)}{2!}h^2 - \dots \quad (4.26)$$

to yield

$$f(x_{i+1}) = f(x_{i-1}) + 2f'(x_i)h + 2\frac{f''(x_i)}{3!}h^3 + \dots$$

which can be solved for

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_{i-1}))}{2h} - \frac{f''(x_i)}{6}h^2 + \dots$$

or

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_{i-1}))}{2h} - O(h^2) \quad (4.27)$$

 **Equation (4.27)** is a *centered finite difference* representation of the first derivative. Notice that the truncation error is of the order  $h^2$  in contrast to the forward and backward approximations that were of the order  $h$ . Consequently, the Taylor series analysis yields the practical information that the centered difference is a more accurate representation of the derivative ( **Fig. 4.10c**). For example, if we halve the step size using a forward or backward difference, we would approximately halve the truncation error, whereas, for the central difference, the error would be quartered.

## EXAMPLE 4.4

### Finite-Difference Approximations of Derivatives

#### Problem Statement

Use the forward and backward difference approximations of  $O(h)$  and a centered difference approximation of  $O(h^2)$  to estimate the first derivative of

$$f(x) = -0.1x^4 - 0.15x^3 - 0.5x^2 - 0.25x + 1.2$$

at  $x = 0.5$  using a step size  $h = 0.5$ . Repeat the computation using  $h = 0.25$ . Note that the derivative can be expressed exactly as

$$f'(x) = -0.4x^3 - 0.45x^2 - 1.0x - 0.25$$

which can be used to compute the true value,  $f'(0.5) = -0.9125$ .

#### Solution

For  $h = 0.5$ , the function can be employed to determine

|                 |                    |
|-----------------|--------------------|
| $x_{i-1} = 0$   | $f(x_{i-1}) = 1.2$ |
| $x_i = 0.5$     | $f(x_i) = 0.925$   |
| $x_{i+1} = 1.0$ | $f(x_{i+1}) = 0.2$ |

These values can be used to compute the forward difference [ **Eq. (4.23)**],

$$f'(0.5) \cong \frac{0.2 - 0.925}{0.5} = -1.45 \quad \varepsilon_t = 58.9\%$$

the backward difference [[Eq. \(4.25\)](#)],

$$f'(0.5) \cong \frac{0.925 - 1.2}{0.5} = -0.55 \quad \varepsilon_t = 39.7\%$$

and the centered difference [[Eq. \(4.27\)](#)],

$$f'(0.5) \cong \frac{0.2 - 1.2}{0.5} = -1.0 \quad \varepsilon_t = 9.6\%$$

For  $h = 0.25$ ,

$$\begin{array}{ll} x_{i-1} = 0.25 & f(x_{i-1}) = 1.10351563 \\ x_i = 0.5 & f(x_i) = 0.925 \\ x_{i+1} = 0.75 & f(x_{i+1}) = 0.63632813 \end{array}$$

which can be used to compute the forward difference,

$$f'(0.5) \cong \frac{0.63632813 - 0.925}{0.25} = -1.155 \quad \varepsilon_t = 26.5\%$$

the backward difference,

$$f'(0.5) \cong \frac{0.925 - 1.10351563}{0.25} = -0.714 \quad \varepsilon_t = 21.7\%$$

and the centered difference,

$$f'(0.5) \cong \frac{0.63632813 - 1.10351563}{0.25} = -0.934 \quad \varepsilon_t = 2.4\%$$

For both step sizes, the centered difference approximation is more accurate than forward or backward differences. Also, as predicted by the Taylor series analysis, halving the step size approximately halves the error for the forward and backward differences and quarters the error for the centered difference.

## Finite-Difference Approximations of Higher Derivatives

Besides first derivatives, the Taylor series expansion can be used to derive numerical estimates of higher derivatives. For example, to obtain an estimate of the second derivative, we can write a forward Taylor series expansion with a step size of  $2h$ ; that is, to estimate  $f'(x_{i+2})$  in terms of  $f(x_i)$ :

$$f(x_{i+2}) = f(x_i) + f'(x_i)(2h) + \frac{f''(x_i)}{2!}(2h)^2 + \dots \quad (4.28)$$

Equation (4.26) can be multiplied by 2 and subtracted from Eq. (4.28) to give

$$f(x_{i+2}) - 2f(x_{i+1}) = -f(x_i) + f''(x_i)h^2 + \dots$$

which can be solved for

$$f''(x_i) = \frac{f(x_{i+2}) - 2f(x_{i+1}) + f(x_i)}{h^2} + O(h) \quad (4.29)$$

This relationship is called the *second forward finite difference*. Similar manipulations can be employed to derive a backward version,

$$f''(x_i) = \frac{f(x_i) - 2f(x_{i-1}) + f(x_{i-2}))}{h^2} + O(h)$$

A centered difference approximation for the second derivative can be derived by adding Eqs. (4.24) and (4.26) and rearranging the result to give

$$f''(x_i) = \frac{f(x_{i+1}) - 2f(x_i) + f(x_{i-1}))}{h^2} + O(h^2)$$

As was the case with the first-derivative approximations, the centered case is more accurate. Notice also that the centered version can be alternatively expressed as

$$f''(x_i) \cong \frac{\frac{f(x_{i+1}) - f(x_i)}{h} - \frac{f(x_i) - f(x_{i-1}))}{h}}{h}$$

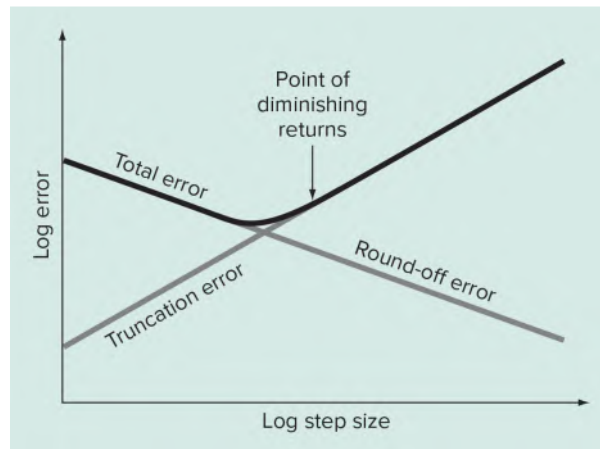
Thus, just as the second derivative is a derivative of a derivative, the second finite difference approximation is a difference of two first finite differences [recall [Eq. \(4.14\)](#)].

## 4.4 TOTAL NUMERICAL ERROR

The *total numerical error* is the summation of the truncation and roundoff errors. In general, the only way to minimize roundoff errors is to increase the number of significant figures of the computer. Further, we have noted that roundoff error may *increase* due to subtractive cancellation or due to an increase in the number of computations in an analysis. In contrast, [Example 4.4](#) demonstrated that the truncation error can be reduced by decreasing the step size. Because a decrease in step size can lead to subtractive cancellation or to an increase in computations, the truncation errors are *decreased* as the roundoff errors are increased.

Therefore, we are faced by the following dilemma: The strategy for decreasing one component of the total error leads to an increase of the other component. In a computation, we could conceivably decrease the step size to minimize truncation errors only to discover in that doing so, the roundoff error begins to dominate the solution and the total error grows! Thus, our remedy becomes our problem ([Fig. 4.11](#)). One challenge that we face is to determine an appropriate step size for a particular computation. We would like to choose a large step size to decrease the amount of calculations and roundoff errors without incurring the penalty of a large truncation error. If the total error is as shown in [Fig. 4.11](#), the challenge is to identify the point of diminishing returns where roundoff error begins to negate the benefits of step-size reduction.

**FIGURE 4.11** A graphical depiction of the trade-off between roundoff and truncation error that sometimes comes into play in the course of a numerical method. The point of diminishing returns is shown, where roundoff error begins to negate the benefits of step-size reduction



When using Python, such situations are relatively uncommon because of its 16-digit precision. Nevertheless, they sometimes do occur and suggest a sort of “numerical uncertainty principle” that places an absolute limit on the accuracy that may be obtained using certain computerized numerical methods. We explore such a case in the following section.



## 4.4.1 Error Analysis of Numerical Differentiation

As described in [Sec. 4.3.4](#), a centered difference approximation of the first derivative can be written as [Eq. \(4.27\)](#)

$$\underbrace{f'(x_i)}_{\text{True/value}} = \underbrace{\frac{f(x_{i+1}) - f(x_{i-1}))}{2h}}_{\text{Finite-difference approximation}} - \underbrace{O(h^2)}_{\text{Truncation error}} \quad (4.30)$$

Thus, if the two function values in the numerator of the finite-difference approximation have no roundoff error, the only error is due to truncation.

However, because we are using digital computers, the function values do include roundoff error as in

$$\begin{aligned} f(x_{i-1}) &= \tilde{f}(x_{i-1}) + e_{i-1} \\ f(x_{i+1}) &= \tilde{f}(x_{i+1}) + e_{i+1} \end{aligned}$$

where the  $\tilde{f}$ 's are the rounded function values and the  $e$ 's are the associated roundoff errors. Substituting these values into [Eq. \(4.30\)](#) gives

$$\underbrace{f'(x_i)}_{\text{True value}} = \underbrace{\frac{\tilde{f}(x_{i+1}) - \tilde{f}(x_{i-1}))}{2h}}_{\text{Finite-difference approximation}} + \underbrace{\frac{e_{i+1} - e_{i-1}}{2h}}_{\text{Roundoff error}} - \underbrace{\frac{f'''(\xi)}{6} h^2}_{\text{Truncation error}}$$

We can see that the total error of the finite-difference approximation consists of a roundoff error that decreases with step size and a truncation error that increases with step size.

Assuming that the absolute value of each component of the roundoff error has an upper bound of  $\varepsilon$ , the maximum possible value of the difference  $e_{i+1} - e_{i-1}$  will be  $2\varepsilon$ . Further, assume that the third derivative has a maximum absolute value of  $M$ . An upper bound on the absolute value of the total error can therefore be represented as

$$\text{Total error} = \left| f'(x_i) - \frac{f(x_{i+1}) - f(x_{i-1}))}{2h} \right| \leq \frac{\varepsilon}{h} + \frac{M}{6} h^2 \quad (4.31)$$

An optimal step size can be determined by differentiating the right side of [Eq. \(4.31\)](#) by  $h$ , setting the result equal to zero and solving for

$$h_{\text{opt}} = \sqrt[3]{\frac{3\varepsilon}{M}} \quad (4.32)$$

## EXAMPLE 4.5

### Roundoff and Truncation errors in Numerical Differentiation

#### Problem Statement

In [Example 4.4](#), we used a centered difference approximation of  $O(h^2)$  to estimate the first derivative of the following function at  $x = 0.5$ ,

$$f(x) = -0.1x^4 - 0.15x^3 - 0.5x^2 - 0.25x + 1.2$$

Perform the same computation starting with  $h = 1$ . Then, progressively divide the step size by a factor of 10 to demonstrate how roundoff becomes dominant as the step size is reduced. Relate your results to [Eq. \(4.32\)](#). Recall that the true value of the derivative is  $-0.9125$ .

#### Solution

We can develop the following Python function and script to perform the computations and plot the results. Notice that we pass both the function name and its analytical derivative function name as arguments to the Python function.

```
import numpy as np
def diffex(func,dfunc,x,n):
    dfttrue = dfunc(x)
    h = 1
    H = np.zeros(n)
    D = np.zeros(n)
    E = np.zeros(n)
    H[0] = h
    D[0] = (func(x+h)-func(x-h))/2/h
    E[0] = abs(dfttrue-D[0])
    for i in range(1,n):
        h = h/10
        H[i] = h
        D[i] = (func(x+h)-func(x-h))/2/h
        E[i] = abs(dfttrue-D[i])
    return H,D,E

ff = lambda x: -0.1*x**4 - 0.15*x**3 - 0.5*x**2 - 0.25*x + 1.2
df = lambda x: -0.4*x**3 - 0.45*x**2 - x - 0.25

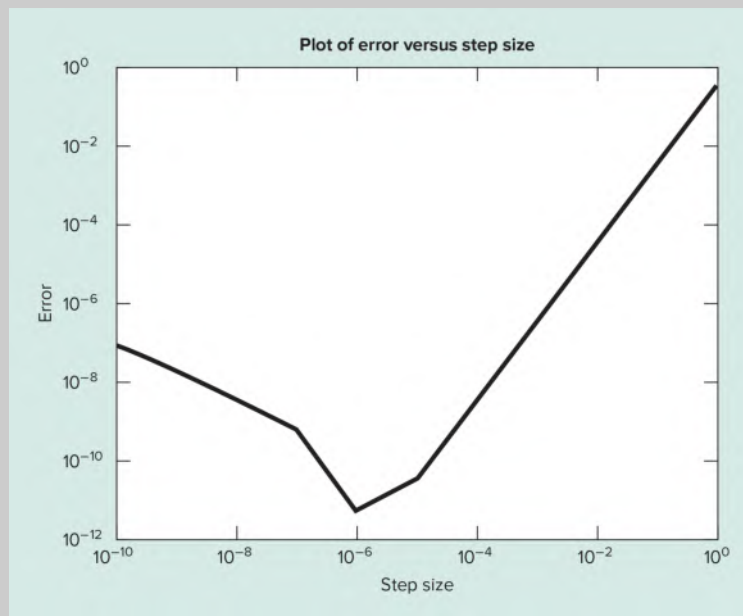
H,D,E = diffex(ff,df,0.5,11)
print('    step size    finite difference    true error')
for i in range(11):
    print(' [0:14.10f] [1:16.14f] [2:16.13f]'.format(H[i],D[i],E[i]))
```

When the script is executed, these are the results:

| step size    | finite difference | true error       |
|--------------|-------------------|------------------|
| 1.0000000000 | -1.26250000000000 | 0.35000000000000 |
| 0.1000000000 | -0.91600000000000 | 0.00350000000000 |
| 0.0100000000 | -0.91253500000000 | 0.00003500000000 |
| 0.0010000000 | -0.91250035000001 | 0.00000035000000 |
| 0.0001000000 | -0.91250000349985 | 0.0000000034998  |
| 0.0000100000 | -0.91250000003318 | 0.0000000000332  |
| 0.0000010000 | -0.91250000000542 | 0.0000000000054  |
| 0.0000001000 | -0.91249999945031 | 0.0000000005497  |
| 0.0000000100 | -0.91250000333609 | 0.0000000033361  |
| 0.0000000010 | -0.91250001998944 | 0.0000000199894  |
| 0.0000000001 | -0.91250007550059 | 0.0000000755006  |

As depicted in [Fig. 4.12](#), the results are as expected. At first, roundoff is minimal and the estimate is dominated by truncation error. Hence, as in [Eq. \(4.31\)](#), the total error drops by a factor of 100 each time we divide the step by 10. However, starting at about  $h = 0.0001$ , we see roundoff error begin to creep in and erode the rate at which the error diminishes. A minimum error is reached at  $h = 10^{-6}$ . Beyond this point, the error increases as roundoff dominates.

**FIGURE 4.12**



Because we are dealing with an easily differentiable function, we can also investigate whether these results are consistent with [Eq. \(4.32\)](#). First, we can estimate  $M$  by evaluating the function's third derivative as

$$M = |f'''(0.5)| = |-2.4 \cdot 0.5 - 0.9| = 2.1$$

Because Python has a floating-point precision of about 16 base-10 digits, a rough estimate of the upper bound on roundoff would be about  $\varepsilon = 1 \times 10^{-16}$ . Substituting these values into  Eq. (4.32) gives

$$h_{\text{opt}} = \sqrt[3]{\frac{3(1 \times 10^{-16})}{2.1}} = 5.2 \times 10^{-6}$$

which is of the same order as the result of  $1 \times 10^{-6}$  obtained with the Python script.

## 4.4.2 Control of Numerical Errors

For most practical cases, we do not know the exact error associated with numerical methods. The exception, of course, is when we know the exact solution, when would then make our numerical approximations unnecessary. Therefore, for most engineering and scientific applications, we must settle for some estimate of the error in our calculations.

There are no systematic and general approaches to evaluating numerical errors for all problems. In many cases, error estimates are based on the experience and judgment of the engineer or scientist.

Although error analysis is an art to a certain extent, there are several practical programming guidelines we can suggest. First and foremost, avoid subtracting two nearly equal numbers. Loss of significance almost always occurs when this is done. Sometimes you can arrange or reformulate a problem to avoid subtractive cancellation. Furthermore, when adding and subtracting numbers, it is best to sort the numbers and work with the smallest numbers first. This avoids loss of significance. At times, computations appear to involve a combination of very large and very small numbers. It may be possible to rescale the numbers so they are of similar magnitude. This will likely avoid loss of significance and may provide more reliable results.

Beyond these computational hints, one can attempt to predict total numerical errors using theoretical formulations. The Taylor series is our primary tool for analysis of such errors. Prediction of total numerical error is very complicated for even moderately sized problems and tends to be pessimistic. Therefore, it is usually attempted for only small-scale tasks.

The tendency is to push forward with the numerical computations and try to estimate the accuracy of your results. This can sometimes be done by seeing if the results satisfy some condition or equation as a check. Or it may be possible to substitute the results back into the original equation to check that it is actually satisfied.

Finally, you should be prepared to perform numerical experiments to increase your awareness of computational errors and possible ill-conditioned problems. Such experiments may involve repeating the computations with a different step size or method and comparing the results. We may employ sensitivity analysis to see how our solution changes when we change model parameters or input values. We may want to try different numerical algorithms that have different theoretical foundations, are based on different computational strategies, or have different convergence properties and stability characteristics.

When the results of numerical computations are extremely critical and involve safety and potential loss of human life or have severe economic ramifications, it is appropriate to take special precautions. This may involve the use of two or more independent groups to solve the same problem so that their results can be compared for validation.

The roles of errors will be a topic of concern and analysis in all sections of this book. We will leave these investigations to specific sections.

## 4.5 BLUNDERS, MODEL ERRORS, AND DATA UNCERTAINTY

Although the following sources of error are not directly connected with most of the numerical methods in this book, they can sometimes have a great impact on the success of a modeling effort. Thus, they must always be kept in mind when applying numerical techniques in the context of real-world problems.

### 4.5.1 Blunders

We are all familiar with gross errors, or blunders. In the early years of computers, erroneous numerical results could sometime be attributed to malfunctions of discrete components of the computer itself. With the advent of highly integrated circuits, that is, the central processor on a chip, this source of error has become unlikely, and most blunders must be attributed to human imperfection.

Blunders are usually disregarded in discussions of numerical methods. This is no doubt because, try as we may, mistakes are to a certain extent unavoidable. However, we believe that there are several ways in which their occurrence can be minimized. In particular, the good programming habits that were outlined in [Chap. 3](#) are extremely useful in mitigating programming blunders. In addition, there are usually simple ways to check whether a particular method is working properly. Throughout this book, we discuss ways to check the results of numerical calculations.

### 4.5.2 Model Errors

*Model errors* relate to bias that can be ascribed to incomplete or erroneous mathematical models. An example of a model error that can be neglected under most circumstances is the application of Newton's laws of motion without accounting for relativistic effects. This does not detract from the adequacy of the solution in [Example 1.1](#) because these errors are minimal on the time and space scales associated with the bungee jumper problem.

However, suppose that resistance is not proportional to the square of the fall velocity, as in [Eq. \(1.7\)](#), but is related to velocity and other factors in a different way. Alternately, imagine that a wind field were important to the bungee jumper scenario, and this was not considered in formulating the mathematical model. In such cases, both the analytical and numerical solutions obtained in [Chap. 1](#) would be erroneous because of model error. The availability of test data would likely reveal this error. You should be cognizant of this type of error and realize that, if you are working with a poorly conceived model, no numerical method will provide adequate results. Mathematical models must always be scrutinized for robustness and possible sources of model error.

### 4.5.3 Data Uncertainty

Errors sometime enter into an analysis because of uncertainty in the physical data upon which a model is based. For instance, suppose that we wanted to test the bungee jumper model by having an individual make repeated jumps and then measuring his or her velocity after a specified time interval. Uncertainty would undoubtedly be associated with these measurements, as the jumper would fall faster during some jumps than during others. Also, there might be errors associated with the instrumentation and human operations used to make the velocity measurements. These errors can exhibit both inaccuracy (bias error) and imprecision (random error). If our instruments consistently under- or overestimate the velocity, we are dealing with an inaccurate, or biased, device. On the other hand, if the measurements vary randomly, we are dealing with noise and a question of precision. Bias can often be countered via calibration of instruments by comparison with accepted standards.

Measurement errors can be quantified by summarizing data with one or more well-chosen statistics that convey as much information as possible regarding specific characteristics of the data. These descriptive statistics are most often selected to represent (1) the central tendency of the data, or location of the center of the data's distribution, and (2) the dispersion or spread of the data corresponding to the width that distribution. There are additional measures associated with temporal changes in the data, such as those caused by instrument drift and environmental factors. We will return to the topic of characterizing data uncertainty when we discuss regression in  **Part Four** of the book.

Although you must be cognizant of blunders, model errors, and uncertain data, the numerical methods used for building models can be studied, for the most part, independently of those errors. Therefore, for most of this book, we will assume that we have not made gross errors, we have a sound model, and we are dealing with error-free measurements. Under these conditions, we can study numerical errors without complicating factors.

# PROBLEMS

---

**4.1** The “divide and average” method, an old-time method for approximating the square root of any positive number  $a$  can be formulated as

$$x_{i+1} = \frac{x_i + a/x_i}{2}$$

Write and test a well-structured Python function to implement this iterative method based on the algorithm outlined in  **Fig. 4.2**.

**4.2** Convert the following base-2 numbers to base 10:

- (a) 1011001
- (b) 0.01011
- (c) 110.01001

**4.3** Convert the following base-8 numbers to base 10:

- (a) 61565<sub>8</sub>
- (b) 2.71<sub>8</sub>

**4.4** For computers, the machine epsilon,  $\varepsilon$ , can also be thought of as the smallest number that when added to one gives a number greater than one. An algorithm based on this idea can be developed as

Step 1. Set  $\varepsilon = 1$ .

Step 2. If  $1 + \varepsilon$  is less than or equal to 1, go to Step 5; otherwise continue to Step 3.

Step 3.  $\varepsilon = \varepsilon/2$ .

Step 4. Return to Step 2.

Step 5.  $\varepsilon = 2\varepsilon$

Write your own Python script based on this algorithm to determine the machine epsilon. Validate the result by comparing it with the value returned from the Python NumPy command, `np.finfo(float).eps`. If the values are different, consider why that might be.

**4.5** In a fashion similar to  **Prob. 4.4**, develop your own script to determine the smallest positive real number used in Python. Base your algorithm on the notion that your computer will be unable to distinguish reliably between zero and a quantity that is smaller than this number. Note that the result you obtain will differ from the value returned by `np.finfo(float).tiny`. Challenge question: Investigate the results by taking the base-2 logarithm of the number generated by your code and those obtained by the NumPy module.

**4.6** For the hypothetical base-10 computer in  **Example 4.2**, prove that the machine epsilon is 0.05.

**4.7** The derivative of  $f(x) = 1/(1 - 3x^2)$  is given by



$$\frac{6x}{(1 - 3x^2)^2}$$

Do you expect to have difficulties evaluating this function at  $x = 0.577$ ? Try it using 3- and 4-digit arithmetic with chopping, not rounding.

**4.8 (a)** Evaluate the polynomial

$$y = x^3 - 7x^2 + 8x - 0.35$$

at  $x = 1.37$ . Use 3-digit arithmetic with chopping, not rounding. Evaluate the percent relative error.

**(b)** Repeat **(a)**, but express  $y$  as

$$y = ((x - 7)x + 8)x - 0.35$$

Evaluate the error and compare to part **(a)**. If there is a significant difference, why do you think that is the case?

**4.9** The following infinite series can be used to approximate  $e^x$ :

$$e^x \cong 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!}$$

- (a)** Prove that this Maclaurin series expansion is a special case of the Taylor series expansion [  **Eq. (4.13)** ] with  $x_i = 0$  and  $h = x$ .
- (b)** Use the Taylor series to estimate  $f(x) = e^{-x}$  at  $x_{i+1}$  for  $x_{i+1} = 0.25$ . Employ zero-, first-, second-, and third-order versions and compute  $|\varepsilon_t|$  for each case.

**4.10** The Maclaurin series expansion for  $\cos(x)$  is

$$\cos(x) = 1 - \frac{x^2}{2} + \frac{x^4}{4!} - \frac{x^6}{6!} + \frac{x^8}{8!} - \cdots$$

Starting with the simplest version,  $\cos(x) \cong 1$ , add terms one at a time to estimate  $\cos(\pi/3)$ . After each new term is added, compute the true and approximate percent relative errors. Use your calculator or Python to determine the true value. Add terms until the absolute value of the approximate error estimate falls below an error criterion corresponding to three significant figures.

**4.11** Perform the same computations as in  **Prob. 4.10**, but use the Maclaurin series expansion  $\sin(x)$  to estimate  $\sin(\pi/3)$  where

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots$$

**4.12** Use zeroth- through third-order Taylor series expansion to predict  $f(3)$  for

$$f(x) = 25x^3 - 6x^2 + 7x - 88$$

using a base point of  $x = 1$ . Compare the true percent relative error for each approximation.

**4.13** Prove that  **Eq. (4.12)** is exact for all values of  $x$  if  $f(x) = ax^2 + bx + c$ .

**4.14** Use zeroth- through fourth-order Taylor series expansions to predict  $f(2)$  for  $f(x) = \ln(x)$  using a base point of  $x = 1$ . Compute the true percent relative error,  $\varepsilon_t$ , for each approximation. Discuss the meaning of the results.

**4.15** Use forward and backward difference approximations of  $O(h)$  and a centered difference approximation of  $O(h^2)$  to estimate the first derivative of the function examined in  **Prob. 4.12**. Evaluate the derivative at  $x = 2$  using a step size of  $h = 0.25$ . Compare your results with the true value of the derivative. Interpret your results in terms of the remainder term of the Taylor series expansion.

**4.16** Use a centered difference approximation of  $O(h^2)$  to estimate the second derivative of the function examined in  **Prob. 4.12**. Perform the evaluation at  $x = 2$  using step sizes of  $h = 0.2$  and  $0.1$ . Compare your estimates with the true value of the second derivative. Interpret your results in terms of the remainder term of the Taylor series expansion.

Page 113

**4.17** If  $|x| < 1$ , it is known that

$$\frac{1}{1-x} = 1 + x + x^2 + x^3 + \cdots$$

Repeat  **Prob. 4.10** for this series for  $x = 0.1$ .

**4.18** To calculate a planet's space coordinates, we have to solve the function

$$f(x) = x - 1 - 0.5 \sin(x)$$

Let the base point be  $a = x_i = \pi/2$  on the interval  $[0, \pi]$ . Determine the highest-order Taylor series expansion resulting in a maximum error of 0.015 on the specified interval. The error is equal to the absolute value of the difference between the given function and the specific Taylor series expansion. Hint: Solve graphically.

**4.19** Consider the function  $f(x) = x^3 - 2x + 4$  on the interval  $[-2, 2]$  with  $h = 0.25$ . Use the forward, backward, and centered difference approximations for the first and second derivatives so as to illustrate graphically which approximation is most accurate. Graph all three first-derivative finite difference approximations along with the theoretical, and do the same for the second derivative as well.

**4.20** Derive  Eq. (4.32).

**4.21** Repeat  Example 4.5, but for  $f(x) = \cos(x)$  at  $x = \pi/6$ .

**4.22** Repeat  Example 4.5, but for the forward divided difference [ Eq. (4.23)].

**4.23** One common instance where subtractive cancellation occurs involves finding the roots of a parabola,  $ax^2 + bx + c$ , with the quadratic formula

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

For cases where  $b^2 \gg 4ac$ , the difference in the numerator can be very small and roundoff errors can occur. In such cases, an alternative formulation can be used to minimize subtractive cancellation:

$$x = \frac{-2c}{b \pm \sqrt{b^2 - 4ac}}$$

Use 5-digit arithmetic with chopping to determine the roots of the following equation with both versions of the quadratic formula.

$$x^2 - 5000.002x + 10$$

**4.24** Develop a well-structured Python function to compute the Maclaurin series expansion for the cosine function as described in  Prob. 4.10. Pattern your function after the one for the exponential function in  Fig. 4.2. Test your program for  $\theta = \pi/3(60^\circ)$  and  $\theta = 2\pi + \pi/3 = 7\pi/3(420^\circ)$ . Explain the difference in the number of iterations required to obtain the correct result with the desired approximate absolute error ( $\varepsilon_a$ ).

**4.25** Develop a well-structured Python function to compute the Maclaurin series expansion for the sine function as described in  Prob. 4.11. Pattern your function after the one for the exponential function in  Fig. 4.2. Test your program for  $\theta = \pi/3(60^\circ)$  and  $\theta = 2\pi + \pi/3 = 7\pi/3(420^\circ)$ . Explain the difference in the number of iterations required to obtain the correct result with the desired approximate absolute error ( $\varepsilon_a$ ).

**4.26** Recall from your calculus class that the Maclaurin series, named after the Scottish mathematician Colin Maclaurin (1698–1746), is a Taylor series expansion of a function about zero. Use the Taylor

series to derive the first four terms of the Maclaurin series expansion for the cosine employed in [Probs. 4.10](#) and [4.24](#).

**4.27** The Maclaurin series expansion for the arctangent of  $x$  is defined for  $|x| \leq 1$  as

$$\tan^{-1}(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1} x^{2n+1}$$

- (a) Write out the first four terms ( $n = 0, \dots, 3$ ).
- (b) Starting with the simplest version,  $\tan^{-1}(x) \cong x$ , add terms one at a time to estimate  $\tan^{-1}(\pi/6)$ . After each new term is added, compute the true and approximate relative errors. Use your calculator or Python to determine the true value. Add terms until the absolute value of the approximate error estimate falls below an error criterion conforming to three significant figures.

## Roots and Optimization

### 2.1 OVERVIEW

Years ago, you learned to use the quadratic formula

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad (\text{PT2.1})$$

to solve

$$f(x) = ax^2 + bx + c = 0 \quad (\text{PT2.2})$$

The values calculated with [Eq. \(PT2.1\)](#) are called the “roots” of [Eq. \(PT2.2\)](#). They represent the values of  $x$  that make [Eq. \(PT2.2\)](#) equal to zero. For this reason, roots are sometimes called the *zeros* of the equation.

Although the quadratic formula is handy for solving [Eq. \(PT2.2\)](#), there are many other functions for which the root cannot be determined so easily. Before the advent of digital computers, there were a number of ways to solve for the roots of such equations. For some cases, the roots could be obtained by direct methods, as with [Eq. \(PT2.1\)](#). Although there were equations like this that could be solved directly, there were many more that could not. In such instances, the only alternative is an approximate solution technique.

One method to obtain an approximate solution is to plot the function and determine where it crosses the  $x$  axis. This point, which represents the  $x$  value for which  $f(x) = 0$ , is the root. Although graphical methods are useful for obtaining rough estimates of roots, they are limited because of their lack of precision. An alternative approach is to use *trial and error*. This “technique” consists of guessing a value of  $x$  and evaluating whether  $f(x)$  is zero. If not (as is almost always the case), another guess is made, and  $f(x)$  is again evaluated to determine whether the new value provides a better estimate of the root. The process is repeated until a guess results in an  $f(x)$  that is close to zero.

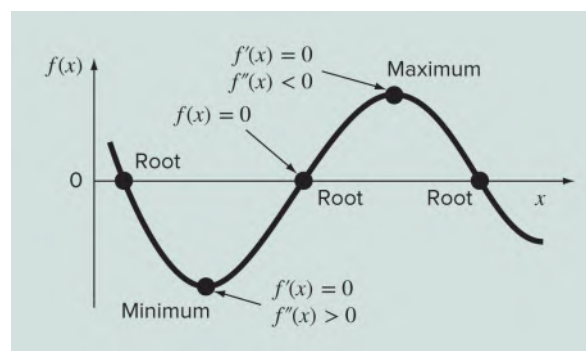
Such haphazard methods are obviously inefficient and inadequate for the requirements of engineering and science practice. Numerical methods represent alternatives that are also approximate but employ systematic strategies to home in on the true root. As elaborated in the following pages, the combination

of these systematic methods and computers makes the solution of most applied roots-of-equations problems a simple and efficient task.



Besides roots, another feature of interest to engineers and scientists are a function's minimum and maximum values. The determination of such optimal values is referred to as *optimization*. As you learned in calculus, such solutions can be obtained analytically by determining the value at which the function is flat; that is, where its derivative is zero. Although such analytical solutions are sometimes feasible, most practical optimization problems require numerical, computer solutions. From a numerical standpoint, such optimization methods are similar in spirit to the root-location methods we just discussed. That is, both involve guessing and searching for a location on a function. The fundamental difference between the two types of problems is illustrated in [Fig. PT2.1](#). Root location involves searching for the location where the function equals zero. In contrast, optimization involves searching for the function's extreme points.

**FIGURE PT2.1** A function of a single variable illustrating the difference between roots and optima.



## 2.2 PART ORGANIZATION

---

The first two chapters in this part are devoted to root location. [Chapter 5](#) focuses on *bracketing methods* for finding roots. These methods start with guesses that bracket, or contain, the root and then systematically reduce the width of the bracket. Two specific methods are covered: *bisection* and *false position*. Graphical methods are used to provide visual insight into the techniques. Error formulations are developed to help you determine how much computational effort is required to estimate the root to a prespecified level of precision.

[Chapter 6](#) covers *open methods*. These methods also involve systematic trial-and-error iterations but do not require that the initial guesses bracket the root. We will discover that these methods are usually more computationally efficient than bracketing methods but that they do not always work. We illustrate several open methods including the *fixed-point iteration*, *Wegstein*, *Newton-Raphson*, and *secant* methods.

Following the description of these individual open methods, we then discuss a hybrid approach called *Brent's root-finding method* that exhibits the reliability of the bracketing methods while exploiting the speed of the open methods. As such, it forms the basis for Python's root-finding function, `brentq`, found in the `optimize` submodule of SciPy. After illustrating how `brentq` can be used for engineering and scientific problems solving, [Chap. 6](#) ends with a brief discussion of special methods devoted to finding the roots of *polynomials*. In particular, we describe Python's convenient built-in capabilities for this task, located in the NumPy module.

[Chapter 7](#) deals with *optimization*. First, we describe two bracketing methods, *golden-section search* and *parabolic interpolation*, for finding the optima of a function of a single variable. Then, we discuss a robust, hybrid approach that combines golden-section search and quadratic interpolation. This approach, which again is attributed to Brent, forms the basis for Python's one-dimensional root-finding function, `minimize_scalar`, found in the `optimize` submodule of SciPy. After describing and illustrating `minimize_scalar`, the last part of the chapter provides a brief description of optimization of multidimensional functions. The emphasis is on describing and illustrating the use of the SciPy `optimize` submodule's capability in this area: the `minimize` function. Finally, the chapter ends with an example of how Python can be employed to solve optimization problems in engineering and science.

# Roots: Bracketing Methods

## CHAPTER OBJECTIVES

The primary objective of this chapter is to acquaint you with bracketing methods for finding a root of a single nonlinear algebraic equation. Specific objectives and topics covered are

- Understanding what roots problems are and where they occur in engineering and science.
- Knowing how to determine a root graphically.
- Understanding the incremental search method and its shortcomings.
- Knowing how to solve a roots problem with the bisection method.
- Knowing how to estimate the error of bisection and why it differs from error estimates for other types of root-location algorithms.
- Understanding false position and how it differs from bisection.

## YOU'VE GOT A PROBLEM

Medical studies have established that a bungee jumper's chance of sustaining a significant vertebrae injury increases significantly if the free-fall velocity exceeds 36 m/s after 4 seconds of free fall. Your boss at the bungee-jumping company wants you to determine the mass at which this criterion is exceeded given a drag coefficient of 0.25 kg/m.

You know from your previous studies that the following analytical solution can be used to predict fall velocity as a function of time:

$$v(t) = \sqrt{\frac{mg}{c_d}} \tanh\left(\sqrt{\frac{c_d g}{m}} t\right) \quad (5.1)$$

Try as you might, you cannot manipulate this equation to solve for  $m$  explicitly—that is, you cannot isolate the mass on the left side of the equation.

An alternative way of looking at the problem involves subtracting  $v(t)$  from both sides of [Eq.\(5.1\)](#) to give a new function:

$$(5.2)$$



$$f(m) = \sqrt{\frac{mg}{c_d}} \tanh\left(\sqrt{\frac{c_d g}{m}} t\right) - v(t)$$

Now, we can see that the answer to the problem is the value of  $m$  that makes the function equal to zero. Hence, we call this a “roots” problem. This chapter will introduce you to how the computer is used as a tool to obtain such solutions.

## 5.1 ROOTS IN ENGINEERING AND SCIENCE

Although they arise in other problem contexts, roots of equations frequently occur in the area of design.

 **Table 5.1** lists several fundamental principles that are routinely used in design work. As introduced in  **Chap. 1**, mathematical equations or models are derived from these principles and employed to predict dependent variables as a function of independent variables, forcing functions, and parameters. Note that, in each case, the dependent variables reflect the state or performance of the system whereas the parameters represent its properties or composition.

**TABLE 5.1** Fundamental principles used in design problems

| <b>Fundamental Principle</b> | <b>Dependent Variable</b>               | <b>Independent Variable</b> | <b>Parameters</b>                                             |
|------------------------------|-----------------------------------------|-----------------------------|---------------------------------------------------------------|
| Thermal energy balance       | Temperature                             |                             | Thermal properties of material, system geometry               |
| Mass balance                 | Concentration or quantity of mass       |                             | Chemical behavior of material, mass transfer, system geometry |
| Force balance                | Magnitude and direction of forces       | Time and position           | Strength of material, structural properties, system geometry  |
| Mechanical energy balance    | Changes in kinetic and potential energy |                             | Mass and properties of material, system geometry              |
| Newton's laws of motion      | Acceleration, velocity and location     |                             | Mass of material, system geometry, dissipative parameters     |
| Kirchoff's laws              | Electrical currents and voltages        | Time                        | Electrical properties (resistance, capacitance, inductance).  |

An example of such a model is the equation for the bungee jumper's velocity. If the parameters are known,  **Eq. (5.1)** can be used to predict the jumper's velocity. Such computations can be performed directly because  $v$  is expressed *explicitly* as a function of the model parameters. That is, it is isolated on one side of the equal sign.

However, as posed at the start of the chapter, suppose that we had to determine the mass for a jumper with a given drag coefficient to attain a prescribed velocity in a set time period. Although  **Eq. (5.1)**

provides a mathematical representation of the interrelationship among the model variables and parameters, it cannot be solved explicitly for mass. In such cases,  $m$  is said to be *implicit*.

This represents a real dilemma because many design problems involve specifying the properties or composition of a system, as represented by its parameters, to ensure that it performs in a desired manner (as represented by its variables). Thus, these problems often require the determination of implicit parameters.

The solution to the dilemma is provided by numerical methods for roots of equations. To solve the problem using numerical methods, it is conventional to re-express [Eq. \(5.1\)](#) by subtracting the dependent variable  $v$  from both sides of the equation to yield [Eq. \(5.2\)](#). The value of  $m$  that makes  $f(m) = 0$  is, therefore, a root of the equation. This value also represents the mass that solves the design problem.

The following pages deal with a variety of numerical and graphical methods for determining roots of relationships such as [Eq. \(5.2\)](#). These techniques can be applied to many other problems confronted routinely in engineering and science.

## 5.2 GRAPHICAL AND TRIAL-AND-ERROR METHODS

A simple method for obtaining an estimate of the root of the equation  $f(x) = 0$  is to make a plot of the function and observe where it crosses the  $x$  axis. This point, which represents the  $x$  value for which  $f(x) = 0$ , provides a rough approximation of the root.

### EXAMPLE 5.1

#### The Graphical Approach

##### Problem Statement

Use the graphical approach to estimate the mass of the bungee jumper with a drag coefficient of  $0.25 \text{ kg/m}$  to have a velocity of  $36 \text{ m/s}$  after  $4 \text{ s}$  of free fall. Note: The acceleration due to gravity is  $9.81 \text{ m/s}^2$ .

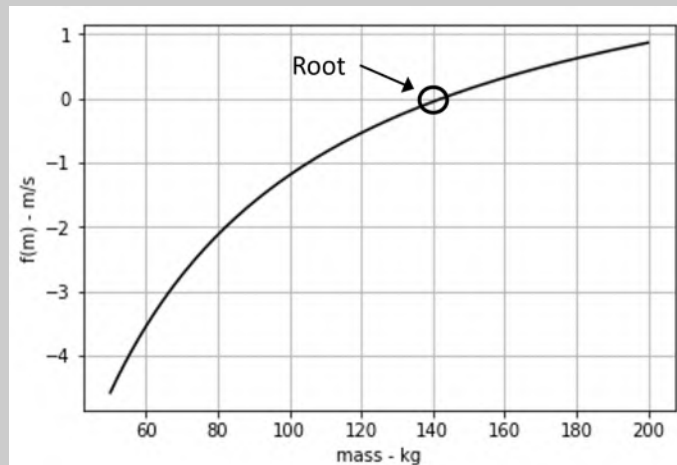
Page 119

##### Solution

The following Python script produces a plot of  Eq. (5.2) versus mass for the range of 50 to 200 kg.

```
"""
Example 5.1
Graphical Approach
"""

import numpy as np
import pylab
cd = 0.25 ; g = 9.81 ; v = 36. ; t = 4.
mp = np.linspace(50.,200.)
fp = np.sqrt(mp*g/cd)*np.tanh(np.sqrt(cd*g/mp)*t)-v
pylab.plot(mp,fp,c='k',lw=0.5)
pylab.grid()
pylab.xlabel(' mass - kg ')
pylab.ylabel(' f(m) - m/s ')
```



The function crosses the  $m$  axis at about 142 kg. This is a reasonable estimate for the root of [Eq. \(5.2\)](#). The validity of the estimate can be checked by substituting it into [Eq. \(5.2\)](#) to yield

```
m = 142
np.sqrt(m*g/cd)*np.tanh(np.sqrt(cd*g/m)*t)-v
-0.0151594086184517
```

which is close to zero. Since the terms in [Eq. \(5.2\)](#) are velocities, we recognize that the graphical solution has a relative error of

```
abs(np.sqrt(m*g/cd)*np.tanh(np.sqrt(cd*g/m)*t)-v)/v*100
0.042109468384588
```

or about 0.04 %.

Graphical techniques are of limited practical value because they are inefficient, and they often do not provide a sufficiently accurate estimate of the root. However, graphical methods can be utilized to provide preliminary estimates of roots. These estimates can then be employed as starting guesses for numerical methods discussed in this chapter.

Another simple method involves trial and error, which consists of making repeated guesses for root until you obtain the desired result. As illustrated in the following example, having access to a computer or calculator makes this approach practically feasible for simple root location.

## EXAMPLE 5.2

### The Trial-and-Error Approach

#### Problem Statement

Use the trial-and-error method to solve the same problem as in [Example 5.1](#).

## Solution

The trial-and-error method can be used with either [Eq. \(5.1\)](#) (guessing the mass that yields the desired velocity) or [Eq. \(5.2\)](#) (guessing the mass that drives the function to zero). Here is an example involving [Eq. \(5.1\)](#), where we guess the mass until we obtain the desired velocity of 36 m/s.

```
"""
Example 5.2
Function for trial-and-error method
"""

import numpy as np
def vtest(mtest):
    cd = 0.25 ; g = 9.81 ; t = 4
    return np.sqrt(mtest*g/cd)*np.tanh(np.sqrt(cd*g/mtest)*t)

vtest(75)
33.57849812941234

vtest(150)
36.14204303065125

vtest(130)
35.71665249183243

vtest(140)
35.943014716272536

vtest(145)
36.045626491526384

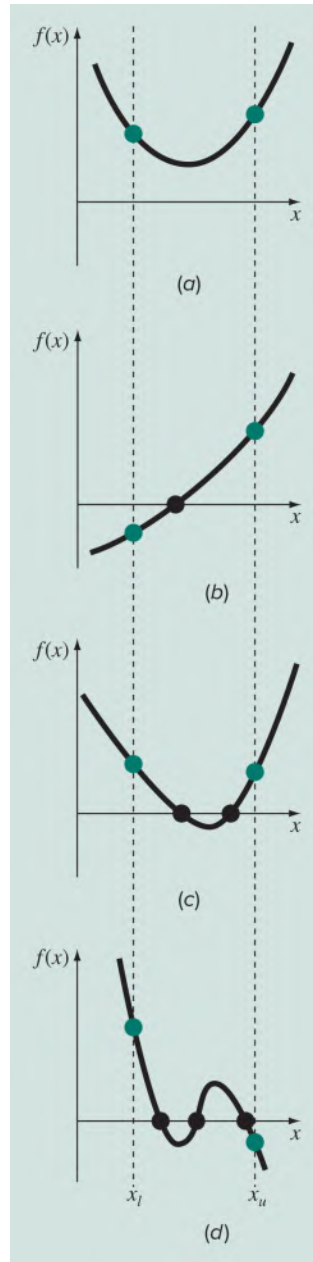
vtest(143)
36.005358021120394
```

You can observe that the sequence proceeds from low to high to low and so on, telescoping toward the desired velocity of 36 m/s. After six estimates, the result of 143 kg provides a result very close to the target velocity. Also, through the sequence, we gain insight into the sensitivity of the velocity to changes in mass. Notice that any mass between 130 and 150 kg provides a velocity reasonably close to the goal.

As with the graphical method, the trial-and-error approach can be a first-time attempt to obtain a solution to the equation that can provide insight on the behavior of the equation and guidance in specifying initial estimates for numerical methods presented later on.

Aside from providing approximate estimates of a root, graphical depictions and trial-and-error experiments are useful for understanding the properties of functions and anticipating the pitfalls of the numerical methods. For example, [Fig. 5.1](#) shows several ways that roots can occur (or be absent) in an interval prescribed by a lower bound  $x_l$  and an upper bound  $x_u$ . [Figure 5.1b](#) depicts the case where a single root is bracketed by negative and positive values of  $f(x)$ . However, [Fig. 5.1d](#), where  $f(x_l)$  and  $f(x_u)$  are also on opposite sides of the  $x$  axis, shows three roots occurring in the interval. In general, if  $f(x_l)$  and  $f(x_u)$  have opposite signs, there is an odd number of roots in the interval. As indicated by [Figs. 5.1a](#) and [5.1c](#), if  $f(x_l)$  and  $f(x_u)$  have the same sign, there are either no roots or an even number of roots between  $x_l$  and  $x_u$ .

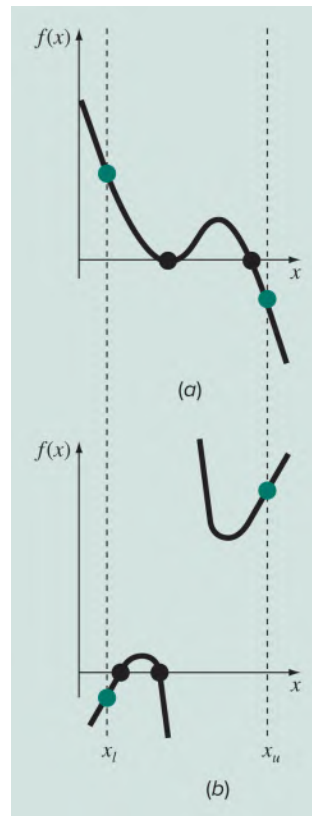
**FIGURE 5.1** Illustration of several general ways that a root may occur in an interval prescribed by a lower bound  $x_l$  and an upper bound  $x_u$ . Subfigures (a) and (c) indicate that, if both  $f(x_l)$  and  $f(x_u)$  have the same sign, either there will be no roots or there will be an even number of roots within the interval. Subfigures (b) and (d) illustrate that, if the function has different signs at the end points, there will be an odd number of roots in the interval.



Although these observations are generally true, there are cases where they do not hold. For example, functions that are tangential to the  $x$  axis  (Fig. 5.2a) and discontinuous functions  (Fig. 5.2b)

violate these principles. An example of a function that is tangential to the axis is the cubic polynomial,  $f(x) = (x - 2)(x - 2)(x - 4)$ . Notice that setting  $x$  equal to two, zeros out two terms in the polynomial. Mathematically,  $x = 2$  is called a *multiple root*. Although it is beyond the scope of this book, there are special techniques that are expressly designed to locate multiple roots (Chapra and Canale, 2021).

**FIGURE 5.2** Illustration of two exceptions to the general cases depicted in [Fig. 5.1](#). **(a)** Multiple roots that occur when the function is tangential to the  $x$  axis. For this case, although the end points are of opposite sign, there is an even number of axis interceptions over the interval. **(b)** Discontinuous functions where the end points are of opposite sign and yet bracket an even number of roots. Special strategies are required for determining the roots for these cases.



The existence of cases of the type depicted in [Fig. 5.2](#) makes it difficult to develop foolproof computer algorithms guaranteed to locate all the roots in an interval. However, when used in conjunction with graphical and trial-and-error approaches, the methods described in the following sections are extremely useful for solving many problems confronted routinely by engineers, scientists, and applied mathematicians.

It is also worth mentioning that many of the problems we solve are based on physical phenomena where the solutions are generally understood and make common sense. If it is well known that



only one solution exists between two reasonable bounds, that assures us that conventional methods will apply and function well.

## 5.3 BRACKETING METHODS AND INITIAL GUESSES

---

Before the era of automatic computers, if you had a roots problem, you would often approach it using the trial-and-error method with manual calculations, by hand or perhaps a slide rule. You would repeatedly make guesses until the function was sufficiently close to zero. This process has been greatly facilitated by the advent of software tools such as spreadsheet programs. By allowing you to evaluate many guesses rapidly, such tools can actually make the trial-and-error approach attractive for some problems, especially if they only need to be solved once or a very few times.

But, for many other problem-solving scenarios, it is preferable, even required, to have methods that come up with the correct solution automatically. Interestingly, as with trial-and-error, these approaches require at least one initial solution estimate or “guess” to get started. Then, they systematically home in on the root in an iterative fashion.

The two major classes of methods available are distinguished by the type of initial guess or guesses. They are

- *Bracketing methods.* As the name implies, these are based on two initial guesses that “bracket” the root—that is, are on either side of the root.
- *Open methods.* These methods can involve one or more initial guesses, but there is no need for them to bracket the root.

For well-posed problems, the bracketing methods are guaranteed to converge to the solution but do so more slowly, that is, they typically take more iterations to home in on the answer. In contrast, the open methods do not always work, that is, they can diverge, but, when they do, they usually converge quicker.

In both cases, initial guesses are required. These may arise naturally from the physical context you are studying. However, in other cases, good initial guesses may not be obvious. In such cases, automated approaches to obtain guesses would be useful. The following section describes one such approach, the incremental search.

### 5.3.1 Incremental Search

When applying the graphical technique in  **Example 5.1**, you observed that  $f(x)$  changed sign on opposite sides of the root. In general, if  $f(x)$  is real and continuous in the interval from  $x_l$  to  $x_u$  and  $f(x_l)$  and  $f(x_u)$  have opposite signs, as indicated by

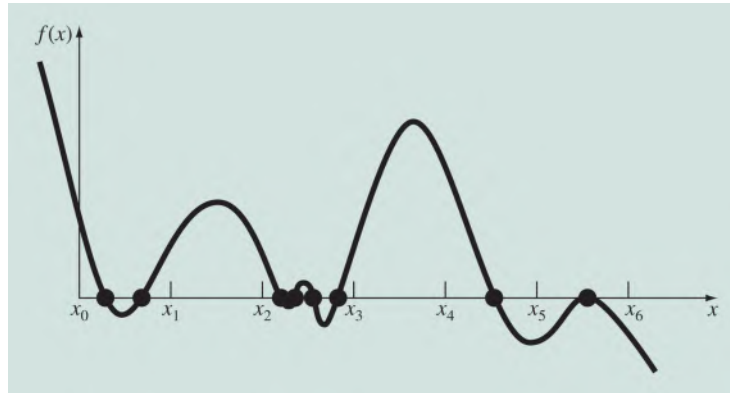
$$f(x_l)f(x_u) < 0 \quad (5.3)$$

then there is at least one real root between  $x_l$  and  $x_u$ .

*Incremental search* methods capitalize on this observation by locating an interval where the function changes sign. A potential problem with an incremental search is the choice of the increment length. If the

length is too small, the search can be time consuming. On the other hand, if the length is too great, there is a possibility that closely spaced roots might be missed (see [Fig. 5.3](#)). The problem is compounded by the possible existence of multiple roots.

**FIGURE 5.3** Cases where roots could be missed because the increment length of the search procedure is too large. Note that the last root on the right is multiple and would be missed regardless of the increment length.



A Python function can be developed<sup>1</sup> that implements an incremental search to locate the roots of a function `func` within the range from `xmin` to `xmax` ([Fig. 5.4](#)). An optional keyword argument `ns` allows the user to specify the number of intervals within the range. If `ns` is omitted, it is set to 50 by default. A `for` loop is used to step through each interval. If a sign change occurs, the upper and lower bounds are stored in an array `xb`.

**FIGURE 5.4** A Python function to implement an incremental search.

```
import numpy as np

def incsearch(func,xmin,xmax,ns=50):
    """
    incsearch: incremental search locator
    incsearch(func,xmin,xmax,ns)
    finds brackets of x that contain sign changes in
    a function of x on an interval
    input:
    func = name of the function
    xmin, xmax = endpoints of the interval
    ns = number of subintervals, default value = 50
    output: a tuple containing
    nb = number of bracket pairs found
    xb = list of bracket pair values
    or returns "no brackets found"
    """
    x = np.linspace(xmin,xmax,ns) # create array of x values
    f = [] # build array of corresponding function values
    for k in range(ns-1):
        f.append(func(x[k]))

    nb = 0
    xb = []
    for k in range(ns-2): # check adjacent pairs of function values
        if func(x[k])*func(x[k+1])<0: # for sign change
```

```

    nb = nb + 1 # increment the bracket counter
    xb.append((x[k],x[k+1])) # save the bracketing pair
if nb==0:
    return 'no brackets found'
else:
    return nb,xb

```

## EXAMPLE 5.3

### Incremental Search

#### Problem Statement

Use the Python `incsearch` function (🔗 Fig. 5.4) to identify brackets within the interval  $[3, 6]$  for the function

$$f(x) = \sin(10x) + \cos(3x) \quad (5.4)$$

#### Solution

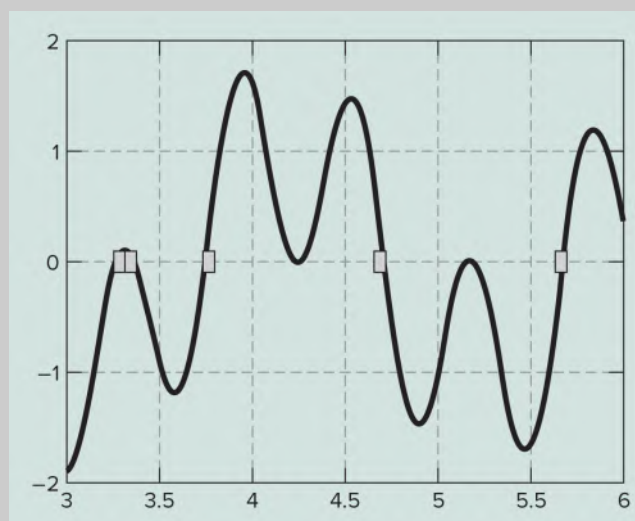
The Console session using the default number of intervals (50) is

```

import numpy as np
(nb,xb) = incsearch(lambda x: np.sin(10*x)+np.cos(3*x),3,6)
nb
5
xb
[(3.2448979591836733, 3.306122448979592),
 (3.306122448979592, 3.36734693877551),
 (3.7346938775510203, 3.795918367346939),
 (4.653061224489796, 4.714285714285714),
 (5.63265306122449, 5.6938775510204085)]

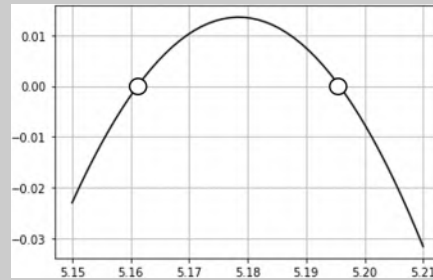
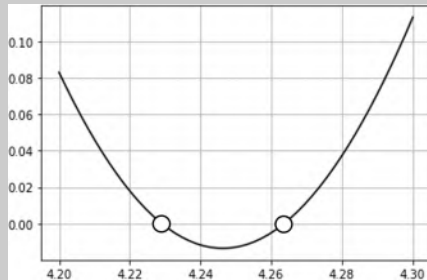
```

A plot of 🔗 Eq. (5.4) along with the root locations is shown here:





Although five sign changes are detected, because the subintervals are too wide, the function missed possible roots at  $x \cong 4.25$  and  $5.2$ . These possible roots look like they might be multiple roots. However, by zooming in, it is clear that each represents two real roots that are very close together. This is shown in the figures below:



The function can be run again with more subintervals with the result that all nine sign changes are located.

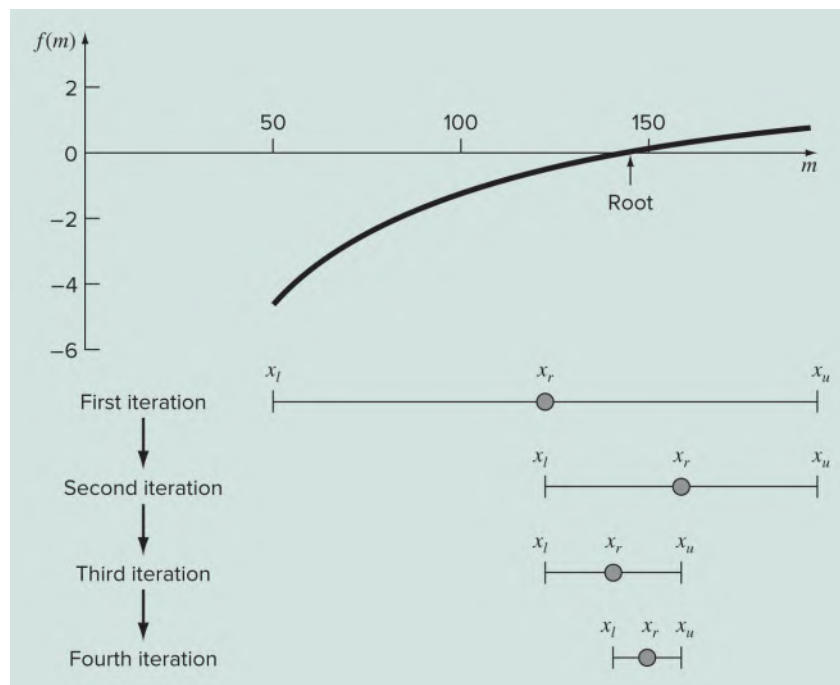
```
(nb,xb) = incsearch(lambda x: np.sin(10*x)+np.cos(3*x),3,6,100)
nb
9
xb
[(3.242424242424242, 3.2727272727272725),
 (3.3636363636363638, 3.393939393939394),
 (3.7272727272727275, 3.757575757575758),
 (4.212121212121212, 4.242424242424242),
 (4.242424242424242, 4.2727272727272725),
 (4.696969696969697, 4.7272727272727275),
 (5.151515151515151, 5.181818181818182),
 (5.181818181818182, 5.212121212121213),
 (5.666666666666667, 5.696969696969697)]
```

The foregoing example illustrates that brute-force methods, such as incremental search, are not foolproof. You would be wise to supplement such automatic techniques with any other information that provides insight into the location of the roots. Such information can be found by plotting the function and through understanding the physical problem from which the equation originated.

## 5.4 BISECTION

The *bisection method* is a variation of the incremental search method in which, for each iteration of the method, the interval containing the root is divided in half. If a function changes sign over an interval, the function value at the midpoint is evaluated. The location of the root is then determined as lying within the half interval where the sign change occurs. That subinterval becomes the interval for the next iteration. The process is repeated until the root is known to the required precision. A graphical depiction of the method is provided in [Fig. 5.5](#). The following example goes through the actual computations involved in the method.

**FIGURE 5.5** A graphical depiction of the bisection method. This plot corresponds to the first four iterations from [Example 5.4](#).



### EXAMPLE 5.4

#### The Bisection Method

#### Problem Statement

Use bisection to solve the same problem approached graphically in [Example 5.1](#).

#### Solution

The first step in bisection is to guess two values of the unknown (in the present problem,  $m$ ) with different signs in the function evaluation. From the graphical solution in [Example 5.1](#), we can see that the function changes sign between values of 50 and 200 kg. The plot obviously suggests better initial guesses, say 140 and 150, but, for illustrative purposes, let's assume we don't have the benefit of the plot and make more conservative guesses. Therefore, the first estimate for the root lies at the midpoint of the interval

$$x_r = \frac{50 + 200}{2} = 125$$

Note that the exact value of the root to four decimal places is 142.7376. This means that the value of 125 calculated above has a true percent relative error of

$$\varepsilon_t = \left| \frac{142.7376 - 125}{142.7376} \right| \times 100 \cong 12.43\%$$

Next, we compute the product of the function value at the lower bound and at the midpoint:

$$f(50)f(125) \cong (-4.579)(-0.409) \cong 1.871$$

which is greater than zero, and hence no sign change occurs between the lower bound and the midpoint. Consequently, the root must be located in the upper interval between 125 and 200. Therefore, we create a new interval by redefining the lower bound as 125.

At this point, the new interval extends from  $x_l = 125$  to  $x_u = 200$ . A revised root estimate can then be calculated as

$$x_r = \frac{125 + 200}{2} = 162.5$$

which represents a true percent error of  $\varepsilon_t \cong 13.85\%$ . This process can be repeated to obtain refined estimates. For example, the next iteration would be

$$f(125)f(162.5) \cong (-0.409)(0.359) \cong -0.147$$

Therefore, the root is now in the lower interval between 125 and 162.5. The upper bound is thus 162.5, and the new root estimate for the third iteration is calculated as

$$x_r = \frac{125 + 162.5}{2} = 143.75$$

which represents a percent relative error of  $\varepsilon_t \cong 0.709\%$ . The method can be repeated until the result is accurate enough to satisfy your needs.

We ended [Example 5.4](#) with the statement that the method could be continued to obtain a refined estimate of the root. We must now develop an objective criterion for deciding when to terminate the method.

An initial suggestion might be to end the calculation when the error falls below some prespecified level. For instance, in [Example 5.4](#), the true relative error dropped from 12.43% to 0.709% during the course of the computation. We might decide that we should terminate when the error drops below, say, 0.5%. This strategy is flawed because the error estimates in the example were based on knowledge of the true root of the function. This would not be the case in an actual situation because there would be no point in using the method if we already knew the root.

Thus, we require an error estimate that is not contingent on foreknowledge of the root. One way to do this by estimating an approximate percent relative error as in [recall [Eq. \(4.5\)](#)]

$$\varepsilon_a = \left| \frac{x_r^{new} - x_r^{old}}{x_r^{new}} \right| 100\% \quad (5.5)$$

where  $x_r^{new}$  is the root estimate for the present iteration, and  $x_r^{old}$  is the root from the previous iteration. When  $\varepsilon_a$  becomes less than a prespecified stopping criterion,  $\varepsilon_s$ , the computation is terminated.

## EXAMPLE 5.5

### Error Estimates for Bisection

#### Problem Statement

Continue [Example 5.4](#) until the approximate error falls below a stopping criterion of  $\varepsilon_s = 0.5\%$ . Use [Eq. \(5.5\)](#) to compute the errors.

#### Solution

The results of the first two iterations for [Example 5.4](#) were 125 and 162.5. Substituting these values into [Eq. \(5.5\)](#) yields

$$|\varepsilon_a| = \left| \frac{162.5 - 125}{162.5} \right| 100\% \cong 23.08\%$$

Recall that the true percent relative error for the root estimate of 162.5 was 13.85%. Therefore,  $\varepsilon_a$  is greater than  $\varepsilon_t$ . This behavior is manifested for the subsequent iterations:

| Iteration | $x_l$   | $x_u$  | $x_r$    | $\varepsilon_a\%$ | $\varepsilon_t\%$ |
|-----------|---------|--------|----------|-------------------|-------------------|
| 1         | 50      | 200    | 125      |                   | 12.43             |
| 2         | 125     | 200    | 162.5    | 23.08             | 13.85             |
| 3         | 125     | 162.5  | 143.75   | 13.04             | 0.71              |
| 4         | 125     | 143.75 | 134.375  | 6.98              | 5.86              |
| 5         | 134.375 | 143.75 | 139.0625 | 3.37              | 2.57              |

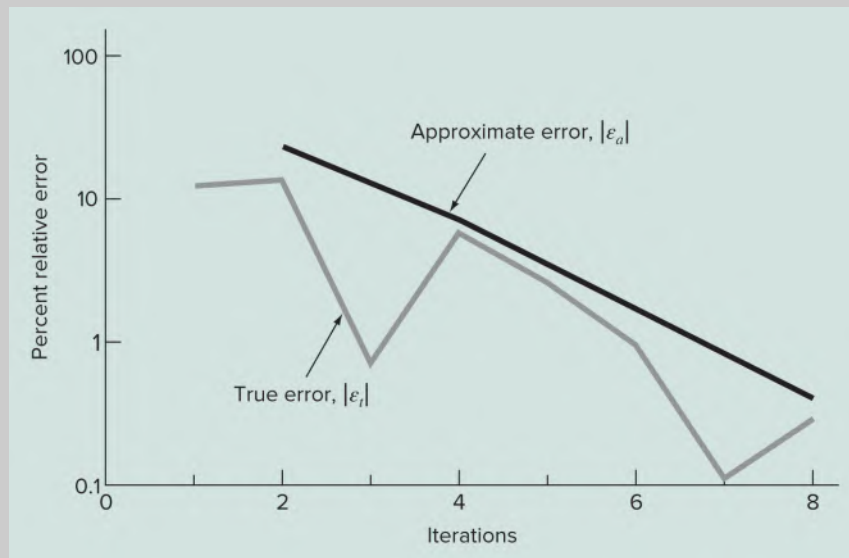


| Iteration | $x_l$    | $x_u$  | $x_r$    | $\varepsilon_a\%$ | $\varepsilon_t\%$ |
|-----------|----------|--------|----------|-------------------|-------------------|
| 6         | 139.0625 | 143.75 | 141.4063 | 1.66              | 0.93              |
| 7         | 141.4063 | 143.75 | 142.5781 | 0.82              | 0.11              |
| 8         | 142.5781 | 143.75 | 143.1641 | 0.41              | 0.30              |

Thus, after eight iterations,  $\varepsilon_a$  finally falls below  $\varepsilon_s = 0.5\%$ , and the computation can be terminated.

These results are summarized in [Fig. 5.6](#). The “ragged” nature of the true error is due to the fact that, for bisection, the true root can lie anywhere within the bracketing interval. The true and approximate errors are far apart when the true root happens to lie near one of the interval ends, as in Iteration 3. They are close when the true root is centered in the interval, as in Iteration 4.

**FIGURE 5.6** Errors for the bisection method. True and approximate errors are plotted versus the number of iterations.



Although the approximate error does not provide an exact estimate of the true error, [Fig. 5.6](#) suggests that  $\varepsilon_a$  captures the downward trend of  $\varepsilon_t$ . In addition, the plot exhibits the attractive characteristic that  $\varepsilon_a$  is always greater than  $\varepsilon_t$ . Thus, when  $\varepsilon_a$  falls below  $\varepsilon_s$ , the computation can be terminated with the assurance that the result for the root is known to be at least as accurate as the prespecified acceptable level.

While it is dangerous to draw general conclusions from a single example, it can be demonstrated that  $\varepsilon_a$  will always be greater than  $\varepsilon_t$  for bisection. This is due to the fact that, each time an approximate root is located using bisection as  $x_r = (x_l + x_u)/2$ , we know that the true root lies somewhere in the interval of  $\Delta x = x_u - x_l$ . Therefore, the root must lie within  $\pm \Delta x/2$  of our estimate. For instance, when [Example 5.5](#) was terminated, we could make the definitive statement that

$$x_r = 143.1641 \pm \frac{143.7500 - 142.5781}{2} \cong 143.1641 \pm 0.5859$$

In essence, [Eq. \(5.5\)](#) provides an upper bound on the true error. For this bound to be exceeded, the true root would have to fall outside the bracketing interval, which by definition could never occur for bisection. Although bisection is generally slower than other methods, the neatness of its error analysis is a positive feature that makes it attractive for certain engineering and scientific applications. Additionally, bisection has the advantage of stability. The method will converge to a solution for the root in the interval.

Another benefit of the bisection method is that the number of iterations required to satisfy an absolute error specification can be computed *a priori*—that is, before starting the computation. One can consider the initial interval of the method as an *interval of uncertainty*. We know that a root exists in the interval but not where it is located. Therefore, the absolute error is

$$E_a^0 = x_u^0 - x_l^0 = \Delta x^0$$

Practically, this interval is too wide, but the bisection method reduces the interval of uncertainty by a factor of 2 for each iteration. For example, after the first iteration,

$$E_a^1 = \frac{\Delta x^0}{2}$$

and after  $n$  iterations,

$$E_a^n = \frac{\Delta x^0}{2^n}$$

Consequently, after ten iterations, the original interval of uncertainty is reduced by a factor of  $2^{10} = 1024$ , about one thousand. And, after twenty iterations,  $2^{20} = 1,048,576$ , about one million.<sup>2</sup> For many practical problems, the assurance of obtaining the estimate of the root to within one part in a million of the original interval is sufficient. This allows the setting of the number of intervals at 20, simplifying the implementation of the algorithm.

If there is a desired specification for the error,  $E_{a,d}$ , the previous equation can be solved for  $n$  using the base-2 logarithm.

$$n \cong \log_2 \left( \frac{\Delta x^0}{E_{a,d}} \right) \quad (5.6)$$

Note that Python's NumPy module has a `log2` function. In addition, you can always determine the logarithm to any base,  $b$ , by recalling that  $\log_b(x) = \log(x)/\log(b)$ .

In solving for the mass,  $m$ , if we wish to meet an error specification of 0.01 kg (10 grams) with the original interval of  $\Delta x^0 = 200 - 50 = 150$  kg,

$$n \cong \log_2\left(\frac{150}{0.01}\right) = \frac{\log_{10}\left(\frac{150}{0.01}\right)}{\log_{10}(2)} \cong 13.87 \Rightarrow 14$$

So, 14 iterations will ensure meeting the specification.

We will illustrate the use of relative errors to determine the completion of the bisection method, but we will also show the alternative of executing a given number of iterations to meet an absolute error to meet a specification. The latter is often facilitated by knowledge of the problem context, as we have illustrated in the example above.

## 5.4.1 Python Functions for Bisection

The first Python function, `bisect1`, is displayed in [Fig. 5.7](#). The arguments passed to the function are three: the name of the function to be solved, along with lower and upper initial guesses. The `bisect1` function first checks to see whether the initial guesses bracket a solution and returns an error message if this is not the case. If the initial guesses check out, the method is then iterated 20 times to provide an estimate of the root to within one part in a million of the original interval.

**FIGURE 5.7** Python function to implement the bisection method for a fixed number of 20 iterations.

```
def bisect1(func,xl,xu,maxit=20):
    """
    Uses the bisection method to estimate a root of func(x).
    The method is iterated maxit (default = 20) times.
    Input:
        func = name of the function
        xl = lower guess
        xu = upper guess
    Output:
        xm = root estimate
        or
        error message if initial guesses do not bracket solution
    """
    if func(xl)*func(xu)>0:
        return 'initial estimates do not bracket solution'
    for i in range(maxit):
        xm = (xl+xu)/2
        if func(xm)*func(xl)>0:
            xl = xm
        else:
            xu = xm
    return xm
```

Notice that the implementation of the bisection method in [Fig. 5.7](#) is very brief, ten lines of code. The code is tested below and produces the desired result.

```
import numpy as np
def f(m):
    g = 9.81
    cd = 0.25
    t = 4
    v = 36
    return np.sqrt(m*g/cd)*np.tanh(np.sqrt(g*cd/m)*t)-v
m = bisect1(f,50,200)
```

```
print('mass = {0:7.3f} kg'.format(m))
mass = 142.738 kg
```

Alternately, we can write a Python function that is based on a relative error criterion and includes a safeguard against too many iterations. This is shown in [Fig. 5.8](#).

**FIGURE 5.8** Implementation of the bisection method including relative error and iteration termination criteria.

```
def bisect(func,xl,xu,es=1.e-7,maxit=30):
    """
    Uses the bisection method to estimate a root of func(x).
    The method is iterated until the relative error from
    one iteration to the next falls below the specified
    value or until the maximum number of iterations is
    reached first.
    Input:
        func = name of the function
        xl = lower guess
        xu = upper guess
        es = relative error specification (default 1.e-7)
        maxit = maximum number of iterations allowed (default 30)
    Output:
        xm = root estimate
        fm = function value at the root estimate
        ea = actual relative error achieved
        i+1 = number of iterations required
    or
        error message if initial guesses do not bracket solution
    """
    if func(xl)*func(xu)>0:
        return 'initial estimates do not bracket solution'
    xmold = xl
    for i in range(maxit):
        xm = (xl+xu)/2
        ea = abs((xm-xmold)/xm)
        if ea < es: break
        if func(xm)*func(xl)>0:
            xl = xm
        else:
            xu = xm
        xmold = xm
    return xm,func(xm),ea,i+1
```

The results from testing the `bisect` function with our example function are

Page 130

```
import numpy as np
def f(m):
    g = 9.81
    cd = 0.25
    t = 4
    v = 36
    return np.sqrt(m*g/cd)*np.tanh(np.sqrt(g*cd/m)*t)-v

(m,fm,ea,iter) = bisect(f,50,200)
print('mass = {0:10.6f} kg'.format(m))
print('function value = {0:7.3g}'.format(fm))
print('relative error = {0:7.3g}'.format(ea))
print('iterations = {0:5d}'.format(iter))

mass = 142.737636 kg
function value = 5.85e-08
relative error = 6.26e-08
iterations = 24
```

Note that the relative error is less than the default specification of  $1 \times 10^{-7}$  (or  $1 \times 10^{-5} \%$ ). To meet the criterion, 24 iterations were required. If instead we had specified a maximum of 20 iterations, the code and results would be

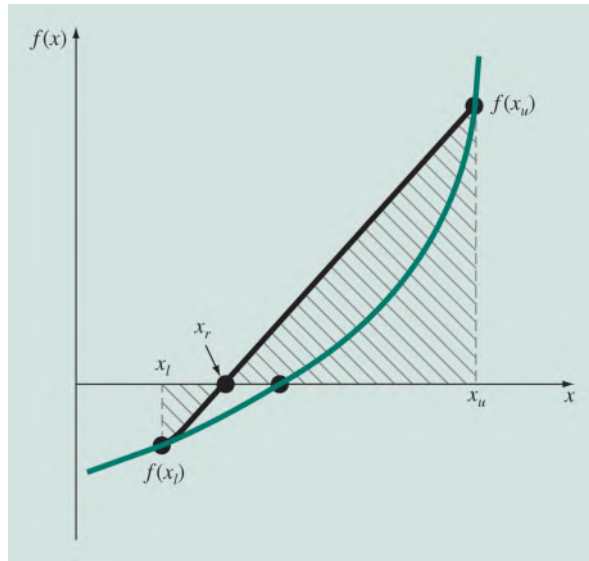
```
(m, fm, ea, iter) = bisect(f, 50, 200, maxit=20)
mass = 142.737627 kg
function value = -1.24e-07
relative error = 1e-06
iterations = 20
```

In this case, the relative error specification of  $1 \times 10^{-7}$  is not met. Regardless, the estimate of the root is precise, agreeing to the fourth decimal fraction.

## 5.5 FALSE POSITION

*False position* (also called linear interpolation or *regula falsi*) is another well-known bracketing method. It is similar to bisection except that it uses a different strategy to come up with the new estimate of the root. Rather than bisecting the interval, it locates the new estimate where a straight line between  $\{x_l, f(x_l)\}$  and  $\{x_u, f(x_u)\}$  crosses the  $f(x) = 0$  axis. This is shown in [Fig. 5.9](#). The curvature of the function influences how close the new estimate,  $x_r$ , is to the true root.

**FIGURE 5.9** False position method.



Observing the two similar cross-hatched triangles in the figure, we can express the equivalency of the straight-line slopes as

$$\frac{0 - f(x_l)}{x_r - x_l} = \frac{f(x_u) - 0}{x_u - x_r}$$

This can be solved algebraically for  $x$  to obtain the *false position* formula

$$x_r = \frac{f(x_u)x_l - f(x_l)x_u}{f(x_u) - f(x_l)} \quad (5.7)$$

The value of  $x_r$  computed with [Eq. \(5.7\)](#) replaces either  $x_l$  or  $x_u$  for the next iteration of the method, depending on the sign of  $f(x_r)$ . As illustrated in the [Fig. 5.9](#),  $x_r$  would become the next  $x_l$ . In this way,  $x_l$  and  $x_u$  will always bracket the root. The process is iterated until the root estimate meets an error specification.

The Python function for false position is similar to the one for bisection except that [Eq. \(5.7\)](#) replaces the midpoint calculation.

$$x_m = (\text{func}(x_u) * x_l - \text{func}(x_l) * x_u) / (\text{func}(x_u) - \text{func}(x_l))$$

Also, with the false position method, we cannot predict the maximum absolute error or the interval of uncertainty using the number of iterations, as we were able to do with the bisection method.

## EXAMPLE 5.6

### The False Position Method

#### Problem Statement

Use false position to solve the same problem approached graphically in [Example 5.1](#) and with bisection in [Example 5.4](#).

#### Solution

As in [Example 5.4](#), initiate the computation with guesses of  $x_l = 50$  and  $x_u = 200$ . The first iteration of the method is

$$\begin{aligned} x_l &= 50 & f(x_l) &= -4.57939 & x_u &= 200 & f(x_u) &= 0.86029 \\ x_r &\cong \frac{(-4.57939)(50) - (0.86029)(200)}{(-4.57939)(0.86029)} \cong 176.2773 & f(x_r) &\cong 0.56617 \end{aligned}$$

This estimate for the root has a true relative error of 23.5%. The table below shows the first eight iterations of the method.

| Iteration | $x_l$ | $x_u$    | $x_r$    | $\epsilon_a\%$ | $\epsilon_t\%$ |
|-----------|-------|----------|----------|----------------|----------------|
| 1         | 50    | 200      | 176.2773 |                | 23.50          |
| 2         | 50    | 176.2773 | 162.3828 | 8.56           | 13.76          |
| 3         | 50    | 162.3828 | 154.2446 | 5.28           | 8.06           |
| 4         | 50    | 154.2446 | 149.4777 | 3.19           | 4.72           |
| 5         | 50    | 149.4777 | 146.6856 | 1.90           | 2.77           |
| 6         | 50    | 146.6856 | 145.0501 | 1.13           | 1.62           |
| 7         | 50    | 145.0501 | 144.0922 | 0.66           | 0.95           |
| 8         | 50    | 144.0922 | 143.5311 | 0.39           | 0.56           |

In the table, notice how the approximate error is always less than the true error, in contrast with bisection as in [Example 5.5](#). This will not always be the case with false position but is so here and is certainly a deficiency. Also, note that the lower guess ( $x_l$ ) does not change. In effect, the method is approaching the solution from above; in a way, “crawling in.” Comparing to the similar bisection table from [Example 5.5](#), we see that the approximate error here is slightly less than

bisection after eight iterations, 0.39% versus 0.41%; however, the true error is greater, 0.56% versus 0.30%.

If we test a false position Python function (we have named it `regfal`), these are the results:

```
(m, fm, ea, iter) = regfal(f, 50, 200)

mass = 142.737651 kg
function value = 3.67e-07
relative error = 8.89e-08
iterations = 28
```

An observation here is that false position required 28 iterations to meet the same relative error specification that bisection met in 24 iterations.

The results for the scenario in [Example 5.4](#) might lead us to conclude that bisection is superior to false position. This is not true. In many cases, false position will converge more quickly than bisection. You will explore this in the problems at the end of this chapter.

Using false position to solve [Example 5.4](#) does reveal a major weakness of the method: its one-sidedness. For many scenarios, as iterations proceed, one of the bracketing points will tend to stay fixed. This can lead to poor convergence, particularly for functions with significant curvature. Possible remedies are available elsewhere (Chapra and Canale, 2010).



## 5.6 CASE STUDY

### GREENHOUSE GASES AND RAINWATER

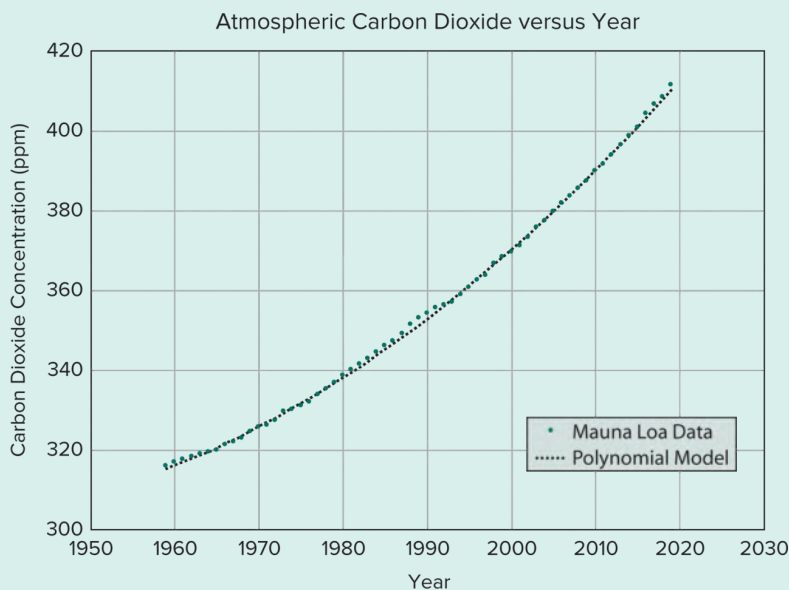
#### Background

It is well documented that the atmospheric levels of several so-called “greenhouse gases” have been increasing over the past 50 years. For example, [Fig. 5.10](#) shows data for partial pressure<sup>3</sup> of carbon dioxide ( $\text{CO}_2$ ) from samples collected at Mauna Loa, Hawai’i, from 1959 through 2019.<sup>4</sup> The trend in these data shows a slight upward curvature from a straight line and can be nicely fit with a second-order polynomial.<sup>5</sup>

$$p_{\text{CO}_2} = 0.012772(t - 1983)^2 + 1.4272(t - 1983) + 342.35$$

where  $p_{\text{CO}_2} = \text{CO}_2$  partial pressure (ppm). These data indicate that levels have increased over 30% over the period from 316 to 411 ppm.

**FIGURE 5.10** Average annual partial pressure of atmospheric carbon dioxide (ppm) measured at Mauna Loa, Hawai’i. Note that commonly,  $\text{CO}_2$  partial pressure is not reported in pressure units but rather in concentration units as parts per million (ppm) on a molar basis, that is,  $\mu\text{mol}/\text{mol}$  as is the case here.



One question we can address is how this trend is affecting the pH of rainwater. Outside of urban and industrial areas, it is well documented that carbon dioxide is the primary determinant of the pH of rain, where pH is the measure of the activity of hydrogen ions ( $\text{H}^+$ ), and therefore the acidity or alkalinity of rainwater. For dilute aqueous solutions, pH can be computed as

$$\text{pH} = -\log_{10} [\text{H}^+] \quad (5.8)$$

where  $[\text{H}^+]$  is the molar concentration of hydrogen ions.

The following five equations govern the chemistry of rainwater:

$$K_1 = \frac{[\text{H}^+][\text{HCO}_3^-]}{K_H p_{\text{CO}_2}} \quad (5.9)$$

$$K_2 = \frac{[\text{H}^+][\text{CO}_3^{2-}]}{[\text{HCO}_3^-]} \quad (5.10)$$

$$K_w = [\text{H}^+][\text{OH}^-] \quad (5.11)$$

$$c_T = \frac{K_H p_{\text{CO}_2}}{10^6} \quad (5.12)$$

$$0 = [\text{HCO}_3^-] + 2[\text{CO}_3^{2-}] + [\text{OH}^-] - [\text{H}^+] \quad (5.13)$$

where  $K_H$  = Henry's constant and  $K_1$ ,  $K_2$ , and  $K_w$  are equilibrium constants. The five unknown concentrations are  $c_T$  = total organic carbon,  $[\text{HCO}_3^-]$  = bicarbonate ion,  $[\text{CO}_3^{2-}]$  = carbonate ion,  $[\text{H}^+]$  = hydrogen ion, and  $[\text{OH}^-]$  = hydroxyl ion. Notice how the partial pressure of  $\text{CO}_2$  appears in [Eqs. \(5.9\)](#) and [\(5.12\)](#).

Use these equations to compute the pH of rainwater given that  $K_H = 10^{-1.46}$ ,  $K_1 = 10^{-6.3}$ ,  $K_2 = 10^{-10.3}$ , and  $K_w = 10^{-14}$ . Compare the results in 1959 when the  $p_{\text{CO}_2}$  was measured at 316 ppm, and in 2019 when the measurement was 411 ppm. When selecting a numerical method for your computation, consider the following:

- You know with certainty that the pH of rain in pristine areas falls between 2 and 12.
- You also know that pH can only be measured to two decimal places of precision.

## Solution

There are a variety of ways to solve this system of five equations. One way is to eliminate unknowns by combining the equations algebraically to produce a single equation that depends only on  $[\text{H}^+]$ . To do this, first rearrange [Eqs. \(5.9\)](#) and [\(5.10\)](#) as follows:

$$[\text{HCO}_3^-] = \frac{K_1}{10^6 [\text{H}^+]} K_H p_{\text{CO}_2} \quad (5.14)$$

$$(5.15)$$

$$[\text{CO}_3^{-2}] = \frac{K_2[\text{HCO}_3^-]}{[\text{H}^+]}$$

Substitute [Eq. \(5.14\)](#) into [Eq. \(5.15\)](#) to get

$$[\text{CO}_3^{-2}] = \frac{K_2 K_1}{10^6 [\text{H}^+]^2} K_{HP\text{CO}_2} \quad (5.16)$$

[Equations \(5.14\)](#) and [Eq. \(5.16\)](#) are then substituted into [Eq. \(5.13\)](#) along with [Eq. \(5.11\)](#) to yield

$$0 = \frac{K_1}{10^6 [\text{H}^+]} K_{HP\text{CO}_2} + 2 \frac{K_2 K_1}{10^6 [\text{H}^+]^2} K_{HP\text{CO}_2} + \frac{K_w}{[\text{H}^+]} - [\text{H}^+] \quad (5.17)$$

Although it might not be immediately apparent, this [Eq. \(5.16\)](#) is a third-order polynomial in  $[\text{H}^+]$ . Thus, its root can be used to compute the pH of the rainwater with [Eq. \(5.8\)](#).

Now, we must decide which numerical method to employ to obtain the solution. There are two reasons why bisection is a good choice. First, the fact that the pH falls within the range from 2 to 12 provides us with two good initial guesses. Second, because pH can only be measured to two decimal places, we will be satisfied with an absolute error of  $E_{a,d} = \pm 0.005$ . Recall that, given the initial bracket or interval and the desired error specification, we can compute *a priori* the required number of iterations for bisection. This number can be computed in the Python Console as

```
import numpy as np
dx = 12 - 2
Ead = 0.005
n = np.log2(dx/Ead)

n
10.965784284662087
```

We see that eleven iterations will provide the desired precision.

First, in the Python Editor, we create a function that will compute [Eq. \(5.16\)](#).

```
def f(pH,pCO2):
    K1 = 1e-6 ; K2 = 1e-10 ; Kw = 1e-14
    KH = 10**(-1.46)
    H = 10**(-pH)
    return K1/1e6/H*KH*pCO2 + 2*K2*K1/1e6/H**2 + Kw/H - H
```

We can then use the `bisect1` function shown in [Fig. 5.7](#), modified to allow an argument specifying the number of iterations, to solve for the pH.

```
def bisect1(func,xl,xu,maxit=30):
    """
    Uses the bisection method to estimate a root of func(x).
    The method is iterated maxit times.
    Input:
        func = name of the function
        xl = lower guess
        xu = upper guess
        maxit = number of iterations
    Output:
```

```

        xm = root estimate
        or
        error message if initial guesses do not bracket solution
    """
    if func(xl)*func(xu)>0:
        return 'initial estimates do not bracket solution'
    for i in range(maxit):
        xm = (xl+xu)/2
        if func(xm)*func(xl)>0:
            xl = xm
        else:
            xu = xm
    return xm

pHsoln = bisect1(lambda pH: f(pH,316),2,12,11)
print('{0:5.2f}'.format(pHsoln))

5.48

```

For 2019, the result is

```

pHsoln = bisect1(lambda pH: f(pH,411),2,12,11)
print('{0:5.2f}'.format(pHsoln))

5.42

```

Interestingly, the results indicate that a 30% rise in atmospheric CO<sub>2</sub> levels has produced only a 1.1% drop in pH of rainwater. Although this is certainly true, remember that pH represents a logarithmic scale as defined by [Eq. \(5.8\)](#). Consequently, a unit drop in pH is a 10-fold increase in the hydrogen ion concentration. That concentration is computed from pH as  $[H^+] = 10^{-pH}$  and its percent change from 1959 to 2019 can be calculated as

```

(10**(-5.42)-10**(-5.48))/10**(-5.42)*100
12.90364100439204

```

So, the hydrogen ion concentration has increased about 13% over the 61 years.

Now that we have a method for determining rainwater pH from atmospheric CO<sub>2</sub> concentration, we prepare a text file, `MaunaLoaCO2Data.txt` from the source data file that contains only the `year`, `ppm` data. We can add Python code to read in the `year` and `ppm` values into arrays, solve for the pH for each year, and plot pH versus year, similar to [Fig. 5.10](#). Here is the Python script that takes advantage of the `bisect1` and `f` functions developed above.

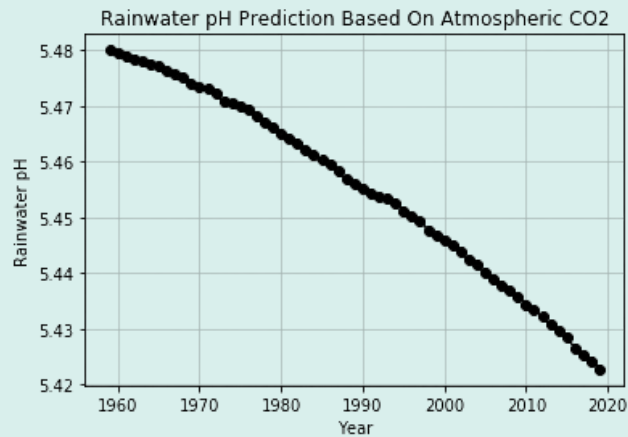
```

import numpy as np
year,ppm = np.loadtxt('MaunaLoaCO2Data.txt',unpack=True)
n = len(year)
pHpred = []
for i in range(n):
    pHsoln = bisect1(lambda pH: f(pH,ppm[i]),2,12,20)
    pHpred.append(pHsoln)

import pylab
pylab.plot(year,pHpred,c='k',marker='o')
pylab.grid()
pylab.xlabel('Year')
pylab.ylabel('Rainwater pH')
pylab.title('Rainwater pH Prediction Based On Atmospheric CO2')

```

and the resulting plot



Greenhouse gas trends represent a controversial topic. Most of the debate has focused on whether the increases are contributing to global warming and climate change. However, regardless of the ultimate implications, it is sobering to realize that something as large as our atmosphere has changed so much over a relatively short period of time. This case study illustrates how numerical methods and Python can be employed to analyze and interpret such trends. Today, engineers and scientists are using such tools to gain increased understanding of such phenomena and help rationalize the debate over their ramifications.

# PROBLEMS

**5.1** Use bisection to determine the drag coefficient needed so that an 95-kg bungee jumper has a velocity of 46 m/s after 9 s of free fall. Note: The acceleration due to gravity is  $9.81 \text{ m/s}^2$ . Start with initial guesses of  $x_l = 0.2$  and  $x_u = 0.5$  and iterate until the approximate relative error falls below 5%. Carry these calculations out by hand using a calculator.

**5.2** Develop your own Python function for bisection similar to that presented in [Fig. 5.7](#). However, rather than using the maximum iterations and [Eq. \(5.5\)](#), employ [Eq. \(5.6\)](#) to determine the number of iterations as your stopping criterion. Make sure to round the result of [Eq. \(5.6\)](#) up to the next highest integer. Python's NumPy `np.ceil` function provides a handy way to do this, but it produces a float type result. You have to use the `int` function to convert it to an integer. Use your last name as part of the function name. The first line of your function should be

```
def LastNameBisect(func,xl,xu,Ea,d)
```

and the last should be

```
return root,Ea,ea,n
```

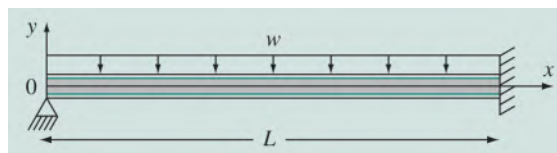
Note that, for the output,  $Ea$  = the approximate absolute error and  $ea$  = the approximate relative error. Then, add your own Python script to solve [Prob. 5.1](#). Note that you must pass the parameters via the arguments. In addition, modify the `def` statement above so that the default value for  $E_{a,d} = 1 \times 10^{-7}$ .

**5.3** [Figure P5.3](#) shows a pinned fixed beam subject to a uniform load. The equation for the resulting deflections is

Page 137

$$y = -\frac{w}{48EI}(2x^4 - 3Lx^3 + L^3x)$$

**FIGURE P5.3** Pinned fixed beam with uniform load.



Develop a Python script that

- plots the function,  $dy/dx$  versus  $x$  (with appropriate labels) and
- uses your `LastNameBisect` function to determine the point of maximum deflection, that is, the value of  $x$ , where  $dy/dx = 0$ . Then, substitute this value into the deflection equation to determine the value of maximum deflection. Employ initial guesses of  $x_l = 0$  and  $x_u = 0.9L$ .

Employ the following parameter values in your computation, making sure you use consistent units:

$L = 400 \text{ cm}$ ,  $E = 52,000 \text{ kN/cm}^2$ ,  $I = 32,000 \text{ cm}^4$ ,  $w = 4 \text{ kN/cm}$ . Also, use

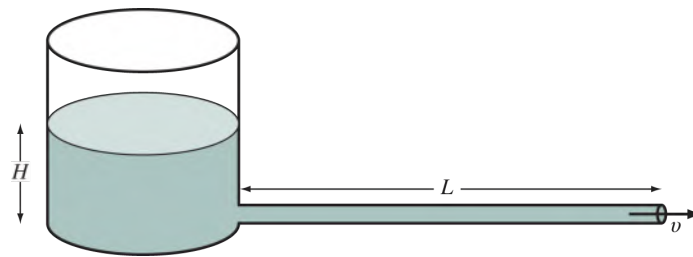
$E_{a,d} = 1 \times 10^{-7}$  cm. Display your results to six decimal places.

**5.4** As shown in  **Fig. P5.4**, water is discharged from a cylindrical tank through a horizontal pipe. Once the pipe is opened, the velocity of the water in the pipe versus time can be computed from

$$v = \sqrt{2gH} \tanh\left(\sqrt{\frac{gH}{L}}t\right)$$

where  $g = 9.81 \text{ m/s}^2$ ,  $H$  = initial liquid level in the tank, m,  $L$  = pipe length, m,  $t$  = elapsed time, s. The equation above assumes that the liquid level in the tank does not vary significantly during the transition time.

**FIGURE P5.4** Tank with horizontal exit pipe.



Develop a Python script that

- plots the function  $v = f(H)$  versus  $H$  for a given time,  $t$ . Present curves for  $t = 1, 2, 3, 4$ , and  $5$  s for  $H = 0$  to  $4$  m on the same plot, and  $L = 50$  m. Label the plot and include a legend.
- uses your `LastNameBisect` function with initial guesses  $x_l = 0$  and  $x_u = 4$  to determine the  $H$  required to achieve  $v = 5 \text{ m/s}$  at  $t = 2.5$  s for a  $4$ -m long pipe. Use  $E_{a,d} = 1 \times 10^{-7}$ . Display Python's full precision in your results.

**5.5** Solve  **Prob. 5.1** using your `LastNamebisection` function.

**5.6** Solve  **Prob. 5.1** using false position. Name your function `LastNameregfal`. Compare the results to those from  **Prob. 5.5**.

**5.7** (a) Determine the roots of  $f(x) = -12 - 21x + 18x^2 - 2.75x^3$  graphically using a Python script. In addition, find the first root (farthest left on the number line) of the function with (b) bisection and (c) false position. Pick reasonable initial guesses and either an iteration limit of 20 or an error criterion of 0.1%. Compare your results in (b) and (c).

**5.8** For the function  $f(x) = \sin(x) - x^2$ , where  $x$  is in radians,

- use an incremental search to bracket any roots in the interval  $0.01 \leq x \leq 2$ ;
- plot the function over this domain of  $x$  to confirm your findings in (a); and
- select appropriate initial guesses based on your observations in (a) and (b), and use bisection to solve for the root.

**5.9** Determine the positive real root of  $\ln(x^2) = 0.7$  (a) graphically, (b) using the bisection method with initial guesses  $x_l = 0.5$  and  $x_u = 2$ , and (c) using false position. Compare your results. Can you solve this equation analytically for  $x$  and find the solution with a simple computation?

**5.10** The saturation concentration of dissolved oxygen in freshwater can be calculated with the equation

$$\ln(o_{sf}) = -139.34 + \frac{157.57}{T_a/1000} - \frac{66.423}{(T_a/1000)^2} + \frac{12.438}{(T_a/1000)^3} - \frac{0.86219}{(T_a/1000)^4}$$

where  $o_{sf}$  = the saturation concentration of dissolved oxygen in freshwater at one atm (mg/L), and  $T_a$  = absolute temperature (K). Recall that  $T_a = T + 273.15$ , where  $T$  = temperature ( $^{\circ}\text{C}$ ). According to this formula, saturation decreases with increasing temperature. For typical natural waters in temperate climates, the formula can be used to determine that oxygen concentration ranges from about 14.6 mg/L at 0  $^{\circ}\text{C}$  to 7.0 mg/L at 35  $^{\circ}\text{C}$ . Given a value of oxygen concentration, this formula and the bisection method can be used to solve for the temperature in  $^{\circ}\text{C}$ .

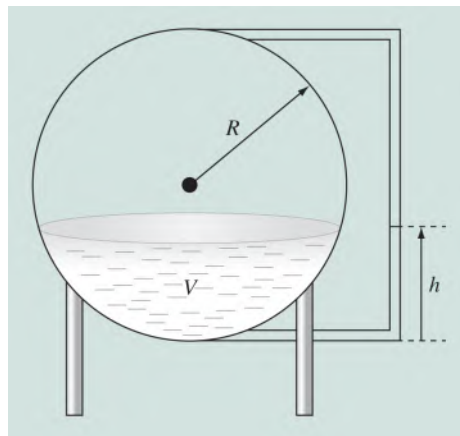
- (a) If the initial guesses are set at 0  $^{\circ}\text{C}$  and 35  $^{\circ}\text{C}$ , how many iterations of the bisection method would be required to determine the temperature within an absolute error of 0.05  $^{\circ}\text{C}$ ?
- (b) Based on (a), use Python to determine  $T$  as a function of given oxygen concentration. Test your code for  $o_{sf} = 8, 10$ , and 14 mg/L.

**5.11** Large quantities of liquids are often stored in spherical tanks. See  **Fig. P5.11**, which illustrates a partially filled tank. The volume of liquid in the tank is given by

$$V = \frac{\pi h^2(3R - h)}{3}$$

where  $R$  = the inside radius of the tank and  $h$  = the depth of the liquid from the liquid surface to the center bottom of the tank. There is a “sight gauge” on the outside of the tank that allows the visual measurement of  $h$ . It would be more desirable to place next to the gauge a scale calibrated directly in volume,  $V$ , rather than  $h$ .

**FIGURE P5.11** Partially filled spherical tank.







- (a) For a tank with  $R = 5$  m, use bisection to solve for  $h$  for a liquid volume of  $300 \text{ m}^3$ . Note: The equation above is a cubic polynomial in  $h$  and has three real roots; however, one root is above the tank and another is below the tank. So, using empty and full tank as initial guesses will guarantee isolation of the meaningful root. Also, note that  $h = 2R$  at the top of the tank.
- (b) Again, using bisection, compute and display a calibration table of  $h$  versus  $V$  for  $10 \text{ m}^3$  intervals from empty to (just before) full tank. Note: The volume of the full tank is the volume of a sphere of radius  $R$ .

**5.12** Water is flowing in an open trapezoidal channel. See the diagram of the channel cross-section in  **Fig. P5.12**. The cross-sectional area of the channel is given by

$$A_c = \frac{1}{2}(B + W)y$$

where, as shown in the diagram,  $W$  = channel width at the surface,  $B$  = channel width at the base, and  $y$  = the fluid depth. Typical channel designs use a  $45^\circ$  slope for the sides of the channel. If this is the case,  $W = B + 2y$  and

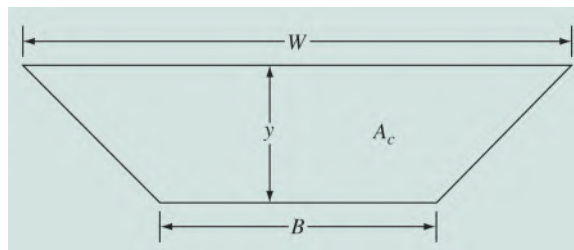
$$A_c = (B + y)y$$

The critical depth  $y$  for such a channel<sup>6</sup> must satisfy the equation

$$\frac{V}{\sqrt{gy}} = 1 \quad \text{or} \quad \frac{Q}{A_c \sqrt{gy}} = 1 \quad \text{or} \quad \frac{Q}{(B + y)y \sqrt{gy}} = 1$$

where  $V$  = flow velocity and  $Q$  = volumetric flow rate.

**FIGURE P5.12** Trapezoidal channel cross-section.



For a flow rate,  $Q = 20 \text{ m}^3/\text{s}$ , and a channel base width,  $B = 3$  m, solve for the critical depth,  $y$ , using (a) the graphical method, (b) bisection, and (c) false position. For (b) and (c), use initial guesses of  $x_l = 0.5$  and  $x_u = 2.5$  m, and iterate until the approximate error falls below  $1 \times 10^{-6}$  or the number of iterations exceeds 20. Compare your results from (b) and (c).

**5.13** The Michaelis-Menten model describes the chemical kinetics of enzyme-mediated reactions:

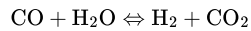
$$\frac{dS}{dt} = -v_m \frac{S}{k_s + S}$$

where  $S$  = substrate concentration (mol/L),  $v_m$  = maximum uptake rate (mol/L/d), and  $k_s$  = half-saturation constant (mol/L), which is the substrate level at which uptake is half of the maximum. If the initial substrate concentration is  $S_0$ , this differential equation can be solved to yield

$$S = S_0 - v_m t + k_s \ln\left(\frac{S_0}{S}\right)$$

Develop a Python script to generate a plot with a family of curves of  $S$  versus  $t$  for  $S_0 = 8$  mol/L,  $v_m = 0.7$  mol/L/d, and  $k_s = 2, 3$ , and 4 mol/L.

**5.14** The *water-gas shift* chemical reaction is given by



At high temperatures, greater than 800 °C, this reaction is governed by equilibrium, which can be represented by

$$\frac{[\text{H}_2][\text{CO}_2]}{[\text{H}_2\text{O}][\text{CO}]} = K_{\text{eq}}(T)$$

where  $[\text{H}_2]$  = the amount of hydrogen present and so forth. The equilibrium constant,  $K_{\text{eq}}$ , is dependent on the absolute temperature,  $T$  (K), and given by the relationship

$$\log_{10}(K_{\text{eq}}) = -1.156 + 0.08849\left(\frac{10000}{T}\right) + 0.004299\left(\frac{10000}{T}\right)^2$$

If a gas mixture with given component flow rates,  $\dot{m}_{\text{H}_2}$  and so forth, is fed to a reactor where equilibrium will be established, the shift in composition,  $x$ , can be represented using the equilibrium relationship above.

$$\frac{(\dot{m}_{\text{H}_2} + x)(\dot{m}_{\text{CO}_2} + x)}{(\dot{m}_{\text{H}_2\text{O}} - x)(\dot{m}_{\text{CO}} - x)} = K_{\text{eq}}(T)$$

For the following component flow rates in the feed, and a temperature of 1200 °C, use bisection to determine the value of the shift  $x$  and the flow rates of the components leaving the reactor in equilibrium. You may wish to use the graphical method to choose suitable initial guesses.

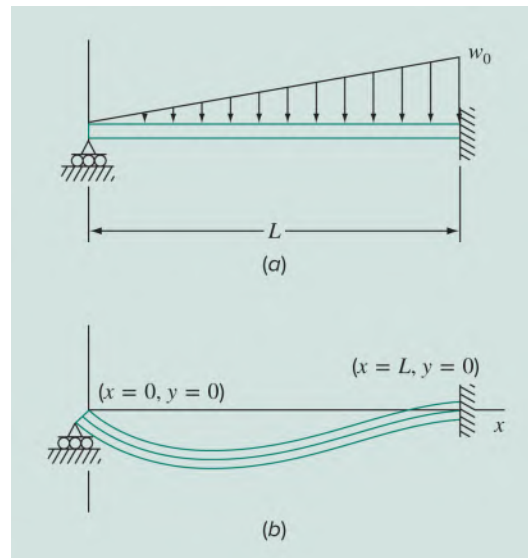
| Component        | kmol/hr |
|------------------|---------|
| H <sub>2</sub>   | 450     |
| CO               | 500     |
| CO <sub>2</sub>  | 50      |
| H <sub>2</sub> O | 1150    |

**5.15**  Figure P5.15a shows a uniform beam subject to a linearly increasing distributed load. The equation for the resulting elastic curve is (see  Fig. P5.15b—the curve is exaggerated for illustration purposes)

$$y = \frac{w_0}{120EI L} (-x^5 + 2L^2 x^3 - L^4 x)$$

where  $y$  = the deflection at position  $x$ ,  $w_0$  = load per unit length of beam,  $E$  = modulus of elasticity of the beam material,  $I$  = beam moment of inertia,  $L$  = length of the beam. For the parameter values  $L = 600$  cm,  $E = 50,000$  kN/cm<sup>2</sup>,  $I = 30,000$  cm<sup>4</sup>, and  $w_0 = 2,500$  N/cm, use bisection to determine the location of maximum deflection, that is, the value of  $x$  where  $dy/dx = 0$ . Then, use the equation above to determine the value of the maximum deflection.

**FIGURE P5.15** Uniform beam subject to a linear increasing load.



**5.16** You plan to buy a Tesla Model S vehicle for \$79,990 by paying 10% down and financing the balance over a 7-year term. You have budgeted a monthly payment of \$1,000. So, now you need to shop for a loan at the required interest rate (or lower, if you can find it!). The formula that governs this is

$$A = P \frac{i(1+i)^n}{(1+i)^n - 1}$$

where  $A$  = the monthly payment,  $P$  = the loan amount,  $i$  = the monthly interest rate expressed as a fraction, not a percentage.

Note: Loan interest rates are typically quoted on an annual basis as “APR.” To obtain the monthly interest rate,  $i$ , you divide the APR by 12. The actual annual interest rate you pay is greater than the

quoted APR. It is  $(1 + i)^{12} - 1$  ( $\times 100$  for %). As an example, for an APR of 6%, with monthly payments, the actual annual interest rate is 6.17%.

Use Python and the bisection method to determine the APR you will need to obtain to meet your goal. Choose initial guesses for APR of 3% and 9%.

**5.17** Many fields of engineering and beyond require accurate population forecasts. For example, transportation engineers might find it necessary to determine separately the population growth trends in a city and an adjacent suburb. The population of the urban area is declining with time according to

$$P_u(t) = P_{u,\max} e^{-k_u t} + P_{u,\min}$$

while the suburban population is growing, described by

$$P_s(t) = \frac{P_{s,\max}}{1 + [P_{s,\max}/P_0 - 1]e^{-k_s t}}$$

where  $P_{u,\max}$ ,  $P_{u,\min}$ ,  $P_{s,\max}$ ,  $P_0$ ,  $k_u$ , and  $k_s$  are parameters whose values are determined empirically for the city/suburb scenario being modeled. For the parameter values below, determine the time and the corresponding values of  $P_u(t)$  and  $P_s(t)$  when the suburb is 20% larger than the urban area. Use (a) the graphical method to determine initial guesses, and (b) the false position method to obtain a precise solution.

$$P_{u,\max} = 110,000, P_{u,\min} = 80,000, P_{s,\max} = 320,000, P_0 = 10,000, k_u = 0.05/\text{yr}, k_s = 0.09/\text{yr}$$

**5.18** The resistivity,  $\rho$ , of doped silicon is based on the charge,  $q$ , on an electron, the electron density,  $n$ , and the electron mobility,  $\mu$ . The electron density is given in terms of the doping density,  $N$ , and the intrinsic carrier density,  $n_i$ . The electron mobility is described by the temperature,  $T$ , the reference temperature,  $T_0$ , and the reference mobility,  $\mu_0$ . The equations required to compute the resistivity are

$$\rho = \frac{1}{qn\mu}$$

where

$$n = \frac{1}{2} \left( N + \sqrt{N^2 + 4n_i^2} \right) \quad \text{and} \quad \mu = \mu_0 \left( \frac{T}{T_0} \right)^{-2.42}$$

Determine  $N$ , given

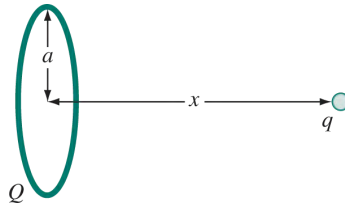
$T_0 = 300 \text{ K}$ ,  $T = 1000 \text{ K}$ ,  $\mu_0 = 1360 \text{ cm}^2/\text{V}\cdot\text{s}$ ,  $q = 1.7 \times 10^{-19} \text{ C}$ ,  $n_i = 6.21 \times 10^9 \text{ cm}^{-3}$ , and a desired  $\rho = 6.5 \times 10^6 \text{ V}\cdot\text{s}\cdot\text{cm}/\text{C}$ . Employ initial guesses of 0 and  $2.5 \times 10^{10}$  for  $N$ . Solve for  $N$  using the bisection method with 20 iterations.

**5.19** A total charge,  $Q$ , is uniformly distributed around a ring-shaped conductor with radius  $a$ . A separate charge,  $q$ , is located at a distance,  $x$ , from the center of the ring (see  Fig. P5.19). The force exerted on the charge by the ring is given by

$$F = \frac{1}{4\pi\epsilon_0} \frac{qQx}{(x^2 + a^2)^{3/2}}$$

where  $\epsilon_0 = 8.9 \times 10^{-12} \text{ C}^2/(\text{N} \cdot \text{m}^2)$ . Find the distance,  $x$ , where the force is 1.25 N, if  $q$  and  $Q$  are both equal to  $2 \times 10^{-5} \text{ C}$  for a ring with radius 0.85 m.

**FIGURE P5.19** Ring charge and point charge.



**5.20** For turbulent flow in pipes, friction can be described by a dimensionless number,  $f$ , the Fanning friction factor.<sup>7</sup> This quantity is dependent on a number of parameters related to the pipe and the fluid. These include the pipe diameter,  $D$ , and wall roughness,  $\epsilon$ , and the Reynolds number,  $\text{Re}$ , describing the flow regime. The Reynolds number is given by

$$\text{Re} = \frac{\rho V D}{\mu}$$

where  $\rho$  is the fluid density,  $V$  is the average fluid velocity in the pipe, and  $\mu$  is the fluid viscosity. The velocity is given by the volumetric flow rate,  $Q$ , divided by the inside cross-sectional area of the pipe,  $A$ , or

$$V = \frac{Q}{A} \quad \text{and} \quad A = \frac{\pi D^2}{4}$$

The *Colebrook equation* is commonly used to predict the *Fanning friction factor*. It takes the form

$$\frac{1}{\sqrt{f}} = -2 \log_{10} \left[ \frac{\epsilon}{3.7D} + \frac{2.51}{\text{Re} \sqrt{f}} \right]$$

You will notice that this equation is implicit in  $f$ . For the formula to be valid, the flow must be turbulent. This can be checked by  $\text{Re} > 2500$ . Values of  $f$  are typically between 0.001 and 0.1.

Use bisection to solve for the Fanning friction factor for the following scenario:

- 4-inch Schedule 40 steel pipe with inside diameter,  $D = 0.1024 \text{ m}$ ,  $\epsilon = 45 \mu\text{m} = 4.5 \times 10^{-5} \text{ m}$
- fluid: ethylene glycol at  $80^\circ\text{F}$ ,  $\mu = 15.5 \text{ centipoise} = 0.015 \text{ Pa}\cdot\text{s}$ ,  $\rho = 1,130 \text{ kg/m}^3$
- flow rate:  $250 \text{ gpm} = 0.01578 \text{ m}^3/\text{s}$

Be sure to check that your units are consistent in the formulas above.

**5.21** Mechanical engineers, as well as most other engineers, use thermodynamics extensively in their work. The following polynomial can be used to relate the zero-gauge-pressure specific heat of dry air,  $c_p$  [kJ/(kg·K)], to absolute temperature,  $T$  [K]:

$$c_p = 0.99403 + 1.671 \times 10^{-4} T + 9.7215 \times 10^{-8} T^2 \\ - 9.5838 \times 10^{-11} T^3 + 1.92520 \times 10^{-14} T^4$$

- (a) Develop plot of  $c_p$  versus  $T$  from 0 to 1200 K.
- (b) Use bisection to determine the temperature that corresponds to a specific heat of 1.1 kJ/(kg·K).

**5.22** The upward velocity of a rocket can be computed with the following formula:

$$v = u \ln\left(\frac{m_0}{m_0 - qt}\right) - gt$$

where  $v$  is the upward velocity,  $u$  = the downward exit velocity of fuel relative to the rocket,  $m_0$  = mass of the rocket at  $t = 0$ ,  $q$  = the fuel consumption rate,  $g$  = downward gravitational acceleration, 9.81 m/s<sup>2</sup>. If  $u = 1800$  m/s,  $m_0 = 160,000$  kg,  $q = 2,600$  kg/s, compute the time at which  $v = 750$  m/s. Hint:  $t$  is somewhere between 10 and 50 s. Determine your result so that it is within 0.01% of the true value.

**5.23** Although we did not mention it in [Sec. 5.6](#), [Eq. \(5.13\)](#) is an expression of *electroneutrality*, that is, positive and negative charges are in balance. This can be seen more clearly by rearranging it as

$$[\text{H}^+] = [\text{HCO}_3^-] + 2[\text{CO}_3^{2-}] + [\text{OH}^-]$$

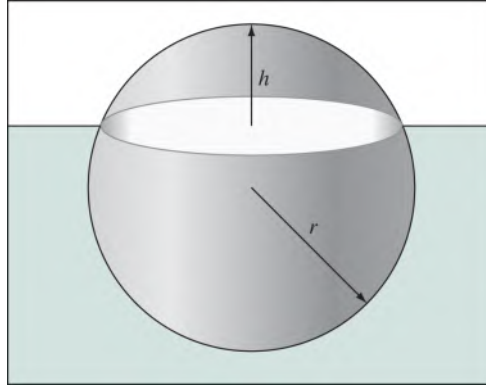
In other words, the positive charges must equal the negative charges. Thus, when you compute a pH of a natural water body, such as a lake, you must also account for other ions that may be present. For the case where these ions originate from nonreactive salts, the net negative minus positive charges due to these ions are lumped together in a quantity called *alkalinity*, and the equation is modified to

$$\text{Alk} + [\text{H}^+] = [\text{HCO}_3^-] + 2[\text{CO}_3^{2-}] + [\text{OH}^-] \quad (\text{P5.23})$$

where  $\text{Alk}$  = alkalinity (eq/L). For example, the average alkalinity of Lake Superior, measured in 2008, was  $0.4 \times 10^{-3}$  eq/L. Perform the same calculations as in [Sec. 5.6](#) to compute the pH of Lake Superior in 2008. Assume that, just like the raindrops, the lake is in equilibrium with atmospheric CO<sub>2</sub> but account for the alkalinity as in [Eq. P5.23](#).

**5.24** According to *Archimedes' principle*, the buoyancy force on a submerged or partially submerged object is equal to the weight of the fluid displaced by the submerged portion of the object. For the sphere depicted in [Fig. P5.24](#), use bisection to determine the height,  $h$ , of the portion that is above water.

**FIGURE P5.24** Partially submerged solid sphere.



The volume of the above-water portion of the sphere can be computed with

$$V = \frac{\pi h^2}{3} (3r - h)$$

Employ the following values for your computation:

$r = 1$  m,  $\rho_s =$  density of the sphere  $= 200$  kg/m<sup>3</sup>,  $\rho_w =$  density of water  $= 1000$  kg/m<sup>3</sup>.

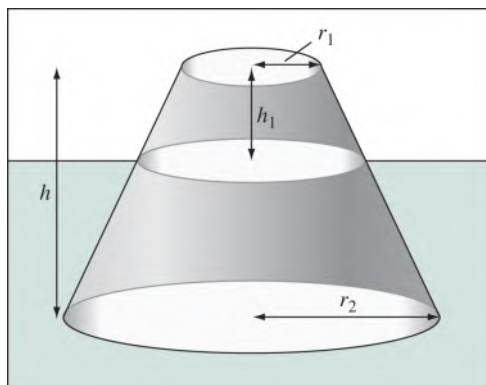
**5.25** Perform the same computations as in [Prob. 5.24](#), but for the frustum of a right circular cone, as depicted in [Fig. P5.25](#). The volume of the entire cone frustum illustrated in the figure is given by

$$V = \frac{\pi h}{3} (r_1^2 + r_2^2 + r_1 r_2)$$

Employ the following values for your computation:

$r_1 = 0.5$  m,  $r_2 = 1$  m,  $h = 1$  m,  $\rho_f = 300$  kg/m<sup>3</sup>,  $\rho_w = 1000$  kg/m<sup>3</sup>.

**FIGURE P5.25** Partially submerged object, frustum of a right, circular cone.



## 6

# Roots: Open Methods

## CHAPTER OBJECTIVES

The primary objective of this chapter is to introduce open methods for finding a root of a single nonlinear algebraic equation. Specific objectives and topics covered are

- Recognizing the difference between bracketing and open methods for root finding.
- Understanding the fixed-point iteration method and how you evaluate its convergence characteristics.
- Implementing the Wegstein method as an extension of fixed-point iteration to enhance convergence and provide stability.
- Knowing how to solve a roots problem with the Newton-Raphson method and appreciating the concept of quadratic convergence.
- Knowing how to implement the secant and modified secant methods.
- Understanding how Brent's method combines reliable bracketing methods with fast open methods to locate roots in a robust and efficient manner.
- Learning how to use the built-in methods in Python's SciPy module to estimate roots.
- Knowing how to use Python's built-in capabilities to manipulate and determine the roots of polynomials.

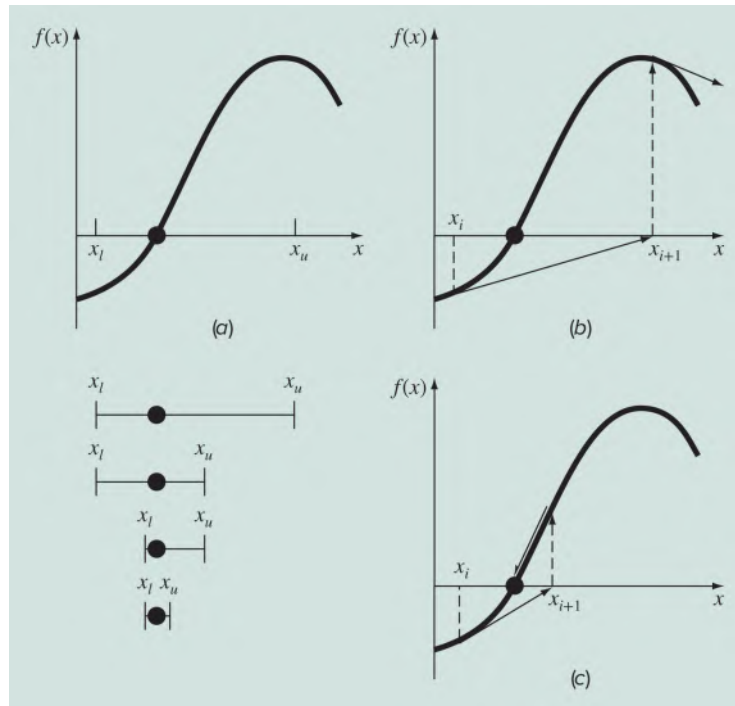
For the bracketing methods in [Chap. 5](#), the root is located within an interval prescribed by a lower and upper bound. Repeated application of these methods always results in closer estimates of the true value of the root. Such methods are said to be *convergent* because they move closer to the truth as the computation progresses ([Fig. 6.1a](#)).

In contrast, the open methods described in this chapter require only one or two starting values that do not necessarily bracket the root. As such, they sometimes diverge or move away from the root as the computation progresses ([Fig. 6.1b](#)). However, when the methods do converge ([Fig. 6.1c](#)), they usually do so much more quickly than the bracketing methods. We will begin our discussion of open techniques with a simple approach that is useful for illustrating their general form and also to demonstrate the concept of convergence.

**FIGURE 6.1** Graphical depiction of the fundamental difference between the bracketing (a) and open methods (b) and (c) for root location. In (a), which is bisection, the root is constrained within the interval prescribed by  $x_l$  and  $x_u$ . In contrast, for the open method depicted in (b) and (c), which is the Newton-



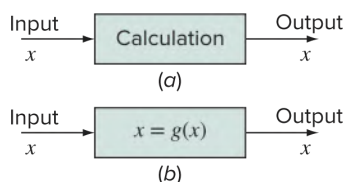
Raphson method, a formula is used to project from  $x_i$  to  $x_{i+1}$  in an iterative fashion. Thus, the method can either diverge, as in (b), or converge rapidly, as in (c), depending on the shape of the function and the value of the initial guess.



## 6.1 FIXED-POINT ITERATION

In carrying out engineering and scientific calculations, it occurs frequently and naturally that, to obtain a result, one must start with an estimate of that result. This is depicted as a general scenario in [Fig. 6.2a](#). When the calculation is represented by a single formula, [Fig. 6.2b](#) is the appropriate illustration.

**FIGURE 6.2** (a) General scenario where an input value of a quantity is required to carry out a calculation to determine an output value of the quantity. (b) Specific scenario when the calculation is a single formula.

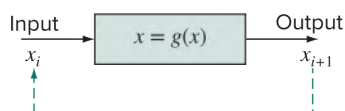


For the calculations in [Fig. 6.2](#) to be valid, the input  $x$  must be the same value as the output  $x$ . That presents a classical dilemma of having to know the answer before one solves for the answer, and raises the question, “What to do?” One approach, based on [Fig. 6.2b](#), is to reformulate the calculation to

$$f(x) = x - g(x)$$

and solve the problem using methods such as bisection from [Chap. 5](#) or methods introduced later in this chapter. Another scheme, illustrated in [Fig. 6.3](#), is to set up a circular calculation, also called *fixed-point iteration* or *successive substitution*. Here, we start with an estimate  $x_i$  and carry out the calculation to determine a result  $x_{i+1}$ . Typically,  $x_{i+1} \neq x_i$ , implying that the calculation or equation in the box is not satisfied.

**FIGURE 6.3** Circular calculation, AKA fixed-point iteration or successive substitution.



The fixed-point iteration method suggests that this scheme is repeated, based on

$$x_{i+1} = g(x_i) \quad (6.1)$$

until  $x_{i+1} \cong x_i$ . It should be noted that a roots problem that is naturally formulated as  $f(x) = 0$  can be reformulated by algebraic manipulation to solve symbolically for an  $x$  in the equation to obtain the  $x = g(x)$  format. As an example,  $f(x) = x - e^{-x}$  could be restated as

$$x = e^{-x} \quad \text{or} \quad x = -\ln(x),$$

both in the  $x = g(x)$  format. We have made an implicit assumption above that the iterative schemes will converge. It turns out that is not always the case and is a major limitation of fixed-point iteration. The scheme will often diverge. We will explore this in the example below.

As with many other iterative formulas in this book, the absolute relative error can be determined using

$$e_a = \left| \frac{x_{i+1} - x_i}{x_{i+1}} \right| 100\% \quad (6.2)$$

## EXAMPLE 6.1

### Simple Fixed-Point Iteration

#### Problem Statement

Use simple fixed-point iteration to estimate the root of  $f(x) = x - e^{-x}$ .

#### Solution

We choose to attempt the solution using the formulation  $x = g(x) = e^{-x}$  starting with an initial guess,  $x_0 = 0$ . The calculations are shown in the table below.

| Iteration | $x_i$  | $g(x_i)$ | $\epsilon_a$ | $\epsilon_t$ | $\epsilon_{t,i}/\epsilon_{t,i-1}$ |
|-----------|--------|----------|--------------|--------------|-----------------------------------|
| 0         | 0      | 1        |              | 100.00%      |                                   |
| 1         | 1      | 0.3679   | 171.83%      | 76.32%       | 0.7632                            |
| 2         | 0.3679 | 0.6922   | 46.85%       | 35.13%       | 0.4603                            |
| 3         | 0.6922 | 0.5005   | 38.31%       | 22.05%       | 0.6276                            |
| 4         | 0.5005 | 0.6062   | 17.45%       | 11.76%       | 0.5331                            |
| 5         | 0.6062 | 0.5454   | 11.16%       | 6.89%        | 0.5865                            |
| 6         | 0.5454 | 0.5796   | 5.90%        | 3.83%        | 0.5562                            |
| 7         | 0.5796 | 0.5601   | 3.48%        | 2.20%        | 0.5734                            |
| 8         | 0.5601 | 0.5711   | 1.93%        | 1.24%        | 0.5636                            |
| 9         | 0.5711 | 0.5649   | 1.11%        | 0.71%        | 0.5692                            |
| 10        | 0.5649 | 0.5684   | 0.62%        | 0.40%        | 0.5660                            |

The true value of the root is 0.567143140453502 (to 15 significant figures), and this is used in the table to compute the absolute error,  $\varepsilon_t$ , and the ratio in the last column. Notice that the latter values in the last column are all close to 0.57. This shows that the error is proportional to that in the previous iteration, in this case, about 60%. This property, called *linear convergence*, is a characteristic of fixed-point iteration.

As a further illustration, we now choose the alternate formulation  $x = g(x) = -\ln(x)$  starting with an initial guess of  $x_0 = 0.5$ , fairly close to the true solution. See the table below.

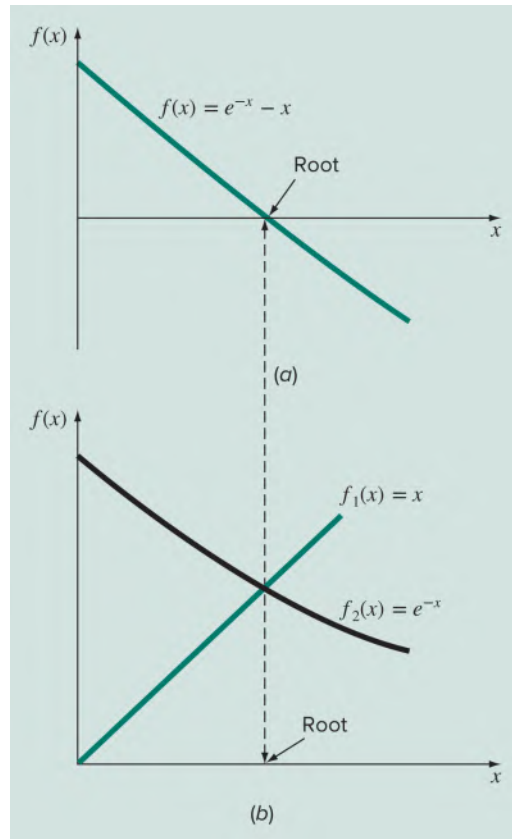
| Iteration | $x_i$  | $g(x_i)$ | $\varepsilon_a$ | $\varepsilon_t$ | $\varepsilon_{t,i}/\varepsilon_{t,i-1}$ |
|-----------|--------|----------|-----------------|-----------------|-----------------------------------------|
| 0         | 0.5    | 0.6931   |                 | 11.84%          |                                         |
| 1         | 0.6931 | 0.3665   | 27.87%          | 22.22%          | 1.8766                                  |
| 2         | 0.3665 | 1.0037   | 89.12%          | 35.38%          | 1.5923                                  |
| 3         | 1.0037 | -0.0037  | 63.48%          | 76.98%          | 2.1760                                  |

It is evident that our fixed-point iteration scheme here diverges.

For  **Example 6.1**, the contrast of the  $f(x) = 0$  and  $x = g(x)$  formulations is depicted in  **Fig. 6.4**. In the (b) subfigure, the  $x = g(x)$  formulation has been decomposed into

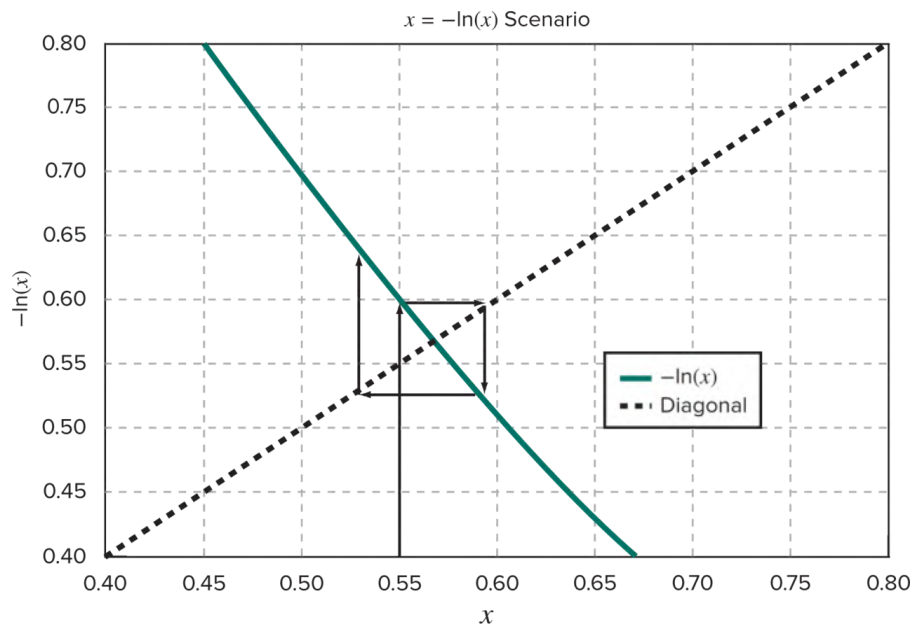
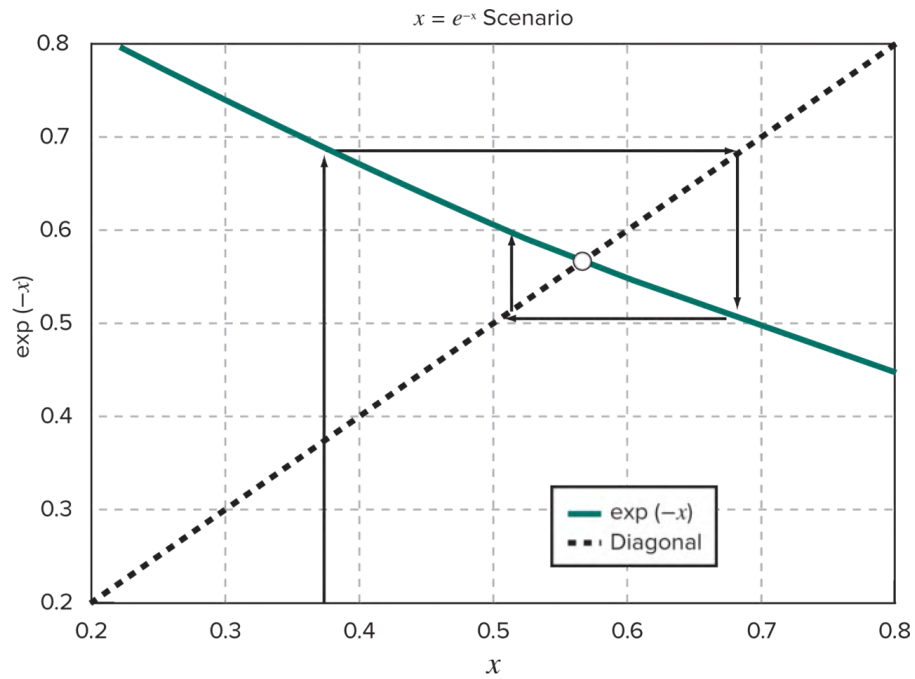
$$f_1(x) = x \quad \text{and} \quad f_2(x) = e^{-x} \quad (6.3)$$

**FIGURE 6.4** Graphical depictions of the  $f(x) = 0$  root-finding scenario (a) and the  $x = g(x)$  fixed-point iteration scenario (b). In (a), the root is located where the  $f(x)$  function curve crosses the  $x$  axis. In (b), it is located where the  $g(x)$  function curve crosses the  $45^\circ$  line.

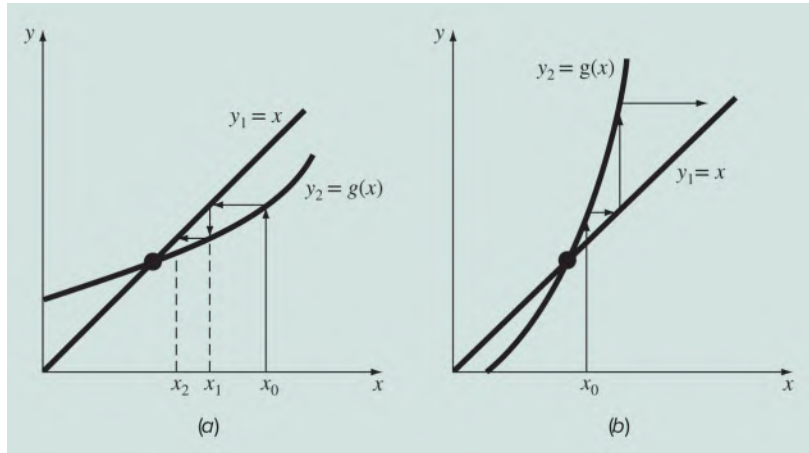


In general, we know that the solution will lie at the intersection of the  $g(x)$  versus  $x$  curve and the  $45^\circ$  line, representing the requirement that  $g(x)$  is equal to  $x$  at the solution. This relationship can be used to explain the difference between a convergent and divergent scheme, as revealed in [Example 6.1](#). In [Fig. 6.5](#), we illustrate this. You can see clearly how the  $x = g(x) = e^{-x}$  is converging on the root and the  $x = g(x) = -\ln(x)$  formulation is diverging. For the example, the two patterns are spiraling. It is also possible to have monotonic convergence or divergence, and this is illustrated in [Fig. 6.6](#).

**FIGURE 6.5** "Cobweb" plots depicting the convergent and divergent scenarios from [Example 6.1](#).



**FIGURE 6.6** Monotonic convergence and divergence for fixed-point iteration scenarios.



A theoretical derivation can be used to gain insight into the fixed-point iteration process. As described in Chapra and Canale (2021), it can be shown that the error for any iteration is linearly proportional to the error from the previous iteration multiplied by the absolute value of the slope of  $g(x)$ , that is,

$$E_{i+1} = E_i |g'(\xi)| \quad (6.4)$$

where  $x = \xi$  is in the vicinity of the root.

Consequently, if  $|g'| < 1$ , the absolute errors will decrease with each iteration. For  $|g'| > 1$ , those errors will grow. Additionally, if  $g' > 0$ , the errors will be positive, and there will be monotonic convergence or divergence, as illustrated in [Fig. 6.6](#). Whereas  $g' < 0$  for [Example 6.1](#), and the errors will change sign each iteration. This will cause the “spiraling in” scenario illustrated in [Fig. 6.5](#).

There are two questions that arise after analyzing, illustrating, and understanding simple fixed-point iteration:

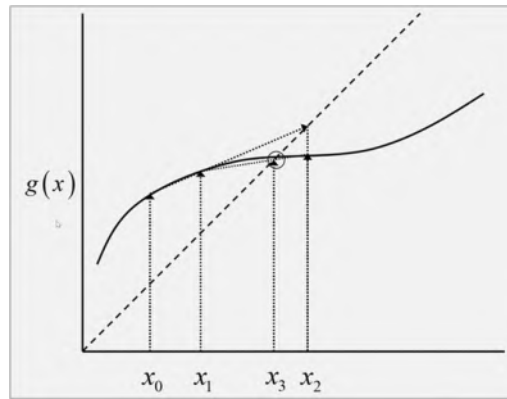
- As with the development of the false-position method as a possible enhancement of the bisection method in [Chap. 5](#), is there a similar possibility for improvement of fixed-point iteration?
- Is there a modification of the fixed-point iteration method that might provide a convergent scheme for a scenario where fixed-point iteration is divergent?

We address these questions in the next section.

## 6.2 THE WEGSTEIN METHOD

In the false position method, we used a linear relationship between two guesses to determine the next estimate as the location where the straight line crossed the  $x$  axis, as we know that the solution also lies on that axis. For the  $x = g(x)$  scenario studied in fixed-point iteration, we know that the solution lies on the  $45^\circ$  line of the plot of  $g(x)$  versus  $x$ , not the  $x$  axis. This raises the possibility of using a straight line through two initial guesses to its intersection with that  $45^\circ$  line to determine the next estimate of the solution. This is the basis for the *Wegstein method*<sup>1</sup> illustrated in  Fig. 6.7.

**FIGURE 6.7** Graphical depiction of the Wegstein method.



We interpret the figure as follows. Two initial guesses,  $x_0$  and  $x_1$ , are required, but, unlike bisection, they do not have to bracket the root. A straight line through the points  $[x_0, g(x_0)]$  and  $[x_1, g(x_1)]$  is projected to its intersection with the  $45^\circ$  line. This locates  $x_2$ . This process is repeated, but now with the points  $[x_1, g(x_1)]$  and  $[x_2, g(x_2)]$  used to locate  $x_3$ . At each iteration, the last two root estimates,  $x_i$  and  $x_{i-1}$ , are used to locate the next estimate,  $x_{i+1}$ . The figure shows a rapid convergence to the root.

How do we express the Wegstein method as an algorithm? We start by writing a general form for the straight line between  $[x_i, g(x_i)]$  and  $[x_{i-1}, g(x_{i-1})]$ :

$$\frac{x - x_i}{g(x) - g(x_i)} = \frac{x - x_{i-1}}{g(x) - g(x_{i-1})}$$

However, we realize that for the projection to the  $45^\circ$  line,  $x_{i+1} = g(x_{i+1})$ . Consequently,

$$\frac{x_{i+1} - x_i}{g(x_{i+1}) - g(x_i)} = \frac{x - x_{i-1}}{g(x_i) - g(x_{i-1})}$$

and solving algebraically for  $x_{i+1}$ , we obtain the Wegstein iterative formula:



$$x_{i+1} = \frac{x_i g(x_{i-1}) - x_{i-1} g(x_i)}{x_i - x_{i-1} - g(x_i) + g(x_{i-1})} \quad (6.5)$$

Given initial guesses, we can apply [Eq. \(6.5\)](#) iteratively to obtain a new estimate. As with fixed-point iteration, we can estimate the relative error with [Eq. \(6.3\)](#).

## EXAMPLE 6.2

### Applying the Wegstein Method

#### Problem Statement

Apply the Wegstein method to the two  $x = g(x)$  functional forms from [Example 6.1](#): (a)  $x = e^{-x}$  and (b)  $x = -\ln(x)$ .

#### Solution

(a) Given initial guesses,  $x_0 = 0$  and  $x_1 = 0.25$ , we can compute  $g(x_0) = 1$  and  $g(x_1) = 0.7780$ . [Equation \(6.5\)](#) can then be employed to calculate,

$$x_2 = \frac{0.25(1) - 0(0.7780)}{0.25 - 0 - 0.7780 + 1} = 0.53056$$

with a percent relative error of

$$\varepsilon_a = \left| \frac{0.53056 - 0.25}{0.53056} \right| = 52.88\%$$

The method can be continued and summarized in the following table:

| $i$ | $x_{i-1}$ | $x_i$   | $x_{i+1}$ | $g(x_{i-1})$ | $g(x_i)$ | $g(x_{i+1})$ | $\varepsilon_a$ | $\varepsilon_t$ | $\varepsilon_{t,i}/\varepsilon_{t,i-1}$ |
|-----|-----------|---------|-----------|--------------|----------|--------------|-----------------|-----------------|-----------------------------------------|
| 1   | 0         | 0.25    | 0.53056   | 1.00000      | 0.77880  | 0.58827      | 52.88%          | 6.45%           |                                         |
| 2   | 0.25      | 0.53056 | 0.56493   | 0.77880      | 0.58827  | 0.56840      | 6.08%           | 0.39%           | 6.04%                                   |
| 3   | 0.53056   | 0.56493 | 0.56713   | 0.58827      | 0.56840  | 0.56715      | 0.39%           | 0.00%           | 0.66%                                   |
| 4   | 0.56493   | 0.56713 | 0.56714   | 0.56840      | 0.56715  | 0.56714      | 0.00%           | 0.00%           | 0.99%                                   |

Compared to the table for fixed-point iteration in [Example 6.1](#), the convergence here is much more rapid. This is confirmed by the relative reduction of absolute true error from one iteration to

the next reaching a value below 1%.

(b) Next, we apply the Wegstein formula to  $x = -\ln(x)$ .

| $i$ | $x_{i-1}$ | $x_i$   | $x_{i+1}$ | $g(x_{i-1})$ | $g(x_i)$ | $g(x_{i+1})$ | $\epsilon_a$ | $\epsilon_t$ | $\epsilon_{t,i}/\epsilon_{t,i-1}$ |
|-----|-----------|---------|-----------|--------------|----------|--------------|--------------|--------------|-----------------------------------|
| 1   | 0.45      | 0.5     | 0.56216   | 0.79851      | 0.69315  | 0.57597      | 11.06%       | 0.88%        |                                   |
| 2   | 0.5       | 0.56216 | 0.56695   | 0.69315      | 0.57597  | 0.56749      | 0.84%        | 0.03%        | 3.95%                             |
| 3   | 0.56216   | 0.56695 | 0.56714   | 0.57597      | 0.56749  | 0.56714      | 0.03%        | 0.00%        | 0.27%                             |

Recall that fixed-point iteration is divergent for this calculation. The Wegstein method “forces” the naturally unstable circular calculation to converge.

For the reasons confirmed by [Example 6.2](#), the Wegstein method is preferred and commonly implemented in commercial software where circular calculations are encountered frequently.

## EXAMPLE 6.3

### Implementing Fixed-Point Iteration and the Wegstein Method in Python

#### Problem Statement

Develop Python functions for fixed-point iteration and the Wegstein method and test these with the functions from [Examples 6.1](#) and [6.2](#).

#### Solution

The Python function for the fixed-position method is shown below.

```
def fixpt(g,x0,Ea=1.e-7,maxit=30):
    """
    This function solves x=g(x) using fixed-point iteration.
    The method is repeated until either the relative error
    falls below Ea (default 1.e-7) or reaches maxit (default 30).
    Input:
        g = name of the function for g(x)
        x0 = initial guess for x
        Ea = relative error threshold
        maxit = maximum number of iterations
    Output:
        x1 = solution estimate
        ea = relative error
        i+1 = number of iterations
    """
    for i in range(maxit):
        x1 = g(x0)
        ea = abs((x1-x0)/x1)
        if ea < Ea: break
        x0 = x1
    return x1,ea,i+1
```

The code is very simple, only seven lines. To use the `fixpt` function to solve

 **Example 6.1**, the following lines are added:

```
import numpy as np
def g(x):
    return np.exp(-x)
x0 = 0
(xsoln,ea,n) = fixpt(g,x0,Ea=1.e-5)
print('Solution = {0:8.5g}'.format(xsoln))
print('Relative error = {0:8.3e}'.format(ea))
print('Number of iterations = {0:5d}'.format(n))
```

and the results displayed on the Console are

```
Solution = 0.56714
Relative error = 6.933e-06
Number of iterations = 23
```

For the Wegstein method, the `fixpt` code is modified to include two initial guesses and the Wegstein formula. Here is the function:

```
def wegstein(g,x0,x1,Ea=1.e-7,maxit=30):
    """
    This function solves x=g(x) using the Wegstein method.
    The method is repeated until either the relative error
    falls below Ea (default 1.e-7) or reaches maxit (default 30).
    Input:
        g = name of the function for g(x)
        x0 = first initial guess for x
        x1 = second initial guess for x
        Ea = relative error threshold
        maxit = maximum number of iterations
    Output:
        x2 = solution estimate
        ea = relative error
        i+1 = number of iterations
    """
    for i in range(maxit):
        x2 = (x1*g(x0)-x0*g(x1))/(x1-x0-g(x1)+g(x0))
        ea = abs((x1-x0)/x1)
        if ea < Ea: break
        x0 = x1
        x1 = x2
    return x2,ea,i+1
```

To test the function for the solution of  $x = e^{-x}$ ,

```
x0 = 0.4
x1 = 0.45
(xsoln,ea,n) = wegstein(g,x0,x1,Ea=1.e-5)
print('Solution = {0:8.5g}'.format(xsoln))
print('Relative error = {0:8.3e}'.format(ea))
print('Number of iterations = {0:5d}'.format(n))
```

with the results

```
Solution = 0.56714
Relative error = 1.038e-08
Number of iterations = 6
```

Note that the Wegstein method achieves the error specification with only six iterations compared to the 23 of the fixed-point method.

The `wegstein` function can also be used to solve the naturally divergent equation,  $x = -\ln(x)$  by modifying the test code to

```
import numpy as np

def g(x):
    return -np.log(x)

x0 = 0.4
x1 = 0.45
(xsoln,ea,n) = wegstein(g,x0,x1,Ea=1.e-5)
print('Solution = {0:8.5g}'.format(xsoln))
print('Relative error = {0:8.3e}'.format(ea))
print('Number of iterations = {0:5d}'.format(n))
```

and produce a successful solution, also in six iterations:

```
Solution = 0.56714
Relative error = 6.805e-09
Number of iterations = 6
```

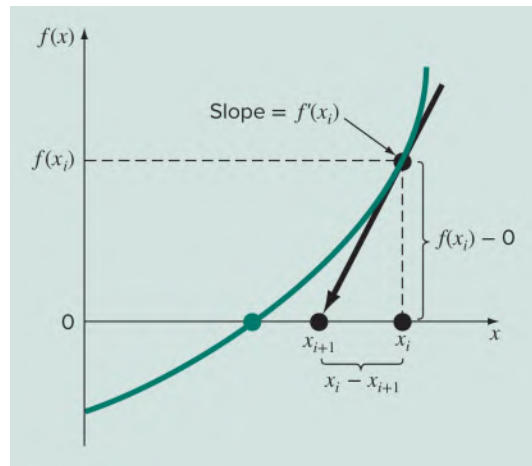
Before we proceed, there are three comments worth mentioning:

- As stated before, many engineering and scientific problems arise naturally in the form  $x = g(x)$  and are often convergent for the fixed-point iteration method. In such cases, it may be attractive to find solutions using the circular methods. The Wegstein method has been shown to accelerate convergence and stabilize a naturally divergent calculation.
- With reference to [Fig. 6.2a](#), circular calculation can involve much more than a single formula, reaching 10s to 100s of lines of code. The same principles and methods introduced in this section apply in such situations; however, the system cannot be easily transformed into the  $f(x) = x - g(x) = 0$  formulation, and will yield a system of nonlinear equations where a numerical solution may be difficult. We will return to such nonlinear systems later in [Chap. 12](#).
- The solution of  $x = g(x)$  or  $f(x) = 0$  equations is often embedded into other iterative calculations and may be required to be performed thousands of times during the solution of the bigger problem. In this case, the importance of efficiency and rapid convergence is amplified. Also, as an advantage, initial guesses are often taken from the last iteration when the equation(s) was solved and is very close to the next solution, again enhancing convergence.

## 6.3 NEWTON-RAPHSON

Perhaps, the most widely used of all root-finding formulas is the *Newton-Raphson method* (🔗 Fig. 6.8). The initial guess or current estimate for the root,  $x_i$ , is extended from the point  $[x_i, f(x_i)]$  via a straight line, which is the tangent at the point, to the  $x$  axis. The point there,  $x_{i+1}$ , usually represents an improved estimate of the root of  $f(x) = 0$ .

**FIGURE 6.8** Graphical depiction of the Newton-Raphson method. A tangent to the function of  $x$ , with slope  $f'(x)$ , is extended from the point  $[x_i, f(x_i)]$  to the  $x$  axis to provide an estimate of the root,  $x_{i+1}$ .



The Newton-Raphson formula can be derived directly from 🔗 Fig. 6.8, where the derivative,  $f'(x_i)$  is given by

$$f'(x_i) = \frac{f(x_i) - 0}{x_i - x_{i+1}}$$

which is rearranged to yield

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \quad (6.6)$$

🔗 Equation (6.6) is the *Newton-Raphson formula*.

### EXAMPLE 6.4

## Newton-Raphson Method

### Problem Statement

Use the Newton-Raphson method to estimate the root of  $f(x) = x - e^{-x}$  employing an initial guess,  $x_0 = 0$ .

### Solution

The first derivative of the function can be evaluated as

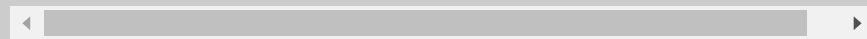
$$f'(x) = 1 + e^{-x}$$

which can be substituted into [Eq. \(6.6\)](#) along with the original function to give

$$x_{i+1} = x_i - \frac{x_i - e^{-x_i}}{1 + e^{-x_i}}$$

Starting with the initial guess of  $x_0 = 0$ , this equation can be applied iteratively to produce the results in the table below.

| Iteration | $x_i$     | $f(x_i)$ | $f'(x_i)$ | $x_{i+1}$ | $\epsilon_t$ |
|-----------|-----------|----------|-----------|-----------|--------------|
| 0         | 0         | -1.00000 | 2.000000  | 0.5       | 11.8%        |
| 1         | 0.500     | -0.10653 | 1.606531  | 0.566311  | 0.147%       |
| 2         | 0.56631   | -0.00130 | 1.567616  | 0.5671432 | 0.000022%    |
| 3         | 0.5671432 | 0.00000  | 1.567143  | 0.5671433 | 0.00000001%  |



Observe how rapidly the scheme converges to the root. Notice that the true absolute error decreases much faster than with fixed-point iteration, and even somewhat faster than the Wegstein method (compare with [Examples 6.1](#) and [6.2](#)).

As with the other root-location methods, [Eq. \(6.2\)](#) can be used as a termination criterion. In addition, a theoretical analysis (Chapra and Canale, 2021) provides insight regarding the rate of convergence expressed by

$$E_{t,i+1} = \frac{-f''(x_r)}{2f'(x_r)} E_{t,i}^2 \quad (6.7)$$

Equation (6.7) shows that the error should be approximately proportional to the square of the previous error. In other words, the number of significant figures of accuracy approximately doubles with each iteration. This behavior is called *quadratic convergence* and is one of the major reasons for the popularity of the method.

Although the Newton-Raphson method is frequently very efficient, there are situations where it performs poorly. A special case—multiple roots—is discussed elsewhere (Chapra and Canale, 2021). However, even when dealing with simple roots, difficulties can also arise. This is illustrated in the following example.

## EXAMPLE 6.5

### A Slowly Converging Function with Newton-Raphson

#### Problem Statement

Determine the positive root of  $f(x) = x^{10} - 1$  using the Newton-Raphson method and an initial guess of  $x = 0.5$ .

#### Solution

The Newton-Raphson formula for this case is

$$x_{i+1} = x_i \frac{x_i^{10} - 1}{10 x_i^9}$$

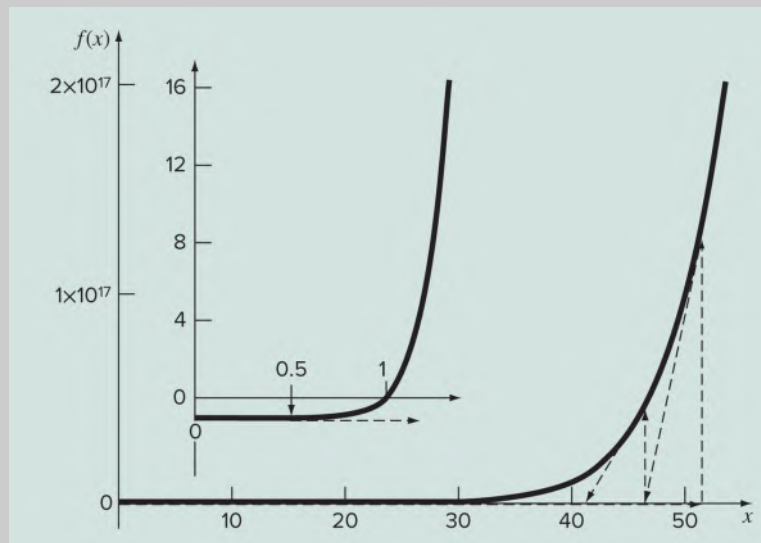
which can be used to develop the table below.

| Iteration | $x_i$   | $f(x_i)$  | $f'(x_i)$ | $x_{i+1}$   | $\epsilon_a$ |
|-----------|---------|-----------|-----------|-------------|--------------|
| 0         | 0.5     | -0.99902  | 0.019531  | 51.65       | 99.0%        |
| 1         | 51.7    | 1.351E+17 | 2.616E+16 | 46.49       | 11.111%      |
| 2         | 46.5    | 4.711E+16 | 1.013E+16 | 41.84       | 11.111%      |
| 3         | 41.8    | 1.643E+16 | 3.926E+15 | 37.65       | 11.111%      |
| 4         | 37.7    | 5.728E+15 | 1.521E+15 | 33.89       | 11.111%      |
|           |         |           | •         |             |              |
|           |         |           | •         |             |              |
|           |         |           | •         |             |              |
| 38        | 1.08    | 1.227E+00 | 2.056E+01 | 1.024       | 5.831%       |
| 39        | 1.024   | 2.635E-01 | 1.234E+01 | 1.0023      | 2.130%       |
| 40        | 1.002   | 2.340E-02 | 1.021E+01 | 1.00002     | 0.229%       |
| 41        | 1.00002 | 2.394E-04 | 1.000E+01 | 1.000000003 | 0.0024%      |

We see that, after the first poor estimate, the technique is converging very slowly to the root. Note the magnitude of the numerator,  $f(x_i)$ , and denominator,  $f'(x_i)$ , terms.

Why does this slow convergence happen? As shown in [Fig. 6.9](#), a plot of the first few iterations is helpful in providing insight. Notice how the first guess is in a region where the slope is near zero. Thus, by dividing by that slope, the first iteration “flings” the estimated solution far away from the initial guess to 51.65 where, as the table shows,  $f(x)$  has a very large value,  $\sim 10^{17}$ . This provides a correspondingly large value for  $f'(x)$ ,  $\sim 10^{16}$ . From there, the solution gradually plods along for over 40 iterations before converging to the root with adequate accuracy.

**FIGURE 6.9** Graphical depiction of the Newton-Raphson method for a case with slow convergence. The inset shows how a near-zero slope initially shoots the solution very slowly converges on the root.



[Example 6.3](#) is a somewhat rare, pathological example, but it illustrates the possibility of slow convergence and the sensitivity of convergence to the initial guess. With an initial guess of 0.9, the method converges within three iterations.

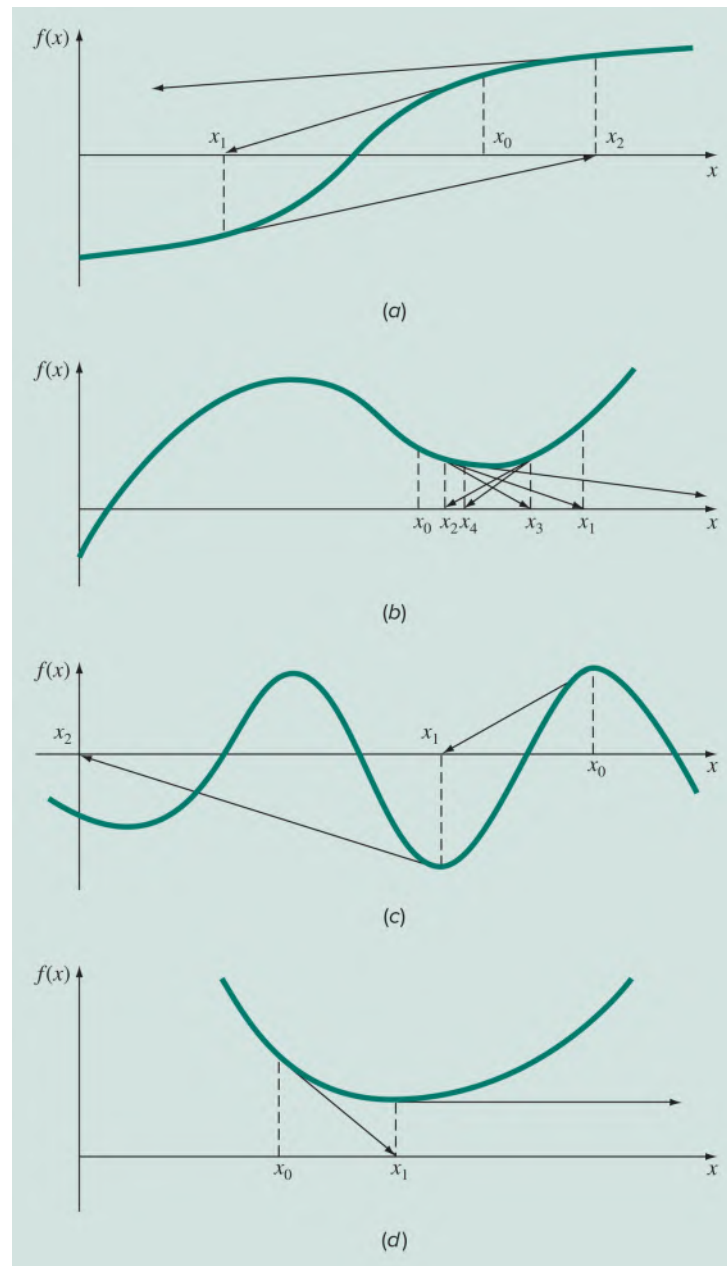
Page 152

Aside from slow convergence due to the nature of the function, other difficulties can arise, as illustrated in [Fig. 6.10](#). For example, [Fig. 6.10a](#) depicts the case where an inflection point, that is,  $f''(x) = 0$ , occurs in the vicinity of a root. [Figure 6.10b](#) illustrates the tendency of the Newton-Raphson technique to oscillate around a local minimum (can also happen around a maximum). Such oscillations may persist, or, as in [Fig. 6.10b](#), a near-zero slope is reached whereupon the estimate is sent far away from the area of interest. [Figure 6.10c](#) shows how an initial guess that is close to one root can jump to a location several roots away. This tendency to move away from the area of interest is because near-zero slopes are encountered. Obviously, a zero slope,  $f'(x) = 0$ , is a real disaster because it causes division



by zero in the Newton-Raphson formula [Eq. (6.6)]. As shown in Fig. 6.10d, the solution shoots off horizontally and never encounters the  $x$  axis.

**FIGURE 6.10** Four cases where the Newton-Raphson method exhibits poor convergence.



There is no general criterion for convergence of the Newton-Raphson method. Its convergence depends on the nature of the function and on the accuracy of the initial guess. The main remedy is to have an initial guess that is in a neighborhood sufficiently close to the solution. And for some functions, no guess will work! Good guesses are usually predicated on knowledge of the physical problem setting or using devices

such as graphs that provide insight into the behavior of the function and its solution(s). As mentioned before, if the Newton-Raphson method is embedded in another calculation that calls it multiple times, the solution from the last time would probably be an excellent initial guess for the next time around. It also suggests that good computer software should be designed to recognize problems of slow convergence or divergence.

## 6.3.1 Python Function: newtraph

An algorithm for the Newton-Raphson method can be readily developed into a Python function (  **Fig. 6.11**). Note that the newtraph function must have access to both  $f$  and  $fp$  functions for  $f(x)$  and  $f'(x)$ , respectively. These can be separate function definitions (def) or lambda functions included directly in the arguments. For example, solving for the root of  $f(x) = x^2 - 9$  with initial guess  $x_0 = 5$ ,

```
def f(x):
    return x**2-9

def fp(x):
    return 2*x

x0 = 5
(xsoln,fxsoln,ea,n) = newtraph(f,fp,x0,Ea=1.e-5)
print('Solution = {0:8.5g}'.format(xsoln))
print('Function value at solution = {0:8.5e}'.format(fxsoln))
print('Relative error = {0:8.3e}'.format(ea))
print('Number of iterations = {0:5d}'.format(n))
```

yields the trivial solution

```
Solution =          3
Function value at solution = 0.00000e+00
Relative error = 4.657e-10
Number of iterations =          5
```

**FIGURE 6.11** A Python function to implement the Newton-Raphson method.

```
def newtraph(f,fp,x0,Ea=1.e-7,maxit=30):
    """
    This function solves f(x)=0 using the Newton-Raphson method.
    The method is repeated until either the relative error
    falls below Ea (default 1.e-7) or reaches maxit (default 30).
    Input:
        f = name of the function for f(x)
        fp = name of the function for f'(x)
        x0 = initial guess for x
        Ea = relative error threshold
        maxit = maximum number of iterations
    Output:
        x1 = solution estimate
        f(x1) = equation error at solution estimate
        ea = relative error
        i+1 = number of iterations
    """
    for i in range(maxit):
        x1 = x0 - f(x0)/fp(x0)
        ea = abs((x1-x0)/x1)
        if ea < Ea: break
        x0 = x1
    return x1,f(x1),ea,i+1
```

Since the  $f$  and  $fp$  functions in the Python code are simple, it would be convenient to use `lambda` functions instead directly in the call to the `newtraph` function as follows:

```
(xsoln,fxsoln,ea,n) = newtraph(lambda x: x**2-9,lambda x: 2*x,x0,Ea=1.e-5)
```

## EXAMPLE 6.6

### Newton-Raphson Method Applied to the Bungee Jumper Problem

#### Problem Statement

Use the Python `newtraph` function from [Fig. 6.11](#) to determine the mass of the bungee jumper with a drag coefficient of 0.25 kg/m that has a velocity of 36 m/s after 4 s of free fall.

#### Solution

The function to be evaluated is

$$f(m) = \sqrt{\frac{mg}{c_d}} \tanh\left(\sqrt{\frac{g c_d}{m}} t\right) - v \quad (\text{E6.6.1})$$

To apply the Newton-Raphson method, we must determine the derivative of  $f(m)$  with respect to  $m$ .

$$\frac{df(m)}{dm} = \frac{1}{2} \sqrt{\frac{g}{m c_d}} \tanh\left(\sqrt{\frac{g c_d}{m}} t\right) - \frac{g}{2m} t \operatorname{sech}^2\left(\sqrt{\frac{g c_d}{m}} t\right) \quad (\text{E6.6.2})$$

Given the complexity of these two formulas, it is preferred to code separate functions for them in Python. These would be

```
def f(m,cd,t,v):
    return np.sqrt(m*g/cd)*np.tanh(np.sqrt(g*cd/m)*t)-v

def fp(m,cd,t,v):
    fp1 = 1/2*np.sqrt(g/m/cd)*np.tanh(np.sqrt(g*cd/m)*t)
    fp2 = g/2/m*t*sech(np.sqrt(g*cd/m)*t)**2
    return fp1-fp2
```

Since the NumPy module does not include a `sech` function, we add

```
def sech(x): # numpy doesn't have sech(x)
    return 1/np.cosh(x)
```

and the code to invoke the `newtraph` function and display the results:

```
cd = 0.25 # kg/m
v = 36 # m/s
t = 4 # s
x0 = 140 # kg
```

```
(xsoln,fxsoln,ea,n) = newtraph(lambda m: f(m,cd,t,v),lambda m:
fp(m,cd,t,v),x0)
print('Solution = {0:8.5g}'.format(xsoln))
print('Function value at solution = {0:8.5e}'.format(fxsoln))
print('Relative error = {0:8.3e}'.format(ea))
print('Number of iterations = {0:5d}'.format(n))
```

The results are

```
Solution = 142.74
Function value at solution = -2.84217e-14
Relative error = 9.908e-08
Number of iterations = 3
```

It is important to note the rapid convergence, although the initial guess is a good one. For a guess farther away, for example,  $x_0 = 70$  kg, the solution is still achieved, but in six iterations. For an initial guess of 280 kg, the solution requires eight iterations. Apparently, the Newton-Raphson method is stable and efficient for a wide range of initial guesses.

The derivation of  $f'(x)$  in this example is not trivial, and it offers the possibility of making analytical errors. It is common for there to be functions where the analytical derivative is complicated or not practical to obtain. Yet, the Newton-Raphson method is still attractive as a solution technique. The next section offers an alternative approach.

## 6.4 SECANT METHODS

For functions whose derivatives may be difficult to evaluate, the derivative can be approximated by a backward difference:

$$f'(x_i) \cong \frac{f(x_i) - f(x_{i-1})}{x_i - x_{i-1}}$$

This approximation can be substituted into the Newton-Raphson formula [[Eq. \(6.6\)](#)] to yield the following iterative formula:

$$x_{i+1} = x_i - \frac{f(x_i)(x_i - x_{i-1})}{f(x_i) - f(x_{i-1})} \quad (6.8)$$

[Eq. \(6.8\)](#) is the formula for the *secant* method. Notice that the method requires two initial estimates of  $x$ . However, these estimates do not need to bracket the root, as in bisection; rather this situation is similar to the Wegstein method.

If two iteration values are not close, the backward difference approximation to the derivative may be poor. Rather than using two iteration values to estimate the derivative, an alternate approach is to use a small perturbation of  $x_i$  to estimate the derivative locally, as in

$$f'(x_i) \cong \frac{f(x_i + \delta x_i) - f(x_i)}{\delta x_i}$$

where  $\delta$  is a small perturbation fraction, for example,  $1 \times 10^{-6}$ . This approximation can be substituted into [Eq. \(6.6\)](#) to yield the following iterative equation:

$$x_{i+1} = x_i - \frac{f(x_i)(\delta x_i)}{f(x_i + \delta x_i) - f(x_i)} \quad (6.9)$$

This is called the *modified secant* method. As shown in the example below, this method provides a convenient means to attain the efficiency of the Newton-Raphson method without having to provide analytical derivatives.

### EXAMPLE 6.7

## Modified Secant Method

### Problem Statement

Use the modified secant method to determine the mass of the bungee jumper with a drag coefficient of 0.25 kg/m to have a velocity of 36 m/s after 4 s of free fall. Use an initial guess of 50 kg and a value of  $1 \times 10^{-6}$  for the perturbation fraction.

### Solution

Inserting parameters into [Eq. \(6.9\)](#) yields

First iteration:

$$\begin{aligned}x_0 &= 50 & f(x_0) &= -4.57938708 \\x_0 + \delta x_0 &= 50.00005 & f(x_0 + \delta x_0) &= -4.57938112 \\x_1 &= 50 - \frac{10^{-6} \times 50(-4.57938708)}{(-4.57938112) - (-4.57938708)} \cong 88.39931 \\ \varepsilon_t &= \left| \frac{142.7376 - 88.3993}{142.7376} \right| \cong 38\% & \varepsilon_a &= \left| \frac{88.3993 - 50}{88.3993} \right|\end{aligned}$$

Second iteration:

$$\begin{aligned}x_1 &= 88.399311 & f(x_1) &= -1.69220770 \\x_1 + \delta x_1 &\cong 88.399396 & f(x_1 + \delta x_1) &\cong -1.69220351 \\x_2 &= 88.3993 - \frac{10^{-6} \times 88.3993(-1.69220770)}{(-1.69220351) - (-1.69220770)} \cong 124.09 \\ \varepsilon_t &= 13\% & \varepsilon_a &= 29\%\end{aligned}$$

The calculations can proceed to produce the table below:

$$\delta = 1 \times 10^{-6}$$

| Iteration | $x_i$      | $f(x_i)$    | $f((1 + \delta)x_i)$ | $x_{i+1}$  | $\varepsilon_t$ | $\varepsilon_a$ |
|-----------|------------|-------------|----------------------|------------|-----------------|-----------------|
| 1         | 50         | -4.57938708 | -4.57938112          | 88.399308  | 38.1%           | 43.4%           |
| 2         | 88.399308  | -1.69220770 | -1.69220351          | 124.089701 | 13.1%           | 28.8%           |
| 3         | 124.089701 | -0.43236988 | -0.43236662          | 140.541723 | 1.54%           | 11.7%           |
| 4         | 140.541723 | -0.04555048 | -0.04554753          | 142.707186 | 0.021%          | 1.52%           |
| 5         | 142.707186 | -0.00062293 | -0.00062001          | 142.737627 | 0.000004%       | 0.02%           |

| $\delta = 1 \times 10^{-6}$ |            |             |                      |            |              |              |
|-----------------------------|------------|-------------|----------------------|------------|--------------|--------------|
| Iteration                   | $x_i$      | $f(x_i)$    | $f((1 + \delta)x_i)$ | $x_{i+1}$  | $\epsilon_t$ | $\epsilon_a$ |
| 6                           | 142.737627 | -1.1918E-07 | 2.80062E-06          | 142.737633 | 0.0000001%   | 0.000004%    |

The choice of an appropriate value for  $\delta$  is not automatic. If  $\delta$  is too small, the method can fail because of roundoff error caused by the subtraction of like quantities in the denominator of [Eq. \(6.9\)](#). If it is too large, the method can become inefficient and even divergent because the estimate of  $f'(x)$  is so poor. However, if chosen correctly, it provides a convenient alternative for cases where evaluating the derivative is difficult and developing two initial guesses is inconvenient.

Further, in its most general sense, a univariate function is merely an entity that returns a single value for values sent to it. Perceived in this way, functions are not always simple formulas like the one-line equations solved in the preceding examples in this chapter. For example, a function might consist of many lines of code that could take a significant amount of time to evaluate. In some cases, the function might even represent a separate software program. In these situations, the secant and modified secant methods can prove to be valuable.

## 6.5 BRENT'S METHOD

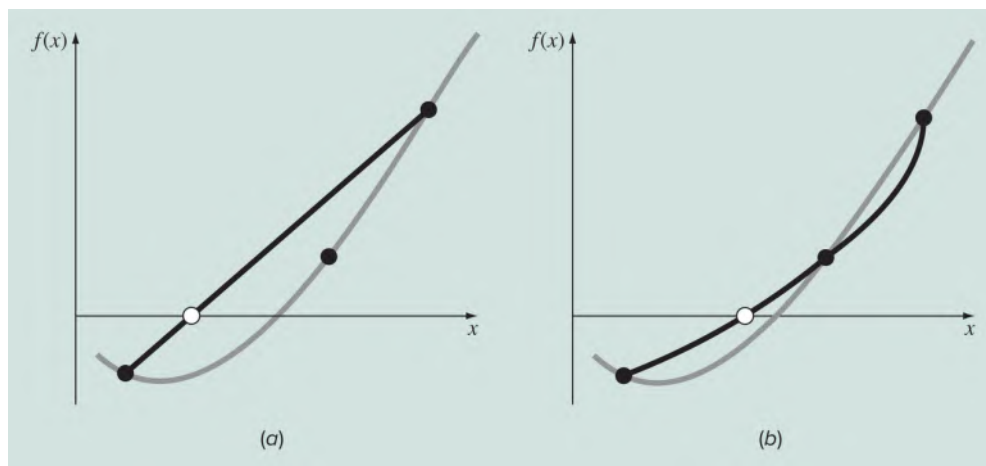
Wouldn't it be nice to have a hybrid approach that combined the reliability of bracketing methods with the speed of open methods? Brent's root-location method is a clever algorithm that does just that by applying a speedy open method wherever possible but reverting to a reliable bracketing method if necessary. This approach was developed by Richard Brent (1973) based on an earlier algorithm due to Theodorus Decker (1969).

The bracketing technique is the trusty bisection method ([Fig. 5.4](#)), whereas two different open methods are employed. The first is the secant method described in [Sec. 6.4](#). As explained next, the second is inverse quadratic interpolation.

### 6.5.1 Inverse Quadratic Interpolation

*Inverse quadratic interpolation* is similar in spirit to the secant method. As illustrated in [Fig. 6.12a](#), the secant method is based on computing a straight line that goes through two guesses ([Eq. 6.8](#)). The intersection of this straight line with the  $x$  axis represents the new root estimate. This is equivalent to false position from [Chap. 5](#), except there the two guesses must bracket the root.

**FIGURE 6.12** Comparison of (a) the secant method and (b) inverse quadratic interpolation. Note that the approach in (b) is called "inverse" because the quadratic function is written in  $y$  rather than in  $x$ . For (a), where the two guesses bracket the root, the situation is equivalent to false position.

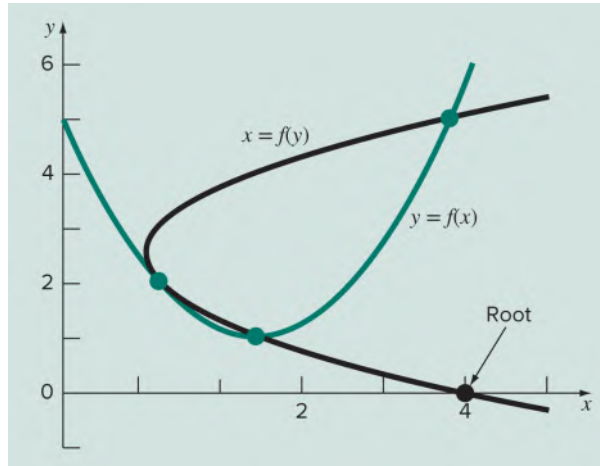


Now, suppose that we had three points. In that case, we could determine a quadratic function of  $x$  that goes through the three points (black circles in [Fig. 6.12b](#)). Just as with the linear secant method, the intersection of this parabola with the  $x$  axis would represent the new root estimate. And, as illustrated in the figure, using a curve rather than a straight line often yields a better estimate.



Although this would seem to represent a great improvement, the approach has a fundamental flaw: it is possible that the parabola might not ever intersect the  $x$  axis! Such would be the case when the resulting parabola had a pair of complex conjugate roots. This is illustrated by the parabola in [Fig. 6.13](#).

**FIGURE 6.13** Two parabolas fit to three points. The parabola written as a function of  $x$ ,  $y = f(x)$ , has complex roots, and hence does not intersect the  $x$  axis. In contrast, if the variables are reversed, and the parabola is developed as  $x = f(y)$ , the curve does intersect the  $x$  axis.



This difficulty can be rectified by employing inverse quadratic interpolation. That is, rather than using a parabola in  $x$ , we can fit the points with a parabola in  $y$ . This amounts to reversing the axes and creating a “sideways” parabola, the  $x = f(y)$  curve in [Fig. 6.13](#).

If the three points are designated as  $(x_{i-2}, y_{i-2})$ ,  $(x_{i-1}, y_{i-1})$ , and  $(x_i, y_i)$ , a quadratic function of  $y$  that passes through the points can be generated as

$$g(y) = \frac{(y - y_{i-1})(y - y_i)}{(y_{i-2} - y_{i-1})(y_{i-2} - y_i)} x_{i-2} + \frac{(y - y_{i-2})(y - y_i)}{(y_{i-1} - y_{i-2})(y_{i-1} - y_i)} x_{i-1} + \frac{(y - y_{i-2})(y - y_{i-1})}{(y_i - y_{i-2})(y_i - y_{i-1})} x_i$$

This equation can be derived by considering a general quadratic polynomial

$$x = ay^2 + by + c$$

and then determining the coefficients  $a$ ,  $b$ , and  $c$ , by requiring the polynomial to pass through the three points noted above,

$$\begin{aligned}
 x_{i-2} &= ay_{i-2}^2 + by_{i-2} + c \\
 x_{i-1} &= ay_{i-1}^2 + by_{i-1} + c \\
 x_i &= ay_i^2 + by_i + c
 \end{aligned}$$

and solving these three linear equations for  $a$ ,  $b$ , and  $c$ .

As we will learn in [Sec. 18.2](#), the form presented in [Eq. \(6.10\)](#) is called a *Lagrange polynomial*. The root  $x_{i+1}$  corresponds to  $y = 0$ , which, when substituted into [Eq. \(6.10\)](#), yields

$$x_{i+1} = \frac{y_{i-1}y_i}{(y_{i-2} - y_{i-1})(y_{i-2} - y_i)} x_{i-2} + \frac{y_{i-2}y_i}{(y_{i-1} - y_{i-2})(y_{i-1} - y_i)} x_{i-1} + \frac{y_{i-2}y_{i-1}}{(y_i - y_{i-2})(y_i - y_{i-1})} x_i$$

As illustrated in [Fig. 6.13](#), such a “sideways” parabola always intersects the  $x$  axis.

## EXAMPLE 6.8

### Inverse Quadratic Interpolation

#### Problem Statement

Develop quadratic equations in both  $x$  and  $y$  for the points depicted in [Fig. 6.13](#):  $(1, 2)$ ,  $(2, 1)$ , and  $(4, 5)$ . For the first,  $y = f(x)$ , illustrate that the roots are complex. For the latter,  $x = g(y)$ , use inverse quadratic interpolation [[Eq. \(6.11\)](#)] to determine the root estimate.

#### Solution

One way to approach the first part of the problem is to write the three simultaneous linear equations noted above, reversing  $x$  and  $y$ , as

$$\begin{aligned}
 2 &= a(1^2) + b(1) + c \\
 1 &= a(2^2) + b(2) + c \\
 5 &= a(4^2) + b(4) + c
 \end{aligned}$$

and solving for  $a = 1$ ,  $b = -4$ , and  $c = 5$ , or

$$y = x^2 - 4x + 5$$

Using the quadratic formula to solve for  $x$  for  $y = 0$ , a complex conjugate pair of roots arises.

$$x = \frac{4 \pm \sqrt{(-4)^2 - (4)(1)(5)}}{(2)(1)} = 2 \pm i$$

Alternately, for the second part of the problem, we can employ [Eq. \(6.10\)](#) to generate the quadratic polynomial in  $y$  as

$$g(y) = \frac{(y-1)(y-5)}{(2-1)(2-5)}(1) + \frac{(y-2)(y-5)}{(1-2)(1-5)}(2) + \frac{(y-2)(y-1)}{(5-2)(5-1)}(4)$$

Collecting terms,

$$g(y) = 0.5y^2 - 2.5y + 4$$

Finally, [Eq. \(6.11\)](#) can be used to determine the root as

$$x_{i+1} = \frac{(-1)(-5)}{(2-1)(2-5)}(1) + \frac{(-2)(-5)}{(1-2)(1-5)}(2) + \frac{(-2)(-1)}{(5-2)(5-1)}(4) = 4$$

Before proceeding to Brent's algorithm, we need to mention one more case where inverse quadratic interpolation does not work. If the three  $y$  values are not distinct, an inverse quadratic polynomial does not exist. So, this is where the secant method comes into play. If we arrive at a situation where the  $y$  values are not distinct, we can always revert to the less efficient secant method to generate a root using two of the points. For example, if  $y_{i-2} = y_{i-1}$ , we use the secant method with  $x_i$  and  $x_{i-1}$ .

## 6.5.2 Brent's Method Algorithm

The general idea behind Brent's<sup>2</sup> root-finding method is to use one of the open, quick methods whenever possible. If these generate an unacceptable result, for example, a root estimate falls outside the bracket, the algorithm reverts to the more conservative bisection method. Although bisection may be slower, it generates an estimate guaranteed to fall within the bracket. This process is repeated until the root is located to within an acceptable tolerance. As might be expected, bisection typically dominates at first, but, as the root is approached, the technique shifts to the faster open methods.

[Figure 6.14](#) presents a Python function `brents` based on a MATLAB m-file developed by Cleve Moler (2004). It represents a simplified version of Brent's method. We will introduce a more sophisticated function, `brentq`, that is available from the Python SciPy module in the next section.

**FIGURE 6.14** Function for Brent's root-finding algorithm.

```

import numpy as np
eps = np.finfo(float).eps
def brentsimp(f,xl,xu):
    a = xl ; b = xu ; fa = f(a) ; fb = f(b)
    c = a ; fc = fa ; d = b - c ; e = d
    while True:
        if fb == 0: break
        if np.sign(fa) == np.sign(fb): # rearrange points as req'd
            a = c ; fa = fc ; d = b - c ; e = d
        if abs(fa) < abs(fb):
            c = b ; b = a ; a = c
            fc = fb ; fb = fa ; fa = fc
        m = (a-b)/2 # termination test and possible exit
        tol = 2 * eps * max(abs(b),1)
        if abs(m) < tol or fb == 0: break
        # choose open methods or bisection
        if abs(e) >= tol and abs(fc) > abs(fb):
            s = fb/fc
            if a == c:
                # secant method here
                p = 2*m*s
                q = 1 - s
            else:
                # inverse quadratic interpolation here
                q = fc/fa ; r = fb/fa
                p = s * (2*m*q*(q-r)-(b-c)*(r-1))
                q = (q-1)*(r-1)*(s-1)
            if p > 0:
                q = -q
            else:
                p = -p
            if 2*p < 3*m*q - abs(tol*q) and p < abs(0.5*e*q):
                e = d ; d = p/q
            else:
                d = m ; e = m
        else:
            # bisection here
            d = m ; e = m
        c = b ; fc = fb
        if abs(d) > tol:
            b = b + d
        else:
            b = b - np.sign(b-a)*tol
        fb = f(b)
    return b

```

The `brentsimp` function has arguments of the function name, `f`, and the two initial guesses, `xl` and `xu`, which must bracket the root. Then, the three variables defining the search interval (`a`, `b`, and `c`) are initialized, and `f` is evaluated at the endpoints.

A main loop is then entered. If necessary, the three points are shuffled to satisfy the conditions required for the algorithm to work effectively. At this point, if one of the stopping criteria is met, the loop is terminated. Otherwise, a decision structure chooses among the three methods and checks whether the outcome is acceptable. A final section then evaluates `f` at the new point `b`, and the loop is repeated. Once one of the stopping criteria is met, the loop terminates, and the final root estimate is returned. Note that there is no iteration limit in place in the algorithm.

## 6.6 PYTHON SCIPY FUNCTION: `brentq`

The `brentq` function is designed to find a real root of a single equation. A simple representation of its syntax is

```
brentq(f,xl,xu)
```

It is available from the `optimize` submodule of SciPy using the Python statement

```
from scipy.optimize import brentq
```

Here is a Console session that solves for a root of a simple quadratic function,  $f(x) = x^2 - 9$ . Clearly, two roots exist,  $x = \pm 3$ . We can test `brentq` for different initial guesses.

```
xsoln = brentq(lambda x: x**2-9,0,4)
print('Solution is ',xsoln)
```

with the result

```
Solution is 3.0000000000000002
```

And, alternately,

```
xsoln = brentq(lambda x: x**2-9,-4,0)
print('Solution is ',xsoln)
```

with the answer being displayed as

```
Solution is -3.0000000000000002
```

If we attempt

```
xsoln = brentq(lambda x: x**2-9,-4,4)
```

an error message is displayed:

```
ValueError: f(a) and f(b) must have different signs
```

The `brentq` function first tests whether the initial guesses bracket a solution. If not, it produces an error message, as shown above. Subsequently, the fast methods, secant and inverse quadratic interpolation, are used unless an unacceptable result occurs, for example, the new root estimate falls outside the original bracket.

It is possible to add arguments to the `brentq` function call. These are described in the “Help” document for the SciPy module and include setting the maximum number of iterations (default = 300), and the convergence tolerances (set very tight at  $10^{-12}$  and  $10^{-16}$ ). The algorithm is based on the book by Brent (1973) and Fortran code from Press et al. (1992).

### EXAMPLE 6.9

#### Solving for a Root with the `brentq` Function

##### Problem Statement

Recall that in [Example 6.3](#), we found the positive root of  $f(x) = x^{10} - 1$  using the Newton-Raphson method with an initial guess of 0.5. The same problem with the `brentq` function and compare results with those of [Example 6.3](#). Also, test the previous `brentsimp` function to find the root.

## Solution

The following Python code is entered into the Spyder editor window.

```
from scipy.optimize import brentq
xsoln = brentq(lambda x: x**10-1,.01,1.1,
               rtol=1.e-7,xtol=1.e-7,maxiter = 20,full_output=True)
print('Solution is ',xsoln)
```

Optional arguments have been added to restrict the iteration limit and relax the convergence tolerances. Also, more output is requested using the `full_output` argument set to `True`. Here is the output in the Console:

```
      Solution is (0.9999999983840661,      converged: True
      flag: 'converged'
function_calls: 11
iterations: 10
      root: 0.9999999983840661)
```

Notice that ten iterations were required to achieve a root estimate more precise than the Newton-Raphson method found in over 40 iterations in [Example 6.3](#).

Below we test the `brentsimp` function with wider initial guesses and find that it successfully finds the root.

```
[xsoln = brentsimp(lambda x: x**10-1,0.1,5)
print('Solution = ',xsoln)

Solution = 1.0
```

## 6.7 POLYNOMIALS

Polynomials represent a special type of nonlinear algebraic equation of the general form

$$f_n(x) = a_1x^n + a_2x^{n-1} + \cdots + a_{n-1}x^2 + a_nx + a_{n+1} \quad (6.12)$$

where  $n$  is the order of the polynomial, and  $a$ 's are the constant coefficients. In many, but not all, cases, the coefficients will be real numbers. For such cases, the roots will be either real or complex conjugate pairs. In general, an  $n^{\text{th}}$ -order polynomial will have  $n$  roots.

Polynomials have many applications in engineering and science. For example, they are used extensively in curve-fitting. However, one of their most interesting and powerful applications is in characterizing dynamic systems—and, in particular, systems represented by linear differential equations. Examples include chemical reactors, mechanical devices, structures, and electrical circuits.

Numerical methods for solving for all the roots of a polynomial fall into two general categories:

1. single root solution coupled with deflation
2. eigenvalue determination

In the first category, a solving technique called Muller's method is combined with a deflation method. *Muller's method* is based on quadratic interpolation (not inverse) and achieves near global convergence and finds both real and complex roots. Once a root is found, it is factored out of the polynomial using a "deflation" technique. The deflation is stopped once the degree of polynomial reaches three or four when closed-form methods apply to finding the remaining roots. Many algorithms use the Newton-Raphson method (or a modified secant method) to "polish" the roots once they have been determined by Muller's method. There is a superb module available for Python called "PyPol"<sup>3</sup> that includes a Muller's method implementation for finding the roots of polynomials.

The second category takes advantage of applied linear algebra methods for finding the eigenvalues of a matrix. It is possible to recast the polynomial root evaluation task as an eigenvalue problem. Because we will be describing eigenvalue problems later in the book, we just provide an overview here.

Suppose, we have a general polynomial

$$a_1x^5 + a_2x^4 + a_3x^3 + a_4x^2 + a_5x + a_6 = 0 \quad (6.13)$$

Dividing by  $a_1$  and rearranging yields

$$x^5 = -\frac{a_2}{a_1}x^4 - \frac{a_3}{a_1}x^3 - \frac{a_4}{a_1}x^2 - \frac{a_5}{a_1}x - \frac{a_6}{a_1}$$

A special matrix can be constructed by using the coefficients from the right-hand side as the first row with 1's and 0's filling in the other rows as shown here:

$$\begin{bmatrix} -\frac{a_2}{a_1} & -\frac{a_3}{a_1} & -\frac{a_4}{a_1} & -\frac{a_5}{a_1} & -\frac{a_6}{a_1} \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \quad (6.14)$$

 **Equation (6.14)** is called the polynomial's *companion matrix*. It has a useful property that its eigenvalues are the roots of the polynomial. The algorithm underlying the `roots` function in the NumPy module, illustrated below, is based on a numerical method for determining the eigenvalues of the companion matrix. The `roots` function has two companion functions that are also useful in polynomial computations:

- `poly`: assembles polynomial coefficients from their root values
- `polyval`: evaluates a polynomial for a given value of  $x$

Also, there is

- `poly1d`: a Python “convenience class” encapsulating various polynomial operations

## EXAMPLE 6.10

### Using Python NumPy to Manipulate Polynomials and Determine Their Roots

#### Problem Statement

Use the following equation to explore how functions in Python's NumPy module can be employed to manipulate and solve polynomials.

$$f(x) = x^5 - 3.5x^4 + 2.75x^3 + 2.125x^2 - 3.875x + 1.25$$

#### Solution

First, we define a list with the coefficients of the polynomial. The list is arranged from highest to lowest power.

```
import numpy as np
a = [ 1, -3.5, 2.75, 2.125, -3.875, 1.25 ]
```

Then, the `roots` function is employed to find the polynomial root values:

```
r = np.roots(a)
print(r)
```



```
[ 2. +0.j -1. +0.j 1. +0.5j 1. -0.5j 0.5+0.j ]
```

Notice that the output is formatted for complex numbers, but, looking closer, you can see that the roots are 2,  $-1$ , 0.5, and  $1 \pm 0.5j$ .

Now, let's say we want to evaluate the polynomial for  $x = \pi$ . We can use the `polyval` function for this.

```
x = np.pi
fx = np.polyval(a,x)
print(fx)
```

with the results

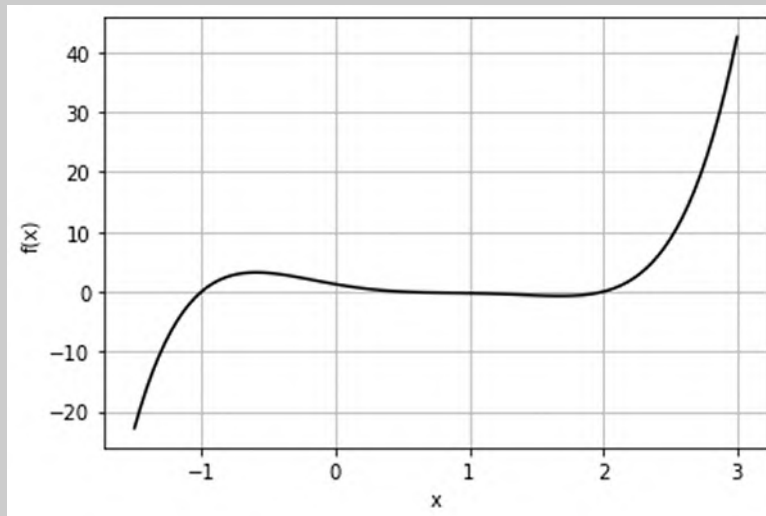
```
60.404364856751855
```

And we can use the same function to create a plot of the polynomial versus  $x$ .

```
xp = np.linspace(-1.5,3,100)
n = len(xp)
fp = []
for i in range(n):
    fp.append(np.polyval(a,xp[i]))
import pylab
pylab.plot(xp,fp,c='k')
pylab.grid()
pylab.xlabel('x')
pylab.ylabel('f(x)')
```

The result is

Page 163



If we would like to create a lower-order polynomial from a pair of the roots, the `poly` function can be used.

```
b = np.poly([0.5,-1])
print(b)
```

resulting in

```
[ 1. 0.5 -0.5]
```

which corresponds to the polynomial

$$x^2 + 0.5x - 0.5$$

What if we would like to divide this latter polynomial into the original? We can use the `poly1d` class for manipulations like addition, subtraction, multiplication, and division of polynomials. See below.

```
p1 = np.poly1d(a)
p2 = np.poly1d(b)
q,r = p1/p2
print('Numerator polynomial = ',p1)
print('Denominator polynomial = ',p2)
print('Quotient = ',q)
print('Remainder = ',r)
```

and the display in the Console is

```
Numerator polynomial =
      5      4      3      2      1
x - 3.5 x + 2.75 x + 2.125 x - 3.875 x + 1.25
Denominator polynomial =
      2      1
x + 0.5 x - 0.5
Quotient =
      3      2      1
x - 4 x + 5.25 x - 2.5
Remainder =
0
```

Note: The integers to the right of the equal signs above represent the corresponding powers of  $x$  in the expressions below.

The original polynomial can be reconstituted by multiplying  $q$  times  $b$ :

```
p1new = q*p2
print('Reconstituted polynomial = ',p1new)
```

and you will see

```
Reconstituted polynomial =
      5      4      3      2      1
x - 3.5 x + 2.75 x + 2.125 x - 3.875 x + 1.25
```

Given the `poly1d` creation of the polynomial variable, instead of using `polyval`, you can simply code the following:

```
print(p1(np.pi))
```

and get

```
60.404364856751855
```

as before.

You will find that the capabilities of Python with the NumPy module to solve and manipulate polynomials come in handy in many engineering and scientific applications.

## PIPE FRICTION

### Background

The study of fluid flow through pipes is important in many applications in engineering and science, from six-foot-diameter pipes carrying water to tiny capillaries with blood flow. Fluids range in character from molten polymers to gases. All flows meet resistance through friction with the walls of the conduits. Fluids are generally categorized into Newtonian and non-Newtonian<sup>4</sup>, and the flow regime as laminar or turbulent<sup>5</sup>. Pipes, tubes, and other conduits are described their size, cross-section shape, and roughness of their surfaces.

The resistance to flow in such conduits is parameterized by a dimensionless quantity, called the *friction factor*. There are two common forms of the friction factor in use, Moody (also called Darcy) and Fanning. They are related by a factor of 4:

$$f_{\text{Moody}} = 4f_{\text{Fanning}}$$

Mechanical engineers tend to use the Moody friction factor, whereas chemical engineers prefer the Fanning friction factor. Below, we will use the symbol  $f$  to represent the Fanning friction factor.

For flow in circular conduits like tubes and pipes, the flow regime is characterized by the Reynolds number, given by

$$\text{Re} = \frac{\rho v D}{\mu}$$

where, with example units,  $\rho$  = fluid density ( $\text{kg}/\text{m}^3$ ),  $v$  = average fluid velocity ( $\text{m}/\text{s}$ ),  $D$  = inside diameter of the pipe ( $\text{m}$ ), and  $\mu$  = fluid viscosity ( $\text{Pa}\cdot\text{s}$ ).<sup>6</sup> There are numerous descriptions of the Fanning friction factor based on the Reynolds number. For laminar flow, based on theory,

$$f = \frac{16}{\text{Re}}$$

and for turbulent flow, there are many empirical correlations. The *Blasius formula* is convenient for pipes with little or no roughness, for example, glass surfaces,

$$f = \frac{0.0791}{\text{Re}^{1/4}}$$

For pipes with surface roughness, the *Colebrook equation* is commonly used.

(6.15)

$$\frac{1}{\sqrt{f}} = -4 \log_{10} \left( \frac{\varepsilon}{3.7D} + \frac{1.26}{Re\sqrt{f}} \right)$$

You will notice that this correlation is implicit in  $f$ . An alternate correlation, the *Haaland equation*, is explicit in  $f$ :

$$f_{\text{Moody}} = 4 f_{\text{Fanning}} = \frac{1}{\left\{ -1.8 \log_{10} \left[ \left( \frac{\varepsilon}{3.7D} \right)^{1.1} + \frac{6.9}{Re} \right] \right\}^2} \quad (6.16)$$

$$f_{\text{Moody}} = 4 f_{\text{Fanning}} = \frac{1}{\left\{ -1.8 \log_{10} \left[ \left( \frac{\varepsilon}{3.7D} \right)^{1.1} + \frac{6.9}{Re} \right] \right\}^2}$$

## Problem Statement

In this case study, we will illustrate how the numerical methods introduced in this part of the book can be employed to determine the Fanning friction factor,  $f$ , for air flow through a thin tube. For this case, the parameters are  $\rho = 1.23 \text{ kg/m}^3$ ,  $\mu = 1.79 \times 10^{-5} \text{ Pa}\cdot\text{s}$ ,  $D = 5 \text{ mm}$  or  $0.005 \text{ m}$ ,  $v = 40 \text{ m/s}$ , and  $\varepsilon = 1.5 \text{ }\mu\text{m}$  or  $= 1.5 \times 10^{-6} \text{ m}$ . For turbulent flow, friction factors typically range between 0.1 and 0.001.

## Solution

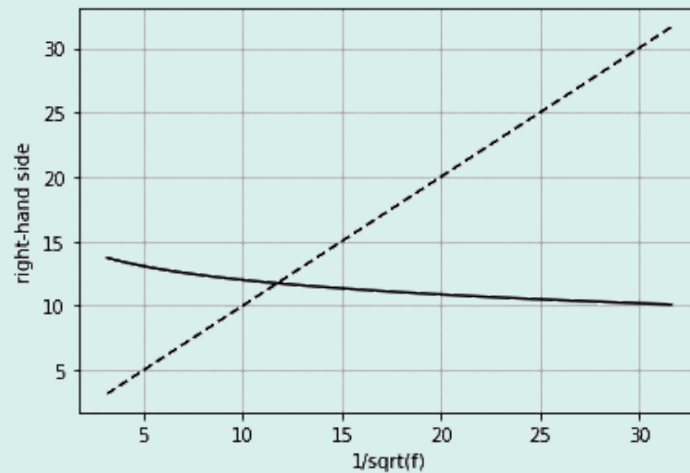
The *Reynolds number* can be computed as

$$Re = \frac{\rho v D}{\mu} = \frac{(1.23)(40)(0.005)}{1.79 \times 10^{-5}} \cong 13,743$$

So, the flow is definitely turbulent.

Before determining the root using a numerical method, it is useful to observe the behavior of [Eq. \(6.15\)](#) for different values of  $f$ . This Python script will do that for us.

```
import numpy as np
import pylab
rho = 1.23 # kg/m3
mu = 1.79e-5 # Pa*s
D = 0.005 # m
V = 40 # m/s
eps = 1.5e-6 # m
Re = rho*V*D/mu # Reynolds number
f = np.logspace(-3,-1,100)
fs = 1/np.sqrt(f)
fr = -4*np.log10(eps/3.7/D+1.26/Re/np.sqrt(f))
pylab.plot(fs,fr,c='k')
pylab.plot(fs,fs,c='k',ls='--')
pylab.grid()
pylab.xlabel('1/sqrt(f)')
pylab.ylabel('right-hand side')
```



Where the solid line crosses the dashed, 45° line, both sides of [Eq. \(6.15\)](#) are equal. This occurs at about a value of 12. That implies a friction factor  $\cong 0.007$ . We also note that the  $|\text{slope}|$  of the solid line is much less than one. This tells us that a circular method, like fixed-point iteration or the Wegstein method, will be convergent. Also, since the slope is very shallow, it indicates that these methods will converge quickly.

Below, we employ our `fixpt` function to obtain a precise value of  $f$ .

```
import numpy as np
rho = 1.23 # kg/m3
mu = 1.79e-5 # Pa*s
D = 0.005 # m
V = 40 # m/s
eps = 1.5e-6 # m
Re = rho*V*D/mu # Reynolds number

def g(f):
    fr = -4*np.log10(eps/3.7/D+1.26/Re/np.sqrt(f))
    return 1/fr**2

f0 = 0.001
fsoln,ea,n = fixpt(g,f0)
print('Fanning friction factor = ',fsoln)
print('Relative error = ',ea)
print('Number of iterations = ',n)
```

with the results

```
Fanning friction factor = 0.007248917198656729
Relative error = 4.330091759458145e-08
Number of iterations = 10
```

The solution agrees with our graphical estimate.

We can also test our `wegstein` function with

```
f0 = 0.001 ; f1 = 0.002
fsoln,ea,n = wegstein(g,f0,f1)
```

and the results are

```
Fanning friction factor = 0.0072489144060813564
Relative error = 1.9428753830183448e-10
Number of iterations = 7
```

Note that fewer iterations are required, and the final relative error is 100 times smaller than for fixed-point iteration. As we would expect, the Wegstein method outperforms fixed-point iteration.

It is notable that the Colebrook equation presents itself in the form convenient for a circular method. For that reason, we employed these methods first. We can also rearrange [Eq. \(6.16\)](#) as

$$g(f) = \frac{1}{\sqrt{f}} + 4\log_{10}\left(\frac{\varepsilon}{3.7D} + \frac{1.26}{\text{Re}\sqrt{f}}\right) = 0 \quad (6.17)$$

and apply bisection, false position, the Newton-Raphson, modified secant, or Brent's methods to solve for  $f$ . To use the Newton-Raphson method, we must derive  $g'(f)$ .

$$g'(f) = -\frac{1}{f^{3/2}} \left[ \frac{1}{2} + \frac{2.52}{\ln(10) \text{Re} \left( \frac{\varepsilon}{3.7D} + \frac{1.26}{\text{Re}\sqrt{f}} \right)} \right]$$

If we would prefer not to face that derivation, we could employ the secant or modified secant method. Here is the code using our `newtraph` function:

```
def g(f):
    return 1/np.sqrt(f)+4*np.log10(eps/3.7/D+1.26/Re/np.sqrt(f))

def gp(f):
    return -1/f**(3/2)*(1/2+2.52/(np.log(10)*Re*(eps/3.7/D+1.26/Re/np.
sqrt(f))))

f0 = 0.001
fsoln,gsoln,ea,n = newtraph(g,gp,f0)
print('Fanning friction factor = ',fsoln)
print('Function value = ',gsoln)
print('Relative error = ',ea)
print('Number of iterations = ',n)
```

Page 167

with the results

```
Fanning friction factor = 0.007248917236610398
Function value = -1.7763568394002505e-15
Relative error = 7.631349518784342e-09
Number of iterations = 7
```

We can see that the performance of the Newton-Raphson method is comparable to the Wegstein method. Of course, with the latter, we did not have to find the derivative.

To complete this case study, now that we have several methods to solve the Colebrook equation for the friction factor, we can compare it graphically to the Haaland explicit equation for a range of Reynolds numbers. Since the latter is a simplified version, if its results are equivalent, we wouldn't need to use root-finding techniques.

For a typical 2-inch, steel pipe,  $D = 0.0605$  m and  $\varepsilon = 45 \mu\text{m}$ . This Python code carries out the comparison:

```
import numpy as np
D = 0.0605 # m
eps = 45e-6 # m

def g(f,Re):
    return 1/np.sqrt(f)+4*np.log10(eps/3.7/D+1.26/Re/np.sqrt(f))

def gp(f,Re):
    return -1/f**(3/2)*(1/2+2.52/(np.log(10)*Re*(eps/3.7/D+1.26/Re/np.
sqrt(f))))
```

```

def Haaland(Re):
    return 1/4*(1/(-1.8*np.log10((eps/3.7/D)**1.1+6.9/Re))**2)

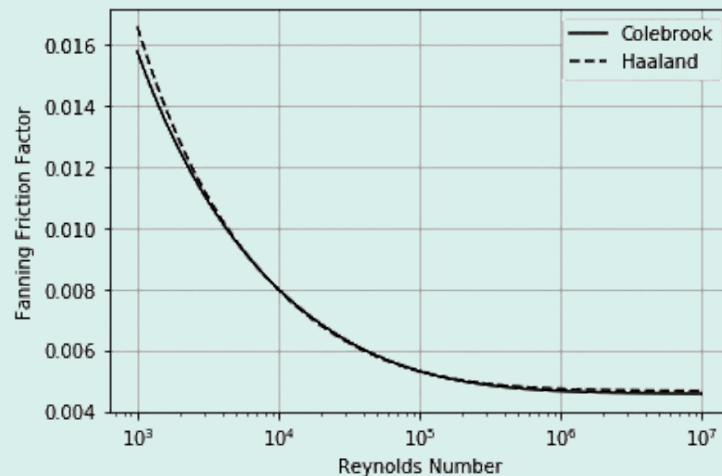
def Colebrook(Re):
    f0 = 0.001
    fsoln,gsoln,ea,n = newtraph(lambda f: g(f,Re),
                                lambda f: gp(f,Re),f0)
    return fsoln

Re = np.logspace(3,7,100)
num = len(Re)
fC = []
fH = []
for i in range(num):
    fC.append(Colebrook(Re[i]))
    fH.append(Haaland(Re[i]))

import pylab
pylab.semilogx(Re,fC,c='k',label='Colebrook')
pylab.semilogx(Re,fH,c='k',ls='--',label='Haaland')
pylab.grid()
pylab.xlabel('Reynolds Number')
pylab.ylabel('Fanning Friction Factor')
pylab.legend()

```

which produces the following plot:



It is evident that the Haaland equation is comparable to the Colebrook equation.

This case study illustrates that, when working with an equation-solving scenario in engineering and science, there are alternative methods available, and one method is not always superior to another. It is important to assess the scenario and perhaps test more than one approach to find what's best. Also, in the latter part of the example, we illustrate that there may be a way around having to use the solving methods by finding an alternate, explicit description of a phenomenon.

# PROBLEMS

---

**6.1** Employ fixed-point iteration to locate the root of

$$f(x) = \sin(\sqrt{x}) - x$$

Use an initial guess of  $x_0 = 0.5$  and iterate until  $\varepsilon_a \leq 0.01\%$ . Verify that the process is linearly convergent as described at the end of [Sec. 6.1](#).

**6.2** Use (a) fixed-point iteration and (b) the Newton-Raphson method to determine a root of

$$f(x) = -0.9x^2 + 1.7x + 2.5$$

using  $x_0 = 5$ . Perform the computations until  $\varepsilon_a \leq 0.01\%$ . Also, check your final answer.

**6.3** Determine the largest positive root of

$$f(x) = x^3 - 6x^2 + 11x - 6.1$$

(a) graphically, (b) using the Newton-Raphson method, three iterations, with  $x_0 = 3.5$ , (c) using the secant method, three iterations, with  $x_{-1} = 2.5$  and  $x_0 = 3.5$ , (d) using the modified secant method, five iterations, with  $x_0 = 3.5$  and  $\delta = 0.01$ , and (e) find all the roots using Python. Comment on your results.

**6.4** Determine the smallest positive root of

$$f(x) = 7\sin(x)e^{-x} - 1$$

(a) graphically, (b) using the Wegstein method, (c) using the Newton-Raphson method, four iterations, with  $x_0 = 0.3$ , and (d) using the modified secant method, with four iterations,  $x_0 = 0.3$  and  $\delta = 0.001$ . Compare your results.

**6.5** Use (a) the Newton-Raphson method, and (b) the modified secant method to determine a root of

$$f(x) = x^5 - 16.05x^4 + 88.75x^3 - 192.0375x^2 + 116.35x + 31.6875$$

using an initial guess,  $x_0 = 0.5825$  and meeting a specification  $\varepsilon_a \leq 0.01\%$ . (c) Then, use the Python NumPy `roots` function to find all the roots. Analyze why (a) and (b) found the root that they did. A



graph may help.

**6.6** Develop a Python function for the secant method. Along with the name of the function to be solved, include the two initial guesses, relative error criterion (default:  $1 \times 10^{-7}$ ) and maximum iteration limit (default 30) as arguments. Test it by solving the function in  **Prob. 6.3**.

**6.7** Develop a Python function for the modified secant method. Along with the name of the function to be solved, include the initial guess, the perturbation fraction,  $\delta$ , relative error criterion (default:  $1 \times 10^{-7}$ ) and maximum iteration limit (default 30) as arguments. Test it by solving the function in  **Prob. 6.3**.

**6.8** Differentiate Eq. (E6.4.1) to derive Eq. (E6.4.2). Show your detailed steps.

**6.9** Employ the Newton-Raphson method to determine a real root for

$$f(x) = -2 + 6x - 4x^2 + 0.5x^3$$

using an initial guess, (a) 4.5 and (b) 4.43. Discuss any peculiarities in your results. Use graphical and analytical methods, as required.

**6.10** The divide-and-average method for approximating the square root (an *olde tyme* method from when calculations were made by hand or with a slide rule) of any positive number  $a$ , is described by

---

Page 169

---

$$x_{i+1} = \frac{x_i + a/x_i}{2}$$

Show that this formula can be derived from the Newton-Raphson method.

**6.11** (a) Apply the Newton-Raphson method to the function

$$f(x) = \tanh(x^2 - 9)$$

to find its known real root at  $x = 3$ . Use an initial guess of  $x_0 = 3.2$  and take at least three iterations. (b) Did the method exhibit convergence to the real root? Sketch a plot with the results of each iteration labeled.

**6.12** The polynomial

$$f(x) = 0.0074x^4 - 0.284x^3 + 3.355x^2 - 12.183x + 5$$

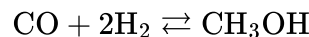
has a real root between 15 and 20. Apply the Newton-Raphson method to this function to find that root using an initial guess of  $x_0 = 16.15$ . Explain your results.

**6.13** Engineers use thermodynamics extensively in their work. An important thermodynamic property is the heat capacity, also called the specific heat. The following polynomial describes the heat capacity of air at atmospheric pressure,  $c_P$  [kJ/(kg·K)], for different temperatures,  $T$  (K):

$$c_P = 0.99403 + 0.1671\left(\frac{T}{1000}\right) + 0.097215\left(\frac{T}{1000}\right)^2 - 0.095838\left(\frac{T}{1000}\right)^3 + 0.01952$$

- (a) Write a Python script to plot  $c_P$  versus  $T$  for the range from 0 to 1200 K.  
 (b) Determine the temperature that corresponds to a heat capacity of 1.1 kJ/(kg·K) using the NumPy polynomial `roots` function.

**6.14** An important industrial chemical reaction is the production of methanol<sup>7</sup> (CH<sub>3</sub>OH, which can also be written as MeOH) from “syngas,” a mixture of carbon monoxide (CO) and hydrogen (H<sub>2</sub>),



The amount of methanol that can be produced is limited by chemical equilibrium, which is described by

$$\frac{y_{\text{MeOH}}}{y_{\text{CO}} y_{\text{H}_2}^2} \frac{1}{P^2} = K_{eq}(T), \quad \text{where} \quad K_{eq}(T) = 4.7914 \times 10^{-13} e^{11,458/T}$$

Here,  $y_i$  = the mole fraction of component  $i$  present,  $P$  = the pressure in atmospheres, and  $T$  = the temperature in kelvins (K). If 100 mols of CO and 200 mols of H<sub>2</sub> are fed to a reactor and, if equilibrium were reached, then we would want to know how much MeOH was produced. If that were designated as  $x$  mols, then there would be  $100 - x$  mols of CO and  $200 - 2x$  mols of H<sub>2</sub> remaining. The total number of mols present would be

$$x + 100 - x + 200 - 2x = 300 - 2x$$

and the mol fractions of the components would be

$$\text{CO: } y_{\text{CO}} = \frac{100 - x}{300 - 2x} \quad \text{H}_2: y_{\text{H}_2} = \frac{200 - 2x}{300 - 2x}$$

and

$$\text{MeOH: } y_{\text{MeOH}} = \frac{x}{300 - 2x}$$

The equilibrium relationship would then be

$$\frac{\frac{x}{300 - 2x}}{\left(\frac{100 - x}{300 - 2x}\right)\left(\frac{200 - 2x}{300 - 2x}\right)} \frac{1}{P^2} = K_{eq}(T)$$

For a temperature of 450 K and a pressure of two atmospheres, determine  $x$  using Brent's method. You may wish to use the `brents imp` function or, from SciPy, the `brentq` function. You should know that  $x$  cannot be greater than 100.

**6.15** The *Soave-Redlich-Kwong* (SRK) *equation of state* is used to describe the behavior of gases at elevated, non-ideal pressures and temperatures. It is a modification of the ideal gas law and is given by

$$P = \frac{RT}{\hat{V} - b} - \frac{\alpha a}{\hat{V}(\hat{V} - b)}$$

where  $P$  = pressure (atm),  $R$  = gas law constant, 0.082057 L·atm/(mol·K),  $T$  = temperature (K),  $\hat{V}$  = specific volume, (L/mol), and  $a$ ,  $\alpha$ , and  $b$  are empirical constants for the gas under consideration given by

$$a = 0.42747 \frac{(RT_c)^2}{P_c} \quad b = 0.08664 \frac{RT_c}{P_c}$$

$$\alpha = \left[ 1 + m \left( 1 - \sqrt{\frac{T}{T_c}} \right) \right]^2$$

where

$$m = 0.48508 + 1.5517\omega - 0.1561\omega^2$$

where  $T_c$  = the critical temperature for the gas (K),  $P_c$  = the critical pressure for the gas (atm), and  $\omega$  = the Pitzer acentric factor for the gas. Note: The ideal gas law is given by the simpler formula

$$P = \frac{RT}{\hat{V}}$$

Consider methane gas with  $T_c = 190.7$  K,  $P_c = 45.8$  atm. and  $\omega = 0.008$ . Methane ( $\text{CH}_4$ ) has a molecular weight of 16.04 g/mol. Determine the mass in kg of methane that can be stored in a tank with volume  $3 \text{ m}^3$  at a temperature of  $-40^\circ\text{C}$  and a pressure of 50 atm. Use the root-finding method of your choice to determine  $\hat{V}$ . From  $\hat{V}$ , you can determine the number of mols of methane, and then the mass using the molecular weight.

**6.16** The volume of a liquid,  $V$ , in a horizontal, cylindrical tank of radius,  $R$ , and length,  $L$ , is given by the formula

$$V = \left[ R^2 \cos^{-1} \left( \frac{R-h}{R} \right) - (R-h) \sqrt{2Rh - h^2} \right] L$$

where  $h$  = the depth of the liquid at the centerline of the cylinder.

- Determine  $h$ , given  $R = 2$  m,  $L = 5$  m, and  $V = 8 \text{ m}^3$ .
- Using your solution technique from (a), create a table of  $h$  versus  $V$  for increments of  $1 \text{ m}^3$ , from  $1 \text{ m}^3$  until just before the tank is filled.

**6.17** A catenary cable is one hung between two points not in the same vertical line. As depicted in  Fig. P6.17a, it is subject to no loads other than its own weight. Thus, its weight acts as a uniform load per unit length along the cable,  $w$  (N/m). A free-body diagram of a section AB is shown in  Fig. 6.17b, where  $T_A$  and  $T_B$  are the tension forces at the ends. Based on horizontal and vertical force balances, the following differential equation model of the cable can be derived

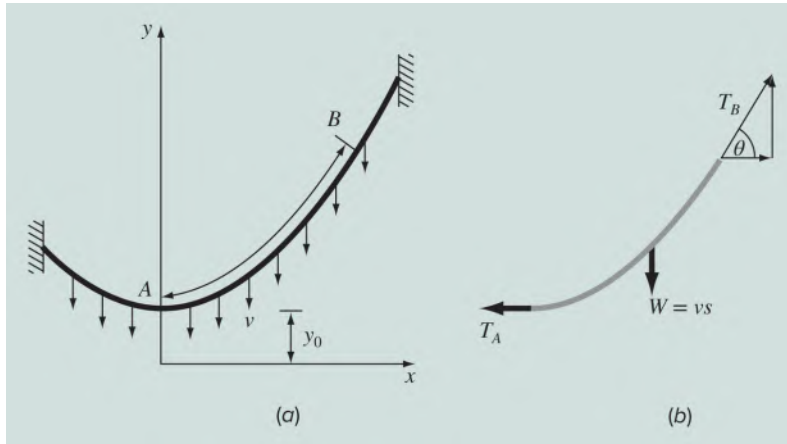
$$\frac{d^2y}{dx^2} = \frac{w}{T_A} \sqrt{1 + \left( \frac{dy}{dx} \right)^2}$$

Calculus can be employed to solve this equation for the height of the cable,  $y$ , as a function of the distance,  $x$ : Page 170

$$y = \frac{T_A}{w} \cosh \left( \frac{w}{T_A} x \right) + y_0 - \frac{T_A}{w}$$

- Use a numerical method to calculate a value for the parameter  $T_A$  given the values  $w = 10$  and  $y_0 = 5$ , such that the cable has a height  $y = 15$  at  $x = 50$ .
- Develop a plot of  $y$  versus  $x$  for  $-150 \leq x \leq 100$ .

**FIGURE P6.17**



**6.18** An oscillating current in an electrical circuit is described by

$$I = 9e^{-t} \sin(2\pi t)$$

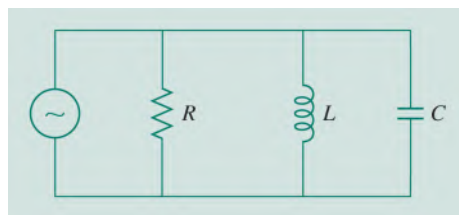
where  $t$  is in seconds. Determine all values of  $t$  such that  $I = 3.5$ .

**6.19** A classical L-R-C electrical circuit is shown in  **Fig. P6.19**. The components are a resistor, inductor (coil), and a capacitor. Using Kirchoff's rules, the impedance<sup>8</sup> of the circuit can be described by the equation

$$\frac{1}{Z} = \sqrt{\frac{1}{R^2} + \left(\omega C - \frac{1}{\omega L}\right)^2}$$

where  $Z = \text{impedance}(\Omega)$  and  $\omega$  is the angular frequency ( $1/s$ ). Find  $\omega$  that results in an impedance of  $100 \Omega$  using the `brentq` function with initial guesses of 1 and 1000 for the following parameter values:  $R = 225 \Omega$ ,  $C = 0.6 \times 10^{-6} \text{ F}$ , and  $L = 0.5 \text{ H}$ .

**FIGURE P6.19**



**6.20** Real mechanical systems may involve the deflection of nonlinear springs. In  **Fig. P6.20**, a mass  $m$  is released a distance  $h$  above a nonlinear spring. The resistance force of the spring is given by

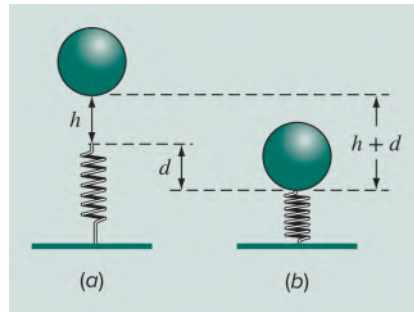
$$F = -\left(k_1 d + k_2 d^{3/2}\right)$$

Conservation of energy can be used to show that

$$0 = \frac{2k_2 d^{5/2}}{5} + \frac{1}{2}k_1 d^2 - mgd - mgh$$

Solve for  $d$  given the following parameter values:  $k_1 = 40,000 \text{ g/s}^2$ ,  $k_2 = 40 \text{ g/(s}^2 \cdot \text{m}^{0.5})$ ,  $m = 95 \text{ g}$ ,  $g = 9.81 \text{ m/s}^2$ , and  $h = 0.43 \text{ m}$ .

**FIGURE P6.20**

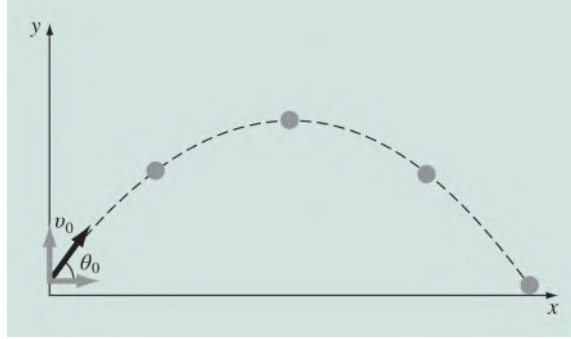


**6.21** Aerospace engineers compute the trajectories of projectiles, such as rockets. A related problem deals with the trajectory of a golf ball for an approach shot to the green of around 100 yards. This trajectory is defined by the  $\{x, y\}$  coordinates, as described in  **Fig. P6.21**. If we neglect such factors as air resistance<sup>9</sup>, the trajectory can be modeled based on elementary kinematics by the equation

$$y = \tan(\theta_0)x - \frac{g}{2v_0^2 \cos^2(\theta_0)}x^2 + y_0$$

Find the launch angle,  $\theta_0$ , if  $v_0 = 125 \text{ mph}$ , for the approach shot distance. Consider an elevated green, 30 feet above the level on the fairway where the shot is taken. Be careful with your units here. Best to convert to SI. Report your result in radians and degrees.

**FIGURE P6.21**

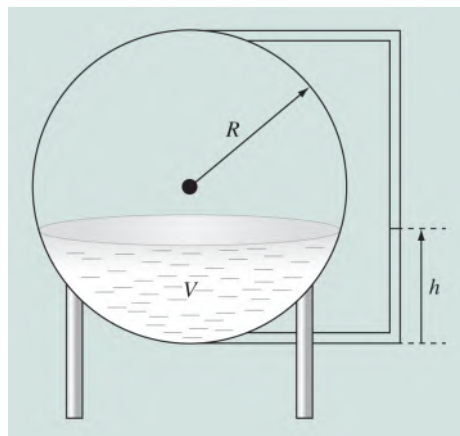


**6.22** You are designing a spherical tank (Fig. P6.22) to hold water for a small village in a developing country. The volume of liquid,  $V$ , for a given depth,  $h$ , is given by the formula

$$V = \pi h^2 \frac{(3R - h)}{3}$$

where  $R$  = the tank radius. A tank is available with radius,  $R = 3$  m. Will the tank hold  $30 \text{ m}^3$  of water? What will be the depth,  $h$ , with this volume? Compare the performance of (a) a bracketing method with (b) an open method for three iterations of the methods. Use relative and absolute error as criteria for comparison. Initial guesses for bracketing methods of 0 and full tank ( $h = 2R$ ) will always enclose a root. For open methods, an initial guess of  $h = R$  will always converge to the root.

**FIGURE P6.22**



**6.23** Use the polynomial capabilities of Python illustrated in Example 6.10 to manipulate and find all the roots of

$$f_5(x) = (x + 2)(x + 5)(x - 6)(x - 4)(x - 8)$$

First and foremost, from the equation above, it is clear that the roots are  $-2$ ,  $-5$ ,  $6$ ,  $4$ , and  $8$ .

Python operations should confirm this.

**6.24** In control systems analysis, *transfer functions* are developed that describe how a system transfers dynamically an input signal to an output signal. Such a transfer function for a robotic positioning system is given by

$$G(s) = \frac{C(s)}{N(s)} = \frac{s^3 + 9s^2 + 26s + 24}{s^4 + 15s^3 + 77s^2 + 153s + 90}$$

where  $G(s)$  is the transfer function,  $C(s)$  is the output signal, and  $N(s)$  is the input signal, all expressed in the Laplace domain with its independent variable  $s$ . Find the roots of the numerator and denominator polynomials and express the transfer function in the form

$$G(s) = \frac{(s + a_1)(s + a_2)(s + a_3)}{(s + b_1)(s + b_2)(s + b_3)(s + b_4)}$$

**6.25** Manning's equation for open channel flow can be written for a rectangular channel as

$$Q = \frac{\sqrt{S}(BH)^{5/3}}{n(B + 2H)^{2/3}}$$

where  $Q$  = the flow rate ( $\text{m}^3/\text{s}$ ),  $S$  = the channel slope ( $\text{m}/\text{m}$ ),  $B$  = the channel width ( $\text{m}$ ),  $H$  = the depth of the flow ( $\text{m}$ ), and  $n$  the Manning roughness coefficient. Develop a fixed-point iteration scheme to solve this equation for  $H$  given  $Q = 5$ ,  $S = 0.0002$ ,  $B = 20$ , and  $n = 0.03$ . Perform the computation until  $\varepsilon_a < 0.05\%$ . Prove that your scheme converges for all initial guesses greater than or equal to zero.

**6.26** See if you can develop a foolproof function to compute the Fanning friction factor,  $f$ , based on the Colebrook equation as described in [\[4\] Sec. 6.7](#), [\[4\] Eq. \(6.15\)](#). Your function should include arguments for the Reynolds number,  $Re$ , pipe inside diameter,  $D$ , and pipe roughness,  $\varepsilon$ . It should return a precise value for the friction factor for Reynolds numbers above 2100, pipe diameter from 10 mm (1/2-inch pipe) to 575 mm (24-inch pipe), and for pipe roughness in the range 0 (glass, plastic) to 260  $\mu\text{m}$  (cast iron). Test it over these ranges. Be careful to use consistent units.

**6.27** Use the Newton-Raphson method to find the root of

$$f(x) = e^{-0.5x}(4 - x) - 2$$



Employ initial guesses of (a) 2, (b) 6, and (c) 8. Explain your results.

**6.28** Given

$$f(x) = -2x^6 - 1.5x^4 + 10x + 2$$

use a root-finding technique of your choice to determine a maximum of this function. Perform iterations until the absolute relative error falls below 1%. If you use a bracketing method, like bisection or false-position, use initial guesses for  $x$  of 0 and 2. If you use the Newton-Raphson or modified secant method, use an initial guess of 1. For the secant method, use initial guesses of 0 and 1. Assuming that convergence is not an issue, choose the technique that is best suited to this problem. Justify your choice.

**6.29** You must determine the root of the following easily differentiable function:

$$e^{0.5x} = 5 - 5x$$

Pick the best numerical technique. Justify your choice. Use that technique to determine the root. It is known that, for all positive initial guesses, all techniques except fixed-point iteration, will eventually converge. Perform iterations until the absolute relative error falls below 2%. If you use a bracketing method, use initial guesses for  $x$  of 0 and 2. For the Newton-Raphson or modified secant method, use an initial guess of 0.7. If you use the secant method, use 0 and 1 as initial guesses.

- 6.30 (a)** Modify the `brentsimp` function from Sec. 6.4.2 to include an absolute relative error specification, default value of  $1 \times 10^{-7}$ , and an iteration limit, default value of 30, in the arguments. Modify the code to replace the tolerance computed based on the machine epsilon, `eps`, with error argument and to count and limit the number of iterations according to the corresponding argument. Also, add code to trap when the initial guesses do not bracket a solution and provide a return error message.
- (b)** Test your function by using it to solve for the root of the function from  **Example 5.6**, specifying a relative error of  $1 \times 10^{-5}$  and an iteration limit of 20.

**6.31**  **Figure P6.31** illustrates the side view of a broad-crested weir in a flowing channel. The symbols shown in the figure are defined as  $H_w$  = height of the weir (m),  $H_h$  = the head above the weir upstream of the drawdown effect (m),  $H$  = the depth of the river upstream of the weir (m), and  $Q_w$  = the channel flow rate over the weir ( $\text{m}^3/\text{s}$ ). The flow rate can be computed as (Munson et al., 2013)

$$Q_w = C_w B_w \sqrt{g} \left( \frac{2}{3} H_h \right)^{3/2}$$

where  $C_w$  = the weir coefficient,  $B_w$  = the weir width (m). The weir coefficient can be determined from

$$C_w = 1.125 \sqrt{\frac{1 + H_h/H_w}{2 + H_h/H_w}}$$

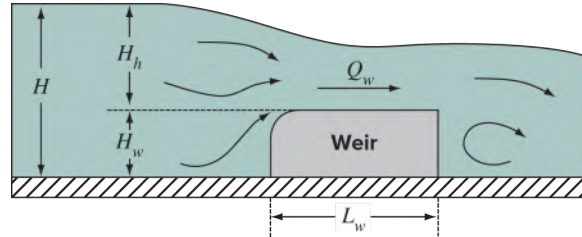
Given  $H_w = 0.8$  m,  $B_w = 8$  m, and  $Q_w = 1.3$  m<sup>3</sup>/s, determine the upstream channel depth,  $H$ , using

Page 172

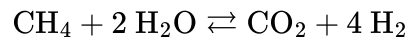
- (a) the modified secant method with  $\delta = 10^{-5}$ ,
- (b) the Wegstein method, and
- (c) the brentq function from Python's SciPy module.

For (a), use  $1.5H_w$  as the initial guess. For (b) and (c), use  $1.01H_w$  and  $1.14H_w$  as initial guesses. Would you expect this problem to be convergent for fixed-point iteration? Explain.

**FIGURE P6.31** A broad-crested weir used to control depth and velocity of streams and canals.



**6.32** Steam-reforming of methane is currently the predominant process to produce hydrogen. The following reversible chemical reaction describes this process.



The reaction is promoted by the presence of a nickel catalyst. The amount of hydrogen produced is limited by chemical equilibrium, which is modeled as

$$\frac{[\text{CO}_2][\text{H}_2]^4}{[\text{CH}_4][\text{H}_2\text{O}]^2} = K_{\text{eq}}$$

where  $K_{\text{eq}}$  = the equilibrium constant, usually dependent on temperature. The brackets,  $[\cdot]$ , denote molar concentrations of the components (mol/L). Molar conservation allows the equilibrium relationship to be reformulated as

$$\frac{\left(\frac{x}{V}\right)\left(\frac{4x}{V}\right)^4}{\left(\frac{M_{\text{CH}_4} - x}{V}\right)\left(\frac{M_{\text{H}_2\text{O}} - 2x}{V}\right)^2} = K_{\text{eq}}$$

where  $x$  = the number of moles of  $\text{CO}_2$  created when molar amounts,  $M_{\text{CH}_4}$  and  $M_{\text{H}_2\text{O}}$ , are fed to a reactor of volume  $V$ . For values of  $K_{\text{eq}} = 7 \times 10^{-3}$ ,  $V = 20$  L,  $M_{\text{CH}_4} = 1$  mol, and  $M_{\text{H}_2\text{O}} = 3$  moles, find the value of  $x$  using (a) the Wegstein method and (b) Brent's method.

**6.33** The concentration of pollutant bacteria,  $c$ , in a lake decreases according to

$$c = 77e^{-1.5t} + 20e^{-0.08t}$$

You can see from above that, at  $t = 0$ ,  $c(0) = 97$ . Determine the time required for the bacteria concentration to be reduced to 15 using the Newton-Raphson method with an initial guess of  $t = 1$  and with a relative error less than 0.1 %. Check your result with the SciPy `brentq` function with initial guesses of 1 and 10.

**6.34** You are asked to solve for the root of the following equation with fixed-point iteration:

$$x^4 = 5x + 10$$

Determine the solution approach that converges for initial guesses in the range  $0 \leq t \leq 7$ . Use either a graphical or analytical approach to prove that your formulation always converges in the given range.

**6.35** A cast iron pipe is used to convey water at a volumetric flow rate,  $Q = 0.3 \text{ m}^3/\text{s}$ . Assume that the flow is steady and fully developed, and water, of course, is relatively incompressible. The head loss (pressure loss in terms of height), friction, and pipe diameter are related by the *Darcy-Weisbach* equation,

$$h_L = f \frac{2Lv^2}{Dg}$$

where  $f$  = the Fanning friction factor,  $L$  = pipe length (m),  $D$  = pipe inside diameter (m), and  $v$  = fluid average velocity (m/s). The velocity is relation to volumetric flow rate by

$$Q = Av$$

where  $A$  = inside cross-sectional area of the pipe  $= \pi D^2/4 (\text{m}^2)$ . The *Fanning friction factor* can be determined from the *Colebrook equation*,

$$\frac{1}{\sqrt{f}} = -4\log_{10}\left(\frac{\varepsilon}{3.7D} + \frac{1.26}{\text{Re}\sqrt{f}}\right)$$

You have a specification that the head loss be no more than 0.006 meter per meter length of pipe. Develop a Python script to achieve this objective. The roughness for cast iron pipe is  $\varepsilon = 0.26$  mm. The Reynolds number is given by

$$\text{Re} = \frac{\rho v D}{\mu}$$

and the viscosity of water,  $\mu$ , at typical temperatures, that is, 20°C, is 1 centipoise or, in SI units, 0.001 Pa·s. The density of water at this temperature is 998 kg/m<sup>3</sup>.

**6.36**  **Figure P6.36** shows an asymmetric, diamond-shaped, supersonic airfoil. The orientation of the airfoil is represented by several angles:  $\alpha$  = the angle of attack,  $\beta$  = the shock angle,  $\theta$  = the deflection angle, with the subscripts “l” and “u” designating the lower and upper surfaces of the airfoil, respectively. The following formula relates the deflection angle to the oblique shock angle and speed:

$$\tan(\theta) = \frac{2 \cot(\beta)(M^2 \sin^2(\beta) - 1)}{M^2(k + \cos(2\beta) + 2)}$$

where  $M$  = Mach number, which is the ratio of the airfoil speed,  $v$ (m/s), to the speed of sound,  $c$ (m/s), and

$$c = \sqrt{kR'T}$$

where  $k$  = the ratio of  $c_P$ , the heat capacity of air at constant pressure, to  $c_V$ , the heat capacity of air at constant volume ( $k = 1.4$  for air), and  $T$  = the absolute temperature (K).  $R'$  is the gas law constant specific to air and is equal to  $R/MW_{\text{air}}$ , where  $R$  is the universal gas law constant and  $MW_{\text{air}}$  is the average molecular weight of air (28.97 kg/kg mol). Consequently,

$$R' = \frac{8.3144 \left[ \frac{\text{kg m}^2}{\text{K mol s}^2} \right]}{28.97 \left[ \frac{\text{kg}}{\text{kgmol}} \right]} 1000 \left[ \frac{\text{mol}}{\text{kgmol}} \right] = 278.35 \left[ \frac{\text{m}^2}{\text{Ks}^2} \right]$$

Given estimates of  $M$ ,  $k$ , and  $\theta$ , the shock angle can be determined as the root of

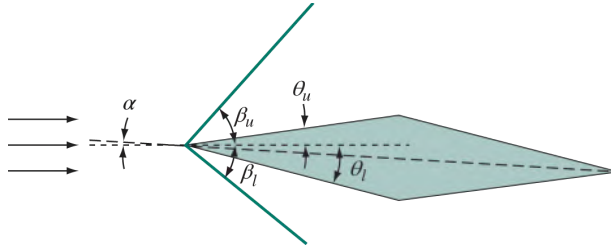
$$f(\beta) = \tan(\theta) - \frac{2 \cot(\beta) (M^2 \sin^2(\beta) - 1)}{M^2(k + \cos(2\beta) + 2)}$$

The pressure on the surface of the airfoil,  $p_a$  (kPa), can then be computed as

$$p_a = p \left( \frac{2k}{k+1} (M \sin^2(\beta)) - \frac{k-1}{k+1} \right)$$

Suppose that the airfoil is attached to an aircraft traveling at a speed,  $v = 625$  m/s, through air with a temperature,  $4^\circ\text{C}$ , pressure, 85 kPa, and  $\theta_u = 4^\circ$ . Develop a Python script to (a) generate a plot of  $f(\beta_u)$  versus  $\beta_u$  over the range from  $2^\circ$  to  $88^\circ$ , and, for the given conditions, compute the pressure on the upper surface of the airfoil.

**FIGURE P6.36** A diamond-shaped airfoil.



**6.37** As described in [Sec. 1.4](#), for objects falling through fluids at very low speeds, the flow regime around the object will be laminar, and the relationship between the drag force and velocity is linear. In addition, in such cases, the buoyancy force must also be considered. A force balance can be written as

$$\frac{dv}{dt} = \underbrace{g}_{\text{gravity}} - \underbrace{\frac{\rho_f V}{m} g}_{\text{buoyancy}} - \underbrace{\frac{c_d}{m} v}_{\text{drag}} \quad (\text{P6.37})$$

where  $v$  = velocity of the object (m/s),  $t$  = time (s),  $m$  = mass of the object (kg),  $\rho_f$  = fluid density ( $\text{kg}/\text{m}^3$ ),  $V$  = particle volume ( $\text{m}^3$ ), and  $c_d$  = linear drag coefficient (kg/m). The mass of the object can be computed from its volume,  $V$ , and density,  $\rho_s$  ( $\text{kg}/\text{m}^3$ ). For a sphere, Stokes developed the following formula for the drag coefficient,  $c_d = 6\pi\mu r$ , where  $\mu$  = the fluid viscosity (Pa·s), and  $r$  = the sphere's radius.

You release an iron sphere at the surface of a container ( $x = 0$ ) filled with honey ([Fig. P6.37](#)) and measure how long it takes to settle to a given depth. Use the following information to estimate the viscosity of the honey:  $\rho_f = 1420 \text{ kg/m}^3$ ,  $\rho_s = 7850 \text{ kg/m}^3$ ,  $r = 20 \text{ mm}$ ,  $L = 0.5 \text{ m}$ , and at  $x = 0.5 \text{ m}$ ,  $t = 3.6 \text{ s}$ . Check the Reynolds number for the flow, given by

$$\text{Re} = \frac{\rho_f v d}{\mu}$$

where  $d$  = the diameter of the sphere (m). The Reynolds number should be well under 2,000 to confirm that laminar conditions exist for the experiment.

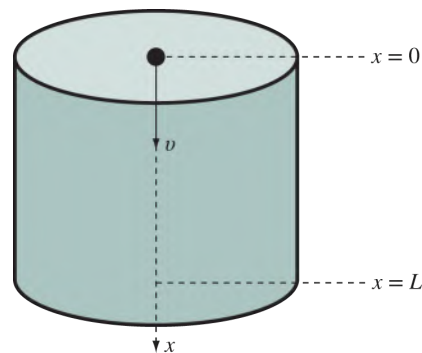
Hint: This problem can be approached by solving the first-order differential equation first, [Eq. \(P6.37\)](#), to yield an equation for  $v$  as a function of  $t$ . And then, noting that  $v = dx/dt$ , integrating the result to obtain an equation for  $x$  as a function of  $t$ . Recall that, for

$$a \frac{dy}{dt} + y = b \quad \text{with} \quad y(0) = y_0$$

the solution is

$$y(t) = y_0 + b(1 - e^{-t/a})$$

**FIGURE P6.37**

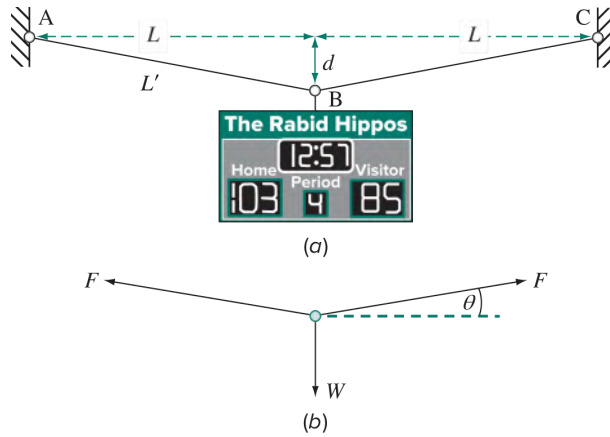


**6.38** As depicted in [Fig. P6.38a](#), a scoreboard is suspended above a sports arena by two cables pinned at  $A$ ,  $B$ , and  $C$ . The cables are initially horizontal and of length  $L$ . After the scoreboard is hung, a free-body diagram at node  $B$  can be developed, as shown in [Fig. P6.38b](#). Assuming that the weight of each cable is negligible compared to that of the scoreboard, determine the deflection,  $d$  (m), that results if the scoreboard weighs  $W = 9,000 \text{ N}$ , about 2,000 lb. Also, compute how much each cable is elongated. Note that each cable obeys Hooke's Law such that the axial elongation is represented by

$$L' - L = \frac{F L}{A_c E}$$

where  $F$  = the axial force (N),  $A_c$  = the cable's cross-sectional area ( $\text{m}^2$ ), and  $E$  = the modulus of elasticity ( $\text{N}/\text{m}^2$ ). Use the following parameter values for your calculations:  $L = 45$  m,  $A_c = 6.362 \times 10^{-4} \text{ m}^2$ , and  $E = 1.5 \times 10^{11} \text{ N}/\text{m}^2$ .

**FIGURE P6.38** (a) Two thin cables pinned at A, B, and C with a scoreboard suspended from B. (b) Free-body diagram of the pin B after the scoreboard is hung.



**6.39** A water tower is connected to a steel pipe with a valve at its end, as shown in  Fig. P6.39. By applying several simplifying assumptions, for example, minor friction losses neglected, the following mechanical energy balance can be written

$$gh - \frac{v^2}{2} = f_M \left( \frac{L + h}{d} + \frac{L_e}{d} + \frac{L_v}{d} \right) \frac{v^2}{2} + K \frac{v^2}{2}$$

where  $h$  = the height from the horizontal pipe to the water level in the tower (m),  $v$  = the average velocity of the water in the pipe (m/s),  $f_M$  = the Moody friction factor,  $4 \times$  the Fanning friction factor,  $L$  = the length of the horizontal section of pipe (m),  $L_e$  = the equivalent pipe length for the elbow fitting (m),  $L_v$  = the equivalent pipe length for the valve (m),  $K$  = the loss coefficient for the contraction at the bottom of the tower, and  $d$  = the inside diameter of the pipe (m).

Write a Python script to determine the flow rate through the pipe and exiting through the valve,  $Q$  ( $\text{m}^3/\text{s}$ ), using the following parameter values:  $h = 24$  m,  $L = 65$  m,  $d = 154.2$  mm (standard 6-inch pipe),  $L_e = 30$  m,  $L_v = 8$  m, and  $K = 0.5$ . The density of water at  $20^\circ\text{C}$  is  $998 \text{ kg}/\text{m}^3$ . The

flow rate is computed by  $v \times A_c$ , where  $A_c$  is the cross-sectional area of the pipe. The Moody friction factor can be determined from the Colebrook equation for this form of the friction factor:

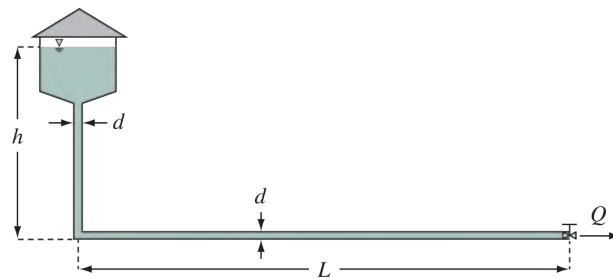
$$\frac{1}{\sqrt{f_M}} = -2.0 \log_{10} \left( \frac{\varepsilon}{3.7d} + \frac{2.51}{Re\sqrt{f}} \right)$$

where the Reynolds number is given by

$$Re = \frac{\rho v d}{\mu}$$

and  $\mu$  = the fluid viscosity. Here, for water at 20°C, the viscosity is 1 centipoise or, in SI units, 0.001 Pa·s. The roughness for a steel pipe is  $\varepsilon = 45 \mu\text{m}$ .

**FIGURE P6.39** A water tower connected to a pipe with a valve at its end.





# Optimization

## CHAPTER OBJECTIVES

The primary objective of this chapter is to introduce you how optimization can be used to determine minima and maxima of both one-dimensional and multidimensional functions. Specific objectives and topics covered are

- Understanding why and where optimization occurs in engineering and science.
- Recognizing the difference between one-dimensional and multidimensional optimization.
- Distinguishing between global and local optima.
- Knowing how to recast a maximization problem so that it can be solved with a minimizing algorithm.
- Being able to define the golden ratio and understand why it makes one-dimensional optimization efficient.
- Locating the optimum of a single-variable function with the golden-section search.
- Locating the optimum of a single-variable function with parabolic interpolation.
- Knowing how to apply the Python `minimize_scalar` function to determine the minimum of a one-dimensional function.
- Being able to develop contour and surface plots to visualize two-dimensional functions.
- Knowing how to apply the Python `minimize` function to determine the minimum of a multidimensional function.

## YOU'VE GOT A PROBLEM

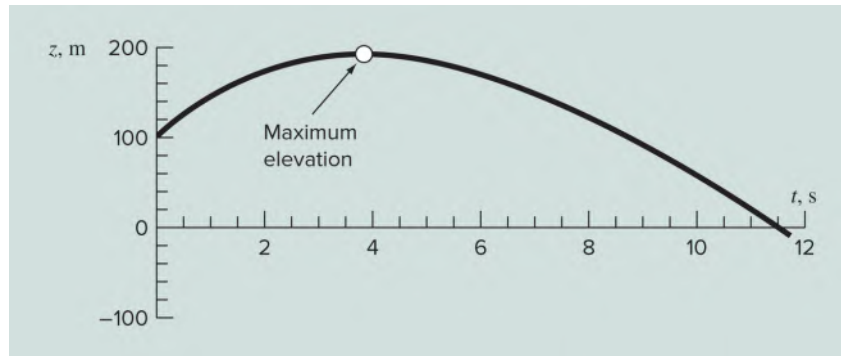
An object like a bungee jumper can be projected upward at a specified velocity. If it is subjected to linear drag, its altitude as a function of time can be computed as

$$z = z_0 + \frac{m}{c} \left( v_0 + \frac{mg}{c} \right) \left( 1 - e^{-\frac{t}{m/c}} \right) - \frac{mg}{c} t \quad (7.1)$$

where  $z$  = altitude (m) above the earth's surface ( $z = 0$ ),  $z_0$  = the initial altitude (m),  $m$  = mass (kg),  $c$  = the linear drag coefficient (kg/s),  $v_0$  = initial velocity (m/s), and  $t$  = time(s). Note that for this formulation, positive velocity is considered to be upward. Given the following parameter values:  $g = 9.81 \text{ m/s}^2$ ,  $z_0 = 100 \text{ m}$ ,  $v_0 = 55 \text{ m/s}$ ,  $m = 80 \text{ kg}$ , and

$c = 15 \text{ kg/s}$ , [Eq. \(7.1\)](#) can be used to calculate the jumper's altitude. As displayed in [Fig. 7.1](#), the jumper rises to a peak elevation of about 190 m at about  $t = 4 \text{ s}$ .

**FIGURE 7.1** Elevation as a function of time for an object initially projected upward with an initial velocity.



Suppose that you are given the job to determine the exact time of the peak elevation. The determination of such extreme values is referred to as optimization. The chapter will introduce you to how the computer is used to make such determination.

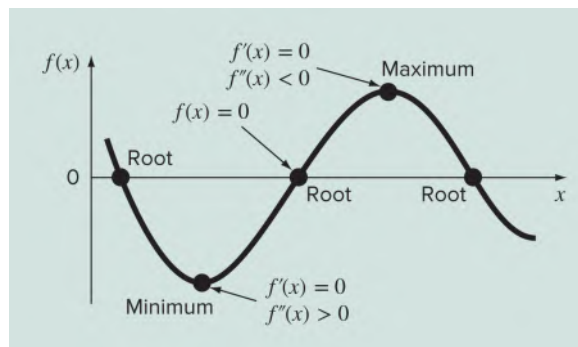
## 7.1 INTRODUCTION AND BACKGROUND

In the most general sense, optimization is the process of creating something that is as effective as possible. As engineers, we must continuously design devices, processes, and products that perform tasks in an efficient fashion for the least cost. Thus, engineers are always confronting optimization problems that attempt to balance performance and limitations. In addition, scientists have interest in optimal phenomena ranging from the peak elevation of projectiles to minimum free energy.

From a mathematical perspective, optimization deals with finding the maxima and minima of a function that depends on one or more variables. The goal is to determine the values of the variables that yield maxima or minima for the function. These can then be substituted back into the function to compute its optimal values.

Although these solutions can sometimes be obtained analytically, most practical optimization problems require numerical, computer solutions. From a numerical standpoint, optimization is similar in spirit to the root-location methods we just covered in [Chaps. 5](#) and [6](#). That is, both involve guessing and searching for a point on a function. The fundamental difference between the two types of problems is shown in [Fig. 7.2](#). Root location involves searching for the location where the function equals zero. In contrast, optimization involves searching for the function's extreme points.

**FIGURE 7.2** A function of a single variable illustrating the difference between roots and optima.



As can be seen in [Fig. 7.2](#), the optima are the points where the curve is flat. In mathematical terms, this corresponds to the  $x$  value where the derivative,  $f'(x)$ , is equal to zero. Additionally, the second derivative,  $f''(x)$ , indicates whether the optimum is a minimum or a maximum: if  $f''(x) < 0$ , the point is a maximum; if  $f''(x) > 0$ , the point is a minimum.

Now, understanding the relationship between roots and optima would suggest a possible strategy for finding the latter. That is, you can differentiate the function and locate its root, that is, the zero, of the new function. In fact, some optimization methods do just this by solving the root problem,  $f'(x) = 0$ .

## EXAMPLE 7.1

### Determining the Optimum Analytically by Root Location

#### Problem Statement

Determine the time and magnitude of the peak elevation based on [Eq. \(7.1\)](#). Use the following parameter values for your calculation:  $g = 9.81 \text{ m/s}^2$ ,  $z_0 = 100 \text{ m}$ ,  $v_0 = 55 \text{ m/s}$ ,  $m = 80 \text{ kg}$ , and  $c = 15 \text{ kg/s}$ .

#### Solution

[Eq. \(7.1\)](#) can be differentiated to give

$$\frac{dz}{dt} = v_0 e^{-\frac{t}{m/c}} - \frac{mg}{c} \left(1 - e^{-\frac{t}{m/c}}\right) \quad (\text{E7.1.1})$$

Note that because  $v = dz/dt$ , this is actually the equation for the velocity. The maximum elevation occurs at the value of  $t$  that drives the equation to zero. Thus, the problem amounts to determining the root. For this case, this can be accomplished by setting the derivative to zero and solving

[Eq. \(E7.1.1\)](#) analytically for

$$t = \frac{m}{c} \ln \left(1 + \frac{cv_0}{mg}\right)$$

Substituting the parameters gives

$$t = \frac{80}{15} \ln \left(1 + \frac{(15)(55)}{(80)(9.81)}\right) \cong 3.83166 \text{ s}$$

This value along with the parameters can then be substituted into [Eq. \(7.1\)](#) to compute the maximum elevation as

$$z = 100 + \frac{80}{15} \left(50 + \frac{(80)(9.81)}{15}\right) \left(1 - e^{-\frac{15}{80/3.83166}}\right) - \frac{(80)(9.81)}{15} 3.83166 = 192.860$$

We can verify that the result is a maximum by differentiating [Eq. \(E7.1.1\)](#) to obtain the second derivative.

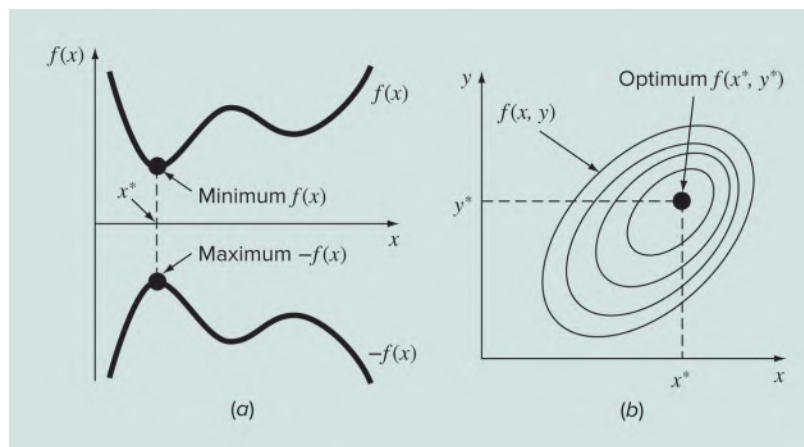
$$\frac{d^2z}{dt^2} = \left(-\frac{c}{m}v_0 - g\right)e^{-\frac{t}{m/c}} = -9.81 \frac{\text{m}}{\text{s}^2}$$

The fact that the second derivative is negative tells us that we have a maximum. Further, the result makes physical sense since the acceleration should be solely equal to the effect of gravity at the maximum when the vertical velocity (and hence drag) is zero.

Although an analytical solution was possible for this case, we could have obtained the same result using the root-finding methods described in [Chaps. 5](#) and [6](#). This will be left as a homework exercise.

Although it is certainly possible to approach optimization as a roots problem, a variety of direct numerical optimization methods are available. These methods are available for both one-dimensional and multidimensional problems. As the name implies, one-dimensional problems involve functions or calculations in terms of a single independent variable. As shown in [Fig. 7.3a](#), the search then consists of climbing or descending one-dimensional peaks and valleys. Multidimensional problems involve functions or calculations that involve two or more independent variables.

**FIGURE 7.3** (a) One-dimensional optimization. This figure also illustrates how minimization of  $f(x)$  is equivalent to the maximization of  $-f(x)$ . (b) Two-dimensional optimization. Note that this figure can be taken to represent either a maximization scenario (contours increase in elevation up to the peak of a mountain) or a minimization case (contours decrease in elevation down to the lowest point of a valley).



In the same spirit, a two-dimensional optimization can again be visualized as searching out peaks and valleys ([Fig. 7.3b](#)). However, just as in hiking, we are not constrained to walk in a single direction. Instead, the topography is examined to reach the goal efficiently.

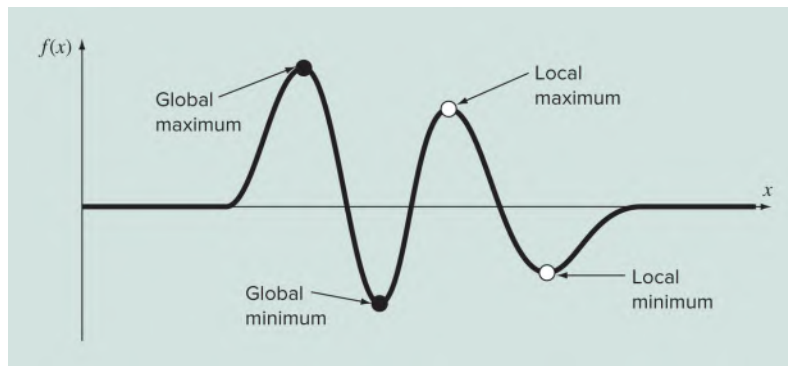
Finally, the process of finding a minimum versus finding a maximum is essentially identical because the same value of  $x^*$  both minimizes  $f(x)$  and maximizes  $-f(x)$ . This equivalence is illustrated graphically for a one-dimensional function in [Fig. 7.3a](#).

In the next section, we will describe common approaches for one-dimensional optimization. Then, we will provide a brief description of how Python and the SciPy `optimize` submodule can be employed to determine optima for one-dimensional functions.

## 7.2 ONE-DIMENSIONAL OPTIMIZATION

This section will describe techniques to find the minimum or maximum of a function of a single variable  $f(x)$ . A useful image in this regard is the one-dimensional “roller coaster,” like the function depicted in [Fig. 7.4](#). Recall from [Chaps. 5](#) and [6](#) that root location was complicated by the fact that several roots can occur for a single function. Similarly, both local and global optima can occur in optimization.

**FIGURE 7.4** A function that asymptotically approaches zero at  $\pm \infty$  and has two maximum and two minimum points in the vicinity of  $x = 0$ . The two points to the right are local optima, whereas the two to the left are global.



A *global optimum* represents the very best solution. A *local optimum*, though not the very best, is the best solution in a neighborhood around it. Cases that include local optima are called *multimodal*. In such cases, we will commonly be interested in finding a global optimum, as long as it represents a feasible solution. In addition, we must be concerned about mistaking a local result for the global optimum.

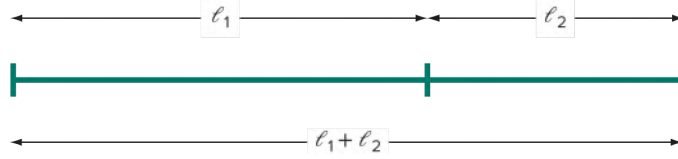
Just as in root location, optimization in one dimension can be divided into bracketing and open methods. As described next, the golden-section search is an example of a bracketing method that is similar in spirit to the bisection method for root location. This is followed by a more sophisticated bracketing approach—parabolic interpolation. We will then illustrate the application of the `minimize_scalar` function from the Python SciPy module’s `optimize` submodule.

## 7.2.1 Golden-Section Search

In many cultures, certain numbers are ascribed magical qualities. For example, in the West, we are familiar with “lucky 7” and “Friday the 13<sup>th</sup>.” Beyond such superstitious quantities, there are several well-known numbers that have such interesting and powerful mathematical properties that they could truly be termed “magical.” The most common of these are the ratio of a circle’s circumference to its diameter,  $\pi$ , and the base of the Napierian natural logarithm,  $e$ .

Although not so widely known, the *golden ratio* should surely be included in the pantheon of remarkable numbers. This number, which is typically represented by the Greek character  $\phi$  (phonetically phi, properly pronounced “fee”) was originally defined by Euclid (ca. 300 BCE) because of its role in the construction of the pentagram or five-pointed star. As depicted in  Fig. 7.5, Euclid’s definition reads “A straight line is said to have been cut in extreme and mean ratio when, as the whole line is to the greater segment, so is the greater to the lesser.”

**FIGURE 7.5** Euclid’s definition of the golden ratio is based on dividing a line into two segments so that the ratio of the whole line to the larger segment is equal to the ratio of the larger segment to the smaller segment. This ratio is called the golden ratio.



The value of the golden ratio can be derived by expressing Euclid’s definition, with regard to  Fig. 7.5, as

$$\frac{l_1 + l_2}{l_1} = \frac{l_1}{l_2} = \phi \quad (7.2)$$

which can be rearranged to yield

$$\phi^2 - \phi - 1 = 0 \quad (7.3)$$

The positive root of this quadratic equation is

$$\phi = \frac{1 + \sqrt{5}}{2} \cong 1.61803 \quad (\text{the golden ratio}) \quad (7.4)$$

An important proportionality property is illustrated by

$$\frac{1}{\phi} = 1 - \phi \cong 0.61803$$

Also, the latter can be computed by

$$\frac{\sqrt{5} - 1}{2} \cong 0.61803$$

further evidence that the golden ratio is truly a magical number.

The golden ratio has long been considered aesthetically pleasing in Western cultures. It also arises in a variety of other contexts, including biology.<sup>1</sup> For our purposes here, it provides the basis for the golden-section search, a simple, general-purpose method for determining the optimum of a single-variable function.

The golden-section search is similar in spirit to the bisection approach for locating roots in [Chap. 5](#). Recall that bisection hinged on defining an interval, specified by a lower guess,  $x_l$ , and an upper guess,  $x_u$ , that bracketed a single root. The presence of a root between these bounds was verified by determining that  $f(x_l)$  and  $f(x_u)$  had different signs. The root was then estimated as the midpoint of this interval:

$$x_r = \frac{x_l + x_u}{2} \quad (7.5)$$

The last step in an iteration of the bisection method involved determining a new smaller bracket. This was done by replacing whichever of the bounds,  $x_l$  or  $x_u$ , had a function value with the same sign as  $f(x_r)$ . A key advantage of this approach was that the new value,  $x_r$ , replaced one of the old bounds.

Page 180

Now suppose that, instead of a root, we were interested in determining the minimum of a one-dimensional function,  $f(x)$ . As with bisection, we can start by defining an interval that contains a single answer. That is, the interval should contain a single minimum, and hence is called *unimodal*. We can adopt the same nomenclature as for bisection, where  $x_l$  and  $x_u$  define the lower and upper bounds of such an interval. However, in contrast to bisection, we need a new strategy for finding a minimum within the interval. Rather than using a single intermediate value, which is sufficient to detect a sign change, we will need two intermediate function values to detect a subinterval in which the minimum is still contained. If one of the intermediate function values is greater than the other, this allows us to eliminate the interval between that point and the bound, as we will illustrate below.

The key to making this approach efficient is the wise choice of the intermediate points. As in bisection, the goal is to minimize function evaluations by replacing old bound values with new ones. For bisection, this was accomplished by choosing the midpoint. For the golden-section search, the two intermediate points are chosen according to the golden ratio:

$$x_1 = x_l + d \quad (7.6)$$

and

$$x_2 = x_u - d \quad (7.7)$$

where

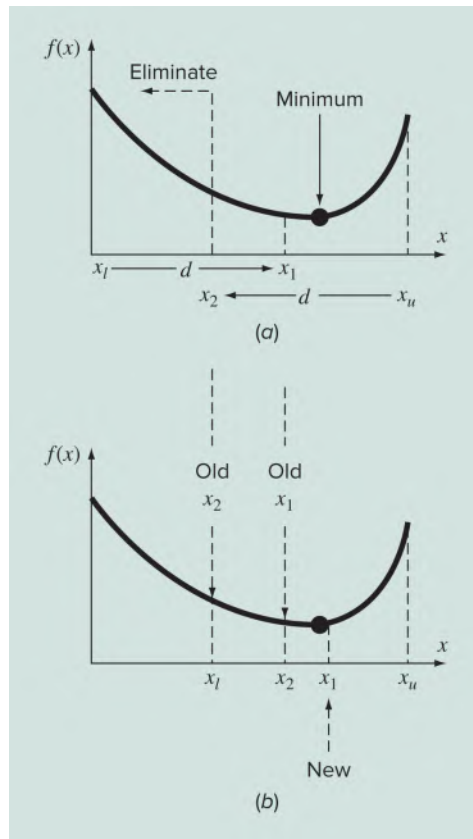
$$d = (1 - \phi)(x_u - x_l) \quad (7.8)$$



As noted in [Fig. 7.6](#), the subintervals overlap, that is,  $x_2$  is less than  $x_1$ . The function is evaluated at the two interior points. Two results can occur for the unimodal scenario<sup>2</sup>:

1. If, as in [Fig. 7.6a](#),  $f(x_1) < f(x_2)$ , then the minimum is contained in the subinterval  $(x_2, x_u)$  and the subinterval  $(x_l, x_2)$  can be eliminated from consideration. In this case,  $x_2$  becomes  $x_l$  for the next round of the method.
2. If  $f(x_2) < f(x_1)$ , then the minimum is contained in the interval  $(x_l, x_1)$  and the subinterval  $(x_1, x_u)$  can be eliminated. In this case,  $x_1$  becomes the  $x_u$  for the next round.

**FIGURE 7.6** (a) The initial step of the golden-section search algorithm involves choosing two interior points according to the golden ratio. (b) The second step defines a new interval that still encompasses the optimum.



We could have used any value of  $d$  greater than 0.5 providing the overlap shown in the figure, but employing the golden ratio provides an important benefit. Because  $x_1$  and  $x_2$  were chosen using the golden ratio, we do not have to recalculate both function values for the next iteration. In the case illustrated in [Fig. 7.6](#), the old  $x_1$  becomes the new  $x_u$  for the next iteration. This leads to already having the value for  $f(x_2)$ , since it is the same as the function value at the old  $x_1$ .

To complete the algorithm, we need only to determine the new  $x_1$  and its function value,  $f(x_1)$ . This is done with [Eq. \(7.6\)](#) with  $d$  computed with [Eq. \(7.8\)](#) based on the new values of  $x_l$  and  $x_u$ . A similar approach would be used for the alternate case where the optimum falls in the left subinterval. For this case, the new  $x_2$  would be computed with [Eq. \(7.7\)](#).

As the iterations are repeated, the interval containing the optimum is reduced rapidly. In fact, after each round, the interval is reduced by a factor of  $1/\phi \cong 61.8\%$ . That means after ten iterations, the interval is shrunk to  $(1/\phi)^{10} \cong 0.8\%$  or one part in 125 of the original interval. Correspondingly, after 20 iterations, that would be 1 part in about 15,000 of the original interval.<sup>3</sup>

## EXAMPLE 7.2

### Golden-Section Search

#### Problem Statement

Use the golden-section search method to find the minimum of

$$f(x) = \frac{x^2}{10} - 2 \sin(x)$$

within the interval from  $x_l = 0$  to  $x_u = 4$ .

#### Solution

First, the golden ratio is used to create the two interior points:

$$\begin{aligned} d &= 0.61803(4 - 0) = 2.4721 \\ x_1 &= 0 + 2.4721 = 2.4721 \\ x_2 &= 4 - 2.4721 = 1.5279 \end{aligned}$$

The function is then evaluated at the two interior points:

$$\begin{aligned} f(x_2) &= \frac{1.5279^2}{10} - 2 \sin(1.5279) = -1.7647 \\ f(x_1) &= \frac{2.4721^2}{10} - 2 \sin(2.4721) = -0.6300 \end{aligned}$$

Because  $f(x_2) < f(x_1)$ , we conclude that the minimum is contained in the interval  $(x_l, x_1) = (0, 2.4721)$ . These define the bounds for the next iteration. And  $x_2$  will become the new  $x_1$  for that iteration.

All that remains then is to use [Eqs. \(7.8\)](#) and [\(7.7\)](#) to compute the new value of  $d$  and  $x_2$ :

$$\begin{aligned} d &= 0.61803(2.4721 - 0) = 1.5279 \\ x_2 &= 2.4721 - 1.5279 = 0.9443 \end{aligned}$$

The function value at the new  $x_2$  is  $f(0.9443) = -1.5310$ . Since this value is less than the value at  $x_1$ , we find that the minimum must be in the reduced interval  $(0.9443, 2.4721)$ . The algorithm can be repeated with the results shown below. The lowest value of  $f(x)$  is highlighted in the table for each iteration.

| Iteration                 | $x_l$   | $x_2$   | $f(x_2)$ | $x_1$   | $f(x_1)$ | Golden<br>Ratio | 1.61803 |
|---------------------------|---------|---------|----------|---------|----------|-----------------|---------|
|                           |         |         |          |         |          | $x_u$           | $d$     |
| 1                         | 0       | 1.52786 | -1.76472 | 2.47214 | -0.62997 | 4               | 2.47214 |
| 2                         | 0       | 0.94427 | -1.53098 | 1.52786 | -1.76472 | 2.47214         | 1.52786 |
| 3                         | 0.94427 | 1.52786 | -1.76472 | 1.88854 | -1.54322 | 2.47214         | 0.94427 |
| 4                         | 0.94427 | 1.30495 | -1.75945 | 1.52786 | -1.76472 | 1.88854         | 0.58359 |
| 5                         | 1.30495 | 1.52786 | -1.76472 | 1.66563 | -1.71358 | 1.88854         | 0.36068 |
| 6                         | 1.30495 | 1.44272 | -1.77547 | 1.52786 | -1.76472 | 1.66563         | 0.22291 |
| 7                         | 1.30495 | 1.39010 | -1.77420 | 1.44272 | -1.77547 | 1.52786         | 0.13777 |
| 8                         | 1.39010 | 1.44272 | -1.77547 | 1.47524 | -1.77324 | 1.52786         | 0.08514 |
| and, after 20 iterations, |         |         |          |         |          |                 |         |
| 15                        | 1.42555 | 1.42736 | -1.77573 | 1.42848 | -1.77572 | 1.43030         | 0.00293 |
| 16                        | 1.42555 | 1.42667 | -1.77572 | 1.42736 | -1.77573 | 1.42848         | 0.00181 |
| 17                        | 1.42667 | 1.42736 | -1.77573 | 1.42779 | -1.77573 | 1.42848         | 0.00112 |
| 18                        | 1.42667 | 1.42710 | -1.77573 | 1.42736 | -1.77573 | 1.42779         | 0.00069 |
| 19                        | 1.42710 | 1.42736 | -1.77573 | 1.42753 | -1.77573 | 1.42779         | 0.00043 |
| 20                        | 1.42736 | 1.42753 | -1.77573 | 1.42763 | -1.77573 | 1.42779         | 0.00026 |

We can see that the minimum occurs between  $x_2 = 1.42753$  and  $x_1 = 1.42763$ , and the interval width for the next iteration would be 0.0001. This level of precision in determining the minimum would be sufficient for most engineering and scientific purposes.

Recall that for bisection ([Sec. 5.4](#)), an exact upper bound on the error can be calculated at each iteration. Using similar reasoning, an upper bound for golden-section search can be derived as follows:

Once an iteration is complete, the optimum will fall in one of the two subintervals. If the comparative minimum function value is at  $x_2$ , it will be in the lower interval  $(x_l, x_2, x_1)$ . Alternately, if that minimum function value is at  $x_1$ , it will be in the upper interval  $(x_2, x_1, x_u)$ . Because the interior points are symmetrical, either case can be used to define the error.

Looking at the upper interval,  $(x_2, x_1, x_u)$ , if the true value were at the far left, the maximum distance from the estimate would be

$$\begin{aligned}
\Delta x_a &= x_1 - x_2 \\
&= x_l + (\phi - 1)(x_u - x_l) - x_u + (\phi - 1)(x_u - x_l) \\
&= -(x_u - x_l) + 2(\phi - 1)(x_u - x_l) \\
&= (2\phi - 3)(x_u - x_l)
\end{aligned}$$

or about  $0.2361 \times (x_u - x_l)$ . If the true value were at the far right, the maximum distance from the estimate would be

$$\begin{aligned}
\Delta x_b &= x_u - x_1 \\
&= x_u - x_l - (\phi - 1)(x_u - x_l) \\
&= (x_u - x_l) - (\phi - 1)(x_u - x_l) \\
&= (2 - \phi)(x_u - x_l)
\end{aligned}$$

or approximately  $0.3820 \times (x_u - x_l)$ . Therefore, this case would represent the maximum error. This result can then be normalized to the minimum value for that iteration,  $x_{\text{opt}}$  to yield

$$\varepsilon_a = (2 - \phi) \left| \frac{x_u - x_l}{x_{\text{opt}}} \right| \quad (7.9)$$

This error estimate provides a basis for terminating the iterations.

A Python function, `goldmin`, is presented in  **Fig. 7.7**. The function returns the location of the minimum, the value of the function, the approximate error, and the number of iterations.

---

**FIGURE 7.7** A Python script to determine the minimum of a function with the golden-section search.

```

import numpy as np

def goldmin(f,xl,xu,Ea=1.e-7,maxit=30):
    """
    use the golden-section search to find the minimum of f(x)
    input:
        f = name of the function
        xl = lower initial guess
        xu = upper initial guess
        Ea = absolute relative error criterion (default = 1.e-7)
        maxit = maximum number of iterations (default = 30)
    output:
        xopt = location of the minimum
        f(xopt) = function value at the minimum
        ea = absolute relative error achieved
        i+1 = number of iterations required
    """
    phi = (1+np.sqrt(5))/2
    d = (phi - 1)*(xu-xl)
    x1 = xl + d ; f1 = f(x1)
    x2 = xu - d ; f2 = f(x2)
    for i in range(maxit):
        xint = xu - x1
        if f1 < f2:
            xopt = x1
            x1 = x2

```

```

        x2 = x1
        f2 = f1
        x1 = x1 + (phi-1)*(xu-x1)
        f1 = f(x1)
    else:
        xopt = x2
        xu = x1
        x1 = x2
        f1 = f2
        x2 = xu - (phi-1)*(xu-x1)
        f2 = f(x2)
    if xopt != 0:
        ea = (2-phi)*abs(xint/xopt)
        if ea <= Ea: break
    return xopt,f(xopt),ea,i+1

```

The `goldmin` function, with the following Python code, can be used to solve the problem from

#### **Example 7.1.**

```

g = 9.81 # m/s2
v0 = 55 # m/s
m = 80 # kg
c = 15 # kg/s
z0 = 100 # m

def f(t):
    return -(z0+m/c*(v0+m*g/c)*(1-np.exp(-t/(m/c)))-m*g/c*t)

t1 = 0
tu = 8
tmin,fmin,ea,n = goldmin(f,t1,tu,Ea=1.e-5)
print('Time at maximum altitude = {0:5.2f} s'.format(tmin))
print('Function value = {0:6.2g} '.format(fmin))
print('Relative error = {0:7.2e} '.format(ea))
print('Iterations required = {0:4.0f} '.format(n))

zmax = z0 + m/c*(v0+m*g/c)*(1-np.exp(-tmin/(m/c)))-m*g/c*tmin
print('Maximum altitude = {0:6.2f} m'.format(zmax))

```

The results displayed are

```

Time at maximum altitude = 3.83 s
Function value = -1.9e+02
Relative error = 7.69e-06
Iterations required = 25
Maximum altitude = 192.86 m

```

Notice how, because this is a maximization (maximum altitude), we have entered the negative of

 **Eq. (7.1)**. Consequently, `fmin` corresponds to an altitude of about 193 meters.

You may have been wondering why we have stressed the reduced function evaluations of the golden-section search. Of course, for solving an optimization problem one time, the speed savings would likely be negligible. However, there are two important contexts where minimizing the number of function evaluations can be important. These are as follows:

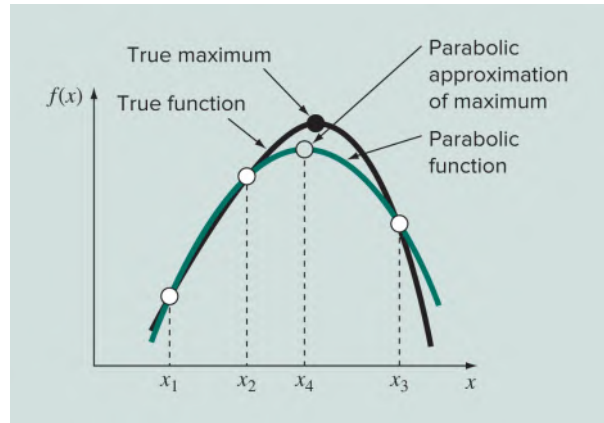
1. Many evaluations. There are cases where an optimization calculation, like the golden-section search, may be part of a much larger computational task. The search method may be called thousands of times. A focus on efficiency by minimizing the number of function evaluations could pay great dividends for such cases.
2. Time-consuming evaluation. For pedagogical reasons, we use simple functions in most of our examples. You should understand that a function may be complex and time consuming to evaluate. It may require hundreds of lines of code or more. For example, optimization is used to estimate the

parameters of a model consisting of a system of differential equations. Here, the “function” involves time-consuming numerical integration of the model. Any optimization method that minimizes evaluations would be advantageous.

## 7.2.2 Parabolic Interpolation

Parabolic interpolation takes advantage of the fact that a second-order polynomial often provides a good approximation to the shape of  $f(x)$  near an optimum (🔗 Fig. 7.8).

**FIGURE 7.8** Graphical depiction of parabolic interpolation.



Just as there is only one straight line connecting two points, there is only one parabola connecting three points. Thus, if we have three points that jointly bracket an optimum, we can fit a parabola to the points. Then, we can differentiate it, set the result equal to zero, and solve for an estimate of the optimal  $x$ . It can be shown through algebraic manipulation that the result is

$$x_4 = x_2 - \frac{1}{2} \frac{(x_2 - x_1)^2 [f(x_2) - f(x_3)] - (x_2 - x_3)^2 [f(x_2) - f(x_1)]}{(x_2 - x_1)[f(x_2) - f(x_3)] - (x_2 - x_3)[f(x_2) - f(x_1)]} \quad (7.10)$$

Page 185

where  $x_1$ ,  $x_2$ , and  $x_3$  are the initial guesses, and  $x_4$  is the value of  $x$  that corresponds to the optimum value of the parabolic fit to the guesses.

### EXAMPLE 7.3

#### Parabolic Interpolation

##### Problem Statement

Use parabolic interpolation to approximate the minimum of

$$f(x) = \frac{x^2}{10} - 2 \sin(x)$$

with initial guesses  $x_1 = 0$ ,  $x_2 = 1$ , and  $x_3 = 4$ .

## Solution

The function values at the three initial guesses can be evaluated as

$$\begin{array}{ll} x_1 = 0 & f(x_1) = 0 \\ x_2 = 1 & f(x_2) \cong -1.5829 \\ x_3 = 4 & f(x_3) \cong 3.1136 \end{array}$$

and substituted into [Eq. \(7.10\)](#) to give

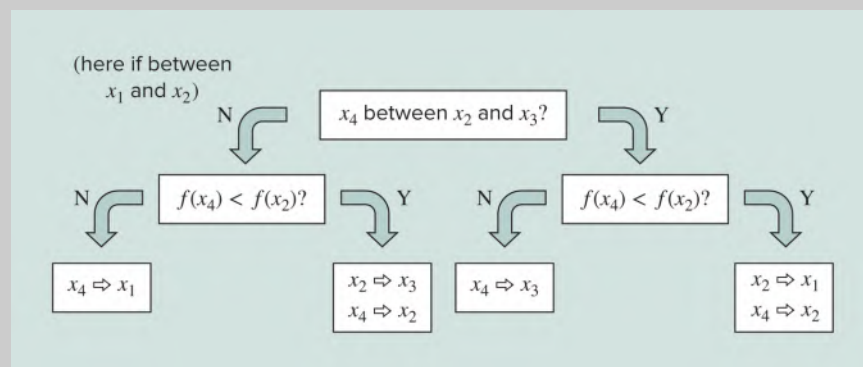
$$x_4 = x_2 - \frac{1}{2} \frac{(1-0)^2[-1.5829-3.1136] - (1-4)^2[-1.5829-0]}{(1-0)[-1.5829-3.1136] - (1-4)[-1.5829-0]} \cong 1.5055$$

which has a function value  $f(1.5055) \cong -1.7691$ .

Next, a similar strategy to the golden-section search can be employed to determine which point should be discarded. This is described in [Fig. 7.9](#). Because the function value for the new point,  $x_4$ , is lower than for the intermediate point,  $x_2$ , and is to the right of that point, the low initial guess,  $x_1$ , is discarded, and the  $x_4$  result becomes the new intermediate guess for the next round. Here are the results for that iteration:

$$\begin{array}{ll} x_1 = 1 & f(x_1) \cong -1.5829 \\ x_2 = 1.5055 & f(x_2) \cong -1.7691 \\ x_3 = 4 & f(x_3) \cong 3.1136 \end{array}$$

**FIGURE 7.9** Strategy for determining the new set of points,  $x_1$ ,  $x_2$ , and  $x_3$ , for the next iteration.



Substituting these values into [Eq. \(7.10\)](#) yields a value  $x_4 \cong 1.4903$ , which has a function value of  $-1.7714$ . This process can be repeated with the results for eight iterations presented in the table on the next page.

| Iteration | $x_1$   | $f(x_1)$ | $x_2$   | $f(x_2)$ | $x_3$   | $f(x_3)$ | $x_4$   |
|-----------|---------|----------|---------|----------|---------|----------|---------|
| 1         | 0       | 0        | 1       | -1.58294 | 4       | 3.11360  | 1.50553 |
| 2         | 1       | -1.58294 | 1.50553 | -1.76908 | 4       | 3.11360  | 1.49025 |
| 3         | 1       | -1.58294 | 1.49025 | -1.77143 | 1.50553 | -1.76908 | 1.42564 |
| 4         | 1       | -1.58294 | 1.42564 | -1.77572 | 1.49025 | -1.77143 | 1.42660 |
| 5         | 1.42564 | -1.77572 | 1.42660 | -1.77572 | 1.49025 | -1.77143 | 1.42755 |
| 6         | 1.42660 | -1.77572 | 1.42755 | -1.77573 | 1.49025 | -1.77143 | 1.42755 |
| 7         | 1.42755 | -1.77573 | 1.42755 | -1.77573 | 1.49025 | -1.77143 | 1.42755 |
| 8         | 1.42755 | -1.77573 | 1.42755 | -1.77573 | 1.49025 | -1.77143 | 1.42755 |

Examining the results, we see that the convergence is rapid and reaches a value of  $x_4 \cong 1.42755$  in the fifth iteration.

## 7.2.3 Python's SciPy Function: `minimize_scalar`

Recall that in [Sec. 6.4](#), we described Brent's method for root location, which combined three root-finding methods in a single algorithm that balanced reliability with efficiency. Because of these qualities, it forms the basis for the `roots` function in the NumPy module.

Brent also developed a similar approach for one-dimensional minimization which is the default solving option in the `minimize_scalar` function in the `optimize` submodule of Python's SciPy module. Brent's algorithm combines the slower, dependable golden-section search with the faster, but unreliable, parabolic interpolation. It first attempts parabolic interpolation and keeps applying it as long as acceptable results are obtained. If not, it reverts to the golden-section search to get matters in hand.

The `minimize_scalar` function will use Brent's algorithm by default, but it is also possible to specify only the golden-section search method. The syntax for using the function is

```
from scipy.optimize import minimize_scalar
result = minimize_scalar(funx)
xmin = result.x
```

where `result` is an *OptimizeResult* object with various attributes. The most important of these is `x`, here providing `xmin`, the solved minimum for `x`.

In this case, the `minimize_scalar` algorithm will determine its own bracketing values. It is also possible to specify those as

```
xmin = minimize_scalar(funx, bracket=(x1, x2, x3))
```

where, `x1`, `x2`, and `x3` are specified for parabolic interpolation or, more simply,

```
xmin = minimize_scalar(funx, bracket=(x1, x2))
```



where the middle point is left out and is calculated as the midpoint of  $x_1$  and  $x_2$ .

If you want to restrict the function to the golden-section search, you can use

```
xmin = minimize_scalar(funx, bracket=(x1, x2), method='golden')
```

Additional arguments are possible, including specifying the convergence tolerance, for example, `tol=1.e-7`, and the maximum number of iterations via the `options` dictionary, for example, `options={'maxiter':20}`.

Here is a Python script that uses `minimize_scalar` to solve the problem from  **Example 7.1**.

```
import numpy as np
from scipy.optimize import minimize_scalar

g = 9.81 # m/s2
v0 = 55 # m/s
m = 80 # kg
c = 15 # kg/s
z0 = 100 # m

def f(t):
    return -(z0+m/c*(v0+m*g/c)*(1-np.exp(-t/(m/c)))-m*g/c*t)
result = minimize_scalar(f)
tmin = result.x

print('Time at maximum altitude = {0:5.2f} s'.format(tmin))
zmax = z0 + m/c*(v0+m*g/c)*(1-np.exp(-tmin/(m/c)))-m*g/c*tmin
print('Maximum altitude = {0:6.2f} m'.format(zmax))
```

with the results

---

Page 187

---

```
Time at maximum altitude = 3.83 s
Maximum altitude = 192.86 m
```

Additional attributes of `result` include

- `success`: a Boolean flag indicating if the solution was successful
- `nit`: number of iterations required

Here is code that has additional specifications and provides more results:

```
result = minimize_scalar(f, bracket=(0, 8),
                        tol=1.e-7, options={'maxiter':20})
tmin = result.x
flag = result.success
n = result.nit

print('Time at maximum altitude = {0:5.2f} s'.format(tmin))
zmax = z0 + m/c*(v0+m*g/c)*(1-np.exp(-tmin/(m/c)))-m*g/c*tmin
print('Maximum altitude = {0:6.2f} m'.format(zmax))
print('Success indicator:', flag)
print('Number of iterations required =', n)
```

with the output in the Console:

```
Time at maximum altitude = 3.83 s
Maximum altitude = 192.86 m
Success indicator True
Number of iterations required = 11
```

Details on `minimize_scalar` and all its features and options are available at

 [https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize\\_scalar.html?highlight=t=minimize\\_scalar#scipy.optimize.minimize\\_scalar](https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize_scalar.html?highlight=t=minimize_scalar#scipy.optimize.minimize_scalar)

You will notice that, in our call to `minimize_scalar` above, we only provided the name of the function. We let the algorithm figure out its initial guesses, and, evidently, it was successful.

## 7.3 MULTIDIMENSIONAL OPTIMIZATION

Aside from one-dimensional functions, optimization often deals with multidimensional functions. Recall from [Fig. 7.3a](#) that our visual image of a one-dimensional search was like a roller coaster. For two-dimensional cases, we can imagine mountains and valleys on a topographic map ([Fig. 7.3b](#)). As we proceed to three independent variables and more, we lose the ability to visualize the situation. For now, we can use Python's capabilities to picture two-dimensional functions.

### EXAMPLE 7.4

#### Visualizing a Two-Dimensional Function

##### Problem Statement

Use Python's graphical capabilities to display the following function and estimate visually its minimum in the range  $-2 \leq x_1 \leq 0$  and  $0 \leq x_2 \leq 3$ :

$$f(x_1, x_2) = 2 + x_1 - x_2 + 2x_1^2 + 2x_1x_2 + x_2^2$$

##### Solution

To create 3D and contour plots, we need to move on from the pylab module to its "big sister" Matplotlib. We will be introducing the use of Matplotlib gradually through the rest of the chapters of the book<sup>4</sup>. In this case, we use a submodule called `pyp1ot`. Here is the code that produces the plots in Fig. 2.9.

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

x1 = np.linspace(-2,0,20)
x2 = np.linspace(0,3,20)
X,Y = np.meshgrid(x1,x2)
Z = 2 + X - Y + 2*X**2 + 2*X*Y + Y**2

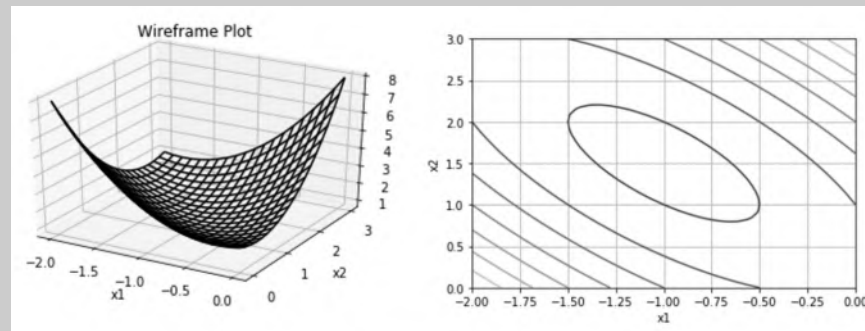
fig = plt.figure()
ax = plt.subplot(111)
ax = fig.add_subplot(111,projection='3d')
ax.plot_wireframe(X,Y,Z,color='k')
ax.set_xticks([-2,-1.5,-1,-0.5,0])
ax.set_yticks([0,1,2,3])
ax.set_xlabel('x1')
ax.set_ylabel('x2')
ax.set_title('Wireframe Plot')
plt.show()

fig2 = plt.figure()
ax2 = fig2.add_subplot(111)
ax2.contour(X,Y,Z)
ax2.set_xlabel('x1')
ax2.set_ylabel('x2')
```

```
ax2.grid()
plt.show()
```

As displayed in [Fig. 7.10](#), both plots lead us to conclude that the function has minimum value between 0 and 1 located at about  $x_1 = 1$  and  $x_2 = 1.5$ .

**FIGURE 7.10** Mesh and contour plots of a two-dimensional function.



Techniques for multidimensional unconstrained optimization can be categorized in several ways. For purposes of the present discussion, we will divide them depending on whether they require derivative evaluation. Those that require derivatives are called *gradient* or *steepest descent* (or ascent) methods. The approaches that do not require derivative calculations are called *nongradient* or *direct* methods. In the next section, we will describe the Python `minimize` function from the SciPy module which, depending on the options specified, can take on either personality. Additionally, we mention another class of optimization methods that incorporate constraints, both inequality and equality, as part of the problem description. Although this is an important class, we do not discuss these here.

## 7.3.1 Python SciPy `minimize` Function

The `minimize` function is the “big brother” of the `minimize_scalar` function in the `optimize` submodule of SciPy. The syntax for using the function is

```
from scipy.optimize import minimize
result = minimize(f,x0)
xmin = result.x
```

where `xmin` is the value of  $x$  that yields the minimum of  $f$ , the name of the function for  $f(x)$ .

Page 189

The initial guess is `x0`. For multidimensional optimization, `x0` is an array of initial guesses and `xmin` is an array of independent variable values for the minimum. The object `result` contains numerous attributes of the solution. Here, we show the `x` attribute, that is, the solution.

The `minimize` function is powerful and versatile. It provides for a choice from 14 different solving methods by the additional `method='solvername'` argument. Here, we illustrate the solution of the problem from [Example 7.4](#) using the Nelder-Mead algorithm, which is a direct-search method that uses only function values (does not require derivatives) and handles non-smooth functions.

```

from scipy.optimize import minimize

def f(x):
    x1 = x[0]
    x2 = x[1]
    return 2+x1-x2+2*x1**2+2*x1*x2+x2**2

x0 = [-0.5,0.5]
result = minimize(f,x0,method='Nelder-Mead',options={'disp':False})
xval = result.x
print(xval)

```

with the displayed output

```
[-0.99996784  1.49997544]
```

It is evident that the solution is converging on  $x_1 = -1$  and  $x_2 = 1.5$ , values we could have obtained analytically using calculus. If `disp` in the `options` dictionary is set to `True`, we get additional information:

```

Optimization terminated successfully.
Current function value: 0.750000
Iterations: 40
Function evaluations: 76

```

The variety of algorithms for the solution of multidimensional optimization problems, as evidenced by the 14 available in the `minimize` function, represents an interesting area of study in the numerical methods field. In fact, entire books have been dedicated to this topic. By necessity, we have chosen to select only a sample here, but we certainly encourage you to expand your knowledge in this area.

## EQUILIBRIUM AND MINIMUM POTENTIAL ENERGY

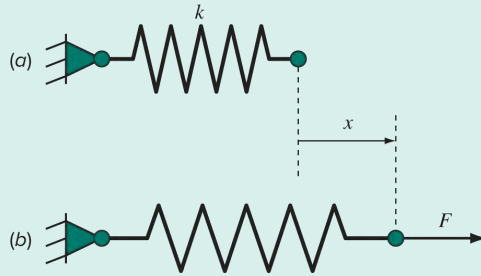
### Background

As shown in [Fig. 7.11a](#), an unloaded spring can be attached to a wall mount. When a horizontal force is applied, the spring stretches. The displacement is related to the force by Hooke's law,  $F = kx$ . The potential energy of the deformed state consists of the difference between the strain energy of the spring and the work done by the force:

$$PE(x) = 0.5kx^2 - Fx \quad (7.11)$$

Page 190

**FIGURE 7.11** (a) An unloaded spring attached to a wall mount. (b) Application of a horizontal force stretches the spring where the relationship between force and displacement is described by Hooke's law.

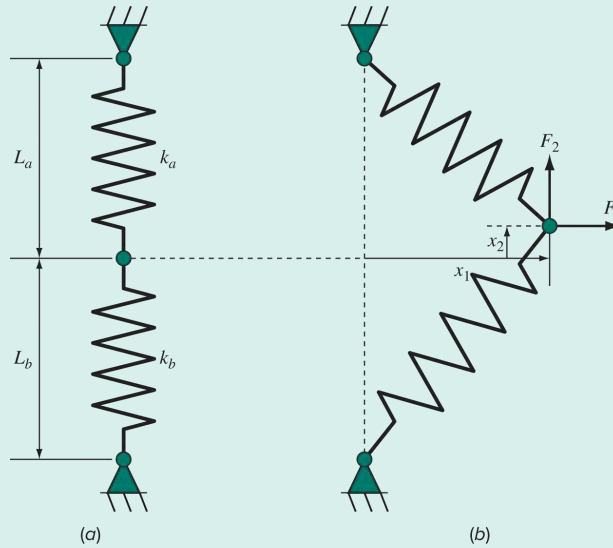


[Equation \(7.11\)](#) defines a parabola. Since the potential energy will be at a minimum at equilibrium, the solution for the displacement can be viewed as a one-dimensional optimization problem. Because this equation is easily differentiated, we can solve for the displacement as  $F' = x/K$ , which is Hooke's law. For example, if  $k = 2 \text{ N/cm}$  and  $F = 5 \text{ N}$ , then  $x = 2.5 \text{ cm}$ .

A more interesting two-dimensional case is shown in [Fig. 7.12](#). In this system, there are two degrees of freedom in that the system is free to move both horizontally and vertically. In the same way that we approached the one-dimensional scenario, the equilibrium deformations are the values of  $x_1$  and  $x_2$  that minimize potential energy:

$$PE(x_1, x_2) = 0.5 k_a \left( \sqrt{x_1^2 + (L_a - x_2)^2} - L_a \right) + 0.5 k_b \left( \sqrt{x_1^2 + (L_b + x_2)^2} - L_b \right) - F_1 x_1 -$$

**FIGURE 7.12** A two-spring system: (a) unloaded and (b) loaded.



If the parameter values are

$k_a = 9 \text{ N/cm}$ ,  $k_b = 2 \text{ N/cm}$ ,  $L_a = 10 \text{ cm}$ ,  $L_b = 10 \text{ cm}$ ,  $F_1 = 2 \text{ N}$ ,  $F_2 = 4 \text{ N}$ , we can write a Python script using the `minimize` function to solve for the displacements and potential energy.

## Solution

A function can be developed to compute the potential energy:

```
import numpy as np

def PE(x,ka,kb,La,Lb,F1,F2):
    x1 = x[0]
    x2 = x[1]
    PEa = 0.5*ka*(np.sqrt(x1**2+(La-x2)**2)-La)**2
    PEb = 0.5*kb*(np.sqrt(x1**2+(Lb+x2)**2)-Lb)**2
    W = F1*x1+F2*x2
    return PEa+PEb-W
```

The solution can be obtained with the `minimize` function:

Page 191

```
from scipy.optimize import minimize
ka = 9 ; kb = 2 # N/cm
La = 10 ; Lb = 10 # cm
F1 = 2 ; F2 = 4 # N
x0 = [-0.5, 0.5] # cm
res = minimize(lambda x: PE(x,ka,kb,La,Lb,F1,F2),x0
               ,method='Nelder-Mead')
xmin = res.x
print('Displacements are x1 =[0:5.2f] and x2 =[1:5.2f] cm'
      .format(xmin[0],xmin[1]))
print('Minimum potential energy =[0:5.2f] N*cm'
      .format(PE(xmin,ka,kb,La,Lb,F1,F2)))
```

And the results are

```
Displacements are x1 = 4.95 and x2 = 1.28 cm
Minimum potential energy = -9.64 N*cm
```

With reference to [Fig. 7.11](#), the connecting point is located 4.95 cm to the right and 1.28 cm above the original position with no forces applied.

# PROBLEMS

---

**7.1** Perform three iterations of the Newton-Raphson method to determine the root of [Eq. \(E7.1.1\)](#). Use the parameter values from [Example 7.1](#) along with an initial guess of  $t = 3$  s.

**7.2** Given the formula

$$f(x) = -x^2 + 8x - 12$$

- (a) Determine the maximum and the corresponding value of  $x$  for this function analytically, that is, using differentiation.
- (b) Verify that [Eq. \(7.10\)](#) yields the same result based on initial guesses of  $x_1 = 0$ ,  $x_2 = 2$ , and  $x_3 = 6$ .

**7.3** Consider the following function:

$$f(x) = 3 + 6x + 5x^2 + 3x^3 + 4x^4$$

Locate the minimum by finding the root of the derivative of this function. Use bisection with initial guesses of  $x_l = -2$  and  $x_u = 1$ .

**7.4** Given

$$f(x) = -1.5x^6 - 2x^4 + 12x$$

- (a) Plot the function.
- (b) Use analytical methods to prove that the function is concave for all values of  $x$ .
- (c) Differentiate the function and then use a root-location method to solve for the maximum  $f(x)$  and the corresponding value of  $x$ .

**7.5** Solve for the value of  $x$  that maximizes  $f(x)$  in [Prob. 7.4](#) using the golden-section search. Employ initial guesses of  $x_l = 0$  and  $x_u = 2$ . Perform three iterations.

**7.6** Repeat [Prob. 7.5](#), except use parabolic interpolation. Employ initial guesses of  $x_1 = 0$ ,  $x_2 = 1$ , and  $x_3 = 2$ . Perform three iterations.

**7.7** Employ the following methods to find the maximum of

$$f(x) = 4x - 1.8x^2 + 1.2x^3 - 0.3x^4$$

- (a) Golden-section search ( $x_l = -2$ ,  $x_u = 4$ ,  $\varepsilon_s = 1\%$ )
- (b) Parabolic interpolation ( $x_1 = 1.75$ ,  $x_2 = 2$ ,  $x_3 = 2.5$ , 5 iterations)

**7.8** Consider the following function:

$$f(x) = x^4 + 2x^3 + 8x^2 + 5x$$



Use analytical and graphical methods to show the function has a minimum for some value of  $x$  in the range  $-2 \leq x \leq 1$ .

**7.9** Employ the following methods to find the minimum of the function from  **Prob. 7.8**:

- (a) Golden-section search ( $x_l = -2$ ,  $x_u = 1$ ,  $\varepsilon_s = 1\%$ ).
- (b) Parabolic interpolation ( $x_1 = -2$ ,  $x_2 = -1$ ,  $x_3 = 1$ , 5 iterations).

**7.10** Consider the following function:

$$f(x) = 2x + \frac{3}{x}$$

Perform ten iterations of parabolic interpolation to locate the minimum. Comment on the convergence of your results. ( $x_1 = 0.1$ ,  $x_2 = 0.5$ , and  $x_3 = 5$ )

**7.11** The following function defines a curve with several unequal minima over the interval  $2 \leq x \leq 20$ ,

$$f(x) = \sin(x) + \sin\left(\frac{2}{3}x\right)$$

Develop a Python script to (a) plot the function over the interval. (b) Determine the minimum with the SciPy `optimize` function `minimum_scalar`, and (c) manually with the golden-section search with a stopping criterion corresponding to three significant figures. For (b) and (c), use initial guesses of  $x_l = 4$  and  $x_u = 8$ .

**7.12** Use the golden-section search method to determine the location of  $x_{\max}$  and the corresponding value of  $f(x_{\max})$  for the following function showing the results of each iteration.

$$f(x) = -0.8x^4 + 2.2x^2 + 0.6$$

Use initial guesses of  $x_l = 0.7$  and  $x_u = 1.4$  and perform sufficient iterations so that  $\varepsilon_s \leq 10\%$ . What is the maximum error of your final result?

**7.13** Develop a single Python script to (a) generate a contour and mesh plots of the following temperature field in a similar fashion to  **Example 7.4**,

$$T(x, y) = 2x^2 + 3y^2 - 4xy - 3x - y$$

and (b) determine the minimum of this surface using the `minimize` function from the SciPy `optimize` module.

**7.14** The head of a groundwater aquifer is described in Cartesian coordinates by

$$h(x, y) = \frac{1}{1 + x^2 + y^2 + xy + x}$$

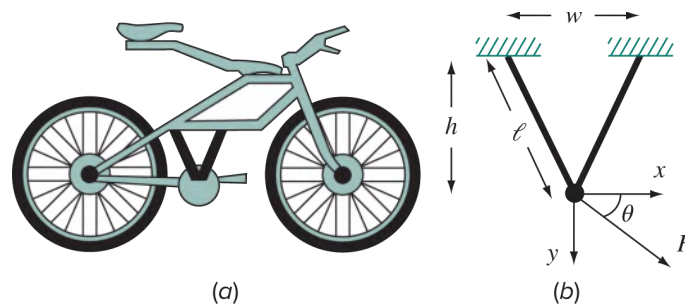
Develop a single Python script to (a) generate a contour and mesh plots of the function in a similar fashion to  **Example 7.4**, and (b) determine the minimum of this surface using the `minimize`

function from the SciPy optimize module.

**7.15** Recent interest in competitive and recreational cycling has meant that engineers have directed their skills toward the design and testing of mountain bikes (Fig. P7.15a). Suppose that you are given the task of predicting the horizontal and vertical displacement of a bike bracketing system in response to a force input. Assume the forces you must analyze can be simplified as depicted in Fig. P7.15b. You are interested in testing the response of the truss to a force exerted in any number of directions designated by the angle  $\theta$ . The parameter values for this problem are

$E = \text{Young's modulus} = 2 \times 10^{11} \text{ Pa}$ ,  $A = \text{cross-sectional area} = 0.0001 \text{ m}^2$ ,  
 $w = \text{width} = 0.44 \text{ m}$ ,  $l = \text{length} = 0.56 \text{ m}$ , and  $h = \text{height} = 0.5 \text{ m}$ . The displacements  $x$  and  $y$  can be solved by determining the values that minimize potential energy. Determine the displacements for a force of 10,000 N and a range of  $\theta$ 's from  $0^\circ$  (horizontal) to  $90^\circ$  (vertical).

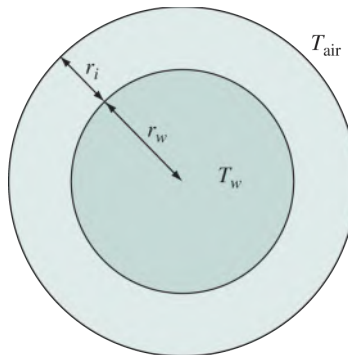
**FIGURE P7.15** (a) A mountain bike along with (b) a free-body diagram for a part of the frame.



**7.16** As electric current moves through a wire (Fig. P7.16), heat generated by resistance is conducted through a layer of insulation and then convected to the surrounding air. The steady-state temperature of the wire can be represented by

$$T = T_{air} + \frac{q}{2\pi} \left[ \frac{1}{k} \ln \left( \frac{r_w + r_i}{r_w} \right) + \frac{1}{h} \frac{1}{r_w + r_i} \right]$$

**FIGURE P7.16** Cross-section of an insulated wire.





Determine the thickness of insulation,  $r_i$  (m), that minimizes the wire's temperature, given the following parameter values:  $q$  = heat generation rate = 75 W/m,  $r_w$  = wire radius = 6 mm,  $k$  = thermal conductivity of the insulation = 0.17 W/(m·K),  $h$  = convective heat transfer coefficient = 12 W/(m<sup>2</sup>·K), and  $T_{air}$  = ambient air temperature = 20°C  $\cong$  293 K.

**7.17** Develop a Python function that implements the golden-section search to find expressly the maximum of a function and not the minimum, as illustrated in  **Fig. 7.7**. The function should include the following features:

- stopping criterion of a threshold on the relative error (default value  $1 \times 10^{-7}$ )
- maximum number of iterations (default value 30)
- return of the optimal  $x$  and  $f(x)$

Test your program with the problem from  **Example 7.1**.

**7.18** Develop a Python function to locate a minimum with the golden-section search. Rather than using the maximum iteration and  **Eq. (7.9)** as stopping criteria, determine the number of iterations needed to attain a desired tolerance. Test your function by solving  **Example 7.2** using  $E_{a,d} = 0.0001$ .

**7.19** Develop a Python function to implement parabolic interpolation to locate a minimum. The function should have the following features:

- Base it on two initial guesses and have the program generate the third initial value at the midpoint of the interval.
- Check whether the guesses bracket a minimum. If not, the function should not implement the algorithm, but return an error message.
- Iterate until the relative error falls below a stopping criterion or exceeds a maximum number of iterations.
- Returns the optimal  $x$  and  $f(x)$ , the relative error achieved, and the number of iterations taken.

Test your function with the problem from  **Example 7.3**.

**7.20** Pressure measurements are taken at certain points behind an airfoil over time. These data are best fit by the curve  $y = 6 \cos(t) - 1.5 \sin(t)$  over the range  $0 \leq t \leq 6$  seconds. Use four iterations of the golden-section search method to find the minimum pressure. Create a plot of the function of this range of time and determine two reasonable starting guesses that bracket the minimum.

**7.21** The trajectory of a ball can be predicted from the formula

$$y = \tan(\theta_0)x - \frac{g}{2v_0^2 \cos^2(\theta_0)}x^2 + y_0$$

where  $y$  = height(m),  $\theta_0$  = initial angle(radians),  $v_0$  = initial velocity(m/s), and  $y_0$  = initial elevation(m). Use the golden-section search to determine the maximum height given

$y_0 = 2$  m,  $v_0 = 20$  m/s, and  $\theta_0 = 45^\circ$ . Iterate until the approximate error falls below  $\varepsilon_s = 10\%$  using initial guesses  $x_l = 10$  m and  $x_u = 30$  m.

**7.22** The deflection of a uniform beam subject to a linearly increasing distributed load can be computed as Page 193

$$y = \frac{w_0}{120EIL}(-x^5 + 2L^2x^3 - L^4x)$$

Given that  $L = 600$  cm,  $E = 50,000$  kN/cm<sup>2</sup>,  $I = 30,000$  cm<sup>4</sup>, and  $w_0 = 2.5$  kN/cm, determine the location of the point of maximum deflection (a) graphically, and (b) using the golden-section search method until the approximate error falls below  $\varepsilon_s = 1\%$ . Employ initial guesses of  $x_l = 0$  and  $x_u = L$ .

**7.23** An object with mass of 90 kg is projected upward from the surface of the earth at a velocity of 60 m/s. If the object is subject to linear drag ( $c = 15$  kg/s), use the golden-section search to determine the maximum height the object attains.

**7.24** The probability density function for the standard normal (Gaussian) distribution is

$$f(z) = \frac{1}{\sqrt{2\pi}} e^{-\frac{z^2}{2}} \quad -\infty \leq z \leq \infty$$

This describes the classic “bell curve,” and  $f(z)$  has significant values in the range  $-4 \leq z \leq 4$ . There are two inflection points in the curve, one for  $z > 0$  and the other for  $z < 0$ . Use the golden-section search to determine the location of the inflection point for positive values of  $z$ .

**7.25** A common practice in the field of experimental design is to fit a quadratic response surface model to the results of a set of experimental runs where two factors are varied. An example model is

$$f(x, y) = 2y^2 - 2.25xy - 1.75y + 1.5x^2$$

Use the `minimize` function from the Python SciPy `optimize` module to determine the values of  $x$  and  $y$  that minimize  $f(x, y)$ . Also, report the optimal value of  $f(x, y)$  and provide a contour plot to confirm your results.

**7.26** Use the `minimize` function from the SciPy `optimize` module to determine the maximum of

$$f(x, y) = 4x + 2y + x^2 - 2x^4 + 2xy - 3y^2$$

**7.27** Given the following function:

$$f(x, y) = -8x + x^2 + 12y + 4y^2 - 2xy$$

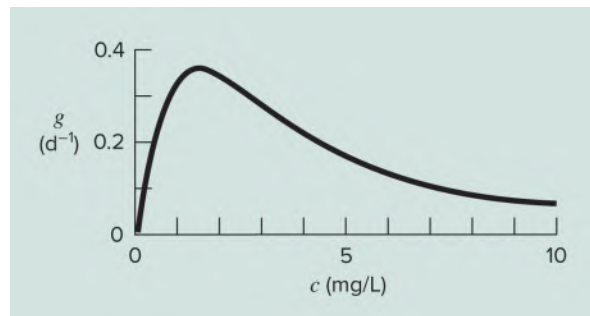
Determine the location of the minimum (a) graphically, using a contour plot, and (b) using the `minimize` function from the SciPy `optimize` module. As part of (b), provide the minimum value of  $f(x, y)$ .

**7.28** The specific growth rate of a yeast that produces an antibiotic is a function of the nutrient concentration  $c$  as given by

$$g = \frac{2c}{4 + 0.8c + c^2 + 0.2c^3}$$

As depicted in  **Fig. P7.28**, growth goes to zero at very low concentrations due to nutrient limitation. It also goes to zero at high concentrations due to toxicity effects. Find the value of  $c$  at which growth is a maximum.

**FIGURE P7.28** The specific growth rate of a yeast that produces an antibiotic versus the nutrient concentration.



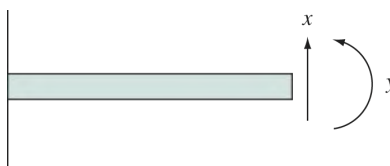
**7.29** A compound A will be converted into B in a stirred tank reactor. The product B is purified in a separator unit by removing unreacted A. The unreacted A is recycled to the reactor. A process engineer has found that the capital cost of the process equipment is a function of the conversion of A,  $x_A$ . This is described by the formula below. Find the conversion that will result in the lowest capital cost. The parameter  $C$  is a proportionality constant.

$$\text{Cost} = C \left[ \left( \frac{1}{(1 - x_A)^2} \right)^{0.6} + 6 \left( \frac{1}{x_A} \right)^{0.6} \right]$$

**7.30** A finite-element model of a cantilever beam subject to loading and moments ( **Fig. P7.30**) is described as

$$f(x, y) = 5x^2 - 5xy + 2.5y^2 - x - 1.5y$$

**FIGURE P7.30** A cantilever beam.

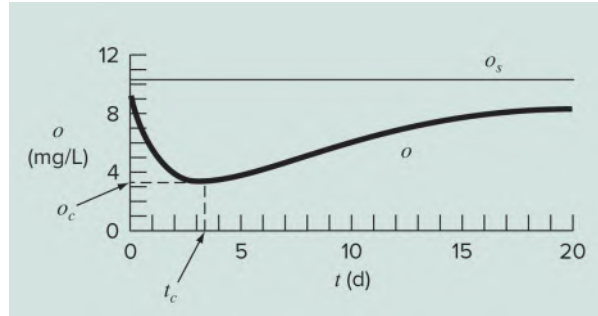


where  $x$  = end displacement and  $y$  = end moment. Find the values of  $x$  and  $y$  that minimize  $f(x, y)$ .

**7.31** The *Streeter-Phelps model* can be used to compute the dissolved oxygen concentration in a river below a point discharge of sewage (Fig. P7.31),

$$o = o_s - \frac{k_d L_o}{k_d + k_s - k_a} (e^{-k_a t} - e^{-(k_d + k_s)t}) - \frac{S_b}{k_a} (1 - e^{-k_a t}) \quad (\text{P7.31})$$

**FIGURE P7.31** A dissolved oxygen “sag” below a point of discharge of sewage into a river.



where  $o$  = dissolved oxygen (mg/L),  $o_s$  = oxygen saturation concentration (mg/L),  $t$  = travel time (d),  $L_o$  = biochemical oxygen demand (BOD) concentration at the mixing point (mg/L),  $k_d$  = rate of decomposition of BOD ( $\text{d}^{-1}$ ),  $k_s$  = rate of settling of BOD ( $\text{d}^{-1}$ ),  $k_a$  = aeration rate ( $\text{d}^{-1}$ ), and  $S_b$  = sediment oxygen demand ( $\text{mg}/(\text{L} \cdot \text{d})$ ).

As indicated in Fig. P7.31, Eq. P7.31 predicts an “oxygen sag” that reaches a critical minimum level,  $o_c$ , at some travel time,  $t_c$ , below the point discharge. This point is termed “critical” because it represents the location where biota that depend on oxygen (like fish) would be the most stressed.

Develop a Python script that (a) generates a plot of  $o$  versus  $t$  and (b) uses `minimize_scalar` to determine the critical travel time and concentration given the following parameter values:  $o_s = 10 \text{ mg/L}$ ,  $k_d = 0.1 \text{ d}^{-1}$ ,  $k_a = 0.6 \text{ d}^{-1}$ ,  $k_s = 0.05 \text{ d}^{-1}$ ,  $L_o = 50 \text{ mg/L}$ , and  $S_b = 1 \text{ mg}/(\text{L} \cdot \text{d})$ .

Page 194

**7.32** The two-dimensional distribution of pollutant concentration in a channel can be described by

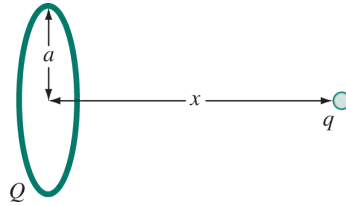
$$c(x, y) = 7.9 + 0.13x + 0.21y - 0.05x^2 - 0.016y^2 - 0.007xy$$

Determine the exact location of the peak (maximum) given the function and the knowledge that the peak lies within the bounds  $-10 \leq x \leq 10$  and  $0 \leq y \leq 20$ .

**7.33** A total charge  $Q$  is uniformly distributed around a ring-shaped conductor of radius  $a$ . A charge  $q$  is located at a distance  $x$  from the center of the ring (Fig. P7.33). The force exerted on the charge by the ring is given by

$$F = \frac{1}{4\pi e_0} \frac{qQx}{(x^2 + a^2)^{3/2}}$$

**FIGURE P7.33**



where  $e_0 = 8.85 \times 10^{-12} \text{ C}^2/(\text{N} \cdot \text{m}^2)$ ,  $q = Q = 2 \times 10^{-5} \text{ C}$ , and  $a = 0.9 \text{ m}$ . Determine the distance  $x$  where the force is a maximum.

**7.34** The torque transmitted to an induction motor is a function of the slip between the rotation of the stator field and the rotor speed,  $s$ , where the slip is defined as

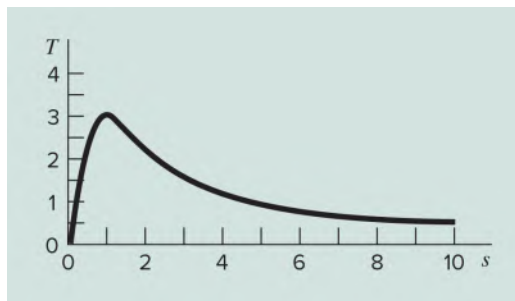
$$s = \frac{n - n_R}{n}$$

where  $n$  = revolutions per second of the rotating stator, and  $n_R$  = revolutions per second of the rotor. Kirchoff's laws can be used to show that the torque, expressed in dimensionless form, and slip are related by

$$T = \frac{15s}{4s^2 - 3s + 4}$$

Figure P7.34 depicts this function graphically. Use a numerical method to determine the slip at which the maximum torque occurs.

**FIGURE P7.34** Torque transmitted to an inductor as a function of slip.

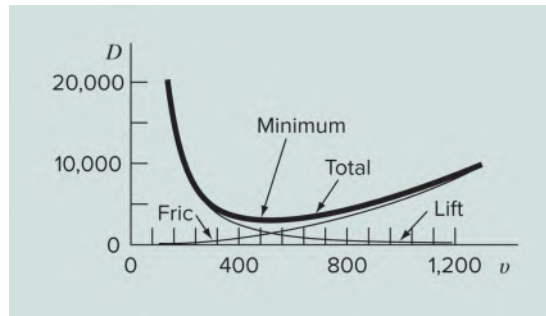


**7.35** The total drag on an airfoil can be estimated by the formula

$$D = \underbrace{0.01\sigma v^2}_{\text{Friction}} + \underbrace{\frac{0.95}{\sigma} \left( \frac{W}{v} \right)^2}_{\text{Lift}}$$

where  $D$  = drag,  $\sigma$  = ratio of the air density between the flight altitude and sea level,  $W$  = weight, and  $v$  = velocity. As seen in [Fig. P7.35](#), the two factors contributing to drag are affected differently as the velocity increases. Whereas friction drag increases with the square of velocity, the drag due to lift decreases. The combination of the two factors leads to a scenario for minimum drag, as depicted in the figure.

**FIGURE P7.35** Plot of drag versus velocity for an airfoil.



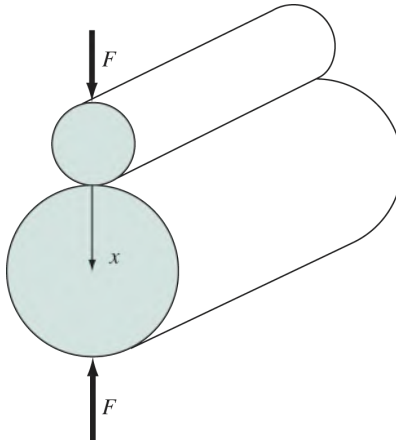
- (a) If  $\sigma = 0.6$  and  $W = 16,000$ , determine the minimum drag and the velocity at which it occurs.
- (b) In addition, develop a sensitivity analysis to determine how this optimum varies over a range of weights from 12,000 to 20,000, again with  $\sigma = 0.6$ .

**7.36** Roller bearings are subject to fatigue failure caused by large contact loads,  $F$  ([Fig. P7.36](#)). The problem of finding the location of maximum stress along the  $x$  axis can be shown to be equivalent to maximizing the function:

$$f(x) = \frac{0.4}{\sqrt{1+x^2}} - \sqrt{1+x^2} \left( 1 - \frac{0.4}{1+x^2} \right) + x$$

**FIGURE P7.36** Roller bearings.





which simplifies to

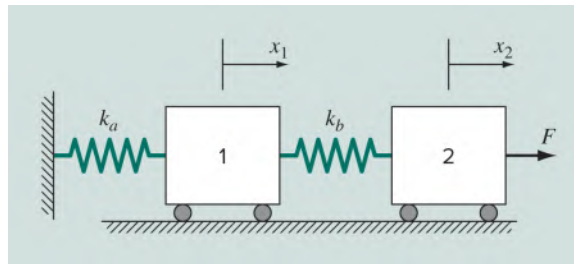
$$f(x) = \frac{0.8}{\sqrt{1+x^2}} - \sqrt{1+x^2} + x$$

Find the value of  $x$  that maximizes  $f(x)$ .

**7.37** In a similar fashion to the case study described in [Fig. P7.37](#), develop the potential energy function for the system depicted in [Fig. P7.37](#). Provide contour and surface plots via Python's Matplotlib module. Minimize the potential energy function to determine equilibrium displacements,  $x_1$  and  $x_2$ , given the forcing function  $F = 100$  N and the parameters  $k_a = 20$  N/m and  $k_b = 15$  N/m.

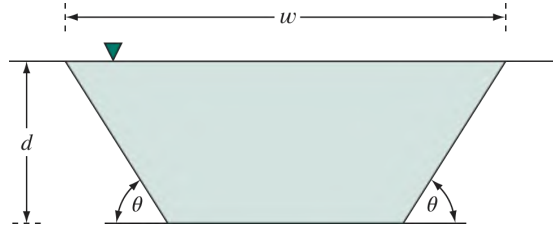
Page 195

**FIGURE P7.37** Two frictionless masses connected to a wall by a pair of linear elastic springs.



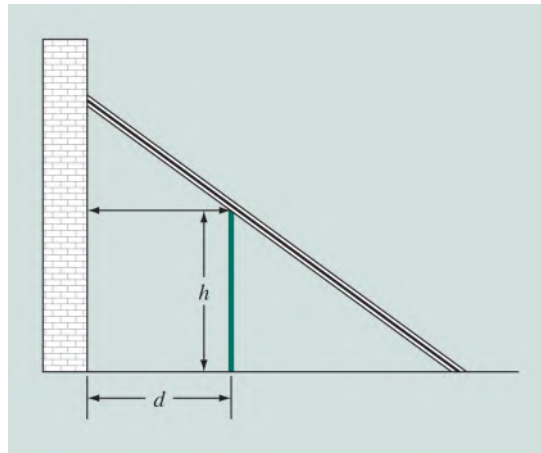
**7.38** In your role as an agricultural engineer, you must design a trapezoidal open channel to carry irrigation water ([Fig. P7.38](#)). Determine the optimal design to minimize the wetted perimeter for a cross-sectional area of  $50 \text{ m}^2$ . Are the relative dimensions universal?

**FIGURE P7.38** Cross-section of a trapezoidal channel.



**7.39** Use the `minimize_scalar` function from the SciPy `optimize` module to minimize the length of a ladder that reaches from the ground over a fence to a building's wall (Fig. P7.39). Test it for the case where  $h = d = 4$  m.

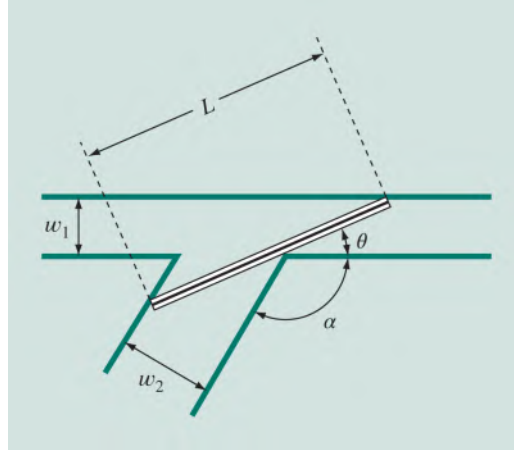
**FIGURE P7.39** A ladder leaning against a fence and just touching a wall.



**7.40** The length of the longest ladder that can negotiate the corner, approaching from below and turning right, as depicted in Fig. P7.40, can be determined by computing the value of  $\theta$  that minimizes the following function:

$$L(\theta) = \frac{w_1}{\sin(\theta)} + \frac{w_2}{\sin(\pi - \alpha - \theta)}$$

**FIGURE P7.40** A ladder negotiating a corner formed by two intersecting hallways.

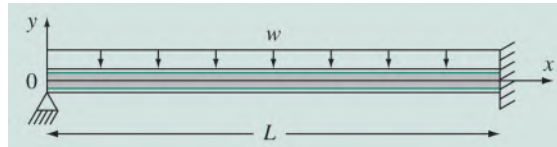


For the case where  $w_1 = 2$  m and  $w_2 = 3$  m, use a numerical method described in this chapter, including Python's built-in capabilities, to develop a plot of the longest  $L$  versus  $\alpha$  for a range  $45^\circ \leq \alpha \leq 135^\circ$ .

**7.41**  **Figure P7.41** shows a pinned fixed beam subject to a uniform load. The equation for the resulting deflection is

$$y = -\frac{w}{48EI} (2x^4 - 3Lx^3 + L^3x)$$

**FIGURE P7.41**

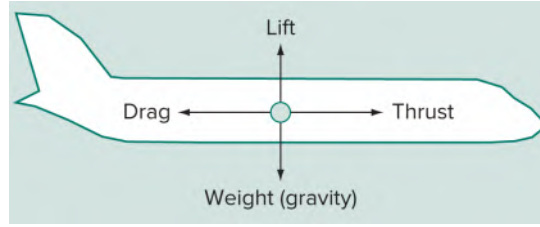


Develop Python scripts that (a) generate a labeled plot of deflection versus distance and (b) determine the location and magnitude of the maximum deflection. Employ the `minimum_scalar` function from the SciPy `optimize` module. Use the following parameter values, making sure that you use consistent units:  $L = 400$  cm,  $E = 52,000$  kN/cm<sup>2</sup>,  $I = 32,000$  cm<sup>4</sup>, and  $w = 4$  kN/cm.

**7.42** For an aircraft in steady, level flight, thrust balances drag and lift balances weight ( **Fig. P7.42**). Under these conditions, the optimal cruise speed occurs when the ratio of drag force to velocity is minimized. The drag coefficient,  $C_D$ , can be computed as

$$C_D = C_{D0} + \frac{C_L^2}{\pi AR}$$

**FIGURE P7.42** The four major forces on an aircraft in steady, level flight.



where  $C_{D0}$  = the drag coefficient at zero lift,  $C_L$  = the lift coefficient, and  $AR$  = the aspect ratio. For steady, level flight, the lift coefficient can be computed from

$$C_L = \frac{2W}{\rho v^2 A}$$

where  $W$  = the aircraft's weight(N),  $\rho$  = the air density( $\text{kg}/\text{m}^3$ ),  
 $v$  = velocity(m/s), and  $A$  = wing planform area( $\text{m}^2$ ). The drag force can then be computed as

Page 196

$$F_D = W \frac{C_D}{C_L}$$

Use these formulas to determine the optimal steady cruise velocity for a 670 kN jet aircraft flying at 10 km above sea level. Employ the following parameter values in your computation:

$A = 150 \text{ m}^2$ ,  $AR = 6.5$ ,  $C_{D0} = 0.018$ , and  $\rho = 0.413 \text{ kg}/\text{m}^3$ .

**7.43** Develop a Python script to generate a plot of the optimal velocity of the aircraft from

 **Prob. 7.42** versus elevation above sea level. Employ a mass of 68,300 kg for the aircraft. Note that gravitational acceleration at  $45^\circ$  latitude can be computed as a function of elevation as follows:

$$g(h) = 9.8066 \left( \frac{r_e}{r_e + h} \right)^2$$

where  $g(h)$  = gravitational acceleration( $\text{m}/\text{s}^2$ ) at elevation  $h$  (m) above sea level, and  $r_e$  = earth's mean radius,  $6.371 \times 10^6$  m. In addition, air density as a function of elevation can be calculated using

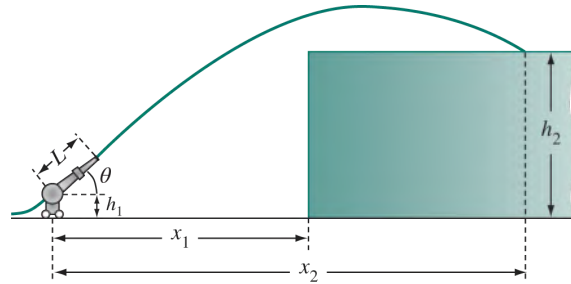
$$\rho(h) = 1.22534 - 1.18951 \times 10^{-4}h + 4.71260 \times 10^{-9}h^2 - 9.57296 \times 10^{-14}h^3$$

Employ the other parameter values from  **Prob. 7.42** and design the plot for elevations ranging from 0 to 12 km above sea level.

**7.44** As depicted in  **Fig. P7.44**, a mobile fire hose projects a stream of water onto the roof of a building. At what angle,  $\theta$ , and how far from the building,  $x_1$ , should the hose be anchored in order to maximize the coverage of the roof, that is, to maximize the distance,  $x_2 - x_1$ ? Note that, due to hose

dimensions and delivery pressure, the water velocity leaving a typical fire hose nozzle has a constant value of 15 m/s regardless of the angle. Other parameter values are  $h_1 = 0.6$  m,  $h_2 = 10$  m, and  $L = 0.4$  m. Hint: Coverage is maximized for the water trajectory that just clears the front edge of the roof. That is, we want to choose an  $x_1$  and  $\theta$  so that the front edge is cleared while maximizing  $x_2 - x_1$ .

**FIGURE P7.44**



## Linear Systems

### 3.1 OVERVIEW

#### What Are Linear Algebraic Equations?

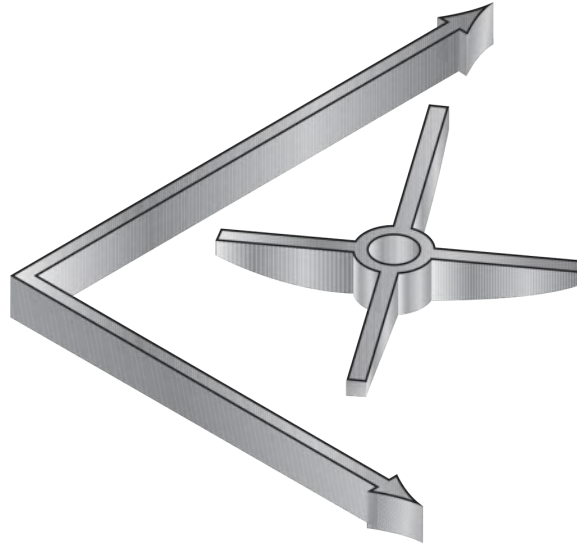
In  **Part Two**, we determined the value  $x$  that satisfied a single equation,  $f(x) = 0$ . Now, we deal with the case of determining the values  $x_1, x_2, \dots, x_n$  that simultaneously satisfy a set of equations:

$$\begin{aligned} f_1(x_1, x_2, \dots, x_n) &= 0 \\ f_2(x_1, x_2, \dots, x_n) &= 0 \\ &\vdots \\ &\vdots \\ f_n(x_1, x_2, \dots, x_n) &= 0 \end{aligned}$$

Such systems are either linear or nonlinear. In Part Three, we deal with *linear algebraic equations* that are of the general form

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n &= b_2 \\ &\vdots \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n &= b_n \end{aligned} \quad (\text{PT3.1})$$

where the  $a$ 's are constant coefficients, the  $b$ 's are constants, the  $x$ 's are unknowns, and  $n$  is the number of equations. All other algebraic equations are nonlinear.



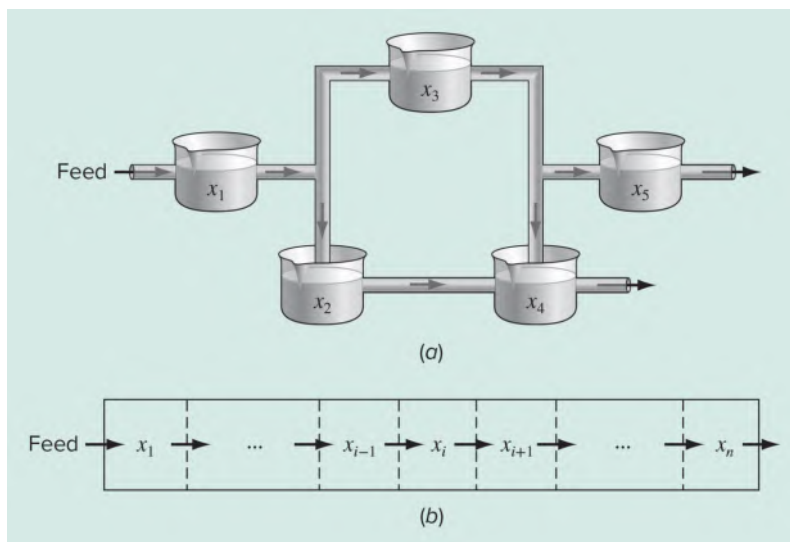
## Linear Algebraic Equations in Engineering and Science

Many of the fundamental equations of engineering and science are based on conservation laws. Some familiar quantities that conform to such laws are mass, energy, and momentum. In mathematical terms, these principles lead to balance or continuity equations that relate system behavior as represented by the levels or response of the quantity being modeled to the properties or characteristics of the system and the external stimuli or forcing functions acting on the system.

As an example, the principle of mass conservation can be used to formulate a model for a series of chemical reactors ([Fig. PT3.1a](#)). For this case, the quantity being modeled is the mass of the chemical in each reactor. The system properties are the reaction characteristics of the chemical and the reactors' sizes and flow rates. The forcing functions are the feed rates of the chemical into the system.

---

**FIGURE PT3.1** Two types of systems that can be modeled using linear algebraic equations: (a) lumped variable system that involves coupled finite components and (b) distributed variable system that involves a continuum.



When we studied roots of equations, you saw how single-component systems result in a single equation that can be solved using root-location techniques. Multicomponent systems result in a coupled set of mathematical equations that must be solved simultaneously. The equations are coupled because the individual parts of the system are influenced by other parts. For example, in [Fig. PT3.1a](#), reactor 4 receives chemical inputs from reactors 2 and 3. Consequently, its response is dependent on the quantity of chemical in these other reactors.

When these dependencies are expressed mathematically, the resulting equations are often of the linear algebraic form of [Eq. \(PT3.1\)](#). The  $x$ 's are usually measures of the magnitudes of the responses of the individual components. Using [Fig. PT3.1a](#) as an example,  $x_1$  might quantify the amount of chemical mass in the first reactor,  $x_2$  might quantify the amount in the second, and so forth. The  $a$ 's typically represent the properties and characteristics that bear on the interactions between components. For instance, the  $a$ 's for [Fig. PT3.1a](#) might be reflective of the flow rates of mass between the reactors. Finally, the  $b$ 's usually represent the forcing functions acting on the system, such as the feed rate.

Multicomponent problems of these types arise from both lumped (macro-) or distributed (micro-) variable mathematical models. *Lumped variable problems* involve coupled finite components. The three interconnected bungee jumpers described at the beginning of [Chap. 8](#) are a lumped system. Other examples include trusses, reactors, and electric circuits.

Conversely, *distributed variable problems* attempt to describe the spatial detail on a continuous or semicontinuous basis. The distribution of chemicals along the length of an elongated, rectangular reactor ([Fig. PT3.1b](#)) is an example of a continuous variable model. Differential equations derived from conservation laws specify the distribution of the dependent variable for such systems. These differential equations can be solved numerically by converting them to an equivalent system of simultaneous algebraic equations.



The solution of such sets of equations represents a major application area for the methods in the following chapters. These equations are coupled because the variables at one location are dependent on the variables in adjoining regions. For example, the concentration at the middle of the reactor in  **Fig. PT3.1b** is a function of the concentration in adjoining regions. Similar examples could be developed for the spatial distribution of temperature, momentum, or electricity.

Aside from physical systems, simultaneous linear algebraic equations also arise in a variety of mathematical problem contexts. These result when mathematical functions are required to satisfy several conditions simultaneously. Each condition results in an equation that contains known coefficients and unknown variables. The techniques discussed in this part can be used to solve for the unknowns when the equations are linear and algebraic. Some widely used numerical techniques that employ simultaneous equations are regression analysis and spline interpolation.

## 3.2 PART ORGANIZATION

---

Due to its importance in formulating and solving linear algebraic equations, [Chap. 8](#) provides a brief overview of *matrix algebra*. Aside from covering the rudiments of matrix representation and manipulation, the chapter also describes how matrices are handled in Python.

[Chapter 9](#) is devoted to the most fundamental technique for solving linear algebraic systems: *Gauss elimination*. Before launching into a detailed discussion of this technique, a preliminary section deals with simple methods for solving small systems. These approaches are presented to provide you with visual insight and because one of the methods—the elimination of unknowns—represents the basis for Gauss elimination.

After this preliminary material, “naive” Gauss elimination is discussed. We start with this “stripped-down” version because it allows the fundamental technique to be elaborated on without complicating details. Then, in subsequent sections, we discuss potential problems of the naive approach and present a number of modifications to minimize and circumvent these problems. The focus of this discussion will be the process of switching rows, or *partial pivoting*. The chapter ends with a brief description of efficient methods for solving *tridiagonal matrices*.

[Chapter 10](#) illustrates how Gauss elimination can be formulated as an *LU factorization*. Such solution techniques are valuable for cases where many right-hand-side vectors need to be evaluated. The chapter ends with a brief outline of how Python solves linear systems.

[Chapter 11](#) starts with a description of how *LU* factorization can be employed to efficiently calculate the *matrix inverse*, which has tremendous utility in analyzing stimulus-response relationships of physical systems. The remainder of the chapter is devoted to the important concept of matrix condition. The condition number is introduced as a measure of the roundoff errors that can result when solving ill-conditioned matrices.

[Chapter 12](#) deals with iterative solution techniques, which are similar in spirit to the approximate methods for roots of equations discussed in [Chap. 6](#). That is, they involve guessing a solution and then iterating to obtain a refined estimate. The emphasis is on the *Gauss-Seidel method*, although a description is provided of an alternative approach, the *Jacobi method*. The chapter ends with a brief description of how *nonlinear simultaneous equations* can be solved.

Finally, [Chap. 13](#) is devoted to *eigenvalue* problems. These have general mathematical relevance as well as many applications in engineering and science. We describe two simple methods as well as Python’s capabilities for determining eigenvalues and *eigenvectors*. In terms of applications, we focus on their use to study the vibrations and oscillations of mechanical systems and structures.

# Linear Algebraic Equations and Matrices

## CHAPTER OBJECTIVES

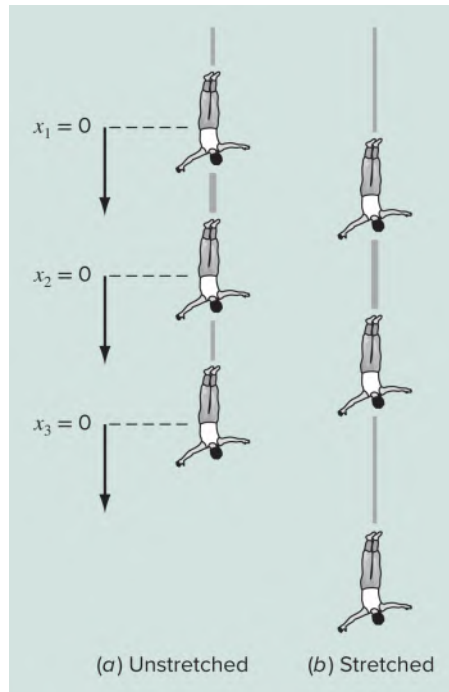
The primary objective of this chapter is to acquaint you with linear algebraic equations and their relationship to matrices and matrix algebra. Specific objectives and topics covered are

- Understanding matrix notation.
- Being able to identify the following types of matrices: identity, diagonal, symmetric, triangular, and tridiagonal.
- Knowing how to perform matrix multiplication and being able to assess when it is feasible.
- Knowing how to represent a system of linear algebraic equations in matrix form.
- Knowing how to solve linear algebraic equations in Python with matrix inversion and the `linalg.solve` function from the NumPy module.

## YOU'VE GOT A PROBLEM

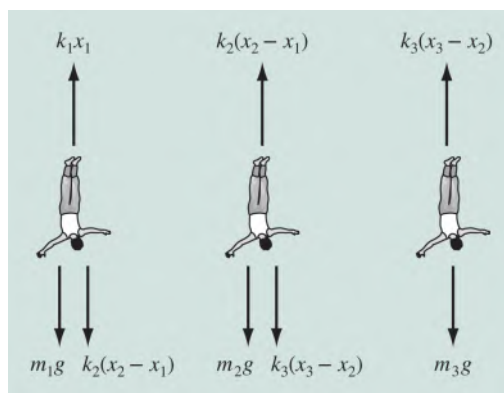
Suppose that three jumpers are connected by bungee cords.  **Figure 8.1a** shows them being held in place vertically so that each cord is fully extended but unstretched. We can define three distances,  $x_1$ ,  $x_2$ , and  $x_3$ , as measured downward from each of their unstretched positions. After they are released, gravity takes hold and the jumpers will eventually come to the equilibrium positions shown in  **Fig. 8.1b**.

**FIGURE 8.1** Three individuals connected by bungee cords.



Suppose that you are asked to compute the displacement of each of the jumpers. If we assume that each cord behaves as a linear spring and follows Hooke's law, free-body diagrams can be developed for each jumper as depicted in [Fig. 8.2](#).

**FIGURE 8.2** Free-body diagrams.



Using Newton's second law, force balances can be written for each jumper:

$$(8.1)$$

$$\begin{aligned}
m_1 \frac{d^2 x_1}{dt^2} &= m_1 g + k_2(x_2 - x_1) - k_1 x_1 \\
m_2 \frac{d^2 x_2}{dt^2} &= m_2 g + k_3(x_3 - x_2) + k_2(x_1 - x_2) \\
m_3 \frac{d^2 x_3}{dt^2} &= m_3 g + k_3(x_2 - x_3)
\end{aligned}$$

where  $m_i$  = the mass of jumper  $i$  (kg),  $t$  = time (s),  
 $k_j$  = the spring constant for cord  $j$  (N/m),  $x_i$  = the displacement of jumper  $i$  measured downward from the equilibrium position (m), and  $g$  = gravitational acceleration ( $9.81 \text{ m/s}^2$ ). Because we are interested in the steady-state solution, the second derivatives can be set to zero. Collecting terms gives

$$\begin{aligned}
(k_1 + k_2)x_1 - k_2x_2 &= m_1g \\
-k_2x_1 + (k_2 + k_3)x_2 - k_3x_3 &= m_2g \\
-k_3x_2 + k_3x_3 &= m_3g
\end{aligned} \quad (8.2)$$

Thus, the problem reduces to solving a system of three simultaneous equations for the three unknown displacements. Because we have used a linear law for the cords, these equations are linear algebraic equations. Chapters 8 through [12](#) will introduce you to how Python is used to solve such systems of equations.

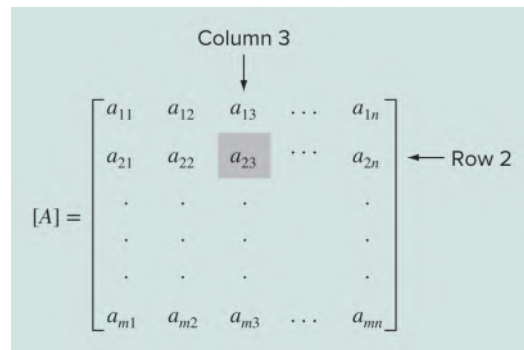
# 8.1 MATRIX ALGEBRA OVERVIEW

Knowledge of matrices is essential for understanding the solution of linear algebraic equations. The following sections outline how matrices provide a concise way to represent and manipulate linear algebraic equations.

## 8.1.1 Matrix Notation

A *matrix* consists of a rectangular array of elements represented by a single symbol. As depicted in  **Fig. 8.3**,  $[A]$  is the shorthand notation for the matrix and  $a_{ij}$  designates an individual *element* of the matrix.

**FIGURE 8.3** A matrix.



A horizontal set of elements is called a *row* and a vertical set is called a *column*. The first subscript  $i$  always designates the number of the row in which the element lies. The second subscript  $j$  designates the column. For example, element  $a_{23}$  is in row 2 and column 3.

The matrix in  **Fig. 8.3** has  $m$  rows and  $n$  columns and is said to have a dimension of  $m$  by  $n$  (or  $m \times n$ ). It is referred to as an  $m$  by  $n$  matrix.

Matrices with row dimension  $m = 1$ , such as

$$[b] = [b_1 \quad b_2 \quad \dots \quad b_n]$$

are called *row vectors*. Note that for simplicity, the first subscript of each element is dropped. Also, it should be mentioned that there are times when it is desirable to employ a special shorthand notation to distinguish a row matrix from other types of matrices. One way to accomplish this is to employ special open-topped brackets, as in  $|b|$ .<sup>1</sup>

Matrices with column dimension  $n = 1$ , such as

$$(8.3)$$

$$[c] = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_m \end{bmatrix}$$

are referred to as *column vectors*. For simplicity, the second subscript is dropped. As with the row vector, there are occasions when it is desirable to employ a special shorthand notation to distinguish a column matrix from other types of matrices. One way to accomplish this is to employ special brackets, as in  $\{c\}$ .

Matrices where  $m = n$  are called *square matrices*. For example, a  $3 \times 3$  matrix is

$$[A] = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

The diagonal consisting of the elements  $a_{11}$ ,  $a_{22}$ , and  $a_{33}$  is termed the *principal* or *main diagonal* of the matrix.

Square matrices are particularly important when solving sets of simultaneous linear equations. For such systems, the number of equations (corresponding to rows) and the number of unknowns (corresponding to columns) must be equal for a unique solution to be possible. Consequently, square matrices of coefficients are encountered when dealing with such systems.

There are a number of special forms of square matrices that are important and should be noted:

A *symmetric matrix* is one where the rows equal the columns—that is,  $a_{ij} = a_{ji}$  for all  $i$ 's and  $j$ 's. For example,

$$[A] = \begin{bmatrix} 5 & 1 & 2 \\ 1 & 3 & 7 \\ 2 & 7 & 8 \end{bmatrix}$$

is a  $3 \times 3$  symmetric matrix.

A *diagonal matrix* is a square matrix where all elements off the main diagonal are equal to zero, as in

$$[A] = \begin{bmatrix} a_{11} & & \\ & a_{22} & \\ & & a_{33} \end{bmatrix}$$

Note that where large blocks of elements are zero, they are left blank.

An *identity matrix* is a diagonal matrix where all elements on the main diagonal are equal to 1, as in

$$[I] = \begin{bmatrix} 1 & & \\ & 1 & \\ & & 1 \end{bmatrix}$$

The identity matrix has properties similar to unity. That is,

$$[A][I] = [I][A] = [A]$$

An *upper triangular matrix* is one where all the elements below the main diagonal are zero, as in

$$[A] = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ & a_{22} & a_{23} \\ & & a_{33} \end{bmatrix}$$

A *lower triangular matrix* is one where all elements above the main diagonal are zero, as in

$$[A] = \begin{bmatrix} a_{11} & & \\ a_{21} & a_{22} & \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

A *banded matrix* has all elements equal to zero, with the exception of a band centered on the main diagonal:

$$[A] = \begin{bmatrix} a_{11} & a_{12} & & \\ a_{21} & a_{22} & a_{23} & \\ & a_{32} & a_{33} & a_{34} \\ & & a_{43} & a_{44} \end{bmatrix}$$

The preceding matrix has a bandwidth of 3 and is given a special name—the *tridiagonal matrix*.

## 8.1.2 Matrix Operating Rules

Now that we have specified what we mean by a matrix, we can define some operating rules that govern its use. Two  $m$  by  $n$  matrices are equal if, and only if, every element in the first is equal to every element in the second—that is,  $[A] = [B]$  if  $a_{ij} = b_{ij}$  for all  $i$  and  $j$ .

Addition of two matrices, say,  $[A]$  and  $[B]$ , is accomplished by adding corresponding terms in each matrix. The elements of the resulting matrix  $[C]$  are computed as

$$c_{ij} = a_{ij} + b_{ij}$$

for  $i = 1, 2, \dots, m$  and  $j = 1, 2, \dots, n$ . Similarly, the subtraction of two matrices, say,  $[E]$  minus  $[F]$ , is obtained by subtracting corresponding terms, as in

$$d_{ij} = e_{ij} - f_{ij}$$

for  $i = 1, 2, \dots, m$  and  $j = 1, 2, \dots, n$ . It follows directly from the preceding definitions that addition and subtraction can be performed only between matrices having the same dimensions.

Both addition and subtraction are commutative:

$$[A] + [B] = [B] + [A]$$



and associative:

$$([A] + [B]) + [C] = [A] + ([B] + [C])$$

The multiplication of a matrix  $[A]$  by a scalar  $g$  is obtained by multiplying every element of  $[A]$  by  $g$ . For example, for a  $3 \times 3$  matrix:

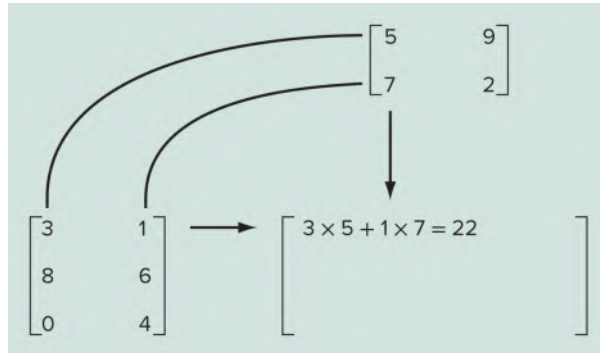
$$[D] = g[A] = \begin{bmatrix} ga_{11} & ga_{12} & ga_{13} \\ ga_{21} & ga_{22} & ga_{23} \\ ga_{31} & ga_{32} & ga_{33} \end{bmatrix}$$

The product of two matrices is represented as  $[C] = [A][B]$ , where the elements of  $[C]$  are defined as

$$c_{ij} = \sum_{k=1}^n a_{ik}b_{kj} \quad (8.4)$$

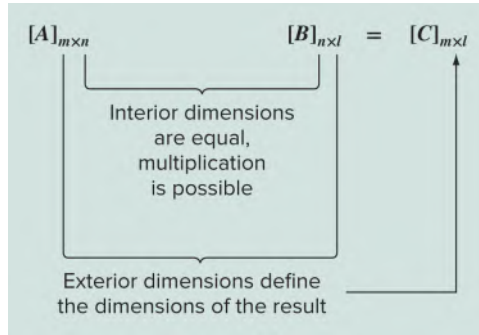
where  $n$  = the column dimension of  $[A]$  and the row dimension of  $[B]$ . That is, the  $c_{ij}$  element is obtained by adding the product of individual elements from the  $i$ th row of the first matrix, in this case  $[A]$ , by the  $j$ th column of the second matrix  $[B]$ . [Figure 8.4](#) depicts how the rows and columns line up in matrix multiplication.

**FIGURE 8.4** Visual depiction of how the rows and columns line up in matrix multiplication.



According to this definition, matrix multiplication can be performed only if the first matrix has as many columns as the number of rows in the second matrix. Thus, if  $[A]$  is an  $m$  by  $n$  matrix,  $[B]$  could be an  $n$  by  $l$  matrix. For this case, the resulting  $[C]$  matrix would have the dimension of  $m$  by  $l$ . However, if  $[B]$  were an  $m$  by  $l$  matrix, the multiplication could not be performed. [Figure 8.5](#) provides an easy way to check whether two matrices can be multiplied.

**FIGURE 8.5** Matrix multiplication can be performed only if the inner dimensions are equal.



If the dimensions of the matrices are suitable, matrix multiplication is *associative*:

$$([A] [B]) [C] = [A] ([B] [C])$$

and *distributive*:

$$[A]([B] + [C]) = [A] [B] + [A] [C]$$

or

$$([A] + [B]) [C] = [A] [C] + [B] [C]$$

However, multiplication is not generally *commutative*:

$$[A] [B] \neq [B] [A]$$

That is, the order of matrix multiplication is important.

Although multiplication is possible, matrix division is not a defined operation. However, if a matrix  $[A]$  is square and nonsingular, there is another matrix  $[A]^{-1}$ , called the *inverse* of  $[A]$ , for which

$$[A][A]^{-1} = [A]^{-1} [A] = [I]$$

Thus, the multiplication of a matrix by the inverse is analogous to division, in the sense that a number divided by itself is equal to 1. That is, multiplication of a matrix by its inverse leads to the identity matrix.

The inverse of a  $2 \times 2$  matrix can be represented simply by

$$[A]^{-1} = \frac{1}{a_{11}a_{22} - a_{12}a_{21}} \begin{bmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{bmatrix}$$

Similar formulas for higher-dimensional matrices are much more involved.  **Chapter 11** will deal with techniques for using numerical methods and the computer to calculate the inverse for such systems.

The *transpose* of a matrix involves transforming its rows into columns and its columns into rows. For example, for the  $3 \times 3$  matrix:

$$[A] = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

the transpose, designated  $[A]^T$ , is defined as

$$[A]^T = \begin{bmatrix} a_{11} & a_{21} & a_{31} \\ a_{12} & a_{22} & a_{32} \\ a_{13} & a_{23} & a_{33} \end{bmatrix}$$

In other words, the element  $a_{ij}$  of the transpose is equal to the  $a_{ji}$  element of the original matrix.

The transpose has a variety of functions in matrix algebra. One simple advantage is that it allows a column vector to be written as a row, and vice versa. For example, if

$$\{c\} = \begin{Bmatrix} c_1 \\ c_1 \\ c_1 \end{Bmatrix}$$

then

$$\{c\}^T = [c_1 \quad c_2 \quad c_3]$$

In addition, the transpose has numerous mathematical applications.

A *permutation matrix* (also called a *transposition matrix*) is an identity matrix with rows and columns interchanged. For example, here is a permutation matrix that is constructed by switching the first and third rows and columns of a  $3 \times 3$  identity matrix:

$$[P] = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

Left multiplying a matrix  $[A]$  by this matrix, as in  $[P][A]$ , will switch the corresponding rows of  $[A]$ . Right multiplying, as in  $[A][P]$ , will switch the corresponding columns. Here is an example of left multiplication:

$$[P][A] = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 2 & -7 & 4 \\ 8 & 3 & -6 \\ 5 & 1 & 9 \end{bmatrix} = \begin{bmatrix} 5 & 1 & 9 \\ 8 & 3 & -6 \\ 2 & -7 & 4 \end{bmatrix}$$

The final matrix manipulation that will have utility in our discussion is *augmentation*. A matrix is augmented by the addition of a column (or columns) to the original matrix. For example, suppose we have a  $3 \times 3$  matrix of coefficients. We might wish to augment this matrix  $[A]$  with a  $3 \times 3$  identity matrix to yield a  $3 \times 6$  dimensional matrix:

$$\left[ \begin{array}{ccc|ccc} a_{11} & a_{11} & a_{11} & 1 & 0 & 0 \\ a_{21} & a_{21} & a_{21} & 0 & 1 & 0 \\ a_{31} & a_{31} & a_{31} & 0 & 0 & 1 \end{array} \right]$$

Such an expression has utility when we must perform a set of identical operations on the rows of two matrices. Thus, we can perform the operations on the single augmented matrix rather than on the two individual matrices.

## EXAMPLE 8.1

### Python Array and Matrix Manipulations

#### Problem Statement

Review the various data structures in Python that contain information and focus on the implementation of matrix manipulations. This example is best approached as a hands-on exercise on the computer.

#### Solution

The simplest data structure in Python that contains a collection of items is the list. Here, the items of interest are numbers. As an example,

```
a = [ 4.3 , 6.5 , 1.9 , -0.4 ]
```

| Name | Type | Size |                       |
|------|------|------|-----------------------|
| a    | list | 4    | [4.3, 6.5, 1.9, -0.4] |

We can certainly reference individual items or subsets of items in a list,

Page 206

```
a [2]      a [1:3]
1.9        [6.5, 1.9]
```

Note: `[1 : 3]` includes only indices 1 and 2 and, since 0 is the first index, we get the 2nd and 3rd items of `a`.

But can we “do math” with a list?

```
3*a
[4.3, 6.5, 1.9, -0.4, 4.3, 6.5, 1.9, -0.4, 4.3, 6.5, 1.9, -0.4]
```

Well, not the way we might think. Here, the list is replicated three times. And,

```
b = [ 12.5 , 33.6 , -129.1 , -11.4 ]
a*b
can't multiply sequence by non-int of type 'list'
```

Similar results occur with the creation and operation on a tuple:

```
a = ( 4.3, 6.5, 1.9, -0.4)
```

| Name | Type  | Size |                       |
|------|-------|------|-----------------------|
| a    | tuple | 4    | (4.3, 6.5, 1.9, -0.4) |

```
3*a
(4.3, 6.5, 1.9, -0.4, 4.3, 6.5, 1.9, -0.4, 4.3, 6.5, 1.9, -0.4)
```

So, these data structures are not the best for implementing array and matrix operations. Via the NumPy module, there is an `ndarray` class. We can create an *array* from a list or a tuple:

```
c = np.array([ 12.5 , 33.6 , -129.1 , -11.4 ])
c
([ 12.5, 33.6, -129.1, -11.4])
a = np.array((4.3, 6.5, 1.9, -0.4))
a
([ 4.3, 6.5, 1.9, -0.4])
```

Now, can we “do math” with arrays?

```
3*c
([ 37.5, 100.8, -387.3, -34.2])
```

Ah, ha! Yes. And,

```
a*c
array([ 53.75, 218.4 , -245.29,  4.56])
```

Here, the respective items of `a` and `c` have been multiplied, that is, an *array operation*.

These arrays are arranged in rows. Can we arrange one in a column?

```
b = np.array([ [12.5] , [33.6] , [-129.1] , [-11.4] ])
b
array([[ 12.5],
       [ 33.6],
       [-129.1],
       [-11.4]])
```

A bit awkward perhaps, but yes. And now, what happens when we multiply `a` times `b`?

```
a*b
array([[ 53.75,  81.25,  23.75, -5.  ],
       [144.48, 218.4 ,  63.84, -13.44],
       [-555.13, -839.15, -245.29,  51.64],
       [-49.02, -74.1 , -21.66,  4.56]])
```

We didn’t expect this. Can you figure out what happened? Most likely, we hoped for an inner product (or “dot” product) of a row vector with a column vector. Operations directly with arrays are called “array operations” for this reason. However, if we try

---

Page 207

```
a.dot(b)
array([31.42])
```

we do get the inner product, and it is still an array type structure. We can also use

```
np.inner(c,a)
31.4200000000000044      The result is not an array type.

np.outer(c,a)
array([[ 53.75,  81.25,  23.75, -5.  ],
       [144.48, 218.4 ,  63.84, -13.44],
       [-555.13, -839.15, -245.29,  51.64],
       [-49.02, -74.1 , -21.66,  4.56]])
```

for inner and outer products, and we can perform a transpose operation and matrix multiplication as shown here:

```
A = np.array([ [ 3 , 2 ] , [ 6 , 8 ] ])
A
array([[3, 2],
       [6, 8]])
```

```

A.transpose()          rows and columns switch
array([[3, 6],
       [2, 8]])

B = np.array( [ [ -6 , 20 ] , [ 7 , -11 ] ] )
B
array([[ -6, 20],
       [ 7, -11]])

A.dot(B)               accomplishes matrix multiplication
array([[ -4, 38],
       [20, 32]])

```

Three useful matrices can be created using the NumPy functions: `zeros` , `ones` and `eye`—notice, for the `eye` function, the `[ ]`'s are not required

```

np.zeros([3,4])
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])

np.ones([4,3])
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.]])

np.eye(5,5)
array([[1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       [0., 0., 1., 0., 0.],
       [0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 1.]])

```

Any math operations involving array types are attempted item by item

```

np.sqrt(A)
array([[1.73205081, 1.41421356],
       [2.44948974, 2.82842712]])

```

Having reviewed all the above, we now introduce the use of another data structure for matrix math, a special version of the array, the *matrix structure*.

```

A = np.matrix([[3.6,1.2,-5.7],[12.9,-9.8,0.4],[10.6,2.9,-4.7 ]])
A
matrix([[ 3.6,  1.2, -5.7],
        [12.9, -9.8,  0.4],
        [10.6,  2.9, -4.7]])

```

A matrix type is always two-dimensional. Even a row or column matrix (or vector) has dimensions `[ 1 , m ]` or `[ n , 1 ]` , respectively.

---

Page 208

---

```

q
matrix([[23, 10, -5]])

q[0,1]
10

```

If you try `q[ 1 ]` , you will get an error.

Using the standard math operators (`+` , `-` , `*` ) , “matrix-like” operations will take place.

```

q*A
matrix([[ 158.8, -84.9, -103.6]])

```

matrix multiplication of A by itself

```

A*A

```

```
matrix([[ -31.98, -23.97,  6.75],
        [-75.74, 112.68, -79.33],
        [ 25.75, -29.33, -37.17]])
```

But not the / operator:

```
1/A
matrix([[ 0.27777778,  0.83333333, -0.1754386 ],
        [ 0.07751938, -0.10204082,  2.5 ],
        [ 0.09433962,  0.34482759, -0.21276596]])
```

This is not the matrix inverse, but the inverse of each item in A .

We can also assign matrices using a simpler syntax, similar to what is used in the MATLAB software.

```
A = np.matrix(' 3.6 1.2 -5.7 ; 12.9 -9.8 0.4 ; 10.6 2.9 -4.7 ')

A
matrix([[ 3.6, 1.2, -5.7],
        [12.9, -9.8, 0.4],
        [10.6, 2.9, -4.7]])

r = np.matrix(' 2 4 6 ')

p = np.matrix(' 1 ; 3 ; 5 ')

r
matrix([[2, 4, 6]])

p
matrix([[1],
        [3],
        [5]])
```

The inverse of a matrix can be computed using the `np.linalg.inv` function.

```
Ainv = np.linalg.inv(A)

Ainv
matrix([[ -0.07934699,  0.01924474,  0.09786717],
        [-0.11463784, -0.07687292,  0.13248649],
        [-0.24968676, -0.0040292 ,  0.08970274]])
```

Then, we can test the inverse `Ainv` by multiplying it with `A`:

```
Ainv = np.linalg.inv(A)

Ainv*A
matrix([[ 1.00000000e+00,  3.29410122e-17,  2.42486610e-17],
        [-2.85766608e-16,  1.00000000e+00, -8.73721529e-17],
        [-1.01257754e-16, -4.26616733e-17,  1.00000000e+00]])
```

The product is essentially the identity matrix with one's on the diagonal and insignificant numbers elsewhere.

And there are other useful functions part of `np.linalg`:

- The Frobenius norm of A—the square root of the sum of squares of all the elements of A

```
np.linalg.norm(A)

21.26875642815066
```

- The determinant of A

```
np.linalg.det(A)

-565.8690000000003
```

- Matrix rank of A, here full rank

```
np.linalg.matrix_rank(A)
3
```

- The sum of the diagonal elements of A

```
np.trace(A)
-10.900000000000002
```

- Transpose—rows and columns are switched

```
np.transpose(A)
matrix([[ 3.6, 12.9, 10.6],
        [ 1.2, -9.8, 2.9],
        [-5.7, 0.4, -4.7]])
```

But we can also use

```
A.trace()
matrix([[ -10.9]])
```

Note that this is returned as a matrix type.

```
A.transpose()
matrix([[ 3.6, 12.9, 10.6],
        [ 1.2, -9.8, 2.9],
        [-5.7, 0.4, -4.7]])
```

which also applies to arrays in general.

If we want to combine arrays or matrices, we can use the “stack” functions. If we define:

```
x = np.array([8, 6, 9])
y = np.array([-5, 8, 1])
z = np.array([4, 8, 2])
```

- Separate rows stacked on top of each other

```
B = np.vstack((x,y,z))
B
array([[ 8, 6, 9],
       [-5, 8, 1],
       [ 4, 8, 2]])
```

- Separate rows stacked side by side

```
C = np.hstack((x,y,z))
C
array([ 8, 6, 9, -5, 8, 1, 4, 8, 2])
```

- Separate columns stacked side by side

```
D = np.dstack((x,y,z))
D
array([[[ 8, -5, 4],
        [ 6, 8, 8],
        [ 9, 1, 2]]])
```

There are also “split” routines for the reverse operations.

For matrices, we can use the colon (:) index for all or ranges of indices.

```
B1 = B[1,:]  
B1
```



```
array([-5, 8, 1])          2nd row, all columns
B2 = B[0:2,:]              0:2 does not include 2, just 0 and 1 (tricky!)
B2
array([[ 8, 6, 9],
       [-5, 8, 1]])
```

Finally, you can extract the dimensions of an array or matrix with

```
B2.shape
(2, 3)                      2 rows, 3 columns
```

The purpose of this example is to show you (and have you experience hands-on) many of the ways to define and operate with arrays and matrices using Python and the NumPy module. Later in the chapter, we will illustrate how to put these to use to solve systems of linear algebraic equations. And you will gain much more familiarity with all of this as the coming chapters proceed. Be patient!

## 8.1.3 Representing Linear Algebraic Equations in Matrix Form

It should be clear that matrices provide a concise notation for representing simultaneous linear equations. For example, a  $3 \times 3$  set of linear equations,

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 &= b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 &= b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3 \end{aligned} \quad (8.5)$$

can be expressed as

$$[A] \{x\} = \{b\} \quad (8.6)$$

where  $[A]$  is the matrix of coefficients:

$$[A] = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

$\{b\}$  is the column vector of constants:

$$\{b\}^T = [b_1 \quad b_2 \quad b_3]$$

and  $\{x\}$  is the column vector of unknowns:

$$\{x\}^T = [x_1 \quad x_2 \quad x_3]$$

Recall the definition of matrix multiplication [Eq. (8.4)] to convince yourself that Eqs. (8.5) and (8.6) are equivalent. Also, realize that Eq. (8.6) is a valid matrix multiplication because the number of columns  $n$  of the first matrix  $[A]$  is equal to the number of rows  $n$  of the second matrix  $\{x\}$ .

This part of the book is devoted to solving Eq. (8.6) for  $\{x\}$ . A formal way to obtain a solution using matrix algebra is to multiply each side of the equation by the inverse of  $[A]$  to yield

$$[A]^{-1} [A] \{x\} = [A]^{-1} \{b\}$$

Because  $[A]^{-1} [A]$  equals the identity matrix, the equation becomes

$$\{x\} = [A]^{-1} \{b\} \quad (8.7)$$

Therefore, the equation has been solved for  $\{x\}$ . This is another example of how the inverse plays a role in matrix algebra that is similar to division. It should be noted that this is not a very efficient way to solve a system of equations. Thus, other approaches are employed in numerical algorithms. However, as discussed in Sec. 11.1.2, the matrix inverse itself has great value in the engineering analyses of such systems.

It should be noted that systems with more equations (rows) than unknowns (columns),  $m > n$ , are said to be *overdetermined*. A typical example is least-squares regression where an equation with  $n$  coefficients is fit to  $m$  data points  $(x, y)$ . Conversely, systems with less equations than unknowns,  $m < n$ , are said to be *underdetermined*. A typical example of underdetermined systems is numerical optimization.

## 8.2 SOLVING LINEAR ALGEBRAIC EQUATIONS WITH PYTHON

Python's NumPy module provides two ways to solve systems of linear algebraic equations. The most efficient way is to employ the `solve` function available in the NumPy `linalg` submodule.

```
x = np.linalg.solve(A,b)
```

The second is to use matrix inversion and matrix multiplication explicitly:

```
x = np.linalg.inv(A)*b
```

As stated at the end of [Sec. 8.1.3](#), the matrix inverse solution is less efficient than using the `solve` function directly. Both options are illustrated in the following example.

### EXAMPLE 8.2

#### Solving the Bungee Jumper Problem with Python

##### Problem Statement

Use Python to solve the bungee jumper problem described at the beginning of this chapter. The parameters for the problem are

| Jumper     | Mass (kg) | Spring Constant (N/m) | Unstretched Cord Length (m) |
|------------|-----------|-----------------------|-----------------------------|
| Top (1)    | 60        | 50                    | 20                          |
| Middle (2) | 70        | 100                   | 20                          |
| Bottom (3) | 80        | 50                    | 20                          |

##### Solution

Substituting these parameter values into [Eq. \(8.2\)](#) gives

$$\begin{bmatrix} 150 & -100 & 0 \\ -100 & 150 & -50 \\ 0 & -50 & -50 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 588.6 \\ 686.7 \\ 784.8 \end{bmatrix}$$

Enter the following Python code to assign the coefficient matrix to `A` and the constant vector to `b`:

```
import numpy as np

A = np.matrix(' 150 -100 0 ; -100 150 -50 ; 0 -50 50 ')
b = np.matrix(' 588.6 ; 686.7 ; 784.8 ')
```

Employing the `solve` function from the NumPy `linalg` submodule yields

```
x = np.linalg.solve(A,b)
```

```
print(' Solution is ',x)
Solution is [[41.202]
            [55.917]
            [71.613]]
```

Alternately, multiplying the inverse of the coefficient matrix times the constant vector gives the same result:

```
x = np.linalg.inv(A)*b
print(' 2nd solution is ',x)
2nd solution is [[41.202]
                [55.917]
                [71.613]]
```

Because the jumpers were connected by 20-m cords, their initial positions relative to the platform can be assigned to a column vector,  $x_i$ ,

```
xi = np.matrix(' 20 ; 40 ; 60 ' )
```

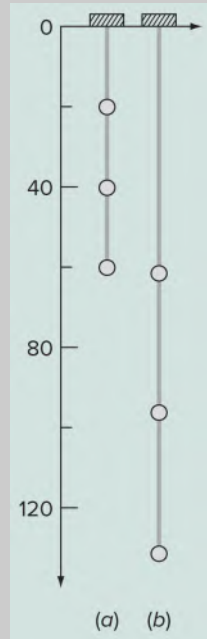
and their final positions can be calculated as

```
xf = x + xi
print(' Final positions:\n ',xf)
Final positions:
[[ 61.202]
 [ 95.917]
 [131.613]]
```

The results, which are displayed in  **Fig. 8.6**, make sense. The first cord is extended the longest because it has a lower value for its spring constant and is subject to the most weight (all three jumpers). Notice that the second and third cords are extended about the same amount. Because it is subject to the weight of two jumpers, one might expect the second cord to be extended longer than the third. However, because it is stiffer, that is, it has a higher spring constant, it stretches less than expected based on the weight it carries.

---

**FIGURE 8.6** Positions of three individuals connected by three bungee cords. (a) Unstretched and (b) stretched.

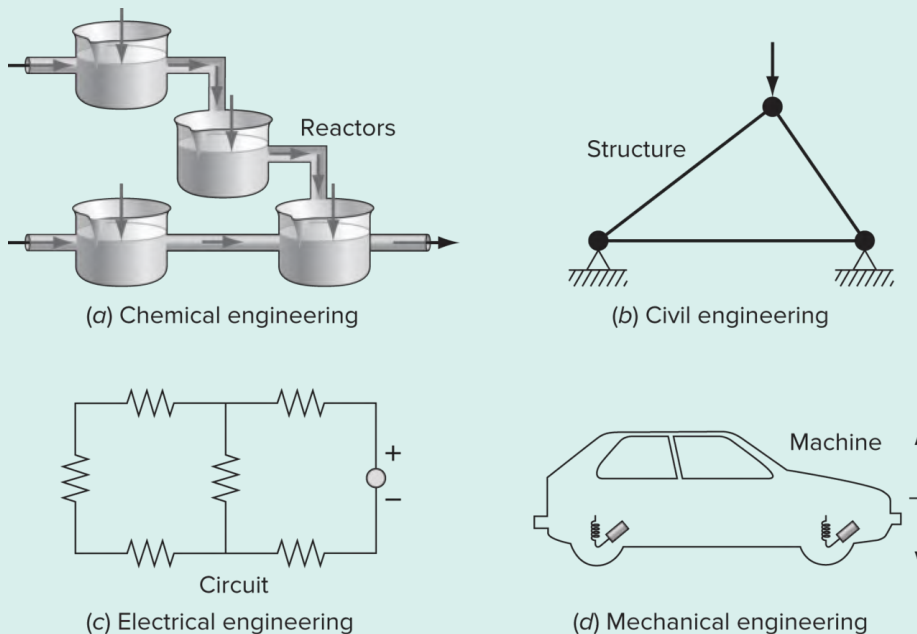


## CURRENTS AND VOLTAGES IN CIRCUITS

### Background

Recall that, in [Chap. 1](#), we summarized some models and associated conservation laws that figure prominently in engineering. As shown in [Fig. 8.7](#), each model represents a system of interacting elements. Consequently, steady-state balances derived from the conservation laws yield systems of simultaneous equations. In many cases, such systems are linear and hence can be expressed in matrix form. The present case study focuses on one such application: circuit analysis.

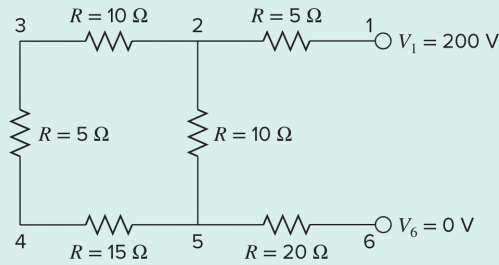
**FIGURE 8.7** Engineering systems which, at steady state, can be modeled with linear algebraic equations.



A common problem in electrical engineering involves determining the currents and voltages at various locations in direct-current circuits with resistor networks. These problems are typically solved using *Kirchoff's current rule* and *Ohm's law*. The *current (or node) rule* states that the algebraic sum of all currents entering a node must be zero [\(Fig. 8.8a\)](#), or

$$\sum i = 0$$

**FIGURE 8.8** A resistive circuit to be solved using linear algebraic equations.



Here, the  $i$  variables are summed, which implies that their values will include both positive and negative quantities. The convention is that  $i$  represents the current into the node, and, if its value is negative, that implies that the current is actually out of the node. The current rule is an application of the principle of *conservation of charge*.

Ohm's law allows us to relate direct current through a resistor to the potential or voltage across the resistor, as in

Page 213

$$\Delta V = iR$$

As we consider these voltage “drops” around a loop of the circuit, they must sum to zero. (Colloquially, you have to return to the place from which you started.) So, Kirchhoff's voltage loop rule suggests that

$$\sum \Delta V_{\text{loop}} = 0$$

This is an expression of the *conservation of energy*. Of course, a static circuit may contain more than resistors. For example, there may be a battery or power supply in the circuit. These contribute to the potential or voltage differences in the circuit, but the sum of all elements in a loop must still be zero.

## Solution

Application of these rules results in systems of simultaneous linear algebraic equations because various loops within the circuit are interconnected. As an example, consider the resistive circuit shown in [Fig. 8.8](#).

Studying the figure, we recognize four nodes that represent points where the currents must sum to zero. These are denoted 2 through 5 on the figure. Nodes 1 and 6 represent imposed potentials, possibly provided by a battery or power supply, as in node 1, or a connection to ground, as in node 6.

Consequently, we will write four equations for the nodes using the convention that currents sum to zero:

Page 214

$$\text{Node 2: } i_{12} + i_{32} + i_{52} = 0$$

$$\text{Node 3: } i_{23} + i_{43} = 0$$

$$\text{Node 4: } i_{34} + i_{54} = 0$$

$$\text{Node 5: } i_{45} + i_{25} + i_{65} = 0$$

The subscripts indicate the direction of the variable into the node. As we stated before, the current values will be mixes of positive and negative numbers to preserve conservation of charge.

Now, we are going to employ Ohm's law to state the equations above in terms of the voltages in the circuit.

$$\begin{aligned}
 \text{Node 2: } & \frac{V_1 - V_2}{R_{12}} + \frac{V_3 - V_2}{R_{23}} + \frac{V_5 - V_2}{R_{25}} = 0 \\
 \text{Node 3: } & \frac{V_2 - V_3}{R_{23}} + \frac{V_4 - V_3}{R_{34}} = 0 \\
 \text{Node 4: } & \frac{V_3 - V_4}{R_{34}} + \frac{V_5 - V_4}{R_{45}} = 0 \\
 \text{Node 5: } & \frac{V_4 - V_5}{R_{45}} + \frac{V_2 - V_5}{R_{25}} + \frac{V_6 - V_5}{R_{56}} = 0
 \end{aligned}$$

Notice that the voltage differences entered for each current term follow the subscript of that term with the node voltage always being the second with the negative sign. This follows the “current into the node” convention. The resistor subscripts do not have a particular order because resistors are not directional.

With some algebraic manipulation, the four equations above can be rearranged into the following matrix form:

$$\begin{bmatrix}
 \left(\frac{1}{R_{12}} + \frac{1}{R_{23}} + \frac{1}{R_{25}}\right) & -\left(\frac{1}{R_{23}}\right) & 0 & -\left(\frac{1}{R_{25}}\right) \\
 -\left(\frac{1}{R_{23}}\right) & \left(\frac{1}{R_{23}} + \frac{1}{R_{34}}\right) & -\left(\frac{1}{R_{34}}\right) & 0 \\
 0 & -\left(\frac{1}{R_{34}}\right) & \left(\frac{1}{R_{34}} + \frac{1}{R_{45}}\right) & -\left(\frac{1}{R_{45}}\right) \\
 -\left(\frac{1}{R_{25}}\right) & 0 & -\left(\frac{1}{R_{45}}\right) & \left(\frac{1}{R_{25}} + \frac{1}{R_{45}} + \frac{1}{R_{56}}\right)
 \end{bmatrix}
 \begin{bmatrix}
 V_2 \\
 V_3 \\
 V_4 \\
 V_5
 \end{bmatrix}
 =
 \begin{bmatrix}
 \frac{1}{R_{12}}V_1 \\
 0 \\
 0 \\
 \frac{1}{R_{56}}V_6
 \end{bmatrix}$$

Once all voltages are known, we can use Ohm’s law to calculate all the currents.

Although impractical to solve by hand, this system is easily handled by Python with the NumPy `linalg.solve` function. The solution is

```

import numpy as np

# set the parameter values
R12 = 5 # ohms
R23 = 10
R25 = 10
R34 = 5
R45 = 15
R56 = 20

V1 = 200 # V
V6 = 0

# set up the coefficient matrix and the constant vector
A = np.matrix([[ 1/R12+1/R23+1/R25, -1/R23,      0,      -1/R25],
               [-1/R23,      1/R23+1/R34, -1/R34,      0],
               [ 0,      -1/R34,      1/R34+1/R45, -1/R45],
               [-1/R25,      0,      -1/R45,      1/R25+1/R45+1/R56]])
b = np.matrix([[V1/R12],[0],[0],[V6/R56]])

# solve for the voltages
V = np.linalg.solve(A,b)

# assign voltages to familiar variables
V2 = V[0,0]
V3 = V[1,0]
V4 = V[2,0]
V5 = V[3,0]

# display voltages
print('\nV2 = {0:6.2f} volts'.format(V2))
print('V3 = {0:6.2f} volts'.format(V3))
print('V4 = {0:6.2f} volts'.format(V4))
print('V5 = {0:6.2f} volts'.format(V5))

# calculate currents
i12 = (V1-V2)/R12
i23 = (V2-V3)/R23

```



```

i25 = (V2-V5)/R25
i34 = (V3-V4)/R34
i45 = (V4-V5)/R45
i56 = (V5-V6)/R56

# display currents
print('\ni12 = {0:6.2f} amps'.format(i12))
print('i23 = {0:6.2f} amps'.format(i23))
print('i25 = {0:6.2f} amps'.format(i25))
print('i34 = {0:6.2f} amps'.format(i34))
print('i45 = {0:6.2f} amps'.format(i45))
print('i56 = {0:6.2f} amps'.format(i56))

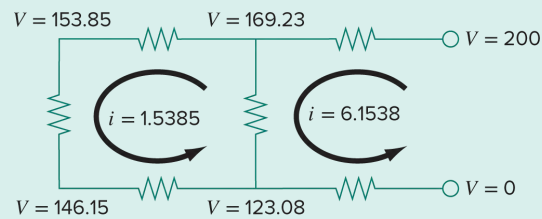
V2 = 169.23 volts
V3 = 153.85 volts
V4 = 146.15 volts
V5 = 123.08 volts

i12 = 6.15 amps
i23 = 1.54 amps
i25 = 4.62 amps
i34 = 1.54 amps
i45 = 1.54 amps
i56 = 6.15 amps

```

The circuit currents and voltages are shown in [Fig. 8.9](#). From the results above, you can see that Kirchhoff's loop current rule is satisfied. The advantages of Python and NumPy for problems of this nature should be evident.

**FIGURE 8.9** The solution for currents and voltages obtained using Python.



# PROBLEMS

**8.1** Given a single square matrix,  $A$ , write Python statement(s) that will create a new matrix,  $Aug$ , that consists of the original  $A$  matrix augmented (to the right) by an identity matrix,  $I$ .

**8.2** Several matrices are defined as follows:

$$A = \begin{bmatrix} 4 & 7 \\ 1 & 2 \\ 5 & 6 \end{bmatrix} \quad B = \begin{bmatrix} 4 & 3 & 7 \\ 1 & 2 & 7 \\ 2 & 0 & 4 \end{bmatrix} \quad C = \begin{bmatrix} 3 \\ 6 \\ 1 \end{bmatrix}$$

$$D = \begin{bmatrix} 9 & 4 & 3 & -6 \\ 2 & -1 & 7 & 5 \end{bmatrix} \quad E = \begin{bmatrix} 1 & 5 & 8 \\ 7 & 2 & 3 \\ 4 & 0 & 6 \end{bmatrix}$$

$$F = \begin{bmatrix} 3 & 0 & 1 \\ 1 & 7 & 3 \end{bmatrix} \quad G = \begin{bmatrix} 7 & 6 & 4 \end{bmatrix}$$

Answer the following questions regarding these matrices:

- What are the dimensions of the matrices?
- Identify the square, column and row matrices?
- What are the values of these elements?  $a_{12}$ ,  $b_{23}$ ,  $d_{32}$ ,  $e_{22}$ ,  $f_{12}$ , and  $g_{12}$
- Perform the following operations:
  - $E + B$
  - $A + F$
  - $B - E$
  - $7 \times B$
  - $C^T$
  - $E \times B$
  - $B \times A$
  - $D^T$
  - $A \times C$
  - $I \times B$
  - $E^T \times E$
  - $C^T \times C$

**8.3** Write the following set of equations in matrix form:

$$\begin{aligned} 50 &= 5x_3 - 7x_2 \\ 4x_2 + 7x_3 + 30 &= 0 \\ x_1 - 7x_3 &= 40 - 3x_2 + 5x_1 \end{aligned}$$

Use Python to solve for the unknowns. In addition, compute the transpose and inverse of the coefficient matrix.

**8.4** Three matrices are defined as

$$A = \begin{bmatrix} 6 & -1 \\ 12 & 8 \\ -5 & 4 \end{bmatrix} \quad B = \begin{bmatrix} 4 & 0 \\ 0.5 & 2 \end{bmatrix} \quad C = \begin{bmatrix} 2 & -2 \\ 3 & 1 \end{bmatrix}$$

- Perform all possible multiplications between pairs of these matrices.
- Justify why the remaining pairs cannot be multiplied.
- Use the results of (a) to illustrate why the order of multiplication is important.

**8.5** Solve the following system of complex linear equations with Python:

$$\begin{bmatrix} 3+2j & 4 \\ -j & 1 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} 2+j \\ 3 \end{bmatrix}$$

Hints: The `solve` function from the NumPy `linalg` submodule will not work with complex linear equations. You can, however, compute the inverse using the `linalg.inv` function and multiply complex matrices. Also, you cannot enter a complex coefficient as `3+2j` in a matrix definition, but rather as `complex(3,2)`.

**8.6** Develop, debug, and test your own Python function called `matmult` to multiply two matrices, that is,  $X = YZ$ , where  $Y$  is  $m \times n$  and  $Z$  is  $n \times p$ . Employ nested *for* loops to implement the multiplication and include error traps to flag bad cases. Test the program, including bad cases, with matrices from [Prob. 8.4](#).

**8.7** Develop, debug, and test your own Python function called `tranmat` to generate the transpose of a matrix. Employ nested *for* loops to implement the transpose. Test it on matrices from [Prob. 8.4](#).

**8.8** Develop, debug, and test your own Python function called `rowperm` to switch the rows of a matrix using a permutation matrix. The first lines of the function should be as follows:

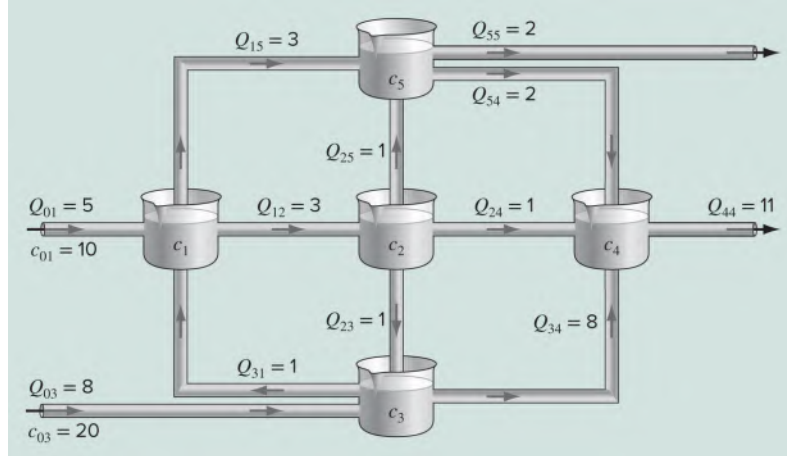
```
import numpy as np
def rowperm(A,r1,r2):
    """
    rowperm: switches rows of matrix A by
    multiplying by a permutation matrix
    PA = rowperm(A,r1,r2)
    input:
    A = original matrix
    r1, r2 = rows to be switched
    output:
    PA = matrix A with rows r1 and r2 switches
    """
```

Include error traps for erroneous inputs, for example, the user specifies rows that exceed the dimensions of the original matrix.

**8.9** Five tanks linked by pipes are shown in [Fig. P8.9](#). The mass flow rate of a constituent through each pipe is computed as the product of the volumetric flow rate,  $Q$  (liters/min), and the concentration of that constituent,  $c$  (grams/liter). At steady-state conditions, the mass flow rate into and out of each reactor must be the same. Since each tank is well mixed, the concentration flowing out of the tank is the same as that in the tank. For example, for the first tank, a *mass balance* can be written as

$$Q_{01}c_{01} + Q_{31}c_3 = Q_{15}c_1 + Q_{12}c_1$$

**FIGURE P8.9**



Write mass balances for the remaining tanks shown in [Fig. P8.9](#) and express the set of linear equations in matrix form. Then, write Python code to solve the equations for the concentrations in each tank.

**8.10** An important problem in structural engineering is that of finding forces in a statically determinate truss ([Fig. P8.10](#)). This type of structure can be described by a system of coupled linear algebraic equations derived from force balances. Based on free-body diagrams for each node, the sum of the forces in both horizontal and vertical directions must be zero at each node because the system is at rest. Therefore, for node 1:

$$\begin{aligned}\sum F_H = 0 &= -F_1 \cos(30^\circ) + F_3 \cos(60^\circ) + F_{1,h} \\ \sum F_V = 0 &= -F_1 \sin(30^\circ) - F_3 \sin(60^\circ) + F_{1,v}\end{aligned}$$

for node 2:

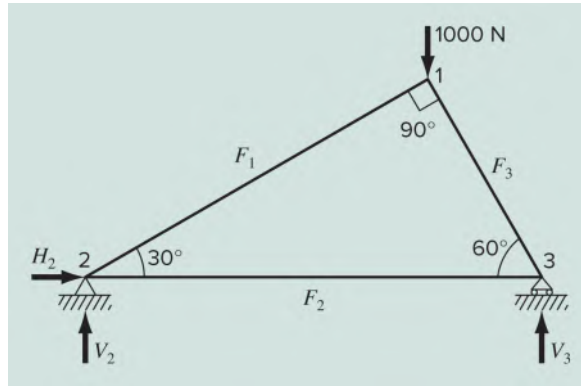
$$\begin{aligned}\sum F_H = 0 &= F_2 - F_1 \cos(30^\circ) + F_{2,h} + H_2 \\ \sum F_V = 0 &= F_1 \sin(30^\circ) + F_{2,v} + V_2\end{aligned}$$

and for node 3:

$$\begin{aligned}\sum F_H = 0 &= -F_3 \cos(60^\circ) + F_{3,h} \\ \sum F_V = 0 &= F_3 \cos(60^\circ) + F_{3,v} + V_3\end{aligned}$$

where  $F_{i,h}$  is the external horizontal force applied to node  $i$ , where positive force is to the right, and  $F_{i,v}$  is the external vertical force applied to node  $i$ , where positive force is upward. Thus, in this problem, the 2000-N downward force on node 1 corresponds to  $F_{1,v} = -2000$ . And for this problem, all other  $F_{i,h}$ 's and  $F_{i,v}$ 's are zero. Express this set of linear algebraic equations in matrix form, and then use Python to solve for the unknowns.

**FIGURE P8.10**



**8.11** Consider the three mass–four spring system shown in [Fig. P8.11](#). Determining the equations of motion from  $\sum F_x = ma_x$  for each mass using its free-body diagram results in the following differential equations:

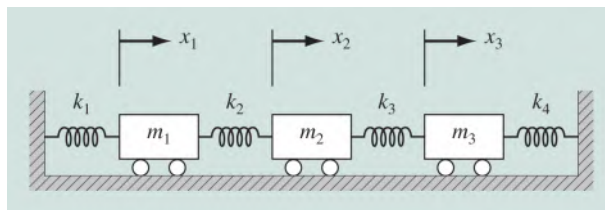
$$\begin{aligned}\ddot{x}_1 + \left(\frac{k_1 + k_2}{m_1}\right)x_1 - \left(\frac{k_2}{m_1}\right)x_2 &= 0 \\ \ddot{x}_2 - \left(\frac{k_2}{m_2}\right)x_1 + \left(\frac{k_2 + k_3}{m_2}\right)x_2 - \left(\frac{k_3}{m_2}\right)x_3 &= 0 \\ \ddot{x}_3 - \left(\frac{k_3}{m_3}\right)x_2 + \left(\frac{k_3 + k_4}{m_3}\right)x_3 &= 0\end{aligned}$$

where  $x_i$  is the displacement to the right of mass  $i$  from equilibrium,

$k_1 = k_4 = 10 \text{ N/m}$ ,  $k_2 = k_3 = 30 \text{ N/m}$ , and  $m_1 = m_2 = m_3 = 1 \text{ kg}$ . The three equations can be written in matrix form:

$$0 = [\text{acceleration vector } \ddot{x}] + [k/m \text{ matrix}][\text{displacement vector } x]$$

**FIGURE P8.11**



The  $k/m$  matrix takes on a *tridiagonal form*. At a specific time,  $x_1 = 0.05 \text{ m}$ ,  $x_2 = 0.04 \text{ m}$ , and  $x_3 = 0.03 \text{ m}$ . Compute the acceleration vector for these conditions.

**8.12** Perform the same computation as in [Example 8.2](#), but use five jumpers with the following characteristics:

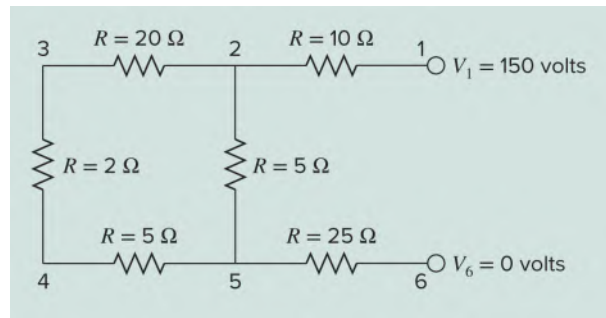
| Jumper | Mass (kg) | Spring Constant (N/m) | Unstretched Cord Length (m) |
|--------|-----------|-----------------------|-----------------------------|
| 1      | 55        | 80                    | 10                          |
| 2      | 75        | 50                    | 10                          |
| 3      | 60        | 70                    | 10                          |
| 4      | 75        | 100                   | 10                          |
| 5      | 90        | 20                    | 10                          |

**8.13** Three masses are suspended vertically by a series of identical springs where mass 1 is at the top and mass 3 is at the bottom. If  $m_1 = 2$  kg,  $m_2 = 3$  kg, and  $m_3 = 2.5$  kg, and the  $k/s = 10$  N/m, write a Python script to solve for the displacements.

Page 218

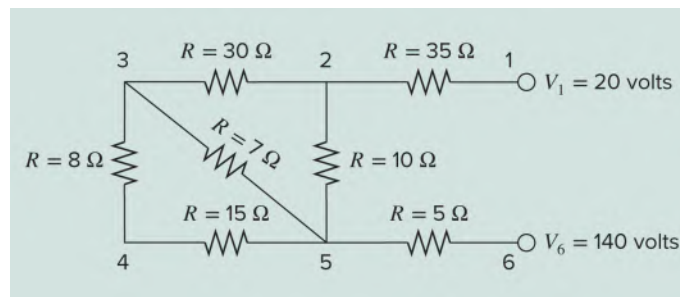
**8.14** Perform the same computation as in [Fig. P8.3](#), but for the circuit in [Fig. P8.14](#).

**FIGURE P8.14**



**8.15** Perform the same computation as in [Fig. P8.3](#), but for the circuit in [Fig. P8.15](#).

**FIGURE P8.15**

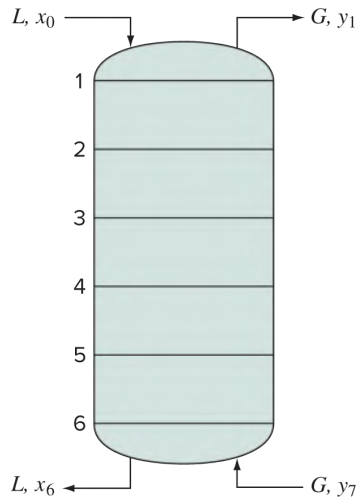


**8.16** Stripper columns are used to remove unwanted pollutants from liquid streams. The process is illustrated in [Fig. P8.16](#) for a six-stage column. The liquid enters at the top and exits the bottom, while the gas enters at the bottom and leaves at the top. Through a series of contact stages, the liquid and gas mix and establish a mass transfer equilibrium on each stage. Gradually, the unwanted compounds

are transferred to the gas stream and can be treated further. Consider a simple model for the six-stage column in [Fig. P8.16](#). The equilibrium relationship between liquid ( $x_m$ ) and vapor ( $y_m$ ) mole fraction on stage  $m$  is given by

$$y_m = ax_m + b$$

**FIGURE P8.16** Stripper column.



The inlet compositions  $x_0$  and  $y_7$ , are specified along with the liquid ( $L$ ) and gas ( $G$ ) molar flow rates. It is desired to calculate the compositions on all stages, in particular, the exit compositions  $y_7$  and  $x_6$ . To do this, a component material balance is written on a representative stage  $n$ ,

$$Lx_{n-1} + Gy_{n+1} = Lx_n + Gy_n$$

Incorporating the equilibrium relationship, this becomes

$$Lx_{n-1} - (L + Ga)x_n + Gax_{n+1} = 0$$

The entire set of equations for the column is

$$\begin{aligned} -(L + Ga)x_1 + Gax_2 &= -Lx_0 \\ Lx_1 - (L + Ga)x_2 + Gax_3 &= 0 \\ Lx_2 - (L + Ga)x_3 + Gax_4 &= 0 \\ Lx_3 - (L + Ga)x_4 + Gax_5 &= 0 \\ Lx_4 - (L + Ga)x_5 + Gax_6 &= 0 \\ Lx_5 - (L + Ga)x_6 &= -G(y_7 + b) \end{aligned}$$

Write a Python script to solve for the liquid compositions,  $x$ 's, then  $y$ 's. Create a plot of these versus stage number. The parameter values are

|                             |                        |
|-----------------------------|------------------------|
| Liquid flow rate:           | $L = 20 \text{ mol/s}$ |
| Inlet liquid mole fraction: | $x_0 = 0$              |

|                          |                        |
|--------------------------|------------------------|
| Gas flow rate:           | $G = 12 \text{ mol/s}$ |
| Inlet gas mole fraction: | $y_7 = 0.1$            |
| Equilibrium model:       | $a = 0.7 \quad b = 0$  |

Suggestion: You will note that the coefficient matrix for this system of equations is tri-diagonal with the same arrangement of values in each row. Create a coefficient matrix with all zeros, and then set the tri-diagonal elements with a *for* loop. In this way, you could just as well set up a problem for 100 stages instead of just six.



# Gauss Elimination

## CHAPTER OBJECTIVES

The primary objective of this chapter is to describe the Gauss elimination algorithm for solving linear algebraic equations. Specific objectives and topics covered are

- Knowing how to solve small sets of linear equations with the graphical method and Cramer's rule.
- Understanding how to implement forward elimination and back substitution as in Gauss elimination.
- Understanding how to count flops to evaluate the efficiency of an algorithm.
- Understanding the concepts of singularity and ill-condition.
- Understanding how partial pivoting is implemented and how it differs from complete pivoting.
- Knowing how to compute the determinant as part of the Gauss elimination algorithm with partial pivoting.
- Recognizing how the banded structure of a tridiagonal system can be exploited to obtain extremely efficient solutions.

At the end of [Chap. 8](#), we showed how Python's NumPy module can be used to solve systems of linear equations via two simple and direct methods:

```
np.linalg.solve(A,b)
```

and

```
np.linalg.inv(A)*b
```

[Chapters 9](#) and [10](#) provide background on how such solutions are obtained. This material is to provide insight into what is “behind the curtain” of the Python NumPy functions above. In addition, it is intended to show how you can build your own solution algorithms in computational environments that do not have Python's built-in capabilities.

The technique described in this chapter is called Gauss elimination because it involves combining equations to eliminate unknowns. Although it is one of the earliest methods for solving simultaneous equations, it remains among the most important algorithms in use today and is the basis for linear equation solving on many popular software packages including Python NumPy.

## 9.1 SOLVING SMALL NUMBERS OF EQUATIONS

Before proceeding to Gauss elimination, we will describe several methods that are appropriate for solving small ( $n \leq 3$ ) sets of simultaneous equations and that do not require a computer. These are the graphical method, Cramer's rule, and the elimination of unknowns.

### 9.1.1 The Graphical Method

A graphical solution is obtainable for two linear equations by plotting them on Cartesian coordinates with one axis corresponding to  $x_1$  and the other to  $x_2$ . Because the equations are linear, each equation will plot as a straight line. For example, suppose that we have the following equations:

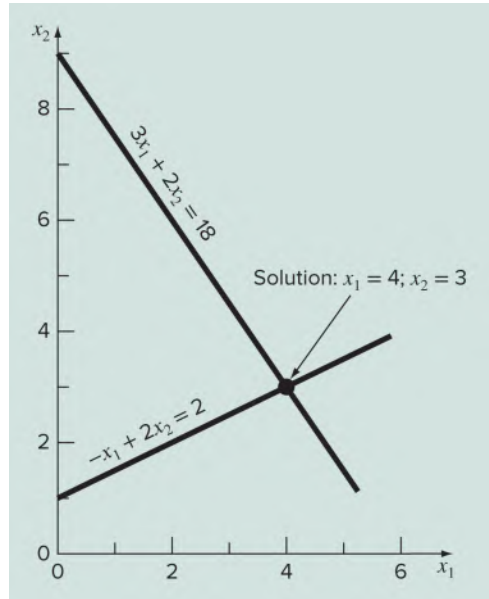
$$\begin{aligned}3x_1 + 2x_2 &= 18 \\ -x_1 + 2x_2 &= 2\end{aligned}$$

If we assume that  $x_1$  is the abscissa, we can solve each of these equations for  $x_2$ :

$$\begin{aligned}x_2 &= -\frac{3}{2}x_1 + 9 \\ x_2 &= \frac{1}{2}x_1 + 1\end{aligned}$$

The equations are now in the form of straight lines—that is,  $x_2 = (\text{slope})x_1 + \text{intercept}$ . When these equations are graphed, the values of  $x_1$  and  $x_2$  at the intersection of the lines represent the solution (  **Fig. 9.1**). For this case, the solution is  $x_1 = 4$  and  $x_2 = 3$ .

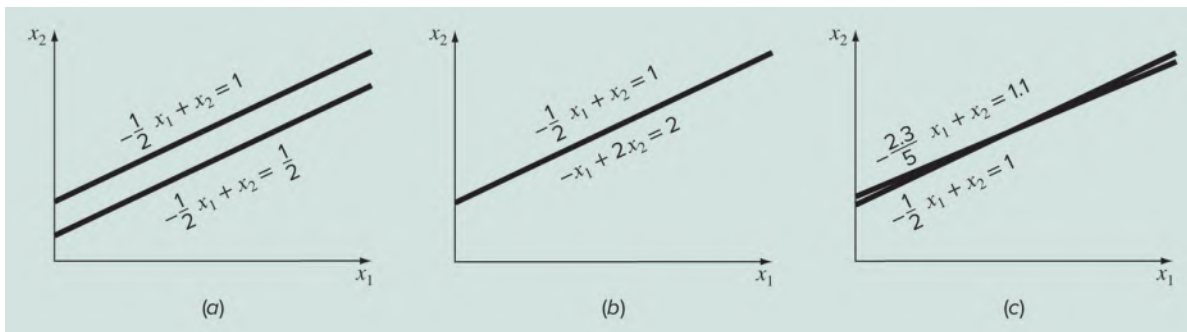
**FIGURE 9.1** Graphical solution of a set of two simultaneous linear algebraic equations. The intersection of the lines represents the solution.



For three simultaneous equations, each equation would be represented by a plane in a three-dimensional coordinate system. The point where the three planes intersect would represent the solution. Beyond three equations, graphical methods break down and, consequently, have little practical value for solving simultaneous equations. However, they are useful in visualizing properties of the solutions.

For example, [Fig. 9.2](#) depicts three cases that can pose problems when solving sets of linear equations. [Figure 9.2a](#) shows the case where the two equations represent parallel lines. For such situations, there is no solution because the lines never cross. [Figure 9.2b](#) depicts the case where the two lines are coincident. For such situations there are an infinite number of solutions. Both types of systems are said to be *singular*.

**FIGURE 9.2** Graphical depiction of singular and ill-conditioned systems: (a) no solution, (b) infinite solutions, and (c) ill-conditioned system where the slopes are so close that the point of intersection is difficult to detect visually.



In addition, systems that are very close to being singular ( Fig. 9.2c) can also cause problems. These systems are said to be *ill-conditioned*. Graphically, this corresponds to the fact that it is difficult to identify the exact point at which the lines intersect. Ill-conditioned systems will also pose problems when they are encountered during the numerical solution of linear equations. This is because they will be extremely sensitive to roundoff error.

## 9.1.2 Determinants and Cramer's Rule

Cramer's rule is another solution technique that is best suited to small numbers of equations. Before describing this method, we will briefly review the concept of the determinant, which is used to implement Cramer's rule. In addition, the determinant has relevance to the evaluation of the ill-conditioning of a matrix.

### Determinants

The determinant can be illustrated for a set of three equations:

$$[A]\{x\} = \{b\}$$

where  $[A]$  is the coefficient matrix

$$[A] = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

The *determinant* of this system is formed from the coefficients of  $[A]$  and is represented as

$$D = \begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix}$$

Although the determinant  $D$  and the coefficient matrix  $[A]$  are composed of the same elements, they are completely different mathematical concepts. That is why they are distinguished visually by using brackets to enclose the matrix and straight lines to enclose the determinant. In contrast to a matrix, the determinant is a single number. For example, the value of the determinant for two simultaneous equations

$$D = \begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix}$$

is calculated by

$$D = a_{11}a_{22} - a_{12}a_{21}$$

For the third-order case, the determinant can be computed as

$$D = a_{11} \begin{vmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{vmatrix} - a_{12} \begin{vmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{vmatrix} + a_{13} \begin{vmatrix} a_{21} & a_{22} \\ a_{31} & a_{32} \end{vmatrix} \quad (9.1)$$

where the  $2 \times 2$  determinants are called *minors*.

## EXAMPLE 9.1

### Determinants

#### Problem Statement

Compute values for the determinants of the systems represented in  Figs. 9.1 and  9.2.

Page 222

#### Solution

For  Fig. 9.1:

$$D = \begin{vmatrix} 3 & 2 \\ -1 & 2 \end{vmatrix} = 3(2) - 2(-1) = 8$$

For  Fig. 9.2a:

$$D = \begin{vmatrix} -\frac{1}{2} & 1 \\ -\frac{1}{2} & 1 \end{vmatrix} = -\frac{1}{2}(1) - 1\left(\frac{-1}{2}\right) = 0$$

For  Fig. 9.2b:

$$D = \begin{vmatrix} -\frac{1}{2} & 1 \\ -1 & 2 \end{vmatrix} = -\frac{1}{2}(2) - 1(-1) = 0$$

For  Fig. 9.2c:

$$D = \begin{vmatrix} -\frac{1}{2} & 1 \\ -\frac{2.3}{5} & 1 \end{vmatrix} = -\frac{1}{2}(1) - 1\left(\frac{-2.3}{5}\right) = -0.04$$

In the foregoing example, the singular systems had zero determinants. Additionally, the results suggest that the system that is almost singular ( **Fig. 9.2c**) has a determinant that is close to zero. These ideas will be pursued further in our subsequent discussion of ill-conditioning in  **Chap. 11**.

## Cramer's Rule

This rule states that each unknown in a system of linear algebraic equations may be expressed as a fraction of two determinants with denominator  $D$  and with the numerator obtained from  $D$  by replacing the column of coefficients of the unknown in question by the constants  $b_1, b_2, \dots, b_n$ . For example, for three equations,  $x_1$  would be computed as

$$x_1 = \frac{\begin{vmatrix} b_1 & a_{12} & a_{13} \\ b_2 & a_{22} & a_{23} \\ b_3 & a_{32} & a_{33} \end{vmatrix}}{D}$$

### EXAMPLE 9.2

#### Cramer's Rule

#### Problem Statement

Use Cramer's rule to solve

$$0.3x_1 + 0.52x_2 + x_3 = -0.01$$

$$0.5x_1 + x_2 + 1.9x_3 = 0.67$$

$$0.1x_1 + 0.3x_2 + 0.5x_3 = -0.44$$

#### Solution

The determinant  $D$  can be evaluated as [ **Eq. (9.1)**]:

$$D = 0.3 \begin{vmatrix} 1 & 1.9 \\ 0.3 & 0.5 \end{vmatrix} - 0.52 \begin{vmatrix} 0.5 & 1.9 \\ 0.1 & 0.5 \end{vmatrix} + 1 \begin{vmatrix} 0.5 & 1 \\ 0.1 & 0.3 \end{vmatrix} = -0.0022$$

The solution can be calculated as

$$x_1 = \frac{\begin{vmatrix} -0.01 & 0.52 & 1 \\ 0.67 & 1 & 1.9 \\ -0.44 & 0.3 & 0.5 \end{vmatrix}}{-0.0022} = \frac{0.03278}{-0.0022} = -14.9$$

$$x_2 = \frac{\begin{vmatrix} 0.3 & -0.01 & 1 \\ 0.5 & 0.67 & 1.9 \\ 0.1 & -0.44 & 0.5 \end{vmatrix}}{-0.0022} = \frac{0.0649}{-0.0022} = -29.5$$

$$x_3 = \frac{\begin{vmatrix} 0.3 & 0.52 & -0.01 \\ 0.5 & 1 & 0.67 \\ 0.1 & 0.3 & -0.44 \end{vmatrix}}{-0.0022} = \frac{-0.04356}{-0.0022} = 19.8$$

## The `linalg.det` Function

The determinant can be computed directly in Python with the `linalg.det` function from the NumPy module. For example, using the system from the previous example:

```
import numpy as np

A = np.matrix('0.3 0.52 1 ; 0.5 1 1.9 ; 0.1 0.3 0.5')
d = np.linalg.det(A)
print('determinant of A is {0:8.4f}'.format(d))

determinant of A is -0.0022
```

Cramer's rule can be applied to compute  $x_1$  as in

```
A[:,0]= [ [-0.01] , [ 0.67] , [-0.44]]
print(A)
x1 = np.linalg.det(A)/d
print('\nx1 = {0:6.2f}'.format(x1))

[[-0.01 0.52 1. ]
 [ 0.67 1. 1.9 ]
 [-0.44 0.3 0.5 ]]

x1 = -14.90
```

For more than three equations, Cramer's rule becomes impractical because, as the number of equations increases, the determinants are time consuming to evaluate by hand (or by computer). Consequently, more efficient alternatives are used. Some of these alternatives are based on the last non-computer solution technique covered in [Sec. 9.1.3](#)—the elimination of unknowns.

## 9.1.3 Elimination of Unknowns

The elimination of unknowns by combining equations is an algebraic approach that can be illustrated for a set of two equations:

$$a_{11}x_1 + a_{12}x_2 = b_1 \quad (9.2)$$

$$a_{21}x_1 + a_{22}x_2 = b_2 \quad (9.3)$$

The basic strategy is to multiply the equations by constants so that one of the unknowns will be eliminated when the two equations are combined. The result is a single equation that can be solved for the remaining unknown. This value can then be substituted into either of the original equations to compute the other variable.

For example, [Eq. \(9.2\)](#) might be multiplied by  $a_{21}$  and [Eq. \(9.3\)](#) by  $a_{11}$  to give

$$a_{21}a_{11}x_1 + a_{21}a_{12}x_2 = a_{21}b_1 \quad (9.4)$$

$$a_{11}a_{21}x_1 + a_{11}a_{22}x_2 = a_{11}b_2 \quad (9.5)$$

Subtracting [Eq. \(9.4\)](#) from [Eq. \(9.5\)](#) will, therefore, eliminate the  $x_1$  term from the equations to yield

$$a_{11}a_{22}x_2 - a_{21}a_{12}x_2 = a_{11}b_2 - a_{21}b_1$$

which can be solved for

$$x_2 = \frac{a_{11}b_2 - a_{21}b_1}{a_{11}a_{22} - a_{21}a_{12}} \quad (9.6)$$

[Equation \(9.6\)](#) can then be substituted into [Eq. \(9.2\)](#), which can be solved for

$$x_1 = \frac{a_{22}b_1 - a_{12}b_2}{a_{11}a_{22} - a_{21}a_{12}} \quad (9.7)$$

Notice that [Eqs. \(9.6\)](#) and [\(9.7\)](#) follow directly from Cramer's rule:



$$x_1 = \frac{\begin{vmatrix} b_1 & a_{12} \\ b_2 & a_{22} \end{vmatrix}}{\begin{vmatrix} a_{11} & a_{12} \\ b_{21} & a_{22} \end{vmatrix}} = \frac{a_{22}b_1 - a_{12}b_2}{a_{11}a_{22} - a_{21}a_{12}}$$

$$x_2 = \frac{\begin{vmatrix} a_{11} & b_1 \\ a_{21} & b_2 \end{vmatrix}}{\begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix}} = \frac{a_{11}b_2 - a_{21}b_1}{a_{11}a_{22} - a_{21}a_{12}}$$

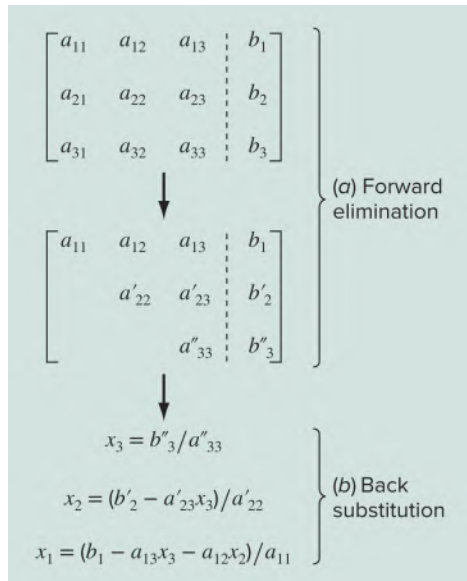
The elimination of unknowns can be extended to systems with more than two or three equations. Page 224  
 However, the numerous calculations that are required for larger systems make the method extremely tedious to implement by hand. However, as described in [Sec. 9.2](#), the technique can be formalized and readily programmed for the computer.

## 9.2 NAIVE GAUSS ELIMINATION

In [Sec. 9.1.3](#), the elimination of unknowns was used to solve a pair of simultaneous equations. The procedure consisted of two steps ([Fig. 9.3](#)):

1. The equations were manipulated to eliminate one of the unknowns from the equations. The result of this elimination step was that we had one equation with one unknown.
2. Consequently, this equation could be solved directly and the result back-substituted into one of the original equations to solve for the remaining unknown.

**FIGURE 9.3** The two phases of Gauss elimination: (a) forward elimination and (b) back substitution.



This basic approach can be extended to large sets of equations by developing a systematic scheme or algorithm to eliminate unknowns and to back-substitute. Gauss elimination is the most basic of these schemes.

This section includes the systematic techniques for forward elimination and back substitution that comprise Gauss elimination. Although these techniques are ideally suited for implementation on computers, some modifications will be required to obtain a reliable algorithm. In particular, the computer program must avoid division by zero. The following method is called “naive” Gauss elimination because it does not avoid this problem. [Section 9.3](#) will deal with the additional features required for an effective computer program.

The approach is designed to solve a general set of  $n$  equations:

(9.8a)

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n = b_1 \quad (9.8b)$$

$$a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + \cdots + a_{2n}x_n = b_2$$

$$\vdots \quad \quad \quad \vdots \quad \quad \quad (9.8c)$$

$$a_{n1}x_1 + a_{n2}x_2 + a_{n3}x_3 + \cdots + a_{nn}x_n = b_n$$

As was the case with the solution of two equations, the technique for  $n$  equations consists of two phases: elimination of unknowns and solution through back substitution.

## Forward Elimination of Unknowns

The first phase is designed to reduce the set of equations to an upper triangular system ( Fig. 9.3a). The initial step will be to eliminate the first unknown  $x_1$  from the second through the  $n$ th equations. To do this, multiply  Eq. (9.8a) by  $a_{21}/a_{11}$  to give

$$a_{21}x_1 + \frac{a_{21}}{a_{11}}a_{12}x_2 + \frac{a_{21}}{a_{11}}a_{13}x_3 + \cdots + \frac{a_{21}}{a_{11}}a_{1n}x_n = \frac{a_{21}}{a_{11}}b_1 \quad (9.9)$$

This equation can be subtracted from  Eq. (9.8b) to give

$$\left(a_{22} - \frac{a_{21}}{a_{11}}a_{12}\right)x_2 + \cdots + \left(a_{2n} - \frac{a_{21}}{a_{11}}a_{1n}\right)x_n = b_2 - \frac{a_{21}}{a_{11}}b_1$$

or

$$a'_{22}x_2 + \cdots + a'_{2n}x_n = b'_2$$

where the prime indicates that the elements have been changed from their original values.

The procedure is then repeated for the remaining equations. For instance,  Eq. (9.8a) can be multiplied by  $a_{31}/a_{11}$  and the result subtracted from the third equation. Repeating the procedure for the remaining equations results in the following modified system:

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n = b_1 \quad (9.10a)$$

$$a'_{22}x_2 + a'_{23}x_3 + \cdots + a'_{2n}x_n = b'_2 \quad (9.10b)$$

$$a'_{32}x_2 + a'_{33}x_3 + \cdots + a'_{3n}x_n = b'_3 \quad (9.10c)$$

$$\vdots \quad \quad \quad \vdots$$

$$a'_{n2}x_2 + a'_{n3}x_3 + \cdots + a'_{nn}x_n = b'_n \quad (9.10d)$$

For the foregoing steps, [Eq. \(9.8a\)](#) is called the *pivot equation* and  $a_{11}$  is called the *pivot element*. Note that the process of multiplying the first row by  $a_{21}/a_{11}$  is equivalent to dividing it by  $a_{11}$  and multiplying it by  $a_{21}$ . Sometimes the division operation is referred to as *normalization*. We make this distinction because a zero pivot element can interfere with normalization by causing a division by zero. We will return to this important issue after we complete our description of naive Gauss elimination.

The next step is to eliminate  $x_2$  from [Eq. \(9.10c\)](#) through [Eq. \(9.10d\)](#). To do this, multiply [Eq. \(9.10b\)](#) by  $a'_{32}/a'_{22}$  and subtract the result from [Eq. \(9.10c\)](#). Perform a similar elimination for the remaining equations to yield

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n &= b_1 \\ a'_{22}x_2 + a'_{23}x_3 + \cdots + a'_{2n}x_n &= b'_2 \\ a''_{33}x_3 + \cdots + a''_{3n}x_n &= b''_3 \\ \vdots & \\ a''_{n3}x_3 + \cdots + a''_{nn}x_n &= b''_n \end{aligned}$$

where the double prime indicates that the elements have been modified twice.

The procedure can be continued using the remaining pivot equations. The final manipulation in the sequence is to use the  $(n - 1)$ th equation to eliminate the  $x_{n-1}$  term from the  $n$ th equation. At this point, the system will have been transformed to an upper triangular system:

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n = b_1 \quad (9.11a)$$

$$a'_{22}x_2 + a'_{23}x_3 + \cdots + a'_{2n}x_n = b'_2 \quad (9.11b)$$

$$a''_{33}x_3 + \cdots + a''_{3n}x_n = b''_3 \quad (9.11c)$$

$$\begin{aligned} \ddots & \\ a^{(n-1)}_{nn}x_n &= b^{(n-1)}_n \end{aligned} \quad (9.11d)$$

## Back Substitution

[Equation \(9.11d\)](#) can now be solved for  $x_n$ :

$$x_n = \frac{b^{(n-1)}_n}{a^{(n-1)}_{nn}} \quad (9.12)$$

This result can be back-substituted into the  $(n - 1)$ th equation to solve for  $x_{n-1}$ . The procedure, which is repeated to evaluate the remaining  $x$ 's, can be represented by the following formula:

$$x_i = \frac{b_i^{(i-1)} - \sum_{j=i+1}^n a_{ij}^{(i-1)} x_j}{a_{ii}^{(i-1)}} \quad \text{for } i = n-1, n-2, \dots, 1 \quad (9.13)$$

## EXAMPLE 9.3

### Naive Gauss Elimination

#### Problem Statement

Use Gauss elimination to solve

$$3x_1 - 0.1x_2 - 0.2x_3 = 7.85 \quad (\text{E9.3.1})$$

$$0.1x_1 + 7x_2 - 0.3x_3 = -19.3 \quad (\text{E9.3.2})$$

$$0.3x_1 - 0.2x_2 + 10x_3 = 71.4 \quad (\text{E9.3.3})$$

#### Solution

The first part of the procedure is forward elimination. Multiply [Eq. \(E9.3.1\)](#) by 0.1/3 and subtract the result from [Eq. \(E9.3.2\)](#) to give

$$7.00333x_2 - 0.293333x_3 = -19.5617$$

Then, multiply [Eq. \(E9.3.1\)](#) by 0.3/3 and subtract it from [Eq. \(E9.3.3\)](#). After these operations, the set of equations is

$$3x_1 - 0.1x_2 - 0.2x_3 = 7.85 \quad (\text{E9.3.4})$$

$$7.00333x_2 - 0.293333x_3 = -19.5617 \quad (\text{E9.3.5})$$

$$-0.190000x_2 + 10.0200x_3 = 70.6150 \quad (\text{E9.3.6})$$

To complete the forward elimination,  $x_2$  must be removed from [Eq. \(E9.3.6\)](#). To accomplish this, multiply [Eq. \(E9.3.5\)](#) by  $-0.190000/7.00333$  and subtract the result from [Eq. \(E9.3.6\)](#). This eliminates  $x_2$  from the third equation and reduces the system to an upper triangular form, as in

$$3x_1 - 0.1x_2 - 0.2x_3 = 7.85 \quad (\text{E9.3.7})$$

$$7.00333x_2 - 0.293333x_3 = -19.5617 \quad (\text{E9.3.8})$$

$$10.0120x_3 = 70.0843 \quad (\text{E9.3.9})$$

We can now solve these equations by back substitution. First, [Eq. \(E9.3.9\)](#) can be solved for

$$x_3 = \frac{70.0843}{10.0120} = 7.00003$$

This result can be back-substituted into [Eq. \(E9.3.8\)](#), which can then be solved for

$$x_2 = \frac{-19.5617 + 0.293333(7.00003)}{7.00333} = -2.50000$$

Finally,  $x_3 = 7.00003$  and  $x_2 = -2.50000$  can be substituted back into [Eq. \(E9.3.7\)](#), which can be solved for

$$x_1 = \frac{7.85 + 0.1(-2.50000) + 0.2(7.00003)}{3} = 3.00000$$

---

Page 227

---

Although there is a slight round off error, the results are very close to the exact solution of  $x_1 = 3$ ,  $x_2 = -2.5$ , and  $x_3 = 7$ . This can be verified by substituting the results into the original equation set:

$$\begin{aligned} 3(3) - 0.1(-2.5) - 0.2(7.00003) &= 7.84999 \cong 7.85 \\ 0.1(3) + 7(-2.5) - 0.3(7.00003) &= -19.30000 = -19.3 \\ 0.3(3) - 0.2(-2.5) + 10(7.00003) &= 71.4003 \cong 71.4 \end{aligned}$$

## Section 9.2.1 Python function: gaussnaive

A Python function that implements naive Gauss elimination is listed in [Fig. 9.4](#). Notice that the coefficient matrix  $A$  and the constant vector  $b$  are combined in the augmented matrix  $Aug$ . Thus, the operations are performed on  $Aug$  rather than separately on  $A$  and  $b$ .

**FIGURE 9.4** A Python function to implement naïve Gauss elimination.

```
def gaussnaive(A,b):
    """
    gaussnaive: naive Gauss elimination
    input:
    A = coefficient matrix
    b = constant vector
    output:
    x = solution vector
    """
    (n,m) = A.shape
    #n = nm[0]
    #m = nm[1]
    if n != m:
        return 'Coefficient matrix A must be square'
    nb = n+1
    # build augmented matrix
    Aug = np.hstack((A,b))
    # forward elimination
    for k in range(n-1):
        for i in range(k+1,n):
            factor = Aug[i,k]/Aug[k,k]
            Aug[i,k:nb]=Aug[i,k:nb]-factor*Aug[k,k:nb]
    # back substitution
    x = np.zeros([n,1]) # create empty x array
    x = np.matrix(x) # convert to matrix type
    x[n-1]=Aug[n-1,nb-1]/Aug[n-1,n-1]
    for i in range(n-2,-1,-1):
        x[i]=(Aug[i,nb-1]-Aug[i,i+1:n]*x[i+1:n,0])/Aug[i,i]
    return x
```

Two nested loops provide a concise representation of the forward elimination step. An outer loop moves down the matrix from one pivot row to the next. The inner loop moves below the pivot row to each of the subsequent rows where elimination is to take place. Finally, the actual elimination is represented by a single line that takes advantage of Python's ability to perform matrix operations.

Two nested loops provide a concise representation of the forward elimination step. An outer loop moves down the matrix from one pivot row to the next. The inner loop moves below the pivot row to each of the subsequent rows where elimination is to take place. Finally, the actual elimination is represented by a single line that takes advantage of MATLAB's ability to perform matrix operations.

The back-substitution step follows directly from [Eqs. \(9.12\)](#) and [\(9.13\)](#). Again, Python's ability to perform matrix operations allows [Eq. \(9.13\)](#) to be programmed as a single statement. This is an efficient Python function with only 15 statements.

The use of subscripts in the Python code may well be confusing. Python, as other computer languages such as C/C++, uses a base subscript of zero, whereas, in typical mathematical notation, a base subscript of one is used. There are other computer languages/packages that adhere to mathematical convention,

such as MATLAB, Fortran and Excel's VBA with Option Base 1. The zero-based subscripting was developed more with a computer science approach of subscripts being a displacement or offset from the base; so a subscript of [3,2] is taken to mean three rows down and 2 columns to the right from the base location. This difference between mathematical and Python representation is illustrated in  Fig. 9.5. One must be careful translating mathematical description of linear algebra to the Python subscripting convention. Figures like this are generally helpful, and this one may help you interpret the code in  Fig. 9.4.

**FIGURE 9.5** Comparison of typical mathematical subscripting with Python zero-based subscripting.

$$\begin{array}{c}
 \begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{n-1,1} & a_{n-1,2} & \cdots & a_{n-1,n-1} & a_{n-1,n} \\ a_{n1} & a_{n2} & \cdots & a_{n,n-1} & a_{nn} \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_{n-1} \\ b_n \end{bmatrix} \quad \text{typical} \\
 \text{mathematical} \\
 \text{representation}
 \end{array}$$
  

$$\begin{array}{c}
 \begin{bmatrix} a_{[0,0]} & a_{[0,1]} & a_{[0,2]} & \cdots & a_{[0,n-1]} \\ a_{[1,0]} & a_{[1,1]} & a_{[1,2]} & \cdots & a_{[1,n-1]} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{[n-2,0]} & a_{[n-2,1]} & \cdots & a_{[n-2,n-2]} & a_{[n-2,n-1]} \\ a_{[n-1,0]} & a_{[n-1,1]} & \cdots & a_{[n-1,n-2]} & a_{[n-1,n-1]} \end{bmatrix} \cdot \begin{bmatrix} x_{[0]} \\ x_{[1]} \\ \vdots \\ x_{[n-2]} \\ x_{[n-1]} \end{bmatrix} = \begin{bmatrix} b_{[0]} \\ b_{[1]} \\ \vdots \\ b_{[n-2]} \\ b_{[n-1]} \end{bmatrix} \quad \text{equivalent} \\
 \text{representation} \\
 \text{with Python} \\
 \text{subscripts}
 \end{array}$$



Also, we need to recall the syntax of the range type, typically used in for loops.

range(n)                      a sequence of integers from 0 to  $n - 1$  ( $n$  integers in all)  
range(1, n)                  from 1 to  $n - 1$  ( $n - 1$  integers)  
range(n, 0, -1)              from  $n$  to 1 in steps of  $-1$  ( $n$  integers)

And, when using the colon (:) in a subscript, the last value is not included.

Aug[0:2, 1:3]    does not include the 2 and 3 subscripts

In other words, this is the 1st and 2nd rows and 2nd and 3rd columns.

## 9.2.2 Operation Counting

The execution time of Gauss elimination depends on the amount of *floating-point operations* (or *flops*) involved in the algorithm. On modern computers using math coprocessors, the time consumed to perform addition/subtraction and multiplication/division is about the same. Therefore, totaling up these



operations provides insight into which parts of the algorithm are most time consuming and how computation time increases as the system gets larger.

Before analyzing naive Gauss elimination, we will first define some quantities that facilitate operation counting:

$$\sum_{i=1}^m cf(i) = c \sum_{i=1}^m f(i) \quad \sum_{i=1}^m f(i) + g(i) = \sum_{i=1}^m f(i) + \sum_{i=1}^m g(i) \quad (9.14a,b)$$

$$\sum_{i=1}^m 1 = 1 + 1 + 1 + \cdots + 1 = m \quad \sum_{i=k}^m 1 = m - k + 1 \quad (9.14c,d)$$

$$\sum_{i=1}^m i = 1 + 2 + 3 + \cdots + m = \frac{m(m+1)}{2} = \frac{m^2}{2} + O(m) \quad (9.14e)$$

$$\sum_{i=1}^m i^2 = 1^2 + 2^2 + 3^2 + \cdots + m^2 = \frac{m(m+1)(2m+1)}{6} = \frac{m^3}{3} + O(m^2) \quad (9.14f)$$

where  $O(m^n)$  means “terms of order  $m^n$  and lower.”

Now, let us examine the naive Gauss elimination algorithm (Fig. 9.4) in detail. We will first count the flops in the elimination stage. On the first pass through the outer loop,  $k = 1$ . Therefore, the limits on the inner loop are from  $i = 2$  to  $n$ . According to Eq. (9.14d), this means that the number of iterations of the inner loop will be

$$\sum_{i=2}^n 1 = n - 2 + 1 = n - 1 \quad (9.15)$$

For every one of these iterations, there is one division to calculate the factor. The next line then performs a multiplication and a subtraction for each column element from 2 to  $nb$ . Because  $nb = n + 1$ , going from 2 to  $nb$  results in  $n$  multiplications and  $n$  subtractions. Together with the single division, this amounts to  $n + 1$  multiplications/divisions and  $n$  addition/subtractions for every iteration of the inner loop. The total for the first pass through the outer loop is therefore  $(n - 1)(n + 1)$  multiplication/divisions and  $(n - 1)(n)$  addition/subtractions.

Similar reasoning can be used to estimate the flops for the subsequent iterations of the outer loop. These can be summarized as

| Outer Loop $k$ | Inner Loop $i$ | Addition/Subtraction Flops | Multiplication/Division Flops |
|----------------|----------------|----------------------------|-------------------------------|
| 1              | 2, $n$         | $(n - 1)(n)$               | $(n - 1)(n + 1)$              |

| Outer Loop $k$ | Inner Loop $i$ | Addition/Subtraction Flops | Multiplication/Division Flops |
|----------------|----------------|----------------------------|-------------------------------|
| 2              | 3, $n$         | $(n-2)(n-1)$               | $(n-2)(n)$                    |
| $\vdots$       | $\vdots$       |                            |                               |
| $k$            | $k+1, n$       | $(n-k)(n+1-k)$             | $(n-k)(n+2-k)$                |
| $\vdots$       | $\vdots$       |                            |                               |
| $n-1$          | $n, n$         | $(1)(2)$                   | $(1)(3)$                      |

Therefore, the total addition/subtraction flops for elimination can be computed as

$$\sum_{k=1}^{n-1} (n-k)(n+1-k) = \sum_{k=1}^{n-1} [n(n+1) - k(2n+1) + k^2] \quad (9.16)$$

or

$$n(n+1) \sum_{k=1}^{n-1} 1 - (2n+1) \sum_{k=1}^{n-1} k + \sum_{k=1}^{n-1} k^2 \quad (9.17)$$

Applying some of the relationships from [Eq. \(9.14\)](#) yields

$$[n^3 + O(n)] - [n^3 + O(n^2)] + \left[ \frac{1}{3} n^3 + O(n^2) \right] = \frac{n^3}{3} + O(n) \quad (9.18)$$

A similar analysis for the multiplication/division flops yields

$$[n^3 + O(n^2)] - [n^3 + O(n)] + \left[ \frac{1}{3} n^3 + O(n^2) \right] = \frac{n^3}{3} + O(n^2) \quad (9.19)$$

Summing these results gives

$$\frac{2n^3}{3} + O(n^2) \quad (9.20)$$

Thus, the total number of flops is equal to  $2n^3/3$  plus an additional component proportional to terms of order  $n^2$  and lower. The result is written in this way because as  $n$  gets large, the  $O(n^2)$  and lower terms become negligible. We are therefore justified in concluding that for large  $n$ , the effort involved in forward elimination converges on  $2n^3/3$ .

Because only a single loop is used, back substitution is much simpler to evaluate. The number of addition/subtraction flops is equal to  $n(n-1)/2$ . Because of the extra division prior to the loop, the number of multiplication/division flops is  $n(n+1)/2$ . These can be added to arrive at a total of

$$n^2 + O(n) \quad (9.21)$$

Thus, the total effort in naive Gauss elimination can be represented as

$$\underbrace{\frac{2n^3}{3} + O(n^2)}_{\text{Forward elimination}} + \underbrace{n^2 + O(n)}_{\text{Back substitution}} \xrightarrow{\text{as } n \text{ increases}} \frac{2n^3}{3} + O(n^2) \quad (9.22)$$

Two useful general conclusions can be drawn from this analysis:

1. As the system gets larger, the computation time increases greatly. As in [Table 9.1](#), the amount of flops increases nearly three orders of magnitude for every order of magnitude increase in the number of equations.
2. Most of the effort is incurred in the elimination step. Thus, efforts to make the method more efficient should probably focus on this step.

**TABLE 9.1** Number of flops for naive Gauss elimination.

| $n$  | Elimination        | Back Substitution | Total Flops        | $2n^3/3$           | Percent Due to Elimination |
|------|--------------------|-------------------|--------------------|--------------------|----------------------------|
| 10   | 705                | 100               | 805                | 667                | 87.58                      |
| 100  | 671550             | 10000             | 681550             | 666667             | 98.53                      |
| 1000 | $6.67 \times 10^8$ | $1 \times 10^6$   | $6.68 \times 10^8$ | $6.67 \times 10^8$ | 99.85                      |

## 9.3 PIVOTING

The primary reason that the foregoing technique is called “naive” is that during both the elimination and the back-substitution phases, it is possible that a division by zero can occur. For example, if we use naive Gauss elimination to solve

$$\begin{aligned}2x_2 + 3x_3 &= 8 \\4x_1 + 6x_2 + 7x_3 &= -3 \\2x_1 - 3x_2 + 6x_3 &= 5\end{aligned}$$

the normalization of the first row would involve division by  $a_{11} = 0$ . Problems may also arise when the pivot element is close, rather than exactly equal, to zero because if the magnitude of the pivot element is small compared to the other elements, then round off errors can be introduced.

Therefore, before each row is normalized, it is advantageous to determine the coefficient with the largest absolute value in the column below the pivot element. The rows can then be switched so that the largest element is the pivot element. This is called *partial pivoting*.

If columns as well as rows are searched for the largest element and then switched, the procedure is called *complete pivoting*. Complete pivoting is rarely used because most of the improvement comes from partial pivoting. In addition, switching columns changes the order of the  $x$ 's and, consequently, adds significant and usually unjustified complexity to the computer program.

The following example illustrates the advantages of partial pivoting. Aside from avoiding division by zero, pivoting also minimizes round off error. As such, it also serves as a partial remedy for ill-conditioning.

### EXAMPLE 9.4

#### Partial Pivoting

#### Problem Statement

Use Gauss elimination to solve

$$\begin{aligned}0.0003x_1 + 3.0000x_2 &= 2.0001 \\1.0000x_1 + 1.0000x_2 &= 1.0000\end{aligned}$$

Note that in this form the first pivot element,  $a_{11} = 0.0003$ , is very close to zero. Then repeat the computation, but partial pivot by reversing the order of the equations. The exact solution is  $x_1 = 1/3$  and  $x_2 = 2/3$ .

## Solution

Multiplying the first equation by  $1/(0.0003)$  yields

$$x_1 + 10,000x_2 = 6667$$

which can be used to eliminate  $x_1$  from the second equation:

$$-9999x_2 = -6666$$

which can be solved for  $x_2 = 2/3$ . This result can be substituted back into the first equation to evaluate  $x_1$ :

$$x_1 = \frac{2.0001 - 3(2/3)}{0.0003} \quad (\text{E9.4.1})$$

Due to subtractive cancellation, the result is very sensitive to the number of significant figures carried in the computation:

| Significant<br>Figures | $x_2$     | $x_1$     | Absolute Value of Percent<br>Relative Error for $x_1$ |
|------------------------|-----------|-----------|-------------------------------------------------------|
| 3                      | 0.667     | -3.33     | 1099                                                  |
| 4                      | 0.6667    | 0.0000    | 100                                                   |
| 5                      | 0.66667   | 0.30000   | 10                                                    |
| 6                      | 0.666667  | 0.330000  | 1                                                     |
| 7                      | 0.6666667 | 0.3330000 | 0.1                                                   |

Note how the solution for  $x_1$  is highly dependent on the number of significant figures. This is because in  Eq. (E9.4.1), we are subtracting two almost-equal numbers.

On the other hand, if the equations are solved in reverse order, the row with the larger pivot element is normalized. The equations are

$$\begin{aligned} 1.0000x_1 + 1.0000x_2 &= 1.0000 \\ 0.0003x_1 + 3.0000x_2 &= 2.0001 \end{aligned}$$

Elimination and substitution again yields  $x_2 = 2/3$ . For different numbers of significant figures,  $x_1$  can be computed from the first equation, as in

$$x_1 = \frac{1 - (2/3)}{1}$$

This case is much less sensitive to the number of significant figures in the computation:

| Significant<br>Figures | $x_2$     | $x_1$     | Absolute Value of Percent Relative Error<br>for $x_1$ |
|------------------------|-----------|-----------|-------------------------------------------------------|
| 3                      | 0.667     | 0.333     | 0.1                                                   |
| 4                      | 0.6667    | 0.3333    | 0.01                                                  |
| 5                      | 0.66667   | 0.33333   | 0.001                                                 |
| 6                      | 0.666667  | 0.333333  | 0.0001                                                |
| 7                      | 0.6666667 | 0.3333333 | 0.0000                                                |

Thus, a pivot strategy is much more satisfactory.

## 9.3.1 Python Function: GaussPivot

A Python function that implements Gauss elimination with partial pivoting is listed in [Fig. 9.6](#). It is identical to the `gaussnaive` function presented previously in [Sec. 9.2.1](#) except for the section of code set off between blank lines that implements partial pivoting.

**FIGURE 9.6** A Python function to implement Gauss elimination with partial pivoting.

```
def gausspivot(A,b):
    """
    gausspivot: Gauss elimination with partial pivoting
    input:
    A = coefficient matrix
    b = constant vector
    output:
    x = solution vector
    """
    (n,m) = A.shape
    if n != m:
        return 'Coefficient matrix A must be square'
    nb = n+1
    # build augmented matrix
    Aug = np.hstack((A,b))
    # forward elimination
    for k in range(n-1):
        # partial pivoting
        imax = maxrow(Aug[k:n,k])
        ipr = imax + k
        if ipr != k: # no row swap if pivot is max
            for j in range(k,nb): # swap rows k and ipr
                temp = Aug[k,j]
                Aug[k,j] = Aug[ipr,j]
                Aug[ipr,j] = temp
            for i in range(k+1,n):
```

```

        factor = Aug[i,k]/Aug[k,k]
        Aug[i,k:nb]=Aug[i,k:nb]-factor*Aug[k,k:nb]
    # back substitution
    x = np.zeros([n,1]) # create empty x array
    x = np.matrix(x) # convert to matrix type
    x[n-1]=Aug[n-1,nb-1]/Aug[n-1,n-1]
    for i in range(n-2,-1,-1):
        x[i]=(Aug[i,nb-1]-Aug[i,i+1:n]*x[i+1:n,0])/Aug[i,i]
    return x
def maxrow(avec):
    # function to determine the row index of the
    # maximum value in a vector
    maxrowind = 0
    n = len(avec)
    amax = abs(avec[0])
    for i in range(1,n):
        if abs(avec[i]) > amax:
            amax = abs(avec[i])
            maxrowind = i
    return maxrowind

```

The `gausspivot` function is supported by the `maxrow` function that determines the row index of the largest coefficient below the pivot. Note: The code contained in the latter function could have been imbedded in the `gausspivot` function; however, it is good practice to break down code into self-contained functions. It leads to more reliable, easier-to-debug, and easier-to-comprehend code.

---

Page 232

## 9.3.2 Determinant Evaluation with Gauss Elimination

At the end of [Sec. 9.1.2](#), we suggested that determinant evaluation by expansion of minors was impractical for large sets of equations. However, because the determinant has value in assessing system condition, it would be useful to have a practical method for computing this quantity.

Fortunately, Gauss elimination provides a simple way to do this. The method is based on the fact that the determinant of a triangular matrix can be simply computed as the product of its diagonal elements:

$$D = a_{11}a_{22}a_{33} \cdots a_{nn}$$

The validity of this formulation can be illustrated for a  $3 \times 3$  system:

---

Page 233

$$D = \begin{vmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22} & a_{23} \\ 0 & 0 & a_{33} \end{vmatrix}$$

where the determinant can be evaluated as [recall [Eq. \(9.1\)](#)]:

$$D = a_{11} \begin{vmatrix} a_{22} & a_{23} \\ 0 & a_{33} \end{vmatrix} - a_{12} \begin{vmatrix} 0 & a_{23} \\ 0 & a_{33} \end{vmatrix} + a_{13} \begin{vmatrix} 0 & a_{22} \\ 0 & 0 \end{vmatrix}$$

or, by evaluating the minors:

$$D = a_{11}a_{22}a_{33} - a_{12}(0) + a_{13}(0) = a_{11}a_{22}a_{33}$$

Recall that the forward-elimination step of Gauss elimination results in an upper triangular system. Because the value of the determinant is not changed by the forward-elimination process, the determinant can be simply evaluated at the end of this step via

$$D = a_{11}a'_{22}a''_{33} \cdots a^{(n-1)}_{nn}$$

where the superscripts signify the number of times that the elements have been modified by the elimination process. Thus, we can capitalize on the effort that has already been expended in reducing the system to triangular form and, in the bargain, come up with a simple estimate of the determinant.

There is a slight modification to the above approach when the program employs partial pivoting. For such cases, the determinant changes sign every time a row is switched. One way to represent this is by modifying the determinant calculation as in

$$D = a_{11}a'_{22}a''_{33} \cdots a^{(n-1)}_{nn} (-1)^p$$

where  $p$  represents the number of times that rows are pivoted. This modification can be incorporated simply into a program by merely keeping track of the number of pivots that take place during the course of the computation.



## 9.4 TRIDIAGONAL SYSTEMS

Certain matrices have a particular structure that can be exploited to develop efficient solution schemes. For example, a banded matrix is a square matrix that has all elements equal to zero, with the exception of a band centered on the main diagonal.

A *tridiagonal* system has a bandwidth of 3 and can be expressed generally as

$$\begin{bmatrix} f_1 & g_1 & & & \\ e_2 & f_2 & g_2 & & \\ & e_3 & f_3 & g_3 & \\ & & \cdot & \cdot & \cdot \\ & & & \cdot & \cdot & \cdot \\ & & & & e_{n-1} & f_{n-1} & g_{n-1} \\ & & & & & e_n & f_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \cdot \\ \cdot \\ x_{n-1} \\ x_n \end{bmatrix} = \begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ \cdot \\ \cdot \\ r_{n-1} \\ r_n \end{bmatrix} \quad (9.23)$$

Notice that we have changed our notation for the coefficients from  $a$ 's and  $b$ 's to  $e$ 's,  $f$ 's,  $g$ 's, and  $r$ 's. This was done to avoid storing large numbers of useless zeros in the square matrix of  $a$ 's. This space-saving modification is advantageous because the resulting algorithm requires less computer memory.

An algorithm to solve such systems can be directly patterned after Gauss elimination—that is, using forward elimination and back substitution. However, because most of the matrix elements are already zero, much less effort is expended than for a full matrix. This efficiency is illustrated in the following example.

### EXAMPLE 9.5

#### Solution of a Tridiagonal System

##### Problem Statement

Solve the following tridiagonal system:

$$\begin{bmatrix} 2.04 & -1 & & \\ -1 & 2.04 & -1 & \\ & -1 & 2.04 & -1 \\ & & -1 & 2.04 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 40.8 \\ 0.8 \\ 0.8 \\ 200.8 \end{bmatrix}$$

## Solution

As with Gauss elimination, the first step involves transforming the matrix to upper triangular form. This is done by multiplying the first equation by the factor  $e_2/f_1$  and subtracting the result from the second equation. This creates a zero in place of  $e_2$  and transforms the other coefficients to new values,

$$f_2 = f_2 - \frac{e_2}{f_1} g_1 = 2.04 - \frac{-1}{2.04}(-1) = 1.550$$
$$r_2 = r_2 - \frac{e_2}{f_1} r_1 = 0.8 - \frac{-1}{2.04}(40.8) = 20.8$$

Notice that  $g_2$  is unmodified because the element above it in the first row is zero.

After performing a similar calculation for the third and fourth rows, the system is transformed to the upper triangular form

$$\left[ \begin{array}{ccc|ccc} 2.04 & -1 & & & & \\ & 1.550 & -1 & & & \\ & & 1.395 & -1 & & \\ & & & 1.323 & & \end{array} \right] \begin{matrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{matrix} = \begin{matrix} 40.8 \\ 20.8 \\ 14.221 \\ 210.996 \end{matrix}$$

Now, back substitution can be applied to generate the final solution:

$$x_4 = \frac{r_4}{f_4} = \frac{210.996}{1.323} = 159.480$$
$$x_3 = \frac{r_3 - g_3 x_4}{f_3} = \frac{14.221 - (-1)159.480}{1.395} = 124.538$$
$$x_2 = \frac{r_2 - g_2 x_3}{f_2} = \frac{20.800 - (-1)124.538}{1.550} = 93.778$$
$$x_1 = \frac{r_1 - g_1 x_2}{f_1} = \frac{40.800 - (-1)93.778}{2.040} = 65.970$$

## 9.4.1 Python Function: `tridiag`

A Python function that solves a tridiagonal system of equations is listed in [Fig. 9.7](#). Note that the algorithm does not include partial pivoting. Although pivoting is sometimes required, most tridiagonal systems routinely solved in engineering and science do not require pivoting. The function is concise, containing only eleven lines of code.

**FIGURE 9.7** A Python function to solve a tridiagonal system of linear equations.

```
def tridiag(e,f,g,r):
    """
    tridiag: solves a set of n linear algebraic equations
             with a tridiagonal-banded coefficient matrix
    input:
    e = subdiagonal vector of length n, first element = 0
    f = diagonal vector of length n
    g = superdiagonal vector of length n, last element = 0
    r = constant vector of length n
    output:
    x = solution vector of length n
    """
    n = len(f)
    # forward elimination
    x = np.zeros([n])
    for k in range(1,n):
        factor = e[k]/f[k-1]
        f[k] = f[k] - factor*g[k-1]
        r[k] = r[k] - factor*r[k-1]
    # back substitution
    x[n-1] = r[n-1]/f[n-1]
    for k in range(n-2,-1,-1):
        x[k] = ( r[k] - g[k]*x[k+1] )/f[k]
    return x
```

Recall that the computational effort for Gauss elimination is proportional to  $n^3$ . Because of its sparseness, the effort involved in solving tridiagonal systems is proportional only to  $n$ . Imagine that comparison for a system of 100 equations, 10,000 to 1. Consequently, the algorithm in [Fig. 9.7](#) executes much faster than Gauss elimination. There is an extension of the algorithm presented here for systems with wider diagonal structures, such as pentadiagonal.

## MODEL OF A HEATED ROD

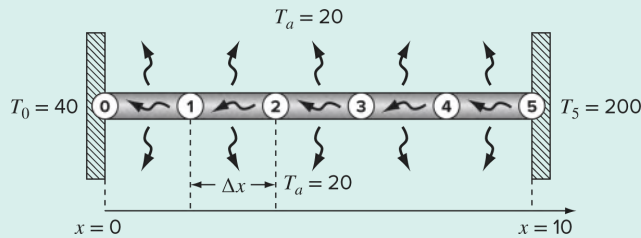
### Background

Linear algebraic equations can arise when modeling distributed systems. For example,  **Fig. 9.8** shows a long, thin rod positioned between two walls that are held at constant temperatures. Heat flows through the rod as well as between the rod and the surrounding air. For the steady-state case, a differential equation based on heat conservation can be written for such a system as

$$\frac{d^2T}{dx^2} + h'(T_a - T) = 0 \quad (9.24)$$

where  $T$  = temperature ( $^{\circ}\text{C}$ ),  $x$  = distance along the rod (m),  $h'$  = a heat transfer coefficient between the rod and the surrounding air ( $\text{m}^{-2}$ ), and  $T_a$  = the air temperature ( $^{\circ}\text{C}$ ).

**FIGURE 9.8** A noninsulated uniform rod positioned between two walls of constant but different temperature. The finite-difference representation employs four interior nodes.



Given values for the parameters, forcing functions, and boundary conditions, calculus can be used to develop an analytical solution. For example, if  $h' = 0.01$ ,  $T_a = 20$ ,  $T(0) = 40$ , and  $T(10) = 200$ , the solution is

$$T = 73.4523e^{0.1x} - 53.4523e^{-0.1x} + 20 \quad (9.25)$$

Although it provided a solution here, calculus does not work for all such problems. In such instances, numerical methods provide a valuable alternative. In this case study, we will use finite differences to transform this differential equation into a tridiagonal system of linear algebraic equations which can be readily solved using the numerical methods described in this chapter.

### Solution

 **Equation (9.24)** can be transformed into a set of linear algebraic equations by conceptualizing the rod as consisting of a series of nodes. For example, the rod in  **Fig. 9.8** is divided into six equispaced nodes. Since the rod has a length of 10, the spacing between nodes is  $\Delta x = 2$ .

Calculus was necessary to solve [Eq. \(9.24\)](#) because it includes a second derivative. As we learned in [Sec. 4.3.4](#), finite-difference approximations provide a means to transform derivatives into algebraic form. For example, the second derivative at each node can be approximated as

$$\frac{d^2T}{dx^2} = \frac{T_{i+1} - 2T_i + T_{i-1}}{\Delta x^2}$$

where  $T_i$  designates the temperature at node  $i$ . This approximation can be substituted into [Eq. \(9.24\)](#) to give

$$\frac{T_{i+1} - 2T_i + T_{i-1}}{\Delta x^2} + h'(T_a - T_i) = 0$$

Collecting terms and substituting the parameters give

Page 236

$$-T_{i-1} + 2.04T_i - T_{i+1} = 0.8 \quad (9.26)$$

Thus, [Eq. \(9.24\)](#) has been transformed from a differential equation into an algebraic equation.

[Equation \(9.26\)](#) can now be applied to each of the interior nodes:

$$\begin{aligned} -T_0 + 2.04T_1 - T_2 &= 0.8 \\ -T_1 + 2.04T_2 - T_3 &= 0.8 \\ -T_2 + 2.04T_3 - T_4 &= 0.8 \\ -T_3 + 2.04T_4 - T_5 &= 0.8 \end{aligned} \quad (9.27)$$

The values of the fixed end temperatures,  $T_0 = 40$  and  $T_5 = 200$ , can be substituted and moved to the right-hand side. The results are four equations with four unknowns expressed in matrix form as

$$\begin{bmatrix} 2.04 & -1 & 0 & 0 \\ -1 & 2.04 & -1 & 0 \\ 0 & -1 & 2.04 & -1 \\ 0 & 0 & -1 & 2.04 \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} = \begin{bmatrix} 40.8 \\ 0.8 \\ 0.8 \\ 200.8 \end{bmatrix} \quad (9.28)$$

So, our original differential equation has been converted into an equivalent system of linear algebraic equations. Consequently, we can use the techniques described in this chapter to solve for the temperatures. For example, using Python's `linalg.solve` function,

```
A = np.matrix('2.04 -1 0 0 ; -1 2.04 -1 0 ; 0 -1 2.04 -1 ; 0 0 -1 2.04')
b = np.matrix(' 40.8 ; 0.8 ; 0.8 ; 200.8 ')
T = np.linalg.solve(A,b)
print('Temperatures in degC are:')
for i in range(4):
    print(' {0:6.2f}'.format(T[i,0]))
Temperatures in degC are:
```

```

65.97
93.78
124.54
159.48

```

A plot can also be developed comparing these results with the analytical solution obtained with [Eq. \(9.25\)](#),

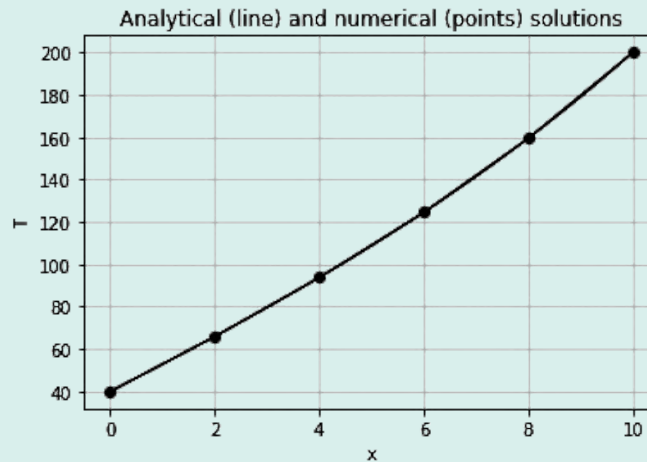
```

import pylab
Tplot = np.zeros((6))
Tplot[0] = 40
Tplot[5] = 200
for i in range(1,5):
    Tplot[i] = T[i-1]
xplot = np.linspace(0,10,6)
xanlyt = np.linspace(0,10)
TT = lambda x: 73.4523*np.exp(0.1*x)-53.4523*np.exp(-0.1*x)+20
Tanlyt = TT(xanlyt)
pylab.plot(xplot,Tplot,c='k',marker='o')
pylab.plot(xanlyt,Tanlyt,c='k')
pylab.grid()
pylab.xlabel('x')
pylab.ylabel('T')
pylab.title('Analytical (line) and numerical (points) solutions')

```

By observing [Fig. 9.9](#), we see that the numerical results are quite close to those obtained with calculus.

**FIGURE 9.9** A plot of temperature versus distance along a heated rod. Both analytical (line) and numerical (points) solutions are displayed.



In addition to being a linear system, notice that [Eq. \(9.28\)](#) is also tridiagonal. We can employ an efficient solution scheme using the `tridiag` function in [Fig. 9.7](#) to obtain the solution:

```

n = 4
e = np.zeros([n])
f = np.zeros([n])
g = np.zeros([n])
for i in range(n):
    f[i] = 2.04
    if i < n-1:
        g[i] = -1
    if i > 0:
        e[i] = -1
r = np.array([40.8],[0.8],[0.8],[200.8])

```

```
T = tridiag(e,f,g,r)
print('Temperatures in degC are:')
for i in range(4):
    print(' {0:6.2f}'.format(T[i]))

Temperatures in degC are:
  65.97
  93.78
 124.54
 159.48
```

The system presents as tridiagonal because each node depends only on its adjacent nodes. Because we numbered the nodes sequentially, the resulting equations are tridiagonal. Such cases occur often when solving systems of equations for one-dimensional systems such as the heated rod. We will explore some of these in the end-of-chapter problems that follow.

# PROBLEMS

**9.1** Determine the number of total flops as a function of the number of equations,  $n$ , for the tridiagonal algorithm ( Fig. 9.7).

**9.2** Use the graphical method to solve

$$\begin{aligned} 4x_1 - 8x_2 &= -24 \\ x_1 + 6x_2 &= 34 \end{aligned}$$

Check your results by substituting them back into the equations.

**9.3** Use the graphical method to solve

$$\begin{aligned} -1.1x_1 + 10x_2 &= 120 \\ -2x_1 + 17.4x_2 &= 174 \end{aligned}$$

- (a) Check your results by substituting them back into the equations.
- (b) On the basis of the graphical solution, what do you expect regarding the condition of the system?

**9.4** Given the system of equations

$$\begin{aligned} -3x_2 + 7x_3 &= 4 \\ x_1 + 2x_2 - x_3 &= 0 \\ 5x_1 - 2x_2 &= 3 \end{aligned}$$

- (a) Compute the determinant.
- (b) Use Cramer's rule to solve for the  $x$ 's.
- (c) Use Gauss elimination with partial pivoting to solve for the  $x$ 's. As part of the computation, calculate the determinant in order to verify the value computed in (a).
- (d) Substitute your results back into the original equations to check your solution.

**9.5** Given the equations

$$\begin{aligned} 0.5x_1 - x_2 &= -9.5 \\ 1.02x_1 - 2x_2 &= -18.8 \end{aligned}$$

- (a) Solve graphically.
- (b) Compute the determinant.
- (c) On the basis of (a) and (b), what would you expect regarding the system's condition?



- (d) Solve by elimination of the unknowns.  
 (e) Solve again, but with  $a_{11}$  modified to 0.52. Interpret your results.

**9.6** Given the equations

$$\begin{aligned} 10x_1 + 2x_2 - x_3 &= 27 \\ -3x_1 - 5x_2 + 2x_3 &= -61.5 \\ x_1 + x_2 + 6x_3 &= -21.5 \end{aligned}$$

- (a) Solve by naive Gauss elimination. Show all the steps of the computation.  
 (b) Substitute your results into the original equations to check your answers.

**9.7** Given the equations

$$\begin{aligned} 2x_1 - 6x_2 - x_3 &= -38 \\ -3x_1 - x_2 + 7x_3 &= -34 \\ -8x_1 + x_2 - 2x_3 &= -20 \end{aligned}$$

- (a) Solve by Gauss elimination with partial pivoting. As part of the computation, use the diagonal elements to calculate the determinant. Show all the steps of the computation.  
 (b) Substitute your results into the original equations to check your answers.

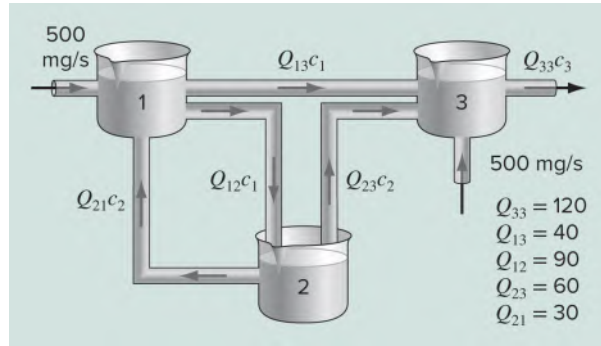
**9.8** Perform the same calculations as in  **Example 9.5**, but for this tridiagonal system:

$$\begin{bmatrix} 0.8 & -0.4 & 0 \\ -0.4 & 0.8 & -0.4 \\ 0 & -0.4 & 0.8 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 41 \\ 25 \\ 105 \end{bmatrix}$$

**9.9**  **Figure P9.9** shows three well-mixed tanks linked by pipes. As indicated, the rate of transfer of a chemical substance through each pipe is equal to the flow rate,  $Q$  (L/s), multiplied by the concentration,  $c$  (mg/L), in that pipe. Since the tanks are well mixed, the concentration in any pipe is the same as that in the tank from which it emanates. If the system is at steady state, the overall transfer rate into a tank will equal the overall transfer rate out. Develop mass balance equations for each tank and solve the three resulting linear algebraic equations to determine the tank concentrations. Use one of the Python solving functions introduced in  **Chap. 9**. What are the volumetric flow rates associated with the external feeds to tank 1 and 3?

---

**FIGURE P9.9** Three well-mixed tanks linked by pipes.



**9.10** A civil engineer involved in a construction project requires 4800, 5800, and 5700 m<sup>3</sup> of sand, fine gravel, and coarse gravel, respectively. There are three pits from which these materials can be obtained. The composition of material in these pits is

|      | Sand % | Fine Gravel % | Coarse Gravel % |
|------|--------|---------------|-----------------|
| Pit1 | 55     | 30            | 15              |
| Pit2 | 25     | 45            | 30              |
| Pit3 | 25     | 20            | 55              |

How many cubic meters must be hauled from each pit in order to meet the engineer's needs? Use one of the Python functions introduced in this chapter to solve the problem.

**9.11** An electrical engineer supervises the production of three types of electrical components. Three kinds of material—metal, plastic, and rubber—are required for production. The amounts needed to produce each component are

| Component | Metal (g/component) | Plastic (g/component) | Rubber (g/Component) |
|-----------|---------------------|-----------------------|----------------------|
| 1         | 15                  | 0.30                  | 1.0                  |
| 2         | 17                  | 0.40                  | 1.2                  |
| 3         | 19                  | 0.55                  | 1.5                  |

If totals of 3.89, 0.095, and 2.82 kg of metal, plastic, and rubber, respectively, are available each day, how many components of each type can be produced per day? Use one of the Python functions introduced in this chapter to solve the problem.

**9.12** Flow of highly viscous fluids through pipes or tubes is governed by a linear relationship between flow rate and pressure drop, given by the *Poiseuille equation*,

$$Q = \frac{\pi R^4}{8\mu L} \Delta P$$

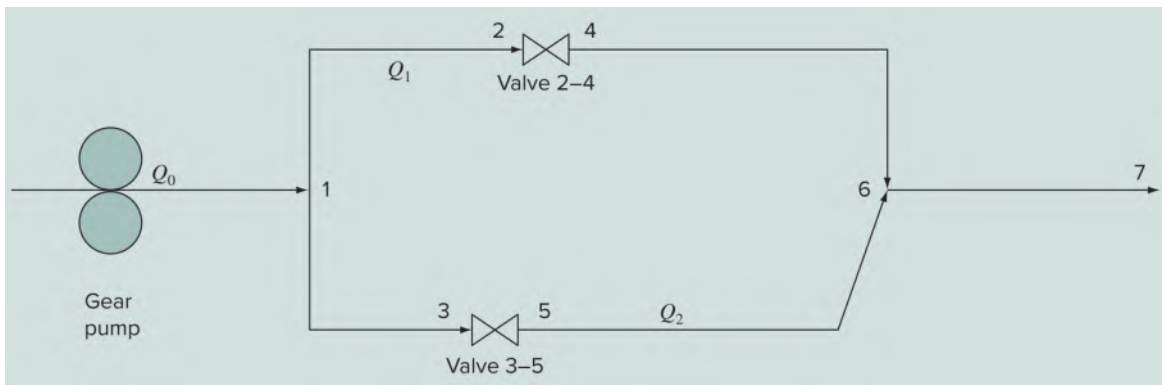
where  $Q$  = volumetric flow rate,  $\text{m}^3/\text{s}$ ,  $\Delta P$  = pressure drop over the length of pipe, Pa,  $R$  = inside radius of the pipe, m,  $\mu$  = fluid viscosity, Pa·s, and  $L$  = length of the pipe, m. Similarly, flow through a resistance, like a valve, also has a linear relationship.

$$Q = C_v \Delta P$$

where  $C_v$  = a valve coefficient depending on the design and manufacturer,  $\text{m}^3/(\text{s} \cdot \text{Pa})$ .

In the flow network shown in  Fig. P9.12, a positive-displacement gear pump discharges a viscous fluid at a given flow rate. The fluid, which is highly incompressible, flows through the network and discharges at the same flow rate but lower, given pressure. It is desired to calculate the flow rates in the two branches of the network and the seven pressures at nodes 0 through 6 on the figure. Solve this network for the following parameter values. Use the Python `gausspivot` function.

**FIGURE P9.12**



| Pipe | Diameter (mm) | Length (cm) |
|------|---------------|-------------|
| 0-1  | 20.8          | 40          |
| 1-2  | 15.7          | 60          |
| 1-3  | 15.7          | 50          |
| 4-6  | 15.7          | 150         |
| 5-6  | 15.7          | 100         |
| 6-7  | 26.7          | 75          |

| Valve | $c_v(\text{m}^3/\text{s}/\text{Pa})$ |
|-------|--------------------------------------|
| 2-4   | 2.00E-09                             |
| 3-5   | 2.75E-09                             |

$$Q_0 = 14 \text{ liters/min}$$

$$P_7 = 200,000 \text{ Pa}$$

Fluid: high-fructose corn syrup at 40°C.

Viscosity: 24 Pa·s = 24,000 centipoise.

Report your flow rates and pressures in additional units, liters/min and psi (1 psi  $\cong$  6895 Pa).

Hints: There are six equations for the pipes, two for the valves, and one equation relating the three flow rates. Be sure to convert the pipe measurements to meters.

**9.13** A multi-stage extraction process is depicted in  **Fig. P9.13**. In such systems, a stream containing a weight fraction,  $y_{\text{in}}$ , of a chemical substance enters from left at a mass flow rate,  $F_1$ . Simultaneously, an immiscible solvent carrying a weight fraction,  $x_{\text{in}}$  (usually zero, or close to it) of the same substance enters from the right at a flow rate,  $F_2$ . In each stage, the two immiscible streams contact and the chemical substance transfers from one to the other. For the  $i$ th stage, a mass balance on the substance can be written

$$F_1 y_{i-1} + F_2 x_{i+1} = F_1 y_i + F_2 x_i \quad (\text{P9.13a})$$

At each stage, because of the mixing contact, an equilibrium is approached between  $y_i$  and  $x_i$  and can be described by a distribution or partition coefficient,

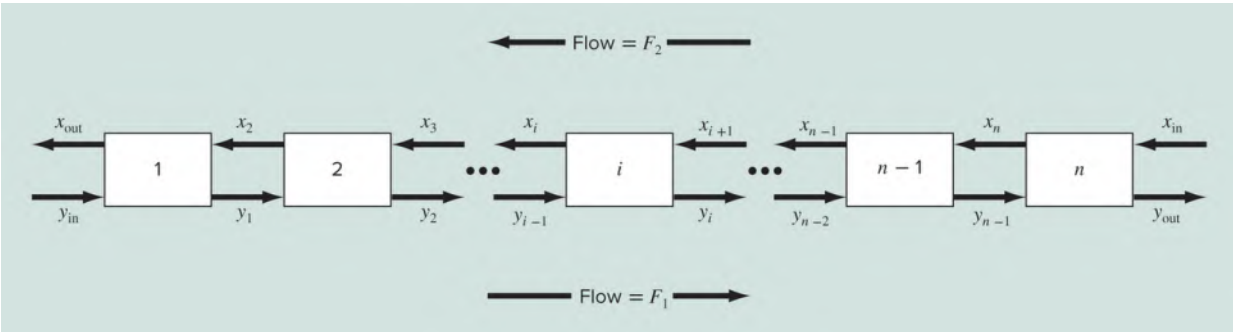
$$K = \frac{x_i}{y_i} \quad (\text{P9.13b})$$

If we solve  **Eq. P9.13b** for  $x_i$  and substitute the result into  **Eq. (P9.13a)**, the result can be written

$$-y_{i-1} + \left(1 + \frac{F_2}{F_1} K\right) y_i - \left(\frac{F_2}{F_1} K\right) y_{i+1} = 0 \quad (\text{P9.13c})$$

If  $F_1 = 500 \text{ kg/hr}$ ,  $F_2 = 1000 \text{ kg/hr}$ ,  $y_{\text{in}} = 0.1$ ,  $x_{\text{in}} = 0$ , and  $K = 4$ , determine the values of  $y_{\text{out}}$  and  $x_{\text{out}}$  for a five-stage extractor. Note that  **Eq. P9.13c** must be modified for the first and last stages to take into account the known inlet fractions. Use the `tridiag` Python function for your solution. Present your results as a plot of both  $y$  and  $x$  versus stage number.

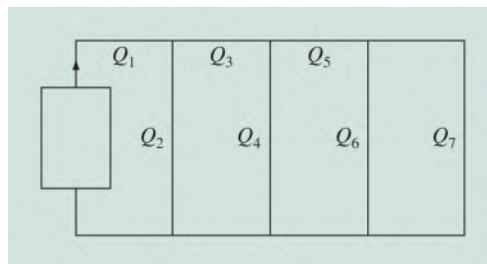
**FIGURE P9.13**



**9.14** A gear pump delivers a flow,  $Q_1 = 100 \text{ mL/min}$ , of a highly viscous fluid to a flow network which is described in [Fig. P9.14](#). Every pipe section has the same length and diameter. The mass and mechanical energy balances can be simplified to obtain the flow rates in every pipe. Solve the following system of equations to determine those flow rates. You can arrange the equations so they form a tridiagonal system and then use the Python `tridiag` function to solve them.

$$\begin{aligned} Q_3 + 2Q_4 - 2Q_2 &= 0 & Q_1 &= Q_2 + Q_3 \\ Q_5 + 2Q_6 - 2Q_4 &= 0 & Q_3 &= Q_4 + Q_5 \\ 3Q_7 - 2Q_6 &= 0 & Q_5 &= Q_6 + Q_7 \end{aligned}$$

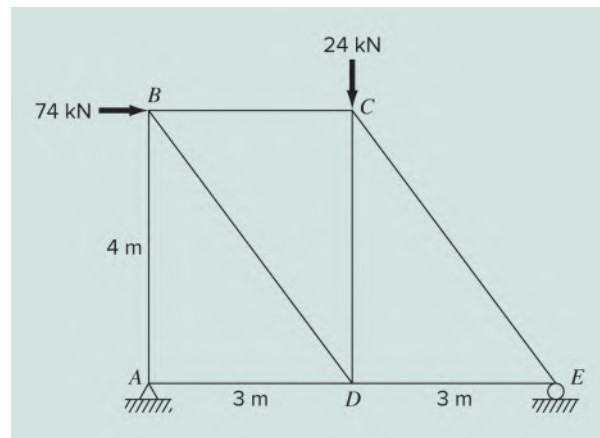
**FIGURE P9.14**



**9.15** A truss is loaded as shown in [Fig. P9.15](#). Using the following set of equations, solve for the ten unknowns,  $AB$ ,  $BC$ ,  $AD$ ,  $BD$ ,  $CD$ ,  $DE$ ,  $CE$ ,  $A_x$ ,  $A_y$ , and  $E_y$ . Use the `gausspivot` function for the solution.

$$\begin{aligned}
A_x + AD &= 0 & -24 - CD - \frac{4}{5}CE &= 0 \\
A_y + AB &= 0 & -AD + DE - \frac{3}{5}BD &= 0 \\
74 + BC + \frac{3}{5}BD &= 0 & CD + \frac{4}{5}BD &= 0 \\
-AB - \frac{4}{5}BD &= 0 & -DE - \frac{3}{5}CE &= 0 \\
-BC + \frac{3}{5}CE &= 0 & E_y + \frac{4}{5}CE &= 0
\end{aligned}$$

**FIGURE P9.15**



**9.16** A *pentadiagonal*, banded system can be represented as

$$\begin{bmatrix}
f_1 & g_1 & h_1 & 0 & 0 & 0 & \cdots & 0 \\
e_2 & f_2 & g_2 & h_2 & 0 & 0 & \cdots & 0 \\
d_3 & e_3 & f_3 & g_3 & h_3 & 0 & \cdots & 0 \\
\vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\
\vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\
0 & \cdots & 0 & d_{n-2} & e_{n-2} & f_{n-2} & g_{n-2} & h_{n-2} \\
0 & \cdots & 0 & 0 & d_{n-1} & e_{n-1} & f_{n-1} & g_{n-1} \\
0 & \cdots & 0 & 0 & 0 & d_n & e_n & f_n
\end{bmatrix}
\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ \vdots \\ x_{n-2} \\ x_{n-1} \\ x_n \end{bmatrix}
=
\begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ \vdots \\ \vdots \\ r_{n-2} \\ r_{n-1} \\ r_n \end{bmatrix}$$

Develop a Python function, `pentadiag`, to solve such a system of linear algebraic equations without pivoting in a similar fashion to the algorithm used for tridiagonal systems in [Sec. 9.4.1](#). Test it

for the following case:

$$\begin{bmatrix} 8 & -2 & -1 & 0 & 0 \\ -2 & 9 & -4 & -1 & 0 \\ -1 & -3 & 7 & -1 & -2 \\ 0 & -4 & -2 & 12 & -5 \\ 0 & 0 & -7 & -3 & -15 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} 5 \\ 2 \\ 1 \\ 1 \\ 5 \end{bmatrix}$$

Check your answer by employing a solution method from [Chap. 8](#), for example, `np.linalg.solve`.

**9.17** Develop a Python function, `gausspivot2`, based on [Fig. 9.6](#), to implement Gauss elimination with partial pivoting. Modify the code so that it computes and returns, in addition to the solution, the determinant (with the correct sign). The function should also check for singularity, or near singularity, by testing the absolute value of each pivot element after the row exchange against a tolerance. The tolerance should be one of the arguments and have a default value of  $1 \times 10^{-12}$ . When the value is below the tolerance, the function returns an error message. Test your program with the equations from [Prob. 9.5](#) and a tolerance value of  $1 \times 10^{-5}$ .

**9.18** As described in [Sec. 9.5](#), linear algebraic equations can arise in the solution of differential equations via finite-difference approximations. For example, the following differential equation results from a steady-state mass balance for a chemical in a stream along the axis of flow,  $x$ :

$$0 = D \frac{d^2 c}{dx^2} - U \frac{dc}{dx} - kc$$

where  $c$  = concentration of the chemical species,  $x$  = axial distance,  $D$  = a dispersion coefficient,  $U$  = fluid velocity, and  $k$  = a first-order decay rate.

- Convert this differential equation into an equivalent system of simultaneous algebraic equations using finite-difference techniques. Use a backward difference approximation for the first derivative and a centered-difference approximation for the second derivative.
- Develop a Python function called `YourLastName_StreamCalc` to solve these equations over the domain  $0 \leq x \leq L$  using  $n$  interior nodes, equivalent to a  $\Delta x = L/(n+1)$ . The function should return the concentrations and distances. The function should contain the arguments  $D$ ,  $U$ ,  $k$ ,  $c_0$ ,  $c_L$ ,  $L$ , and  $n$ .
- Given the following values, solve the equations over the domain  $0 \leq x \leq 10$  m and produce a plot of concentration versus distance. To obtain a good approximation, use 25 interior nodes.  
 $D = 2 \text{ m}^2/\text{d}$ ,  $U = 1 \text{ m/d}$ ,  $k = 0.2 \text{ 1/d}$ ,  $c(0) = 80 \text{ mg/L}$ ,  
 $c(10) = 20 \text{ mg/L}$

**9.19** The following results from a force balance for a beam with uniform loading.

$$0 = EI \frac{d^2 y}{dx^2} - \frac{wL}{2}x + \frac{w}{2}x^2$$

where  $x$  = distance along the beam (m),  $y$  = deflection (m),  $L$  = beam length (m),

$E$  = modulus of elasticity ( $\text{N/m}^2$ ),  $I$  = moment of inertia ( $\text{m}^4$ ), and  $w$  = uniform load ( $\text{N/m}$ )

- (a) Convert this differential equation into a set of simultaneous algebraic equations using a centered-difference approximation to the second derivative.
- (b) Develop a Python function called `YourLastName_BeamCalc` function to solve these equations over the domain  $0 \leq x \leq L$  and return the resulting distances and deflections. The first line of your function should be that returns the deflections and distances. The arguments to the function should be  $E$ ,  $I$ ,  $w$ ,  $y_0$ ,  $y_L$ ,  $L$ , and  $n$ , where  $n$  = the number of interior node points.
- (c) Write a Python script that invokes the function and plots  $y$  versus  $x$ .
- (d) Test your script with the following parameter values:  $L = 3$  m,  $\Delta x = 0.2$  m,  $E = 250 \times 10^9$  N/m<sup>2</sup>,  $I = 3 \times 10^{-4}$  m<sup>4</sup>,  $w = 22,500$  N/m,  $y(0)$  and  $y(L) = 0$ .

**9.20** Heat is conducted along a metal rod positioned between two walls, each at a fixed temperature. Aside from conduction through the metal, heat is transferred between the rod and the surrounding air. Based on a thermal energy balance, the distribution of temperature along the rod is described by the following second-order differential equation:

$$0 = \frac{d^2 T}{dx^2} + h'(T_a - T)$$

where  $T$  = rod temperature,  $h'$  = a bulk heat transfer coefficient reflecting the relative importance of convection to conduction,  $x$  = distance long the rod, and  $T_a$  = ambient temperature.

- (a) Convert this differential equation to an equivalent system of simultaneous algebraic equations using a centered-difference approximation for the second derivative.
- (b) Develop a Python function called `YourLastName_RodCalc` to solve these equations over a domain  $0 \leq x \leq L$  that returns the resulting temperatures and distances. The arguments to the function should be  $h'$ ,  $T_a$ ,  $T_0$ ,  $T_L$ ,  $L$ , and  $n$ , where  $n$  is the number of interior nodes.
- (c) Write a Python script that invokes this function and plots the results.
- (d) Test your script and function with the following parameter values:  $h' = 0.0425$  m<sup>-2</sup>,  $L = 12$  m,  $T_a = 30^\circ\text{C}$ ,  $T(0) = 60^\circ\text{C}$ ,  $T(L) = 200^\circ\text{C}$ , and  $n = 50$ .



# LU Factorization

## CHAPTER OBJECTIVES

The primary objective of this chapter is to acquaint you with *LU* factorization. Specific objectives and topics covered are

- Understanding that *LU* factorization involves decomposing the coefficient matrix into two triangular matrices that can then be used to efficiently evaluate different right-hand-side vectors.
- Knowing how to express Gauss elimination as an *LU* factorization.
- Given an *LU* factorization, knowing how to evaluate multiple right-hand-side vectors.
- Recognizing that Cholesky's method provides an efficient way to decompose a symmetric matrix and that the resulting triangular matrix and its transpose can be used to evaluate right-hand-side vectors efficiently.
- Understanding in general terms what happens when the `linalg.solve` function from Python's NumPy module is used to solve a set of linear algebraic equations.

As described in [Chap. 9](#), Gauss elimination is designed to solve systems of linear algebraic equations:

$$[A] \{x\} = \{b\} \quad (10.1)$$

Although it certainly represents a sound way to solve such systems, it becomes inefficient when solving equations with the same coefficients  $[A]$ , but with different right-hand-side constants  $\{b\}$ .

Recall that Gauss elimination involves two steps: forward elimination and back substitution ([Fig. 9.3](#)). As we learned in [Sec. 9.2.2](#), the forward-elimination step comprises the bulk of the computational effort. This is particularly true for large systems of equations.

*LU* factorization methods separate the time-consuming elimination of the matrix  $[A]$  from the manipulations of the right-hand side  $\{b\}$ . Thus, once  $[A]$  has been “factored” or “decomposed,” multiple right-hand-side vectors can be evaluated in an efficient manner.

Interestingly, Gauss elimination itself can be expressed as an *LU* factorization. Before showing how this can be done, let us first provide a mathematical overview of the factorization strategy.

# 10.1 OVERVIEW OF *LU* FACTORIZATION

Just as was the case with Gauss elimination, *LU* factorization requires pivoting to avoid division by zero. However, to simplify the following description, we will omit pivoting. In addition, the following explanation is limited to a set of three simultaneous equations. The results can be directly extended to *n*-dimensional systems.

 **Equation (10.1)** can be rearranged to give

$$[A] \{x\} - \{b\} = 0 \quad (10.2)$$

Suppose that  **Eq. (10.2)** could be expressed as an upper triangular system. For example, for a  $3 \times 3$  system:

Page 243

$$\begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \end{bmatrix} \quad (10.3)$$

Recognize that this is similar to the manipulation that occurs in the first step of Gauss elimination. That is, elimination is used to reduce the system to upper triangular form.  **Equation (10.3)** can also be expressed in matrix notation and rearranged to give

$$[U] \{x\} - \{d\} = 0 \quad (10.4)$$

Now, assume that there is a lower diagonal matrix with 1's on the diagonal,

$$[L] = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \quad (10.5)$$

that has the property that when  **Eq. (10.4)** is premultiplied by it,  **Eq. (10.2)** is the result. That is,

$$[L] \{[U] \{x\} - \{d\}\} = [A] \{x\} - \{b\} \quad (10.6)$$

If this equation holds, it follows from the rules for matrix multiplication that

$$[L][U] = [A] \quad (10.7)$$

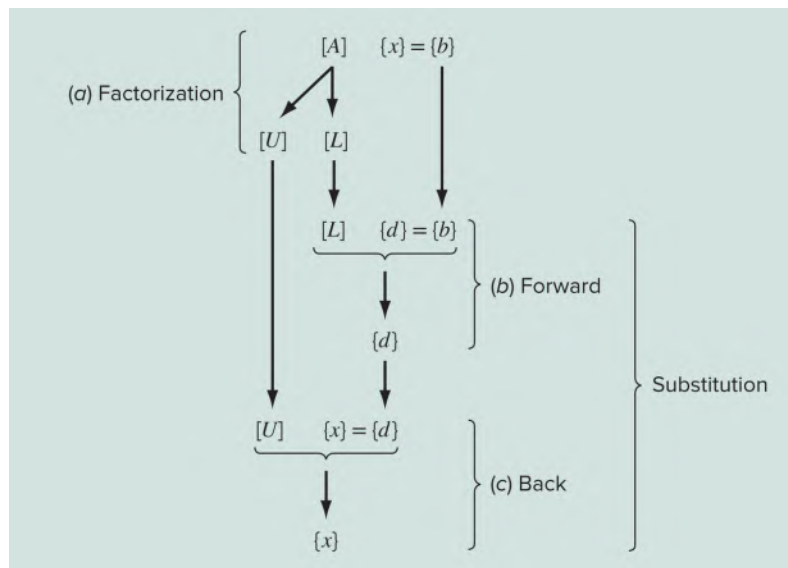
and

$$[L]\{d\} = \{b\} \quad (10.8)$$

A two-step strategy (see [Fig. 10.1](#)) for obtaining solutions can be based on [Eqs. \(10.3\)](#), [\(10.7\)](#), and [\(10.8\)](#):

1. *LU* factorization step.  $[A]$  is factored or “decomposed” into lower  $[L]$  and upper  $[U]$  triangular matrices.
2. Substitution step.  $[L]$  and  $[U]$  are used to determine a solution  $\{x\}$  for a right-hand side  $\{b\}$ . This step itself consists of two steps. First, [Eq. \(10.8\)](#) is used to generate an intermediate vector  $\{d\}$  by forward substitution. Then, the result is substituted into [Eq. \(10.3\)](#), which can be solved by back substitution for  $\{x\}$ .

**FIGURE 10.1** The steps in *LU* factorization.



Now, let us show how Gauss elimination can be implemented in this way.

## 10.2 GAUSS ELIMINATION AS $LU$ FACTORIZATION

Although it might appear at face value to be unrelated to  $LU$  factorization, Gauss elimination can be used to decompose  $[A]$  into  $[L]$  and  $[U]$ . This can be easily seen for  $[U]$ , which is a direct product of the forward elimination. Recall that the forward-elimination step is intended to reduce the original coefficient matrix  $[A]$  to the form

$$[U] = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a'_{22} & a'_{23} \\ 0 & 0 & a''_{33} \end{bmatrix} \quad (10.9)$$

which is in the desired upper triangular format.

Though it might not be as apparent, the matrix  $[L]$  is also produced during the step. This can be readily illustrated for a three-equation system,

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{Bmatrix} x_1 \\ x_2 \\ x_3 \end{Bmatrix} = \begin{Bmatrix} b_1 \\ b_2 \\ b_3 \end{Bmatrix}$$

The first step in Gauss elimination is to multiply row 1 by the factor [recall [Eq. \(9.9\)](#)]

$$f_{21} = \frac{a_{21}}{a_{11}}$$

and subtract the result from the second row to eliminate  $a_{21}$ . Similarly, row 1 is multiplied by

$$f_{31} = \frac{a_{31}}{a_{11}}$$

and the result subtracted from the third row to eliminate  $a_{31}$ . The final step is to multiply the modified second row by

$$f_{32} = \frac{a'_{32}}{a'_{22}}$$

and subtract the result from the third row to eliminate  $a'_{32}$ .

Now, suppose that we merely perform all these manipulations on the matrix  $[A]$ . Clearly, if we do not want to change the equations, we also have to do the same to the right-hand side  $\{b\}$ . But, there is absolutely no reason that we have to perform the manipulations simultaneously. Thus, we could save the  $f$ 's and manipulate  $\{b\}$  later.

Where do we store the factors  $f_{21}$ ,  $f_{31}$ , and  $f_{32}$ ? Recall that the whole idea behind the elimination was to create zeros in  $a_{21}$ ,  $a_{31}$ , and  $a_{32}$ . Thus, we can store  $f_{21}$  in  $a_{21}$ ,  $f_{31}$  in  $a_{31}$ , and  $f_{32}$  in  $a_{32}$ . After elimination, the  $[A]$  matrix can therefore be written as

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ f_{21} & a'_{22} & a'_{23} \\ f_{31} & f_{32} & a''_{33} \end{bmatrix} \quad (10.10)$$

This matrix, in fact, represents an efficient storage of the  $LU$  factorization of  $[A]$ ,

$$[A] \rightarrow [L][U] \quad (10.11)$$

where

$$[U] = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a'_{22} & a'_{23} \\ 0 & 0 & a''_{33} \end{bmatrix} \quad (10.12)$$

and

$$[L] = \begin{bmatrix} 1 & 0 & 0 \\ f_{21} & 1 & 0 \\ f_{31} & f_{32} & 1 \end{bmatrix} \quad (10.13)$$

The following example confirms that  $[A] = [L][U]$ .

## EXAMPLE 10.1

### ***LU* Factorization with Gauss Elimination**

#### **Problem Statement**

Derive an  $LU$  factorization based on the Gauss elimination performed previously in

 **Example 9.3.**

#### **Solution**

In  **Example 9.3**, we used Gauss elimination to solve a set of linear algebraic equations that had the following coefficient matrix:

$$[A] = \begin{bmatrix} 3 & -0.1 & -0.2 \\ 0.1 & 7 & -0.3 \\ 0.3 & -0.2 & 10 \end{bmatrix}$$

After forward elimination, the following upper triangular matrix was obtained:

$$[U] = \begin{bmatrix} 3 & -0.1 & -0.2 \\ 0 & 7.00333 & -0.293333 \\ 0 & 0 & 10.0120 \end{bmatrix}$$

The factors employed to obtain the upper triangular matrix can be assembled into a lower triangular matrix. The elements  $a_{21}$  and  $a_{31}$  were eliminated by using the factors

$$f_{21} = \frac{0.1}{3} = 0.0333333 \quad f_{31} = \frac{0.3}{3} = 0.100000$$

and the element  $a_{32}$  was eliminated by using the factor

$$f_{32} = \frac{-0.19}{7.00333} = -0.0271300$$

Thus, the lower triangular matrix is

$$[L] = \begin{bmatrix} 1 & 0 & 0 \\ 0.0333333 & 1 & 0 \\ 0.100000 & -0.0271300 & 1 \end{bmatrix}$$

Consequently, the  $LU$  factorization is

$$[A] = [L][U] = \begin{bmatrix} 1 & 0 & 0 \\ 0.0333333 & 1 & 0 \\ 0.100000 & -0.0271300 & 1 \end{bmatrix} \begin{bmatrix} 3 & -0.1 & -0.2 \\ 0 & 7.00333 & -0.293333 \\ 0 & 0 & 10.0120 \end{bmatrix}$$

This result can be verified by performing the multiplication of  $[L][U]$  to give

$$[L][U] = \begin{bmatrix} 3 & -0.1 & -0.2 \\ 0.0999999 & 7 & -0.3 \\ 0.3 & -0.2 & 9.99996 \end{bmatrix}$$

where the minor discrepancies are due to roundoff.

After the matrix is decomposed, a solution can be generated for a particular right-hand-side vector  $\{b\}$ . This is done in two steps. First, a forward-substitution step is executed by solving [Eq. \(10.8\)](#) for  $\{d\}$ . It is important to recognize that this merely amounts to performing the elimination manipulations on  $\{b\}$ . Thus, at the end of this step, the right-hand side will be in the same state that it would have been had we performed forward manipulation on  $[A]$  and  $\{b\}$  simultaneously.

The forward-substitution step can be represented concisely as

$$d_i = b_i - \sum_{j=1}^{i-1} l_{ij}d_j \quad \text{for } i = 1, 2, \dots, n$$

The second step then merely amounts to implementing back substitution to solve [Eq. \(10.3\)](#). Again, it is important to recognize that this is identical to the back-substitution phase of conventional Gauss elimination [compare with [Eqs. \(9.12\)](#) and [\(9.13\)](#)]:

$$x_n = d_n / u_{nn}$$

$$x_i = \frac{d_i - \sum_{j=i+1}^n u_{ij}x_j}{u_{ii}} \quad \text{for } i = n-1, n-2, \dots, 1$$

## EXAMPLE 10.2

### The Substitution Steps

#### Problem Statement

Complete the problem initiated in [Example 10.1](#) by generating the final solution with forward and back substitution.

#### Solution

As just stated, the intent of forward substitution is to impose the elimination manipulations that we had formerly applied to  $[A]$  on the right-hand-side vector  $\{b\}$ . Recall that the system being solved is

$$\begin{bmatrix} 3 & -0.1 & -0.2 \\ 0.1 & 7 & -0.3 \\ 0.3 & -0.2 & 10 \end{bmatrix} \begin{Bmatrix} x_1 \\ x_2 \\ x_3 \end{Bmatrix} = \begin{Bmatrix} 7.85 \\ -19.3 \\ 71.4 \end{Bmatrix}$$

and that the forward-elimination phase of conventional Gauss elimination resulted in

$$\begin{bmatrix} 3 & -0.1 & -0.2 \\ 0 & 7.00333 & -0.293333 \\ 0 & 0 & 10.0120 \end{bmatrix} \begin{Bmatrix} x_1 \\ x_2 \\ x_3 \end{Bmatrix} = \begin{Bmatrix} 7.85 \\ -19.5617 \\ 70.0843 \end{Bmatrix}$$

The forward-substitution phase is implemented by applying [Eq. \(10.8\)](#):

$$\begin{bmatrix} 1 & 0 & 0 \\ 0.0333333 & 1 & 0 \\ 0.100000 & -0.0271300 & 1 \end{bmatrix} \begin{Bmatrix} d_1 \\ d_2 \\ d_3 \end{Bmatrix} = \begin{Bmatrix} 7.85 \\ -19.3 \\ 71.4 \end{Bmatrix}$$

or multiplying out the left-hand side:

$$\begin{aligned} d_1 &= 7.85 \\ 0.0333333d_1 + d_2 &= -19.3 \\ 0.100000d_1 - 0.0271300d_2 + d_3 &= 71.4 \end{aligned}$$

We can solve the first equation for  $d_1 = 7.85$ , which can be substituted into the second equation to solve for

$$d_2 = -19.3 - 0.0333333(7.85) = -19.5617$$

Both  $d_1$  and  $d_2$  can be substituted into the third equation to give

$$d_3 = 71.4 - 0.1(7.85) + 0.02713(-19.5617) = 70.0843$$

Thus,

$$\{d\} = \begin{Bmatrix} 7.85 \\ -19.5617 \\ 70.0843 \end{Bmatrix}$$

This result can then be substituted into [Eq. \(10.3\)](#),  $[U]\{x\} = \{d\}$ :

$$\begin{bmatrix} 3 & -0.1 & -0.2 \\ 0 & 7.00333 & -0.293333 \\ 0 & 0 & 10.0120 \end{bmatrix} \begin{Bmatrix} x_1 \\ x_2 \\ x_3 \end{Bmatrix} = \begin{Bmatrix} 7.85 \\ -19.5617 \\ 70.0843 \end{Bmatrix}$$

which can be solved by back substitution (see [Example 9.3](#) for details) for the final solution:

$$\{x\} = \begin{Bmatrix} 3 \\ -2.5 \\ 7.00003 \end{Bmatrix}$$

## 10.2.1 LU Factorization with Pivoting

Just as for standard Gauss elimination, partial pivoting is necessary to obtain reliable solutions with  $LU$  factorization. One way to do this involves using a permutation matrix (recall [Sec. 8.1.2](#)). The approach consists of the following steps:

1. Elimination. The  $LU$  factorization with pivoting of a matrix  $[A]$  can be represented in matrix form as

$$[P][A] = [L][U]$$

The upper triangular matrix,  $[U]$ , is generated by elimination with partial pivoting, while storing the multiplier factors in  $[L]$  and employing the permutation matrix,  $[P]$ , to keep track of the row switches.



2. Forward substitution. The matrices  $[L]$  and  $[P]$  are used to perform the elimination step with pivoting on  $\{b\}$  in order to generate the intermediate right-hand-side vector,  $\{d\}$ . This step can be represented concisely as the solution of the following matrix formulation:

$$[L] \{d\} = [P] \{b\}$$

3. Back substitution. The final solution is generated in the same fashion as done previously for Gauss elimination. This step can also be represented concisely as the solution of the matrix formulation:

$$[U] \{x\} = \{d\}$$

The approach is illustrated in the following example.

## EXAMPLE 10.3

### LU Factorization with Pivoting

#### Problem Statement

Compute the  $LU$  factorization and find the solution for the same system analyzed in [Example 9.4](#)

$$\begin{bmatrix} 0.0003 & 3.0000 \\ 1.0000 & 1.0000 \end{bmatrix} \begin{Bmatrix} x_1 \\ x_2 \end{Bmatrix} = \begin{Bmatrix} 2.0001 \\ 1.0000 \end{Bmatrix}$$

#### Solution

Before elimination, we set up the initial permutation matrix:

$$[P] = \begin{bmatrix} 1.0000 & 0.0000 \\ 0.0000 & 1.0000 \end{bmatrix}$$

We immediately see that pivoting is necessary, so prior to elimination we switch the rows:

$$[A] = \begin{bmatrix} 1.0000 & 1.0000 \\ 0.0003 & 3.0000 \end{bmatrix}$$

At the same time, we keep track of the pivot by switching the rows of the permutation matrix:

$$[P] = \begin{bmatrix} 0.0000 & 1.0000 \\ 1.0000 & 0.0000 \end{bmatrix}$$

We then eliminate  $a_{21}$  by subtracting the factor  $l_{21} = a_{21}/a_{11} = 0.0003/1 = 0.0003$  from the second row of  $A$ . In so doing, we compute that the new value of  $a'_{22} = 3 - 0.0003(1) = 2.9997$ . Thus, the elimination step is complete with the result:

$$[U] = \begin{bmatrix} 1 & 1 \\ 0 & 2.9997 \end{bmatrix} \quad [L] = \begin{bmatrix} 1 & 0 \\ 0.0003 & 1 \end{bmatrix}$$

Before implementing forward substitution, the permutation matrix is used to reorder the right-hand-side vector to reflect the pivots as in

$$[P] \{b\} = \begin{bmatrix} 0.0000 & 1.0000 \\ 1.0000 & 0.0000 \end{bmatrix} \begin{Bmatrix} 2.0001 \\ 1 \end{Bmatrix} = \begin{Bmatrix} 1 \\ 2.0001 \end{Bmatrix}$$

Then, forward substitution is applied as in

$$\begin{bmatrix} 1 & 0 \\ 0.0003 & 1 \end{bmatrix} \begin{Bmatrix} d_1 \\ d_2 \end{Bmatrix} = \begin{Bmatrix} 1 \\ 2.0001 \end{Bmatrix}$$

which can be solved for  $d_1 = 1$  and  $d_2 = 2.0001 - 0.0003(1) = 1.9998$ . At this point, the system is

Page 248

$$\begin{bmatrix} 1 & 1 \\ 0 & 2.9997 \end{bmatrix} \begin{Bmatrix} x_1 \\ x_2 \end{Bmatrix} = \begin{Bmatrix} 1 \\ 1.9998 \end{Bmatrix}$$

Applying back substitution gives the final result:

$$x_2 = \frac{1.9998}{2.9997} = 0.66667$$

$$x_1 = \frac{1 - 1(0.66667)}{1} = 0.33333$$

The  $LU$  factorization algorithm requires the same total flops as for Gauss elimination. The only difference is that a little less effort is expended in the factorization phase since the operations are not applied to the right-hand side. Conversely, the substitution phase takes a little more effort.

## 10.2.2 $LU$ Factorization with Python

The SciPy module, available in Python, has a built-in function in the `linalg` submodule that computes the  $LU$  factorization. It has the general syntax<sup>1</sup>:

```
from scipy.linalg import lu
P,L,U = lu(A)
```

where  $P$  is a permutation matrix,  $L$  is the lower triangular matrix, and  $U$  is the upper triangular matrix. The permutation matrix is arranged in columns to show the row swaps. This function uses partial pivoting to avoid division by zero and the  $P$  matrix indicates the rows that were swapped. The following example shows how it can be employed to generate both the  $LU$  factorization and a solution for the problem that was solved in [Examples 10.1](#) and [10.2](#).

### EXAMPLE 10.4

## LU Factorization with Python

### Problem Statement

Use Python to compute the  $LU$  factorization and to find the solution of the linear system of equations analyzed in [Examples 10.1](#) and [10.2](#):

$$\begin{bmatrix} 3 & -0.1 & -0.2 \\ 0.1 & 7 & -0.3 \\ 0.3 & -0.2 & 10 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 7.85 \\ -19.3 \\ 71.4 \end{bmatrix}$$

### Solution

First, the SciPy and NumPy modules are imported and the coefficient matrix and constant vector assigned to the variables  $A$  and  $b$ , respectively.

```
import scipy as sc
import numpy as np

A = np.matrix(' 3 -0.1 -0.2 ; 0.1 7 -0.3 ; 0.3 -0.2 10 ')
b = np.matrix(' 7.85 ; -19.3 ; 71.4 ')
```

Next, the  $LU$  factorization is computed with

```
P,L,U = sc.linalg.lu(A)
```

where  $P$ ,  $L$ , and  $U$  are arrays (`ndarray` class) and the results are displayed as follows:

Page 249

```
print('L=\n',L)
print('U=\n',U)
print('P=\n',P)

L=
[[ 1. 0. 0. ]
 [ 0.03333333 1. 0. ]
 [ 0.1 -0.02712994 1. ]]
U=
[[ 3. -0.1 -0.2 ]
 [ 0. 7.00333333 -0.29333333]
 [ 0. 0. 10.01204188]]
P=
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

The  $P$  matrix indicates that there were no pivot swaps. We can check the factorization by multiplying  $L$  and  $U$  to reconstruct  $A$ .

```
print(L.dot(U))

[[ 3. -0.1 -0.2]
 [ 0.1 7. -0.3]
 [ 0.3 -0.2 10. ]]
```

Note:  $L * U$  will not produce the desired result because these are in the `ndarray` class and multiplication will be an array operation, that is, item per item.

To generate the solution, following the steps in [Sec. 10.2.1](#),

```
d = np.linalg.solve(L,b)
x = np.linalg.solve(U,d)
```

```
print(x)
```

```
[[ 3. ]
```

```
 [-2.5]
```

```
 [ 7. ]]
```

These results agree with those obtained by hand in [Example 10.2](#).

## 10.3 CHOLSKY FACTORIZATION

Recall from [Chap. 8](#) that a symmetric matrix is one where  $a_{ij} = a_{ji}$  for all  $i$  and  $j$ . In other words,  $[A] = [A]^T$ . Such systems occur commonly in both mathematical and engineering/science problem contexts.

Special solution techniques are available for such systems. They offer computational advantages because only half the storage is needed and only half the computation time is required for their solution.

One of the most popular approaches involves *Cholesky factorization* (also called Cholesky decomposition). This algorithm is based on the fact that a symmetric matrix can be decomposed, as in

$$[A] = [U]^T [U] \quad (10.14)$$

That is, the resulting triangular factors are the transpose of each other.

The terms of [Eq. \(10.14\)](#) can be multiplied out and set equal to each other. The factorization can be generated efficiently by recurrence relations. For the  $i$ th row:

$$u_{ii} = \sqrt{a_{ii} - \sum_{k=1}^{i-1} u_{ki}^2} \quad (10.15)$$

$$u_{ij} = \frac{a_{ij} - \sum_{k=1}^{i-1} u_{ki} u_{kj}}{u_{ii}} \quad \text{for } j = i + 1, \dots, n \quad (10.16)$$

### EXAMPLE 10.5

#### Cholesky Factorization

#### Problem Statement

Compute the Cholesky factorization for the symmetric matrix

$$[A] = \begin{bmatrix} 6 & 15 & 55 \\ 15 & 55 & 225 \\ 55 & 225 & 979 \end{bmatrix}$$

## Solution

For the first row ( $i = 1$ ), [Eq. \(10.15\)](#) is employed to compute

$$u_{11} = \sqrt{a_{11}} = \sqrt{6} = 2.44949$$

Then, [Eq. \(10.16\)](#) can be used to determine

$$u_{12} = \frac{a_{12}}{u_{11}} = \frac{15}{2.44949} = 6.123724$$
$$u_{13} = \frac{a_{13}}{u_{11}} = \frac{55}{2.44949} = 22.45366$$

For the second row ( $i = 2$ ):

$$u_{22} = \sqrt{a_{22} - u_{12}^2} = \sqrt{55 - (6.123724)^2} = 4.1833$$
$$u_{23} = \frac{a_{23} - u_{12}u_{13}}{u_{22}} = \frac{225 - 6.123724(22.45366)}{4.1833} = 20.9165$$

For the third row ( $i = 3$ ):

$$u_{33} = \sqrt{a_{33} - u_{13}^2 - u_{23}^2} = \sqrt{979 - (22.45366)^2 - (20.9165)^2} = 6.110101$$

Thus, the Cholesky factorization yields

$$[U] = \begin{bmatrix} 2.44949 & 6.123724 & 22.45366 \\ & 4.1833 & 20.9165 \\ & & 6.110101 \end{bmatrix}$$

The validity of this factorization can be verified by substituting it and its transpose into [Eq. \(10.14\)](#) to see if their product yields the original matrix  $[A]$ . This is left for an exercise.

After obtaining the factorization, it can be used to determine a solution for a right-hand-side vector  $\{b\}$  in a manner similar to  $LU$  factorization. First, an intermediate vector  $\{d\}$  is created by solving

$$[U]^T \{d\} = \{b\} \quad (10.17)$$

Then, the final solution can be obtained by solving

$$[U] \{x\} = \{d\} \quad (10.18)$$

## 10.3.1 Python Function: `scipy.linalg.cholesky`

Python's SciPy module has a function `cholesky` that generates Cholesky factorization. It has the general syntax,

```
import scipy as sc
U = sc.linalg.cholesky(A)
```

where  $U$  is an upper triangular matrix such that  $U^T U = X$ . The following example shows that it can be employed to generate both the factorization and a solution for the same matrix we looked at in the previous example.

Page 251

### EXAMPLE 10.6

#### Cholesky Factorization with Python

##### Problem Statement

Use Python to compute the Cholesky factorization for the same matrix we analyzed in

 [Example 10.5](#).

$$A = \begin{bmatrix} 6 & 15 & 55 \\ 15 & 55 & 225 \\ 55 & 225 & 979 \end{bmatrix}$$

Also, obtain a solution for a constant vector that is the sum of the rows of  $A$ . Note that, for this case, the answer will be a vector of ones.

##### Solution

First, import the SciPy and NumPy modules and create the  $A$  matrix and the  $b$  vector:

```
import numpy as np
from scipy.linalg import cholesky
A = np.matrix(' 6 15 55 ; 15 55 225 ; 55 225 979')

b = np.matrix([ [np.sum(A[0,:])],[np.sum(A[1,:])],[np.sum(A[2,:])]])
print(b)
```

with the resulting display

```
[[ 76]
 [295]
[1259]]
```

Next, the Cholesky factorization can be computed and displayed with

```
U = cholesky(A)
print(U)

[[ 2.44948974  6.12372436 22.45365598]
 [ 0.    4.18330013 20.91650066]
 [ 0.    0.    6.11010093]]
```

The original matrix can be reconstructed with

```
Ut = U.transpose()
Atest = Ut.dot(U)
print(' Atest=\n',Atest)
```

with the result

```
Atest=
[[ 6. 15. 55.]
 [ 15. 55. 225.]
 [ 55. 225. 979.]]
```

To generate the solution, we execute the commands

```
d = np.linalg.solve(Ut,b)
x = np.linalg.solve(U,d)
print(' Solution is\n',x)
```

and, as expected,

```
Solution is
[[1.]
 [1.]
 [1.]]
```

is the result.



## 10.4 THE PYTHON `np.linalg.solve` FUNCTION

---

We previously introduced and used the `solve` function from the NumPy `linalg` submodule without any clear explanation of how it works. Now that we have some background on matrix solution techniques, we can provide a simplified description of its operation.

The `solve` function is derived from another routine in the LAPACK library. It employs, as introduced in this chapter, *LU* factorization with partial pivoting to produce three matrices,  $P$ ,  $L$ , and  $U$ , such that the matrix product  $P \times L \times U$  reconstructs the coefficient matrix  $A$ . LAPACK, an acronym for **L**inear **A**lgebra **P**ackage, is a software library for numerical linear algebra. This library has a rich and long history dating back to the 1970s and was written originally in the Fortran language.

There is also a function in the SciPy `linalg` submodule named `lu_solve` which can be preceded by calling the `lu_factor` function to solve sets of linear equations via *LU* factorization<sup>2</sup>.

# PROBLEMS

**10.1** Determine the number of total flops as a function of the number of equations,  $n$ , for the (a) factorization, (b) forward substitution, and (c) back-substitution phases of the  $LU$  factorization version of Gauss elimination.

**10.2** Use the rules of matrix multiplication to prove that [Eqs. \(10.7\)](#) and [\(10.8\)](#) follow from [Eq. \(10.6\)](#).

**10.3** Use naive Gauss elimination to factor the following system according to the description in [Sec. 10.2](#).

$$\begin{aligned} 10x_1 + 2x_2 - x_3 &= 27 \\ -3x_1 - 6x_2 + 2x_3 &= -61.5 \\ x_1 + x_2 + 5x_3 &= -21.5 \end{aligned}$$

Then, multiply the resulting  $L$  and  $U$  matrices to determine that  $A$  is the result.

**10.4** (a) Use  $LU$  factorization to solve the system of equations in [Prob. 10.3](#). Show all the steps in the computation. (b) Also, solve the system for an alternate constant vector

$$b^T = [12 \quad 18 \quad -6]$$

**10.5** Solve the following system of equations using  $LU$  factorization with partial pivoting.

$$\begin{aligned} 2x_1 - 6x_2 - x_3 &= -38 \\ -3x_1 - x_2 + 7x_3 &= -34 \\ -8x_1 + x_2 - 2x_3 &= -40 \end{aligned}$$

**10.6** Develop your own Python function, called `lu_factor`, to carry out the  $LU$  factorization of a square matrix without partial pivoting. That is, the function takes one argument, the matrix  $A$ , and returns two results, the  $L$  and  $U$  matrices. Test your function by using it to solve the system in [Prob. 10.3](#). Confirm that your function is working properly by verifying that  $LU = A$  and by using the `linalg.lu` function from SciPy.

**10.7** Confirm the validity of the Cholesky factorization of [Example 10.5](#) by substituting the results of [Eq. \(10.14\)](#) to verify that the product of  $U^T$  and  $U$  yields  $A$ .

## 10.8

(a) Perform a Cholesky factorization of the following symmetric system by hand.

$$\begin{bmatrix} 8 & 20 & 15 \\ 20 & 80 & 50 \\ 15 & 50 & 60 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 50 \\ 250 \\ 100 \end{bmatrix}$$

- (b) Verify your hand calculation with the `cholesky` function from the SciPy `linalg` submodule.
- (c) Employ the results of factorization, the  $U$  matrix, to determine the solution for the given constant vector.

**10.9** Develop your own Python function, called `chol`, to determine the Cholesky factorization of a symmetric matrix without pivoting. The function should accept a symmetric matrix as its argument and return the Cholesky  $U$  matrix. It should check that the input matrix is square and symmetric, and return appropriate error messages if the tests fail. Test your function with the coefficient matrix from [Prob. 10.8](#) and use the `cholesky` function from the SciPy `linalg` submodule to confirm that your function is working properly.

**10.10** Solve the following set of equations via  $LU$  factorization with pivoting. Use the Python `linalg.lu` function from the SciPy module. Determine which row swaps took place from the resulting permutation,  $P$ , matrix.

$$\begin{aligned} -x_1 - 2x_2 + 5x_3 &= -26 \\ 3x_1 - 2x_2 + x_3 &= -10 \\ 2x_1 + 6x_2 - 4x_3 &= 44 \end{aligned}$$

### 10.11

- (a) Determine the  $LU$  factorization without pivoting by hand for the following matrix and check your results by validating that  $LU = A$ .

$$\begin{bmatrix} 8 & 2 & 1 \\ 3 & 7 & 2 \\ 2 & 3 & 9 \end{bmatrix}$$

- (b) Employ the result of (a) to compute the determinant.
- (c) Repeat (a) and (b) using Python.

**10.12** Use the following  $LU$  factorization to

- (a) compute the determinant of  $A$ , and
- (b) solve  $Ax = b$  with  $b^T = [-10 \quad 44 \quad -26]$ .

$$A = LU = \begin{bmatrix} 1 & 0 & 0 \\ 0.6667 & 1 & 0 \\ -0.3333 & -0.3636 & 1 \end{bmatrix} \begin{bmatrix} 3 & -2 & 1 \\ 0 & 7.3333 & -4.6667 \\ 0 & 0 & 3.6364 \end{bmatrix}$$

**10.13** Use Cholesky factorization to determine  $U$  so that

$$A = U^T U = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$$

Confirm that the above product does indeed reconstruct the  $A$  matrix.

**10.14** Compute the Cholesky factorization of

$$A = \begin{bmatrix} 9 & 0 & 0 \\ 0 & 25 & 0 \\ 0 & 0 & 4 \end{bmatrix}$$

Do your results make sense in terms of [Eqs. \(10.15\)](#) and [\(10.16\)](#)?

# Matrix Inverse and Condition

## CHAPTER OBJECTIVES

The primary objective of this chapter is to show how to compute the matrix inverse and to illustrate how it can be used to analyze complex linear systems that occur in engineering and science. In addition, a method to assess a matrix solution's sensitivity to roundoff error is described. Specific objectives and topics covered are

- Knowing how to determine the matrix inverse in an efficient manner based on  $LU$  factorization.
- Understanding how the matrix inverse can be used to assess stimulus-response characteristics of engineering systems.
- Understanding the meaning of matrix and vector norms and how they are computed.
- Knowing how to use norms to compute the matrix condition number.
- Understanding how the magnitude of the condition number can be used to estimate the precision of solutions of linear algebraic equations.

# 11.1 THE MATRIX INVERSE

In our discussion of matrix operations ([Sec. 8.1.2](#)), we introduced the notion that, if a matrix  $[A]$  is square, there may be another matrix  $[A]^{-1}$ , called the inverse of  $[A]$ , for which

$$[A][A]^{-1} = [A]^{-1}[A] = [I] \quad (11.1)$$

Now, we will focus on how the inverse can be computed numerically. Then, we will explore how it can be used for engineering analysis.

## 11.1.1 Calculating the Inverse

The inverse can be computed in a column-by-column fashion by generating solutions with unit vectors as the right-hand-side constants. For example, if the right-hand-side constant has a 1 in the first position and zeros elsewhere,

$$\{b\} = \begin{Bmatrix} 1 \\ 0 \\ 0 \end{Bmatrix} \quad (11.2)$$

the resulting solution will be the first column of the matrix inverse. Similarly, if a unit vector with a 1 at the second row is used

$$\{b\} = \begin{Bmatrix} 0 \\ 1 \\ 0 \end{Bmatrix} \quad (11.3)$$

the result will be the second column of the matrix inverse.

The best way to implement such a calculation is with  $LU$  factorization. Recall that one of the great strengths of  $LU$  factorization is that it provides a very efficient means to evaluate multiple right-hand-side vectors. Thus, it is ideal for evaluating the multiple unit vectors needed to compute the inverse.

### EXAMPLE 11.1

#### Matrix Inversion

#### Problem Statement

Employ  $LU$  factorization to determine the matrix inverse for the system from [Example 10.1](#):

$$[A] = \begin{bmatrix} 3 & -0.1 & -0.2 \\ 0.1 & 7 & -0.3 \\ 0.3 & -0.2 & 10 \end{bmatrix}$$

Recall that the factorization resulted in the following lower and upper triangular matrices:

$$[U] = \begin{bmatrix} 3 & -0.1 & -0.2 \\ 0 & 7.00333 & -0.293333 \\ 0 & 0 & 10.0120 \end{bmatrix} \quad [L] = \begin{bmatrix} 1 & 0 & 0 \\ 0.0333333 & 1 & 0 \\ 0.100000 & -0.0271300 & 1 \end{bmatrix}$$

## Solution

The first column of the matrix inverse can be determined by performing the forward-substitution solution procedure with a unit vector (with 1 in the first row) as the right-hand-side vector. Thus, the lower triangular system can be set up as [recall [Eq. \(10.8\)](#)]

$$\begin{bmatrix} 1 & 0 & 0 \\ 0.0333333 & 1 & 0 \\ 0.100000 & -0.0271300 & 1 \end{bmatrix} \begin{Bmatrix} d_1 \\ d_2 \\ d_3 \end{Bmatrix} = \begin{Bmatrix} 1 \\ 0 \\ 0 \end{Bmatrix}$$

and solved with forward substitution for  $\{d\}^T = [1 - 0.3333 - 0.1009]$ . This vector can then be used as the right-hand side of the upper triangular system [recall [Eq. \(10.3\)](#)]

$$\begin{bmatrix} 3 & -0.1 & -0.2 \\ 0 & 7.00333 & -0.293333 \\ 0 & 0 & 10.0120 \end{bmatrix} \begin{Bmatrix} x_1 \\ x_2 \\ x_3 \end{Bmatrix} = \begin{Bmatrix} 1 \\ -0.03333 \\ -0.1009 \end{Bmatrix}$$

which can be solved by back substitution for  $\{x\}^T = [0.33249 - 0.00518 - 0.01008]$ , which is the first column of the matrix inverse:

$$[A]^{-1} = \begin{bmatrix} 0.33249 & 0 & 0 \\ -0.00518 & 0 & 0 \\ -0.01008 & 0 & 0 \end{bmatrix}$$

To determine the second column, [Eq. \(10.8\)](#) is formulated as

$$\begin{bmatrix} 1 & 0 & 0 \\ 0.0333333 & 1 & 0 \\ 0.100000 & -0.0271300 & 1 \end{bmatrix} \begin{Bmatrix} d_1 \\ d_2 \\ d_3 \end{Bmatrix} = \begin{Bmatrix} 0 \\ 1 \\ 0 \end{Bmatrix}$$

This can be solved for  $\{d\}$ , and the results are used with [Eq. \(10.3\)](#) to determine  $\{x\}^T = [0.004944 \ 0.142903 \ 0.00271]$ , which is the second column of the matrix inverse:

$$[A]^{-1} = \begin{bmatrix} 0.33249 & 0.004944 & 0 \\ -0.00518 & 0.142903 & 0 \\ -0.01008 & 0.002710 & 0 \end{bmatrix}$$

Finally, the same procedures can be implemented with  $\{b\}^T = [0 \ 0 \ 1]$  to solve for  $\{x\}^T = [0.006798 \ 0.004183 \ 0.09988]$ , which is the final column of the matrix inverse:

$$[A]^{-1} = \begin{bmatrix} 0.33249 & 0.004944 & 0.006798 \\ -0.00518 & 0.142903 & 0.004183 \\ -0.01008 & 0.002710 & 0.099880 \end{bmatrix}$$

The validity of this result can be checked by verifying that  $[A][A]^{-1} = [I]$ .

## 11.1.2 Stimulus-Response Computations

As discussed in [PT 3.1](#), many of the linear systems of equations arising in engineering and science are derived from conservation laws. The mathematical expression of these laws is some form of balance equation to ensure that a particular property—mass, force, heat, momentum, electrostatic potential—is conserved. For a force balance on a structure, the properties might be horizontal or vertical components of the forces acting on each node of the structure. For a mass balance, the properties might be the mass in each reactor of a chemical process. Other fields of engineering and science would yield similar examples.

A single balance equation can be written for each part of the system, resulting in a set of equations defining the behavior of the property for the entire system. These equations are interrelated, or coupled, in that each equation may include one or more of the variables from the other equations. For many cases, these systems are linear and, therefore, of the exact form dealt with in this chapter:

$$[A] \{x\} = \{b\} \quad (11.4)$$

Now, for balance equations, the terms of [Eq. \(11.4\)](#) have a definite physical interpretation. For example, the elements of  $\{x\}$  are the levels of the property being balanced for each part of the system. In a force balance of a structure, they represent the horizontal and vertical forces in each member. For the mass balance, they are the mass of chemical in each reactor. In either case, they represent the system's *state* or *response*, which we are trying to determine.

The right-hand-side vector  $\{b\}$  contains those elements of the balance that are independent of behavior of the system—that is, they are constants. In many problems, they represent the *forcing functions* or *external stimuli* that drive the system.

Finally, the matrix of coefficients  $[A]$  usually contains the *parameters* that express how the parts of the system *interact* or are coupled. Consequently, [Eq. \(11.4\)](#) might be reexpressed as

$$[\text{Interactions}] \{\text{response}\} = \{\text{stimuli}\}$$



As we know from previous chapters, there are a variety of ways to solve [Eq. \(11.4\)](#). However, using the matrix inverse yields a particularly interesting result. The formal solution can be expressed as

$$\{x\} = [A]^{-1}\{b\}$$

or (recalling our definition of matrix multiplication from [Sec. 8.1.2](#))

$$\begin{aligned}x_1 &= a_{11}^{-1}b_1 + a_{12}^{-1}b_2 + a_{13}^{-1}b_3 \\x_2 &= a_{21}^{-1}b_1 + a_{22}^{-1}b_2 + a_{23}^{-1}b_3 \\x_3 &= a_{31}^{-1}b_1 + a_{32}^{-1}b_2 + a_{33}^{-1}b_3\end{aligned}$$

Thus, we find that the inverted matrix itself, aside from providing a solution, has extremely useful properties. That is, each of its elements represents the response of a single part of the system to a unit stimulus of any other part of the system.

Notice that these formulations are linear and, therefore, superposition and proportionality hold.

*Superposition* means that, if a system is subject to several different stimuli (the  $b$ 's), the responses can be computed individually and the results summed to obtain a total response. *Proportionality* means that multiplying the stimuli by a quantity results in the response to those stimuli being multiplied by the same quantity. Thus, the coefficient  $a_{11}^{-1}$  is a proportionality constant that gives the value of  $x_1$  due to a unit level of  $b_1$ . This result is independent of the effects of  $b_2$  and  $b_3$  on  $x_1$ , which are reflected in the coefficients  $a_{12}^{-1}$  and  $a_{13}^{-1}$ , respectively. Therefore, we can draw the general conclusion that the element  $a_{ij}^{-1}$  of the inverted matrix represents the value of  $x_i$  due to a unit quantity of  $b_j$ .

Using the example of the structure, element  $a_{ij}^{-1}$  of the matrix inverse would represent the force in member  $i$  due to a unit external force at node  $j$ . Even for small systems, such behavior of individual stimulus-response interactions would not be intuitively obvious. As such, the matrix inverse provides a powerful technique for understanding the interrelationships of component parts of complicated systems.

## EXAMPLE 11.2

### Analyzing the Bungee Jumper Problem

#### Problem Statement

At the beginning of [Chap. 8](#), we set up a problem involving three individuals suspended vertically connected by bungee cords. We derived a system of linear algebraic equations based on force balances on each jumper.

$$\begin{bmatrix} 150 & -100 & 0 \\ -100 & 150 & -50 \\ 0 & -50 & 50 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 588.6 \\ 686.7 \\ 784.8 \end{bmatrix}$$

In  **Example 8.2**, we used Python to solve this system for the vertical positions of the jumpers (the  $x$ 's). In the present example, use Python to compute the matrix inverse and interpret what it means.

## Solution

Here is the Python code and result for computing the inverse matrix.

```
import numpy as np

A = np.matrix(' 150. -100. 0. ; -100. 150. -50. ; 0. -50. 50. ')
Ainv = np.linalg.inv(A)
print(Ainv)

[[0.02 0.02 0.02]
 [0.02 0.03 0.03]
 [0.02 0.03 0.05]]
```

Each element of the inverse,  $ainv_{ij}$ , represents the vertical change in position (in meters) of jumper  $i$  in response to a unit change in force (in Newtons) applied to jumper  $j$ .

First, observe that the numbers in the first column ( $j = 1$ ) indicate that the position of all the three jumpers would increase by 0.02 m (20 cm) if the force on the first jumper were increased by 1 N. This makes sense, as the additional force would only elongate the first cord by that amount.

In contrast, the numbers in the second column ( $j = 2$ ) indicate that applying a force of 1 N to the second jumper would move the first jumper down by 0.2 m, but the second and third jumpers by 0.3 m. The 0.02 m elongation of the first jumper makes sense because the first cord is subject to an extra 1 N force regardless of whether the force is applied to the first or second jumper. However, for the second jumper, the elongation is now 0.03 m because, along with the first cord, the second cord also elongates due to the additional force. And, of course, the third jumper shows the identical translation as the second jumper as there is no additional force on the third cord that connects them.

As expected, the third column ( $j = 3$ ) indicates that applying a force of 1 N to the third jumper results in the first and second jumpers moving the same distances as occurred when the force was applied to the second jumper. However, now because of the additional elongation of the third cord, the third jumper is moved farther downward.

Superposition and proportionality can be demonstrated by using the inverse to determine how much farther the third jumper would move if additional forces of 10, 50, and 20 N were applied to the first, second, and third jumpers, respectively. This can be done simply by using the appropriate elements of the third row of the inverse to compute,

$$\Delta x_3 = kinv_{31} \Delta F_1 + kinv_{32} \Delta F_2 + kinv_{33} \Delta F_3 = 0.02 \cdot 10 + 0.03 \cdot 50 + 0.05 \cdot 20 \cong 2.7 \text{ m}$$

# 11.2 ERROR ANALYSIS AND SYSTEM CONDITION

---

Aside from its engineering and scientific applications, the inverse also provides a means to discern whether systems are ill-conditioned. Three direct methods can be devised for this purpose:

1. Scale the matrix of coefficients  $[A]$  so that the largest element in each row is 1. Invert the scaled matrix and if there are elements of  $[A]^{-1}$  that are several orders of magnitude greater than one, it is likely that the system is ill-conditioned.
2. Multiply the inverse by the original coefficient matrix and assess whether the result is close to the identity matrix. If not, it indicates ill-conditioning.
3. Invert the inverted matrix and assess whether the result is sufficiently close to the original coefficient matrix. If not, it again indicates that the system is ill-conditioned.

---

Page 257

Although these methods can indicate ill-conditioning, it would be preferable to obtain a single number that could serve as an indicator of the problem. Attempts to formulate such a matrix condition number are based on the mathematical concept of the norm.

## 11.2.1 Vector and Matrix Norms

A *norm* is a real-valued function that provides a measure of the size or “length” of multi-component mathematical entities such as vectors and matrices.

A simple example is a vector in three-dimensional Euclidean space ( Fig. 11.1) that can be represented as

$$[F] = [a \ b \ c]$$

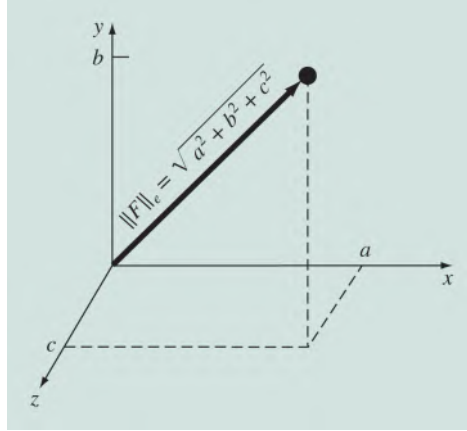
where  $a$ ,  $b$ , and  $c$  are the distances along the  $x$ ,  $y$ , and  $z$  axes, respectively. The length of this vector—that is, the distance from the coordinate  $(0, 0, 0)$  to  $(a, b, c)$ —can be simply computed as

$$\|F\|_e = \sqrt{a^2 + b^2 + c^2}$$

where the nomenclature  $\|F\|_e$  indicates that this length is referred to as the *Euclidean norm* of  $[F]$ .

---

**FIGURE 11.1** Graphical depiction of a vector in Euclidean space.



Similarly, for an  $n$ -dimensional vector  $[X] = [x_1 \ x_2 \ \cdots \ x_n]$ , a Euclidean norm would be computed as

$$\|X\|_e = \sqrt{\sum_{i=1}^n x_i^2}$$

The concept can be extended further to a matrix  $[A]$ , as in

$$\|A\|_f = \sqrt{\sum_{i=1}^n \sum_{j=1}^n a_{i,j}^2} \quad (11.5)$$

which is given a special name—the *Frobenius norm*. As with the other vector norms, it provides a single value to quantify the “size” of  $[A]$ .

It should be noted that there are alternatives to the Euclidean and Frobenius norms. For vectors, there are alternatives called  $p$  norms that can be represented generally by

$$\|X\|_p = \left( \sum_{i=1}^n |x_i|^p \right)^{1/p}$$

We can see that the Euclidean norm and the 2 norm,  $\|X\|_2$ , are identical for vectors.

Other important examples are ( $p = 1$ )

$$\|X\|_1 = \sum_{i=1}^n |x_i|$$

which represents the norm as the sum of the absolute values of the elements. Another is the maximum-magnitude or uniform-vector norm ( $p = \infty$ ),

$$\|X\|_{\infty} = \max_{1 \leq i \leq n} |x_i|$$

which defines the norm as the element with the largest absolute value.

Using a similar approach, norms can be developed for matrices. For example,

$$\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}|$$

That is, a summation of the absolute values of the coefficients is performed for each column, and the largest of these summations is taken as the norm. This is called the *column-sum norm*.

A similar determination can be made for the rows, resulting in a uniform-matrix or *row-sum norm*:

$$\|A\|_{\infty} = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{ij}|$$

It should be noted that, in contrast to vectors, the 2 norm and the Frobenius norm for a matrix are not the same. Whereas the Frobenius norm  $\|A\|_f$  can be easily determined by [Eq. \(11.5\)](#), the matrix 2 norm  $\|A\|_2$  is calculated as

$$\|A\|_2 = (\mu_{\max})^{1/2}$$

where  $\mu_{\max}$  is the largest eigenvalue of  $[A]^T[A]$ . In [Chap. 13](#), we will learn more about eigenvalues. For the time being, the important point is that the  $\|A\|_2$ , or *spectral norm*, is the minimum norm and, therefore, provides the tightest measure of size (Ortega, 1972).

## 11.2.2 Matrix Condition Number

Now that we have introduced the concept of the norm, we can use it to define

$$\text{Cond}[A] = \|A\| \|A^{-1}\|$$

where  $\text{Cond}[A]$  is called the *matrix condition number*. Note that for a matrix  $[A]$ , this number will be greater than or equal to 1. It can be shown (Ralston and Rabinowitz, 1978; Gerald and Wheatley, 1989) that

$$\frac{\|\Delta X\|}{\|X\|} \leq \text{Cond}[A] \frac{\|\Delta A\|}{\|A\|}$$

That is, the relative error of the norm of the computed solution can be as large as the relative error of the norm of the coefficients of  $[A]$  multiplied by the condition number. For example, if the coefficients of  $[A]$  are known to  $t$ -digit precision (i.e., rounding errors are on the order of  $10^{-t}$ ) and  $\text{Cond}[A] = 10^c$ , the solution  $[X]$  may be valid to only  $t - c$  digits (rounding errors  $\approx 10^{c-t}$ ).

## EXAMPLE 11.3

### Matrix Condition Evaluation

#### Problem Statement

The Hilbert matrix, which is notoriously ill-conditioned, can be represented generally as

$$\begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \cdots & \frac{1}{n} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \cdots & \frac{1}{n+1} \\ \vdots & \vdots & \vdots & & \vdots \\ \frac{1}{n} & \frac{1}{n+1} & \frac{1}{n+2} & \cdots & \frac{1}{2n-1} \end{bmatrix}$$

Use the row-sum norm to estimate the matrix condition number for the  $3 \times 3$  Hilbert matrix:

Page 259

$$[A] = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \end{bmatrix}$$

#### Solution

First, the matrix can be normalized so that the maximum element in each row is 1:

$$[A] = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ 1 & \frac{2}{3} & \frac{1}{2} \\ 1 & \frac{3}{4} & \frac{3}{5} \end{bmatrix}$$

Summing each of the rows gives 1.833, 2.1667, and 2.35. Thus, the third row has the largest sum and the row-sum norm is

$$\|A\|_{\infty} = 1 + \frac{3}{4} + \frac{3}{5} = 2.35$$

The inverse of the scaled matrix can be computed as

$$[A]^{-1} = \begin{bmatrix} 9 & -18 & 10 \\ -36 & 96 & -60 \\ 30 & -90 & 60 \end{bmatrix}$$

Note that the elements of this matrix are larger than the original matrix. This is also reflected in its row-sum norm, which is computed as

$$\|A^{-1}\|_{\infty} = |-36| + |96| + |-60| = 192$$

Thus, the condition number can be calculated as

$$\text{Cond}[A] = 2.35(192) = 451.2$$

The fact that the condition number is much greater than unity suggests that the system is ill-conditioned. The extent of the ill-conditioning can be quantified by calculating  $c = \log 451.2 = 2.65$ . Hence, the last three significant digits of the solution could exhibit rounding errors. Note that such estimates almost always overpredict the actual error. However, they are useful in alerting you to the possibility that roundoff errors may be significant.

## 11.2.3 Norms and Condition Number with Python

The Python NumPy module has two functions that compute norms and condition numbers, `linalg.norm` and `linalg.cond`, respectively. The syntax for their use is

`norm(x)`      or      `norm(x,ord)`

and

`cond(x)`      or      `cond(x,p)`

In each case, `x` can be a matrix or a vector. The extra argument, either `ord` or `p` can be used to specify the type of norm. The default is the Frobenius norm for matrices ( 'fro' ) or the 2-norm for vectors. Also available are 1, 2, and `np.inf`, plus others.

### EXAMPLE 11.4

#### Matrix Condition Evaluation with Python

##### Problem Statement

Use Python to compute both the norms and condition numbers for the scaled Hilbert matrix previously analyzed in [Example 11.3](#):

$$[A] = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ 1 & \frac{2}{3} & \frac{1}{2} \\ 1 & \frac{3}{4} & \frac{3}{5} \end{bmatrix}$$

- (a) As in [Example 11.3](#), first compute the row-sum versions (`np.inf` option).
- (b) Also, compute the Frobenius and the 2-norm condition numbers.

## Solution

(a)

```
import numpy as np

A = np.matrix([ [1., 1/2, 1/3] , [1., 2/3, 1/2] , [1., 3/4, 3/5] ])
print('inf norm = ', np.linalg.norm(A, np.inf))
print('cond no. = ', np.linalg.cond(A, np.inf))

inf norm = 2.35
cond no. = 451.20000000000025
```

These results correspond to those that were calculated by hand in [Example 11.3](#).

(b)

```
print('Frobenius norm = ', np.linalg.norm(A, 'fro'))
print('cond no. = ', np.linalg.cond(A, 'fro'))
print('2-norm = ', np.linalg.norm(A, 2))
print('cond no. = ', np.linalg.cond(A, 2))

Frobenius norm = 2.231155654712498
cond no. = 368.08659043159537
2-norm = 2.221599755938623
cond no. = 366.35032323670333
```

And note that the following code gives the same results as specifying 'fro', since that is the default.

```
print('Frobenius norm = ', np.linalg.norm(A))
print('cond no. = ', np.linalg.cond(A))

Frobenius norm = 2.231155654712498
cond no. = 366.35032323670333
```



## 11.3 CASE STUDY

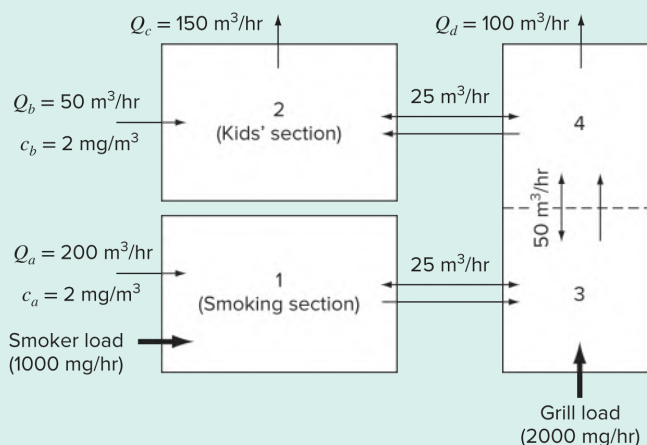
### INDOOR AIR POLLUTION

#### Background

As the name implies, indoor air pollution deals with air contamination in enclosed spaces such as homes, offices, and commercial spaces. Suppose that you are studying the ventilation system for Bubba's Gas 'N Guzzle, a truck stop restaurant located adjacent to an eight-lane freeway.

As depicted in [Fig. 11.2](#), the restaurant serving area consists of a room for smokers, another for families with kids, and one elongated room. Room 1 and Section 3 have sources of carbon monoxide from smokers and a faulty grill, respectively. In addition, rooms 1 and 2 gain carbon monoxide from air intakes that unfortunately are positioned alongside the freeway.

**FIGURE 11.2** Plan view of rooms in a restaurant. The one-way arrows represent volumetric air flows, whereas the two-way arrows indicate dispersive mixing. The smoker and grill loads add carbon monoxide mass to the system but negligible air flow.



#### Problem Statement

Write steady-state mass balances for each room and solve the resulting linear algebraic equations for the concentration of carbon monoxide in each room. In addition, generate the matrix inverse, and use it to analyze how the various sources affect the family/kids' room. For example, determine what percentage of the carbon monoxide there is in that room due to (1) the smokers, (2) the grill, and (3) the intake vents. Also, compute the improvement in the concentration in the family/kids' section if the carbon monoxide load is decreased by banning smoking and renovating the grill. Finally, analyze how the concentration in the kids' area would change if a barrier is constructed to decrease the exchange between areas 2 and 4 to  $5 \text{ m}^3/\text{hr}$ .

## Solution

Steady-state mass balances can be written for each room. For example, the balance for the smoking section (room 1) is

$$0 = W_{\text{smoker}} + Q_a c_a - Q_a c_1 + E_{13}(c_3 - c_1)$$

Similar balances can be written for the other three rooms:

$$\begin{aligned} 0 &= Q_b c_b + (Q_a - Q_d) c_4 - Q_c c_2 + E_{24}(c_4 - c_2) \\ 0 &= W_{\text{grill}} + Q_a c_1 + E_{13}(c_1 - c_3) + E_{34}(c_4 - c_3) - Q_a c_3 \\ 0 &= Q_a c_3 + E_{34}(c_3 - c_4) + E_{24}(c_2 - c_4) - Q_a c_4 \end{aligned}$$

The set of four linear equations can be rearranged in matrix form as follows:

$$\begin{bmatrix} Q_a + E_{13} & 0 & -E_{13} & 0 \\ 0 & Q_c + E_{24} & 0 & -(Q_a + E_{24}) \\ -(Q_a + E_{13}) & 0 & E_{13} + E_{34} + Q_a & -E_{34} \\ 0 & -E_{24} & -(Q_a + E_{34}) & E_{34} + E_{24} + Q_a \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{bmatrix} = \begin{bmatrix} W_{\text{smoker}} + Q_a c_a \\ Q_b c_b \\ W_{\text{grill}} \\ 0 \end{bmatrix}$$

Substituting the parameter values from  Fig. 11.2, we have

$$\begin{bmatrix} 225 & 0 & -25 & 0 \\ 0 & 175 & 0 & -125 \\ -225 & 0 & 275 & -50 \\ 0 & -25 & -250 & 275 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{bmatrix} = \begin{bmatrix} 1400 \\ 100 \\ 2000 \\ 0 \end{bmatrix}$$

Python can be used to generate the solution. First, we can compute the inverse.

```
import numpy as np

Qa = 200 ; Qb = 50 ; Qc = 150 ; Qd = 100 # m3/hr
E13 = 25 ; E24 = 25 ; E34 = 50 # m3/hr
Wsm = 1000 ; Wgr = 2000 # m3/hr
ca = 2 ; cb = 2 # mg/m3

A = np.matrix([[Qa+E13, 0, -E13, 0], [0, Qc+E24, 0, -(Qa-Qd+E24)],
               [-(Qa+E13), 0, E13+E34+Qa, -E34],
               [0, -E24, -(Qa+E34), E34+E24+Qa]])
b = np.matrix([[Wsm+Qa*ca],
               [Qb*cb], [Wgr], [0]])

Ainv = np.linalg.inv(A)
np.set_printoptions(precision=4)
print('Inverse of A is\n',Ainv)

Inverse of A is
[[4.9962e-03  1.5326e-05  5.5172e-04  1.0728e-04]
 [3.4483e-03  6.2069e-03  3.4483e-03  3.4483e-03]
 [4.9655e-03  1.3793e-04  4.9655e-03  9.6552e-04]
 [4.8276e-03  6.8966e-04  4.8276e-03  4.8276e-03]]
```

You will note that we used the `set_printoptions` function to control the number of decimal fraction digits displayed in the output.

The solution can then be generated with

```
c = Ainv*b
print('Solution is\n',c)

Solution is
[[ 8.0996]
 [12.3448]]
```

```
[16.8966]
[16.4828]]
```

We observe a surprising result. The smoking room has the lowest CO levels. The highest concentrations occur in rooms 3 and 4 and the family/kids' room has more than twice the concentration of the smoking room! These results take place because (a) carbon monoxide is conservative and (b) the only air exhausts are out of sections 2 and 4 ( $Q_c$  and  $Q_d$ ). Room 3 is particularly bad because, not only does it get the load from the faulty grill, but it also receives the effluent from room 1.

Although the foregoing is interesting, the real power of linear systems comes from using the elements of the matrix inverse to understand how the parts of the system interact. For example, the matrix inverse can be used to determine the percentage of the CO in the family/kids' section due to each source:

The smoker source:

$$c_{2,\text{smoker}} = \text{ainv}_{21} W_{\text{smoker}} \cong 3.4483$$
$$\%_{\text{smoker}} \cong \frac{3.4483}{12.345} 100 \cong 28\%$$

The grill source:

$$c_{2,\text{grill}} = \text{ainv}_{23} W_{\text{grill}} \cong 6.897$$
$$\%_{\text{smoker}} \cong \frac{6.897}{12.345} 100 \cong 56\%$$

The intakes by difference from 100%:

$$\%_{\text{intakes}} = 100 - 28 - 56 = 16\%$$

Clearly, the faulty grill is the most significant source.

The inverse can also be employed to determine the impact of proposed remedies such as banning smoking and fixing the grill. Because the model is linear, superposition holds, and the results can be determined individually and then summed.

$$\Delta c_2 = \text{ainv}_{21} W_{\text{smoker}} + \text{ainv}_{23} W_{\text{grill}} \cong -3.45 - 6.90 = -10.35 \text{ mg/m}^3$$

Implementing both remedies would bring the family/kids' room concentration down by  $10.35 \text{ mg/m}^3$  with the resulting level being  $2 \text{ mg/m}^3$ . This makes sense because, in the absence of the smoker and grill loads, the only sources are the air intakes, which are at  $2 \text{ mg/m}^3$ .

Because all the foregoing calculations involved changing forcing functions, it was not necessary to recompute the solution. However, if the mixing between the kids' area and room 4 is decreased from 25 to  $5 \text{ m}^3/\text{hr}$ , then the coefficient matrix would change, and we would have a new solution.

```
import numpy as np
Qa = 200 ; Qb = 50 ; Qc = 150 ; Qd = 100 # m3/hr
E13 = 25 ; E24 = 5 ; E34 = 50 # m3/hr
Wsm = 1000 ; Wgr = 2000 # m3/hr
ca = 2 ; cb = 2 # mg/m3
A = np.matrix([[Qa+E13, 0, -E13, 0],[0, Qc+E24, 0, -(Qa-Qd+E24)],
               [-(Qa+E13), 0, E13+E34+Qa, -E34],
               [0, -E24, -(Qa+E34), E34+E24+Qa]])
b = np.matrix([[Wsm+Qa*ca], [Qb*cb], [Wgr], [0]])
Ainv = np.linalg.inv(A)
np.set_printoptions(precision=4)
print('Inverse of A is\n',Ainv)
```

```
c = Ainv*b
print('Solution is\n',c)

Solution is
[[ 8.1084]
 [12.08 ]
 [16.976 ]
 [16.88 ]]
```

Therefore, this remedy would only have a small effect on the concentration in the family/kids' room, about  $0.27 \text{ mg/m}^3$ .

Note that, if our only interest were solving the set of equations, we would have used the `np.linalg.solve` function, which is more efficient than computing the inverse and multiplying it with the constant vector. But, since we were interested in studying the effects of forcing functions, having the inverse matrix in hand is of value.

# PROBLEMS

---

**11.1** Determine the matrix inverse of the following system:

$$\begin{aligned}10x_1 + 2x_2 - x_3 &= 27 \\ -3x_1 - 6x_2 + 2x_3 &= -61.5 \\ x_1 + x_2 + 5x_3 &= -21.5\end{aligned}$$

Check your results by verifying that  $A^{-1}A = I$ . Do not use a pivoting strategy.

**11.2** Determine the matrix inverse for the following system:

$$\begin{aligned}-8x_1 + x_2 - 2x_3 &= -20 \\ 2x_1 - 6x_2 - x_3 &= -38 \\ -3x_1 - x_2 + 7x_3 &= -34\end{aligned}$$

**11.3** The following system of equations is designed to determine the concentrations ( $c$ 's in  $\text{g}/\text{m}^3$ ) in a series of coupled, well-mixed tanks as a function of mass input to each tank. The right-hand sides of the equations below represent these inputs in  $\text{g}/\text{day}$ .

$$\begin{aligned}15c_1 - 3c_2 - c_3 &= 4000 \\ -3c_1 + 18c_2 - 6c_3 &= 1200 \\ -4c_1 - c_2 + 12c_3 &= 2350\end{aligned}$$

- (a) Determine the inverse of the coefficient matrix.
- (b) Use that inverse to determine the solution.
- (c) Determine how much the rate of mass input to tank 3 must be increased to induce a  $10 \text{ g}/\text{m}^3$  rise in the concentration in tank 1.
- (d) How much will the concentration in tank 3 be reduced if the rate of mass input to tanks 1 and 2 are reduced by 500 and 250  $\text{g}/\text{day}$ , respectively?

**11.4** Determine the matrix inverse for the system described in  **Prob. 8.9**. Use the inverse to determine the concentration in tank 5 if the inflow concentrations are changed to  $c_{01} = 20$  and  $c_{03} = 50$ .

**11.5** Determine the matrix inverse for the system described in  **Prob. 8.10**. Use the matrix inverse to determine the force in the three members ( $F_1$ ,  $F_2$ , and  $F_3$ ) if the vertical load at node 1 is doubled to  $F_{1,v} = -2000 \text{ N}$  and a horizontal load of  $F_{3,h} = -500 \text{ N}$  is applied to node 3.

**11.6** Determine  $\|A\|_f$ ,  $\|A\|_1$ , and  $\|A\|_\infty$  for

$$A = \begin{bmatrix} 8 & 2 & -10 \\ -9 & 1 & 3 \\ 15 & -1 & 6 \end{bmatrix}$$

Before determining these norms, scale the matrix by making the maximum element in each row, by absolute value, equal to 1 or  $-1$ .

**11.7** Determine the Frobenius and row-sum norms for the systems in  **Probs. 11.2** and  **11.3**.

**11.8** Use Python to determine the spectral condition number of the following matrix. Do not normalize the matrix.

$$\begin{bmatrix} 1 & 4 & 9 & 16 & 25 \\ 4 & 9 & 16 & 25 & 36 \\ 9 & 16 & 25 & 36 & 49 \\ 16 & 25 & 36 & 49 & 64 \\ 25 & 36 & 49 & 64 & 81 \end{bmatrix}$$

**11.9** Aside from the Hilbert matrix, there are other matrices that are inherently ill-conditioned. One such is the *Vandermonde matrix*, which has the following form as a  $3 \times 3$  matrix:

$$\begin{bmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ x_3^2 & x_3 & 1 \end{bmatrix}$$

- (a) Determine the condition number based on the row-sum norm for the case where  $x_1 = 4$ ,  $x_2 = 2$ , and  $x_3 = 7$ .  
 (b) Use Python to compute the spectral and Frobenius condition numbers.

Page 264

**11.10** Use Python to determine the spectral condition number for a 10-dimensional Hilbert matrix. How many digits of precision are expected to be lost due to ill-conditioning? Determine the solution for this system for the case where each element of the constant vector,  $b$ , consists of the summation of the coefficients in each row. In other words, solve for the case where all the unknowns should be exactly one. Compare the resulting errors with those expected based on the condition number.

**11.11** Repeat  **Prob. 11.10**, but for the case of a six-dimensional Vandermonde matrix (see  **Prob. 11.9**) where  $x_1 = 4$ ,  $x_2 = 2$ ,  $x_3 = 7$ ,  $x_4 = 10$ ,  $x_5 = 3$ , and  $x_6 = 5$ .

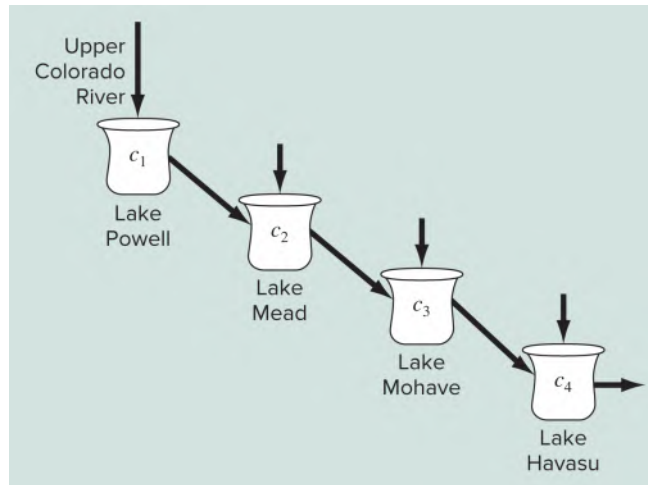
**11.12** The Lower Colorado River consists of a series of four major reservoirs as shown in  **Fig. P11.12**. Mass balances can be written for each reservoir, and the following set of linear algebraic equations results:

$$\begin{bmatrix} 13.422 & 0 & 0 & 0 \\ -13.422 & 12.252 & 0 & 0 \\ 0 & -12.252 & 12.377 & 0 \\ 0 & 0 & -12.377 & 11.797 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{bmatrix} = \begin{bmatrix} 750.5 \\ 300 \\ 102 \\ 30 \end{bmatrix}$$

where the constant vector consists of the loadings of chloride ion to each of the four lakes and the unknown  $c$ 's are the chloride concentrations for Lakes Powell, Mead, Mohave, and Havasu, respectively.

- (a) Use the matrix inverse to solve for the concentrations in each of the four lakes.  
 (b) How much must the loading to Lake Powell be reduced for the concentration in Lake Havasu to be 75?  
 (c) Using the column-sum norm, compute the condition number and how many suspect digits would be generated by solving this system.

**FIGURE P11.12** The Lower Colorado River



**11.13** (a) The matrix below is called a “circular” matrix. Determine the matrix inverse and condition number. (b) Repeat (a) but change  $a_{33}$  slightly to 9.1. What do you conclude about circular matrices?

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

**11.14** Polynomial interpolation consists of determining the unique  $(n - 1)$ th-order polynomial that passes exactly through  $n$  data points. Such polynomials have the general form

$$y = p_1x^{n-1} + p_2x^{n-2} + \cdots + p_{n-1}x + p_n \quad (\text{P11.14})$$

where the  $p$ 's are constant coefficients. A straightforward way to compute the coefficients is to generate  $n$  linear algebraic equations that we can solve simultaneously. The following table describes the density of methanol-water solutions at various %-wt. methanol concentrations at 20°C.

| Density of Methanol-Water Solutions at 20°C |                              |
|---------------------------------------------|------------------------------|
| Wt% Methanol                                | Density (kg/m <sup>3</sup> ) |
| 0                                           | 998.2                        |
| 10                                          | 981.5                        |
| 20                                          | 966.6                        |
| 30                                          | 951.5                        |
| 40                                          | 934.5                        |

Each of these pairs can be substituted into [Eq. \(P11.14\)](#) to yield a system of five linear equations in five unknowns (the  $p$ 's). Use this approach to solve for the coefficients. Use your resulting interpolating polynomial to predict the density at 25°C. In addition, determine the condition number of the coefficient matrix and interpret it.

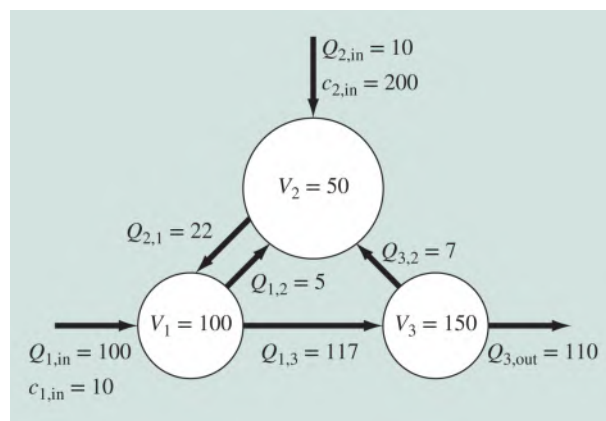
**11.15** A chemical constituent flows between three well-mixed tanks, also called reactors, as depicted in [Fig. P11.15](#). Steady-state mass balances can be written for a substance that reacts with first-order kinetics. For example, the mass balance in reactor 1 is

$$Q_{1,\text{in}}c_{1,\text{in}} - Q_{1,2}c_1 - Q_{1,3}c_1 + Q_{2,1}c_2 - kV_1c_1 = 0$$

where  $Q_{1,\text{in}}$  = volumetric inflow to reactor 1 (L/min),  $c_{1,\text{in}}$  inflow mass concentration to reactor 1 (g/L),  $Q_{i,j}$  = volumetric flow from reactor  $i$  to reactor  $j$  (L/min),  $c_i$  = concentration in reactor  $i$  (g/L),  $k$  = first-order decay rate ( $\text{min}^{-1}$ ), and  $V_i$  = volume of reactor  $i$  (L).

- Write the mass balances for reactors 2 and 3.
- If  $k = 0.1 \text{ min}^{-1}$ , write the mass balances for all three reactors as a system of linear algebraic equations.
- Compute the  $LU$  factorization for this system.
- Use the  $LU$  factorization to compute the matrix inverse.
- Use the matrix inverse to address the following questions:
  - What are the steady-state concentrations in the three reactors?
  - If the inflow in the second reactor is set to zero, what is the resulting reduction in the concentration in reactor 1?
  - If the inflow concentration to reactor 1 is doubled, and the inflow concentration to reactor 2 is halved, what will be the concentration in reactor 3?

**FIGURE P11.15**



**11.16** As described in [Examples 8.2](#) and [11.2](#), use the matrix inverse to answer the following:



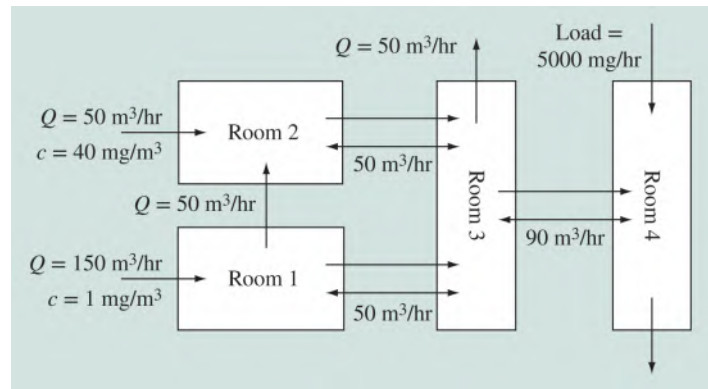
- (a) Determine the change in position of the first jumper if the mass of the third jumper is increased to 100 kg.
- (b) What force must be applied to the third jumper so that the final position of that jumper is 140 m?

**11.17** Determine the matrix inverse for the electrical circuit formulated in [Sec. 8.3](#). Use the inverse to determine the new voltage at node 3 if 200 V is applied at node 6 and the voltage at node 1 is halved.

### 11.18

- (a) Using the same approach as described in [Sec. 11.3](#), derive steady-state mass balances for the room configuration depicted in [Fig. P11.18](#).
- (b) Determine the matrix inverse and use it to calculate the resulting concentrations in the rooms.
- (c) Use the matrix inverse to determine how much the room 4 load must be reduced to maintain a concentration of  $20 \text{ mg/m}^3$  in room 2.

**FIGURE P11.18**

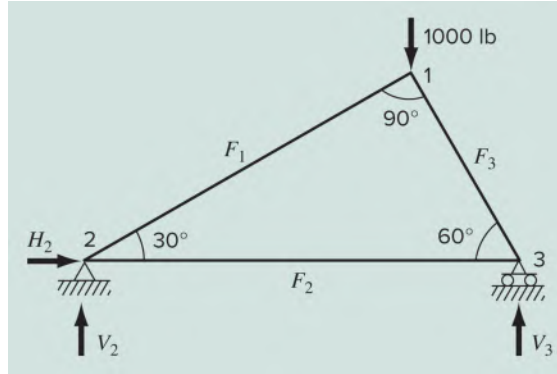


**11.19** Write your own well-structured Python function, named `fro_norm`, to calculate the Frobenius norm of an  $m \times n$  matrix. Use for loops. Test your function with the following matrix.

$$\begin{bmatrix} 5 & 7 & -9 \\ 1 & 8 & 4 \\ 7 & 6 & 2 \end{bmatrix}$$

**11.20** [Figure P11.20](#) shows a statically determinate truss. This type of structure can be described by a system of coupled linear algebraic equations by developing free-body force diagrams at each node in the figure. The sum of forces in the horizontal direction and in the vertical direction must both equal zero, because the system is at rest.

**FIGURE P11.20** Forces on a statically determinate truss.



Therefore, for node 1,

$$\begin{aligned}\sum F_H = 0 &= -F_1 \cos(30^\circ) + F_3 \cos(60^\circ) + F_{1h} \\ \sum F_V = 0 &= -F_1 \sin(30^\circ) - F_3 \sin(60^\circ) + F_{1v}\end{aligned}$$

for node 2,

$$\begin{aligned}\sum F_H = 0 &= F_2 + F_1 \cos(30^\circ) + F_{2h} + H_2 \\ \sum F_V = 0 &= F_1 \sin(30^\circ) + F_{2v} + V_2\end{aligned}$$

and for node 3,

$$\begin{aligned}\sum F_H = 0 &= -F_2 - F_3 \cos(60^\circ) + F_{3h} \\ \sum F_V = 0 &= F_3 \sin(60^\circ) + F_{3v} + V_3\end{aligned}$$

where  $F_{ih}$  is the external horizontal force applied to node  $i$  (where a positive force is to the right), and  $F_{iv}$  is the external vertical force applied to node  $i$  (where a positive force is upward). Thus, in this problem, the 1000-N downward force on node 1 corresponds to  $F_{1v} = -1000$ . For this case, all other  $F_{iv}$ 's and  $F_{ih}$ 's are zero. Note that the directions of the internal forces and reactions are unknown. Proper applications of Newton's laws only require consistent conventions regarding direction. Also note that in this problem the forces in all members are assumed to be in tension and act to pull the adjoining nodes apart. A negative solution therefore corresponds to compression. When numerical values for the external forces are substituted and the trigonometric functions evaluated, this problem reduces to a set of six linear algebraic equations with six unknowns.

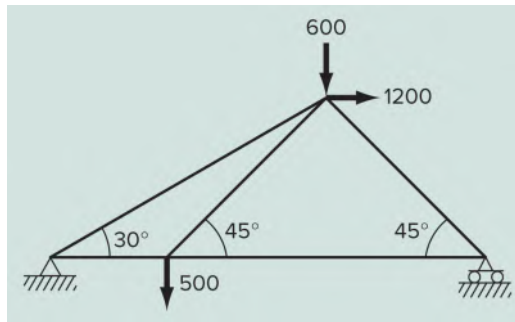
- Solve for the forces and reactions for the case displayed in  Fig. P11.20.
- Determine the system's matrix inverse. What is your interpretation of the zeros in the second row of the inverse?
- Use the elements of the inverse to answer the following questions:
  - If the force at node 1 were reversed, that is, directed upward, compute the impact on  $H_2$  and  $V_2$ .

- (ii) If the force at node 1 were set to zero, and horizontal forces of 1500 N were applied at nodes 1 and 2 ( $F_{1h} = F_{2h} = 1500$ ), what would be the vertical reaction at node 3 ( $V_3$ )?

**11.21** Employing the same approach as in [Prob. 11.20](#),

- (a) Compute the forces and reactions for the members and supports for the truss depicted in [Fig. P11.21](#).  
(b) Compute the matrix inverse.  
(c) Determine the changes in reactions at the two supports if the force at the peak is directed upward.

**FIGURE P11.21**



# Iterative Methods

## CHAPTER OBJECTIVES

The primary objective of this chapter is to acquaint you with iterative methods for solving simultaneous equations. Specific objectives and topics covered are

- Understanding the difference between the Gauss-Seidel and Jacobi methods.
- Knowing how to assess diagonal dominance and knowing what it means.
- Recognizing how relaxation can be used to improve the convergence of iterative methods.
- Understanding how to solve systems of nonlinear equations with successive substitution, Newton-Raphson, and Python's SciPy `optimize.root` function.

Iterative or approximate methods provide an alternative to the elimination methods described to this point. Such approaches are similar to the techniques we developed to obtain the roots of a single equation in [Chaps. 5](#) and [6](#). Those approaches consisted of guessing a value and then using a systematic method to obtain a refined estimate of the root. Because the present part of the book deals with a similar problem—obtaining the values that simultaneously satisfy a set of equations—we might suspect that such approximate methods could be useful in this context. In this chapter, we will present approaches for solving both linear and nonlinear simultaneous equations.

# 12.1 LINEAR SYSTEMS: GAUSS-SEIDEL

The *Gauss-Seidel method* is the most commonly used iterative method for solving linear algebraic equations. Assume that we are given a set of  $n$  equations:

$$[A]\{x\} = \{b\}$$

Suppose that for conciseness we limit ourselves to a  $3 \times 3$  set of equations. If the diagonal elements are all nonzero, the first equation can be solved for  $x_1$ , the second for  $x_2$ , and the third for  $x_3$  to yield

$$x_1^j = \frac{b_1 - a_{12}x_2^{j-1} - a_{13}x_3^{j-1}}{a_{11}} \quad (12.1a)$$

$$x_2^j = \frac{b_2 - a_{21}x_1^j - a_{23}x_3^{j-1}}{a_{22}} \quad (12.1b)$$

$$x_3^j = \frac{b_3 - a_{31}x_1^j - a_{32}x_2^j}{a_{33}} \quad (12.1c)$$

where  $j$  and  $j - 1$  are the present and previous iterations, respectively.

To start the solution process, initial guesses must be made for the  $x$ 's. A simple approach is to assume that they are all zero. These zeros can be substituted into [Eq. \(12.1a\)](#), which can be used to calculate a new value for  $x_1 = b_1/a_{11}$ . Then, we substitute this new value of  $x_1$  along with the previous guess of zero for  $x_3$  into [Eq. \(12.1b\)](#) to compute a new value for  $x_2$ . The process is repeated for [Eq. \(12.1c\)](#) to calculate a new estimate for  $x_3$ . Then, we return to the first equation and repeat the entire procedure until our solution converges closely enough to the true values. Convergence can be checked using the criterion that for all  $i$

$$\varepsilon_{a,i} = \left| \frac{x_i^j - x_i^{j-1}}{x_i^j} \right| \times 100\% \leq \varepsilon_s \quad (12.2)$$

## EXAMPLE 12.1

### Gauss-Seidel Method

## Problem Statement

Use the Gauss-Seidel method to obtain the solution for

$$\begin{aligned}3x_1 - 0.1x_2 - 0.2x_3 &= 7.85 \\0.1x_1 + 7x_2 - 0.3x_3 &= -19.3 \\0.3x_1 - 0.2x_2 + 10x_3 &= 71.4\end{aligned}$$

Note that the solution is  $x_1 = 3$ ,  $x_2 = -2.5$ , and  $x_3 = 7$ .

## Solution

First, solve each of the equations for its unknown on the diagonal:

$$x_1 = \frac{7.85 + 0.1x_2 + 0.2x_3}{3} \quad (\text{E12.1.1})$$

$$x_2 = \frac{-19.3 - 0.1x_1 + 0.3x_3}{7} \quad (\text{E12.1.2})$$

$$x_3 = \frac{71.4 - 0.3x_1 + 0.2x_2}{10} \quad (\text{E12.1.3})$$

By assuming that  $x_2$  and  $x_3$  are zero,  Eq. (E12.1.1) can be used to compute

$$x_1 = \frac{7.85 + 0.1(0) + 0.2(0)}{3} = 2.616667$$

This value, along with the assumed value of  $x_3 = 0$ , can be substituted into  Eq. (E12.1.2) to calculate

$$x_2 = \frac{-19.3 - 0.1(2.616667) + 0.3(0)}{7} = -2.794524$$

The first iteration is completed by substituting the calculated values for  $x_1$  and  $x_2$  into  Eq. (E12.1.3) to yield

$$x_3 = \frac{71.4 - 0.3(2.616667) + 0.2(-2.794524)}{10} = 7.005610$$

For the second iteration, the same process is repeated to compute

$$\begin{aligned}x_1 &= \frac{7.85 + 0.1(-2.794524) + 0.2(7.005610)}{3} = 2.990557 \\x_2 &= \frac{-19.3 - 0.1(2.990557) + 0.3(7.005610)}{7} = -2.499625 \\x_3 &= \frac{71.4 - 0.3(2.990557) + 0.2(-2.499625)}{10} = 7.000291\end{aligned}$$

The method is, therefore, converging on the true solution. Additional iterations could be applied to improve the answers. However, in an actual problem, we would not know the true answer *a priori*. Consequently, [Eq. \(12.2\)](#) provides a means to estimate the error. For example, for  $x_1$ :

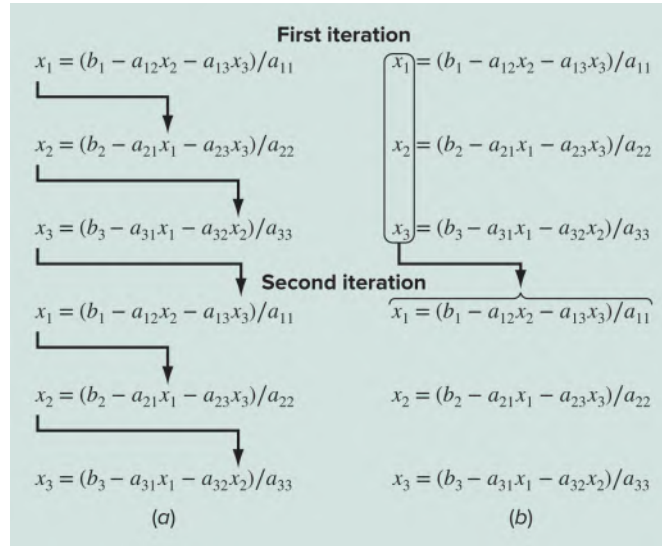
$$\varepsilon_{a,1} = \left| \frac{2.990557 - 2.616667}{2.990557} \right| \times 100\% = 12.5\%$$

For  $x_2$  and  $x_3$ , the error estimates are  $\varepsilon_{a,2} = 11.8\%$  and  $\varepsilon_{a,3} = 0.076\%$ . Note that, as was the case when determining roots of a single equation, formulations such as [Eq. \(12.2\)](#) usually provide a conservative appraisal of convergence. Thus, when they are met, they ensure that the result is known to at least the tolerance specified by  $\varepsilon_s$ .

As each new  $x$  value is computed for the Gauss-Seidel method, it is immediately used in the next equation to determine another  $x$  value. Thus, if the solution is converging, the best available estimates will be employed. An alternative approach, called *Jacobi iteration*, utilizes a somewhat different tactic. Rather than using the latest available  $x$ 's, this technique uses [Eq. \(12.1\)](#) to compute a set of new  $x$ 's on the basis of a set of old  $x$ 's. Thus, as new values are generated, they are not immediately used but rather are retained for the next iteration.

The difference between the Gauss-Seidel method and Jacobi iteration is depicted in [Fig. 12.1](#). Although there are certain cases where the Jacobi method is useful, Gauss-Seidel's utilization of the best available estimates usually makes it the method of preference.

**FIGURE 12.1** Graphical depiction of the difference between (a) the Gauss-Seidel and (b) the Jacobi iterative methods for solving simultaneous linear algebraic equations.



## 12.1.1 Convergence and Diagonal Dominance

Note that the Gauss-Seidel method is similar in spirit to the technique of simple fixed-point iteration that was used in [Sec. 6.1](#) to solve for the roots of a single equation. Recall that simple fixed-point iteration was sometimes nonconvergent. That is, as the iterations progressed, the answer moved farther and farther from the correct result.

Although the Gauss-Seidel method can also diverge, because it is designed for linear systems, its ability to converge is much more predictable than for fixed-point iteration of nonlinear equations. It can be shown that if the following condition holds, Gauss-Seidel will converge:

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| \quad (12.3)$$

That is, the absolute value of the diagonal coefficient in each of the equations must be larger than the sum of the absolute values of the other coefficients in the equation. Such systems are said to be *diagonally dominant*. This criterion is sufficient but not necessary for convergence. That is, although the method may sometimes work if [Eq. \(12.3\)](#) is not met, convergence is guaranteed if the condition is satisfied. Fortunately, many engineering and scientific problems of practical importance fulfill this requirement. Therefore, Gauss-Seidel represents a feasible approach to solve many problems in engineering and science.



## 12.1.2 Python Function: gaussseidel

Before developing an algorithm, let us first recast Gauss-Seidel in a form that is compatible with Python's ability to perform matrix operations. This is done by expressing [Eq. \(12.1\)](#) as

$$\begin{aligned}x_1^{\text{new}} &= \frac{b_1}{a_{11}} - \frac{a_{12}}{a_{11}}x_2^{\text{old}} - \frac{a_{13}}{a_{11}}x_3^{\text{old}} \\x_2^{\text{new}} &= \frac{b_2}{a_{22}} - \frac{a_{21}}{a_{22}}x_1^{\text{new}} - \frac{a_{23}}{a_{22}}x_3^{\text{old}} \\x_3^{\text{new}} &= \frac{b_3}{a_{33}} - \frac{a_{31}}{a_{33}}x_1^{\text{new}} - \frac{a_{32}}{a_{33}}x_2^{\text{new}}\end{aligned}$$

Notice that the method can then be expressed concisely in matrix form as

$$\mathbf{x} = \mathbf{d} - \mathbf{C}\mathbf{x} \quad (12.4)$$

where

$$\mathbf{d} = \begin{bmatrix} b_1/a_{11} \\ b_2/a_{22} \\ b_3/a_{33} \end{bmatrix} \quad \text{and} \quad \mathbf{C} = \begin{bmatrix} 0 & a_{12}/a_{22} & a_{13}/a_{22} \\ a_{21}/a_{22} & 0 & a_{23}/a_{22} \\ a_{31}/a_{33} & a_{32}/a_{33} & 0 \end{bmatrix}$$

A Python script to implement [Eq. \(12.4\)](#) is listed in [Fig. 12.2](#).

**FIGURE 12.2** Python function to implement the Gauss-Seidel method.

```
def GaussSeidel(A,b,es=1.e-7,maxit=50):
    """
    Implements the Gauss-Seidel method
    to solve a set of linear algebraic equations
    without relaxation
    Input:
    A = coefficient matrix
    b = constant vector
    es = stopping criterion (default = 1.e-7)
    maxit = maximum number of iterations (default=50)
    Output:
    x = solution vector
    """
    n,m = np.shape(A)
    if n != m :
        return 'Coefficient matrix must be square'
    C = np.zeros((n,n))
    x = np.zeros((n,1))
    for i in range(n): # set up C matrix with zeros on the diagonal
        for j in range(n):
            if i != j:
                C[i,j] = A[i,j]
```

```

d = np.zeros((n,1))
for i in range(n): # divide C elements by A pivots
    C[i,0:n] = C[i,0:n]/A[i,i]
    d[i] = b[i]/A[i,i]
ea = np.zeros((n,1))
xold = np.zeros((n,1))
for it in range(maxit): # Gauss-Seidel method
    for i in range(n):
        xold[i] = x[i] # save the x's for convergence test
    for i in range(n):
        x[i] = d[i] - C[i,:].dot(x) # update the x's 1-by-1
        if x[i] != 0:
            ea[i] = abs((x[i]-xold[i])/x[i]) # compute change error
    if np.max(ea) < es: # exit for loop if stopping criterion met
        break
    if it == maxit: # check for maximum iteration exit
        return 'maximum iterations reached'
    else:
        return x

```

## 12.1.3 Relaxation

Relaxation represents a slight modification of the Gauss-Seidel method that is designed to enhance convergence. After each new value of  $x$  is computed using [Eq. \(12.1\)](#), that value is modified by a weighted average of the results of the previous and the present iterations:

$$x_i^{\text{new}} = \lambda x_i^{\text{new}} + (1 - \lambda)x_i^{\text{old}} \quad (12.5)$$

where  $\lambda$  is a weighting factor that is assigned a value between 0 and 2. It is typically employed to make a nonconvergent system converge or to hasten convergence by dampening out oscillations.

If  $\lambda = 1$ ,  $(1 - \lambda)$  is equal to 0 and the result is unmodified. However, if  $\lambda$  is set at a value between 0 and 1, the result is a weighted average of the present and the previous results. This type of modification is called *underrelaxation*.

For values of  $\lambda$  from 1 to 2, extra weight is placed on the present value. In this instance, there is an implicit assumption that the new value is moving in the correct direction toward the true solution but at too slow a rate. Thus, the added weight of  $\lambda$  is intended to improve the estimate by pushing it closer to the truth. Hence, this type of modification, which is called *overrelaxation*, is designed to accelerate the convergence of an already convergent system. The approach is also called *successive overrelaxation*, or SOR.

The choice of a proper value for  $\lambda$  is highly problem-specific and is often determined empirically. For a single solution of a set of equations, it is often unnecessary. However, if the system under study is to be solved repeatedly, the efficiency introduced by a wise choice of  $\lambda$  can be extremely important. Good examples are the very large systems of linear algebraic equations that can occur when solving partial differential equations in a variety of engineering and scientific problem contexts.

### EXAMPLE 12.2

## Gauss-Seidel Method with Relaxation

### Problem Statement

Solve the following system with Gauss-Seidel using overrelaxation ( $\lambda = 1.2$ ) and a stopping criterion of  $\varepsilon_s = 10\%$ :

$$\begin{aligned}-3x_1 + 12x_2 &= 9 \\ 10x_1 - 2x_2 &= 8\end{aligned}$$

### Solution

First rearrange the equations so that they are diagonally dominant and solve the first equation for  $x_1$  and the second for  $x_2$ :

$$\begin{aligned}x_1 &= \frac{8 + 2x_2}{10} = 0.8 + 0.2x_2 \\ x_2 &= \frac{9 + 3x_1}{12} = 0.75 + 0.25x_1\end{aligned}$$

First iteration: Using initial guesses of  $x_1 = x_2 = 0$ , we can solve for  $x_1$ :

$$x_1 = 0.8 + 0.2(0) = 0.8$$

Before solving for  $x_2$ , we first apply relaxation to our result for  $x_1$ :

$$x_{1,r} = 1.2(0.8) - 0.2(0) = 0.96$$

We use the subscript  $r$  to indicate that this is the “relaxed” value. This result is then used to compute  $x_2$ :

$$x_2 = 0.75 + 0.25(0.96) = 0.99$$

We then apply relaxation to this result to give

$$x_{2,r} = 1.2(0.99) - 0.2(0) = 1.188$$

At this point, we could compute estimated errors with [Eq. \(12.2\)](#). However, since we started with assumed values of zero, the errors for both variables will be 100%.

Second iteration: Using the same procedure as for the first iteration, the second iteration yields

$$\begin{aligned}x_1 &= 0.8 + 0.2(1.188) = 1.0376 \\x_{1,r} &= 1.2(1.0376) - 0.2(0.96) = 1.05312 \\ \varepsilon_{a,1} &= \left| \frac{1.05312 - 0.96}{1.05312} \right| \times 100\% = 8.84\% \\x_2 &= 0.75 + 0.25(1.05312) = 1.01328 \\x_{2,r} &= 1.2(1.01328) - 0.2(1.188) = 0.978336 \\ \varepsilon_{a,2} &= \left| \frac{0.978336 - 1.188}{0.978336} \right| \times 100\% = 21.43\%\end{aligned}$$

Because we now have nonzero values from the first iteration, we can compute approximate error estimates as each new value is computed. At this point, although the error estimate for the first unknown has fallen below the 10% stopping criterion, the second has not. Hence, we must implement another iteration.

Third iteration:

$$\begin{aligned}x_1 &= 0.8 + 0.2(0.978336) = 0.995667 \\x_{1,r} &= 1.2(0.995667) - 0.2(1.05312) = 0.984177 \\ \varepsilon_{a,1} &= \left| \frac{0.984177 - 1.05312}{0.984177} \right| \times 100\% = 7.01\% \\x_2 &= 0.75 + 0.25(0.984177) = 0.996044 \\x_{2,r} &= 1.2(0.996044) - 0.2(0.978336) = 0.999586 \\ \varepsilon_{a,2} &= \left| \frac{0.999586 - 0.978336}{0.999586} \right| \times 100\% = 2.13\%\end{aligned}$$

At this point, we can terminate the computation because both error estimates have fallen below the 10% stopping criterion. The results at this juncture,  $x_1 = 0.984177$  and  $x_2 = 0.999586$ , are converging on the exact solution of  $x_1 = x_2 = 1$ .

## 12.2 NONLINEAR SYSTEMS

The following is a set of two simultaneous nonlinear equations with two unknowns:

$$x_1^2 + x_1x_2 = 10 \quad (12.6a)$$

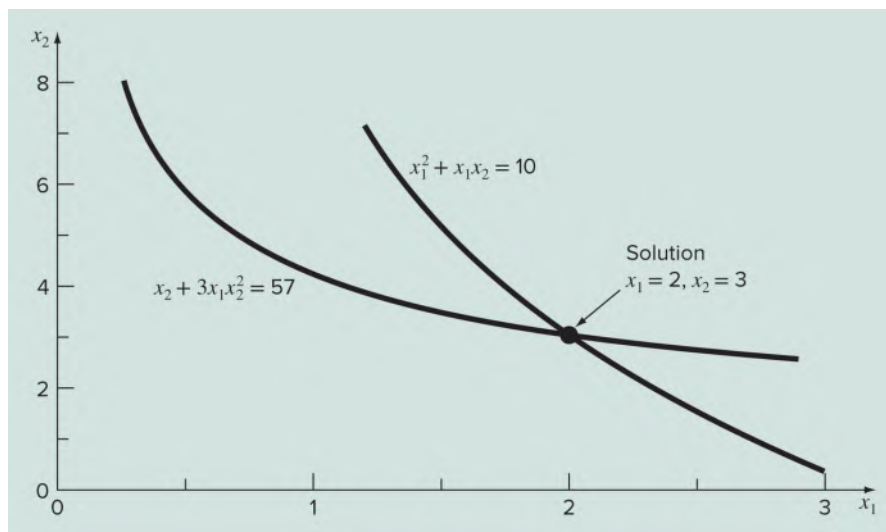
$$x_2 + 3x_1x_2^2 = 57 \quad (12.6b)$$

In contrast to linear systems which plot as straight lines (recall [Fig. 9.1](#)), these equations plot as curves on an  $x_2$  versus  $x_1$  graph. As in [Fig. 12.3](#), the solution is the intersection of the curves.

Just as we did when we determined roots for single nonlinear equations, such systems of equations can be expressed generally as

$$\begin{aligned} f_1(x_1, x_2, \dots, x_n) &= 0 \\ f_2(x_1, x_2, \dots, x_n) &= 0 \\ &\vdots \\ f_n(x_1, x_2, \dots, x_n) &= 0 \end{aligned} \quad (12.7)$$

**FIGURE 12.3** Graphical depiction of the solution of two simultaneous nonlinear equations.



Therefore, the solutions are the values of the  $x$ 's that make the equations equal to zero.

## 12.2.1 Successive Substitution

A simple approach for solving  Eq. (12.7) is to use the same strategy that was employed for fixed-point iteration and the Gauss-Seidel method. That is, each one of the nonlinear equations can be solved for one of the unknowns. These equations can then be implemented iteratively to compute new values which (hopefully) will converge on the solutions. This approach, which is called *successive substitution*, is illustrated in the following example.

### EXAMPLE 12.3

#### Successive Substitution for a Nonlinear System

##### Problem Statement

Use successive substitution to determine the roots of  Eq. (12.6). Note that a correct pair of roots is  $x_1 = 2$  and  $x_2 = 3$ . Initiate the computation with guesses of  $x_1 = 1.5$  and  $x_2 = 3.5$ .

##### Solution

 Equation (12.6a) can be solved for

$$x_1 = \frac{10 - x_1^2}{x_2} \quad (\text{E12.3.1})$$

and  Eq. (12.6b) can be solved for

$$x_2 = 57 - 3x_1x_2^2 \quad (\text{E12.3.2})$$

On the basis of the initial guesses,  Eq. (E12.3.1) can be used to determine a new value of  $x_1$ :

$$x_1 = \frac{10 - (1.5)^2}{3.5} = 2.21429$$

This result and the initial value of  $x_2 = 3.5$  can be substituted into  Eq. (E12.3.2) to determine a new value of  $x_2$ :

$$x_2 = 57 - 3(2.21429)(3.5)^2 = -24.37516$$

Thus, the approach seems to be diverging. This behavior is even more pronounced on the second iteration:

$$x_1 = \frac{10 - (2.21429)^2}{-24.37516} = -0.20910$$

$$x_2 = 57 - 3(-0.20910)(-24.37516)^2 = 429.709$$

Obviously, the approach is deteriorating.

Now, we will repeat the computation but with the original equations set up in a different format. For example, an alternative solution of [Eq. \(12.6a\)](#) is

$$x_1 = \sqrt{10 - x_1 x_2}$$

and of [Eq. \(12.6b\)](#) is

$$x_2 = \sqrt{\frac{57 - x_2}{3x_1}}$$

Now, the results are more satisfactory:

Page 273

$$x_1 = \sqrt{10 - 1.5(3.5)} = 2.17945$$

$$x_2 = \sqrt{\frac{57 - 3.5}{3(2.17945)}} = 2.86051$$

$$x_1 = \sqrt{10 - 2.17945(2.86051)} = 1.94053$$

$$x_2 = \sqrt{\frac{57 - 2.86051}{3(1.94053)}} = 3.04955$$

Thus, the approach is converging on the true values of  $x_1 = 2$  and  $x_2 = 3$ .

The previous example illustrates the most serious shortcoming of successive substitution—that is, convergence often depends on the manner in which the equations are formulated. Additionally, even in those instances where convergence is possible, divergence can occur if the initial guesses are insufficiently close to the true solution. These criteria are so restrictive that fixed-point iteration has limited utility for solving nonlinear systems.

## 12.2.2 Newton-Raphson

Just as fixed-point iteration can be used to solve systems of nonlinear equations, other open root location methods such as the Newton-Raphson method can be used for the same purpose. Recall that the Newton-Raphson method was predicated on employing the derivative (i.e., the slope) of a function to estimate its intercept with the axis of the independent variable—that is, the root. In [Chap. 6](#), we used a graphical derivation to compute this estimate. An alternative is to derive it from a first-order Taylor series expansion:

$$f(x_{i+1}) = f(x_i) + (x_{i+1} - x_i)f'(x_i) \quad (12.8)$$

where  $x_i$  is the initial guess at the root and  $x_{i+1}$  is the point at which the slope intercepts the  $x$  axis. At this intercept,  $f(x_{i+1})$  by definition equals zero and [Eq. \(12.8\)](#) can be rearranged to yield

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \quad (12.9)$$

which is the single-equation form of the Newton-Raphson method.

The multiequation form is derived in an identical fashion. However, a multivariable Taylor series must be used to account for the fact that more than one independent variable contributes to the determination of the root. For the two-variable case, a first-order Taylor series can be written for each nonlinear equation as

$$f_{1,i+1} = f_{1,i} + (x_{1,i+1} - x_{1,i})\frac{\partial f_{1,i}}{\partial x_1} + (x_{2,i+1} - x_{2,i})\frac{\partial f_{1,i}}{\partial x_2} \quad (12.10a)$$

$$f_{2,i+1} = f_{2,i} + (x_{1,i+1} - x_{1,i})\frac{\partial f_{2,i}}{\partial x_1} + (x_{2,i+1} - x_{2,i})\frac{\partial f_{2,i}}{\partial x_2} \quad (12.10b)$$

Just as for the single-equation version, the root estimate corresponds to the values of  $x_1$  and  $x_2$ , where  $f_{1,i+1}$  and  $f_{2,i+1}$  equal zero. For this situation, [Eq. \(12.10\)](#) can be rearranged to give

$$\frac{\partial f_{1,i}}{\partial x_1} x_{1,i+1} + \frac{\partial f_{1,i}}{\partial x_2} x_{2,i+1} = -f_{1,i} + x_{1,i}\frac{\partial f_{1,i}}{\partial x_1} + x_{2,i}\frac{\partial f_{1,i}}{\partial x_2} \quad (12.11a)$$

$$\frac{\partial f_{2,i}}{\partial x_1} x_{1,i+1} + \frac{\partial f_{2,i}}{\partial x_2} x_{2,i+1} = -f_{2,i} + x_{1,i}\frac{\partial f_{2,i}}{\partial x_1} + x_{2,i}\frac{\partial f_{2,i}}{\partial x_2} \quad (12.11b)$$



Because all values subscripted with  $i$ s are known (they correspond to the latest guess or approximation), the only unknowns are  $x_{1,i+1}$  and  $x_{2,i+1}$ . Thus, [Eq. \(12.11\)](#) is a set of two linear equations with two unknowns. Consequently, algebraic manipulations (e.g., Cramer's rule) can be employed to solve for

$$x_{1,i+1} = x_{1,i} - \frac{f_{1,i} \frac{\partial f_{2,i}}{\partial x_2} - f_{2,i} \frac{\partial f_{1,i}}{\partial x_2}}{\frac{\partial f_{1,i}}{\partial x_1} \frac{\partial f_{2,i}}{\partial x_2} - \frac{\partial f_{1,i}}{\partial x_2} \frac{\partial f_{2,i}}{\partial x_1}} \quad (12.12a)$$

$$x_{2,i+1} = x_{2,i} - \frac{f_{2,i} \frac{\partial f_{1,i}}{\partial x_1} - f_{1,i} \frac{\partial f_{2,i}}{\partial x_1}}{\frac{\partial f_{1,i}}{\partial x_1} \frac{\partial f_{2,i}}{\partial x_2} - \frac{\partial f_{1,i}}{\partial x_2} \frac{\partial f_{2,i}}{\partial x_1}} \quad (12.12b)$$

The denominator of each of these equations is formally referred to as the determinant of the *Jacobian* of the system.

[Equation \(12.12\)](#) is the two-equation version of the Newton-Raphson method. As in the following example, it can be employed iteratively to home in on the roots of two simultaneous equations.

## EXAMPLE 12.4

### Newton-Raphson for a Nonlinear System

#### Problem Statement

Use the multiple-equation Newton-Raphson method to determine roots of [Eq. \(12.6\)](#). Initiate the computation with guesses of  $x_1 = 1.5$  and  $x_2 = 3.5$ .

#### Solution

First compute the partial derivatives and evaluate them at the initial guesses of  $x$  and  $y$ :

$$\begin{aligned} \frac{\partial f_{1,0}}{\partial x_1} &= 2x_1 + x_2 = 2(1.5) + 3.5 = 6.5 & \frac{\partial f_{1,0}}{\partial x_2} &= x_1 = 1.5 \\ \frac{\partial f_{2,0}}{\partial x_1} &= 3x_2^2 = 3(3.5)^2 = 36.75 & \frac{\partial f_{2,0}}{\partial x_2} &= 1 + 6x_1x_2 = 1 + 6(1.5)(3.5) = \end{aligned}$$

Thus, the determinant of the Jacobian for the first iteration is

$$6.5(32.5) - 1.5(36.75) = 156.125$$

The values of the functions can be evaluated at the initial guesses as

$$\begin{aligned} f_{1,0} &= (1.5)^2 + 1.5(3.5) - 10 = -2.5 \\ f_{2,0} &= 3.5 + 3(1.5)(3.5)^2 - 57 = 1.625 \end{aligned}$$

These values can be substituted into [Eq. \(12.12\)](#) to give

$$\begin{aligned} x_1 &= 1.5 - \frac{-2.5(32.5) - 1.625(1.5)}{156.125} = 2.03603 \\ x_2 &= 3.5 - \frac{1.625(6.5) - (-2.5)(36.75)}{156.125} = 2.84388 \end{aligned}$$

Thus, the results are converging to the true values of  $x_1 = 2$  and  $x_2 = 3$ . The computation can be repeated until an acceptable accuracy is obtained.

When the multi-equation Newton-Raphson works, it exhibits the same speedy quadratic convergence as the single-equation version. However, just as with successive substitution, it can diverge if the initial guesses are not sufficiently close to the true roots. Whereas graphical methods could be employed to derive good guesses for the single-equation case, no such simple procedure is available for the multiequation version. Although there are some advanced approaches for obtaining acceptable first estimates, often the initial guesses must be obtained on the basis of trial and error and knowledge of the physical system being modeled.

The two-equation Newton-Raphson approach can be generalized to solve  $n$  simultaneous equations. To do this, [Eq. \(12.11\)](#) can be written for the  $k$ th equation as

$$\frac{\partial f_{k,i}}{\partial x_1} x_{1,i+1} + \frac{\partial f_{k,i}}{\partial x_2} x_{2,i+1} + \cdots + \frac{\partial f_{k,i}}{\partial x_n} x_{n,i+1} = -f_{k,i} + x_{1,i} \frac{\partial f_{k,i}}{\partial x_1} + x_{2,i} \frac{\partial f_{k,i}}{\partial x_2} + \cdots + x_{n,i} \frac{\partial f_{k,i}}{\partial x_n}$$

where the first subscript  $k$  represents the equation or unknown and the second subscript denotes whether the value or function in question is at the present value ( $i$ ) or at the next value ( $i + 1$ ). Notice that the only unknowns in [Eq. \(12.13\)](#) are the  $x_{k,i+1}$  terms on the left-hand side. All other quantities are located at the present value ( $i$ ) and, thus, are known at any iteration. Consequently, the set of equations generally represented by [Eq. \(12.13\)](#) (i.e., with  $k = 1, 2, \dots, n$ ) constitutes a set of linear simultaneous equations that can be solved numerically by the elimination methods elaborated in previous chapters.

Matrix notation can be employed to express [Eq. \(12.13\)](#) concisely as

$$[J]\{x_{i+1}\} = -\{f\} + [J]\{x_i\} \quad (12.14)$$

where the partial derivatives evaluated at  $i$  are written as the *Jacobian matrix* consisting of the partial derivatives:

$$[J] = \begin{bmatrix} \frac{\partial f_{1,i}}{\partial x_1} & \frac{\partial f_{1,i}}{\partial x_2} & \cdots & \frac{\partial f_{1,i}}{\partial x_n} \\ \frac{\partial f_{2,i}}{\partial x_1} & \frac{\partial f_{2,i}}{\partial x_2} & \cdots & \frac{\partial f_{2,i}}{\partial x_n} \\ \vdots & \vdots & \vdots & \vdots \\ \frac{\partial f_{n,i}}{\partial x_1} & \frac{\partial f_{n,i}}{\partial x_2} & \cdots & \frac{\partial f_{n,i}}{\partial x_n} \end{bmatrix} \quad (12.15)$$

The initial and final values are expressed in vector form as

$$\{x_i\}^T = [x_{1,i} \quad x_{2,i} \quad \cdots \quad x_{n,i}]$$

and

$$\{x_{i+1}\}^T = [x_{1,i+1} \quad x_{2,i+1} \quad \cdots \quad x_{n,i+1}]$$

Finally, the function values at  $i$  can be expressed as

$$\{f\}^T = [f_{1,i} \quad f_{2,i} \quad \cdots \quad f_{n,i}]$$

 **Equation (12.14)** can be solved using a technique such as Gauss elimination. This process can be repeated iteratively to obtain refined estimates in a fashion similar to the two-equation case in

 **Example 12.4.**

Insight into the solution can be obtained by solving  **Eq. (12.14)** with matrix inversion. Recall that the single-equation version of the Newton-Raphson method is

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \quad (12.16)$$

If [Eq. \(12.14\)](#) is solved by multiplying it by the inverse of the Jacobian, the result is

$$\{x_{i+1}\} = \{x_i\} - [J]^{-1}\{f\} \quad (12.17)$$

Comparison of [Eqs. \(12.16\)](#) and [\(12.17\)](#) clearly illustrates the parallels between the two equations. In essence, the Jacobian is analogous to the derivative of a multivariate function.

Such matrix calculations can be implemented in Python. We can illustrate this by using Python Page 276 to duplicate the calculations from [Example 12.4](#). After defining the initial guesses, we can compute the Jacobian and the function values.

```
import numpy as np
x = np.matrix([ [1.5 ] , [3.5] ])
f = np.matrix([ [ x[0,0]**2+x[0,0]*x[1,0]-10 ] ,
                [ x[1,0]+3*x[0,0]*x[1,0]**2-57 ] ])
print('Function values are:\n',f)
J = np.matrix([ [ 2*x[0,0]+x[1,0], x[0,0] ] ,
                [ 3*x[1,0]**2, 1+6*x[0,0]*x[1,0] ] ])
print('Jacobian is:\n',J)
xnew = x-np.linalg.inv(J)*f
print('New values for x:\n' ,xnew)

Function values are:
[[-2.5 ]
 [ 1.625]]
Jacobian is:
[[ 6.5 1.5 ]
 [36.75 32.5 ]]
New values for x:
[[2.03602882]
 [2.8438751 ]]
```

These Python statements can be incorporated into a function, called `newtmult`. As shown in [Fig. 12.4](#), the arguments to the function include another function that computes the  $f$  and  $J$  values, initial  $x$  guesses, an optional convergence criterion and iteration maximum. The `newtmult` function iterates the Newton-Raphson method until either the convergence is satisfied, or the iteration limit is reached.

**FIGURE 12.4** Python function to implement the Newton-Raphson method for systems of nonlinear equations.

```
def newtmult(fandJ,x0,es=1.e-7,maxit=20):
    """
    Newton-Raphson solution of sets of nonlinear algebraic equations
    Input:
    fandJ = function name that supplies f and Jacobian values
    x0 = initial guesses for x
    es = convergence tolerance (default = 1.3-7)
    maxit = iteration limit (default = 20)
    Output:
    x = solution
    f = function values at the solution
    ea = relative error
    iter = number of iterations taken
    """
    n,m = np.shape(x0) # get the number of equations in n
    x = np.zeros((n,m))
```

```

for i in range(n): # initialize x
    x[i] = x0[i]
for i in range(maxit):
    f,J = fandJ(x) # get the function values and the Jacobian
    dx = np.linalg.inv(J).dot(f) # Newton-Raphson iteration
    x = x - dx
    ers = dx/x
    ea = max(abs(ers))
    if ea < es: break # check for convergence
if i == maxit:
    return 'iteration limit reached'
else:
    return x,f,ea,i+1 # here if solution successful

```

We should note that there are two shortcomings to the foregoing approach. First,

 **Eq. (12.15)** is sometimes inconvenient to evaluate. Therefore, variations of the Newton-Raphson approach have been developed to circumvent this dilemma. As might be expected, most are based on using finite-difference approximations for the partial derivatives that comprise the Jacobian matrix.

The second shortcoming of the multiequation Newton-Raphson method is that excellent initial guesses are usually required to ensure convergence. Because these are sometimes difficult or inconvenient to obtain, alternate approaches that are slower than Newton-Raphson, but have more stable convergence behavior have been developed. One approach is to reformulate the nonlinear system as a single function:

$$F(\mathbf{x}) = \sum_{i=1}^n [f_i(\mathbf{x})]^2$$

where  $\mathbf{x}$  is a vector of the individual unknown  $x$ 's, and  $f_i(\mathbf{x})$  is the  $i$ th equation of the original system of  **Eq. (12.7)**. The values of  $\mathbf{x}$  that minimize this function also represent the solution of the nonlinear system. With this approach, one must take care that the  $f_i(\mathbf{x})$  values are not of widely differing magnitudes. If so, it would be necessary to rescale one or more of the functions. In this approach, multivariate nonlinear optimization techniques can be employed to obtain solutions.

Another empirical approach, when the standard Newton-Raphson method is unstable and does not converge, is to employ a *decelerator* in the multivariable method formula as follows:

$$\mathbf{x}_{i+1} = \mathbf{x}_i - \text{decel} \mathbf{J}^{-1}(\mathbf{x}_i) \mathbf{f}(\mathbf{x}_i)$$

The value of *decel* is adjusted between zero and one to provide stable convergence. Note that this is akin to our use of underrelaxation for the Gauss-Seidel method ( **Sec. 12.1.3**).

It is worth mentioning that these techniques can fail when they adjust the  $\mathbf{x}$  variables in a way that cause the function or Jacobian evaluations to fail. An example of this is when square roots are taken and an  $x$  variable is adjusted into the negative region. This introduces the general concept of constraining the ranges of acceptable values for variables, and that involves modifications of these techniques to accommodate such constraints.

## 12.2.3 Python SciPy Function: root

The `root` function in the SciPy `optimize` module solves systems of nonlinear equations. A general representation of the syntax is

```
result = root(f,x0)
```

The `result` variable contains summary information on the performance of the root function. The solution values are found in

```
result.x
```

There are several optional arguments. Commonly used are

- |                     |                                                                                                                                                                                                                            |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>method</code> | allows for the selection of one of ten numerical methods.<br>The default method is 'hybr' using a modified Powell's method which minimizes the single objective function, $F(\mathbf{x})$ , described in the last section. |
| <code>jac</code>    | allows the user to supply a separate function that computes the Jacobian; otherwise, the Jacobian is estimated numerically.                                                                                                |
| <code>tol</code>    | specifies the convergence criterion.                                                                                                                                                                                       |

As an example, we can solve the system from [Eq. \(12.6\)](#),

---

Page 278

---

$$\begin{aligned}f_1(\mathbf{x}) &= 2x_1 + x_1x_2 - 10 \\f_2(\mathbf{x}) &= x_2 + 3x_1x_2^2 - 57\end{aligned}$$

with initial guesses  $x_1 = 1.5$  and  $x_2 = 3.5$ . Here is the Python code and the displayed solution:

```
import numpy as np
from scipy.optimize import root

def f(x):
    x1 = x[0]
    x2 = x[1]
    f1 = x1**2+x1*x2-10
    f2 = x2+3*x1*x2**2-57
    return np.array([f1,f2])

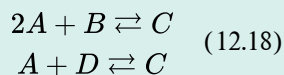
x0 = np.matrix(' 1.5 ; 3.5 ')
result = root(f,x0)
xsoln = result.x
print('Solution is:\n' ,xsoln)

Solution is:
[2. 3.]
```

## CHEMICAL REACTIONS

### Background

Nonlinear systems of equations occur frequently in the characterization of chemical reactions. As an example, the following chemical reactions take place in a closed, batch reactor vessel:



At equilibrium, they can be characterized by

$$\begin{aligned} K_1 &= \frac{c_c}{c_a^2 c_b} \\ K_2 &= \frac{c_c}{c_a c_d} \end{aligned} \quad (12.19)$$

where  $c_i$  represents the molar concentration of chemical component  $i$  and  $K_1$  and  $K_2$  are equilibrium constants.

### Problem Statement

If  $x_1$  and  $x_2$  are the concentrations of component  $C$  produced by the first and second reactions, respectively, formulate the equilibrium relationships as a pair of two nonlinear equations. Example parameter values are

$$K_1 = 4 \times 10^{-4}, K_2 = 3.7 \times 10^{-2}, c_{a0} = 50, c_{b0} = 20, c_{c0} = 5, c_{d0} = 10$$

where  $c_{i,0}$  is the initial concentration of component  $i$  when the reactor is charged. Employ the Newton-Raphson method to solve for the  $x$ 's.

### Solution

Using the stoichiometry of [Eq. \(12.18\)](#), the concentrations of each constituent can be represented in terms of  $x_1$  and  $x_2$  as

$$\begin{aligned} c_a &= c_{a0} - 2x_1 - x_2 \\ c_b &= c_{b0} - x_1 \\ c_c &= c_{c0} + x_1 + x_2 \\ c_d &= c_{d0} - x_2 \end{aligned} \quad (12.20)$$

These expressions can be substituted into the equilibrium relationships, [Eq. \(12.19\)](#), to yield two nonlinear equations in  $x_1$  and  $x_2$ . Upon solving for the  $x$ 's, the formulas of [Eq. \(12.20\)](#) can be utilized to calculate the equilibrium concentrations.

$$\begin{aligned} f_1(x_1, x_2) &= \frac{c_{c0} + x_1 + x_2}{(c_{a0} - 2x_1 - x_2)^2(c_{b0} - x_1)} - K_1 = 0 \\ f_2(x_1, x_2) &= \frac{c_{c0} + x_1 + x_2}{(c_{a0} - 2x_1 - x_2)(c_{d0} - x_1)} - K_2 = 0 \end{aligned} \quad (12.21)$$

We will utilize the `newtmult` function to solve these equations. Instead of deriving the Jacobian matrix analytically, we will illustrate the approximation of the Jacobian by finite differences. This is similar to the technique used for the modified secant method in [Sec. 6.3](#). For the current set of two nonlinear equations, this approximation is

$$\mathbf{J} \cong \begin{bmatrix} \frac{f_1(x_1 + \delta x_1, x_2) - f_1(x_1, x_2)}{\delta x_1} & \frac{f_1(x_1, x_2 + \delta x_2) - f_1(x_1, x_2)}{\delta x_2} \\ \frac{f_2(x_1 + \delta x_1, x_2) - f_2(x_1, x_2)}{\delta x_1} & \frac{f_2(x_1, x_2 + \delta x_2) - f_2(x_1, x_2)}{\delta x_2} \end{bmatrix} \quad (12.22)$$

The relationships in [Eqs. \(12.21\)](#) and [\(12.22\)](#) can be expressed in the following Python functions and associated statements.

```
K1 = 4e-4
K2 = 3.7e-2
ca0 = 50
cb0 = 20
cc0 = 5
cd0 = 10

def f1(x1,x2):
    return (cc0+x1+x2)/(ca0-2*x1-x2)**2/(cb0-x1)-K1

def f2(x1,x2):
    return (cc0+x1+x2)/(ca0-2*x1-x2)/(cd0-x2)-K2

def my_fandJ(x):
    delta = 1.e-6
    x1 = x[0,0]
    x2 = x[1,0]
    f1t = f1(x1,x2)
    f2t = f2(x1,x2)
    f = np.matrix( [ [ f1t ] , [ f2t ] ] )
    J11 = (f1(x1+delta,x2)-f1(x1,x2))/delta
    J12 = (f1(x1,x2+delta)-f1(x1,x2))/delta
    J21 = (f2(x1+delta,x2)-f1(x1,x2))/delta
    J22 = (f2(x1,x2+delta)-f1(x1,x2))/delta
    J = np.matrix([ [ J11 , J12 ] , [ J21 , J22 ] ] )
    return f,J
```

The function `newtmult` can then be employed to determine the roots given the initial guesses,  $x_1 = 3$  and  $x_2 = 3$ . The additional Python code is

```
x0 = np.matrix([[3.],[3.]])
xsoln,fsoln,esoln,itsoln = newtmult(my_fandJ,x0,es=1.e-6,maxit=50)
print('Solution is:\n',xsoln)
print('Function values are:\n',fsoln)
print('Relative error achieved =',esoln)
```



```

print('Iterations taken =',itsoln)
x1 = xsoln[0]
x2 = xsoln[1]
ca = ca0 - 2*x1 - x2
cb = cb0 - x1
cc = cc0 + x1 + x2
cd = cd0 - x2
print('\nConcentrations are:\n')
print('A: ',ca)
print('B: ',cb)
print('C: ',cc)
print('D: ',cd)

```

The results are

---

Page 280

---

```

Solution is:
[[3.35427349]
 [2.64572392]]
Function values are:
[[ 2.21665134e-16]
 [-2.00839913e-04]]
Relative error achieved = [[8.62640718e-07]]
Iterations taken = 4

Concentrations are:
A: [[40.6457291]]
B: [[16.64572651]]
C: [[10.99999741]]
D: [[7.35427608]]

```

The SciPy `optimize.root` function can also be used to find the solution as follows:

```

from scipy.optimize import root

def my_func(x):
    x1 = x[0]
    x2 = x[1]
    f1t = f1(x1,x2)
    f2t = f2(x1,x2)
    f = np.array( [ f1t , f2t ] )
    return f

result = root(my_func,x0)
xsoln = result.x
print('Solution is:\n' ,xsoln)

```

This code produces the solution:

```

Solution is:
[3.33660129 2.67718089]

```

which meets a tighter error criterion than what was specified in the `newtmult` function.

## PROBLEMS

---

**12.1** Solve the following system using three iterations of the Gauss-Seidel method. If necessary, rearrange the equations. Show all the steps in your solution including your error estimates. At the end of your computations, compute the true error of your final results.

$$\begin{aligned}3x_1 + 8x_2 &= 11 \\7x_1 - x_2 &= 5\end{aligned}$$

### 12.2

(a) Use the Gauss-Seidel method to solve the following system until the maximum percent relative error,  $\varepsilon_s$ , falls below 5%.

$$\begin{bmatrix} 0.8 & -0.4 & 0 \\ -0.4 & 0.8 & -0.4 \\ 0 & -0.4 & 0.8 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 41 \\ 25 \\ 105 \end{bmatrix}$$

(b) Repeat (a) but use relaxation with  $\lambda = 1.2$ .

**12.3** Use the Gauss-Seidel method to solve the following system of linear equations until the percent relative error,  $\varepsilon_s$ , falls below 5%.

$$\begin{aligned}10x_1 + 2x_2 - x_3 &= 27 \\-3x_1 - 6x_2 + 2x_3 &= -61.5 \\x_1 + x_2 + 5x_3 &= -21.5\end{aligned}$$

**12.4** Repeat  **Prob. 12.3** but use Jacobi iteration.

**12.5** The following system of equations determines the concentrations ( $c$ 's in  $\text{kg/m}^3$ ) in a series of well-mixed, interconnected tanks as a function of the mass input to each tank ( $\text{kg/day}$ ), shown on the right side of each equation.

$$\begin{aligned}15c_1 - 3c_2 - c_3 &= 3800 \\-3c_1 + 18c_2 - 6c_3 &= 1200 \\-4c_1 - c_2 + 12c_3 &= 2350\end{aligned}$$

Solve this problem with the Gauss-Seidel method to  $\varepsilon_s \leq 5\%$ .

**12.6** Use the Gauss-Seidel method (a) without relaxation and (b) with relaxation ( $\lambda = 1.2$ ) to solve the following set of linear equations to meet an error tolerance of  $\varepsilon_s = 5\%$ . If necessary, rearrange the equations to achieve convergence.

$$\begin{aligned}2x_1 - 6x_2 - x_3 &= -38 \\ -3x_1 - x_2 + 7x_3 &= -34 \\ -8x_1 + x_2 - 2x_3 &= -20\end{aligned}$$

**12.7** Of the following three sets of linear equations, identify the set(s) that you could not solve using an iterative method such as Gauss-Seidel. Show, using any number of iterations required, that your solution does not converge. State your convergence criterion clearly (how you know that it is not converging).

| Set One            | Set Two            | Set Three            |
|--------------------|--------------------|----------------------|
| $8x + 3y + z = 13$ | $x + y + 6z = 8$   | $-3x + 4y + 5z = 6$  |
| $-6x + 8z = 2$     | $x + 5y - z = 5$   | $-2x + 2y - 3z = -3$ |
| $2x + 5y - z = 6$  | $4x + 2y - 2z = 4$ | $2y - z = 1$         |

**12.8** Determine the solution of the simultaneous nonlinear equations

$$\begin{aligned}y &= -x^2 + x + 0.75 \\ y &= x^2 - 5xy\end{aligned}$$

Use the Newton-Raphson method and employ initial guesses  $x = y = 1.2$ . Carry out three iterations.

**12.9** Determine the solution of the simultaneous nonlinear equations

$$\begin{aligned}x^2 &= 5 - y^2 \\ y + 1 &= x^2\end{aligned}$$

(a) graphically, and (b) using successive substitution with initial guesses,  $x = y = 1.5$ .

**12.10**  **Figure P12.10** depicts a chemical mass transfer process consisting of a series of well-mixed tanks through which a gas is flowing from left to right and is passed over a liquid flowing from right to left. The transfer of a chemical species from the gas into the liquid occurs at a rate that is proportional to the difference between the adjacent gas and liquid concentrations in each tank. At steady state, a mass balance for the first tank can be written for the gas as

$$Q_G c_{G0} - Q_G c_{G1} + D(c_{L1} - c_{G1}) = 0$$

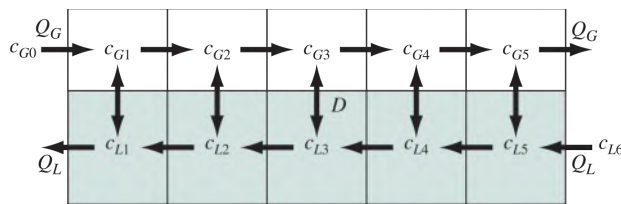
and for the liquid as

$$Q_L c_{L2} - Q_L c_{L1} - D(c_{L1} - c_{G1}) = 0$$

where  $Q_G$  and  $Q_L$  are the gas and liquid volumetric flow rates, respectively, and  $D$  is the mass transfer rate. Similar balances can be written for the other tanks. Use Gauss-Seidel without relaxation to solve for the concentrations given the following values:

$$Q_G = 2, \quad Q_L = 1, \quad D = 0.8, \quad c_{G0} = 100, \quad c_{L6} = 10$$

**FIGURE P12.10**

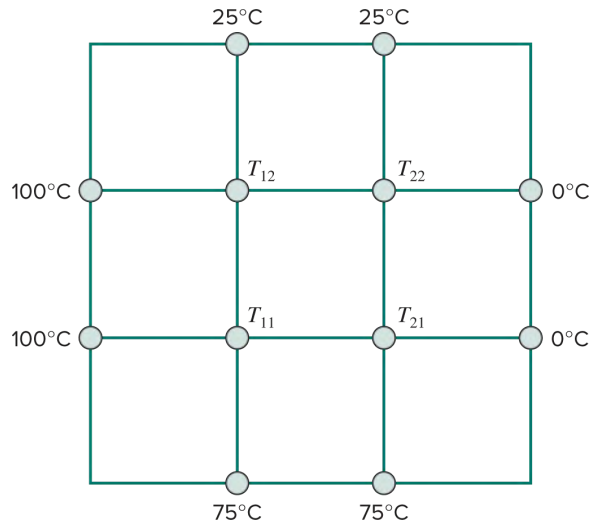


**12.11** The two-dimensional, steady-state distribution of temperature on a heated plate can be modeled by *Laplace's equation*:

$$0 = \frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2}$$

If a grid is laid out on the plate, as shown in  **Fig. P12.11**, it forms a series of nodes. By approximating the second-order partial derivatives with centered finite-differences, one can form a set of linear algebraic equations. Assuming that the grid spacing is the same vertically as horizontally, use the Gauss-Seidel method to solve for the temperatures of the interior nodes shown on the figure.

**FIGURE P12.11**



**12.12** Modify the `GaussSeidel` function presented in [Fig. 12.2](#) to include a return of the relative error and the number of iterations taken. Call the new function `GaussSeidel2`. Test it by replicating [Example 12.1](#) with an error criterion of  $1 \times 10^{-3}$ , and then use it to solve [Prob. 12.2a](#) with an error criterion of  $5 \times 10^{-2}$ . For each, report the solution, relative error achieved, and the number of iterations taken.

**12.13** Enhance the `GaussSeidel2` function from [Prob. 12.12](#) by adding relaxation as a feature. Name the new function `GaussSeidelR`. Add `lam` (for  $\lambda$ ) as an argument to the function with the default value of one (no relaxation). Test the new function by replicating [Example 12.2](#), and then apply it to solve [Prob. 12.2b](#) with the default error criterion.

**12.14** Duplicate the Python function `newtmult` shown in [Fig. 12.4](#). Test the function by solving [Example 12.4](#) and [Prob. 12.8](#). Use the default error criterion and iteration maximum for the former. For [Prob. 12.8](#), adjust the maximum iterations allowed so that the default error criterion is met.

**12.15** Determine the roots of the following simultaneous nonlinear equations using (a) fixed-point iteration, (b) the Newton-Raphson method, and (c) the SciPy `optimize.root` function.

$$\begin{aligned} y &= -x^2 + x + 0.75 \\ y + 5xy &= x^2 \end{aligned}$$

Employ initial guesses of  $x = y = 1.2$ . Discuss and compare the results.

**12.16** Determine the roots of the simultaneous nonlinear equations

$$\begin{aligned}(x-4)^2 + (y-4)^2 &= 5 \\ x^2 + y^2 &= 16\end{aligned}$$

Use a graphical approach to obtain your initial guesses. Determine refined estimates with (a) the Newton-Raphson method using the `newtmult` function, and (b) the `root` function from SciPy `optimize`.

**12.17** Repeat  **Prob. 12.16**, but determine the positive root of

$$\begin{aligned}y &= x^2 + 1 \\ y &= 2 \cos(x)\end{aligned}$$

**12.18** The equilibrium between liquid water and water vapor in a steam boiler at moderate pressures and temperatures can be described by two equations:

$$\text{the ideal gas law: } PV = \frac{m}{MW} R(T + 273.15)$$

$$\text{and the Antoine equation for vapor pressure: } \log_{10} P = A - \frac{B}{T + C}$$

where  $P$  = pressure in Pa,  $V$  = the volume of the vapor space in  $\text{m}^3$ ,  $m$  = the mass of the vapor in kg,  $MW$  = the molecular weight of water, 18.02 kg/kmol,  $R$  = the gas law constant, 8314 ( $\text{Pa} \cdot \text{m}^3$ )/(kmol $\cdot$ K), and  $T$  = the temperature in  $^{\circ}\text{C}$ . The constant parameters in the Antoine equation for water are  $A = 11.21$ ,  $B = 2354.7$ , and  $C = 280.71$ . For the following operating conditions of a boiler, determine the equilibrium temperature and pressure:  $m = 3.775$  kg and  $V = 3.142$   $\text{m}^3$ . Use the Newton-Raphson method and confirm your solution via the SciPy `optimize.root` function. Use initial guesses of  $P = 200,000$  Pa and  $T = 110$   $^{\circ}\text{C}$ .

Note: The terms in the ideal gas law, for the units given, are much, much greater than the terms in the Antoine equation. A good strategy then is to scale down the ideal gas law equation by a factor of 100,000.

**12.19** As previously described in  **Sec. 5.6**, the following system of five nonlinear equations governs the chemistry of rainwater:

$$\begin{aligned}K_1 &= \frac{[\text{H}^+][\text{HCO}_3^-]}{K_{HP_{\text{CO}_2}}} & K_2 &= \frac{[\text{H}^+][\text{CO}_3^{2-}]}{[\text{HCO}_3^-]} & K_w &= [\text{H}^+][\text{OH}^-] \\ c_T &= \frac{K_{HP_{\text{CO}_2}}}{10^6} + [\text{HCO}_3^-] + [\text{CO}_3^{2-}] \\ 0 &= [\text{HCO}_3^-] + 2[\text{CO}_3^{2-}] + [\text{OH}^-] - [\text{H}^+]\end{aligned}$$

where  $K_H$  = Henry's constant,  $K_1$ ,  $K_2$ , and  $K_3$  = equilibrium constants,  $c_T$  = total organic carbon,  $[\text{HCO}_3^-]$  = bicarbonate,  $[\text{CO}_3^{2-}]$  = carbonate,  $[\text{H}^+]$  = hydrogen ion, and  $[\text{OH}^-]$  = hydroxyl ion. Notice how the partial pressure of  $\text{CO}_2$  shows up in the equations indicating the impact of this greenhouse gas on the acidity of rain. Use these equations and the `root` function to compute the pH of rainwater given

$$K_H = 10^{-1.46}, K_1 = 10^{-6.3}, K_2 = 10^{-10.3}, \text{ and } K_w = 10^{-14}$$

Compare the results in 1958 when  $p_{\text{CO}_2}$  was 315 to 2015 when it was 400 ppm. Note that this is a difficult problem to solve because the concentrations tend to be very small and vary over many orders of magnitude. Therefore, it is useful to use the trick of expressing the unknowns on a negative log scale,  $pK = -\log_{10}(K)$ . That is, the five unknowns can be re-expressed as follows:

$$\begin{aligned} \text{pH} &= -\log_{10}([\text{H}^+]) & \text{pOH} &= -\log_{10}([\text{OH}^-]) \\ \text{pHCO}_3 &= -\log_{10}([\text{HCO}_3^-]) & \text{pCO}_3 &= \log_{10}([\text{CO}_3^{2-}]) \\ \text{pc}_T &= -\log_{10}(c_T) \end{aligned}$$

In addition, it is helpful to reduce the error tolerance to a number like  $10^{-12}$ . In the order presented above, use initial guesses of 7, 7, 3, 7, and 3.

# Eigenvalues

## CHAPTER OBJECTIVES

The primary objective of this chapter is to introduce you to eigenvalues and their roles in engineering and scientific applications.

- Understanding the mathematical definition of eigenvalues and eigenvectors.
- Gaining the concepts necessary to interpret system behavior based on eigenvalues.
- Seeing how eigenvalues and eigenvectors arise in the study of differential equation models.
- Understanding the role of eigenvalues in the study of pure oscillatory, vibrating systems.
- Knowing how to determine eigenvalues through the solution of the characteristic polynomial.
- Understanding the power method and its application to find the largest and smallest eigenvalues.
- Knowing how to use and interpret the Python NumPy `eigvals` and `eig` functions.

## YOU'VE GOT A PROBLEM

At the beginning of [Chap. 8](#), we used Newton's second law and force balances to predict the equilibrium positions of three bungee jumpers connected by cords. Because we assumed that the cords behaved like ideal springs, that is, followed Hooke's law, the steady-state solution reduced to solving a system of linear algebraic equations [recall [\(Eq. 8.1\)](#) and [Example 8.2](#)]. In mechanics, this is referred to as a *statics* problem.

Now, let's consider a *dynamics* problem involving the same system. That is, we'll study the jumpers' motion as a function of time. To do this, their initial conditions, positions and velocities, must be known. For example, we can set the jumpers' initial positions at the equilibrium values computed in [Example 8.2](#). If we then set their initial velocities to zero, there would be no movement because, by definition, the system is at equilibrium.

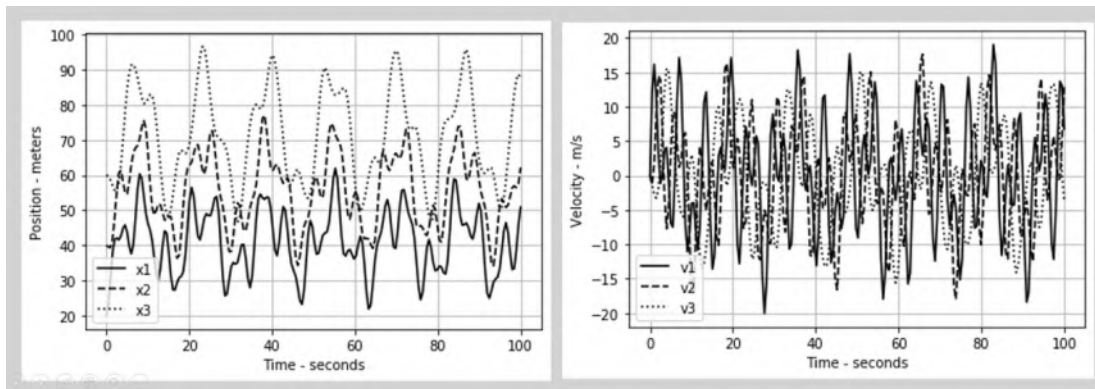
Because we are now interested in examining the system's dynamics, we must set the initial conditions to values that induce motion. For example, we might start with the jumpers located at the lengths of the cords when not stretched. We would then expect an abrupt downward motion followed by upward motion and so on. You may have witnessed these oscillations personally (brave you!) or at least by watching a video. Happily, the oscillations eventually die out because of damping effects, including air drag. If a dynamic model of the jumpers does not include damping, the oscillations reach a consistent shape and continue *ad infinitum*. We can use Python to solve the differential equations governing this system and generate the resulting positions and velocities. You have already seen this in [Chap. 1](#) and will learn



much more about the subject when we describe numerical solution of ordinary differential equations in [Part Six](#).

This behavior is shown in [Fig. 13.1](#). By observing the curves closely, you will recognize patterns. For example, the distances between peaks and troughs might be constant even though the amplitudes seem to vary. But, it is difficult to perceive whether there is anything systematic and predictable going on.

**FIGURE 13.1** The positions (m) and velocities (m/s) versus time for the system of the three interconnected bungee jumpers from [Example 8.2](#).



In this chapter, we deal with one approach for extracting fundamental information from mathematical models in engineering and science, such as the one describing the jumpers. This entails determining characteristic values, called *eigenvalues*, for such systems. We will see that eigenvalues and their corresponding *eigenvectors* provide a basis for understanding the dynamics of systems of differential equations. To do so, we need to provide you with the mathematical background required for this analysis. This may or may not be review for you, but it's good to start with a solid foundation.

Following this introduction, we will consider different scenarios where eigenvalues come into play. And, finally, we will provide a focus on the understanding of purely oscillatory or vibrational systems, which are very important in various fields of engineering and science, and, in particular, structural and mechanical engineering.

# 13.1 EIGENVALUES AND EIGENVECTORS—THE BASICS

Eigenvalues and eigenvectors are quantities that characterize a square matrix. They are of interest in numerous science and engineering applications, in particular, those involving differential equations. If  $\mathbf{A}$  is an  $n \times n$  matrix, an equation of interest is<sup>1</sup>

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x} \quad (13.1)$$

where  $\lambda$  = a scalar value, either real or complex, and  $\mathbf{x}$  is an  $n \times 1$  vector. It is obvious that, for any value of  $\lambda$ , a trivial solution to [Eq. \(13.1\)](#) is  $\mathbf{x} = \mathbf{0}$ . Where there is a value of  $\lambda$  that satisfies the equation for a vector  $\mathbf{x} \neq \mathbf{0}$ , that value of  $\lambda$  is an eigenvalue and the corresponding  $\mathbf{x}$  is a related eigenvector.

[Eq. \(13.1\)](#) can be rearranged to give

$$(\mathbf{A} - \lambda\mathbf{I})\mathbf{x} = \mathbf{0} \quad (13.2)$$

For the  $n$  linear equations represented by [Eq. \(13.2\)](#) to have a solution  $\mathbf{x} \neq \mathbf{0}$ , the matrix  $(\mathbf{A} - \lambda\mathbf{I})$  must be singular, that is, its determinant,  $|\mathbf{A} - \lambda\mathbf{I}|$ , must equal zero.

Page 285

For a simple case where  $n = 2$ , [Eq. \(13.2\)](#) can be written as

$$\begin{bmatrix} a_{11} - \lambda & a_{12} \\ a_{21} & a_{22} - \lambda \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad (13.3)$$

The determinant can be computed and set equal to zero

$$|\mathbf{A} - \lambda\mathbf{I}| = (a_{11} - \lambda)(a_{22} - \lambda) - a_{12}a_{21} = 0 \quad (13.4)$$

Thus, the eigenvalues are the roots of

$$\lambda^2 - (a_{11} + a_{22})\lambda - a_{12}a_{21} = 0$$

Employing the quadratic formula, we obtain

$$(13.5)$$

$$\lambda = \frac{(a_{11} + a_{22})}{2} \pm \frac{\sqrt{(a_{11} + a_{22})^2 + 4a_{12} a_{21}}}{2}$$

It is clear that the values of  $\lambda$  will be either two real quantities or a complex conjugate pair. By substituting each value from [Eq. \(13.5\)](#) into [Eq. \(13.3\)](#), we can solve for the corresponding  $\mathbf{x}$  vector. This vector, the *eigenvector*, establishes a relationship between  $x_1$  and  $x_2$ , which may be scaled by any factor. It is often scaled so that the magnitude of the vector,  $|\mathbf{x}| = \sqrt{x_1^2 + x_2^2} = 1$ .

For larger matrices,  $\mathbf{A}$ , the characteristic polynomial, represented in [Eq. \(13.4\)](#), has the order  $n$ , and solution by computer is required.

## EXAMPLE 13.1

### Eigenvalues and Eigenvectors of a Matrix

#### Problem Statement

Consider the square matrix

$$\mathbf{A} = \begin{bmatrix} 4 & 2 & 3 & 7 \\ 2 & 8 & 5 & 1 \\ 3 & 5 & 12 & 9 \\ 7 & 1 & 9 & 7 \end{bmatrix}$$

Determine the eigenvalues and corresponding eigenvectors of  $\mathbf{A}$ . Scale the eigenvectors to a magnitude of unity.

#### Solution

The characteristic equation is given by the determinant

$$\begin{vmatrix} 4 - \lambda & 2 & 3 & 7 \\ 2 & 8 - \lambda & 5 & 1 \\ 3 & 5 & 12 - \lambda & 9 \\ 7 & 1 & 9 & 7 - \lambda \end{vmatrix} = \lambda^4 - 31\lambda^3 + 175\lambda^2 + 282\lambda - 2076$$

To find the eigenvalues, we need to determine the roots of this polynomial. We can use the `roots` function in Python's NumPy module.

```
import numpy as np
coeff = np.array([1., -31., 175., 282., -2076.])
r = np.roots(coeff)
```

```
print('Roots are:\n',r)
```

Roots are:

```
[23.04467254 -3.23389171 7.4501013 3.73911787]
```

So, the eigenvalues are approximately 23.04, -3.234, 7.450, and 3.739.

Let's find the eigenvector corresponding to the first eigenvalue from

$$\begin{bmatrix} 4 - 23.04 & 2 & 3 & 7 \\ 2 & 8 - 23.04 & 5 & 1 \\ 3 & 5 & 12 - 23.04 & 9 \\ 7 & 1 & 9 & 7 - 23.04 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

or

Page 286

$$\begin{bmatrix} -19.04 & 2 & 3 & 7 \\ 2 & -15.04 & 5 & 1 \\ 3 & 5 & -11.04 & 9 \\ 7 & 1 & 9 & -16.04 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Here, we can solve the first equation algebraically for  $x_1$ ,

$$x_1 = \frac{2}{19.04}x_2 + \frac{3}{19.04}x_3 + \frac{7}{19.04}x_4 \cong 0.1050x_2 + 0.1576x_3 + 0.3677x_4$$

This can be substituted into the second equation to yield

$$x_2 = \frac{2}{15.04}(0.1050x_2 + 0.1576x_3 + 0.3677x_4) + \frac{5}{15.04}x_3 + \frac{1}{15.04}x_4$$

and

$$x_2 = 0.3584x_3 + 0.1170x_4$$

When this is substituted into the third equation and solved for  $x_3$ , the result is

$$x_3 = 1.2381x_4$$

At this point, an arbitrary value can be assigned to  $x_4$ , 1, and the rest of the  $x$ 's can be calculated as

$$x_3 = 1.238, x_2 = 0.561, x_1 = 0.622$$

and the first eigenvector is

$$\mathbf{x} = \begin{bmatrix} 0.622 \\ 0.561 \\ 1.238 \\ 1 \end{bmatrix} \text{ which is scaled to magnitude one as } \begin{bmatrix} 0.346 \\ 0.312 \\ 0.688 \\ 0.556 \end{bmatrix}$$

The same method is used to determine the eigenvectors associated with the other three eigenvalues. They are summarized here in a matrix where the columns are associated with the eigenvalues:

$$\begin{bmatrix} 0.346 & -0.581 & 0.679 & 0.287 \\ 0.312 & 0.204 & 0.375 & -0.849 \\ 0.688 & -0.365 & -0.617 & -0.108 \\ 0.516 & 0.698 & 0.132 & 0.430 \end{bmatrix}$$

We note that eigenvectors are orthogonal, one to another. The dot product of the first two columns, computed using Python, is

```
vtest = v[:,0].dot(v[:,1])
print(' {0:7.5f} '.format(vtest))
-0.00000
```

Now that we have introduced eigenvalues and eigenvectors and how they relate to a square matrix, we would like to see where they occur in applications in engineering and science.

## 13.2 APPLICATIONS OF EIGENVALUES AND EIGENVECTORS

---

The most frequent setting for engineering and scientific applications of eigenvalues is differential equations. However, there are other examples that relate strictly to algebraic equations, but we do not consider these in this chapter.

The use of eigenvalues to study the behavior of differential equation models is essential to many applications in engineering and science, especially in the design of equipment, processes, and structures. When the independent variable of ordinary differential equations (or an independent variable in partial differential equations) is time, eigenvalues reveal much about the dynamic behavior of the system being modeled.

Many practical differential equation models contain only first or second derivatives. This is because of the physical basis of the models, for example, Newton's second law relates the second derivative of position to applied forces. A first observation we will make is that it is possible to reduce ordinary differential equations with second derivatives to equivalent systems with only first derivatives. It is also possible to extend this technique to higher-order derivatives where they occur. We will follow that by studying the eigenvalues of systems of first-order differential equations, and then return to the special case of second-order equations that have no first-order derivatives.

---

Page 287

---

### 13.2.1 First-Order Equivalents to Second-Order Differential Equations

As a simple example, consider that we have a single, second-order differential equation in time that also contains a first-derivative term. For now, we will look at the homogeneous form of this equation.

$$\frac{d^2y}{dt^2} + a\frac{dy}{dt} + by = 0 \quad (13.6)$$

We will define two new dependent variables,

$$x_1 = y \quad \text{and} \quad x_2 = \frac{dy}{dt} = \frac{dx_1}{dt}$$

and rewrite the original differential equation,  **Eq. (13.6)**, in terms of the two first-order equations,

$$(13.7)$$

$$\begin{aligned}\frac{dx_1}{dt} &= x_2 \\ \frac{dx_2}{dt} &= -ax_2 - bx_1\end{aligned}$$

We can also write these last two equations in matrix form as

$$\frac{d\mathbf{x}}{dt} = \begin{bmatrix} 0 & 1 \\ -b & -a \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \mathbf{A}\mathbf{x} \quad (13.8)$$

Note that, if we do not have a first-derivative term, the  $a_{22}$  term is equal to zero.

## 13.2.2 Role of Eigenvalues and Eigenvectors in the Solution of Differential Equations

The homogeneous solution to the general form of a system of first-order, ordinary differential equations is

$$\mathbf{x} = \exp(\mathbf{A}t)\mathbf{x}_0 \quad (13.9)$$

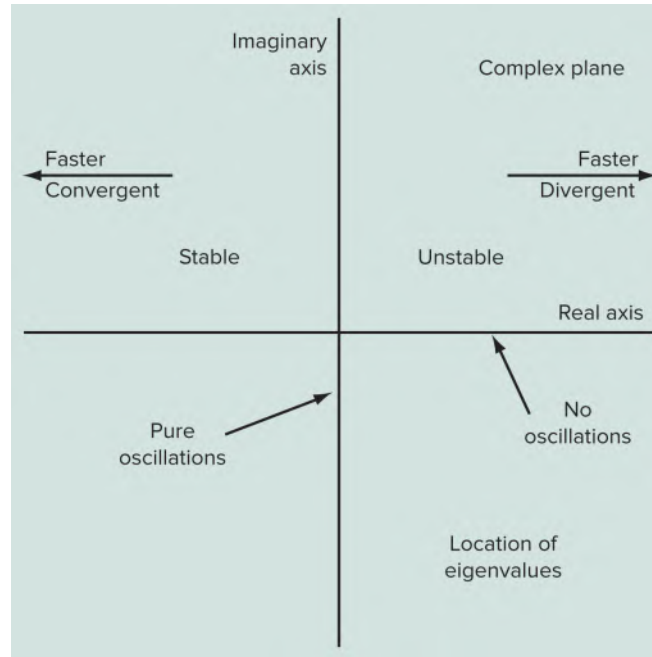
where  $\exp(\mathbf{A}t)$  is called the *matrix exponential* and can be expressed as  $\mathbf{U}\exp(\mathbf{D}t)\mathbf{U}^{-1}$ . Here,  $\mathbf{U}$  is the matrix of eigenvectors, and  $\mathbf{D}$  is a diagonal matrix of the eigenvalues. The solution for each  $x_i$  in  $\mathbf{x}$  will be a linear combination of terms in  $\exp(\lambda_j t)$ ,  $j = 1, \dots, n$ .

At this point, we recognize that eigenvalues are revealed through the solution of the characteristic polynomial. Since this polynomial has real coefficients, the eigenvalues will occur as real numbers, both positive and negative, and complex numbers, occurring as complex conjugate pairs with the same real part and opposite imaginary parts. Based on this, we make some important qualitative observations:

1. positive real eigenvalues are unstable and lead to exploding exponentials,
2. negative real eigenvalues die out,
3. eigenvalues with zero real part and finite imaginary parts oscillate without amplification or attenuation, often called a pure harmonic oscillator,
4. a zero eigenvalue corresponds to a linear solution behavior for a constant input,
5. complex eigenvalues provide oscillatory behavior,
6. an alternate representation of the solution for complex eigenvalues is a combination of sine and cosine terms,
7. the larger the absolute value of the real part of the eigenvalue, the faster its convergent or divergent behavior, and
8. the larger the imaginary part of a complex eigenvalue, the higher the frequency of its oscillations.

These observations are illustrated neatly in  **Fig. 13.2**.

**FIGURE 13.2** Location of eigenvalues and the behavior of solutions depicted on complex plane.



## EXAMPLE 13.2

### Eigenvalues of a System of First-Order Differential Equations

#### Problem Statement

Determine the eigenvalues and predict the behavior of the following system of first-order differential equations.

$$\begin{aligned}\frac{dx_1}{dt} &= -3x_1 + x_2 \\ \frac{dx_2}{dt} &= x_1 - 3x_2\end{aligned}$$

#### Solution

We represent the differential equations in matrix form.



$$\frac{d\mathbf{x}}{dt} = \begin{bmatrix} -3 & 1 \\ 1 & -3 \end{bmatrix} \mathbf{x} = \mathbf{A}\mathbf{x}$$

To determine the eigenvalues, we can derive the characteristic polynomial.

$$\begin{vmatrix} -3 - \lambda & 1 \\ 1 & -3 - \lambda \end{vmatrix} = (\lambda + 3)(\lambda + 3) - 1 = \lambda^2 + 6\lambda + 8 = 0$$

and determine its roots:

$$\lambda = -\frac{6}{2} \pm \frac{\sqrt{36 - 32}}{2} = -2, -4$$

Since both eigenvalues are negative, the solutions for the  $x$ 's will be stable and die out to zero without oscillations.

It is important to note that we did not have to derive the solution of the differential equations in

Page 289

 **Example 13.2** in order to make a judgment about the behavior of the system.

## 13.2.3 Eigenvalues and Ordinary Differential Equations with Pure Oscillations

In real systems, natural oscillations either amplify or die out. If they amplify, something will eventually break. To sustain oscillations, it requires an input or forcing function that is periodic. Even so, we can study important properties of a system with a model that proposes pure oscillations that sustain. Here is an example of a single, second-order, ordinary differential equation.

$$\frac{d^2y}{dt^2} = -ay \quad (13.10)$$

Using the approach from  **Sec. 13.2.1**, we can convert this single, second-order equation into two first order equations,

$$\begin{aligned} \frac{dx_1}{dt} &= x_2 \\ \frac{dx_2}{dt} &= -ax_1 \end{aligned} \quad (13.11)$$

and represent it in matrix format as

$$\frac{d\mathbf{x}}{dt} = \begin{bmatrix} 0 & 1 \\ -a & 0 \end{bmatrix} \mathbf{x} = \mathbf{A}\mathbf{x} \quad (13.12)$$

Again, we can determine the eigenvalues of  $\mathbf{A}$ .

$$\begin{vmatrix} -\lambda & 1 \\ -a & -\lambda \end{vmatrix} = \lambda^2 + a = 0 \quad \lambda = \pm i\sqrt{a}$$

Since the eigenvalues are on the imaginary axis, the solution will be purely oscillatory, and the frequency of the oscillations will be  $\sqrt{a}$  as radians/time, not cycles/time. Recognizing that there are  $2\pi$  radians per cycle, the frequency in cycles per time would be  $\sqrt{a}/(2\pi)$ .

So, for this simple system, we can see how the coefficient  $a$  relates to the eigenvalue and how that relates to the frequency of oscillations. The eigenvectors for the system are also complex.

$$\mathbf{U} = \begin{bmatrix} -i 0.4082 & i 0.4082 \\ 0.9129 & 0.9129 \end{bmatrix}$$

There is a shorter approach to determining the eigenvalues and frequencies of pure oscillatory systems. We can illustrate this with a system of two second-order equations.

$$\begin{aligned} \frac{d^2 y_1}{dt^2} &= -a_{11}y_1 + a_{12}y_2 \\ \frac{d^2 y_2}{dt^2} &= a_{21}y_1 - a_{22}y_2 \end{aligned} \quad (13.13)$$

which can be written in matrix form as

$$\frac{d^2 \mathbf{y}}{dt^2} = \begin{bmatrix} -a_{11} & a_{12} \\ a_{21} & -a_{22} \end{bmatrix} \mathbf{y} \quad (13.14)$$

Since we know that these systems have periodic solutions, we can propose a solution of the form<sup>2</sup>

$$\mathbf{y} = \mathbf{x}e^{i\omega t} \quad (13.15)$$

where  $\omega$  = frequency, and  $\mathbf{x}$  is a vector of coefficients. Substituting this into the differential equation, we obtain

$$-\omega^2 \mathbf{x} = \mathbf{A} \mathbf{x} \quad (13.16)$$

which we recognize as an eigenvalue problem with  $\lambda = -\omega^2$  and  $\mathbf{x}$  as associated eigenvectors.

Since we know that eigenvalues will be the negative of the square of frequencies, we can determine the eigenvalues of this system directly,

Page 290

$$\begin{vmatrix} -a_{11} - \lambda & a_{12} \\ a_{21} & -a_{22} - \lambda \end{vmatrix} = (\lambda + a_{11})(\lambda + a_{22}) - a_{12} a_{21} = \lambda^2 + (a_{11} + a_{22})\lambda + (a_{11} a_{22} - a_{12} a_{21})$$

So, the eigenvalues are given by

$$\lambda = -\frac{(a_{11} + a_{22})}{2} \pm \frac{\sqrt{(a_{11} + a_{22})^2 - 4(a_{11} a_{22} - a_{12} a_{21})}}{2}$$

and the frequencies will be given by  $\omega = \sqrt{-\lambda}$ . We should note that we could have written [Eq. \(13.17\)](#) as

$$\begin{vmatrix} -a_{11} + \omega^2 & a_{12} \\ a_{21} & -a_{22} + \omega^2 \end{vmatrix} = \begin{vmatrix} a_{11} + \omega^2 & -a_{12} \\ -a_{21} & a_{22} - \omega^2 \end{vmatrix}$$

The frequencies determined here are called *natural* or *resonant* frequencies and are often designated by  $\omega_n$ , as they are inherent to the system and not dependent on external influences.

## EXAMPLE 13.3

### Eigenvalues a Pure Oscillatory System

#### Problem Statement

Study the behavior of the system described by

$$\begin{aligned}\frac{d^2 y_1}{dt^2} &= -5y_1 + 2y_2 \\ \frac{d^2 y_2}{dt^2} &= 2y_1 - 2y_2\end{aligned}$$

## Solution

The eigenvalues can be determined from

$$\begin{vmatrix} -5 - \lambda & 2 \\ 2 & -2 - \lambda \end{vmatrix} = (\lambda + 5)(\lambda + 2) - 4 = \lambda^2 + 7\lambda + 6 = 0$$

In this case,

$$\lambda = -\frac{7}{2} \pm \frac{\sqrt{49 - 24}}{2} = -1, -6$$

We can then solve for the eigenvectors. They are

$$\mathbf{x}_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \mathbf{x}_2 = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \text{ which can be scaled to suit.}$$

From the eigenvalues, we see that the frequencies are  $\omega = \pm 1, \pm \sqrt{6}$ .

To study the behavior, we will put together the general solution for this system. There are four possible components to the solution.

$$\mathbf{x}_1 e^{\pm it} \quad \text{and} \quad \mathbf{x}_2 e^{\pm i\sqrt{6}t}$$

Recalling the *Euler identity*,  $e^{\pm i\theta} \equiv \cos(\theta) \pm i \sin(\theta)$ ,

$$\begin{aligned}\mathbf{x}_1 e^{\pm it} &= \mathbf{x}_1 (\cos(t) \pm i \sin(t)) \\ \mathbf{x}_2 e^{\pm i\sqrt{6}t} &= \mathbf{x}_2 \left( \cos(\sqrt{6}t) \pm i \sin(\sqrt{6}t) \right)\end{aligned}$$

By adding and subtracting each solution pair and then dividing by two, we obtain the general solution:

$$\mathbf{y} = \mathbf{x}_1(a_1\cos(t) + b_1\sin(t)) + \mathbf{x}_2\left(a_2\cos(\sqrt{6}t) + b_2\sin(\sqrt{6}t)\right)$$

where  $a_1$ ,  $b_1$ ,  $a_2$ , and  $b_2$  are constants to be determined by the initial conditions on  $\mathbf{y}$  and its derivative. Recalling the eigenvectors, this becomes

$$\begin{aligned} y_1 &= a_1 \cos(t) + b_1 \sin(t) + 2a_2 \cos(\sqrt{6}t) + 2b_2 \sin(\sqrt{6}t) \\ y_2 &= 2a_1 \cos(t) + 2b_1 \sin(t) - a_2 \cos(\sqrt{6}t) - b_2 \sin(\sqrt{6}t) \end{aligned}$$

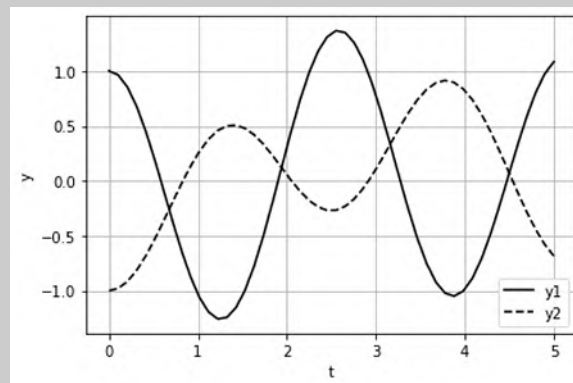
To specify the homogeneous solution completely, we include the following example initial conditions: Page 291

$$y_1(0) = 1, y_2(0) = -1, \frac{dy_1}{dt}(0) = 0, \text{ and } \frac{dy_2}{dt}(0) = 0$$

This allows us to determine the values for  $a_1$ ,  $b_1$ ,  $a_2$ ,  $b_2$ , write the solution as

$$\begin{aligned} y_1 &= -\frac{1}{5}\cos(t) + \frac{6}{5}\cos(\sqrt{6}t) \\ y_2 &= -\frac{2}{5}\cos(t) - \frac{3}{5}\cos(\sqrt{6}t) \end{aligned}$$

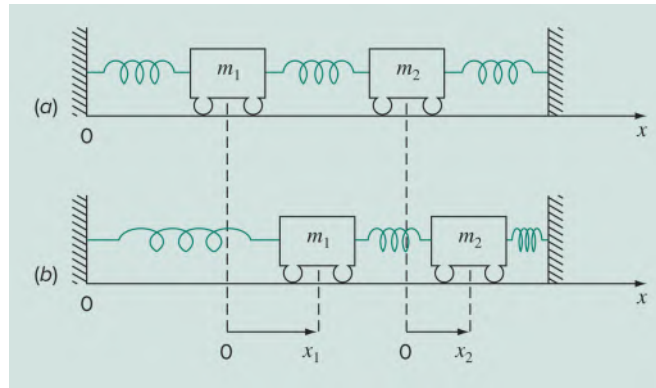
and a plot of the solution is



## 13.3 PHYSICAL SETTINGS—MASS-SPRING SYSTEMS

With the background provided in [Secs. 13.1](#) and [13.2](#), we can now study in more detail systems that focus on vibration. A model that is useful in such studies is depicted in [Fig. 13.3](#).

**FIGURE 13.3** A two-mass, three-spring system with frictionless rollers vibrating between two walls. The position of the masses can be referenced to local coordinates with origins at their respective equilibrium positions, as in (a). Positioning the masses away from equilibrium creates forces in the springs that on release lead to oscillations of the masses.



To simplify the analysis, assume that each mass has no external or damping forces acting on it. In addition, assume that each spring has the same natural length,  $l$ , at rest and the same spring constant,  $k$ . Finally, assume that the displacement of each spring is measured relative to its own local coordinate system with an origin at the spring's equilibrium position. Under these assumptions, Newton's second law can be employed to develop a dynamic force balance for each mass:

$$\begin{aligned} m_1 \frac{d^2 x_1}{dt^2} &= -kx_1 + k(x_2 - x_1) \\ m_2 \frac{d^2 x_2}{dt^2} &= -kx_2 + k(x_1 - x_2) \end{aligned} \quad (13.18)$$

We can rearrange the model in [Eq. \(13.18\)](#) into the following matrix form:

$$\frac{d^2 \mathbf{x}}{dt^2} = \begin{bmatrix} -\frac{2k}{m_1} & \frac{k}{m_1} \\ \frac{k}{m_2} & -\frac{2k}{m_2} \end{bmatrix} \mathbf{x} \quad (13.19)$$

Referring to [Eq. \(13.17\)](#), we have that

$$\begin{vmatrix} -\frac{2k}{m_1} - \lambda & \frac{k}{m_1} \\ \frac{k}{m_2} & -\frac{2k}{m_2} - \lambda \end{vmatrix} = \lambda^2 + 2k\left(\frac{1}{m_1} + \frac{1}{m_2}\right)\lambda + \left(\frac{4k^2}{m_1m_2} - \frac{k^2}{m_1m_2}\right) \quad (13.20)$$

and we can solve for the eigenvalues as

$$\lambda = -k \frac{m_1 + m_2 \pm \sqrt{m_1^2 - m_1m_2 + m_2^2}}{m_1m_2} \quad (13.21)$$

From [Sec. 13.2.3](#), we can determine the frequencies as

$$\omega = \sqrt{k \frac{m_1 + m_2 \pm \sqrt{m_1^2 - m_1m_2 + m_2^2}}{m_1m_2}} \quad (13.22)$$

## EXAMPLE 13.4

### Interpretation of the Eigenvalues for the Spring-Mass System

#### Problem Statement

For  $m_1 = m_2 = 40$  kg and  $k = 200$  N/m, interpret the results for the two-mass, three-spring system.

#### Solution

Using [Eq. \(13.21\)](#), we can compute the eigenvalues as  $-5$  and  $-15$ . The corresponding angular frequencies are then

$$\omega_1 = \sqrt{5} \text{ rad/s}, \omega_2 = \sqrt{15} \text{ rad/s}$$

and the cycle frequencies (where Hz = cycles/s) are

$$f_1 = \frac{\sqrt{5}}{2\pi} \cong 0.356 \text{ Hz}, f_2 = \frac{\sqrt{15}}{2\pi} \cong 0.616 \text{ Hz}$$

The periods of the oscillations are

$$P_1 = \frac{1}{f_1} \cong 2.81 \text{ s}, P_2 = \frac{1}{f_2} \cong 1.62 \text{ s}$$

The eigenvectors for this system are

$$\mathbf{U} = \begin{bmatrix} \sqrt{2}/2 & -\sqrt{2}/2 \\ \sqrt{2}/2 & \sqrt{2}/2 \end{bmatrix} \text{ which can be scaled to } \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$$

If we follow the pattern of [Example 13.3](#), the general solution has the form

$$\mathbf{y} = \mathbf{x}_1(a_1\cos(\omega_1 t) + b_1\sin(\omega_1 t)) + \mathbf{x}_2(a_2\cos(\omega_2 t) + b_2\sin(\omega_2 t))$$

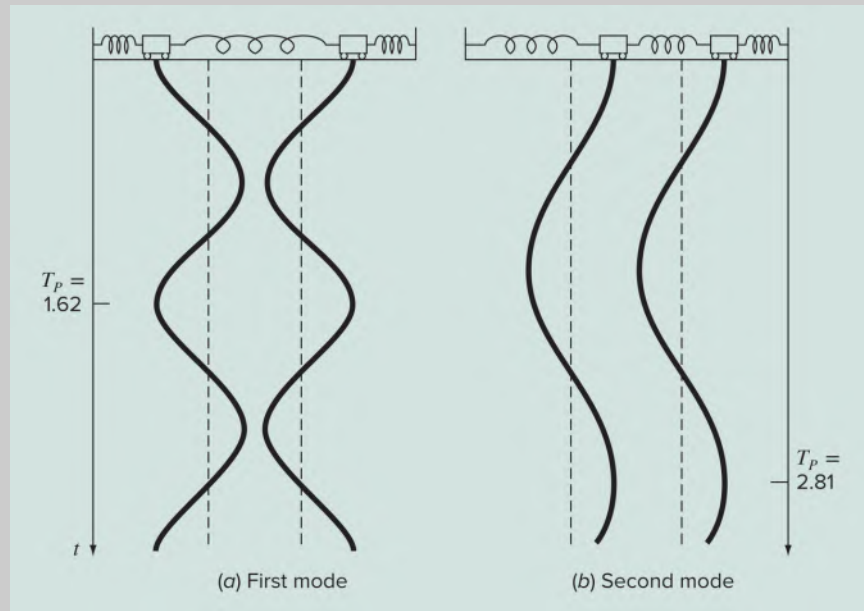
And with our eigenvectors, this becomes the two equations

$$\begin{aligned} y_1 &= (a_1\cos(\omega_1 t) + b_1\sin(\omega_1 t)) - (a_2\cos(\omega_2 t) + b_2\sin(\omega_2 t)) \\ y_2 &= (a_1\cos(\omega_1 t) + b_1\sin(\omega_1 t)) + (a_2\cos(\omega_2 t) + b_2\sin(\omega_2 t)) \end{aligned}$$

We recognize the first and second terms of each equation as two distinct frequencies or two different modes of vibration. Independent of the initial conditions, the first modes are “in sync” with each other, and the second modes, because of the sign change, are out of phase by one-half cycle,  $\pi$ , or  $180^\circ$ . This is illustrated in [Fig. 13.4](#). If we combine the two modes and choose the initial conditions judiciously, we can achieve the responses in the figure where in (a) only the first mode is seen and in (b) only the second mode.

**FIGURE 13.4** The principal modes of vibration of two equal masses connected by three identical springs between fixed walls.





## 13.4 THE POWER METHOD

The power method is an iterative approach that can be employed to determine the largest or dominant eigenvalue. With slight modification, it can also be employed to determine the smallest value. It has the additional benefit that the corresponding eigenvector is obtained as a by-product of the method. To implement the power method, the system being analyzed is expressed in the form

$$\mathbf{Ax} = \lambda\mathbf{x} \quad (13.23)$$

As illustrated by the following example, [Eq. \(13.23\)](#) forms the basis for an iterative solution technique that eventually yields the highest eigenvalue and its associated eigenvector.

### EXAMPLE 13.5

#### Power Method for Highest Eigenvalue

##### Problem Statement

Using the same approach as in [Sec. 13.3](#), we can derive the following homogeneous set of equations for a three mass–four spring system between two fixed walls:

$$\begin{aligned} \left(\frac{2k}{m_1} - \omega^2\right)X_1 - \frac{k}{m_1}X_2 &= 0 \\ -\frac{k}{m_2}X_1 + \left(\frac{2k}{m_2} - \omega^2\right)X_2 - \frac{k}{m_2}X_3 &= 0 \\ -\frac{k}{m_3}X_2 + \left(\frac{k}{m_3} - \omega^2\right)X_3 &= 0 \end{aligned}$$

If all the masses  $m = 1$  kg and all the spring constants  $k = 20$  N/m, the system can be expressed in the matrix format of [Eq. \(13.2\)](#) as

Page 294

$$\begin{bmatrix} 40 & -20 & 0 \\ -20 & 40 & -20 \\ 0 & -20 & 40 \end{bmatrix} - \lambda\mathbf{I} = 0$$

where the eigenvalue  $\lambda$  is the square of the angular frequency  $\omega^2$ . Employ the power method to determine the highest eigenvalue and its associated eigenvector.

## Solution

The system is first written in the form of [Eq. \(13.23\)](#):

$$\begin{aligned} 40X_1 - 20X_2 &= \lambda X_1 \\ -20X_1 + 40X_2 - 20X_3 &= \lambda X_2 \\ -20X_2 + 40X_3 &= \lambda X_3 \end{aligned}$$

At this point, we can make initial values of the  $X$ 's and use the left-hand side to compute an eigenvalue and eigenvector. A good first choice is to assume that all the  $X$ 's on the left-hand side of the equation are equal to one:

$$\begin{aligned} 40(1) - 20(1) &= 20 \\ -20(1) + 40(1) - 20(1) &= 0 \\ -20(1) + 40(1) &= 20 \end{aligned}$$

Next, the right-hand side is normalized by 20 to make the largest element equal to one:

$$\begin{bmatrix} 20 \\ 0 \\ 20 \end{bmatrix} = 20 \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

Thus, the normalization factor is our first estimate of the eigenvalue (20) and the corresponding eigenvector is  $[1 \ 0 \ 1]^T$ . This iteration can be expressed concisely in matrix form as

$$\begin{bmatrix} 40 & -20 & 0 \\ -20 & 40 & -20 \\ 0 & -20 & 40 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 20 \\ 0 \\ 20 \end{bmatrix} = 20 \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

The next iteration consists of multiplying the matrix by the eigenvector from the last iteration,  $[1 \ 0 \ 1]^T$  to give

$$\begin{bmatrix} 40 & -20 & 0 \\ -20 & 40 & -20 \\ 0 & -20 & 40 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 40 \\ -40 \\ 40 \end{bmatrix} = 40 \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}$$

Therefore, the eigenvalue estimate for the second iteration is 40, which can be employed to determine an error estimate:

$$\varepsilon_a = \left| \frac{40 - 20}{40} \right| \times 100\% = 50\%$$

The process can then be repeated.

Third iteration:

$$\begin{bmatrix} 40 & -20 & 0 \\ -20 & 40 & -20 \\ 0 & -20 & 40 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 60 \\ -80 \\ 60 \end{bmatrix} = -80 \begin{bmatrix} -0.75 \\ 1 \\ -0.75 \end{bmatrix}$$

where  $\varepsilon_a = 150\%$  (which is high because of the sign change).

Fourth iteration:

$$\begin{bmatrix} 40 & -20 & 0 \\ -20 & 40 & -20 \\ 0 & -20 & 40 \end{bmatrix} \begin{bmatrix} -0.75 \\ 1 \\ -0.75 \end{bmatrix} = \begin{bmatrix} -50 \\ 70 \\ -50 \end{bmatrix} = 70 \begin{bmatrix} -0.71429 \\ 1 \\ -0.71429 \end{bmatrix}$$

where  $\varepsilon_a = 214\%$  (another sign change).

Fifth iteration:

$$\begin{bmatrix} 40 & -20 & 0 \\ -20 & 40 & -20 \\ 0 & -20 & 40 \end{bmatrix} \begin{bmatrix} -0.71429 \\ 1 \\ -0.71429 \end{bmatrix} = \begin{bmatrix} -48.51714 \\ 68.51714 \\ -48.51714 \end{bmatrix} = 68.51714 \begin{bmatrix} -0.70833 \\ 1 \\ -0.70833 \end{bmatrix}$$

where  $\varepsilon_a = 2.08\%$ .

Thus, the eigenvalue is converging. After several more iterations, it stabilizes on a value of 68.28427 with a corresponding eigenvector of  $[-0.707107 \ 1 \ -0.707107]^T$ .

Note that there are some instances where the power method will converge to the second-largest eigenvalue instead of to the largest. James, Smith, and Wolford (1985) provide an illustration of such a case. Other special cases are discussed in Fadeev and Fadeeva (1963).

In addition, there are sometimes cases where we are interested in determining the smallest eigenvalue. This can be done by applying the power method to the matrix inverse of  $\mathbf{A}$ . For this case, the power method

will converge on the largest value of  $1/\lambda$ —in other words, the smallest value of  $\lambda$ . An application to find the smallest eigenvalue will be left as a problem exercise.

Finally, after finding the largest eigenvalue, it is possible to determine the next highest by replacing the original matrix by one that includes only the remaining eigenvalues. The process of removing the largest known eigenvalue is called deflation.

We should mention that although the power method can be used to locate intermediate values, better methods are available for cases where we need to determine all the eigenvalues as described in the next section. Thus, the power method is primarily used when we want to locate the largest or the smallest eigenvalue.

## 13.5 PYTHON NUMPY FUNCTIONS: eig AND eigvals

Python has powerful and robust capabilities for evaluating eigenvalues and eigenvectors. The function `eigvals` in the NumPy module can be employed to generate an array of eigenvalues as in

```
import numpy as np
e = linalg.eigvals(A)
```

where  $A$  is a square matrix. Alternately, the `eig` function can be invoked as

```
e, v = linalg.eig(A)
```

where  $v$  is a square array whose columns are the corresponding eigenvectors.

It should be noted that the `eig` function normalizes the eigenvectors by dividing them by their Euclidean distance (square root of the sum of squares). Thus, as shown in the following example, although their magnitude may be different from values computed with the power method, the relative magnitudes of their elements will be the same.

### EXAMPLE 13.6

#### Eigenvalues and Eigenvectors with Python

##### Problem Statement

Use Python to determine the eigenvalues and eigenvectors of the system described in

 [Example 13.5](#).

##### Solution

Recall that the matrix to be analyzed is

$$\begin{bmatrix} 40 & -20 & 0 \\ -20 & 40 & -20 \\ 0 & -20 & 40 \end{bmatrix}$$

The matrix can be entered as an *ndarray* in Python,

```
import numpy as np
A = np.array([[40., -20., 0.], [-20., 40., -20.], [0., -20., 40.]])
```

If we just want to determine the eigenvalues, we can enter

```
np.set_printoptions(precision=5)
print(np.linalg.eigvals(A))
```

and the result displayed is

```
[68.28427 40. 11.71573]
```

Note that the largest eigenvalue is consistent with the value previously determined with the power method in [Example 13.5](#).

If we want both the eigenvalues and eigenvectors, we can add the statements

```
e,v = np.linalg.eig(A)
print('Eigenvalues are\n',e)
print('\nEigenvector matrix is\n',v)
```

and the display is

```
Eigenvalues are
[68.28427 40. 11.71573]

Eigenvector matrix is
[[-5.00000e-01 -7.07107e-01 5.00000e-01]
 [ 7.07107e-01 -1.28296e-16 7.07107e-01]
 [-5.00000e-01 7.07107e-01 5.00000e-01]]
```

Although the results are scaled differently from those of [Example 13.5](#), the eigenvector corresponding to the largest eigenvalue,  $[-0.5 \ 0.7071 \ -0.5]^T$ , is consistent with the result from the power method,  $[-0.7071 \ 1 \ -0.7071]^T$ . This can be demonstrated by dividing the eigenvector from the power method by its Euclidean norm

```
e_power = np.array([-np.sqrt(2)/2, 1., -np.sqrt(2)/2])
print(e_power)
e_power_norm = np.linalg.norm(e_power)
print(e_power/e_power_norm)

[-0.70711  1. -0.70711]
[-0.5   0.70711 -0.5   ]
```

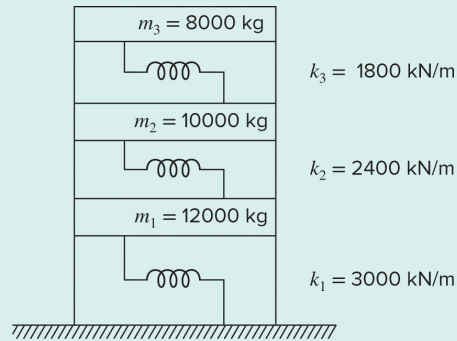
Although the magnitudes of the elements differ, their ratios are identical. Another way of stating this is that the two vectors point in the same direction in three-dimensional space.

## EIGENVALUES AND EARTHQUAKES

### Background

Engineers and scientists use mass-spring models to gain insight into the dynamics of structures under the influence of disturbances such as earthquakes. [Figure 13.5](#) shows such a model for a three-story building. Each floor mass is represented by  $m_i$ , and each floor stiffness is represented by  $k_i$  for  $i = 1$  to 3.

**FIGURE 13.5** A three-story building modeled as a mass-spring system.



For this case, the analysis is limited to horizontal motion of the structure as it is subjected to horizontal base motion due to earthquakes. Using the same approach as in [Sec. 13.3](#), dynamic force balances can be developed for this system as

$$\begin{aligned} m_1 \frac{d^2 x_1}{dt^2} &= -k_1 x_1 + k_2 (x_2 - x_1) \\ m_2 \frac{d^2 x_2}{dt^2} &= -k_2 (x_2 - x_1) + k_3 (x_3 - x_2) \\ m_3 \frac{d^2 x_3}{dt^2} &= -k_3 (x_3 - x_2) \end{aligned}$$

This system can be analyzed, similar to that of [Sec. 13.3](#), by solving the eigenvalue problem in terms of the square of the natural frequencies,  $\omega_n^2$ ,

$$\begin{bmatrix} \frac{k_1 + k_2}{m_1} - \omega_n^2 & -\frac{k_2}{m_1} & 0 \\ -\frac{k_2}{m_2} & \frac{k_2 + k_3}{m_2} - \omega_n^2 & -\frac{k_3}{m_2} \\ 0 & -\frac{k_3}{m_3} & \frac{k_3}{m_3} - \omega_n^2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = 0$$



where the  $x_i$  represent floor translations (m), and  $\omega_n$  is the natural or resonant frequency (radians/s). The resonant frequency is commonly expressed in Hertz ( $\text{Hz} = \text{cycles/s}$ ) and can be obtained from  $\omega_n$  by dividing by  $2\pi$  radians/cycle.

## Problem Statement

Use Python to determine the eigenvalues and eigenvectors for this system. Graphically represent the modes of vibration for the structure by displaying the amplitudes versus height for each of the eigenvectors. Normalize the amplitudes so that the translation of the third floor is one.

## Solution

The parameter values from  **Fig. 13.5** can be substituted into the force balances to give

$$\begin{bmatrix} 450 - \omega_n^2 & -200 & 0 \\ -240 & 420 - \omega_n^2 & -180 \\ 0 & -225 & 225 - \omega_n^2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = 0$$

A Python script can be written to evaluate the eigenvalues and eigenvectors as follows:

```
import numpy as np
A = np.array([[450.,-200.,0.],[-240.,420.,-180.],[0.,-225.,225.]])
e,v = np.linalg.eig(A)
print('\nEigenvalues are:\n',e)
print('\nEigenvectors are:\n',v)
```

with the results displayed as

```
Eigenvalues are:
[698.59819 339.47789 56.92392]

Eigenvectors are:
[[-0.58785 -0.63436 0.2913 ]
 [ 0.7307 -0.35055 0.57251]
 [-0.34714 0.68899 0.76641]]
```

Based on the eigenvalues, the resonant frequencies are

```
wn = np.sqrt(e)/2/np.pi
print('\nResonant frequencies in Hz:\n',wn)
Resonant frequencies in Hz:
[4.20663 2.93242 1.20079]
```

The corresponding eigenvectors can be scaled so that the amplitude for the third floor is one.

```
vt = np.zeros((3,3))
for i in range(3):
    vt[:,i] = v[:,i]/v[2,i]

print('\nScaled eigenvectors are:\n',vt)

Scaled eigenvectors are:
[[ 1.6934 -0.9207 0.38008]
 [-2.10488 -0.50879 0.747 ]
 [ 1.      1.      1.     ]]
```

Following  **Example 13.5**, we see that the general solution can be written as

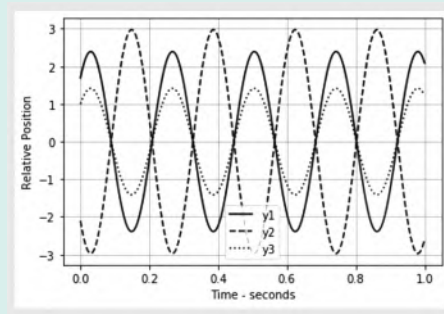
$$\begin{aligned}\mathbf{x} = & \mathbf{u}_1(a_1\cos(\omega_1t) + b_1\sin(\omega_1t)) \\ & + \mathbf{u}_2(a_2\cos(\omega_2t) + b_2\sin(\omega_2t)) \\ & + \mathbf{u}_3(a_3\cos(\omega_3t) + b_3\sin(\omega_3t))\end{aligned}$$

where the  $\mathbf{u}$ 's are the eigenvectors. By specifying initial conditions for  $\mathbf{x}$  and  $d\mathbf{x}/dt$ , the six  $a$  and  $b$  coefficients can be determined, and the specific solution is found. On the other hand, if we wanted to study just the first frequency or mode, we could determine the initial conditions that would zero out  $a_2$ ,  $b_2$ ,  $a_3$ , and  $b_3$ . This would leave us with

$$\mathbf{x} = \mathbf{u}_1(a_1\cos(\omega_1t) + b_1\sin(\omega_1t))$$

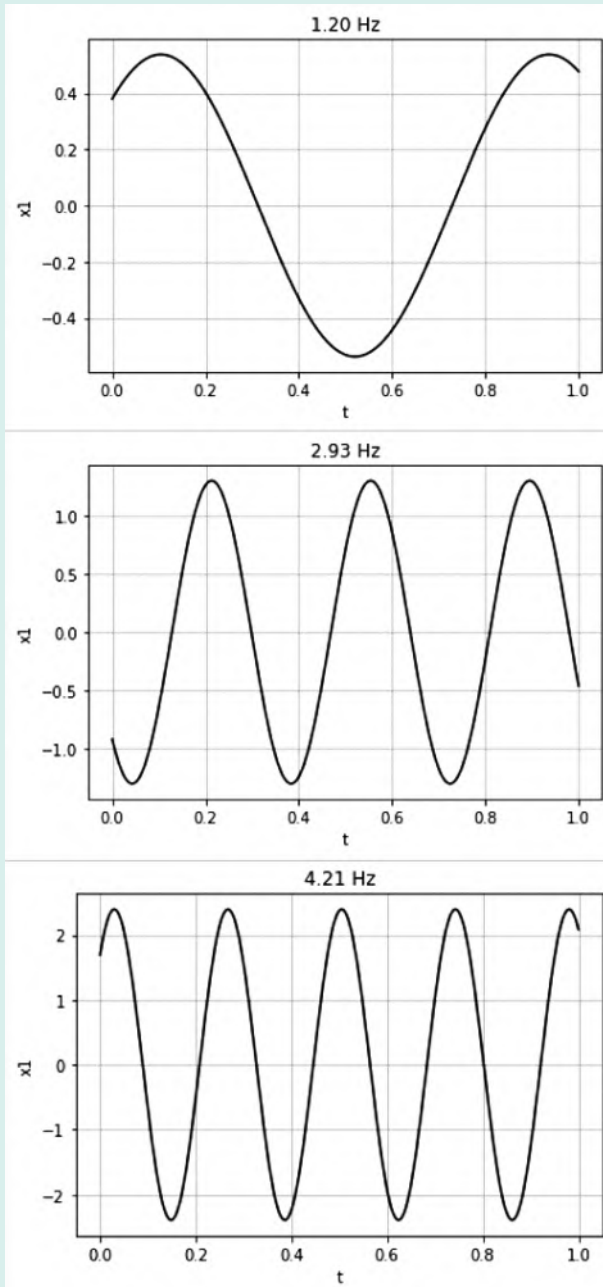
where  $a_1$  and  $b_1$  can be arbitrarily set to one. That says that the relative amplitude of the oscillations for  $x_1$ ,  $x_2$ , and  $x_3$  is determined by the values of the eigenvector corresponding to  $\omega_1$ . A plot of this solution for the time range,  $0 \leq t \leq 1$  s, is shown in  **Fig. 13.6**.

**FIGURE 13.6**



A graph can also be constructed to depict the three modes ( **Fig. 13.7**). Note that we have ordered them from the lowest to the highest natural frequency as is customary in structural engineering. This graph is for the  $x_1$  component.

**FIGURE 13.7** The three primary modes of vibration of the three-story building.



Natural frequencies and mode shapes are characteristics of structures in terms of their tendencies to resonate at these frequencies. The frequency content of an earthquake's vibrations typically has the most energy between 0 and 20 Hz and is influenced by the earthquake's magnitude, the distance from the epicenter, and other factors. Rather than vibrate at a single frequency, they contain a spectrum of frequencies with varying strengths or amplitudes. Buildings are more susceptible to vibration at their lower modes due to their simpler deformed shapes and requiring less strain energy to deform in the lower modes. When the earthquake's strong frequencies are close to the natural frequencies of buildings, large, destructive dynamic responses are induced, creating large stresses and strains in the structure's beams, columns, and foundations. Based on analyses like the simple one in

this case study, structural engineers can design buildings more wisely to withstand earthquakes with a good safety factor.

## PROBLEMS

**13.1** For the following symmetrix matrix,

$$\begin{bmatrix} 20 & 3 & 2 \\ 3 & 9 & 4 \\ 2 & 4 & 12 \end{bmatrix}$$

- (a) Determine the eigenvalues from the characteristic polynomial.
- (b) Use the power method to find the largest eigenvalue and compare this with the results from (a). Perform three iterations by hand.
- (c) Use the power method to find the smallest eigenvalue and compare this with the results from (a). Perform three iterations by hand.

**13.2** Repeat  **Example 13.1** but for three masses with the  $m/s = 40$  kg, and the  $k/s = 240$  N/m. Produce a plot like  **Fig. 13.4** to identify the principal modes of vibration.

**13.3** Use the power method to determine the largest eigenvalue and corresponding eigenvector for

$$\begin{bmatrix} 2 - \lambda & 8 & 10 \\ 8 & 4 - \lambda & 5 \\ 10 & 5 & 7 - \lambda \end{bmatrix}$$

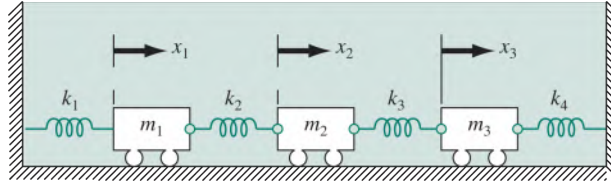
**13.4** Use the power method to determine the smallest eigenvalue and corresponding eigenvector for the system of  **Prob. 13.3**.

**13.5** Derive the set of differential equations for a three-mass, four-spring system ( **Fig. P13.5**) that describes their time motion. Write the three differential equations in matrix form

$$\begin{bmatrix} \text{acceleration} \\ \text{vector} \end{bmatrix} + \begin{bmatrix} \text{k/m} \\ \text{matrix} \end{bmatrix} \begin{bmatrix} \text{displacement} \\ \text{vector} \end{bmatrix} = 0$$

Note that each equation has been divided by the mass to leave the acceleration term as a second derivative alone. Solve for the eigenvalues and natural frequencies for the following values of masses and spring constants:  $k_1 = k_4 = 15$  N/m,  $k_2 = k_3 = 35$  N/m, and  $m_1 = m_2 = m_3 = 1.5$  kg.

**FIGURE P13.5**



**13.6** Consider the mass-spring system in  Fig. P13.6. By applying the vector/matrix equation,  $\mathbf{M}\ddot{\mathbf{x}} + \mathbf{K}\mathbf{x} = 0$ , one can obtain

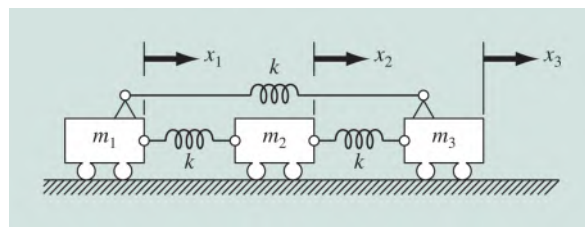
$$\begin{bmatrix} m_1 & 0 & 0 \\ 0 & m_2 & 0 \\ 0 & 0 & m_3 \end{bmatrix} \begin{bmatrix} \ddot{x}_1 \\ \ddot{x}_2 \\ \ddot{x}_3 \end{bmatrix} + \begin{bmatrix} 2k & -k & -k \\ -k & 2k & -k \\ -k & -k & 2k \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Applying the trial solution,  $x_i = x_{i0}e^{i\omega t}$ , we get the following matrix:

$$\begin{bmatrix} 2k - m_1\omega^2 & -k & -k \\ -k & 2k - m_2\omega^2 & -k \\ -k & -k & 2k - m_3\omega^2 \end{bmatrix} \begin{bmatrix} x_{01} \\ x_{02} \\ x_{03} \end{bmatrix} e^{i\omega t} = 0$$

Use Python's `eigvals` function to solve for the eigenvalues of the matrix above. Then, from the eigenvalues, solve for the frequencies ( $\omega$ ) in Hz. Let  $m_1 = m_2 = m_3 = 1$  kg and  $k = 2$  N/m.

**FIGURE P13.6**



**13.7** A viscous fluid flows by gravity through a system of three, standard, 55-gal. drums, as shown in  Fig. P13.7. The relationship of flow rate out of each tank to the level in the tank is linear with a proportionality constant,  $K$ . Three volume-balance, differential equations are written to describe this system:

$$\begin{aligned}
 A \frac{dh_1}{dt} &= q_0 - q_1 & q_1 &= kh_1 \\
 A \frac{dh_2}{dt} &= q_1 - q_2 & q_2 &= kh_2 \\
 A \frac{dh_3}{dt} &= q_2 - q_3 & q_3 &= kh_3
 \end{aligned}$$

- (a) Introduce the flow rate–level relationships into the differential equations, and rearrange the equations into matrix form as

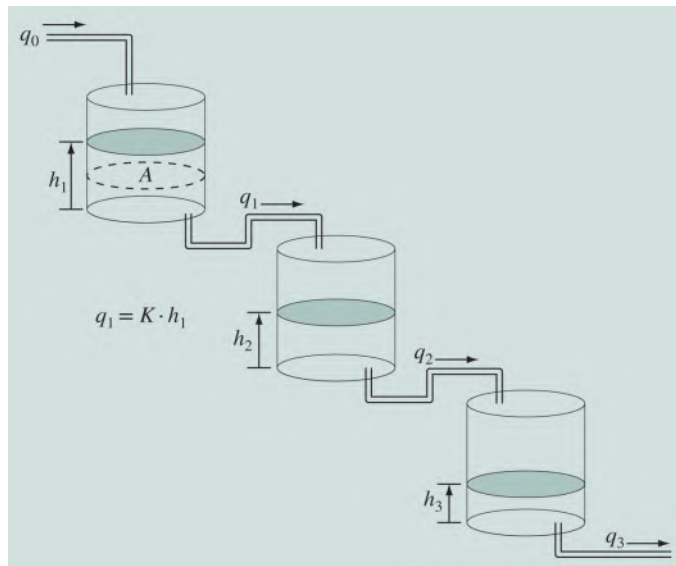
$$\frac{d\mathbf{h}}{dt} = \mathbf{BH} + \mathbf{b}$$

where the  $\mathbf{b}$  vector contains the input term with the inlet flow rate,  $q_0$ . By setting the derivatives equal to zero, solve for the steady-state levels and flow rates, assuming a value of  $q_0 = 20 \text{ L/min}$ .

- (b) For the following parameter values, determine the eigenvalues of this system and comment qualitatively on the system's dynamic behavior.

$$A = 0.21 \text{ m}^2, K = 42 \frac{\text{L/min}}{\text{m}}$$

**FIGURE P13.7** A viscous liquid flowing through three drums.

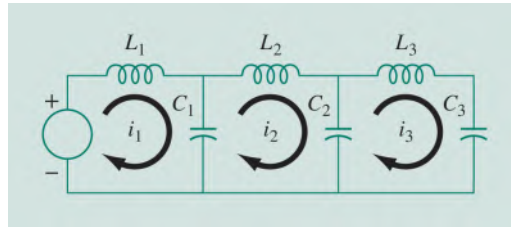


**13.8** As displayed in  **Fig. P13.8**, an LC electrical circuit can be modeled by following system of differential equations describing the currents in the three loops shown in the figure:

$$\begin{aligned} L_1 \frac{d^2 i_1}{dt^2} + \frac{1}{C_1} (i_1 - i_2) &= 0 \\ L_2 \frac{d^2 i_2}{dt^2} + \frac{1}{C_2} (i_2 - i_3) - \frac{1}{C_1} (i_1 - i_2) &= 0 \\ L_3 \frac{d^2 i_3}{dt^2} + \frac{1}{C_3} i_3 - \frac{1}{C_2} (i_2 - i_3) &= 0 \end{aligned}$$

where  $L$  = the coil inductance (henries, H),  $t$  = time (seconds, s),  $i$  = current (amperes, A), and  $C$  = capacitance (farads, F). Assuming a solution is of the form  $i_j = I_j \sin(\omega t)$ , determine the eigenvalues and eigenvectors for this system with all  $L$ 's = 1 H, and all  $C$ 's = 0.25 F. Draw the network, indicating how the currents oscillate in their primary modes.

**FIGURE P13.8**



**13.9** Repeat  **Prob. 13.8**, but with only the first two loops. Draw the network, illustrating how the currents oscillate in their primary mode.

**13.10** Control engineers use dynamic, differential equations models to study the behavior of systems and processes and design stable, high-performing control systems. Traditionally, the Laplace transform is used to produce algebraic relationships between input and output variables. These are called transfer functions and are ratios of polynomials in the Laplace complex variable,  $s$ , which has replaced the time independent variable via the transform. The denominator polynomial of a transfer function is equivalent to the characteristic polynomial of the differential equation, so its roots are the eigenvalues of the system modeled. An example transfer function for a chemical process is

$$\frac{s + 1.7}{s^2 - 0.9s - 0.7}$$

- (a) What are the eigenvalues of this system and how would you characterize its behavior?
- (b) By applying feedback control to this system, the transfer function is modified to the form



$$\frac{K \frac{s + 1.7}{s^2 - 0.9s - 0.7}}{1 + K \frac{s + 1.7}{s^2 - 0.9s - 0.7}}$$

and, after consolidation to a rational polynomial, the denominator polynomial is modified with new associated eigenvalues a function of the choice of  $K$ , called the feedback gain. Use Python to investigate the change in the eigenvalues for a range of  $K$  values and plot the location of the eigenvalues on the complex plane. Comment qualitatively on the transitions in system behavior.

**13.11** Repeat the problem from the case study described in  **Sec. 13.6**, but leave off the third floor.

**13.12** The curvature of a slender column subject to an axial load,  $P$  ( **Fig. P13.12**), can be modeled by

$$\frac{d^2y}{dx^2} + p^2y = 0$$

where

$$p^2 = \frac{P}{EI}$$

Here,  $E$  = modulus of elasticity and  $I$  = moment of inertia of the cross section about its neutral axis. This model can be converted into an eigenvalue problem by substituting a centered finite-difference approximation for the second derivative to give

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{\Delta x^2} + p^2y_i = 0$$

where  $i$  = a node located at a position along the rod's interior and  $\Delta x$  = the spacing between nodes. This equation can be rearranged as

$$y_{i-1} - (2 - \Delta x^2 p^2)y_i + y_{i+1} = 0$$

Writing this equation for a series of interior nodes along the axis of the column yields a homogeneous system of equations. For example, if the column is divided into five segments, that is, four interior nodes, the result is

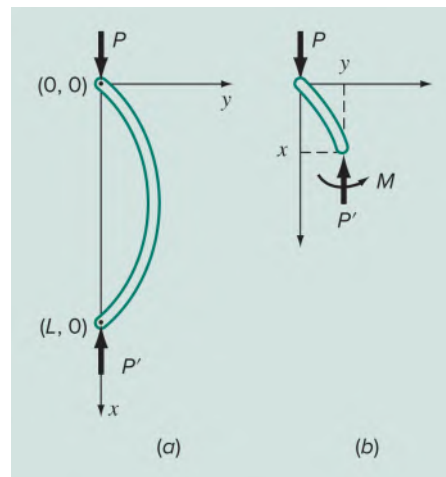
$$\begin{bmatrix} (2 - \Delta x^2 p^2) & -1 & 0 & 0 \\ -1 & (2 - \Delta x^2 p^2) & -1 & 0 \\ 0 & -1 & (2 - \Delta x^2 p^2) & -1 \\ 0 & 0 & -1 & (2 - \Delta x^2 p^2) \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = 0$$

An axially loaded wooden column has the following characteristics:

$E = 1 \times 10^{10}$  Pa,  $I = 1.25 \times 10^{-5}$  m<sup>4</sup>, and  $L = 3$  m. For the five-segment, four-node representation:

- Implement the polynomial method with Python to determine the eigenvalues of the system.
- Use the Python `eig` function to determine the eigenvalues and eigenvectors.
- Use the power method to determine the largest eigenvalue and its corresponding eigenvector. Does your answer agree with parts (a) and (b)? If not, can you adjust the method to achieve agreement?

**FIGURE P13.12** (a) A slender rod and (b) a free-body diagram of a rod.



**13.13** A system of two homogeneous linear ordinary differential equations with constant coefficients is written as

$$\begin{aligned} \frac{dy_1}{dt} &= -5y_1 + 3y_2 & y_1(0) &= 50 \\ \frac{dy_2}{dt} &= 100y_1 - 301y_2 & y_2(0) &= 100 \end{aligned}$$

If you have taken a course in differential equations, you know that the solutions for such equations have the form  $e^{\lambda t}$ . Substituting this solution and its derivative into the original equations convert the system into an eigenvalue problem. The resulting eigenvalues and eigenvectors can then be used to derive the

general solution to the differential equations. For example, for the two-equation case, the general solution can be written in terms of vectors as

$$\mathbf{y} = c_1 \mathbf{v}_1 e^{\lambda_1 t} + c_2 \mathbf{v}_2 e^{\lambda_2 t}$$

where  $\mathbf{v}_i$  = the eigenvector corresponding to the  $i$ th eigenvalue,  $\lambda_i$ , and the  $c$ 's are unknown coefficients that can be determined with the initial conditions.

- (a) Convert the system into an eigenvalue problem.
- (b) Use Python to solve for the eigenvalues and eigenvectors. Based on the eigenvalues, what do you expect the character of the solution to be?
- (c) Employ the results from (b) and the given initial conditions to determine the general solution.
- (d) Use Python to produce a plot of the solution over the range,  $0 \leq t \leq 1$ .

**13.14** Water flows between the North American Great Lakes as depicted in  **Fig. P13.14**. Based on mass balances, the following differential equations can be written for the concentrations in each of the lakes for a pollutant that decays with first-order kinetics:

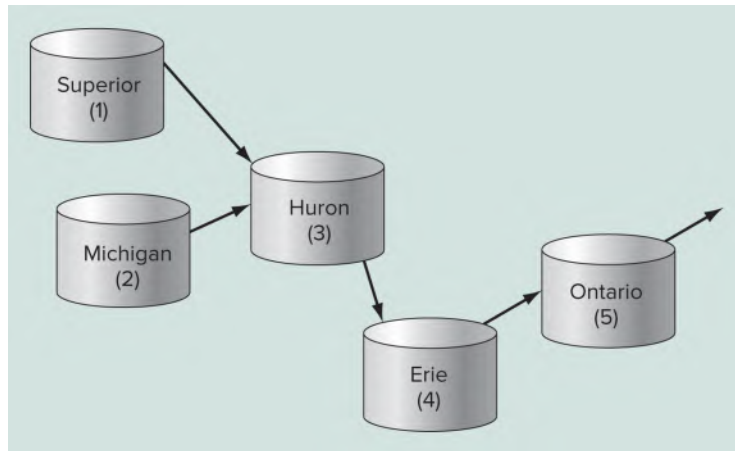
$$\begin{aligned}\frac{dc_1}{dt} &= -(0.0056 + k)c_1 \\ \frac{dc_2}{dt} &= -(0.01 + k)c_2 \\ \frac{dc_3}{dt} &= 0.01902c_1 + 0.01387c_2 - (0.047 + k)c_3 \\ \frac{dc_4}{dt} &= 0.33597c_3 - (0.376 + k)c_4 \\ \frac{dc_5}{dt} &= 0.11364c_4 - (0.133 + k)c_5\end{aligned}$$

where  $k$  = a first-order decay rate ( $\text{yr}^{-1}$ ), which is equal to  $0.69315/\text{half-life}$ . Note that the constants in the equations account for the flow rates between the lakes and their respective volumes. As a result of testing nuclear weapons in the atmosphere in the decade following WW-II, the concentrations of strontium-90 ( $^{90}\text{Sr}$ ) in the five lakes in 1963 were approximately

$$[17.7, 30.5, 43.9, 136.3, 30.1] \text{ in units of Bq/m}^3.$$

Assuming that no additional  $^{90}\text{Sr}$  entered the system afterwards, use Python and the approach outlined in  **Prob. 13.13** to compute the concentrations in each of the lakes from 1963 through 2020. Note that  $^{90}\text{Sr}$  has a half-life of 28.8 years.

**FIGURE P13.14** The North American Great Lakes. The arrows indicate how water flows between the lakes.



**13.15** Develop a Python function to determine the largest eigenvalue and its associated eigenvector using the power method. Test the function by duplicating [Example 13.5](#) and then use it to solve [Prob. 13.3](#).

**13.16** Recall that, at the start of [Chap. 8](#), we suspended three bungee jumpers with zero drag from frictionless cords that followed Hooke's law. Determine the resulting eigenvalues and eigenvectors that would eventually characterize the jumpers' oscillating motions and relative positions if they were instantaneously released from their starting positions as depicted in [Fig. 8.1a](#), that is, with each cord fully extended, but not stretched. Although the bungee cords do not actually behave like ideal springs, assume that they stretch and compress in linear proportion to the applied force. Use the parameters from [Example 8.2](#).

## Curve Fitting

### 4.1 OVERVIEW

---

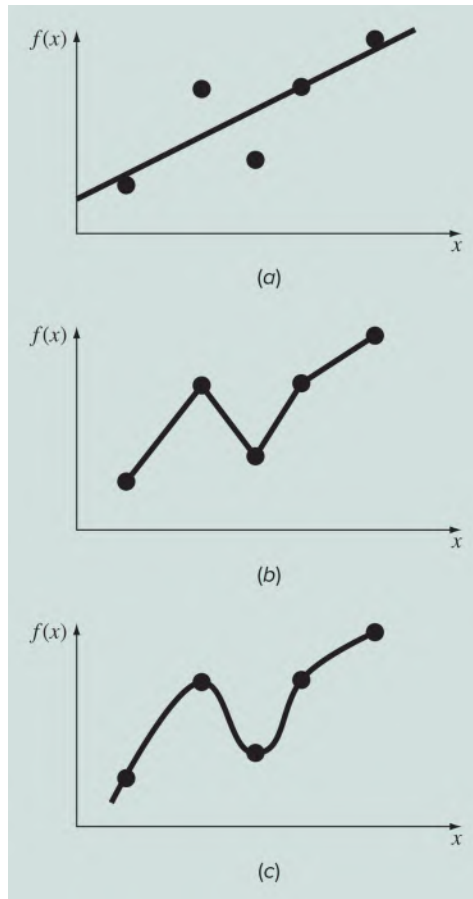
#### What Is Curve Fitting?

Data are often given for discrete values along a continuum. However, you may require estimates at points between the discrete values.  Chapters 14 through  18 describe techniques to fit curves to such data to obtain intermediate estimates. In addition, you may require a simplified version of a complicated function. One way to do this is to compute values of the function at a number of discrete values along the range of interest. Then, a simpler function may be derived to fit these values. Both of these applications are known as *curve fitting*.

There are two general approaches for curve fitting that are distinguished from each other on the basis of the amount of error associated with the data. First, where the data exhibit a significant degree of error or “scatter,” the strategy is to derive a single curve that represents the general trend of the data. Because any individual data point may be incorrect, we make no effort to intersect every point. Rather, the curve is designed to follow the pattern of the points taken as a group. One approach of this nature is called *least-squares regression* ( Fig. PT4.1a).

---

**FIGURE PT4.1** Three attempts to fit a “best” curve through five data points: (a) least-squares regression, (b) linear interpolation, and (c) curvilinear interpolation.

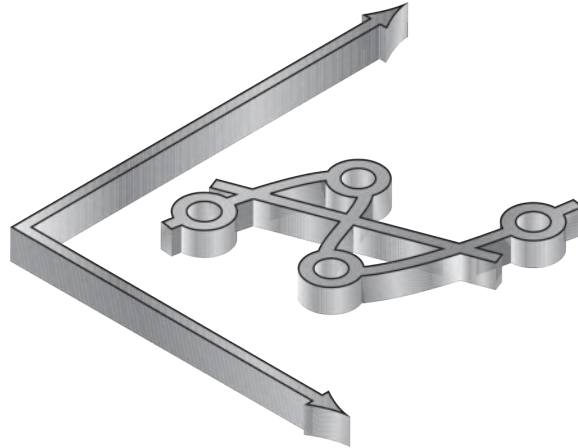


Second, where the data are known to be very precise, the basic approach is to fit a curve or a series of curves that pass directly through each of the points. Such data usually originate from property tables. Examples are values for the density of water or for the heat capacity of gases as a function of temperature. The estimation of values between well-known discrete points is called *interpolation* ([Fig. PT4.1b](#) and [c](#)).

## Curve Fitting and Engineering and Science

Your first exposure to curve fitting may have been to determine intermediate values from tabulated data—for instance, from interest tables for engineering economics or from steam tables for thermodynamics. Throughout the remainder of your career, you will have frequent occasion to estimate intermediate values from such tables.

Although many of the widely used engineering and scientific properties have been tabulated, there are a great many more that are not available in this convenient form. Special cases and new problem contexts often require that you measure your own data and develop your own predictive relationships. Two types of applications are generally encountered when fitting experimental data: trend analysis and hypothesis testing.



*Trend analysis* represents the process of using the pattern of the data to make predictions. For cases where the data are measured with high precision, you might utilize interpolating polynomials. Imprecise data are often analyzed with least-squares regression.

Trend analysis may be used to predict or forecast values of the dependent variable. This can involve extrapolation beyond the limits of the observed data or interpolation within the range of the data. All fields of engineering and science involve problems of this type.

A second application of experimental curve fitting is *hypothesis testing*. Here, an existing mathematical model is compared with measured data. If the model coefficients are unknown, it may be necessary to determine values that best fit the observed data. On the other hand, if estimates of the model coefficients are already available, it may be appropriate to compare predicted values of the model with observed values to test the adequacy of the model. Often, alternative models are compared and the “best” one is selected on the basis of empirical observations.

---

Page 304

In addition to the foregoing engineering and scientific applications, curve fitting is important in other numerical methods such as integration and the approximate solution of differential equations. Finally, curve-fitting techniques can be used to derive simple functions to approximate complicated functions.

## 4.2 PART ORGANIZATION

---

After a brief review of statistics, [Chap. 14](#) focuses on straight-line *linear regression*; that is, how to determine the “best” straight line through a set of uncertain data points. Besides discussing how to calculate the slope and intercept of this straight line, we also present quantitative and visual methods for evaluating the validity of the results. In addition, we describe *random number generation* as well as several approaches for the linearization of nonlinear equations.

[Chapter 15](#) begins with brief discussions of polynomial and multiple linear regression. *Polynomial regression* deals with developing a best fit of parabolas, cubics, or higher-order polynomials. This is followed by a description of *multiple linear regression*, which is designed for the case where the dependent variable  $y$  is a linear function of two or more independent variables  $x_1, x_2, \dots, x_m$ . This approach has special utility for evaluating experimental data where the variable of interest is dependent on a number of different factors.

After multiple regression, we illustrate how polynomial and multiple regression are both subsets of a *general linear least-squares model*. Among other things, this will allow us to introduce a concise matrix representation of regression and discuss its general statistical properties. Finally, the last sections of [Chap. 15](#) are devoted to *nonlinear regression*. This approach is designed to compute a least-squares fit of an equation nonlinear in its parameters to data.

---

Page 305

[Chapter 16](#) deals with *Fourier analysis* which involves fitting periodic functions to data. Our emphasis will be on the *fast Fourier transform* or *FFT*. This method, which is readily implemented with Python, has many engineering applications, ranging from vibration analysis of structures to signal processing.

In [Chap. 17](#), the alternative curve-fitting technique called *interpolation* is described. As discussed previously, interpolation is used for estimating intermediate values between precise data points. In [Chap. 17](#), polynomials are derived for this purpose. We introduce the basic concept of polynomial interpolation by using straight lines and parabolas to connect points. Then, we develop a generalized procedure for fitting an  $n$ th-order polynomial. Two formats are presented for expressing these polynomials in equation form. The first, called *Newton's interpolating polynomial*, is preferable when the appropriate order of the polynomial is unknown. The second, called the *Lagrange interpolating polynomial*, has advantages when the proper order is known beforehand.

Finally, [Chap. 18](#) presents an alternative technique for fitting precise data points. This technique, called *spline interpolation*, fits polynomials to data but in a piecewise fashion. As such, it is particularly well suited for fitting data that are generally smooth but exhibit abrupt local changes. An overview of how piecewise interpolation is implemented in Python. The chapter concludes with an introduction to two important data smoothing techniques: cubic spline smoothing and LOESS. Implementation via Python functions is presented.



# Straight-Line Linear Regression

## CHAPTER OBJECTIVES

The primary objective of this chapter is to introduce you to how least-squares regression is applied to fit a straight line through data.

- Familiarizing yourself with some basic descriptive statistics and the normal distribution.
- Knowing how to compute the slope and intercept of a best-fit straight line.
- Learning a technique for scribing a straight line through data without computations.
- Knowing how to generate random numbers with Python and how they can be employed for Monte Carlo simulations.
- Knowing how to compute and understand the meaning of the coefficient of determination and the standard error of the estimate.
- Learning how to make initial assessments of residual errors for model adequacy.
- Learning how to estimate confidence intervals on the slope and intercept and use these to determine model validity.
- Understanding how to transform some nonlinear models to straight-line form and apply regression to fit the models.
- Knowing how to implement straight-line linear regression with Python.

## YOU'VE GOT A PROBLEM

In  **Chap. 1**, we noted that a free-falling object in air, such as a bungee jumper, is subject to the upward force of air resistance or drag. As a first approximation, we assumed that this force was proportional to the square of the velocity as in

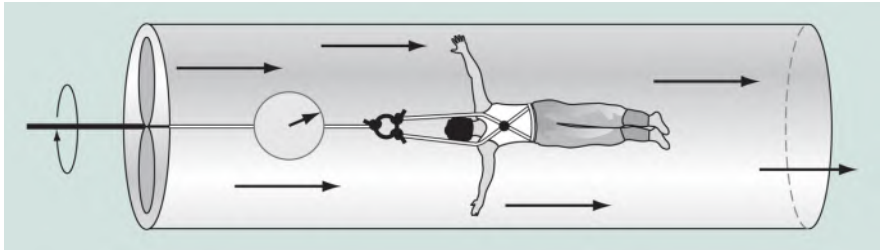
$$F_U = c_d v^2 \quad (14.1)$$

where  $F_U$  = the upward force of air resistance or drag ( $\text{N} = \text{kg} \cdot \text{m}/\text{s}^2$ ),  $c_d$  = a drag coefficient ( $\text{kg}/\text{m}$ ), and  $v$  = velocity ( $\text{m}/\text{s}$ ).

Expressions such as  **Eq. (14.1)** come from the field of fluid mechanics. Although such relationships derive in part from theory, experiments play a critical role in their formulation. One such experiment is

depicted in [Fig. 14.1](#). An individual is suspended in a wind tunnel (any volunteers?) and the force is measured for various levels of wind velocity. Typical results are listed in [Table 14.1](#).

**FIGURE 14.1** Wind tunnel experiment to measure how the force of air resistance depends on velocity.

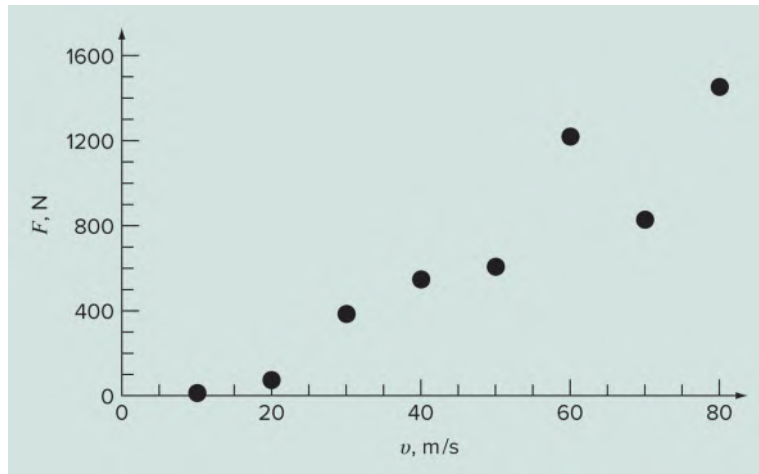


**TABLE 14.1** Experimental data for drag force (N) and velocity (m/s) from a wind tunnel experiment.

| $v, \text{ m/s}$ | 10 | 20 | 30  | 40  | 50  | 60   | 70  | 80   |
|------------------|----|----|-----|-----|-----|------|-----|------|
| $F, \text{ N}$   | 25 | 70 | 380 | 550 | 610 | 1220 | 830 | 1450 |

The relationship can be visualized by plotting force versus velocity. As displayed in [Fig. 14.2](#), several features of the relationship bear mention. First, the points indicate that the force increases as the velocity increases. Second, the points do not increase smoothly, but exhibit scatter, and the noise appears to increase with velocity. Finally, although it may not be obvious, the relationship between force and velocity may not be linear. Of course, noting [Eq. \(14.1\)](#), we might expect this. And this conclusion becomes more apparent if we assume that force starts out at zero at zero velocity and increases from there.

**FIGURE 14.2** Plot of drag force versus wind velocity for an object suspended in a wind tunnel.



In Chaps. 14 and [15](#), we will explore how to fit a “best” line or curve to such data. In doing so, we will illustrate how relationships like [Eq. \(14.1\)](#) arise from experimental data.

# 14.1 STATISTICS REVIEW

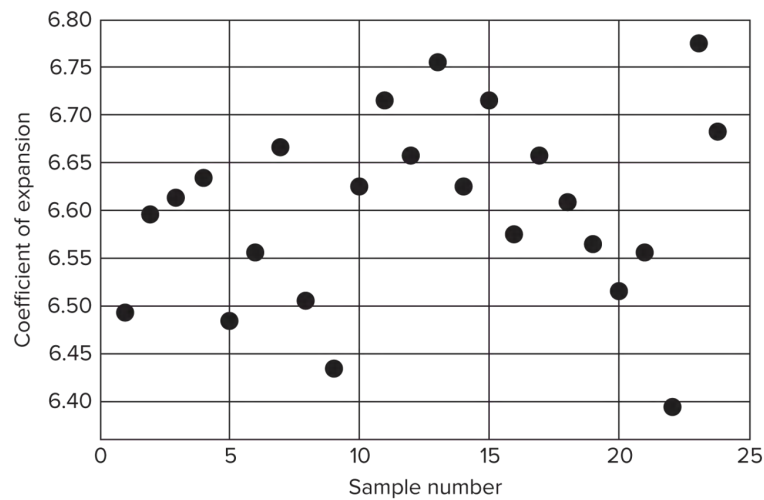
Before describing least-squares regression, we will first review some basic concepts from the field of statistics. These include observing the distribution of a set of data via a histogram chart and computing sample statistics for the mean, standard deviation, and residual sum of squares. We will also consider the normal distribution, most commonly used to describe data variability. In addition, we will illustrate how Python can be used to compute these statistics and produce illustrative plots. If you are thoroughly familiar with these topics, feel free to skip to [Sec. 14.2](#), but we suggest you pay attention to the Python code used here.

## 14.1.1 Descriptive Statistics

Suppose that, during an engineering study, several measurements were made of a particular quantity. For example, [Table 14.2](#) contains 24 readings of the coefficient of thermal expansion measured on samples of a structural steel. We do not anticipate that there will be a trend in these readings as a function of some factor or variable, for example, the order in which the samples were measured, any significant changes in composition of the samples (not likely), but, as required, we could examine that. If the samples were measured in the order across each row, here is a plot of measurement versus sample order.

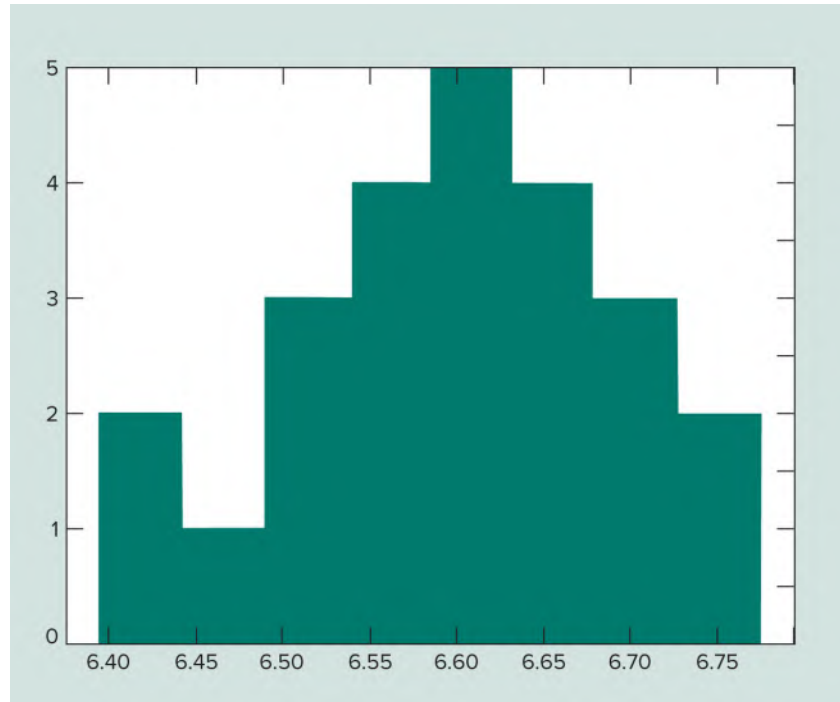
**TABLE 14.2** Measurements of the coefficient of thermal expansion of structural steel.

|       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|
| 6.495 | 6.595 | 6.615 | 6.635 | 6.485 | 6.555 |
| 6.665 | 6.505 | 6.435 | 6.625 | 6.715 | 6.655 |
| 6.755 | 6.625 | 6.715 | 6.575 | 6.655 | 6.605 |
| 6.565 | 6.515 | 6.555 | 6.395 | 6.775 | 6.685 |





We do not observe definite trends in the data. At most, we might claim that the variability increases toward the end, and there are a couple runs of three data either increasing or decreasing. Additionally, the data range is from 6.395 to 6.775, and it is of interest to get a picture of how many data points are in each interval of measurement, perhaps intervals of 0.05. If we count these and create a bar chart, we are viewing what is called a *histogram* of the data.



Here, we note that the data display a central tendency and a somewhat symmetric spread. This invites us to estimate what the central location of the measurements is and to quantify the spread of the data. If we were able to take thousands of measurements and use tiny intervals in the histogram, we would see a nearly continuous curve and be able to characterize the location and spread accurately; however, given only a small set of samples, which is typical in practice, we have to estimate what these “true” values are. That is the nature of *sample statistics*. An infinitely large set of measurements would represent what is called the *population*.

## Measure of Location

The “true” value of the central tendency is called the *mean*. Our estimate of it is called the “sample mean,” and the most common, but not only, statistic used is the *sample average*,

$$y = \frac{\sum y_i}{n} \quad (14.2)$$

where the summation (and all succeeding summations in this section) is from  $i = 1$  to  $n$ .

There are numerous alternatives to the average for measuring central tendency. One of the most common is the *median*. A concern with data is that one or more data points might be *outliers*. These are measurements often associated with a blunder or mistake. They affect the average more than they do the median. The median is the midpoint of the data. It is found or calculated by sorting the data set and picking the middle value. If the number of data is even, as in our data in [Table 14.2](#), one takes the average of the two middle points for the median. The median is sometimes called the 50th percentile.

The *mode* is the most frequently occurring value. It is the peak on the histogram chart. In the chart above, it is about 6.62. If one looks at individual measurements, the mode is not that useful for small data sets like ours. There are four measurements, 6.555, 6.625, 6.655, and 6.715, which occur twice. The mode is useful when the distribution, as evidenced by the histogram, is not symmetric. That is not the case with the steel coefficient data.

There are other estimators for the mean that discount further the influence of measurements that are far from the center. One of these is called the *m-estimate*, and we do not introduce that here.

## Measures of Spread or Dispersion

It should be intuitive that the greater the spread of the data, the more difficult it is to estimate accurately the mean. The simplest measure of spread is the *range*, the difference between the largest and smallest data point. The range, however, is not considered a reliable estimate because it is highly sensitive to sample size and greatly influenced by only two data points. It is sometime used for very small data sets, for example, two-to-five points, and divided by the square root of the number of data points.

The most common measure of spread is the *sample standard deviation* ( $s_y$ ), which is calculated as

$$s_y = \sqrt{\frac{SS_T}{n-1}} \quad (14.3)$$

where  $SS_T$  = the total sum of squares of the differences between the data points and the estimate of the mean, also called the *total corrected sum of squares*, and is computed as

$$SS_T = \sum (y_i - y)^2 \quad (14.4)$$

The total sum of squares is considered to be a useful measure of the variability of the data. Data which are far from the average inflate  $SS_T$  greatly; consequently,  $SS_T$  is very sensitive to outliers in the data. Another common measure of the dispersion of the data is the *sample variance*, which is

$$s_y^2 = \frac{\sum (y_i - y)^2}{n-1} \quad (14.5)$$

Note that the denominator in [Eqs. \(14.3\)](#) and [\(14.5\)](#) is  $n - 1$ . You might think it should be  $n$ . The divisor  $n - 1$  is called the *degrees of freedom*. If we imagine taking  $n$  measurements over and over, and, for each set of measurements, we calculate  $y$  and  $s_y$ , we would expect these estimates to cluster about the true values of the mean, denoted  $\mu$ , and the standard deviation, denoted  $\sigma$ . It turns out that this is valid for the mean. However, if we were to use  $n$  in the denominator of [Eq. \(14.3\)](#), we would find that our cluster of  $s_y$  estimates would be shifted, or biased, to the left from the true value by a factor of  $(n - 1)/n$ . Using the  $n - 1$  denominator, our cluster would be aligned properly. By computing  $y$  from the data and using it in [Eq. \(14.3\)](#), we have removed a degree of freedom from the data, and thus, the  $n - 1$  denominator. This can be proven mathematically by using a technique called expectation, which we do not illustrate here.

We note that there is an alternate, equivalent formula for  $s_y^2$ , which is more convenient for manual, calculator-based operations.

Page 310

$$s_y^2 = \frac{\sum y_i^2 - ny^2}{n - 1} \quad (14.6)$$

This formula does not require calculating the individual  $(y_i - y)$  differences, and thus saves computational effort; however, fair warning that the numerator can represent a small difference between two very large numbers, and thus is subject to roundoff errors (recall discussion of subtractive cancellation from [Chap. 4](#)). [Equation \(14.6\)](#) is generally not used in computer code, rather the prior formulas.

As we stated above, these measures of dispersion or spread,  $s_y$ ,  $SS_T$ , and  $s_y^2$ , are vulnerable to outliers in the data and will be greatly inflated by such. If this occurs, the estimates will be much greater than  $\sigma$ . Is there a measure of spread, and an estimator of  $\sigma$ , that is not so susceptible to outliers?<sup>1</sup> The answer is yes, and the measure is called the *median absolute deviation*, or the MAD. These are the steps in calculating the MAD:

1. compute the median,  $\tilde{y}$ , of the data set
2. compute the absolute values of the data from the median, AKA the absolute deviations
3. compute the median of the absolute deviations
4. divide this quantity by 0.6745 to obtain an unbiased estimate of  $\sigma$

A final statistic that has utility in quantifying the spread of data is the coefficient of variation, *c.v.* This statistic is the ratio of the sample standard deviation to the sample mean estimate. It is often expressed as a percentage as in

$$c.v. = \frac{s_y}{y} 100\% \quad (14.7)$$

## EXAMPLE 14.1

## Calculating Sample Statistics

### Problem Statement

Compute the sample mean, median, total sum of squares, variance, standard deviation, coefficient of variation, and the MAD for the data in [Table 14.2](#).

### Solution

The data can be organized in tabular form for calculating the desired sample statistics. This is presented in Table 4.3.

The sample mean can be calculated as the arithmetic average using [Eq. \(14.2\)](#),

$$\bar{y} = \frac{158.4}{24} = 6.600$$

With an even number of data, the median,  $\tilde{y}$ , is the average of items 12 and 13 from the first sorted column,  $(6.605 + 6.615)/2 = 6.61$ .

The total corrected sum of squares,  $SS_T$ , is the sum of squares of the  $y_i - \bar{y}$  column, noted in the table as 0.217. The estimated variance is therefore

$$s_y^2 = \frac{SS_T}{n - 1} \cong 0.009435 \quad \text{and then} \quad s_y \cong 0.097133$$

The coefficient of variation is then  $s_y/\bar{y} \cong 1.47\%$ .

As far as the MAD is concerned, the table presents the absolute deviations from the median,  $|y_i - \tilde{y}|$  and then sorts this column. The MAD is the median of the last column divided by 0.6745, or

$$MAD = \frac{0.055}{0.6745} \cong 0.0815$$

Notice that the MAD is lower than the sample standard deviation as an estimate of  $\sigma$ .

In reviewing these calculations, we note that sorting data (this was done twice in [Table 14.3](#)) and calculating the sum of squares of the deviations from the average are both time consuming. You will see that we note in the table the sum of squares of the sorted  $y_i$  column (same as that of the  $y_i$  column). This allows us to use the short-cut formula from [Eq. \(14.6\)](#),

$$s_y^2 = \frac{1045.7 - 24 \cdot 6.6^2}{23} \cong 0.0113 \quad \text{and} \quad s_y \cong 0.1063$$



**TABLE 14.3** Data arranged for computing sample statistics.

| $i$  | $y_i$ | sorted   | $y_i - \bar{y}$ | $ y_i - \bar{y} $ | sorted |
|------|-------|----------|-----------------|-------------------|--------|
| 1    | 6.495 | 6.395    | -0.205          | 0.215             | 0.005  |
| 2    | 6.595 | 6.435    | -0.165          | 0.175             | 0.005  |
| 3    | 6.615 | 6.485    | -0.115          | 0.125             | 0.015  |
| 4    | 6.635 | 6.495    | -0.105          | 0.115             | 0.015  |
| 5    | 6.485 | 6.505    | -0.095          | 0.105             | 0.015  |
| 6    | 6.555 | 6.515    | -0.085          | 0.095             | 0.025  |
| 7    | 6.665 | 6.555    | -0.045          | 0.055             | 0.035  |
| 8    | 6.505 | 6.555    | -0.045          | 0.055             | 0.045  |
| 9    | 6.435 | 6.565    | -0.035          | 0.045             | 0.045  |
| 10   | 6.625 | 6.575    | -0.025          | 0.035             | 0.045  |
| 11   | 6.715 | 6.595    | -0.005          | 0.015             | 0.055  |
| 12   | 6.655 | 6.605    | 0.005           | 0.005             | 0.055  |
| 13   | 6.755 | 6.615    | 0.015           | 0.005             | 0.055  |
| 14   | 6.625 | 6.625    | 0.025           | 0.015             | 0.075  |
| 15   | 6.715 | 6.625    | 0.025           | 0.015             | 0.095  |
| 16   | 6.575 | 6.635    | 0.035           | 0.025             | 0.105  |
| 17   | 6.655 | 6.655    | 0.055           | 0.045             | 0.105  |
| 18   | 6.605 | 6.655    | 0.055           | 0.045             | 0.105  |
| 19   | 6.565 | 6.665    | 0.065           | 0.055             | 0.115  |
| 20   | 6.515 | 6.685    | 0.085           | 0.075             | 0.125  |
| 21   | 6.555 | 6.715    | 0.115           | 0.105             | 0.145  |
| 22   | 6.395 | 6.715    | 0.115           | 0.105             | 0.165  |
| 23   | 6.775 | 6.755    | 0.155           | 0.145             | 0.175  |
| 24   | 6.685 | 6.775    | 0.175           | 0.165             | 0.215  |
| Sums | 158.4 | 1045.657 | 0.217           | Sum of squares    |        |

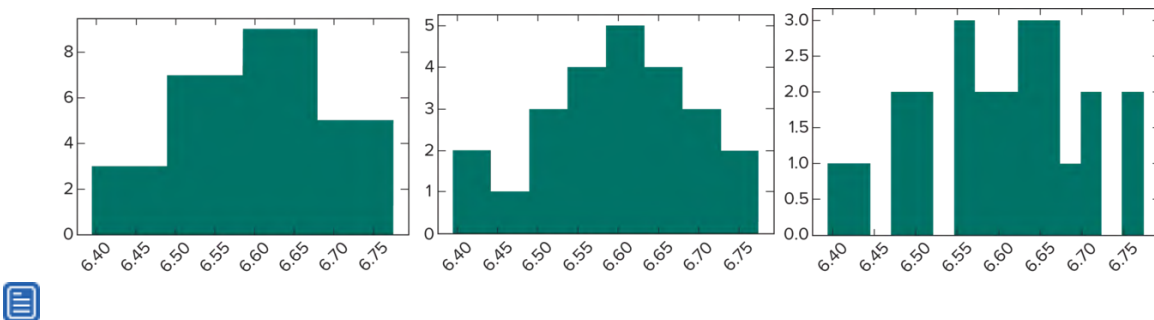
You will note that these last values differ from the previous ones in this example. The reason for this is the loss of precision in the numerator term of [Eq. \(14.6\)](#). If we use instead 1045.657, the answers will agree. This points out the need for care of precision in using the short-cut formula and others like it.

## 14.1.2 The Normal Distribution

We illustrated briefly in the previous section the use of the histogram chart to depict the data. It is always useful to look at your data graphically before diving into numerical estimates. This allows us to address important questions first, such as,

- Do the data exhibit a single, central tendency?
- What is the apparent spread or dispersion of the data?
- Is the dispersion approximately symmetric about the center?
- Are there any apparent outliers that bear further investigation?

Here is some more information about the construction of a histogram and its associated bar chart. The *histogram* is a set of frequency data, that is, it represents the counts of the number of data that fall within given intervals spanning the range of the data. The intervals chosen are, by convention, even in width. And then comes the question: How many intervals to use? This sounds like a simple question, but interval choice is a common mistake in constructing histograms. Use too many or too few intervals, and you will not be able to discern overall patterns. A general guideline, when there are 50 or more data, is to use a number of intervals between  $\sqrt{n}$  and  $\log_2(n) - 1$ , rounding each one of these numbers down to the nearest integer. For smaller sets of data, some experimentation and judgment may be required. The charts below contrast the use of 4, 8, and 15 intervals, or *bins* as they are commonly called, for our data set from the previous section.



You can see that the shape of the distribution is more clearly revealed in the intermediate case.

It is common to attempt to model the histogram with a mathematical function called a distribution. The function used most frequently is the *normal* or *Gaussian distribution*, represented by the classic “bell curve.” Not all, but many phenomena are adequately described by the normal distribution, and, importantly, if we collect samples of  $n$  data repeatedly, the histogram of the sequence of averages,  $y$ , will always tend to the normal distribution following a principle known as the *central limit theorem*.

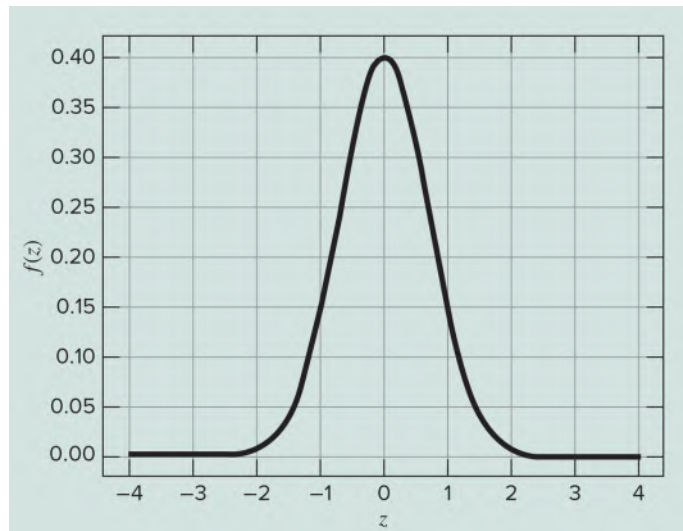
The *normal distribution* is defined mathematically by a density function,  $f$ ,

$$f(y) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(y-\mu)^2}{2\sigma^2}} \quad -\infty \leq y \leq \infty \quad (14.8)$$

where  $\mu$  = the mean and  $\sigma$  = the standard deviation. The area under the density curve is probability, so the total area is equal to one and the height of the curve at  $y = \mu$  is  $1/(\sigma\sqrt{2\pi})$ . If we standardize the

normal distribution by defining  $z \triangleq (y - \mu)/\sigma$ , the density function becomes much simpler:

$$f(z) = \frac{1}{\sqrt{2\pi}} e^{-z^2} \quad -\infty \leq z \leq \infty \quad (14.9)$$



We observe that most of the area under the curve is within the limits  $-3 \leq z \leq 3$ , which is equivalent to  $\pm 3\sigma$ , about  $y = \mu$ . The area between  $z = -1$  and  $z = 1$  (or  $\mu - \sigma \leq y \leq \mu + \sigma$ ) is about 68%. Likewise, between  $z = \pm 2$ , 95%, and  $z = \pm 3$ , 99.7%. These observations taken together are known as the *normal probability rule*.

The curves described in [Eqs. \(14.8\)](#) and [\(14.9\)](#) will not plot properly on the histogram chart because the ordinate there is frequency. To compare the normal density to our histogram chart, we need to scale either the normal density up or the histogram frequency down.

## EXAMPLE 14.2

### Construction of a histogram chart and comparison with the normal distribution

#### Problem Statement

Construct a histogram chart for the data in [Table 14.2](#) and provide a graphical comparison with the normal distribution. Use evenly spaced intervals of width 0.05, starting at 6.39.

#### Solution

We can make reference to the first sorted column in [Table 14.3](#). First, list the bin boundaries.

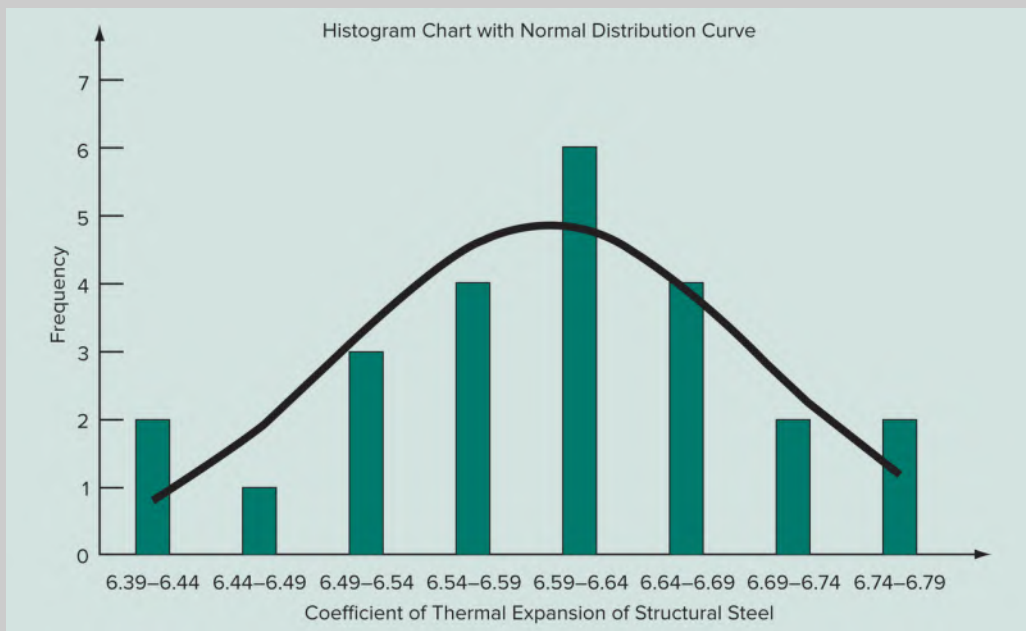
[ 6.39, 6.44, 6.49, 6.54, 6.59, 6.64, 6.69, 6.74, 6.79 ]

Next, we can count data in the intervals using [Table 14.3](#) and produce the histogram table.

| Bin Center | Bin Interval | Frequency |
|------------|--------------|-----------|
| 6.415      | 6.39–6.44    | 2         |
| 6.465      | 6.44–6.49    | 1         |
| 6.515      | 6.49–6.54    | 3         |
| 6.565      | 6.54–6.59    | 4         |
| 6.615      | 6.59–6.64    | 6         |
| 6.665      | 6.64–6.69    | 4         |
| 6.715      | 6.69–6.74    | 2         |
| 6.765      | 6.74–6.79    | 2         |
|            |              | 24        |

To make the comparison with the normal distribution, we need values for  $\mu$  and  $\sigma$ . The best we can do here is to use our corresponding estimates,  $\bar{y}$  and  $s_y$ . To compute the values for the chart, it is a reasonable approximation to use [Eq. \(14.8\)](#) and scale the result by multiplying by  $n$  and by the bin width, here = 0.05. With that, we can construct the comparative plot, shown in [Fig. 14.3](#).

**FIGURE 14.3** A histogram used to depict the distribution of the data in [Table 14.2](#). If the sample size were to increase greatly, the histogram bars would likely approach the bell-shaped curve shown on the graph, the normal distribution density curve.



One can see a general trend in the histogram bars that resembles the normal distribution curve. With a much larger data set and correspondingly much narrower bin widths, the resemblance would be closer.

## 14.1.3 Descriptive Statistics Using Python

Python provides a suite of functions in its NumPy and SciPy modules to facilitate the calculation of descriptive statistics. The following code illustrates their use.

```
import numpy as np
from scipy import stats

ce = np.array([6.495, 6.595, 6.615, 6.635, 6.485, 6.555,
               6.665, 6.505, 6.435, 6.625, 6.715, 6.655,
               6.755, 6.625, 6.715, 6.575, 6.655, 6.605,
               6.565, 6.515, 6.555, 6.395, 6.775, 6.685])

cebar = np.mean(ce)
print('mean estimate = {0:5.3f} '.format(cebar))
cemed = np.median(ce)
print('sample median = {0:5.3f} '.format(cemed))
cemode = stats.mode(ce,axis=None)
print('sample mode = ',cemode)
cevar = np.var(ce)
print('sample variance = {0:5.3e} '.format(cevar))
ces = np.std(ce)
print('sample standard deviation = {0:7.5f} '.format(ces))

def S(a):
    abar = np.mean(a)
    adev = a - abar
    return np.sum(adev**2)

cv = ces/cebar
print('coefficient of variation = {0:5.3f} %'.format(cv))

Sce = S(ce)
print('total corrected sum of squares = {0:5.3f} '.format(Sce))

MADce = stats.median_absolute_deviation(ce)
print('MAD = {0:5.3e}'.format(MADce))

mean estimate = 6.600
sample median = 6.610
sample mode = ModeResult(mode=array([6.555]), count=array([2]))
sample variance = 9.042e-03
sample standard deviation = 0.09509
coefficient of variation = 0.014 %
total corrected sum of squares = 0.217
MAD = 8.154e-02
```

These results are consistent with those obtained previously in  **Example 14.1**. Notice the format of the result from the `stats.mode` function in the SciPy module. Only the first occurrence of two identical values is noted. You will also note that a function `S` was programmed to compute the total corrected sum of squares, as this is not available in the modules, although it can be computed indirectly in Python via `np.var(ce)*(n-1)`, where `n` is the number of data.

Python can also be used to generate histogram data and charts. The `histogram` function in the NumPy module has this syntax:

```
hist, bin_edges = np.histogram(data_array, bins=10, range=)
```

There are additional arguments. The `bins` and `range` arguments are optional. For the data and specifications from [Example 14.2](#),

Page 315

```
import numpy as np
#from scipy import stats

ce = np.array([6.495, 6.595, 6.615, 6.635, 6.485, 6.555,
               6.665, 6.505, 6.435, 6.625, 6.715, 6.655,
               6.755, 6.625, 6.715, 6.575, 6.655, 6.605,
               6.565, 6.515, 6.555, 6.395, 6.775, 6.685])

hist, bin_edges = np.histogram(ce, bins=8, range=[6.39, 6.79])
print('\nHistogram data:\n', hist)
print('\nBin boundaries:\n', bin_edges)

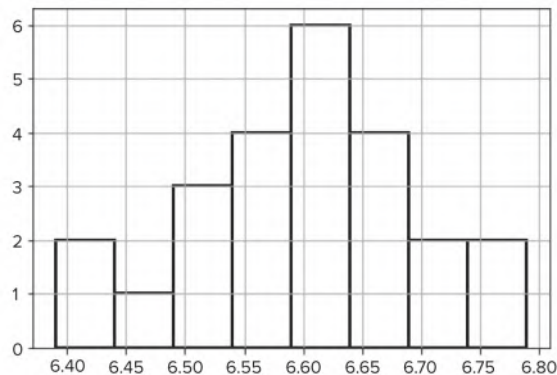
Histogram data:
[2 1 3 4 6 4 2 2]

Bin boundaries:
[6.39 6.44 6.49 6.54 6.59 6.64 6.69 6.74 6.79]
```

With these data, it is possible to create a bar chart using the `bar` function from the Matplotlib `pyplot` module and have a decent amount of control over the appearance of the chart.

```
bin_width = bin_edges[1]-bin_edges[0]
n = len(hist)
bin_centers = np.zeros((n))
for i in range(n):
    bin_centers[i] = (bin_edges[i]+bin_edges[i+1])/2

import matplotlib.pyplot as plt
plt.bar(bin_centers, hist, width=bin_width, color='w', edgecolor='k')
```

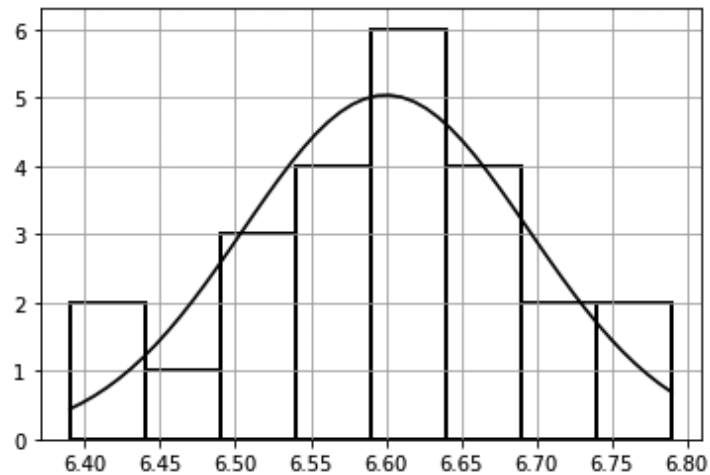


We can add a normal distribution curve to the plot by entering the following commands, which include using the `norm.pdf` function from the SciPy `stats` module:

```
from scipy import stats
cebar = np.mean(ce)
ces = np.std(ce)
x = np.linspace(6.39, 6.79)
y = stats.norm.pdf(x, cebar, ces)
plt.plot(x, 24*0.05*y, color='k')
```

with the result,

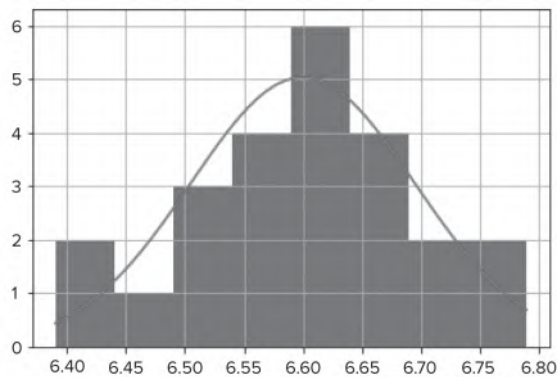
Page 316



Notice how the distribution curve is scaled to the bar chart by multiplying by the number of data, 24, and the bin width, 0.05.

For convenience, Python's `pylab` module provides a `hist` function that computes the histogram data and produces a bar chart. Here, we can also add the normal distribution curve.

```
import pylab
pylab.figure()
pylab.hist(ce,bins=8,range=[6.39,6.79])
pylab.grid()
pylab.plot(x,1.2*y,color='r')
```



With the `hist` function, you do not have as much control over the presentation, but it is often the method of choice.

## 14.2 RANDOM NUMBERS AND SIMULATION

In this section, we will describe functions in Python's NumPy `random` submodule that can be used to produce a sequence of random numbers following a given statistical distribution. We will highlight the uniform and normal distributions, although options are available for many others.

### 14.2.1 Random Numbers from the Uniform Distribution

In general, the density function for the *uniform distribution* is

$$f(x) = \frac{1}{b-a} \quad a \leq x \leq b$$

This means that the probability of a value of  $x$  being in any subinterval between  $a$  and  $b$  is a constant.

Note: For distributions of real, random variables, the probability for a specific value of  $x$  is zero, corresponding to the area below  $f$  for a single point on the  $x$ -axis.

Page 317

A built-in function in Python that provides  $n$  random numbers as an `ndarray` drawn from a uniform distribution has the syntax,

```
xvals = numpy.random.uniform(a,b,n)
```

#### EXAMPLE 14.3

##### Generating Uniform Random Values of Drag

##### Problem Statement

If the initial velocity is zero, the downward velocity of the free-falling bungee jumper can be predicted by the following analytical solution ( Eq. 1.9):

$$v = \sqrt{\frac{mg}{c_d}} \tanh\left(\sqrt{\frac{gc_d}{m}} t\right)$$

Suppose that  $m = 68.1$  kg, but  $c_d$  is not known precisely. For example, you might suppose that it has a random value between 0.225 and 0.275, that is, about  $\pm 10\%$  around a mean value of 0.25 kg/m. Use the Python NumPy `random.uniform` function to generate 1,000 random values for



$c_d$  in the given range, and then use these values to compute 1,000 values of velocity at  $t = 4$  s. Display the  $c_d$  and  $v$  values in histogram charts.

## Solution

Before generating the random numbers for  $c_d$ , we can first compute the mean velocity:

$$v_{\text{mean}} = \sqrt{\frac{9.81 \cdot 68.1}{0.25}} \tanh\left(\sqrt{\frac{9.81 \cdot 0.25}{68.1}} 4\right) \cong 33.1118 \frac{\text{m}}{\text{s}}$$

We can also generate the range of velocities:

$$v_{\text{low}} = \sqrt{\frac{9.81 \cdot 68.1}{0.275}} \tanh\left(\sqrt{\frac{9.81 \cdot 0.275}{68.1}} 4\right) \cong 32.6223 \frac{\text{m}}{\text{s}}$$

$$v_{\text{high}} = \sqrt{\frac{9.81 \cdot 68.1}{0.225}} \tanh\left(\sqrt{\frac{9.81 \cdot 0.225}{68.1}} 4\right) \cong 33.6198 \frac{\text{m}}{\text{s}}$$

With these results, we can see that for a 20% change in the drag coefficient, the velocity only changes by

$$\Delta v = \frac{33.6198 - 32.6223}{33.1118} \cong 3.01\%$$

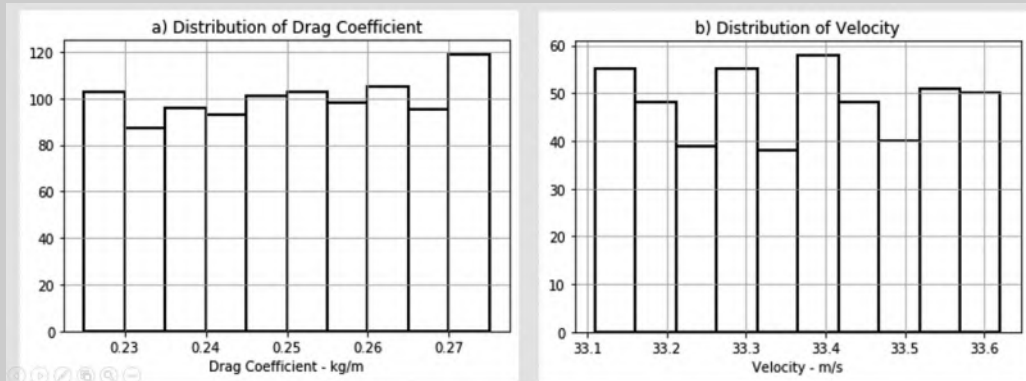
That tells us that velocity is not highly sensitive to errors in drag coefficient.

The following Python script generates the random values for  $c_d$  along with the sample mean and standard deviation. Then, a histogram chart is generated, shown in  Fig. 14.4a. For comparison purposes, the true mean is computed as  $\mu = (a + b)/2 = 0.25$  and the standard deviation is

$$\sigma = \sqrt{(b - a)^2/12} \cong 0.0144.$$

---

**FIGURE 14.4** Histograms of (a) uniformly distributed drag coefficients, and (b) the resulting distribution of velocity.



```
import numpy as np
n = 1000
a = 0.225
b = 0.275

mu_cd = (a+b)/2
sigma_cd = np.sqrt((b-a)**2/12)

cdrand = np.random.uniform(a,b,n)

avg_cd = np.mean(cdrand)
s_cd = np.std(cdrand)

print('\nMean = {0:7.4f}'.format(mu_cd))
print('Sigma = {0:7.4f}'.format(sigma_cd))
print('Sample mean = {0:7.4f}'.format(avg_cd))
print('Sample std dev = {0:7.4f}'.format(s_cd))

import pylab
pylab.figure()
pylab.hist(cdrand,bins=10,range=[a,b],color='w',edgecolor='k',linewidth=2.)
pylab.grid()

Mean = 0.2500
Sigma = 0.0144
Sample mean = 0.2509
Sample std dev = 0.0144
```

Observe that the “experimental” estimates for the mean and standard deviation are very close to the theoretical values. If our sample size were much smaller, there would be more deviation in the histogram and the estimates would not be so close.

Page 318

The 1,000 values of  $c_d$  can be employed via the analytical solution to compute the resulting distribution of velocities at  $t = 4$  s.

```
g = 9.81 # m/s2
m = 68.1 # kg
t = 4 # s
vrand = np.sqrt(m*g/cdrand)*np.tanh(np.sqrt(g*cdrand/m)*t)

pylab.figure()
pylab.hist(vrand,bins=10,range=[33.11,33.62],color='w',edgecolor='k',linewidth=2.)
pylab.grid()
pylab.xlabel('Velocity - m/s ')
pylab.title('b) Distribution of Velocity')

avg_v = np.mean(vrand)
range_v = np.max(vrand)-np.min(vrand)
print('Average velocity = {0:7.4f} m/s '.format(avg_v))
```

```
print('% Variation of velocity = {0:7.2f}'.format(range_v/avg_v*100))

Average velocity = 33.1084 m/s
% Variation of velocity =      3.01
```

The distribution of velocities is shown in the histogram chart in [Fig. 14.4b](#). The average and % variation calculated from the samples compares favorably to the previously calculated values.

The foregoing example is formally referred to as a *Monte Carlo simulation*. The term, which is a reference to Monaco's Monte Carlo casino, was first used by physicists on nuclear weapons projects in the 1940s. Although this approach yields intuitive results for this simple example, there are instances where such computer simulations yield surprising outcomes and provide insights that would otherwise be impossible to determine. This approach is feasible only because of the computer's ability to implement tedious, repetitive computations in an efficient manner.

Page 319

## 14.2.2 Random Numbers from the Normal Distribution

A built-in function in Python that provides  $n$  random numbers as an `ndarray` drawn from a normal distribution has the syntax,

```
xvals = numpy.random.normal(mu,sigma,n)
```

where `mu` = the mean of the distribution and `sigma` the standard deviation. It is possible to return a two-dimensional  $n \times m$  array by a third argument of `...,n,m)` instead of `...,n)`.

### EXAMPLE 14.4

#### Generating Normally Distributed Random Values of Drag

##### Problem Statement

Analyze the same case as in [Example 14.2](#), but rather than employing a uniform distribution, generate 1,000 normally distributed drag coefficient values with a mean of 0.25 and a standard deviation of 0.0144.

##### Solution

The following Python script generates the random values for  $c_d$ , along with calculating the sample mean, standard deviation and plotting a histogram of the data. Code is added to compute the corresponding velocities, as well as their sample mean and standard deviation.

```
import numpy as np

n = 1000

mu_cd = 0.25
sigma_cd = 0.0144
lowlim = mu_cd - 3.5*sigma_cd
hilim = mu_cd + 3.5*sigma_cd

cdnorm = np.random.normal(mu_cd,sigma_cd,n)
```

```

avg_cd = np.mean(cdnorm)
s_cd = np.std(cdnorm)

print('\nMean = [0:7.4f]'.format(mu_cd))
print('Sigma = [0:7.4f]'.format(sigma_cd))
print('Sample mean = [0:7.4f]'.format(avg_cd))
print('Sample std dev = [0:7.4f]'.format(s_cd))

import pylab
pylab.figure()
pylab.hist(cdnorm,bins=10,range=[lowlim,hilim],color='w', \
           edgecolor='k',linewidth=2.)

pylab.grid()
pylab.xlabel('Drag Coefficient - kg/m')
pylab.title('a) Distribution of Drag Coefficient')

g = 9.81 # m/s2
m = 68.1 # kg
t = 4 # s
vnorm = np.sqrt(m*g/cdnorm)*np.tanh(np.sqrt(g*cdnorm/m)*t)
vmin = np.min(vnorm)
vmax = np.max(vnorm)

pylab.figure()
pylab.hist(vnorm,bins=10,range=[vmin,vmax],color='w', \
           edgecolor='k',linewidth=2.)

pylab.grid()
pylab.xlabel('Velocity - m/s')
pylab.title('b) Distribution of Velocity')

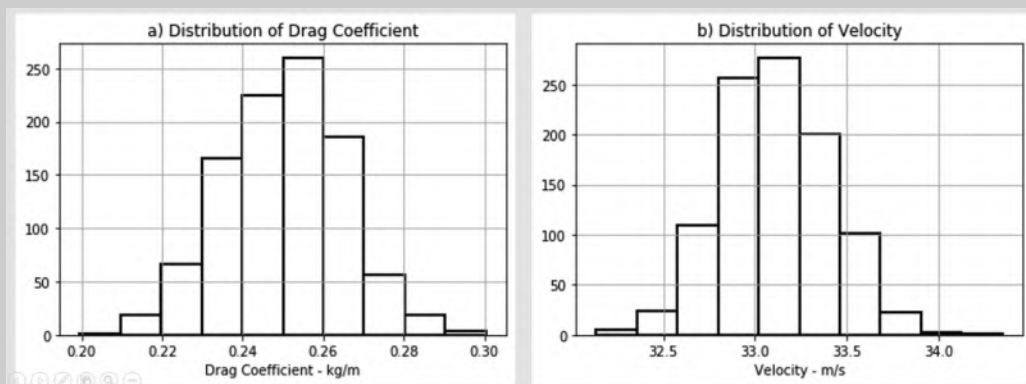
avg_v = np.mean(vnorm)
range_v = np.max(vnorm)-np.min(vnorm)
print('Average velocity = [0:7.4f] m/s'.format(avg_v))
print('% Variation of velocity = [0:7.2f] %'.format(range_v/avg_v*100))

Mean = 0.2500
Sigma = 0.0144
Sample mean = 0.2495
Sample std dev = 0.0145
Average velocity = 33.1251 m/s
% Variation of velocity = 3.01 %

```

The histogram displayed in  **Fig. 14.5a** shows a typical normal distribution, and the sample mean and standard deviation compare closely to the ideal values used to generate the random numbers.

**FIGURE 14.5** Histograms of (a) normally distributed drag coefficient values and (b) the resulting distribution of velocity.





The results for velocity are shown in [Fig. 14.5b](#), and again the distribution looks like the normal. The average velocity corresponds to that calculated in [Example 14.2](#) from the mean drag coefficient.

Although simple, the foregoing two examples illustrate how random numbers can be created easily with Python. We will explore additional applications in the problems at the end of this chapter.

## 14.3 STRAIGHT-LINE LEAST SQUARES REGRESSION

---

Where substantial error is associated with data, the best curve-fitting strategy is to specify an approximating function that fits the shape or general trend of the data without attempting to match each data point. One approach to do this is to plot the data against the factor or independent variable of interest, inspect the plotted data visually and then sketch a line through the points using your best judgment. Although such “eyeball” approaches have commonsense appeal and are valid for “back-of-the-envelope” calculations, they are deficient because they are arbitrary and subjective. That is, unless the points define a perfect straight line (in which case, interpolation would be appropriate), different analysts would draw different lines. A bit later, we will describe a more detailed method to sketch a straight line through data that is more reliable and repeatable.

It turns out that the human eye is a very good judge of patterns though. If observation tells us that there is no pattern or correlation between the measurements and a factor or independent variable, statistical calculations will likely confirm that. If we see a pattern, and it looks linear, then a straight-line fit is called for. However, if we see a pattern, and it definitely has curvature, we will be wasting our time trying to fit a straight line to the data.

To remove the subjectivity from fitting a line to data, a criterion must be established as a basis for the fit. A logical way to proceed is to derive a curve that minimizes the discrepancy between the data points and the line. To do this, we must first quantify the term discrepancy. The simplest example is fitting a straight line to a set of paired observations,  $\{x_1, y_1\}, \{x_2, y_2\}, \dots, \{x_n, y_n\}$ . The mathematical expression for the straight line is

$$y_i = a_0 + a_1 x_i + e_i$$

where  $a_0$  and  $a_1$  are chosen coefficients for the intercept and slope, respectively, and  $e_i$  is the error or residual that is required to amend the straight-line prediction to equal the actual measurement  $y_i$ . Another way to write this, recognizing that the straight-line formula represents a predicted value of the measurement,  $\hat{y}_i$ , is

$$\hat{y}_i = a_0 + a_1 x_i$$

and then the residual as the difference between the actual measured value and the prediction,

$$e_i = y_i - \hat{y}_i$$

So, with the values for  $a_0$  and  $a_1$  in hand, we can calculate a set of residuals,  $e_i, i = 1, \dots, n$ .

## 14.3.1 Criteria for a “Best” Fit

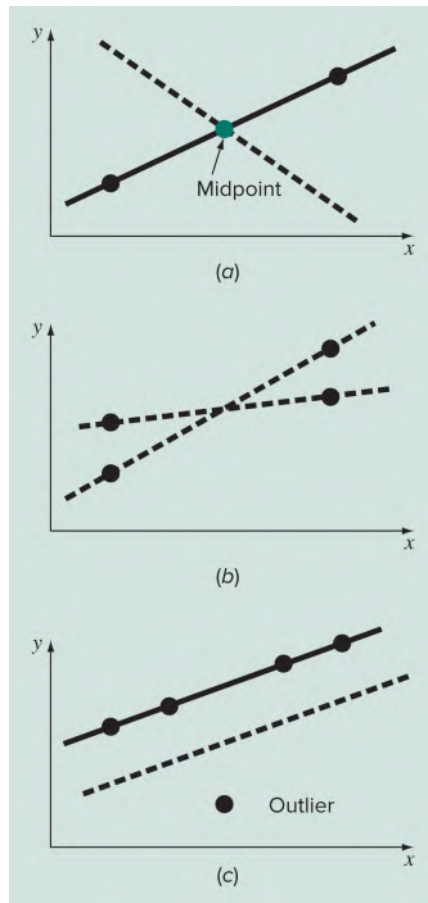
One strategy for fitting a “best” line through the data would be to minimize the sum of the residual errors, as in

$$\min_{a_0, a_1} \sum e_i \quad \text{or} \quad \min_{a_0, a_1} \sum (y_i - a_0 - a_1 x_i) \quad (14.11)$$

However, this criterion is inadequate, as illustrated in [Fig. 14.6a](#), which depicts the fit of a straight line to two points. Obviously, the best fit is the line connecting the two points; however, any straight line passing through the midpoint of the connecting line, except a vertical line, results in the same zero minimum value in [Eq. \(14.11\)](#) because the positive and negative errors cancel.

**FIGURE 14.6** Examples of criteria for “best fit” that are inadequate for regression:

- (a) minimizes the sum of the residuals,
- (b) minimizes the sum of the absolute values of the residuals, and
- (c) minimizes the maximum error of any individual point.





One way to remove the effect of the signs of the errors might be to minimize the absolute values of the residuals, that is,

$$\min_{a_0, a_1} \sum |e_i| \quad \text{or} \quad \min_{a_0, a_1} \sum |y_i - a_0 - a_1 x_i| \quad (14.12)$$

 **Figure 14.6b** demonstrates why this criterion is also inadequate. For the four points shown, any straight line falling within the dashed lines will minimize the sum of the absolute values of the residuals. Thus, this criterion also does not yield a unique best fit.

A third strategy for fitting the best line is called *minimax*. In this technique, the line is chosen that minimizes the maximum absolute value residual, that is, the maximum distance that an individual point falls from the line. As depicted in  **Fig. 14.6c**, this strategy is ill-suited for regression because it gives undue influence to an outlier, a single point with a large error. It should be noted, however, that the minimax criterion can be well-suited to fitting a simple function to a complicated analytical function (Carnahan, Luther, and Wilkes, 1969).

A fourth strategy that overcomes the shortcomings of the aforementioned approaches is to minimize the sum of the squares of the residuals:

$$\min_{a_0, a_1} \sum e_i^2 \quad \text{or} \quad \min_{a_0, a_1} \sum (y_i - a_0 - a_1 x_i)^2 \quad (14.13)$$

For convenience, we define  $SS_E \equiv \sum e_i^2$ . This minimization criterion, which is called *least squares*, has several advantages, including that it yields a unique line for a set of data.

It is worth mentioning, however, that, because an outlier has a large residual value, the square of that value may influence  $SS_E$  unduly and affect the fitted line. Such outliers are said to have strong *leverage* on the resulting best fit. Detecting and possibly removing outliers from data sets is an important topic and a tricky affair. If the residual data appear to follow a normal distribution, a good criterion for identifying an outlier is it is so far away from the sample mean that it would occur too rarely, such as 4-to-5 times the *standard error of the estimate*,  $s_e$ . This quantity, which we will describe shortly, is a measure of the standard deviation of the measurements around the best-fit line.

There is another common graphical approach to identifying outliers called a *boxplot*. It is based on using the median, not the mean estimate, and the interquartile range (25th to 75th percentile), and not the sample standard deviation. We do not introduce that here, but you should be aware of it. Python will produce a boxplot conveniently with the `matplotlib.pyplot.boxplot` function.

As we have stated before, removing outliers from data is discouraged unless there is clear evidence of the cause, such as a failure in a measuring instrument or operator error. You should always document that you have removed an outlier and be prepared to defend the action. Do keep in mind that an extreme outlier will affect a least-squares fit.



## 14.3.2 Least-Squares Fit of a Straight Line

To determine values for  $a_0$  and  $a_1$ ,  $SS_E$  is differentiated with respect to each unknown coefficient:

$$\begin{aligned}\frac{\partial SS_E}{\partial a_0} &= -2 \sum (y_i - a_0 - a_1 x_i) \\ \frac{\partial SS_E}{\partial a_1} &= -2 \sum (y_i - a_0 - a_1 x_i) x_i\end{aligned}$$

Note that we have simplified the summation symbols; unless otherwise indicated, all summations are from  $i = 1$  to  $n$ . Setting these derivatives equal to zero will result in a minimum  $SS_E$ . If this is done, the equations can be expressed as

$$\begin{aligned}0 &= \sum y_i - \sum a_0 - \sum a_1 x_i \\ 0 &= \sum x_i y_i - \sum a_0 x_i - \sum a_1 x_i^2\end{aligned}$$

Now, realizing that  $\sum a_0 = na_0$ , we can express the equations as two simultaneous linear equations with two unknowns,  $a_0$  and  $a_1$ :

Page 323

$$\begin{bmatrix} n & \sum x_i \\ \sum x_i & \sum x_i^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum x_i y_i \end{bmatrix} \quad (14.14)$$

These are called the *normal equations*. They can be solved simultaneously to obtain

$$a_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2} \quad (14.15)$$

This result can then be used in conjunction with the equation represented by the first row of

 **Eq. (14.14)** to solve for

$$a_0 = \bar{y} - a_1 \bar{x} \quad (14.16)$$

where  $\bar{y}$  and  $\bar{x}$  are the sample means of  $y$  and  $x$ , respectively.

### EXAMPLE 14.5

## Straight-Line Linear Regression

### Problem Statement

Fit a straight line to the values in [Table 14.1](#).

### Solution

In this application, force is the dependent variable,  $y$ , and velocity is the independent variable,  $x$ . The data are set up in tabular form and the necessary sums are computed and displayed in [Table 14.4](#).

**TABLE 14.4** Data and summations required to compute the best-fit straight line for the data from [Table 14.1](#)

| $i$      | $x_i$     | $y_i$        | $x_i^2$      | $x_i y_i$      |
|----------|-----------|--------------|--------------|----------------|
| 1        | 10        | 25           | 100          | 250            |
| 2        | 20        | 70           | 400          | 1,400          |
| 3        | 30        | 380          | 900          | 11,400         |
| 4        | 40        | 550          | 1,600        | 22,000         |
| 5        | 50        | 610          | 2,500        | 30,500         |
| 6        | 60        | 1,220        | 3,600        | 73,200         |
| 7        | 70        | 830          | 4,900        | 58,100         |
| 8        | <u>80</u> | <u>1,450</u> | <u>6,400</u> | <u>116,000</u> |
| $\Sigma$ | 360       | 5,135        | 20,400       | 312,850        |

The sample means are computed as

$$x = \frac{360}{8} = 45 \quad y = \frac{5135}{8} = 641.875$$

The slope and intercept can then be calculated using [Eqs. \(14.15\)](#) and [\(14.16\)](#) as

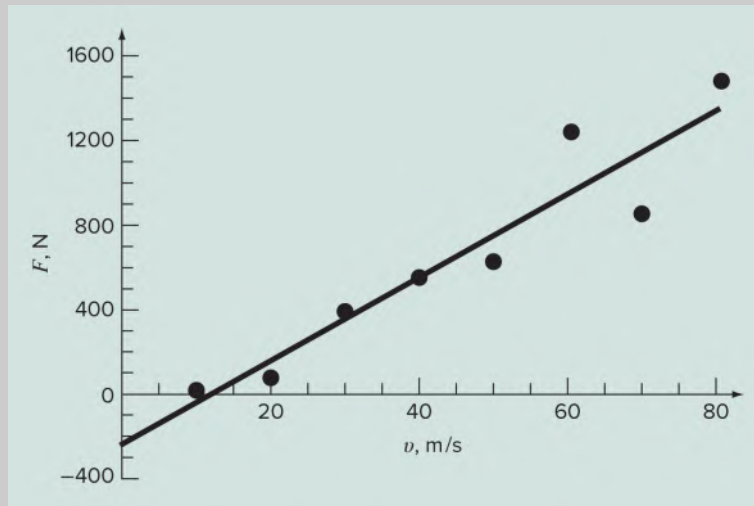
$$a_1 = \frac{8 \cdot 312850 - 360 \cdot 5135}{8 \cdot 20400 - 360^2} \cong 19.47$$
$$a_0 \cong 641.875 - 19.47 \cdot 45 \cong -234.29$$

Using the force and velocity symbols in place of  $y$  and  $x$ , the least squares fitted model is

$$F = -234.29 + 19.47v$$

The fitted line and the data are shown in [Fig. 14.7](#).

**FIGURE 14.7** Least-squares fit of a straight line to the data from [Table 14.1](#).



Note that, although the line appears to fit the trend of the data well, the intercept at zero velocity predicts negative forces, which is unrealistic. In [Sec. 14.4](#), we will show how models which are nonlinear in the independent variable yet linear in the fitted parameters can be employed to develop an alternative best-fit line that is more physically realistic.

### 14.3.3 A “Desert Island” Method to Sketch a Straight Line

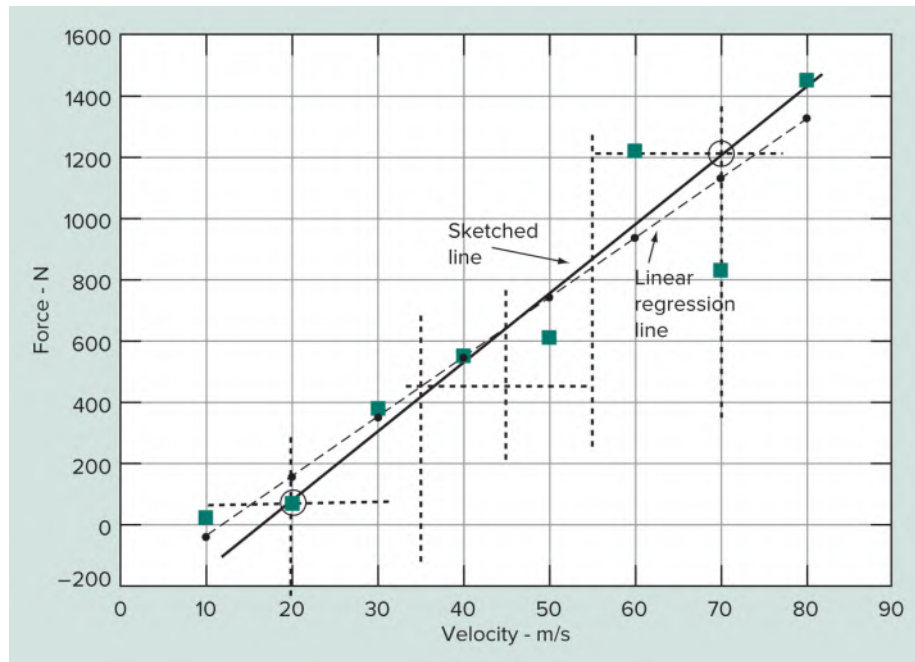
At this point, we will take a brief diversion from the mathematical/numerical approaches to fitting a straight line and present a “desert island” method you can use if you are caught in the field with no computational aids and need to sketch a straight line through a collection of data. Several investigators have concocted such approaches. The original version of the method described here was proposed by Tukey (1971) and is sometimes referred to as the *Median-Median Line Algorithm*.

We will illustrate the method using the data from [Table 14.1](#). Here are the steps:

1. Divide the data points on the graph into three, roughly equal groups with vertical lines.
2. Locate the intersection of the vertical and horizontal medians in the end groups.
3. Draw a line between the two intersections.

4. As a possible improvement, shift the line one-third of the way to the median intersection of the middle group of data.

As applied to the data in [Table 14.1](#), the method is illustrated in the figure below. The eight data are divided into groups of 3, 2, and 3. The medians of the right and left group are located and shown by the circles. That establishes the points for the sketched line. The median of the middle group of two points is located and the sketched line might be moved toward that point. Also, the regressed line is shown for comparison purposes. Based on principles, the regressed line is likely better, but the sketched line might have to do, if you didn't have the ability to compute the regression with a computer or with a calculator. With larger data sets, the sketched line tends toward the regressed line.



## 14.3.4 Quantification of the Error in Linear Regression

Any straight line, other than the one computed in [Example 14.4](#), results in a larger sum of squares of the residuals. Thus, the line is unique and, in terms of our chosen criterion, is a “best” fit line through the points. Several additional properties of this fit can be elucidated by examining more closely the way in which residuals were computed and how they relate to the total corrected sum of squares.

From [Eq. \(14.13\)](#), we can define the *error sum of squares*,  $SS_E$ , as

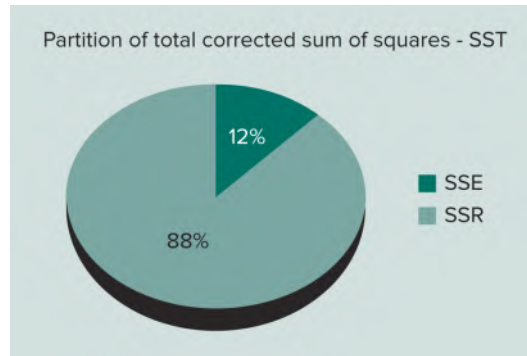
$$SS_E = \sum (y_i - \hat{y}_i)^2 \quad (14.17)$$

and we can recall [Eq. \(14.4\)](#) and split that artificially into two terms.

$$\begin{aligned}
 SS_T &= \sum (y_i - \bar{y})^2 = \sum (\hat{y}_i - \bar{y})^2 + \sum (y_i - \hat{y}_i)^2 \quad (14.18) \\
 &= SS_R + SS_E
 \end{aligned}$$

This is called a *partition of the sum of squares* and can be represented neatly with an example pie chart (  **Fig. 14.8** ). The  $SS_R$  term is called the *regression sum of squares*, and it represents the portion of the total sum of squares explained by the model.

**FIGURE 14.8** Partition of  $SS_T$  into  $SS_E$  and  $SS_R$ .



The partition is subject to several interpretations. First, if  $SS_R$  is a very small piece of the pie, it is likely that the model is not valid, that is, it doesn't explain enough of the behavior. On the other hand, if  $SS_E$  is relatively large, that may indicate that the measurements are just noisy, and this is random behavior we have to live with. So, this leaves us perhaps in the gray area of having to decide when the model "brings enough to the party" to be considered seriously. And, further down the road, if we have competing models, how do we judge which one is the best choice.

There is an important concept associated with regression, and that is called *model adequacy*. It is our expectation that the model accounts for all the systematic behavior in the data, and that the residual errors are random and described by the normal distribution with a given variance,  $\sigma_e^2$ . Additionally, the variance should be consistent and not vary with the independent variable,  $x$ . These requirements for adequacy should be checked and are often overlooked by engineers and scientists. A qualitative way to do this is to create two plots: a histogram chart of the residuals and a graph of the residuals versus  $x$ , or, more commonly, versus the predicted values of  $y$ . The histogram should "look like" a normal distribution, and the graph should not show systematic behavior of the residuals versus the independent variable nor should it show a dramatic change in the spread of the residuals along the  $x$  axis.

The assessment of model adequacy then begs the question, "What to do if the model is inadequate?" These are typical remedies:

1. If there is significant, systematic behavior in the residuals, a straight-line model is not adequate, and a more complicated model should be used. This is usually obvious from a graph of the line and the data. This is dealt with in  **Chap. 15**.
2. If the variance of error changes significantly with  $x$ , the  $y$  data should be transformed, commonly using the logarithm or a power, so that this does not occur.

3. If the distribution of the errors does not agree with the normal distribution, again, a transformation of  $y$  may be in order. If this cannot be resolved, linear regression should not be used.

There are quantitative statistical tests associated with the checks for adequacy, but it is beyond our scope here to introduce those. Commonly, good qualitative judgment is sufficient.

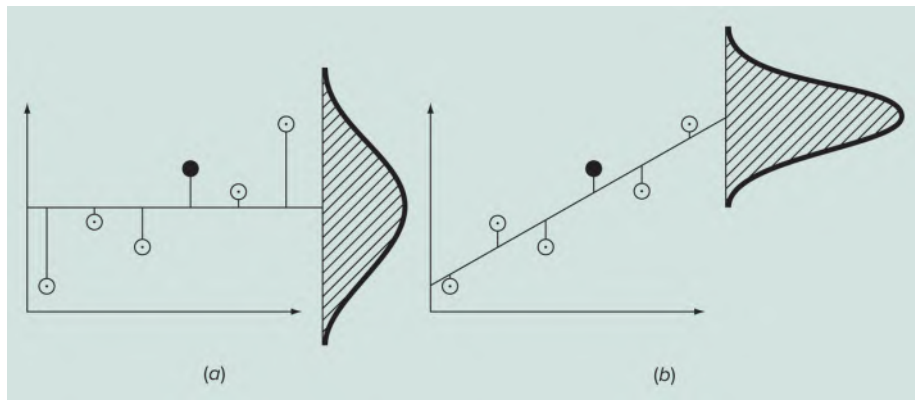
Beyond adequacy, we want to consider how well our straight-line model performs in fitting the data. In a very general sense, if  $SS_R$  is large and  $SS_E$  is tiny, that indicates that the model is worthwhile. If the requirements for adequacy are met, it can be demonstrated that least-squares regression will provide the best, that is, the most likely, estimates for  $a_0$  and  $a_1$  (Draper and Smith, 1981). This is called the *maximum likelihood principle* in statistics. In addition, if these criteria are met, a standard deviation from the regressed line can be computed as

$$s_e = \sqrt{\frac{SS_E}{n - 2}} \quad (14.19)$$

where  $s_e$  is called the *standard error of the estimate*. The subscript  $e$  indicates that this is an estimate of the standard deviation of the true error term in the model,  $\sigma_e$ . Notice that we divide by  $n - 2$  and not  $n$ , since two parameters,  $a_0$  and  $a_1$ , were estimated using the data, and, thus, we have lost two *degrees of freedom*. Note that these calculations make no sense for  $n = 2$ , as with two data points there is a unique straight line passing through them with no residual errors.

Just as was the case with the sample standard deviation [Eq. (14.3)], the standard error of the estimate quantifies the spread of data; however,  $s_e$  describes the spread around the regression line as shown in Fig. 14.9b, in contrast to the sample standard deviation that quantified the spread around the sample mean, as shown in Fig. 14.9a.

**FIGURE 14.9** Regression data showing (a) the spread of the data around the mean of the dependent variable and (b) the spread of the data around the best-fit line. The reduction in the spread in going from (a) to (b), as indicated in the bell-shaped curves to the right, represents the improvement due to linear regression.



These concepts can help us describe the “goodness of fit” of the straight-line model. This will be particularly useful for comparing the performance of several candidate models, as we will see in [Chap. 15](#). A common quantity used to quantify model performance is the *coefficient of determination*,  $R^2$ , which is the fraction of the total corrected sum of squares that is explained by the model,

$$R^2 = \frac{SS_R}{SS_T} = 1 - \frac{SS_E}{SS_T} \quad (14.20)$$

In common statistical lingo, this is called “*R-squared*.” The square root of the coefficient of determination,  $R$ , is called the Pearson correlation coefficient. An alternative formula that can be used to compute  $R$  for the straight-line case is

$$R = \frac{n \sum (x_i y_i) - \sum x_i \sum y_i}{\sqrt{n \sum x_i^2 - (\sum x_i)^2} \sqrt{n \sum y_i^2 - (\sum y_i)^2}} \quad (14.21)$$

## EXAMPLE 14.6

### Estimation of Errors for the Linear Least-Squares Fit

#### Problem Statement

Compute the total corrected sum of squares, the sample standard deviation, the error sum of squares, the regression sum of squares, and the  $R^2$  value for the regression scenario of [Example 14.4](#). Create a plot of the residuals versus the  $x$  values. State your observations on these results.

#### Solution

The data can be set up in tabular form and the necessary sums computed. This is shown in [Table 14.5](#).

**TABLE 14.5** Data and summations needed to compute goodness-of-fit statistics for the data from [Table 14.1](#).

| $i$ | $x_i$ | $y_i$ | $\hat{y}_i$ | $e_i$   | $e_i^2$ | $(y_i - \bar{y})^2$ | $(\hat{y}_i - \bar{y})^2$ |
|-----|-------|-------|-------------|---------|---------|---------------------|---------------------------|
| 1   | 10    | 25    | -39.58      | 64.58   | 4171.0  | 380534.8            | 464385.5                  |
| 2   | 20    | 70    | 155.12      | -85.12  | 7245.3  | 327041.0            | 236931.4                  |
| 3   | 30    | 380   | 349.82      | 30.18   | 910.7   | 68578.5             | 85295.3                   |
| 4   | 40    | 550   | 544.52      | 5.48    | 30.0    | 8441.0              | 9477.3                    |
| 5   | 50    | 610   | 739.23      | -129.23 | 16699.4 | 1016.0              | 9477.3                    |

| $i$ | $x_i$ | $y_i$ | $\hat{y}_i$ | $e_i$   | $e_i^2$        | $(y_i - \bar{y})^2$ | $(\hat{y}_i - \bar{y})^2$ |
|-----|-------|-------|-------------|---------|----------------|---------------------|---------------------------|
| 6   | 60    | 1220  | 933.93      | 286.07  | 81836.9        | 334228.5            | 85295.3                   |
| 7   | 70    | 830   | 1128.63     | -298.63 | 89180.4        | 35391.0             | 236931.4                  |
| 8   | 80    | 1450  | 1323.33     | 126.67  | <u>16044.4</u> | <u>653066.0</u>     | <u>464385.5</u>           |
|     |       |       |             |         | 216118.2       | 1808296.9           | 1592178.7                 |

The total corrected sum of squares is, directly from the table,

$$SS_T = 1,808,296.9$$

The sample standard deviation is then

$$s_y = \sqrt{\frac{SS_T}{n-1}} = \sqrt{\frac{1808297}{7}} \cong 508.26$$

The error sum of squares and the regression sum of squares are, taken from the table,

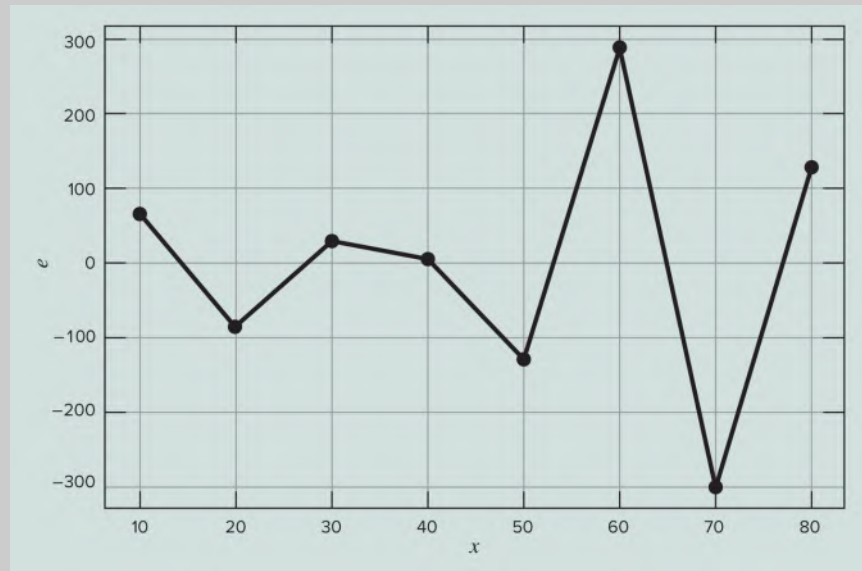
$$SS_E = 216118.2 \quad SS_R = 1,592,178.7$$

Finally, the coefficient of determination is calculated to be

$$R^2 = \frac{SS_R}{SS_T} = \frac{1,592,178.7}{1,808,296.9} \cong 0.88 \text{ or } 88\%$$

Here is the plot of residuals versus  $x$  values.





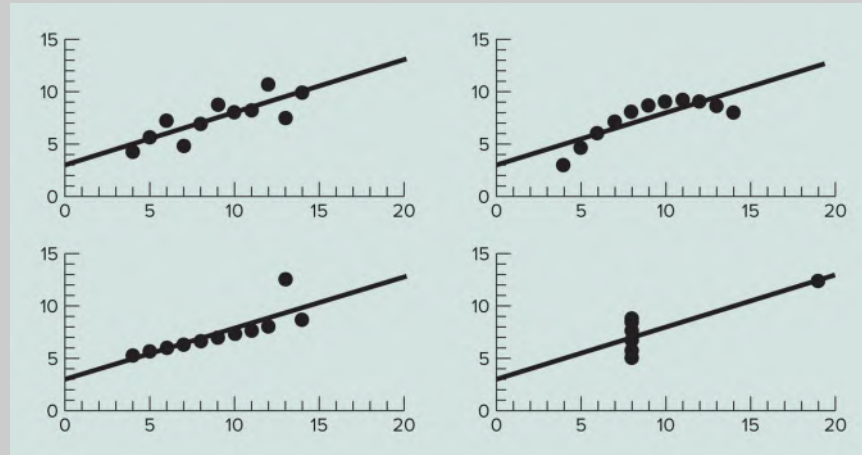
We first observe, via the  $R^2$  value, that the model explains 88% of the variability in the data. It is likely that the model is significant, confirming what is observed in [Fig. 14.7](#).

A matter of concern here, as observed in the plot, is that the variability of the residuals appears to be higher toward the right, at the higher values of  $x$ . It is difficult to support this observation statistically because of the small number of data, but we can ask whether this makes sense physically—that the velocity is more variable at higher body weights.

Before proceeding, a word of caution is in order. Although the coefficient of determination,  $R^2$ , provides a handy measure of goodness-of-fit, you should be careful not to ascribe more meaning to it than is warranted. Just because “R-squared” is close to one does not mean that the fit is necessarily good. For example, it is possible to obtain a relatively high value of  $R^2$  when the underlying relationship between  $x$  and  $y$  is not even linear! However, in such a case, the regressed model will clearly fail adequacy tests. And, as we will see later, in comparing different regression models, it is not advisable to use  $R^2$  as a criterion for selecting the best model.

A nice example was developed by Anscombe (1973). As shown in [Fig. 14.10](#), he came up with four data sets consisting of 11 data points each. Although their graphs are very different, all have the same best-fit equation,  $y = 3 + 0.5x$ , and the same coefficient of determination,  $R^2 = 67\%$ ! It is clear from observation, however, that only the plot in the upper left-hand corner will likely satisfy adequacy requirements. The figure also illustrates why it is important to study your data and regressed models graphically.

**FIGURE 14.10** Anscombe’s four data sets along with the best fit line,  $y = 3 + 0.5x$ .



Always keep in mind the basic definition of  $R^2$ ; that is, it describes the fraction of the variability in the data explained by the model. If you are faced with a situation where measurements have a high content of random noise,  $R^2$  may have a relatively low value, yet a model you develop may be adequate, and, yes, valuable.

Finally, in introducing important concepts related to regression, we want to consider the certainty (or uncertainty) of the parameter estimates (intercept and slope for a straight-line model) that arise from the curve-fitting calculation. We see clearly that there is uncertainty or noise associated with our data. Depending on the situation, data may be “polluted” with noise to a greater or lesser extent. If our model is adequate, this uncertainty is embodied in the variance estimate of the residuals,  $s_e^2$ .  **Figure 14.11** illustrates the concept that this uncertainty is propagated through to our parameter estimates. The question then is how we quantify this propagation.

**FIGURE 14.11** Propagation of uncertainty from the residual variance to the parameter variances.



Without diving too far into applied statistics, we will describe here the formulas used to calculate this propagation. First, we calculate the corrected sum of squares for  $x$ ,

$$S_{xx} = \sum (x_i - \bar{x})^2$$

The variance estimates for the slope,  $a_1$ , and the intercept,  $a_{22}$ , are then

$$s_{a1}^2 = \frac{s_e^2}{S_{xx}} \quad \text{and} \quad s_{a0}^2 = s_e^2 \left( \frac{1}{n} + \frac{x^2}{S_{xx}} \right)$$

The square root of each of these is the standard error for the estimated parameter values.

How can we use these standard errors? Again, without introducing detail, we can be highly confident that the true value of the parameter is within  $\pm$  three standard errors of the estimated value.

Let's see how this applies to the results from the previous  **Example 14.5**.

$$\begin{aligned} S_{xx} &= \sum (x_i - \bar{x})^2 = 6225 \\ s_{a1}^2 &= \frac{s_e^2}{S_{xx}} = \frac{36020}{6225} \cong 5.79 & s_{a1} &\cong 2.41 \\ a_{a0}^2 &= s_e^2 \left( \frac{1}{n} + \frac{\bar{x}^2}{S_{xx}} \right) = 36020 \left( \frac{1}{8} + \frac{2025}{6225} \right) \cong 16220 & s_{a0} &\cong 127 \end{aligned}$$

Using our regressed values for  $a_0$  and  $a_1$  along with three times their standard errors, we come up with

$$-615 \leq a_0 \leq 147 \quad \text{and} \quad 12.24 \leq a_1 \leq 26.7$$

You will note that these intervals are very wide, especially the intercept interval. This has to do with the inadequacy of the straight-line model and the noisiness of the measurements. But, as they say, "It is what it is." With a better and more precise measurements (and perhaps a better model!), the intervals become much tighter.

## 14.4 LINEARIZATION OF NONLINEAR RELATIONSHIPS

Straight-line regression provides a powerful tool for fitting a best line to data. However, it is predicated on the relationship between the dependent (or response) variable and the independent (or regressor) variable being linear. This is often not the case, and the first step in regression modeling is to plot the data to see whether a straight line would be appropriate. In some cases, polynomial models are appropriate. These are described in the next chapter. For others, transformations may be possible to express the data in a form compatible with a straight-line model.

An important distinction in regression modeling is whether the model being considered is based on fundamental knowledge or is on a purely empirical basis. For example, the physics describing the relationship between force and velocity for the bungee jumper proposes

$$F \propto v^2 \quad \text{or} \quad F = c_d v^2$$

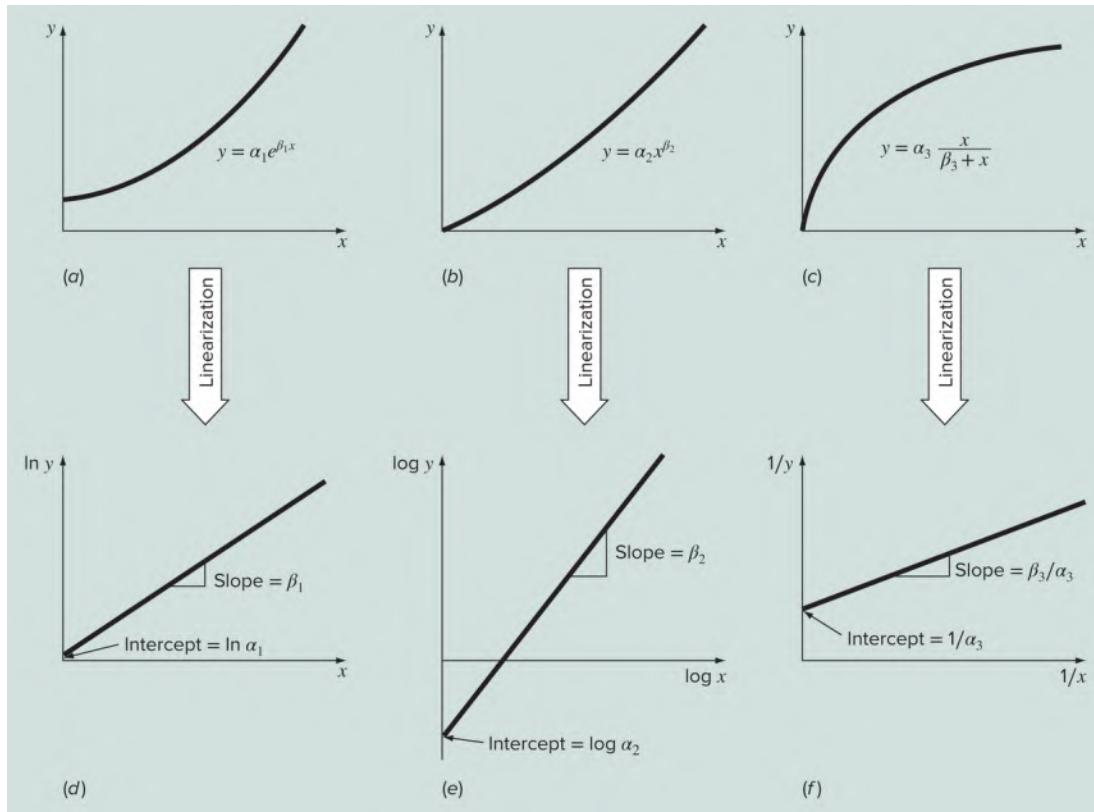
This is a fundamental model as it is predicated on theory, whereas our straight-line model fit to the data earlier in this chapter is empirical. Generally, when a fundamental model is available, we want to use it. At times, the fundamental model is complicated, and it is of interest to replace it with an empirical approximation.

An example of a model that can be transformed into a straight-line form is the *exponential model*:

$$y = \alpha_1 e^{\beta_1 x} \quad (14.22)$$

where  $\alpha_1$  and  $\beta_1$  are constants. This model is used in many fields of engineering and science to characterize quantities that increase (positive  $\beta_1$ ) or decrease (negative  $\beta_1$ ) at a rate that is directly proportional to their magnitude. For example, population growth or radioactive decay can exhibit such behavior. As depicted in  Fig. 14.12a, the equation represents a nonlinear relationship (for  $\beta_1 \neq 0$ ) between  $y$  and  $x$ .

**FIGURE 14.12** (a) The exponential equation, (b) the power equation, and (c) the saturation-growth-rate equation. Parts (d), (e), and (f) are linearized versions of these equations that result from simple transformations.



Another example of a nonlinear model is the *power equation*:

$$y = \alpha_2 x^{\beta_2} \quad (14.23)$$

where  $\alpha_2$  and  $\beta_2$  are constants. This model has wide applicability in all fields of engineering and science. It is frequently used to fit experimental data when the underlying model is not known. As shown in [Fig. 14.12b](#), the equation (for  $\beta_2 \neq 0$ ) is nonlinear and has a zero intercept. This has the form of our fundamental equation for force versus velocity of the bungee jumper, except there the exponent is fixed at two.

A third example of a nonlinear model is the *saturation-growth-rate equation*:

$$y = \frac{\alpha_3 x}{\beta_3 + x} \quad (14.24)$$

where  $\alpha_2$  and  $\beta_2$  are constant coefficients. This model, which is particularly well suited for characterizing population growth rate under limiting conditions, also represents a nonlinear relationship between  $y$  and  $x$  that levels off or “saturates” as  $x$  increases ([Fig. 14.12c](#)). It has many applications, particularly in biologically related areas of both engineering and science.

There are many other nonlinear models that are chosen most often based on observation of the behavior of the data. The three models shown above are, as stated, nonlinear in the  $\alpha$  and  $\beta$  parameters. These models can be fit to data using nonlinear regression, which is presented in [Chap. 15](#). An alternative is to transform these equations into a form linear in the parameters and then apply straight-line regression.

For example, [Eq. \(14.22\)](#) can be linearized by taking its natural logarithm to yield

$$\ln(y) = \ln(\alpha_1) + \beta_1 x \quad (14.25)$$

Thus, a plot of  $\ln(y)$  versus  $x$  will be a straight line with slope  $\beta_1$  and intercept  $\ln(\alpha_1)$ . See

[Fig. 14.12d](#).

[Equation \(14.23\)](#) can be linearized by taking the base-10 logarithm of both sides to give

$$\log_{10}(y) = \log_{10}(\alpha_2) + \beta_2 \log_{10}(x) \quad (14.26)$$

So, a plot of  $\log_{10}(y)$  versus  $\log_{10}(x)$  will yield a straight line with slope  $\beta_2$  and intercept  $\log_{10}(\alpha_2)$ , as shown in [Fig. 14.12e](#). Note that any base logarithm can be used to linearize this model; however, as done here, the base-10 logarithm is commonly applied.

[Equation \(14.24\)](#) is linearized by inverting it with the result

$$\frac{1}{y} = \frac{1}{\alpha_3} + \frac{\beta_3}{\alpha_3} \frac{1}{x} \quad (14.27)$$

Thus, a plot of  $1/y$  versus  $1/x$  will be a straight line with intercept  $1/\alpha_3$  and slope  $\beta_3/\alpha_3$  (

[Fig. 14.11f](#)).

In their transformed versions, these models can be fit with straight-line linear regression to evaluate the coefficients. Then, they can be transformed back to their original format and used for predictive purposes.

There is one cautionary aspect of fitting transformed models that bears mention. If [Eq. \(14.22\)](#) is written so the left-hand side is the actual value of  $y$  and not the model prediction, it would be

$$y = \alpha_1 e^{\beta_1 x} + \varepsilon \quad (14.28)$$

where  $\varepsilon$  represents the error term which must be added to the model to achieve the actual measured  $y$ . If this model is adequate,  $\varepsilon$  might be a random variable representing the noise in the measurement process.

It would have a mean of zero and a variance of  $\sigma_e^2$ . When [Eq. \(14.28\)](#) is applied to a set of  $\{x_i, y_i, i = 1, \dots, n\}$  values, it yields, for chosen values of  $\alpha_1$  and  $\beta_1$ , a set of residual errors,  $\{e_i, i = 1, \dots, n\}$ . These residuals can be analyzed for model adequacy and used to estimate  $\sigma_e^2$ .

It should be clear that, when a transformation is applied to a model such as [Eq. \(14.22\)](#), the error term for the resulting model is not the  $\varepsilon$  in [Eq. \(14.28\)](#)—it is different, and, for example, it no longer represents the noise in the measurement process. So, when there is a desire to characterize and estimate the error term in the original model, it is more typical to apply nonlinear regression directly to that model and not carry out the transformation. Additionally, there is no guarantee that a transformed model deemed adequate will assure that the original model is adequate, and vice versa.

Back to transformation, the following example illustrates this procedure for the power model.

## EXAMPLE 14.7

### Fitting Data with the Power Equation

#### Problem Statement

Fit [Eq. \(14.23\)](#) to the data in [Table 14.1](#) using a logarithmic transformation.

#### Solution

The data can be set up in tabular form and the necessary sums computed as in [Table 14.6](#). The sample means (averages) of the log terms are computed from the sums in the table as

$$\log_{10}(x_i) = \frac{12.6055}{8} \cong 1.5757 \quad \log_{10}(y_i) = \frac{20.5153}{8} \cong 2.5644$$

**TABLE 14.6** Data, summations, and results of the fit of the power model to the data from [Table 14.1](#) :

| $i$ | $x_i$ | $y_i$ | $\log_{10}(x_i)$ | $\log_{10}(y_i)$ | $(\log_{10}(x_i))^2$ | $\log_{10}(x_i)\log_{10}(y_i)$ | $\log_{10}(\hat{y}_i)$ | $\hat{y}_i$ |
|-----|-------|-------|------------------|------------------|----------------------|--------------------------------|------------------------|-------------|
| 1   | 10    | 25    | 1.0000           | 1.3979           | 1.0000               | 1.3979                         | 1.4221                 | 26.43       |
| 2   | 20    | 70    | 1.3010           | 1.8451           | 1.6927               | 2.4005                         | 2.0194                 | 104.58      |
| 3   | 30    | 380   | 1.4771           | 2.5798           | 2.1819               | 3.8107                         | 2.3688                 | 233.80      |
| 4   | 40    | 550   | 1.6021           | 2.7404           | 2.5666               | 4.3902                         | 2.6167                 | 413.75      |
| 5   | 50    | 610   | 1.6990           | 2.7853           | 2.8865               | 4.7322                         | 2.8090                 | 644.21      |
| 6   | 60    | 1220  | 1.7782           | 3.0864           | 3.1618               | 5.4880                         | 2.9661                 | 924.98      |
| 7   | 70    | 830   | 1.8451           | 2.9191           | 3.4044               | 5.3860                         | 3.0990                 | 1255.94     |
| 8   | 80    | 1450  | <u>1.9031</u>    | <u>3.1614</u>    | <u>3.6218</u>        | <u>6.0164</u>                  | 3.2140                 | 1636.95     |
|     |       |       | 12.6055          | 20.5153          | 20.5156              | 33.6219                        |                        |             |

The slope and intercept can then be calculated using [Eqs. \(14.15\)](#) and [\(14.16\)](#).

$$\beta_2 = \frac{8 \cdot 33.6219 - 12.6055 \cdot 20.5153}{8 \cdot 20.5156 - 12.6055^2} \cong 1.9842$$

and

$$\log_{10}(\alpha_2) = 2.5644 - 1.9842 \cdot 1.5757 \cong -0.5620$$

The least-squares fit of the transformed model is then

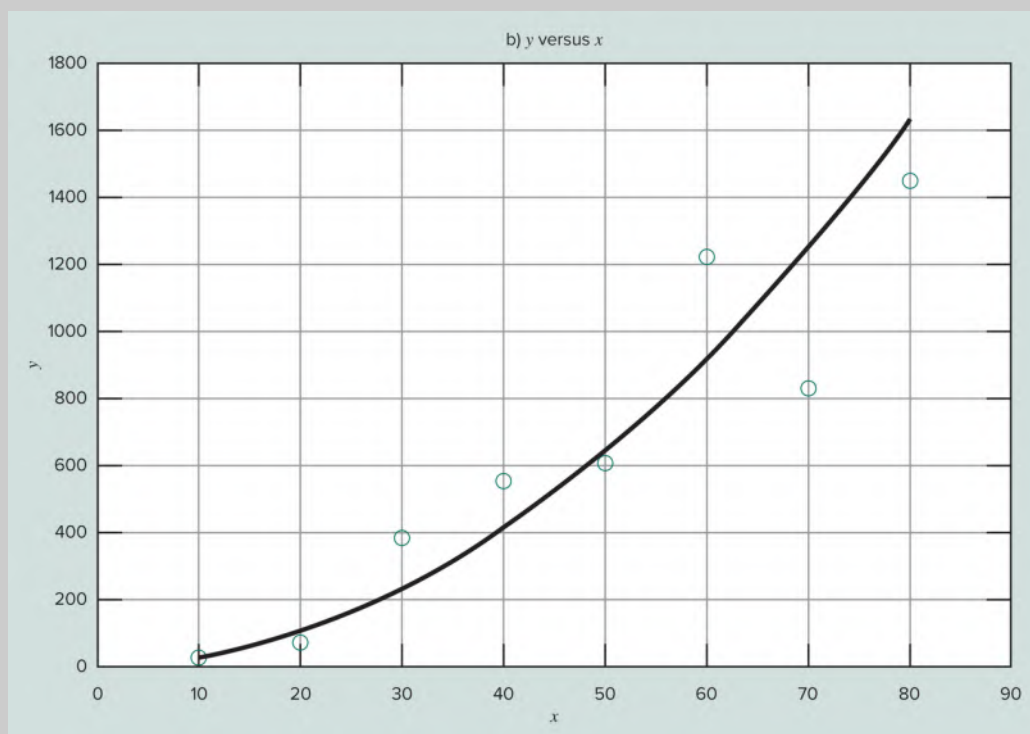
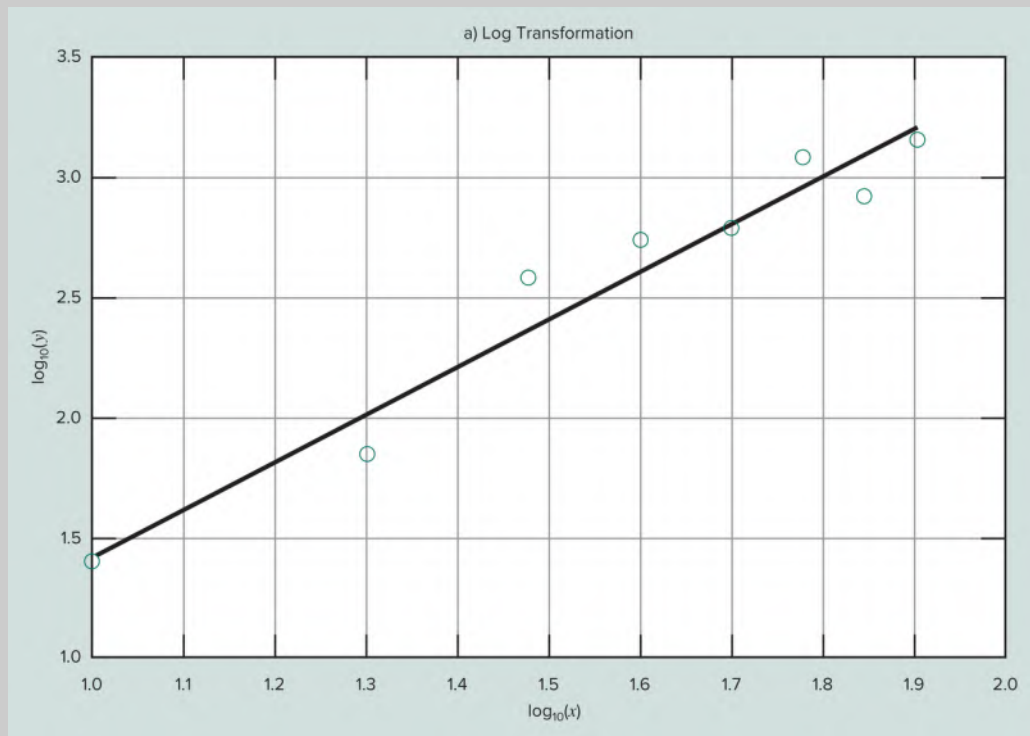
$$\log_{10}(y) = -0.5620 + 1.9842 \log_{10}(x)$$

This fit along with the log-transformed data is shown in [Fig. 14.13a](#). We can also display the fit using the original coordinates. To do this, the value of  $\alpha_2$  is computed as  $10^{-0.5620} \cong 0.2741$ . Using force and velocity in place of  $y$  and  $x$ , the fitted model becomes

$$F = 0.2741v^{1.9842}$$

**FIGURE 14.13** Least-squares fit of a power model to the data from [Table 14.1](#). On the left, the fit of the transformed data, and, on the right, the fitted power equation along with the original data.

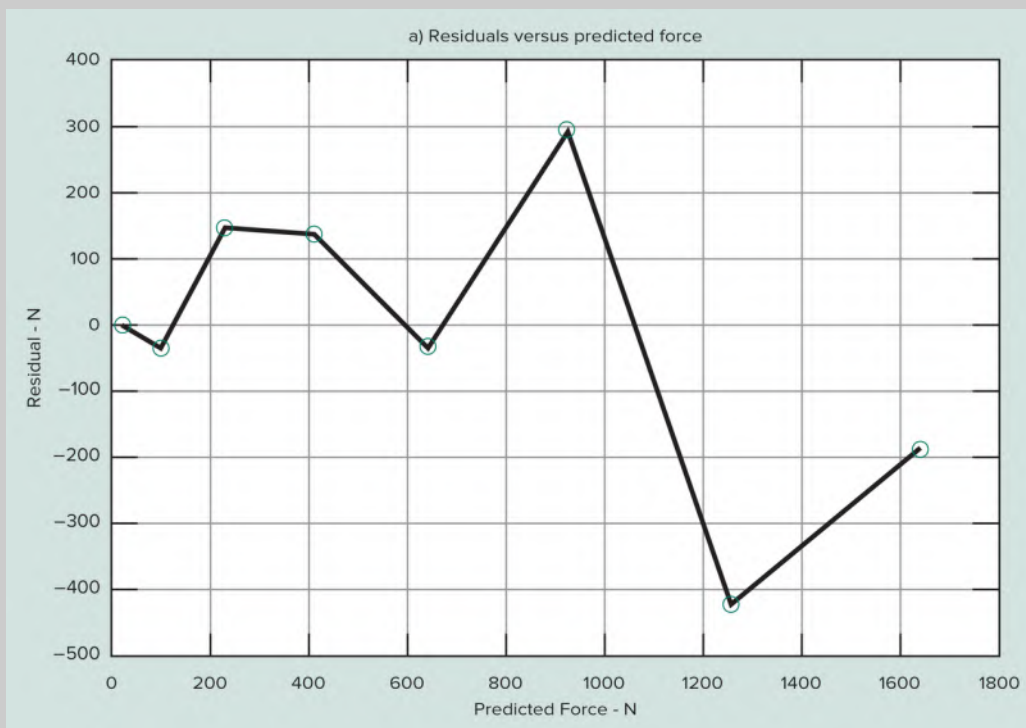


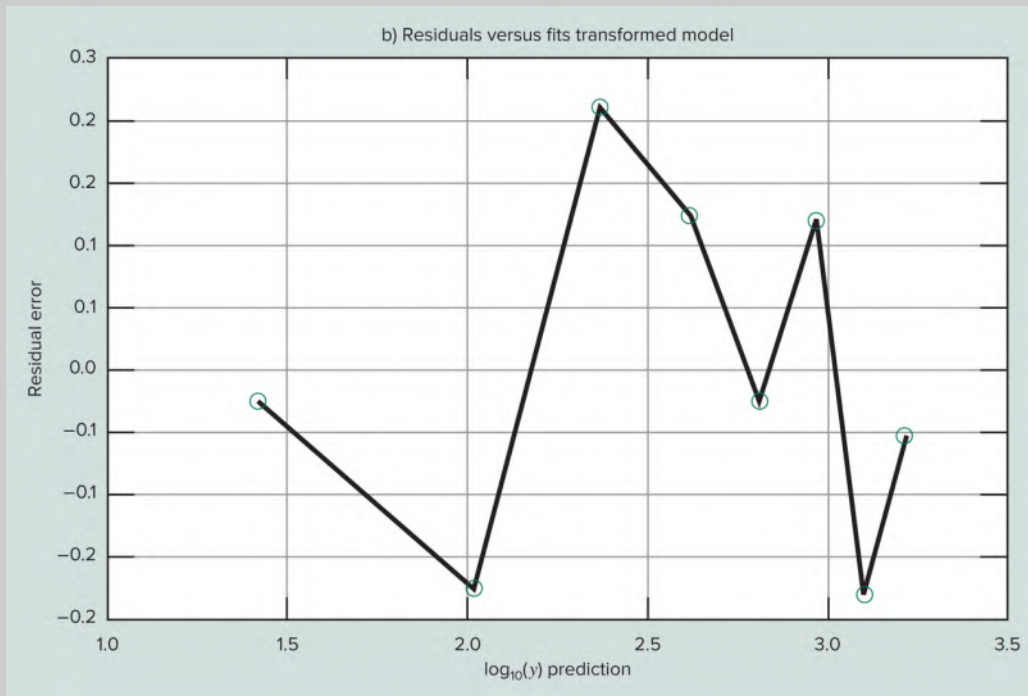


A line plot of this equation along with the original data is shown in [Fig. 14.13b](#). We note here that, based on physics, we expected the exponent to be two, and, statistically, 1.9842 cannot be distinguished from 2.0000; consequently, our fit has served to confirm theory and provide an estimate for the drag coefficient,  $c_d = 0.2741 \text{ kg/m}$ . An issue we will address later is how certain this estimate is.

Additionally, in [Fig. 14.14a](#), a plot of the residuals versus the predicted  $y$  values is presented. In statistical jargon, this is called a *residuals versus fits plot* and is typically presented to analyze adequacy. Although there is no systematic variation observed, there remains a concern that the variability increases at the higher  $y$  values; however, that concern is diminished by the plot of the residuals of the transformed model versus the predicted  $\log_{10}(y)$  values. Here, the variability is more consistent. This shows that we have been able to improve adequacy through fitting the transformed model, a fortunate by-product.

**FIGURE 14.14** (a) Residuals versus the predicted force and (b) residuals of the transformed model versus the predicted  $\log_{10}(y)$  values.





A question that might arise is why we didn't present a histogram of the residuals. The answer here is quite simple. We really don't have enough data for a histogram to be meaningful. There are other ways to check whether the residuals adhere to a normal distribution, but we do not take the time for that here.

The fit in [Example 14.6](#) can be compared to the straight-line fit in [Example 14.4](#). The  $R^2$  value in [Example 14.6](#) is 95%, and it was 88% in [Example 14.4](#). This indicates that the power equation accounts for more of the total corrected sum of squares than the original linear model. We might expect this, since the power model aligns with our fundamental equation from physics. As we will see in [Chap. 15](#), it is straightforward to fit the power model with a given exponent of 2.

## 14.4.1 General Comments on Straight-Line Linear Regression

Before proceeding to a more general approach to linear regression, including curvilinear and multivariable models, we must emphasize the introductory nature of the foregoing material on straight-line regression. We have focused on the simple derivation and practical use of straight-line formulas to fit data. You should be cognizant that there are theoretical aspects of regression that are of practical importance but are beyond the scope of this book. For example, several statistical assumptions inherent to the least-squares procedure we have introduced are

1. Each  $x$  has a fixed value. It is deterministic, not statistical in nature. It is known without error.
2. The  $y$  values are independent variables and their variance is consistent.
3. The  $y$  values for a given  $x$  are normally distributed.

Such assumptions are relevant to the proper application of regression. We have touched on (2) and (3) in our discussion on model adequacy. You are encouraged to consult other references, such as Montgomery and Runger (2014), and, of course, we encourage our students to take a full-length course in applied statistics.

## 14.5 COMPUTER APPLICATIONS

Straight-line regression is so commonplace that it can be implemented on most pocket calculators. In this section, we will show how a simple Python function can be developed to determine the slope and intercept, the  $R^2$  value, and the standard error of the estimate. We will also show how straight-line regression can be implemented with the `polyfit` function from the NumPy `stats` submodule.

### 14.5.1 Python function: `strlinregr`

A Python function for straight-line regression can be developed easily (Fig. 14.15). The required summations are computed either with the NumPy `sum` function or with a `for` loop. These are then used to compute the slope and intercept with Eqs. (14.15) and (14.16). The residuals are computed internally and used to calculate  $SS_E$ . The total corrected sum of squares,  $SS_T$ , is computed and then the regression sum of squares,  $SS_R$ . These are used to compute  $R^2$  and the standard error of the estimate,  $SE$ . The function returns the intercept, slope,  $R^2$ , and  $SE$ .

**FIGURE 14.15** Python function `strlinregr` to implement straight-line linear regression.

```
def strlinregr(x,y):
    n = len(x)
    if len(y) != n: return 'x and y must be of same length'
    sumx = np.sum(x)
    xbar = sumx/n
    sumy = np.sum(y)
    ybar = sumy/n
    sumsqx = 0
    sumxy = 0
    for i in range(n):
        sumsqx = sumsqx + x[i]**2
        sumxy = sumxy + x[i]*y[i]
    a1 = (n*sumxy-sumx*sumy)/(n*sumsqx-sumx**2)
    a0 = ybar - a1*xbar
    e = np.zeros((n))
    SST = 0
    SSE = 0
    for i in range(n):
        e[i] = y[i] - (a0+a1*x[i])
        SST = SST + (y[i]-ybar)**2
        SSE = SSE + e[i]**2
    SSR = SST - SSE
    Rsq = SSR/SST
    SE = np.sqrt(SSE/(n-2))
    return a0,a1,Rsq,SE
```

To check the performance of the `strlinregr` function, the following Python code is added.

```
x = np.array([10., 20., 30., 40., 50., 60., 70., 80.])
y = np.array([25., 70., 380., 550., 610., 1220., 830., 1450.])
a0,a1,Rsq,SE = strlinregr(x,y)
print('Intercept = {0:7.2f}'.format(a0))
print('Slope = {0:7.3f}'.format(a1))
print('R-squared = {0:5.3f}'.format(Rsq))
```

```
print('Standard error = {0:7.2f}'.format(SE))
```

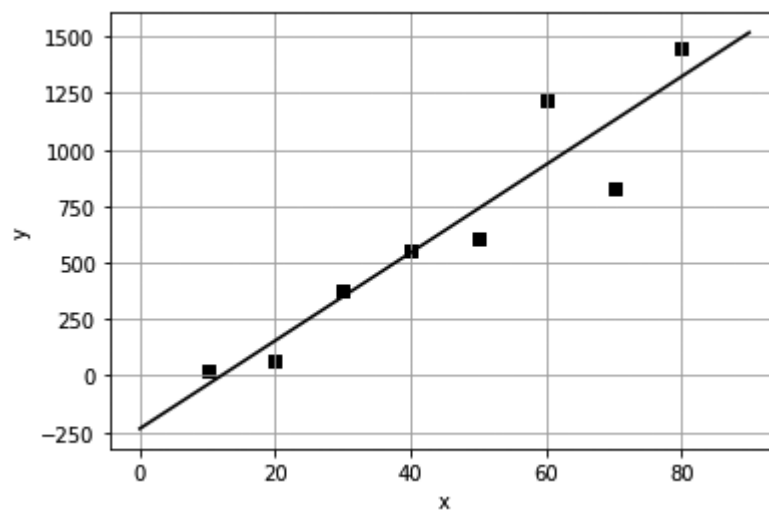
with the results displayed as

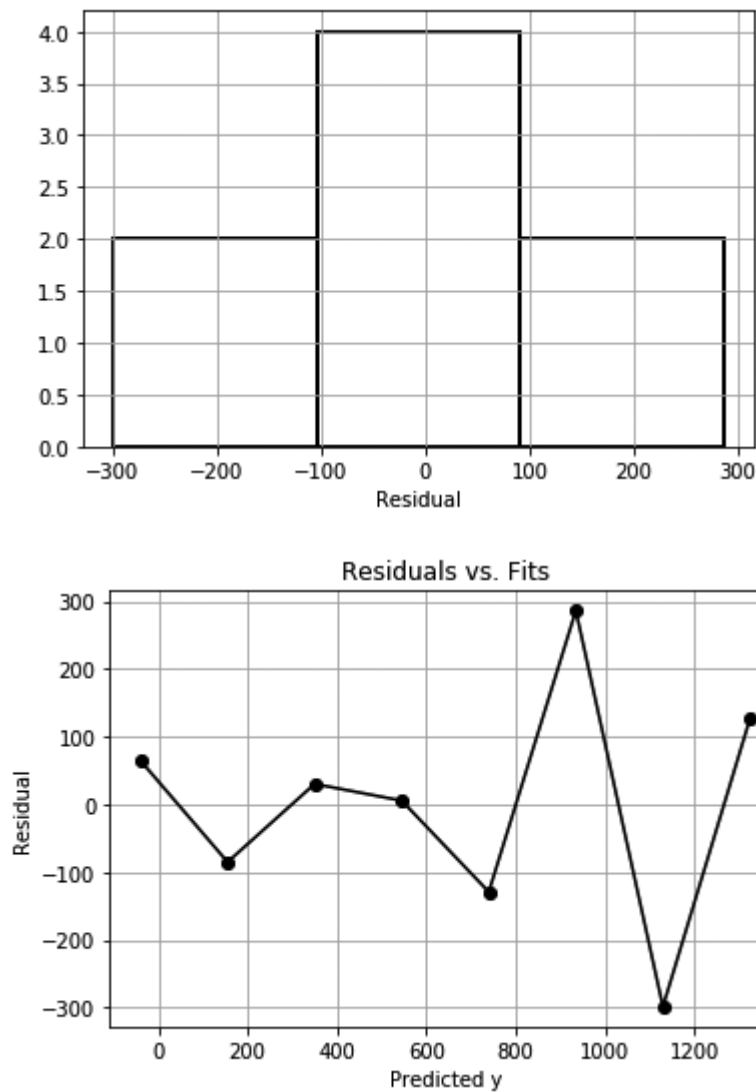
```
Intercept = -234.29  
Slope = 19.470  
R-squared = 0.880  
Standard error = 189.79
```

Additional code can be added for graphical displays of the fitted line and the data, a histogram of the residuals (not very informative here because of the low  $n$ ), and a plot of the residuals versus the fits.

```
xline = np.linspace(0,90,10)  
yline = a0 + a1*xline  
yhat = a0 + a1*x  
e = y - yhat  
import pylab  
pylab.scatter(x,y,c='k',marker='s')  
pylab.plot(xline,yline,c='k')  
pylab.grid()  
pylab.xlabel('x')  
pylab.ylabel('y')  
pylab.figure()  
pylab.hist(e,bins=3,color='w',edgecolor='k',linewidth=2.)  
pylab.grid()  
pylab.xlabel('Residual')  
pylab.figure()  
pylab.plot(yhat,e,c='k',marker='o')  
pylab.grid()  
pylab.xlabel('Predicted y')  
pylab.ylabel('Residual')  
pylab.title('Residuals vs. Fits')
```

These duplicate the results from [Examples 14.4](#) and [14.5](#).





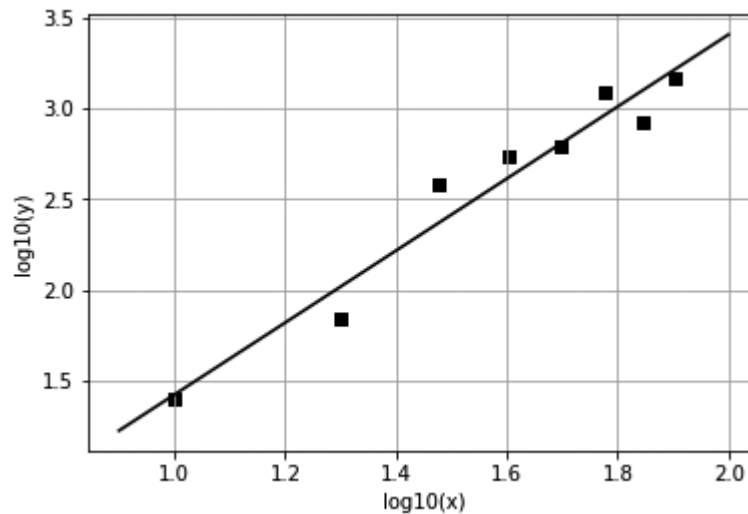
We can also apply the `strlinregr` function with the power equation model and the transformed  $x$  and  $y$  data.

```
logx = np.log10(x)
logy = np.log10(y)
a0,a1,Rsq,SE = strlinregr(logx,logy)
print('Intercept = {0:7.2f}'.format(a0))
print('Slope = {0:7.3f}'.format(a1))
print('R-squared = {0:5.3f}'.format(Rsq))
print('Standard error = {0:7.2f}'.format(SE))
logxline = np.linspace(0.9,2,10)
logyline = a0 + a1*logxline
logyhat = a0 + a1*logx
loge = logy - logyhat
pylab.figure()
pylab.scatter(logx,logy,c='k',marker='s')
pylab.plot(logxline,logyline,c='k')
pylab.grid()
pylab.xlabel('log10(x)')
```

```
pylab.ylabel('log10(y)')
```

The results and the plot are

```
Intercept = -0.56  
Slope = 1.984  
R-squared = 0.948  
Standard error = 0.15
```



## 14.5.2 Python NumPy functions: `polyfit` and `polyval`

Python's NumPy module has a built-in function, `polyfit`, that fits an  $n$ th-order polynomial to data using linear least squares. It can be applied as

```
p = np.polyfit(x,y,n)
```

where  $x$  and  $y$  are arrays containing values for the independent and dependent variables, respectively, and  $n$  is the degree of the polynomial. The function returns the polynomial coefficients in an array,  $p$ , from highest to lowest order term. We should note that it represents the polynomial in decreasing powers of  $x$  as in

$$f(x) = p[0]x^n + p[1]x^{n-1} + \cdots + p[n-1]x + p[n]$$

Because a straight line is a first-order polynomial, `polyfit(x,y,1)` will return the slope and intercept of the best-fit straight line. Here is Python code applied to our previous example.

```
import numpy as np  
x = np.array([10., 20., 30., 40., 50., 60., 70., 80.])  
y = np.array([25., 70., 380., 550., 610., 1220., 830., 1450.])  
p = np.polyfit(x,y,1)
```



```
print('Straight-line coefficients are:\n',p)
```

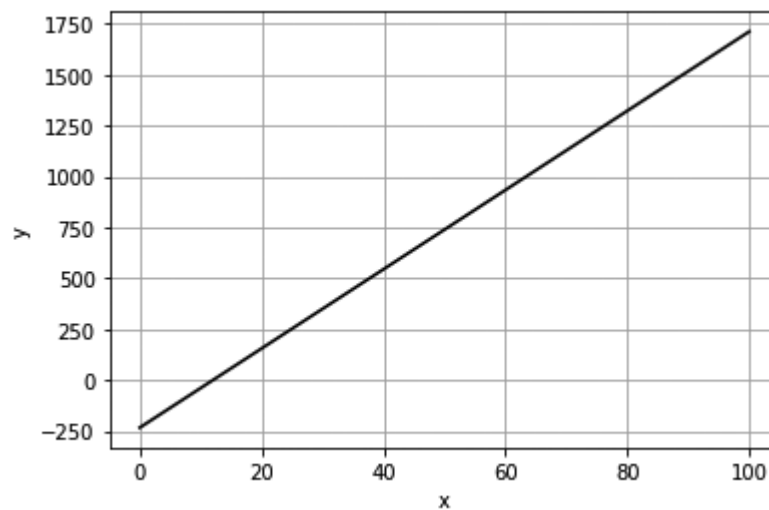
And the results displayed are

```
Straight-line coefficients are:  
[ 19.4702381 -234.28571429]
```

So, the slope is approximately 19.47 and the intercept is -234.3.

Another function, `polyval`, can then be used to compute predicted values of  $y$  given values of  $x$ . These can be used in a plot.

```
x = np.linspace(0,100,2)  
import pylab  
pylab.plot(x,np.polyval(p,x),c='k')  
pylab.grid()  
pylab.xlabel('x')  
pylab.ylabel('y')
```



## ENZYME KINETICS

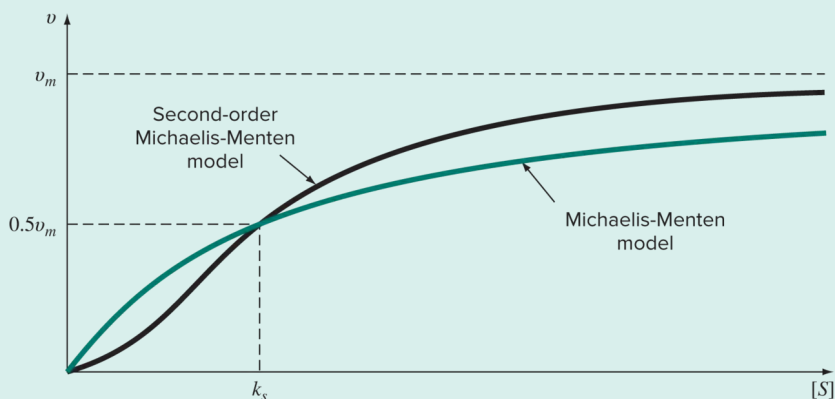
### Background

*Enzymes* act as catalysts to speed up the rate of chemical reactions taking place in living cells. In most cases, they convert one chemical, the *substrate*, into another, the *product*. The *Michaelis-Menten* model is commonly used to describe such reactions:

$$v = \frac{v_m S}{k_s + S} \quad (14.29)$$

where  $v$  = the reaction rate,  $v_m$  = the maximum reaction rate,  $S$  = the substrate concentration, and  $k_s$  = the *half-saturation constant*. As shown in  **Fig. 14.16**, the equation describes a saturating relationship which levels off as  $S$  increases. The figure also illustrates that the half-saturation constant corresponds to the substrate concentration at which the reaction rate is one half of the maximum.

**FIGURE 14.16** Two versions of the Michaelis-Menten model of enzyme kinetics.



Although the Michaelis-Menten model provides a nice starting point, it has been refined and extended to incorporate additional features of enzyme kinetics. One simple extension involves so-called *allosteric enzymes*, where the binding of a substrate molecule at one site leads to enhanced binding of subsequent molecules at other sites. For cases with two interacting binding sites, the following second-order version often results in a better fit:

$$v = \frac{v_m S^2}{k_s^2 + S^2} \quad (14.30)$$

This model also describes a saturating curve, but, as shown in [Fig. 14.16](#), the squared concentrations produce a curve with zero initial slope and an inflection point at  $S = k_s$ . The curve is more S-shaped or *sigmoidal*.

## Problem Statement

Suppose you are provided with the following data:

|          |       |       |       |       |       |       |       |
|----------|-------|-------|-------|-------|-------|-------|-------|
| <b>S</b> | 1.3   | 1.8   | 3.0   | 4.5   | 6.0   | 8.0   | 9.0   |
| <b>v</b> | 0.070 | 0.130 | 0.220 | 0.275 | 0.335 | 0.350 | 0.360 |

Employ linear regression to fit these data with linearized versions of [Eqs. \(14.29\)](#) and [\(14.30\)](#). Aside from estimating the model parameters, assess the comparative performance of the models and their adequacy.

## Solution

[Equation \(14.29\)](#), which is in the format of the saturation-growth-rate model of [Eq. \(14.24\)](#) can be linearized, as was done in [Eq. \(14.27\)](#), to give

$$\frac{1}{v} = \frac{1}{v_m} + \frac{k_s}{v_m} \frac{1}{S}$$

The `strlinregr` function from [Fig. 14.15](#) can then be used to determine the least squares fit for the model,

```
S = np.array([1.3, 1.8, 3., 4.5, 6., 8., 9.])
v = np.array([0.07, 0.13, 0.22, 0.275, 0.335, 0.35, 0.36])
x = 1./S
y = 1./v

a0,a1,Rsq,SE = strlinregr(x,y)
print('Intercept = {0:7.4f}'.format(a0))
print('Slope = {0:7.3f}'.format(a1))
print('R-squared = {0:5.3f}'.format(Rsq))
print('Standard error = {0:7.3f}'.format(SE))

vm = 1/a0
ks = a1*vm

print('Estimated maximum rate = {0:5.3f}'.format(vm))
print('Estimated half-saturation constant = {0:6.3f}'.format(ks))
```

with the results

```
Intercept = 0.1902
Slope = 16.402
R-squared = 0.934
Standard error = 1.185
Estimated maximum rate = 5.257
Estimated half-saturation constant = 86.226
```

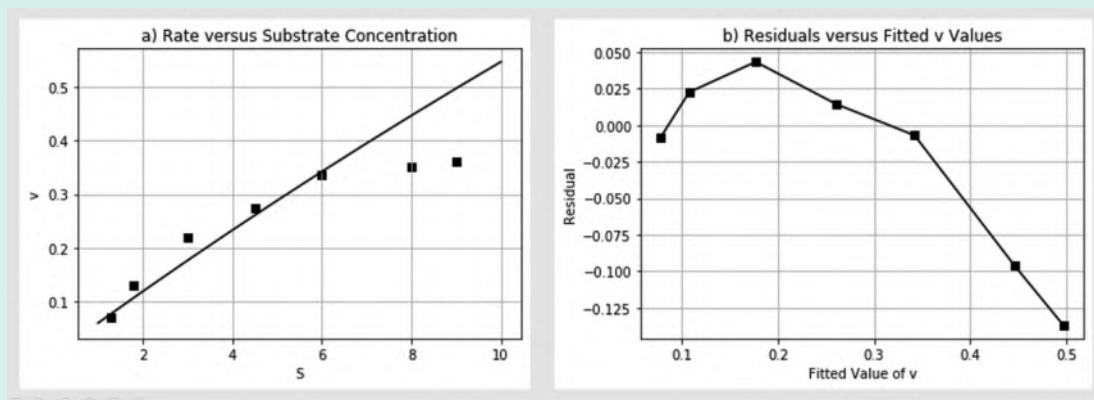
The best-fit model is then

$$v = \frac{5.257S}{86.226 + S}$$

Although the  $R^2$  value is encouraging in terms of the fraction of the variability in  $1/v$  accounted for by the model, inspection of the estimated coefficients might raise doubts. For example, the maximum rate, 5.257 is much higher than the highest observed rate, 0.36. Also, the half-saturation rate, 86.226, is much greater than the maximum substrate concentration, 9.

This problem is underscored when we plot the fitted line along with the data in [Fig. 14.17a](#). Although the line captures the upward trend, it cannot align with the curvature and saturation shown in the data. Also, in [Fig. 14.17b](#), it is clear that there is systematic behavior in the residuals plot. This indicates that the model is inadequate despite the relatively high  $R^2$  value.

**FIGURE 14.17** Results from regression of transformed, first-order Michaelis-Menten model. (a) Rate versus substrate concentration—data and model line and (b) residuals versus fitted  $v$  values.



The results from the first-order model lead us to consider the second-order model, which can be transformed for straight-line regression by inverting the untransformed model,

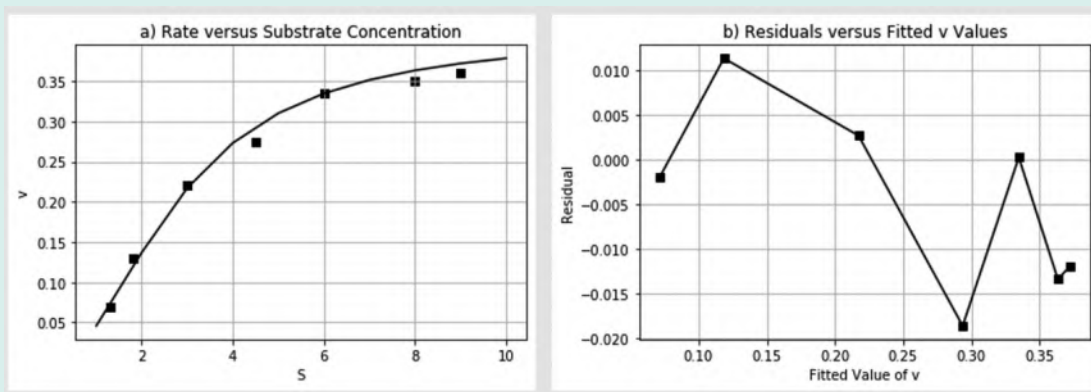
$$\frac{1}{v} = \frac{1}{v_m} + \frac{k_s^2}{v_m} \frac{1}{S^2}$$

If we apply the `strlinregr` function to this form of the model, here are the results:

```
Intercept = 2.4492
Slope = 19.376
R-squared = 0.993
Standard error = 0.389
Estimated maximum rate = 0.408
Estimated half-saturation constant = 2.813
```

You will note immediately the improvement in the  $R^2$  and standard error values. This is encouraging. Also, the plots in [Fig. 14.18](#) are revealing. The model captures the curvature and saturation of the data much more closely in [Fig. 14.18a](#), and the residuals versus fits plot in [Fig. 14.18b](#) is absent the systematic behavior observed in [Fig. 14.17b](#). The model satisfies adequacy requirements.

**FIGURE 14.18** Results from regression of transformed, second-order Michaelis-Menten model. (a) Rate versus substrate concentration—data and model line, (b) residuals versus fitted  $v$  values.



The fitted, second-order, Michaelis-Menten model is

$$v = \frac{0.408S^2}{2.813^2 + S^2}$$

We also observe that our parameter estimates are more sensible here than with the first-order model. We conclude that the second-order model provides a much better fit of the data set. This suggests that we are dealing with an allosteric enzyme.

Page 342

Beyond this result, there are other general conclusions we can draw from this case study. First, we should not rely on performance statistics, such as  $R^2$ , as the sole basis for evaluating goodness of fit of a single model. Second, we should always analyze residuals to assess adequacy of the model. Third, our analysis should always be accompanied by plots of the results. With larger data sets, in the 10s or 100s of data points, we would include a histogram chart of the residuals.

Finally, as we have stated earlier in the chapter, we must be aware that the residuals of the transformed model are not the same as those for the original model. Adequacy should be checked for both, and consistency of variance of the residuals, in particular, should be checked for the transformed model. The conflict between analysis of residuals for the original and transformed model suggests that there are occasions where it would be preferable to fit the original model, which is nonlinear in the parameters, or use an empirical model, linear in its parameters, that accommodates curvature, such as a polynomial function. We will describe how this is done in [Chap. 15](#).

# PROBLEMS

---

## 14.1 Given the data

|      |      |      |      |      |
|------|------|------|------|------|
| 0.90 | 1.42 | 1.30 | 1.55 | 1.63 |
| 1.32 | 1.35 | 1.47 | 1.95 | 1.66 |
| 1.96 | 1.47 | 1.92 | 1.35 | 1.05 |
| 1.85 | 1.74 | 1.65 | 1.78 | 1.71 |
| 2.29 | 1.82 | 2.06 | 2.14 | 1.27 |

Determine (a) the sample mean, (b) median, (c) mode, (d) range, (e) sample standard deviation, (f) sample variance, (g) the MAD, and (h) coefficient of variation. Do you find any outliers in the data? If so, justify your decision.

**14.2** Construct a histogram chart for the data in  **Prob. 14.1**. Use five intervals from 0.85 to 2.35. If a value falls on a bin boundary, devise a scheme for assigning it to either the lower or higher bin.

## 14.3 Given the data

|       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|
| 29.65 | 28.55 | 28.65 | 30.15 | 29.35 | 29.75 | 29.25 |
| 30.65 | 28.15 | 29.85 | 29.05 | 30.25 | 30.85 | 28.75 |
| 29.65 | 30.45 | 29.15 | 30.45 | 34.65 | 29.35 | 29.75 |
| 31.25 | 29.45 | 30.15 | 29.65 | 30.55 | 29.65 | 29.25 |

- (a) Determine the following sample statistics: mean, median, mode, variance, standard deviation, MAD, and coefficient of variation. If there are any suspicious values that appear to be outliers, identify them. Remove them and repeat the calculations, noting any significant differences. Leave them out for the subsequent parts of this problem.
- (b) Construct a histogram chart of the data. Determine the number of bins to use following the guidelines. Choose bin boundaries so that no data fall on them.
- (c) Based on the sample mean and standard deviation, construct a scaled normal distribution curve on the chart. Comment on the comparison.
- (d) Evaluate the 68% normal probability rule for these data. Is it reasonable?

It is suggested that you use Python for this problem.

---

Page 343

---

**14.4** Using the same approach as was employed to derive  **Eqs. (14.15)** and  **(14.16)**, derive the least-squares fit for the following model:

$$y = a_1x + \varepsilon$$

That is, determine the slope that results from the least-squares calculation for a straight line with zero intercept. Fit the following data with this model and present the result graphically.

|          |   |   |   |   |    |    |    |    |    |
|----------|---|---|---|---|----|----|----|----|----|
| <b>x</b> | 2 | 4 | 6 | 7 | 10 | 11 | 14 | 17 | 20 |
| <b>y</b> | 4 | 5 | 6 | 5 | 8  | 8  | 6  | 9  | 12 |

**14.5** Use least-squares regression to fit a straight line to the data

|          |   |   |   |   |   |    |    |    |    |    |
|----------|---|---|---|---|---|----|----|----|----|----|
| <b>x</b> | 0 | 2 | 4 | 6 | 9 | 11 | 12 | 15 | 17 | 19 |
| <b>y</b> | 5 | 6 | 7 | 6 | 9 | 8  | 8  | 10 | 12 | 12 |

Along with the slope and intercept, compute the standard error of the estimate,  $s_e$ , and the coefficient of determination,  $R^2$ . Then, repeat the calculations switching the two variables. Interpret your results.

**14.6** Fit a power model to the data from [Table 14.1](#), but use the natural logarithm for the transformation of the data.

**14.7** The following data were gathered to investigate the relationship between pressure and volume for 10 kg sulfur dioxide ( $\text{SO}_2$ ) gas at 400 K.

|                         |       |       |       |       |       |       |       |       |       |
|-------------------------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| <b>V(m<sup>3</sup>)</b> | 0.156 | 0.234 | 0.312 | 0.390 | 0.468 | 0.546 | 0.624 | 0.702 | 0.780 |
| <b>P (MPa)</b>          | 3.326 | 2.217 | 1.663 | 1.330 | 1.109 | 0.950 | 0.831 | 0.739 | 0.665 |

It is desired to investigate whether the ideal gas law is valid over the range of conditions measured. To do that, fit a straight-line model to the data, but  $P$  versus  $1/V$ , evaluate the fit including a plot of residuals versus  $1/V$ , and compare the model to the pressure predicted by the ideal gas law,

$$PV = nRT$$

where  $P$  = pressure in MPa,  $V$  = volume in  $\text{m}^3$ ,  $n$  = number of kmols of gas,  $R$  = the gas law constant,  $8.3145 \times 10^{-3} (\text{MPa} \cdot \text{m}^3 / (\text{kmol} \cdot \text{K}))$ , and  $T$  = temperature in kelvins (K). Note: The molecular weight of  $\text{SO}_2$  is 64.07 kg/kmol. The MPa unit is megapascals or  $10^6$  Pa. A pascal is  $1 \text{ N/m}^2$ . A kmol is 1000 mols.

**14.8** Beyond the examples in [Fig. 14.12](#), there are other models that can be linearized using transformations. Here is one:

$$y = \alpha_4 x e^{\beta_4 x}$$

Linearize this model and use it to estimate  $\alpha_4$  and  $\beta_4$  based on the following data. Present plots of your fitted  $y$  versus  $x$  line along with the data and the residuals versus predicted  $y$ 's.

|                       |      |      |      |      |      |      |      |      |      |
|-----------------------|------|------|------|------|------|------|------|------|------|
| <b><math>x</math></b> | 0.1  | 0.2  | 0.4  | 0.6  | 0.9  | 1.3  | 1.5  | 1.7  | 1.8  |
| <b><math>y</math></b> | 0.75 | 1.25 | 1.45 | 1.25 | 0.85 | 0.55 | 0.35 | 0.28 | 0.18 |

**14.9** The concentration of *E. coli* (*Escherichia coli*) bacteria in a swimming area is monitored after a rainstorm:

|                                    |      |      |      |     |     |     |
|------------------------------------|------|------|------|-----|-----|-----|
| <b><math>t</math> (hr)</b>         | 4    | 8    | 12   | 16  | 20  | 24  |
| <b><math>c</math> (CFU/100 mL)</b> | 1600 | 1320 | 1000 | 890 | 650 | 560 |

The time is measured in hours after the storm ended, and the unit CFU is a “colony-forming unit.” Use the data to estimate (a) the concentration at the storm’s end ( $t = 0$ ), and (b) the time at which the concentration will reach 200 CFU/mL. Note: Your choice of model should be consistent with the fact that negative concentrations are not possible, and that the bacteria concentration always decreases with time.

**14.10** Rather than using the base- $e$  exponential model, [Eq. \(14.22\)](#), a common alternative is to employ a base-10 equivalent,

$$y = \alpha_5 10^{\beta_5 x}$$

When used for curve-fitting, this equation yields identical results to the base- $e$  version, but the value of the exponent coefficient,  $\beta_5$ , will differ from the  $\beta_1$  estimated using [Eq. \(14.22\)](#). Use the base-10 version to solve [Prob. 14.9](#). Also, develop a formula to relate  $\beta_5$  and  $\beta_1$ .

**14.11** Determine an equation to predict metabolism rate as a function of mass based on the following data. Use it to predict the metabolism rate of a 200-kg tiger.

| <b>Animal</b> | <b>Mass (kg)</b> | <b>Metabolism (watts)</b> |
|---------------|------------------|---------------------------|
| Cow           | 400              | 270                       |
| Human         | 70               | 82                        |
| Sheep         | 45               | 50                        |
| Hen           | 2                | 4.8                       |
| Rat           | 0.3              | 1.45                      |
| Dove          | 0.16             | 0.97                      |



**14.12** On average, the surface area,  $A$ , of a human is related to weight,  $W$ , and height,  $H$ . Measurements for several individuals of height 180 cm and different weights (kg), give values of area ( $\text{m}^2$ ) in the following table.

|                                                 |      |      |      |      |      |      |      |      |
|-------------------------------------------------|------|------|------|------|------|------|------|------|
| <b><math>W</math> (kg)</b>                      | 70   | 75   | 77   | 80   | 82   | 84   | 87   | 90   |
| <b><math>A</math> (<math>\text{m}^2</math>)</b> | 2.10 | 2.12 | 2.15 | 2.20 | 2.22 | 2.23 | 2.26 | 2.30 |

Show that a power law,  $A = aW^b$ , fits these data reasonably well. Present plots of the data along with the model line, and residuals versus predicted  $A$  values. Predict what the surface area is for a 95-kg person. What is the certainty of this prediction, expressed as  $\pm$  two times the standard error of the estimate?

**14.13** Fit an exponential model to

|                       |     |     |      |      |      |      |
|-----------------------|-----|-----|------|------|------|------|
| <b><math>x</math></b> | 0.4 | 0.8 | 1.2  | 1.6  | 2    | 2.3  |
| <b><math>y</math></b> | 800 | 985 | 1490 | 1950 | 2850 | 3600 |

Use Python to compute the fit and plot the data and the equation on both standard and semi-logarithmic graphs.

**14.14** An investigator has reported the data tabulated below for an experiment to determine the growth rate,  $k$ , of bacteria per day as a function of oxygen concentration,  $c$ , in mg/L. It is known that this growth can be modeled by the following equation:

$$k = \frac{k_{\max} c^2}{c_s + c^2}$$

where  $c_s$  and  $k_{\max}$  are parameters. Use a transformation to linearize this equation. Then, use straight-line regression to estimate the two parameters. Finally, use the model to predict growth rate at  $c = 2$  mg/L.

Page 344

|                       |     |     |     |     |     |
|-----------------------|-----|-----|-----|-----|-----|
| <b><math>c</math></b> | 0.5 | 0.8 | 1.5 | 2.5 | 4   |
| <b><math>k</math></b> | 1.1 | 2.5 | 5.3 | 7.6 | 8.9 |

**14.15** Develop a Python function to compute and return descriptive statistics for an array of numerical values. The function should return the number of values, sample mean, median, mode, range, sample standard deviation, sample variance, MAD, and coefficient of variation. In addition, the function should generate and display a histogram of the data. Use the square root of the number of data rounded down to the nearest integer as your number of bins. Test your function with the following data representing the viscosity of a chemical product:

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 8.2 | 8.6 | 8.9 | 8.2 | 9.3 |
| 8.2 | 8.6 | 9.1 | 8.1 | 9.3 |
| 8.2 | 8.6 | 9.5 | 8.3 | 9.5 |
| 8.4 | 8.6 | 8.5 | 8.4 | 9.3 |
| 8.4 | 8.6 | 8.4 | 8.7 | 9.5 |
| 8.4 | 8.8 | 8.3 | 8.8 | 9.5 |

**14.16** Modify the `strlinregr` function in  Fig. 14.15 so that, in addition, it

- (a) computes and returns the standard errors of the slope and intercept,
- (b) displays a plot of the data with the fitted line, and
- (c) displays a plot of the residuals versus the predicted values.

Test the modified function with the data from  Prob. 14.5.

**14.17** Develop a Python function, called `power_regr`, to fit a power model to a set of data. Have the function return the best-fit coefficients  $\alpha_2$  and  $\beta_2$  along with the  $R^2$  for the original, untransformed model. Return also the standard errors for the coefficients back-transformed from the standard errors for the straight-line coefficients. Test your function with the data from  Prob. 14.11.

**14.18** The following data show the relationship between the viscosity of SAE 70 oil and temperature. Plot the data and choose a transformation to use so that you can perform straight-line regression. Present plots of the data and fitted line and the residuals versus the predicted values. Comment on the apparent adequacy of the model.

|                        |       |       |        |        |
|------------------------|-------|-------|--------|--------|
| <b>Temperature, °C</b> | 26.67 | 93.33 | 148.89 | 315.66 |
| <b>Viscosity, Pa·s</b> | 1.35  | 0.085 | 0.012  | 0.0075 |

**14.19** You perform experiments in the laboratory and determine the following values of heat capacity,  $c_P$  in J/(kg·°C), at various temperatures,  $T$  in °C, for a gas:

|                       |      |      |      |      |      |      |
|-----------------------|------|------|------|------|------|------|
| <b><math>T</math></b> | -50  | -30  | 0    | 60   | 90   | 110  |
| <b><math>c</math></b> | 1250 | 1280 | 1350 | 1480 | 1580 | 1700 |

Use regression to determine a model to predict heat capacity at a given temperature. Test your model for a temperature of 30 °C.

**14.20** It is known that the tensile strength of a plastic increases as a function of the time it is heat treated. The following data have been collected as evidence of this effect.

|                         |    |    |    |    |    |    |    |    |    |
|-------------------------|----|----|----|----|----|----|----|----|----|
| <b>Time</b>             | 10 | 15 | 20 | 25 | 40 | 50 | 55 | 60 | 75 |
| <b>Tensile Strength</b> | 5  | 20 | 18 | 40 | 33 | 54 | 70 | 60 | 78 |

- (a) Fit a straight line to these data, and use the model to predict the tensile strength at a time of 32 minutes.
- (b) Repeat the analysis, but for a line with a zero intercept. Reference [Prob. 14.4](#).
- (c) From part (a), calculate a “confidence interval” for the slope and intercept as the estimates  $\pm 2$  standard errors. Does the interval for the intercept include zero? If so, what do you think that implies?

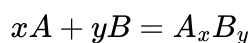
**14.21** The following data were measured during operation of a batch, stirred-tank chemical reactor for the generic reaction,  $A \rightarrow B$ . Use the data to determine the best possible estimates for  $k_0$  and  $E$  for the following chemical kinetic model:

$$-\frac{dA}{dt} = k_0 e^{-\frac{E}{RT}} A$$

where  $R$  = gas law constant, 8.3145 J/(mol·K). Compute two-standard error limits around your parameter estimates. What do you observe?

|                                              |     |     |      |      |      |
|----------------------------------------------|-----|-----|------|------|------|
| $-dA/dt[\text{mol}/(\text{L}\cdot\text{s})]$ | 460 | 960 | 2485 | 1600 | 1245 |
| $A(\text{mol/L})$                            | 200 | 150 | 50   | 20   | 10   |
| $T(\text{K})$                                | 280 | 320 | 450  | 500  | 550  |

**14.22** Concentration data were collected at 15 points in time for the polymerization reaction:



We assume that the reaction occurs via a complex mechanism consisting of many steps. Several models have been hypothesized, and the sum of squares of the residuals has been calculated for the fits of the models of the data. The results are shown below. Which model would you choose to explain the data best? Explain your choice. Would you want any additional information?

|                                         | <b>Models</b> |          |          |
|-----------------------------------------|---------------|----------|----------|
|                                         | <b>A</b>      | <b>B</b> | <b>C</b> |
| Sum of squares of the residuals, $SS_E$ | 135           | 105      | 100      |
| Number of model parameters, $n$         | 2             | 3        | 5        |

**14.23** Below are data taken from a batch reactor of bacterial growth, after the lag phase was over. The bacteria are allowed to grow as fast as possible for the first 2.5 hours, and then they are induced to produce a recombinant protein, the production of which slows the bacterial growth significantly. The theoretical growth of bacteria can be described by

$$\frac{dX}{dt} = \mu X$$

where  $X$  = the number of bacteria and  $\mu$  = the specific growth rate of the bacteria during the exponential growth period. Based on the data in the table below, estimate the specific growth rate during the first two hours of growth and, separately, during the next four hours of growth. Create a plot of the data and the two models.

|                     |       |       |       |       |       |       |       |
|---------------------|-------|-------|-------|-------|-------|-------|-------|
| <b>Time, h</b>      | 0     | 1     | 2     | 3     | 4     | 5     | 6     |
| <b>[Cells], g/L</b> | 0.100 | 0.335 | 1.102 | 1.655 | 2.453 | 3.702 | 5.460 |

**14.24** A transportation engineering study was conducted to determine the proper design of bicycle lanes. Data were gathered on bike-lane widths and average distance between bikes and passing cars. The data from nine streets are

Page 345

|                      |     |     |     |     |     |     |     |     |     |
|----------------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| <b>Distance, m</b>   | 2.4 | 1.5 | 2.4 | 1.8 | 1.8 | 2.9 | 1.2 | 3   | 1.2 |
| <b>Lane Width, m</b> | 2.9 | 2.1 | 2.3 | 2.1 | 1.8 | 2.7 | 1.5 | 2.9 | 1.5 |

- Plot the data. Make any observations.
- Fit a straight line to the data using regression. Add this line to the plot.
- If the minimum safe distance between bikes and passing cars is to be two meters, determine the design width of the bike lane.

**14.25** In water-resources engineering, the sizing of reservoirs depends on accurate estimates of water flow in the river that is being impounded. For some rivers, long-term historical records of such flow data are difficult to obtain. In contrast, meteorological data on precipitation are often available for many past years. Therefore, it is often useful to determine a relationship between river flow and precipitation. This relationship can then be used to estimate flows for years when only precipitation measurements were taken. The following data are available for a river that is to be dammed to create a reservoir:

|                                  |      |       |       |       |      |      |       |      |
|----------------------------------|------|-------|-------|-------|------|------|-------|------|
| <b>Precip.,<br/>cm/yr</b>        | 88.9 | 108.5 | 104.1 | 139.7 | 127  | 94   | 116.8 | 99.1 |
| <b>Flow,<br/>m<sup>3</sup>/s</b> | 14.6 | 16.7  | 15.3  | 23.2  | 19.5 | 16.1 | 18.1  | 16.6 |

- Plot the data and comment on what you observe.
- Fit a straight line to the data and add the line to your plot.
- Plot the residuals versus the predicted flows and comment on adequacy of your model.
- Use the best-fit line to predict the annual water flow if the precipitation is 120 cm.
- Given the precipitation in part (d), if the drainage area is  $1,100 \text{ km}^2$ , estimate what fraction of the precipitation is lost via processes such as evaporation, deep groundwater infiltration, and consumptive use.

**14.26** The mast of a sailboat has a cross-sectional area of  $10.65 \text{ cm}^2$  per centimeter length and is constructed of an experimental composite material. Tests were performed to define the relationship between stress and strain. The test results are

|                                 |        |        |        |        |        |        |
|---------------------------------|--------|--------|--------|--------|--------|--------|
| <b>Strain, cm/cm</b>            | 0.0032 | 0.0045 | 0.0055 | 0.0016 | 0.0085 | 0.0005 |
| <b>Stress, N/cm<sup>2</sup></b> | 4970   | 5170   | 5500   | 3590   | 6900   | 1240   |

The stress caused by the wind can be computed as  $F/A_c$ , where  $F$  = the force on the mast (N), and  $A_c$  = the mast's cross-sectional area per unit length. This value can then be substituted into Hooke's law to determine the mast's deflection,  $\Delta L_{\text{strain}} \times L$ , where  $L$  = the mast's length. If the wind force is 25,000 N, use the data to estimate the deflection of a 9-meter mast.

**14.27** The following data were taken from an experiment that measured the current in a wire for various imposed voltages:

|             |     |     |      |    |      |      |
|-------------|-----|-----|------|----|------|------|
| <b>V, V</b> | 2   | 3   | 4    | 5  | 7    | 10   |
| <b>i, A</b> | 5.2 | 7.8 | 10.7 | 13 | 19.3 | 27.5 |

- On the basis of straight-line regression of the data, predict the current for a voltage of 3.5 V. Plot the model line and the data, and plot the residuals versus the predicted current values. Evaluate the fit.
- Redo the regression with a zero intercept.

**14.28** An experiment is performed to determine the % elongation of electrical conducting material as a function of temperature. The resulting data are listed below. Predict the % elongation for a temperature of  $400^\circ\text{C}$ .

|                        |     |     |     |     |      |      |      |
|------------------------|-----|-----|-----|-----|------|------|------|
| <b>Temperature, °C</b> | 200 | 250 | 300 | 375 | 425  | 475  | 600  |
| <b>% Elongation</b>    | 7.5 | 8.6 | 8.7 | 10  | 11.3 | 12.7 | 15.3 |

**14.29** The population,  $p$ , of a small community on the outskirts of a city grows rapidly over a 20-year period:

---

|                 |     |     |     |     |      |
|-----------------|-----|-----|-----|-----|------|
| <b><i>t</i></b> | 0   | 5   | 10  | 15  | 20   |
| <b><i>p</i></b> | 100 | 200 | 450 | 950 | 2000 |

As an engineer working for a utility company, you must forecast the population five years into the future in order to anticipate the demand for power. Employ an exponential model and straight-line regression to make this prediction.

**14.30** The velocity,  $u$ , of air flowing past a flat surface is measured at several distances,  $y$ , away from the surface. Fit a curve to these data, assuming that velocity at the surface,  $y = 0$ , is zero. Use your result to determine the shear stress,  $\mu du/dy$ , at the surface where the viscosity of air is  $1.8 \times 10^{-5}$  Pa·s.

|                      |       |       |       |       |       |
|----------------------|-------|-------|-------|-------|-------|
| <b><i>y, m</i></b>   | 0.002 | 0.006 | 0.012 | 0.018 | 0.024 |
| <b><i>u, m/s</i></b> | 0.287 | 0.899 | 1.915 | 3.048 | 4.299 |

**14.31** *Andrade's equation* has been proposed as a model of the effect of temperature on viscosity:

$$\mu = De^{B/T}$$

where  $\mu$  = viscosity (Pa·s),  $T$  = absolute temperature (K), and  $D$  and  $B$  are parameters. The following data have been collected for water:

| <b>Temperature (°C)</b> | <b>Viscosity (cP)</b> |
|-------------------------|-----------------------|
| 0.0                     | 1.794                 |
| 4.4                     | 1.546                 |
| 10.0                    | 1.310                 |
| 15.6                    | 1.129                 |
| 21.1                    | 0.982                 |
| 26.7                    | 0.862                 |
| 32.2                    | 0.764                 |
| 37.8                    | 0.682                 |
| 48.9                    | 0.559                 |
| 60.0                    | 0.470                 |
| 71.1                    | 0.401                 |
| 82.2                    | 0.347                 |
| 93.3                    | 0.305                 |

Fit Andrade's equation to these data. Evaluate the adequacy of the model. Note: cP = centipoise, and  $1 \text{ cP} = 0.001 \text{ Pa}\cdot\text{s}$ . Also,  $T(K) = T(^{\circ}\text{C}) + 273.15$ .

**14.32** Perform the same computation as in [Example 14.2](#), but in addition to the drag coefficient, vary the mass uniformly by  $\pm 10\%$ .

**14.33** Perform the same computation as in [Example 14.3](#), but, in addition to the drag coefficient, also vary the mass following the normal distribution around its mean with a coefficient of variation of 5.7887%.

**14.34** *Manning's formula* for a rectangular channel is

$$Q = \frac{1}{n_m} \frac{(BH)^{5/3}}{(B + 2H)^{2/3}} \sqrt{S}$$

where  $Q$  = volumetric flow rate ( $\text{m}^3/\text{s}$ ),  $n_m$  = a roughness factor for the channel surfaces,  $B$  = channel width (m),  $H$  = fluid depth (m), and  $S$  = the slope. You are applying the formula to a stream where you know the width, 20 m, and the depth, 0.3 m, but, as is typical, you are unsure of the values for the roughness and the slope. You believe the roughness is about 0.03 and the slope is about 0.0003 with an uncertainty of about  $\pm 10\%$ .

Assuming uniform distributions, use a Monte Carlo analysis with 10,000 samples to characterize the distribution of flow. To be clear, generate random sequences for  $n_m$  and  $S$ , and then use corresponding pairs from these sequences to compute the flow using Manning's formula. Then, produce a histogram chart and sample statistics for the flow rate. Use an appropriate number of bins for the sample size.

**14.35** A Monte Carlo analysis can be used for optimization. For example, the trajectory of a projectile, not subject to significant drag, can be modeled with

$$y = \tan(\theta_0)x - \frac{g}{2v_0^2 \cos^2(\theta_0)}x^2 + y_0$$

where  $y$  = the elevation of the trajectory (m),  $\theta_0$  = the launch angle (radians),  $v_0$  = the launch velocity (m/s), and  $y_0$  = the initial elevation.

Imagine a professional golfer, for example, Rory McIlroy, hits a drive from an elevated tee box, 2.5 m high, with an initial angle of  $20^{\circ}$ , and a launch velocity of 170 mph. Determine the maximum height of the drive and the distance where this occurs. Of course, ignoring air drag is leaving out an important effect in this case.

- (a) Determine the answers using calculus.
- (b) Use Monte Carlo simulation to find the answers.

Suggestions: You can determine the distance at which the drive returns to the fairway by setting  $y = 0$  and solving for  $x$ . Then, create 10,000 values of  $x$  between 0 and this  $x$  distance. You can calculate  $y$  for

each of these  $x$ 's and then find the maximum  $y$  and its location, that is,  $x$  value. For the location, you might look up the NumPy `where` function.

**14.36** *Stokes Settling law* provides a means to compute the settling velocity of spherical particles under laminar flow conditions.

$$v_s = \frac{g}{18} \frac{\rho_s - \rho}{\mu} d^2$$

where  $v_s$  = the terminal settling velocity (m/s),  $g$  = gravitational acceleration ( $9.81 \text{ m/s}^2$ ),  $\rho$  = the fluid density ( $\text{kg/m}^3$ ),  $\rho_s$  = the particle density ( $\text{kg/m}^3$ ),  $\mu$  = the fluid viscosity (Pa·s), and  $d$  = the particle diameter (m).

Suppose you conduct an experiment where you measure the terminal settling velocity of several 500- $\mu\text{m}$ -diameter spherical beads for different materials in corn syrup at room temperature (20 °C) and collect the data in the table below.

| <b>Bead<br/>Material</b> | <b>Density<br/>(<math>\text{kg/m}^3</math>)</b> | <b>Settling Velocity<br/>(mm/s)</b> |
|--------------------------|-------------------------------------------------|-------------------------------------|
| Glass                    | 2500                                            | 1.06                                |
| Aluminum                 | 2700                                            | 1.28                                |
| Carbon                   | 3510                                            | 1.62                                |
| Titanium                 | 4500                                            | 2.22                                |
| Zinc                     | 7120                                            | 5.39                                |
| Iron                     | 7200                                            | 5.52                                |
| Steel                    | 7800                                            | 4.58                                |
| Copper                   | 8800                                            | 6.62                                |
| Lead                     | 11350                                           | 7.68                                |
| Tungsten                 | 19200                                           | 12.93                               |

- Generate a labeled plot of the settling velocity versus density. Use markers, not lines.
- Fit a straight line to the data using the `polyfit` function and add the line to the plot.
- Use your model to predict the settling velocity of a similar-sized tantalum bead of density  $16,400 \text{ kg/m}^3$ .
- Use the slope and intercept values to estimate the fluid's viscosity and density. In published literature, the viscosity of corn syrup at 20 °C is reported to be about 0.16 Pa·s and the density is to be about  $1350 \text{ kg/m}^3$ . Explain any differences between these values and your calculated values.



**14.37** Beyond the examples in [Fig. 14.12](#), there are many other models that can be reformulated for straight-line regression. For example, the following model applies to third-order chemical reactions in batch reactors:

$$c = c_0 \frac{1}{\sqrt{1 + 2kc_0^2 t}}$$

where  $c$  = concentration (mol/L),  $c_0$  = initial concentration (mol/L),  $k$  = reaction rate  $\left((\text{L/mol})^2/\text{d}\right)$ , and  $t$  = time in days (d). Transform this model for straight-line regression, and, based on the data in the table, use it to estimate  $k$  and  $c_0$ . Given the standard errors for your straight-line parameters, estimate the uncertainty of your  $k$  and  $c_0$  values. Provide plot of the data and your fitted model for both the transformed and original model.

|          |      |      |      |      |      |      |     |      |
|----------|------|------|------|------|------|------|-----|------|
| <b>t</b> | 0    | 0.5  | 1    | 1.5  | 2    | 3    | 4   | 5    |
| <b>c</b> | 3.26 | 2.09 | 1.62 | 1.48 | 1.17 | 1.06 | 0.9 | 0.85 |

**14.38** In [Chap. 7](#), we presented optimization techniques to find the optimal values of one- and multi-dimensional functions. Random numbers provide an alternative means to solve the same types of problems (recall [Prob. 14.35](#)). This is done repeatedly evaluating the function at randomly selected values of the independent variable (or variables) and keeping track of the one that yields the best value of the function being optimized. If a sufficient number of samples are taken, the optimum will eventually be located. Also, there is an advantage for systems that have more than one local optimum. The random search will usually find the best of the local optima. Develop a Python function, called RandOpt, that uses random numbers to locate the maximum of the “humps” function,

$$f(x) = \frac{1}{(x - 0.3)^2 + 0.01} + \frac{1}{(x - 0.9)^2 + 0.04} - 6$$

in the domain  $0 \leq x \leq 2$ . Here is the Python script you can use to test your function:

```
import numpy as np
import pylab
xmin = 0
xmax = 2
xplot = np.linspace(xmin,xmax,200)
yplot = f(xplot)
pylab.plot(xplot,yplot,c='k')
pylab.grid()
n = 10000
xopt,yopt = RandOpt(f,n,xmin,xmax)
print('xopt =',xopt)
print('yopt = ',yopt)
```

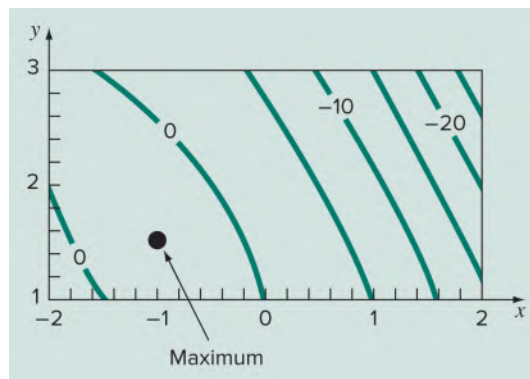
**14.39** Using the same approach as described in [Prob. 14.38](#), develop a function that uses random numbers to search for and determine the maximum and corresponding  $x$  and  $y$  values of the following two-dimensional function:

$$f(x, y) = y - x - 2x^2 - 2xy - y^2$$

in the domain  $-2 \leq x \leq 2$  and  $1 \leq y \leq 3$ . A contour plot of the function over the domain is presented in [Fig. P14.39](#). The maximum can be determined readily using calculus to be located at  $y = 1.5$  and  $x = -1$ . Here is a Python script you can use to test your function:

```
xint = np.array([-2.,2.])
yint = np.array([1.,3.])
n = 10000
xopt,yopt,fopt = RandOpt2D(fxy,n,xint,yint)
print('xopt =',xopt)
print('yopt = ',yopt)
print('fopt = ',fopt)
```

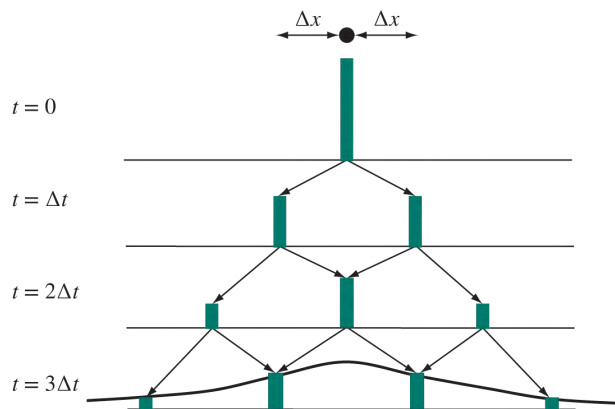
**FIGURE P14.39** A two-dimensional function with a maximum of 1.25 at  $x = -1$  and  $y = 1.5$ .



**14.40** Suppose that a population of particles is confined to motion along a one-dimensional line. This is illustrated in [Fig. P14.40](#). Assume that each particle has an equal likelihood of moving a distance,  $\Delta x$ , either to the right or the left at each time step,  $\Delta t$ . At  $t = 0$ , all particles are aligned at  $x = 0$  and are allowed to take one step in either direction randomly. After  $\Delta t$ , approximately 50% will have stepped to the right and 50% to the left. After  $2\Delta t$ , 25% will be two steps to the left, 25% would be two steps to the right, and 50% would have stepped back to the origin. With many additional time steps, the particles would spread out with the population greater near the origin and diminishing at the greater distances. The net result is that the distribution of particles approaches a spreading, bell-shaped distribution. This process, formally called a *random walk*, describes many phenomena in engineering and science with a common example being Brownian motion. Develop a Python function that is given a step size,  $\Delta x$ , a total number of particles,  $n$ , and steps,  $m$ . At each step, determine the location along the  $x$  axis of each particle, and use these results to generate an animated histogram that displays how the distribution's

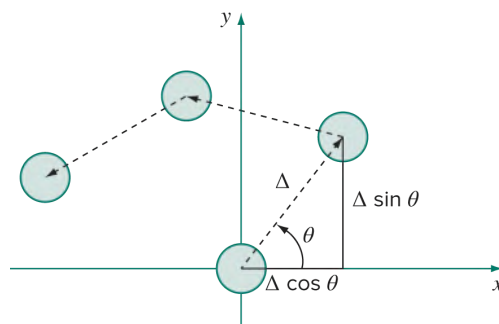
shape evolves as the computation progresses. Note: You might want to look into the Python `sleep` function in the `time` module to implement time delays and thereby slow down the simulation.

**FIGURE P14.40** The one-dimensional random walk.



**14.41** Repeat [Prob. 14.40](#), but for a two-dimensional random walk. As depicted in [Fig. P14.41](#), have each particle take a random step of length  $\Delta$  at a random angle  $\theta$  ranging from 0 to  $2\pi$ . Generate a pair of plots, animated, with the location of the particles in one and a histogram of the  $x$  coordinates in another. Challenge: For the final state of the simulation, create a 2D histogram chart using the NumPy `histogram2D` function and the `bar3D` function from the Matplotlib module.

**FIGURE P14.41** Depiction of the steps of a two-dimensional random walk.



**14.42** The table below shows the 2020 world record times and holders for outdoor running distances. The last two entries are more commonly known as the half marathon and marathon, respectively. Note that all but the 100 m and half-marathon and marathon are run on oval tracks. Fit a power model for each gender and use it to predict the world record for the 2000 m race. Note that the actual records for this race are 284.79 s (El Guerrouj) and 323.75 s (Dibaba) for men and women, respectively. Create a single plot of the data and the model lines and include a legend. Comment on the models and the predictions.

| Event<br>(m) | Time<br>(s) | Men                  | Time<br>(s) | Women           |
|--------------|-------------|----------------------|-------------|-----------------|
|              |             | Record Holder        |             | Record Holder   |
| 100          | 9.58        | Bolt                 | 10.49       | Griffith-Joyner |
| 200          | 19.19       | Bolt                 | 21.34       | Griffith-Joyner |
| 400          | 43.03       | Wayde van<br>Niekerk | 47.60       | Koch            |
| 800          | 100.91      | Rudisha              | 113.28      | Kratochvilova   |
| 1000         | 131.96      | Ngeny                | 148.98      | Masterkova      |
| 1500         | 326.00      | El Guerrouj          | 230.07      | Dibaba          |
| 5000         | 757.35      | Bekele               | 851.15      | Dibaba          |
| 10000        | 1577.53     | Bekele               | 1757.45     | Ayana           |
| 21098        | 3481.00     | Kamworor             | 3891.00     | Jepkosgei       |
| 42195        | 7283.40     | Kipchoge             | 8044.00     | Kosgei          |

# General Linear and Nonlinear Regression

## CHAPTER OBJECTIVES

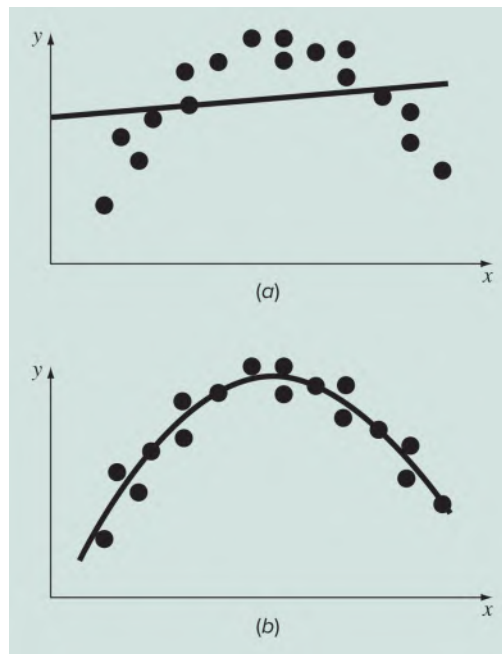
This chapter takes the concept of fitting a straight line and extends it to (a) fitting a polynomial in one factor or independent variable and (b) fitting a model that involves more than one factor or independent variable. For these examples, the models will be linear in the fitted parameters. We will show how such applications can be generalized and applied to a broader category of problems. Finally, we will show how optimization techniques can be used to fit models that are nonlinear in the parameters. Specific objectives and topics covered are

- Knowing how to implement polynomial regression and determine the best order and structure of polynomial to fit the data given the collinearity inherent to polynomial models.
- Understanding how to implement regression involving multiple factors or independent variables.
- Learning the vector-matrix formulation general to linear regression and knowing how to program Python to solve the general formulation.
- Knowing how to compute parameter confidence intervals for general linear regression.
- Understanding the distinction between linear and nonlinear regression.
- Learning how to apply optimization methods to the solution of nonlinear regression problems.
- Learning how to estimate parameter confidence intervals in nonlinear regression.

# 15.1 POLYNOMIAL REGRESSION

In [Chap. 14](#), a procedure was developed to derive the equation of a straight line using the least-squares criterion. Some data display a pattern that clearly will not be well fit by a straight line, such as seen in [Fig. 15.1](#). For these situations, a model with curvature is attractive. We illustrated nonlinear models in [Chap. 14](#) that could be transformed into straight-line form. An alternative that is commonly practiced is to fit polynomials to data, that is, *polynomial regression*.

**FIGURE 15.1** Data that show curvature and are ill-suited to a straight-line model (a) but may be modeled better by a parabola (b).



The least-squares procedure can be readily extended to fit the data to a higher-order polynomial. As a first example, suppose that we want to fit a second-order polynomial model:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \varepsilon \quad (15.1)$$

Here,  $y$  represents the actual measured variable, and the  $\beta$ 's are unknown parameters. The  $\varepsilon$  is the model error which is assumed to be zero mean and random, described by the normal distribution with variance  $\sigma_e^2$ . Given a set of data,  $\{x_i, y_i, i = 1, \dots, n\}$ , we want to generate specific estimates for the  $\beta$ 's, which we will designate as  $b$ 's, and this will give rise to a specific set of residual errors,  $e$ 's, that can be tested against the assumptions associated with  $\varepsilon$ . So, for each of the  $n$  pairs in our data set, we can write

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \varepsilon_i \quad (15.2)$$

These  $n$  expressions are called the *model realizations*. This allows us to formulate the sum of squares of the residual errors as

$$SS_E = \sum [y_i - (\beta_0 + \beta_1 x_i + \beta_2 x_i^2)]^2 \quad (15.3)$$

The least-squares problem then becomes finding the  $b$ 's that minimize  $SS_E$ . This can be readily solved using calculus by forming the partial derivative of  $SS_E$  with respect to each of the three  $\beta$ 's and setting these derivative expressions equal to zero, yielding three equations in the three unknown  $b$ 's:

$$\begin{aligned} \frac{\partial SS_E}{\partial \beta_0} &= -2 \sum [y_i - (\beta_0 + \beta_1 x_i + \beta_2 x_i^2)] \\ \frac{\partial SS_E}{\partial \beta_1} &= -2 \sum x_i [y_i - (\beta_0 + \beta_1 x_i + \beta_2 x_i^2)] \\ \frac{\partial SS_E}{\partial \beta_2} &= -2 \sum x_i^2 [y_i - (\beta_0 + \beta_1 x_i + \beta_2 x_i^2)] \end{aligned}$$

leading to

$$\begin{aligned} [n]b_0 + [\sum x_i]b_1 + [\sum x_i^2]b_2 &= \sum y_i \\ [\sum x_i]b_0 + [\sum x_i^2]b_1 + [\sum x_i^3]b_2 &= \sum x_i y_i \\ [\sum x_i^2]b_0 + [\sum x_i^3]b_1 + [\sum x_i^4]b_2 &= \sum x_i^2 y_i \end{aligned} \quad (15.4)$$

As with straight-line regression, [Eqs. \(15.4\)](#) are called the *normal equations*. They can be solved for the  $b$ 's readily using the techniques we learned in [Chap. 9](#). This then yields a predictive model in the form

$$\hat{y} = b_0 + b_1 x + b_2 x^2 \quad (15.5)$$

and the residuals and residual sum of squares can be computed as

$$e_i = y_i - \hat{y}_i \quad SS_E = \sum e_i^2$$

The standard error of the estimate is then

$$s_e = \sqrt{\frac{SS_E}{n-3}} \quad (15.6)$$

Note the difference in the denominator from straight-line regression ( $n-2$ ) with two estimated parameters, as here we have estimated three parameters.

The foregoing methodology can be extended to higher-order polynomials, and to polynomials with missing terms, but using the derivation of individual linear equations becomes cumbersome. You will see later in this chapter how we can formulate regression of polynomials of any order conveniently using vectors and matrices. We will also introduce the parameter standard errors there.

## EXAMPLE 15.1

### Polynomial Regression

#### Problem Statement

Fit a second-order polynomial to the data in the first two columns of [Table 15.1](#).

**TABLE 15.1** Computations for regression of a second-order polynomial.

|      | <b>x</b> | <b>y</b>    | <b>x<sup>2</sup></b> | <b>x<sup>3</sup></b> | <b>x<sup>4</sup></b> | <b>xy</b>    | <b>x<sup>2</sup>y</b> |
|------|----------|-------------|----------------------|----------------------|----------------------|--------------|-----------------------|
|      | 0        | 2.1         | 0                    | 0                    | 0                    | 0            | 0                     |
|      | 1        | 7.7         | 1                    | 1                    | 1                    | 7.7          | 7.7                   |
|      | 2        | 13.6        | 4                    | 8                    | 16                   | 27.2         | 54.4                  |
|      | 3        | 27.2        | 9                    | 27                   | 81                   | 81.6         | 244.8                 |
|      | 4        | 40.9        | 16                   | 64                   | 256                  | 163.6        | 654.4                 |
|      | <u>5</u> | <u>61.1</u> | <u>25</u>            | <u>125</u>           | <u>625</u>           | <u>305.5</u> | <u>1527.5</u>         |
| Sums | 15       | 152.6       | 55                   | 225                  | 979                  | 585.6        | 2488.8                |

#### Solution

Using the sums from [Table 15.1](#), we can solve [Eqs. \(15.4\)](#) quantitatively.

$$\begin{aligned}[6]b_0 + [15]b_1 + [55]b_2 &= 152.6 \\ [15]b_0 + [55]b_1 + [225]b_2 &= 585.6 \\ [55]b_0 + [225]b_1 + [979]b_2 &= 2488.8\end{aligned}$$

We can use the Python code below to solve for the parameter estimates.

```
import numpy as np

X = np.matrix('6., 15., 55. ; 15., 55., 225. ; 55., 225., 979.')
const = np.matrix('152.6 ; 585.6 ; 2488.8')

b = np.linalg.solve(X,const)
np.set_printoptions(precision=4)
print('Estimated parameters are\n',b)
```

with the results

```
Estimated parameters are
[[2.4786]
 [2.3593]
 [1.8607]]
```

Our model is then

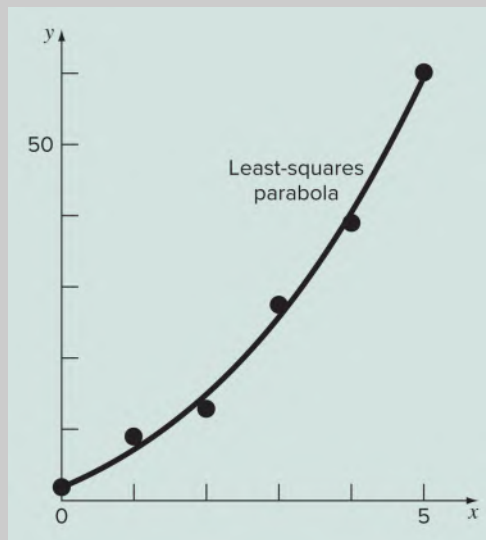
$$\hat{y} = 2.4786 + 2.3593x + 1.8607x^2$$



| $\hat{y}$ | $e$    |
|-----------|--------|
| 2.479     | -0.379 |
| 6.699     | 1.001  |
| 14.640    | -1.040 |
| 26.303    | 0.897  |
| 41.687    | -0.787 |
| 60.793    | 0.307  |
| 3.75      | $SS_E$ |
| 2513.4    | $SS_T$ |
| 2509.6    | $SS_R$ |
| 1.118     | $S_e$  |
| 0.9985    | $R^2$  |

These results indicate that a very high percentage, 99.85%, of the variability in  $y$  is explained by the model. The lack of any systematic behavior in the residual series supports a claim that the model is adequate. The fit is illustrated in [Fig. 15.2](#).

**FIGURE 15.2** Fit of a second-order polynomial.



There are remaining issues associated with fitting polynomials. A major one is choosing an appropriate order of polynomial. A fourth-order polynomial with five parameters will provide a perfect fit to five data points, but, generally, it will be a poor model because its predictions between the data points will be

flawed, even bizarre. A second issue is deciding whether all the terms in a polynomial model are significant, possibly leading to a simplified model that is adequate. This emphasizes the concept of *parsimony*, which, when applied to regression, suggests that the best model is the simplest or most frugal one that yet does an adequate job of representing the phenomenon.<sup>1</sup> The contrasting concept is *overfitting*, the use of models that are too complex with too many terms for the task at hand. We will explore parsimony and overfitting later in this chapter.

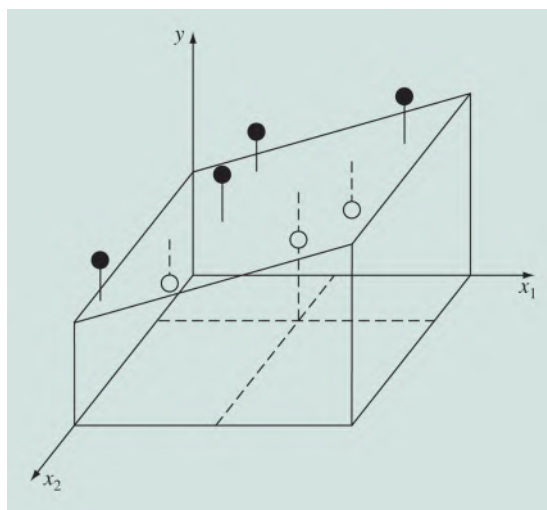
## 15.2 MULTIPLE LINEAR REGRESSION

Another useful extension of straight-line regression is the case where  $y$  is a linear function of two or more independent variables or factors. For example,  $y$  might be a function of  $x_1$  and  $x_2$ , as in

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \varepsilon$$

This equation fits data with two independent variables with a model that can be envisioned as a plane in  $x_1 - x_2 - y$  three-dimensional space (📄 Fig. 15.3).

**FIGURE 15.3** Graphical depiction of multiple linear regression where  $y$  is a function of  $x_1$  and  $x_2$ , and a model defines a plane in three-dimensional space.



As with the previous cases, the “best” estimates for the model parameters are determined by formulating the sum of the squares of the residuals:

$$SS_E = \sum e_i^2 = \sum_{i=1}^n [y_i - (b_0 + b_1 x_{1,i} + b_2 x_{2,i})]^2 \quad (15.7)$$

and take the partial derivatives of  $SS_E$  with respect to each of the  $b$ ’s.

$$\begin{aligned}\frac{\partial SS_E}{\partial b_0} &= -2 \sum [y_i - (b_0 + b_1 x_{1,i} + b_2 x_{2,i})] \\ \frac{\partial SS_E}{\partial b_1} &= -2 \sum x_{1,i} [y_i - (b_0 + b_1 x_{1,i} + b_2 x_{2,i})] \\ \frac{\partial SS_E}{\partial b_2} &= -2 \sum x_{2,i} [y_i - (b_0 + b_1 x_{1,i} + b_2 x_{2,i})]\end{aligned}$$

As before, by setting these derivatives equal to zero, the results can be rearranged into a set of three linear equations in the  $b$ 's. We can express these in vector-matrix form as

$$\begin{bmatrix} n & \sum x_{1,i} & \sum x_{2,i} \\ \sum x_{1,i} & \sum x_{1,i}^2 & \sum x_{1,i}x_{2,i} \\ \sum x_{2,i} & \sum x_{1,i}x_{2,i} & \sum x_{2,i}^2 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum x_{1,i}y_i \\ \sum x_{2,i}y_i \end{bmatrix} \quad (15.8)$$

The solution of these normal equations yields the parameter estimates.

As with polynomial regression, our multivariable model can be extended to  $m$  dimensions, as in

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_m x_m + \varepsilon$$

The total corrected sum of squares,  $SS_T$ , can be partitioned into the sum of squares of the residuals,  $SS_E$ , and the regression sum of squares,  $SS_R$ . As with straight-line regression, this partition provides the calculation of the coefficient of determination,  $R^2 = SS_R/SS_T$ , and the standard error of the estimate,

$$s_e = \sqrt{\frac{SS_E}{n - (m + 1)}}$$

## EXAMPLE 15.2

### Multiple Linear Regression

#### Problem Statement

The following data were created from the model,  $y = 5 + 4x_1 - 3x_2$ , where random errors were added to the model predictions.

| $x_1$ | $x_2$ | $y$  |
|-------|-------|------|
| 0     | 0     | 5.16 |
| 2     | 1     | 9.91 |
| 2.5   | 2     | 8.91 |
| 1     | 3     | 0.27 |

| $x_1$ | $x_2$ | $y$   |
|-------|-------|-------|
| 4     | 6     | 3.07  |
| 7     | 2     | 27.04 |

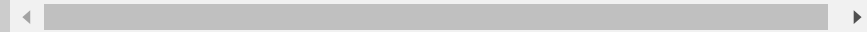
Use multiple linear regression to fit these data. Assess how close the fit comes to the “true” model.

## Solution

The summations required to formulate the normal equations are shown in [Table 15.2](#).

**TABLE 15.2** Computations required for the normal equations for [Example 15.2](#).

| $i$      | $x_1$    | $x_2$    | $y$          | $x_1^2$   | $x_2^2$  | $x_1x_2$  | $x_1y$       | $x_2y$       |
|----------|----------|----------|--------------|-----------|----------|-----------|--------------|--------------|
| 1        | 0        | 0        | 5.16         | 0         | 0        | 0         | 0            | 0            |
| 2        | 2        | 1        | 9.91         | 4         | 1        | 2         | 19.81        | 9.905        |
| 3        | 2.5      | 2        | 8.91         | 6.25      | 4        | 5         | 22.28        | 17.82        |
| 4        | 1        | 3        | 0.27         | 1         | 9        | 3         | 0.2728       | 0.8183       |
| 5        | 4        | 6        | 3.07         | 16        | 36       | 24        | 12.27        | 18.41        |
| <u>6</u> | <u>7</u> | <u>2</u> | <u>27.04</u> | <u>49</u> | <u>4</u> | <u>14</u> | <u>189.3</u> | <u>54.08</u> |
| $n$      | 16.5     | 14       | 54.36        | 76.25     | 54       | 48        | 243.91       | 101.03       |



Substituting the appropriate summations into [Eq. \(15.8\)](#) yields

$$\begin{bmatrix} 6 & 16.5 & 14 \\ 16.5 & 76.25 & 48 \\ 14 & 48 & 54 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} 54.36 \\ 243.91 \\ 101.03 \end{bmatrix}$$

We can solve these three linear equations to obtain  $b_0 \cong 5.081$ ,  $b_1 \cong 3.975$ , and  $b_2 \cong 2.98$ . These estimates are consistent with and close to the original model that was used to generate the data.

Although there are many cases where linear models are sufficient and adequate, multiple linear regression can be extended to multivariable power models, such as

$$y = \beta_0 x_1^{\beta_1} x_2^{\beta_2} \cdots x_m^{\beta_m} + \varepsilon$$

via the logarithmic transformation

$$\log(y) = \log(\beta_0) + \beta_1 \log(x_1) + \beta_2 \log(x_2) + \cdots + \beta_m \log(x_m) + \varepsilon'$$

As was mentioned in the previous chapter [recall the discussion following [Eq. \(14.28\)](#)], we must recognize the difference between the residual error terms in the original ( $\varepsilon$ ) and transformed ( $\varepsilon'$ ) models.

A similar situation arises with multilinear regression as with polynomial regression vis-à-vis parsimony versus overfitting. Here, though, in determining whether a model term should be included, we are concluding whether an input variable is significant or not. We should also mention that it is possible to partition  $SS_R$  into sum-of-squares components associated with each input variable. This provides further insight to help with model determination via a technique called *analysis of variance*.

As we will now see, it is useful to generalize all our linear regression cases into one consistent framework that can accommodate a great variety of model structures with the requirement that the model parameters to be estimated enter linearly into the model.

## 15.3 GENERAL LINEAR LEAST-SQUARES REGRESSION

In [Chap. 14](#) and the preceding pages of this chapter, we have introduced three types of linear regression: straight-line, polynomial, and multiple linear. In fact, all three fall under the following description of the general linear model:

$$y = \beta_0 f_0(x_1, x_2, \dots, x_p) + \beta_1 f_1(x_1, x_2, \dots, x_p) + \dots + \beta_m f_m(x_1, x_2, \dots, x_p) + \varepsilon \quad (15.9)$$

where  $f_0, f_1, \dots, f_m$  are  $m + 1$  basis functions. In the cases of straight-line and polynomial regression, there is only one  $x$  variable; however, in multilinear regression, there are more than one  $x$  variables. As we have mentioned before, the requirement for linear regression is that the model be linear in the parameters to be estimated, as shown in [Eq. \(15.9\)](#). Many of our models have a leading parameter that is an intercept. In these cases,  $f_0 \equiv 1$ .

In terms of the independent variables,  $x_k$ , the model terms may be highly nonlinear. For example,

$$y = \beta_0 + \beta_1 \cos(\omega x) + \beta_2 \sin(\omega x) + \varepsilon$$

is suitable for linear regression. This model is the basis for *Fourier analysis*. On the other hand, the Antoine equation for vapor pressure,  $P$ , as a function of temperature,  $T$ , is

$$P = 10^{A - \frac{B}{C+T}} + \varepsilon$$

where  $A$ ,  $B$ , and  $C$  are model parameters to be estimated. This model is nonlinear in these parameters, and, even with a log transformation,

$$\log_{10}(P) = A - \frac{B}{C+T} + \varepsilon'$$

it is still nonlinear in the parameters. Fitting such a model requires methods of nonlinear regression that we will introduce later in the chapter.

Back to [Eq. \(15.9\)](#), consider the situation where we have collected  $n$  data points,

$$\{y_i, x_{1,i}, x_{2,i}, \dots, x_{p,i}, i = 1, \dots, n\}$$

where  $y_i$  = the value of the dependent variable for the  $i$ th data point,  $x_{j,i}$  = the value of the  $j$ th independent variable for the  $i$ th data point,  $i = 1, \dots, n$ , and  $j = 1, \dots, p$ . Each of these data points can be introduced into the model to create  $n$  model realizations, and we can store the values of the  $m$  basis functions evaluated at each of the  $p$  independent variables in an  $n \times (m + 1)$  matrix,  $\mathbf{X}$ ,

$$\mathbf{X} \equiv \begin{bmatrix} f_0(x_{1,1}, x_{2,1}, \dots, x_{p,1}) & f_1(x_{1,1}, x_{2,1}, \dots, x_{p,1}) & \cdots & f_m(x_{1,1}, x_{2,1}, \dots, x_{p,1}) \\ f_0(x_{1,2}, x_{2,2}, \dots, x_{p,2}) & f_1(x_{1,2}, x_{2,2}, \dots, x_{p,2}) & \cdots & f_m(x_{1,2}, x_{2,2}, \dots, x_{p,2}) \\ \vdots & \vdots & \ddots & \vdots \\ f_0(x_{1,n}, x_{2,n}, \dots, x_{p,n}) & f_1(x_{1,n}, x_{2,n}, \dots, x_{p,n}) & \cdots & f_m(x_{1,n}, x_{2,n}, \dots, x_{p,n}) \end{bmatrix}$$

It is important to recognize that the  $\mathbf{X}$  matrix is distinct from the  $x$  variables. It is a collection of evaluations of functions involving the  $x$  variables, which, in the simplest form, may be just a sole  $x$  variable, but may involve nonlinear combinations or functions of the  $x$  variables. This matrix of realizations is denoted as  $\mathbf{X}$  commonly in applied statistics, as we do here. Also defining

$$\mathbf{y} \equiv \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \quad \boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_m \end{bmatrix} \quad \text{and} \quad \mathbf{e} = \begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_n \end{bmatrix}$$

we have

Page 356

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \mathbf{e} \quad (15.11)$$

Observe here that  $\mathbf{y}$  is a vector of the  $n$  actual measured responses. In the  $\mathbf{X}$  matrix, we have substituted in each of the  $n$  rows, the independent variable values for that element in the data set. Then, for a given set of  $m + 1$  parameter values, the model predictions are calculated as  $\mathbf{X}\boldsymbol{\beta}$ , and the residual vector,  $\mathbf{e}$ , contains the errors required to be added to the model predictions to yield the actual responses,  $\mathbf{y}$ .

To proceed with a least-squares derivation, we need the sum of the squares of the residual errors, or

$$SS_E = \sum_{i=1}^n e_i^2 = \mathbf{e}'\mathbf{e}$$

The latter is the inner product of the error vector with itself that yields the sum of squares. Since, from [Eq. \(15.10\)](#),  $\mathbf{e} = \mathbf{y} - \mathbf{X}\boldsymbol{\beta}$ , we can formulate the least-squares minimization in terms of the partial derivatives,

$$\frac{\partial SS_E}{\partial \boldsymbol{\beta}} = \frac{\partial}{\partial \boldsymbol{\beta}} (\mathbf{e}'\mathbf{e}) = \frac{\partial}{\partial \boldsymbol{\beta}} ((\mathbf{y} - \mathbf{X}\boldsymbol{\beta})'(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})) = -2\mathbf{X}'(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$$

By setting this latter expression equal to zero, we derive the general set of normal equations for linear regression, which now involve the parameter estimates,  $\mathbf{b}$ .

$$[\mathbf{X}'\mathbf{X}]\mathbf{b} = \mathbf{X}'\mathbf{y} \quad (15.12)$$

and we can write the analytical solution for the parameter estimates as



$$\mathbf{b} = [\mathbf{X}/\mathbf{X}]^{-1} \mathbf{X}/\mathbf{y} \quad (15.13)$$

To determine the  $\mathbf{b}$  values, we use established numerical techniques to solve the  $m$  linear normal equations. With specific  $\mathbf{b}$  values in hand, we can compute statistics related to the fit,

$$\begin{aligned} \mathbf{e} &= \mathbf{y} - \mathbf{X}\mathbf{b}, & SS_E &= \sum_{i=1}^n e_i^2 = \mathbf{e}/\mathbf{e}, & \hat{\mathbf{y}} &= \mathbf{X}\mathbf{b} \\ SS_T &= \sum_{i=1}^n (y_i - \bar{y})^2, & SS_R &= \sum_{i=1}^n (y_i - \hat{y}_i)^2, & R^2 &= \frac{SS_R}{SS_T} \end{aligned}$$

We note also in the above, from the partition of the total corrected sum of squares,  $SS_T$ , that  $SS_R = SS_T - SS_E$ . Rather than compute the regression sum of squares,  $SS_R$ , we usually compute it using this difference. As before, the standard error of the estimate is given by

$$s_e = \sqrt{\frac{SS_E}{n - (m + 1)}}$$

At this point, we introduce the standard error estimates for the model parameters. As we discussed in [Chap. 14](#), this has to do with how the uncertainty represented by the variance of the error,  $\sigma_e^2$ , propagates through the model to corresponding uncertainties in the parameter estimates. In vector-matrix form, this is based on the *covariance matrix* of the parameter estimates,  $\text{cov}(\hat{\boldsymbol{\beta}})$ , given by

$$\text{cov}(\hat{\boldsymbol{\beta}}) = \sigma_e^2 [\mathbf{X}/\mathbf{X}]^{-1} \quad (15.14)$$

For a specific regression calculation,  $\hat{\boldsymbol{\beta}}$  becomes  $\mathbf{b}$ , and  $\sigma_e^2$  is estimated as

$$s_e^2 = \frac{SS_E}{n - (m + 1)}$$

The standard error estimate for each parameter is taken as the square root of the corresponding diagonal element of the matrix inverse from [Eq. \(15.13\)](#),  $[\mathbf{X}/\mathbf{X}]^{-1}$ , multiplied by  $s_e^2$ . This calculation is important because it provides us with the basis for judging the significance of individual terms in the model and for developing parsimonious model structures.

At this point, we hope that you appreciate the power and elegance of the matrix formulation of general linear least squares described above. Recall that the earlier derivations for polynomial and multiple regression involved using partial differentiation to generate the normal equations which themselves consisted of numerous summations. In contrast, the matrix approach generates the same result but in a much simpler and concise fashion. The power of the approach is illustrated by the following example where it is applied to estimate the parameters of a model commonly used in optics.

## EXAMPLE 15.3

## General Linear Regression—Refractive Index of Borosilicate Glass

### Problem Statement

*Cauchy's equation* is known to be a good model for relating refractive index of a translucent material,  $n$ , to wavelength of the incident light,  $\lambda$ , and has the form

$$n = \beta_0 + \beta_1 \frac{1}{\lambda^2} + \beta_2 \frac{1}{\lambda^4}$$

The data in [Table 15.3](#) represent careful laboratory measurements of refractive index for a sample of borosilicate glass. Fit Cauchy's equation to these data using general linear regression and present the statistics associated with the fit.

**TABLE 15.3** Measurements of the refractive index of borosilicate glass at different wavelengths.

| Wavelength ( $\mu\text{m}$ ) | Refractive Index |
|------------------------------|------------------|
| 0.6563                       | 1.50883          |
| 0.6439                       | 1.50917          |
| 0.5890                       | 1.51124          |
| 0.5338                       | 1.51386          |
| 0.5086                       | 1.51534          |
| 0.4861                       | 1.51690          |
| 0.4340                       | 1.52136          |
| 0.3988                       | 1.52546          |

### Solution

For this scenario, the model realization matrix is

$$\mathbf{X} = \begin{bmatrix} f_0(x_1) & f_1(x_1) & \cdots & f_k(x_1) \\ f_0(x_2) & f_1(x_2) & \cdots & f_k(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ f_0(x_n) & f_1(x_n) & \cdots & f_k(x_n) \end{bmatrix} = \begin{bmatrix} 1 & \frac{1}{\lambda_1^2} & \frac{1}{\lambda_1^4} \\ 1 & \frac{1}{\lambda_2^2} & \frac{1}{\lambda_2^4} \\ \vdots & \vdots & \vdots \\ 1 & \frac{1}{\lambda_8^2} & \frac{1}{\lambda_8^4} \end{bmatrix}$$

and for the data in [Table 15.3](#),

$$\mathbf{X} = \begin{bmatrix} 1 & 2.322 & 5.590 \\ 1 & 2.412 & 5.817 \\ 1 & 2.883 & 8.309 \\ 1 & 3.509 & 12.316 \\ 1 & 3.866 & 14.945 \\ 1 & 4.232 & 17.910 \\ 1 & 5.309 & 28.186 \\ 1 & 6.288 & 39.535 \end{bmatrix}$$

The response vector is

$$\mathbf{y} = \mathbf{n} = \begin{bmatrix} 1.50883 \\ 1.50917 \\ 1.51124 \\ 1.51386 \\ 1.51534 \\ 1.51690 \\ 1.52136 \\ 1.52546 \end{bmatrix}$$

The normal equations are given by

Page 358

$$\begin{bmatrix} 8 & 30.82 & 132.41 \\ 30.82 & 132.41 & 625.52 \\ 132.41 & 625.52 & 3185.2 \end{bmatrix} \mathbf{b} = \begin{bmatrix} 12.122 \\ 46.758 \\ 201.12 \end{bmatrix}$$

and solving these for the model parameters yields

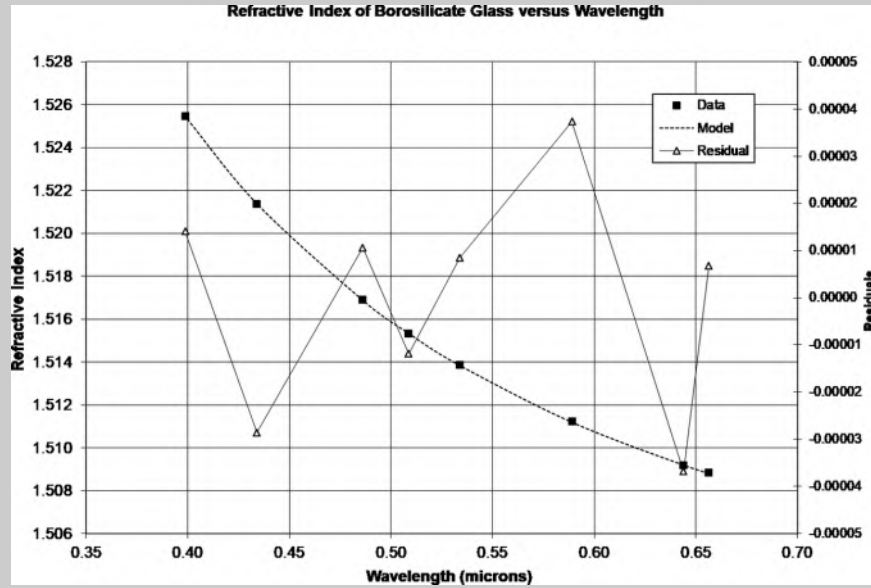
$$\mathbf{b} = \begin{bmatrix} 1.499 \\ 4.321 \times 10^{-3} \\ -1.509 \times 10^{-5} \end{bmatrix}$$

Consequently, our Cauchy's equation model is

$$n = 1.499 + 4.321 \times 10^{-3} \frac{1}{\lambda^2} - 1.509 \times 10^{-5} \frac{1}{\lambda^4}$$

How well does the model fit the data? [Figure 15.4](#) provides a picture.

**FIGURE 15.4** Cauchy's equation model performance.



The model fit appears very good. The residuals are tiny, and they are random with respect to wavelength.

For the additional statistics,  $SS_T = 2.405 \times 10^{-4}$ ,  $SS_E = 4.133 \times 10^{-9}$ ,  $R^2 = 0.999983$ , and  $s_e = 2.875 \times 10^{-5}$ . The covariance matrix estimate is

$$\text{cov}(\hat{\beta}) = \sigma_e^2 [\mathbf{X}'\mathbf{X}]^{-1} = 8.267 \times 10^{-10} \begin{bmatrix} 14.20 & -7.150 & 0.8137 \\ -7.150 & 3.704 & -0.4302 \\ 0.8137 & -0.4302 & 0.05096 \end{bmatrix}$$

Then, the standard error for each parameter is calculated as

$$\begin{aligned} s_{b_0} &= \sqrt{8.267 \times 10^{-10} \cdot 14.20} \cong 1.083 \times 10^{-4} \\ s_{b_1} &= \sqrt{8.267 \times 10^{-10} \cdot 3.704} \cong 5.534 \times 10^{-5} \\ s_{b_2} &= \sqrt{8.267 \times 10^{-10} \cdot 0.05096} \cong 6.491 \times 10^{-6} \end{aligned}$$

The standard errors are small. For example, if we take an interval about the  $b_2$  parameter value of  $\pm 3s_{b_2}$ , that yields  $-3.456 \times 10^{-5} \leq \beta_2 \leq 4.382 \times 10^{-6}$ . The interval contains zero which makes it difficult for us to claim that  $\beta_2$  is different from zero. In other words, it suggests that we might leave the third term out of our model and remake the fit. That is not the case with the other two parameters where the intervals are very small compared to the parameter values.

## 15.4 MODEL BUILDING AND SELECTION IN REGRESSION

We have introduced various models that can be fit to data using the technique of linear least-squares regression. That immediately raises the question how to decide between competing models. One of the best examples of this is fitting polynomials. How many, and which, polynomial terms provide the best fit? It turns out that this is not an easy and direct question to answer, and that is the subject we address in this section.

There are two general criteria of concern in model fitting. One is performance, as evidenced by criteria like the coefficient of determination,  $R^2$ , and the standard error of the estimate,  $s_e$ . The other is adequacy, that is, has the model accounted for all the systematic behavior in the data? We analyze this by looking at histogram charts of the residuals and plots of the residuals versus fitted values and independent variables. Further, a criterion like  $R^2$  will increase toward unity (a perfect fit) as we add terms to the model until, when the number of parameters is equal to the number of data, the model will pass through each data point exactly; however, it is likely a very poor model in between the data points.

A well-tested approach to model building is called *stepwise regression*. Here, there are the *forward selection* and *backward elimination* techniques. In forward selection, one starts with a simple model and adds terms one by one, evaluating both performance and adequacy at each step. It is most likely that the optimal performance and meeting adequacy will occur at different steps. Then, choices must be made. In backward elimination, one starts with a model with many terms and finds a term that is a candidate for removal. This process repeats until all remaining terms are justified and, hopefully, adequacy is satisfied. This process has the possibility of pulling out intermediate terms in the full model; whereas forward selection does not. A final exhaustive technique provided by statistical software is to regress and evaluate all possible sub-models of a full model and select the model with the highest performance.

Before we illustrate model building via regression, it is important to address performance criteria. We have already pointed out the disadvantages associated with  $R^2$ . Essentially, there is no penalty for adding terms to the model. A common alternative is called *adjusted  $R^2$*  and is defined as

$$R_{\text{adj}}^2 = 1 - \frac{SS_E / (n - (m + 1))}{SS_T / (n - 1)} = 1 - (1 - R^2) \frac{n - 1}{n - (m + 1)}$$

With this criterion, as we increase the number of model terms, the approach of  $R^2$  to 1 is retarded by the  $n - (m + 1)$  denominator and  $R_{\text{adj}}^2$  may pass through a maximum, an optimal value.

Another measure of interest is the standard error of the estimate,  $s_e$ ,

$$s_e = \sqrt{\frac{SS_E}{n - (m + 1)}}$$

As we increase the number of terms in the model,  $SS_E$  will generally decrease, but, since  $m$  increases, so will the denominator decrease. We can then look for a minimum  $s_e$ .

These two performance criteria,  $R_{\text{adj}}^2$  and  $s_e$ , are useful as long as our data set is not too large. If  $n \gg m$ , the compensation for adding model terms will be insignificant and another measure is needed.

There are many performance criteria for regression, and we cannot introduce too many here, but one of interest is the *prediction error sum of squares*, or *PRESS* statistic, and the related *predicted*  $R^2$ . These are useful for two reasons: (1) they are effective for larger data sets and (2) they detect overfitting (too many model terms) and prevent such. The *PRESS* concept might be stated in colloquial terms as “making regression compete on a level playing field.” The steps in the method are to

1. remove the first data point from the data set,
2. carry out the regression using just the remaining points,
3. use the resulting model to predict the response,  $\hat{y}$ , for the left-out data point,
4. compute the residual associated with this prediction,  $e_{(i)}$ , and
5. repeat steps 1 through 4 removing subsequent data points.

Then,

$$PRESS = \sum_{i=1}^n e_{(i)}^2$$

and the predicted  $R^2$  is

$$R_{\text{pred}}^2 = 1 - \frac{PRESS}{SS_T}$$

An overfitted model doesn't do a good job predicting the left-out points. Consequently, a maximum in  $R_{\text{pred}}^2$  tends to select a sound model. But, that model yet must satisfy adequacy.

The procedure for computing the *PRESS* statistic sounds terribly labor-intensive ( $n$  regression calculations!), but there is a neat short-cut involving the “hat” matrix,  $\mathbf{H}$ . It goes as follows:

$$\mathbf{H} = \mathbf{X}(\mathbf{X}'\mathbf{X})\mathbf{X}'$$

and

$$PRESS = \sum_{i=1}^n \left( \frac{e_i}{1 - h_{ii}} \right)^2$$

Here, the  $e_i$  are the residuals from a single regression with the entire data set, and the  $h_{ii}$  are the corresponding diagonal elements of  $\mathbf{H}$ .

We will now illustrate these concepts with an example involving the fitting of a polynomial model.

## EXAMPLE 15.4

## Regression of a Polynomial Model for the Viscosity of Water versus Temperature

### Problem Statement

Data on the viscosity of water (centipoise, cP) at different temperatures (°C) are presented in [Table 15.4](#). Build a polynomial model that performs well in fitting these data and meets adequacy requirements. Use stepwise regression with forward selection.

**TABLE 15.4** Viscosity of water measured at different temperatures.

| Temperature (°C) | Viscosity (cP) |
|------------------|----------------|
| 0.00             | 1.794          |
| 4.44             | 1.546          |
| 10.00            | 1.310          |
| 15.56            | 1.129          |
| 21.11            | 0.982          |
| 26.67            | 0.862          |
| 32.22            | 0.764          |
| 37.78            | 0.682          |
| 48.89            | 0.559          |
| 60.00            | 0.470          |
| 71.11            | 0.401          |
| 82.22            | 0.347          |
| 93.33            | 0.305          |

### Solution

First, if we look at an 8th-order polynomial based on the temperatures in the table, comparing the 8th-order term for the second and last temperature in the table, respectively, we get

$$1.52 \times 10^5 \quad \text{and} \quad 5.76 \times 10^{15}$$

and note ten orders of magnitude difference in the terms, a *scaling* issue. This reveals a problem with polynomial models that can cause difficulties in solving the normal equations, a badly conditioned **X** matrix.

There is an additional issue with polynomial models. The terms of the polynomial are naturally highly correlated one with another—they are all a function of the single independent variable. This is called *collinearity*.

A common strategy to combat both poor scaling and collinearity is to standardize the independent variable by the transformation

$$z = \frac{x - \bar{x}}{s_x}$$

which will create a vector of values centered about zero and in a range of  $\pm 3$ . Applying this transformation to the temperature data in [Table 15.4](#) produces the following values:

|       |       |       |       |       |       |       |       |      |      |      |      |
|-------|-------|-------|-------|-------|-------|-------|-------|------|------|------|------|
| -1.28 | -1.13 | -0.95 | -0.76 | -0.58 | -0.40 | -0.21 | -0.03 | 0.34 | 0.70 | 1.07 | 1.44 |
|-------|-------|-------|-------|-------|-------|-------|-------|------|------|------|------|

We will use these values to build the model. Then, the penalty we pay is that we must transform a temperature first before using the model to predict viscosity.

The Python code in [Fig. 15.5](#) is used to compute the regression and associated performance statistics.

**FIGURE 15.5** Python code for complete polynomial regression of viscosity of water data.

```
import numpy as np
import pylab
# set table data
x = np.array([0., 4.44, 10., 15.56, 21.11, 26.67, 32.22,
              37.78, 48.89, 60., 71.11, 82.22, 93.33])
y = np.array([1.794, 1.546, 1.31, 1.129, 0.982, 0.862,
              0.764, 0.682, 0.559, 0.47, 0.401, 0.347, 0.305])

# standardize x to z
xbar = np.mean(x)
sx = np.std(x)
z = (x-xbar)/sx
n = len(x)
m = 9
# order of polynomial
# calculate the X matrix
X = np.zeros((n,m+1))
for i in range(n):
    for j in range(m+1):
        X[i,j] = z[i]**j
# formulate and solve the normal equations
Xt = np.transpose(X)
A = np.dot(Xt,X)
const = np.dot(Xt,np.transpose(y))
b = np.linalg.solve(A,const)
print('Estimated model parameters are:\n',b)
#compute model predictions and residuals
yhat = np.zeros((n))
e = np.zeros((n))
for i in range(n):
    for j in range(m+1):
        yhat[i] = yhat[i] + b[j]*z[i]**j
    e[i] = y[i] - yhat[i]
#compute the sum of squares
SSE = np.dot(e,e) # residuals
SST = np.var(y)*(n-1) # total corrected
SSR = SST - SSE # regression
print('SSE = ',SSE)
```



```

print('SST = ',SST)
print('SSR = ',SSR)
# R2 and the standard error of the estimate
R2 = SSR/SST
se2 = SSE/(n-(m+1))
se = np.sqrt(se2)
print('R-squared = ',R2)
print('Standard error of the estimate = ',se)
# adjusted R2
R2adj = 1 - (SSE/(n-(m+1)))/(SST/(n-1))
print('Adjusted R-squared = ',R2adj)
# covariance matrix and parameter standard errors
XtXinv = np.linalg.inv(np.dot(Xt,X))
covb = XtXinv*se2
seb = np.zeros((m+1))
for j in range(m+1):
    seb[j] = np.sqrt(se2*covb[j,j])
print('Standard errors of the parameter estimates:\n',seb)
# hat matrix, PRESS statistic and predicted R2
H = np.dot(np.dot(X,XtXinv),Xt)
PRESS = 0
for i in range(n):
    PRESS = PRESS + (e[i]/(1-H[i,i]))**2
R2pred = 1 - PRESS/SST
print('PRESS = ',PRESS)
print('Predicted R-squared = ',R2pred)
# plots
def ypred(x,xavg,xstd,b,m):
    yp = 0
    zvar = (x-xavg)/xstd
    for j in range(m+1):
        yp = yp + b[j]*zvar**j
    return yp
xplot = np.linspace(0,95.)
npt = len(xplot)
yplot = np.zeros((npt))
for k in range(npt):
    yplot[k] = ypred(xplot[k],xbar,sx,b,m)
pylab.scatter(x,y,c='k',marker='s')
pylab.plot(xplot,yplot,c='k')
pylab.grid()
pylab.xlabel('Temperature - deg C')
pylab.ylabel('Viscosity - cP')
pylab.figure()
pylab.plot(yhat,e,c='k',ls='-',marker='s')
pylab.grid()
pylab.xlabel('Predicted Viscosity - cP')
pylab.ylabel('Residual - cP')

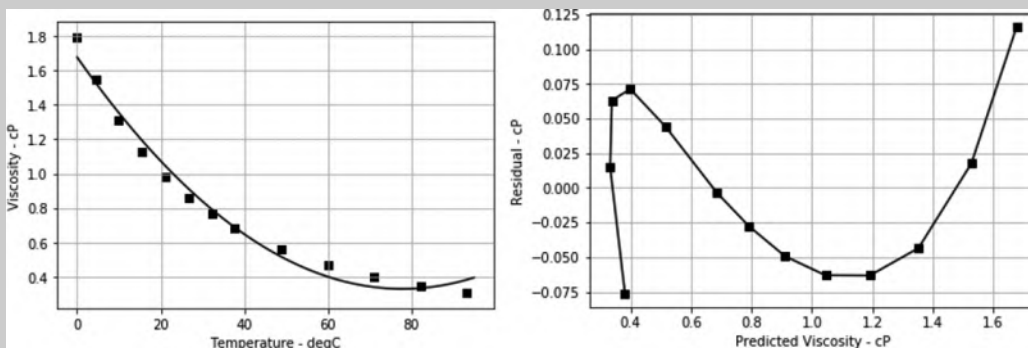
```

The code in  Fig. 15.5 fits a 2nd-order polynomial to the data. Here are the results, both quantitative and graphical.

```

Estimated model parameters are:
[ 0.66898204 -0.50752418 0.18878719]
SSE = 0.04387720062491488
SST = 2.4905652071005924
SSR = 2.4466880064756773
R-squared = 0.982382633267412
Standard error of the estimate = 0.06623986762133124
Adjusted R-squared = 0.9788591599208946
Standard errors of the parameter estimates:
[0.00184482 0.00137498 0.00138652]
PRESS = 0.11329036057233799
Predicted R-squared = 0.9545121885388315

```



We can tell from the plots that a quadratic polynomial is not adequate, even though the three  $R^2$  values are fairly high. The model just can't fit the pattern of the data, and this is readily evident in the residuals versus fits plot. Notice that the  $R^2$  value is greater than the  $R_{\text{adj}}^2$  value which is in turn greater than the  $R_{\text{pred}}^2$  value.

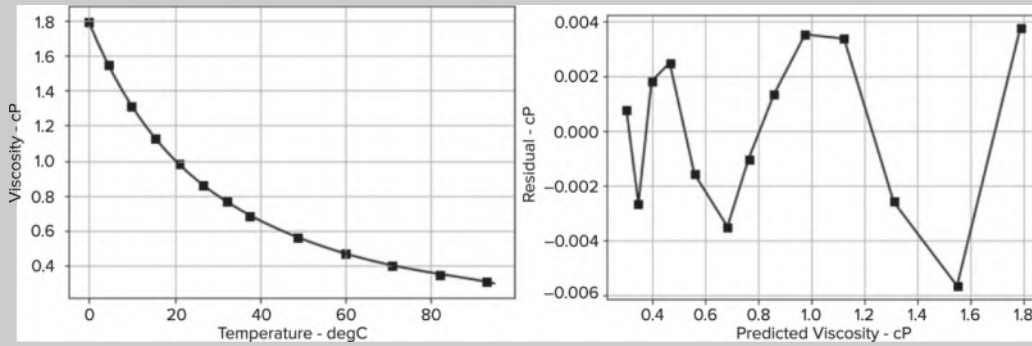
Rather than take up a lot of space showing the detailed results for our forward selection process, we present summary results for each polynomial order below and highlight the optimal selections for the various criteria.

Page 362

| Order | $R^2$    | $\text{adj}R^2$ | $\text{pred}R^2$ | $s_e$    |
|-------|----------|-----------------|------------------|----------|
| 2     | 0.98238  | 0.97886         | 0.95451          | 0.066240 |
| 3     | 0.99840  | 0.99787         | 0.99171          | 0.021040 |
| 4     | 0.99982  | 0.99973         | 0.99803          | 0.007493 |
| 5     | 0.99996  | 0.99992         | 0.99843          | 0.004000 |
| 6     | 0.99999  | 0.99998         | 0.99707          | 0.002065 |
| 7     | 0.999999 | 0.999998        | 0.99898          | 0.000601 |
| 8     | 0.999999 | 0.999999        | 0.99995          | 0.000474 |
| 9     | 0.999999 | 0.999999        | 0.97529          | 0.000547 |

The  $R_{\text{pred}}^2$  measure goes through two maxima for the 5th- and 8th-order polynomials, respectively. The standard error of the estimate,  $s_e$ , has a minimum for the 8th-order model. Both  $R^2$  and  $R_{\text{adj}}^2$  point to the highest-order polynomial. Checking performance and adequacy with the plots for the 5th-order fit, we have

Page 363



The fit appears to be excellent and the pattern of the residuals oscillates some but is fairly random. Also, the residuals are on the order of  $10^{-3}$  for viscosity values around unity. The 5th-order polynomial appears to be a good choice, whereas the 8th-order model is perhaps overkill. Here are the complete results for the 5th-order model:

Estimated model parameters are:  
 [ 0.67339962 -0.37086365 0.15887252 -0.07404558 0.04001281 -0.01029659]  
 SSE = 0.000119464027988233  
 SST = 2.4905652071005924  
 SSR = 2.4904532606977936  
 R-squared = 0.9999550518081279  
 Standard error of the estimate = 0.003999042792591091  
 Adjusted R-squared = 0.9999229459567907  
 Standard errors of the parameter estimates:  
 [8.85397093e-06 1.90877863e-05 2.53260683e-05 2.50813895e-05  
 1.34210643e-05 8.96731652e-06]  
 PRESS = 0.00391224304370201  
 Predicted R-squared = 0.9984291746176538

The model is then

Page 364

$$\mu = 0.6734 - 0.3709z + 0.1589z^2 - 0.07405z^3 + 0.04001z^4 - 0.01030z^5$$

It is fair to mention that, although we have presented detail regarding model building and selection, we have only scratched the surface of this topic. More will be learned via independent study or enrolling in a course in applied statistics for engineers and scientists.

## 15.5 NONLINEAR REGRESSION

There are many cases in engineering and science where models that are nonlinear in the parameters must be fit to data. For example, as we have cited before, the *Antoine equation* relating vapor pressure of a liquid substance to temperature is, in logarithmic form,

$$\log_{10}(P_V) = A - \frac{B}{C + T} \quad (15.15)$$

where  $P_V$  = vapor pressure,  $T$  = temperature, and  $A$ ,  $B$ , and  $C$  are adjustable parameters whose values will depend on the units used for pressure and temperature, for example, mmHg and °C, respectively. This formula cannot be manipulated to be linear in the parameters.

Another scenario where nonlinear regression is useful involves estimating parameters for a multi-step numerical method that produces predictions of the response variable,  $y$ . The numerical solution of the parameters of differential equations is a common example of this type of scenario. We will illustrate such applications later in Part Six.

As with linear least-squares regression, nonlinear regression is based on determining the values of the parameters that minimize the sum of squares of the residual errors. However, here, we cannot invoke calculus to provide a closed-form solution, as we found before with the normal equations for linear regression. We must proceed in an iterative fashion to find the minimum.

There are numerical techniques for finding minima that are well-suited to nonlinear regression. One is the Gauss-Newton method based on a Taylor series linearization of the original nonlinear equation. Then, least-squares theory is used to obtain new estimates that, if the equation were actually linear, would determine the best parameter estimates in one step. Of course, with nonlinear models, this doesn't occur, and iterative methods are required. Another method moves, at times slowly, toward the minimum following the slope or gradient of the sum-of-squares function. A well-known technique, called the Marquardt method after its author, combines the Gauss-Newton and gradient approaches. Details on these methods are available elsewhere (Chapra and Canale, 2010).

A practical alternative is to use an optimization technique embedded in existing software, such as the `minimize` function in the SciPy `optimize` module in Python, as was presented in [Chap. 7](#). Here, there are nine well-developed algorithms available to minimize the residual sum-of-squares, and thus achieve a least-squares nonlinear regression result. The `minimize` function also includes the possibility of incorporating constraints on the parameter estimates so that the numerical procedure doesn't wander off into parameter values that are physically unrealistic or cause problems. One aspect of nonlinear regression that is different is that initial estimates of the parameters must be provided. These should be reasonable values; otherwise, the optimization technique may not be able to find the solution.

### EXAMPLE 15.5

#### Nonlinear Regression of the Antoine Equation with Python

## Problem Statement

Data on the vapor pressure of concentrated sulfuric acid (95 wt% aqueous solution) versus temperature are presented in [Table 15.5](#). Fit the Antoine equation [[Eq. \(15.15\)](#)] to these data using nonlinear regression.

**TABLE 15.5** Vapor pressure of 95 wt% sulfuric acid (torr or mmHg) versus temperature (°C)

| Temperature<br>(degC) | Vapor<br>Pressure<br>(torr) | Temperature<br>(degC) | Vapor<br>Pressure<br>(torr) | Temperature<br>(degC) | Vapor<br>Pressure<br>(torr) |
|-----------------------|-----------------------------|-----------------------|-----------------------------|-----------------------|-----------------------------|
| 35                    | 0.0015                      | 125                   | 1.07                        | 215                   | 66.9                        |
| 40                    | 0.00235                     | 130                   | 1.42                        | 220                   | 79.8                        |
| 45                    | 0.0037                      | 135                   | 1.87                        | 225                   | 95.5                        |
| 50                    | 0.0058                      | 140                   | 2.4                         | 230                   | 115                         |
| 55                    | 0.00877                     | 145                   | 3.11                        | 235                   | 137                         |
| 60                    | 0.0133                      | 150                   | 4.02                        | 240                   | 164                         |
| 65                    | 0.0196                      | 155                   | 5.13                        | 245                   | 193                         |
| 70                    | 0.0288                      | 160                   | 6.47                        | 250                   | 229                         |
| 75                    | 0.0415                      | 165                   | 8.39                        | 255                   | 268                         |
| 80                    | 0.0606                      | 170                   | 10.3                        | 260                   | 314                         |
| 85                    | 0.0879                      | 175                   | 12.9                        | 265                   | 363                         |
| 90                    | 0.123                       | 180                   | 15.9                        | 270                   | 430                         |
| 95                    | 0.172                       | 185                   | 20.2                        | 275                   | 500                         |
| 100                   | 0.237                       | 190                   | 24.8                        | 280                   | 580                         |
| 105                   | 0.321                       | 195                   | 30.7                        | 285                   | 682                         |
| 110                   | 0.437                       | 200                   | 36.7                        | 290                   | 790                         |
| 115                   | 0.59                        | 205                   | 45.3                        |                       |                             |
| 120                   | 0.788                       | 210                   | 55                          |                       |                             |

## Solution

First, we note that the ratio of the last vapor pressure value to the first is about 527000:1. The log transformation of the Antoine equation is important here; otherwise, the optimization technique will depreciate the residual errors at the low end of the range.

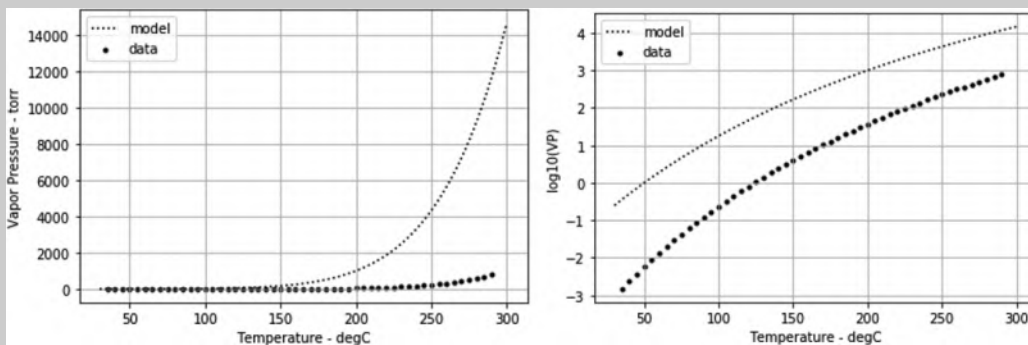
First, we use Python's `pylab` module to create plots of the original data and the log-transformed vapor pressure versus temperature along with a model curve based on some typical Antoine equation parameters (as initial guesses).

```
import numpy as np
import pylab
# read data from text file
T,VP =
```

```

np.loadtxt(fname='H2SO4_VaporPressureData.csv',delimiter=',',unpack=True)
# plot data
pylab.scatter(T,VP,c='k',marker='.',label='data')
pylab.grid()
pylab.xlabel('Temperature - degC')
pylab.ylabel('Vapor Pressure - torr')
# initial guess for parameters
A = 10
B = 3500
C = 300
# plot model curve
Tplot = np.linspace(30.,300.,100)
VPplot = 10**(A-B/(C+Tplot))
pylab.plot(Tplot,VPplot,c='k',ls=':',label='model')
pylab.legend()
# log-transform VP data and generate similar plot
logVP = np.log10(VP)
pylab.figure()
pylab.scatter(T,logVP,c='k',marker='.',label='data')
pylab.grid()
pylab.xlabel('Temperature - degC')
pylab.ylabel('log10(VP)')
logVPplot = np.log10(VPplot)
pylab.plot(Tplot,logVPplot,c='k',ls=':',label='model')
pylab.legend()

```



You note that the data are acquired from a (comma-delimited, csv) text file. This is typical when dealing with larger data sets. We see that, as expected, the model diverges from the data. The difference is not clear on the vapor pressure plot for low temperatures, but the  $\log_{10}$  plot clearly “levels the playing field.” Using another sports analogy, we can see that our initial guesses place the model “in the ballpark.” These guesses could be “tweaked” further to get the initial model closer to the data. Initial, manual “tuning” of parameter guesses is common in nonlinear regression. We now follow with the code for the nonlinear regression and associated statistics.

```

def SSE(params):
    A = params[0] ; B = params[1] ; C = params[2]
    logVPpred = A - B/(C+T)
    e = logVP - logVPpred
    return np.dot(e,e)

from scipy.optimize import minimize
result = minimize(SSE,(A,B,C))
print(result.x)

# plot data
pylab.figure()
pylab.scatter(T,VP,c='k',marker='.',label='data')
pylab.grid()

```

```

pylab.xlabel('Temperature - degC')
pylab.ylabel('Vapor Pressure - torr')
# regressed estimates for parameters
A = result.x[0]
B = result.x[1]
C = result.x[2]
# plot model curve
Tplot = np.linspace(30.,300.,100)
VPplot = 10**(A-B/(C+Tplot))
pylab.plot(Tplot,VPplot,c='k',ls=':',label='model')
pylab.legend()
pylab.title('Nonlinear Regression')

pylab.figure()
pylab.scatter(T,logVP,c='k',marker='.',label='data')
pylab.grid()
pylab.xlabel('Temperature - degC')
pylab.ylabel('log 10(VP)')
pylab.title('Nonlinear Regression')

# regression statistics
SSE = SSE(result.x)
n = len(T)
SST = np.var(logVP)*(n-1)
R2 = 1 - SSE/SST
se = np.sqrt(SSE/(n-3))
print('R-squared = ',R2)
print('Standard error of the estimate = ',se)

logVPpred = A - B/(C+T)
e = logVP - logVPpred
pylab.figure()
pylab.plot(logVPpred,e,c='k')
pylab.grid()
pylab.xlabel('Predicted log 10(VP)')
pylab.ylabel('Residual')
pylab.title('Residuals vs. Fits')

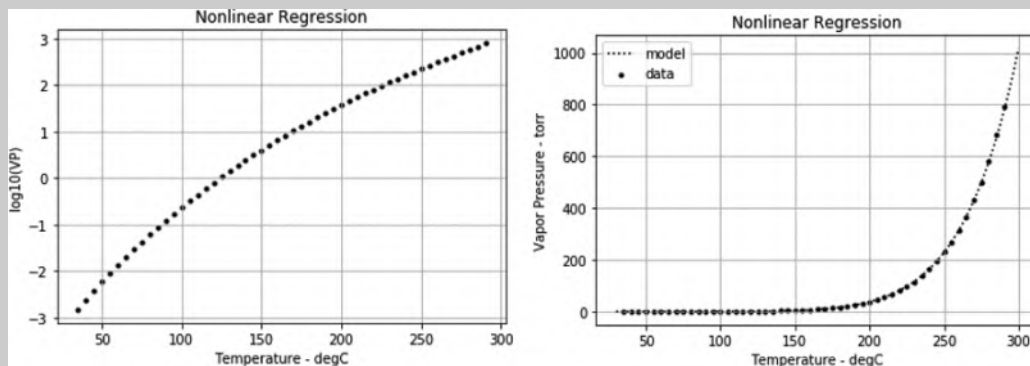
```

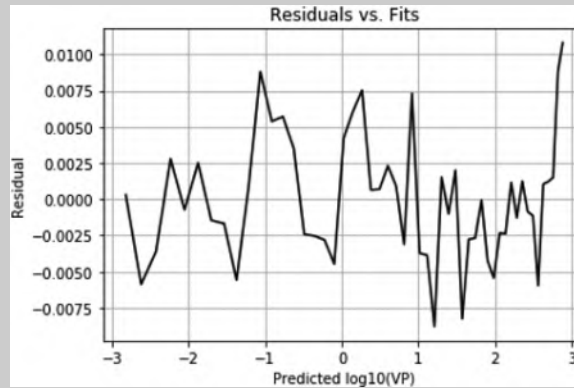
And, here are the results:

```

[ 9.80565591 3901.73546568 273.9292448 ]
R-squared = 0.9999929675994859
Standard error of the estimate = 0.004450499116009375

```





We observe that the fit is excellent. The residuals in  $\log_{10}(P)$  are very small, and the residuals versus fits plot shows no pattern. The model appears to be adequate. This example reveals why the Antoine equation is used to model the effect of temperature on vapor pressure.

There is one remaining issue we will consider regarding nonlinear regression. With linear regression, we were able to estimate standard errors for the parameter estimates using the error variance,  $s_e^2$ , and  $[\mathbf{X}/\mathbf{X}]^{-1}$ . What can we do in the case of nonlinear regression? There are two common approaches, approximation via linearization to provide  $[\mathbf{X}/\mathbf{X}]^{-1}$  and Monte Carlo simulation of random error. We will illustrate the former here. For the Antoine equation illustrated in Example 15.6, we formulate an approximate  $\mathbf{X}$  matrix as

$$\mathbf{X} = \begin{bmatrix} \frac{\partial y}{\partial A}(T_1) & \frac{\partial y}{\partial B}(T_1) & \frac{\partial y}{\partial C}(T_1) \\ \frac{\partial y}{\partial A}(T_2) & \frac{\partial y}{\partial B}(T_2) & \frac{\partial y}{\partial C}(T_2) \\ \vdots & \vdots & \vdots \\ \frac{\partial y}{\partial A}(T_n) & \frac{\partial y}{\partial B}(T_n) & \frac{\partial y}{\partial C}(T_n) \end{bmatrix} \quad y = \log_{10}(P) = A - \frac{B}{C + T}$$

In this case, the partial derivatives can be derived analytically; however, it is common that these must be estimated with finite-differences. We have

$$\frac{\partial y}{\partial A} = 1 \quad \frac{\partial y}{\partial B} = \frac{1}{C + T} \quad \frac{\partial y}{\partial C} = -\frac{B}{(C + T)^2}$$

We recall then that the estimated covariance matrix of the parameters is  $s_e^2 [\mathbf{X}/\mathbf{X}]^{-1}$ , and the standard errors of the parameters are taken as the square roots of the diagonal elements of this matrix. Here, we add code for that from Example 15.6 to compute and display the standard errors.

```
# formulate the X matrix
X = np.zeros((n,3))
for i in range(n):
    X[i,0] = 1
    X[i,1] = 1/(C+T[i])
    X[i,2] = - B/(C+T[i])**2
```



```
# compute the covariance matrix
XtXinv = np.linalg.inv(np.dot(np.transpose(X),X))
cov = se**2 * XtXinv
seA = cov[0,0]
seB = cov[1,1]
seC = cov[2,2]
print(' Estimated parameter standard errors:')
print(' A: ',seA)
print(' B: ',seB)
print(' C: ',seC)
```

The results are

Page 368

```
Estimated parameter standard errors:
A: 0.0004991211539259338
B: 346.5871827501103
C: 0.9397624976948085
```

It is interesting to evaluate the coefficient of variation (%) for each of these results.

```
# coefficients of variation
print(' Coefficients of variation (%):')
print(' A: {0:7.3g}'.format(seA/A*100))
print(' B: {0:7.3g}'.format(seB/B*100))
print(' C: {0:7.3g}'.format(seC/C*100))

Coefficients of variation (%):
A: 0.00509
B: 8.88
C: 0.343
```

We observe that the intervals of certainty are quite different for the three parameters with  $B$  being the least certain at about 9% CV.

This section provides you with a good basis for carrying out nonlinear regression calculations, including the associated statistics that are helpful in validating the model. Nonlinear models often arise from efforts to represent phenomena based on fundamental considerations, and, as such, may not be subject to model building, whereas empirical models, often polynomial, are valid candidates for model building. There are occasions where fundamental models contain terms that need to be evaluated to determine whether they are significant enough to be retained. In such cases, parameter certainty intervals are used. Also, for nonlinear regression, the certainty of parameter estimates can be very important in reporting results, for example, in the published literature.

## FITTING EXPERIMENTAL DATA

### Background

As mentioned at the end of [Sec. 15.2](#), although there are many cases where a response variable is related linearly to two or more input factors, multiple linear regression has additional utility in being applied to multivariable power equations of the general form

$$y = b_0 x_1^{b_1} x_2^{b_2} \cdots x_m^{b_m} \quad (15.16)$$

Such equations are extremely useful when fitting certain types of experimental data. To apply linear regression, [Eq. \(15.16\)](#) is transformed by taking the logarithm of both sides, as in

$$\log_{10}(y) = \log_{10}(b_0) + b_1 \log_{10}(x_1) + b_2 \log_{10}(x_2) + \cdots + b_m \log_{10}(x_m) \quad (15.17)$$

Thus, the logarithm of the response or dependent variable is linearly dependent on the logarithms of the input factors or independent variables.

### Problem Statement

An example relates to gas transfer in natural waters, such as rivers, lakes, and estuaries. In particular, it Page 369  
has been found that the mass-transfer coefficient of dissolved oxygen,  $K_L$  (m/d), is related to a river's mean water velocity,  $U$  (m/s), and mean depth,  $H$  (m), by

$$K_L = b_0 U^{b_1} H^{b_2} \quad (15.18)$$

Transforming [Eq. \(15.18\)](#) with the base-10 logarithm yields

$$\log_{10}(K_L) = \log_{10}(b_0) + b_1 \log_{10}(U) + b_2 \log_{10}(H) \quad (15.19)$$

The following data were collected in a laboratory flume with water at 20 °C:

|       |      |      |      |      |     |     |     |     |     |
|-------|------|------|------|------|-----|-----|-----|-----|-----|
| $U$   | 0.5  | 2    | 10   | 0.5  | 2   | 10  | 0.5 | 2   | 10  |
| $H$   | 0.15 | 0.15 | 0.15 | 0.3  | 0.3 | 0.3 | 0.5 | 0.5 | 0.5 |
| $K_L$ | 0.48 | 3.9  | 57   | 0.85 | 5   | 77  | 0.8 | 9   | 92  |

Use these data and general linear regression to fit the model described in [Eqs. \(15.16\)](#) and [Eq. \(15.17\)](#).

### Solution

We can adapt the Python code in Fig. 15.15 to create the  $\mathbf{X}$  matrix and solve for the parameters in [Eq. \(15.17\)](#):

```
import numpy as np
import pylab
# create the data arrays
```

```

U = np.array([0.5, 2., 10., 0.5, 2., 10., 0.5, 2., 10.])
H = np.array([0.15, 0.15, 0.15, 0.3, 0.3, 0.3, 0.5, 0.5, 0.5])
KL = np.array([0.48, 3.9, 57., 0.85, 5., 77., 0.8, 9., 92.])
# log transformation
logU = np.log10(U)
logH = np.log10(H)
logKL = np.log10(KL)
n = len(U)
# calculate the X matrix
X = np.zeros((n,3))
for i in range(n):
    X[i,0] = 1
    X[i,1] = logU[i]
    X[i,2] = logH[i]
# formulate and solve the normal equations
Xt = np.transpose(X)
A = np.dot(Xt,X)
const = np.dot(Xt,np.transpose(logKL))
b = np.linalg.solve(A,const)
print('Estimated model parameters are:\n',b)
# back-transform the intercept
b0 = 10**b[0]
print('b0 = ',b0)

```

The resulting parameter estimates are

```

Estimated model parameters are:
[0.57627483 1.5620453 0.50742446]
b0 = 3.769422594695167

```

The best-fit model, as transformed, is

$$\log_{10}(K_L) = 0.5763 + 1.562 \log_{10}(U) + 0.5074 \log_{10}(H)$$

or, in original form,

$$K_L = 3.769 U^{1.562} H^{0.5074}$$

As is our custom, we can also compute statistics related to the fit with the Python code:

---

Page 370

---

```

# model predictions
logKLpred = b[0] + b[1]*logU + b[2]*logH
# residuals and SSE
e = logKL - logKLpred
SSE = np.dot(e,e)
print('SSE = ',SSE)
# SST
SST = np.var(logKL)*(n-1)
# R2 and se
R2 = 1 - SSE/SST
se = np.sqrt(SSE/(n-3))
print('R-squared = ',R2)
print('Standard error = ',se)

```

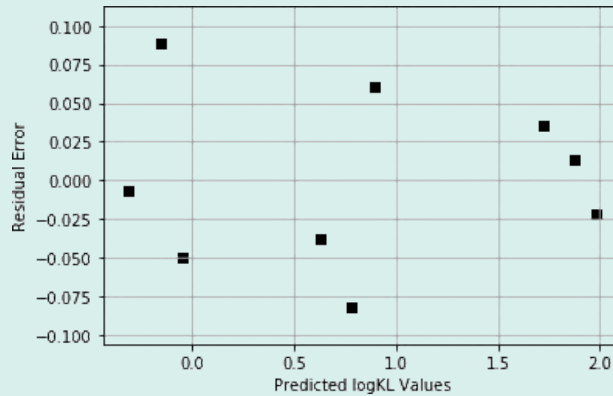
These statistics are

```

SSE = 0.02417133664466078
R-squared = 0.9957090256143829
Standard error = 0.06347090756751052

```

Notice that we compute these results in terms of the log-transformed model. A plot of the residuals versus predicted  $\log_{10}(K_L)$  values can be used to judge adequacy.



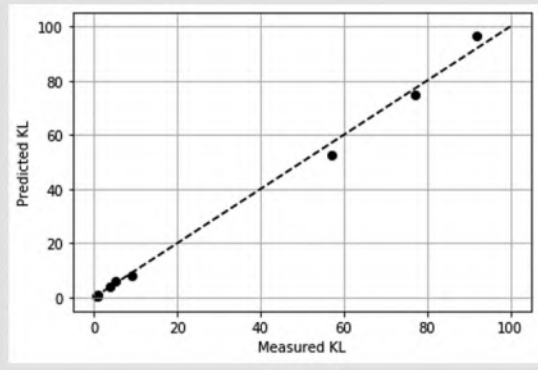
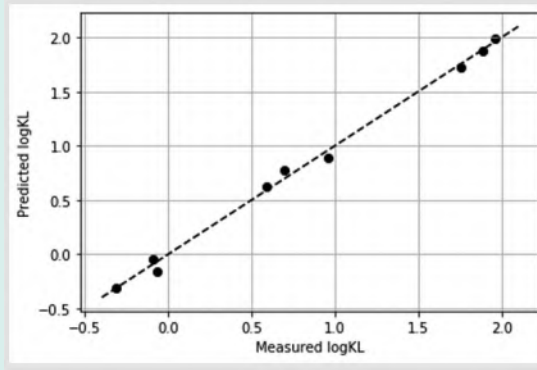
There is no distinct pattern other than the natural grouping of data triplets. The model appears to be adequate.

If we want to present the fit results in a two-dimensional plot, we cannot use the format employed for polynomials because there is more than one input factor. In these situations, it is customary to plot predicted responses versus actual measurements where perfect agreement would be along a 45°-line. Here is the Python code for plots of the transformed and original data and model predictions.

```
# performance plots
pylab.figure()
pylab.scatter(logKL, logKLpred, c='k', marker='o')
pylab.plot([-0.4, 2.1], [-0.4, 2.1], c='k', ls='--')
pylab.grid()
pylab.xlabel('Measured logKL')
pylab.ylabel('Predicted logKL')
pylab.figure()
KLpred = 10**logKLpred
pylab.scatter(KL, KLpred, c='k', marker='o')
pylab.plot([0., 100.], [0., 100.], c='k', ls='--')
pylab.grid()
pylab.xlabel('Measured KL')
pylab.ylabel('Predicted KL')
```

The resulting plots are presented in [Fig. 15.6](#). Notice how the log transformation spreads the data out, and, as with the vapor pressure data in Example 15.6, the transformation spreads out and scales the data, so the smaller values are not depreciated in the fit. Also, if there were a pattern in the data points with respect to the 45°-line, that would indicate model inadequacy, not the case here.

**FIGURE 15.6** Plots of predicted versus measured values of the oxygen mass-transfer coefficient based on regression of the multivariable power model with input variables of water velocity and stream depth. Results are shown for (a) the log-transformed data upon which the regression was performed, and (b) the original data and power model. The dashed 45°-line shows perfect agreement of model predictions with measured values.



# PROBLEMS

**15.1** Fit a parabola to the data from [Table 14.1](#). Determine the  $R^2$  for the fit and evaluate its adequacy.

**15.2** Using the same approach as was employed to derive [Eqs. \(14.15\)](#) and [\(14.16\)](#), derive the least-squares fit of the following model:

$$y = \beta_1 x + \beta_2 x^2 + \varepsilon$$

That is, determine the normal equations for the least-squares fit of a second-order polynomial with zero intercept. Test the approach by using it to fit the data from [Table 14.1](#).

**15.3** Find an appropriate polynomial model for the following data. Use  $R_{\text{adj}}^2$  and  $s_e$  as criteria for selecting the order of the polynomial of your choice. Create plots of the data with your chosen model curve and the residuals versus the predicted  $y$  values.

|          |     |     |     |     |     |     |     |     |
|----------|-----|-----|-----|-----|-----|-----|-----|-----|
| <b>x</b> | 3   | 4   | 5   | 7   | 8   | 9   | 11  | 12  |
| <b>y</b> | 1.6 | 3.6 | 4.4 | 3.4 | 2.2 | 2.8 | 3.8 | 4.6 |

**15.4** For the data from [Prob. 15.3](#), determine the *PRESS* statistic and  $R_{\text{pred}}^2$  for polynomials of order 3, 4, and 5. Do these results confirm your choice of polynomial in [Prob. 15.3](#).

**15.5** For the data in [Table P15.5](#), use polynomial regression to develop a predictive equation for dissolved oxygen concentration as a function of temperature for the case where the chloride concentration is equal to zero. Employ a polynomial of sufficiently high order that the predictions match the measurements to the number of significant digits displayed in the table.

**TABLE P15.5** Dissolved oxygen concentration in water as a function of temperature (°C) and chloride concentration (mg/L).

| Dissolved Oxygen (mg/L) for Temperature (°C) and Concentration of Chloride (g/L) |                     |                      |                      |
|----------------------------------------------------------------------------------|---------------------|----------------------|----------------------|
| $T, ^\circ\text{C}$                                                              | $c = 0 \text{ g/L}$ | $c = 10 \text{ g/L}$ | $c = 20 \text{ g/L}$ |
| 0                                                                                | 14.6                | 12.9                 | 11.4                 |
| 5                                                                                | 12.8                | 11.3                 | 10.3                 |
| 10                                                                               | 11.3                | 10.1                 | 8.96                 |
| 15                                                                               | 10.1                | 9.03                 | 8.08                 |
| 20                                                                               | 9.09                | 8.17                 | 7.35                 |
| 25                                                                               | 8.26                | 7.46                 | 6.73                 |

**Dissolved Oxygen (mg/L) for Temperature (°C) and Concentration of Chloride (g/L)**

| $T, ^\circ\text{C}$ | $c = 0 \text{ g/L}$ | $c = 10 \text{ g/L}$ | $c = 20 \text{ g/L}$ |
|---------------------|---------------------|----------------------|----------------------|
| 30                  | 7.56                | 6.85                 | 6.20                 |

**15.6** For the data from [Table P15.5](#), use multiple linear regression to develop a model to predict dissolved oxygen concentration as a function of temperature and chloride concentration. Use the model equation to estimate the concentration of dissolved oxygen for a chloride concentration of 15 g/L at  $T = 12^\circ\text{C}$ . Note that the measured value, not reported in the table, is 9.09 mg/L. Compute the percent relative error for your prediction. Explain possible causes for the discrepancy.

**15.7** Based on the models developed in [Probs. 15.5](#) and [15.6](#), a somewhat more complicated model that accounts for the effects of temperature and chloride concentration on dissolved oxygen saturation can be hypothesized in the form

$$DO = \beta_0 + \beta_1 T + \beta_2 c + \beta_3 T^2 + \beta_4 T^3 + \varepsilon$$

Because it involves higher polynomial powers in one (or more) input variables, this is called a Page 372  
*response surface* model. Use general linear regression to estimate the parameters of this model with the data in [Table P15.5](#). Use the resulting model equation to estimate the concentration of dissolved oxygen for a chloride concentration of 15 g/L at  $T = 12^\circ\text{C}$ . Note that the measured value, not reported in the table, is 9.09 mg/L. Compute the percent relative error for your prediction. If you completed [Prob. 15.6](#), compare the errors.

**15.8** Use multiple linear regression to fit a model to the following data:

|       |      |      |      |      |      |      |      |      |      |
|-------|------|------|------|------|------|------|------|------|------|
| $x_1$ | 0    | 1    | 1    | 2    | 2    | 3    | 3    | 4    | 4    |
| $x_2$ | 0    | 1    | 2    | 1    | 2    | 1    | 2    | 1    | 2    |
| $y$   | 15.1 | 17.9 | 12.7 | 25.6 | 20.5 | 35.1 | 29.7 | 45.4 | 40.2 |

Compute the model coefficients, the standard error of the estimate, the  $R^2$  value, and the standard errors of the parameter estimates. Produce plots of the predicted versus measured  $y$  values along with the 45°-line, and of the residuals versus predicted  $y$  values. Comment on the performance and adequacy of the model.

**15.9** The following data were collected for the steady flow of water in a concrete circular culvert:

| Experiment | Diameter, m | Slope, m/m | Flow, m <sup>3</sup> /s |
|------------|-------------|------------|-------------------------|
| 1          | 0.3         | 0.001      | 0.04                    |
| 2          | 0.6         | 0.001      | 0.24                    |
| 3          | 0.9         | 0.001      | 0.69                    |

| Experiment | Diameter, m | Slope, m/m | Flow, m <sup>3</sup> /s |
|------------|-------------|------------|-------------------------|
| 4          | 0.3         | 0.01       | 0.13                    |
| 5          | 0.6         | 0.01       | 0.82                    |
| 6          | 0.9         | 0.01       | 2.38                    |
| 7          | 0.3         | 0.05       | 0.31                    |
| 8          | 0.6         | 0.05       | 1.95                    |
| 9          | 0.9         | 0.05       | 5.66                    |

Use multiple linear regression to fit the following model to these data:

$$Q = \beta_0 D^{\beta_1} S^{\beta_2} + \varepsilon$$

where  $Q$  = flow rate,  $D$  = diameter, and  $S$  = slope.

**15.10** Three disease-carrying organisms decay exponentially in seawater according to the following model:

$$p(t) = Ae^{-1.5t} + Be^{-0.3t} + Ce^{-0.05t}$$

Use general linear least squares to estimate the initial concentration of each organism ( $A$ ,  $B$ , and  $C$ ) given the following measurements:

| $t$    | 0.5 | 1   | 2   | 3   | 4 | 5   | 6   | 7   | 9   |
|--------|-----|-----|-----|-----|---|-----|-----|-----|-----|
| $p(t)$ | 6   | 4.4 | 3.2 | 2.7 | 2 | 1.9 | 1.7 | 1.4 | 1.1 |

Compute and report the  $R^2$  value and standard error of the estimate. Present plots of the data and the model curve and the residuals versus the predicted  $p$  values. Comment on the performance and adequacy of the model.

**15.11** The following model is used to represent the effect of solar radiation on the photosynthesis rate of aquatic plants:

$$P = P_m \frac{I}{I_{\text{sat}}} e^{-\frac{I}{I_{\text{sat}}}} + 1$$

where  $P$  = photosynthesis rate, ( $\text{mg} / (\text{m}^3 \cdot \text{d})$ ),  $P_m$  = the maximum photosynthesis rate,  $I$  = solar radiation ( $\mu\text{E} / (\text{m}^2 \cdot \text{s})$ ), and  $I_{\text{sat}}$  = optimal solar radiation. Use nonlinear regression to estimate  $P_m$  and  $I_{\text{sat}}$  based on the following data:

| $I$ | 50 | 80  | 130 | 200 | 250 | 350 | 450 | 550 | 700 |
|-----|----|-----|-----|-----|-----|-----|-----|-----|-----|
| $P$ | 99 | 177 | 202 | 248 | 229 | 219 | 173 | 142 | 72  |



Present plots of the data and the model line and of the residuals versus the predicted  $P$  values. Comment on the performance and adequacy of the model.

**15.12** The following data are provided:

|                       |     |     |     |     |     |
|-----------------------|-----|-----|-----|-----|-----|
| <b><math>x</math></b> | 1   | 2   | 3   | 4   | 5   |
| <b><math>y</math></b> | 2.2 | 2.8 | 3.6 | 4.5 | 5.5 |

Fit the following model to these data using Python.

$$y = a + bx + \frac{c}{x}$$

Provide the following statistics associated with your fit:  $R^2$ ,  $s_e$ , and standard error estimates for the parameters,  $a$ ,  $b$ , and  $c$ . Create a plot of the data with the model line. Comment on the fit.

**15.13** In [Prob. 14.8](#), we used a transformation to linearize and fit the following model:

$$y = \alpha_4 x e^{\beta_4 x}$$

Use nonlinear regression to estimate  $\alpha_4$  and  $\beta_4$  based on the following data. Provide a plot of your fit with the data and a plot of the residual errors versus the model predictions.

|                       |      |      |      |      |      |      |      |      |      |
|-----------------------|------|------|------|------|------|------|------|------|------|
| <b><math>x</math></b> | 0.1  | 0.2  | 0.4  | 0.6  | 0.9  | 1.3  | 1.5  | 1.7  | 1.8  |
| <b><math>y</math></b> | 0.75 | 1.25 | 1.45 | 1.25 | 0.85 | 0.55 | 0.35 | 0.28 | 0.18 |

**15.14** Enzymatic reactions are used extensively to characterize biologically mediated reactions. The following is an example of a model that is used to fit such reactions:

$$v_0 = \frac{k_m S^3}{K + S^3}$$

where  $v_0$  = the initial reaction rate (M/s),  $S$  = the substrate concentration (M), where  $k_m$  and  $K$  are parameters. The following data have been acquired and can be used to fit this model:

| <b><math>[S], M</math></b> | <b><math>v_0, M/s</math></b> |
|----------------------------|------------------------------|
| 0.01                       | $6.078 \times 10^{-11}$      |
| 0.05                       | $7.595 \times 10^{-9}$       |
| 0.1                        | $6.063 \times 10^{-8}$       |
| 0.5                        | $5.788 \times 10^{-6}$       |
| 1                          | $1.737 \times 10^{-5}$       |

| $[S], M$ | $v_0, M/s$             |
|----------|------------------------|
| 5        | $2.423 \times 10^{-5}$ |
| 10       | $2.430 \times 10^{-5}$ |
| 50       | $2.431 \times 10^{-5}$ |
| 100      | $2.431 \times 10^{-5}$ |

- (a) Use a transformation so that the model is suitable for linear regression. Estimate the parameters. Display the data and model line in on a log-log plot.
- (b) Fit the model using nonlinear regression and display a similar plot. Comment on any differences in parameter values and the plots.

**15.15** Given the data

| $x$ | 5  | 10 | 15 | 20 | 25 | 30 | 35 | 40 | 45 | 50 |
|-----|----|----|----|----|----|----|----|----|----|----|
| $y$ | 17 | 24 | 31 | 33 | 37 | 37 | 40 | 40 | 42 | 41 |

use least-squares regression to fit (a) a straight line, (b) a power equation, (c) a saturation-growth-rate model, and (d) a parabola. For (b) and (c), employ transformations to obtain model forms linear in the parameters. Plot the data along with the curves. Compute  $R_{adj}^2$  for each and use it as a basis for selection of the best model. For your best model, plot the residuals versus the predicted  $y$  values and judge adequacy.

**15.16** The following data represent bacterial growth in a liquid culture over several days:

Page 373

| Day                  | 0     | 4     | 8     | 12    | 16     | 20     |
|----------------------|-------|-------|-------|-------|--------|--------|
| Amount $\times 10^6$ | 67.38 | 74.67 | 82.74 | 91.69 | 101.60 | 112.58 |

Find a best-fit model for these data. Try several possibilities—linear, quadratic, and exponential. Use your selected model to predict the number of bacteria after 35 days.

**15.17** The heat capacity of sulfuric acid solutions is related to the concentration of  $H_2SO_4$ , and this property has been measured carefully in the laboratory with the data presented in the table below.

| wt% $H_2SO_4$ | $C_p$ kJ/(kg·K) | wt% $H_2SO_4$ | $C_p$ kJ/(kg·K) |
|---------------|-----------------|---------------|-----------------|
| 0.34          | 4.173           | 35.25         | 3.030           |
| 0.68          | 4.160           | 37.69         | 2.940           |
| 1.34          | 4.135           | 40.49         | 2.834           |
| 2.65          | 4.087           | 43.75         | 2.711           |
| 3.50          | 4.056           | 47.57         | 2.576           |
| 5.16          | 3.998           | 52.13         | 2.429           |

| wt% H <sub>2</sub> SO <sub>4</sub> | C <sub>p</sub> kJ/(kg·K) | wt% H <sub>2</sub> SO <sub>4</sub> | C <sub>p</sub> kJ/(kg·K) |
|------------------------------------|--------------------------|------------------------------------|--------------------------|
| 9.82                               | 3.842                    | 57.65                              | 2.269                    |
| 15.36                              | 3.671                    | 64.47                              | 2.098                    |
| 21.40                              | 3.491                    | 73.13                              | 1.938                    |
| 22.27                              | 3.465                    | 77.91                              | 1.892                    |
| 23.22                              | 3.435                    | 81.33                              | 1.876                    |
| 24.25                              | 3.403                    | 82.49                              | 1.870                    |
| 25.39                              | 3.367                    | 84.48                              | 1.846                    |
| 26.63                              | 3.326                    | 85.48                              | 1.820                    |
| 28.00                              | 3.281                    | 89.36                              | 1.681                    |
| 29.52                              | 3.231                    | 91.81                              | 1.586                    |
| 30.34                              | 3.202                    | 94.82                              | 1.488                    |
| 31.20                              | 3.173                    | 97.44                              | 1.425                    |
| 33.11                              | 3.107                    | 100.00                             | 1.403                    |

Develop a polynomial model for this relationship using Python. As in [Example 15.4](#), it is appropriate to standardize the wt% values before building the polynomial. Start off with a plot of the data. Also, it is suggested that you read in the table values from a comma-delimited text file (.csv). Use  $R^2_{\text{pred}}$  as the criterion for model selection. Assess adequacy. Discuss any issues that you observe. Note: We will revisit this problem in [Chap. 18](#).

**15.18** Use general least squares to estimate the best possible virial coefficients,  $A_1$  and  $A_2$ , for the following equation of state.  $R = 82.05(\text{mL} \cdot \text{atm})/(\text{mol} \cdot \text{K})$ , and  $T = 303 \text{ K}$ .

$$\frac{P\hat{V}}{RT} = 1 + \frac{A_1}{\hat{V}} + \frac{A_2}{\hat{V}^2}$$

Here,  $\hat{V}$  = specific volume (mL/mol).

| P (atm)                  | 0.985  | 1.108  | 1.363  | 1.631  |
|--------------------------|--------|--------|--------|--------|
| $\hat{v}(\text{mL/mol})$ | 25,000 | 22,200 | 18,000 | 15,000 |

**15.19** Environmental scientists and engineers dealing with the impacts of acid rain must determine the value of the ion product of water,  $K_w$ , as a function of temperature. They have suggested that the following equation can be used to model this relationship:

$$-\log_{10}(K_w) = \frac{a}{T} + b \log_{10}(T) + cT + d$$

where  $T$  = absolute temperature (K), and  $a$ ,  $b$ ,  $c$ , and  $d$  are parameters. Use Python and the following data to carry out regression to estimate the parameters. Create plots of the data and model curve versus temperature and the residual errors versus the predicted values. Comment on the fit.

| $T$ (°C) | $K_w$                   |
|----------|-------------------------|
| 0        | $1.164 \times 10^{-15}$ |
| 10       | $2.950 \times 10^{-15}$ |
| 20       | $6.846 \times 10^{-15}$ |
| 30       | $1.467 \times 10^{-14}$ |
| 40       | $2.929 \times 10^{-14}$ |

**15.20** The distance required to stop an automobile is a factor of both thinking and braking, each of which is a function of vehicle speed. The following experimental data were collected to quantify this relationship. Develop best-fit equations for the thinking and braking components separately, then use these to estimate the total stopping distance for a car traveling 110 km/hr. Also, make this prediction for 150 km/hr. Comment on your results.

| Speed, km/hr | 30  | 45   | 60   | 75   | 90   | 120  |
|--------------|-----|------|------|------|------|------|
| Thinking, m  | 5.6 | 8.5  | 11.1 | 14.5 | 16.7 | 22.4 |
| Braking, m   | 5.0 | 12.3 | 21.0 | 32.9 | 47.6 | 84.7 |

**15.21** The data below have been collected, and a model has been proposed,

$$y = a + be^{cx}$$

Use nonlinear regression to fit this model to the data, and check adequacy of your model. Show plots of the data and the model line and the residuals versus the predicted  $y$  values.

| $x$ | 0.4  | 1.4  | 5.4  | 19.5 | 48.2 | 95.9 |
|-----|------|------|------|------|------|------|
| $y$ | 51.6 | 53.4 | 20.0 | -4.2 | -3.0 | -4.8 |

**15.22** It is known that the data below can be modeled with the following equation:

$$y = \left( a + \frac{b}{\sqrt{x}} \right)^2$$

Estimate the parameters using nonlinear regression. Use your model to predict  $y$  for  $x = 1.6$ . Could you use this model and apply linear regression? If so, carry this out and compare its prediction to that of the nonlinear regression.

|          |      |     |     |     |   |
|----------|------|-----|-----|-----|---|
| <b>x</b> | 0.5  | 1   | 2   | 3   | 4 |
| <b>y</b> | 10.4 | 5.8 | 3.3 | 2.4 | 2 |

**15.23** An investigator has reported the data tabulated below for an experiment to determine the growth rate of bacteria,  $k$  (per day), as a function of oxygen concentration,  $c$  (mg/L). It is common for such growth processes to be modeled by the following equation:

$$k = \frac{k_{\max} c^2}{c_s + c^2}$$

Use nonlinear regression to estimate  $c_s$  and  $k_{\max}$  and predict the growth rate at  $c = 2$  mg/L. Page 374

|          |     |     |     |     |     |
|----------|-----|-----|-----|-----|-----|
| <b>c</b> | 0.5 | 0.8 | 1.5 | 2.5 | 4   |
| <b>k</b> | 1.1 | 2.4 | 5.3 | 7.6 | 8.9 |

Show plots of the data and the model line and the residuals versus the predicted  $k$  values.

**15.24** A material is tested for cyclic fatigue whereby a stress (MPa) is applied to the material and the number of cycles needed to cause failure is measured. The results are in the table below.

|                     |      |      |     |       |        |         |           |
|---------------------|------|------|-----|-------|--------|---------|-----------|
| <b>Stress (MPa)</b> | 1    | 10   | 100 | 1,000 | 10,000 | 100,000 | 1,000,000 |
| <b>Cycles</b>       | 1100 | 1000 | 925 | 800   | 625    | 550     | 420       |

Examine the data, create plots as needed, and select a model that you think might be appropriate. Determine the model parameters via regression and test the model for adequacy.

**15.25** The following data show the relationship between the viscosity of SAE 70 oil and temperature. A common model used to represent this dependency is

$$\mu = b_0 e^{b_1/T}$$

where  $\mu$  = viscosity (Pa·s) and  $T$  = absolute temperature (K). Use nonlinear regression to estimate the parameters  $b_0$  and  $b_1$ . Compute  $R^2$  and  $s_e$ . Compute the standard error estimates for the parameters. Comment on the uncertainty of the parameter estimates. Provide a plot of the data and model line and of the residuals versus the predicted  $\mu$  values.

|                         |       |       |        |         |
|-------------------------|-------|-------|--------|---------|
| <b>Temperature (°C)</b> | 26.67 | 93.33 | 148.89 | 315.56  |
| <b>Viscosity (Pa·s)</b> | 1.35  | 0.085 | 0.012  | 0.00075 |

**15.26** The concentration of *E. coli* bacteria in a swimming pool is measured at different times after a rainstorm:

|                             |      |      |      |     |     |     |
|-----------------------------|------|------|------|-----|-----|-----|
| <b><i>t</i>(hr)</b>         | 4    | 8    | 12   | 16  | 20  | 24  |
| <b><i>c</i>(CFU/100 mL)</b> | 1590 | 1320 | 1000 | 900 | 650 | 560 |

Employ nonlinear regression to fit directly an exponential model [Eq. (14.22)] to these data. Use the model to estimate (a) the concentration at the end of the storm ( $t = 0$ ), and (b) the time at which the concentration will reach 200 CFU/(100 mL).

**15.27** Nitric oxide (NO) is absorbed into a reacting solution to produce a product. The data below are measurements taken from this process. Use nonlinear regression to estimate the parameters,  $b_1$ ,  $b_2$ , and  $b_3$ , in the model

$$y = b_0 e^{b_1 x} x^{b_2}$$

where  $y$  = concentration of product (g/L) and  $x$  = NO absorbed (g/L).

|                 |      |      |       |       |       |       |       |       |       |
|-----------------|------|------|-------|-------|-------|-------|-------|-------|-------|
| <b><i>x</i></b> | 0.09 | 0.32 | 0.69  | 1.51  | 2.29  | 3.06  | 3.39  | 3.63  | 3.77  |
| <b><i>y</i></b> | 15.1 | 57.3 | 103.3 | 174.6 | 191.5 | 193.2 | 178.7 | 172.3 | 167.5 |



Use your model to predict the amount of NO absorbed that provides the highest product concentration. Create a plot of the data and your model line and a plot of the residuals versus the predicted  $y$  values. Compute the  $R^2$  value and the standard error of the estimate. Comment on the goodness of fit and adequacy of the model.

**15.28** The *Soave-Redlich-Kwong* (SRK) equation of state is used to describe the behavior of gases at elevated, non-ideal pressures and temperatures. It is a modification of the ideal gas law and is given by

$$P = \frac{RT}{\hat{V} - b} - \frac{\alpha a}{\hat{V}(\hat{V} - b)}$$

where  $P$  = pressure (atm),  $R$  = gas law constant, 0.082057 L atm/(mol·K),  $T$  = temperature (K),  $\hat{V}$  = specific volume, L/mol, and  $a$ ,  $\alpha$ , and  $b$  are empirical constants for the gas under consideration given by,

$$a = 0.42747 \frac{(RT_c)^2}{P_c} \quad b = 0.08664 \frac{RT_c}{P_c}$$

$$\alpha = \left[ 1 + m \left( 1 - \sqrt{\frac{T}{T_c}} \right) \right]^2$$

where

$$m = 0.48508 + 1.5517\omega - 0.1561\omega^2$$

where  $T_c$  = the critical temperature for the gas (K),  $P_c$  = the critical pressure for the gas (atm), and  $\omega$  = the Pitzer acentric factor for the gas.

Note: The ideal gas law is given by the simpler formula

$$P = \frac{RT}{\hat{V}}$$

The data presented in the table were obtained via careful laboratory experiments for sulfur dioxide gas (SO<sub>2</sub>).

| $\hat{V}(\text{L/mol})$ | $T(\text{K})$ | $P(\text{atm})$ |
|-------------------------|---------------|-----------------|
| 4.345                   | 323.2         | 5.651           |
| 3.260                   | 323.2         | 7.338           |
| 2.901                   | 323.2         | 8.118           |
| 4.674                   | 348.2         | 5.767           |
| 3.178                   | 348.2         | 8.237           |
| 1.495                   | 348.2         | 15.71           |
| 5.136                   | 373.2         | 5.699           |
| 2.926                   | 373.2         | 9.676           |
| 1.620                   | 373.2         | 16.345          |
| 0.979                   | 373.2         | 24.401          |
| 5.419                   | 398.2         | 5.812           |
| 2.734                   | 398.2         | 11.12           |
| 1.504                   | 398.2         | 19.017          |
| 0.944                   | 398.2         | 27.921          |
| 1.532                   | 423.2         | 20.314          |
| 1.169                   | 423.2         | 25.695          |
| 1.325                   | 473.2         | 26.617          |
| 0.679                   | 473.2         | 47.498          |

Sulfur dioxide (SO<sub>2</sub>) has a molecular weight of 64.07 g/mol. Its critical properties are  $T_c = 430.7$  K and  $P_c = 77.8$  atm. For this gas,  $\omega = 0.251$ . Employ nonlinear regression to determine the values of the parameters  $a$  and  $b$  and compare these values to those predicted by theory, as shown by the relationships above. Use the value of  $a$  computed with the formulas above.

Finally, using your model, plot the predicted pressures for the conditions in the table based on the SRK equation of state and ideal gas law versus the measured pressures. Include the 45°-line on the plot and comment on agreement (or lack of agreement).

Note: You may find it convenient to enter the data in a text file and read the file into your Python script.



# Fourier Analysis

## CHAPTER OBJECTIVES

The primary objective of this chapter is to introduce you to Fourier analysis. This topic, which is named after Joseph Fourier<sup>1</sup>, involves identifying cycles or patterns within a time series of data. Specific objectives and topics covered are

- Understanding sinusoids and how they can be used in curve fitting.
- Knowing how to use least-squares regression to fit a sinusoid to data.
- Understanding the relationship between sinusoids and complex exponentials based on Euler's formula.
- Recognizing the benefits of analyzing mathematical functions or signals in the frequency domain, that is, as a function of frequency rather than time.
- Understanding how the Fourier integral and transform extend Fourier analysis to aperiodic functions and signals.
- Understanding how the discrete Fourier transform (DFT) extends Fourier analysis to sampled signals.
- Recognizing how discrete sampling limits the ability of the DFT to distinguish frequencies. In particular, knowing how to compute and interpret the Nyquist frequency.
- Recognizing how the fast Fourier transform (FFT) provides a highly efficient means to compute the DFT for cases where the length of the data record is a power of 2.
- Knowing how to use Python to compute the DFT and FFT and understanding how to interpret the results.
- Knowing how to compute and interpret the power spectrum.

## YOU'VE GOT A PROBLEM

At the beginning of  **Chap. 8**, we used Newton's second law and force balances to predict the equilibrium positions of three bungee jumpers connected by cords. Then, in  **Chap. 13**, we determined the same system's eigenvalues and eigenvectors in order to identify its resonant frequencies and principal modes of vibration. Although this analysis certainly provided useful results, it required detailed system information including knowledge of the underlying model and parameters (i.e., the jumpers' masses and the cords' spring constants).

So suppose that you have measurements of the jumpers' positions or velocities at discrete, equally spaced times (recall  **Fig. 13.1**). Such information is referred to as a *time series*. However, suppose further that you do not know the underlying model or the parameters needed to compute the eigenvalues. For such cases, is there any way to use the time series to learn something fundamental about the system's dynamics?

In this chapter, we describe such an approach, *Fourier analysis*, which provides a way to accomplish this objective. The approach is based on the premise that more complicated functions (e.g., a time series) can be represented by the sum of simpler trigonometric functions. As a prelude to outlining how this is done, it is useful to explore how data can be fit with sinusoidal functions.

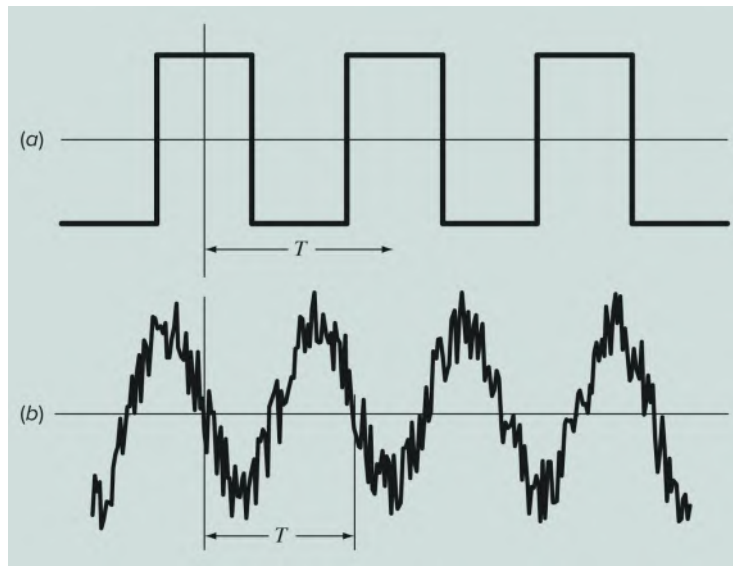
# 16.1 CURVE FITTING WITH SINUSOIDAL FUNCTIONS

A periodic function  $f(t)$  is one for which

$$f(t) = f(t + T) \quad (16.1)$$

where  $T$  is a constant called the *period* that is the smallest value of time for which [Eq. \(16.1\)](#) holds. Common examples include both artificial and natural signals ([Fig. 16.1a](#)).

**FIGURE 16.1** Aside from trigonometric functions such as sines and cosines, periodic functions include idealized waveforms like the square wave depicted in (a). Beyond such artificial forms, periodic signals in nature can be contaminated by noise like the air temperatures shown in (b).



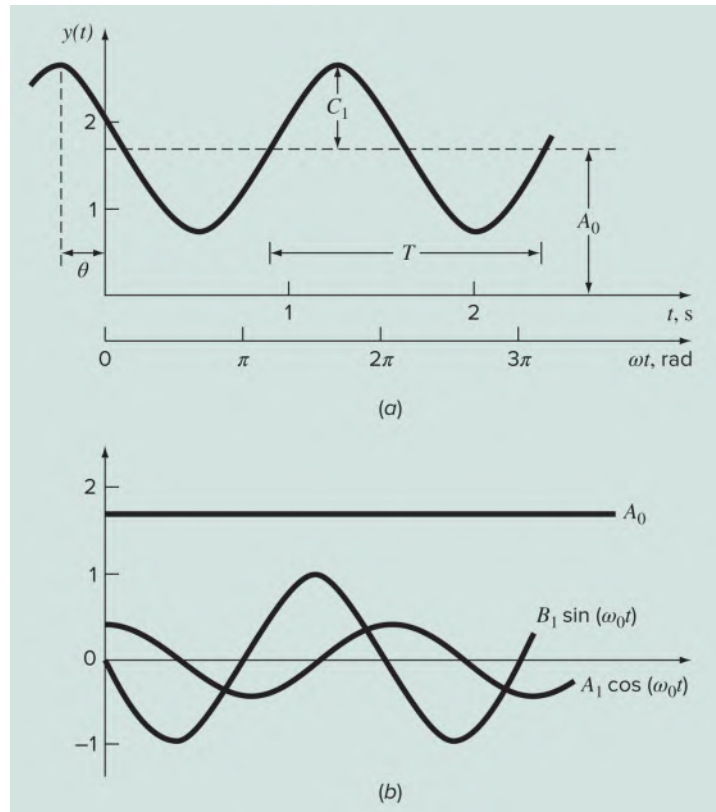
The most fundamental are sinusoidal functions. In this discussion, we will use the term *sinusoid* to represent any waveform that can be described as a sine or cosine. There is no clear-cut convention for choosing either function, and in any case, the results will be identical because the two functions are simply offset in time by  $\pi/2$  radians. For this chapter, we will use the cosine, which can be expressed generally as

$$f(t) = A_0 + C_1 \cos(\omega_0 t + \theta) \quad (16.2)$$

Inspection of  **Eq. (16.2)** indicates that four parameters serve to uniquely characterize the sinusoid ( **Fig. 16.2a**):

- The *mean value*  $A_0$  sets the average height above the abscissa.
- The *amplitude*  $C_1$  specifies the height of the oscillation.
- The *angular frequency*  $\omega_0$  characterizes how often the cycles occur.
- The *phase angle* (or *phase shift*)  $\theta$  parameterizes the extent to which the sinusoid is shifted horizontally.

**FIGURE 16.2** (a) A plot of the sinusoidal function  $y(t) = A_0 + C_1 \cos(\omega_0 t + \theta)$ . For this case,  $A_0 = 1.7$ ,  $C_1 = 1$ ,  $\omega_0 = 2\pi/T = 2\pi/(1.5 \text{ s})$ , and  $\theta = \pi/3 \text{ radians} = 1.0472 (= 0.25 \text{ s})$ . Other parameters used to describe the curve are the frequency  $f = \omega_0/(2\pi)$ , which for this case is 1 cycle/(1.5 s) = 0.6667 Hz and the period  $T = 1.5 \text{ s}$ . (b) An alternative expression of the same curve is  $y(t) = A_0 + A_1 \cos(\omega_0 t) + B_1 \sin(\omega_0 t)$ . The three components of this function are depicted in (b), where  $A_1 = 0.5$  and  $B_1 = -0.866$ . The summation of the three curves in (b) yields the single curve in (a).



Note that the *angular frequency* (in radians/time) is related to the *ordinary frequency*  $f$  (in cycles/time)<sup>2</sup> by

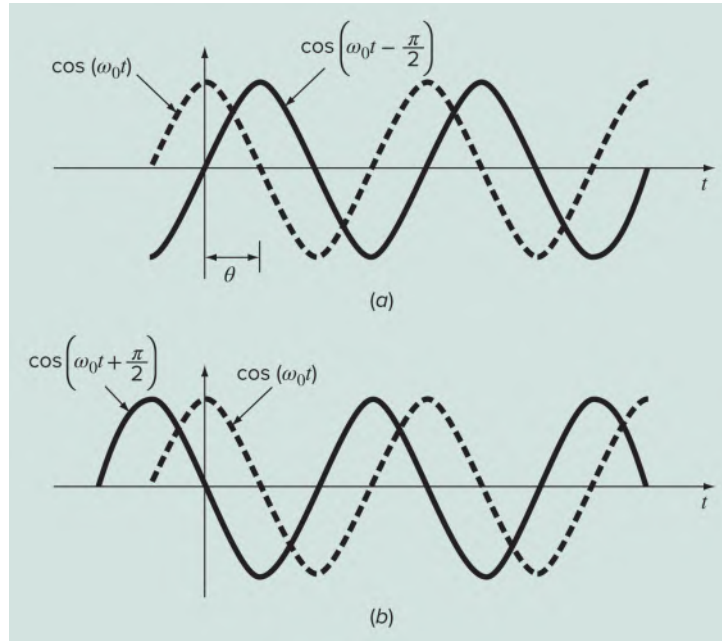
$$\omega_0 = 2\pi f \quad (16.3)$$

and the ordinary frequency in turn is related to the period  $T$  by

$$f = \frac{1}{T} \quad (16.4)$$

In addition, the *phase angle* represents the distance in radians from  $t = 0$  to the point at which the cosine function begins a new cycle. As depicted in [Fig. 16.3a](#), a negative value is referred to as a *lagging phase angle* because the curve  $\cos(\omega_0 t - \theta)$  begins a new cycle  $\theta$  radians after  $\cos(\omega_0 t)$ . Thus,  $\cos(\omega_0 t - \theta)$  is said to lag  $\cos(\omega_0 t)$ . Conversely, as in [Fig. 16.3b](#), a positive value is referred to as a *leading phase angle*.

**FIGURE 16.3** Graphical depictions of (a) a lagging phase angle and (b) a leading phase angle. Note that the lagging curve in (a) can be alternatively described as  $\cos(\omega_0 t + 3\pi/2)$ . In other words, if a curve lags by an angle of  $\alpha$ , it can also be represented as leading by  $2\pi - \alpha$ .



Although [Eq. \(16.2\)](#) is an adequate mathematical characterization of a sinusoid, it is awkward to work with from the standpoint of curve fitting because the phase shift is included in the argument of the cosine function. This deficiency can be overcome by invoking the trigonometric identity:

$$C_1 \cos(\omega_0 t + \theta) = C_1 [\cos(\omega_0 t) \cos(\theta) - \sin(\omega_0 t) \sin(\theta)] \quad (16.5)$$

Substituting [Eq. \(16.5\)](#) into [Eq. \(16.2\)](#) and collecting terms gives ([Fig. 16.2b](#))

$$(16.6)$$

$$f(t) = A_0 + A_1 \cos(\omega_0 t) + B_1 \sin(\omega_0 t)$$

where

$$A_1 = C_1 \cos(\theta) \quad B_1 = -C_1 \sin(\theta) \quad (16.7)$$

Dividing the two parts of [Eq. \(16.7\)](#) gives

$$\theta = \arctan \left( -\frac{B_1}{A_1} \right) \quad (16.8)$$

where, if  $A_1 < 0$ , add  $\pi$  to  $\theta$ . Squaring and summing [Eq. \(16.7\)](#) lead to

$$C_1 = \sqrt{A_1^2 + B_1^2} \quad (16.9)$$

Thus, [Eq. \(16.6\)](#) represents an alternative formulation of [Eq. \(16.2\)](#) that still requires four parameters but that is cast in the format of a general linear model [recall [Eq. \(15.7\)](#)]. As we will discuss in the next section, it can be simply applied as the basis for a least-squares fit.

Before proceeding to the next section, however, we should stress that we could have employed a sine rather than a cosine as our fundamental model of [Eq. \(16.2\)](#). For example,

$$f(t) = A_0 + C_1 \sin(\omega_0 t + \delta)$$

could have been used. Simple relationships can be applied to convert between the two forms:

$$\sin(\omega_0 t + \delta) = \cos\left(\omega_0 t + \delta - \frac{\pi}{2}\right)$$

and

$$\cos(\omega_0 t + \delta) = \sin\left(\omega_0 t + \delta + \frac{\pi}{2}\right) \quad (16.10)$$

In other words,  $\theta = \delta - \pi/2$ . The only important consideration is that one or the other format should be used consistently. Thus, we will use the cosine version throughout our discussion.

## 16.1.1 Least-Squares Fit of a Sinusoid

Equation (16.6) can be thought of as a linear least-squares model:

$$y = A_0 + A_1 \cos(\omega_0 t) + B_1 \sin(\omega_0 t) + e \quad (16.11)$$

which is just another example of the general model [recall [Eq. \(15.9\)](#)]

$$y = \beta_0 f_0(x_1, x_2, \dots, x_p) + \beta_1 f_1(x_1, x_2, \dots, x_p) + \dots + \beta_m f_m(x_1, x_2, \dots, x_p) + \varepsilon$$

where  $f_0 = 1$ ,  $f_1 = \cos(\omega_0 t)$ ,  $f_2 = \sin(\omega_0 t)$ , and all other  $f$ 's = 0. Thus, our goal is to determine coefficient values that minimize

Page 379

$$S_r = \sum_{i=1}^N \{y_i - [A_0 + A_1 \cos(\omega_0 t) + B_1 \sin(\omega_0 t)]\}^2$$

The normal equations to accomplish this minimization can be expressed in matrix form as [recall [Eq. \(15.10\)](#)]

$$\begin{bmatrix} N & \sum \cos(\omega_0 t) & \sum \sin(\omega_0 t) \\ \sum \cos(\omega_0 t) & \sum \cos^2(\omega_0 t) & \sum \cos(\omega_0 t) \sin(\omega_0 t) \\ \sum \sin(\omega_0 t) & \sum \cos(\omega_0 t) \sin(\omega_0 t) & \sum \sin^2(\omega_0 t) \end{bmatrix} \begin{bmatrix} A_0 \\ B_1 \\ B_1 \end{bmatrix} = \begin{bmatrix} \sum y \\ \sum y \cos(\omega_0 t) \\ \sum y \sin(\omega_0 t) \end{bmatrix} \quad (16.12)$$

These equations can be employed to solve for the unknown coefficients. However, rather than do this, we can examine the special case where there are  $N$  observations equispaced at intervals of  $\Delta t$  and with a total record length of  $T = (N - 1)\Delta t$ . For this situation, the following average values can be determined (see [Prob. 16.5](#)):

$$\frac{\sum \sin(\omega_0 t)}{N} = 0 \quad \frac{\sum \cos(\omega_0 t)}{N} = 0 \quad (16.13)$$

$$\begin{aligned} \frac{\sum \sin^2(\omega_0 t)}{N} &= \frac{1}{2} & \frac{\sum \cos^2(\omega_0 t)}{N} &= \frac{1}{2} \\ \frac{\sum \cos(\omega_0 t) \sin(\omega_0 t)}{N} &= 0 \end{aligned}$$

Thus, for equispaced points the normal equations become

$$\begin{bmatrix} N & 0 & 0 \\ 0 & N/2 & 0 \\ 0 & 0 & N/2 \end{bmatrix} \begin{bmatrix} A_0 \\ B_1 \\ B_2 \end{bmatrix} = \begin{bmatrix} \sum y \\ \sum y \cos(\omega_0 t) \\ \sum y \sin(\omega_0 t) \end{bmatrix}$$

The inverse of a diagonal matrix is merely another diagonal matrix whose elements are the reciprocals of the original. Thus, the coefficients can be determined as

$$\begin{bmatrix} A_0 \\ B_1 \\ B_2 \end{bmatrix} = \begin{bmatrix} 1/N & 0 & 0 \\ 0 & N/2 & 0 \\ 0 & 0 & N/2 \end{bmatrix} \begin{bmatrix} \sum y \\ \sum y \cos(\omega_0 t) \\ \sum y \sin(\omega_0 t) \end{bmatrix}$$

or

$$A_0 = \frac{\sum y}{N} \quad (16.14)$$

$$A_1 = \frac{2}{N} \sum y \cos(\omega_0 t) \quad (16.15)$$

$$B_1 = \frac{2}{N} \sum y \sin(\omega_0 t) \quad (16.16)$$

Notice that the first coefficient represents the function's average value.

## EXAMPLE 16.1

### Least-Squares Fit of a Sinusoid

#### Problem Statement

The curve in  Fig. 16.2a is described by  $y = 1.7 + \cos(4.189t + 1.0472)$ . Generate 10 discrete values for this curve at intervals of  $\Delta t = 0.15$  for the range  $t = 0$  to 1.35. Use this information to evaluate the coefficients of  Eq. (16.11) by a least-squares fit.



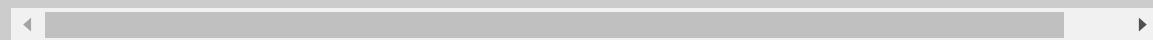
## Solution

The data required to evaluate the coefficients with  $\omega = 4.189$  are

| $t$        | $y$    | $y \cos(\omega_0 t)$ | $y \sin(\omega_0 t)$ |
|------------|--------|----------------------|----------------------|
| 0          | 2.200  | 2.200                | 0.000                |
| 0.15       | 1.595  | 1.291                | 0.938                |
| 0.30       | 1.031  | 0.319                | 0.980                |
| 0.45       | 0.722  | -0.223               | 0.687                |
| 0.60       | 0.786  | -0.636               | 0.462                |
| 0.75       | 1.200  | -1.200               | 0.000                |
| 0.90       | 1.805  | -1.460               | -1.061               |
| 1.05       | 2.369  | -0.732               | -2.253               |
| 1.20       | 2.678  | 0.829                | -2.547               |
| 1.35       | 2.614  | 2.114                | -1.536               |
| $\Sigma =$ | 17.000 | 2.502                | -4.330               |

These results can be used to determine [[Eqs. \(16.14\)](#) through [\(16.16\)](#)]

$$A_0 = \frac{17.000}{10} = 1.7 \quad A_1 = \frac{2}{10} 2.502 = 0.500 \quad B_1 = \frac{2}{10} (-4.330) = -0.866$$



Thus, the least-squares fit is

$$y = 1.7 + 0.500 \cos(\omega_0 t) - 0.866 \sin(\omega_0 t)$$

The model can also be expressed in the format of [Eq. \(16.2\)](#) by calculating [[Eq. \(16.8\)](#)]

$$\theta = \arctan \left( \frac{-0.866}{0.500} \right) = 1.0472$$

and [[Eq. \(16.9\)](#)]

$$C_1 = \sqrt{0.5^2 + (-0.866)^2} = 1.00$$

to give

$$y = 1.7 + \cos(\omega_0 t + 1.0472)$$

or alternatively, as a sine by using [[Eq. \(16.10\)](#)]

$$y = 1.7 + \sin(\omega_0 t + 2.618)$$

The foregoing analysis can be extended to the general model

$$f(t) = A_0 + A_1 \cos(\omega_0 t) + B_1 \sin(\omega_0 t) + A_2 \cos(2\omega_0 t) + B_2 \sin(2\omega_0 t) + \cdots + A_m \cos(m\omega_0 t)$$



where, for equally spaced data, the coefficients can be evaluated by

$$\begin{aligned} A_0 &= \frac{\sum y}{N} \\ A_j &= \frac{2}{N} \sum y \cos(j\omega_0 t) \\ B_j &= \frac{2}{N} \sum y \sin(j\omega_0 t) \end{aligned} \quad \left\{ \begin{array}{l} j = 1, 2, \dots, m \end{array} \right.$$

Although these relationships can be used to fit data in the regression sense (i.e.,  $N > 2m + 1$ ), an alternative application is to employ them for interpolation or collocation—that is, to use them for the case where the number of unknowns  $2m + 1$  is equal to the number of data points  $N$ . This is the approach used in the continuous Fourier series, as described next.

## 16.2 CONTINUOUS FOURIER SERIES

In the course of studying heat-flow problems, Fourier showed that an arbitrary periodic function can be represented by an infinite series of sinusoids of harmonically related frequencies. For a function with period  $T$ , a continuous Fourier series can be written

$$f(t) = a_0 + a_1 \cos(\omega_0 t) + b_1 \sin(\omega_0 t) + a_2 \cos(2\omega_0 t) + b_2 \sin(2\omega_0 t) + \dots$$

or more concisely,

$$f(t) = a_0 + \sum_{k=1}^{\infty} [a_k \cos(k\omega_0 t) + b_k \sin(k\omega_0 t)] \quad (16.17)$$

where the angular frequency of the first mode ( $\omega_0 = 2\pi/T$ ) is called the *fundamental frequency* and its constant multiples  $2\omega_0$ ,  $3\omega_0$ , etc., are called *harmonics*. Thus, [Eq. \(16.17\)](#) expresses  $f(t)$  as a linear combination of the basis functions:  $1$ ,  $\cos(\omega_0 t)$ ,  $\sin(\omega_0 t)$ ,  $\cos(2\omega_0 t)$ ,  $\sin(2\omega_0 t)$ ,  $\dots$

The coefficients of [Eq. \(16.17\)](#) can be computed via

$$a_k = \frac{2}{T} \int_0^T f(t) \cos(k\omega_0 t) dt \quad (16.18)$$

and

$$b_k = \frac{2}{T} \int_0^T f(t) \sin(k\omega_0 t) dt \quad (16.19)$$

for  $k = 1, 2, \dots$  and

$$a_0 = \frac{1}{T} \int_0^T f(t) dt \quad (16.20)$$

### EXAMPLE 16.2

#### Continuous Fourier Series Approximation

## Problem Statement

Use the continuous Fourier series to approximate the square or rectangular wave function (  **Fig. 16.1a**) with a height of 2 and a period  $T = 2\pi/\omega_0$ :

$$f(t) = \begin{cases} -1 & -T/2 < t < -T/4 \\ 1 & -T/4 < t < T/4 \\ -1 & T/4 < t < T/2 \end{cases}$$

## Solution

Because the average height of the wave is zero, a value of  $a_0 = 0$  can be obtained directly. The remaining coefficients can be evaluated as [  **Eq. (16.18)**]

$$\begin{aligned} a_k &= \frac{2}{T} \int_{-T/2}^{T/2} f(t) \cos(k\omega_0 t) dt \\ &= \frac{2}{T} \left[ - \int_{-T/2}^{-T/4} \cos(k\omega_0 t) dt + \int_{-T/4}^{T/4} \cos(k\omega_0 t) dt - \int_{T/4}^{T/2} \cos(k\omega_0 t) dt \right] \end{aligned}$$

The integrals can be evaluated to give

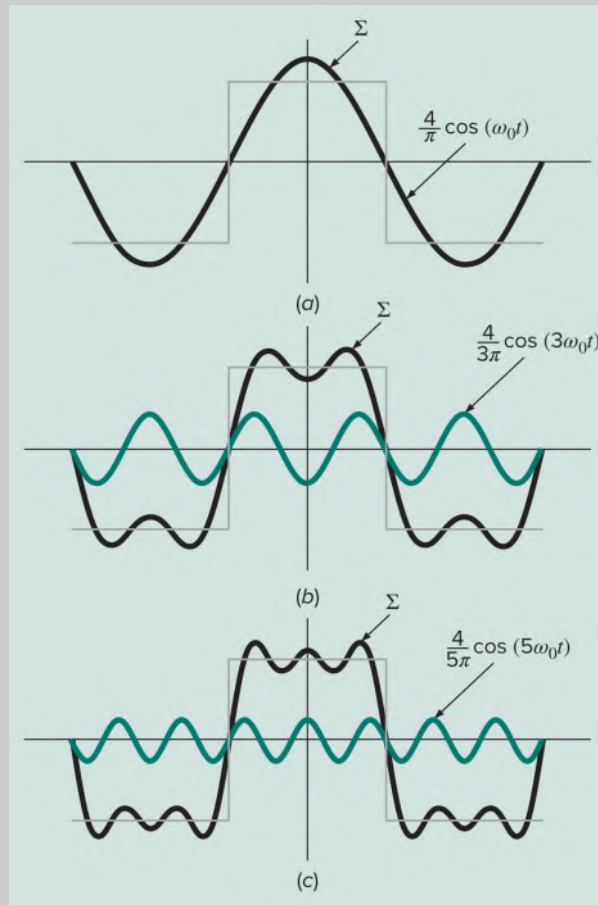
$$a_k = \begin{cases} 4/(k\pi) & \text{for } k = 1, 5, 9, \dots \\ -4/(k\pi) & \text{for } k = 3, 7, 11, \dots \\ 0 & \text{for } k = \text{even integers} \end{cases}$$

Similarly, it can be determined that all the  $b_k$ 's = 0. Therefore, the Fourier series approximation is

$$f(t) = \frac{4}{\pi} \cos(\omega_0 t) - \frac{4}{3\pi} \cos(3\omega_0 t) + \frac{4}{5\pi} \cos(5\omega_0 t) - \frac{4}{7\pi} \cos(7\omega_0 t) + \dots$$

The results up to the first three terms are shown in  **Fig. 16.4**.

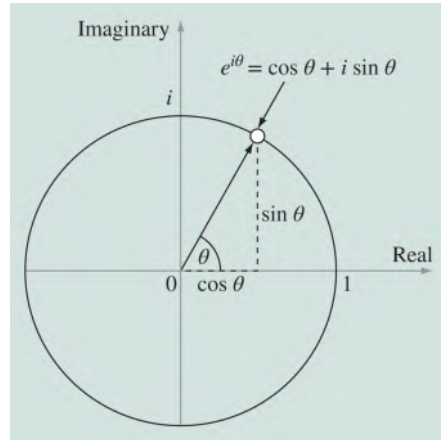
**FIGURE 16.4** The Fourier series approximation of a square wave. The series of plots shows the summation up to and including the (a) first, (b) second, and (c) third terms. The individual terms that were added or subtracted at each stage are also shown.



Before proceeding, the Fourier series can also be expressed in a more compact form using complex notation. This is based on *Euler's formula* ([Fig. 16.5](#)):

$$e^{\pm ix} = \cos x \pm i \sin x \quad (16.21)$$

**FIGURE 16.5** Graphical depiction of Euler's formula. The rotating vector is called a phasor.



where  $i = \sqrt{-1}$  and  $x$  is in radians. [Equation \(16.21\)](#) can be used to express the Fourier series concisely as (Chapra and Canale, 2021)

$$f(t) = \sum_{k=-\infty}^{\infty} \tilde{c}_k e^{ik\omega_0 t} \quad (16.22)$$

where the coefficients are

$$\tilde{c}_k = \frac{1}{T} \int_{-T/2}^{T/2} f(t) e^{-ik\omega_0 t} dt \quad (16.23)$$

Note that the tildes  $\sim$  are included to stress that the coefficients are complex numbers. Because it is more concise, we will primarily use the complex form in the rest of the chapter. Just remember that it is identical to the sinusoidal representation.

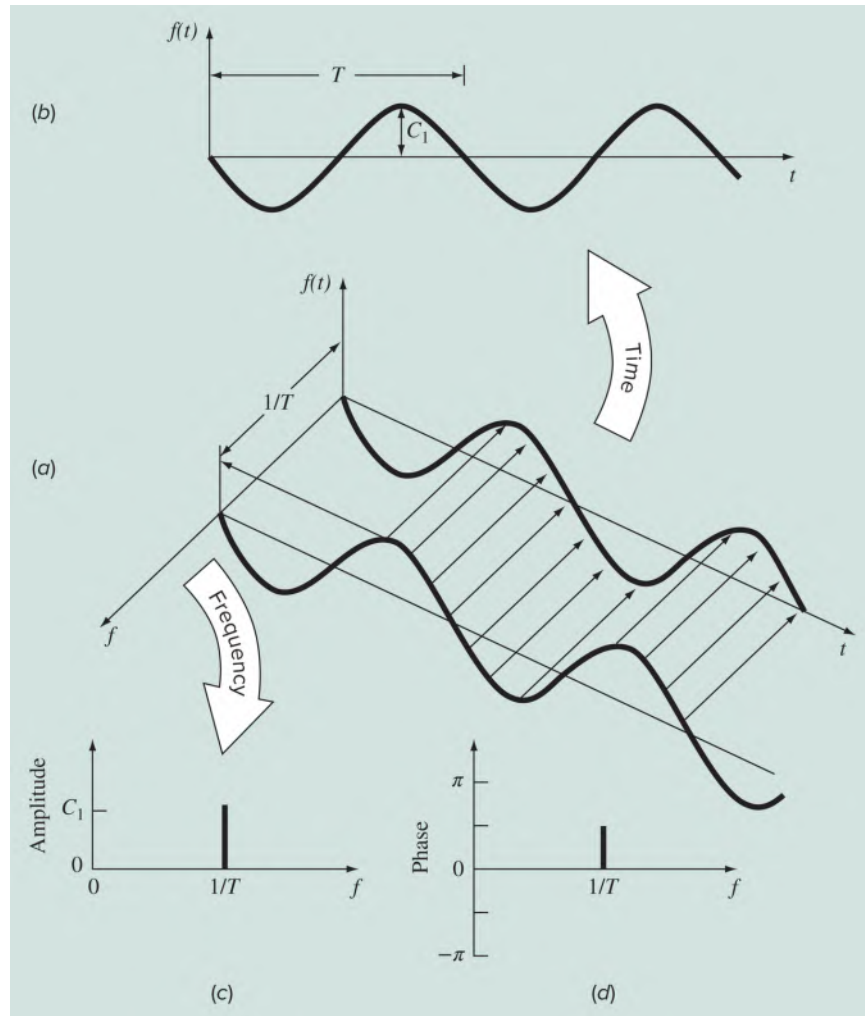
## 16.3 FREQUENCY AND TIME DOMAINS

To this point, our discussion of Fourier analysis has been limited to the *time domain*. We have done this because most of us are fairly comfortable conceptualizing a function's behavior in this dimension. Although it is not as familiar, the *frequency domain* provides an alternative perspective for characterizing the behavior of oscillating functions.

Just as amplitude can be plotted versus time, it can also be plotted versus frequency. Both types of expression are depicted in  **Fig. 16.6a**, where we have drawn a three-dimensional graph of a sinusoidal function:

$$f(t) = C_1 \cos\left(t + \frac{\pi}{2}\right)$$

**FIGURE 16.6** (a) A depiction of how a sinusoid can be portrayed in the time and the frequency domains. The time projection is reproduced in (b), whereas the amplitude-frequency projection is reproduced in (c). The phase-frequency projection is shown in (d).



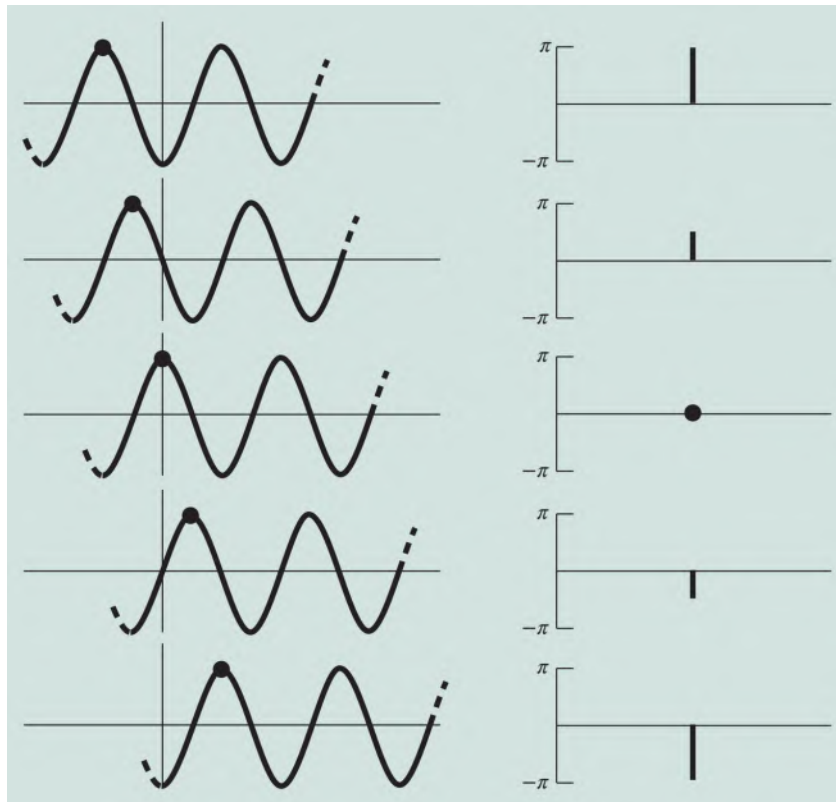
In this plot, the magnitude or amplitude of the curve  $f(t)$  is the dependent variable, and time  $t$  and frequency  $f = \omega_0/2\pi$  are the independent variables. Thus, the amplitude and the time axes form a *time plane*, and the amplitude and the frequency axes form a *frequency plane*. The sinusoid can, therefore, be conceived of as existing a distance  $1/T$  out along the frequency axis and running parallel to the time axes. Consequently, when we speak about the behavior of the sinusoid in the time domain, we mean the projection of the curve onto the time plane (Fig. 16.6b). Similarly, the behavior in the frequency domain is merely its projection onto the frequency plane.

As in Fig. 16.6c, this projection is a measure of the sinusoid's maximum positive amplitude  $C_1$ . The full peak-to-peak swing is unnecessary because of the symmetry. Together with the location  $1/T$  along the frequency axis, Fig. 16.6c now defines the amplitude and frequency of the sinusoid. This is enough information to reproduce the shape and size of the curve in the time domain. However, one more parameter—namely, the phase angle—is required to position the curve relative to  $t = 0$ . Consequently, a phase diagram, as shown in Fig. 16.6d, must also be included. The phase angle is determined as the distance (in radians) from zero to the point at which the positive peak occurs. If the peak occurs after



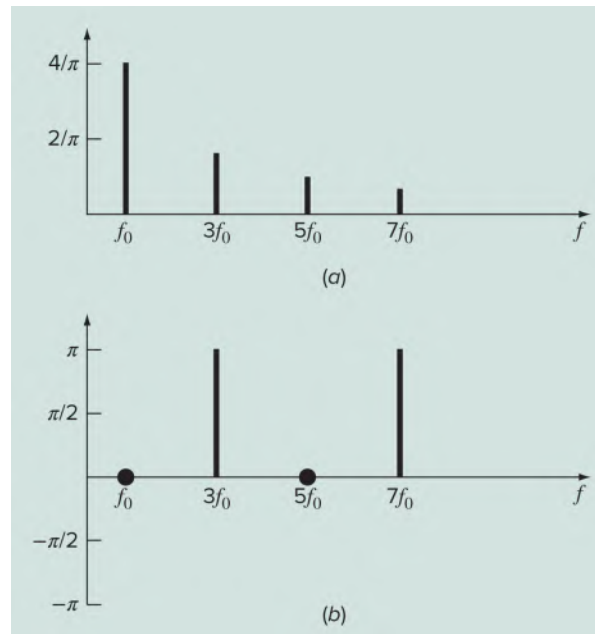
zero, it is said to be delayed (recall our discussion of lags and leads in [Sec. 16.1](#)), and by convention, the phase angle is given a negative sign. Conversely, a peak before zero is said to be advanced and the phase angle is positive. Thus, for [Fig. 16.6](#), the peak leads zero and the phase angle is plotted as  $+\pi/2$ . [Figure 16.7](#) depicts some other possibilities.

**FIGURE 16.7** Various phases of a sinusoid showing the associated phase line spectra.



We can now see that [Fig. 16.6c](#) and [d](#) provide an alternative way to present or summarize the pertinent features of the sinusoid in [Fig. 16.6a](#). They are referred to as *line spectra*. Admittedly, for a single sinusoid they are not very interesting. However, when applied to a more complicated situation—say, a Fourier series—their true power and value are revealed. For example, [Fig. 16.8](#) shows the amplitude and phase line spectra for the square-wave function from [Example 16.2](#).

**FIGURE 16.8** (a) Amplitude and (b) phase line spectra for the square wave from [Fig. 16.4](#).



Such spectra provide information that would not be apparent from the time domain. This can be seen by contrasting [Figs. 16.4](#) and [Fig. 16.8](#). [Figure 16.4](#) presents two alternative time domain perspectives. The first, the original square wave, tells us nothing about the sinusoids that comprise it. The alternative is to display these sinusoids—that is,  $(4/\pi) \cos(\omega_0 t)$ ,  $-(4/3\pi) \cos(3\omega_0 t)$ ,  $(4/5\pi) \cos(5\omega_0 t)$ , etc. This alternative does not provide an adequate visualization of the structure of these harmonics. In contrast, [Fig. 16.8a](#) and [Fig. 16.8b](#) provide a graphic display of this structure. As such, the line spectra represent “fingerprints” that can help us to characterize and understand a complicated waveform. They are particularly valuable for nonidealized cases where they sometimes allow us to discern structure in otherwise obscure signals. In the next section, we will describe the Fourier transform that will allow us to extend such analyses to nonperiodic waveforms.

## 16.4 FOURIER INTEGRAL AND TRANSFORM

Although the Fourier series is a useful tool for investigating periodic functions, there are many waveforms that do not repeat themselves regularly. For example, a lightning bolt occurs only once (or at least it will be a long time until it occurs again), but it will cause interference with receivers operating on a broad range of frequencies—for example, TVs, radios, and shortwave receivers. Such evidence suggests that a nonrecurring signal such as that produced by lightning exhibits a continuous frequency spectrum. Because such phenomena are of great interest to engineers, an alternative to the Fourier series would be valuable for analyzing these aperiodic waveforms.

The *Fourier integral* is the primary tool available for this purpose. It can be derived from the exponential form of the Fourier series [Eq. (16.22) and Eq. (16.23)]. The transition from a periodic to a nonperiodic function can be effected by allowing the period to approach infinity. In other words, as  $T$  becomes infinite, the function never repeats itself and thus becomes aperiodic. If this is allowed to occur, it can be demonstrated (e.g., Van Valkenburg, 1974; Hayt and Kemmerly, 1986) that the Fourier series reduces to

Page 386

$$f(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} F(\omega) e^{i\omega t} d\omega \quad (16.24)$$

and the coefficients become a continuous function of the frequency variable  $\omega$ , as in

$$F(\omega) = \int_{-\infty}^{\infty} f(t) e^{-i\omega t} dt \quad (16.25)$$

The function  $F(\omega)$ , as defined by Eq. (16.25), is called the *Fourier integral* of  $f(t)$ . In addition, Eqs. (16.24) and (16.25) are collectively referred to as the *Fourier transform pair*. Thus, along with being called the Fourier integral,  $F(\omega)$  is also called the *Fourier transform* of  $f(t)$ . In the same spirit,  $f(t)$ , as defined by Eq. (16.24), is referred to as the *inverse Fourier transform* of  $F(\omega)$ . Thus, the pair allows us to transform back and forth between the time and the frequency domains for an aperiodic signal.

The distinction between the Fourier series and transform should now be quite clear. The major difference is that each applies to a different class of functions—the series to periodic and the transform to nonperiodic waveforms. Beyond this major distinction, the two approaches differ in how they move between the time and the frequency domains. The Fourier series converts a continuous, periodic time-domain function to frequency-domain magnitudes at discrete frequencies. In contrast, the Fourier transform converts a continuous time-domain function to a continuous frequency-domain function. Thus,

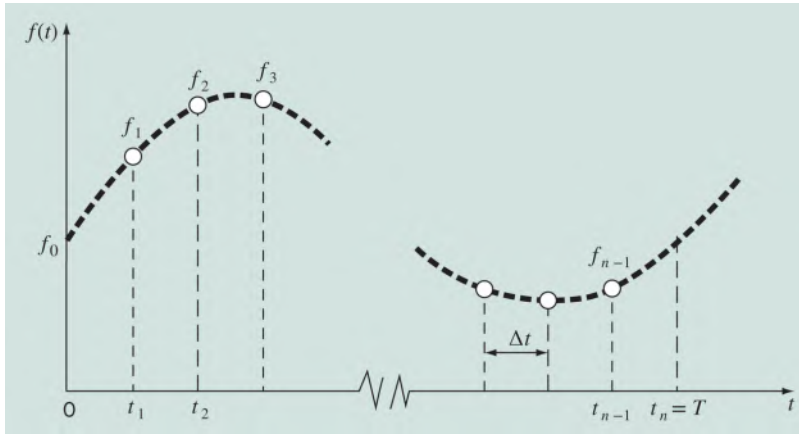
the discrete frequency spectrum generated by the Fourier series is analogous to a continuous frequency spectrum generated by the Fourier transform.

Now that we have introduced a way to analyze an aperiodic signal, we will take the final step in our development. In the next section, we will acknowledge the fact that a signal is rarely characterized as a continuous function of the sort needed to implement  **Eq. (16.25)**. Rather, the data are invariably in a discrete form. Thus, we will now show how to compute a Fourier transform for such discrete measurements.

## 16.5 DISCRETE FOURIER TRANSFORM (DFT)

In engineering, functions are often represented by a finite set of discrete values. Additionally, data are often collected in or converted to such a discrete format. As depicted in [Fig. 16.9](#), an interval from 0 to  $T$  can be divided into  $n$  equispaced subintervals with widths of  $\Delta t = T/n$ . The subscript  $j$  is employed to designate the discrete times at which samples are taken. Thus,  $f_j$  designates a value of the continuous function  $f(t)$  taken at  $t_j$ . Note that the data points are specified at  $j = 0, 1, 2, \dots, n - 1$ . A value is not included at  $j = n$ . (See Ramirez, 1985, for the rationale for excluding  $f_n$ .)

**FIGURE 16.9** The sampling points of the discrete Fourier series.



For the system in [Fig. 16.9](#), a discrete Fourier transform can be written as

$$F_k = \sum_{j=0}^{n-1} f_j e^{-ik\omega_0 j} \quad \text{for } k = 0 \text{ to } n - 1 \quad (16.26)$$

and the inverse Fourier transform as

$$f_j = \frac{1}{n} \sum_{k=0}^{n-1} F_k e^{-ik\omega_0 j} \quad \text{for } j = 0 \text{ to } n - 1 \quad (16.27)$$

where  $\omega_0 = 2\pi/n$ .

[Equations \(16.26\)](#) and [\(16.27\)](#) represent the discrete analogs of [Eqs. \(16.25\)](#) and [\(16.24\)](#), respectively. As such, they can be employed to compute both a direct and an inverse Fourier transform for

discrete data. Note that the factor  $1/n$  in [Eq. \(16.27\)](#) is merely a scale factor that can be included in either [Eq. \(16.26\)](#) or [Eq. \(16.27\)](#), but not both. For example, if it is shifted to [Eq. \(16.26\)](#), the first coefficient  $F_0$  (which is the analog of the constant  $a_0$ ) is equal to the arithmetic mean of the samples.

Before proceeding, several other aspects of the DFT bear mentioning. The highest frequency that can be measured in a signal, called the *Nyquist frequency*, is half the sampling frequency. Periodic variations that occur more rapidly than the shortest sampled time interval cannot be detected. The lowest frequency you can detect is the inverse of the total sample length.

As an example, suppose that you take 100 samples of data ( $n = 100$  samples) at a sample frequency of  $f_s = 1000$  Hz (i.e., 1000 samples per second). This means that the sample interval is

$$\Delta t = \frac{1}{f_s} = \frac{1}{1000 \text{ samples/s}} = 0.001 \text{ s/sample}$$

The total sample length is

$$t_n = \frac{n}{f_s} = \frac{100 \text{ sample}}{1000 \text{ samples/s}} = 0.1 \text{ s}$$

and the frequency increment is

$$\Delta f = \frac{f_s}{n} = \frac{1000 \text{ samples/s}}{1000 \text{ samples}} = 10 \text{ Hz}$$

The Nyquist frequency is

$$f_{\max} = 0.5 f_s = 0.5(1000 \text{ Hz}) = 500 \text{ Hz}$$

and the lowest detectable frequency is

$$f_{\min} = \frac{1}{0.1 \text{ s}} = 10 \text{ Hz}$$

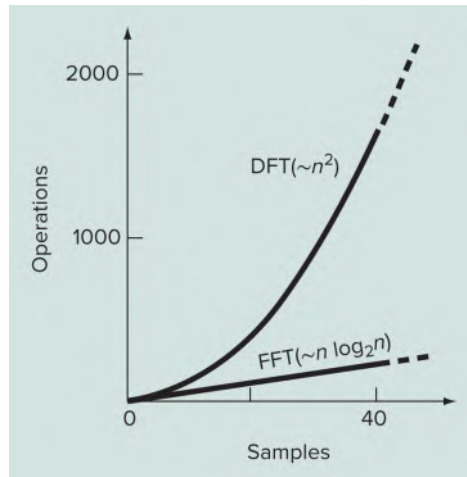
Thus, for this example, the DFT could detect signals with periods from  $1/500 = 0.002$  s up to  $1/10 = 0.1$  s.

## 16.5.1 Fast Fourier Transform (FFT)

Although an algorithm can be developed to compute the DFT based on [Eq. \(16.26\)](#), it is computationally burdensome because  $n^2$  operations are required. Consequently, for data samples of even moderate size, the direct determination of the DFT can be extremely time consuming.

The *fast Fourier transform*, or *FFT*, is an algorithm that has been developed to compute the DFT in an extremely economical fashion. Its speed stems from the fact that it utilizes the results of previous computations to reduce the number of operations. In particular, it exploits the periodicity and symmetry of trigonometric functions to compute the transform with approximately  $n \log_2 n$  operations ( [Fig. 16.10](#)). Thus, for  $n = 50$  samples, the FFT is about 10 times faster than the standard DFT. For  $n = 1000$ , it is about 100 times faster.

**FIGURE 16.10** Plot of number of operations versus sample size for the standard DFT and the FFT.



The first FFT algorithm was developed by Gauss in the early nineteenth century (Heideman et al., 1984). Major contributions were made by Runge, Danielson, Lanczos, and others in the early twentieth century. However, because discrete transforms often took days to weeks to calculate by hand, they did not attract broad interest prior to the development of the modern digital computer.

In 1965, J. W. Cooley and J. W. Tukey published a key paper in which they outlined an algorithm for calculating the FFT. This scheme, which is similar to those of Gauss and other earlier investigators, is called the Cooley-Tukey algorithm. Today, there are a host of other approaches that are offshoots of this method. As described next, Python's SciPy module offers a function called `fft` that employs such efficient algorithms to compute the DFT.

## 16.5.2 Python SciPy Function: `fft`

The SciPy module in Python provides an efficient way to compute a DFT. There is an `fft` submodule that provides a variety of functions for Fourier transforms. Here, we will focus on the 1-D discrete

Fourier transform function `fft`. The syntax for this is

```
from scipy.fft import fft
F = fft(f)
```

where  $F$  = an array containing the DFT and  $f$  = an array containing input samples. There are optional arguments, one being  $n$  = the length of the output  $F$ . If  $n$  is not specified, the length of the input is used for the output. If  $n$  is less than the length of  $f$ , the input is cropped accordingly. If  $n$  is greater than the length of  $f$ , the input is padded with zeros.

Note that the elements of  $F$  are sequenced in what is called *reverse-wraparound order*. The first half of the values are the positive frequencies (starting with a constant), and the second half are the negative frequencies. Thus, if  $n = 8$ , the order is  $[0, 1, 2, 3, -4, -3, -2, -1]$ . The following example illustrates the function's use to calculate the DFT of a simple sinusoid.

## EXAMPLE 16.3

### Compute the DFT of a Simple Sinusoid with Python

#### Problem Statement

Apply the Python `fft` function to determine the discrete Fourier transform for a simple sinusoid:

$$f(t) = 5 + \cos(2\pi 12.5t) + \sin(2\pi 18.75t)$$

Generate eight equispaced points with  $\Delta t = 0.02$  s. Plot the result versus frequency.

Page 389

#### Solution

Before generating the DFT, we can compute several relevant quantities. The sampling frequency is

$$f_s = \frac{1}{\Delta t} = \frac{1}{0.02 \text{ s}} = 50 \text{ Hz}$$

The total sample length is

$$t_n = \frac{n}{f_s} = \frac{8}{50} = 0.16 \text{ s}$$

The Nyquist sampling frequency is

$$f_{\max} = \frac{f_s}{2} = 25 \text{ Hz}$$



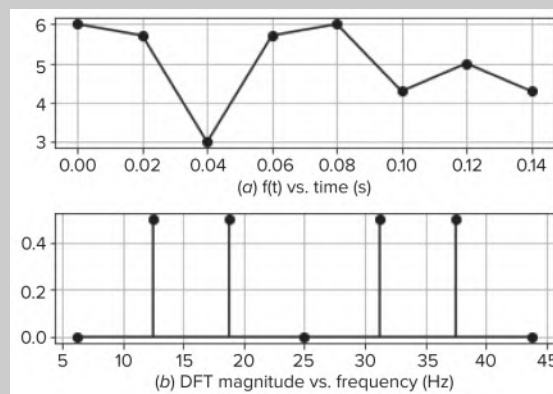
The lowest detectable frequency is

$$f_{\min} = \frac{1}{t_n} = \frac{1}{0.16 \text{ s}} = 6.25 \text{ Hz}$$

Thus, the analysis can detect signals with periods from  $1/25 = 0.04 \text{ s}$  up to  $1/6.25 = 0.16 \text{ s}$ . So, we should be able to resolve both the 12.5 and 18.75 Hz signals.

The following Python script is used to generate and plot the time-domain sample ([Fig. 16.11a](#)). Note: We choose to use the Matplotlib module here instead of Pylab in order to provide multiple subplots in the same frame.

**FIGURE 16.11** Results of computing the DFT with the Python SciPy `fft` function: (a) the time domain sample and (b) the magnitude of the DFT versus frequency.



```
import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft

# compute the time series
n = 8 ; dt = 0.02 ; fs = 1/dt ; T = 0.16
tspan = np.arange(0,n)/fs
y = 5 + np.cos(2*np.pi*12.5*tspan) + np.sin(2*np.pi*18.75*tspan)

# create a frame for 2 subplots
# and plot the time samples
# as the first subplot
fig=plt.figure()
ax1 = fig.add_subplot(211)
fig.subplots_adjust(hspace=0.5)
ax1.plot(tspan,y,c='k',marker='o')
ax1.grid()
ax1.set_xlabel('a) f(t) vs. time (s)')
```

As was mentioned at the beginning of [Sec. 16.5](#), notice that `tspan` omits the last point.

The `fft` function can be used to compute the DFT and display the results.

```
# compute and print the DFT
np.set_printoptions(precision=3,suppress=True)
Y = fft(y)/n
print(Y)

[ 5.-0.j 0.+0.j 0.5-0.j -0.-0.5j 0.-0.j -0.+0.5j 0.5+0.j 0.-0.j ]
```

The results can be summarized in the following table:

| Index | Frequency | Period   | Real | Imaginary |
|-------|-----------|----------|------|-----------|
| 0     | 0         | constant | 5    | 0         |
| 1     | 6.25      | 0.16     | 0    | 0         |
| 2     | 12.5      | 0.08     | 0.5  | 0         |
| 3     | 18.75     | 0.0533   | 0    | -0.5      |
| 4     | 25        | 0.04     | 0    | 0         |
| 5     | 31.25     | 0.032    | 0    | 0.5       |
| 6     | 37.5      | 0.0267   | 0.5  | 0         |
| 7     | 43.75     | 0.0229   | 0    | 0         |

Notice that the `fft` has detected the 12.5 Hz and 18.75 Hz signals. In addition, we have highlighted the Nyquist frequency to indicate that the frequencies greater than it in the table are redundant. That is, they are merely reflections of the results less than the Nyquist frequency.

We add a plot of the magnitude of the DFT values versus frequency using the `stem` plot format from the Matplotlib module,

```
# plot the magnitude of the DFT
# as the second subplot
Ymag = abs(Y[1:])
freq = np.arange(1,n)/T
ax2 = fig.add_subplot(212)
ax2.stem(freq,Ymag,basefmt='k-',linefmt='k-',markerfmt='ko',
         use_line_collection=True)
ax2.set_xlabel('b) DFT magnitude vs. frequency (Hz)')
ax2.grid()
```

The reflection across the Nyquist frequency is evident in the plot ([Fig. 16.11b](#)). For this reason, we often plot just the first half of the DFT.

## 16.6 THE POWER SPECTRUM

Beyond the complex DFT, the magnitude (amplitude), and phase spectra, another common way to depict the underlying frequency content is the power spectrum. As its name implies, it derives from the analysis of power output of electrical systems, where power is generally proportional to voltage or current squared. In terms of the DFT, a power spectrum consists of a plot of the power versus the frequency. For aperiodic signals, this is a continuous curve; whereas, for periodic signals, as in [Example 16.3](#), it is discrete. The power can be computed as the sum (or integral) of the squares of the Fourier coefficients:

$$P_k = |\tilde{c}_k|^2 \quad \text{or} \quad P(\omega) = |F(\omega)|^2$$

where  $P_k$  = the power associated with the discrete frequency,  $k\omega_0$ , and  $P(\omega)$  = the power associated with the continuous frequency,  $\omega$ , in the case of the Fourier integral transform.

### EXAMPLE 16.4

#### Computing the Power Spectrum with Python

##### Problem Statement

Compute the spectrum for the sinusoidal function for which the DFT was computed in

[Example 16.3](#).

Page 391

##### Solution

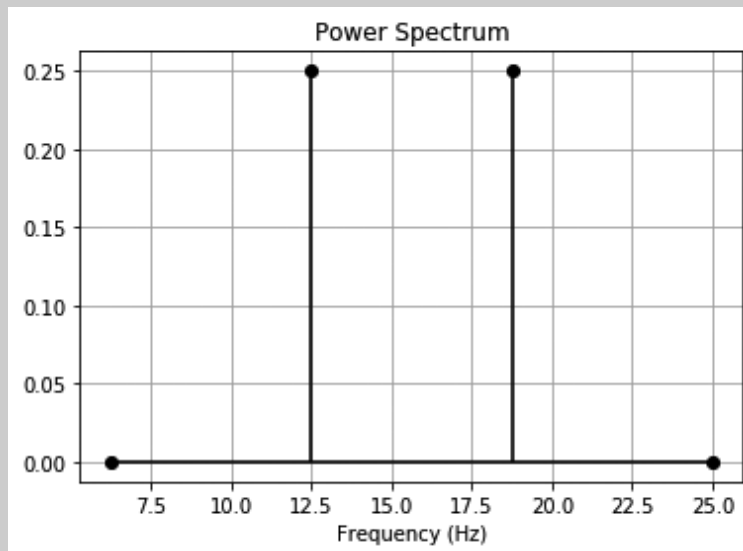
The following Python script computes and displays the power spectrum:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft

# compute the DFT
n = 8 ; dt = 0.02 ; fs = 1/dt ; T = 0.16
tspan = np.arange(0,n)/fs
y = 5 + np.cos(2*np.pi*12.5*tspan) + np.sin(2*np.pi*18.75*tspan)
Y = fft(y)/n
freq = np.arange(1,n)/T
# compute and plot the power spectrum
nyquist = fs/2
n2 = int(n/2)
fP = freq[0:n2]
Pyy = abs(Y[1:n2+1])**2
fig=plt.figure()
plt.stem(fP,Pyy,basefmt='k-',linefmt='k-',markerfmt='ko',
         use_line_collection=True)
plt.xlabel('Frequency (Hz)')
plt.title('Power Spectrum')
plt.grid()
```

As indicated, the first section repeats the computation of the DFT with pertinent statements from [Example 16.3](#). The second section then computes and plots the power spectrum. The power spectrum is computed for only the first half of the DFT, since the second half is the reflection. As shown in [Fig. 16.12](#), the resulting graph indicates that peaks occur at both 12.5 Hz and 18.75 Hz and at no other frequencies, as expected.

**FIGURE 16.12** Power spectrum for a sinusoidal function with frequencies 12.5 Hz and 18.75 Hz.

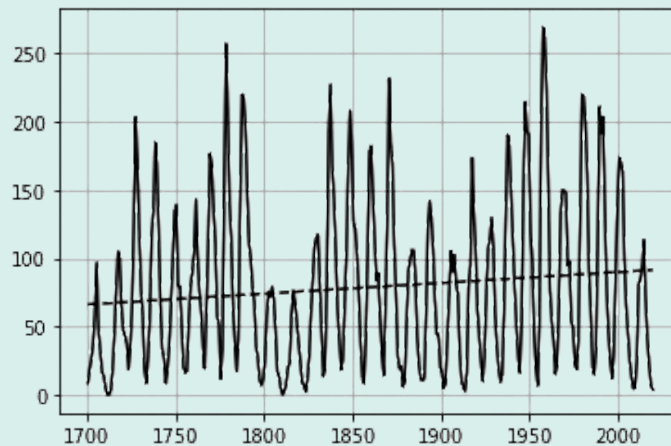


## SUNSPOTS

In 1848, Johann Rudolph Wolf devised a method for quantifying solar activity by counting the number of individual dark spots or groups of spots on the sun's surface. He computed a quantity, now called the Wolf sunspot number,<sup>3</sup> by adding ten times the number of groups plus the total count of individual spots in a year. As shown in  Fig. 16.13, the data set for annual counts extends back to 1700. Based on the early records, Wolf determined the cycle's length to be 11.1 years. Use Fourier analysis to confirm this result by applying an FFT to the data.

Page 392

**FIGURE 16.13** Wolf sunspot number versus year from 1700 through 2019. The dashed line is a straight-line fit of the data and reveals an upward trend.



## Solution

The data are available in .csv format for download from  <http://www.sidc.be/silso/datafiles>, as filename `SN_y_tot_V2.0.csv`. First, we can write a Python script to load the data from the file, extract the series, and create the plot displayed in  Fig. 16.13. We have also included here a straight-line fit through the data that reveals a slight upward trend in the data. This is an indication that the data are not *stationary* (with a constant mean).

Consequently, we transform the data by subtracting away the straight-line value.

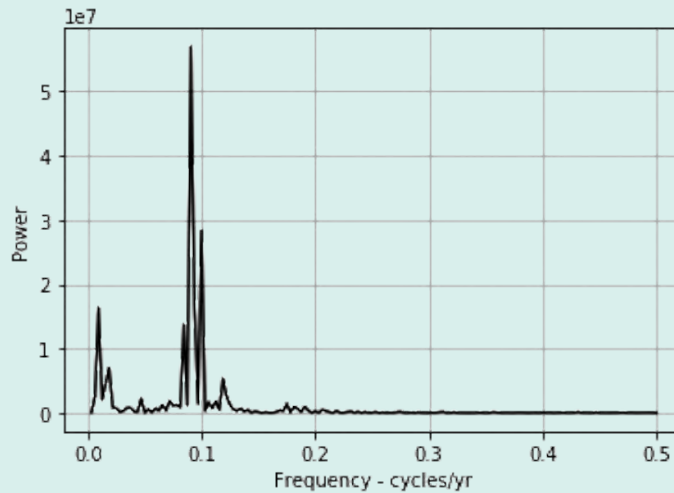
```
import numpy as np
import pylab

yr,numspots,sd,n1,n1 =
np.loadtxt(fname='SN_y_tot_V2.0.csv',delimiter=';',unpack=True)
pylab.plot(yr,numspots,c='k')
pylab.grid()

coef = np.polyfit(yr,numspots,1)
pylab.plot(yr,np.polyval(coef,yr),c='k',ls='--')
```

Next, we can apply the `fft` function to the stationary data series, compute and plot the power spectrum, as shown in [Fig. 16.14](#).

**FIGURE 16.14** Power spectrum for the Wolf sunspot number versus year, 1700–2019.



```
y = numspots - np.polyval(coef,yr)
pylab.figure()
pylab.plot(yr,y,c='k')
pylab.grid()
pylab.xlabel('Year')
pylab.ylabel('Sunspot Number')

from scipy.fft import fft
Y = fft(y)
fs = 1 # 1/yr
n = len(yr)
f = np.arange(1,n)*fs/n
n2 = int(n/2)
f2 = f[0:n2]
Y2 = Y[1:n2+1]
Pyy = abs(Y2)**2
pylab.figure()
pylab.plot(f2,Pyy,c='k')
pylab.grid()
pylab.xlabel('Frequency - cycles/yr')
pylab.ylabel('Power')
```

There is a maximum in the frequency content just below 0.1 cycles/yr. We can find this more precisely with the script and output:

```
pmax = np.max(Pyy)
for i in range(n2):
    if Pyy[i] >= pmax:
        imax = i
        fmax = f2[i]
        break

print('Frequency at max power = ',fmax,' 1/yr')
print('Period at max power = ',1/fmax,' years')

Frequency at max power = 0.090625 1/yr
Period at max power = 11.03448275862069 years
```

and we see that our period of 11.03 years is close to the value of 11.1 years observed by Wolf over 170 years ago. This is an excellent example of the use of the Fourier transform to detect and estimate a fundamental sinusoidal frequency in a data series.

# PROBLEMS

**16.1** The following equation describes the variations of temperature in °C with time in days in a tropical lake:

$$T(t) = 12.8 + 4 \cos\left(\frac{2\pi}{365}t\right) + 3 \sin\left(\frac{2\pi}{365}t\right)$$

What is (a) the mean temperature, (b) the amplitude, and (c) the period?

**16.2** The temperature in a pond varies sinusoidally over the course of a year. Use linear regression to fit [Eq. \(16.11\)](#) to the following data. Use your fitted equation to determine the mean, amplitude, and the day and value of the maximum temperature. Since this is an annual cycle, the period is 365 days. To confirm your fit visually, create a plot of the model line and the data.

|                            |     |     |     |      |     |      |      |      |      |     |     |
|----------------------------|-----|-----|-----|------|-----|------|------|------|------|-----|-----|
| $t,$<br>$d$                | 15  | 45  | 75  | 105  | 135 | 165  | 225  | 255  | 285  | 315 | 345 |
| $T,$<br>$^{\circ}\text{C}$ | 3.4 | 4.7 | 8.5 | 11.7 | 16  | 18.7 | 19.7 | 17.1 | 12.7 | 7.7 | 5.1 |

**16.3** The pH in a waste stream varies sinusoidally over the course of a day. Regress the data in the table to fit [Eq. \(16.11\)](#). Use your model to determine the mean, amplitude, and time and value of the maximum pH. Note that the period is 24 hours.

|          |     |     |     |     |     |     |
|----------|-----|-----|-----|-----|-----|-----|
| Time, hr | 0   | 2   | 4   | 5   | 7   | 9   |
| pH       | 7.6 | 7.2 | 7   | 6.5 | 7.5 | 7.2 |
| Time, hr | 12  | 15  | 20  | 22  | 24  |     |
| pH       | 8.9 | 9.1 | 8.9 | 7.9 | 7   |     |

**16.4** The solar radiation for Tucson, Arizona, has been tabulated as

|                             |     |     |     |     |     |     |
|-----------------------------|-----|-----|-----|-----|-----|-----|
| Time, mo                    | J   | F   | M   | A   | M   | J   |
| Radiation, W/m <sup>2</sup> | 144 | 188 | 245 | 311 | 351 | 359 |
| Time, mo                    | J   | A   | S   | O   | N   | D   |
| Radiation, W/m <sup>2</sup> | 308 | 287 | 260 | 211 | 159 | 131 |



Assuming each month is 30 days long, fit a sinusoid to these data. Consider each data point to be representative of a mid-month time. Use the resulting model to predict the radiation at the end of August. Comment on model adequacy.

**16.5** The following time series data were generated by summing several sinusoids with a small amount of noise added to the summation. Determine the frequencies (in Hz) of these sinusoids and whether they were sines or cosines.

| $f(t)$ | $t$    | $f(t)$ | $t$    | $f(t)$ | $t$    | $f(t)$ | $t$    |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 0.00   | 21.497 | 0.08   | 10.002 | 0.16   | 2.499  | 0.24   | 13.999 |
| 0.01   | 11.887 | 0.09   | 10.223 | 0.17   | 12.120 | 0.25   | 13.774 |
| 0.02   | 16.405 | 0.10   | 22.630 | 0.18   | 7.601  | 0.26   | 1.374  |
| 0.03   | 10.826 | 0.11   | 10.655 | 0.19   | 13.178 | 0.27   | 13.337 |
| 0.04   | 3.865  | 0.12   | 17.299 | 0.20   | 20.132 | 0.28   | 6.692  |
| 0.05   | 15.567 | 0.13   | 9.853  | 0.21   | 8.439  | 0.29   | 14.148 |
| 0.06   | 5.075  | 0.14   | 9.128  | 0.22   | 18.929 | 0.30   | 14.865 |
| 0.07   | 12.998 | 0.15   | 16.045 | 0.23   | 11.000 | 0.31   | 7.961  |

**16.6** In electrical circuits, it is common to see current behavior in the form of a rectangular wave as shown in  **Fig. P16.6**. Note that this wave differs from the one described in  **Example 16.2**. From the functional description

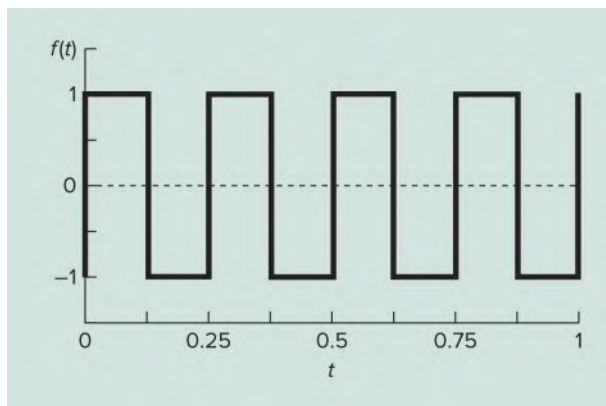
$$f(t) = \begin{cases} A_0 & 0 \leq t \leq T/2 \\ -A_0 & T/2 \leq t \leq T \end{cases}$$

the Fourier series can be represented as

$$f(t) = \sum_{n=1}^{\infty} \left( \frac{4A_0}{(2n-1)\pi} \right) \sin\left( \frac{2\pi(2n-1)t}{T} \right)$$

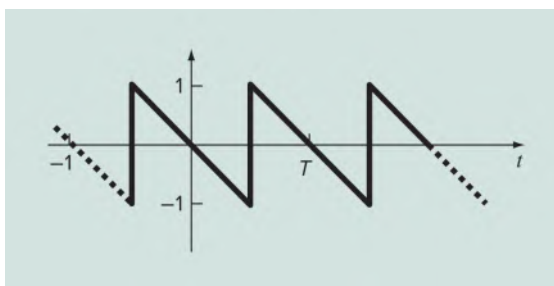
Develop a Python script to generate and plot the first six terms of the series. Also, plot the sum of these terms. Do this for the range  $0 \leq t \leq 4T$ . Use dotted black lines for the individual terms and a solid black line for the sum. As shown in the figure,  $A_0 = 1$  and  $T = 0.25$ .

**FIGURE P16.6**



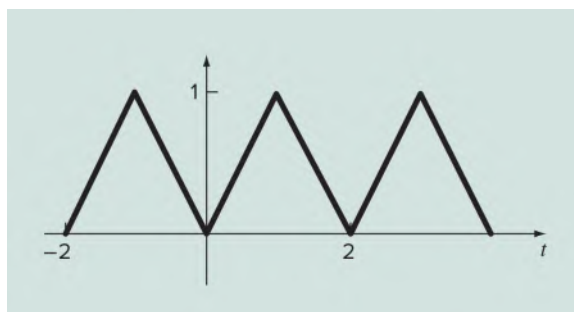
**16.7** Use a continuous Fourier series to approximate the sawtooth wave in [Fig. P16.7](#). Plot the first four terms of the series individually along with the summation. Create the plot from  $-T$  to  $T$  and use  $T = 1$ . Create another plot of the amplitude of the harmonics versus the term number.

**FIGURE P16.7** A sawtooth wave.



**16.8** Use a continuous Fourier series to approximate the triangular waveform in [Fig. P16.8](#). Plot the first four terms of the series individually along with the summation. Create the plot from  $-T$  to  $T$  and use  $T = 2$ . Create another plot of the amplitude of the harmonics versus the term number.

**FIGURE P16.8** A triangular wave.



**16.9** Use the Maclaurin series expansions for  $e^x$ ,  $\cos(x)$ , and  $\sin(x)$  to prove Euler's formula, [Eq. \(16.21\)](#).

**16.10** A half-wave rectifier can be characterized by

$$C_1 = \frac{1}{\pi} + \frac{1}{2}\sin(t) - \frac{2}{3\pi}\cos(2t) - \frac{2}{15\pi}\cos(4t) - \frac{2}{35\pi}\cos(6t) - \dots$$

where  $C_1$  describes the wave. Plot the first four terms involving time along with the summation. Pick a reasonable time range for the plot so that the waveform is well displayed.

**16.11** Rework Example 16.3, but for 64 points sampled at a rate of  $\Delta t = 0.01$  s and based on the function

$$f(t) = \cos(2\pi 12.5t) + \cos(2\pi 25t)$$

Use the Python SciPy `fft` function to generate a DFT of these values and plot the results as a power spectrum.

**16.12** Create a Python script to generate 64 points of the function

$$f(t) = \cos(10t) + \sin(3t)$$

from  $t = 0$  to  $2\pi$ . Add a random component to the signal with the signal using the Numpy `random.normal` function with a mean of zero and a standard deviation = 0.25. Use the SciPy `fft` function to produce a DFT of the signal values and plot the results in the form of a power spectrum. Is the DFT able to detect the two fundamental frequencies?

**16.13** Create a Python script to generate 32 points for the sinusoid depicted in [Fig. 16.2](#) from  $t = 0$  to 6 s. Compare the DFT and create three subplots of (a) the original signal, (b) the real part, and (c) the imaginary part of the DFT versus frequency.

Page 395

**16.14** Use the `fft` function to compute a DFT for the sawtooth wave from [Prob. 16.7](#). Sample the wave from  $t = 0$  to  $4T$  using 128 sample points. Create a plot of the power spectrum.

**16.15** Develop a Python function that generates a power spectrum plot using the `fft` function and given a function name, period, and number of sample points as arguments. The leading statement of the function definition should be

```
def powerspec(f,T,n):
```

Use your function in the solution of [Prob. 16.11](#).

**16.16** Use the `fft` function from the Python SciPy module to compute the DFT for the following function:

$$f(t) = 1.5 + 1.8 \cos(2\pi 12t) + 0.8 \sin(2\pi 20t) - 1.25 \cos(2\pi 28t)$$

Take 64 samples with a frequency of 128 samples/s. Have your script compute and display values of  $\Delta t$ ,  $t_n$ ,  $\Delta f$ ,  $f_{\min}$ , and  $f_{\max}$ . As illustrated in  **Examples 16.3** and  **16.4**, have your script generate plots as in  **Figs. 16.11** and  **16.12**.

**16.17** If you take 128 samples of data ( $n = 128$ ) with a total sample length,  $t_n = 0.4$  s, compute the following: (a) the sample frequency,  $f_s$ , samples/s, (b) the sample interval,  $\Delta t$ , s/sample, (c) the Nyquist frequency,  $f_{\max}$  in Hz, and (d) the minimum frequency,  $f_{\min}$  in Hz.

**16.18** Discovering and analyzing periodic behavior in nature is an important endeavor, and Fourier analysis provides a window through which to view data. Data are available, from various sources, for example,  <https://www.nhc.noaa.gov/climo/images/AtlanticStormTotalsTable.pdf>, from 1851 forward on the number of named hurricanes in the Atlantic Basin per year. Apply the Fourier transform to reveal any significant cyclic frequencies. Compensate for any long-term linear trends in the data, as illustrated in  **Case Study 16.7**.

# Polynomial Interpolation

## CHAPTER OBJECTIVES

The primary objective of this chapter is to introduce you to polynomial interpolation. Specific objectives and topics covered are

- Recognizing that evaluating polynomial coefficients with simultaneous equations is an ill-conditioned problem.
- Knowing how to evaluate polynomial coefficients and interpolate with the `polyfit` and `polyval` functions from Python's Numpy module.
- Knowing how to perform an interpolation with Newton's polynomial.
- Being able to perform an interpolation with a Lagrange polynomial.
- Knowing how to solve an inverse interpolation problem by recasting it into a roots problem.
- Appreciating the dangers of extrapolation.
- Recognizing that higher-order polynomials can exhibit large oscillations that may affect interpolations adversely.

## YOU'VE GOT A PROBLEM

If we want to improve the velocity prediction for the free-falling bungee jumper, we might expand our model to account for other factors beyond mass and the drag coefficient. As was previously mentioned in [Sec. 1.4](#), the drag coefficient can itself be formulated as a function of other factors such as the area of the jumper and characteristics such as the air's density and viscosity.

Air density and viscosity are commonly presented in tabular form as a function of temperature. For example, [Table 17.1](#) is reprinted from a popular fluid mechanics textbook (White, 2015).

**TABLE 17.1** Density ( $\rho$ ) and viscosity ( $\mu$ ) of air for a wide range of temperatures ( $T$ ), as reported by White (2015).

| $T$ (°C) | $\rho$ (kg/m <sup>3</sup> ) | $\mu$ (Pa · s) $\times 10^5$ |
|----------|-----------------------------|------------------------------|
| -40      | 1.52                        | 1.51                         |

| $T (^{\circ}\text{C})$ | $\rho (\text{kg}/\text{m}^3)$ | $\mu (\text{Pa} \cdot \text{s}) \times 10^5$ |
|------------------------|-------------------------------|----------------------------------------------|
| 0                      | 1.29                          | 1.71                                         |
| 20                     | 1.20                          | 1.80                                         |
| 50                     | 1.09                          | 1.95                                         |
| 100                    | 0.95                          | 2.17                                         |
| 150                    | 0.84                          | 2.38                                         |
| 200                    | 0.75                          | 2.57                                         |
| 250                    | 0.68                          | 2.75                                         |
| 300                    | 0.62                          | 2.93                                         |
| 400                    | 0.53                          | 3.25                                         |
| 500                    | 0.46                          | 3.55                                         |

Suppose that you desired the density at a temperature not included in the table. In such a case, you would have to interpolate. That is, you would have to estimate the value at the desired temperature based on the densities that bracket it. The simplest approach is to determine the equation for the straight line connecting the two adjacent values and use this equation to estimate the density at the desired intermediate temperature. Although such *linear interpolation* is perfectly adequate in many cases, error can be introduced when the data exhibit significant curvature. In this chapter, we will explore a number of different approaches for obtaining adequate estimates for such situations.

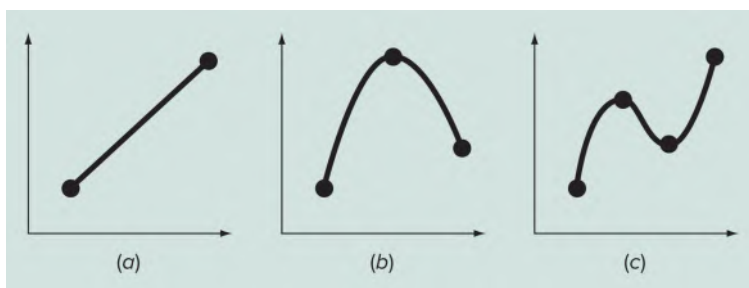
# 17.1 INTRODUCTION TO INTERPOLATION

You will frequently have occasion to estimate intermediate values of a property between precise data points. We have illustrated in [Chaps. 14](#) and [15](#) one approach, regression, which is to fit a model to the data and use that model to provide the estimates. Another common method used for this purpose is *polynomial interpolation*. A general formula for an  $(n - 1)$ th-order polynomial is

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1} \quad (17.1)$$

For  $n$  data points, there is only one polynomial of order  $(n - 1)$  that passes through all the points. For example, there is only one straight line, that is, a first-order polynomial, that connects two points ([Fig. 17.1a](#)). Similarly, only one parabola connects a set of three points ([Fig. 17.1b](#)). Polynomial interpolation consists of determining the unique  $(n - 1)$ th-order polynomial that fits  $n$  data points. This polynomial then provides a formula to compute intermediate values.

**FIGURE 17.1** Examples of interpolating polynomials: (a) first-order (linear) connecting two points, (b) second-order (quadratic or parabolic) connecting three points, and (c) third-order (cubic) connecting four points.



Before proceeding, we should note that Python NumPy's polynomial functions represent coefficients using decreasing powers of  $x$ , as in

$$f(x) = p_0x^{n-1} + p_1x^{n-2} + \cdots + p_{n-2}x + p_{n-1} \quad (17.2)$$

To be consistent with Python, we will adopt the scheme of [Eq. \(17.2\)](#) in the following section.

## 17.1.1 Determining Polynomial Coefficients

A straightforward way for computing the coefficients of [Eq. \(17.2\)](#) is based on the fact that  $n$  data points are required to determine the  $n$  coefficients. As shown in the following example, this allows us to generate  $n$  linear algebraic equations that we can solve simultaneously for the coefficients.

## EXAMPLE 17.1

### Determining Polynomial Coefficients with Simultaneous Equations

#### Problem Statement

Suppose that we want to determine the coefficients of the parabola,  $f(x) = p_0x^2 + p_1x + p_2$ , that passes through the last three density values from [Table 17.1](#):

$$\begin{array}{ll} x_1 = 300 & f(x_1) = 0.616 \\ x_2 = 400 & f(x_2) = 0.525 \\ x_3 = 500 & f(x_3) = 0.457 \end{array}$$

#### Solution

Each of these pairs can be substituted into [Eq. \(17.2\)](#) to yield a system of three equations:

$$\begin{array}{l} 0.616 = p_0 300^2 + p_1 300 + p_2 \\ 0.525 = p_0 400^2 + p_1 400 + p_2 \\ 0.457 = p_0 500^2 + p_1 500 + p_2 \end{array}$$

or, in vector/matrix form

$$\begin{bmatrix} 90000 & 300 & 1 \\ 160000 & 400 & 1 \\ 250000 & 500 & 1 \end{bmatrix} \begin{bmatrix} p_0 \\ p_1 \\ p_2 \end{bmatrix} = \begin{bmatrix} 0.616 \\ 0.525 \\ 0.457 \end{bmatrix}$$

Thus, the problem reduces to solving three simultaneous linear algebraic equations for the three unknown coefficients. A simple Python script can be used to obtain the coefficient values:

```
import numpy as np
A = np.matrix('90000., 300., 1.; 160000., 400., 1.; 250000., 500., 1.')
b = np.matrix('0.616 ; 0.525 ; 0.457 ')
p = np.linalg.solve(A,b)
print(p)
```

with the result



```
[[ 1.150e-06]
 [-1.715e-03]
 [ 1.027e+00]]
```

Consequently, the parabola that passes exactly through the three points is

$$f(x) = (1.150 \times 10^{-6})x^2 - (1.715 \times 10^{-3})x + 1.027$$

This polynomial then provides a means to estimate intermediate values. For example, the density at 350 °C can be estimated as

$$f(350) = (1.150 \times 10^{-6})350^2 - (1.715 \times 10^{-3})350 + 1.027 \cong 0.568$$

Although the approach in [Example 17.1](#) provides an easy way to perform interpolation, it has a serious deficiency. To understand this flaw, notice the coefficient matrix in the example has a decided structure. This can be seen clearly by expressing it in general terms:

$$\begin{bmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ x_3^2 & x_3 & 1 \end{bmatrix} \begin{bmatrix} p_0 \\ p_1 \\ p_2 \end{bmatrix} = \begin{bmatrix} f(x_1) \\ f(x_2) \\ f(x_3) \end{bmatrix} \quad (17.3)$$

Coefficient matrices of this form are referred to as *Vandermonde matrices*. Such matrices are very ill-conditioned. That is, their solutions are subject to round-off errors. This can be illustrated by using Python to compute the condition number for the coefficient matrix in [Example 17.1](#):

```
print('{0:7.5g}'.format(np.linalg.cond(A)))
5.8932e+06
```

As per our discussion in [Sec. 11.2.2](#), this condition number, which is large for a  $3 \times 3$  matrix, implies that rounding errors in the solution may be on the order of  $10^{6-t}$ , where  $t$  is the precision of the coefficients. We encountered this problem in another setting in Chap. 15 with polynomial regression and provided a scaling method to combat it. There, we were also dealing with the issue of collinearity in regression computations. Using the scaling method, we would define a new variable, such as

$$z \equiv \frac{x - 400}{100}$$

and our coefficient matrix for [Example 17.1](#) would then be

$$\begin{bmatrix} 1 & -1 & 1 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix} \text{ with a condition number equal to } 3.26.$$

We could solve the problem in this way, but, then, in order to use our interpolating polynomial, we would first have to convert the temperature to the  $z$  form shown above. Also, it is worth noting that our original ill-conditioning situation gets worse as the order of the interpolating polynomial increases.

Rather than approaching this predicament using scaling, there are alternative methods that do not manifest this shortcoming. In this chapter, we will describe two techniques that are well-suited for computer implementation: the Newton and Lagrange polynomials. Before doing this, we will review briefly how the coefficients of the interpolating polynomial can be estimated directly with Python's built-in functions.

## 17.1.2 Python NumPy Functions: `polyfit` and `polyval`

Recalling from [Sec. 14.5.2](#), the `polyfit` function can be used to perform polynomial regression. In such applications, the number of data points is greater than the number of coefficients being estimated. Consequently, the least-squares fit line does not necessarily pass through any of the points, but rather follows the general trend of the data.

For the case where the number of data points equals the number of coefficients, `polyfit` will determine the coefficients of the interpolating polynomial, that is, the polynomial that passes directly through the data points. For example, `polyfit` can be used to determine the coefficients of the parabola that passes through the last three density values from [Table 17.1](#):

```
T = np.array([300., 400., 500.])
rho = np.array([0.616, 0.525, 0.457])
coef = np.polyfit(T,rho,2)
print(coef)

[ 1.150e-06 -1.715e-03 1.027e+00]
```

We can then use the `polyval` function to perform an interpolation as in

```
dens = np.polyval(coef,350.)
print('{0:7.5g}'.format(dens))

0.56762
```

These results agree with those obtained in [Example 17.1](#), using simultaneous linear equations.

# 17.2 NEWTON INTERPOLATING POLYNOMIAL

There are a variety of alternative forms for expressing an interpolating polynomial beyond the familiar format of [Eq. \(17.2\)](#). Newton's interpolating polynomial is among the most popular and useful forms. Before presenting the general equation, we will introduce the first- and second-order versions because of their simple visual interpretation.

Page 400

## 17.2.1 Linear Interpolation

The simplest form of interpolation is to connect two data points with a straight line. This technique, called *linear interpolation*, is depicted graphically in [Fig. 17.2](#). Using similar triangles,

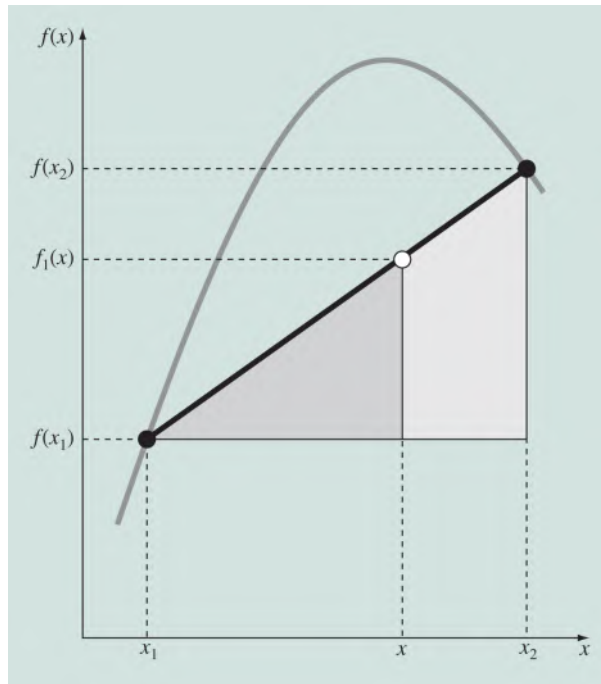
$$\frac{f_1(x) - f(x_1)}{x - x_1} = \frac{f(x_2) - f(x_1)}{x_2 - x_1} \quad (17.4)$$

which can be rearranged to yield

$$f_1(x) = f(x_1) + \frac{f(x_2) - f(x_1)}{x_2 - x_1}(x - x_1) \quad (17.5)$$

which is the *Newton linear-interpolation formula*. The notation  $f_1(x)$  designates that this is a first-order interpolating polynomial. Notice that besides representing the slope of the line connecting the points, the term  $[f(x_2) - f(x_1)]/(x_2 - x_1)$  is a finite-difference approximation of the first derivative [recall [Eq. \(4.20\)](#)]. In general, the smaller the interval between the data points, the better the approximation. This is due to the fact that, as the interval decreases, a continuous function will be better approximated by a straight line. This characteristic is demonstrated in the following example.

**FIGURE 17.2** Graphical depiction of linear interpolation. The shaded areas indicate the similar triangles used to derive the Newton linear-interpolation formula [[Eq. \(17.5\)](#)].



## EXAMPLE 17.2

### Linear Interpolation

#### Problem Statement

Estimate the natural logarithm of 2 using linear interpolation. First, perform the computation by interpolating between  $\ln 1 = 0$  and  $\ln 6 = 1.791759$ . Then, repeat the procedure, but use a smaller interval from  $\ln 1$  to  $\ln 4$  (1.386294). Note that the true value of  $\ln 2$  is 0.6931472.

#### Solution

We use [Eq. \(17.5\)](#) from  $x_1 = 1$  to  $x_2 = 6$  to give

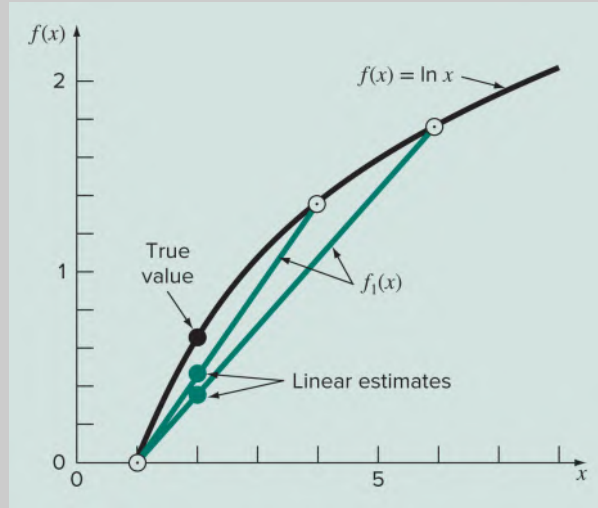
$$f_1(2) = 0 + \frac{1.791759 - 0}{6 - 1}(2 - 1) = 0.3583519$$

which represents an error of  $\varepsilon_t = 48.3\%$ . Using the smaller interval from  $x_1 = 1$  to  $x_2 = 4$  yields

$$f_1(2) = 0 + \frac{1.386294 - 0}{4 - 1}(2 - 1) = 0.4620981$$

Thus, using the shorter interval reduces the percent relative error to  $\varepsilon_t = 33.3\%$ . Both interpolations are shown in [Fig. 17.3](#), along with the true function.

**FIGURE 17.3** Two linear interpolations to estimate  $\ln 2$ . Note how the smaller interval provides a better estimate.



## 17.2.2 Quadratic Interpolation

The error in [Example 17.2](#) resulted from approximating a curve with a straight line. Consequently, a strategy for improving the estimate is to introduce some curvature into the line connecting the points. If three data points are available, this can be accomplished with a second-order polynomial (also called a quadratic polynomial or a parabola). A particularly convenient form for this purpose is

$$f_2(x) = b_1 + b_2(x - x_1) + b_3(x - x_1)(x - x_2) \quad (17.6)$$

A simple procedure can be used to determine the values of the coefficients. For  $b_1$ , [Eq. \(17.6\)](#) with  $x = x_1$  can be used to compute

$$b_1 = f(x_1) \quad (17.7)$$

[Equation \(17.7\)](#) can be substituted into [Eq. \(17.6\)](#), which can be evaluated at  $x = x_2$  for

$$b_2 = \frac{f(x_2) - f(x_1)}{x_2 - x_1} \quad (17.8)$$

Finally, [Eqs. \(17.7\)](#) and [\(17.8\)](#) can be substituted into [Eq. \(17.6\)](#), which can be evaluated at  $x = x_3$  and solved (after some algebraic manipulations) for

$$b_3 = \frac{\frac{f(x_3) - f(x_2)}{x_3 - x_2} - \frac{f(x_2) - f(x_1)}{x_2 - x_1}}{x_3 - x_1} \quad (17.9)$$

Notice that, as was the case with linear interpolation,  $b_2$  still represents the slope of the line connecting points  $x_1$  and  $x_2$ . Thus, the first two terms of [Eq. \(17.6\)](#) are equivalent to linear interpolation between  $x_1$  and  $x_2$ , as specified previously in [Eq. \(17.5\)](#). The last term,  $b_3(x - x_1)(x - x_2)$ , introduces the second-order curvature into the formula.

Before illustrating how to use [Eq. \(17.6\)](#), we should examine the form of the coefficient  $b_3$ . It is very similar to the finite-difference approximation of the second derivative introduced previously in [Eq. \(4.27\)](#). Thus, [Eq. \(17.6\)](#) is beginning to manifest a structure that is very similar to the Taylor series expansion. That is, terms are added sequentially to capture increasingly higher-order curvature.

## EXAMPLE 17.3

### Quadratic Interpolation

#### Problem Statement

Employ a second-order Newton polynomial to estimate  $\ln 2$  with the same three points used in [Example 17.2](#):

$$\begin{array}{ll} x_1 = 1 & f(x_1) = 0 \\ x_2 = 4 & f(x_2) = 1.386294 \\ x_3 = 6 & f(x_3) = 1.791759 \end{array}$$

#### Solution

Applying [Eq. \(17.7\)](#) yields

$$b_1 = 0$$

[Equation \(17.8\)](#) gives

$$b_2 = \frac{1.386294 - 0}{4 - 1} = 0.4620981$$

and [Eq. \(17.9\)](#) yields

$$b_3 = \frac{\frac{1.791759 - 1.386294}{6 - 4}}{6 - 1} = 0.4620981$$

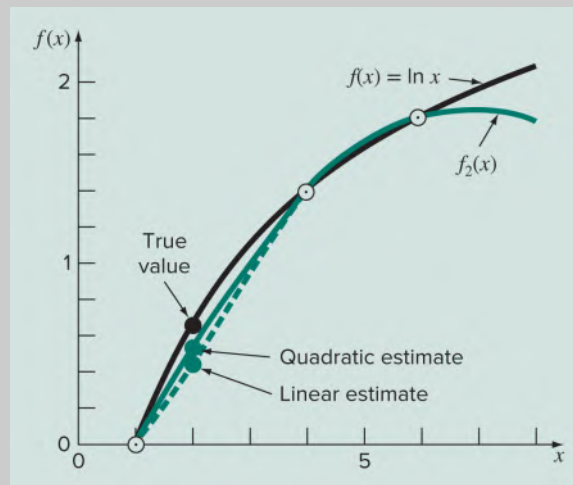
$$b_3 = \frac{1.791759 - 1.386294}{6 - 1} = -0.0518731$$

Substituting these values into [Eq. \(17.6\)](#) yields the quadratic formula

$$f_2(x) = 0 + 0.4620981(x - 1) - 0.0518731(x - 1)(x - 4)$$

which can be evaluated at  $x = 2$  for  $f_2(2) = 0.5658444$ , which represents a relative error of  $\varepsilon_t = 18.4\%$ . Thus, the curvature introduced by the quadratic formula ([Fig. 17.4](#)) improves the interpolation compared with the result obtained using straight lines in [Example 17.2](#) and [Fig. 17.3](#).

**FIGURE 17.4** The use of quadratic interpolation to estimate  $\ln 2$ . The linear interpolation from  $x = 1$  to  $4$  is also included for comparison.



## 17.2.3 General Form of Newton's Interpolating Polynomials

The preceding analysis can be generalized to fit an  $(n - 1)$ th-order polynomial to  $n$  data points. The  $(n - 1)$ th-order polynomial is

$$f_{n-1}(x) = b_1 + b_2(x - x_1) + \cdots + b_n(x - x_1)(x - x_2) \cdots (x - x_{n-1}) \quad (17.10)$$

As was done previously with linear and quadratic interpolation, data points can be used to evaluate the coefficients  $b_1, b_2, \dots, b_n$ . For an  $(n - 1)$ th-order polynomial,  $n$  data points are required:  $[x_1, f(x_1)], [x_2, f(x_2)], \dots, [x_n, f(x_n)]$ . We use these data points and the following equations to evaluate the coefficients:

$$b_1 = f(x_1) \quad (17.11)$$

$$b_2 = f[x_2, x_1] \quad (17.12)$$

$$b_3 = f[x_3, x_2, x_1] \quad (17.13)$$

.

.

.

$$b_n = f[x_n, x_{n-1}, \dots, x_2, x_1] \quad (17.14)$$

where the bracketed function evaluations are finite divided differences. For example, the first finite divided difference is represented generally as

$$f[x_i, x_j] = \frac{f(x_i) - f(x_j)}{x_i - x_j} \quad (17.15)$$

The second finite divided difference, which represents the difference of two first divided differences, is expressed generally as

$$f[x_i, x_j, x_k] = \frac{f[x_i, x_j] - f[x_j, x_k]}{x_i - x_k} \quad (17.16)$$

Similarly, the  $n$ th finite divided difference is

$$f[x_n, x_{n-1}, \dots, x_2, x_1] = \frac{f[x_n, x_{n-1}, \dots, x_2] - f[x_{n-1}, x_{n-2}, \dots, x_1]}{x_n - x_1} \quad (17.17)$$

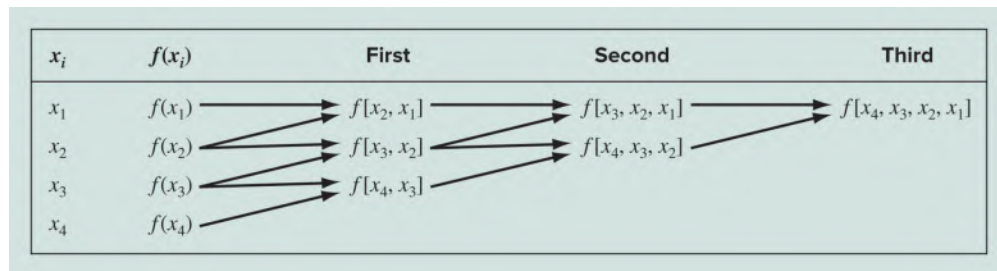
These differences can be used to evaluate the coefficients in [Eqs. \(17.11\)](#) through [\(17.14\)](#), which can then be substituted into [Eq. \(17.10\)](#) to yield the general form of Newton's interpolating polynomial:



$$f_{n-1}(x) = f(x_1) + (x - x_1)f[x_2, x_1] + (x - x_1)(x - x_2)f[x_3, x_2, x_1] + \cdots + (x - x_1)(x - x_2)\cdots(x - x_{n-2})f[x_n, x_{n-1}, \dots, x_1]$$

We should note that it is not necessary that the data points used in [Eq. \(17.18\)](#) be equally spaced or that the abscissa values necessarily be in ascending order, as illustrated in the following example. However, the points should be ordered so that they are centered around and as close as possible to the unknown. Also, notice how [Eqs. \(17.15\)](#) through [\(17.17\)](#) are recursive—that is, higher-order differences are computed by taking differences of lower-order differences ([Fig. 17.5](#)). This property will be exploited when we develop an efficient Python script to implement the method.

**FIGURE 17.5** Graphical depiction of the recursive nature of finite divided differences. This representation is referred to as a divided difference table.



## EXAMPLE 17.4

### Newton Interpolating Polynomial

#### Problem Statement

In [Example 17.3](#), data points at  $x_1 = 1$ ,  $x_2 = 4$ , and  $x_3 = 6$  were used to estimate  $\ln 2$  with a parabola. Now, adding a fourth point  $[x_4 = 5; f(x_4) = 1.609438]$ , estimate  $\ln 2$  with a third-order Newton's interpolating polynomial.

Page 404

#### Solution

The third-order polynomial, [Eq. \(17.10\)](#) with  $n = 4$ , is

$$f_3(x) = b_1 + b_2(x - x_1) + b_3(x - x_1)(x - x_2) + b_4(x - x_1)(x - x_2)(x - x_3)$$

The first divided differences for the problem are [[Eq. \(17.15\)](#)]

$$f[x_2, x_1] = \frac{1.386294 - 0}{4 - 1} = 0.4620981$$

$$f[x_3, x_2] = \frac{1.791759 - 1.386294}{6 - 4} = 0.2027326$$

$$f[x_4, x_3] = \frac{1.609438 - 1.791759}{5 - 6} = 0.1823216$$

The second divided differences are [ Eq. (17.16)]

$$f[x_3, x_2, x_1] = \frac{0.2027326 - 0.4620981}{6 - 1} = -0.05187311$$

$$f[x_4, x_3, x_2] = \frac{0.1823216 - 0.2027326}{5 - 4} = -0.02041100$$

The third divided difference is [ Eq. (17.17) with  $n = 4$ ]

$$f[x_4, x_3, x_2, x_1] = \frac{-0.02041100 - (-0.05187311)}{5 - 1} = 0.007865529$$

Thus, the divided difference table is

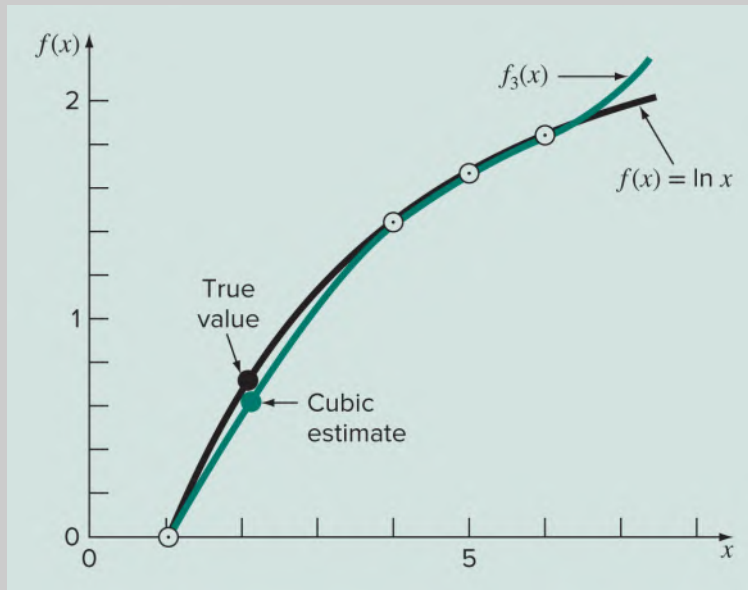
| $x_i$ | $f(x_i)$ | First     | Second      | Third       |
|-------|----------|-----------|-------------|-------------|
| 1     | 0        | 0.4620981 | -0.05187311 | 0.007865529 |
| 4     | 1.386294 | 0.2027326 | -0.02041100 |             |
| 6     | 1.791759 | 0.1823216 |             |             |
| 5     | 1.609438 |           |             |             |

The results for  $f(x_1)$ ,  $f[x_2, x_1]$ ,  $f[x_3, x_2, x_1]$ , and  $f[x_4, x_3, x_2, x_1]$  represent the coefficients  $b_1$ ,  $b_2$ ,  $b_3$ , and  $b_4$ , respectively, of  Eq. (17.10). Thus, the interpolating cubic is

$$f_3(x) = 0 + 0.4620981(x - 1) - 0.05187311(x - 1)(x - 4) + 0.007865529(x - 1)(x - 4)(x - 6)$$

which can be used to evaluate  $f_3(2) = 0.6287686$ , which represents a relative error of  $\varepsilon_t = 9.3\%$ . The complete cubic polynomial is shown in  Fig. 17.6.

**FIGURE 17.6** The use of cubic interpolation to estimate  $\ln 2$ .



## 17.2.4 Python Function: Newtint

It is straightforward to develop a Python function to implement Newton interpolation. As shown in [Fig. 17.7](#), the first step is to compute the divided differences and store them in an array. The differences are then used in conjunction with [Eq. \(17.18\)](#) to perform the interpolation.

**FIGURE 17.7** A Python function to implement Newton interpolation.

```
def Newtint(x,y,xx):
    """
    Newtint: Newton interpolating polynomial
    Uses an (n-1)th-order Newton interpolating polynomial
    based on n data pairs to return a value of the
    dependent variable, yint, at a given value of the
    independent variable, xx.
    Input:
        x = array of independent variable values
        y = array of dependent variable values
        xx = value of independent variable at which
            the interpolation is calculated
    Output:
        yint = interpolated value of the dependent variable
    """
    # compute the finite divided differences in the
    # form of a difference table
    n = len(x)
    if len(y) != n:
        return 'x and y must be of same length'
    b = np.zeros((n,n))
    # assign the dependent variables to the first column of b
    b[:,0] = np.transpose(y)
    for j in range(1,n):
```

```

        for i in range(n-j):
            b[i,j] = (b[i+1,j-1]-b[i,j-1])/(x[i+j]-x[i])
    # use the finite divided differences to interpolate
    xt = 1
    yint = b[0,0]
    for j in range(n-1):
        xt = xt * (xx - x[j])
        yint = yint + b[0,j+1]*xt
    return yint

```

An example Python script using the function would be to duplicate the calculation we performed in

 **Example 17.3:**

```

x = np.array([1., 4., 6., 5.])
y = np.log(x)
yi = Newtint(x,y,2.)
print(yi)

```

with the result

0.6287685789084135

## 17.3 LAGRANGE INTERPOLATING POLYNOMIAL

---

Suppose that we formulate a linear interpolating polynomial as the weighted average of the two values that we are connecting by a straight line:

$$f(x) = L_1 f(x_1) + L_2 f(x_2) \quad (17.19)$$

where the  $L$ 's are the weighting coefficients. It is logical that the first weighting coefficient is the straight line that is equal to 1 at  $x_1$  and 0 at  $x_2$ :

$$L_1 = \frac{x - x_2}{x_1 - x_2}$$

Similarly, the second coefficient is the straight line that is equal to 1 at  $x_2$  and 0 at  $x_1$ :

---

Page 406

$$L_2 = \frac{x - x_1}{x_2 - x_1}$$

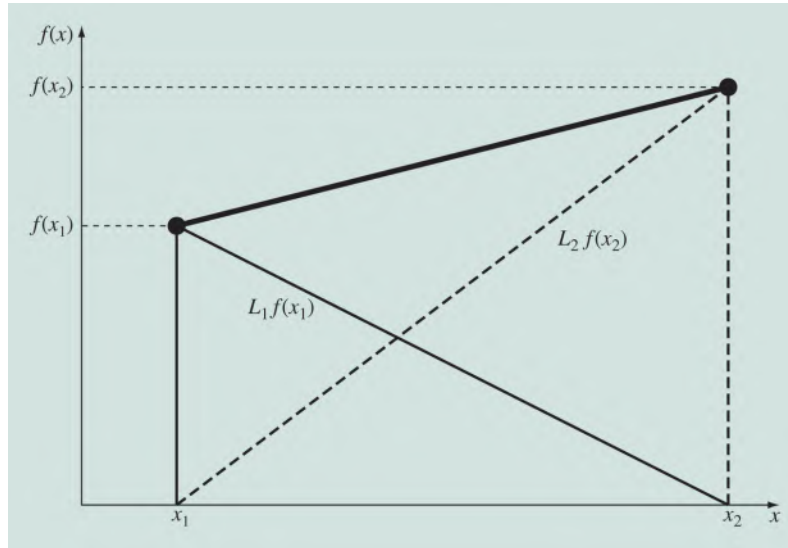
Substituting these coefficients into  **Eq. (17.19)** yields the straight line that connects the points ( **Fig. 17.8**):

$$f_1(x) = \frac{x - x_2}{x_1 - x_2} f(x_1) + \frac{x - x_1}{x_2 - x_1} f(x_2) \quad (17.20)$$

where the nomenclature  $f_1(x)$  designates that this is a first-order polynomial.  **Equation (17.20)** is referred to as the *linear Lagrange interpolating polynomial*.

---

**FIGURE 17.8** A visual depiction of the rationale behind Lagrange interpolating polynomials. The figure shows the first-order case. Each of the two terms of  **Eq. (17.20)** passes through one of the points and is zero at the other. The summation of the two terms must, therefore, be the unique straight line that connects the two points.



The same strategy can be employed to fit a parabola through three points. For this case, three parabolas would be used with each one passing through one of the points and equaling zero at the other two. Their sum would then represent the unique parabola that connects the three points. Such a second-order Lagrange interpolating polynomial can be written as

$$f_2(x) = \frac{(x - x_2)(x - x_3)}{(x_1 - x_2)(x_1 - x_3)} f(x_1) + \frac{(x - x_1)(x - x_3)}{(x_2 - x_1)(x_2 - x_3)} f(x_2) + \frac{(x - x_1)(x - x_2)}{(x_3 - x_1)(x_3 - x_2)} f(x_3)$$



Notice how the first term is equal to  $f(x_1)$  at  $x_1$  and is equal to zero at  $x_2$  and  $x_3$ . The other terms work in a similar fashion.

Both the first- and second-order versions as well as higher-order Lagrange polynomials can be represented concisely as

$$f_{n-1}(x) = \sum_{i=1}^n L_i(x) f(x_i) \quad (17.22)$$

where

$$L_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j} \quad (17.23)$$

where  $n$  = the number of data points and  $\Pi$  designates the “product of.”

## EXAMPLE 17.5

### Lagrange Interpolating Polynomial

#### Problem Statement

Use a Lagrange interpolating polynomial of the first and second orders to evaluate the density of unused motor oil at  $T = 15^\circ\text{C}$  based on the following data:

$$\begin{array}{ll} x_1 = 0 & f(x_1) = 3.85 \\ x_2 = 20 & f(x_2) = 0.800 \\ x_3 = 40 & f(x_3) = 0.212 \end{array}$$

#### Solution

The first-order polynomial [Eq. (17.20)] can be used to obtain the estimate at  $x = 15$ :

$$f_1(x) = \frac{15 - 20}{0 - 20} 3.85 + \frac{15 - 0}{20 - 0} 0.800 = 1.5625$$

In a similar fashion, the second-order polynomial is developed as [Eq. (17.21)]

$$f_2(x) = \frac{(15 - 20)(15 - 40)}{(0 - 20)(0 - 40)} 3.85 + \frac{(15 - 0)(15 - 40)}{(20 - 0)(20 - 40)} 0.800 + \frac{(15 - 0)(15 - 20)}{(40 - 0)(40 - 20)} 0.$$

## 17.3.1 Python Function: Lagrange

We can create a Python function based on Eqs. (17.22) and (17.23). As shown in Fig. 17.9, the function has three arguments, identical to those in the previous `NewtInt` function: an array of independent variable values,  $x$ , a matching array of dependent variable values,  $y$ , and a value of the independent variable,  $xx$ , to be used for the interpolation. The order of the polynomial is based on the length of the  $x$  array. If there are  $n$  values, an  $(n - 1)$ th-order polynomial is fit.

**FIGURE 17.9** A Python function to implement Lagrange interpolation.

```
def Lagrange(x,y,xx):
```

```

"""
Lagrange interpolating polynomial
Uses an (n-1)th-order Lagrange interpolating polynomial
based on n data pairs to return a value of the
dependent variable, yint, at a given value of the
independent variable, xx.
Input:
    x = array of independent variable values
    y = array of dependent variable values
    xx = value of independent variable at which
        the interpolation is calculated
Output:
    yint = interpolated value of the dependent variable
"""
n = len(x)
if len(y) != n:
    return 'x and y must be of same length'
s = 0
for i in range(n):
    product = y[i]
    for j in range(n):
        if i != j:
            product = product * (xx - x[j])/(x[i]-x[j])
    s = s + product
yint = s
return yint

```

An application of this function would be to predict the density of air at atmospheric pressure and at a temperature of 15 °C based on the first four values from [Table 17.1](#). Because four values are used for the independent and dependent variable arrays, a third-order polynomial would be implemented by the Lagrange function using the following script:

```

T = np.array([-40., 0., 20., 50.])
rho = np.array([1.52, 1.29, 1.2, 1.09])
rho_int = Lagrange(T,rho,15.)
print(rho_int)

```

with the result

```
1.2211284722222222
```



## 17.4 INVERSE INTERPOLATION

As the nomenclature implies, the  $f(x)$  and  $x$  values in most interpolation contexts are the dependent and independent variables, respectively. As a consequence, the values of the  $x$ 's are typically uniformly spaced. A simple example is a table of values derived for the function  $f(x) = 1/x$ :

| $x$    | 1 | 2   | 3      | 4    | 5   | 6      | 7      |
|--------|---|-----|--------|------|-----|--------|--------|
| $f(x)$ | 1 | 0.5 | 0.3333 | 0.25 | 0.2 | 0.1667 | 0.1429 |

Now suppose that you must use the same data, but you are given a value for  $f(x)$  and must determine the corresponding value of  $x$ . For instance, for the data above, suppose that you were asked to determine the value of  $x$  that corresponded to  $f(x) = 0.3$ . For this case, because the function is available and easy to manipulate, the correct answer can be determined directly as  $x = 1/0.3 = 3.3333$ .

Such a problem is called *inverse interpolation*. For a more complicated case, you might be tempted to switch the  $f(x)$  and  $x$  values [that is, merely plot  $x$  versus  $f(x)$ ] and use an approach like Newton or Lagrange interpolation to determine the result. Unfortunately, when you reverse the variables, there is no guarantee that the values along the new abscissa [the  $f(x)$ 's] will be evenly spaced. In fact, in many cases, the values will be “telescoped.” That is, they will have the appearance of a logarithmic scale with some adjacent points bunched together and others spread out widely. For example, for  $f(x) = 1/x$  the result is

| $f(x)$ | 0.1429 | 0.1667 | 0.2 | 0.25 | 0.3333 | 0.5 | 1 |
|--------|--------|--------|-----|------|--------|-----|---|
| $x$    | 7      | 6      | 5   | 4    | 3      | 2   | 1 |

Such nonuniform spacing on the abscissa often leads to oscillations in the resulting interpolating polynomial. This can occur even for lower-order polynomials. An alternative strategy is to fit an  $n$ th-order interpolating polynomial,  $f_n(x)$ , to the original data [that is, with  $f(x)$  versus  $x$ ]. In most cases, because the  $x$ 's are evenly spaced, this polynomial will not be ill-conditioned. The answer to your problem then amounts to finding the value of  $x$  that makes this polynomial equal to the given  $f(x)$ . Thus, the interpolation problem reduces to a roots problem!

For example, for the problem just outlined, a simple approach would be to fit a quadratic polynomial to the three points: (2, 0.5), (3, 0.3333), and (4, 0.25). The result would be

$$f_2(x) = 0.041667x^2 - 0.375x + 1.08333$$

The answer to the inverse interpolation problem of finding the  $x$  corresponding to  $f(x) = 0.3$  would therefore involve determining the root of

$$0.3 = 0.041667x^2 - 0.375x + 1.08333$$

For this simple case, the quadratic formula can be used to calculate

$$x = \frac{0.375 \pm \sqrt{(-0.375)^2 - 4(0.041667)(0.78333)}}{2(0.041667)} = \begin{matrix} 5.704158 \\ 3.295842 \end{matrix}$$

Thus, the second root, 3.296, is a good approximation of the true value of 3.333. If additional accuracy were desired, a third- or fourth-order polynomial along with one of the root-location methods from [Chaps. 5](#) or [6](#) could be employed.

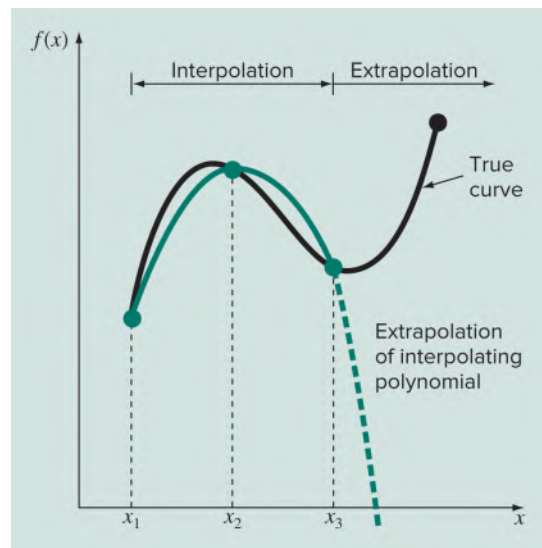
# 17.5 EXTRAPOLATION AND OSCILLATIONS

Before leaving this chapter, there are two issues related to polynomial interpolation that must be addressed. These are extrapolation and oscillations.

## 17.5.1 Extrapolation

*Extrapolation* is the process of estimating a value of  $f(x)$  that lies outside the range of the known base points,  $x_1, x_2, \dots, x_n$ . As depicted in  Fig. 17.10, the open-ended nature of extrapolation represents a step into the unknown because the process extends the curve beyond the known region. As such, the true curve could easily diverge from the prediction. Extreme care should, therefore, be exercised whenever a case arises where one must extrapolate.

**FIGURE 17.10** Illustration of the possible divergence of an extrapolated prediction. The extrapolation is based on fitting a parabola through the first three known points.



### EXAMPLE 17.6

#### Dangers of Extrapolation

#### Problem Statement

This example is patterned after one originally developed by Forsythe, Malcolm, and Moler (1977). The population in millions of the United States from 1920 to 2020<sup>1</sup> is tabulated as

| Year                  | 1920   | 1930  | 1940   | 1950   | 1960   | 1970   | 1980   | 1990   |
|-----------------------|--------|-------|--------|--------|--------|--------|--------|--------|
| Population ( $10^6$ ) | 106.02 | 123.2 | 132.17 | 151.33 | 179.32 | 203.21 | 226.55 | 248.71 |

Fit a ninth-order polynomial to the first ten points, 1920 to 2010. Use the polynomial to predict the population in 2020 and compare it to the projected value in the table.

Page 410

## Solution

First, here is the Python code that attempts to fit a seventh-order polynomial.

```
import numpy as np

yr = np.arange(1920.,2020.,10.)
pop = np.array([106.02,123.2,132.17,151.33,179.32,203.21,226.55,248.71,
                281.42,308.75])

coef = np.polyfit(yr,pop,7)
np.set_printoptions(precision=5)
print(coef)
```

However, the results displayed are cautionary:

```
[-2.62750e-19  1.89237e-15 -3.36857e-12 -4.03934e-09  1.04524e-05
  2.07940e-02 -3.03478e+01 -1.00419e+05  2.16510e+08 -1.15968e+11]
RankWarning: Polyfit may be poorly conditioned
```

This warning occurs because the polynomial is raising years to high powers. We have encountered this before, and a suggested approach is to scale the independent variable. In [Chap. 15](#), we used the scale transformation

$$z = \frac{x - x}{s_x}$$

Applying this to our example,  $z = (yr - 1965)/30$ . If we incorporate this into our Python code to fit the ninth-order polynomial, the modified code is

```
z = (yr-1965.)/30.
coef = np.polyfit(z,pop,9)
np.set_printoptions(precision=5)
print(coef)
```

And the results, without warnings, are

```
[-18.12465 -2.0495 79.56966 -1.04477 -105.68631 23.29964
 40.06379 -16.82524 70.63697 191.71441]
```

We can then use the polyval function to predict the population in 2020:

```
z20 = (2020.-1965.)/30.
pop20 = np.polyval(coef,z20)
print(' {0:7.2f}'.format(pop20))
```

And the prediction is

-417.05

which is dramatically lower than the table value of 332.64 and obviously erroneous.

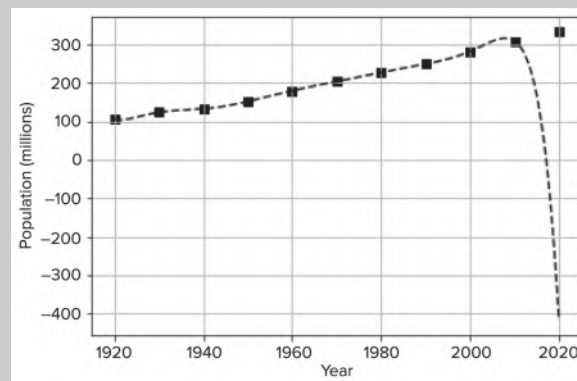
Insight can be gained by generating a plot of the table data along with the model curve.

```
import pylab
yr2 = np.append(yr,[2020.])
pop2 = np.append(pop,[332.64])
yrplot = np.linspace(1920.,2020.,200)
zplot = (yrplot-1965.)/30.
pop2plot = np.polyval(coef,zplot)
pylab.scatter(yr2,pop2,c='k',marker='s')
pylab.plot(yrplot,pop2plot,c='k',ls='--')
pylab.grid()
pylab.xlabel('Year')
pylab.ylabel('Population (millions)')
```

 **Figure 17.11** reveals the problem. The polynomial fits the data perfectly through 2010; however, when it is used to extrapolate to 2020, its upward pattern deviates drastically downward and produces a faulty prediction. A more subtle observation is the slight oscillation in the model curve between 1920 and 1940. This leads us to the topic of the next section.

Page 411

**FIGURE 17.11** Use of a ninth-order polynomial to make a prediction of U.S. population in 2020 based on data from 1920 through 2010.



## 17.5.2 Oscillations

We have seen, in the case of overfitting polynomials in regression, that model lines can oscillate between data points and provide spurious predictions. This is also the case for polynomial interpolation—“more is not always better.” A higher-order polynomial, which interpolation requires to pass through data points, may demonstrate significant oscillations in between those points. The following example illustrates this point and emphasizes the concept of *parsimony*, which we discussed in regression, that is, use the simplest model or polynomial that yet provides an adequate tool for interpolation.

## EXAMPLE 17.7

### Dangers of Higher-Order Polynomial Interpolation

#### Problem Statement

In 1901, Carl Runge<sup>2</sup> published a study on the dangers of higher-order polynomial interpolation. He studied the following, deceptively simple function:

$$f(x) = \frac{1}{1 + 25x^2} \quad (17.24)$$

which is now called *Runge's function*. Runge took equidistant-spaced  $x$  values over the interval,  $[-1, 1]$ . He then used interpolating polynomials of increasing order and found that, as he took more points, the polynomials and the original curve differed considerably. Further, the situation deteriorated greatly as the order was increased. This behavior is referred to as *Runge's phenomenon*. Duplicate Runge's result by using the Python NumPy `polyfit` and `polyval` functions to fit fourth- and tenth-order polynomials to five and eleven equally spaced points, respectively, generated using [Eq. \(17.24\)](#). Create plots of your results along with the sample values and the curve for Runge's function.

#### Solution

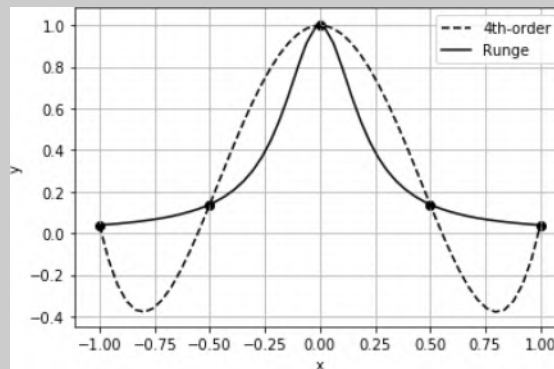
For the first case, the following Python code generates the five  $x$  values and corresponding  $f(x)$  values, interpolates a range of 50 values of  $x$  based on a fourth-order polynomial fit, and creates a plot including the Runge's function curve.

```
import numpy as np
import pylab
# generate 5 equally-spaced points and function values
x = np.linspace(-1.,1.,5)
y = 1./(1.+25.*x**2)
# 50 interpolation points
xx = np.linspace(-1.,1.)
# fit 4th-order polynomial
coef = np.polyfit(x,y,4)
# use polynomial to interpolate
y4 = np.polyval(coef,xx)
# Runge's function values
yr = 1./(1.+25.*xx**2)
# generate plot
pylab.scatter(x,y,c='k',marker='o')
pylab.plot(xx,y4,c='k',ls='--',label='4th-order')
pylab.plot(xx,yr,c='k',label='Runge')
pylab.grid()
pylab.xlabel('x')
pylab.ylabel('y')
pylab.legend()
```

Page 412

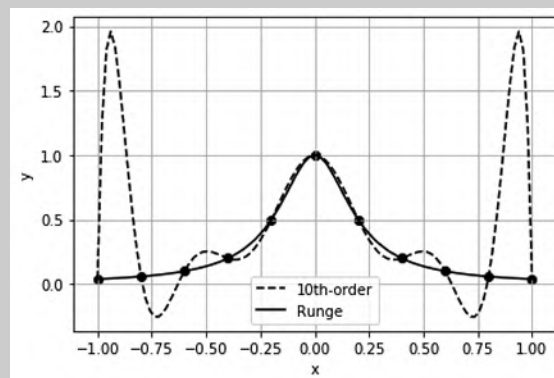
The plot is shown in [Fig. 17.12](#), and we can see that the polynomial does a poor job of representing Runge's function and providing interpolations.

**FIGURE 17.12** Comparison of Runge's function and a fourth-order interpolating polynomial based on five equally spaced points sample from the function.



We can easily modify the Python code to provide the plot for the second case. The result is shown in [Fig. 17.13](#).

**FIGURE 17.13** Comparison of Runge's function and a tenth-order interpolating polynomial based on eleven equally spaced points sample from the function.



```
import numpy as np
import pylab
# generate 11 equally-spaced points and function values
x = np.linspace(-1.,1.,11)
y = 1./(1.+25.*x**2)
# 100 interpolation points
xx = np.linspace(-1.,1.,100)
# fit 10th-order polynomial
coef = np.polyfit(x,y,10)
# use polynomial to interpolate
y10 = np.polyval(coef,xx)
# Runge's function values
yr = 1./(1.+25.*xx**2)
# generate plot
pylab.scatter(x,y,c='k',marker='o')
pylab.plot(xx,y10,c='k',ls='--',label='10th-order')
```

```
pylab.plot(xx,yy,c='k',label='Runge')
pylab.grid()
pylab.xlabel('x')
pylab.ylabel('y')
pylab.legend()
```

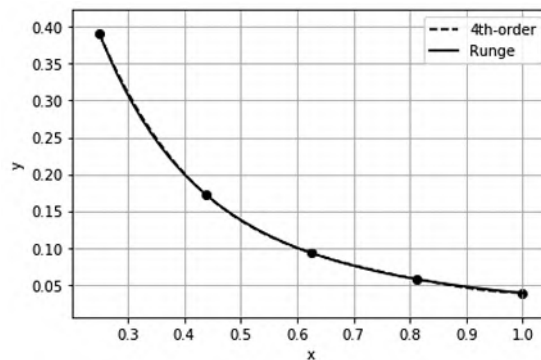
The oscillations in [Fig. 17.13](#) are worse, particularly at the ends of the interval.

Although there may be certain contexts where high-order polynomials are justified, they are generally to be avoided. In most engineering and scientific scenarios, lower-order polynomials of the type described in this chapter can be used effectively to capture curving trends in data without suffering from oscillations.

Page 413

A case in point is to consider Runge's function over the interval  $[0.25, 1.0]$  with five equidistant points and a fourth-order interpolating polynomial. The results, dramatically better, are shown in [Fig. 17.14](#). You will note here that we have considered a portion of the original interval rather than attempting to fit a polynomial from  $-1$  to  $1$ . We will be considering the concept of subintervals or piecewise interpolation in the next chapter.

**FIGURE 17.14** A fourth-order interpolating polynomial and Runge's function over the interval  $[0.25, 1.0]$ .





## PROBLEMS

**17.1** The following data were measured with high precision. Use the best numerical method (for this type of problem) to estimate  $y$  at  $x = 3.5$ . Note that a polynomial will yield an exact value. Your solution should prove that your result is exact.

|     |    |        |       |     |       |      |    |
|-----|----|--------|-------|-----|-------|------|----|
| $x$ | 0  | 1.8    | 5     | 6   | 8.2   | 9.2  | 12 |
| $y$ | 26 | 16.415 | 5.375 | 3.5 | 2.015 | 2.54 | 8  |

**17.2** Based on the data in the table, use Newton's interpolating polynomial to determine  $y$  at  $x = 3.5$  to the best possible accuracy. Compute the finite divided differences as in [Fig. 17.5](#) and order your points to attain optimal accuracy and convergence. That is, the points should be centered around and as close as possible to the unknown.

|     |   |        |        |        |        |        |    |
|-----|---|--------|--------|--------|--------|--------|----|
| $x$ | 0 | 1      | 2.5    | 3      | 4.5    | 5      | 6  |
| $y$ | 2 | 5.4375 | 7.3516 | 7.5625 | 8.4453 | 9.1875 | 12 |

**17.3** Given the data in the table below, use Newton's interpolating polynomial to estimate  $y$  at  $x = 8$ . Compute the finite divided differences as in [Fig. 17.5](#) and order your points to attain optimal accuracy and convergence. That is, the points should be centered around and as close as possible to the unknown. Then, repeat the process, but use the order of data in the table. Comment on any differences in your results.

|     |     |       |     |     |      |      |     |     |
|-----|-----|-------|-----|-----|------|------|-----|-----|
| $x$ | 0   | 1     | 2   | 5.5 | 11   | 13   | 16  | 18  |
| $y$ | 0.5 | 3.134 | 5.3 | 9.9 | 10.2 | 9.35 | 7.2 | 6.2 |

**17.4** Given the data

|        |   |   |     |     |   |   |
|--------|---|---|-----|-----|---|---|
| $x$    | 1 | 2 | 2.5 | 3   | 4 | 5 |
| $f(x)$ | 0 | 5 | 7   | 6.5 | 2 | 0 |

- Plot the data and estimate  $f(3.4)$  from the plot.
- Calculate  $f(3.4)$  using Newton's interpolating polynomials of order one through five.
- Analyze your results to determine what order of polynomial provides a reasonably accurate interpolation.

**17.5** Given the data in the table below, estimate  $f(4)$  using Newton's interpolating polynomials of order one through four. Choose your base points to be centered around and as close as possible to the unknown. What do your results indicate regarding the order of the polynomial used to generate the data in the table?

| $x$    | 1    | 2 | 3    | 5     | 6  |
|--------|------|---|------|-------|----|
| $f(x)$ | 4.75 | 4 | 5.25 | 19.75 | 36 |

**17.6** Repeat  **Problem 17.5** using Lagrange polynomials of order one through three.

**17.7**  **Table P15.5** lists values for dissolved oxygen concentration in water as a function of temperature and chloride concentration.

- Use quadratic and cubic interpolation to determine the oxygen concentration for  $T = 12^\circ\text{C}$  and  $c = 10\text{ g/L}$ .
- Use linear interpolation to determine the oxygen concentration for  $T = 12^\circ\text{C}$  and  $c = 15\text{ g/L}$ .
- Repeat (b) but use quadratic interpolation.

**17.8** Employ inverse interpolation using a cubic polynomial and bisection to determine the value of  $x$  that corresponds to  $f(x) = 1.7$  for the following tabulated data:

| $x$    | 1   | 2   | 3   | 4   | 5    | 6   | 7       |
|--------|-----|-----|-----|-----|------|-----|---------|
| $f(x)$ | 3.6 | 1.8 | 1.2 | 0.9 | 0.72 | 1.5 | 0.51429 |

**17.9** Employ inverse interpolation to determine the value of  $x$  that corresponds to  $f(x) = 0.93$  for the following tabulated data:

| $x$    | 0 | 1   | 2   | 3   | 4        | 5        |
|--------|---|-----|-----|-----|----------|----------|
| $f(x)$ | 0 | 0.5 | 0.8 | 0.9 | 0.941176 | 0.961538 |

Note that the values in the table were generated with the function  $f(x) = x^2/(1 + x^2)$ .

- Determine the correct value analytically.
- Use quadratic inverse interpolation and the quadratic formula to determine the value numerically.
- Use cubic inverse interpolation and bisection to determine the value numerically.

**17.10** A portion of the steam table for specific volume,  $v$  ( $\text{m}^3/\text{kg}$ ), and entropy,  $s$  ( $\text{kJ}/[\text{kg} \cdot \text{K}]$ ), at a pressure of 200 MPa is presented in the table below.

- Estimate the entropy for  $v = 0.118$  using linear interpolation.
- Estimate the same as in (a) but using quadratic interpolation.
- Estimate the specific volume corresponding to an entropy of 6.45 using quadratic inverse interpolation.

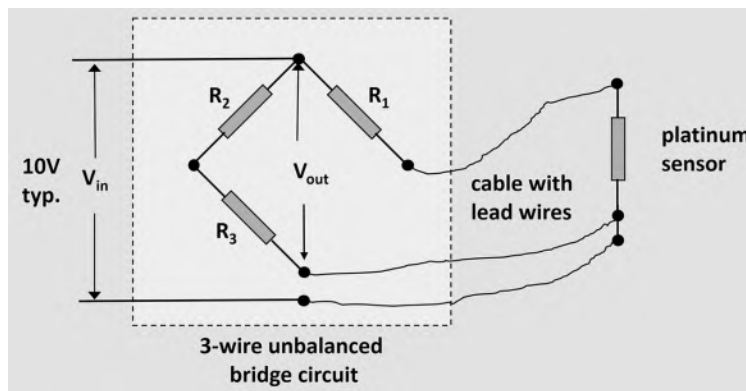
|                              |         |         |         |
|------------------------------|---------|---------|---------|
| $\nu, \text{m}^3/\text{kg}$  | 0.10377 | 0.11144 | 0.12547 |
| $s, \text{kJ}/(\text{kg K})$ | 6.4147  | 6.5453  | 6.7664  |

**17.11** The following data for the density of nitrogen gas versus temperature come from a table with values that were measured with high precision. Use first- through fifth-order polynomials to estimate the density at a temperature of 330 K. What is your best estimate? Use your choice and inverse interpolation to determine the corresponding temperature.

|                                 |       |       |       |       |       |       |
|---------------------------------|-------|-------|-------|-------|-------|-------|
| $T, \text{K}$                   | 200   | 250   | 300   | 350   | 400   | 450   |
| Density, $\text{kg}/\text{m}^3$ | 1.708 | 1.367 | 1.139 | 0.967 | 0.854 | 0.759 |

**17.12** One of the most accurate devices for measuring temperature is the platinum resistance thermometer, AKA RTD. In this device, a long, thin wire of platinum is wound around a bobbin and embedded in a stainless-steel sheath. The resistance of the wire changes very linearly with temperature. A typical length of wire gives a resistance around  $100 \Omega$  at  $0^\circ\text{C}$  and changes about  $0.39 \Omega/^\circ\text{C}$ . Consequently, in order to measure temperatures accurately, one must be able to measure changes in resistance to a high resolution. Traditionally, the unbalanced bridge circuit has been used to do this. It is illustrated in [Fig. P17.12](#) and provides a small voltage output that is proportional (but not linearly so) to the sensor's resistance.

**FIGURE P17.12** A three-wire unbalanced bridge circuit for measuring resistance.



Here are measurements taken from such a circuit, with  $R_1 = R_2 = 5 \text{ k}\Omega$  and  $R_3 = 120 \Omega$ :

|                         |         |         |          |          |          |          |
|-------------------------|---------|---------|----------|----------|----------|----------|
| Sensor                  | 100     | 115     | 130      | 145      | 160      | 175      |
| Resistance ( $\Omega$ ) |         |         |          |          |          |          |
| Bridge Output (V)       | 0.45244 | 0.10592 | -0.19918 | -0.46988 | -0.71167 | -0.92896 |

A standard 100  $\Omega$  platinum resistance thermometer has a resistance of 139.27  $\Omega$  at 100 °C. Use polynomial interpolation to estimate the bridge output voltage for these conditions. If the bridge output is measured to be 0 volts, use inverse interpolation to estimate the measured resistance. What do you conclude?

**17.13** Bessel functions often arise in advanced engineering analyses, such as the study of electric fields. Here are some selected values for the zeroth-order Bessel function of the first kind:

| $x$      | 1.8    | 2.0    | 2.2    | 2.4    | 2.6    |
|----------|--------|--------|--------|--------|--------|
| $J_1(x)$ | 0.5815 | 0.5767 | 0.5560 | 0.5202 | 0.4708 |

Estimate  $J_1(2.1)$  using third- and fourth-order interpolating polynomials. Determine the percent error for each case compared to the true value, which can be determined with the `special.jv` function in Python's SciPy module.

**17.14** Repeat [Example 17.6](#), but using first-, second- and third-order interpolating polynomials to predict the population in 2020 based on the most recent data. That is, for the linear prediction, use data from 2000 and 2010, and so on. Compare your results to the value from the US Census Bureau, which is also a projection. Which of your results compares best?

**17.15** The specific volume of superheated steam at a given pressure is listed in the table below, with values extracted from the steam tables.

| $T, ^\circ\text{C}$ | 370    | 382    | 394    | 406   | 418     |
|---------------------|--------|--------|--------|-------|---------|
| $v, \text{L/kg}$    | 5.9313 | 7.5838 | 8.8428 | 9.796 | 10.5311 |

Estimate  $v$  at  $T = 400$  °C using polynomial interpolation.

**17.16** The vertical stress,  $\sigma_z$ , under the corner of a rectangular area subjected to a uniform load of intensity,  $q$ , is given by the solution of Boussinesq's equation:

$$\sigma = \frac{q}{4\pi} \left[ \frac{2mn\sqrt{m^2 + n^2 + 1}}{m^2 + n^2 + 1 + m^2n^2} \frac{m^2 + n^2 + 2}{m^2 + n^2 + 1} + \sin^{-1} \left( \frac{2mn\sqrt{m^2 + n^2 + 1}}{m^2 + n^2 + 1 + m^2n^2} \right) \right]$$

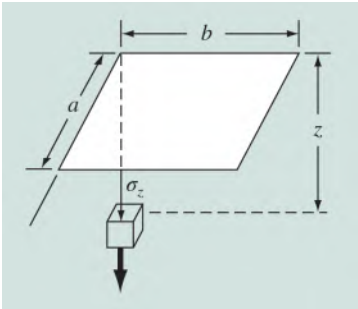
Historically, since this equation is inconvenient to solve manually, it has been reformulated as

$$\sigma_z = qf_z(m, n)$$

where  $f_z(m, n)$  is called the influence value, and  $m$  and  $n$  are dimensionless ratios, with  $m = a/z$ ,  $n = b/z$ , and  $a$  and  $b$  are defined in  Fig. P17.16. The influence value is then tabulated, a portion of which is given in Table P17.16. If  $a = 4.6$  m and  $b = 14$  m, use a third-order interpolating polynomial to compute  $\sigma_z$  at a depth of 10 m below the corner of a rectangular footing that is subject to a total load of 100 tonnes (metric tons). Express your answer in tonnes per square meter. Note that  $q$  is equal to the load per unit area.

| <i>m</i> | <i>n</i> = 1.2 | <i>n</i> = 1.4 | <i>n</i> = 1.6 |
|----------|----------------|----------------|----------------|
| 0.1      | 0.02926        | 0.03007        | 0.03058        |
| 0.2      | 0.05733        | 0.05894        | 0.05994        |
| 0.3      | 0.08323        | 0.08561        | 0.08709        |
| 0.4      | 0.10631        | 0.10941        | 0.11135        |
| 0.5      | 0.12626        | 0.13003        | 0.13241        |
| 0.6      | 0.14309        | 0.14749        | 0.15027        |
| 0.7      | 0.15703        | 0.16199        | 0.16515        |
| 0.8      | 0.16843        | 0.17389        | 0.17739        |

**FIGURE P17.16**



**17.17** The density of water versus temperature is displayed in the table below.

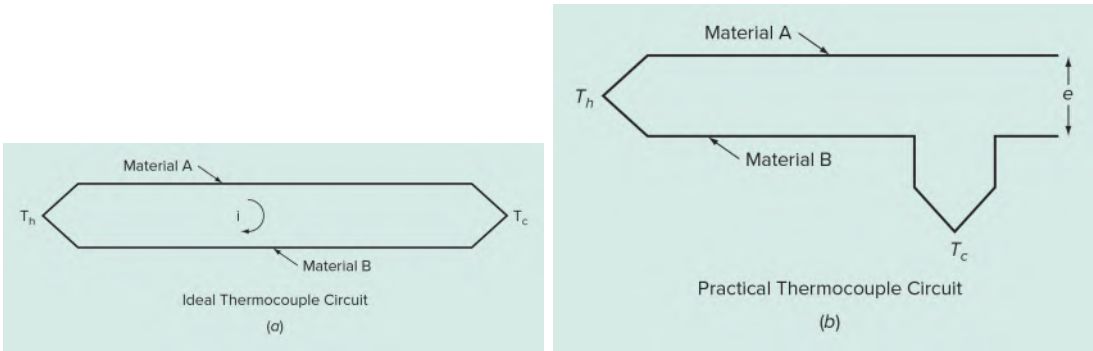
| Temperature (°C) | Density (kg/m <sup>3</sup> ) |
|------------------|------------------------------|
| 0                | 999.87                       |
| 2                | 999.97                       |
| 6                | 999.97                       |
| 10               | 999.73                       |

Water has the curious property that its density goes through a maximum between 0 and 10 °C. An artifact of this is that its specific volume increases as it freezes, a well-known cause for breaks in

domestic water piping. Use a 3rd-order interpolating polynomial and to estimate the location of the maximum and its value. By the way, the value is well known as  $1000 \text{ kg/m}^3$ .

**17.18** The thermocouple is the most common device for measuring temperature in industrial applications. It is based on the concept of a small “electromotive force,” or e.m.f., generated at the junction of two distinct metals or alloys. If this junction is balanced by another at the same temperature, the e.m.f.’s cancel each other out, but, if the junctions are at different temperatures,  $T_h$  and  $T_c$ , as shown in  **Fig. P17.18a**, a small current is created that is proportional to the temperature difference.

**FIGURE P17.18** Ideal (a) and practical (b) thermocouple pair circuits.



The thermocouple circuit can be made practical, as shown in  **Fig. P17.18b**, by controlling the temperature,  $T_c$ , called the reference junction temperature, and breaking the circuit to measure the net e.m.f. For typical thermocouple materials, the measured e.m.f. is in millivolts (mV) and, in order to measure tenths of a degree, one must be able to measure at microvolt ( $\mu\text{V}$ ) resolution.

The relationship between the net e.m.f. and temperature for common material pairs has been measured carefully in the laboratory and is available in tables, for example, from NIST. It is somewhat linear, but not entirely. Here are some table values for the common type K thermocouple, chromel–alumel (chromel –90% nickel/10% chromium alloy, alumel –95% nickel/2% aluminum/2% manganese/1% silicon alloy), based on the reference junction temperature of  $0^\circ\text{C}$ :

| Temperature ( $^\circ\text{C}$ ) | –100   | 0 | 100   | 200   | 300    | 400    |
|----------------------------------|--------|---|-------|-------|--------|--------|
| e.m.f. (mV)                      | –3.554 | 0 | 4.096 | 8.138 | 12.209 | 16.397 |

Use polynomial interpolation to estimate the e.m.f. corresponding to a temperature of  $144^\circ\text{C}$ . Find the exact value from table references and compute the percent error.

**17.19** Gravitational acceleration is less as the distance above the surface of the earth increases. Estimate  $g$  at 55,000 m, based on the data in the table below:

|                           |        |        |        |        |         |
|---------------------------|--------|--------|--------|--------|---------|
| <b>y, m</b>               | 0      | 30,000 | 60,000 | 90,000 | 120,000 |
| <b>g, m/s<sup>2</sup></b> | 9.8100 | 9.7487 | 9.6879 | 9.6278 | 9.5682  |

**17.20** Temperatures are measured precisely at various points on a heated plate ([Table P17.20](#)). Estimate the temperature at (a)  $x = 4$ ,  $y = 3.2$ , and (b)  $x = 4.3$ ,  $y = 2.7$ .

**TABLE P17.20** Temperatures ( $^{\circ}\text{C}$ ) at various points on a square heated plate.

|              | $x = 0$ | $x = 2$ | $x = 4$ | $x = 6$ | $x = 8$ |
|--------------|---------|---------|---------|---------|---------|
| <b>y = 0</b> | 100.00  | 90.00   | 80.00   | 70.00   | 60.00   |
| <b>y = 2</b> | 85.00   | 64.49   | 53.50   | 48.15   | 50.00   |
| <b>y = 4</b> | 70.00   | 48.90   | 38.43   | 35.03   | 40.00   |
| <b>y = 6</b> | 55.00   | 38.78   | 30.39   | 27.07   | 30.00   |
| <b>y = 8</b> | 40.00   | 35.00   | 30.00   | 25.00   | 20.00   |

**17.21** The refractive index of borosilicate glass at different wavelengths of light is presented in the table. Use inverse interpolation to estimate the wavelength where the refractive index is 1.520.

| <b>Wavelength <math>\lambda</math> (Å)</b> | <b>Refractive Index <math>-n</math></b> |
|--------------------------------------------|-----------------------------------------|
| 6563                                       | 1.50883                                 |
| 6439                                       | 1.50917                                 |
| 5890                                       | 1.51124                                 |
| 5338                                       | 1.51386                                 |
| 5086                                       | 1.51534                                 |
| 4861                                       | 1.51690                                 |
| 4340                                       | 1.52136                                 |
| 3988                                       | 1.52546                                 |

**17.22** Develop a Python function, using `polyfit` and `polyval` from the NumPy module, that performs polynomial interpolation. The function should look like

```
def polyint(x,y,n,xval):
    .
    .
    .
    return yint
```

where  $x$  and  $y$  are the input arrays,  $n$  is the polynomial order, and  $xval$  is the value of  $x$  to be interpolated. The function should check that the argument  $n$  is acceptable, given the length of the  $x$  and  $y$  arrays. The function should also check that the  $x$  and  $y$  arrays are the same length.

Test your function with the  $x$  values, 1., 2., 4., and 8., and the  $y$  values computed from

$$y = 10e^{-x/5}$$

interpolating the  $y$  value for  $x = 3$ . Compute and display the percentage error of your interpolation.

**17.23** The following data come from a table where the measurements were taken with high precision. Use the Newton interpolating polynomial to estimate  $y$  at  $x = 3.5$ . Properly order all the points and develop the divided finite-difference table. Note that a polynomial will yield an exact value. Your solution should prove that your result is exact.

|                       |    |      |       |     |       |     |   |
|-----------------------|----|------|-------|-----|-------|-----|---|
| <b><math>x</math></b> | 0  | 1    | 2.5   | 3   | 4.5   | 5   | 6 |
| <b><math>y</math></b> | 26 | 15.5 | 5.375 | 3.5 | 2.375 | 3.5 | 8 |

**17.24** The following data describe the U.S. Standard Atmosphere pressure (kPa) versus altitude from sea level.

Page 417

|                           |        |       |       |       |       |       |       |       |       |       |       |
|---------------------------|--------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| <b>Altitude<br/>(m)</b>   | 0      | 1000  | 2000  | 3000  | 4000  | 5000  | 6000  | 7000  | 8000  | 9000  | 10000 |
| <b>Pressure<br/>(kPa)</b> | 101.33 | 89.88 | 79.50 | 70.12 | 61.66 | 54.05 | 47.22 | 41.11 | 35.65 | 30.80 | 26.50 |

Select the last five entries in the table and use the Newton and Lagrange functions, described in [Figs. 17.7](#) and [Fig. 17.9](#), respectively, to estimate the pressure at the summit of Mt. Everest.



# Splines and Piecewise Interpolation

## CHAPTER OBJECTIVES

The primary objective of this chapter is to introduce you to splines and the contrast between piecewise interpolation and smoothing. Specific objectives and topics covered are

- Understanding that splines minimize oscillations by fitting lower-order polynomials to data in a piecewise fashion.
- Knowing how to develop code to perform a table lookup.
- Recognizing why cubic polynomials are preferable to quadratic and higher-order splines.
- Understanding the continuity conditions that underlie a cubic spline fit.
- Understanding the difference between natural, clamped, and not-a-knot end conditions.
- Knowing how to fit a spline to data using the `interpolate` functions in Python's SciPy module.
- Understanding how two-dimensional interpolation is implemented with Python.
- Seeing how splines can be extended as a smoothing mechanism for noisy data.
- Understanding the alternative LOESS method for smoothing of noisy data.

## 18.1 INTRODUCTION TO SPLINES

---

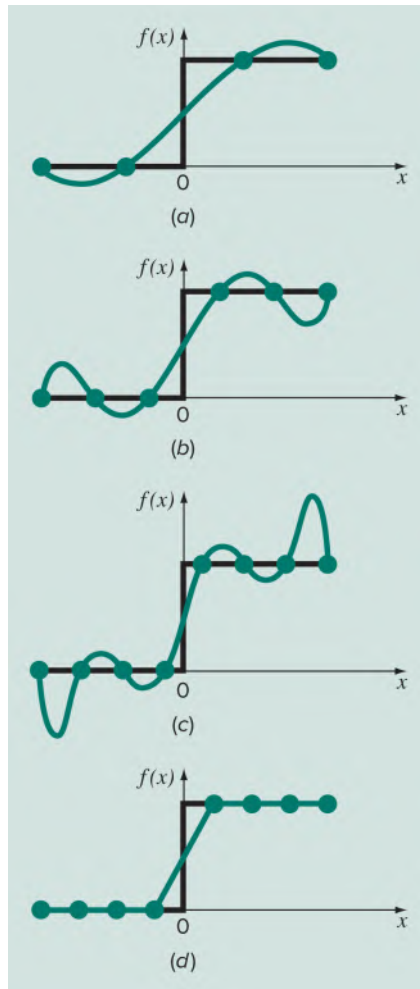
In [Chap. 17](#),  $(n - 1)$ th-order polynomials were used to interpolate between  $n$  data points. For example, for eight points, we can derive a perfect seventh-order polynomial. This curve would capture all the meanderings (at least up to and including seventh derivatives) suggested by the points. However, there are cases where these functions can lead to erroneous results because of round-off error and oscillations. An alternative approach is to apply lower-order polynomials in a piecewise fashion to subsets of data points. Such connecting polynomials are called *spline functions*.

For example, third-order curves employed to connect each pair of data points are called *cubic splines*. These functions can be constructed so that the connections between adjacent cubic equations are visually smooth. On the surface, it would seem that the third-order approximation of the splines would be inferior to the seventh-order expression. You might wonder why a spline would ever be preferable.

[Figure 18.1](#) illustrates a situation where a spline performs better than a higher-order polynomial. This is the case where a function is generally smooth but undergoes an abrupt change somewhere along the region of interest. The step increase depicted in [Fig. 18.1](#) is an extreme example of such a change but serves to illustrate the point.

---

**FIGURE 18.1** A visual representation of a situation where splines are superior to higher-order interpolating polynomials. The function to be fit undergoes an abrupt increase at  $x = 0$ . Parts (a) through (c) indicate that the abrupt change induces oscillations in interpolating polynomials. In contrast, because it is limited to straight-line connections, a linear spline (d) provides a much more acceptable approximation.

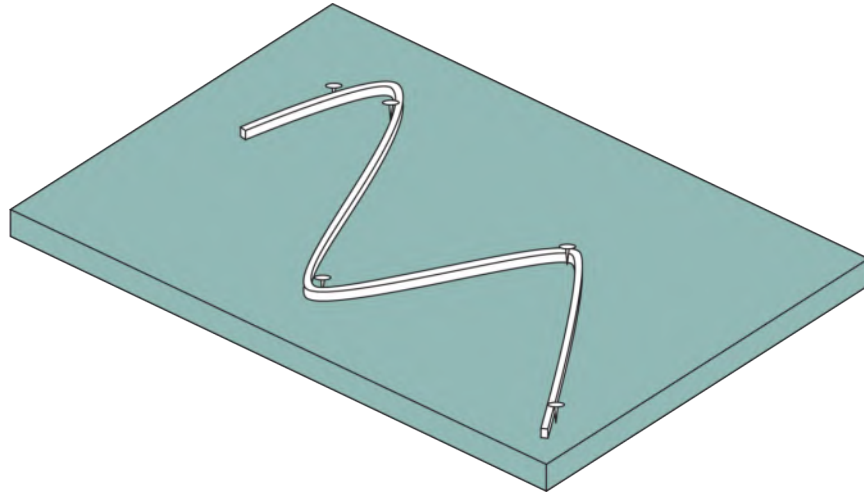


 **Figure 18.1a** through  **c** illustrates how higher-order polynomials tend to swing through wild oscillations in the vicinity of an abrupt change. In contrast, the spline also connects the points, but because it is limited to lower-order changes, the oscillations are kept to a minimum. As such, the spline usually provides a superior approximation of the behavior of functions that have local, abrupt changes.

The concept of the spline originated from the drafting technique of using a thin, flexible strip Page 419 (called a *spline*) to draw smooth curves through a series of points. The process is depicted in  **Fig. 18.2** for a series of five pins (data points). In this technique, the drafter places paper over a wooden board and hammers nails or pins into the paper (and board) at the location of the data points. A smooth cubic curve results from interweaving the strip between the pins. Hence, the name “cubic spline” has been adopted for polynomials of this type.

---

**FIGURE 18.2** The drafting technique of using a spline to draw smooth curves through a series of points. Notice how, at the end points, the spline straightens out. This is called a “natural” spline.



In this chapter, simple linear functions will first be used to introduce some basic concepts and issues associated with spline interpolation. Then, we derive an algorithm for fitting quadratic splines to data. This is followed by material on the cubic spline, which is the most common and useful version in engineering and science. Finally, we will describe the capabilities of the `interpolate` submodule of Python's SciPy module for piecewise interpolation including its ability to generate splines.

## 18.2 LINEAR SPLINES

The notation used for splines is displayed in [Fig. 18.3](#). For  $n$  data points ( $i = 1, 2, \dots, n$ ), there are  $n - 1$  intervals. Each interval  $i$  has its own spline function,  $s_i(x)$ . For linear splines, each function is merely the straight line connecting the two points at each end of the interval, which is formulated as

$$s_i(x) = a_i + b_i(x - x_i) \quad (18.1)$$

where  $a_i$  is the intercept, which is defined as

$$a_i = f_i \quad (18.2)$$

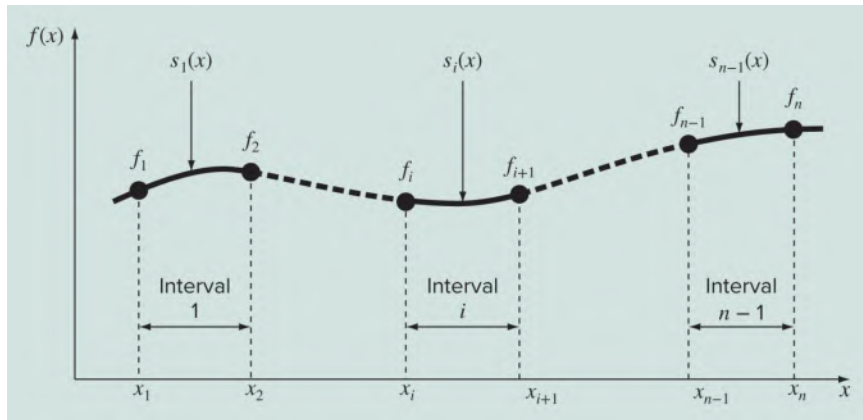
and  $b_i$  is the slope of the straight line connecting the points:

$$b_i = \frac{f_{i+1} - f_i}{x_{i+1} - x_i} \quad (18.3)$$

where  $f_i$  is shorthand for  $f(x_i)$ . Substituting [Eqs. \(18.1\)](#) and [\(18.2\)](#) into [Eq. \(18.3\)](#) gives

$$s_i(x) = f_i + \frac{f_{i+1} - f_i}{x_{i+1} - x_i}(x - x_i) \quad (18.4)$$

**FIGURE 18.3** Notation used to derive splines. Notice that there are  $n - 1$  intervals and  $n$  data points.



These equations can be used to evaluate the function at any point between  $x_1$  and  $x_n$  by first locating the interval within which the point lies. Then, the appropriate equation is used to determine the function value within the interval. Inspection of [Eq. \(18.4\)](#) indicates that the linear spline amounts to using Newton's first-order polynomial [[Eq. \(17.5\)](#)] to interpolate within each interval.

## EXAMPLE 18.1

### First-Order Splines

#### Problem Statement

Fit the data in [Table 18.1](#) with first-order splines. Evaluate the function at  $x = 5$ .

**TABLE 18.1** Data to be fit with spline functions.

| $i$ | $x_i$ | $f_i$ |
|-----|-------|-------|
| 1   | 3.0   | 2.5   |
| 2   | 4.5   | 1.0   |
| 3   | 7.0   | 2.5   |
| 4   | 9.0   | 0.5   |

#### Solution

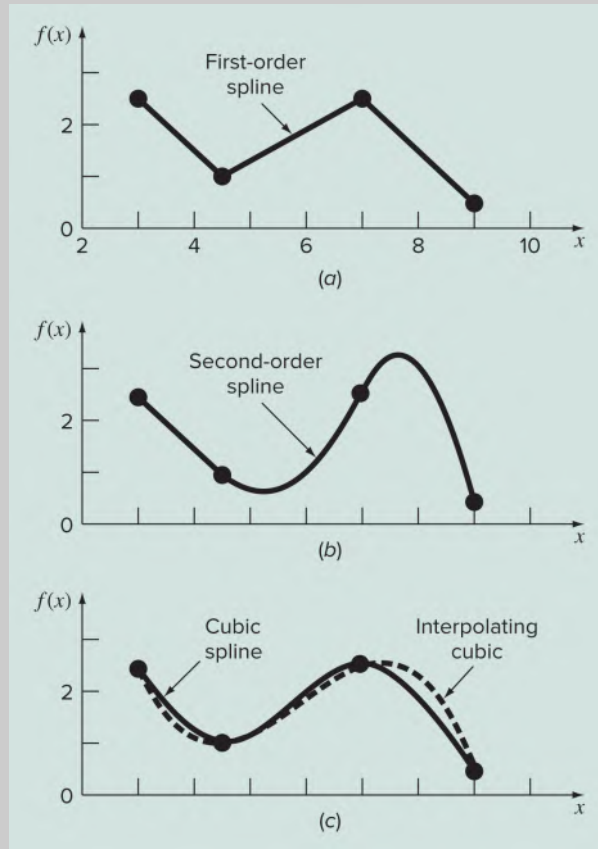
The data can be substituted into [Eq. \(18.4\)](#) to generate the linear spline functions. For example, for the second interval from  $x = 4.5$  to  $x = 7$ , the function is

$$s_2(x) = 1.0 + \frac{2.5 - 1.0}{7.0 - 4.5}(x - 4.5)$$

The equations for the other intervals can be computed, and the resulting first-order splines are plotted in [Fig. 18.4a](#). The value at  $x = 5$  is 1.3.

$$s_2(x) = 1.0 + \frac{2.5 - 1.0}{7.0 - 4.5}(5 - 4.5) = 1.3$$

**FIGURE 18.4** Spline fits of a set of four points. (a) Linear spline, (b) quadratic spline, and (c) cubic spline, with a cubic interpolating polynomial also plotted.



Visual inspection of [Fig. 18.4a](#) indicates that the primary disadvantage of first-order splines is that they are not smooth. In essence, at the data points where two splines meet (called a *knot*), the slope changes abruptly. In formal terms, the first derivative of the function is discontinuous at these points. This deficiency is overcome by using higher-order polynomial splines that ensure smoothness at the knots by equating derivatives at these points, as will be discussed subsequently. Before doing that, the following section provides an application where linear splines are useful.

## 18.2.1 Table Lookup

A table lookup is a common task frequently encountered in engineering and computer science applications. There are two types of table lookup. One has to do with *discrete tables*. These are tables of information related to discrete objects. Examples would be properties of specific materials listed by their names, such as a table of angle of repose of particulate materials. The other, which we call *continuous*

*tables*, are a discrete abstraction of a continuous property, such as the density of air at different temperatures, represented in [Table 17.1](#). The approaches to table lookup are different for each type.

First, the entries in a discrete table can generally be in any order, and the index or independent variable(s) can be numerical or names (strings). In continuous tables, we expect the independent variable(s) to be ordered, usually by ascending value. One does not interpolate a discrete table. Either the item is there, or it is not. There is also the possibility that the item occurs more than once, and there must be a strategy of what to do when that occurs. Even though discrete tables are not necessarily numerical in nature, it is worthwhile to present an example using Python because the extraction of information from those tables is common.

## EXAMPLE 18.2

### Extraction of Information from a Discrete Table

#### Problem Statement

[Table 18.2](#) presents a set of properties for various granular materials. Develop a Python function that has arguments of the name of the material and the name of the property and returns the appropriate value.

**TABLE 18.2** Properties for various granular materials.

| Bed<br>Material | Absolute<br>Density<br>(kg/m <sup>3</sup> ) | Bulk<br>Density<br>(kg/m <sup>3</sup> ) | Percent<br>Void | Particle<br>Diameter<br>(mm) | Shape<br>Factor |
|-----------------|---------------------------------------------|-----------------------------------------|-----------------|------------------------------|-----------------|
| Wheat           | 1400                                        | 865                                     | 39.2            | 3.61                         | 1.07            |
| Rice            | 1457                                        | 905                                     | 37.9            | 2.72                         | 1.04            |
| Millet          | 1180                                        | 727                                     | 38.4            | 1.99                         | 1.07            |
| Polyethylene    | 922                                         | 592                                     | 35.8            | 3.43                         | 1.02            |
| Corn            | 1342                                        | 743                                     | 44.6            | 7.26                         | 1.50            |
| Polystyrene     | 1058                                        | 641                                     | 39.4            | 1.56                         | 1.14            |
| Barley          | 1279                                        | 725                                     | 43.4            | 3.70                         | 1.14            |
| Flaxseeds       | 1129                                        | 703                                     | 37.8            | 2.09                         | 1.05            |

#### Solution

There are different ways to approach this problem using Python. The approach here relates well to the methods for continuous tables. First, we define the material and property names. Then, the table entries.

```
import numpy as np

Matl = np.array(['Wheat', 'Rice', 'Millet', 'Polyethylene',
                 'Corn', 'Polystyrene', 'Barley', 'Flaxseeds'])
Prop = np.array(['Absolute Density', 'Bulk Density', 'Percent Void',
```



```

        'Particle Diameter', 'Shape Factor'])
TableData = np.array([[1400., 865., 39.2, 3.61, 1.07],
                      [1457., 905., 37.9, 2.72, 1.04],
                      [1180., 727., 38.4, 1.99, 1.07],
                      [ 922., 592., 35.8, 3.43, 1.02],
                      [1342., 743., 44.6, 7.26, 1.50],
                      [1058., 641., 39.4, 1.56, 1.14],
                      [1279., 725., 43.4, 3.70, 1.14],
                      [1129., 703., 37.8, 2.09, 1.05]])

```

Note: For longer tables, it may be useful to read in the data table information from an external text file.

Next, we program the lookup function:

```

def TableLookup2(Row,Col,RowNames,ColNames,TableData):
    """
    Function for lookup in a two-dimensional table.
    Input:
        Row = name in row array
        Col = name in column array
        RowNames = row array
        ColNames = column array
        TableData = two-dimensional array of data in table
    Output:
        TableValue = value extracted from table
    """
    n = len(RowNames) ; m = len(ColNames)
    nt = np.size(TableData,0)
    mt = np.size(TableData,1)
    if n != nt or m != mt:
        return 'table information does not conform in size'
    ifind = False
    for i in range(n):
        if Row == RowNames[i]:
            isel = i
            ifind = True
            break
    if not ifind: return 'row name not found in table'
    jfind = False
    for j in range(m):
        if Col == ColNames[j]:
            jsel = j
            jfind = True
            break
    if not jfind: return 'column name not found in table'
    return TableData[isel,jsel]

```

Here is an example application of the function:

```

MatlName = 'Corn'
PropName = 'Bulk Density'
TableValue = TableLookup2(MatlName,PropName,Matl,Prop,TableData)
if type(TableValue) == type(str()):
    print(TableValue)
else:
    print(' {0:1} of {1:11} is
{2:7.5g}'.format(PropName,MatlName,TableValue))

Bulk Density      of Corn      is      743

```

And, when an erroneous name is used:

```

MatlName = 'Corn'
PropName = 'BulkDensity'
TableValue = TableLookup2(MatlName,PropName,Matl,Prop,TableData)

```

```

if type(TableValue) == type(str()):
    print(TableValue)
else:
    print(' {0:1} of {1:11} is
{2:7.5g} '.format(PropName,MatlName,TableValue))

column name not found in table

```

To conclude this example, we note that, if there were multiple entries in the table for whatever reason, this function returns the first entry.

Now, back to continuous tables, a more relevant topic for this chapter. We will look at one-way tables first where there is a single independent and dependent variable. Considering again the data

Page 424

in [Table 17.1](#), one way to approach table lookup would be to have a Python function whereby you provide the desired independent variable value, for example, 75 °C, and the two pairs of temperature and density at table values of 50 °C and 100 °C; then the function would compute a simple linear interpolation and return the estimate. This way presents the challenge of reading the table manually before each interpolation. A more general approach, similar to the `TableLookup2` function in [Example 18.2](#), is to provide the independent variable value and arrays of the independent and dependent values and have the function find the interval in the table that contains the independent variable value.

So, the Python function would perform two tasks: first, search in the table to find the two values of the independent variable that bracket the provided value; second, carry out the linear interpolation calculation and return the result. Of course, the function should check to determine whether the provided value is within the range of the table, providing an error message return if it is not and, if it coincides with a table value, no interpolation is necessary.

This approach requires that data in the table be arranged so that the independent or index variable is ordered, usually ascending. This is a typical arrangement in most cases. There are two common ways to find the appropriate interval in the table: a sequential search and a binary search. As the name implies, a *sequential search* involves comparing the desired value of the independent variable with the values in the input array until the appropriate interval is found. For data in ascending order, which is most common, the search through the input array proceeds until a value is found which is greater than the provided value. This then locates the larger value for the interval, and the smaller value is the one before it. If an exact match is found, the interpolation calculation can be avoided.

Here is a Python function that accomplished the sequential search and linear interpolation:

```

import numpy as np
def TableLookup(x,y,xx):
    n = len(x)
    if n != len(y): return 'input arrays must be the same length'
    if xx < x[0] or xx > x[n-1]:
        return 'input value out of range of table'
    for i in range(n):
        if xx == x[i]: # check for an exact match
            return y[i]
        elif x[i] > xx: # check for upper interval
            i2 = i
            break
    xint = (xx-x[i2-1])/(x[i2]-x[i2-1])*(y[i2]-y[i2-1])+y[i2-1]
    return xint

```

We can test this function several times with the data from [Table 17.1](#):

```

T = np.array([-40, 0., 20., 50., 100., 150., 200., 250.,
              300., 400., 500.])
rho = np.array([1.52, 1.29, 1.20, 1.09, 0.946, 0.935,
                0.746, 0.675, 0.616, 0.525, 0.457])

Tx = 350.
rhox = TableLookup(T,rho,Tx)
print(rhox)

0.5705

```

The results for an exact match are

```

Tx = -40.
rhox = TableLookup(T,rho,Tx)
print(rhox)

1.52

```

and for an erroneous entry:

---

Page 425

---

```

Tx = -501.
rhox = TableLookup(T,rho,Tx)
print(rhox)

input value out of range of table

```

For situations where the data table is lengthy, perhaps thousands of entries, the sequential sort is inefficient because it must search through all the preceding points before arriving at the bracketing pair. For a one-time calculation, the sequential technique might still be suitable, but imagine if the table lookup task were imbedded in another routine with thousands of iterations. A good alternative in these cases is the *binary search*. Here is a Python function that performs a binary search followed by a linear interpolation:

```

def TableLookupBin(x,y,xx):
    n = len(x)
    if n != len(y): return 'input arrays must be the same length'
    if xx < x[0] or xx > x[n-1]:
        return 'input value out of range of table'
    iL = 0 ; iU = n-1
    while True:
        if iU - iL <= 1: break # exit when the subscript index is 1
        iM = int((iL+iU)/2) # compute the midpoint index
        if x[iM] == xx: # check for a match
            return y[iM]
        elif x[iM] < xx: # adjust upper or lower index
            iL = iM
        else:
            iU = iM
    xint = (xx-x[iL])/(x[iU]-x[iL])*(y[iU]-y[iL])+y[iL]
    return xint

```

The approach here is similar to that of the bisection method for root location. The index at the midpoint,  $i_M$ , is computed as the average of the first or lower index,  $i_L = 0$ , and the last or upper index,  $i_U = n-1$ . The  $xx$  value is then compared with the value from the  $x$  array at the midpoint,  $x_M$ , to assess whether it is a match or in the upper or lower half of the array. Depending on where it lies, either the upper or lower index takes on the value of the midpoint index. This process is repeated until the difference between the upper and lower index is less than or equal to zero. At this point, the lower index lies at the lower bound of the interval containing  $xx$ , the upper index is the upper bound, and the linear interpolation is calculated.

Similar code as before produces the same interpolated result:

```

T = np.array([-40, 0., 20., 50., 100., 150., 200., 250.,
              300., 400., 500.])

```

```

rho = np.array([[1.52, 1.29, 1.20, 1.09, 0.946, 0.935,
                 0.746, 0.675, 0.616, 0.525, 0.457]])

Tx = 350.
rhox = TableLookupBin(T,rho,Tx)
print(rhox)

0.5705

```

This result can be verified by the following hand calculation:

$$f(350) = \frac{350 - 300}{400 - 300} (0.525 - 0.616) + 0.616 \cong 0.5705 \quad \checkmark$$

## 18.3 QUADRATIC SPLINES

To ensure that the  $n$ th derivatives are continuous at the knots, a spline of at least  $(n + 1)$ th order must be used. Third-order polynomials or cubic splines that ensure continuous first and second derivatives are most frequently used in practice. Although third and higher derivatives can be discontinuous when using cubic splines, they usually cannot be detected visually and consequently are ignored.

Because the derivation of cubic splines is somewhat involved, we have decided to first illustrate the concept of spline interpolation using second-order polynomials. These “quadratic splines” have continuous first derivatives at the knots. Although quadratic splines are not of practical importance, they serve nicely to demonstrate the general approach for developing higher-order splines.

Page 426

The objective in quadratic splines is to derive a second-order polynomial for each interval between data points. The polynomial for each interval can be represented generally as

$$s_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 \quad (18.5)$$

where the notation is as in  **Fig. 18.3**. For  $n$  data points ( $i = 1, 2, \dots, n$ ), there are  $n - 1$  intervals and, consequently,  $3(n - 1)$  unknown constants (the  $a$ 's,  $b$ 's, and  $c$ 's) to evaluate. Therefore,  $3(n - 1)$  equations or conditions are required to evaluate the unknowns. These can be developed as follows:

1. The function must pass through all the points. This is called a *continuity condition*. It can be expressed mathematically as

$$f_i = a_i + b_i(x_i - x_i) + c_i(x_i - x_i)^2$$

which simplifies to

$$a_i = f_i \quad (18.6)$$

Therefore, the constant in each quadratic must be equal to the value of the dependent variable at the beginning of the interval. This result can be incorporated into  **Eq. (18.5)**:

$$s_i(x) = f_i + b_i(x - x_i) + c_i(x - x_i)^2$$

Note that because we have determined one of the coefficients, the number of conditions to be evaluated has now been reduced to  $2(n - 1)$ .

2. The function values of adjacent polynomials must be equal at the knots. This condition can be written for knot  $i + 1$  as

$$f_i + b_i(x_{i+1} - x_i) + c_i(x_{i+1} - x_i)^2 = f_{i+1} + b_{i+1}(x_{i+1} - x_{i+1}) + c_{i+1}(x_{i+1} - x_{i+1})^2$$

This equation can be simplified mathematically by defining the width of the  $i$ th interval as

$$h_i = x_{i+1} - x_i$$

Thus, [Eq. \(18.7\)](#) simplifies to

$$f_i + b_i h_i + c_i h_i^2 = f_{i+1} \quad (18.8)$$

This equation can be written for the nodes,  $i = 1, \dots, n - 1$ . Since this amounts to  $n - 1$  conditions, it means that there are  $2(n - 1) - (n - 1) = n - 1$  remaining conditions.

3. The first derivatives at the interior nodes must be equal. This is an important condition, because it means that adjacent splines will be joined smoothly, rather than in the jagged fashion that we saw for the linear splines. [Equation \(18.5\)](#) can be differentiated to yield

$$s'_i(x) = b_i + 2c_i(x - x_i)$$

The equivalence of the derivatives at an interior node,  $i + 1$  can therefore be written as

$$b_i + 2c_i h_i = b_{i+1} \quad (18.9)$$

Writing this equation for all the interior nodes amounts to  $n - 2$  conditions. This means that there is  $n - 1 - (n - 2) = 1$  remaining condition. Unless we have some additional information regarding the functions or their derivatives, we must make an arbitrary choice to successfully compute the constants. Although there are a number of different choices that can be made, we select the following condition.

4. Assume that the second derivative is zero at the first point. Because the second derivative of [Eq. \(18.5\)](#) is  $2c_i$ , this condition can be expressed mathematically as

$$c_1 = 0$$

The visual interpretation of this condition is that the first two points will be connected by a straight line.

## EXAMPLE 18.2

## Quadratic Splines

### Problem Statement

Fit quadratic splines to the same data employed in [Example 18.1](#) ([Table 18.1](#)). Use the results to estimate the value at  $x = 5$ .

### Solution

For the present problem, we have four data points and  $n = 3$  intervals. Therefore, after applying the continuity condition and the zero second-derivative condition, this means that

$2(4 - 1) - 1 = 5$  conditions are required. [Equation \(18.8\)](#) is written for  $i = 1$  through 3 (with  $c_1 = 0$ ) to give

$$\begin{aligned}f_1 + b_1 h_1 &= f_2 \\f_2 + b_2 h_2 + c_2 h_2^2 &= f_3 \\f_3 + b_3 h_3 + c_3 h_3^2 &= f_4\end{aligned}$$

Continuity of derivatives, [Eq. \(18.9\)](#), creates an additional  $3 - 1 = 2$  conditions (again, recall that  $c_1 = 0$ ):

$$\begin{aligned}b_1 &= b_2 \\b_2 + 2c_2 h_2 &= b_3\end{aligned}$$

The necessary function and interval width values are

$$\begin{aligned}f_1 &= 2.5 & h_1 &= 4.5 - 3.0 = 1.5 \\f_2 &= 1.0 & h_2 &= 7.0 - 4.5 = 2.5 \\f_3 &= 2.5 & h_3 &= 9.0 - 7.0 = 2.0 \\f_4 &= 0.5\end{aligned}$$

These values can be substituted into the conditions which can be expressed in matrix form as

$$\begin{bmatrix} 1.5 & 0 & 0 & 0 & 0 \\ 0 & 2.5 & 6.25 & 0 & 0 \\ 0 & 0 & 0 & 2 & 4 \\ 1 & -1 & 0 & 0 & 0 \\ 0 & 1 & 5 & -1 & 0 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ c_2 \\ b_3 \\ c_3 \end{bmatrix} = \begin{bmatrix} -1.5 \\ 1.5 \\ -2 \\ 0 \\ 0 \end{bmatrix}$$

These equations can be solved using Python with the results:

$$\begin{aligned}b_1 &= -1 \\b_2 &= -1 & c_2 &= 0.64 \\b_3 &= 2.2 & c_3 &= -1.6\end{aligned}$$

These results, along with the values for the  $a$ 's [[Eq. \(18.6\)](#)], can be substituted into the original quadratic equations to develop the following quadratic splines for each interval:

$$\begin{aligned}s_1(x) &= 2.5 - (x - 3) \\s_2(x) &= 1.0 - (x - 4.5) + 0.64(x - 4.5)^2 \\s_3(x) &= 2.5 + 2.2(x - 7.0) - 1.6(x - 7.0)^2\end{aligned}$$

Because  $x = 5$  lies in the second interval, we use  $s_2$  to make the prediction,

$$s_2(5) = 1.0 - (5 - 4.5) + 0.64(5 - 4.5)^2 = 0.66$$

The total quadratic spline fit is depicted in [Fig. 18.4b](#). Notice that there are two shortcomings that detract from the fit: (1) the straight line connecting the first two points and (2) the spline for the last interval seems to swing too high. The cubic splines in the next section do not exhibit these shortcomings and, as a consequence, are better methods for spline interpolation.



## 18.4 CUBIC SPLINES

As stated at the beginning of the previous section, cubic splines are most frequently used in practice. The shortcomings of linear and quadratic splines have already been discussed. Quartic or higher-order splines are not used because they tend to exhibit the instabilities inherent in higher-order polynomials. Cubic splines are preferred because they provide the simplest representation that exhibits the desired appearance of smoothness.

The objective in cubic splines is to derive a third-order polynomial for each interval between knots as represented generally by

$$s_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 \quad (18.10)$$

Thus, for  $n$  data points ( $i = 1, 2, \dots, n$ ), there are  $n - 1$  intervals and  $4(n - 1)$  unknown coefficients to evaluate. Consequently,  $4(n - 1)$  conditions are required for their evaluation.

The first conditions are identical to those used for the quadratic case. That is, they are set up so that the functions pass through the points and that the first derivatives at the knots are equal. In addition to these, conditions are developed to ensure that the second derivatives at the knots are also equal. This greatly enhances the fit's smoothness.

After these conditions are developed, two additional conditions are required to obtain the solution. This is a much nicer outcome than occurred for quadratic splines where we needed to specify a single condition. In that case, we had to arbitrarily specify a zero second derivative for the first interval, hence making the result asymmetric. For cubic splines, we are in the advantageous position of needing two additional conditions and can, therefore, apply them evenhandedly at both ends.

For cubic splines, these last two conditions can be formulated in several ways. A very common approach is to assume that the second derivatives at the first and last knots are equal to zero. The visual interpretation of these conditions is that the function becomes a straight line at the end nodes. Specification of such an end condition leads to what is termed a “natural” spline. It is given this name because the drafting spline naturally behaves in this fashion ( Fig. 18.2).

There are a variety of other end conditions that can be specified. Two of the more popular are the clamped and the not-a-knot conditions. We will describe these options in  Sec. 18.4.2. For the following derivation, we will limit ourselves to natural splines.

Once the additional end conditions are specified, we would have the  $4(n - 1)$  conditions needed to evaluate the  $4(n - 1)$  unknown coefficients. Whereas it is certainly possible to develop cubic splines in this fashion, we will present an alternative approach that requires the solution of only  $n - 1$  equations. Further, the simultaneous equations will be tridiagonal and hence can be solved very efficiently. Although the derivation of this approach is less straightforward than for quadratic splines, the gain in efficiency is well worth the effort.

## 18.4.1 Derivation of Cubic Splines

As was the case with quadratic splines, the first condition is that the spline must pass through all the data points:

$$f_i = a_i + b_i(x_i - x_i) + c_i(x_i - x_i)^2 + d_i(x_i - x_i)^3$$

which simplifies to

$$a_i = f_i \quad (18.11)$$

Therefore, the constant in each cubic must be equal to the value of the dependent variable at the beginning of the interval. This result can be incorporated into [Eq. \(18.10\)](#):

$$s_i(x) = f_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 \quad (18.12)$$

Next, we will apply the condition that each of the cubics must join at the knots. For knot  $i + 1$ , this can be represented as

$$f_i + b_i h_i + c_i h_i^2 + d_i h_i^3 = f_{i+1} \quad (18.13)$$

where

---

Page 429

---

$$h_i = x_{i+1} - x_i$$

The first derivatives at the interior nodes must be equal. [Equation \(18.12\)](#) is differentiated to yield

$$s'_i(x) = b_i + 2c_i(x - x_i) + 3d_i(x - x_i)^2 \quad (18.14)$$

The equivalence of the derivatives at an interior node,  $i + 1$  can therefore be written as

$$b_i + 2c_i h_i + 3d_i h_i^2 = b_{i+1} \quad (18.15)$$

The second derivatives at the interior nodes must also be equal. [Equation \(18.14\)](#) can be differentiated to yield

$$s_i''(x) = 2c_i + 6d_i(x - x_i) \quad (18.16)$$

The equivalence of the second derivatives at an interior node,  $i + 1$  can therefore be written as

$$c_i + 3d_i h_i = c_{i+1} \quad (18.17)$$

Next, we can solve [Eq. \(18.17\)](#) for  $d_i$

$$d_i = \frac{c_{i+1} - c_i}{3h_i} \quad (18.18)$$

This can be substituted into [Eq. \(18.13\)](#) to give

$$f_i + b_i h_i + \frac{h_i^2}{3}(2c_i + c_{i+1}) = f_{i+1} \quad (18.19)$$

[Equation \(18.18\)](#) can also be substituted into [Eq. \(18.15\)](#) to give

$$b_{i+1} = b_i + h_i(c_i + c_{i+1}) \quad (18.20)$$

[Equation \(18.19\)](#) can be solved for

$$b_i = \frac{f_{i+1} - f_i}{h_i} - \frac{h_i}{3}(2c_i + c_{i+1}) \quad (18.21)$$

The index of this equation can be reduced by 1:

$$b_{i-1} = \frac{f_i - f_{i-1}}{h_{i-1}} - \frac{h_{i-1}}{3}(2c_{i-1} + c_i) \quad (18.22)$$

The index of [Eq. \(18.20\)](#) can also be reduced by 1:

$$b_i = b_{i-1} + h_{i-1}(c_{i-1} + c_i) \quad (18.23)$$

Equations (18.21) and (18.22) can be substituted into Eq. (18.23) and the result simplified to yield

$$h_{i-1}c_{i-1} + 2(h_{i-1} + h_i)c_i + h_ic_{i+1} = 3\frac{f_{i+1} - f_i}{h_i} - 3\frac{f_i - f_{i-1}}{h_{i-1}} \quad (18.24)$$

This equation can be made a little more concise by recognizing that the terms on the right-hand side are finite differences [recall Eq. (17.15)]:

$$f[x_i, x_j] = \frac{f_i - f_j}{x_i - x_j}$$

Therefore, Eq. (18.24) can be written as

$$h_{i-1}c_{i-1} + 2(h_{i-1} + h_i)c_i + h_ic_{i+1} = 3(f[x_{i+1}, x_i] - f[x_i, x_{i-1}]) \quad (18.25)$$

Equation (18.25) can be written for the interior knots,  $i = 2, 3, \dots, n-2$ , which results in  $n-3$  simultaneous tridiagonal equations with  $n-1$  unknown coefficients,  $c_1, c_2, \dots, c_{n-1}$ . Therefore, if we have two additional conditions, we can solve for the  $c$ 's. Once this is done, Eqs. (18.21) and (18.18) can be used to determine the remaining coefficients,  $b$  and  $d$ .

As stated previously, the two additional end conditions can be formulated in a number of ways. Page 430  
One common approach, the natural spline, assumes that the second derivatives at the end knots are equal to zero. To see how these can be integrated into the solution scheme, the second derivative at the first node [Eq. (18.16)] can be set to zero as in

$$s''_1(x_1) = 0 = 2c_1 + 6d_1(x_1 - x_1)$$

Thus, this condition amounts to setting  $c_1$  equal to zero.

The same evaluation can be made at the last node:

$$s''_{n-1}(x_n) = 0 = 2c_{n-1} + 6d_{n-1}h_{n-1} \quad (18.26)$$

Recalling Eq. (18.17), we can conveniently define an extraneous parameter  $c_n$ , in which case Eq. (18.26) becomes

$$c_{n-1} + 3d_{n-1}h_{n-1} = c_n = 0$$

Thus, to impose a zero second derivative at the last node, we set  $c_n = 0$ .

The final equations can now be written in matrix form as

$$\begin{bmatrix} 1 & & & & \\ h_1 & 2(h_1 + h_2) & h_2 & & \\ & \ddots & \ddots & \ddots & \\ & & h_{n-2} & 2(h_{n-2} + h_{n-1}) & h_{n-1} \\ & & & & 1 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_{n-1} \\ c_n \end{bmatrix} = \begin{bmatrix} 0 \\ 3(f[x_3, x_2] - f[x_2, x_1]) \\ \vdots \\ 3(f[x_n, x_{n-1}] - f[x_{n-1}, x_{n-2}]) \\ 0 \end{bmatrix} \quad (18.27)$$

As shown, the system is tridiagonal and hence efficient to solve.

## EXAMPLE 18.3

### Natural Cubic Splines

#### Problem Statement

Fit cubic splines to the same data used in [Examples 18.1](#) and [18.2](#) ([Table 18.1](#)). Utilize the results to estimate the value at  $x = 5$ .

#### Solution

The first step is to employ [Eq. \(18.27\)](#) to generate the set of simultaneous equations that will be utilized to determine the  $c$  coefficients:

$$\begin{bmatrix} 1 & & & \\ h_1 & 2(h_1 + h_2) & h_2 & \\ & h_2 & 2(h_2 + h_3) & h_3 \\ & & & 1 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 3(f[x_3, x_2] - f[x_2, x_1]) \\ 3(f[x_4, x_3] - f[x_3, x_2]) \\ 0 \end{bmatrix}$$

The necessary function and interval width values are

$$\begin{aligned} f_1 &= 2.5 & h_1 &= 4.5 - 3.0 = 1.5 \\ f_2 &= 1.0 & h_2 &= 7.0 - 4.5 = 2.5 \\ f_3 &= 2.5 & h_3 &= 9.0 - 7.0 = 2.0 \\ f_4 &= 0.5 \end{aligned}$$

These can be substituted to yield

$$\begin{bmatrix} 1 & & & \\ & 1.5 & 8 & 2.5 \\ & & 2.5 & 9 \\ & & & 1 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 4.8 \\ -4.8 \\ 0 \end{bmatrix}$$

These equations can be solved using Python with the results:

$$\begin{aligned} c_1 &= 0 & c_2 &= 0.839543726 \\ c_3 &= -0.766539924 & c_4 &= 0 \end{aligned}$$

Equations (18.21) and (18.18) can be used to compute the  $b$ 's and  $d$ 's

$$\begin{aligned} b_1 &= -1.419771863 & d_1 &= 0.186565272 \\ b_2 &= -0.160456274 & d_2 &= -0.214144487 \\ b_3 &= 0.022053232 & d_3 &= 0.127756654 \end{aligned}$$

These results, along with the values for the  $a$ 's [Eq. (18.11)], can be substituted into Eq. (18.10) to develop the following cubic splines for each interval:

Page 431

$$\begin{aligned} s_1(x) &= 2.5 - 1.419771863(x - 3) + 0.186565272(x - 3)^3 \\ s_2(x) &= 1.0 - 0.160456274(x - 4.5) + 0.839543726(x - 4.5)^2 - 0.214144487(x - 4.5)^3 \\ s_3(x) &= 2.5 + 0.022053232(x - 7.0) - 0.766539924(x - 7.0)^2 + 0.127756654(x - 7.0)^3 \end{aligned}$$

The three equations can then be employed to compute values within each interval. For example, the value at  $x = 5$ , which falls within the second interval, is calculated as

$$\begin{aligned} s_2(5) &= 1.0 - 0.160456274(5 - 4.5) + 0.839543726(5 - 4.5)^2 - 0.214144487(5 - 4.5)^3 \\ &= 1.102889734. \end{aligned}$$

The total cubic spline fit is depicted in Fig. 18.4c.

The results of Examples 18.1 through 18.3 are summarized in Fig. 18.4. Notice the progressive improvement of the fit as we move from linear to quadratic to cubic splines. We have also superimposed a cubic interpolating polynomial on Fig. 18.4c. Although the cubic spline consists of a series of third-

order curves, the resulting fit differs from that obtained using the third-order polynomial. This is due to the fact that the natural spline requires zero second derivatives at the end knots, whereas the cubic polynomial has no such constraint.

## 18.4.2 End Conditions

Although its graphical basis is appealing, the natural spline is only one of several end conditions that can be specified for splines. Two of the most popular are as follows:

- *Clamped End Condition.* This option involves specifying the first derivatives at the first and last nodes. This is sometimes called a “clamped” spline because it is what occurs when you clamp the end of a drafting spline so that it has a desired slope. For example, if zero first derivatives are specified, the spline will level off or become horizontal at the ends.
- *“Not-a-Knot” End Condition.* A third alternative is to force continuity of the third derivative at the second and the next-to-last knots. Since the spline already specifies that the function value and its first and second derivatives are equal at these knots, specifying continuous third derivatives means that the same cubic functions will apply to each of the first and last two adjacent segments. Since the first internal knots no longer represent the junction of two different cubic functions, they are no longer true knots. Hence, this case is referred to as the “not-a-knot” condition. It has the additional property that for four points, it yields the same result as is obtained using an ordinary cubic interpolating polynomial of the sort described in [Chap. 17](#).

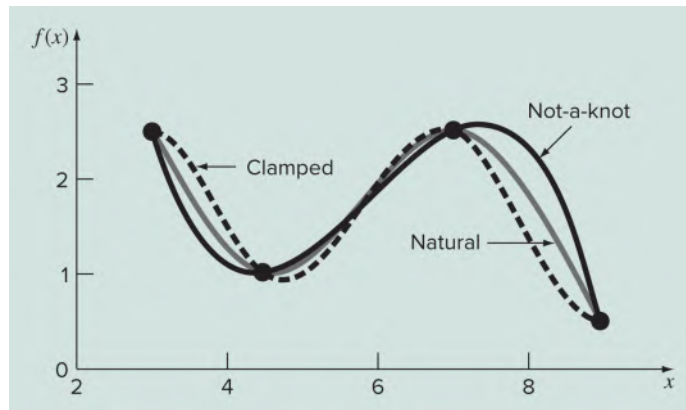
These conditions can be readily applied by using [Eq. \(18.25\)](#) for the interior knots,  $i = 2, 3, \dots, n - 2$ , and using first (1) and last equations ( $n - 1$ ) as written in [Table 18.3](#).

**TABLE 18.3** The first and last equations needed to specify some commonly use conditions for cubic splines.

| Condition                                                                                                                                  | First and Last Equations                                                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Natural                                                                                                                                    | $c_1 = 0, c_n = 0$                                                                                       |
| Clamped<br>(where $f_1'$<br>and $f_n'$ are<br>the specified<br>first<br>derivatives<br>at the first<br>and last<br>nodes,<br>respectively) | $2h_1c_1 + h_1c_2 = 3f_{[x_2, x_1]} - 3f_1'h_{n-1}c_{n-1} + 2h_{n-1}c_n = 3f_n' - 3f_{[x_n, x_{n-1}]}$   |
| Not-a-knot                                                                                                                                 | $h_2c_1 - (h_1 + h_2)c_2 + h_1c_3 = 0$<br>$h_{n-1}c_{n-2} - (h_{n-2} + h_{n-1})c_{n-1} + h_{n-2}c_n = 0$ |

 **Figure 18.5** shows a comparison of the three end conditions as applied to fit the data from **Table 18.1**. The clamped case is set up so that the derivatives at the ends are equal to zero.

**FIGURE 18.5** Comparison of the clamped (with zero first derivatives), not-a-knot, and natural splines for the data from **Table 18.1**.



As expected, the spline fit for the clamped case levels off at the ends. In contrast, the natural and not-a-knot cases follow the trend of the data points more closely. Notice how the natural spline tends to straighten out as would be expected because the second derivatives go to zero at the ends. Because it has nonzero second derivatives at the ends, the not-a-knot exhibits more curvature.



## 18.5 PIECEWISE INTERPOLATION IN PYTHON

Given the derivation in the previous section, we can compose a function in Python to compute the natural spline and carry out an interpolation. The code is presented in [Fig. 18.6](#) and depends on the use of the `tridiag` function from [Chap. 9](#).

**FIGURE 18.6** Python function, `cspine`, to implement cubic spline interpolation with natural end conditions.

```
import numpy as np

def cspine(x,y,xx):
    """
    Cubic Spline Interpolation with Natural End Conditions
    input:
        x = array of independent variable values
        y = array of dependent variable values
        xx = input value for interpolation
    output:
        yy = interpolated value of y
    """
    n = len(x)
    if len(y) != n: return 'input arrays must be the same length '
    if xx < x[0] or xx > x[n-1]: return 'input value out of range of table '
    h = np.zeros((n-1)) # x interval widths
    for i in range(n-1):
        h[i] = x[i+1] - x[i]
    df = np.zeros((n-1)) # y over h finite differences
    for i in range(n-1):
        df[i] = (y[i+1]-y[i])/h[i]
    e = np.zeros((n)) # diagonals of coefficient matrix
    f = np.zeros((n))
    f[0] = 1 ; f[n-1] = 1
    g = np.zeros((n))
    for i in range(1,n-1):
        e[i] = h[i-1]
        f[i] = 2*(h[i-1] + h[i])
        g[i] = h[i]
    const = np.zeros((n)) # constant vector
    for i in range(1,n-1):
        const[i] = 3*(df[i]-df[i-1])
    c = tridiag(e,f,g,const) # solve tridiagonal system
    b = np.zeros((n-1)) # calculate b coefficients from c
    for i in range(n-1):
        b[i] = (y[i+1]-y[i])/h[i]-h[i]/3*(2*c[i]+c[i+1])
    d = np.zeros((n-1)) # calculate d coefficients from c
    for i in range(n-1):
        d[i] = (c[i+1]-c[i])/3/h[i]
    for i in range(n): # calculate interpolation
        if xx == x[i]: # check for an exact match
            return y[i]
        elif x[i] > xx: # check for upper interval
            i2 = i-1
            break
    yy = y[i2] + b[i2]*(xx-x[i2]) + c[i2]*(xx-x[i2])**2 + d[i2]*(xx-x[i2])**3
    return yy
```

We can test the `cspline` function by first interpolating our example value of  $T = 350$  °C, and then plotting an interpolation curve for an array of 100 temperature values from the density versus temperature data of [Table 17.1](#).

```
Tx = 350.
rhox = cspline(T,rho,Tx)
print(rhox)

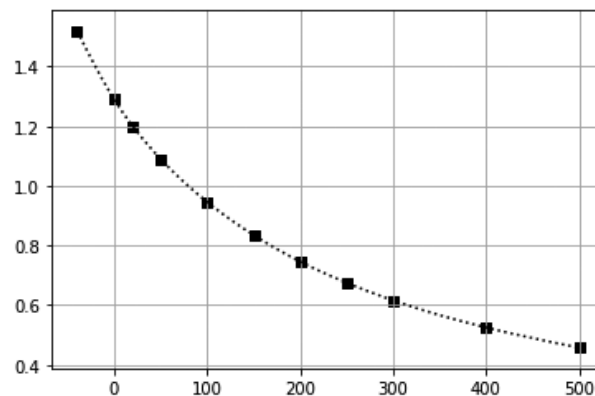
import pylab

Tplot = np.linspace(-40.,500.,100)
k = len(Tplot)
rhoplot = np.zeros((k))
for i in range(k):
    rhoplot[i] = cspline(T,rho,Tplot[i])
pylab.scatter(T,rho,c='k',marker='s')
pylab.plot(Tplot,rhoplot,c='k',ls = ':')
pylab.grid()
```

The resulting interpolation is

```
0.5665375927793199
```

and the plot is shown below.



## 18.5.1 Python SciPy interpolate Function: CubicSpline

Cubic spline interpolation can also be computed using built-in functions from the `interpolate` submodule in Python's SciPy module. This versatile function allows specification of boundary conditions, including natural, clamped, and not-a-knot. The syntax is as follows, choosing here natural end conditions:

```
from scipy.interpolate import CubicSpline
cs = CubicSpline(x,y,bc_type='natural')
```

The CS return is called a *PPoly instance* with breakpoints matching the data. To compute an interpolation, the syntax is

```
yy = cs(xx)
```

where  $xx$  is the desired  $x$ -value for interpolation and  $yy$  is the interpolated value. The  $xx$  argument can be an array of values.

## EXAMPLE 18.4

### Splines Using Python's Built-in Functions

#### Problem Statement

Runge's function is a notorious example of a function that cannot be fit well with polynomials.

Recall  [Example 17.7](#).

$$f(x) = \frac{1}{1 + 25x^2}$$

Use Python to fit a cubic spline to nine, equally spaced data points sampled from this function in the interval  $[-1, 1]$ . Compare the results for the end conditions: natural, not-a-knot, and clamped with end slopes of  $f'_1 = 1$  and  $f'_{n-1} = -4$ .

#### Solution

The nine, equally spaced points can be generated as

```
import numpy as np
from scipy.interpolate import CubicSpline

def Runge(x):
    return 1./(1. + 25.*x**2)

x = np.linspace(-1.,1.,9)
y = Runge(x)
```

Next, a more finely spaced array of  $x$ -values can be generated so that we can create an array of 50 interpolated values and function values for plotting.

```
xx = np.linspace(-1.,1)
```

Now, we can use the `CubicSpline` function for the interpolated values related to the array of  $xx$  values.

```
xx = np.linspace(-1.,1)
cs = CubicSpline(x,y,bc_type='natural')
yy = cs(xx)
```

And, we can compute function values corresponding to the  $xx$  values.

```
yR = Runge(xx)
```

This gives us the results we need to generate a plot, comparing the interpolated values to the true function values.

```
import pylab
pylab.scatter(x,y,c='k',marker='o')
pylab.plot(xx,yy,c='k',ls='--',label='Spline')
pylab.plot(xx,yR,c='k',label='Runge')
pylab.grid()
```

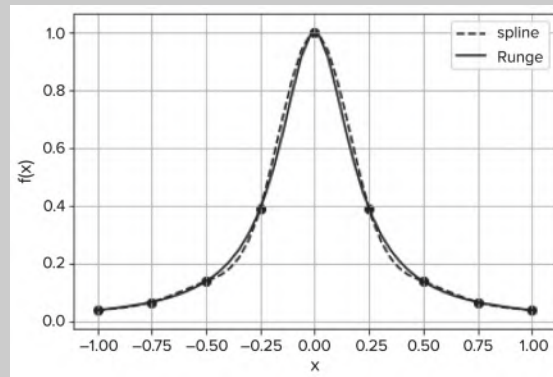
```

pylab.xlabel('x')
pylab.ylabel('f(x)')
pylab.legend()

```

The plot is shown in [Fig. 18.7](#). The natural spline does an adequate job of following Runge's function without showing the wild oscillations we saw in [Chap. 17](#). With more than nine points, the interpolations would be closer to the true function.

**FIGURE 18.7** Comparison of Runge's function with a nine-point natural spline generated with Python's `CubicSpline` function.



We can now add interpolations based on the not-a-knot and clamped end conditions and generate a single plot of the three interpolation types.

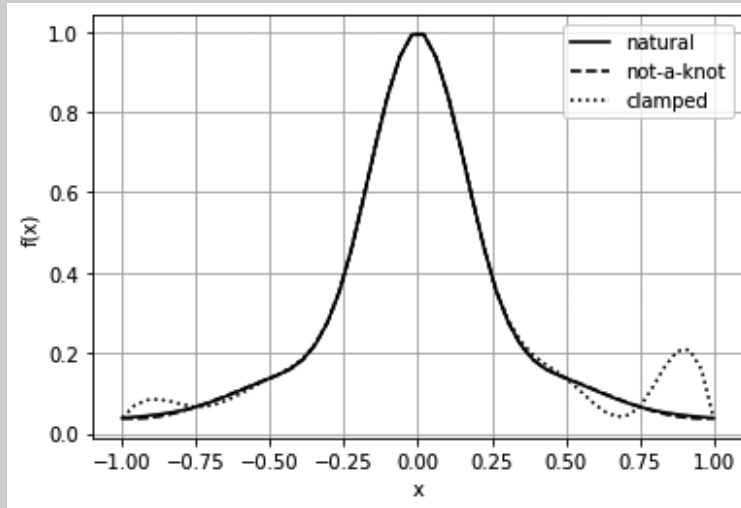
```

# not-a-knot'
csk = CubicSpline(x,y,bc_type='not-a-knot')
yyk = csk(xx)
# clamped with derivatives spec'd
csc = CubicSpline(x,y,bc_type=((1,1.),(1,-4.)))
yyc = csc(xx)
# plot natural, not-a-knot, and clamped
pylab.figure()
pylab.plot(xx,yy,c='k',label='natural')
pylab.plot(xx,yyk,c='k',ls='--',label='not-a-knot')
pylab.plot(xx,yyk,c='k',ls=':',label='clamped')
pylab.grid()
pylab.xlabel('x')
pylab.ylabel('f(x)')
pylab.legend()

```

We observe, in [Fig. 18.8](#), that there is little difference between the natural and not-a-knot end conditions; however, the clamped conditions with our specified derivative values deviate significantly at the ends of the interpolation range. Our derivative specifications here were artificial. In other situations, we might have reliable derivative values, and the clamped spline would then perform well.

**FIGURE 18.8** Comparison of the cubic spline interpolation with the three common end conditions.



## 18.5.2 Additional Python SciPy Interpolate Functions: `interp1d` and `PChipInterpolator`

The built-in `interp1d` function provides a handy means to implement a number of different types of piecewise one-dimensional interpolation. An example syntax for linear interpolation is

```
from scipy.interpolate import interp1d
f = interp1d(x,y,kind='linear')
```

The return, `f`, is a function that can be used for interpolation, as in

```
yy = f(xx)
```

Page 436

Here are a few of the various methods available with `interp1d`:

- `linear` – linear interpolation
- `nearest` – finds the nearest data point. This is sometimes called *nearest neighbor interpolation*.
- `quadratic` – quadratic spline
- `cubic` – cubic spline

Repeating the interpolation from [Section 18.5](#), but using `interp1d`, we have

```
import numpy as np
from scipy.interpolate import interp1d
T = np.array([-40., 0., 20., 50., 100., 150., 200., 250.,
              300., 400., 500.])
rho = np.array([1.52, 1.29, 1.20, 1.09, 0.946, 0.835,
                0.746, 0.675, 0.616, 0.525, 0.457])

Tx = 350.
frho = interp1d(T,rho,kind='cubic')
rhox = frho(Tx)
```

```
print(rhox)
```

with the result

```
0.5666131930705898
```

which is very close to the result from the natural spline.

An alternative interpolation function, among many, is `PchipInterpolator`, which stands for “piecewise cubic Hermite interpolation.” This method uses cubic polynomials to connect data points with continuous first derivatives. However, it differs from cubic splines in that the second derivatives are not necessarily continuous. Further, the first derivatives at the knots are not the same as those for the cubic spline. Rather, they are expressly chosen so that the interpolation is “shape preserving.” That is, the interpolated values do not tend to overshoot the data points as can sometimes happen with cubic splines.

Consequently, there are trade-offs between the `interp1d/cubic` and `PchipInterpolator` functions. The results from using cubic spline interpolation will generally appear smoother because the human eye can detect discontinuities in the second derivative. In addition, cubic splines will be more accurate if the data values are smooth. On the other hand, *pchip* interpolation has fewer overshoots and less oscillation if the data are not smooth. The next example evaluates *pchip* interpolation for the Runge function. Later on, we will see that there are specific smoothing techniques which should be applied when data have measurable random error.

## EXAMPLE 18.5

### Applying the `PchipInterpolator` to the Runge Function

#### Problem Statement

Extend  [Example 18.4](#) by using the `PchipInterpolator` function from the `interpolation` submodule of the Python SciPy module. Compare the results with the natural spline.

#### Solution

We can use the following Python script to generate interpolations for the two methods and plot them along with the Runge function curve over the range  $x = [-1, 1]$ .

```
import numpy as np
import scipy.interpolate as intr

def Runge(x):
    return 1./(1. + 25.*x**2)

x = np.linspace(-1.,1.,9)
y = Runge(x)

xx = np.linspace(-1.,1)
cs = intr.CubicSpline(x,y,bc_type='natural')
yy = cs(xx)

cchip = intr.PchipInterpolator(x,y)
ychip = cchip(xx)

yR = Runge(xx)

import pylab
```

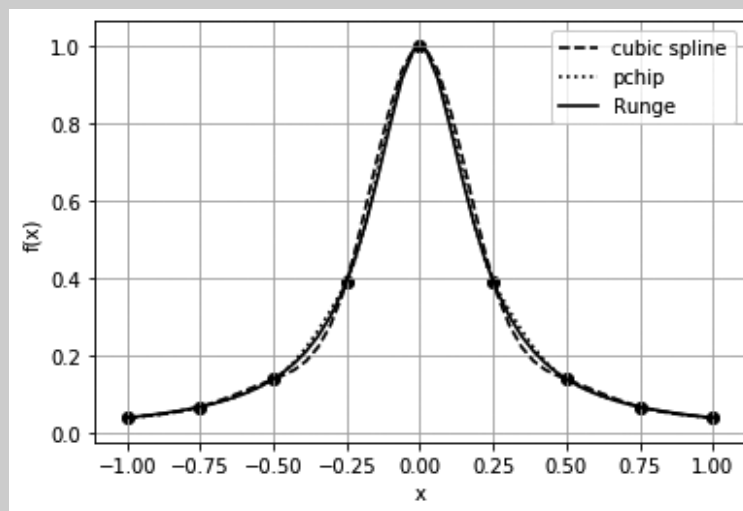
```

pylab.scatter(x,y,c='k',marker='o')
pylab.plot(xx,yy,c='k',ls='--',label='cubic spline')
pylab.plot(xx,ychip,c='k',ls=':',label='pchip')
pylab.plot(xx,yR,c='k',label='Runge')
pylab.grid()
pylab.xlabel('x')
pylab.ylabel('f(x)')
pylab.legend()

```

The results are presented in [Fig. 18.9](#). By close observation, we note that the *pchip* curve is closer to the Runge function true curve. We see that the over/undershoot of the cubic spline has been moderated by the *pchip* method.

**FIGURE 18.9** Comparison of the natural cubic spline with the pchip interpolator for the Runge function.



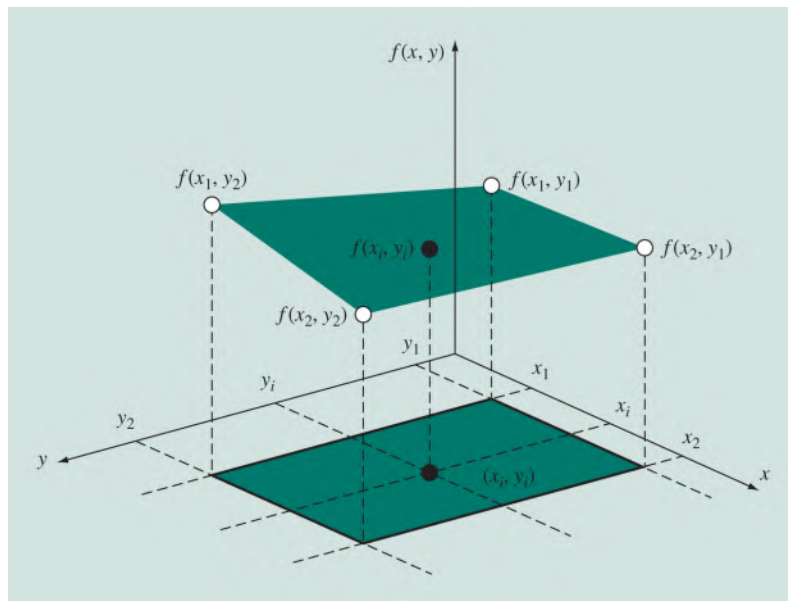
## 18.6 MULTIDIMENSIONAL INTERPOLATION

The interpolation methods for one-dimensional problems can be extended to multidimensional interpolation. In this section, we will describe the simplest case of two-dimensional interpolation in Cartesian coordinates. In addition, we will describe Python's capabilities for multidimensional interpolation.

### 18.6.1 Bilinear Interpolation

*Two-dimensional interpolation* deals with determining intermediate values for functions of two variables  $z = f(x_i, y_i)$ . As depicted in [Fig. 18.10](#), we have values at four points:  $f(x_1, y_1)$ ,  $f(x_2, y_1)$ ,  $f(x_1, y_2)$ , and  $f(x_2, y_2)$ . We want to interpolate between these points to estimate the value at an intermediate point  $f(x_i, y_i)$ . If we use a linear function, the result is a plane connecting the points as in [Fig. 18.10](#). Such functions are called *bilinear*.

**FIGURE 18.10** Graphical depiction of two-dimensional bilinear interpolation where an intermediate value (filled circle) is estimated on the basis of four given values (open circles).



A simple approach for developing the bilinear function is depicted in [Fig. 18.11](#). First, we can hold the  $y$  value fixed and apply one-dimensional linear interpolation in the  $x$  direction. Using the Lagrange form, the result at  $(x_i, y_1)$  is

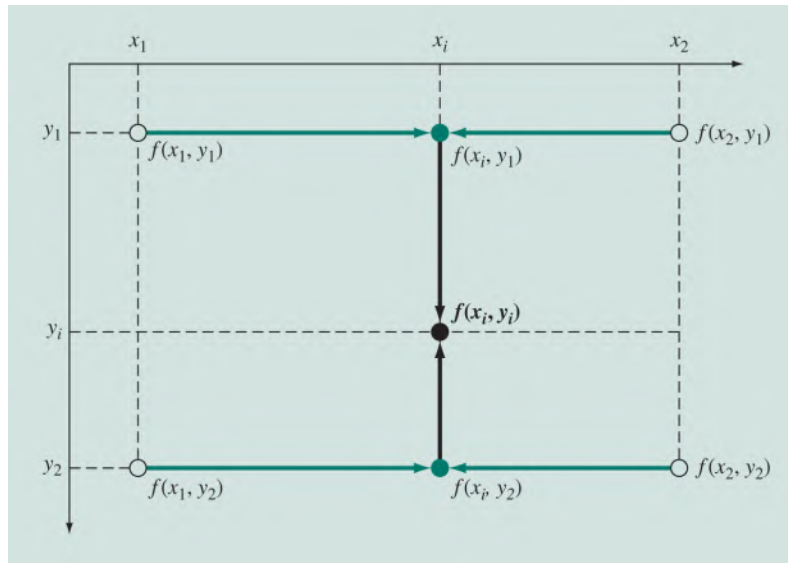


$$f(x_i, y_1) = \frac{x_i - x_2}{x_1 - x_2} f(x_1, y_1) + \frac{x_i - x_1}{x_2 - x_1} f(x_2, y_1) \quad (18.29)$$

and at  $(x_i, y_2)$  is

$$f(x_i, y_2) = \frac{x_i - x_2}{x_1 - x_2} f(x_1, y_2) + \frac{x_i - x_1}{x_2 - x_1} f(x_2, y_2) \quad (18.30)$$

**FIGURE 18.11** Two-dimensional bilinear interpolation can be implemented by first applying one-dimensional linear interpolation along the x dimension to determine values at  $x_i$ . These values can then be used to linearly interpolate along the y dimension to yield the final result at  $x_i, y_i$ .



These points can then be used to linearly interpolate along the y dimension to yield the final result:

$$f(x_i, y_i) = \frac{y_i - y_2}{y_1 - y_2} f(x_i, y_1) + \frac{y_i - y_1}{y_2 - y_1} f(x_i, y_2) \quad (18.31)$$

A single equation can be developed by substituting [Eq. \(18.29\)](#) and [Eq. \(18.30\)](#) into [Eq. \(18.31\)](#) to give

$$f(x_i, y_i) = \frac{x_i - x_2}{x_1 - x_2} \frac{y_i - y_2}{y_1 - y_2} f(x_1, y_1) + \frac{x_i - x_1}{x_2 - x_1} \frac{y_i - y_2}{y_1 - y_2} f(x_2, y_1) + \frac{x_i - x_2}{x_1 - x_2} \frac{y_i - y_1}{y_2 - y_1} f(x_1, y_2) + \frac{x_i - x_1}{x_2 - x_1} \frac{y_i - y_1}{y_2 - y_1} f(x_2, y_2)$$

## EXAMPLE 18.6

### Bilinear Interpolation

#### Problem Statement

Suppose you have measured temperatures at a number of coordinates on the surface of a rectangular heated plate:

$$\begin{array}{ll} T(2, 1) = 60 & T(9, 1) = 57.5 \\ T(2, 6) = 55 & T(9, 6) = 70 \end{array}$$

Use bilinear interpolation to estimate the temperature at  $x_i = 5.25$  and  $y_i = 4.8$ .

#### Solution

Substituting these values into [Eq. \(18.32\)](#) gives

$$f(5.25, 4.8) = \frac{5.25 - 9}{2 - 9} \frac{4.8 - 6}{1 - 6} 60 + \frac{5.25 - 2}{9 - 2} \frac{4.8 - 6}{1 - 6} 57.5 + \frac{5.25 - 9}{2 - 9} \frac{4.8 - 1}{6 - 1} 55 +$$

## 18.6.2 Multidimensional Interpolation in Python

The `interpolate` submodule in Python's SciPy module contains numerous functions for multidimensional interpolation<sup>1</sup>. A function that we select here to illustrate this family is `interp2d`. This function extends the `interp1d` function, illustrated before, to two dimensions. Its syntax is

```
f = interp2d(x,y,z,kind='linear')
```

There are also `cubic` and `quintic` type options, but `linear` is the default.

The `x` and `y` arguments are arrays of the two independent variables, and the `z` array is typically two-dimensional with dependent variable values corresponding to the `x` and `y` axes. A simple Python script to illustrate this function is

```
import numpy as np
from scipy.interpolate import interp2d

x = np.array([2., 9.])
y = np.array([1., 6.])
z = np.array([[60., 57.5],[55., 70.]])

f = interp2d(x,y,z)
print(f(5.25,4.8))
```

And the interpolated result for  $x = 5.25$  and  $y = 4.8$  is

[61.21428571]

## 18.7 SMOOTHING OF DATA SERIES

The methods introduced so far in this chapter are best applied when the data series are not corrupted by experimental error/noise and are smooth in appearance. This is often the case when data are collected under carefully controlled laboratory conditions. Many physical and chemical property tables adhere to this requirement. When a data series exhibits non-smooth random behavior, we need another approach for interpolation or prediction of intermediate values. In [Chaps. 14](#) and [15](#), regression illustrated one method of dealing with noisy data. In this section, we will illustrate another useful approach, *data smoothing*.

The basis of regression is the assumption that there is an underlying model that represents the data series. In some cases, we can postulate a model based on fundamental principles and then use regression to validate the model and estimate its parameters. However, when we use a purely empirical model, we are stretching the assumption that this truly represents the underlying process that produces the data. In these instances, we might better employ smoothing techniques.

Page 440

In this section, we will consider two common smoothing methods. First, we will extend the application of cubic splines to accomplish smoothing. Second, we will introduce LOESS smoothing as a viable alternative.

### 18.7.1 Cubic Spline Smoothing

In order to accomplish smoothing with cubic splines, we must relax the requirement that the cubic equation satisfy continuity with the noisy data points,  $\{x_i, y_i, i = 0, \dots, n\}$ . The smoothing spline function,  $s_i(x)$ , is instead chosen to minimize a “smoothing” objective function,

$$L = \lambda \sum_{i=0}^n \left[ \frac{y_i - s_i(x_i)}{\sigma_i} \right]^2 + (1 - \lambda) \sum_{i=0}^{n-1} \int_{x_i}^{x_{i+1}} [s''(x)]^2 dx \quad (18.33)$$

where, recalling

$$s_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 \quad (18.34)$$

The first term of [Eq. \(18.33\)](#) becomes small as the spline function meets the continuity requirement of interpolating cubic splines, and likewise gets larger as the function departs from the data to achieve smoothness. The second term is small when the overall spline function is smooth and becomes large when the segment splines exhibit roughness to meet continuity. The lambda parameter,  $\lambda$ , is for tuning the amount of smoothing. When  $\lambda = 1$ , there is no smoothing, and we have the interpolating cubic spline. As  $\lambda \rightarrow 0$ , the smoothing is extreme. The  $\sigma_i$  values are generally estimates of the standard deviation of the  $y_i$  series, and it is common to use a single estimate,  $\sigma$ .

Without presenting the details of the derivation here<sup>4</sup>, the strategy is to solve for the coefficients of the spline cubic polynomials, minimizing the objective function. To do this, we first express the solution for the smoothing spline functions as a tridiagonal system of linear equations. Here, the  $a$  parameter values are no longer equal to the  $y_i$  values, as they were with the interpolating spline, because of the departure from continuity, so they must be determined another way.

$$\begin{bmatrix} p_1 & h_1 & 0 & \cdots & 0 & 0 \\ h_1 & p_2 & h_2 & \cdots & 0 & 0 \\ 0 & h_2 & p_3 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & p_{n-2} & h_{n-2} \\ 0 & 0 & 0 & \cdots & h_{n-2} & p_{n-1} \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ \vdots \\ c_{n-2} \\ c_{n-1} \end{bmatrix} = \begin{bmatrix} r_0 & f_1 & r_1 & 0 & \cdots & 0 & 0 \\ 0 & r_1 & f_2 & r_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & r_{n-2} & 0 \\ 0 & 0 & 0 & 0 & \cdots & f_{n-1} & r_{n-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_4 \\ \vdots \\ a_{n-1} \\ a_n \end{bmatrix}$$

where  $p_i = 2(h_{i-1} - h_i)$ ,  $r_i = 3/h_i$ ,  $f_i = -3(1/h_{i-1} + 1/h_i)$ . With appropriate definitions of matrices, these equations can be written in compact form as

$$\mathbf{R}\mathbf{c} = \mathbf{Q}'\mathbf{a}$$

By incorporating this relationship into  $L$  and differentiating with respect to the  $\mathbf{a}$  vector, we are able to derive a set of linear equations to solve for  $\mathbf{c}$ ,

$$(\mu\mathbf{Q}'\Sigma\mathbf{Q} + \mathbf{R})\mathbf{c} = \mathbf{Q}'\mathbf{y}$$

where

$$\mu = \frac{2(1-\lambda)}{3\lambda} \text{ and } \Sigma = \begin{bmatrix} \sigma_0 & 0 & 0 & 0 \\ 0 & \sigma_1 & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \sigma_n \end{bmatrix}$$

Given the  $\mathbf{c}$  values, we can then solve for the  $\mathbf{a}$ 's with

$$\mathbf{a} = \mathbf{y} - \mu\Sigma\mathbf{Q}\mathbf{c}$$

Noting from the natural end conditions that  $c_0 = c_n = 0$ , the polynomials' **d** and **b** coefficients are computed using

$$d_i = \frac{c_{i+1} - c_i}{3h_i} \quad \text{and} \quad b_i = \frac{a_{i+1} - a_i}{h_i} - \frac{1}{3}(c_{i+1} - 2c_i)h_i \quad \text{for } i = 0, \dots, n-1$$

We then have all the coefficients for the polynomials

$$s_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 \quad i = 0, \dots, n-1$$

A `csplinesm` function coded in Python is presented in  **Fig. 18.12**. The input arguments to the function are the **x** and **y** arrays, an **xx** value for a specific interpolation, the **λ** value, **lam**, and a standard deviation estimate, **sdest**. The function returns an interpolated **yy** value corresponding to the **xx** input and the coefficient arrays for the set of cubic spline smoothing polynomials. The latter can be used for subsequent interpolations outside the function without having to invoke the function and all its calculations, each time an interpolation is desired for a given set of **x** and **y** data.

**FIGURE 18.12** A Python function, `csplinesm`, for smoothing cubic splines.

```
import numpy as np

def csplinesm(x,y,xx,lam,sdest):
    mu = 2*(1-lam)/lam/3
    m = len(x)
    if len(y) != m: return 'x and y arrays must be the same length'
    n = m - 1
    h = np.zeros((n)) # compute x intervals
    for i in range(n):
        h[i] = x[i+1] - x[i]
    # compute elements for R and Q' matrices
    p = np.zeros((n))
    for i in range(1,n):
        p[i] = 2*(-h[i]+h[i-1])
    r = np.zeros((n))
    for i in range((n)):
        r[i] = 3/h[i]
    f = np.zeros((n))
    for i in range(1,n):
        f[i] = - 3*(1/h[i-1]+1/h[i])
    # compose R matrix
    R = np.zeros((n-1,n-1))
    for i in range(n-2):
        R[i,i] = p[i]
        R[i+1,i] = h[i]
        R[i,i+1] = h[i]
    R[n-2,n-2] = p[n-2]
    Qp = np.zeros((n-1,n+1))
    for i in range(n-2):
        Qp[i,i] = r[i]
        Qp[i,i+1] = f[i+1]
        Qp[i,i+2] = r[i+1]
    Qp[n-2,n-1] = f[n-1]
```

```

Qp[n-2,n-2] = r[n-2]
Qp[n-2,n] = r[n-1]
# Q from Q'
Q = np.transpose(Qp)
# diagonal matrix of sigma estimates
SigMat = np.zeros((n+1,n+1))
for i in range(n+1):
    SigMat[i,i] = sdest
# set up linear equations to solve for c
Qt = np.dot(Qp,SigMat)
Qcoef = np.dot(Qt,Q)
Qcoef = Qcoef*mu + R
const = np.dot(Qp,y)
# solve for c
c = np.linalg.solve(Qcoef,const)
# solve for d
SigQ = np.dot(SigMat,Q)
yc = np.dot(SigQ,c)
yc = yc*mu
a = y - yc
# solve for d and b
d = np.zeros((n))
b = np.zeros((n))
cx = np.zeros((n))
for i in range(1,n-1):
    cx[i] = c[i-1]
for i in range(0,n-1):
    d[i] = (cx[i+1]-cx[i])/3/h[i]
    b[i] = (a[i+1]-a[i]/h[i])
# compute interpolation
for i in range(n): # calculate interpolation
    if xx == x[i]: # check for an exact match
        return y[i],a,b,c,d
    elif x[i] > xx: # check for upper interval
        i2 = i-1
        break
yy = a[i2] + b[i2]*(xx-x[i2]) + c[i2]*(xx-x[i2])**2 + d[i2]*(xx-x[i2])**3
# return interpolated value and spline coefficient arrays
return yy,a,b,c,d

```

We will test the function with a noisy set of chemical concentration measurements. These are listed in [Appendix B](#), and we have them available in a text file for input to our Python script.

```

conc = np.loadtxt(fname='ConcentrationData.txt')
n = len(conc)
samp = np.zeros((n))
for i in range(n):
    samp[i] = float(i)
xx = 100.5
lam = 0.001
sconc = np.std(conc)

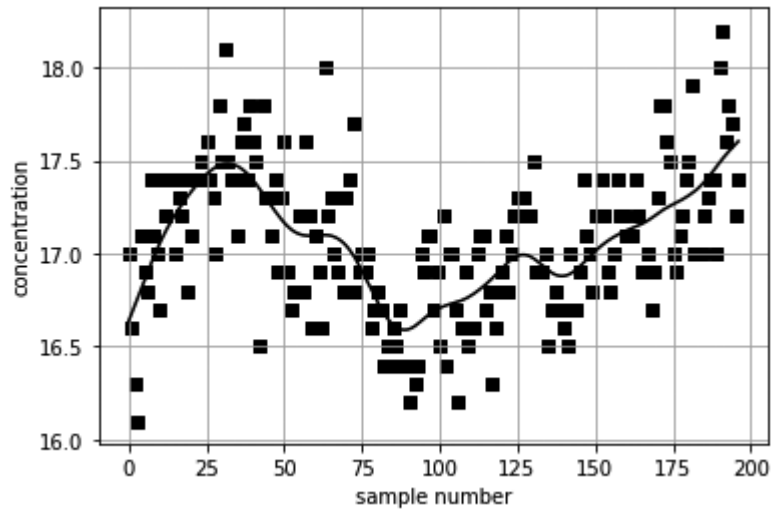
concint,a,b,c,d = csplinesm(samp,conc,xx,lam,sconc)
print(concint)

import pylab
pylab.scatter(samp,conc,c='k',marker='s')
pylab.plot(samp,a,c='k')
pylab.grid()
pylab.xlabel('sample number')
pylab.ylabel('concentration')

```

The script requests an interpolation for an  $x$ -value of 100.5 and then plots the smoothed values corresponding to the data values. The standard deviation estimate is calculated from the data, and the lambda weighting factor has been set very small, at 0.001, to provide a significant amount of smoothing.

The lambda factor can be tuned to provide the amount of smoothing desired. For this example, the interpolated value at  $x = 100.5$  is 16.710944281895063, and a plot of the data along with the smoothing spline is



Page 443

As with many numerical methods in Python, there are resources available to carry out cubic spline smoothing. One of these is the `csaps` function available from <https://pypi.org/project/csaps/>.

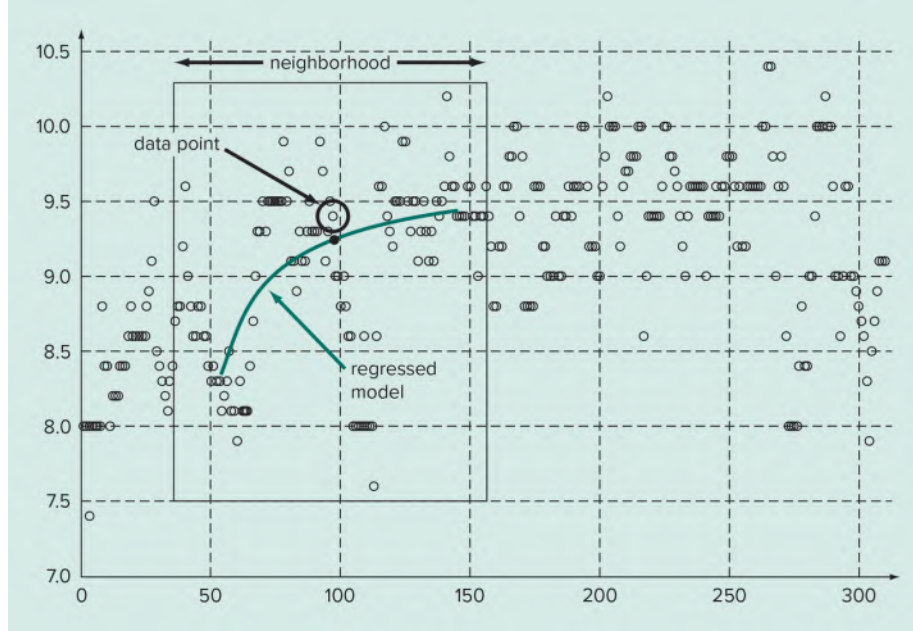
## 18.7.2 LOESS Smoothing

The basis for the *LOESS*<sup>3</sup> method is an intuitively attractive smoothing technique. One fits a low-order polynomial, typically a quadratic, to the data, but only in the neighborhood of the interpolation. The concept is illustrated in [Fig. 18.13](#).

Page 444

**FIGURE 18.13** LOESS method concept.





An additional feature is that weighted least squares is applied where the points closer to the interpolation location,  $x_0$ , have more influence on the fit than those farther away. The width of the neighborhood is governed by a parameter,  $\alpha$ , the fraction of the entire data set to be used for the fit. The closer  $\alpha$  is to one, the greater the smoothing. Apart from the order of the polynomial, it is the only tunable parameter.

For weighting of the data in the neighborhood, a *tricube function* is typically used. It is defined as

$$W(z) = (1 - z^3)^3 \quad 0 \leq z \leq 1$$

$$W(-z) = W(z)$$

and depicted in  Fig. 18.14. For a quadratic polynomial, the least squares criterion to be minimized is

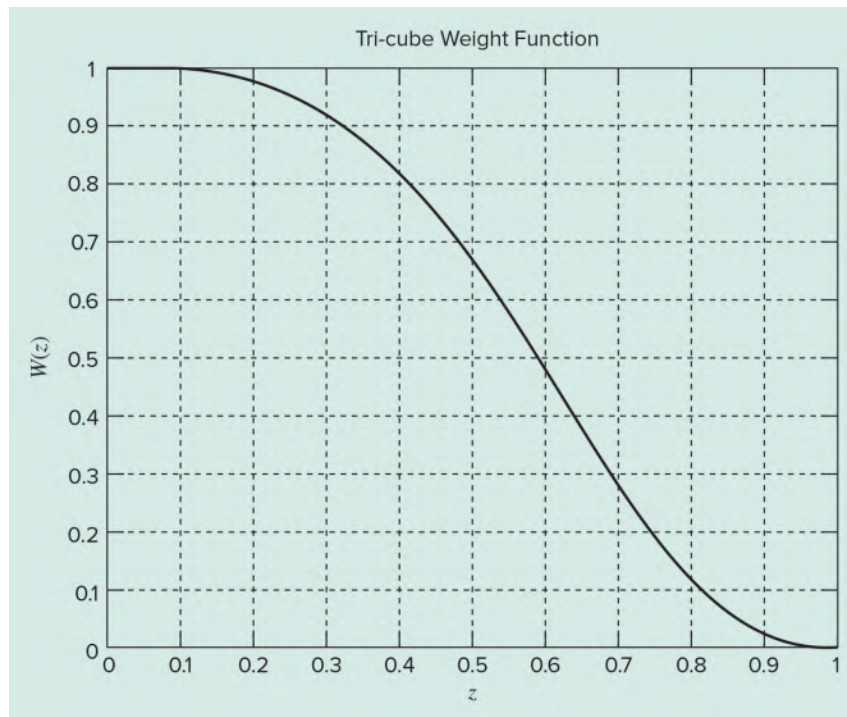
$$L = \sum_{i=1}^k w_i(x_0) (y_i - (b_0 + b_1 x_i + b_2 x_i^2))^2$$

where  $k$  is the greatest integer  $\geq \alpha \cdot n$  and

$$w_i(x_0) = W\left(\frac{|x_i - x_0|}{\Delta_k(x_0)}\right)$$

with  $\Delta_k(x_0)$  being the distance from  $x_0$  to the  $k$ th nearest neighbor.

**FIGURE 18.14** Tri-cube weight function used with least squares fit in the LOESS method.



The weighted least squares solution can then be described in vector-matrix form as

$$L = \mathbf{e}^T \mathbf{W} \mathbf{e} \quad \text{where} \quad \mathbf{e} = \mathbf{y} - \mathbf{X}\mathbf{b} \quad \text{and} \quad \mathbf{W} = \text{diag}(\mathbf{w})$$

Then,  $\partial L / \partial \mathbf{b} \Rightarrow 0$  yields the weighted normal equations that can be solved for the  $\mathbf{b}$  values. The resulting polynomial is employed to compute the smoothed value,  $\hat{y}$ , given the  $x$  value,  $x_0$ .

 **Figure 18.15** presents a Python function, `loess`, that computes smoothed  $y$  values for a given data set,  $\{x_i, y_i, i = 1, \dots, n\}$ . We can apply this function to the same concentration data from the previous section using an alpha value of 0.35. The code and results are shown below, including a figure similar to that of the previous section.

**FIGURE 18.15** Python code for function `loess`.

```
import numpy as np

def loess(x0,x,y,alpha):
    """
    loess smoothing applied to the series [x,y]
    Input:
        x0 = independent variable value for interpolation
        x = independent variable series
        y = dependent variable series
        alpha = smoothing parameter between 0 and 1
```

Output:

```
ys = smoothed values of y
"""
n = len(x)
if n != len(y): return 'x and y series must be of same length'
k = int(alpha*n) # how many data for loess?
# compute distance array
dist = np.zeros((n))
for i in range(n):
    dist[i] = abs(x0-x[i])
# sort the distance array
# with an accompanying index array
sdist,ind = sortdist(dist,n)
# extract nearest x and y data to x0
Nx = np.zeros((k))
Ny = np.zeros((k))
for i in range(k):
    Nx[i] = x[ind[i]]
    Ny[i] = y[ind[i]]
# set value for weight calculation
# based on farthest distance
delx0 = sdist[k]
# zero out the rest of the sdist array
for i in range(k,n):
    sdist[i] = 0
# compute the weights and the diagonal
# weight matrix
z = np.zeros((k))
w = np.zeros((k))
for i in range(k):
    z[i] = sdist[i]/delx0
    w[i] = (1-z[i]**3)**3
W = np.zeros((k,k))
for i in range((k)):
    W[i,i] = w[i]
# build the X matrix
X = np.zeros((k,3))
for i in range(k):
    X[i,0] = Nx[i]**2
    X[i,1] = Nx[i]
    X[i,2] = 1
# formulate the coefficient matrix
# and constant vector
# for the normal equations
Xt = np.transpose(X)
XtW = np.dot(Xt,W)
Xcoef = np.dot(XtW,X)
const = np.dot(XtW,Ny)
# solve the normal equations
b = np.linalg.solve(Xcoef,const)
# use the coefficients for the interpolation
yhat0 = b[0]*x0**2 + b[1]*x0 + b[2]
return yhat0

def sortdist(dist,n):
    """
    function to sort the dist array
    into the sdist array
    using an insertion sort
    the original indices of the distances
    are retained in ind
    """
    pos = np.zeros((n),dtype=np.bool)
    ind = np.zeros((n),dtype=np.int16)
    sdist = np.zeros((n))
    for i in range(n):
```

```

        pos[j] = False
    maxdist = np.max(dist)
    for i in range(n):
        mindist = maxdist
        for j in range(n):
            if dist[j] <= mindist and not pos[j]:
                mindist = dist[j]
                minloc = j
        sdist[i] = mindist
        ind[i] = minloc
        pos[minloc] = True
    return sdist, ind

```

```

conc = np.loadtxt(fname='ConcentrationData.txt')
n = len(conc)
samp = np.zeros((n))
for i in range(n):
    samp[i] = float(i)
xx = 100.5
alpha = 0.35

concint = loess(xx, samp, conc, alpha)
print(concint)

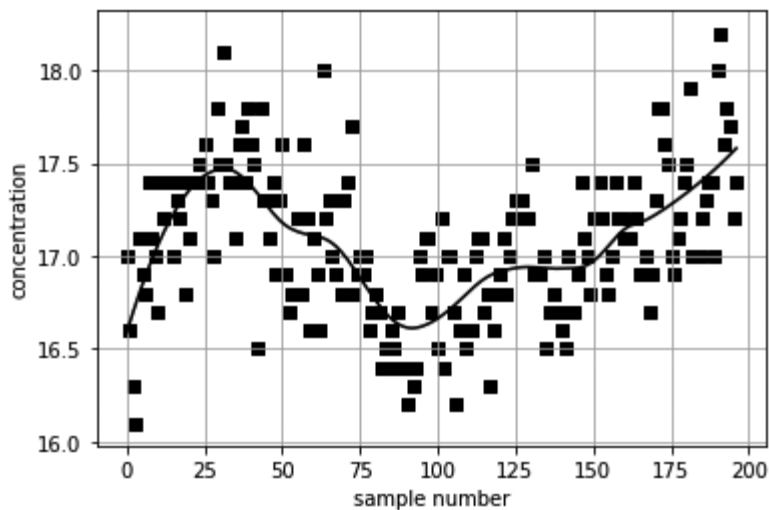
concs = np.zeros((n))
for i in range(n):
    concs[i] = loess(samp[i], samp, conc, alpha)

import pylab
pylab.scatter(samp, conc, c='k', marker='s')
pylab.plot(samp, concs, c='k')
pylab.grid()
pylab.xlabel('sample number')
pylab.ylabel('concentration')

```

and here are the results:

16.66655022336913



We note that the `statsmodels` module for Python provides a `lowess` function.

Now that we have introduced two popular smoothing methods, questions arise. Which is best? We have noted similar results between cubic spline and LOESS smoothing. When considering smoothing of an entire data series, cubic spline smoothing requires fewer calculations. One solves a single set of linear equations to obtain a set of polynomial coefficients that can then be used for multiple predictions throughout the range of the data. LOESS smoothing requires the solution of a smaller set of normal equations, but these equations must be solved for each interpolated point as the neighborhood moves. When the number of interpolations is few, LOESS is attractive. Given the speed of modern computers, one may not observe significant differences in execution time between the two techniques.

When to use smoothing instead of empirical regression? Using, for example, polynomial regression is a common way to develop models that are then used to predict interpolated values. Often, there is no strong claim of an underlying “true” model for the process producing the data. In many cases, regression of empirical models is inadequate. It is useful to consider smoothing techniques as an alternative to regression, but not necessarily always preferred. And, for smooth data, interpolation techniques such as cubic splines should be considered.

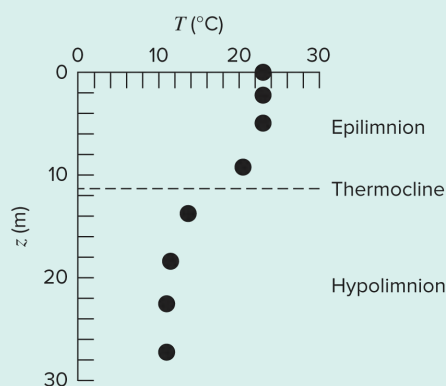
## 18.8 CASE STUDY

### HEAT TRANSFER IN A LAKE

#### Background

Lakes in the temperate zone can become thermally stratified during the summer months. As depicted in [Fig. 18.16](#), warm, buoyant water near the lake surface overlies colder, denser bottom water. Such stratification effectively divides the lake vertically into two layers: the *epilimnion* and the *hypolimnion*, separated by a plane called the *thermocline*.

**FIGURE 18.16** Temperature versus depth during the summer for Platte Lake.



Thermal stratification has great significance for environmental engineers and scientists studying such systems. In particular, the thermocline greatly diminishes mixing between the two layers. As a result, decomposition of organic matter can lead to severe depletion of oxygen in the isolated bottom waters. It has been hypothesized that global warming will intensify this problem in the coming decades.

The location of the thermocline can be defined as the inflection point of the temperature versus depth curve; that is, the point at which  $d^2T/dz^2 = 0$ . It is also the point at which the absolute value of the derivative or gradient is at a maximum.

The temperature gradient is important in its own right because it can be used in conjunction with Fourier's law to determine the heat flux across the thermocline:

$$J = -D\rho c_P \frac{dT}{dz} \quad (18.35)$$

where  $J$  = heat flux [ $\text{J}/(\text{m}^2 \cdot \text{s})$ ],  $D$  = an eddy diffusion coefficient ( $\text{m}^2/\text{s}$ ),  $\rho$  = density ( $\cong 1000 \text{ kg}/\text{m}^3$ ), and  $c_P$  = heat capacity [ $\cong 4186 \text{ J}/(\text{kg} \cdot ^\circ\text{C})$ ].

## Problem Statement

Temperature data versus depth for Platte Lake in Michigan are presented in [Table 18.4](#). Using the natural cubic spline, determine the thermocline depth and temperature gradient for this lake. These data can also be used to determine the heat flux given  $D = 10^{-6} \text{ m}^2/\text{s}$ .

**TABLE 18.4** Temperature measurements versus depth during the summer months for Platte Lake.

| $z, \text{ m}$      | 0    | 2.3  | 4.9  | 9.1  | 13.7 | 18.3 | 22.9 | 27.2 |
|---------------------|------|------|------|------|------|------|------|------|
| $T, ^\circ\text{C}$ | 22.8 | 22.8 | 22.8 | 20.6 | 13.9 | 11.7 | 11.1 | 11.1 |

## Solution

In [Fig. 18.6](#), the code for a Python function, `cspline`, is presented for interpolation with cubic splines with natural end conditions (zero second derivatives at the boundaries). In order to estimate first and second derivatives, we also need the coefficients of the cubic spline polynomials. These can be obtained easily by modifying the two return statements of the `cspline` function to

```
return y[i],b,c,d
.
.
return yy,b,c,d
```

With these coefficients in hand, we can compute function, first derivative, and second derivative values from

$$\begin{aligned}
 f_i(x) &= y_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 \\
 f_i'(x) &= b_i + 2c_i(x - x_i) + 3d_i(x - x_i)^2 \\
 f_i''(x) &= 2c_i + 6d_i(x - x_i)
 \end{aligned}$$

The following Python script employs the modified `cspline` function, computes and plots the temperature and derivative profiles.

```
z = np.array([0., 2.3, 4.9, 9.1, 13.7, 18.3, 22.9, 27.2])
T = np.array([22.8, 22.8, 22.8, 20.6, 13.9, 11.7, 11.1, 11.1])

zz = 10.
TT,b,c,d = cspline(z,T,zz)

zplot = np.linspace(0.,27.2)
n = len(zplot)
m = len(z)
Tplot = np.zeros((n))
dTplot = np.zeros((n))
d2Tplot = np.zeros((n))
for i in range(n):
    for j in range(m-1):
        if z[j] > zplot[i]:
            j2 = j-1
            break
    Tplot[i] = T[j2]+b[j2]*(zplot[i]-z[j2])+c[j2]*(zplot[i]-z[j2])**2 \
+ d[j2]*(zplot[i]-z[j2])**3
    dTplot[i] = b[j2]+2*c[j2]*(zplot[i]-z[j2])+3*d[j2]*(zplot[i]-z[j2])**2
    d2Tplot[i] = 2*c[j2]+6*d[j2]*(zplot[i]-z[j2])

import matplotlib.pyplot as plt
```

```

fig = plt.figure()
ax1 = fig.add_subplot(131)
fig.subplots_adjust(wspace=0.5)
ax1.plot(Tplot,zplot,c='k')
ax1.scatter(T,z,c='k',marker='s')
ax1.set_ylim(30.,0.)
ax1.set_title('T vs. z')
ax1.set_ylabel('Depth, z in m')
ax1.set_xlabel('degC')
ax1.grid()

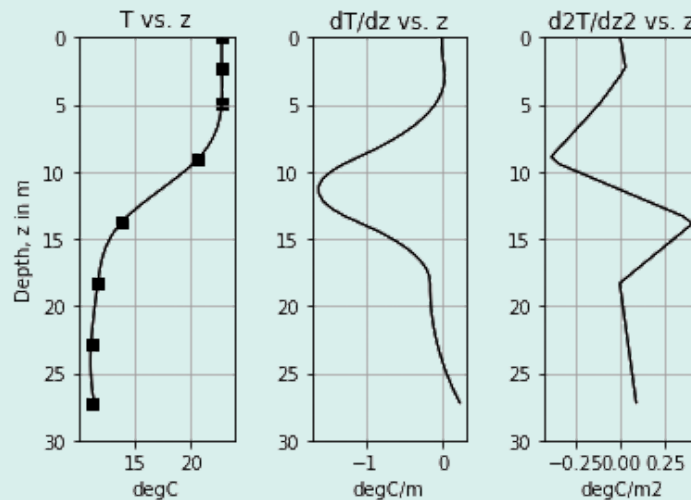
ax2 = fig.add_subplot(132)
ax2.plot(dTplot,zplot,c='k')
ax2.set_ylim(30.,0.)
ax2.set_title('dT/dz vs. z')
ax2.set_xlabel('degC/m')
ax2.grid()

ax3 = fig.add_subplot(133)
ax3.plot(d2Tplot,zplot,c='k')
ax3.set_ylim(30.,0.)
ax3.set_title('d2T/dz2 vs. z')
ax3.set_xlabel('degC/m2')
ax3.grid()

```

In [Fig. 18.17](#), the three profiles are presented. The Python code illustrates how to use the Matplotlib module and `pyplot` submodule to create vertical plots with the vertical axis scale reversed. Noting the maximum in the first derivative plot and zero-crossing of the second derivative plot, the thermocline appears to be located at a depth of about 11.5 m. We could use root-finding for the second derivative to find a zero or optimization for the first derivative to find the maximum and refine this estimate. The result would be that the thermocline is located at a depth of 11.35 m where the gradient is  $-1.61\text{ }^{\circ}\text{C/m}$ .

**FIGURE 18.17** Plots of temperature, gradient, and second derivative versus depth (m) generated by natural cubic spline interpolation.



The gradient can be used to compute the heat flux across the thermocline with [Eq. \(18.35\)](#):

$$J = -\left(1 \times 10^{-6} \frac{\text{m}^2}{\text{s}}\right) \left(1,000 \frac{\text{kg}}{\text{m}^3}\right) \left(-1.61 \frac{^{\circ}\text{C}}{\text{m}}\right) \left(\frac{86,400 \text{ s}}{\text{d}}\right) \cong 5.82 \times 10^5 \frac{\text{J}}{\text{m}^2 \cdot \text{s}}$$



The foregoing analysis demonstrates how spline interpolation can be used for engineering and scientific problem solving. However, it is also an example of numerical differentiation. As such, it illustrates how numerical approaches from different topical areas can be used in tandem for problem solving. We will be describing numerical differentiation in detail in [🔗 Chap. 21](#).

# PROBLEMS

**18.1** Given the data

|        |   |   |     |   |   |   |
|--------|---|---|-----|---|---|---|
| $x$    | 1 | 2 | 2.5 | 3 | 4 | 5 |
| $f(x)$ | 1 | 5 | 7   | 8 | 2 | 1 |

Fit these data with (a) cubic splines with natural end conditions, (b) cubic splines with not-a-knot end conditions, and (c) piecewise cubic Hermite interpolation. Present comparative plots of 50 equally spaced interpolation points over the domain,  $1 \leq x \leq 5$  and comment on your observations. Use Python capabilities as you see fit.

**18.2** A vertical, cylindrical vessel is used as a chemical reactor. The liquid in the reactor is not stirred and, as a small amount of catalyst is added at the top surface, a temperature gradient is established as the exothermic reaction proceeds from the top to the bottom. A temperature profile captured early in the reaction process is depicted in the table below.

|                 |    |     |    |     |    |     |    |
|-----------------|----|-----|----|-----|----|-----|----|
| Depth, m        | 0  | 0.5 | 1  | 1.5 | 2  | 2.5 | 3  |
| Temperature, °C | 70 | 70  | 55 | 22  | 13 | 10  | 10 |

Based on these temperatures, the reactor can be modeled as two zones separated by a strong temperature gradient or *thermocline*. The depth of the thermocline can be defined as the inflection point of the temperature-depth curve—that is, the point at which  $d^2T/dz^2 = 0$ . At this depth, the heat flux from the surface layer to the bottom layer can be computed with *Fourier's law*:

$$J = -k \frac{dT}{dz}$$

Use a clamped cubic spline fit with zero end derivatives to determine the thermocline depth. If  $k = 4.187 \text{ J}/(\text{s} \cdot \text{m} \cdot ^\circ\text{C})$ , compute the flux across this interface.

**18.3** The following is a specific version of the *humps* function:

$$f(x) = \frac{1}{(x - 0.3)^2 + 0.01} + \frac{1}{(x - 0.9)^2 + 0.04} - 6$$

The humps function exhibits both flat and steep regions over a relatively short domain for  $x$ . Here are some values that have been generated at intervals of 0.1 over the domain  $0 \leq x \leq 1$ .

|        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|
| $x$    | 0      | 0.1    | 0.2    | 0.3    | 0.4    | 0.5    |
| $f(x)$ | 5.176  | 15.471 | 45.887 | 96.500 | 47.448 | 19.000 |
| $x$    | 0.6    | 0.7    | 0.8    | 0.9    | 1      |        |
| $f(x)$ | 11.692 | 12.382 | 17.846 | 21.703 | 16.000 |        |

Fit these data with (a) a cubic spline with not-a-knot end conditions and (b) piecewise cubic Hermite interpolation. Create a plot comparing the two fits with the exact function.

**18.4** Develop a plot of a cubic spline fit of the following data with (a) natural end conditions and (b) not-a-knot end conditions. Also, (c) add to the plot a curve based on piecewise cubic Hermite interpolation (pchip). Comment on the fits.

|        |         |         |         |         |
|--------|---------|---------|---------|---------|
| $x$    | 0       | 100     | 200     | 400     |
| $f(x)$ | 0       | 0.82436 | 1.00000 | 0.73576 |
| $x$    | 600     | 800     | 1000    |         |
| $f(x)$ | 0.40601 | 0.19915 | 0.09158 |         |

**18.5** The following data are sampled from the unit step function depicted in [Fig. 18.1](#).

|        |    |      |      |     |     |   |
|--------|----|------|------|-----|-----|---|
| $x$    | -1 | -0.6 | -0.2 | 0.2 | 0.6 | 1 |
| $f(x)$ | 0  | 0    | 0    | 1   | 1   | 1 |

Fit these data with (a) a cubic spline with not-a-knot end conditions, (b) a cubic spline with clamped end conditions (zero derivative), and (c) piecewise cubic Hermite interpolation (pchip). Plot the results, comparing to the ideal step curve.

**18.6** Develop a Python function, similar to that in [Fig. 18.6](#), that implements the clamped-end cubic spline method and returns only the arrays with the cubic polynomial coefficients. Apply your function to the data in [Table 18.3](#) with zero derivative specifications. Your function should look like

```
def clmpspline(x,y,d0,dn):
    .
    .
    .
    return b,c,d
```

where d0 and dn are the derivative specifications. Note: The a coefficients are the smoothed y values because of the spline requirements. Use the `tridiag` function to solve the linear equations.

**18.7** The following data were generated with the fifth-order polynomial,

$$f(x) = 0.0185x^5 - 0.444x^4 + 3.9125x^3 - 15.456x^2 + 27.069x - 14.1$$

|        |       |       |       |       |       |       |
|--------|-------|-------|-------|-------|-------|-------|
| $x$    | 1     | 3     | 5     | 6     | 7     | 9     |
| $f(x)$ | 1.000 | 2.172 | 4.220 | 5.430 | 4.912 | 9.120 |

(a) Fit these data with a cubic spline with natural end conditions. Create a plot comparing the fit with the function. (b) Repeat (a), but use clamped end conditions where the end derivatives are set at the exact values determined by differentiating the function.

**18.8** Bessel functions often arise in advanced engineering and scientific analyses, such as the study of electromagnetic fields. These functions are usually not amenable to straightforward evaluation and, therefore, are often compiled in standard mathematical tables. For example,

|          |        |        |       |        |        |
|----------|--------|--------|-------|--------|--------|
| $x$      | 1.8    | 2      | 2.2   | 2.4    | 2.6    |
| $J_1(x)$ | 0.5815 | 0.5767 | 0.556 | 0.5202 | 0.4708 |

Estimate  $J_1(2.1)$  using (a) an interpolating polynomial, and (b) using cubic splines. Note that the true value to four significant figures is 0.5683.

**18.9** The following data define the sea-level concentration of dissolved oxygen for fresh water as a function of temperature:

|                     |        |        |       |       |       |       |
|---------------------|--------|--------|-------|-------|-------|-------|
| $T, ^\circ\text{C}$ | 0      | 8      | 16    | 24    | 32    | 40    |
| $o, \text{mg/L}$    | 14.621 | 11.843 | 9.870 | 8.418 | 7.305 | 6.413 |

Use Python to fit the data with (a) piecewise linear interpolation, (b) a fifth-order polynomial, and (c) cubic spline interpolation. Display the results graphically, and use each approach to estimate  $o(27)$ . Note that the exact result is 7.986 mg/L.

**18.10** (a) Use Python to fit a cubic spline with natural end conditions to the following data to determine  $x = 1.5$ .

|     |    |    |    |   |    |    |
|-----|----|----|----|---|----|----|
| $x$ | 0  | 2  | 4  | 7 | 10 | 12 |
| $y$ | 20 | 20 | 12 | 7 | 6  | 6  |

(b) Repeat, but with clamped end conditions as zero first derivatives.

**18.11** Runge's function is written as

$$f(x) = \frac{1}{1 + 25x^2}$$

Generate five equidistant values of this function over the interval  $[-1, 1]$ . Fit these data with (a) a fourth-order polynomial, (b) a linear spline, and (c) a cubic spline with not-a-knot end conditions. Present your results graphically on the same plot.

**18.12** Use Python to generate eight equally spaced points sampled from the function

$$f(t) = \sin^2(t)$$

over the domain  $0 \leq t \leq 2\pi$ . Fit these data using (a) cubic spline with not-a-knot end conditions, (b) cubic spline with derivative end conditions equal to the exact values calculated from the differentiated function, and (c) piecewise cubic Hermite interpolation. Present the fits on one plot along with the function and a comparative plot of the absolute error ( $| \text{approximation} - \text{true value} |$ ) for each method.

**18.13** The drag coefficient for spheres, such as sporting balls, is known to vary as a function of the Reynolds number,  $Re$ , a dimensionless quantity that represents the ratio of inertial to viscous forces:

$$Re = \frac{\rho V D}{\mu}$$

where  $\rho$  = fluid density ( $\text{kg/m}^3$ ),  $V$  = velocity (m/s),  $D$  = diameter (m), and  $\mu$  = fluid viscosity ( $\text{Pa} \cdot \text{s}$ ). Although the relationship of drag to Reynolds number is sometimes available in equation form, it is frequently tabulated from careful experimental measurements in a wind tunnel. For example, the following table provides values for a smooth spherical ball:

| $Re(\times 10^{-4})$ | 2    | 5.8   | 16.8  | 27.2 | 29.9 | 33.9 |      |
|----------------------|------|-------|-------|------|------|------|------|
| $C_D$                | 0.52 | 0.52  | 0.52  | 0.5  | 0.49 | 0.44 |      |
| $Re(\times 10^{-4})$ | 36.3 | 40    | 46    | 60   | 100  | 200  | 400  |
| $C_D$                | 0.18 | 0.074 | 0.067 | 0.08 | 0.12 | 0.16 | 0.19 |

(a) Develop a Python function that employs an appropriate interpolation method to return a value of  $C_D$  as a function of the Reynolds number. The first line of the function definition should be

```
def Drag(ReCDTable, ReIn):
```

and the last line

```
return CD
```

where `ReCDTable` is two-row array containing the table data and `ReIn` is the Reynolds number at which you want to estimate the drag. The return `CD` is the estimated drag coefficient corresponding to `ReIn`.

- (b) Write a Python script that uses the function developed in (a) to generate a labeled plot of drag force versus velocity (recall [Sec. 1.4](#)). Use the following parameter values for the script:  $D = 22 \text{ cm}$ ,  $\rho = 1.3 \text{ kg/m}^3$ , and  $\mu = 1.78 \times 10^{-5} \text{ Pa} \cdot \text{s}$ . Employ a range of velocities from 4 to 40 m/s for your plot.

**18.14** The following function describes the temperature distribution on a rectangular plate for the range  $-2 \leq x \leq 0$  and  $0 \leq y \leq 3$

$$T = 2 + x - y + 2x^2 + 2xy + y^2$$

Develop a Python script to (a) generate a meshplot of this function using the Matplotlib module. Employ the NumPy `linspace` function with default spacing (50 interior points) to generate the  $x$  and  $y$  values. (b) Use the Python function `interp2d` from the SciPy `interpolate` submodule, with `kind='linear'` to estimate the temperature at  $x = -1.63$  and  $y = 1.627$ . Determine the percent relative error of your result. (c) Repeat (b) but with `kind='cubic'`. Note: For parts (b) and (c), develop your sampled points from the function using `np.linspace` with nine interior points.

**18.15** The U.S. Standard Atmosphere specifies atmospheric properties as a function of altitude above sea level. The following table shows selected values of temperature, pressure, and density.

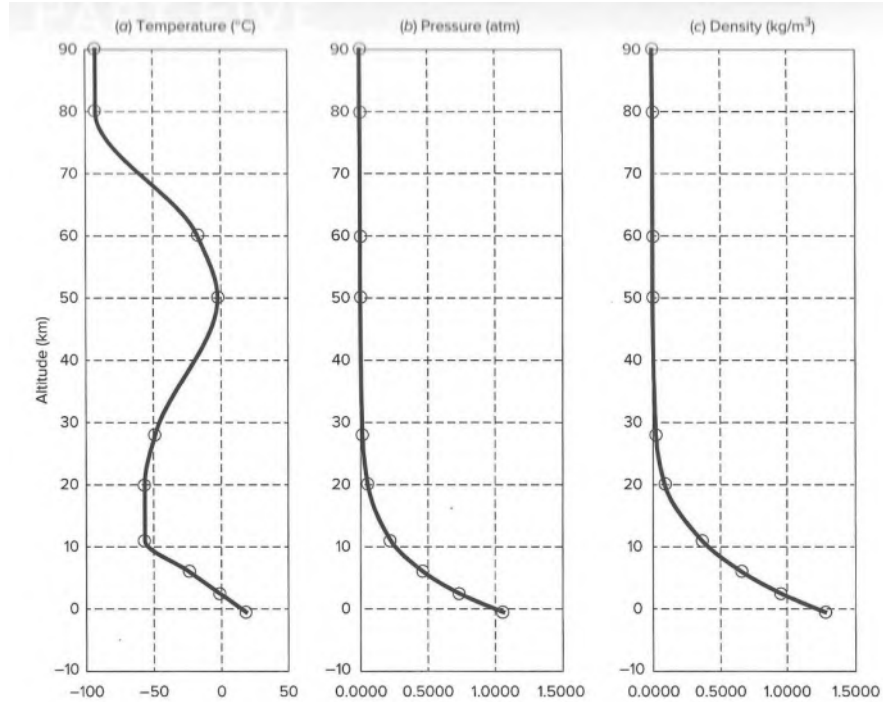
| Altitude (km) | $T(^{\circ}\text{C})$ | $p(\text{atm})$         | $\rho(\text{kg/m}^3)$   |
|---------------|-----------------------|-------------------------|-------------------------|
| -0.5          | 18.4                  | 1.0607                  | 1.2850                  |
| 2.5           | -1.1                  | 0.73702                 | 0.95697                 |
| 6             | -23.8                 | 0.46589                 | 0.66015                 |
| 11            | -56.2                 | 0.22394                 | 0.36481                 |
| 20            | -56.3                 | 0.054557                | 0.088911                |
| 28            | -48.5                 | 0.015946                | 0.025076                |
| 50            | -2.3                  | $7.8721 \times 10^{-4}$ | $1.0269 \times 10^{-3}$ |
| 60            | -17.2                 | $2.2165 \times 10^{-4}$ | $3.0588 \times 10^{-4}$ |
| 80            | -92.3                 | $1.0227 \times 10^{-5}$ | $1.9992 \times 10^{-5}$ |
| 90            | -92.3                 | $1.6216 \times 10^{-6}$ | $3.1703 \times 10^{-6}$ |

Develop a Python function called `StdAtm` to return the values of the three properties for a given altitude. Base `StdAtm` on the SciPy `interpolate` function `PchipInterpolator`. Check the function input altitude to make certain it is within the range of the table. If not, return an error message. The function definition should look like

```
def StdAtm(z,T,p,rho,zz):
    .
    .
    .
    return Tint,pint,rho,int
```

Combine your function with a script that produces three subplots similar to those presented in [Fig. P18.15](#).

**FIGURE P18.15**



**18.16** Felix Baumgartner ascended to 39 km in a stratospheric balloon and made a free-fall jump rushing toward the earth at supersonic speed before parachuting to the ground. As he fell, his drag coefficient changed primarily because of changes in the air density. Recall from [Chap. 1](#) that the terminal velocity,  $v_{\text{terminal}}$  (m/s), of a free-falling object can be computed as

$$v_{\text{terminal}} = \sqrt{\frac{mg}{C_d}}$$

where  $m$  = mass of the object (kg),  $g$  = gravitational acceleration ( $\text{m/s}^2$ ), and  $c_d$  = the drag coefficient ( $\text{kg/m}$ ). The drag coefficient can be computed from

$$c_d = 0.5\rho AC_d$$

where  $\rho$  = the fluid density ( $\text{kg/m}^3$ ),  $A$  = the projected area ( $\text{m}^2$ ), and  $C_d$  = a dimensionless drag coefficient. Note that gravitational acceleration,  $g$ , can be related to elevation by

$$g = g_0 \left( \frac{r_e}{r_e + z} \right)^2$$

where  $g_0$  = gravitational acceleration at sea level,  $9.80667 \text{ m/s}^2$ ,  $r_e$  = mean radius of the earth, 6,371 km,  $z$  = elevation above the earth's surface (km), and the density of air at various elevations is tabulated as

| $z \text{ (km)}$ | $\rho \text{ (kg/m}^3\text{)}$ | $z \text{ (km)}$ | $\rho \text{ (kg/m}^3\text{)}$ | $z \text{ (km)}$ | $\rho \text{ (kg/m}^3\text{)}$ |
|------------------|--------------------------------|------------------|--------------------------------|------------------|--------------------------------|
| -1               | 1.347                          | 6                | 0.6601                         | 25               | 0.04008                        |
| 0                | 1.225                          | 7                | 0.5900                         | 30               | 0.01841                        |
| 1                | 1.112                          | 8                | 0.5258                         | 40               | 0.003996                       |
| 2                | 1.007                          | 9                | 0.4671                         | 50               | 0.001027                       |
| 3                | 0.9093                         | 10               | 0.4135                         | 60               | 0.0003097                      |
| 4                | 0.8194                         | 15               | 0.1948                         | 70               | $8.283 \times 10^{-5}$         |
| 5                | 0.7364                         | 20               | 0.08891                        | 80               | $1.846 \times 10^{-5}$         |

Assume that  $m = 80 \text{ kg}$ ,  $A = 0.55 \text{ m}^2$ , and  $C_d = 1.1$ . Develop a Python script that creates a plot of terminal velocity versus elevation (0 to 40 km in steps of 0.1 km). Use a spline method to generate the required densities needed to construct the plot. What would you have to know to estimate the elevation at which the transition from supersonic to subsonic speed occurs? Oh, by the way, Felix survived.

**18.17** From  **Prob. 15.17**, the heat capacity of sulfuric acid solutions is related to the concentration of  $\text{H}_2\text{SO}_4$ , and this property has been measured carefully in the laboratory with the data presented in the table below.

| wt% $\text{H}_2\text{SO}_4$ | $c_p \text{ kJ/(kg}\cdot\text{K)}$ | wt% $\text{H}_2\text{SO}_4$ | $c_p \text{ kJ/(kg}\cdot\text{K)}$ |
|-----------------------------|------------------------------------|-----------------------------|------------------------------------|
| 0.34                        | 4.173                              | 35.25                       | 3.030                              |
| 0.68                        | 4.160                              | 37.69                       | 2.940                              |
| 1.34                        | 4.135                              | 40.49                       | 2.834                              |
| 2.65                        | 4.087                              | 43.75                       | 2.711                              |
| 3.50                        | 4.056                              | 47.57                       | 2.576                              |
| 5.16                        | 3.998                              | 52.13                       | 2.429                              |
| 9.82                        | 3.842                              | 57.65                       | 2.269                              |
| 15.36                       | 3.671                              | 64.47                       | 2.098                              |
| 21.40                       | 3.491                              | 73.13                       | 1.938                              |
| 22.27                       | 3.465                              | 77.91                       | 1.892                              |
| 23.22                       | 3.435                              | 81.33                       | 1.876                              |
| 24.25                       | 3.403                              | 82.49                       | 1.870                              |



| wt% H <sub>2</sub> SO <sub>4</sub> | $c_p$ kJ/(kg·K) | wt% H <sub>2</sub> SO <sub>4</sub> | $c_p$ kJ/(kg·K) |
|------------------------------------|-----------------|------------------------------------|-----------------|
| 25.39                              | 3.367           | 84.48                              | 1.846           |
| 26.63                              | 3.326           | 85.48                              | 1.820           |
| 28.00                              | 3.281           | 89.36                              | 1.681           |
| 29.52                              | 3.231           | 91.81                              | 1.586           |
| 30.34                              | 3.202           | 94.82                              | 1.488           |
| 31.20                              | 3.173           | 97.44                              | 1.425           |
| 33.11                              | 3.107           | 100.00                             | 1.403           |

In [Prob. 15.17](#), we found that it was not possible to fit these data with a high-order polynomial and achieve adequacy. Nature has provided a little “bump” in the curve which gives polynomial regression difficulties. Now that we are in [Chap. 18](#), we have learned alternate approaches to regression for retrieving and interpolating values from tabular property data. Apply the natural cubic spline to these data and present a plot of the spline-interpolated curve and the data. Comment on the ability of the spline fit to accomplish what polynomial regression could not.

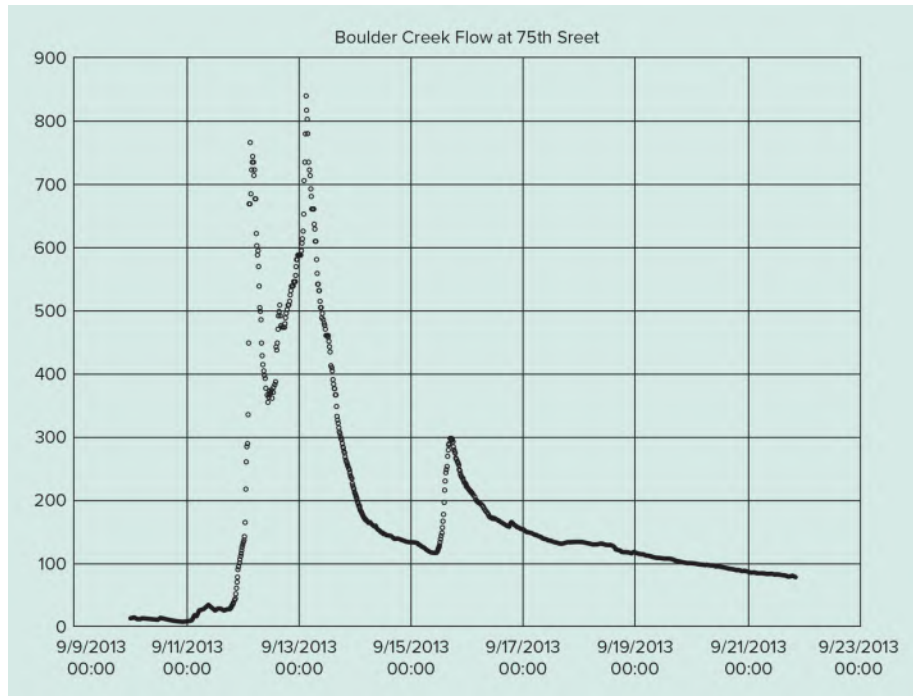
**18.18** Select a river or stream on the [waterwatch.USGS.gov](#) site. Stream flow data in cubic feet per second (cfs) are available for different time bases, from yearly, monthly, and daily averages to readings every 15 minutes. Choose a basis for your stream and a time interval you would consider interesting. Generate a text file of the flow data and load it into a Python script. Apply both LOESS and spline smoothing to the data. Write about the results of your analysis. Compare the two methods. Relate your results to any circumstances that would have affected stream flow during the period that you studied.

Here are a few suggestions:

- pick a state that contains a river or stream of interest and double-click it on the map or select it from the dropdown list
- scan the markers on the state map and find one of interest, click it, and click the USGS number in blue that shows up in the window for that site
- pick the type of flow data (discharge) you wish to extract—if you want measurements taken frequently, choose Current/Historical Observations
- pick a time interval and generate a Tab-separated data file—you can also display a graph to determine whether your selected data are “interesting” for a smoothing exercise
- the data are in a text file—you may want to “trim” this file with Notepad, import this file into Excel to remove extraneous columns, before you attempt to “load” it into Python
- from there, you can use the Python examples to carry out the smoothing methods

Here is an extreme example, Boulder Creek, Colorado, in mid-September 2013.

**FIGURE P18.18**



**18.19** The tragic coronavirus Covid-19 pandemic in 2020–2021 has provided us with a plethora of data on reported cases, hospitalizations, and deaths. Pick a geographic region (country, state/province, city) of interest and seek downloadable frequency data (not cumulative) for a selected statistic. Apply either cubic spline or LOESS smoothing to the data and tune the smoothing to represent the general trend of the data. Comment on the shape of the smoothed curve and how it relates to societal conditions during the pandemic.

## Integration and Differentiation

### 5.1 OVERVIEW

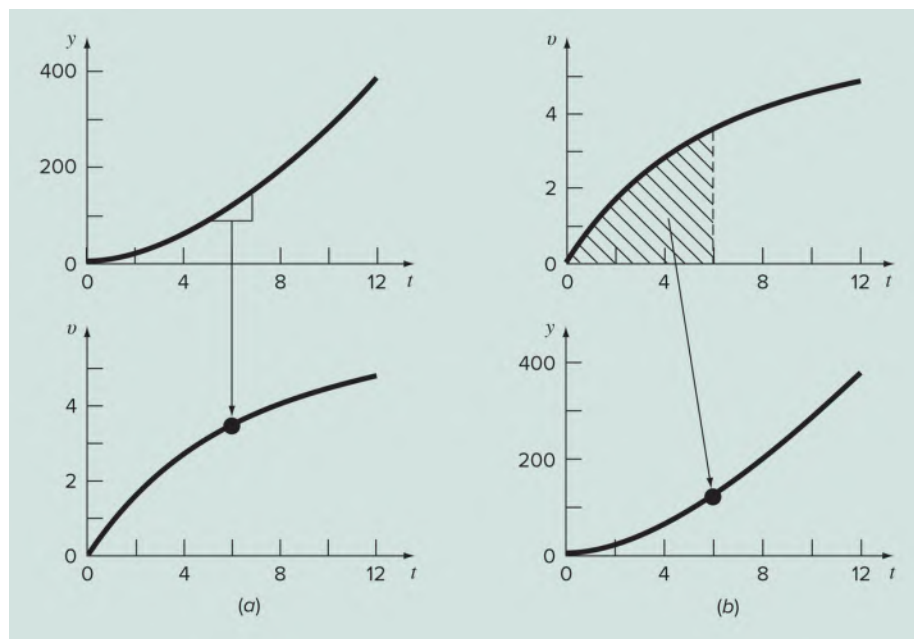
In high school or during your first year of college, you were introduced to differential and integral calculus. There you learned techniques to obtain analytical or exact derivatives and integrals.

Mathematically, the *derivative* represents the rate of change of a dependent variable with respect to an independent variable. For example, if we are given a function  $y(t)$  that specifies an object's position as a function of time, differentiation provides a means to determine its velocity, as in

$$v(t) = \frac{d}{dt}y(t)$$

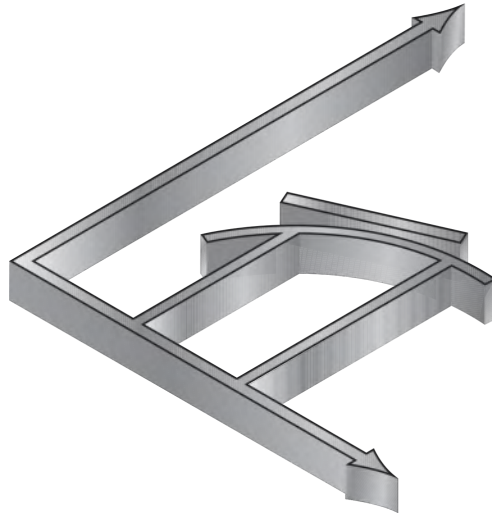
As in  **Fig. PT5.1a**, the derivative can be visualized as the slope of a function.

**FIGURE PT5.1** The contrast between (a) differentiation and (b) integration.



Integration is the inverse of differentiation. Just as differentiation uses differences to quantify an instantaneous process, integration involves summing instantaneous information to give a total result over an interval. Thus, if we are provided with velocity as a function of time, integration can be used to determine the distance traveled:

$$y(t) = \int_0^t v(t) dt$$



As in  **Fig. PT5.1b**, for functions lying above the abscissa, the integral can be visualized as the area under the curve of  $v(t)$  from 0 to  $t$ . Consequently, just as a derivative can be thought of as a slope, an integral can be envisaged as a summation.

Because of the close relationship between differentiation and integration, we have opted to devote this part of the book to both processes. Among other things, this will provide the opportunity to highlight their similarities and differences from a numerical perspective. In addition, the material will have relevance to the next part of the book where we will cover differential equations.

Although differentiation is taught before integration in calculus, we reverse their order in the following chapters. We do this for several reasons. First, we have already introduced you to the basics of numerical differentiation in  **Chap. 4**. Second, in part because it is much less sensitive to roundoff errors, integration represents a more highly developed area of numerical methods. Finally, although numerical differentiation is not as widely employed, it does have great significance for the solution of differential equations. Hence, it makes sense to cover it as the last topic prior to describing differential equations in  **Part Six**.

## 5.2 PART ORGANIZATION

---

 **Chapter 19** is devoted to the most common approaches for numerical integration—the *Newton-Cotes formulas*. These relationships are based on replacing a complicated function or tabulated data with a simple polynomial that is easy to integrate. Three of the most widely used Newton-Cotes formulas are discussed in detail: the *trapezoidal rule*, *Simpson's 1/3 rule*, and *Simpson's 3/8 rule*. All these formulas are designed for cases where the data to be integrated are evenly spaced. In addition, we also include a discussion of numerical integration of unequally spaced data. This is a very important topic because many real-world applications deal with data that are in this form.

All the above material relates to *closed integration*, where the function values at the ends of the limits of integration are known. At the end of  **Chap. 19**, we present *open integration formulas*, where the integration limits extend beyond the range of the known data. Although they are not commonly used for definite integration, open integration formulas are presented here because they are utilized in the solution of ordinary differential equations in  **Part Six**.

The formulations covered in  **Chap. 19** can be employed to analyze both tabulated data and equations.  **Chapter 20** deals with two techniques that are expressly designed to integrate equations and functions: *Romberg integration* and *Gauss quadrature*. Computer algorithms are provided for both of these methods. In addition, *adaptive integration* is discussed.

In  **Chap. 21**, we present additional information on *numerical differentiation* to supplement the introductory material from  **Chap. 4**. Topics include *high-accuracy finite-difference formulas*, *Richardson extrapolation*, and the differentiation of unequally spaced data. The effect of errors on both numerical differentiation and integration is also discussed.

# Numerical Integration Formulas

## CHAPTER OBJECTIVES

The primary objective of this chapter is to introduce you to numerical integration. Specific objectives and topics covered are

- Recognizing that Newton-Cotes integration formulas are based on the strategy of replacing a complicated function or tabulated data with a polynomial that is easy to integrate.
- Knowing how to implement the following single application Newton-Cotes formulas:
  - Trapezoidal rule
  - Simpson's 1/3 rule
  - Simpson's 3/8 rule
- Knowing how to implement the following composite Newton-Cotes formulas:
  - Trapezoidal rule
  - Simpson's 1/3 rule
- Recognizing that even-segment-odd-point formulas like Simpson's 1/3 rule achieve higher than expected accuracy.
- Knowing how to use the trapezoidal rule to integrate unequally spaced data.
- Understanding the difference between open and closed integration formulas.

## YOU'VE GOT A PROBLEM

Recall that the velocity of a free-falling bungee jumper as a function of time can be computed as

$$v(t) = \sqrt{\frac{gm}{c_d}} \tanh\left(\sqrt{\frac{gc_d}{m}}t\right) \quad (19.1)$$

Suppose that we would like to know the vertical distance  $z$  the jumper has fallen after a certain time  $t$ . This distance can be evaluated by integration:

$$(19.2)$$

$$x(t) = \int_0^t v(t) dt$$

Substituting [Eq. \(19.1\)](#) into [Eq. \(19.2\)](#) gives

$$x(t) = \int_0^t \sqrt{\frac{gm}{c_d}} \tanh\left(\sqrt{\frac{gc_d}{m}}t\right) dt \quad (19.3)$$

Thus, integration provides the means to determine the distance from the velocity. Calculus can be used to solve [Eq. \(19.3\)](#) for

$$x(t) = \frac{m}{c_d} \ln\left[\cosh\left(\sqrt{\frac{gc_d}{m}}t\right)\right] \quad (19.4)$$

Although a closed form solution can be developed for this case, there are other functions that cannot be integrated analytically. Further, suppose that there was some way to measure the jumper's velocity at various times during the fall. These velocities along with their associated times could be assembled as a table of discrete values. In this situation, it would also be possible to integrate the discrete data to determine the distance. In both these instances, numerical integration methods are available to obtain solutions. [Chapters 19](#) and [20](#) will introduce you to some of these methods.

# 19.1 INTRODUCTION AND BACKGROUND

---

## 19.1.1 What Is Integration?

According to the dictionary definition, to integrate means “to bring together, as parts, into a whole; to unite; to indicate the total amount. . . .” Mathematically, definite integration is represented by

$$I = \int_a^b f(x) dx \quad (19.5)$$

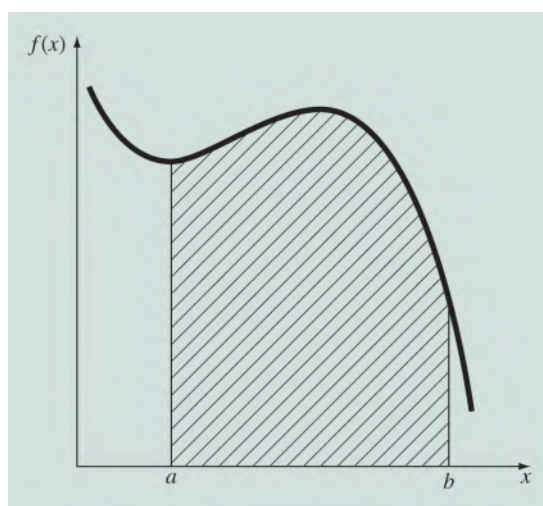
which stands for the integral of the function  $f(x)$  with respect to the independent variable  $x$ , evaluated between the limits  $x = a$  and  $x = b$ .

As suggested by the dictionary definition, the “meaning” of [Eq. \(19.5\)](#) is the total value, or summation, of  $f(x) dx$  over the range  $x = a$  to  $b$ . In fact, the symbol  $\int$  is actually a stylized capital S that is intended to signify the close connection between integration and summation.

[Figure 19.1](#) represents a graphical manifestation of the concept. For functions lying above the  $x$  axis, the integral expressed by [Eq. \(19.5\)](#) corresponds to the area under the curve of  $f(x)$  between  $x = a$  and  $b$ .

---

**FIGURE 19.1** Graphical representation of the integral of  $f(x)$  between the limits  $x = a$  and  $b$ . The integral is equivalent to the area under the curve.



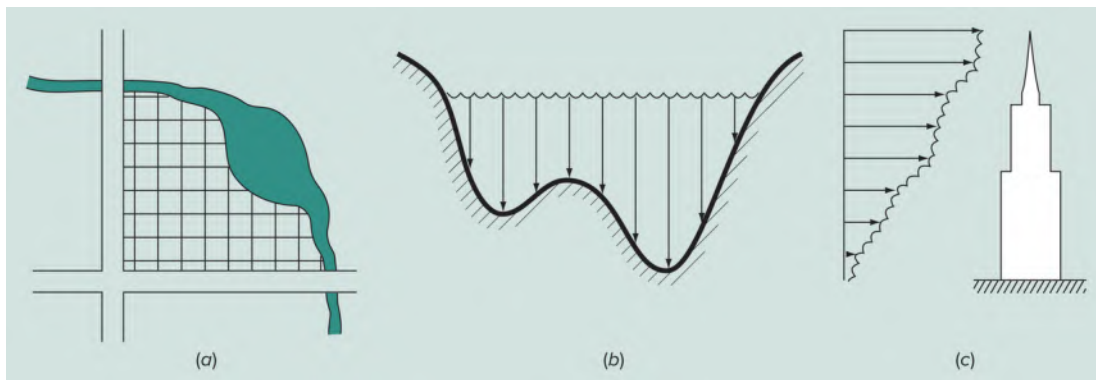


Numerical integration is sometimes referred to as quadrature. This is an archaic term that originally meant the construction of a square having the same area as some curvilinear figure. Today, the term *quadrature* is generally taken to be synonymous with numerical definite integration.

## 19.1.2 Integration in Engineering and Science

Integration has so many engineering and scientific applications that you were required to take integral calculus in your first year at college. Many specific examples of such applications could be given in all fields of engineering and science. A number of examples relate directly to the idea of the integral as the area under a curve.  **Figure 19.2** depicts a few cases where integration is used for this purpose.

**FIGURE 19.2** Examples of how integration is used to evaluate areas in engineering and scientific applications. (a) A surveyor might need to know the area of a field bounded by a meandering stream and two roads. (b) A hydrologist might need to know the cross-sectional area of a river. (c) A structural engineer might need to determine the net force due to a nonuniform wind blowing against the side of a skyscraper.

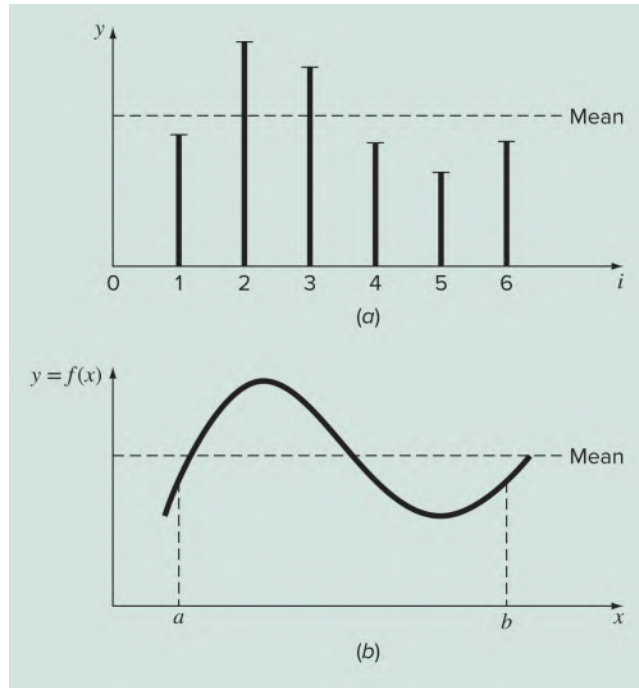


Other common applications relate to the analogy between integration and summation. For example, a common application is to determine the mean of a continuous function. Recall that the mean of  $n$  discrete data points can be calculated by [ **Eq. (14.2)**].

$$\text{Mean} = \frac{\sum_{i=1}^n y_i}{n} \quad (19.6)$$

where  $y_i$  are individual measurements. The determination of the mean of discrete points is depicted in  **Fig. 19.3a**.

**FIGURE 19.3** An illustration of the mean for (a) discrete and (b) continuous data.



In contrast, suppose that  $y$  is a continuous function of an independent variable  $x$ , as depicted in [Fig. 19.3b](#). For this case, there are an infinite number of values between  $a$  and  $b$ . Just as [Eq. \(19.6\)](#) can be applied to determine the mean of the discrete readings, you might also be interested in computing the mean or average of the continuous function  $y = f(x)$  for the interval from  $a$  to  $b$ . Integration is used for this purpose, as specified by

$$\text{Mean} = \frac{\int_a^b f(x) dx}{b - a} \quad (19.7)$$

This formula has hundreds of engineering and scientific applications. For example, it is used to calculate the center of gravity of irregular objects in mechanical and civil engineering and to determine the root-mean-square current in electrical engineering.

Integrals are also employed by engineers and scientists to evaluate the total amount or quantity of a given physical variable. The integral may be evaluated over a line, an area, or a volume. For example, the total mass of chemical contained in a reactor is given as the product of the concentration of chemical and the reactor volume, or

$$\text{Mass} = \text{concentration} \times \text{volume}$$

where concentration has units of mass per volume. However, suppose that concentration varies from location to location within the reactor. In this case, it is necessary to sum the products of local concentrations  $c_i$  and corresponding elemental volumes  $\Delta V_i$ :

$$\text{Mass} = \sum_{i=1}^n c_i \Delta V_i$$

where  $n$  is the number of discrete volumes. For the continuous case, where  $c(x, y, z)$  is a known function and  $x$ ,  $y$ , and  $z$  are independent variables designating position in Cartesian coordinates, integration can be used for the same purpose:

$$\text{Mass} = \iiint c(x, y, z) \, dx \, dy \, dz$$

or

$$\text{Mass} = \iiint_V c(V) \, dV$$

which is referred to as a *volume integral*. Notice the strong analogy between summation and integration.

Similar examples could be given in other fields of engineering and science. For example, the total rate of energy transfer across a plane where the flux (in joules per square centimeter per second) is a function of position is given by

$$\text{Flux} = \iint_A \text{flux} \, dA$$

which is referred to as an *areal integral*, where  $A$  = area.

These are just a few of the applications of integration that you might face regularly in the pursuit of your profession. When the functions to be analyzed are simple, you will normally choose to evaluate them analytically. However, it is often difficult or impossible when the function is complicated, as is typically the case in more realistic examples. In addition, the underlying function is often unknown and defined only by measurement at discrete points. For both these cases, you must have the ability to obtain approximate values for integrals using numerical techniques as described next.

## 19.2 NEWTON-COTES FORMULAS

The *Newton-Cotes formulas* are the most common numerical integration schemes. They are based on the strategy of replacing a complicated function or tabulated data with a polynomial that is easy to integrate:

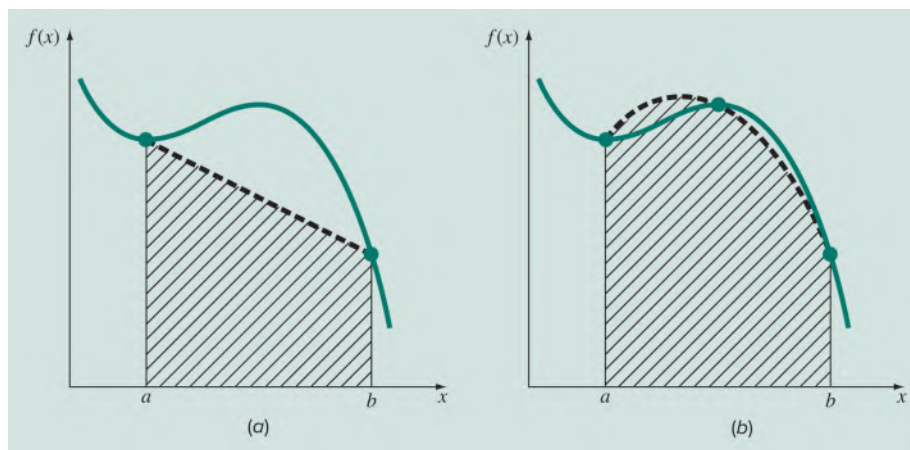
$$I = \int_a^b f(x) dx \cong \int_a^b f_n(x) dx \quad (19.8)$$

where  $f_n(x)$  = a polynomial of the form

$$f_n(x) = a_0 + a_1x + \cdots + a_{n-1}x^{n-1} + a_nx^n \quad (19.9)$$

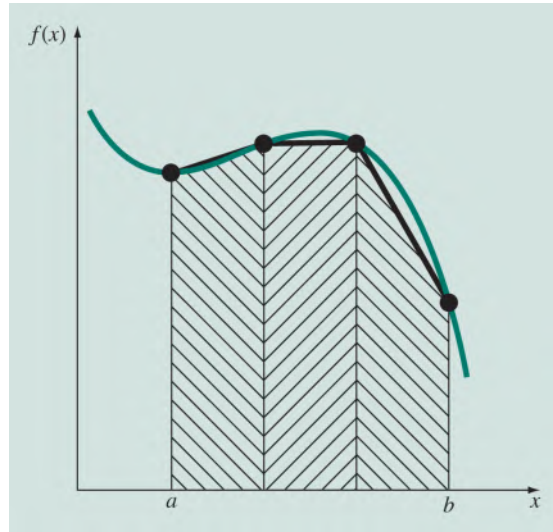
where  $n$  is the order of the polynomial. For example, in [Fig. 19.4a](#), a first-order polynomial (a straight line) is used as an approximation. In [Fig. 19.4b](#), a parabola is employed for the same purpose.

**FIGURE 19.4** The approximation of an integral by the area under (a) a straight line and (b) a parabola.



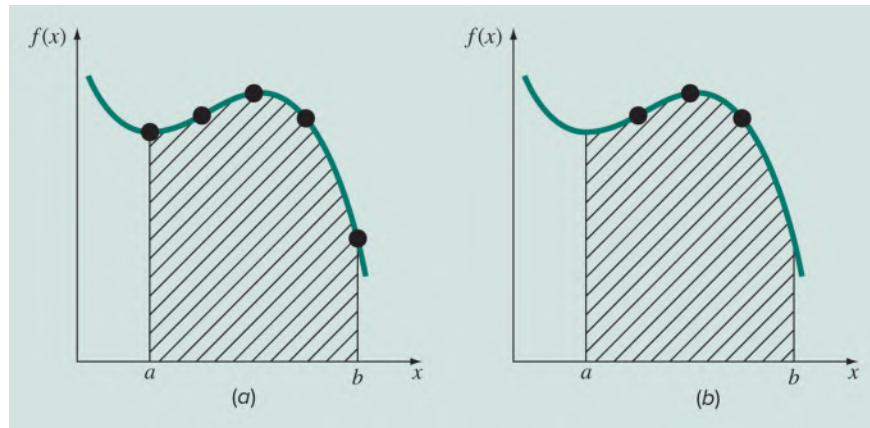
The integral can also be approximated using a series of polynomials applied piecewise to the function or data over segments of constant length. For example, in [Fig. 19.5](#), three straight-line segments are used to approximate the integral. Higher-order polynomials can be utilized for the same purpose.

**FIGURE 19.5** The approximation of an integral by the area under three straight-line segments.



Closed and open forms of the Newton-Cotes formulas are available. The *closed forms* are those where the data points at the beginning and end of the limits of integration are known (Fig. 19.6a). The *open forms* have integration limits that extend beyond the range of the data (Fig. 19.6b). This chapter emphasizes the closed forms. However, material on open Newton-Cotes formulas is briefly introduced in Sec. 19.7.

**FIGURE 19.6** The difference between (a) closed and (b) open integration formulas.



## 19.3 THE TRAPEZOIDAL RULE

---

The *trapezoidal rule* is the first of the Newton-Cotes closed integration formulas. It corresponds to the case where the polynomial in [Eq. \(19.8\)](#) is first-order:

$$I = \int_a^b \left[ f(a) + \frac{f(b) - f(a)}{b - a} (x - a) \right] dx \quad (19.10)$$

The result of the integration is

$$I = (b - a) \frac{f(a) + f(b)}{2} \quad (19.11)$$

which is called the *trapezoidal rule*.

Geometrically, the trapezoidal rule is equivalent to approximating the area of the trapezoid under the straight line connecting  $f(a)$  and  $f(b)$  in [Fig. 19.7](#). Recall from geometry that the formula for computing the area of a trapezoid is the height times the average of the bases. In our case, the concept is the same but the trapezoid is on its side. Therefore, the integral estimate can be represented as

$$I = \text{width} \times \text{average height} \quad (19.12)$$

---

Page 463

---

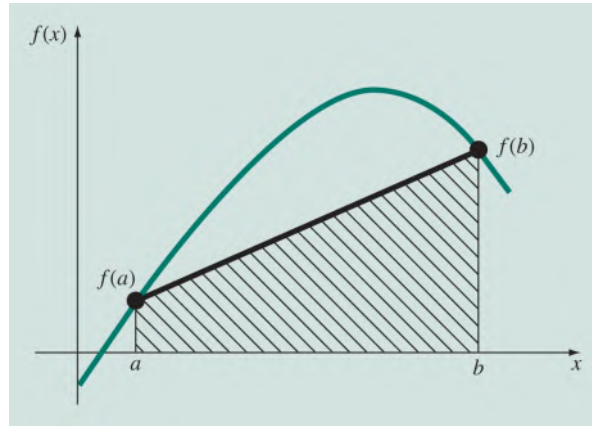
or

$$I = (b - a) \times \text{average height} \quad (19.13)$$

where, for the trapezoidal rule, the average height is the average of the function values at the end points, or  $[f(a) + f(b)]/2$ .

---

**FIGURE 19.7** Graphical depiction of the trapezoidal rule.



All the Newton-Cotes closed formulas can be expressed in the general format of [Eq. \(19.13\)](#). That is, they differ only with respect to the formulation of the average height.

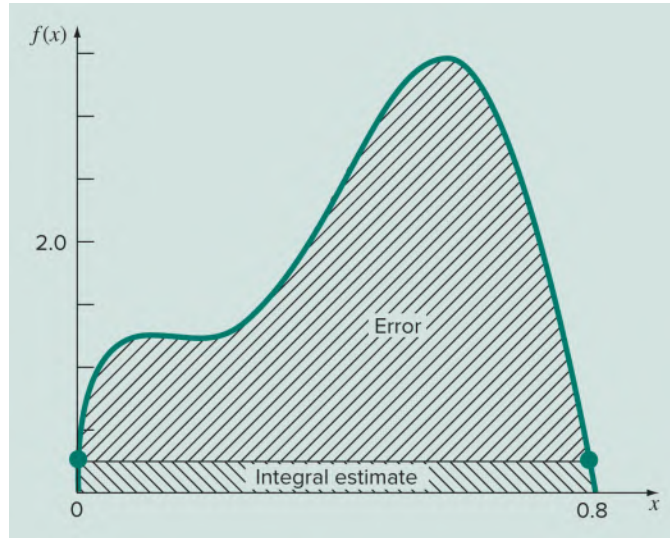
## 19.3.1 Error of the Trapezoidal Rule

When we employ the integral under a straight-line segment to approximate the integral under a curve, we obviously can incur an error that may be substantial ([Fig. 19.8](#)). An estimate for the local truncation error of a single application of the trapezoidal rule is

$$E_t = -\frac{1}{12} f''(\xi)(b-a)^3 \quad (19.14)$$

where  $\xi$  lies somewhere in the interval from  $a$  to  $b$ . [Equation \(19.14\)](#) indicates that if the function being integrated is linear, the trapezoidal rule will be exact because the second derivative of a straight line is zero. Otherwise, for functions with second- and higher-order derivatives (i.e., with curvature), some error can occur.

**FIGURE 19.8** Graphical depiction of the use of a single application of the trapezoidal rule to approximate the integral of  $f(x) = 0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5$  from  $x = 0$  to  $0.8$ .



## EXAMPLE 19.1

### Single Application of the Trapezoidal Rule

#### Problem Statement

Use [Eq. \(19.11\)](#) to numerically integrate

$$f(x) = 0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5$$

from  $a = 0$  to  $b = 0.8$ . Note that the exact value of the integral can be determined analytically to be 1.640533.

Page 464

#### Solution

The function values  $f(0) = 0.2$  and  $f(0.8) = 0.232$  can be substituted into [Eq. \(19.11\)](#) to yield

$$I = (0.8 - 0) \frac{0.2 + 0.232}{2} = 0.1728$$

which represents an error of  $E_t = 1.640533 - 0.1728 = 1.467733$ , which corresponds to a percent relative error of  $\varepsilon_t = 89.5\%$ . The reason for this large error is evident from the graphical depiction in [Fig. 19.8](#). Notice that the area under the straight line neglects a significant portion of the integral lying above the line.



In actual situations, we would have no foreknowledge of the true value. Therefore, an approximate error estimate is required. To obtain this estimate, the function's second derivative over the interval can be computed by differentiating the original function twice to give

$$f''(x) = -400 + 4,050x - 10,800x^2 + 8,000x^3$$

The average value of the second derivative can be computed as [Eq. (19.7)]

$$\bar{f}''(x) = \frac{\int_0^{0.8} (-400 + 4,050x - 10,800x^2 + 8,000x^3) dx}{0.8 - 0} = -60$$

which can be substituted into Eq. (19.14) to yield

$$E_a = -\frac{1}{12}(-60)(0.8)^3 = 2.56$$

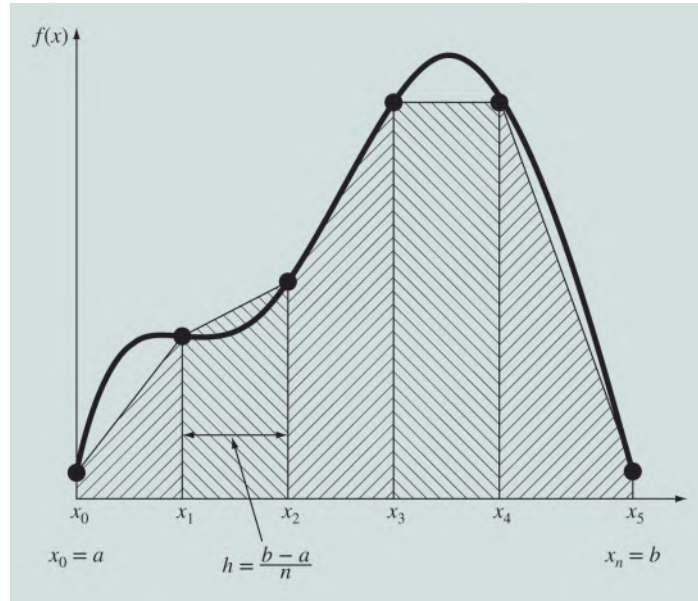
which is of the same order of magnitude and sign as the true error. A discrepancy does exist, however, because of the fact that for an interval of this size, the average second derivative is not necessarily an accurate approximation of  $f''(\xi)$ . Thus, we denote that the error is approximate by using the notation  $E_a$ , rather than exact by using  $E_t$ .

## 19.3.2 The Composite Trapezoidal Rule

One way to improve the accuracy of the trapezoidal rule is to divide the integration interval from  $a$  to  $b$  into a number of segments and apply the method to each segment (Fig. 19.9). The areas of individual segments can then be added to yield the integral for the entire interval. The resulting equations are called *composite*, or *multiple-segment, integration formulas*.

---

**FIGURE 19.9** Composite trapezoidal rule.



**Figure 19.9** shows the general format and nomenclature we will use to characterize composite integrals. There are  $n + 1$  equally spaced base points  $(x_0, x_1, x_2, \dots, x_n)$ . Consequently, there are  $n$  segments of equal width:

$$h = \frac{b - a}{n} \quad (19.15)$$

If  $a$  and  $b$  are designated as  $x_0$ , and  $x_n$ , respectively, the total integral can be represented as

$$I = \int_{x_0}^{x_1} f(x) dx + \int_{x_1}^{x_2} f(x) dx + \dots + \int_{x_{n-1}}^{x_n} f(x) dx$$

Substituting the trapezoidal rule for each integral yields

$$I = h \frac{f(x_0) + f(x_1)}{2} + h \frac{f(x_1) + f(x_2)}{2} + \dots + h \frac{f(x_{n-1}) + f(x_n)}{2} \quad (19.16)$$

or, grouping terms:

$$I = \frac{h}{2} [f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n)] \quad (19.17)$$

or, using [Eq. \(19.15\)](#) to express [Eq. \(19.17\)](#) in the general form of [Eq. \(19.13\)](#):

$$I = \underbrace{(b-a)}_{\text{width}} \underbrace{\frac{f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n)}{2n}}_{\text{Average height}} \quad (19.18)$$

Because the summation of the coefficients of  $f(x)$  in the numerator divided by  $2n$  is equal to 1, the average height represents a weighted average of the function values. According to [Eq. \(19.18\)](#), the interior points are given twice the weight of the two end points  $f(x_0)$  and  $f(x_n)$ .

An error for the composite trapezoidal rule can be obtained by summing the individual errors for each segment to give

$$E_t = -\frac{(b-a)^3}{12n^3} \sum_{i=1}^n f''(\xi_i) \quad (19.19)$$

where  $f''(\xi_i)$  is the second derivative at a point  $\xi_i$  located in segment  $i$ . This result can be simplified by estimating the mean or average value of the second derivative for the entire interval as

$$f'' \cong \frac{\sum_{i=1}^n f''(\xi_i)}{n} \quad (19.20)$$

Therefore  $\sum f''(\xi_i) \cong n f''$  and [Eq. \(19.19\)](#) can be rewritten as

$$E_a = -\frac{(b-a)^3}{12n^2} f'' \quad (19.21)$$

Thus, if the number of segments is doubled, the truncation error will be quartered. Note that [Eq. \(19.21\)](#) is an approximate error because of the approximate nature of [Eq. \(19.20\)](#).

## EXAMPLE 19.2

### Composite Application of the Trapezoidal Rule

## Problem Statement

Use the two-segment trapezoidal rule to estimate the integral of

$$f(x) = 0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5$$

from  $a = 0$  to  $b = 0.8$ . Employ [Eq. \(19.21\)](#) to estimate the error. Recall that the exact value of the integral is 1.640533.

## Solution

For  $n = 2$  ( $h = 0.4$ ):

$$f(0) = 0.2 \quad f(0.4) = 2.456 \quad f(0.8) = 0.232$$

$$I = 0.8 \frac{0.2 + 2(2.456) + 0.232}{4} = 1.0688$$

$$E_t = 1.640533 - 1.0688 = 0.57173 \quad \varepsilon_t = 34.9\%$$

$$E_a = -\frac{0.8^3}{12(2)^2}(-60) = 0.64$$

where  $-60$  is the average second derivative determined previously in [Example 19.1](#).

The results of the previous example, along with three- through ten-segment applications of the trapezoidal rule, are summarized in [Table 19.1](#). Notice how the error decreases as the number of segments increases. However, also notice that the rate of decrease is gradual. This is because the error is inversely related to the square of  $n$  [[Eq. \(19.21\)](#)]. Therefore, doubling the number of segments quarters the error. In subsequent sections we develop higher-order formulas that are more accurate and that converge more quickly on the true integral as the segments are increased. However, before investigating these formulas, we will first discuss how Python can be used to implement the trapezoidal rule.

**TABLE 19.1** Results for the composite trapezoidal rule to estimate the integral of

$f(x) = 0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5$   
from  $x = 0$  to  $0.8$ . The exact value is 1.640533.

| $n$ | $h$    | $I$    | $\varepsilon_t(\%)$ |
|-----|--------|--------|---------------------|
| 2   | 0.4    | 1.0688 | 34.9                |
| 3   | 0.2667 | 1.3695 | 16.5                |
| 4   | 0.2    | 1.4848 | 9.5                 |
| 5   | 0.16   | 1.5399 | 6.1                 |
| 6   | 0.1333 | 1.5703 | 4.3                 |

| $n$ | $h$    | $I$    | $\varepsilon_t(\%)$ |
|-----|--------|--------|---------------------|
| 7   | 0.1143 | 1.5887 | 3.2                 |
| 8   | 0.1    | 1.6008 | 2.4                 |
| 9   | 0.0889 | 1.6091 | 1.9                 |
| 10  | 0.08   | 1.6150 | 1.6                 |

## 19.3.3 Python Function: trap

A simple algorithm to implement the composite trapezoidal rule can be written as shown in

 **Fig. 19.10**. The function to be integrated is passed into the `trap` function along with the limits of integration and the number of segments. A loop is then employed to generate the integral following  **Eq. (19.18)**.

**FIGURE 19.10** Python function `trap` to implement the composite trapezoidal rule.

```
def trap(func,a,b,n=100):
    """
    Composite trapezoidal rule quadrature
    Input:
        func = name of function to be integrated
        a,b = integration limits
        n = number of segments (default = 100)
    Output:
        I = estimate of integral
    """
    if b <= a: return 'upper bound must be greater than lower bound'
    x = a
    h = (b-a)/n
    s = func(a)
    for i in range(n-1):
        x = x + h
        s = s + 2*func(x)
    s = s + func(b)
    I = (b-a)*s/2/n
    return I
```

An application of the Python function can be developed to determine the distance fallen by the free-falling bungee jumper in the first three seconds by evaluating the integral of  **Eq. (19.3)**. For this example, assume the following parameter values:  $m = 68.1$  kg and  $c_d = 0.25$  kg/m. Note that the exact value of the integral can be computed with  **Eq. (19.4)** as 41.94805.

The function to be integrated can be written in separate `def` format or in `lambda` format. Here, Page 467 we use the `def` format.

```
import numpy as np
def zint(t):
    return np.sqrt(m*g/cd)*np.tanh(np.sqrt(g*cd/m)*t)
```

or

```
zint = lambda t: np.sqrt(m*g/cd)*np.tanh(np.sqrt(g*cd/m)*t)
```

In the latter case, we could embed the `lambda` definition directly in our call to `trap`. We do this here for a five-segment integration.

```
z = trap(lambda t: np.sqrt(m*g/cd)*np.tanh(np.sqrt(g*cd/m)*t),0.,3.,5)
print(z)
```

with the result

```
41.86992959072735
```

This result has a true absolute error of 0.186%. To obtain a more accurate result, we can use the default 100 intervals.

```
z = trap(lambda t: np.sqrt(m*g/cd)*np.tanh(np.sqrt(g*cd/m)*t),0.,3.)
print(z)
```

and the result is much closer to the true value:

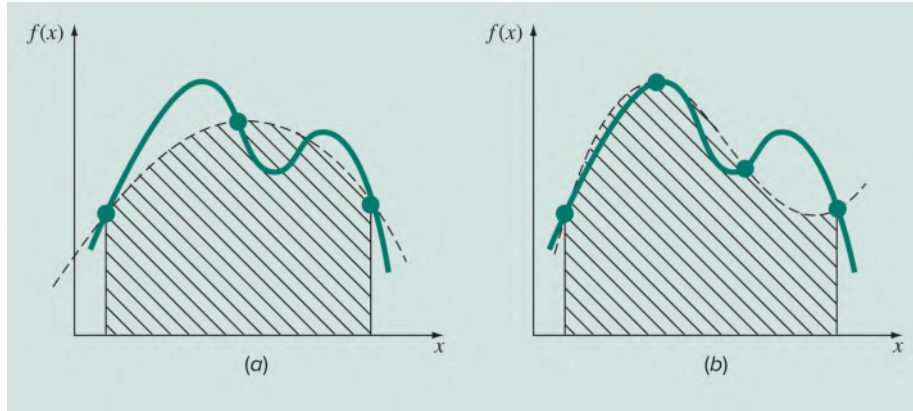
```
41.94785498810134
```

## 19.4 SIMPSON'S RULES

Aside from applying the trapezoidal rule with finer segmentation, another way to obtain a more accurate estimate of an integral is to use higher-order polynomials to connect the points. For example, if there is an extra point midway between  $f(a)$  and  $f(b)$ , the three points can be connected with a parabola (  Fig. 19.11a). If there are two points equally spaced between  $f(a)$  and  $f(b)$ , the four points can be connected with a third-order polynomial (  Fig. 19.11b). The formulas that result from taking the integrals under these polynomials are called *Simpson's rules*.

Page 468

**FIGURE 19.11** (a) Graphical depiction of Simpson's 1/3 rule: It consists of taking the area under a parabola connecting three points. (b) Graphical depiction of Simpson's 3/8 rule: It consists of taking the area under a cubic equation connecting four points.



### 19.4.1 Simpson's 1/3 Rule

Simpson's 1/3 rule corresponds to the case where the polynomial in  Eq. (19.8) is second-order:

$$I = \int_{x_0}^{x_2} \left[ \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} f(x_0) + \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} f(x_1) + \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} f(x_2) \right] dx$$

where  $a$  and  $b$  are designated as  $x_0$  and  $x_2$ , respectively. The result of the integration is

$$I = \frac{h}{3} [f(x_0) + 4f(x_1) + f(x_2)] \quad (19.22)$$

where, for this case,  $h = (b - a)/2$ . This equation is known as *Simpson's 1/3 rule*. The label “1/3” stems from the fact that  $h$  is divided by 3 in [Eq. \(19.22\)](#). Simpson's 1/3 rule can also be expressed using the format of [Eq. \(19.13\)](#):

$$I = (b - a) \frac{f(x_0) + 4f(x_1) + f(x_2)}{6} \quad (19.23)$$

where  $a = x_0$ ,  $b = x_2$ , and  $x_1$  = the point midway between  $a$  and  $b$ , which is given by  $(a + b)/2$ . Notice that, according to [Eq. \(19.23\)](#), the middle point is weighted by two-thirds and the two end points by one-sixth.

It can be shown that a single-segment application of Simpson's 1/3 rule has a truncation error of

$$E_t = -\frac{1}{90} h^5 f^{(4)}(\xi)$$

or, because  $h = (b - a)/2$ :

$$E_t = -\frac{(b - a)^5}{2880} f^{(4)}(\xi) \quad (19.24)$$

where  $\xi$  lies somewhere in the interval from  $a$  to  $b$ . Thus, Simpson's 1/3 rule is more accurate than the trapezoidal rule. However, comparison with [Eq. \(19.14\)](#) indicates that it is more accurate than expected. Rather than being proportional to the third derivative, the error is proportional to the fourth derivative. Consequently, Simpson's 1/3 rule is third-order accurate even though it is based on only three points. In other words, it yields exact results for cubic polynomials even though it is derived from a parabola!

## EXAMPLE 19.3

### Single Application of Simpson's 1/3 Rule

#### Problem Statement

Use [Eq. \(19.23\)](#) to integrate

$$f(x) = 0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5$$



from  $a = 0$  to  $b = 0.8$ . Employ [Eq. \(19.24\)](#) to estimate the error. Recall that the exact integral is 1.640533.

### Solution

$n = 2(h = 0.4)$ :

$$\begin{aligned} f(0) &= 0.2 & f(0.4) &= 2.456 & f(0.8) &= 0.232 \\ I &= 0.8 \frac{0.2 + 4(2.456) + 0.232}{6} = 1.367467 \\ E_t &= 1.640533 - 1.367467 = 0.2730667 & \varepsilon_t &= 16.6\% \end{aligned}$$

which is approximately five times more accurate than for a single application of the trapezoidal rule ([Example 19.1](#)). The approximate error can be estimated as

$$E_a = -\frac{0.8^5}{2880}(-2400) = 0.2730667$$

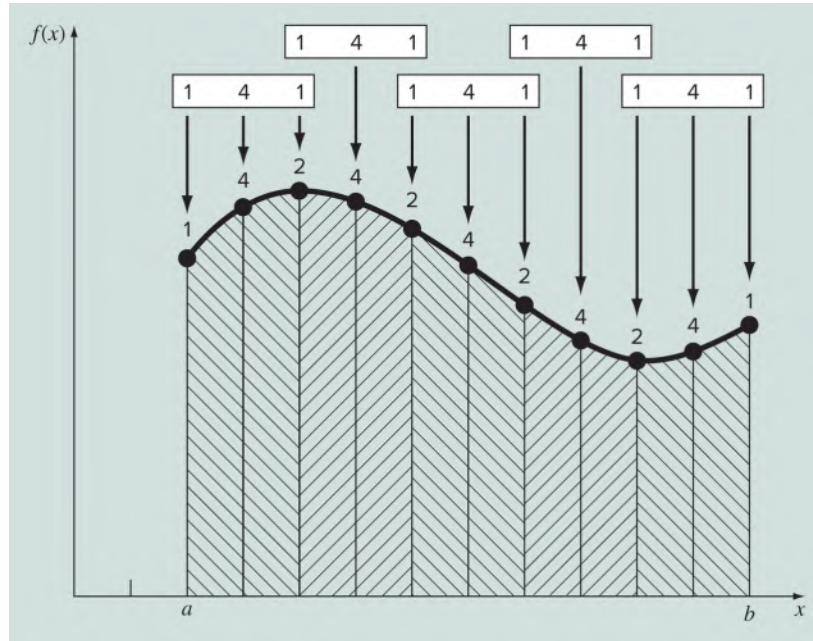
where  $-2400$  is the average fourth derivative for the interval. As was the case in [Example 19.1](#), the error is approximate ( $E_a$ ) because the average fourth derivative is generally not an exact estimate of  $f^{(4)}(\xi)$ . However, because this case deals with a fifth-order polynomial, the result matches exactly.

## 19.4.2 The Composite Simpson's 1/3 Rule

Just as with the trapezoidal rule, Simpson's rule can be improved by dividing the integration interval into a number of segments of equal width ([Fig. 19.12](#)). The total integral can be represented as

$$I = \int_{x_0}^{x_2} f(x) dx + \int_{x_2}^{x_4} f(x) dx + \cdots + \int_{x_{n-2}}^{x_n} f(x) dx \quad (19.25)$$

**FIGURE 19.12** Composite Simpson's 1/3 rule. The relative weights are depicted above the function values. Note that the method can be employed only if the number of segments is even.



Substituting Simpson's 1/3 rule for each integral yields

$$I = 2h \frac{f(x_0) + 4f(x_1) + f(x_2)}{6} + 2h \frac{f(x_2) + 4f(x_3) + f(x_4)}{6} + \cdots + 2h \frac{f(x_{n-2}) + 4f(x_{n-1})}{6}$$



or, grouping terms and using [Eq. \(19.15\)](#):

$$I = (b - a) \frac{f(x_0) + 4 \sum_{i=1,3,5}^{n-1} f(x_i) + 2 \sum_{j=2,4,6}^{n-2} f(x_j) + f(x_n)}{3n} \quad (19.26)$$

Notice that, as illustrated in [Fig. 19.12](#), an even number of segments must be utilized to implement the method. In addition, the coefficients “4” and “2” in [Eq. \(19.26\)](#) might seem peculiar at first glance. However, they follow naturally from Simpson's 1/3 rule. As illustrated in [Fig. 19.12](#), the odd points represent the middle term for each application and hence carry the weight of 4 from [Eq. \(19.23\)](#). The even points are common to adjacent applications and hence are counted twice.

An error estimate for the composite Simpson's rule is obtained in the same fashion as for the trapezoidal rule by summing the individual errors for the segments and averaging the derivative to yield

$$(19.27)$$

$$E_a = -\frac{(b-a)^5}{180n^4} f^{(4)}$$

where  $f^{(4)}$  is the average fourth derivative for the interval.

## EXAMPLE 19.4

### Composite Simpson's 1/3 Rule

#### Problem Statement

Use [Eq. \(19.26\)](#) with  $n = 4$  to estimate the integral of

$$f(x) = 0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5$$

from  $a = 0$  to  $b = 0.8$ . Employ [Eq. \(19.27\)](#) to estimate the error. Recall that the exact integral is 1.640533.

#### Solution

$n = 4 (h = 0.2)$ :

$$\begin{aligned} f(0) &= 0.2 & f(0.2) &= 1.288 \\ f(0.4) &= 2.456 & f(0.6) &= 3.464 \\ f(0.8) &= 0.232 \end{aligned}$$

From [Eq. \(19.26\)](#):

$$\begin{aligned} I &= 0.8 \frac{0.2 + 4(1.288 + 3.464) + 2(2.456) + 0.232}{12} = 1.623467 \\ E_t &= 1.640533 - 1.623467 = 0.017067 \quad \varepsilon_t = 1.04\% \end{aligned}$$

The estimated error [[Eq. \(19.27\)](#)] is

$$E_a = -\frac{(0.8)^5}{180(4)^4}(-2400) = 0.017067$$

which is exact (as was also the case for [Example 19.3](#)).

As in [Example 19.4](#), the composite version of Simpson's 1/3 rule is considered superior to the trapezoidal rule for most applications. However, as mentioned previously, it is limited to cases where the values are equispaced. Further, it is limited to situations where there are an even number of segments and an odd number of points. Consequently, as discussed in [Sec. 19.4.3](#), an odd-segment-even-point formula known as Simpson's 3/8 rule can be used in conjunction with the 1/3 rule to permit evaluation of both even and odd numbers of equispaced segments.

## 19.4.3 Simpson's 3/8 Rule

In a similar manner to the derivation of the trapezoidal and Simpson's 1/3 rule, a third-order Lagrange polynomial can be fit to four points and integrated to yield

$$I = \frac{3h}{8} [f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)]$$

where  $h = (b - a)/3$ . This equation is known as *Simpson's 3/8 rule* because  $h$  is multiplied by 3/8. It is the third Newton-Cotes closed integration formula. The 3/8 rule can also be expressed in the form of

[Eq. \(19.13\)](#):

$$I = (b - a) \frac{f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)}{8} \quad (19.28)$$

Thus, the two interior points are given weights of three-eighths, whereas the end points are weighted with one-eighth. Simpson's 3/8 rule has an error of

$$E_t = -\frac{3}{80} h^5 f^{(4)}(\xi)$$

or, because  $h = (b - a)/3$ :

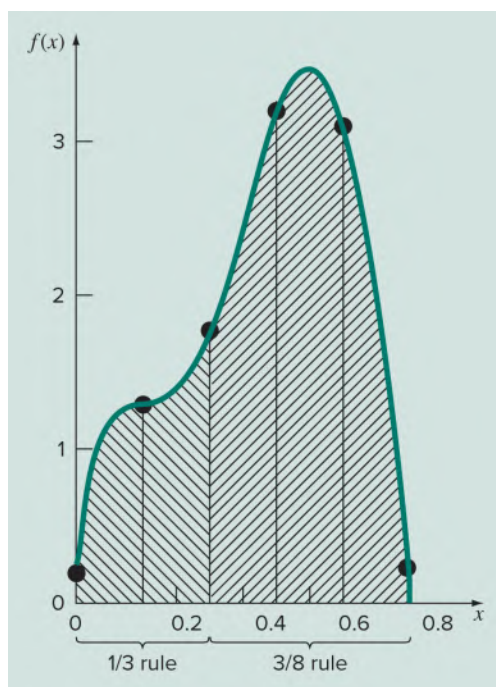
$$E_t = -\frac{(b - a)^5}{6480} f^{(4)}(\xi) \quad (19.29)$$

Because the denominator of [Eq. \(19.29\)](#) is larger than that for [Eq. \(19.24\)](#), the 3/8 rule is somewhat more accurate than the 1/3 rule.

Simpson's 1/3 rule is usually the method of preference because it attains third-order accuracy with three points rather than the four points required for the 3/8 version. However, the 3/8 rule has utility when the

number of segments is odd. For instance, in [Example 19.4](#) we used Simpson's rule to integrate the function for four segments. Suppose that you desired an estimate for five segments. One option would be to use a composite version of the trapezoidal rule as was done in [Example 19.2](#). This may not be advisable, however, because of the large truncation error associated with this method. An alternative would be to apply Simpson's 1/3 rule to the first two segments and Simpson's 3/8 rule to the last three ( [Fig. 19.13](#)). In this way, we could obtain an estimate with third-order accuracy across the entire interval.

**FIGURE 19.13** Illustration of how Simpson's 1/3 and 3/8 rules can be applied in tandem to handle multiple applications with odd numbers of intervals.



## EXAMPLE 19.5

### Simpson's 3/8 Rule

#### Problem Statement

(a) Use Simpson's 3/8 rule to integrate

$$f(x) = 0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5$$

from  $a = 0$  to  $b = 0.8$ . (b) Use it in conjunction with Simpson's 1/3 rule to integrate the same function for five segments.

## Solution

(a) A single application of Simpson's 3/8 rule requires four equally spaced points:

$$\begin{array}{ll} f(0) = 0.2 & f(0.2667) = 1.432724 \\ f(0.5333) = 3.487177 & f(0.8) = 0.232 \end{array}$$

Using  Eq. (19.28):

$$I = 0.8 \frac{0.2 + 3(1.432724 + 3.487177) + 0.232}{8} = 1.51917$$

(b) The data needed for a five-segment application ( $h = 0.16$ ) are

Page 472

$$\begin{array}{ll} f(0) = 0.2 & f(0.16) = 1.296919 \\ f(0.32) = 1.743393 & f(0.48) = 3.186015 \\ f(0.64) = 3.181929 & f(0.80) = 0.232 \end{array}$$

The integral for the first two segments is obtained using Simpson's 1/3 rule:

$$I = 0.32 \frac{0.2 + 4(1.296919) + 1.743393}{6} = 0.3803237$$

For the last three segments, the 3/8 rule can be used to obtain

$$I = 0.48 \frac{1.743393 + 3(3.186015 + 3.181929) + 0.232}{8} = 1.264754$$

The total integral is computed by summing the two results:

$$I = 0.3803237 + 1.264754 = 1.645077$$

# 19.5 HIGHER-ORDER NEWTON-COTES FORMULAS

As noted previously, the trapezoidal rule and both of Simpson's rules are members of a family of integrating equations known as the Newton-Cotes closed integration formulas. Some of the formulas are summarized in [Table 19.2](#) along with their truncation-error estimates.

**TABLE 19.2** Newton-Cotes closed integration formulas. The formulas are presented in the format of [Eq. 19.1](#). The weighting of the data points to estimate the average height is apparent. The step size is given by  $h = (b - a) / (n - 1)$ .

| Segments<br>(n) | Points | Name               | Formula                                                                               |   |
|-----------------|--------|--------------------|---------------------------------------------------------------------------------------|---|
| 1               | 2      | Trapezoidal rule   | $(b - a) \frac{f(x_0) + f(x_1)}{2}$                                                   | - |
| 2               | 3      | Simpson's 1/3 rule | $(b - a) \frac{f(x_0) + 4f(x_1) + f(x_2)}{6}$                                         | - |
| 3               | 4      | Simpson's 3/8 rule | $(b - a) \frac{f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)}{8}$                               | - |
| 4               | 5      | Boole's rule       | $(b - a) \frac{7f(x_0) + 32f(x_1) + 12f(x_2) + 32f(x_3) + 7f(x_4)}{90}$               | - |
| 5               | 6      |                    | $(b - a) \frac{19f(x_0) + 75f(x_1) + 50f(x_2) + 50f(x_3) + 75f(x_4) + 19f(x_5)}{288}$ | - |

Notice that, as was the case with Simpson's 1/3 and 3/8 rules, the five- and six-point formulas have the same order error. This general characteristic holds for the higher-point formulas and leads to the result that the even-segment-odd-point formulas (e.g., 1/3 rule and Boole's rule) are usually the methods of preference.

However, it must also be stressed that, in engineering and science practice, the higher-order (i.e., greater than four-point) formulas are not commonly used. Simpson's rules are sufficient for most applications. Accuracy can be improved by using the composite version. Furthermore, when the function is known and high accuracy is required, methods such as Romberg integration or Gauss quadrature, described in [Chap. 20](#), offer viable and attractive alternatives.

## 19.6 INTEGRATION WITH UNEQUAL SEGMENTS

To this point, all formulas for numerical integration have been based on equispaced data points. In practice, there are many situations where this assumption does not hold and we must deal with unequal-sized segments. For example, experimentally derived data are often of this type. For these cases, one method is to apply the trapezoidal rule to each segment and sum the results:

$$I = h_1 \frac{f(x_0) + f(x_1)}{2} + h_2 \frac{f(x_1) + f(x_2)}{2} + \cdots + h_n \frac{f(x_{n-1}) + f(x_n)}{2} \quad (19.30)$$

where  $h_i$  = the width of segment  $i$ . Note that this was the same approach used for the composite trapezoidal rule. The only difference between [Eq. \(19.16\)](#) and [\(19.30\)](#) is that the  $h$ 's in the former are constant.

### EXAMPLE 19.6

#### Trapezoidal Rule with Unequal Segments

##### Problem Statement

The information in [Table 19.3](#) was generated using the same polynomial employed in [Example 19.1](#). Use [Eq. \(19.30\)](#) to determine the integral for these data. Recall that the correct answer is 1.640533.

**TABLE 19.3** Data for  
 $f(x) = 0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5$   
 , with unequally spaced values of  $x$ .

| $x$  | $f(x)$   | $x$  | $f(x)$   |
|------|----------|------|----------|
| 0.00 | 0.200000 | 0.44 | 2.842985 |
| 0.12 | 1.309729 | 0.54 | 3.507297 |
| 0.22 | 1.305241 | 0.64 | 3.181929 |
| 0.32 | 1.743393 | 0.70 | 2.363000 |
| 0.36 | 2.074903 | 0.80 | 0.232000 |
| 0.40 | 2.456000 |      |          |

##### Solution



Applying [Eq. \(19.30\)](#) yields

$$I = 0.12 \frac{0.2 + 1.309729}{2} + 0.10 \frac{1.309729 + 1.305241}{2} + \cdots + 0.10 \frac{2.363 + 0.232}{2} =$$

which represents an absolute percent relative error of  $\varepsilon_t = 2.8\%$ .

## 19.6.1 Python Function: trapuneq

A Python function to implement the trapezoidal rule for unequally spaced data can be written as shown in [Fig. 19.14](#). Two arrays,  $x$  and  $y$ , holding the independent and dependent variable values are passed to the function. There are two error traps to ensure that (a) the two arrays are of the same length, and (b) the  $x$ 's are in ascending order. A for loop is employed to sum up the integral.

Page 474

**FIGURE 19.14** Python function trapuneq to implement trapezoidal rule for unequally spaced data.

```
def trapuneq(x,y):
    """
    trapezoidal rule for unequally spaced data
    returns an array of cumulative sums
    Input:
        x = array of independent variable values
        y = array of dependent variable values
        x and y arrays must be of equal length
        and in ascending order of x
    Output:
        s = array of sums
    """
    n = len(x)
    if len(y) != n: return 'x and y arrays must be of equal length'
    for i in range(n-1):
        if x[i+1] < x[i]: return 'x array not in ascending order'
    s = 0
    for k in range(0,n-1):
        s = s + (x[k+1]-x[k])*(y[k+1]+y[k])/2
    return s
```

An application of the trapuneq function to the problem solved in [Example 19.6](#) is

```
import numpy as np
x = np.array([0.,0.12,0.22,0.32,0.36,0.4,0.44,0.54,0.64,0.7,0.8])
y = 0.2 + 25.*x - 200.*x**2 + 675.*x**3 - 900.*x**4 + 400.*x**5
Iest = trapuneq(x,y)
print('Integral estimate = {0:6.4f}'.format(Iest))
```

with the result identical to that obtained in [Example 19.6](#):

Integral estimate = 1.5948

## 19.6.2 Python Functions: trapz and trap\_cumulative

The NumPy module in Python has a built-in function that evaluates integrals for data in the same fashion as the `trapuneq` function of [Fig. 19.14](#). Notably, the `x` and `y` arguments are reversed in calls to this function, `trapz`. Here, we use `trapz` to integrate the data from [Table 19.3](#).

```
import numpy as np
x = np.array([0.,0.12,0.22,0.32,0.36,0.4,0.44,0.54,0.64,0.7,0.8])
y = 0.2 + 25.*x - 200.*x**2 + 675.*x**3 - 900.*x**4 + 400.*x**5
Iest = np.trapz(y,x)
print('Integral estimate = {0:6.4f}'.format(Iest))
```

The result is the same as we obtained with `trapuneq`.

```
Integral estimate = 1.5948
```

There are applications where we would like to keep track of the cumulative integral over the domain of integration. An example of this is creating cumulative probability from probability density. The following is a modification of the `trapuneq` function of [Fig. 19.14](#) to return an array of cumulative integral sums instead of only the final sum.

```
def trap_cumulative(x,y):
    """
    trapezoidal rule for unequally spaced data
    returns an array of cumulative sums
    Input:
        x = array of independent variable values
        y = array of dependent variable values
        x and y arrays must be of equal length
        and in ascending order of x
    Output:
        s = array of sums
    """
    n = len(x)
    if len(y) != n: return 'x and y arrays must be of equal length'
    for i in range(n-1):
        if x[i+1] < x[i]: return 'x array not in ascending order'
    s = np.zeros((n))
    for k in range(1,n):
        s[k] = s[k-1] + (x[k]-x[k-1])*(y[k]+y[k-1])/2
    return s
```

---

Page 475

---

When this function is applied to the data from [Table 19.3](#), the result is

```
import numpy as np
x = np.array([0.,0.12,0.22,0.32,0.36,0.4,0.44,0.54,0.64,0.7,0.8])
y = 0.2 + 25.*x - 200.*x**2 + 675.*x**3 - 900.*x**4 + 400.*x**5
cumI = trap_cumulative(x,y)
print(cumI)

[0.          0.09058376  0.22133228  0.37376401  0.45012994  0.540748
 0.6467277  0.9642418  1.29870309  1.46505096  1.59480096]
```

You will note that the last element of the array is the final result we obtained using previous functions like `trapuneq` and `trapz`.

## EXAMPLE 19.7

### Using Numerical Integration to Compute the Cumulative Normal Probability

#### Problem Statement

One of the most important functions in probability and statistics is the normal or Gaussian density function, given by

$$f(x) = \frac{1}{\sqrt{2\pi} \sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \quad -\infty \leq x \leq +\infty$$

This function defines the classic “bell-shaped curve,” and to compute the probability of  $x$  falling in the interval  $[a, b]$ , one must evaluate the integral

$$P[a \leq x \leq b] = \int_a^b f(x) dx$$

It turns out that there is not an analytical solution to this integral; consequently, it must be evaluated numerically. Additionally, statisticians like to use a table of cumulative probabilities developed from

$$P[x \leq b] = F(x) = \int_{-\infty}^x f(x) dx$$

A practical result in this case is that

$$P[a \leq x \leq b] = F(b) - F(a)$$

Again, practically speaking, the density values for  $(x - \mu)/\sigma < -5$  are negligible.

- (a) Develop a Python function called `normprob` that has input arguments of  $a$ ,  $b$ ,  $\mu$ , and  $\sigma$  and returns  $P[a \leq x \leq b]$ . Test the function for  $a = -6$ ,  $b = 12$ ,  $\mu = 4$ , and  $\sigma = 5$ . Use 100 intervals. Express the probability in percent.
- (b) Develop a Python script that computes and plots the cumulative standard normal distribution ( $\mu = 0$  and  $\sigma = 1$ ) for  $-5 \leq x \leq 5$ . Use 100 intervals.

#### Solution

Since we know the analytical form of the Gaussian density, we can use our `trap` function for part (a).

```
mu = 4.  
sigma = 5.  
a = -6.  
b = 12.  
  
import numpy as np  
  
def normal_density(x):  
    f = 1/np.sqrt(2*np.pi)/sigma*np.exp(-(x-mu)**2/sigma**2/2)  
    return f  
  
P = trap(normal_density,a,b)*100  
print(P)
```

with the result

```
92.24197472531215
```

So, there is about a 92% probability that a random sample of  $x$  will fall between  $-6$  and  $12$  for these values of  $\mu$  and  $\sigma$ .

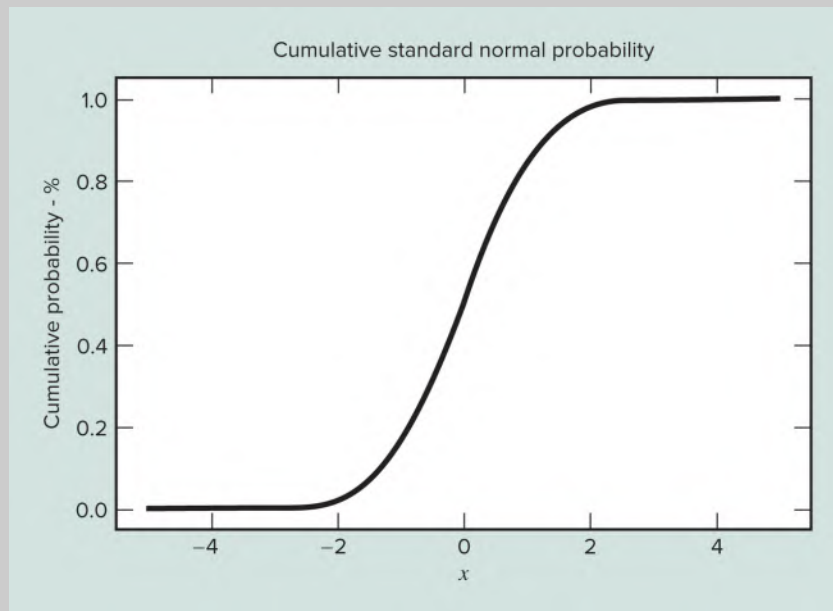
For part (b), we can use our `trap_cumulative` function as follows:

```
import numpy as np  
  
mu = 0.  
sigma = 1.  
a = -5.  
b = 5.  
x = np.linspace(a,b,100)  
f = 1/np.sqrt(2*np.pi)/sigma*np.exp(-(x-mu)**2/sigma**2/2)  
F = trap_cumulative(x,f)  
  
import pylab  
pylab.plot(x,F,c='k')  
pylab.grid()  
pylab.xlabel('x')  
pylab.ylabel('cumulative probability - %')  
pylab.title('Cumulative Standard Normal Probability')
```

The plot produced by this script is shown in  **Fig. 19.15**.

---

**FIGURE 19.15** Integration of the normal probability density to produce the cumulative probability.



# 19.7 OPEN METHODS

Recall from [Fig. 19.6b](#) that open integration formulas have limits that extend beyond the range of the data. [Table 19.4](#) summarizes the *Newton-Cotes open integration formulas*. The formulas are expressed in the form of [Eq. \(19.13\)](#) so that the weighting factors are evident. As with the closed versions, successive pairs of the formulas have the same-order error. The even-segment-odd-point formulas are usually the methods of preference because they require fewer points to attain the same accuracy as the odd-segment-even-point formulas.

**TABLE 19.4** Newton-Cotes open integration formulas. The formulas are presented in the format of [Eq. \(19.13\)](#). The weighting of the data points to estimate the average height is apparent. The step size is  $h = (b - a)/n$ .

| Segments ( <i>n</i> ) | Points | Name            | Formula                                                                      |
|-----------------------|--------|-----------------|------------------------------------------------------------------------------|
| 2                     | 1      | Midpoint method | $(b - a)f(x_1)$ (1)                                                          |
| 3                     | 2      |                 | $(b - a)\frac{f(x_1) + f(x_2)}{2}$ (3)                                       |
| 4                     | 3      |                 | $(b - a)\frac{2f(x_1) - f(x_2) + 2f(x_3)}{3}$ (1)                            |
| 5                     | 4      |                 | $(b - a)\frac{11f(x_1) + f(x_2) + f(x_3) + 11f(x_4)}{24}$ (5)                |
| 6                     | 5      |                 | $(b - a)\frac{11f(x_1) - 14f(x_2) + 26f(x_3) - 14f(x_4) + 11f(x_5)}{20}$ (4) |



The open formulas are not often used for definite integration. However, they have utility for analyzing improper integrals. In addition, they will have relevance to our discussion of methods for solving ordinary differential equations in [Chaps. 22](#) and [23](#).

## 19.8 MULTIPLE INTEGRALS

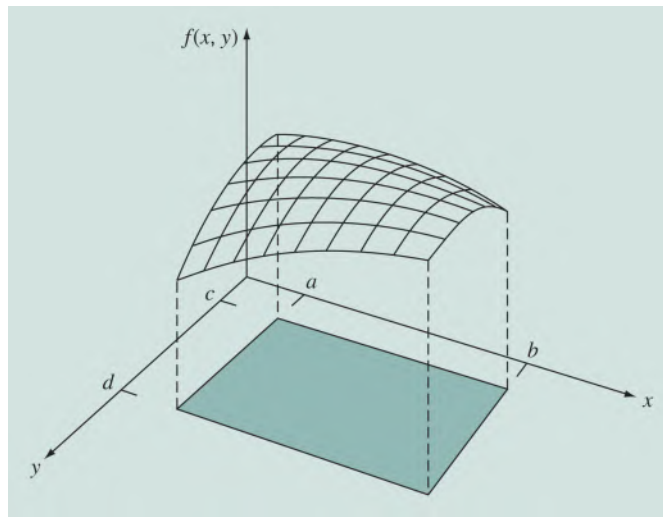
Multiple integrals are widely used in engineering and science. For example, a general equation to compute the average of a two-dimensional function can be written as [recall [Eq. \(19.7\)](#)]

$$\bar{f} = \frac{\int_c^d \left( \int_a^b f(x, y) \, dx \right) dy}{(d - c)(b - a)} \quad (19.31)$$

The numerator is called a *double integral*.

The techniques discussed in this chapter (and [Chap. 20](#)) can be readily employed to evaluate multiple integrals. A simple example would be to take the double integral of a function over a rectangular area ([Fig. 19.16](#)).

**FIGURE 19.16** Double integral as the area under the function surface.



Recall from calculus that such integrals can be computed as iterated integrals:

$$\int_c^d \left( \int_a^b f(x, y) \, dx \right) dy = \int_a^b \left( \int_c^d f(x, y) \, dy \right) dx \quad (19.32)$$

Thus, the integral in one of the dimensions is evaluated first. The result of this first integration is integrated in the second dimension. [Equation \(19.32\)](#) states that the order of integration is not important.

A numerical double integral would be based on the same idea. First, methods such as the composite trapezoidal or Simpson's rule would be applied in the first dimension with each value of the second dimension held constant. Then, the method would be applied to integrate the second dimension. The approach is illustrated in the following example.

## EXAMPLE 19.8

### Using Double Integral to Determine Average Temperature

#### Problem Statement

Suppose that the temperature of a rectangular heated plate is described by the following function:

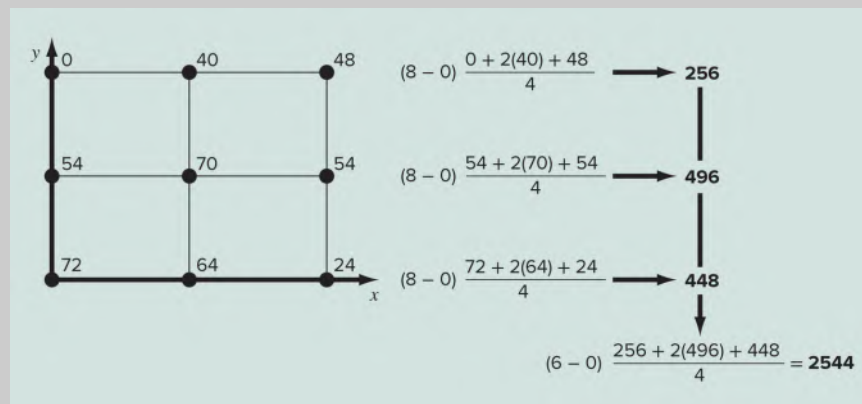
$$T(x, y) = 2xy + 2x - x^2 - 2y^2 + 72$$

If the plate is 8 m long ( $x$  dimension) and 6 m wide ( $y$  dimension), compute the average temperature.

#### Solution

First, let us merely use two-segment applications of the trapezoidal rule in each dimension. The temperatures at the necessary  $x$  and  $y$  values are depicted in [Fig. 19.17](#). Note that a simple average of these values is 47.33. The function can also be evaluated analytically to yield a result of 58.66667.

**FIGURE 19.17** Numerical evaluation of a double integral using the two-segment trapezoidal rule.



To make the same evaluation numerically, the trapezoidal rule is first implemented along the  $x$  dimension for each  $y$  value. These values are then integrated along the  $y$  dimension to give the final result of 2544. Dividing this by the area yields the average temperature as  $2544 / (6 \times 8) = 53$ .



Now, we can apply a single-segment Simpson's 1/3 rule in the same fashion. This results in an integral of 2816 and an average of 58.66667, which is exact. Why does this occur? Recall that Simpson's 1/3 rule yielded perfect results for cubic polynomials. Since the highest-order term in the function is second order, the same exact result occurs for the present case.

For higher-order algebraic functions as well as transcendental functions, it would be necessary to use composite applications to attain accurate integral estimates. In addition, [Chap. 20](#) introduces techniques that are more efficient than the Newton-Cotes formulas for evaluating integrals of given functions. These often provide a superior means to implement the numerical integrations for multiple integrals.

## 19.8.1 Python Functions: dblquad and tplquad

The SciPy `integrate` submodule in Python has functions to implement both double and triple integrations.

The `dblquad` function evaluates the integral

$$\int_a^b \int_{g(x)}^{h(x)} f(x, y) dy dx$$

It allows for the limits of the integral on  $y$  to be determined as functions of  $x$ . When these limits are constants, one defines the  $g(x)$  and  $h(x)$  functions to be constants. The syntax for `dblquad` is

```
from scipy.integrate import dblquad
(y, abserr) = dblquad(func, a, b, gfun, hfun)
```

We can use this function as follows to compute the double integral from [Example 19.8](#):

```
from scipy.integrate import dblquad
def f(y,x):
    return 2*x*y + 2*x - x**2 - 2*y**2 + 72.
Iest, Ierr = dblquad(f, 0., 8., lambda x: 0., lambda x: 6.)
print(Iest)
print(Ierr)
```

The results are

```
2816.0
3.126388037344441e-11
```

SciPy also includes a `tplquad` function to integrate in three dimensions. The syntax for use is similar and details can be found in the SciPy Reference Guide available via Spyder Help.

## COMPUTING WORK WITH NUMERICAL INTEGRATION

### Background

The calculation of work is an important component of many areas of engineering and science. The general formula is

$$\text{Work} = \text{force} \times \text{distance}$$

When you were introduced to this concept in high school physics, simple applications were presented using forces that remained constant throughout the displacement. For example, if a force of 10 N was used to pull a block a distance of 5 m, the work would be calculated as 50 J (1 joule = 1 N·m).

Although such a simple computation is useful for introducing the concept, realistic problem settings are usually more complex. For example, suppose that the force varies during the course of the calculation. In such cases, the work equation is reexpressed as

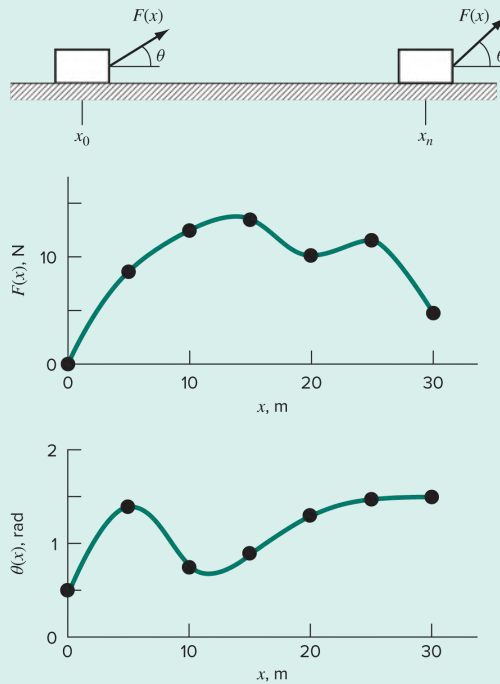
$$W = \int_{x_0}^{x_n} F(x) dx \quad (19.33)$$

where  $W$  = work (J),  $x_0$  and  $x_n$  = the initial and final positions (m), respectively, and  $F(x)$  = a force that varies as a function of position (N). If  $F(x)$  is easy to integrate, [Eq. \(19.33\)](#) can be evaluated analytically. However, in a realistic problem setting, the force might not be expressed in such a manner. In fact, when analyzing measured data, the force might be available only in tabular form. For such cases, numerical integration is the only viable option for the evaluation.

Further complexity is introduced if the angle between the force and the direction of movement also varies as a function of position ([Fig. 19.18](#)). The work equation can be modified further to account for this effect, as in

$$W = \int_{x_0}^{x_n} F(x) \cos[\theta(x)] dx \quad (19.34)$$

**FIGURE 19.18** The case of a variable force acting on a block. For this case the angle, as well as the magnitude, of the force varies.



Again, if  $F(x)$  and  $\theta(x)$  are simple functions, [Eq. \(19.34\)](#) might be solved analytically. However, as in [Fig. 19.18](#), it is more likely that the functional relationship is complicated. For this situation, numerical methods provide the only alternative for determining the integral.

Suppose that you have to perform the computation for the situation depicted in [Fig. 19.18](#). Although the figure shows the continuous values for  $F(x)$  and  $\theta(x)$ , assume that, because of experimental constraints, you are provided with only discrete measurements at  $x = 5\text{ m}$  intervals ([Table 19.5](#)). Use single and composite versions of the trapezoidal rule and Simpson's 1/3 and 3/8 rules to compute work for these data.

**TABLE 19.5** Data for force  $F(x)$  and angle  $\theta(x)$  as a function of position  $x$ .

| $x, \text{ m}$ | $F(x), \text{ N}$ | $\theta, \text{ rad}$ | $F(x)\cos\theta$ |
|----------------|-------------------|-----------------------|------------------|
| 0              | 0.0               | 0.50                  | 0.0000           |
| 5              | 9.0               | 1.40                  | 1.5297           |
| 10             | 13.0              | 0.75                  | 9.5120           |
| 15             | 14.0              | 0.90                  | 8.7025           |
| 20             | 10.5              | 1.30                  | 2.8087           |
| 25             | 12.0              | 1.48                  | 1.0881           |
| 30             | 5.0               | 1.50                  | 0.3537           |

## Solution

The results of the analysis are summarized in [Table 19.6](#). A percent relative error  $\varepsilon_t$  was computed in reference to a true value of the integral of 129.52 that was estimated on the basis of values taken from [Fig. 19.18](#) at 1-m intervals.

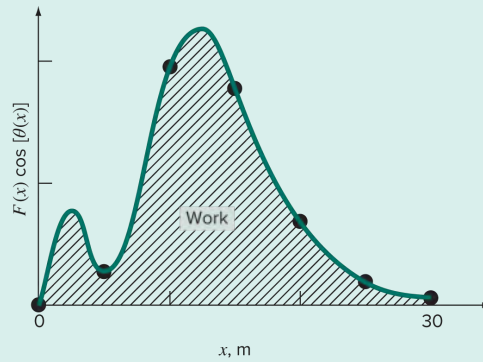
**TABLE 19.6** Estimates of work calculated using the trapezoidal rule and Simpson’s rules. The percent relative error  $\varepsilon_t$  as computed in reference to a true value of the integral (129.52 J) that was estimated on the basis of values at 1-m intervals.

| Technique          | Segments | Work   | $\varepsilon_t$ , % |
|--------------------|----------|--------|---------------------|
| Trapezoidal rule   | 1        | 5.31   | 95.9                |
|                    | 2        | 133.19 | 2.84                |
|                    | 3        | 124.98 | 3.51                |
|                    | 6        | 119.09 | 8.05                |
| Simpson’s 1/3 rule | 2        | 175.82 | 35.75               |
|                    | 6        | 117.13 | 9.57                |
| Simpson’s 3/8 rule | 3        | 139.93 | 8.04                |

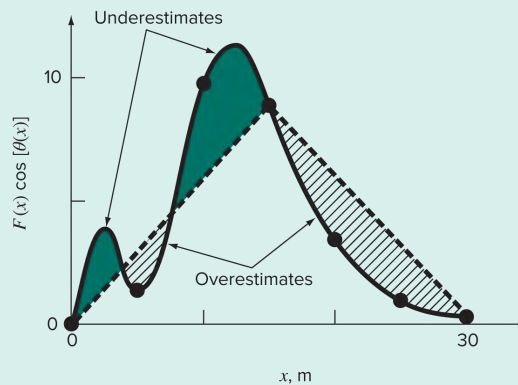
The results are interesting because the most accurate outcome occurs for the simple two-segment trapezoidal rule. More refined estimates using more segments, as well as Simpson’s rules, yield less accurate results.

The reason for this apparently counterintuitive result is that the coarse spacing of the points is not adequate to capture the variations of the forces and angles. This is particularly evident in [Fig. 19.19](#), where we have plotted the continuous curve for the product of  $F(x)$  and  $\cos[\theta(x)]$ . Notice how the use of seven points to characterize the continuously varying function misses the two peaks at  $x = 2.5$  and 12.5 m. The omission of these two points effectively limits the accuracy of the numerical integration estimates in [Table 19.6](#). The fact that the two-segment trapezoidal rule yields the most accurate result is due to the chance positioning of the points for this particular problem ([Fig. 19.20](#)).

**FIGURE 19.19** A continuous plot of  $F(x)\cos[\theta(x)]$  versus position with the seven discrete points used to develop the numerical integration estimates in [Table 19.6](#). Notice how the use of seven points to characterize this continuously varying function misses two peaks at  $x = 2.5$  and 12.5 m.



**FIGURE 19.20** Graphical depiction of why the two-segment trapezoidal rule yields a good estimate of the integral for this particular case. By chance, the use of two trapezoids happens to lead to an even balance between positive and negative errors.



The conclusion to be drawn from [Fig. 19.20](#) is that an adequate number of measurements must be made to compute integrals accurately. For the present case, if data were available at  $F(2.5) \cos(\theta(2.5)) = 3.9007$  and  $F(12.5) \cos(\theta(12.5)) = 11.3940$ , we could determine an improved integral estimate. For example, using the Python `trapz` function, we could compute

```
import numpy as np
x = np.array([0., 2.5, 5., 10., 12.5, 15., 20., 25., 30.])
y = np.array([0., 3.9007, 1.5297, 9.5120, 11.3940,
              8.7025, 2.8087, 1.0881, 0.3537])

Iest = np.trapz(y,x)
print(Iest)
```

with the result displayed as

```
132.64575000000002
```

Including the two additional points yields an improved integral estimate of  $132.64575(\varepsilon_t = 2.16\%)$ . Thus, the inclusion of the additional data incorporates peaks that were missed previously and, consequently, leads to a better result.

# PROBLEMS

---

**19.1** Derive [Eq. \(19.4\)](#) by integrating [Eq. \(19.3\)](#).

**19.2** Evaluate the following integral:

$$\int_0^4 (1 - e^{-x}) dx$$

(a) analytically, (b) single application of the trapezoidal rule, (c) composite trapezoidal rule with  $n = 2$  and 4, (d) single application of Simpson's 1/3 rule, (e) composite Simpson's 1/3 rule with  $n = 4$ , (f) Simpson's 3/8 rule, and (g) composite Simpson's rule with  $n = 5$ . For each of the numerical estimates (b) through (g), determine the true percent relative error based on (a).

**19.3** Evaluate the following integral:

$$\int_0^{\pi/2} (8 + 4 \cos(x)) dx$$

(a) analytically, (b) single application of the trapezoidal rule, (c) composite trapezoidal rule with  $n = 2$  and 4, (d) single application of Simpson's 1/3 rule, (e) composite Simpson's 1/3 rule with  $n = 4$ , (f) Simpson's 3/8 rule, and (g) composite Simpson's rule with  $n = 5$ . For each of the numerical estimates (b) through (g), determine the true percent relative error based on (a).

**19.4** Evaluate the following integral:

$$\int_{-2}^4 [1 - x - 4x^3 + 2x^5] dx$$

(a) analytically, (b) single application of the trapezoidal rule, (c) composite trapezoidal rule with  $n = 2$  and 4, (d) single application of Simpson's 1/3 rule, (e) Simpson's 3/8 rule, (f) Boole's rule. For each of the numerical estimates (b) through (f), determine the true percent relative error based on (a).

**19.5** The function

$$f(x) = e^{-x}$$

can be used to generate the following table of unequally spaced data:

|             |   |        |        |        |        |        |        |
|-------------|---|--------|--------|--------|--------|--------|--------|
| <b>x</b>    | 0 | 0.1    | 0.3    | 0.5    | 0.7    | 0.95   | 1.2    |
| <b>f(x)</b> | 1 | 0.9048 | 0.7408 | 0.6065 | 0.4966 | 0.3867 | 0.3012 |

Evaluate the integral of this function from  $a = 0$  to  $b = 1.2$  using (a) analytical means, (b) the trapezoidal rule, and (c) a combination of trapezoidal and Simpson's rules wherever possible to attain the highest accuracy. For (b) and (c), compute the true percent relative error.

**19.6** Evaluate the double integral

$$\int_{-2}^2 \int_0^4 [x^2 - 3y^2 + xy^3] dx dy$$

(a) analytically, (b) using the composite trapezoidal rule with  $n = 2$ , (c) using single applications of Simpson's 1/3 rule, and (d) using the `dblquad` function. For (b), (c), and (d), compute the true percent relative error.

**19.7** Evaluate the triple integral

$$\int_{-4}^4 \int_0^6 \int_{-1}^3 [x^3 - 2yz] dx dy dz$$

(a) analytically, (b) using single applications of Simpson's 1/3 rule, and (c) using the `tplquad` function. For (b) and (c), compute the true percent relative error.

**19.8** Determine the distance traveled from the following velocity data:

|          |   |   |      |     |     |   |   |     |   |    |
|----------|---|---|------|-----|-----|---|---|-----|---|----|
| <b>t</b> | 1 | 2 | 3.25 | 4.5 | 6   | 7 | 8 | 8.5 | 9 | 10 |
| <b>v</b> | 5 | 6 | 5.5  | 7   | 8.5 | 8 | 6 | 7   | 7 | 5  |

- (a) Use the trapezoidal rule. In addition, determine the average velocity.
- (b) Fit the data with a cubic equation using polynomial regression. Integrate the polynomial to determine the distance.

**19.9** Water exerts pressure on the upstream face of a dam as shown in  **Fig. P19.9**. The pressure can be described by the hydrostatic relationship

$$p(z) = \rho g(D - z)$$

where  $p(z)$  = pressure (Pa) exerted on the dam face at elevation  $z$ (m) above the reservoir bottom,  $\rho$  = density of water  $\left(1000 \text{ kg/m}^3\right)$ , and  $D$  = elevation of the water surface (m) above the reservoir bottom. According to the relationship above, pressure increases linearly with depth, as illustrated in [Fig. 19.9a](#). Omitting atmospheric pressure, also known as gauge pressure, the force on the dam,  $f_t$ , can be determined by multiplying pressure times the area of the dam face, as shown in [Fig. P19.9b](#). Because both pressure and area vary with depth, the total force is obtained by evaluating

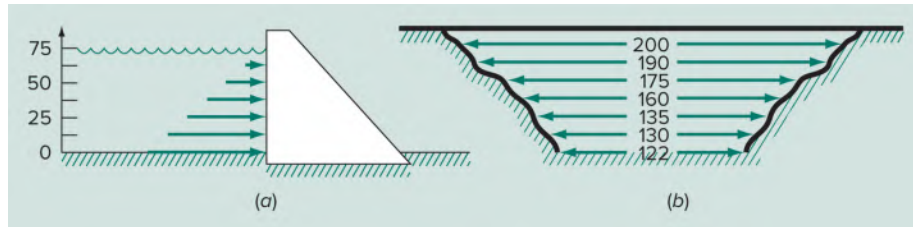
$$f_t = \int_0^D [\rho g u(z)(D - z)] dz$$

where  $u(z)$  = width of the dam face at elevation  $z$ . The *line of action* can also be obtained by evaluating

$$d = \frac{\int_0^D [\rho g z u(z)(D - z)] dz}{\int_0^D [\rho g u(z)(D - z)] dz}$$

Use Simpson's rule to compute  $f_t$  and  $d$ .

**FIGURE P19.9**



**19.10** The force on a sailboat mast can be represented by the following function:

$$f(z) = 200 \left( \frac{z}{5 + z} \right) e^{-2z/H}$$

where  $f(z)$  = force per unit mast height,  $z$  = elevation above the deck, and  $H$  = height of the mast. The total force exerted on the mast can be determined by integrating this function over the height of the mast:

$$F = \int_0^H f(z) dz$$



The line of action can also be determined by integration:

$$d = \frac{\int_0^H z f(z) dz}{\int_0^H f(z) dz}$$

- (a) Use the composite trapezoidal rule to compute  $F$  and  $d$  for the case where  $H = 30$  with  $n = 6$ .  
 (b) Repeat (a) but use the composite Simpson's 1/3 rule.

**19.11** A wind force distributed against the side of a skyscraper is measured as

|                                      |      |      |      |      |      |
|--------------------------------------|------|------|------|------|------|
| <b>Height <math>l</math>, m</b>      | 0    | 30   | 60   | 90   | 120  |
| <b>Force, <math>F(l)</math>, N/m</b> | 0    | 340  | 1200 | 1550 | 2700 |
| <b>Height <math>l</math>, m</b>      | 150  | 180  | 210  | 240  |      |
| <b>Force, <math>F(l)</math>, N/m</b> | 3100 | 3200 | 3500 | 3750 |      |

Compute the net force and line of action due to this distributed wind.

**19.12** An 11-m beam is subjected to a load, and the shear force follows the equation

$$V(x) = 5 + 0.25x^2$$

where  $V$  = the shear force, and  $x$  is the distance along the beam. We know that  $V = dM/dx$ , where  $M$  is the bending moment. Integration yields the relationship

$$M = M_0 + \int_0^x V dx$$

If  $M_0$  is zero and  $x = 11$ , calculate  $M$  using (a) analytical integration, (b) composite trapezoidal rule, and (c) composite Simpson's rule. For (b) and (c), use 1-m increments.

**19.13** The total mass of a variable density rod of variable cross-section is given by

$$m = \int_0^L \rho(x) A(x) dx$$

where  $m$  = mass,  $\rho$  = density,  $A$  = cross-sectional area,  $x$  = the distance along the rod, and  $L$  = the total length of the rod. The following data have been measured for a 20-m rod. Determine the mass of the rod

in grams using the Python functions `trapz` and `simps`. What is the percentage difference between the two results?

|                                             |      |      |      |      |      |      |      |
|---------------------------------------------|------|------|------|------|------|------|------|
| <b><i>x</i>, m</b>                          | 0    | 4    | 6    | 8    | 12   | 16   | 20   |
| <b><i>ρ</i>, g/cm<sup>3</sup></b>           | 4.00 | 3.95 | 3.89 | 3.80 | 3.60 | 3.41 | 3.30 |
| <b><i>A<sub>c</sub></i>, cm<sup>2</sup></b> | 100  | 103  | 106  | 110  | 120  | 133  | 150  |

**19.14** A transportation engineering study requires that you determine the number of cars that pass through an intersection during morning rush hour. You stand at the side of the road and count the number of cars that pass every four minutes at several times as tabulated below. Use the best numerical method to determine (a) the total number of cars that pass between 7:30 and 9:15, and (b) the rate of cars going through the intersection per minute. Hint: Be careful with units.

Page 484

|                              |      |      |      |      |      |      |
|------------------------------|------|------|------|------|------|------|
| <b>Time (hr)</b>             | 7:30 | 7:45 | 8:00 | 8:15 | 8:45 | 9:15 |
| <b>Rate (cars per 4 min)</b> | 18   | 23   | 14   | 24   | 20   | 9    |

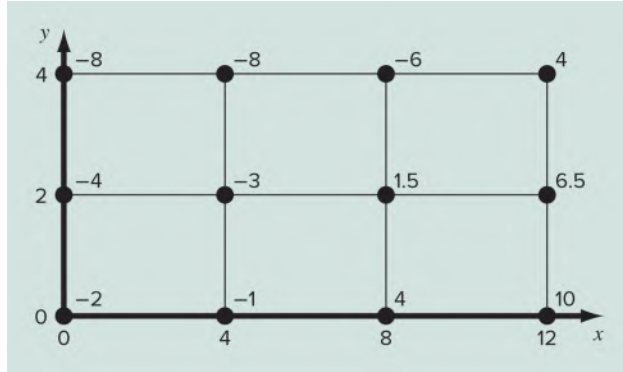
**19.15** Determine the average value for the data shown in  **Fig. P19.15**. Perform the integral needed for the average in the order shown by the following equation:

$$I = \int_{x_0}^{x_n} \left[ \int_{y_0}^{y_m} f(x, y) dy \right] dx$$

Note that the average value of a function,  $f(x)$ , over a domain,  $a \leq x \leq b$  is

$$f = \frac{\int_a^b f(x) dx}{b - a}$$

**FIGURE P19.15**



**19.16** In the transportation of fluids through pipelines, it is critical to know how much fluid has passed over a given time period. This amount is used by the supplier of the fluid to invoice the recipient. Examples include the distribution of water for agricultural irrigation and the transfer of an intermediate chemical product like ethylene from the plant that produces it from natural gas to the plant that uses it to make polyethylene. This accounting procedure is called flow totalization and amounts to an integration of flow rate measurements over a given period of time. Since monetary transfer charges are involved, this is a matter of high interest. Another way of stating this is that the total amount transferred is equal to the area under the flow rate versus time curve. The table below represents flow rate measurements of irrigation water transferred from a channel to a farmer's land over a period of 24 hours. Use numerical integration to estimate the total amount of water delivered.

| Time (h)                         | 0     | 4     | 8     | 12    | 16    | 20    | 24    |
|----------------------------------|-------|-------|-------|-------|-------|-------|-------|
| Flow Rate<br>(m <sup>3</sup> /s) | 0.906 | 0.595 | 1.331 | 1.841 | 1.218 | 2.095 | 0.538 |

**19.17** The cross-sectional area of a channel can be computed as

$$A = \int_0^B H(y) dy$$

where  $B$  = the total channel width (m),  $H$  = the channel depth (m) at a given  $y$ , and  $y$  = the distance from the bank (m). In a similar fashion, the channel flow rate,  $Q$  (m<sup>3</sup>/s), can be computed as

$$Q = \int_0^B U(y)H(y) dy$$

and  $U$  = the water velocity (m/s) at distance  $y$ . Use these relationships and a numerical method to determine  $A$  and  $Q$  for the following data:

|               |      |      |      |      |      |      |
|---------------|------|------|------|------|------|------|
| <b>y, m</b>   | 0    | 2    | 4    | 5    | 6    | 9    |
| <b>H, m</b>   | 0.5  | 1.3  | 1.25 | 1.8  | 1    | 0.25 |
| <b>U, m/s</b> | 0.03 | 0.06 | 0.05 | 0.13 | 0.11 | 0.02 |

**19.18** The average concentration of a substance,  $\bar{c}$  (g/m<sup>3</sup>) in a lake where the area,  $A$ (m<sup>2</sup>), varies with depth,  $z$ (m) can be computed by integration as follows:

$$\bar{c} = \frac{\int_0^Z c(z)A(z)dz}{\int_0^Z A(z)dz}$$

where  $Z$  = the total depth (m). Determine the average concentration based on the following data:

|                                       |        |        |        |        |        |
|---------------------------------------|--------|--------|--------|--------|--------|
| <b>z, m</b>                           | 0      | 4      | 8      | 12     | 16     |
| <b>A, 10<sup>6</sup>m<sup>2</sup></b> | 9.8175 | 5.1051 | 1.9635 | 0.3927 | 0.0000 |
| <b>c, g/m<sup>3</sup></b>             | 10.2   | 8.5    | 7.4    | 5.2    | 4.1    |

**19.19** As was done in [Section 19.9](#), determine the work performed if a constant force of 1 N applied at an angle  $\theta$  results in the following displacements. Use the `trap_cumulative` function to determine the cumulative work and plot the result versus  $\theta$ .

|                                 |   |    |     |     |     |     |     |
|---------------------------------|---|----|-----|-----|-----|-----|-----|
| <b>x, m</b>                     | 0 | 1  | 2.8 | 3.9 | 3.8 | 3.2 | 1.3 |
| <b><math>\theta</math>, deg</b> | 0 | 30 | 60  | 90  | 120 | 150 | 180 |

**19.20** Compute work, as described in [Section 19.9](#), but use the following equations for  $F(x)$  and  $\theta(x)$ :

$$F(x) = 1.6x - 0.045x^2$$

$$\theta(x) = -0.00055x^3 + 0.0123x^2 + 0.13x$$

The force is in newtons and the angle is in radians. Perform the integration from  $x = 0$  to 30 m.

**19.21** As specified in the following table, a manufactured spherical particle has a density that varies as a function of the distance from its center ( $r = 0$ ):

---

|                       |      |      |      |      |      |
|-----------------------|------|------|------|------|------|
| $r, \text{ mm}$       | 0    | 0.12 | 0.24 | 0.36 | 0.49 |
| $\rho(\text{g/cm}^3)$ | 6    | 5.81 | 5.14 | 4.29 | 3.39 |
| $r, \text{ mm}$       | 0.62 | 0.79 | 0.86 | 0.93 | 1    |
| $\rho(\text{g/cm}^3)$ | 2.7  | 2.19 | 2.1  | 2.04 | 2    |

Use numerical integration to estimate the particle's mass (g) and average density ( $\text{g/cm}^3$ ).

**19.22** As specified in the following table, the earth's density varies as a function of the distance from its center ( $r = 0$ ).

Page 485

|                       |      |      |      |      |      |      |
|-----------------------|------|------|------|------|------|------|
| $r, \text{ km}$       | 0    | 1100 | 1500 | 2450 | 3400 | 3630 |
| $\rho(\text{g/cm}^3)$ | 13   | 12.4 | 12   | 11.2 | 9.7  | 5.7  |
| $r, \text{ km}$       | 4500 | 5380 | 6060 | 6280 | 6380 |      |
| $\rho(\text{g/cm}^3)$ | 5.2  | 4.7  | 3.6  | 3.4  | 3    |      |

Use numerical integration to estimate the earth's mass (in metric tonnes), and average density (in  $\text{kg/m}^3$ ). Present vertically stacked subplots of density versus radius and mass versus radius.

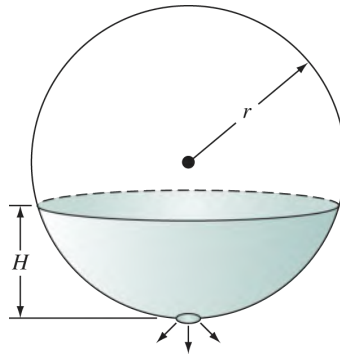
**19.23** A spherical tank has a circular orifice in its bottom through which the liquid drains (  Fig. P19.23). The following data have been collected for the flow rate through the orifice as a function of time. At the final time, the tank is empty.

|                            |       |       |       |       |       |       |
|----------------------------|-------|-------|-------|-------|-------|-------|
| $t, \text{ s}$             | 0     | 500   | 1000  | 1500  | 2200  | 2900  |
| $Q, \text{ m}^3/\text{hr}$ | 10.55 | 9.576 | 9.072 | 8.640 | 8.100 | 7.560 |
| $t, \text{ s}$             | 3600  | 4300  | 5200  | 6500  | 7000  | 7500  |
| $Q, \text{ m}^3/\text{hr}$ | 7.020 | 6.480 | 5.688 | 4.752 | 3.348 | 1.404 |

Write a Python script (a) to estimate the volume of fluid (in liters) drained over the entire period and (b) estimate the initial liquid level. For this tank,  $r = 1.5 \text{ m}$ . Note: The relationship between liquid volume,  $V$ , radius,  $r$ , and depth,  $H$ , is given by

$$V = \pi H^2 \frac{3r - H}{3}$$

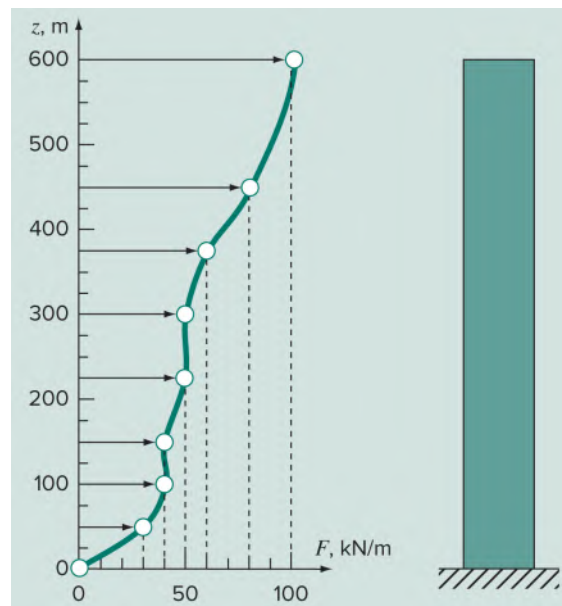
**FIGURE P19.23**



**19.24** Develop a Python function, called `simp3`, to implement the composite Simpson's 1/3 rule for equispaced data. The function should check for (1) the input arrays being the same length, and (2) the data being equispaced, and return error messages if this is not the case. If there are only two data points, use trapezoidal rule. If there are an even number of data points and an odd number of segments, use Simpson's 3/8 rule for the last three data points. Test your function with the data from [Prob. 19.11](#). Compare your result with that from using the `simps` function from the SciPy `integrate` submodule. Check all the features of your function by creating errors (1) and (2), reducing the data set to just two points, and changing from an even to odd number of intervals.

**19.25** During a storm, a high wind blows along one side of a rectangular skyscraper as depicted in [Fig. P19.25](#). As described in [Prob. 19.9](#), use the best lower-order Newton-Cotes formulas (trapezoidal, Simpson's 1/3, and Simpson's 3/8 rules) to determine (a) the force on the building in kN, and the line of force in meters.

**FIGURE P19.25**



**19.26** The following data are provided for the velocity of an object as a function of time:

|                                    |   |    |    |    |    |    |    |    |    |
|------------------------------------|---|----|----|----|----|----|----|----|----|
| <b><math>t, \text{ s}</math></b>   | 0 | 4  | 8  | 12 | 16 | 20 | 24 | 28 | 30 |
| <b><math>v, \text{ m/s}</math></b> | 0 | 18 | 31 | 42 | 50 | 56 | 61 | 65 | 70 |

- (a) Limiting yourself to trapezoidal rule and Simpson's 1/3 and 3/8 rules, make the best estimate of how far the object travels from 0 to 30 s.
- (b) Employ the results from (a) to compute the average velocity.

**19.27** A gas is expanded as a piston retracts in an engine cylinder. The expansion follows the law

$$PV^\gamma = \text{constant}$$

where  $P$  = pressure,  $V$  = volume,  $\gamma$  = the ratio of  $C_P/C_V$ . The latter are the heat capacities at constant pressure and at constant volume. If this ratio is 1.3, and the gas is expanded from  $P = 2550 \text{ kPa}$  to  $210 \text{ kPa}$ , with the final volume being  $0.75 \text{ m}^3$ , compute the work done by the gas. Note: Work is given by the thermodynamic relationship

$$W = \int_{V_1}^{V_2} P dV$$

**19.28** The pressure  $p$  and volume  $v$  of a given mass of gas are connected by the relation

$$\left(p + \frac{a}{v^2}\right)(v - b) = k$$

where  $a$ ,  $b$ , and  $k$  are constants. Express  $p$  in terms of  $v$  and write a Python script to compute work done by the gas in expanding from an initial to final volume. Test your solution with  $a = 0.01 \text{ kPa}\cdot\text{m}^3$ ,  $b = 0.001 \text{ m}^3$ , initial pressure and volume of  $100 \text{ kPa}$  and  $1 \text{ m}^3$ , respectively, and final volume  $= 2 \text{ m}^3$ . Compute the final pressure also.

# Numerical Integration of Functions

## CHAPTER OBJECTIVES

The primary objective of this chapter is to introduce you to numerical methods for integrating given functions. Specific objectives and topics covered are

- Understanding how Richardson extrapolation provides a means to create a more accurate integral estimate by combining two less accurate estimates.
- Understanding how Gauss quadrature provides superior integral estimates by picking optimal abscissas at which to evaluate the function.
- Understanding how adaptive quadrature efficiently evaluates integrals by using refined segmentation where functions change rapidly and coarse segmentation where they change gradually.
- Knowing how to use Python's SciPy `integrate` submodule's family of functions for integrating mathematical functions.



## 20.1 INTRODUCTION

---

In [Chap. 19](#), we noted that functions to be integrated numerically will typically be of two forms: a table of values or a function. The form of the data has an important influence on the approaches that can be used to evaluate the integral. For tabulated information, you are limited by the number of points that are given. In contrast, if the function is available, you can generate as many values of  $f(x)$  as are required to attain acceptable accuracy.

At face value, the composite Simpson's  $1/3$  rule might seem to be a reasonable tool for such problems. Although it is certainly adequate for many problems, there are more efficient methods that are available. This chapter is devoted to three such techniques, which capitalize on the ability to generate function values to develop efficient schemes for numerical integration.

The first technique is based on *Richardson extrapolation*, which is a method for combining two numerical integral estimates to obtain a third, more accurate value. The computational algorithm for implementing Richardson extrapolation in a highly efficient manner is called *Romberg integration*. This technique can be used to generate an integral estimate within a prespecified error tolerance.

The second method is called *Gauss quadrature*. Recall that, in [Chap. 19](#), values of  $f(x)$  for the Newton-Cotes formulas were determined at specified values of  $x$ . For example, if we used the trapezoidal rule to determine an integral, we were constrained to take the weighted average of  $f(x)$  at the ends of the interval. Gauss-quadrature formulas employ  $x$  values that are positioned between the integration limits in such a manner that a much more accurate integral estimate results.

The third approach is called *adaptive quadrature*. This technique applies the composite Simpson's  $1/3$  rule to subintervals of the integration range in a way that allows error estimates to be computed. These error estimates are then used to determine whether more refined estimates are required for a subinterval. In this way, more refined segmentation is only used where it is necessary.

## 20.2 ROMBERG INTEGRATION

Romberg integration is one technique that is designed to attain efficient numerical integrals of functions. It is quite similar to the techniques discussed in [Chap. 19](#) in the sense that it is based on successive application of the trapezoidal rule. However, through mathematical manipulations, superior results are attained for less effort.

### 20.2.1 Richardson Extrapolation

Techniques are available to improve the results of numerical integration on the basis of the integral estimates themselves. Generally called *Richardson extrapolation*, these methods use two estimates of an integral to compute a third, more accurate approximation.

The estimate and the error associated with the composite trapezoidal rule can be represented generally as

$$I = I(h) + E(h)$$

where  $I$  = the exact value of the integral,  $I(h)$  = the approximation from an  $n$ -segment application of the trapezoidal rule with step size  $h = (b - a)/n$ , and  $E(h)$  = the truncation error. If we make two separate estimates using step sizes of  $h_1$  and  $h_2$  and have exact values for the error:

$$I(h_1) + E(h_1) = I(h_2) + E(h_2) \quad (20.1)$$

Now, recall that the error of the composite trapezoidal rule can be represented approximately by

[Eq. \(19.21\)](#) [with  $n = (b - a)/h$ ]:

$$E \cong -\frac{b-a}{12} h^2 f'' \quad (20.2)$$

If it is assumed that  $f''$  is constant regardless of step size, [Eq. \(20.2\)](#) can be used to determine that the ratio of the two errors will be

$$\frac{E(h_1)}{E(h_2)} \cong \frac{h_1^2}{h_2^2} \quad (20.3)$$

This calculation has the important effect of removing the term  $f''$  from the computation. In so doing, we have made it possible to utilize the information embodied by [Eq. \(20.2\)](#) without prior knowledge of

the function's second derivative. To do this, we rearrange [Eq. \(20.3\)](#) to give

$$E(h_1) \cong E(h_2) \left( \frac{h_1}{h_2} \right)^2$$

which can be substituted into [Eq. \(20.1\)](#):

$$I(h_1) + E(h_2) \left( \frac{h_1}{h_2} \right)^2 = I(h_2) + E(h_2)$$

which can be solved for

$$E(h_2) = \frac{I(h_1) - I(h_2)}{1 - (h_1/h_2)^2}$$

Thus, we have developed an estimate of the truncation error in terms of the integral estimates and their step sizes. This estimate can then be substituted into

$$I = I(h_2) + E(h_2)$$

to yield an improved estimate of the integral:

$$I = I(h_2) + \frac{1}{(h_1/h_2)^2 - 1} [I(h_2) - I(h_1)] \quad (20.4)$$

It can be shown (Ralston and Rabinowitz, 1978) that the error of this estimate is  $O(h^4)$ . Thus, we have combined two trapezoidal rule estimates of  $O(h^2)$  to yield a new estimate of  $O(h^4)$ . For the special case where the interval is halved ( $h_2 = h_1/2$ ), this equation becomes

$$I = \frac{4}{3} I(h_2) - \frac{1}{3} I(h_1) \quad (20.5)$$

## EXAMPLE 20.1

## Richardson Extrapolation

### Problem Statement

Use Richardson extrapolation to evaluate the integral of

$f(x) = 0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5$  from  $a = 0$  to  $b = 0.8$ .

### Solution

Single and composite applications of the trapezoidal rule can be used to evaluate the integral:

| Segments | $h$ | Integral | $\epsilon_t$ |
|----------|-----|----------|--------------|
| 1        | 0.8 | 0.1728   | 89.5%        |
| 2        | 0.4 | 1.0688   | 34.9%        |
| 4        | 0.2 | 1.4848   | 9.5%         |

Richardson extrapolation can be used to combine these results to obtain improved estimates of the integral. For example, the estimates for one and two segments can be combined to yield

$$I = \frac{4}{3}(1.0688) - \frac{1}{3}(0.1728) = 1.367467$$

The error of the improved integral is  $E_t = 1.640533 - 1.367467 = 0.273067$  ( $\epsilon_t = 16.6\%$ ), which is superior to the estimates upon which it was based.

In the same manner, the estimates for two and four segments can be combined to give

$$I = \frac{4}{3}(1.4848) - \frac{1}{3}(1.0688) = 1.623467$$

which represents an error of  $E_t = 1.640533 - 1.623467 = 0.017067$  ( $\epsilon_t = 1.0\%$ ).

 **Equation (20.4)** provides a way to combine two applications of the trapezoidal rule with error  $O(h^2)$  to compute a third estimate with error  $O(h^4)$ . This approach is a subset of a more general method for combining integrals to obtain improved estimates. For instance, in  **Example 20.1**, we computed two improved integrals of  $O(h^4)$  on the basis of three trapezoidal rule estimates. These two improved integrals can, in turn, be combined to yield an even better value with  $O(h^6)$ . For the special case where the original trapezoidal estimates are based on successive halving of the step size, the equation used for  $O(h^6)$  accuracy is

(20.6)

$$I = \frac{16}{15}I_m - \frac{1}{15}I_l$$

where  $I_m$  and  $I_l$  are the more and less accurate estimates, respectively. Similarly, two  $O(h^6)$  results can be combined to compute an integral that is  $O(h^8)$  using

$$I = \frac{64}{63}I_m - \frac{1}{63}I_l \quad (20.7)$$

## EXAMPLE 20.2

### Higher-Order Corrections

#### Problem Statement

In [Example 20.1](#), we used Richardson extrapolation to compute two integral estimates of  $O(h^4)$ . Utilize [Eq. \(20.6\)](#) to combine these estimates to compute an integral with  $O(h^6)$ .

#### Solution

The two integral estimates of  $O(h^4)$  obtained in [Example 20.1](#) were 1.367467 and 1.623467. These values can be substituted into [Eq. \(20.6\)](#) to yield

$$I = \frac{16}{15}(1.623467) - \frac{1}{15}(1.367467) = 1.640533$$

which is the exact value of the integral.

## 20.2.2 The Romberg Integration Algorithm

Notice that the coefficients in each of the extrapolation equations [[Eqs. \(20.5\)](#), [\(20.6\)](#), and [\(20.7\)](#)] add up to 1. Thus, they represent weighting factors that, as accuracy increases, place relatively greater weight on the superior integral estimate. These formulations can be expressed in a general form that is well suited for computer implementation:

$$I_{j,k} = \frac{4^{k-1} I_{j+1,k-1} - I_{j,k-1}}{4^{k-1} - 1} \quad (20.8)$$

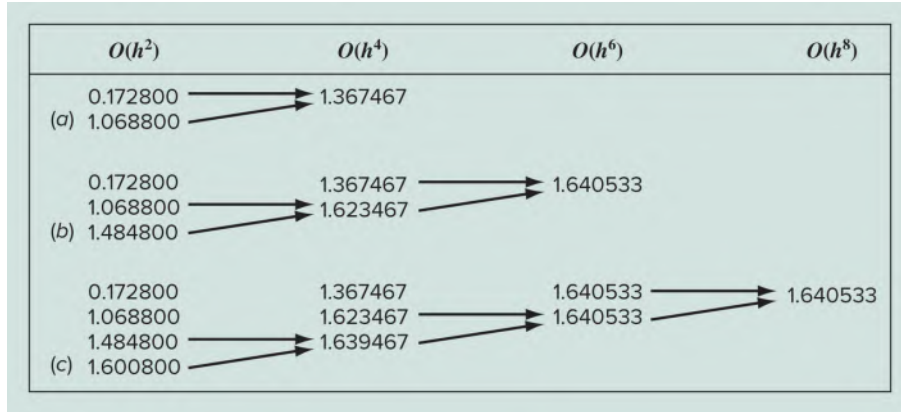
where  $I_{j+1,k-1}$  and  $I_{j,k-1}$  = the more and less accurate integrals, respectively, and  $I_{j,k}$  = the improved integral. The index  $k$  signifies the level of the integration, where  $k = 1$  corresponds to the original trapezoidal rule estimates,  $k = 2$  corresponds to the  $O(h^4)$  estimates,  $k = 3$  to the  $O(h^6)$ , and so forth. The index  $j$  is used to distinguish between the more ( $j + 1$ ) and the less ( $j$ ) accurate estimates. For example, for  $k = 2$  and  $j = 1$ , Eq. (20.8) becomes

$$I_{1,2} = \frac{4I_{2,1} - I_{1,1}}{3}$$

which is equivalent to Eq. (20.5).

The general form represented by Eq. (20.8) is attributed to Romberg, and its systematic application to evaluate integrals is known as *Romberg integration*. Figure 20.1 is a graphical depiction of the sequence of integral estimates generated using this approach. Each step ( $a$ ,  $b$ , and  $c$ ) corresponds to a single iteration. The first column contains the trapezoidal rule evaluations that are designated  $I_{j,1}$ , where  $j = 1$  is for a single-segment application (step size is  $b - a$ ),  $j = 2$  is for a two-segment application [step size is  $(b - a)/2$ ],  $j = 3$ , is for a four-segment application [step size is  $(b - a)/4$ ], and so forth. The other columns of the matrix are generated by systematically applying Eq. (20.8) to obtain successively better estimates of the integral.

**FIGURE 20.1** Graphical depiction of the sequence of integral estimates generated using Romberg integration. (a) First iteration. (b) Second iteration. (c) Third iteration.



For example, the first iteration (Fig. 20.1a) involves computing the one- and two-segment trapezoidal rule estimates ( $I_{1,1}$  and  $I_{2,1}$ ). Equation (20.8) is then used to compute the element  $I_{1,2} = 1.367467$ , which has an error of  $O(h^4)$ .

Now, we must check to determine whether this result is adequate for our needs. As in other approximate methods in this book, a termination, or stopping, criterion is required to assess the accuracy of the

results. One method that can be employed for the present purposes is

$$|\varepsilon_a| = \left| \frac{I_{1,k} - I_{2,k-1}}{I_{1,k}} \right| \times 100\% \quad (20.9)$$

where  $\varepsilon_a$  = an estimate of the percent relative error. Thus, as was done previously in other iterative processes, we compare the new estimate with a previous value. For [Eq. \(20.9\)](#), the previous value is the most accurate estimate from the previous level of integration (i.e., the  $k - 1$  level of integration with  $j = 2$ ). When the change between the old and new values as represented by  $\varepsilon_a$  is below a prespecified error criterion  $\varepsilon_s$ , the computation is terminated. For [Fig. 20.1a](#), this evaluation indicates the following percent change over the course of the first iteration:

$$|\varepsilon_a| = \left| \frac{1.367467 - 1.068800}{1.367467} \right| \times 100\% = 21.8\%$$

The object of the second iteration ([Fig. 20.1b](#)) is to obtain the  $O(h^6)$  estimate— $I_{1,3}$ . To do this, a four-segment trapezoidal rule estimate,  $I_{3,1} = 1.4848$ , is determined. Then, it is combined with  $I_{2,1}$  using [Eq. \(20.8\)](#) to generate  $I_{2,2} = 1.623467$ . The result is, in turn, combined with  $I_{1,2}$  to yield  $I_{1,3} = 1.640533$ . [Equation \(20.9\)](#) can be applied to determine that this result represents a change of 1.0% when compared with the previous result  $I_{2,2}$ .

The third iteration ([Fig. 20.1c](#)) continues the process in the same fashion. In this case, an eight-segment trapezoidal estimate is added to the first column, and then [Eq. \(20.8\)](#) is applied to compute successively more accurate integrals along the lower diagonal. After only three iterations, because we are evaluating a fifth-order polynomial, the result ( $I_{1,4} = 1.640533$ ) is exact.

Romberg integration is more efficient than the trapezoidal rule and Simpson's rules. For example, for determination of the integral as shown in [Fig. 20.1](#), Simpson's 1/3 rule would require about a 48-segment application in double precision to yield an estimate of the integral to seven significant digits: 1.640533. In contrast, Romberg integration produces the same result based on combining one-, two-, four-, and eight-segment trapezoidal rules—that is, with only 15 function evaluations!

[Figure 20.2](#) presents a Python function for Romberg integration. By using loops, this algorithm implements the method in an efficient manner. Note that the function uses another function, `trap`, to implement the composite trapezoidal rule evaluations (recall [Fig. 19.10](#)). By studying the function, you will observe that it is tricky to translate the index subscripts in [Eqs. \(20.8\)](#) and [\(20.9\)](#) to Python's zero-based array indices. Here is additional Python code showing how the `romberg` function can be used to determine the integral of the polynomial from [Example 20.1](#):

```
def f(x):
    return 0.2 + 25.*x - 200.*x**2 + 675.*x**3 - 900*x**4 + 400*x**5

Ival,errel,iter = romberg(f,0.,0.8)
print(Ival)
print(errel)
```

```
print(iter)
```

**FIGURE 20.2** Python function to implement Romberg integration.

```
def romberg(func,a,b,es=1.e-8,maxit=30):
    """
    Romberg integration quadrature
    input:
        func = name of function to be integrated
        a, b = integration limits
        es = desired relative error (default = 1.e-8)
        maxit = iteration limit (default = 30)
    output:
        q = integral estimate
        ea = approximate relative error achieved
        iter = iterations taken
    """
    n = 1
    I = np.zeros((2*maxit,maxit+1))
    I[0,0] = trap(func,a,b,n)
    for iter in range(1,maxit+1):
        n = 2*iter
        I[iter,0] = trap(func,a,b,n)
        for k in range(1,iter+1):
            j = iter-k
            I[j,k] = (4**(k)*I[j+1,k-1] - I[j,k-1])/(4**(k)-1)
        ea = abs((I[0,iter]-I[1,iter-1])/I[0,iter])
        if ea <= es: break
    q = I[0,iter]
    return q,ea,iter
```

Here are the results displayed:

Page 491

```
1.6405333333333318
0.0
3
```

As was illustrated in [Example 20.2](#), the function returns the exact result in three iterations, and, consequently, the error displayed is zero.

The `integrate` submodule of SciPy also has a `romberg` function with the following syntax:

```
from scipy.integrate import romberg
result = romberg(func,a,b)
```

There are also optional arguments for absolute and relative error tolerances and an argument allowing the passing of arguments through to `func`. We can test the function using the code

```
from scipy.integrate import romberg
result = romberg(f,0.,0.8)
print(result)
```

and obtain

```
1.6405333333333363
```

which is virtually identical to the result we obtained using the function in [Fig. 20.2](#).



## 20.3 GAUSS QUADRATURE

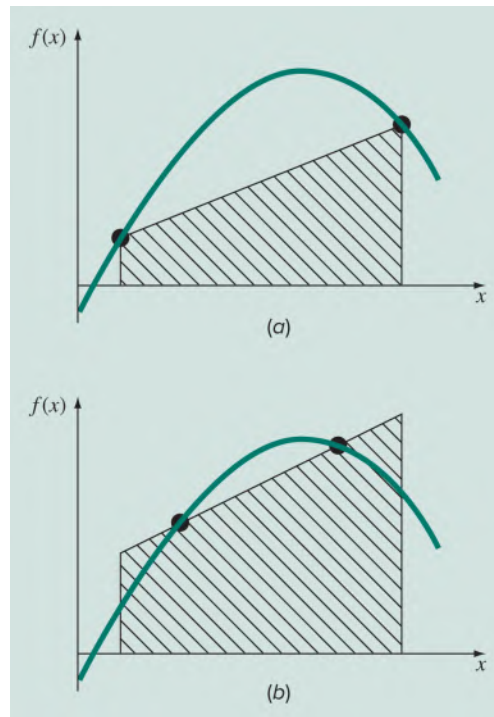
In [Fig. 19](#), we employed the Newton-Cotes equations. A characteristic of these formulas (with the exception of the special case of unequally spaced data) was that the integral estimate was based on evenly spaced function values. Consequently, the location of the base points used in these equations was predetermined or fixed.

For example, as depicted in [Fig. 20.3a](#), the trapezoidal rule is based on taking the area under the straight line connecting the function values at the ends of the integration interval. The formula that is used to compute this area is

$$I \cong (b - a) \frac{f(a) + f(b)}{2} \quad (20.10)$$

where  $a$  and  $b$  = the limits of integration and  $b - a$  = the width of the integration interval. Because the trapezoidal rule must pass through the end points, there are cases such as [Fig. 20.3a](#), where the formula results in a large error.

**FIGURE 20.3** (a) Graphical depiction of the trapezoidal rule as the area under the straight line joining fixed end points. (b) An improved integral estimate obtained by taking the area under the straight line passing through two intermediate points. By positioning these points wisely, the positive and negative errors are better balanced, and an improved integral estimate results.





Now, suppose that the constraint of fixed base points was removed and we were free to evaluate the area under a straight line joining any two points on the curve. By positioning these points wisely, we could define a straight line that would balance the positive and negative errors. Hence, as in [Fig. 20.3b](#), we would arrive at an improved estimate of the integral.

*Gauss quadrature* is the name for a class of techniques to implement such a strategy. The particular Gauss quadrature formulas described in this section are called *Gauss-Legendre* formulas. Before describing the approach, we will show how numerical integration formulas such as the trapezoidal rule can be derived using the method of undetermined coefficients. This method will then be employed to develop the Gauss-Legendre formulas.

## 20.3.1 Method of Undetermined Coefficients

In [Chap. 19](#), we derived the trapezoidal rule by integrating a linear interpolating polynomial and by geometrical reasoning. The method of undetermined coefficients offers a third approach that also has utility in deriving other integration techniques such as Gauss quadrature.

To illustrate the approach, [Eq. \(20.10\)](#) is expressed as

Page 492

$$I \cong c_0 f(a) + c_1 f(b) \quad (20.11)$$

where the  $c$ 's = constants. Now realize that the trapezoidal rule should yield exact results when the function being integrated is a constant or a straight line. Two simple equations that represent these cases are  $y = 1$  and  $y = x$  ([Fig. 20.4](#)). Thus, the following equalities should hold:

$$c_0 + c_1 = \int_{-(b-a)/2}^{(b-a)/2} 1 \, dx$$

and

$$-c_0 \frac{b-a}{2} + c_1 \frac{b-a}{2} = \int_{-(b-a)/2}^{(b-a)/2} x \, dx$$

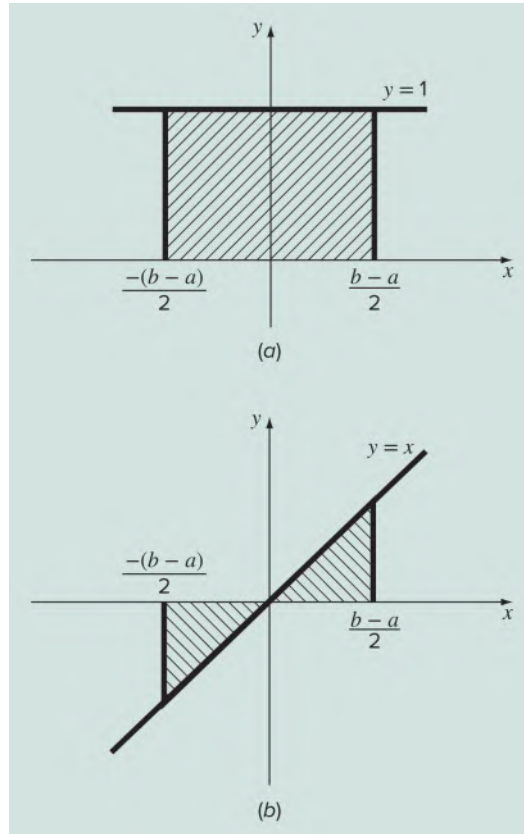
or, evaluating the integrals,

$$c_0 + c_1 = b - a$$

and

$$-c_0 \frac{b-a}{2} + c_1 \frac{b-a}{2} = 0$$

**FIGURE 20.4** Two integrals that should be evaluated exactly by the trapezoidal rule: (a) a constant and (b) a straight line.



These are two equations with two unknowns that can be solved for

$$c_0 = c_1 = \frac{b-a}{2}$$

which, when substituted back into [Eq. \(20.11\)](#), gives

$$I = \frac{b-a}{2} f(a) + \frac{b-a}{2} f(b)$$

which is equivalent to the trapezoidal rule.

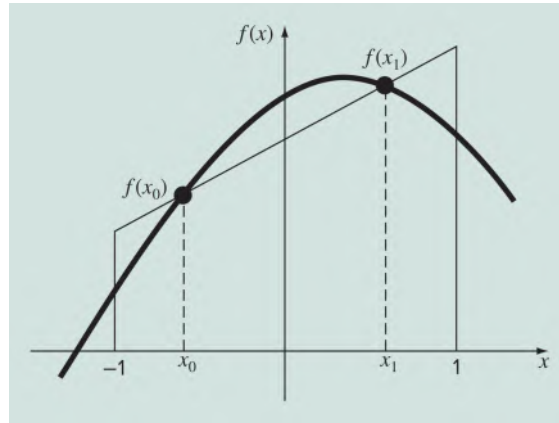
## 20.3.2 Derivation of the Two-Point Gauss-Legendre Formula

Just as was the case for the previous derivation of the trapezoidal rule, the object of Gauss quadrature is to determine the coefficients of an equation of the form

$$I \cong c_0 f(x_0) + c_1 f(x_1) \quad (20.12)$$

where the  $c$ 's = the unknown coefficients. However, in contrast to the trapezoidal rule that used fixed end points  $a$  and  $b$ , the function arguments  $x_0$  and  $x_1$  are not fixed at the end points, but are unknowns ( [Fig. 20.5](#)). Thus, we now have a total of four unknowns that must be evaluated, and consequently, we require four conditions to determine them exactly.

**FIGURE 20.5** Graphical depiction of the unknown variables  $x_0$  and  $x_1$  for integration by Gauss quadrature.



Just as for the trapezoidal rule, we can obtain two of these conditions by assuming that [Eq. \(20.12\)](#) fits the integral of a constant and a linear function exactly. Then, to arrive at the other two conditions, we merely extend this reasoning by assuming that it also fits the integral of a parabolic ( $y = x^2$ ) and a cubic ( $y = x^3$ ) function. By doing this, we determine all four unknowns and in the bargain derive a linear two-point integration formula that is exact for cubics. The four equations to be solved are

$$c_0 + c_1 = \int_{-1}^1 1 \, dx = 2 \quad (20.13)$$

$$(20.14)$$

$$c_0x_0 + c_1x_1 = \int_{-1}^1 x \, dx = 0$$

$$c_0x_0^2 + c_1x_1^2 = \int_{-1}^1 x^2 \, dx = \frac{2}{3} \quad (20.15)$$

$$c_0x_0^3 + c_1x_1^3 = \int_{-1}^1 x^3 \, dx = 0 \quad (20.16)$$

Equations (20.13) through (20.16) can be solved simultaneously for the four unknowns. First, solve Eq. (20.14) for  $c_1$  and substitute the result into Eq. (20.16), which can be solved for

$$x_0^2 = x_1^2$$

Since  $x_0$  and  $x_1$  cannot be equal, this means that  $x_0 = -x_1$ . Substituting this result into Eq. (20.14) yields  $c_0 = c_1$ . Consequently from Eq. (20.13) it follows that

$$c_0 = c_1 = 1$$

Substituting these results into Eq. (20.15) gives

$$x_0 = -\frac{1}{\sqrt{3}} = -0.5773503 \dots$$

$$x_1 = \frac{1}{\sqrt{3}} = 0.5773503 \dots$$

Therefore, the two-point Gauss-Legendre formula is

$$I = f\left(\frac{-1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right) \quad (20.17)$$

Thus, we arrive at the interesting result that the simple addition of the function values at  $x = -1/\sqrt{3}$  and  $1/\sqrt{3}$  yields an integral estimate that is third-order accurate.

Notice that the integration limits in [Eqs. \(20.13\)](#) through [\(20.16\)](#) are from  $-1$  to  $1$ . This was done to simplify the mathematics and to make the formulation as general as possible. A simple change of variable can be used to translate other limits of integration into this form. This is accomplished by assuming that a new variable  $x_d$  is related to the original variable  $x$  in a linear fashion, as in

$$x = a_1 + a_2 x_d \quad (20.18)$$

If the lower limit,  $x = a$ , corresponds to  $x_d = -1$ , these values can be substituted into [Eq. \(20.18\)](#) to yield

$$a = a_1 + a_2(-1) \quad (20.19)$$

Similarly, the upper limit,  $x = b$ , corresponds to  $x_d = 1$ , to give

$$b = a_1 + a_2(1) \quad (20.20)$$

[Equations \(20.19\)](#) and [\(20.20\)](#) can be solved simultaneously for

$$a_1 = \frac{b+a}{2} \quad \text{and} \quad a_2 = \frac{b-a}{2} \quad (20.21)$$

which can be substituted into [Eq. \(20.18\)](#) to yield

$$x = \frac{(b+a) + (b-a)x_d}{2} \quad (20.22)$$

This equation can be differentiated to give

$$dx = \frac{b-a}{2} dx_d \quad (20.23)$$

[Equations \(20.22\)](#) and [\(20.23\)](#) can be substituted for  $x$  and  $dx$ , respectively, in the equation to be integrated. These substitutions effectively transform the integration interval without changing the value of the integral. The following example illustrates how this is done in practice.

## EXAMPLE 20.3

### Two-Point Gauss-Legendre Formula

#### Problem Statement

Use  Eq. (20.17) to evaluate the integral of

$$f(x) = 0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5$$

between the limits  $x = 0$  to  $0.8$ . The exact value of the integral is 1.640533.

#### Solution

Before integrating the function, we must perform a change of variable so that the limits are from  $-1$  to  $+1$ . To do this, we substitute  $a = 0$  and  $b = 0.8$  into  Eqs. (20.22) and  (20.23) to yield

$$x = 0.4 + 0.4x_d \quad \text{and} \quad dx = 0.4dx_d$$

Both of these can be substituted into the original equation to yield

$$\begin{aligned} & \int_0^{0.8} (0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5) dx \\ &= \int_{-1}^1 \left[ 0.2 + 25(0.4 + 0.4x_d) - 200(0.4 + 0.4x_d)^2 + 675(0.4 + 0.4x_d)^3 - 900(0.4 + \right. \end{aligned}$$

Therefore, the right-hand side is in the form that is suitable for evaluation using Gauss quadrature.

The transformed function can be evaluated at  $x_d = -1/\sqrt{3}$  as 0.516741 and at  $x_d = 1/\sqrt{3}$  as

1.305837. Therefore, the integral according to  Eq. (20.17) is

$0.516741 + 1.305837 = 1.822578$ , which represents a percent relative error of  $-11.1\%$ . This result is comparable in magnitude to a four-segment application of the trapezoidal rule or a single application of Simpson's  $1/3$  and  $3/8$  rules. This latter result is to be expected because Simpson's rules are also third-order accurate. However, because of the clever choice of base points, Gauss quadrature attains this accuracy on the basis of only two function evaluations.

## 20.3.3 Higher-Point Formulas

Beyond the two-point formula described in the previous section, higher-point versions can be developed in the general form

$$I \cong c_0 f(x_0) + c_1 f(x_1) + \cdots + c_{n-1} f(x_{n-1}) \quad (20.24)$$

where  $n$  = the number of points. Values for  $c$ 's and  $x$ 's for up to and including the six-point formula are summarized in [Table 20.1](#).

**TABLE 20.1** Weighting factors and function arguments used in Gauss-Legendre formulas.

| Points | Weighting Factors                                                                                                                                                          | Function Arguments                                                                                                                                                            | Truncation Error      |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| 1      | $c_0 = 2$                                                                                                                                                                  | $x_0 = 0.0$                                                                                                                                                                   | $\cong f^{(2)}(\xi)$  |
| 3      | $c_0 = 1$<br>$c_1 = 1$                                                                                                                                                     | $x_0 = -1/\sqrt{3}$<br>$x_1 = 1/\sqrt{3}$                                                                                                                                     | $\cong f^{(4)}(\xi)$  |
| 3      | $c_0 = 5/9$<br>$c_1 = 8/9$<br>$c_2 = 5/9$                                                                                                                                  | $x_0 = -\sqrt{3/5}$<br>$x_1 = 0.0$<br>$x_2 = \sqrt{3/5}$                                                                                                                      | $\cong f^{(6)}(\xi)$  |
| 4      | $c_0 = (18 - \sqrt{30})/36$<br>$c_1 = (18 + \sqrt{30})/36$<br>$c_2 = (18 + \sqrt{30})/36$<br>$c_3 = (18 - \sqrt{30})/36$                                                   | $x_0 = -\sqrt{525 + 70\sqrt{30}}/35$<br>$x_1 = -\sqrt{525 - 70\sqrt{30}}/35$<br>$x_2 = \sqrt{525 - 70\sqrt{30}}/35$<br>$x_3 = \sqrt{525 + 70\sqrt{30}}/35$                    | $\cong f^{(8)}(\xi)$  |
| 5      | $c_0 = (322 - 13\sqrt{70})/900$<br>$c_1 = (322 + 13\sqrt{70})/900$<br>$c_2 = 128/225$<br>$c_3 = (322 + 13\sqrt{70})/900$<br>$c_4 = (322 - 13\sqrt{70})/900$                | $x_0 = -\sqrt{245 + 14\sqrt{70}}/21$<br>$x_1 = -\sqrt{245 - 14\sqrt{70}}/21$<br>$x_2 = 0.0$<br>$x_3 = \sqrt{245 - 14\sqrt{70}}/21$<br>$x_4 = \sqrt{245 + 14\sqrt{70}}/21$     | $\cong f^{(10)}(\xi)$ |
| 6      | $c_0 = 0.171324492379170$<br>$c_1 = 0.360761573048139$<br>$c_2 = 0.467913934572691$<br>$c_3 = 0.467913934572691$<br>$c_4 = 0.360761573048131$<br>$c_5 = 0.171324492379170$ | $x_0 = -0.932469514203152$<br>$x_1 = -0.661209386466265$<br>$x_2 = -0.238619186083197$<br>$x_3 = 0.238619186083197$<br>$x_4 = 0.661209386466265$<br>$x_5 = 0.932469514203152$ | $\cong f^{(12)}(\xi)$ |

## EXAMPLE 20.4

### Three-Point Gauss-Legendre Formula

#### Problem Statement

Use the three-point formula from [Table 20.1](#) to estimate the integral for the same function as in [Example 20.3](#).



## Solution

According to  **Table 20.1**, the three-point formula is

$$I = 0.5555556 f(-0.7745967) + 0.8888889 f(0) + 0.5555556 f(0.7745967)$$

which is equal to

$$I = 0.2813013 + 0.8732444 + 0.4859876 = 1.640533$$

which is exact.

Because Gauss quadrature requires function evaluations at nonuniformly spaced points within the integration interval, it is not appropriate for cases where the function is unknown. Thus, it is not suited for engineering problems that deal with tabulated data. However, where the function is known, its efficiency can be a decided advantage. This is particularly true when numerous integral evaluations must be performed.

## 20.4 ADAPTIVE QUADRATURE

Although Romberg integration is more efficient than the composite Simpson's 1/3 rule, both use equally spaced points. This constraint does not take into account that some functions have regions of relatively abrupt changes where more refined spacing might be required. Hence, to achieve a desired accuracy, fine spacing must be applied everywhere even though it is only needed for the regions of sharp change.

Adaptive quadrature methods remedy this situation by automatically adjusting the step size so that small steps are taken in regions of sharp variations and larger steps are taken where the function changes gradually.

### 20.4.1 Python function: quadadapt

*Adaptive quadrature* methods accommodate the fact that many functions have regions of high variability along with other sections where change is gradual. They accomplish this by adjusting the step size so that small intervals are used in regions of rapid variations and larger intervals are used where the function changes gradually. Many of these techniques are based on applying the composite Simpson's 1/3 rule to subintervals in a fashion that is very similar to the way in which the composite trapezoidal rule was used in Richardson extrapolation. That is, the 1/3 rule is applied at two levels of refinement, and the difference between these two levels is used to estimate the truncation error. If the truncation error is acceptable, no further refinement is required, and the integral estimate for the subinterval is deemed acceptable. If the error estimate is too large, the step size is refined and the process repeated until the error falls to acceptable levels. The total integral is then computed as the summation of the integral estimates for the subintervals.

The theoretical basis of the approach can be illustrated for an interval  $x = a$  to  $x = b$  with a width of  $h_1 = b - a$ . A first estimate of the integral can be estimated with Simpson's 1/3 rule:

Page 497

$$I(h_1) = \frac{h_1}{6} [f(a) + 4f(c) + f(b)] \quad (20.25)$$

where  $c = (a + b)/2$ .

As in Richardson extrapolation, a more refined estimate can be obtained by halving the step size. That is, by applying the composite Simpson's 1/3 rule with  $n = 4$ :

$$I(h_2) = \frac{h_2}{6} [f(a) + 4f(d) + 2f(c) + 4f(e) + f(b)] \quad (20.26)$$

where  $d = (a + c)/2$ ,  $e = (c + b)/2$ , and  $h_2 = h_1/2$ .

Because both  $I(h_1)$  and  $I(h_2)$  are estimates of the same integral, their difference provides a measure of the error. That is,

$$E \cong I(h_2) - I(h_1) \quad (20.27)$$

In addition, the estimate and error associated with either application can be represented generally as

$$I = I(h) + E(h) \quad (20.28)$$

where  $I$  = the exact value of the integral,  $I(h)$  = the approximation from an  $n$ -segment application of the Simpson's 1/3 rule with step size  $h = (b - a)/n$ , and  $E(h)$  = the corresponding truncation error.

Using an approach similar to Richardson extrapolation, we can derive an estimate in the error of the more refined estimate  $I(h_2)$  as a function of the difference between the two integral estimates:

$$E(h_2) = \frac{1}{15} [I(h_2) - I(h_1)] \quad (20.29)$$

The error can then be added to  $I(h_2)$  to generate an even better estimate:

$$I = I(h_2) + \frac{1}{15} [I(h_2) - I(h_1)] \quad (20.30)$$

This result is equivalent to *Boole's rule* ([Table 19.2](#)).

The equations just developed can now be combined into an efficient algorithm. [Figure 20.6](#) presents a Python function that is based on an algorithm originally developed by Cleve Moler (2004).

**FIGURE 20.6** A Python function to implement an adaptive quadrature algorithm based on an algorithm originally developed by Cleve Moler (2004).

```
def quadadapt(func,a,b,tol=1.e-8):
    """
    Evaluates the definite integral of f(x) from a to b
    """
    c = (a+b)/2
    fa = func(a) ; fb = func(b) ; fc = func(c)
    q = quadstep(func,a,b,tol,fa,fc,fb)
    return q
def quadstep(func,a,b,tol,fa,fc,fb):
    h = b - a ; c = (a+b)/2
    fd = func((a+c)/2) ; fe = func((c+b)/2)
    q1 = h/6 * (fa + 4*fc + fb)
    q2 = h/12 * (fa + 4*fd + 2*fc + 4*fe + fb)
    if abs(q1-q2) < tol:
        q = q2 + (q2-q1)/15
    else:
        qa = quadstep(func,a,c,tol,fa,fd,fc)
        qb = quadstep(func,c,b,tol,fc,fe,fb)
```

```
q = qa + qb
return q
```

The function consists of a main calling function, `quadadapt`, along with a recursive function, `quadstep`, that actually performs the integration. The main calling function is passed the function `func` and the integration limits, `a` and `b`. The convergence tolerance, `tol`, can also be passed to the function but has a default value of  $1 \times 10^{-8}$  if that argument is omitted. The function values for the initial application of Simpson's 1/3 rule [Eq. (20.25)] are computed. These values along with the integration limits, the function name, `func`, and the tolerance are then passed to `quadstep`. Within `quadstep`, the remaining step sizes and function values are determined, and the two integral estimates [Eqs. (20.25) and (20.26)] are computed.

At this point, the error is estimated as the absolute difference between the integral estimates. Depending on the value of the error, there are two alternatives:

1. If the error is less than the tolerance, `tol`, Boole's rule is computed; the function terminates and passes the result back to `quadadapt`.
2. If the error is greater than or equal to the tolerance, `quadstep` is invoked twice to evaluate each of the two subintervals of the current call.

The two recursive calls (`quadstep` calling itself) in the second step represent the real beauty of this algorithm. They just keep subdividing until the tolerance is met. Once this occurs, their results are passed back up the recursive path, combining with the other integral estimates along the way. This process ends when the final call is satisfied, the total integral is evaluated and returned to the main calling function.

It should be stressed that the algorithm in Fig. 20.6 is a stripped-down version of the `quad` function in the SciPy `integrate` submodule. It does not guard against exception conditions, such as cases where integrals do not exist. Also, it doesn't provide for a relative error tolerance. The `quad` function provides for both absolute and relative error tolerances. Nevertheless, this `quadadapt` function works well for many applications and serves to illustrate how adaptive quadrature works. Here is a Python script showing how `quadadapt` can be used to determine the integral of the polynomial from Example 20.1:

```
def f(x):
    return 0.2 + 25.*x - 200.*x**2 + 675.*x**3 - 900*x**4 + 400*x**5

fint = quadadapt(f,0.,0.8)
print(fint)
```

The result displayed is

```
1.6405333333333347
```

## 20.4.2 Python SciPy integrate Function: `quad`

The `integrate` submodule of Python SciPy has a function for implementing adaptive quadrature:

```
from scipy.integrate import quad
Ival,abserr = quad(func,a,b)
```

where `func` is the function to be integrated and `a` and `b` are the integration limits. The function returns the integral estimate, `Ival`, and the absolute error estimate, `abserr`. There are optional arguments, `epsabs` and `epsrel`, for the absolute and relative tolerance limits, respectively, with default values of  $1.5 \times 10^{-8}$ . There is also a limit on the number of subintervals used, `limit`, with a default value of 50. There are provisions allowing the passing of additional arguments of `func` through to its evaluations by `quad`. Details are available via Spyder Help on the SciPy module.

## EXAMPLE 20.5

### Adaptive Quadrature

#### Problem Statement

Use the `quad` function to integrate the following function:

$$f(x) = \frac{1}{(x - q)^2 + 0.01} + \frac{1}{(x - r)^2 + 0.04} - s$$

between the limits  $x = 0$  to 1. Note that, for  $q = 0.3$ ,  $r = 0.9$ , and  $s = 6$ , this is a version of the humps function<sup>1</sup>. The humps function exhibits both flat and steep regions over a relatively short  $x$  range. Hence, it is useful for demonstrating and testing numerical methods for integration, such as `quad`. Note that this version of the humps function can be integrated analytically between the given limits to yield a value of 29.85832539549867.

Page 499

#### Solution

Here is the Python script and resulting display for evaluating this integral:

```
from scipy.integrate import quad

q = 0.3
r = 0.9
s = 6.

def f(x):
    return 1./((x-q)**2+0.01) + 1./((x-r)**2+0.04) - s

Ival,abserr = quad(f,0.,1.)
print(Ival)
print(abserr)

29.85832539549867
4.346562361912858e-11
```

The solution replicates the analytical value to machine precision, 16 significant digits.

## ROOT-MEAN-SQUARE CURRENT

### Background

Since the late 1800s, alternating current (AC), as opposed to direct current (DC), has been used as the common method for transmission of electricity. The current in an AC circuit can be represented in the form of a sine wave:

$$i = i_{\text{peak}} \sin(\omega t)$$

where  $i$  = the current (A = C/s),  $i_{\text{peak}}$  = the peak current (A),  $\omega$  = the angular frequency (radians/s), and  $t$  = time (s). The angular frequency is related to the period  $T$ (s) by  $\omega = 2\pi/T$ .

The power generated is related to the magnitude of the current. Integration can be used to determine the average current over one cycle:

$$\bar{i} = \frac{1}{T} \int_0^T i_{\text{peak}} \sin(\omega t) dt = \frac{i_{\text{peak}}}{T} (-\cos(2\pi) + \cos(0)) = 0$$

Despite the fact that the average is zero, such a current is capable of generating power. Therefore, an alternative to the average current must be derived.

To do this, electrical engineers and scientists determine the root-mean-square current  $i_{\text{rms}}$  (A), which is calculated as

$$i_{\text{rms}} = \sqrt{\frac{1}{T} \int_0^T i_{\text{peak}}^2 \sin^2(\omega t) dt} = \frac{i_{\text{peak}}}{\sqrt{2}} \quad (20.31)$$

Thus, as the name implies, the rms current is the square root of the mean of the squared current. Because  $1/\sqrt{2} = 0.70707$ ,  $i_{\text{rms}}$  is equal to about 70% of the peak current for our assumed sinusoidal wave form.

This quantity has meaning because it is directly related to the average power absorbed by an element in an AC circuit. To understand this, recall that *Joule's law* states that the instantaneous power absorbed by a circuit element is equal to product of the voltage across it and the current through it:

$$P = iV \quad (20.32)$$

where  $P$  = the power (W = J/s), and  $V$  = voltage (V = J/C). For a resistor, *Ohm's law* states that the voltage is directly proportional to the current: Page 500

$$V = iR \quad (20.33)$$

where  $R = \text{the resistance } (\Omega = \text{V/A} = \text{J}\cdot\text{s/C}^2)$ . Substituting [Eq. \(20.33\)](#) into [\(20.32\)](#) gives

$$P = i^2 R \quad (20.34)$$

The average power can be determined by integrating [Eq. \(20.34\)](#) over a period with the result:

$$\overline{P} = i_{\text{rms}}^2 R$$

Thus, the AC circuit generates the equivalent power as a DC circuit with a constant current of  $i_{\text{rms}}$ .

Now, although the simple sinusoid is widely employed, it is by no means the only waveform that is used. For some of these forms, such as triangular or square waves, the  $i_{\text{rms}}$  can be evaluated analytically with closed-form integration. However, some waveforms must be analyzed with numerical integration methods.

In this case study, we will calculate the root-mean-square current of a non-sinusoidal wave form. We will use both the Newton-Cotes formulas from [Chap. 19](#) as well as the approaches described in this chapter.

## Solution

The integral that must be evaluated is

$$i_{\text{rms}}^2 = \int_0^{1/2} [10e^{-t}\sin(2\pi t)] dt \quad (20.35)$$

For comparative purposes, the exact value of this integral to sixteen significant digits is 15.41260804810169.

Integral estimates for various applications of trapezoidal rule and Simpson's 1/3 rule are listed in [Table 20.2](#). Notice that Simpson's rule is more accurate than trapezoidal rule. The value for the integral to 11 significant figures obtained by a 128-segment trapezoidal rule is roughly equivalent to that using a 32-segment Simpson's rule. Neither quite matches the exact value in the last two digits.

**TABLE 20.2** Values for the integral calculated using Newton-Cotes formulas.

| Technique        | Segments | Integral     | $\varepsilon_t(\%)$   |
|------------------|----------|--------------|-----------------------|
| Trapezoidal rule | 1        | 0.0          | 100.0000              |
|                  | 2        | 15.163266493 | 1.6178                |
|                  | 4        | 15.401429095 | 0.0725                |
|                  | 8        | 15.411958360 | $4.22 \times 10^{-3}$ |
|                  | 16       | 15.412568151 | $2.59 \times 10^{-4}$ |
|                  | 32       | 15.412605565 | $1.61 \times 10^{-5}$ |
|                  | 64       | 15.412607893 | $1.01 \times 10^{-6}$ |

| Technique          | Segments | Integral     | $\varepsilon_t(\%)$   |
|--------------------|----------|--------------|-----------------------|
| Simpson's 1/3 rule | 128      | 15.412608038 | $6.28 \times 10^{-8}$ |
|                    | 2        | 20.217688657 | 31.1763               |
|                    | 4        | 15.480816629 | 0.4426                |
|                    | 8        | 15.415468115 | 0.0186                |
|                    | 16       | 15.412771415 | $1.06 \times 10^{-3}$ |
|                    | 32       | 15.412618037 | $6.48 \times 10^{-5}$ |

The function we developed in [Fig. 20.2](#) can be used to evaluate the integral with Romberg integration:

```
f = lambda t: (10.*np.exp(-t)*np.sin(2*np.pi*t))**2
Ival,errel,iter = romberg(f,0.,0.5)
print('integral estimate = ',Ival)
print('relative error = ',errel)
print('iterations required = ',iter)

integral estimate = 15.412608042889765
relative error = 1.4800587873269456e-10
iterations required = 5
```

With the default stopping criterion,  $1 \times 10^{-8}$ , we obtain a result that is accurate to ten significant figures in five iterations. We can obtain an even better result if we impose a more stringent stopping criterion:

Page 501

```
Ival,errel,iter = romberg(f,0.,0.5,es=1.e-15)
integral estimate = 15.412608048101685
relative error = 0.0
iterations required = 7
```

Gauss quadrature can also be used to make the same estimate. First, a change of variable is performed by applying [Eqs. \(20.22\)](#) and [\(20.23\)](#) to yield

$$t = \frac{1}{4} + \frac{1}{4}t_d \quad \text{and} \quad dt = \frac{1}{4}dt_d \frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i}$$

These relationships can be substituted into [Eq. \(20.35\)](#) to yield

$$i_{\text{rms}}^2 = \int_{-1}^1 [10 e^{-(0.25+0.25t_d)} \sin(2\pi[0.25 + 0.25t_d])]^2 0.25 dt_d$$

For the two-point Gauss-Legendre formula, this function is evaluated at  $t_d = -1/\sqrt{3}$  and  $1/\sqrt{3}$  with the results being 7.684096 and 4.313728, respectively. These values can be substituted into [Eq. \(20.17\)](#) to yield an integral estimate of 11.99782, which represents an error of  $\varepsilon_t = 22.1\%$ .

The three-point formula is ([Table 20.1](#))

$$I = 0.555556 \cdot 1.237449 + 0.888889 \cdot 15.16327 + 0.555556 \cdot 2.684915 = 15.65755$$

which has  $\varepsilon_t = 1.6\%$ . The results of using the higher-point formulas are summarized in [Table 20.3](#).



**TABLE 20.3** Results of using various-point Gauss quadrature formulas to approximate the integral.

| Points | Estimate   | $\varepsilon_t(\%)$   |
|--------|------------|-----------------------|
| 2      | 11.9978243 | 22.1                  |
| 3      | 15.6575502 | 1.59                  |
| 4      | 15.4058023 | $4.42 \times 10^{-2}$ |
| 5      | 15.4126391 | $2.01 \times 10^{-4}$ |
| 6      | 15.4126109 | $1.82 \times 10^{-5}$ |

Finally, the integral can be evaluated with the SciPy `integrate quad` function:

```
from scipy.integrate import quad
irms2,irmserr = quad(f,0.,0.5)
print(irms2)
15.412608048101674
```

This result is accurate to 15 significant figures.

We can now compute  $i_{\text{rms}}$  by merely taking the square root of the integral estimate. For example, using the result computed with `quad`, we have

```
irms = np.sqrt(irms2)
print(irms)
3.925889459485796
```

This result could then be employed to guide other aspects of the design and operation of the circuit, such as power dissipation computations.

As we did for the simple sinusoid in [Eq. \(20.31\)](#), an interesting calculation involves comparing this result with the peak current. Recognizing that this is an optimization problem, we can readily employ the `minimize_scalar` function from the `optimize` submodule of SciPy. Because we are looking for a maximum, we evaluate the negative of the function:

```
from scipy.optimize import minimize_scalar
func = lambda t: -10.*np.exp(-t)*np.sin(2.*np.pi*t)
result = minimize_scalar(func,bounds=(0.,0.5),method='bounded')
print(result.x)
print(-func(result.x))
0.22487940319319893
7.886853873932577
```

A maximum current of 7.887 A occurs at  $t = 0.2249$  s. Hence, for this particular waveform, the root-mean-square value is about 49.8% of the maximum.

# PROBLEMS

---

**20.1** Use Romberg integration to evaluate

$$I = \int_1^2 \left( x + \frac{1}{x} \right)^2 dx$$

to an accuracy of  $\varepsilon_s = 0.5\%$ . Your results should be presented in the format of  **Fig. 20.1**. Use the analytical solution of the integral to determine the percent relative error of the result obtained with Romberg integration. Check that  $\varepsilon_t$  is less than  $\varepsilon_s$ .

**20.2** Evaluate the following integral (a) analytically, (b) using Romberg integration,  $\varepsilon_s = 0.5\%$ , (c) using the three-point Gauss quadrature formula, and (d) with the `quad` function from Python's SciPy `integrate` submodule.

$$I = \int_0^8 (-0.055x^4 + 0.86x^3 - 4.2x^2 + 6.3x + 2) dx$$

**20.3** Evaluate the following integral with (a) Romberg integration ( $\varepsilon_s = 0.5\%$ ), (b) the two-point Gauss quadrature formula, and (c) the `quad` function from Python's SciPy `integrate` submodule:

$$I = \int_0^3 x e^{2x} dx$$

**20.4** There is no closed-form solution for the *error function*

$$\text{erf}(a) = \frac{2}{\sqrt{\pi}} \int_0^a e^{-x^2} dx$$

Use the (a) two-point and (b) three-point Gauss-Legendre formulas to estimate  $\text{erf}(1.5)$ . Determine the percent error for each case based on the true value which can be determined with the `erf` function in the `special` submodule of SciPy.

**20.5** The force on a sailboat mast can be modeled by the following integral:

$$F = \int_0^H \left[ 200 \left( \frac{z}{5+z} \right) e^{-\frac{2z}{H}} \right] dz$$

where  $z$  = elevation above the deck, and  $H$  = height of the mast. Compute  $F$  for the case where  $H = 30$  using (a) Romberg integration to a tolerance of  $\varepsilon_s = 0.5\%$ , (b) the two-point Gauss-Legendre formula, and (c) the quad function from Python's SciPy `integrate` submodule.

**20.6** The root-mean-square current can be computed from

$$I_{\text{rms}} = \sqrt{\frac{1}{T} \int_0^T i^2(t) dt}$$

For  $T = 1$ , suppose that  $i(t)$  is defined as

$$\begin{aligned} i(t) &= 8e^{-t/T} \sin\left(2\pi \frac{t}{T}\right) & 0 \leq t \leq T/2 \\ i(t) &= 0 & T/2 \leq t \leq T \end{aligned}$$

Evaluate the  $I_{\text{rms}}$  for this case using (a) Romberg integration to a tolerance of 0.1%, (b) the two- and three-point Gauss-Legendre formulas, and (c) the quad function from Python's SciPy `integrate` submodule.

**20.7** The heat required,  $\Delta H(\text{kJ})$ , to induce a temperature change,  $\Delta T = T - T_0(^{\circ}\text{C})$ , in a material can be computed as

$$\Delta H = m \int_{T_0}^T C_P(T) dT$$

where  $m$  = mass (kg), and  $C_P(T)$  = heat capacity [ $\text{kJ}/(\text{kg} \cdot ^{\circ}\text{C})$ ]. The heat capacity increases with temperature,  $T(^{\circ}\text{C})$ , and is typically modeled with a polynomial. For methane gas ( $\text{CH}_4$ ) at lower pressures where the ideal gas law applies, this model is

$$C_P(T) = 3.431 \times 10^{-2} + 5.469 \times 10^{-5}T + 3.661 \times 10^{-9}T^2 - 1.10 \times 10^{-11}T^3$$

Write a Python script that uses the `quad` function to generate a plot of  $\Delta H$  versus  $\Delta T$ , where the base temperature is  $0^{\circ}\text{C}$  and  $\Delta T$  ranges from 100 to 1200  $^{\circ}\text{C}$ .

**20.8** The amount of a trace substance transported by a pipeline over a period of time can be computed as

$$M = \int_{t_1}^{t_2} Q(t) c(t) dt$$

where  $M$  = mass of the substance,  $t_1$  = initial time (min),  $t_2$  = final time (min),  $Q(t)$  = volumetric flow rate ( $\text{m}^3/\text{min}$ ), and  $c(t)$  = concentration of the substance ( $\text{mg}/\text{m}^3$ ). The following functional representations define the temporal variations in flow rate and concentration:

$$\begin{aligned} Q(t) &= 9 + 5\cos^2(0.4t) \\ c(t) &= 5e^{-0.5t} + 2e^{0.15t} \end{aligned}$$

Determine the mass of substance transported between  $t_1 = 2$  and  $t_2 = 8$  min with (a) Romberg integration with a tolerance of 0.1% and (b) the quad function from the SciPy `integrate` submodule.

### 20.9 Evaluate the double integral

$$\int_{-2}^2 \int_0^4 (x^2 - 3y^2 + xy^3) dx dy$$

(a) analytically and (b) with the `dblquad` function from the SciPy `integrate` submodule.

### 20.10 Compute work, as described in [Sec. 20.5](#), but use the following equations for $F(x)$ and $\theta(x)$ :

$$\begin{aligned} F(x) &= 1.6x - 0.045x^2 \\ \theta(x) &= -0.00055x^3 + 0.0123x^2 + 0.13x \end{aligned}$$

The force is in newtons, and the angle is in radians. Perform the integration from  $x = 0$  to 30 m.

### 20.11 Perform the same computation as in [Sec. 20.5](#), but for the current as specified by

$$\begin{aligned} I &= 6e^{-1.25t} \sin(2\pi t) & 0 \leq t \leq T/2 \\ I &= 0 & T/2 \leq t \leq T \end{aligned}$$

where  $T = 1$ .

### 20.12 Compute the average power absorbed by an element in a circuit as described in [Sec. 20.5](#), but for a simple sinusoidal current, $I = \sin(2\pi t/T)$ where $T = 1$ s.

(a) Assume that Ohm's law holds and  $R = 5\Omega$ .

- (b) Assume that Ohm's law does not hold, and that voltage and current are related by the nonlinear relationship,  $V = 5i - 1.25i^3$ .

**20.13** Suppose that the current through a resistor is described by the function

$$i(t) = (60 - t)^2 + (60 - t)\sin(\sqrt{t})$$

and the resistance is a function of the current:

$$R = 10i + 2i^{2/3}$$

Compute the average voltage over  $t = 0$  to 60 s using the composite Simpson's 1/3 rule.

**20.14** If a capacitor initially holds no charge, the voltage across it as a function of time can be computed as

$$V(t) = \frac{1}{C} \int_0^t i(t) dt$$

Use Python to fit the data in the table below with a fifth-order polynomial. Then, use a numerical integration technique along with the value of  $C = 10^{-5}$  farad to generate a plot of voltage versus time.

| $t, s$         | 0      | 0.2    | 0.4    | 0.6    |
|----------------|--------|--------|--------|--------|
| $i, 10^{-3} A$ | 0.2    | 0.3683 | 0.3819 | 0.2282 |
| $t, s$         | 0.8    | 1      | 1.2    |        |
| $i, 10^{-3} A$ | 0.0486 | 0.0082 | 0.1441 |        |

**20.15** The work done on an object is equal to the force times the distance moved in the direction of the force. The velocity of an object in the direction of the force is given by

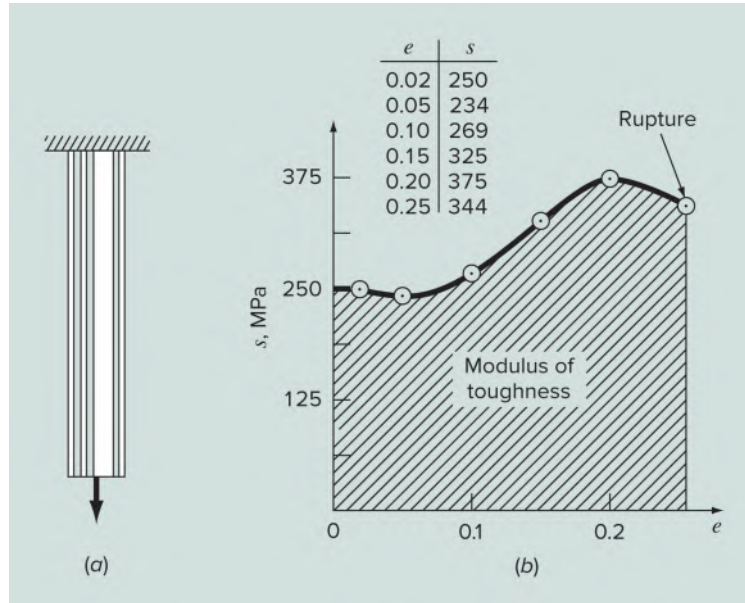
$$\begin{aligned} v &= 4t & 0 \leq t \leq 5 \\ v &= 20 + (5 - t)^2 & 5 \leq t \leq 15 \end{aligned}$$

where  $v$  is in m/s. Determine the work if a constant force of 200 N is applied for all  $t$ .

**20.16** A rod subject to an axial load ( **Fig. P20.16a**) will be deformed, as shown in the stress-strain curve in  **Fig. P20.16b**. The area under the curve from zero stress out to the point of rupture is called the *modulus of toughness* of the material. It provides a measure of the energy per unit volume required to

cause the material to rupture. As such, it is representative of the material's ability to withstand an impact load. Use numerical integration to compute the modulus of toughness for the stress-strain curve presented in  **Fig. P20.16b**.

**FIGURE P20.16** (a) A rod under axial loading, and (b) the resulting stress-strain curve, where stress is in megapascals (MPa), and strain is dimensionless.



**20.17** If the velocity distribution of a fluid flowing through a pipe is known ( **Fig. P20.17**), the flow rate,  $Q$  (the volume of fluid passing through the pipe per unit time) can be computed as

$$Q = \int v dA$$

where  $v$  is the velocity, and  $dA$  is a differential of the pipe's cross-section. For a circular pipe,  $A = \pi r^2$ , and then  $dA = 2\pi r dr$ . Therefore,

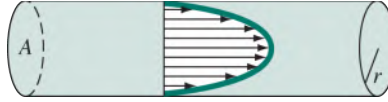
$$Q = \int_0^{r_0} v(2\pi r) dr$$

where  $r_0$  is the inside radius of the pipe. A typical model for the velocity profile in turbulent flow is

$$v = 2 \left( 1 - \frac{r}{r_0} \right)^{1/6}$$

For  $r_0 = 3$  cm and  $v$  in cm/s, compute  $Q$  in  $\text{cm}^3/\text{s}$  using the composite trapezoidal rule. Discuss the results.

**FIGURE P20.17**



**20.18** Using the following data, calculate the work done by stretching a spring to a displacement of  $x = 0.35$  m. To do this, first fit the data with a polynomial and then integrate the polynomial numerically to compute the work.

|                     |       |       |       |       |
|---------------------|-------|-------|-------|-------|
| $F, 10^3 \text{ N}$ | 0     | 0.01  | 0.028 | 0.046 |
| $x, \text{ m}$      | 0     | 0.05  | 0.10  | 0.15  |
| $F, 10^3 \text{ N}$ | 0.063 | 0.082 | 0.11  | 0.13  |
| $x, \text{ m}$      | 0.20  | 0.25  | 0.30  | 0.35  |

**20.19** Evaluate the vertical distance traveled by a rocket if the vertical velocity,  $v$ , is given by

$$v = \begin{cases} 11t^2 - 5t & 0 \leq t \leq 10 \\ 1100 - 5t & 10 \leq t \leq 20 \\ 50t + 2(t - 20)^2 & 20 \leq t \leq 30 \end{cases}$$

Create a plot of distance traveled versus time.

**20.20** The upward velocity,  $v$ , of a rocket can be computed from the following formula:

$$v = u \ln\left(\frac{m_0}{m_0 - qt}\right) - gt$$

where  $u$  = the velocity at which fuel is expelled downward from the rocket,  $m_0$  = the initial mass of the rocket at time = 0,  $q$  = the fuel consumption rate, and  $g$  is the acceleration due to gravity ( $9.81 \text{ m/s}^2$ ). If  $u = 1850 \text{ m/s}$ ,  $m_0 = 160,000 \text{ kg}$ , and  $q = 2500 \text{ kg/s}$ , determine the elevation that the rocket will reach at  $t = 30 \text{ s}$ .

**20.21** The *Weibull* statistical distribution is used frequently to model reliability in processes and equipment. It is described by its density function:

$$f(x) = \frac{\beta}{\delta} \left( \frac{x}{\delta} \right)^{\beta-1} e^{-\left(\frac{x}{\delta}\right)^\beta}$$

where  $\delta > 0$  = a scale parameter, and  $\beta > 0$  = a shape parameter. These parameters are fit to the process or equipment under study. The time to failure in hours of a bearing in a mechanical shaft,  $t$ , is modeled by the Weibull distribution with  $\beta = 0.5$  and  $\delta = 5000$  hours. Use integration of the Weibull density to determine the probability that the bearing's life will exceed 7500 hours. Recall that

$$P[\text{failure} < t_f] = \int_0^{t_f} f(t) dt$$

**20.22** Use Romberg integration to evaluate

$$\int_0^2 \frac{e^x \sin(x)}{1+x^2} dx$$

to an accuracy of  $\varepsilon_s = 0.5\%$ . Your results should be presented in the format of  **Fig. 20.1**.

**20.23** Recall that the velocity of the free-falling bungee jumper can be derived analytically as  **Eq. (1.9)** :

$$v(t) = \sqrt{\frac{mg}{c_d}} \tanh\left(\sqrt{\frac{gc_d}{m}} t\right)$$

where  $v(t)$  = velocity (m/s),  $t$  = time (s),  $g = 9.81$  m/s<sup>2</sup>,  $m$  = mass (kg),  $c_d$  = drag coefficient (kg/m).

- (a) Use Romberg integration to compute how far the jumper travels during the first eight seconds of free fall given  $m = 80$  kg and  $c_d = 0.2$  kg/m. Compute the answer to  $\varepsilon_s = 1\%$ .
- (b) Perform the same computation with the `quad` function from the SciPy `integrate` submodule.

**20.24** Prove that  **Eq. (20.30)** is equivalent to Boole's rule.

**20.25** As specified in the following table, the earth's density varies as a function of the distance from its center ( $r = 0$ ).

|                                            |       |       |       |       |      |      |      |
|--------------------------------------------|-------|-------|-------|-------|------|------|------|
| <b><math>r</math>, km</b>                  | 0     | 1100  | 1500  | 2450  | 3400 | 3630 | 4500 |
| <b><math>\rho</math>, kg/m<sup>3</sup></b> | 13000 | 12400 | 12000 | 11200 | 9700 | 5700 | 5200 |
| <b><math>r</math>, km</b>                  | 5380  | 6060  | 6280  | 6380  |      |      |      |
| <b><math>\rho</math>, kg/m<sup>3</sup></b> | 4700  | 3600  | 3400  | 3000  |      |      |      |



Develop a script to fit these data with the `PchipInterpolator` function from the SciPy `interpolate` submodule. Generate a plot showing the resulting fit along with the data points. Then, use one of the functions from the SciPy `integrate` submodule to estimate the earth's mass in metric tonnes by integrating the output of the `interpld` function.

**20.26** (a) Employ the `romberg` function shown in [Fig. 20.2](#) to determine the integral of the polynomial from [Example 20.1](#). (b) Then, use the function to solve [Prob. 20.1](#) to high precision.

**20.27** Employ the adaptive quadrature function of [Fig. 20.6](#) to solve [Prob. 20.20](#).

**20.28** The average flow in a river channel,  $Q(\text{m}^3/\text{s})$ , with an irregular cross-section can be computed as the integral of the product of velocity and depth over the width of the channel at the water surface.

$$Q = \int_0^B U(y)H(y)dy$$

where  $U(y)$  = water velocity (m/s) at a distance  $y(\text{m})$  from the bank, and  $H(y)$  = water depth (m) at a distance  $y$  from the bank. Use the Python SciPy `quad` function along with cubic spline fits of  $U$  and  $H$  to the data collected and shown below to estimate the flow rate in the river.

| $y$ (m) | $H$ (m) | $Y$ (m) | $U$ (m/s) |
|---------|---------|---------|-----------|
| 0       | 0       | 0       | 0         |
| 1.1     | 0.21    | 1.6     | 0.08      |
| 2.8     | 0.78    | 4.1     | 0.61      |
| 4.6     | 1.87    | 4.8     | 0.68      |
| 6       | 1.44    | 6.1     | 0.55      |
| 8.1     | 1.28    | 6.8     | 0.42      |
| 9       | 0.2     | 9       | 0         |

**20.29** Use the four-point Gauss quadrature method to estimate the average value of the following function between  $x = 1$  and  $x = 5$ .

$$f(x) = \frac{2}{1+x^2}$$

**20.30** Evaluate the following integral:

$$I = \int_0^4 x^3 dx$$

(a) analytically and (b) using the `quad` function from the SciPy `integrate` submodule.

**20.31** An example version of the *humps* function is

$$f(x) = \frac{1}{(x - 0.3)^2 + 0.01} + \frac{1}{(x - 0.9)^2 + 0.04} - 6$$

Use the `quad` function from SciPy's `integrate` submodule to estimate

$$\int_0^2 f(x) dx$$

**20.32** Evaluate the following double integral:

$$I = \int_0^2 \int_{-3}^1 y^4 (x^2 + xy) dx dy$$

- (a) using a single application of Simpson's 1/3 rule across each dimension, and
- (b) via the `dblquad` function from the SciPy `integrate` submodule.

# Numerical Differentiation

## CHAPTER OBJECTIVES

The primary objective of this chapter is to introduce you to numerical differentiation. Specific objectives and topics covered are

- Understanding how to apply high-accuracy numerical differentiation formulas for equispaced data.
- Knowing how to evaluate derivatives for unequally spaced data.
- Understanding how Richardson extrapolation is applied for numerical differentiation.
- Recognizing the sensitivity of numerical differentiation to random content in data.
- Knowing how to evaluate derivatives in Python with the `diff` and `gradient` functions in the NumPy module.
- Knowing how to create a two-dimensional contour plot in Python with gradient arrows to visualize a vector field.

## YOU'VE GOT A PROBLEM

Recall that the velocity of a free-falling bungee jumper as a function of time can be computed as

$$v(t) = \sqrt{\frac{gm}{c_d}} \tanh\left(\sqrt{\frac{gc_d}{m}} t\right) \quad (21.1)$$

At the beginning of [Chap. 19](#), we used calculus to integrate this equation to determine the vertical distance  $z$  the jumper has fallen after a time  $t$ .

$$z(t) = \frac{m}{c_d} \ln \left[ \cosh \left( \sqrt{\frac{gc_d}{m}} t \right) \right] \quad (21.2)$$

Now suppose that you were given the reverse problem. That is, you were asked to determine velocity based on the jumper's position as a function of time. Because it is the inverse of integration, differentiation could be used to make the determination:

$$v(t) = \frac{d\mathbf{x}(t)}{dt} \quad (21.3)$$

Substituting [Eq. \(21.2\)](#) into [Eq. \(21.3\)](#) and differentiating would bring us back to [Eq. \(21.1\)](#).

Beyond velocity, you might also be asked to compute the jumper's acceleration. To do this, we could either take the first derivative of velocity, or the second derivative of displacement:

$$a(t) = \frac{dv(t)}{dt} = \frac{d^2\mathbf{x}(t)}{dt^2} \quad (21.4)$$

In either case, the result would be

$$a(t) = g \operatorname{sech}^2\left(\sqrt{\frac{gc_d}{m}} t\right) \quad (21.5)$$

Although a closed-form solution can be developed for this case, there are other functions that may be difficult or impossible to differentiate analytically. Further, suppose that there was some way to measure the jumper's position at various times during the fall. These distances along with their associated times could be assembled as a table of discrete values. In this situation, it would be useful to differentiate the discrete data to determine the velocity and the acceleration. In both these instances, numerical differentiation methods are available to obtain solutions. This chapter will introduce you to some of these methods.

# 21.1 INTRODUCTION AND BACKGROUND

---

## 21.1.1 What Is Differentiation?

*Calculus* is the mathematics of change. Because engineers and scientists must continuously deal with systems and processes that change, calculus is an essential tool of our profession. Standing at the heart of calculus is the mathematical concept of differentiation.

According to the dictionary definition, to *differentiate* means “to mark off by differences; distinguish; . . . to perceive the difference in or between.” Mathematically, the *derivative*, which serves as the fundamental vehicle for differentiation, represents the rate of change of a dependent variable with respect to an independent variable. As depicted in  **Fig. 21.1**, the mathematical definition of the derivative begins with a difference approximation:

$$\frac{\Delta y}{\Delta x} = \frac{f(x_i + \Delta x) - f(x_i)}{\Delta x} \quad (21.6)$$

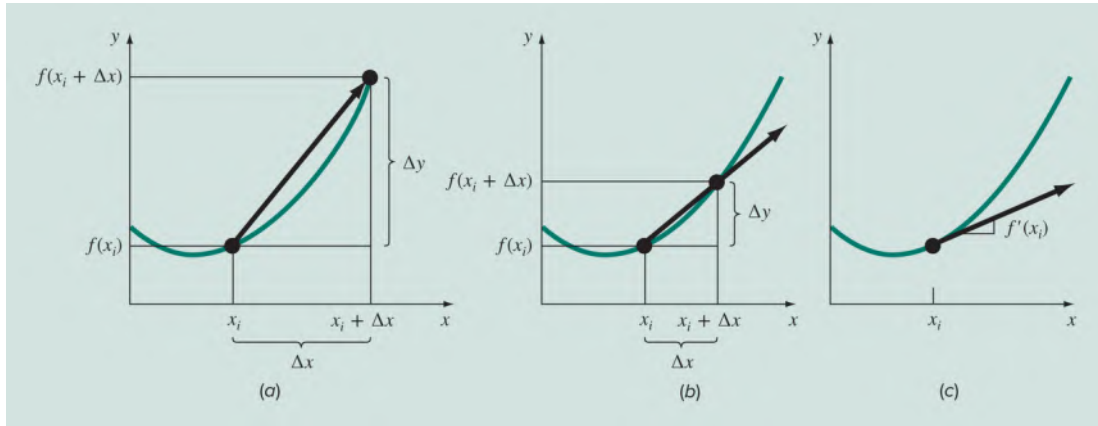
where  $y$  and  $f(x)$  are alternative representatives for the dependent variable and  $x$  is the independent variable. If  $\Delta x$  is allowed to approach zero, as occurs in moving from  **Fig. 21.1a** to  **c**, the difference becomes a derivative:

$$\frac{dy}{dx} = \lim_{\Delta x \rightarrow 0} \frac{f(x_i + \Delta x) - f(x_i)}{\Delta x} \quad (21.7)$$

where  $dy/dx$  [which can also be designated as  $y'$  or  $f'(x_i)$ ]<sup>1</sup> is the first derivative of  $y$  with respect to  $x$  evaluated at  $x_i$ . As seen in the visual depiction of  **Fig. 21.1c**, the derivative is the slope of the tangent to the curve at  $x_i$ .

---

**FIGURE 21.1** The graphical definition of a derivative: as  $\Delta x$  approaches zero in going from (a) to (c), the difference approximation becomes a derivative.



The second derivative represents the derivative of the first derivative,

$$\frac{d^2y}{dx^2} = \frac{d}{dx} \left( \frac{dy}{dx} \right) \quad (21.8)$$

Thus, the second derivative tells us how fast the slope is changing. It is commonly referred to as the *curvature*, because a high value for the second derivative means high curvature.

Finally, partial derivatives are used for functions that depend on more than one variable. Partial derivatives can be thought of as taking the derivative of the function at a point with all but one variable held constant. For example, given a function  $f$  that depends on both  $x$  and  $y$ , the partial derivative of  $f$  with respect to  $x$  at an arbitrary point  $(x, y)$  is defined as

$$\frac{\partial f}{\partial x} = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x, y) - f(x, y)}{\Delta x} \quad (21.9)$$

Similarly, the partial derivative of  $f$  with respect to  $y$  is defined as

$$\frac{\partial f}{\partial y} = \lim_{\Delta y \rightarrow 0} \frac{f(x, y + \Delta y) - f(x, y)}{\Delta y} \quad (21.10)$$

To get an intuitive grasp of partial derivatives, recognize that a function that depends on two variables is a surface rather than a curve. Suppose you are mountain climbing and have access to a function  $f$  that yields elevation as a function of longitude (the east-west oriented  $x$  axis) and latitude (the north-south oriented  $y$  axis). If you stop at a particular point  $(x_0, y_0)$ , the slope to the east would be  $\partial f(x_0, y_0) / \partial x$ , and the slope to the north would be  $\partial f(x_0, y_0) / \partial y$ .

## 21.1.2 Differentiation in Engineering and Science

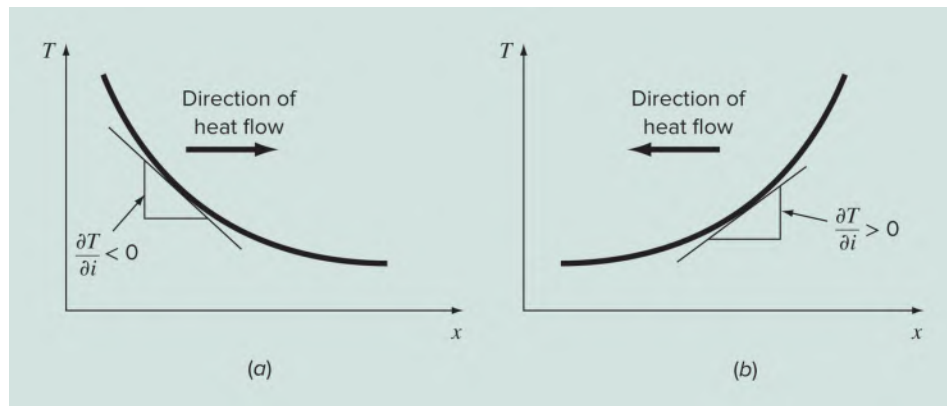
The differentiation of a function has so many engineering and scientific applications that you were required to take differential calculus in your first year at college. Many specific examples of such applications could be given in all fields of engineering and science. Differentiation is commonplace in engineering and science because so much of our work involves characterizing the changes of variables in both time and space. In fact, many of the laws and other generalizations that figure so prominently in our work are based on the predictable ways in which change manifests itself in the physical world. A prime example is Newton's second law, which is not couched in terms of the position of an object but rather in its change with respect to time.

Aside from such temporal examples, numerous laws involving the spatial behavior of variables are expressed in terms of derivatives. Among the most common of these are the *constitutive laws* that define how potentials or gradients influence physical processes. For example, *Fourier's law of heat conduction* quantifies the observation that heat flows from regions of high to low temperature. For the one-dimensional case, this can be expressed mathematically as

$$q = -k \frac{dT}{dx} \quad (21.11)$$

where  $q(x)$  = heat flux ( $\text{W}/\text{m}^2$ ),  $k$  = coefficient of thermal conductivity [ $\text{W}/(\text{m}\cdot\text{K})$ ],  $T$  = temperature (K), and  $x$  = distance (m). Thus, the derivative, or *gradient*, provides a measure of the intensity of the spatial temperature change, which drives the transfer of heat (📖 Fig. 21.2).

**FIGURE 21.2** Graphical depiction of a temperature gradient. Because heat moves “downhill” from high to low temperature, the flow in (a) is from left to right. However, due to the orientation of Cartesian coordinates, the slope is negative for this case. Thus, a negative gradient leads to a positive flow. This is the origin of the minus sign in Fourier's law of heat conduction. The reverse case is depicted in (b), where the positive gradient leads to a negative heat flow from right to left.



Similar laws provide workable models in many other areas of engineering and science, including the modeling of fluid dynamics, mass transfer, chemical reaction kinetics, electricity, and solid mechanics ( [Table 21.1](#) ). The ability to accurately estimate derivatives is an important facet of our capability to work effectively in these areas.

**TABLE 21.1** The one-dimensional forms of some constitutive laws commonly used in engineering and science.

| Law                    | Equation                        | Physical Area             | Gradient      | Flux         | Proportionality         |
|------------------------|---------------------------------|---------------------------|---------------|--------------|-------------------------|
| Fourier’s law          | $q = -k \frac{dT}{dx}$          | Heat conduction           | Temperature   | Heat flux    | Thermal conductivity    |
| Fick’s law             | $J = -D \frac{dc}{dx}$          | Mass diffusion            | Concentration | Mass flux    | Diffusivity             |
| Darcy’s law            | $q = -k \frac{dh}{dx}$          | Flow through porous media | Head          | Flow flux    | Hydraulic conductivity  |
| Ohm’s law              | $J = -\sigma \frac{dV}{dx}$     | Current flow              | Voltage       | Current flux | Electrical conductivity |
| Newton’s viscosity law | $\tau = \mu \frac{du}{dx}$      | Fluids                    | Velocity      | Shear Stress | Dynamic viscosity       |
| Hooke’s law            | $\sigma = E \frac{\Delta L}{L}$ | Elasticity                | Deformation   | Stress       | Young’s modulus         |

Beyond direct engineering and scientific applications, numerical differentiation is also important in a variety of general mathematical contexts including other areas of numerical methods. For example, recall that in [Chap. 6](#) the secant method was based on a finite-difference approximation of the derivative. In addition, probably the most important application of numerical differentiation involves the solution of differential equations. We have already seen an example in the form of Euler’s method in [Chap. 1](#). In [Chap. 24](#), we will investigate how numerical differentiation provides the basis for solving boundary-value problems of ordinary differential equations.

These are just a few of the applications of differentiation that you might face regularly in the pursuit of your profession. When the functions to be analyzed are simple, you will normally choose to evaluate them analytically. However, it is often difficult or impossible when the function is complicated. In addition, the underlying function is often unknown and defined only by measurement at discrete points. For both these cases, you must have the ability to obtain approximate values for derivatives, using numerical techniques as described next.



## 21.2 HIGH-ACCURACY DIFFERENTIATION FORMULAS

We have already introduced the notion of numerical differentiation in [Chap. 4](#). Recall that we employed Taylor series expansions to derive finite-difference approximations of derivatives. In [Chap. 4](#), we developed forward, backward, and centered difference approximations of first and higher derivatives. Remember that, at best, these estimates had errors that were  $O(h^2)$ —that is, their errors were proportional to the square of the step size. This level of accuracy is due to the number of terms of the Taylor series that were retained during the derivation of these formulas. We will now illustrate how high-accuracy finite-difference formulas can be generated by including additional terms from the Taylor series expansion.

For example, the forward Taylor series expansion can be written as [recall [Eq. \(4.13\)](#)]

Page 510

$$f(x_{i+1}) = f(x_i) + f'(x_i)h + \frac{f''(x_i)}{2!}h^2 + \dots \quad (21.12)$$

which can be solved for

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{h} - \frac{f''(x_i)}{2!}h + O(h^2) \quad (21.13)$$

In [Chap. 4](#), we truncated this result by excluding the second- and higher-derivative terms and were thus left with a forward-difference formula:

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{h} + O(h) \quad (21.14)$$

In contrast to this approach, we now retain the second-derivative term by substituting the following forward-difference approximation of the second derivative [recall [Eq. \(4.27\)](#)]:

$$f''(x_i) = \frac{f(x_{i+2}) - 2f(x_{i+1}) + f(x_i))}{h^2} + O(h) \quad (21.15)$$

into [Eq. \(21.13\)](#) to yield

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{h} - \frac{f(x_{i+2}) - 2f(x_{i+1}) + f(x_i))}{2h^2}h + O(h^2) \quad (21.16)$$

or, by collecting terms:

$$f'(x_i) = \frac{-f(x_{i+2}) + 4f(x_{i+1}) - 3f(x_i)}{2h} + O(h^2) \quad (21.17)$$

Notice that inclusion of the second-derivative term has improved the accuracy to  $O(h^2)$ . Similar improved versions can be developed for the backward and centered formulas as well as for the approximations of higher-order derivatives. The formulas are summarized in [Fig. 21.3](#) through [Fig. 21.5](#) along with the lower-order versions from [Chap. 4](#). The following example illustrates the utility of these formulas for estimating derivatives.

**FIGURE 21.3** Forward finite-difference formulas: two versions are presented for each derivative. The latter version incorporates more terms of the Taylor series expansion and is, consequently, more accurate.

|                                                                                                                 |          |
|-----------------------------------------------------------------------------------------------------------------|----------|
| First Derivative                                                                                                | Error    |
| $f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{h}$                                                                       | $O(h)$   |
| $f'(x_i) = \frac{-f(x_{i+2}) + 4f(x_{i+1}) - 3f(x_i)}{2h}$                                                      | $O(h^2)$ |
| Second Derivative                                                                                               |          |
| $f''(x_i) = \frac{f(x_{i+2}) - 2f(x_{i+1}) + f(x_i)}{h^2}$                                                      | $O(h)$   |
| $f''(x_i) = \frac{-f(x_{i+3}) + 4f(x_{i+2}) - 5f(x_{i+1}) + 2f(x_i)}{h^2}$                                      | $O(h^2)$ |
| Third Derivative                                                                                                |          |
| $f'''(x_i) = \frac{f(x_{i+3}) - 3f(x_{i+2}) + 3f(x_{i+1}) - f(x_i)}{h^3}$                                       | $O(h)$   |
| $f'''(x_i) = \frac{-3f(x_{i+4}) + 14f(x_{i+3}) - 24f(x_{i+2}) + 18f(x_{i+1}) - 5f(x_i)}{2h^3}$                  | $O(h^2)$ |
| Fourth Derivative                                                                                               |          |
| $f^{(4)}(x_i) = \frac{f(x_{i+4}) - 4f(x_{i+3}) + 6f(x_{i+2}) - 4f(x_{i+1}) + f(x_i)}{h^4}$                      | $O(h)$   |
| $f^{(4)}(x_i) = \frac{-2f(x_{i+5}) + 11f(x_{i+4}) - 24f(x_{i+3}) + 26f(x_{i+2}) - 14f(x_{i+1}) + 3f(x_i)}{h^4}$ | $O(h^2)$ |

**FIGURE 21.4** Backward finite-difference formulas: two versions are presented for each derivative. The latter version incorporates more terms of the Taylor series expansion and is, consequently, more accurate.

|                                                                                                                     |          |
|---------------------------------------------------------------------------------------------------------------------|----------|
| First Derivative                                                                                                    | Error    |
| $f'(x_i) = \frac{f(x_i) - f(x_{i-1}))}{h}$                                                                          | $O(h)$   |
| $f'(x_i) = \frac{3f(x_i) - 4f(x_{i-1})) + f(x_{i-2}))}{2h}$                                                         | $O(h^2)$ |
| Second Derivative                                                                                                   |          |
| $f''(x_i) = \frac{f(x_i) - 2f(x_{i-1})) + f(x_{i-2}))}{h^2}$                                                        | $O(h)$   |
| $f''(x_i) = \frac{2f(x_i) - 5f(x_{i-1})) + 4f(x_{i-2})) - f(x_{i-3}))}{h^2}$                                        | $O(h^2)$ |
| Third Derivative                                                                                                    |          |
| $f'''(x_i) = \frac{f(x_i) - 3f(x_{i-1})) + 3f(x_{i-2})) - f(x_{i-3}))}{h^3}$                                        | $O(h)$   |
| $f'''(x_i) = \frac{5f(x_i) - 18f(x_{i-1})) + 24f(x_{i-2})) - 14f(x_{i-3})) + 3f(x_{i-4}))}{2h^3}$                   | $O(h^2)$ |
| Fourth Derivative                                                                                                   |          |
| $f^{(4)}(x_i) = \frac{f(x_i) - 4f(x_{i-1})) + 6f(x_{i-2})) - 4f(x_{i-3})) + f(x_{i-4}))}{h^4}$                      | $O(h)$   |
| $f^{(4)}(x_i) = \frac{3f(x_i) - 14f(x_{i-1})) + 26f(x_{i-2})) - 24f(x_{i-3})) + 11f(x_{i-4})) - 2f(x_{i-5}))}{h^4}$ | $O(h^2)$ |

**FIGURE 21.5** Centered finite-difference formulas: two versions are presented for each derivative. The latter version incorporates more terms of the Taylor series expansion and is, consequently, more accurate.

|                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------|
| First Derivative                                                                                                    |
| $f'(x_i) = \frac{f(x_{i+1})) - f(x_{i-1}))}{2h}$                                                                    |
| $f'(x_i) = \frac{-f(x_{i+2})) + 8f(x_{i+1})) - 8f(x_{i-1})) + f(x_{i-2}))}{12h}$                                    |
| Second Derivative                                                                                                   |
| $f''(x_i) = \frac{f(x_{i+1})) - 2f(x_i) + f(x_{i-1}))}{h^2}$                                                        |
| $f''(x_i) = \frac{-f(x_{i+2})) + 16f(x_{i+1})) - 30f(x_i) + 16f(x_{i-1})) - f(x_{i-2}))}{12h^2}$                    |
| Third Derivative                                                                                                    |
| $f'''(x_i) = \frac{f(x_{i+2})) - 2f(x_{i+1})) + 2f(x_{i-1})) - f(x_{i-2}))}{2h^3}$                                  |
| $f'''(x_i) = \frac{-f(x_{i+3})) + 8f(x_{i+2})) - 13f(x_{i+1})) + 13f(x_{i-1})) - 8f(x_{i-2})) + f(x_{i-3}))}{8h^3}$ |
| Fourth Derivative                                                                                                   |

$$f'''(x_i) = \frac{f(x_{i+2}) - 4f(x_{i+1}) + 6f(x_i) - 4f(x_{i-1}) + f(x_{i-2}))}{h^4}$$

$$f'''(x_i) = \frac{-f(x_{i+3}) + 12f(x_{i+2}) - 39f(x_{i+1}) + 56f(x_i) - 39f(x_{i-1}) + 12f(x_{i-2}) - f(x_{i-3}))}{6h^4}$$

## EXAMPLE 21.1

### High-Accuracy Differentiation Formulas

#### Problem Statement

Recall that in [Example 4.4](#) we estimated the derivative of

$$f(x) = -0.1x^4 - 0.15x^3 - 0.5x^2 - 0.25x + 1.2$$

at  $x = 0.5$  using finite differences and a step size of  $h = 0.25$ . The results are summarized in the following table. Note that the errors are based on the true value of  $f'(0.5) = -0.9125$ .

|                                | Backward<br>$\mathcal{O}(h)$ | Centered<br>$\mathcal{O}(h^2)$ | Forward<br>$\mathcal{O}(h)$ |
|--------------------------------|------------------------------|--------------------------------|-----------------------------|
| <b>Estimate</b>                | -0.714                       | -0.934                         | -1.155                      |
| <b><math>\epsilon_t</math></b> | 21.7%                        | -2.4%                          | -26.5%                      |

Repeat this computation, but employ the high-accuracy formulas from [Fig. 21.3](#) through [Fig. 21.5](#).

#### Solution

The data needed for this example are

$$\begin{array}{ll} x_{i-2} = 0 & f(x_{i-2}) = 1.2 \\ x_{i-1} = 0.25 & f(x_{i-1}) = 1.1035156 \\ x_i = 0.5 & f(x_i) = 0.925 \\ x_{i+1} = 0.75 & f(x_{i+1}) = 0.6363281 \\ x_{i+2} = 1 & f(x_{i+2}) = 0.2 \end{array}$$

The forward difference of accuracy  $\mathcal{O}(h^2)$  is computed as ([Fig. 21.3](#))

$$f'(0.5) = \frac{-0.2 + 4(0.6363281) - 3(0.925)}{2(0.25)} = -0.859375 \quad \varepsilon_t = 5.82\%$$

The backward difference of accuracy  $O(h^2)$  is computed as ( **Fig. 21.4**)

$$f'(0.5) = \frac{3(0.925) - 4(1.1035156) + 1.2}{2(0.25)} = -0.878125 \quad \varepsilon_t = 3.77\%$$

The centered difference of accuracy  $O(h^4)$  is computed as ( **Fig. 21.5**)

$$f'(0.5) = \frac{-0.2 + 8(0.6363281) - 8(1.1035156) + 1.2}{12(0.25)} = -0.9125 \quad \varepsilon_t = 0\%$$

As expected, the errors for the forward and backward differences are considerably lower than the results from  **Example 4.4**. However, surprisingly, the centered difference yields the exact derivative at  $x = 0.5$ . This is because the formula based on the Taylor series is equivalent to passing a fourth-order polynomial through the data points.

## 21.3 RICHARDSON EXTRAPOLATION

To this point, we have seen that there are two ways to improve derivative estimates when employing finite differences: (1) decrease the step size or (2) use a higher-order formula that employs more points. A third approach, based on Richardson extrapolation, uses two derivative estimates to compute a third, more accurate, approximation.

Recall from [Sec. 20.2.1](#) that Richardson extrapolation provided a means to obtain an improved integral estimate by the formula [[Eq. \(20.4\)](#)]

$$I = \mathcal{I}(h_2) + \frac{1}{(h_1/h_2)^2 - 1} [\mathcal{I}(h_2) - \mathcal{I}(h_1)] \quad (21.18)$$

where  $\mathcal{I}(h_1)$  and  $\mathcal{I}(h_2)$  are integral estimates using two step sizes:  $h_1$  and  $h_2$ . Because of its convenience when expressed as a computer algorithm, this formula is usually written for the case where  $h_2 = h_1/2$ , as in

$$I = \frac{4}{3} \mathcal{I}(h_2) - \frac{1}{3} \mathcal{I}(h_1) \quad (21.19)$$

In a similar fashion, [Eq. \(21.19\)](#) can be written for derivatives as

Page 513

$$D = \frac{4}{3} D(h_2) - \frac{1}{3} D(h_1) \quad (21.20)$$

For centered difference approximations with  $\mathcal{O}(h^2)$ , the application of this formula will yield a new derivative estimate of  $\mathcal{O}(h^4)$ .

### EXAMPLE 21.2

#### Richardson Extrapolation

##### Problem Statement

Using the same function as in [Example 21.1](#), estimate the first derivative at  $x = 0.5$  employing step sizes of  $h_1 = 0.5$  and  $h_2 = 0.25$ . Then use [Eq. \(21.20\)](#) to compute an improved estimate with Richardson extrapolation. Recall that the true value is  $-0.9125$ .

##### Solution

The first-derivative estimates can be computed with centered differences as

$$D(0.5) = \frac{0.2 - 1.2}{1} = -1.0 \quad \varepsilon_t = -9.6\%$$

and

$$D(0.25) = \frac{0.6363281 - 1.103516}{0.5} = -0.934375 \quad \varepsilon_t = -2.4\%$$

The improved estimate can be determined by applying [Eq. \(21.20\)](#) to give

$$D = \frac{4}{3}(-0.934375) - \frac{1}{3}(-1) = -0.9125$$

which for the present case is exact.

The previous example yielded an exact result because the function being analyzed was a fourth-order polynomial. The exact outcome was due to the fact that Richardson extrapolation is actually equivalent to fitting a higher-order polynomial through the data and then evaluating the derivatives by centered divided differences. Thus, the present case matched the derivative of the fourth-order polynomial precisely. For most other functions, of course, this would not occur, and our derivative estimate would be improved but not exact. Consequently, as was the case for the application of Richardson extrapolation, the approach can be applied iteratively using a Romberg algorithm until the result falls below an acceptable error criterion.

## 21.4 DERIVATIVES OF UNEQUALLY SPACED DATA

The approaches discussed to this point are primarily designed to determine the derivative of a given function. For the finite-difference approximations of [Sec. 21.2](#), the data had to be evenly spaced. For the Richardson extrapolation technique of [Sec. 21.3](#), the data also had to be evenly spaced and generated for successively halved intervals. Such control of data spacing is usually available only in cases where we can use a function to generate a table of values.

In contrast, empirically derived information—that is, data from experiments or field studies—is often collected at unequal intervals. Such information cannot be analyzed with the techniques discussed to this point.

One way to handle nonequispaced data is to fit a Lagrange interpolating polynomial [recall [Eq. \(17.21\)](#)] to a set of adjacent points that bracket the location value at which you want to evaluate the derivative. Remember that this polynomial does not require that the points be equispaced. The polynomial can then be differentiated analytically to yield a formula that can be used to estimate the derivative.

For example, you can fit a second-order Lagrange polynomial to three adjacent points  $(x_0, y_0)$ ,  $(x_1, y_1)$ , and  $(x_2, y_2)$ . Differentiating the polynomial yields:

$$f'(x) = f(x_0) \frac{2x - x_1 - x_2}{(x_0 - x_1)(x_0 - x_2)} + f(x_1) \frac{2x - x_0 - x_2}{(x_1 - x_0)(x_1 - x_2)} + f(x_2) \frac{2x - x_0 - x_1}{(x_2 - x_0)(x_2 - x_1)}$$

where  $x$  is the value at which you want to estimate the derivative. Although this equation is certainly more complicated than the first-derivative approximation from [Fig. 21.3](#) through [Fig. 21.5](#), it has some important advantages. First, it can provide estimates anywhere within the range prescribed by the three points. Second, the points themselves do not have to be equally spaced. Third, the derivative estimate is of the same accuracy as the centered difference [[Eq. \(4.25\)](#)]. In fact, for equispaced points, [Eq. \(21.21\)](#) evaluated at  $x = x_1$  reduces to [Eq. \(4.25\)](#).

Page 514

### EXAMPLE 21.3

#### Differentiating Unequally Spaced Data

##### Problem Statement

As in [Fig. 21.6](#), a temperature gradient can be measured down into the soil. The heat flux at the soil-air interface can be computed with Fourier's law ([Table 21.1](#)):



$$q(z=0) = -k \left. \frac{dT}{dz} \right|_{z=0}$$

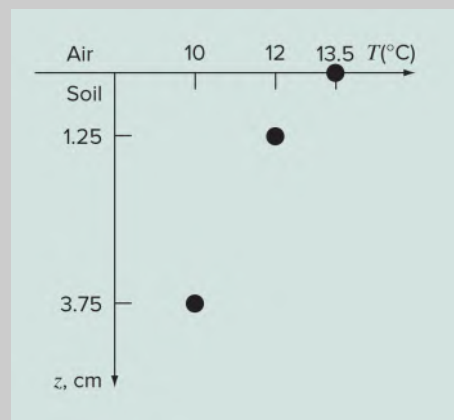
where  $q(z)$  = heat flux ( $\text{W}/\text{m}^2$ ),

$k$  = coefficient of thermal conductivity for soil [=  $0.5 \text{ W}/(\text{m}\cdot\text{K})$ ],

$T$  = temperature (K), and  $z$  = distance measured down from the surface into the soil (m).

Note that a positive value for flux means that heat is transferred from the air to the soil. Use numerical differentiation to evaluate the gradient at the soil-air interface and employ this estimate to determine the heat flux into the ground.

**FIGURE 21.6** Temperature versus depth into the soil.



## Solution

 Equation (21.21) can be used to calculate the derivative at the air-soil interface as

$$\begin{aligned} f'(0) &= 13.5 \frac{2(0) - 0.0125 - 0.0375}{(0 - 0.0125)(0 - 0.0375)} + 12 \frac{2(0) - 0 - 0.0375}{(0.0125 - 0)(0.0125 - 0.0375)} \\ &\quad + 10 \frac{2(0) - 0 - 0.0125}{(0.0375 - 0)(0.0375 - 0.0125)} \\ &= -1440 + 1440 - 133.333 = -133.333 \text{ K/m} \end{aligned}$$

which can be used to compute

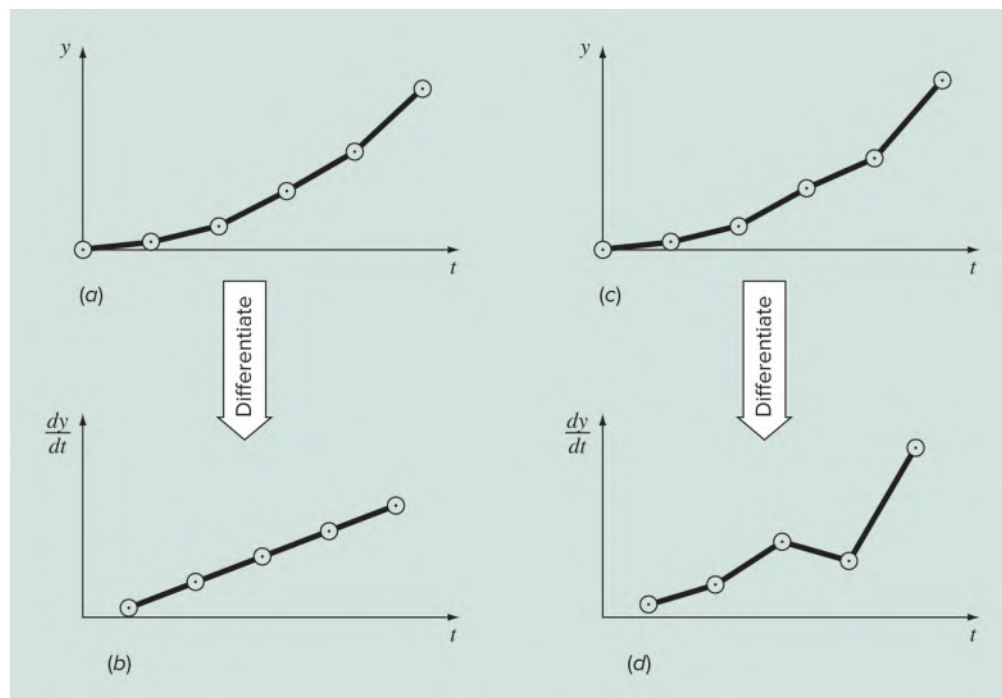
$$q(z=0) = -0.5 \frac{\text{W}}{\text{m K}} \left( -133.333 \frac{\text{K}}{\text{m}} \right) = 66.667 \frac{\text{W}}{\text{m}^2}$$

## 21.5 DERIVATIVES AND INTEGRALS FOR DATA WITH ERRORS

Aside from unequal spacing, another problem related to differentiating empirical data is that these data usually include measurement error. A shortcoming of numerical differentiation is that it tends to amplify errors in the data.

 **Figure 21.7a** shows smooth, error-free data that when numerically differentiated yield a smooth result ( **Fig. 21.7b**). In contrast,  **Fig. 21.7c** uses the same data, but with alternating points raised and lowered slightly. This minor modification is barely apparent from  **Fig. 21.7c**. However, the resulting effect in  **Fig. 21.7d** is significant.

**FIGURE 21.7** Illustration of how small data errors are amplified by numerical differentiation: (a) data with no error, (b) the resulting numerical differentiation of curve (a), (c) data modified slightly, and (d) the resulting differentiation of curve (c) manifesting increased variability. In contrast, the reverse operation of integration [moving from (d) to (c) by taking the area under (d)] tends to attenuate or smooth data errors.



The error amplification occurs because differentiation is subtractive. Hence, random positive and negative errors tend to add. In contrast, the fact that integration is a summing process makes it very

forgiving with regard to uncertain data. In essence, as points are summed to form an integral, random positive and negative errors cancel out.

As might be expected, the primary approach for determining derivatives for imprecise data is to use least-squares regression to fit a smooth, differentiable function to the data. In the absence of any other information, a lower-order polynomial regression might be a good first choice. Obviously, if a fundamental functional relationship between the dependent and independent variable is known, this relationship should form the basis for the least-squares fit.

## 21.6 PARTIAL DERIVATIVES

Partial derivatives along a single dimension are computed in the same fashion as ordinary derivatives. For example, suppose that we want to determine the partial derivatives for a two-dimensional function  $f(x, y)$ . For equally spaced data, the partial first derivatives can be approximated with centered differences:

$$\frac{\partial f}{\partial x} = \frac{f(x + \Delta x, y) - f(x - \Delta x, y)}{2\Delta x} \quad (21.22)$$

$$\frac{\partial f}{\partial y} = \frac{f(x, y + \Delta y) - f(x, y - \Delta y)}{2\Delta y} \quad (21.23)$$

All the other formulas and approaches discussed to this point can be applied to evaluate partial derivatives in a similar fashion.

For higher-order derivatives, we might want to differentiate a function with respect to two or more different variables. The result is called a *mixed partial derivative*. For example, we might want to take the partial derivative of  $f(x, y)$  with respect to both independent variables

$$\frac{\partial^2 f}{\partial x \partial y} = \frac{\partial}{\partial x} \left( \frac{\partial f}{\partial y} \right) \quad (21.24)$$

To develop a finite-difference approximation, we can first form a difference in  $x$  of the partial derivatives in  $y$ .

Page 516

$$\frac{\partial^2 f}{\partial x \partial y} = \frac{\frac{\partial f}{\partial y}(x + \Delta x, y) - \frac{\partial f}{\partial y}(x - \Delta x, y)}{2\Delta x} \quad (21.25)$$

Then, we can use finite differences to evaluate each of the partials in  $y$ .

$$\frac{\partial^2 f}{\partial x \partial y} = \frac{\frac{f(x + \Delta x, y + \Delta y) - f(x + \Delta x, y - \Delta y)}{2\Delta y} - \frac{f(x - \Delta x, y + \Delta y) - f(x - \Delta x, y - \Delta y)}{2\Delta y}}{2\Delta x}$$

Collecting terms yields the final result

$$\frac{\partial^2 f}{\partial x \partial y} = \frac{f(x + \Delta x, y + \Delta y) - f(x + \Delta x, y - \Delta y) - f(x - \Delta x, y + \Delta y) + f(x - \Delta x, y - \Delta y)}{4\Delta x \Delta y}$$



## 21.7 NUMERICAL DIFFERENTIATION WITH PYTHON

The NumPy module in Python provides the ability to determine derivatives of data based primarily on two functions: `diff` and `gradient`.

### 21.7.1 Python NumPy Function: `diff`

It is certainly possible to create a simple “home grown” function in Python to difference an array of data. A possible version would be

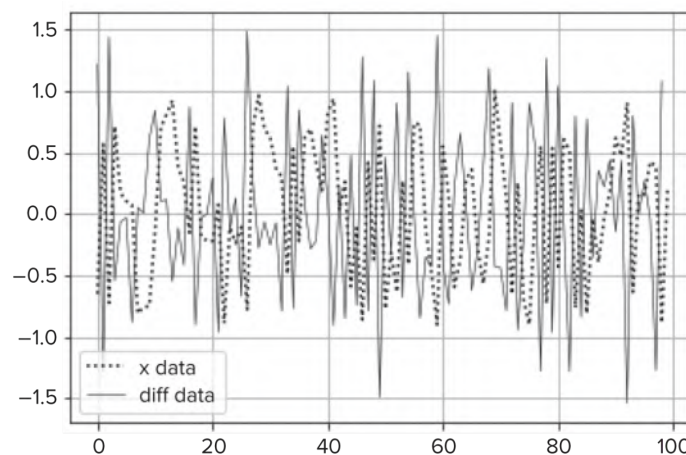
```
import numpy as np
def diffdata(x):
    n = len(x)
    xd = np.zeros(n-1)
    for i in range(n-1):
        xd[i] = x[i+1] - x[i]
    return xd
```

This code can be tested with a Python script that uses an array of random numbers drawn from a uniform distribution.

```
x = np.random.uniform(-1.,1.,100)
xdiff = diffdata(x)

import pylab
pylab.plot(x,c='k',ls=':',label='x data')
pylab.plot(xdiff,c='k',lw=0.5,label='diff data')
pylab.grid()
pylab.legend()
```

and yields the plot



Careful observation reveals that the differenced data have a higher variance than the original data. Such is the nature of differencing data with random content. Integration attenuates noise, and differentiation amplifies it.

The `diff` function from Python's NumPy module operates just like the above `diffdata` function. As described in the following example, `diff` can be employed to determine finite-difference approximations of first derivatives.

## EXAMPLE 21.4

### Using `diff` for Differentiation

#### Problem Statement

Explore how the Python `diff` function can be employed to differentiate the function

$$f(x) = 0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5$$

from  $x = 0$  to  $0.8$ . Compare your results with the exact solution:

$$f'(x) = 25 - 400x + 2025x^2 - 3600x^3 + 2000x^4$$

#### Solution

We can first express  $f(x)$  as an anonymous function:

```
f = lambda x: 0.2 + 25.*x - 200.*x**2 + 675*x**3 - 900.x**4 + 400.*x**5
```

We can then generate a series of equally spaced values of the independent and dependent variables:

```
import numpy as np
x = np.arange(0.,0.9,0.1)
y = f(x)
```

The `diff` function can then be used to determine the differences between the adjacent elements of each vector. For example,

```
dx = np.diff(x)
print(dx)

[0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1]
```

As expected, the result represents the differences between each pair of elements of  $x$ . To compute the divided-difference approximations of the derivative, we merely perform an array division of the  $y$  differences by the  $x$  differences by adding

```
dy = np.diff(y)
dydx = dy/dx
print(dydx)

[ 1.089e+01 -1.000e-02 3.190e+00 8.490e+00 8.690e+00 1.390e+00
 -1.101e+01 -2.131e+01]
```

Note that, because we are using equally spaced values, after generating the  $x$  values, we could have simply performed the above computation concisely as

```
dydx = np.diff(f(x))/0.1
```

The array `dydx` now contains derivative estimates corresponding to the midpoint between adjacent elements. Therefore, in order to create a plot of our results, we should first generate an array holding the  $x$  values for the midpoint of each interval:

```
n = len(x)
xmid = []
for i in range(n-1):
    xmid.append((x[i]+x[i+1])/2.)
```

As a final step, we can compute values for the analytical derivative as a finer level of resolution to include on the plot for comparison.

Page 518

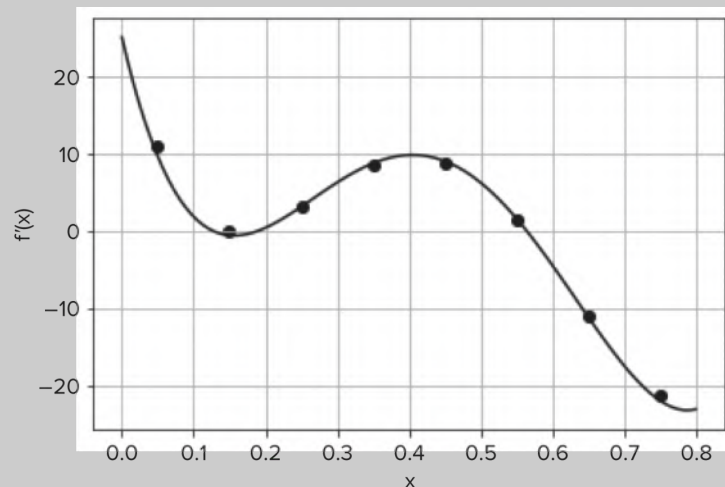
```
xa = np.linspace(0.,0.8,100)
ya = 25. - 400.*xa + 2025.*xa**2 - 3600.*xa**3 + 2000.*xa**4
```

A plot of numerical and analytical estimates can be generated with

```
import pylab
pylab.scatter(xmid,dydx,c='k')
pylab.plot(xa, ya, c='k')
pylab.grid()
pylab.xlabel('x')
pylab.ylabel("f'(x)")
```

As displayed in  **Fig. 21.8**, the results compare favorably for this case.

**FIGURE 21.8** Comparison of the exact derivative (line) and numerical estimates (•) computed with Python NumPy's `diff` function.



Note that, aside from evaluating derivatives, the `diff` function comes in handy as a programming tool for testing certain characteristics of arrays. For example, the following line of code displays an error message if the array has unequal spacing:

```
if np.any(np.diff(np.diff(x)) >= 1.e-15): print('unequal spacing')
```

One must be careful in testing “not equal to zero” because of round-off differences at the limit of precision. If we modify the above statement to



```
if np.any(np.diff(np.diff(x)) != 0 ): print('unequal spacing')
```

we get an “unequal spacing” display even for an  $x$  array created with (ostensibly) equal spacing.

Another common use is to detect whether an array is in ascending or descending order. For example, the following line of code rejects an array that is not in ascending order:

```
if np.any(np.diff(x) < 0): print('not in ascending order')
```

By reversing the  $<$  to  $>$ , we can test for descending order. If we want to avoid duplicate adjacent values, we would include the equals sign with the test operator, that is,  $<=$  or  $>=$ .

---

Page 519

---

## 21.7.2 Python NumPy Function: `gradient`

NumPy’s `gradient` function also returns differences. However, it does so in a manner that is more compatible with evaluating derivatives at the independent variable values themselves rather than in the intervals between  $x$  values. The syntax for the `gradient` function is

```
fx = np.gradient(f)
```

where  $f$  = a one-dimensional array of length  $n$ , and  $fx$  is an array of length  $n$  containing differences based on  $f$ . Just as with the `diff` function, the first value returned is the difference between the first and second values. However, for the interior values, a centered difference based on adjacent values is returned:

$$diff_i = \frac{f_{i+1} - f_{i-1}}{2}$$

The last value is then computed as the difference between the final two values. Hence, the results are akin to using centered differences for all the intermediate values with forward and backward differences at the ends.

Note that the spacing between points is assumed to be one. If the array represents equally spaced data, the following version divides all the results by the interval and hence returns the actual values of the derivatives:

```
fx = gradient(f,h)
```

where  $h$  = the spacing between the points. An alternate syntax can be used for unequally spaced data,

```
fx = gradient(f,x)
```

where  $x$  = an array with  $n$  values corresponding to the independent variable values corresponding to the  $n$   $f$  values.

### EXAMPLE 21.5

#### Using `gradient` for Differentiation

## Problem Statement

Use the `gradient` function to differentiate the same function that we analyzed in [Example 21.4](#) with the `diff` function.

## Solution

In the same fashion as [Example 21.4](#), we can generate a series of equally spaced values of the independent and dependent variables:

```
f = lambda x: 0.2 + 25.*x - 200.*x**2 + 675.*x**3 - 900.*x**4 + 400.*x**5
import numpy as np
x = np.arange(0.,0.9,0.1)
y = f(x)
```

We can then use the `gradient` function to determine the derivatives as

```
dydx = np.gradient(y,0.1)
print(dydx)

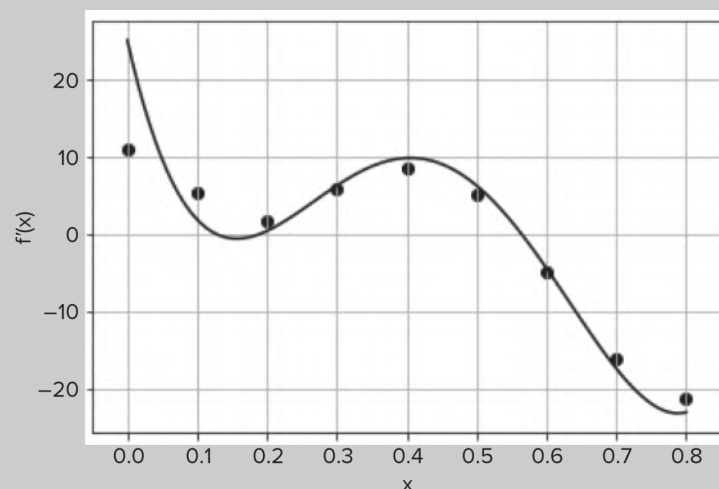
[ 10.89   5.44   1.59   5.84   8.59   5.04  -4.81 -16.16 -21.31]
```

As in [Example 21.4](#), we can generate values for the analytical derivative and display both the numerical and analytical estimates on a plot:

```
import pylab
pylab.scatter(x,dydx,c='k')
pylab.plot(xa,ya,c='k')
pylab.grid()
pylab.xlabel('x')
pylab.ylabel("f'(x)")
```

As displayed in [Fig. 21.9](#), the results are not as accurate as those obtained with the `diff` function in [Example 21.4](#). This occurs because `gradient` employs intervals that are two times (0.2) as wide as for those used for `diff` (0.1).

**FIGURE 21.9** Comparison of the exact derivative (line) with numerical estimates (•) computed with Python NumPy's `gradient` function.



Beyond one-dimensional arrays, the `gradient` function has the capability of determining the partial derivatives of multidimensional arrays. For example, for a two-dimensional array, `f`, the function can be invoked as

```
fx, fy = np.gradient(f,h)
```

where `fx` corresponds to the differences in the  $x$  (column) direction, `fy` corresponds to the differences in the  $y$  (row) direction, and `h` = the spacing between points. If `h` is omitted, the spacing between points is assumed to be one. Alternately, one can specify `hx` and `hy` as different spacing in those dimensions, and it is possible to provide arrays for  $x$  and  $y$  with unequal spacing. In the next section, we will illustrate how the `gradient` function can be used to visualize vector fields.

## VISUALIZING FIELDS

### Background

Beyond the determination of derivatives in one dimension, the gradient function is also useful for determining partial derivatives in two or more dimensions. In particular, it can be used in conjunction with the Python Matplotlib module to produce visualizations of vector fields.

To understand how this is done, we can return to our discussion of partial derivatives at the end of [Sec. 21.1.1](#). Recall that we used mountain elevation as an example of a two-dimensional function. We can represent such a function mathematically as

$$z = f(x, y)$$

where  $z$  = elevation,  $x$  = distance measured along the east-west axis, and  $y$  = distance measured along the north-south axis.

For this example, the partial derivatives provide the slopes in the directions of the axes. However, if you were mountain climbing, you would probably be much more interested in determining the direction of the maximum slope. If we think of the two partial derivatives as component vectors, the answer is provided by

$$\nabla f = \frac{\partial f}{\partial x} \mathbf{i} + \frac{\partial f}{\partial y} \mathbf{j}$$

where  $\nabla f$  is referred to as the *gradient* of  $f$ . This vector, which represents the steepest slope, has a magnitude

$$|\nabla f| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

and a direction

$$\angle(\nabla f) = \theta = \tan^{-1}\left(\frac{\partial f/\partial y}{\partial f/\partial x}\right)$$

where  $\theta$  = the angle measured clockwise from the  $x$  axis.

Now, suppose that we generate a grid of points in the  $xy$  plane and use the foregoing equations to draw the gradient vector at each point. The result would be a field of arrows indicating the steepest direction to the peak from any point. Conversely, if we plotted the negative of the gradient, it would indicate how a ball would travel if it were released to roll downhill from any point.

Such graphical representations are so useful that the Python Matplotlib has a special function, `quiver`, to create such plots. A simple representation of its syntax is

```
import matplotlib.pyplot as plt
plt.quiver([X,Y],U,V,[C])
```

where  $X$  and  $Y$  are two-dimensional arrays containing the position coordinates and  $U$  and  $V$  are two-dimensional arrays containing the partial derivatives. There is an optional  $C$  argument to specify color. The following example demonstrates the use of `quiver` to visualize a field.

## Problem Statement

Employ the `gradient` function to determine partial derivatives for the following two-dimensional function:

$$f(x, y) = y - x - 2x^2 - 2xy - y^2$$

from  $x = -2$  to  $2$  and  $y = 1$  to  $3$ . Then, use `quiver` to superimpose a vector field on a contour plot of the function.

## Solution

We can first express  $f(x, y)$  as an anonymous `lambda` function:

```
f = lambda x,y: y - x - 2.*x**2 - 2.*x*y - y**2
```

A series of equally spaced values of the independent variables  $x$  and  $y$  along with the corresponding  $z$  values can be generated with these lines of code:

```
import numpy as np
x = np.arange(-2.,0.25,0.25)
y = np.arange(1.,3.25,0.25)
X,Y = np.meshgrid(x,y)
Z = f(X,Y)
```

The `gradient` function can be utilized to determine the partial derivatives:

```
fx,fy = np.gradient(Z,0.25)
```

We can then create a contour plot of the results:

```
fig = plt.figure()
ax = fig.add_subplot(111)
ax.contour(X,Y,Z,colors='k')
ax.grid()
```

As a final step, the resultant of the partial derivatives can be superimposed as vector arrows on the contour plot:

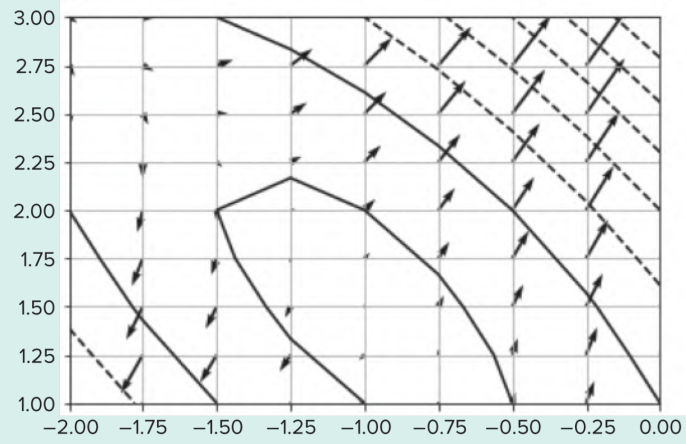
```
plt.quiver(X,Y,-fx,-fy,color='k',width=0.005)
```

Note that we have displayed the negative of the resultants in order that they point “downhill.”

The resulting plot is shown in [Fig. 21.10](#). The function’s peak occurs at  $x = -1$  and  $y = 1.5$  and then drops off in all directions. As indicated by the lengthening arrows, the gradient drops off more steeply to the northeast and southwest.

---

**FIGURE 21.10** Python Matplotlib contour plot of a two-dimensional function with the resultant of the partial derivatives displayed as arrows.



## PROBLEMS

**21.1** Compute forward and backward difference approximations of  $O(h)$  and  $O(h^2)$ , and centered difference approximations of  $O(h^2)$  and  $O(h^4)$  for the first derivative of  $y = \sin(x)$  at  $x = \pi/4$  using a value of  $h = \pi/12$ . Estimate the true percent relative error  $\varepsilon_t$  for each approximation.

**21.2** Use centered-difference approximations to estimate the first and second derivatives of  $y = e^x$  at  $x = 2$  for  $h = 0.1$ . Employ both  $O(h^2)$  and  $O(h^4)$  formulas for your estimates. Estimate the true percent relative error  $\varepsilon_t$  for each approximation.

**21.3** Use a Taylor series expansion to derive a centered-difference approximation to the third derivative that is second-order accurate. To do this, you will have to use four different expansions for the points  $x_{i-2}$ ,  $x_{i-1}$ ,  $x_{i+1}$ , and  $x_{i+2}$ . In each case, the expansion will be around the point  $x_i$ . The interval  $\Delta x$  will be used in each case of  $i - 1$  and  $i + 1$ , and  $2\Delta x$  will be used in each case of  $i - 2$  and  $i + 2$ . The four equations must then be combined in a way to eliminate the first and second derivatives. Carry enough terms that will be truncated in order to determine the order of the approximation.

**21.4** Use Richardson extrapolation to estimate the first derivative of  $y = \cos(x)$  at  $x = \pi/4$  using step sizes of  $h_1 = \pi/3$  and  $h_2 = \pi/6$ . Employ centered differences of  $O(h^2)$  for the initial estimates.

**21.5** Repeat  **Prob. 21.4**, but for the first derivative of  $\ln(x)$  at  $x = 5$  using  $h_1 = 2$  and  $h_2 = 1$ .

**21.6** Employ  **Eq. (21.21)** to determine the first derivative of  $y = 2x^4 - 6x^3 - 12x - 8$  at  $x = 0$  based on values at  $x_0 = -0.5$ ,  $x_1 = 1$ , and  $x_2 = 2$ . Compare this result with the true value and with an estimate obtained using a centered-difference approximation based on  $h = 1$ .

**21.7** Prove that, for equispaced points,  **Eq. (21.21)** reduces to  **Eq. (4.25)** at  $x = x_1$ .

**21.8** Develop a Python function, called `romdiff`, to apply the Romberg algorithm to estimate the derivative of a given function. The function should have arguments including the name of the function, the independent variable value at which the derivative is estimated, a stopping criterion, and an iteration limit. The latter should have default values of  $1 \times 10^{-8}$  and 30, respectively. Test your function by estimating the derivative of  $f(x) = e^{-0.5x}$  at  $x = 1$ . Compare your result with the true value. Hint: Make reference to  **Fig. 20.2**.

**21.9** Develop a Python script to obtain first-derivative estimates for unequally spaced data. Test it with the following data:

| $x$    | 0.6    | 1.5    | 1.6    | 2.5     | 3.5     |
|--------|--------|--------|--------|---------|---------|
| $f(x)$ | 0.9036 | 0.3734 | 0.3261 | 0.08422 | 0.01596 |

Create a plot showing the derivative estimates versus  $x$  as markers and a continuous curve for the true derivative versus  $x$ , where  $f(x) = 5e^{-2x} - 10xe^{-2x}$ . Compare the estimates and the true values.

**21.10** Develop a Python function, called `diffeq`, that computes and returns first- and second-derivative estimates of order  $O(h^2)$  based on the formulas in [Figs. 21.3](#) through [21.5](#). The function's first line should be

```
def diffeq(x,y):
```

where  $x$  and  $y$  are input arrays of length  $n$  containing values of the independent and dependent variables, respectively. The function should return  $dydx$  and  $dydx2$  as output arrays of length  $n$  containing first- and second-derivative estimates at each value of the independent variable. The function should return an error message if the input arrays are not the same length, if the values of the independent variable are not equally spaced, or if there are not enough data to complete the estimates. Test your program with the data from [Prob. 21.11](#). Create a plot with two horizontal subplots of the first- and second-derivative estimates versus  $x$ . Also, create tests to demonstrate your error checks.

**21.11** The following data were collected for the distance traveled versus time for a rocket:

|                     |   |    |    |    |     |     |
|---------------------|---|----|----|----|-----|-----|
| <b><i>t</i>, s</b>  | 0 | 25 | 50 | 75 | 100 | 125 |
| <b><i>y</i>, km</b> | 0 | 32 | 58 | 78 | 92  | 100 |

Use numerical differentiation to estimate the rocket's velocity and acceleration at each time. Carry out your calculations manually.

**21.12** A jet fighter's position on an aircraft carrier's runway was timed during landing:

|                    |     |      |      |      |      |      |      |
|--------------------|-----|------|------|------|------|------|------|
| <b><i>t</i>, s</b> | 0   | 0.52 | 1.04 | 1.75 | 2.37 | 3.25 | 3.83 |
| <b><i>x</i>, m</b> | 153 | 185  | 208  | 249  | 261  | 271  | 273  |

where  $x$  is the distance from the beginning of the carrier deck. Estimate the velocity and acceleration using numerical differentiation.

**21.13** Use the following data to find the velocity and acceleration at  $t = 10$  seconds.

|                              |   |     |     |     |     |     |     |     |     |
|------------------------------|---|-----|-----|-----|-----|-----|-----|-----|-----|
| <b>Time, <i>t</i>, s</b>     | 0 | 2   | 4   | 6   | 8   | 10  | 12  | 14  | 16  |
| <b>Position, <i>x</i>, m</b> | 0 | 0.7 | 1.8 | 3.4 | 5.1 | 6.3 | 7.3 | 8.0 | 8.4 |

Use second-order correct (a) centered, (b) forward, and (c) backward finite-difference methods.

**21.14** A plane is being tracked by radar, and data are taken every second in polar coordinates  $\theta$  and  $r$ .

|                                   |      |      |      |      |      |      |
|-----------------------------------|------|------|------|------|------|------|
| <b><i>t</i>, s</b>                | 200  | 202  | 204  | 206  | 208  | 210  |
| <b><math>\theta</math>, (rad)</b> | 0.75 | 0.72 | 0.70 | 0.68 | 0.67 | 0.66 |



|                    |      |      |      |      |      |      |
|--------------------|------|------|------|------|------|------|
| <b><i>r</i>, m</b> | 5120 | 5370 | 5560 | 5800 | 6030 | 6240 |
|--------------------|------|------|------|------|------|------|

At 206 seconds, use the centered difference (second-order correct) to find the vector expressions for velocity,  $\vec{v}$ , and acceleration,  $\vec{a}$ . The velocity and acceleration, expressed in polar coordinates, are

$$\vec{v} = \dot{r}\vec{e}_r + r\dot{\theta}\vec{e}_\theta \quad \text{and} \quad \vec{a} = (\ddot{r} - r\dot{\theta}^2)\vec{e}_r + (r\ddot{\theta} + 2\dot{r}\dot{\theta})\vec{e}_\theta$$

**21.15** For the data in the table below, use regression to estimate the acceleration at each time for the following data with second-, third-, and fourth-order polynomials. Plot the results.

|                 |    |    |      |     |    |    |    |     |     |    |
|-----------------|----|----|------|-----|----|----|----|-----|-----|----|
| <b><i>t</i></b> | 1  | 2  | 3.25 | 4.5 | 6  | 7  | 8  | 8.5 | 9.3 | 10 |
| <b><i>v</i></b> | 10 | 12 | 11   | 14  | 17 | 16 | 12 | 14  | 14  | 10 |

**21.16** The standard normal (Gaussian) distribution is defined by the density function

$$f(z) = \frac{1}{\sqrt{2\pi}} \cdot e^{-z^2/2}$$

Develop a Python script to determine the inflection points of this function via numerical estimates of the second derivative.

**21.17** The following data were generated from the standard normal distribution. Use Python to estimate the inflection points of the distribution.

|                    |         |         |         |         |         |
|--------------------|---------|---------|---------|---------|---------|
| <b><i>z</i></b>    | -2      | -1.5    | -1      | -0.5    | 0       |
| <b><i>f(z)</i></b> | 0.05399 | 0.12952 | 0.24197 | 0.35207 | 0.39894 |
| <b><i>z</i></b>    | 0.5     | 1       | 1.5     | 2       |         |
| <b><i>f(z)</i></b> | 0.35207 | 0.24197 | 0.12952 | 0.05399 |         |

**21.18** Use the `diff` function to develop a Python function to compute finite-difference approximations to the first and second derivatives at each  $x$  value in the table below. Use finite-difference approximations that are second-order correct,  $O(h^2)$ .

|                 |     |     |     |     |     |     |     |     |     |      |
|-----------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|
| <b><i>x</i></b> | 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9    |
| <b><i>y</i></b> | 1.4 | 2.1 | 3.3 | 4.8 | 6.8 | 6.6 | 8.6 | 7.5 | 8.9 | 10.9 |



**21.19** The objective of this problem is to compare second-order-accurate forward, backward, and centered finite-difference approximations of the first derivative of a function to the actual value of the derivative. This will be done for

$$f(x) = e^{-2x} - x$$

- (a) Use calculus to determine the correct value of the derivative at  $x = 2$ .
- (b) Develop a Python function to evaluate centered finite-difference approximations, starting with  $\Delta x = 0.5$ . Thus, for the first evaluation, the  $x$  values for the centered-difference approximation will be  $x = 2 \pm 0.5$  or  $x = 1.5$  and  $2.5$ . Then, decrease in increments of  $0.01$  down to a minimum of  $0.01$ .
- (c) Repeat (b) for the second-order forward and backward differences. Note: These can be done at the same time as the centered difference is computed in a loop.
- (d) Plot the results of (b) and (c) versus  $\Delta x$ . Include the exact result on your plot for comparison.

**21.20** You have to measure the flow rate of water through a small pipe. In order to accomplish that you place a graduated cylinder at the pipe's outlet and measure the volume in the cylinder as a function of time, as tabulated below. Estimate the flow rate at  $t = 7$  s.

|                               |   |   |   |      |
|-------------------------------|---|---|---|------|
| <b>Time, s</b>                | 0 | 1 | 5 | 8    |
| <b>Volume, cm<sup>3</sup></b> | 0 | 1 | 8 | 16.4 |

**21.21** The velocity,  $v(\text{m/s})$ , of air flowing past a flat surface is measured at several distances,  $y(\text{m})$ , perpendicular to the surface. The data are shown in the table below. Determine the shear stress,  $\tau(\text{Pa})$ , at the surface ( $y = 0$ ) using Newton's law of viscosity:

$$\tau = \mu \frac{du}{dy}$$

where  $\mu$  is the viscosity and equals  $1.8 \times 10^{-5} \text{ Pa}\cdot\text{s}$ .

|               |   |       |       |       |       |       |
|---------------|---|-------|-------|-------|-------|-------|
| <b>y, m</b>   | 0 | 0.002 | 0.006 | 0.012 | 0.018 | 0.024 |
| <b>u, m/s</b> | 0 | 0.287 | 0.899 | 1.915 | 3.048 | 4.299 |

**21.22** Fick's first law of diffusion states that

$$\text{mass flux} = -D \frac{dc}{dx} \quad (\text{P21.22})$$

where *mass flux* = the quantity of mass that passes across a unit area per unit time ( $\text{g}/\text{cm}^2/\text{s}$ ),  $D$  = a diffusion coefficient ( $\text{cm}^2/\text{s}$ ),  $c$  = concentration ( $\text{g}/\text{cm}^3$ ), and  $x$  = distance (cm). An environmental engineer measures the following concentrations of a pollutant in the pore waters of sediments underlying a lake ( $x = 0$  at the sediment-water interface and increases downward):

|                                                      |      |      |     |
|------------------------------------------------------|------|------|-----|
| <b><math>x, \text{ cm}</math></b>                    | 0    | 1    | 3   |
| <b><math>c, 10^{-6} \text{ g}/\text{cm}^3</math></b> | 0.06 | 0.32 | 0.6 |

Use the best numerical differentiation technique available to estimate the derivative at  $x = 0$ . Employ this estimate in conjunction with [Eq. \(P21.22\)](#) to compute the mass flux of pollutant out of the sediments and into the overlying waters. Assume a diffusivity of  $1.52 \times 10^{-6} \text{ cm}^2/\text{s}$ . For a lake with a sediment surface area of  $3.6 \times 10^6 \text{ m}^2$ , how much pollutant would be transported into the lake from the sediment in a year's time?

**21.23** The following data were collected when a large oil tanker was loading:

Page 524

|                                             |     |     |      |      |      |      |      |
|---------------------------------------------|-----|-----|------|------|------|------|------|
| <b><math>t, \text{ min}</math></b>          | 0   | 10  | 20   | 30   | 45   | 60   | 75   |
| <b><math>v, 10^6 \text{ barrels}</math></b> | 0.4 | 0.7 | 0.77 | 0.88 | 1.05 | 1.17 | 1.35 |

Calculate the flow rate,  $Q (= dV/dt)$  for each time to the order of  $h^2$ .

**21.24** Fourier's law is used routinely by architectural engineers to determine heat flow through walls by conduction.

$$q = -k \frac{dT}{dx}$$

where  $q$  is the heat flux ( $\text{W}/\text{m}^2$ ),  $k$  is the thermal conductivity [ $\text{W}/(\text{m} \cdot ^\circ\text{C})$ ],  $T$  is the temperature ( $^\circ\text{C}$ ), and  $x$  is the distance (m). The following temperatures are measured from the surface ( $x = 0$ ) into a stone wall:

|                                       |      |      |      |
|---------------------------------------|------|------|------|
| <b><math>x, \text{ m}</math></b>      | 0    | 0.08 | 0.16 |
| <b><math>T, ^\circ\text{C}</math></b> | 20.2 | 17   | 15   |

If the flux at  $x = 0$  is  $60 \text{ W}/\text{m}^2$ , compute  $k$ .

**21.25** The horizontal surface area,  $A_s(\text{m}^2)$ , of a lake at a particular depth can be computed from volume by differentiation:

$$A(z) = -\frac{dV}{dz}(z)$$

where  $V = \text{volume}(\text{m}^3)$  and  $z = \text{depth}(\text{m})$  as measured from the surface down to the bottom. The average concentration of a substance that varies with depth,  $\bar{c}(\text{g}/\text{m}^3)$ , can be computed by integration:

$$\bar{c} = \frac{\int_0^Z c(z)A(z)dz}{V_T}$$

where  $Z = \text{the total depth}(\text{m})$ , and  $V_T = \text{the total volume of the lake}$ , also given by

$$V_T = \int_0^Z A(z)dz$$

Based on the following data, determine the average concentration.

| $z, \text{ m}$            | 0      | 4      | 8      | 12     | 16     |
|---------------------------|--------|--------|--------|--------|--------|
| $v, 10^6 \text{ m}^3$     | 9.8175 | 5.1051 | 1.9635 | 0.3927 | 0.0000 |
| $c, \text{ g}/\text{m}^3$ | 10.2   | 8.5    | 7.4    | 5.2    | 4.1    |

**21.26** Faraday's law characterizes the voltage drop across an inductor as

$$V_L = L \frac{di}{dt}$$

where  $V_L = \text{voltage drop}(\text{V})$ ,  $L = \text{inductance}(\text{henrys}, 1 \text{ H} = 1 \text{ V}\cdot\text{s}/\text{A})$ , and  $t = \text{time}(\text{s})$ . Determine the voltage drop as a function of time from the following data for an inductance of 4 H.

| $t$ | 0 | 0.1  | 0.2  | 0.3  | 0.5  | 0.7 |
|-----|---|------|------|------|------|-----|
| $i$ | 0 | 0.16 | 0.32 | 0.56 | 0.84 | 2   |

**21.27** The rate of cooling of a body ( Fig. P21.27) can be expressed as

$$\frac{dT}{dt} = -k(T - T_a)$$

where  $T$  = temperature of the body ( $^{\circ}\text{C}$ ),

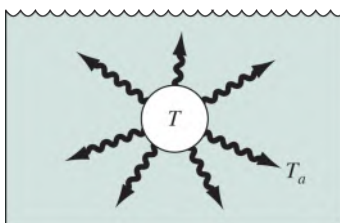
$T_a$  = temperature of the surrounding medium ( $^{\circ}\text{C}$ ), and

$k$  = a proportionality constant ( $\text{min}^{-1}$ ). Thus, this equation, called *Newton's law of cooling*, specifies that the rate of cooling is proportional to the difference between the temperature of the body, assumed to be relatively constant throughout the body, and of the surrounding medium. If a metal ball heated to  $80^{\circ}\text{C}$  is suspended in a large volume of water, held constant at  $20^{\circ}\text{C}$  by stirring, the temperature of the ball changes, as in the example data below:

| Time, min             | 0  | 5    | 10   | 15   | 20   | 25   |
|-----------------------|----|------|------|------|------|------|
| $T, ^{\circ}\text{C}$ | 80 | 44.5 | 30.0 | 24.1 | 21.7 | 20.7 |

Utilize numerical differentiation to estimate  $dT/dt$  at each value of time in the table. Plot  $dT/dt$  versus  $(T - T_a)$ . Use straight-line regression to estimate  $k$ .

**FIGURE P21.27**



**21.28** The enthalpy of a real gas is a function of pressure as described below. The data shown in the table were obtained for such a real gas. Estimate the enthalpy of the gas at 400 K and 50 atm (evaluate the integral from 0.1 to 50 atm). Note: Here, enthalpy has units of L·atm.

$$H = \int_0^P \left( V - T \left( \frac{\partial V}{\partial T} \right)_P \right) dP$$

| $P, \text{atm}$ | $V, L$              |                     |                     |
|-----------------|---------------------|---------------------|---------------------|
|                 | $T = 350 \text{ K}$ | $T = 400 \text{ K}$ | $T = 450 \text{ K}$ |
| 0.1             | 220                 | 250                 | 282.5               |
| 5               | 4.1                 | 4.7                 | 5.23                |
| 10              | 2.2                 | 2.5                 | 2.7                 |
| 20              | 1.35                | 1.49                | 1.55                |

| <i>P</i> , atm | <i>V</i> , L     |                  |                  |
|----------------|------------------|------------------|------------------|
|                | <i>T</i> = 350 K | <i>T</i> = 400 K | <i>T</i> = 450 K |
| 25             | 1.1              | 1.2              | 1.24             |
| 30             | 0.90             | 0.99             | 1.03             |
| 40             | 0.68             | 0.75             | 0.78             |
| 45             | 0.61             | 0.675            | 0.7              |
| 50             | 0.54             | 0.6              | 0.62             |

**21.29** For fluid flow over a flat surface, the heat flux to the surface due to conduction can be computed with Fourier's law,

$$q = -k \frac{dT}{dy}$$

where  $q$  = heat flux ( $\text{J}/\text{m}^2$ ),  $k$  = thermal conductivity [ $\text{W}/(\text{m}\cdot\text{K})$ ],  $T$  = temperature (K), and  $y$  = distance normal to the surface. Here are data for one such scenario:

| <i>y</i> , cm | 0   | 1   | 3   | 5   |
|---------------|-----|-----|-----|-----|
| <i>T</i> , K  | 900 | 480 | 270 | 210 |

If the plate's dimensions are 200 cm long by 50 cm wide, and  $k = 0.028 \text{ W}/(\text{m}\cdot\text{K})$ , determine the flux at the surface, and the overall heat transferred in watts.

**21.30** The pressure gradient for laminar flow through a smooth pipe of constant radius is given by the *Hagen-Poiseuille equation*:

Page 525

$$\frac{dp}{dx} = -\frac{8\mu Q}{\pi r^4}$$

where  $p$  = pressure (Pa),  $x$  = axial distance (m),  $\mu$  = fluid viscosity ( $\text{Pa}\cdot\text{s}$ ),  $Q$  = volumetric flow rate ( $\text{m}^3/\text{s}$ ), and  $r$  = pipe radius (m).

- (a) Determine the pressure drop (Pa) for a 10-cm-long tube and a viscous liquid ( $\mu = 0.005 \text{ Pa}\cdot\text{s}$ ), for a flow rate of  $Q = 1 \times 10^{-5} \text{ m}^3/\text{s}$ . Here, the pipe has a varying radius along its length given in samples the table below:

| <i>x</i> , cm | 0 | 2    | 4    | 5   | 6    | 7    | 10 |
|---------------|---|------|------|-----|------|------|----|
| <i>r</i> , mm | 2 | 1.35 | 1.34 | 1.6 | 1.58 | 1.42 | 2  |

- (b) Compare your result with the pressure drop that would have occurred if the tube had a constant radius equal to the average radius.
- (c) Determine the Reynolds number ( $Re$ ) for the flow at the average radius to verify that it is in the laminar region, where

$$Re = \frac{\rho v D}{\mu}$$

and  $\rho$  = fluid density, here = 1000 kg/m<sup>3</sup>,  $v$  = average fluid velocity (volumetric flow rate divided by the cross-sectional area).

**21.31** The following data for the heat capacity of benzene were generated from a polynomial model of this property. Use numerical differentiation to determine the order,  $n$ , of the polynomial used.

|                          |         |         |         |         |
|--------------------------|---------|---------|---------|---------|
| $T, K$                   | 300     | 400     | 500     | 600     |
| $c_p, kJ/(kmol \cdot K)$ | 82.888  | 112.136 | 136.933 | 157.744 |
| $T, K$                   | 700     | 800     | 900     | 1000    |
| $c_p, kJ/(kmol \cdot K)$ | 175.036 | 189.273 | 200.923 | 210.450 |

**21.32** The heat capacity at constant pressure,  $C_P[kJ/(kg \cdot K)]$ , of an ideal gas is related to the enthalpy by

$$C_P = \frac{dh}{dT}$$

where  $h$  = specific enthalpy (kJ/kg) and  $T$  = absolute temperature (K). The following enthalpy data are provided for carbon dioxide, CO<sub>2</sub>, at several temperatures. Use these to determine the heat capacity in kJ/(kg·K) for each of the temperatures. Note that the atomic weights of carbon and oxygen are 12.011 and 15.994 kg/kmol, respectively.

|              |        |        |        |        |
|--------------|--------|--------|--------|--------|
| $T, K$       | 750    | 800    | 900    | 1000   |
| $h, kJ/kmol$ | 29,629 | 32,179 | 37,405 | 42,769 |

**21.33** An  $n$ th-order rate law is often used to model chemical reactions that solely depend on the concentration of a single reactant:

$$\frac{dc}{dt} = -kc^n$$

where  $c$  = concentration (mol/L),  $t$  = time (min),  $n$  = reaction order (dimensionless), and  $k$  = reaction rate  $\left[\text{min}^{-1} \cdot (\text{mol/L})^{1-n}\right]$ . The differential method can be used to evaluate the parameters  $k$  and  $n$ . This involves applying a logarithmic transformation to the rate equation to obtain

$$\log\left(-\frac{dc}{dt}\right) = \log(k) + n \log(c)$$

Note: Either the common logarithm ( $\log_{10}$ ) or natural logarithm ( $\ln$ ) can be used above. If the rate expression is valid, a plot of

$$\log\left(-\frac{dc}{dt}\right) \text{ vs. } \log(c)$$

should yield a straight line with a slope  $n$  and intercept  $\log(k)$ . Use the differential method and straight-line regression to determine  $k$  and  $n$  for the following data for the conversion of ammonium cyanate to urea:

|                        |       |       |       |       |       |
|------------------------|-------|-------|-------|-------|-------|
| <b><i>t</i>, min</b>   | 0     | 5     | 15    | 30    | 45    |
| <b><i>c</i>, mol/L</b> | 0.750 | 0.594 | 0.420 | 0.291 | 0.223 |

**21.34** The sediment oxygen demand, *SOD* in units of g/(m<sup>2</sup>·d), is an important parameter in determining the dissolved oxygen content of a natural water. It is measured by placing a sediment core in a cylindrical container ( **Fig. P21.34**). After carefully introducing a layer of distilled, oxygenated water above the sediment, the container is covered to prevent gas transfer. A stirrer is used to mix the water layer gently, and an oxygen probe tracks how the water’s oxygen concentration decreases over time. The *SOD* can then be computed as

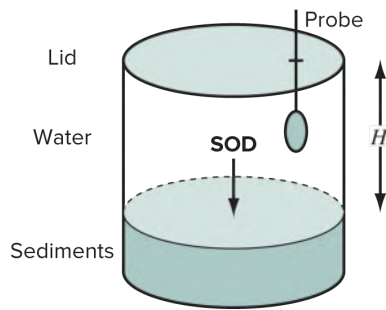
$$\text{SOD} = -H \frac{do}{dt}$$

where  $H$  = the depth of the water layer (m),  $o$  = the oxygen concentration (g/m<sup>3</sup>), and  $t$  = time (d). Based on the following data and  $H = 0.1$  m, use numerical differentiation to generate plots of (a) *SOD* versus time, and (b) *SOD* versus oxygen concentration:

|                       |    |       |      |       |      |       |      |
|-----------------------|----|-------|------|-------|------|-------|------|
| <b><i>t</i>, d</b>    | 0  | 0.125 | 0.25 | 0.375 | 0.5  | 0.625 | 0.75 |
| <b><i>o</i>, mg/L</b> | 10 | 7.11  | 4.59 | 2.57  | 1.15 | 0.33  | 0.03 |



**FIGURE P21.34**



**21.35** The following relationships can be used to analyze uniform beams subject to distributed loads:

$$\frac{dy}{dx} = \theta(x) \quad \frac{d\theta}{dx} = \frac{M(x)}{EI} \quad \frac{dM}{dx} = V(x) \quad \frac{dV}{dx} = -u(x)$$

where  $x$  = distance along the beam (m),  $y$  = deflection (m),  $\theta(x)$  = slope(m/m),  $E$  = modulus of elasticity (Pa),  $I$  = moment of inertia ( $\text{m}^4$ ),  $M(x)$  = moment ( $\text{N}\cdot\text{m}$ ),  $V(x)$  = shear (N), and  $u(x)$  = distributed load ( $\text{N/m}$ ). For the case of a linearly increasing load (recall P5.15), the slope can be computed analytically to be

$$\theta(x) = \frac{w_0}{120 EIL} (-5x^4 + 6 \cdot L^2 x^2 - L^4) \quad (\text{P21.35})$$

Employ (a) numerical integration to compute the deflection,  $y$ , and (b) numerical differentiation to compute the moment,  $M$ , and  $V$ . Produce plots versus distance. Base your numerical values on the slope computed with Eq. (P21.35) at equally spaced intervals of  $\Delta x = 0.125$  m along a 3-m beam. Use the following parameter values in your computation:  $E = 200$  GPa,  $I = 0.0003$   $\text{m}^4$ , and  $w_0 = 2.5$  kN/cm. In addition, the deflections at the ends of the beam are set at  $y(0) = y(L) = 0$ . Take care with units.

**21.36** You measure the following deflections along the length of a simply supported uniform beam (see Prob. 21.35).

|                           |         |         |         |         |         |
|---------------------------|---------|---------|---------|---------|---------|
| <b><math>x</math>, m</b>  | 0       | 0.375   | 0.75    | 1.125   | 1.5     |
| <b><math>y</math>, cm</b> | 0       | -0.2571 | -0.9484 | -1.9689 | -3.2262 |
| <b><math>x</math>, m</b>  | 1.875   | 2.25    | 2.625   | 3       |         |
| <b><math>y</math>, cm</b> | -4.6414 | -6.1503 | -7.7051 | -9.275  |         |

---

Employ numerical differentiation to compute the slope, the moment (N·m), the shear (N), and the distributed load (N/m). Use the following parameters in your computation:  $E = 200$  GPa and  $I = 0.0003$  m<sup>4</sup>. Produce plots of these quantities versus distance,  $x$ .

**21.37** Evaluate  $\partial f/\partial x$ ,  $\partial f/\partial y$ , and  $\partial^2 f/(\partial x \partial y)$  for the following function at  $x = y = 1$  (a) analytically and (b) numerically with  $\Delta x = \Delta y = 0.0001$ ,

$$f(x, y) = 3xy + 3x - x^3 - 3y^3$$

**21.38** Develop a Python script to generate the same computations and plots as in [Sec. 21.8](#) but for the following functions and  $-3 \leq x \leq 3$  and  $-3 \leq y \leq 3$ :

(a)  $f(x, y) = e^{-(x^2+y^2)}$

(b)  $f(x, y) = xe^{-(x^2+y^2)}$

**21.39** The velocity,  $v$ (m/s), of an object at time,  $t$ (s), is given by

$$v = \frac{2t}{\sqrt{1+t^2}}$$

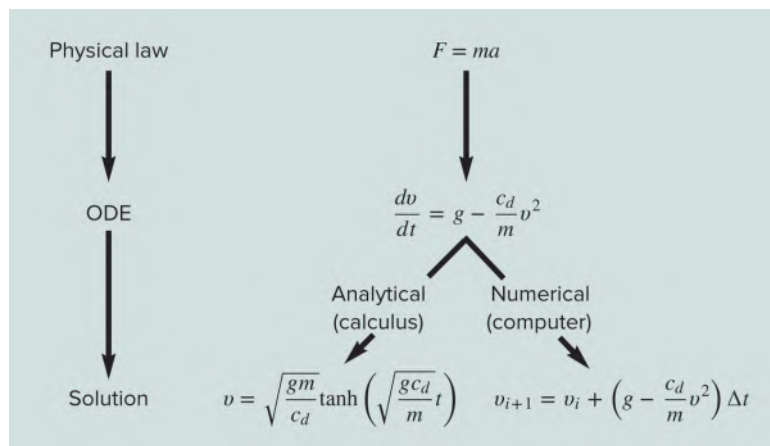
Using Richardson's extrapolation, find the acceleration of the object at  $t = 5$  s using  $h = 0.5$  and  $0.25$ . Employ the exact solution to compute the true percent relative error of each estimate.

## Ordinary Differential Equations

### 6.1 OVERVIEW

The fundamental laws of physics, mechanics, electricity, and thermodynamics are usually based on empirical observations that explain variations in physical properties and states of systems. Rather than describing the state of physical systems directly, the laws are usually couched in terms of spatial and temporal changes. These laws define mechanisms of change. When combined with continuity laws for energy, mass, or momentum, differential equations result. Subsequent integration of these differential equations results in mathematical functions that describe the spatial and temporal state of a system in terms of energy, mass, or velocity variations. As in  **Fig. PT6.1**, the integration can be implemented analytically with calculus or numerically with the computer.

**FIGURE PT6.1** The sequence of events in the development and solution of ODEs for engineering and science. The example shown is for the velocity of the free-falling bungee jumper.

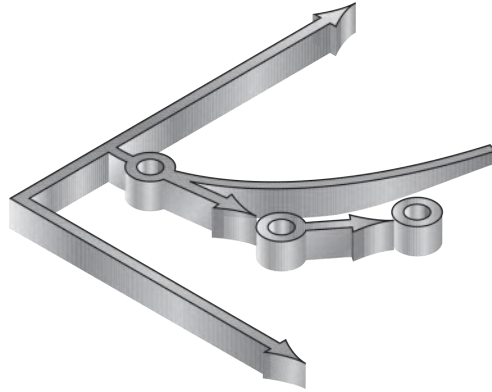


The free-falling bungee jumper problem introduced in  **Chap. 1** is an example of the derivation of a differential equation from a fundamental law. Recall that Newton's second law was used to develop an ODE describing the rate of change of velocity of a falling bungee jumper:

$$\frac{dv}{dt} = g - \frac{c_d}{m}v^2 \quad (\text{PT6.1})$$

where  $g$  is the gravitational constant,  $m$  is the mass, and  $c_d$  is a drag coefficient. Such equations, which are composed of an unknown function and its derivatives, are called *differential equations*. They are sometimes referred to as *rate equations* because they express the rate of change of a variable as a function of variables and parameters.

In [Eq. \(PT6.1\)](#), the quantity being differentiated  $v$  is called the *dependent variable*. The quantity with respect to which  $v$  is differentiated  $t$  is called the *independent variable*. When the function involves one independent variable, the equation is called an *ordinary differential equation* (or *ODE*). This is in contrast to a *partial differential equation* (or *PDE*) that involves two or more independent variables.



Differential equations are also classified as to their *order*. For example, [Eq. \(PT6.1\)](#) is called a *first-order equation* because the highest derivative is a first derivative. A *second-order equation* would include a second derivative. For example, the equation describing the position  $x$  of an unforced mass-spring system with damping is the second-order equation:

$$m \frac{d^2 x}{dt^2} + c \frac{dx}{dt} + kx = 0 \quad (\text{PT6.2})$$

where  $m$  is mass,  $c$  is a damping coefficient, and  $k$  is a spring constant. Similarly, an  $n$ th-order equation would include an  $n$ th derivative.

Higher-order differential equations can be reduced to a system of first-order equations. This is accomplished by defining the first derivative of the dependent variable as a new variable. For

[Eq. \(PT6.2\)](#), this is done by creating a new variable  $v$  as the first derivative of displacement

$$v = \frac{dx}{dt} \quad (\text{PT6.3})$$

where  $v$  is velocity. This equation can itself be differentiated to yield

$$(\text{PT6.4})$$

$$\frac{dv}{dt} = \frac{d^2x}{dt^2}$$

Equations (PT6.3) and (PT6.4) can be substituted into Eq. (PT6.2) to convert it into a first-order equation:

$$m \frac{dv}{dt} + cv + kx = 0 \quad (\text{PT6.5})$$

As a final step, we can express Eqs. (PT6.3) and (PT6.5) as rate equations:

$$\frac{dx}{dt} = v \quad (\text{PT6.6})$$

$$\frac{dv}{dt} = -\frac{c}{m}v - \frac{k}{m}x \quad (\text{PT6.7})$$

Thus, Eqs. (PT6.6) and (PT6.7) are a pair of first-order equations that are equivalent to the original second-order equation [Eq. (PT6.2)]. Because other  $n$ th-order differential equations can be similarly reduced, this part of our book focuses on the solution of first-order equations.

A solution of an ordinary differential equation is a specific function of the independent variable and parameters that satisfies the original differential equation. To illustrate this concept, let us start with a simple fourth-order polynomial,

$$y = -0.5x^4 + 4x^3 - 10x^2 + 8.5x + 1 \quad (\text{PT6.8})$$

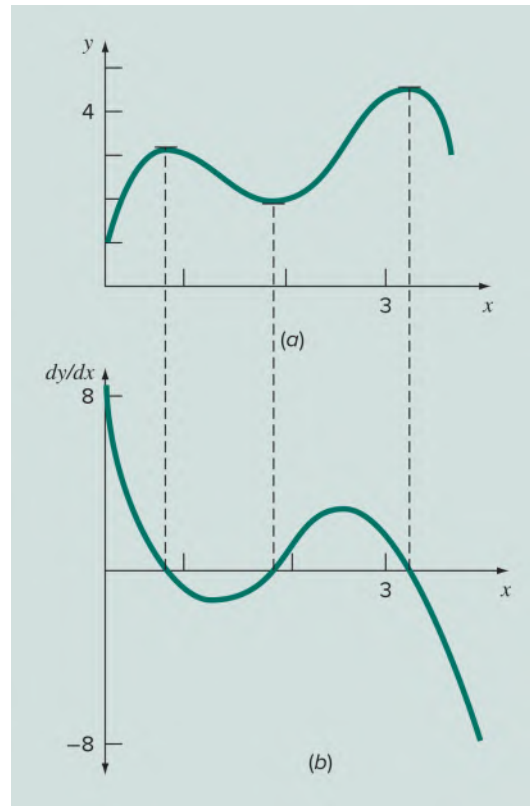
Now, if we differentiate Eq. (PT6.8), we obtain an ODE:

$$\frac{dy}{dx} = -2x^3 + 12x^2 - 20x + 8.5 \quad (\text{PT6.9})$$

This equation also describes the behavior of the polynomial, but in a manner different from Eq. (PT6.8). Rather than explicitly representing the values of  $y$  for each value of  $x$ , Eq. (PT6.9) gives the rate of change of  $y$  with respect to  $x$  (i.e., the slope) at every value of  $x$ . Figure PT6.2 shows both the function and the derivative plotted versus  $x$ . Notice how the zero values of the derivatives correspond to the point at which the original function is flat—that is, where it has a zero slope. Also, the

maximum absolute values of the derivatives are at the ends of the interval where the slopes of the function are greatest.

**FIGURE PT6.2** Plots of (a)  $y$  versus  $x$  and (b)  $dy/dx$  versus  $x$  for the function  $y = -0.5x^4 + 4x^3 - 10x^2 + 8.5x + 1$ .



Although, as just demonstrated, we can determine a differential equation given the original function, the object here is to determine the original function given the differential equation. The original function then represents the solution.

Without computers, ODEs are usually solved analytically with calculus. For example, [Eq. \(PT6.9\)](#) could be multiplied by  $dx$  and integrated to yield

$$y = \int (-2x^3 + 12x^2 - 20x + 8.5) dx \quad (\text{PT6.10})$$

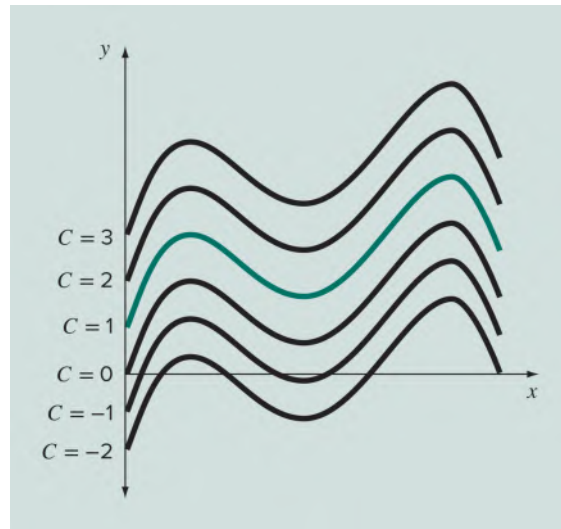
The right-hand side of this equation is called an *indefinite integral* because the limits of integration are unspecified. This is in contrast to the *definite integrals* discussed previously in Part Five [compare [Eq. \(PT6.10\)](#) with [Eq. \(19.5\)](#)].

An analytical solution for [Eq. \(PT6.10\)](#) is obtained if the indefinite integral can be evaluated exactly in equation form. For this simple case, it is possible to do this with the result:

$$y = -0.5x^4 + 4x^3 - 10x^2 + 8.5x + C \quad (\text{PT6.11})$$

which is identical to the original function with one notable exception. In the course of differentiating and then integrating, we lost the constant value of 1 in the original equation and gained the value  $C$ . This  $C$  is called a *constant of integration*. The fact that such an arbitrary constant appears indicates that the solution is not unique. In fact, it is but one of an infinite number of possible functions (corresponding to an infinite number of possible values of  $C$ ) that satisfy the differential equation. For example, [Fig. PT6.3](#) shows six possible functions that satisfy [Eq. \(PT6.11\)](#).

**FIGURE PT6.3** Six possible solutions for the integral of  $-2x^3 + 12x^2 - 20x + 8.5$ . Each conforms to a different value of the constant of integration  $C$ .



Therefore, to specify the solution completely, a differential equation is usually accompanied by auxiliary conditions. For first-order ODEs, a type of auxiliary condition called an initial value is required to determine the constant and obtain a unique solution. For example, the original differential equation could be accompanied by the initial condition that at  $x = 0$ ,  $y = 1$ . These values could be substituted into [Eq. \(PT6.11\)](#) to determine  $C = 1$ . Therefore, the unique solution that satisfies both the differential equation and the specified initial condition is

$$y = -0.5x^4 + 4x^3 - 10x^2 + 8.5x + 1$$

Thus, we have “pinned down” [Eq. \(PT6.11\)](#) by forcing it to pass through the initial condition, and in so doing, we have developed a unique solution to the ODE and have come full circle to the original function [[Eq. \(PT6.8\)](#)].

Initial conditions usually have very tangible interpretations for differential equations derived from physical problem settings. For example, in the bungee jumper problem, the initial condition was reflective of the physical fact that at time zero the vertical velocity was zero. If the bungee jumper had already been in vertical motion at time zero, the solution would have been modified to account for this initial velocity.

When dealing with an  $n$ th-order differential equation,  $n$  conditions are required to obtain a unique solution. If all conditions are specified at the same value of the independent variable (e.g., at  $x$  or  $t = 0$ ), then the problem is called an *initial-value problem*. This is in contrast to *boundary-value problems* where specification of conditions occurs at different values of the independent variable. [Chapters 22](#) and [23](#) will focus on initial-value problems. Boundary-value problems are covered in [Chap. 24](#).



## 6.2 PART ORGANIZATION

---

 **Chapter 22** is devoted to one-step methods for solving initial-value ODEs. As the name suggests, *one-step methods* compute a future prediction  $y_{i+1}$ , based only on information at a single point  $y_i$  and no other previous information. This is in contrast to *multistep approaches* that use information from several previous points as the basis for extrapolating to a new value.

With all but a minor exception, the one-step methods presented in  **Chap. 22** belong to what are called *Runge-Kutta techniques*. Although the chapter might have been organized around this theoretical notion, we have opted for a more graphical, intuitive approach to introduce the methods. Thus, we begin the chapter with *Euler's method*, which has a very straightforward graphical interpretation. In addition, because we have already introduced Euler's method in  **Chap. 1**, our emphasis here is on quantifying its truncation error and describing its stability.

Next, we use visually oriented arguments to develop two improved versions of Euler's method—the *Heun* and the *midpoint* techniques. After this introduction, we formally develop the concept of Runge-Kutta (or RK) approaches and demonstrate how the foregoing techniques are actually first- and second-order RK methods. This is followed by a discussion of the higher-order RK formulations that are frequently used for engineering and scientific problem solving. In addition, we cover the application of one-step methods to *systems of ODEs*. Note that all the applications in  **Chap. 22** are limited to cases with a fixed step size.

In  **Chap. 23**, we cover more advanced approaches for solving initial-value problems. First, 

Page 531

 we describe *adaptive RK methods* that automatically adjust the step size in response to the truncation error of the computation. These methods are especially pertinent as they are employed by Python to solve ODEs.

Next, we discuss *multistep methods*. As mentioned above, these algorithms retain information of previous steps to more effectively capture the trajectory of the solution. They also yield the truncation error estimates that can be used to implement step-size control. We describe a simple method—the *non-self-starting Heun* method—to introduce the essential features of the multistep approaches.

Finally, the chapter ends with a description of *stiff ODEs*. These are both individual and systems of ODEs that have both fast and slow components to their solution. As a consequence, they require special solution approaches. We introduce the idea of an *implicit solution* technique as one commonly used remedy. We also describe Python's built-in functions for solving stiff ODEs.

In  **Chap. 24**, we focus on two approaches for obtaining solutions to *boundary-value problems*: the *shooting* and *finite-difference methods*. Aside from demonstrating how these techniques are implemented, we illustrate how they handle *derivative boundary conditions* and *nonlinear ODEs*.

# Initial-Value Problems

## CHAPTER OBJECTIVES

The primary objective of this chapter is to introduce you to solving initial-value problems for ordinary differential equations (ODEs). Specific objectives and topics covered are

- Understanding the meaning of local and global truncation errors and their relationship to step size for one-step methods for solving ODEs.
- Knowing how to implement the following Runge-Kutta (RK) methods for a single ODE:
  - Euler
  - Heun
  - Midpoint
  - Fourth-order RK
- Knowing how to iterate the corrector of Heun's method.
- Knowing how to implement the following Runge-Kutta methods for systems of ODEs:
  - Euler
  - Fourth-order RK

## YOU'VE GOT A PROBLEM

We started this book with the problem of simulating the velocity of a free-falling bungee jumper. This problem amounted to formulating and solving an ordinary differential equation, the topic of this chapter. Now let's return to this problem and make it more interesting by computing what happens when the jumper reaches the end of the bungee cord.

To do this, we should recognize that the jumper will experience different forces depending on whether the cord is slack or stretched. If it is slack, the situation is that of free fall where the only forces are gravity and drag. However, because the jumper can now move up as well as down, the sign of the drag force must be modified so that it always tends to retard velocity,

$$\frac{dv}{dt} = g - \text{sign}(v) \frac{c_d}{m} v^2 \quad (22.1a)$$

where  $v$  is velocity (m/s),  $t$  is time (s),  $g$  is the acceleration due to gravity ( $9.81 \text{ m/s}^2$ ),  $c_d$  is the drag coefficient (kg/m), and  $m$  is mass (kg). The *signum function*,<sup>1</sup>  $\text{sign}$ , returns a  $-1$  or a  $1$ , depending on whether its argument is negative or positive, respectively. Thus, when the jumper is falling downward (positive velocity,  $\text{sign} = 1$ ), the drag force will be negative and hence will act to reduce velocity. In contrast, when the jumper is moving upward (negative velocity,  $\text{sign} = -1$ ), the drag force will be positive so that it again reduces the velocity.

Once the cord begins to stretch, it obviously exerts an upward force on the jumper. As done previously in [Chap. 8](#), Hooke's law can be used as a first approximation of this force. In addition, a dampening force should also be included to account for frictional effects as the cord stretches and contracts. These factors can be incorporated along with gravity and drag into a second force balance that applies when the cord is stretched. The result is the following differential equation:

$$\frac{dv}{dt} = g - \text{sign}(v) \frac{c_d}{m} v^2 - \frac{k}{m} (x - L) - \frac{\gamma}{m} v \quad (22.1b)$$

where  $k$  is the cord's spring constant (N/m),  $x$  is vertical distance measured downward from the bungee jump platform (m),  $L$  is the length of the unstretched cord (m), and  $\gamma$  is a dampening coefficient (N·s/m).

Because [Eq. \(22.1b\)](#) only holds when the cord is stretched ( $x > L$ ), the spring force will always be negative. That is, it will always act to pull the jumper back up. The dampening force increases in magnitude as the jumper's velocity increases and always acts to slow the jumper down.

If we want to simulate the jumper's velocity, we would initially solve [Eq. \(22.1a\)](#) until the cord was fully extended. Then, we could switch to [Eq. \(22.1b\)](#) for periods that the cord is stretched. Although this is fairly straightforward, it means that knowledge of the jumper's position is required. This can be done by formulating another differential equation for distance:

$$\frac{dx}{dt} = v \quad (22.2)$$

Thus, solving for the bungee jumper's velocity amounts to solving two ordinary differential equations where one of the equations takes different forms depending on the value of one of the dependent variables.

[Chapters 22](#) and [23](#) explore methods for solving this and similar problems involving ODEs.

## 22.1 OVERVIEW

---

This chapter is devoted to solving ordinary differential equations of the form

$$\frac{dy}{dt} = f(t, y) \quad (22.3)$$

In [Chap. 1](#), we developed a numerical method to solve such an equation for the velocity of the free-falling bungee jumper. Recall that the method was of the general form

$$\text{New value} = \text{old value} + \text{slope} \times \text{step size}$$

or, in mathematical terms,

$$y_{i+1} = y_i + \phi h \quad (22.4)$$

where the slope  $\phi$  is called an *increment function*. According to this equation, the slope estimate of  $\phi$  is used to extrapolate from an old value  $y_i$  to a new value  $y_{i+1}$  over a distance  $h$ . This formula can be applied step by step to trace out the trajectory of the solution into the future. Such approaches are called *one-step methods* because the value of the increment function is based on information at a single point  $i$ . They are also referred to as *Runge-Kutta methods* after the two applied mathematicians who first discussed them in the early 1900s. Another class of methods called *multistep methods* uses information from several previous points as the basis for extrapolating to a new value. We will describe multistep methods briefly in [Chap. 23](#).

All one-step methods can be expressed in the general form of [Eq. \(22.4\)](#), with the only difference being the manner in which the slope is estimated. The simplest approach is to use the differential equation to estimate the slope in the form of the first derivative at  $t_i$ . In other words, the slope at the beginning of the interval is taken as an approximation of the average slope over the whole interval. This approach, called Euler's method, is discussed next. This is followed by other one-step methods that employ alternative slope estimates that result in more accurate predictions.

## 22.2 EULER'S METHOD

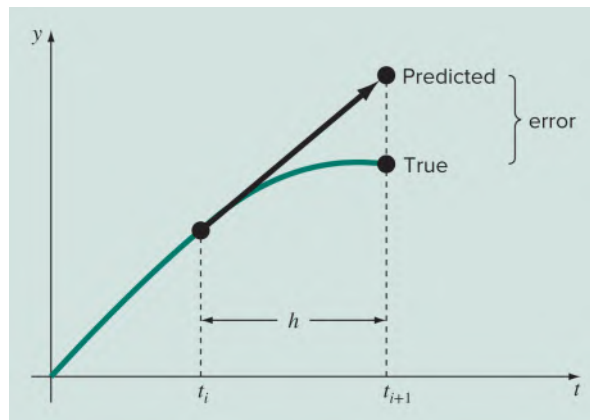
The first derivative provides a direct estimate of the slope at  $t_i$  (Fig. 22.1):

$$\phi = f(t_i, y_i)$$

where  $f(t_i, y_i)$  is the differential equation evaluated at  $t_i$  and  $y_i$ . This estimate can be substituted into Eq. (22.1):

$$y_{i+1} = y_i + f(t_i, y_i)h \quad (22.5)$$

**FIGURE 22.1** Euler's method.



This formula is referred to as *Euler's method* (or the Euler-Cauchy or point-slope method). A new value of  $y$  is predicted using the slope (equal to the first derivative at the original value of  $t$ ) to extrapolate linearly over the step size  $h$  (Fig. 22.1).

### EXAMPLE 22.1

#### Euler's Method

#### Problem Statement

Use Euler's method to integrate  $y' = 4e^{0.8t} - 0.5y$  from  $t = 0$  to 4 with a step size of 1. The initial condition at  $t = 0$  is  $y = 2$ . Note that the exact solution can be determined analytically as

$$y = \frac{4}{1.3} (e^{0.8t} - e^{-0.5t}) + 2e^{-0.5t}$$

## Solution

 Equation (22.5) can be used to implement Euler's method:

$$y(1) = y(0) + f(0, 2)(1)$$

where  $y(0) = 2$  and the slope estimate at  $t = 0$  is

$$f(0, 2) = 4e^0 - 0.5(2) = 3$$

Therefore,

$$y(1) = 2 + 3(1) = 5$$

The true solution at  $t = 1$  is

$$y = \frac{4}{1.3} (e^{0.8(1)} - e^{-0.5(1)}) + 2e^{-0.5(1)} = 6.19463$$

Thus, the percent relative error is

$$\varepsilon_t = \left| \frac{6.19463 - 5}{6.19463} \right| \times 100\% = 19.28\%$$

For the second step:

Page 535

$$\begin{aligned} y(2) &= y(1) + f(1, 5)(1) \\ &= 5 + [4e^{0.8(1)} - 0.5(5)](1) = 11.40216 \end{aligned}$$

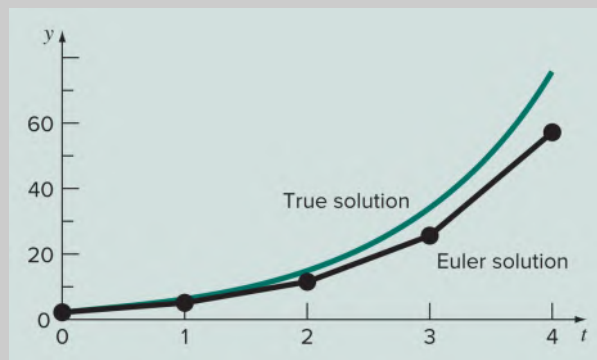
The true solution at  $t = 2.0$  is 14.84392 and, therefore, the true percent relative error is 23.19%.

The computation is repeated, and the results are compiled in  Table 22.1 and  Fig. 22.2. Note that although the computation captures the general trend of the true solution, the error is considerable. As discussed in the next section, this error can be reduced by using a smaller step size.

**TABLE 22.1** Comparison of true and numerical values of the integral of  $y' = 4e^{0.8t} - 0.5y$ , with the initial condition that  $y = 2$  at  $t = 0$ . The numerical values were computed using Euler's method with a step size of 1.

| $t$ | $y_{\text{true}}$ | $y_{\text{Euler}}$ | $ \epsilon_t (\%)$ |
|-----|-------------------|--------------------|--------------------|
| 0   | 2.00000           | 2.00000            |                    |
| 1   | 6.19463           | 5.00000            | 19.28              |
| 2   | 14.84392          | 11.40216           | 23.19              |
| 3   | 33.67717          | 25.51321           | 24.24              |
| 4   | 75.33896          | 56.84931           | 24.54              |

**FIGURE 22.2** Comparison of the true solution with a numerical solution using Euler's method for the integral of  $y' = 4e^{0.8t} - 0.5y$  from  $t = 0$  to 4 with a step size of 1.0. The initial condition at  $t = 0$  is  $y = 2$ .



## 22.2.1 Error Analysis for Euler's Method

The numerical solution of ODEs involves two types of error (recall [Chap. 4](#)):

1. *Truncation*, or discretization, errors caused by the nature of the techniques employed to approximate values of  $y$ .
2. *Roundoff* errors caused by the limited numbers of significant digits that can be retained by a computer.

The truncation errors are composed of two parts. The first is a *local truncation error* that results from an application of the method in question over a single step. The second is a *propagated truncation error* that results from the approximations produced during the previous steps. The sum of the two is the total error. It is referred to as the *global truncation error*.

Insight into the magnitude and properties of the truncation error can be gained by deriving Euler's method directly from the Taylor series expansion. To do this, realize that the differential equation being integrated will be of the general form of [Eq. \(22.3\)](#), where  $dy/dt = y'$ , and  $t$  and  $y$  are the independent and dependent variables, respectively. If the solution—that is, the function describing the behavior of  $y$ —has continuous derivatives, it can be represented by a Taylor series expansion about a starting value  $(t_i, y_i)$ , as in [recall [Eq. \(4.13\)](#)]:

$$y_{i+1} = y_i + y'_i h + \frac{y''_i}{2!} h^2 + \cdots + \frac{y^{(n)}_i}{n!} h^n + R_n \quad (22.6)$$

where  $h = t_{i+1} - t_i$  and  $R_n$  = the remainder term, defined as

$$R_n = \frac{y^{(n+1)}(\xi)}{(n+1)!} h^{n+1} \quad (22.7)$$

where  $\xi$  lies somewhere in the interval from  $t_i$  to  $t_{i+1}$ . An alternative form can be developed by substituting [Eq. \(22.3\)](#) into [Eqs. \(22.6\)](#) and [\(22.7\)](#) to yield

$$y_{i+1} = y_i + f(t_i, y_i)h + \frac{f'(t_i, y_i)}{2!} h^2 + \cdots + \frac{f^{(n-1)}(t_i, y_i)}{n!} h^n + O(h^{n+1}) \quad (22.8)$$

where  $O(h^{n+1})$  specifies that the local truncation error is proportional to the step size raised to the  $(n+1)$ th power.

By comparing [Eqs. \(22.5\)](#) and [\(22.8\)](#), it can be seen that Euler's method corresponds to the Taylor series up to and including the term  $f(t_i, y_i)h$ . Additionally, the comparison indicates that a truncation error occurs because we approximate the true solution using a finite number of terms from the Taylor series. We thus truncate, or leave out, a part of the true solution. For example, the truncation error in Euler's method is attributable to the remaining terms in the Taylor series expansion that were not included in [Eq. \(22.5\)](#). Subtracting [Eq. \(22.5\)](#) from [Eq. \(22.8\)](#) yields

$$E_t = \frac{f'(t_i, y_i)}{2!} h^2 + \cdots + O(h^{n+1}) \quad (22.9)$$



where  $E_t$  = the true local truncation error. For sufficiently small  $h$ , the higher-order terms in [Eq. \(22.9\)](#) are usually negligible, and the result is often represented as

$$E_a = \frac{f''(t_i, y_i)}{2!} h^2 \quad (22.10)$$

or

$$E_a = O(h^2) \quad (22.11)$$

where  $E_a$  = the approximate local truncation error.

According to [Eq. \(22.11\)](#), we see that the local error is proportional to the square of the step size and the first derivative of the differential equation. It can also be demonstrated that the global truncation error is  $O(h)$ —that is, it is proportional to the step size (Carnahan et al., 1969). These observations lead to some useful conclusions:

1. The global error can be reduced by decreasing the step size.
2. The method will provide error-free predictions if the underlying function (i.e., the solution of the differential equation) is linear, because for a straight line the second derivative would be zero.

This latter conclusion makes intuitive sense because Euler's method uses straight-line segments to approximate the solution. Hence, Euler's method is referred to as a *first-order method*.

It should also be noted that this general pattern holds for the higher-order one-step methods described in the following pages. That is, an  $n$ th-order method will yield perfect results if the underlying solution is an  $n$ th-order polynomial. Further, the local truncation error will be  $O(h^{n+1})$  and the global error  $O(h^n)$ .

## 22.2.2 Stability of Euler's Method

In the preceding section, we learned that the truncation error of Euler's method depends on the step size in a predictable way based on the Taylor series. This is an accuracy issue.

The stability of a solution method is another important consideration that must be considered when solving ODEs. A numerical solution is said to be unstable if errors grow exponentially for a problem for which there is a bounded solution. The stability of a particular application can depend on three factors: the differential equation, the numerical method, and the step size.

Insight into the step size required for stability can be examined by studying a very simple ODE: Page 537

$$\frac{dy}{dt} = -ay \quad (22.12)$$

If  $y(0) = y_0$ , calculus can be used to determine the solution as

$$y = y_0 e^{-at}$$

Thus, the solution starts at  $y_0$  and asymptotically approaches zero.

Now suppose that we use Euler's method to solve the same problem numerically:

$$y_{i+1} = y_i + \frac{dy_i}{dt} h$$

Substituting  Eq. (22.12) gives

$$y_{i+1} = y_i - ay_i h$$

or

$$y_{i+1} = y_i(1 - ah) \quad (22.13)$$

The parenthetical quantity  $1 - ah$  is called an *amplification factor*. If its absolute value is greater than unity, the solution will grow in an unbounded fashion. So clearly, the stability depends on the step size  $h$ . That is, if  $h > 2/a$ ,  $|y_i| \rightarrow \infty$  as  $i \rightarrow \infty$ . Based on this analysis, Euler's method is said to be *conditionally stable*.

Note that there are certain ODEs where errors always grow regardless of the method. Such ODEs are called *ill-conditioned*.

Inaccuracy and instability are often confused. This is probably because (a) both represent situations where the numerical solution breaks down and (b) both are affected by step size. However, they are distinct problems. For example, an inaccurate method can be very stable. We will return to the topic when we discuss stiff systems in  Chap. 23.

## 22.2.3 Python Function: `eulode`

We have already developed a simple Python script to implement Euler's method for the falling bungee jumper problem in  Chap. 3. Recall from  Sec. 3.6 that this function used Euler's method to compute the velocity after a given time of free fall. Now, let's develop a more general, all-purpose algorithm.

 **Figure 22.3** shows a Python function, `eulode`, that uses Euler's method to compute values of the dependent variable,  $y$ , over a range of values of the independent variable,  $t$ . The name of the function that evaluates the right-hand side of the differential equation, `dydt`, is passed to the `eulode` function. The initial and final values of the independent variable are passed as an array, `tspan`. The initial value and the

desired step size are passed as `y0` and `h`, respectively. The `eulode` function also allows for additional arguments, `*args`, that can be passed through to the `dydt` function.

**FIGURE 22.3** A Python function to implement Euler's method.

```
import numpy as np

def eulode(dydt,tspan,y0,h,*args):
    """
    solve initial-value single ODEs with the Euler method
    input:
        dydt = function name that evaluates the derivative
        tspan = array of [ti,tf] where
            ti and tf are the initial and final values
            of the independent variable
        y0 = initial value of the dependent variable
        h = step size
        *args = additional argument to be passed to dydt
    output:
        t = an array of independent variable values
        y = an array of dependent variable values
    """
    ti = tspan[0] ; tf = tspan[1]
    if not(tf>ti+h): return 'upper limit must be greater than lower limit'
    t = []
    t.append(ti) # start the t array with ti
    nsteps = int((tf-ti)/h)
    for i in range(nsteps): # add the rest of the t values
        t.append(ti+(i+1)*h)
    n = len(t)
    if t[n-1] < tf: # check if t array is short of tf
        t.append(tf)
        n = n+1
    y = np.zeros(n) ; y[0] = y0 # initialize y array
    for i in range(n-1):
        y[i+1] = y[i] + dydt(t[i],y[i],*args)*(t[i+1]-t[i]) # Euler step
    return t,y
```

The function first generates an array `t` over the desired range of the independent variable using an increment `h`. In the event the step size is not evenly divisible into the range, the last value will fall short of the final value of the range. If this occurs, the final value is appended to the `t` array so that the series spans the entire range. In addition, an array of the dependent variable, `y`, is created and its first element is set to `y0`.

At this point, the Euler method [[Eq. \(22.5\)](#)] is implemented by a simple loop:

```
for i in range(n-1):
    y[i+1] = y[i] + dydt(t[i],y[i],*args)*h
```

Notice how the function `dydt` is used to generate a value for the derivative at the appropriate values of the independent and dependent variables. Also notice how the time step is automatically calculated based on the difference in the adjacent values in the `t` array. It is important not to use `h` here because of the possible adjustment to the final interval.

An ODE being solved can be set up in two ways. First, the differential equation can be defined via a `lambda` anonymous function. For example, for the ODE from [Example 22.1](#):

```
for i in range(n-1):
    y[i+1] = y[i] + dydt(t[i],y[i],*args)*(t[i+1]-t[i])
    dydt = lambda t,y,a,b,c: b*np.exp(a*t) - c*y
```

where `a`, `b`, and `c` are additional arguments that will have to be passed through `eulode` to `dydt`.

The solution can then be generated with the script

```
tspan = np.array([0.,4.])
y0 = 2.
h = 1.0
a = 0.8
b = 4.
c = 0.5
t,y = eulode(dydt,tspan,y0,h,a,b,c)
n = len(t)
for i in range(n):
    print('{0:4.1f} {1:7.4f}'.format(t[i],y[i]))
```

with the result

```
0.0  2.0000
1.0  5.0000
2.0 11.4022
3.0 25.5132
4.0 56.8493
```

We can also check the situation where `h` doesn't provide intervals that span exactly `tf-ti`:

```
h = 0.9
0.0  2.0000
0.9  4.7000
1.8  9.9810
2.7 20.6840
3.6 42.5923
4.0 62.5767
```

Although using the `lambda` function is feasible for the present case, there will be more complex problems where the definition of the ODE requires several lines of code. In such instances, creating a `dydt` function with `def` will be required.

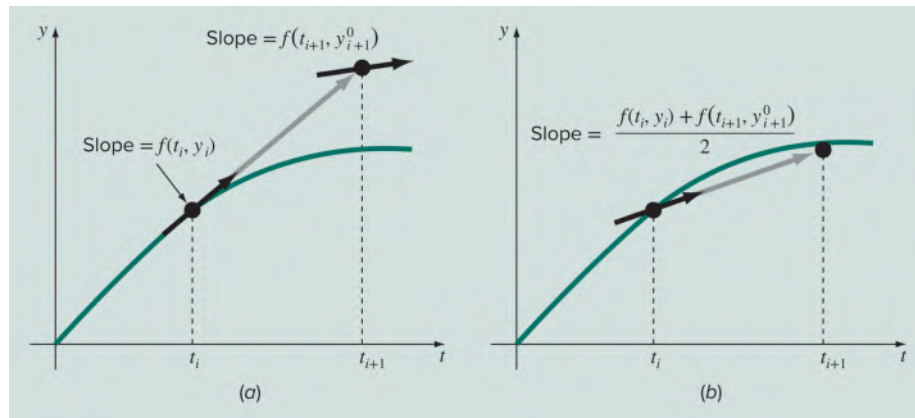
## 22.3 IMPROVEMENTS OF EULER'S METHOD

A fundamental source of error in Euler's method is that the derivative at the beginning of the interval is assumed to apply across the entire interval. Two simple modifications are available to help circumvent this shortcoming. As will be demonstrated in [Sec. 22.4](#), both modifications (as well as Euler's method itself) actually belong to a larger class of solution techniques called Runge-Kutta methods. However, because they have very straightforward graphical interpretations, we will present them prior to their formal derivation as Runge-Kutta methods.

### 22.3.1 Heun's Method

One method to improve the estimate of the slope involves the determination of two derivatives for the interval—one at the beginning and another at the end. The two derivatives are then averaged to obtain an improved estimate of the slope for the entire interval. This approach, called *Heun's method*, is depicted graphically in [Fig. 22.4](#).

**FIGURE 22.4** Graphical depiction of Heun's method. (a) Predictor and (b) corrector.



Recall that in Euler's method, the slope at the beginning of an interval

$$y_i^- = f(t_i, y_i) \quad (22.14)$$

is used to extrapolate linearly to  $y_{i+1}$ :

$$y_{i+1}^0 = y_i + f(t_i, y_i)h \quad (22.15)$$

For the standard Euler method, we would stop at this point. However, in Heun's method the  $y_{i+1}^0$  calculated in [Eq. \(22.15\)](#) is not the final answer, but an intermediate prediction. This is why we have distinguished it with a superscript 0. [Equation \(22.15\)](#) is called a *predictor equation*. It provides an estimate that allows the calculation of a slope at the end of the interval:

$$y'_{i+1} = f(t_{i+1}, y_{i+1}^0) \quad (22.16)$$

Thus, the two slopes [[Eqs. \(22.14\)](#) and [\(22.16\)](#)] can be combined to obtain an average slope for the interval:

$$y^- = \frac{f(t_i, y_i) + f(t_{i+1}, y_{i+1}^0)}{2}$$

This average slope is then used to extrapolate linearly from  $y_i$  to  $y_{i+1}$  using Euler's method:

$$y_{i+1} = y_i + \frac{f(t_i, y_i) + f(t_{i+1}, y_{i+1}^0)}{2} h \quad (22.17)$$

which is called a *corrector equation*.

The Heun method is a *predictor-corrector approach*. As just derived, it can be expressed concisely as

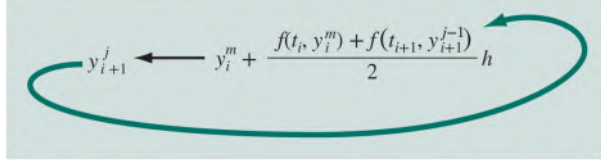
Predictor ([Fig. 22.4a](#)): 
$$y_{i+1}^0 = y_i^m + f(t_i, y_i) h \quad (22.18)$$

Corrector ([Fig. 22.4b](#)): 
$$y_{i+1}^j = y_i^m + \frac{f(t_i, y_i^m) + f(t_{i+1}, y_{i+1}^{j-1})}{2} h \quad (22.19)$$

$$(\text{for } j = 1, 2, \dots, m)$$

Note that because [Eq. \(22.19\)](#) has  $y_{i+1}$  on both sides of the equal sign, it can be applied in an iterative fashion as indicated. That is, an old estimate can be used repeatedly to provide an improved estimate of  $y_{i+1}$ . The process is depicted in [Fig. 22.5](#).

**FIGURE 22.5** Graphical representation of iterating the corrector of Heun's method to obtain an improved estimate.



As with similar iterative methods discussed in previous sections of the book, a termination criterion for convergence of the corrector is provided by

$$|\varepsilon_a| = \left| \frac{y_{i+1}^j - y_{i+1}^{j-1}}{y_{i+1}^j} \right| \times 100\%$$

where  $y_{i+1}^{j-1}$  and  $y_{i+1}^j$  are the result from the prior and the present iteration of the corrector, respectively. It should be understood that the iterative process does not necessarily converge on the true answer but will converge on an estimate with a finite truncation error, as demonstrated in the following example.

## EXAMPLE 22.2

### Heun's Method

#### Problem Statement

Use Heun's method with iteration to integrate  $y' = 4e^{0.8t} - 0.5y$  from  $t = 0$  to 4 with a step size of 1. The initial condition at  $t = 0$  is  $y = 2$ . Employ a stopping criterion of 0.00001% to terminate the corrector iterations.

#### Solution

First, the slope at  $(t_0, y_0)$  is calculated as

$$y_0' = 4e^0 - 0.5(2) = 3$$

Then, the predictor is used to compute a value at 1.0:

$$y_1^0 = 2 + 3(1) = 5$$

Note that this is the result that would be obtained by the standard Euler method. The true value in

**Table 22.2** shows that it corresponds to a percent relative error of 19.28%.

**TABLE 22.2** Comparison of true and numerical values of the integral of  $y' = 4e^{0.8t} - 0.5y$ , with the initial condition that  $y = 2$  at  $t = 0$ . The numerical values were computed using the Euler and Heun methods with a step size of 1. The Heun method was implemented both without and with iteration of the corrector.

|     | Without Iteration |                    |                       |                   | With Iteration        |                   |                       |
|-----|-------------------|--------------------|-----------------------|-------------------|-----------------------|-------------------|-----------------------|
| $t$ | $y^{\text{true}}$ | $y^{\text{Euler}}$ | $ \varepsilon_t (\%)$ | $y^{\text{Heun}}$ | $ \varepsilon_t (\%)$ | $y^{\text{Heun}}$ | $ \varepsilon_t (\%)$ |
| 0   | 2.00000           | 2.00000            |                       | 2.00000           |                       | 2.00000           |                       |
| 1   | 6.19463           | 5.00000            | 19.28                 | 6.70108           | 8.18                  | 6.36087           | 2.68                  |
| 2   | 14.84392          | 11.40216           | 23.19                 | 16.31978          | 9.94                  | 15.30224          | 3.09                  |
| 3   | 33.67717          | 25.51321           | 24.24                 | 37.19925          | 10.46                 | 34.74328          | 3.17                  |
| 4   | 75.33896          | 56.84931           | 24.54                 | 83.33777          | 10.62                 | 77.73510          | 3.18                  |

Now, to improve the estimate for  $y_{i+1}$ , we use the value  $y_1^0$  to predict the slope at the end of the interval,

$$y_1' = f(x_1, y_1^0) = 4e^{0.8(1)} - 0.5(5) = 6.402164$$

which can be combined with the initial slope to yield an average slope over the interval from  $t = 0$  to 1:

$$y^- = \frac{3 + 6.402164}{2} = 4.701082$$

This result can then be substituted into the corrector [Eq. (22.19)] to give the prediction at  $t = 1$ :

$$y_1^1 = 2 + 4.701082(1) = 6.701082$$

which represents a true percent relative error of  $-8.18\%$ . Thus, the Heun method without iteration of the corrector reduces the absolute value of the error by a factor of about 2.4 as compared with Euler's method. At this point, we can also compute an approximate error as

$$|e_a| = \left| \frac{6.701082 - 5}{6.701082} \right| \times 100\% = 25.39\%$$



Now, the estimate of  $y_1$  can be refined by substituting the new result back into the right-hand side of [Eq. \(22.19\)](#) to give

$$y_1^2 = 2 + \frac{3 + 4e^{0.8(1)} - 0.5(6.701082)}{2} \cdot 1 = 6.275811$$

which represents a true percent relative error of 1.31 percent and an approximate error of

$$|\varepsilon_a| = \left| \frac{6.275811 - 6.701082}{6.275811} \right| \times 100\% = 6.776\%$$

The next iteration gives

$$y_1^2 = 2 + \frac{3 + 4e^{0.8(1)} - 0.5(6.275811)}{2} \cdot 1 = 6.382129$$

which represents a true error of 3.03% and an approximate error of 1.666%.

The approximate error will keep dropping as the iterative process converges on a stable final result. In this example, after 12 iterations the approximate error falls below the stopping criterion. At this point, the result at  $t = 1$  is 6.36087, which represents a true relative error of 2.68%. [Table 22.2](#) shows results for the remainder of the computation along with results for Euler's method and for the Heun method without iteration of the corrector.

Insight into the local error of the Heun method can be gained by recognizing that it is related to the trapezoidal rule. In the previous example, the derivative is a function of both the dependent variable  $y$  and the independent variable  $t$ . For cases such as polynomials, where the ODE is solely a function of the independent variable, the predictor step [[Eq. \(22.18\)](#)] is not required and the corrector is applied only once for each iteration. For such cases, the technique is expressed concisely as

$$y_{i+1} = y_i + \frac{f(t_i) + f(t_{i+1})}{2} h \quad (22.20)$$

Notice the similarity between the second term on the right-hand side of [Eq. \(22.20\)](#) and the trapezoidal rule [[Eq. \(19.11\)](#)]. The connection between the two methods can be formally demonstrated by starting with the ordinary differential equation

$$\frac{dy}{dt} = f(t) \quad (22.21)$$

This equation can be solved for  $y$  by integration:

$$\int_{y_i}^{y_{i+1}} dy = \int_{t_i}^{t_{i+1}} f(t) dt \quad (22.22)$$

which yields

$$y_{i+1} - y_i = \int_{t_i}^{t_{i+1}} f(t) dt \quad (22.23)$$

or

$$y_{i+1} = y_i + \int_{t_i}^{t_{i+1}} f(t) dt \quad (22.24)$$

Now, recall that the trapezoidal rule [Eq. (19.11)] is defined as

$$\int_{t_i}^{t_{i+1}} f(t) dt = \frac{f(t_i) + f(t_{i+1})}{2} h \quad (22.25)$$

where  $h = t_{i+1} - t_i$ . Substituting Eq. (22.25) into Eq. (22.24) yields

$$y_{i+1} = y_i + \frac{f(t_i) + f(t_{i+1})}{2} h \quad (22.26)$$

which is equivalent to Eq. (22.20). For this reason, Heun's method is sometimes referred to as the trapezoidal rule.

Because Eq. (22.26) is a direct expression of the trapezoidal rule, the local truncation error is given by [recall Eq. (19.14)]

$$E_t = -\frac{f''(\xi)}{12} h^3 \quad (22.27)$$

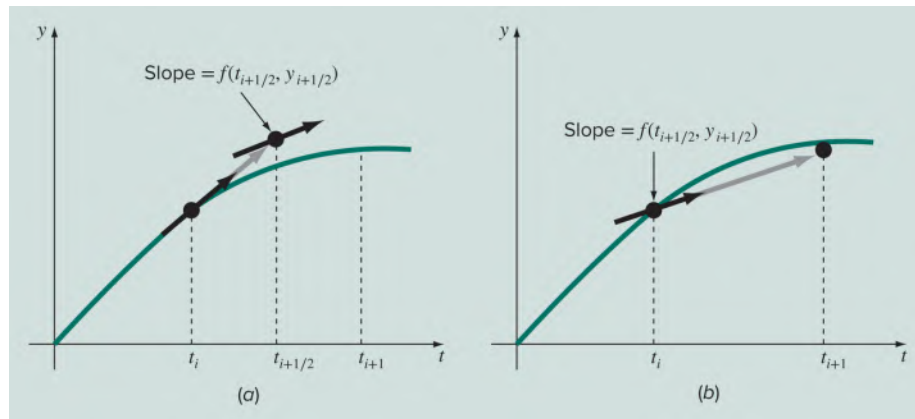
where  $\xi$  is between  $t_i$  and  $t_{i+1}$ . Thus, the method is second order because the second derivative of the ODE is zero when the true solution is a quadratic. In addition, the local and global errors are  $O(h^3)$  and  $O(h^2)$ , respectively. Therefore, decreasing the step size decreases the error at a faster rate than for Euler's method.

## 22.3.2 The Midpoint Method

 **Figure 22.6** illustrates another simple modification of Euler's method. Called the *midpoint method*, this technique uses Euler's method to predict a value of  $y$  at the midpoint of the interval ( **Fig. 22.6a**):

$$y_{i+1/2} = y_i + f(t_i, y_i) \frac{h}{2} \quad (22.28)$$

**FIGURE 22.6** Graphical depiction of midpoint method. (a) Predictor and (b) corrector.



Then, this predicted value is used to calculate a slope at the midpoint:

$$y'_{i+1/2} = f(t_{i+1/2}, y_{i+1/2}) \quad (22.29)$$

which is assumed to represent a valid approximation of the average slope for the entire interval. This slope is then used to extrapolate linearly from  $t_i$  to  $t_{i+1}$  ( **Fig. 22.6b**):

$$y_{i+1} = y_i + f(t_{i+1/2}, y_{i+1/2})h \quad (22.30)$$

Observe that because  $y_{i+1}$  is not on both sides, the corrector [ **Eq. (22.30)**] cannot be applied iteratively to improve the solution as was done with Heun's method.

As in our discussion of Heun's method, the midpoint method can also be linked to Newton-Cotes integration formulas. Recall from [Table 19.4](#) that the simplest Newton-Cotes open integration formula, which is called the midpoint method, can be represented as

$$\int_a^b f(x)dx \cong (b-a)f(x_1) \quad (22.31)$$

where  $x_1$  is the midpoint of the interval  $(a, b)$ . Using the nomenclature for the present case, it can be expressed as

$$\int_{t_i}^{t_{i+1}} f(t)dt \cong hf(t_{i+1/2}) \quad (22.32)$$

Substitution of this formula into [Eq. \(22.24\)](#) yields [Eq. \(22.30\)](#). Thus, just as the Heun method can be called the trapezoidal rule, the midpoint method gets its name from the underlying integration formula on which it is based.

The midpoint method is superior to Euler's method because it utilizes a slope estimate at the midpoint of the prediction interval. Recall from our discussion of numerical differentiation in [Sec. 4.3.4](#) that centered finite differences are better approximations of derivatives than either forward or backward versions. In the same sense, a centered approximation such as [Eq. \(22.29\)](#) has a local truncation error of  $O(h^2)$  in comparison with the forward approximation of Euler's method, which has an error of  $O(h)$ . Consequently, the local and global errors of the midpoint method are  $O(h^3)$  and  $O(h^2)$ , respectively.

## 22.4 RUNGE-KUTTA METHODS

Runge-Kutta (RK) methods achieve the accuracy of a Taylor series approach without requiring the calculation of higher derivatives. Many variations exist but all can be cast in the generalized form of

 Eq. (22.4):

$$y_{i+1} = y_i + \phi h \quad (22.33)$$

where  $\phi$  is called an *increment function*, which can be interpreted as a representative slope over the interval. The increment function can be written in general form as

$$\phi = a_1 k_1 + a_2 k_2 + \cdots + a_n k_n \quad (22.34)$$

where the  $a$ 's are constants and the  $k$ 's are

Page 544

$$k_1 = f(t_i, y_i) \quad (22.34a)$$

$$k_2 = f(t_i + p_1 h, y_i + q_{11} k_1 h) \quad (22.34b)$$

$$k_3 = f(t_i + p_2 h, y_i + q_{21} k_1 h + q_{22} k_2 h) \quad (22.34c)$$

$$\vdots$$

$$k_n = f(t_i + p_{n-1} h, y_i + q_{n-1,1} k_1 h + q_{n-1,2} k_2 h + \cdots + q_{n-1,n-1} k_{n-1} h) \quad (22.34d)$$

where the  $p$ 's and  $q$ 's are constants. Notice that the  $k$ 's are recurrence relationships. That is,  $k_1$  appears in the equation for  $k_2$  which appears in the equation for  $k_3$  and so forth. Because each  $k$  is a functional evaluation, this recurrence makes RK methods efficient for computer calculations.

Various types of RK methods can be devised by employing different numbers of terms in the increment function as specified by  $n$ . Note that the first-order RK method with  $n = 1$  is, in fact, Euler's method.

Once  $n$  is chosen, values for the  $a$ 's,  $p$ 's, and  $q$ 's are evaluated by setting  Eq. (22.33) equal to terms in a Taylor series expansion. Thus, at least for the lower-order versions, the number of terms  $n$  usually represents the order of the approach. For example, in  Sec. 22.4.1, second-order RK methods use an increment function with two terms ( $n = 2$ ). These second-order methods will be exact if the solution to the differential equation is quadratic. In addition, because terms with  $h^3$  and higher are dropped during the derivation, the local truncation error is  $O(h^3)$  and the global error is  $O(h^2)$ . In  Sec. 22.4.2, the fourth-order RK method ( $n = 4$ ) is presented for which the global truncation error is  $O(h^4)$ .

### 22.4.1 Second-Order Runge-Kutta Methods

The second-order version of [Eq. \(22.33\)](#) is

$$y_{i+1} = y_i + (a_1 k_1 + a_2 k_2)h \quad (22.35)$$

where

$$k_1 = f(t_i, y_i) \quad (22.35a)$$

$$k_2 = f(t_i + p_1 h, y_i + q_{11} k_1 h) \quad (22.35b)$$

The values for  $a_1$ ,  $a_2$ ,  $p_1$ , and  $q_{11}$  are evaluated by setting [Eq. \(22.35\)](#) equal to a second-order Taylor series. By doing this, three equations can be derived to evaluate the four unknown constants (see Chapra and Canale, 2010, for details). The three equations are

$$a_1 + a_2 = 1 \quad (22.36)$$

$$a_2 p_1 = 1/2 \quad (22.37)$$

$$a_2 q_{11} = 1/2 \quad (22.38)$$

Because we have three equations with four unknowns, these equations are said to be underdetermined. We, therefore, must assume a value of one of the unknowns to determine the other three. Suppose that we specify a value for  $a_2$ . Then, [Eqs. \(22.36\)](#) through [Eq. \(22.38\)](#) can be solved simultaneously for

$$a_1 = 1 - a_2 \quad (22.39)$$

$$p_1 = q_{11} = \frac{1}{2a_2} \quad (22.40)$$

Because we can choose an infinite number of values for  $a_2$ , there are an infinite number of second-order RK methods. Every version would yield exactly the same results if the solution to the ODE were quadratic, linear, or a constant. However, they yield different results when (as is typically the case) the solution is more complicated. Three of the most commonly used and preferred versions are presented next.

## Heun Method without Iteration ( $a_2 = 1/2$ )

If  $a_2$  is assumed to be  $1/2$ , [Eqs. \(22.39\)](#) and [Eq. \(22.40\)](#) can be solved for  $a_1 = 1/2$  and  $p_1 = q_{11} = 1$ . These parameters, when substituted into [Eq. \(22.35\)](#), yield

$$y_{i+1} = y_i + \left( \frac{1}{2} k_1 + \frac{1}{2} k_2 \right) h \quad (22.41)$$

where

$$k_1 = f(t_i, y_i) \quad (22.41a)$$

$$k_2 = f(t_i + h, y_i + k_1 h) \quad (22.41b)$$

Note that  $k_1$  is the slope at the beginning of the interval and  $k_2$  is the slope at the end of the interval. Consequently, this second-order RK method is actually Heun's technique without iteration of the corrector.

## The Midpoint Method ( $a_2 = 1$ )

If  $a_2$  is assumed to be 1, then  $a_1 = 0$ ,  $p_1 = q_{11} = 1/2$ , and [Eq. \(22.35\)](#) becomes

$$y_{i+1} = y_i + k_2 h \quad (22.42)$$

where

$$k_1 = f(t_i, y_i) \quad (22.42a)$$

$$k_2 = f(t_i + h/2, y_i + k_1 h/2) \quad (22.42b)$$

This is the midpoint method.

## Ralston's Method ( $a_2 = 3/4$ )

Ralston (1962) and Ralston and Rabinowitz (1978) determined that choosing  $a_2 = 3/4$  provides a minimum bound on the truncation error for the second-order RK algorithms. For this version,  $a_1 = 1/4$  and  $p_1 = q_{11} = 2/3$ , and [Eq. \(22.35\)](#) becomes

$$y_{i+1} = y_i + \left( \frac{1}{4} k_1 + \frac{3}{4} k_2 \right) h \quad (22.43)$$

where

$$k_1 = f(t_i, y_i) \quad (22.43a)$$

$$k_2 = f\left(t_i + \frac{2}{3}h, y_i + \frac{2}{3}k_1 h\right) \quad (22.43b)$$

## 22.4.2 Classical Fourth-Order Runge-Kutta Method

The most popular RK methods are fourth order. As with the second-order approaches, there are an infinite number of versions. The following is the most commonly used form, and we therefore call it the *classical fourth-order RK method*:

$$y_{i+1} = y_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)h \quad (22.44)$$

where

$$k_1 = f(t_i, y_i) \quad (22.44a)$$

$$k_2 = f\left(t_i + \frac{1}{2}h, y_i + \frac{1}{2}k_1h\right) \quad (22.44b)$$

$$k_3 = f\left(t_i + \frac{1}{2}h, y_i + \frac{1}{2}k_2h\right) \quad (22.44c)$$

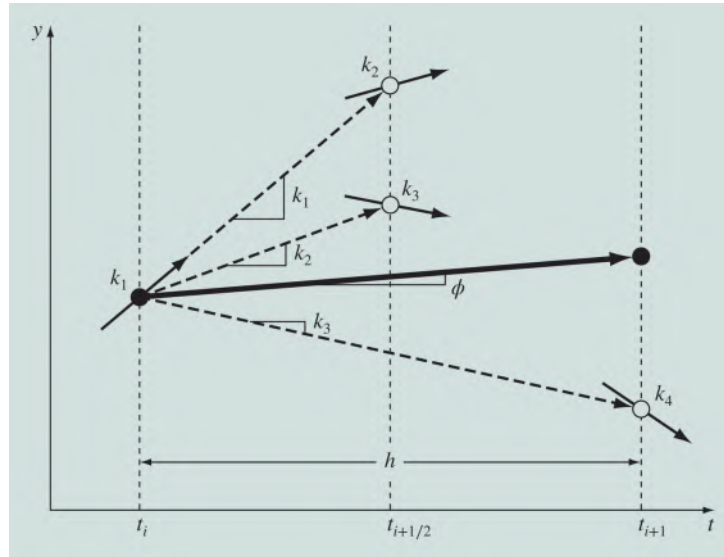
$$k_4 = f(t_i + h, y_i + k_3h) \quad (22.44d)$$

Notice that for ODEs that are a function of  $t$  alone, the classical fourth-order RK method is similar to Simpson's 1/3 rule. In addition, the fourth-order RK method is similar to the Heun approach in that multiple estimates of the slope are developed to come up with an improved average slope for the interval. As depicted in  **Fig. 22.7**, each of the  $k$ 's represents a slope.  **Equation (22.44)** then represents a weighted average of these to arrive at the improved slope.

---

**FIGURE 22.7** Graphical depiction of the slope estimates comprising the fourth-order RK method.





## EXAMPLE 22.3

### Classical Fourth-Order RK Method

#### Problem Statement

Employ the classical fourth-order RK method to integrate  $y' = 4e^{0.8t} - 0.5y$  from  $t = 0$  to 1 using a step size of 1 with  $y(0) = 2$ .

#### Solution

For this case, the slope at the beginning of the interval is computed as

$$k_1 = f(0, 2) = 4e^{0.8(0)} - 0.5(2) = 3$$

This value is used to compute a value of  $y$  and a slope at the midpoint:

$$y(0.5) = 2 + 3(0.5) = 3.5$$

$$k_2 = f(0.5, 3.5) = 4e^{0.8(0.5)} - 0.5(3.5) = 4.217299$$

This slope in turn is used to compute another value of  $y$  and another slope at the midpoint:

$$y(0.5) = 2 + 4.217299(0.5) = 4.108649$$

$$k_3 = f(0.5, 4.108649) = 4e^{0.8(0.5)} - 0.5(4.108649) = 3.912974$$

Next, this slope is used to compute a value of  $y$  and a slope at the end of the interval:

$$\begin{aligned} y(1.0) &= 2 + 3.912974(1.0) = 5.912974 \\ k_4 &= f(1.0, 5.912974) = 4e^{0.8(1.0)} - 0.5(5.912974) = 5.945677 \end{aligned}$$

Finally, the four slope estimates are combined to yield an average slope. This average slope is then used to make the final prediction at the end of the interval.

$$\begin{aligned} \phi &= \frac{1}{6}[3 + 2(4.217299) + 2(3.912974) + 5.945677] = 4.201037 \\ y(1.0) &= 2 + 4.201037(1.0) = 6.201037 \end{aligned}$$

which compares favorably with the true solution of 6.194631 ( $\varepsilon_t = 0.103\%$ ).

It is certainly possible to develop fifth- and higher-order RK methods. For example, Butcher's (1964) fifth-order RK method is written as

$$y_{i+1} = y_i + \frac{1}{90}(7k_1 + 32k_3 + 12k_4 + 32k_5 + 7k_6)h \quad (22.45)$$

where

Page 547

$$k_1 = f(t_i, y_i) \quad (22.45a)$$

$$k_2 = f\left(t_i + \frac{1}{4}h, y_i + \frac{1}{4}k_1h\right) \quad (22.45b)$$

$$k_3 = f\left(t_i + \frac{1}{4}h, y_i + \frac{1}{8}k_1h + \frac{1}{8}k_2h\right) \quad (22.45c)$$

$$k_4 = f\left(t_i + \frac{1}{2}h, y_i - \frac{1}{2}k_2h + k_3h\right) \quad (22.45d)$$

$$k_5 = f\left(t_i + \frac{3}{4}h, y_i + \frac{3}{16}k_1h + \frac{9}{16}k_4h\right) \quad (22.45e)$$

$$k_6 = f\left(t_i + h, y_i - \frac{3}{7}k_1h + \frac{2}{7}k_2h + \frac{12}{7}k_3h - \frac{12}{7}k_4h + \frac{8}{7}k_5h\right) \quad (22.45f)$$

Note the similarity between Butcher's method and Boole's rule in [Table 19.2](#). As expected, this method has a global truncation error of  $O(h^5)$ .

Although the fifth-order version provides more accuracy, notice that six function evaluations are required. Recall that up through the fourth-order versions,  $n$  function evaluations are required for an  $n$ th-order RK method. Interestingly, for orders higher than four, one or two additional function evaluations are necessary. Because the function evaluations account for the most computation time, methods of order five and higher are usually considered relatively less efficient than the fourth-order versions. This is one of the main reasons for the popularity of the fourth-order RK method.

## 22.5 SYSTEMS OF EQUATIONS

Many practical problems in engineering and science require the solution of a system of simultaneous ordinary differential equations rather than a single equation. Such systems may be represented generally as

$$\begin{aligned}\frac{dy_1}{dt} &= f_1(t, y_1, y_2, \dots, y_n) \\ \frac{dy_2}{dt} &= f_2(t, y_1, y_2, \dots, y_n) \\ &\vdots \\ \frac{dy_n}{dt} &= f_n(t, y_1, y_2, \dots, y_n)\end{aligned}\quad (22.46)$$

The solution of such a system requires that  $n$  initial conditions be known at the starting value of  $t$ .

An example is the calculation of the bungee jumper's velocity and position that we set up at the beginning of this chapter. For the free-fall portion of the jump, this problem amounts to solving the following system of ODEs:

$$\frac{dx}{dt} = v \quad (22.47)$$

$$\frac{dv}{dt} = g - \frac{c_d}{m} v^2 \quad (22.48)$$

If the stationary platform from which the jumper launches is defined as  $x = 0$ , the initial conditions would be  $x(0) = v(0) = 0$ .

### 22.5.1 Euler's Method

All the methods discussed in this chapter for single equations can be extended to systems of ODEs. Engineering applications can involve thousands of simultaneous equations. In each case, the procedure for solving a system of equations simply involves applying the one-step technique for every equation at each step before proceeding to the next step. This is best illustrated by the following example for Euler's method.

#### EXAMPLE 22.4

##### Solving Systems of ODEs with Euler's Method

## Problem Statement

Solve for the velocity and position of the free-falling bungee jumper using Euler's method. Assuming that at  $t = 0$ ,  $x = v = 0$ , and integrate to  $t = 10$  s with a step size of 2 s. As was done previously in [Examples 1.1](#) and [1.2](#), the gravitational acceleration is  $9.81 \text{ m/s}^2$ , and the jumper has a mass of 68.1 kg with a drag coefficient of  $0.25 \text{ kg/m}$ .

Recall that the analytical solution for velocity is [[Eq. \(1.9\)](#)]:

$$v(t) = \sqrt{\frac{gm}{c_d}} \tanh \left( \sqrt{\frac{gc_d}{m}} t \right)$$

This result can be substituted into [Eq. \(22.47\)](#), which can be integrated to determine an analytical solution for distance as

$$x(t) = \frac{m}{c_d} \ln \left[ \cosh \left( \sqrt{\frac{gc_d}{m}} t \right) \right]$$

Use these analytical solutions to compute the true relative errors of the results.

## Solution

The ODEs can be used to compute the slopes at  $t = 0$  as

$$\begin{aligned} \frac{dx}{dt} &= 0 \\ \frac{dv}{dt} &= 9.81 - \frac{0.25}{68.1} (0)^2 = 9.81 \end{aligned}$$

Euler's method is then used to compute the values at  $t = 2$  s,

$$\begin{aligned} x &= 0 + 0(2) = 0 \\ v &= 0 + 9.81(2) = 19.62 \end{aligned}$$

The analytical solutions can be computed as  $x(2) = 19.16629$  and  $v(2) = 18.72919$ . Thus, the percent relative errors are 100% and 4.756%, respectively.

The process can be repeated to compute the results at  $t = 4$  as

$$x = 0 + 19.62(2) = 39.24$$

$$v = 19.62 + \left( 9.81 - \frac{0.25}{68.1}(19.62)^2 \right) 2 = 36.41368$$

Proceeding in a like manner gives the results displayed in [Table 22.3](#).

**TABLE 22.3** Distance and velocity of a free-falling bungee jumper as computed numerically with Euler's method.

| $t$ | $x_{\text{true}}$ | $v_{\text{true}}$ | $x_{\text{Euler}}$ | $v_{\text{Euler}}$ | $\varepsilon_t(x)$ | $\varepsilon_t(v)$ |
|-----|-------------------|-------------------|--------------------|--------------------|--------------------|--------------------|
| 0   | 0                 | 0                 | 0                  | 0                  |                    |                    |
| 2   | 19.1663           | 18.7292           | 0                  | 19.6200            | 100.00%            | 4.76%              |
| 4   | 71.9304           | 33.1118           | 39.2400            | 36.4137            | 45.45%             | 9.97%              |
| 6   | 147.9462          | 42.0762           | 112.0674           | 46.2983            | 24.25%             | 10.03%             |
| 8   | 237.5104          | 46.9575           | 204.6640           | 50.1802            | 13.83%             | 6.86%              |
| 10  | 334.1782          | 49.4214           | 305.0244           | 51.3123            | 8.72%              | 3.83%              |

Although the foregoing example illustrates how Euler's method can be implemented for systems of ODEs, the results are not very accurate because of the large step size. In addition, the results for distance are a bit unsatisfying because  $x$  does not change until the second iteration. Using a much smaller step greatly mitigates these deficiencies. As described next, using a higher-order solver provides decent results even with a relatively large step size.

## 22.5.2 Runge-Kutta Methods

Note that any of the higher-order RK methods in this chapter can be applied to systems of equations.

However, care must be taken in determining the slopes. [Figure 22.7](#) is helpful in visualizing the proper way to do this for the fourth-order method. That is, we first develop slopes for all variables at the initial value. These slopes (a set of  $k_1$ 's) are then used to make predictions of the dependent variables at the midpoint of the interval. These midpoint values are in turn used to compute a set of slopes at the midpoint (the  $k_2$ 's). These new slopes are then taken back to the starting point to make another set of midpoint predictions that lead to new slope predictions at the midpoint (the  $k_3$ 's). These are then employed to make predictions at the end of the interval that are used to develop slopes at the end of the interval (the  $k_4$ 's). Finally, the  $k$ 's are combined into a set of increment functions [as in [Eq. \(22.44\)](#)] that are brought back to the beginning to make the final predictions. The following example illustrates the approach.

### EXAMPLE 22.5

#### Solving Systems of ODEs with the Fourth-Order RK Method

## Problem Statement

Use the fourth-order RK method to solve for the same problem we addressed in [Example 22.4](#).

## Solution

First, it is convenient to express the ODEs in the functional format of [Eq. \(22.46\)](#) as

$$\begin{aligned}\frac{dx}{dt} &= f_1(t, x, v) = v \\ \frac{dv}{dt} &= f_2(t, x, v) = g - \frac{c_d}{m} v^2\end{aligned}$$

The first step in obtaining the solution is to solve for all the slopes at the beginning of the interval:

$$\begin{aligned}k_{1,1} &= f_1(0, 0, 0) = 0 \\ k_{1,2} &= f_2(0, 0, 0) = 9.81 - \frac{0.25}{68.1}(0)^2 = 9.81\end{aligned}$$

where  $k_{i,j}$  is the  $i$ th value of  $k$  for the  $j$ th dependent variable. Next, we must calculate the first values of  $x$  and  $v$  at the midpoint of the first step:

$$\begin{aligned}x(1) &= x(0) + k_{1,1} \frac{h}{2} = 0 + 0 \frac{2}{2} = 0 \\ v(1) &= v(0) + k_{1,2} \frac{h}{2} = 0 + 9.81 \frac{2}{2} = 9.81\end{aligned}$$

which can be used to compute the first set of midpoint slopes:

$$\begin{aligned}k_{2,1} &= f_1(1, 0, 9.81) = 9.8100 \\ k_{2,2} &= f_2(1, 0, 9.81) = 9.4567\end{aligned}$$

These are used to determine the second set of midpoint predictions:

$$\begin{aligned}x(1) &= x(0) + k_{2,1} \frac{h}{2} = 0 + 9.8100 \frac{2}{2} = 9.8100 \\ v(1) &= v(0) + k_{2,2} \frac{h}{2} = 0 + 9.4567 \frac{2}{2} = 9.4567\end{aligned}$$

which can be used to compute the second set of midpoint slopes:

$$k_{3,1} = f_1(1, 9.8100, 9.4567) = 9.4567$$

$$k_{3,2} = f_2(1, 9.8100, 9.4567) = 9.4817$$

These are used to determine the predictions at the end of the interval:

$$x(2) = x(0) + k_{3,1}h = 0 + 9.4567(2) = 18.9134$$

$$v(2) = v(0) + k_{3,2}h = 0 + 9.4817(2) = 18.9634$$

which can be used to compute the endpoint slopes:

$$k_{4,1} = f_1(2, 18.9134, 18.9634) = 18.9634$$

$$k_{4,2} = f_2(2, 18.9134, 18.9634) = 8.4898$$

The values of  $k$  can then be used to compute [Eq. (22.44)]:

Page 550

$$x(2) = 0 + \frac{1}{6} [0 + 2(9.8100 + 9.4567) + 18.9634] 2 = 19.1656$$

$$v(2) = 0 + \frac{1}{6} [9.8100 + 2(9.4567 + 9.4817) + 8.4898] 2 = 18.7256$$

Proceeding in a like manner for the remaining steps yields the values displayed in Table 22.4. In contrast to the results obtained with Euler's method, the fourth-order RK predictions are much closer to the true values. Further, a highly accurate, nonzero value is computed for distance on the first step.

**TABLE 22.4** Distance and velocity of a free-falling bungee jumper as computed numerically with the fourth-order RK method.

| $t$ | $x_{\text{true}}$ | $v_{\text{true}}$ | $x_{\text{RK4}}$ | $v_{\text{RK4}}$ | $\epsilon_t(x)$ | $\epsilon_t(v)$ |
|-----|-------------------|-------------------|------------------|------------------|-----------------|-----------------|
| 0   | 0                 | 0                 | 0                | 0                |                 |                 |
| 2   | 19.1663           | 18.7292           | 19.1656          | 18.7256          | 0.004%          | 0.019%          |
| 4   | 71.9304           | 33.1118           | 71.9311          | 33.0995          | 0.001%          | 0.037%          |
| 6   | 147.9462          | 42.0762           | 147.9521         | 42.0547          | 0.004%          | 0.051%          |
| 8   | 237.5104          | 46.9575           | 237.5104         | 46.9345          | 0.000%          | 0.049%          |
| 10  | 334.1782          | 49.4214           | 334.1626         | 49.4027          | 0.005%          | 0.038%          |



## 22.5.3 Python Function: rk4sys

 **Figure 22.8** shows a Python function, `rk4sys`, that implements the fourth-order RK method to solve a system of ODEs. The code here is similar in many ways to the function developed earlier ( **Fig. 22.3**) to solve a single ODE with Euler's method. For example, the function name, `dydt`, defining the derivatives is passed as its first argument.

---

**FIGURE 22.8** A Python function, `rk4sys`, to implement the RK4 method for a system of ODEs.

```
import numpy as np

def rk4sys(dydt,tspan,y0,h= -1.,*args):
    """
    fourth-order Runge-Kutta method
    for solving a system of ODEs
    input:
        dydt = function name that evaluates the derivatives
        tspan = array of independent variable values where either
                ti and tf are the initial and final values
                of the independent variable when h is specified,
                or the array specifies the values of t for
                solution (h is not specified)
        y0 = initial value of the dependent variable
        h = step size, default = 0.1
        *args = additional argument to be passed to dydt
    output:
        t = array of independent variable values
        y = array of dependent variable values
    """
    if np.any(np.diff(tspan) < 0): return 'tspan times must be ascending'
    # check if only ti and tf spec'd and no value for h
    if len(tspan) == 2 and h != -1.:
        ti = tspan[0] ; tf = tspan[1]
        nsteps = int((tf-ti)/h)
        t = []
        t.append(ti)
        for i in range(nsteps): # add the rest of the t values
            t.append(ti+(i+1)*h)
        n = len(t)
        if t[n-1] < tf: # check if t array is short of tf
            t.append(tf)
            n = n+1
    else:
        n = len(tspan) # here if tspan contains step times
        t = tspan
    neq = len(y0)
    y = np.zeros((n,neq)) # set up 2-D array for dependent variables
    for j in range(neq):
        y[0,j] = y0[j] # set first elements to initial conditions
    for i in range(n-1): # 4th order RK
        hh = t[i+1] - t[i]
        k1 = dydt(t[i],y[i,:],*args)
        ymid = y[i,:] + k1*hh/2.
        k2 = dydt(t[i]+hh/2.,ymid,*args)
        ymid = y[i,:] + k2*hh/2.
        k3 = dydt(t[i]+hh/2.,ymid,*args)
        yend = y[i,:] + k3*hh
        k4 = dydt(t[i]+hh,yend,*args)
        phi = (k1 + 2.*(k2+k3) + k4)/6.
        y[i+1,:] = y[i,:] + phi*hh
    return t,y
```

However, it has an additional feature that allows you to generate output in two ways, depending on whether the step size,  $h$ , is specified in the call to `rk4sys`. If  $h$  is specified with a value, the `tspan` array must contain only two values, `ti` and `tf`. If  $h$  is not included as an argument, the solution will take place at the values in the `tspan` array. In this case, if the `tspan` array has only two values, `ti` and `tf`, the RK4 method will take only one step. However, if the `tspan` array has multiple values, those will be used as the steps for the solution.

## EXAMPLE 22.6

### Solving Coupled ODEs with `rk4sys`

#### Problem Statement

Employ the `rk4sys` function to solve the following coupled differential equations:

$$\begin{aligned}\frac{dy_1}{dt} &= -2y_1^2 + 2y_1 + y_2 - 1 & y_1(0) &= 2 \\ \frac{dy_2}{dt} &= -y_1 - 3y_2^2 + 2y_2 + 2 & y_2(0) &= 0\end{aligned}$$

over the domain  $0 \leq t \leq 2$  with a step size of  $h = 0.01$ .

#### Solution

Here is the Python script to carry out the solution and plot the results:

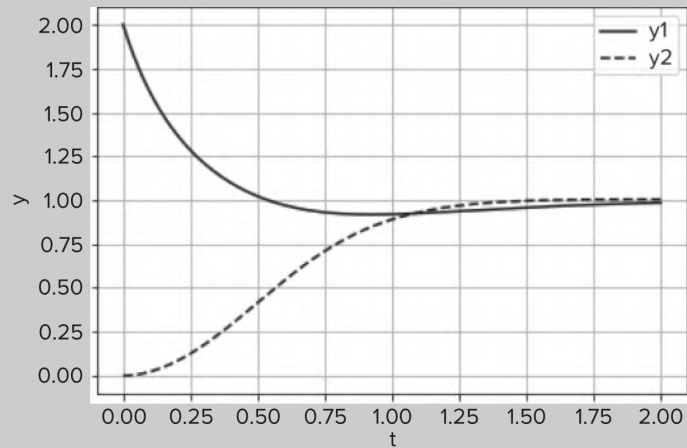
```
def dydtsys(t,y):
    n = len(y)
    dy = np.zeros((n))
    dy[0] = -2.*y[0]**2 + 2.*y[0] + y[1] - 1.
    dy[1] = -y[0] - 3*y[1]**2 + 2.*y[1] + 2.
    return dy

h = 0.01
ti = 0.; tf = 2.
tspan = [0.,2.]
y0 = np.array([2.,0.])
t,y = rk4sys(dydtsys,tspan,y0,h)

import pylab
pylab.plot(t,y[:,0],c='k',label='y1')
pylab.plot(t,y[:,1],c='k',ls='--',label='y2')
pylab.grid()
pylab.xlabel('t')
pylab.ylabel('y')
pylab.legend()
```

The plot of the solution is provided in  **Fig. 22.9**.

**FIGURE 22.9** Plot of the solution of two simultaneous ODEs using the `rk4sys` function.



Alternately, we can specify a set of `t` values in `tspan` and carry out a solution that uses those values. You will note that we leave out the specification of `h`, since that is irrelevant.

```
tspan = [0.,0.1,0.2,0.3,0.5,0.75,1.,1.25,1.5,1.75,2.]
y0 = np.array([2.,0.])
t,y = rk4sys(dydtsys,tspan,y0)

n = len(t)
print(' t  y1  y2')
for i in range(n):
    print('{0:5.2f} {1:7.4f} {2:7.4f}'.format(t[i],y[i,0],y[i,1]))
```

In this case, we choose to display the results in a table:

| t    | y1     | y2     |
|------|--------|--------|
| 0.00 | 2.0000 | 0.0000 |
| 0.10 | 1.6151 | 0.0225 |
| 0.20 | 1.3745 | 0.0830 |
| 0.30 | 1.2124 | 0.1744 |
| 0.50 | 1.0225 | 0.4160 |
| 0.75 | 0.9306 | 0.7103 |
| 1.00 | 0.9199 | 0.8899 |
| 1.25 | 0.9376 | 0.9683 |
| 1.50 | 0.9585 | 0.9958 |
| 1.75 | 0.9746 | 1.0033 |
| 2.00 | 0.9853 | 1.0041 |

## PREDATOR-PREY MODELS AND CHAOS

### Background

Engineers and scientists deal with a variety of problems involving systems of nonlinear ordinary differential equations. This case study focuses on two of these applications. The first relates to predator-prey models that are used to study species interactions. The second are equations derived from fluid dynamics that are used to simulate the atmosphere.

*Predator-prey models* were developed independently in the early part of the twentieth century by Italian mathematician Vito Volterra and the American biologist Alfred Lotka. These equations are therefore commonly called *Lotka-Volterra equations*. The simplest version is the following pair of ODEs:

$$\frac{dx}{dt} = ax - bxy \quad (22.49)$$

$$\frac{dy}{dt} = -cy + dxy \quad (22.50)$$

where  $x$  and  $y$  = the number of prey and predators, respectively,  $a$  = the prey growth rate,  $c$  = the predator death rate, and  $b$  and  $d$  = rates characterizing the effect of predator-prey interactions on the prey death and predator growth, respectively. The bilinear (multiplicative, involving  $x \cdot y$ ) terms are what makes such equations nonlinear. Since each equation involves both dependent variables, the equations are also coupled.

---

 Page 553

An example of a simple nonlinear model based on atmospheric fluid dynamics is the *Lorenz equations* created by the American meteorologist Edward Lorenz:

$$\begin{aligned} \frac{dx}{dt} &= -ax + ay \\ \frac{dy}{dt} &= rx - y - xz \\ \frac{dz}{dt} &= -bz + xy \end{aligned}$$

Lorenz developed these equations to relate the intensity of atmospheric fluid motion  $x$  to temperature variations  $y$  and  $z$  in the horizontal and vertical directions, respectively. As with the predator-prey model, the nonlinearities stem from the bilinear terms,  $x \cdot z$  and  $x \cdot y$ , and the equations are coupled.

### Problem Statement

Use numerical methods to obtain solutions for the Lotka-Volterra equations. Plot the results to visualize how the dependent variables change temporally. In addition, create plots of the dependent variables versus each other to

see whether any interesting patterns emerge.

The following parameter values can be used for the predator-prey simulation:  $a = 1.2$ ,  $b = 0.6$ ,  $c = 0.8$ , and  $d = 0.3$ . The initial conditions will be  $x = 2$  and  $y = 1$ , and the solution will be found over the time range from 0 to 30, using a step size of 0.0625.

## Solution

First, we can develop a function to compute the derivative values using the differential equations:

```
import numpy as np

def predprey(t,y,a,b,c,d):
    dy = np.zeros((2))
    dy[0] = a*y[0] - b*y[0]*y[1]
    dy[1] = -c*y[1] + d*y[0]*y[1]
    return dy
```

The following script employs this function to generate solutions with both the Euler and fourth-order RK methods. Note that the function `eulersys` is based on modifying the `rk4sys` function (Fig. 22.8). We will leave the development of such a function as a homework problem. In addition to displaying the solution as a time-series plot ( $x$  and  $y$  versus  $t$ ), the script also generates a plot of  $y$  versus  $x$ . Such *phase-plane* plots are often useful in elucidating features of the model's underlying structure that may not be evident from the time series plot.

```
import matplotlib.pyplot as plt

h = 0.0625
tspan = np.array([0.,40.])
y0 = np.array([2.,1.])
a = 1.2 ; b = 0.6 ; c = 0.8 ; d = 0.3

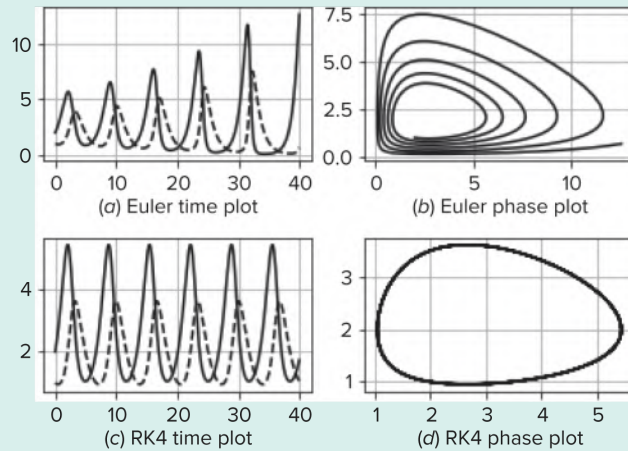
t,y = eulersys(predprey,tspan,y0,h,a,b,c,d)
fig = plt.figure()
ax1 = fig.add_subplot(221)
ax1.plot(t,y[:,0],c='k',label='prey')
ax1.plot(t,y[:,1],c='k',ls='--',label='predator')
ax1.grid()
ax1.set_xlabel('a) Euler time plot')
ax2 = fig.add_subplot(222)
ax2.plot(y[:,0],y[:,1],c='k')
ax2.grid()
ax2.set_xlabel('b) Euler phase plot')

t,y = rk4sys(predprey,tspan,y0,h,a,b,c,d)
ax3 = fig.add_subplot(223)
ax3.plot(t,y[:,0],c='k',label='prey')
ax3.plot(t,y[:,1],c='k',ls='--',label='predator')
ax3.grid()
ax3.set_xlabel('c) RK4 time plot')
ax4 = fig.add_subplot(224)
ax4.plot(y[:,0],y[:,1],c='k')
ax4.grid()
ax4.set_xlabel('d) RK4 phase plot')
plt.subplots_adjust(hspace=0.5)
```

Page 554

The solution obtained with Euler's method is shown at the top of Fig. 22.10. The time series (Fig. 22.10a) indicates that the amplitudes of the oscillations are expanding. This is reinforced by the phase-plane plot (Fig. 22.10b). Hence, these results indicate that the Euler method would require a much smaller time step to obtain accurate results.

**FIGURE 22.10** Solution for the Lotka-Volterra model. Euler's method (a) time series and (b) phase-plane plots. RK4 method (a) time series and (b) phase-plane plots.



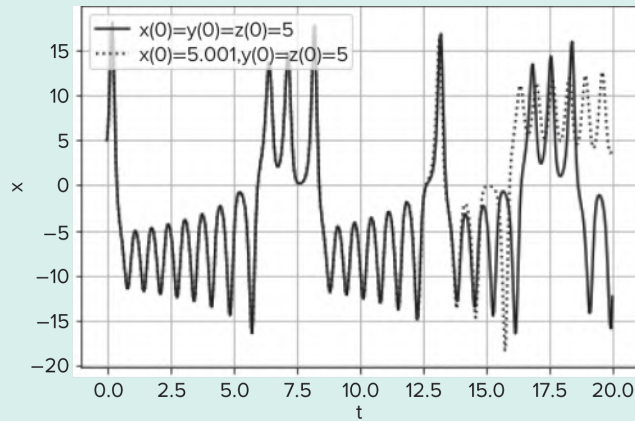
In contrast, because of its much smaller truncation error, the RK4 method yields good results with the same time step. As in [Fig. 22.10c](#), a cyclical pattern emerges in time. Because the predator population is initially small, the prey grows exponentially. At a certain point, the prey become so numerous that the predator population begins to grow. Eventually, the increased predators cause the prey to decline. This decrease, in turn, leads to a decrease of the predators. Eventually again, the process repeats. Notice that, as expected, the predator peak lags the prey. Also, observe that the process has a fixed period; that is, it repeats in a set time.

The phase-plane representation for the more accurate RK4 solution ([Fig. 22.9d](#)) indicates that the interaction between the predator and the prey amounts to a closed, counterclockwise orbit. Interestingly, there is a resting or *critical point* at the center of the orbit. The exact location of this point can be determined by setting [Eqs. \(22.49\)](#) and [\(22.50\)](#) to steady state ( $dy/dt = dx/dt = 0$ ) and solving  $(x, y) = (0, 0)$  and  $(c/d, a/b)$ . The former is the trivial result that if we start with the neither predators nor prey, nothing will happen. The latter is the more interesting outcome that, if the initial conditions are set at  $x = c/d$  and  $y = a/b$ , the derivatives will be zero, and the populations will remain constant.

Now, let's use the same approach to investigate the trajectories of the Lorenz equations with the following parameter values:  $a = 10$ ,  $b = 8/3$ , and  $r = 28$ . Employ the initial conditions of  $x = y = z = 5$  and integrate from  $t = 0$  to 20. For this case, we will use the RK4 method to obtain the solution with a time step of 0.03125.

The results are quite different from the behavior of the Lotka-Volterra equations. As shown in [Fig. 22.11](#), the variable  $x$  seems to be undergoing an almost random pattern of oscillations, bouncing around from negative to positive values. The other variables,  $y$  and  $z$ , exhibit similar behavior. However, even though the patterns have a random character, the frequency of the oscillations and the amplitudes seem fairly consistent.

**FIGURE 22.11** Time-domain representation of  $x$  versus  $t$  for the Lorenz equations. The solid-line time series is for the initial conditions (5, 5, 5). The dashed line is where the initial condition for  $x$  is perturbed slightly (5.001, 5, 5).

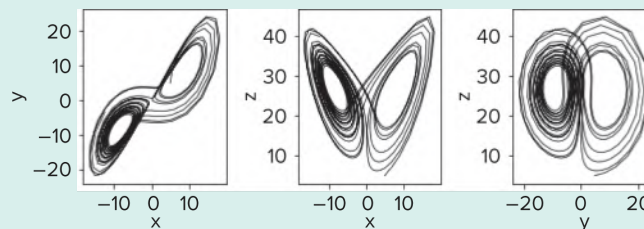


An interesting feature of such solutions can be illustrated by changing the initial condition for  $x$  slightly from 5 to 5.001. The results are superimposed as the dashed line in [Fig. 22.11](#). Although the solutions track each other for a time, about  $t = 15$ , they diverge significantly. Thus, we can see that the Lorenz equations are quite sensitive to their initial conditions. The term *chaotic* is used to describe such solutions. In his original study, this led Lorenz to the conclusion that long-range weather forecasts are difficult, if not impossible!

The sensitivity of a dynamical system to small perturbations in its initial conditions is sometimes called the *butterfly effect*. The idea is that the flapping of a butterfly's wings might induce tiny changes in the atmosphere that ultimately leads to a large-scale weather phenomenon like a tornado or cyclone.

Although the time series plots are chaotic, phase-plane plots can reveal underlying structure. Because we are dealing with three dependent variables, we can generate projections. [Figure 22.12](#) shows projections on the  $xy$ ,  $xz$ , and  $yz$  planes. Notice how patterns and structure emerge when observed from the phase-plane perspective. The solution forms orbits around what appear to be two critical points. These points are called *strange attractors* in the jargon of mathematicians who study such nonlinear systems.

**FIGURE 22.12** Phase-plane representation for the Lorenz equations: (a)  $xy$ , (b)  $xz$ , and (c)  $yz$  projections.



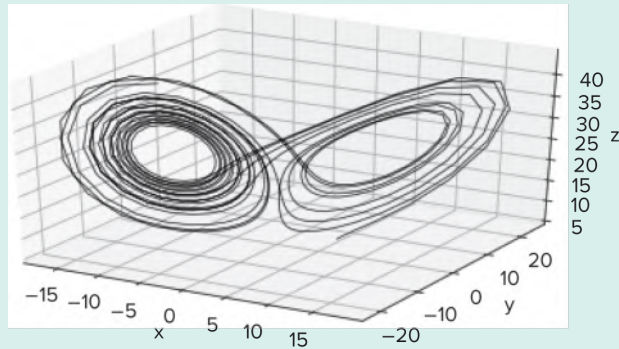
Beyond the two-variable projections, Python's Matplotlib module provides the capability to display a three-dimensional phase-plane plot with the code:

```
from mpl_toolkits.mplot3d import Axes3D
fig1 = plt.figure(figsize=[7.,4.])
ax = fig1.add_subplot(111,projection='3d')
ax.plot(y[:,0],y[:,1],y[:,2],c='k',lw=0.7)
ax.grid()
ax.set_xlabel('x')
ax.set_ylabel('y')
```

```
ax.set_zlabel('z')
```

As was the case in [Fig. 22.12](#), the three-dimensional plot ([Fig. 22.13](#)) depicts trajectories in a cyclic pattern that switches between two patterns around a pair of critical points.

**FIGURE 22.13** Three-dimensional phase-plane representation for the Lorenz equations generated with Python's Matplotlib module.



As a final note, the sensitivity of chaotic systems to initial conditions has implications for numerical computations. Beyond the initial conditions themselves, different step sizes or different algorithms can introduce small differences in the solutions. In a similar fashion to [Fig. 22.11](#), these discrepancies may eventually lead to large deviations. Some of the problems in this chapter and in [Chap. 23](#) are designed to illustrate this issue.



# PROBLEMS

---

**22.1** Solve the following initial value problem over the interval from  $t = 0$  to 2 where  $y(0) = 1$ . Display all your results on the same graph.

$$\frac{dy}{dt} = yt^2 - 1.1y$$

- (a) Analytically.
- (b) Using Euler's method with  $h = 0.5$  and  $0.25$ .
- (c) Using the midpoint method with  $h = 0.5$ .
- (d) Using the fourth-order RK method with  $h = 0.5$ .

You may choose to make these calculations manually or create a Python script with function definitions for the methods.

**22.2** Solve the following problem over the interval from  $x = 0$  to 1 using a step size of 0.25 where  $y(0) = 1$ . Display all your results on the same graph.

$$\frac{dy}{dx} = (1 + 2 \cdot x)\sqrt{y}$$

- (a) Analytically.
- (b) Using Euler's method.
- (c) Using Heun's method without iteration.
- (d) Using Ralston's method.
- (e) Using the fourth-order RK method.

You may choose to make these calculations manually or create a Python script with function definitions for the methods.

---

Page 557

---

**22.3** Solve the following problem over the interval from  $t = 0$  to 3 using a step size of 0.5 with  $y(0) = 1$ . Display all your results on a graph.

$$\frac{dy}{dt} = -y + t^2$$

Obtain your solutions with (a) Heun's method without iterating the corrector, (b) Heun's method iterating the corrector until  $\varepsilon_s < 0.1\%$ , (c) the midpoint method, and (d) Ralston's method. You may choose to make these calculations manually or create a Python script with function definitions for the methods.

**22.4** The growth of populations of organisms has many engineering and scientific applications. One of the simplest models assumes that the rate of change of the population,  $p$ , is proportional to the existing population at any time,  $t$ :

$$\frac{dp}{dt} = k_g p \quad (\text{P22.4})$$

The world population in billions ( $10^9$ ) from 1960 through 2020 is recorded in the table below.

|                                 |       |       |       |       |       |       |       |
|---------------------------------|-------|-------|-------|-------|-------|-------|-------|
| <b>year</b>                     | 1960  | 1965  | 1970  | 1975  | 1980  | 1985  | 1990  |
| <b>Population</b><br>( $10^6$ ) | 3.035 | 3.340 | 3.700 | 4.079 | 4.458 | 4.871 | 5.327 |
| <b>year</b>                     | 1995  | 2000  | 2005  | 2010  | 2015  | 2020  |       |
| <b>Population</b><br>( $10^6$ ) | 5.744 | 6.143 | 6.542 | 6.957 | 7.380 | 7.795 |       |

- (a) Assuming that [Eq. \(P22.4\)](#) holds, use the data from 1960 through 2020 to estimate  $k_g$ .
- (b) Use the fourth-order RK method along with the result from (a) to simulate world population from 1960 to 2060 with a step size of five years. Display your simulation results along with the data on a plot.

**22.5** Although the model in [Prob. 22.4](#) may work adequately when population growth is unlimited, it breaks down when there are constraining factors, such as food shortages, pollution, disease, and lack of space, which inhibit growth. In such cases, the growth rate is not a constant, but can be modeled by

$$k_g = k_{gm} \left( 1 - \frac{p}{p_{\max}} \right)$$

where  $k_{gm}$  = the maximum growth rate under no constraining conditions,  $p$  = the population, and  $p_{\max}$  = the maximum population attainable given the constraints. Note the  $p_{\max}$  is sometimes called the *carrying capacity*. Thus, at low population density,  $p \ll p_{\max}$ ,  $k_g \rightarrow k_{gm}$ . As  $p$  approaches  $p_{\max}$ , the growth rate approaches zero. Using this growth rate formulation, the rate of change of population can be modeled as

$$\frac{dp}{dt} = k_{gm} \left( 1 - \frac{p}{p_{\max}} \right) p$$

This is referred to as a *logistic model*. The analytical solution to this model is

$$p = p_0 \frac{p_{\max}}{p_0 + (p_{\max} - p_0) \cdot e^{-k_{gm}t}}$$

Simulate the world's population from 1960 to 2060 using (a) the analytical solution, and (b) the fourth-order RK method with a step size of five years. Employ the following initial conditions and parameter values for your simulation:

$$p_0 = 3.035 \text{ billion} \quad k_{gm} = 0.0281 \quad p_{\max} = 12.06 \text{ billion}$$

Note: For the value of  $k_{gm}$ ,  $t = 0$  at year 1960. Display your results as a plot of population versus year along with the data from  **Prob. 22.4**.

**22.6** Suppose that a projectile is launched upward from the earth's surface. Assume that the only force acting on the object is the downward force of gravity. Under these conditions, a force balance can be used to derive

$$\frac{dv}{dt} = -g_0 \frac{R^2}{(R + x)^2}$$

where  $v$  = upward velocity (m/s),  $t$  = time (s),  $x$  = altitude (m) measure upward from the earth's surface,  $g_0$  = the gravitational acceleration at the earth's surface ( $\cong 9.81 \text{ m/s}^2$ ), and  $R$  = earth's radius ( $\cong 6.36 \times 10^6 \text{ m}$ ). Recognizing that  $dx/dt = v$ , use the `rk4sys` function ( **Fig. 22.8**) to determine the maximum height that would be obtained if  $v(t = 0) = 1500 \text{ m/s}$ .

**22.7** Solve the following pair of ODEs over the interval from  $t = 0$  to 4 using a step size of 0.1. The initial conditions are  $y(0) = 2$  and  $z(0) = 4$ . Obtain your solution with (a) Euler's method, and (b) the fourth-order RK method. Display your results in a plot.

$$\begin{aligned} \frac{dy}{dt} &= -2y + 4e^{-t} \\ \frac{dz}{dt} &= -\frac{yz^2}{3} \end{aligned}$$

**22.8** The *van der Pol equation* is a model of an electronic circuit that arose back in the days of vacuum tubes:

$$\frac{d^2y}{dt^2} - (1 - y^2) \frac{dy}{dt} + y = 0$$

Given the initial conditions,  $y(0) = y'(0) = 1$ , solve this equation from  $t = 0$  to 10 using Euler's method with a step size of (a) 0.2 and (b) 0.1. Plot both solutions on the same graph.

**22.9** Given the initial conditions,  $y(0) = 1$ , and  $y'(0) = 0$ , solve the following initial value problem from  $t = 0$  to 4:

$$\frac{d^2y}{dt^2} + 9y = 0$$

Obtain your solutions with (a) Euler's method and (b) the fourth-order RK method. In both cases, use a step size of 0.1. Plot both solutions on the same graph along with the exact solution,  $y = \cos(3t)$ .

**22.10** Develop a Python function to solve a single ODE with Heun's method with iteration. The function definition statement should be

```
def heuniter(dydt,tspan,y0,h,es=1.e-8,maxiter=30,*args)
```

where `dydt` = function to compute the derivative with arguments of values of the independent and dependent variables, and any additional arguments required by `dydt`, `tspan` = an array containing the initial and final values of the independent variable, `y0` = the initial value of the dependent variable, `h` = the step size, `es` = the relative error tolerance, `maxiter` = the maximum number of iterations allowed to converge the Heun corrector, and `*args` = any additional arguments that will be passed to `dydt`.

Test your function by solving the differential equation described in [Prob. 22.2](#). Use instead a step size of 0.05. Compare using the default error tolerance with a tolerance specified at 1%. Plot your results on the same graph.

**22.11** Develop a Python function to solve a single ODE with midpoint method. The function definition statement should be

```
def midpt(dydt,tspan,y0,h,*args)
```

where `dydt` = function to compute the derivative with arguments of values of the independent and dependent variables, and any additional arguments required by `dydt`, `tspan` = an array containing the initial and final values of the independent variable, `y0` = the initial value of the dependent variable, `h` = the step size, and `*args` = any additional arguments that will be passed to `dydt`. Test your function by solving the differential equation described in [Prob. 22.2](#). Use instead a step size of 0.05. Plot your results.

**22.12** Develop a Python function to solve a single ODE with the fourth-order RK method. The function definition statement should be

```
def rk4(dydt,tspan,y0,h,*args)
```

where `dydt` = function to compute the derivative with arguments of values of the independent and dependent variables, and any additional arguments required by `dydt`, `tspan` = an array containing the initial and final values of the independent variable, `y0` = the initial value of the dependent variable, `h` = the step size, and `*args` = any additional arguments that will be passed to `dydt`. Test your function by

solving the differential equation described in [Prob. 22.2](#). Use instead a step size of 0.1. Plot your results.

**22.13** Develop a Python function to solve a system of ODEs with Euler's method. The function definition statement should be

```
def eulsys(dydt,tspan,y0,h,*args)
```

where `dydt` = function to compute the derivative with arguments of values of the independent and dependent variables, and any additional arguments required by `dydt`, `tspan` = an array containing the initial and final values of the independent variable, `y0` = the initial value of the dependent variable, `h` = the step size, and `*args` = any additional arguments that will be passed to `dydt`. Test your function by solving the differential equations described in [Prob. 22.7](#). Use instead a step size of 0.005. Plot your results.

**22.14** Isle Royale National Park is a 540-square-kilometer archipelago composed of a single large island and many small islands in Lake Superior. Moose arrived there around 1900, and, by 1930, their population approached 3,000, ravaging vegetation in the park. In 1949, wolves crossed an ice bridge from Ontario. Since the late 1950s, the numbers of the moose and wolves have been tracked. The table presents the data from 1959 through 2006.

- Integrate the Lotka-Volterra equations ([Sec. 22.6](#)) from 1960 through 2020 using the following coefficient values:  $a = 0.23$ ,  $b = 0.0133$ ,  $c = 0.4$ , and  $d = 0.0004$ . Compare your simulation with the data using a time-series plot.
- Present your simulation results in a phase-plane plot.

| Year | Moose | Wolves | Year | Moose | Wolves | Year | Moose | Wol |
|------|-------|--------|------|-------|--------|------|-------|-----|
| 1959 | 563   | 20     | 1975 | 1355  | 41     | 1991 | 1313  | 1   |
| 1960 | 610   | 22     | 1976 | 1282  | 44     | 1992 | 1590  | 1   |
| 1961 | 628   | 22     | 1977 | 1143  | 34     | 1993 | 1879  | 1   |
| 1962 | 639   | 23     | 1978 | 1001  | 40     | 1994 | 1770  | 1   |
| 1963 | 663   | 20     | 1979 | 1028  | 43     | 1995 | 2422  | 1   |
| 1964 | 707   | 26     | 1980 | 910   | 50     | 1996 | 1163  | 2   |
| 1965 | 733   | 28     | 1981 | 863   | 30     | 1997 | 500   | 2   |
| 1966 | 765   | 26     | 1982 | 872   | 14     | 1998 | 699   | 1   |
| 1967 | 912   | 22     | 1983 | 932   | 23     | 1999 | 750   | 2   |
| 1968 | 1042  | 22     | 1984 | 1038  | 24     | 2000 | 850   | 2   |
| 1969 | 1268  | 17     | 1985 | 1115  | 22     | 2001 | 900   | 1   |
| 1970 | 1295  | 18     | 1986 | 1192  | 20     | 2002 | 1100  | 1   |
| 1971 | 1439  | 20     | 1987 | 1268  | 16     | 2003 | 900   | 1   |
| 1972 | 1493  | 23     | 1988 | 1335  | 12     | 2004 | 750   | 2   |
| 1973 | 1435  | 24     | 1989 | 1397  | 12     | 2005 | 540   | 3   |
| 1974 | 1467  | 31     | 1990 | 1216  | 15     | 2006 | 450   | 3   |

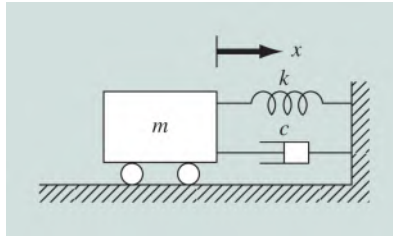
You may wish to provide the data via a text file.

**22.15** The motion of a damped spring-mass system ( **Fig. P22.15**) is described by the following ordinary differential equation:

$$m \frac{d^2x}{dt^2} + c \frac{dx}{dt} + kx = 0$$

where  $x$  = displacement from equilibrium position (m),  $t$  = time (s),  $c$  = damping coefficient (N·s/m). The damping coefficient,  $c$ , takes on three values of 5 (underdamped), 40 (critically damped), and 200 (overdamped). The spring constant is 20 N/m. The initial velocity is zero, and the initial displacement is  $x = 1$  m. Solve this equation using a numerical method of your choice over the period of time from 0 to 15 s. Plot the displacement versus time for the three cases of the damping coefficient on the same plot.

**FIGURE P22.15**



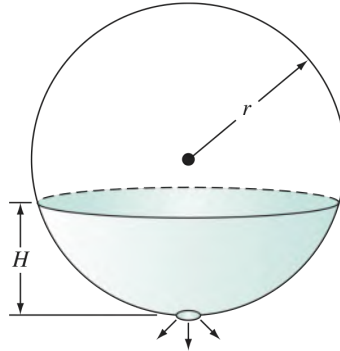
**22.16** A spherical tank has a circular orifice at its bottom center through which liquid flows out ( **Fig. P22.16**). The flow rate through the orifice is described by *Torricelli's law*:

$$Q = CA\sqrt{gh}$$

where  $Q$  = volumetric exit flow rate (m<sup>3</sup>/s),  $C$  = a flow coefficient, typical value around 0.6,  $A$  = orifice area (m<sup>2</sup>), and  $h$  = liquid depth (m). Use one of the numerical methods described in this chapter to determine how long it will take for water to drain completely out of a 3-m diameter tank with an initial level of 2.75 m. The orifice diameter is 0.1 m. The volume of liquid,  $V$ , in a sphere as shown in  **Fig. P22.16** is given by

$$V = \pi H^2 \frac{(3r - H)}{3} \quad \text{and} \quad \frac{dV}{dt} = -Q$$

**FIGURE P22.16** Spherical tank with orifice in the bottom center draining a liquid.



**22.17** In the investigation of a homicide or accidental death, it is often important to estimate the time of death. From experimental observations, it is known that the surface temperature of an object changes at a rate proportional to the difference between the temperature of the object and that of the surrounding environment or ambient temperature. This is known as Newton's law of cooling. Thus, if  $T(t)$  is the temperature of the surface of the object at time  $t$ ,  $T_a$  is a constant ambient temperature, then

Page 559

$$\frac{dT}{dt} = -K(T - T_a)$$

where  $K > 0$  is a constant of proportionality. Suppose that at  $t = 0$  a corpse is discovered, and its temperature is measured to be  $T_0$ . We assume that, at the time of death, the body temperature,  $T_d$ , was at a normal, standard value of  $37^\circ\text{C}$  ( $98.6^\circ\text{F}$ ). Suppose that the temperature of the corpse when it was discovered was  $29.5^\circ\text{C}$ , and that two hours later, it is  $23.5^\circ\text{C}$ . The ambient temperature is  $20^\circ\text{C}$ .

- Determine  $K$  and the time of death.
- Solve the ODE numerically and plot the body temperature versus time.

**22.18** A chemical reaction represented by  $A \rightarrow B$  takes place in two reactor tanks that are connected in series. The reactors are well mixed, but they do not operate at steady state. The unsteady-state mass balances for the two reactors are as follows:

$$\begin{aligned}\frac{dC_{A1}}{dt} &= \frac{q}{V_1}(C_{A0} - C_{A1}) - kC_{A1} \\ \frac{dC_{B1}}{dt} &= -\frac{q}{V_1}C_{B1} + kC_{A1} \\ \frac{dC_{A2}}{dt} &= \frac{q}{V_2}(C_{A1} - C_{A2}) - kC_{A2} \\ \frac{dC_{B2}}{dt} &= \frac{q}{V_2}(C_{B1} - C_{B2}) + kC_{A2}\end{aligned}$$

where  $V_1$  is the volume of the first reactor (L),  $q$  is the volumetric flow rate through the reactors (L/min),  $C_{A0}$  is the concentration of the reactant in the feed stream to the first reactor,  $C_{A1}$  is the concentration of  $A$  in the first reactor and at the inlet of the second reactor,  $C_{B1}$  is the concentration of  $B$  in the first reactor and at the inlet of the second reactor,  $C_{A2}$  is the concentration of  $A$  in the second reactor,  $C_{B2}$  is the concentration of  $B$  in the second reactor, and  $k$  is the reaction rate parameter. All concentrations are measured in mol/L, and  $k$  carries the units of 1/min. There is no  $B$  in the feed stream to the first reactor. The  $V/q$  ratios represent the average residence time of material in a particular reactor in minutes.

If  $V_1 = 1000$  L,  $V_2 = 500$  L,  $q = 150$  L/min,  $C_{A0} = 20$  mol/L,  $k = 0.12$  min<sup>-1</sup>, and the initial concentrations in the reactors are zero (start-up conditions), simulate the reactor behavior for the initial 30 minutes of operation. Depict your result graphically.

**22.19** A nonisothermal batch reactor is described by the following equations:

$$\begin{aligned}\frac{dC}{dt} &= -0.35e^{-\frac{10}{T+273}} C \\ \frac{dT}{dt} &= 1000e^{-\frac{10}{T+273}} C - 15(T - 20)\end{aligned}$$

where  $C$  is the concentration of the reactant (mol/L),  $T$  is the temperature of the reactor (°C), and  $t$  is time in minutes. Initially, the reactor is at 15 °C and a concentration of reactant  $C$  of 1.0 mol/L. Determine the concentration and temperature in the reactor as a function of time. Present your results graphically.

**22.20** The following equation can be used to model the deflection of a sailboat mast subject to a wind force:

$$\frac{d^2y}{dz^2} = \frac{f(z)}{2EI}(L - z)^2$$

where  $f(z)$  = wind force,  $E$  = modulus of elasticity,  $L$  = mast length, and  $I$  = moment of inertia. The force on the mast varies with height,  $z$ , according to

$$f(z) = \frac{200z}{5 + z} e^{-2z/30}$$

Calculate the deflection versus height if  $y = 0$  and  $dy/dz = 0$  at  $z = 0$ . Use parameter values of  $L = 30$ ,  $E = 1.25 \times 10^8$ , and  $I = 0.05$  for your computation. Present your results in a plot.

**22.21** A pond drains through a pipe as shown in  **Fig. P22.21**. Using several simplifying assumptions, the following differential equation describes how depth changes with time:

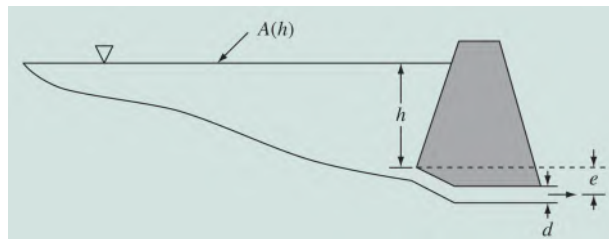


$$\frac{dh}{dt} = -\frac{\pi d^2}{4A(h)} \sqrt{2gh + e}$$

where  $h$  = depth (m),  $t$  = time (s),  $d$  = pipe diameter (m),  $A(h)$  = pond surface area as a function of depth ( $\text{m}^2$ ), and  $e$  = depth of pipe outlet below pond bottom (m). Based on the following area-depth table, solve this differential equation to determine how long it takes for the pond to empty, given that  $h(0) = 6$  m,  $d = 0.25$  m, and  $e = 1$  m. Also, display a plot of pond level versus time.

| $h, \text{ m}$           | 6    | 5    | 4    | 3    | 2    | 1    | 0 |
|--------------------------|------|------|------|------|------|------|---|
| $A(h), 10^4 \text{ m}^2$ | 1.17 | 0.97 | 0.67 | 0.45 | 0.32 | 0.18 | 0 |

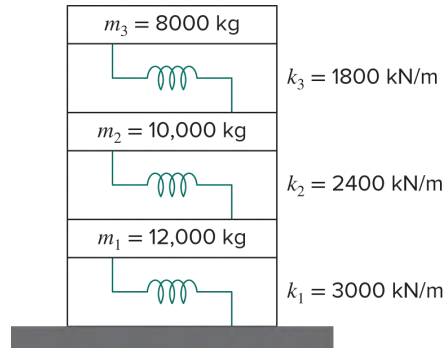
**FIGURE P22.21**



**22.22** Engineers and scientists use mass-spring models to gain insight into the dynamics of structures under the influence of disturbances such as earthquakes.  **Figure P22.22** shows such a representation for a three-story building. For this case, the analysis is limited to horizontal motion of the structure. Using Newton's second law, force balances can be developed for this system as

$$\begin{aligned} \frac{d^2 x_1}{dt^2} &= -\frac{k_1}{m_1} x_1 + \frac{k_2}{m_1} (x_2 - x_1) \\ \frac{d^2 x_2}{dt^2} &= -\frac{k_2}{m_2} (x_1 - x_2) + \frac{k_3}{m_2} (x_3 - x_2) \\ \frac{d^2 x_3}{dt^2} &= \frac{k_3}{m_3} (x_2 - x_3) \end{aligned}$$

**FIGURE P22.22**



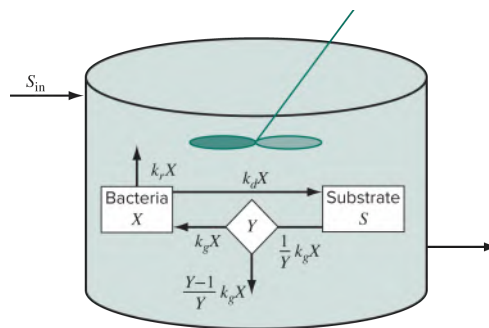
Simulate the dynamics of this structure from  $t = 0$  to 20 s, given the initial condition that the velocity of the ground floor is  $dx_1/dt = 1$  m/s, and all other initial values of displacements and velocities are zero. Present your results as two time-series plots of (a) displacements and (b) velocities. In addition, develop a three-dimensional phase-plane plot of the displacements.

**22.23** Repeat the same simulations as in [Sec. 22.6](#) for the Lorenz equations but generate the solutions with the midpoint method.

**22.24** Perform the same simulation as in [Sec. 22.6](#) for the Lorenz equations, but use a value of  $r = 99.96$ . Employ a step size of  $h = 0.001$  to achieve a “smoother” solution. Compare your results with those obtained in the section.

**22.25** [Figure P22.25](#) shows the kinetic interactions governing the concentration of a bacterial culture and its nutrition source (substrate) in a continuous-flow, stirred-tank reactor (CSTR).

**FIGURE P22.25** Continuous-flow, stirred-tank reactor (CSTR) to grow a bacterial culture.



The mass balances for the bacterial biomass,  $X$  (gC/m<sup>3</sup>), and the substrate concentration,  $S$  (gC/m<sup>3</sup>), can be written as

$$\begin{aligned}\frac{dX}{dt} &= \left( k_{g,\max} \frac{S}{K_s + S} - k_d - k_r - \frac{1}{\tau_w} \right) X \\ \frac{dS}{dt} &= \left( -\frac{1}{Y} k_{g,\max} \frac{S}{K_s + S} + k_d \right) X + \frac{1}{\tau_w} (S_{\text{in}} - S)\end{aligned}$$

where  $t$  = time (h),  $k_{g,\max}$  = maximum bacterial growth rate (1/h),  $K_s$  = half-saturation constant (gC/m<sup>3</sup>),  $k_d$  = death rate (1/h),  $k_r$  = respiration rate (1/h),  $Y$  = yield coefficient gC-cell/gC-substrate,  $S_{\text{in}}$  = inflow substrate concentration (mgC/m<sup>3</sup>),  $\tau_w$  = reactor residence time (h), given by

$$\tau_w = \frac{V}{Q}$$

where  $V$  = reactor volume (m<sup>3</sup>), and  $Q$  = volumetric flow rate (m<sup>3</sup>/h). Simulate how the bacteria and substrate change over 100 hours of operation in this reactor for three residence times: 20, 10, and 5 h. Employ the following parameters for the simulation:  $X(0) = 100$  gC/m<sup>3</sup>,  $S(0) = 0$ ,  $k_{g,\max} = 0.2$  1/h,  $K_s = 150$  gC/m<sup>3</sup>,  $k_d = k_r = 0.01$  1/h,  $Y = 0.5$  gC-cell/gC-substrate, and  $S_{\text{in}} = 1000$  gC/m<sup>3</sup>. Display your results graphically. Attempt to present the result on one plot with the  $X$  scale on the left axis and the  $S$  scale on the right axis (Hint: See [Sec. 3.2](#)).

# Adaptive Methods and Stiff Systems

## CHAPTER OBJECTIVES

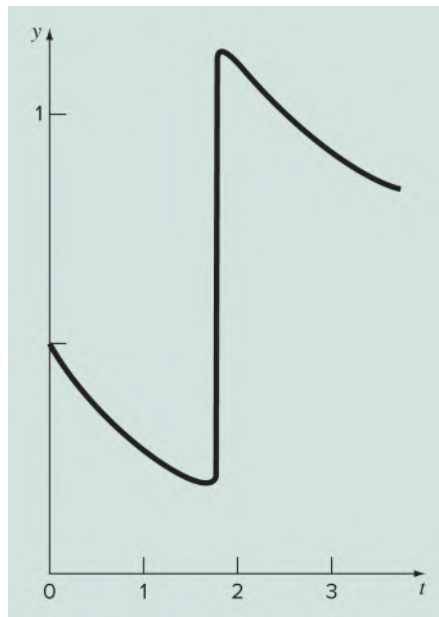
The primary objective of this chapter is to introduce you to more advanced methods for solving initial-value problems for ordinary differential equations. Specific objectives and topics covered are

- Understanding how the Runge-Kutta-Fehlberg (RKF) methods use RK methods of different orders to provide error estimates that are used to adjust the step size.
- Seeing how the RKF45 method is implemented in Python with fourth- and fifth-order Runge-Kutta formulas.
- Familiarizing yourself with the built-in Python functions for solving ODE available from the SciPy module's `integrate` submodule.
- Learning how to adjust the options of the Python `solve_ivp` function.
- Understanding how to pass parameters to the `solve_ivp` function required by the function that evaluates the derivatives.
- Understanding the difference between one-step and multistep methods for solving ODEs.
- Understanding what is meant by stiffness and its implications.

## 23.1 ADAPTIVE RUNGE-KUTTA METHODS

To this point, we have presented methods for solving ODEs that employ a constant step size. For a significant number of real problems, this can represent a serious limitation. For example, suppose that we are integrating an ODE with a solution of the type illustrated in [Fig. 23.1](#). For most of the range, the solution changes gradually. Such behavior suggests that a fairly large step size could be employed to obtain adequate results. However, for a localized region from  $t = 1.75$  to  $2.25$ , the solution undergoes an abrupt change. The practical consequence of dealing with such behavior is that a very small step size would be required to capture the impulsive behavior accurately. If a constant step-size algorithm were employed, the smaller step size required for the region of abrupt change would have to be applied to the entire computation. As a consequence, a much smaller step size than necessary, and, therefore, many more calculations, would be wasted on the regions of gradual change.

**FIGURE 23.1** An example of a solution of an ODE that exhibits an abrupt change. Automatic step-size adjustment has great advantages for such cases.



Algorithms that automatically adjust the step size can avoid such overkill and inefficiency and hence be of great advantage. Because they “adapt” to the solution’s trajectory, they are said to have adaptive step-size control. Implementation of such approaches requires that an estimate of the local truncation error be obtained at each step. This error estimate can then serve as a basis for either shortening or lengthening the step size.

A scenario that occurs frequently in engineering and science that suggests the use of adaptive methods is the simulation of the dynamic behavior of processes or phenomena. Typically, changes are very rapid with

the departure from initial conditions, but later, the response slows down as the simulation approaches a steady-state condition. Small, perhaps tiny, step sizes are required initially; however, these can stretch out to long step sizes at the end and still maintain the accuracy of the solution. The efficiency gained by adaptive methods is amplified when the solution is embedded in another algorithm, such as optimization or nonlinear regression, which requires the ODEs to be solved over and over again, perhaps hundreds or thousands of times.

Before proceeding, we should mention that, aside from solving ODEs, the methods described in Page 562 this chapter can also be used to evaluate definite integrals. The evaluation of

$$I = \int_a^b f(x)dx$$

is equivalent to solving the differential equation

$$\frac{dy}{dx} = f(x)$$

for  $y(b)$  given the initial condition  $y(a) = 0$ . Thus, the following techniques can be employed to evaluate efficiently definite integrals involving functions that are generally smooth but exhibit regions of abrupt change.

There are two primary approaches to incorporate adaptive step-size control into one-step methods. Step halving involves taking each step twice, once as a full-step and then as two half steps. The difference in the two results represents an estimate of the local truncation error. The step size can then be adjusted based on this error estimate. One naturally thinks of reducing the step size in order to meet an error criterion, but there is also a consideration of increasing the step size, on a trial basis, when the error criterion is first met. In other words, the algorithm must accommodate step-size adjustment in both directions, smaller and larger.

In the second approach, call embedded RK methods, the local truncation error is estimated as the difference between two predictions using different-order RK methods. These are currently the methods of choice because they are more efficient than step halving.

The embedded methods were first developed by Fehlberg. Hence, they are often referred to as *RKF methods*. At face value, the idea of using two predictions of different order might seem too computationally expensive. For example, a fourth- and fifth-order prediction amounts to a total of ten function evaluations per step [recall [Eqs. \(22.44\)](#) and [\(22.45\)](#)]. Fehlberg cleverly circumvented this problem by deriving a fifth-order RK method that employs most of the same function evaluations required for an accompanying fourth-order RK method. Thus, the approach yielded the error estimate based on only six function evaluations!

## 23.1.1 A Python Function for the Runge-Kutta-Fehlberg 4/5 Algorithm: rkf45

As described above, the basis for the RKF algorithm is to generate two different estimates for a given step size and compare those to determine whether the step size should be reduced or increased. The most popular version of the RKF algorithm uses fourth- and fifth-order Runge-Kutta methods formulated to minimize the number of function evaluations (to six).

Consider the differential equation

$$\frac{dy}{dx} = f(x, y)$$

and the current step  $h$  from  $\{x_k, y_k\}$  to  $\{x_{k+1}, y_{k+1}\}$ , where  $h = x_{k+1} - x_k$ .

The six function evaluations, each building on the previous, required are

$$\begin{aligned} k_1 &= hf(x_k, y_k) \\ k_2 &= hf\left(x_k + \frac{1}{4}h, y_k + \frac{1}{4}k_1\right) \\ k_3 &= hf\left(x_k + \frac{3}{8}h, y_k + \frac{3}{32}k_1 + \frac{9}{32}k_2\right) \\ k_4 &= hf\left(x_k + \frac{12}{13}h, y_k + \frac{1932}{2197}k_1 - \frac{7200}{2197}k_2 + \frac{7296}{2197}k_3\right) \\ k_5 &= hf\left(x_k + h, y_k + \frac{439}{216}k_1 - 8k_2 + \frac{3680}{513}k_3 - \frac{845}{4104}k_4\right) \\ k_6 &= hf\left(x_k + \frac{1}{2}h, y_k - \frac{8}{27}k_1 + 2k_2 - \frac{3544}{2565}k_3 + \frac{1859}{4104}k_4 - \frac{11}{40}k_5\right) \end{aligned} \quad (23.1)$$

A single-step calculation for fourth-order and fifth-order RK methods is then

$$y_{k+1}^{[4]} = y_k + \frac{25}{216}k_1 + \frac{1408}{2565}k_3 + \frac{2197}{4101}k_4 - \frac{1}{5}k_5 \quad (23.2)$$

$$y_{k+1}^{[5]} = y_k + \frac{16}{135}k_1 + \frac{6656}{12825}k_3 + \frac{28561}{56430}k_4 - \frac{9}{50}k_5 + \frac{2}{55}k_6 \quad (23.3)$$

Note that in [Eqs. \(23.2\)](#) and [\(23.3\)](#), the  $k_2$  factor is not used explicitly.

The fractions in the expressions of [Eqs. \(23.1\)](#), [\(23.2\)](#), and [\(23.3\)](#) are computed *a priori* to improve the efficiency of the algorithm. The local relative error,  $e_{k+1}$ , is computed as

$$e_{k+1} = \left| \frac{y_{k+1}^{[5]} - y_{k+1}^{[4]}}{y_{k+1}^{[5]}} \right|$$

The decision then has to be made whether and how to adjust  $h$ . A scheme has been developed to do this using a scale factor,  $s$ , which is computed as

$$s = \left( \frac{\varepsilon_s}{2e_{k+1}} \right)^{1/4} \cong 0.84 \left( \frac{\varepsilon_s}{e_{k+1}} \right)^{1/4}$$

where  $\varepsilon_s$  is the specified local error tolerance. The scale factor is typically constrained from making changes in  $h$  which are too large or small, for example,  $0.25 \leq s \leq 4$ . The new value of  $h$  is then given by

$$h^{\text{new}} = sh$$

There are several other practical considerations in composing a Python function to carry out the RKF45 algorithm:

1. There will be specific values of the independent variable that have to be determined during the calculations. In this case, if the new value of  $h$  exceeds one of these values, for example, the final value, it must be reset to meet that value.
2. If the function is to be able to integrate one or a set of more than one differential equation, the error criterion needs to be modified to accommodate more than one local error result. A common way to do this would be to base the scale factor,  $s$ , on the norm of those local errors.
3. There should be a provision in the function for passing through additional arguments to the user function that computes the derivative values based on the differential equation(s).
4. On occasion, we want the solution of one or more differential equations to take note of a particular change or “event” and provide a precise result where that event occurs. An example is where a dependent variable reaches a given value. We might also want the option of terminating the solution when that event occurs. More advanced versions of RKF45 have means for accommodating events, as we shall see. The more elementary Python function here does not.
5. There should be consideration of a minimum value for the step size, with an appropriate default value, and based on or somewhat greater than the computer’s precision capability (machine epsilon).

---

Page 564

With these considerations in mind,  **Fig. 23.2** presents the function `rkf45`. The arguments to the function include the name of the function that provides the derivative values, `dydx`, the span argument, `xspan`, which must be an array containing two or more values including the starting value of the independent variable (often zero) and values of the independent variable where the solution will be determined and returned, the initial values of the dependent variable(s), `y0`, a local error tolerance value, `es`, and minimum step size value, `hmin`, both with default values, and a provision, via `*args`, for additional arguments to be passed through to `dydx`.



**FIGURE 23.2** Python function rkf45 to implement the Runge-Kutta-Fehlberg algorithm.

```
def rkf45(dydx,xspan,y0,es=1.e-6,maxit=50,hmin=1.e-15,*args):
    """
    Runge-Kutta-Fehlberg 4/5 algorithm for the
    solution of one or more ODEs
    input:
        dydx = function name that evaluates the derivatives
        xspan = an array of independent variable values
                where the solution will be returned
        y0 = an array initial values of the dependent variable(s)
        es = local relative error tolerance (default = 1.e-10)
        hmin = minimum step size
        *args = additional arguments to be passed to dydt
    output:
        t = array of independent variable values
        y = array of dependent variable values
    """
    # compute all coefficients
    a2 = 0.25 ; a3 = 0.375 ; a4 = 12./13. ; a5 = 1. ; a6 = 0.5
    b21 = 0.25 ;
    b31 = 3./32. ; b32 = 9./32.
    b41 = 1932./2197. ; b42 = -7200./2197. ; b43 = 7296./2197.
    b51 = 439./216. ; b52 = -8. ; b53 = 3680./513. ; b54 = -845./4104.
    b61 = -8./27. ; b62 = 2. ; b63 = -3544./2565. ; b64 = 1859./4104.
    b65 = -11./40.
    c1 = 25./216. ; c3 = 1408./2565. ; c4 = 2197./4101. ; c5 = -0.2
    d1 = 16./135. ; d3 = 6656./12825. ; d4 = 28561./56430.
    d5 = -9./50. ; d6 = 2./55.

    n = len(xspan) # here if tspan contains step times
    x = xspan
    neq = np.size(y0) # determine no. of ODEs
    y = np.zeros((n,neq)) # set up 2-D array for dependent variables
    if neq > 1: # set initial conditions
        y[0,:] = y0[:]
    else:
        y[0] = y0
    cd = -1 # set code to h < hmin exit
    hnew = x[1]-x[0]
    for i in range(n-1): # integrate steps given in tspan
        h = hnew
        xk = x[i] ; yk = y[i,:]
        while True: # while loop until next x[i+1] met
            for k in range(maxit): # for loop to meet tolerance
                if xk + h > x[i+1]: # if necessary, reduce h to
                    h = x[i+1] - xk # meet x[i+1]
                k1 = h*dydx(xk,yk,*args) # compute k factors
                k2 = h*dydx(xk+a2*h,yk+b21*k1,*args)
                k3 = h*dydx(xk+a3*h,yk+b31*k1+b32*k2,*args)
                k4 = h*dydx(xk+a4*h,yk+b41*k1+b42*k2+b43*k3,*args)
                k5 = h*dydx(xk+a5*h,yk+b51*k1+b52*k2+b53*k3+b54*k4,*args)
                k6 = h*dydx(xk+a6*h,yk+b61*k1+b62*k2+b63*k3+b64*k4+b65*k5,*ar
                y4 = yk + c1*k1 + c3*k3 + c4*k4 + c5*k5 # 4th-order
                y5 = yk + d1*k1 + d3*k3 + d4*k4 + d5*k5 + d6*k6 # 5th order
                yerr = y5 - y4
                ynorm = np.linalg.norm(y4)
                # error, perhaps array of errors
                yerrm = np.linalg.norm(yerr)/ynorm # normed error
                s = 1.
                if yerrm != 0:
                    s = 0.84 * (es/yerrm)**0.25 # s factor
                    if s < 0.125: # clamp s factor, if necessary
                        s = 0.125
                    elif s > 4.0:
                        s = 4.0
```

```

        hnew = s*h # adjust h for next iteration
        if yerrm < es: break # check if tolerance met
        if hnew < hmin: return x,y,cd # bail out if hnew < hmin
        h = hnew # set h for next iteration
        if k == maxit-1: # check iteration limit
            cd = 0 # exit with this code if limit reached
            return x,y,cd
        if abs(xk + h - x[i+1]) < hmin: break # check if at x[i+1]
        xk = xk + h ; yk = y4 ; h = hnew # update for next step
        y[i+1,:] = y4 # store y
        cd=1 # set success code value
        return x,y,cd

```

To test the `rkf45` function, we will use [Example 22.6](#) with the ODEs to be solved:

Page 565

$$\frac{dy_1}{dt} = -2y_1^2 + 2y_1 + y_2 - 1 \quad y_1(0) = 2$$

$$\frac{dy_2}{dt} = -y_1 - 3y_2^2 + 2y_2 + 2 \quad y_2(0) = 0$$

The required Python script is

```

def dydtsys(t,y):
    n = len(y)
    dy = np.zeros((n))
    dy[0] = -2.*y[0]**2 + 2.*y[0] + y[1] - 1.
    dy[1] = -y[0] - 3*y[1]**2 + 2.*y[1] + 2.
    return dy

tspan = np.linspace(0.,2.)
y0 = np.array([2.,0.])
t,y,cd = rkf45(dydtsys,tspan,y0)
print(cd)

```

By displaying the `cd` value, 1, we confirm that the solution was successful. Codes of -1 (failed to converge in `maxit` iterations) and 0 (reached minimum step size) would indicate that the algorithm was not successful. The results here, when plotted, replicate [Fig. 22.9](#).

Page 566

Although the `rkf45` is adequate and illustrates the implementation of the Runge-Kutta-Fehlberg, it lacks several features that are part of more sophisticated algorithms, such as the one from the SciPy `integrate` submodule described in the next section.

## 23.1.2 Python Function to Solve IVP ODEs: `solve_ivp` from SciPy `integrate`

There are numerous functions available in the SciPy `integrate` module for the solution of ODEs. Their description is available in SciPy documentation. One that has been popular, and you may find referenced elsewhere, is `odeint` based on the LSODA Fortran routine. A more current and versatile function is `solve_ivp`, which we choose to illustrate here. A simplified version of the syntax for `solve_ivp` is

```
result = solve_ivp(dydt,(ti,tf),y0,method='RK45',t_eval=tspan)
```

where `dydt` is the function that evaluates the derivatives given `t`, `y`, and possible additional arguments, `(ti,tf)` are the initial and final values of the independent variable, `y0` is an array of initial conditions of the dependent variable, `method` defines the algorithm to be used. There are six from which to choose with RK45 as the default. The RK45 method is not quite the RKF algorithm but is similar. The `t_eval` argument specifies the independent variable values where the solution will be returned. There is a possible additional argument, `args=(...)`, to provide parameter values that are passed to `dydt`.

Here is an example of using `solve_ivp` to solve a single ODE with three parameters, `a`, `b`, and `c`, to be passed to the `dydt` function,

$$\frac{dy}{dt} = ae^{bt} - cy \quad 0 \leq t \leq 4 \quad y(0) = 2$$

$$a = 4, b = 0.8, c = 0.5$$

We will request that the solution be reported on intervals of 1.0.

```
import numpy as np
from scipy.integrate import solve_ivp

a = 4.
b = 0.8
c = 0.5

def dydt(t,y,a,b,c):
    return a*np.exp(b*t)-c*y

ti = 0 ; tf = 4.
y0 = [2.]
tspan = np.array([0.,1.,2.,3.,4.])
result = solve_ivp(dydt,(ti,tf),y0,t_eval=tspan,args=(a,b,c))
t = result.t
y = result.y

print(' t  y')
for i in range(5):
    print('{0:4.1f} {1:6.3f}'.format(t[i],y[0,i]))
```

The results displayed are

| t   | y      |
|-----|--------|
| 0.0 | 2.000  |
| 1.0 | 6.195  |
| 2.0 | 14.838 |
| 3.0 | 33.678 |
| 4.0 | 75.345 |

## EXAMPLE 23.1

### Using Python to Solve a System of ODEs

#### Problem Statement

Employ `solve_ivp` to solve the following set of nonlinear ODEs from  $t = 0$  to 20.

$$\frac{dy_1}{dt} = 1.2y_1 - 0.6y_1y_2 \qquad \frac{dy_2}{dt} = -0.8y_2 + 0.3y_1y_2$$

with  $y_1(0) = 2$  and  $y_2(0) = 1$ . Such equations are described as predator-prey descriptions.

## Solution

Before employing the `solve_ivp` function, you must define a function to compute the derivatives above. Even though these expressions do not include  $t$ , the independent variable must be included in the function definition. Here, we use Python's `def` statement:

```
def dydt(t,y):
    n = len(y)
    dy = np.zeros((n))
    dy[0] = 1.2*y[0] - 0.6*y[0]*y[1]
    dy[1] = -0.8*y[1] + 0.3*y[0]*y[1]
    return dy
```

In this case, as an illustration, we have chosen to save this function in a separate file, `Example231dydt.py`.

Next, we create a script in a separate file as

```
import numpy as np
from scipy.integrate import solve_ivp
import pylab
from Example231dydt import dydt

ti = 0. ; tf = 20.
y0 = np.array([2.,1.])
tspan = np.linspace(ti,tf,100)
result = solve_ivp(dydt,(ti,tf),y0,t_eval=tspan)
t = result.t
y = result.y

pylab.plot(t,y[0,:],c='k',label='y1')
pylab.plot(t,y[1,:],c='k',ls='--',label='y2')
pylab.grid()
pylab.xlabel('t')
pylab.ylabel('y')
pylab.legend()
```

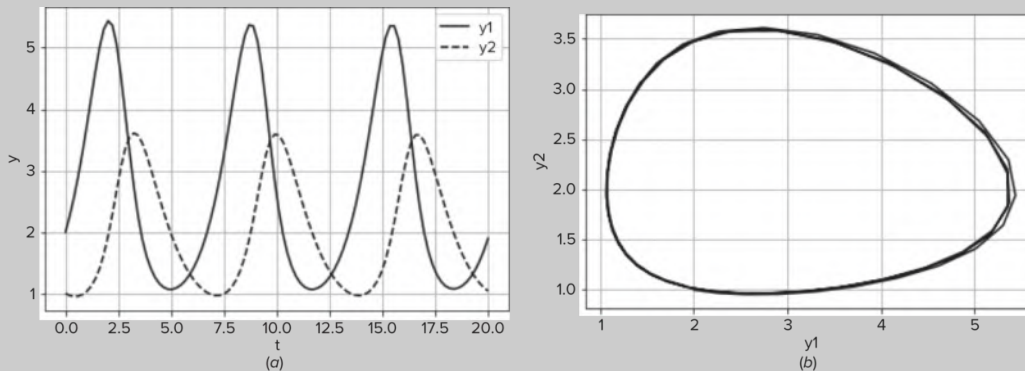
This illustrates how you can compartmentalize functions in separate `py` files. This is important for longer scripts. It provides a modular organization where elements are stored in “bitesize” segments that can, in most cases, be tested and verified separately.

The plot created above is included as  **Fig. 23.3a**. In addition, we can create a phase plot of  $y_2$  versus  $y_1$  by adding the code

```
pylab.figure()
pylab.plot(y[0,:],y[1,:],c='k')
pylab.grid()
pylab.xlabel('y1')
pylab.ylabel('y2')
```

---

**FIGURE 23.3** (a) Solution of predator-prey model with Python's `solve_ivp` function. (b) Phase-plane plot of predator-prey model.



The resulting plot is shown in [Fig. 23.3](#).

There are additional parameters for `solve_ivp` that control various aspects of the integration. Page 568  
Four of particular interest are

|                         |                           |
|-------------------------|---------------------------|
| <code>rtol</code>       | relative error tolerance  |
| <code>atol</code>       | absolute error tolerance  |
| <code>first_step</code> | initial step size         |
| <code>max_step</code>   | maximum allowed step size |

The solver maintains the local error less than  $\text{atol} + \text{rtol} * \text{abs}(y)$ .

## EXAMPLE 23.2

### Setting Parameters to Control Integration Options

#### Problem Statement

Use the RK23 method and `solve_ivp` to solve the following ODE from  $t = 0$  to 4:

$$\frac{dy}{dt} = 10e^{-(t-2)^2/(2 \cdot 0.075^2)} - 0.6y \quad y(0) = 0.5$$

Obtain solutions for the default relative error tolerance ( $10^{-3}$ ) and for a more stringent value ( $10^{-5}$ ). Compare the results.

#### Solution

First, we will create a Python function to compute the right-hand side of the ODE:

```
dydt = lambda t,y: 10.*np.exp(-(t-2.)**2/(2*0.075**2)) - 0.6*y
```

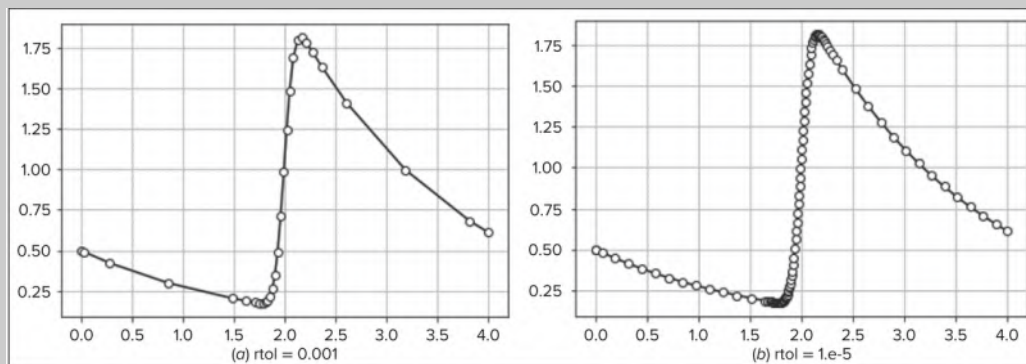
Then, we invoke the `solve_ivp` function without setting any special parameters.

```
ti = 0.; tf = 4.  
tspan = np.linspace(0.,4.,100)  
y0 = [0.5]  
  
res = solve_ivp(dydt,(ti,tf),y0,method='RK23')  
t = res.t  
y = res.y[0,:]
```

Note that we have not specified the `t_eval` argument. In this case, the RK23 routine selects its own points for returning the solution in proportion to the number of steps required. We create a plot, displayed in [Fig. 23.4a](#), with the code

```
pylab.plot(t,y,c='k',marker='o',markerfacecolor='w')  
pylab.grid()  
pylab.xlabel('a') rtol = 0.001')
```

**FIGURE 23.4** Solution of an ODE with Python's `solve_ivp` function. For (b), a relative tolerance 100 times smaller is used and hence many more steps are taken.



Subsequently, we solve the ODE again, but with `rtol` set equal to  $10^{-5}$ .

```
res = solve_ivp(dydt,(ti,tf),y0,method='RK23',rtol=1.e-5)  
t1 = res.t  
y1 = res.y[0,:]
```

The resulting plot is displayed in [Fig. 23.4b](#). Notice that more points are displayed as an indicator that the solver used smaller step sizes to achieve the more stringent `rtol` tolerance.

## 23.1.3 Events

Python's ODE solvers, including `solve_ivp`, are commonly implemented for a prespecified integration interval. That is, they are most often used to obtain a solution from an initial to a final value of the independent variable. However, there are many problems where we do not know the final time of the integration.

A nice example relates to the free-falling bungee jumper that we have been using throughout this book. Suppose that the jump master inadvertently neglects to attach the cord to the jumper. The final time for this case, which corresponds unfortunately to the jumper hitting the ground, is not given. In fact, the objective of solving the ODEs would be to determine the time at which this occurred.

The `solve_ivp` function provides an `events` argument as a means to solve such problems, and others where abrupt changes take place during a simulation. It works by testing a condition to see if it is zero or has had a sign change over an integration step. The solver will then find an accurate value of the independent variable at which the zero occurs. There are two options which may be set

- `terminal`            the integration will end if this is set True (default False)
- `direction`           if +1, the event will only trigger when going from negative to positive, and vice versa for -1.

In order to illustrate the application of the events feature, we will use our bungee jumper problem. The system of ODEs is formulated as

$$\begin{aligned}\frac{dx}{dt} &= v \\ \frac{dv}{dt} &= g - \frac{c_d}{m}|v|\end{aligned}$$

where  $x$  = distance (m),  $t$  = time (s),  $v$  = velocity (m/s) where positive velocity is in the downward direction,  $c_d$  = a second-order drag coefficient (kg/m), and  $m$  = mass (kg). Note that, in this formulation, distance and velocity are both positive in the downward direction, and ground level is defined as  $x = 0$ .

For the present example, we will assume that the jumper is initially located 200 m above the ground, and the initial velocity is 20 m/s in the upward direction, that is,  $x(0) = -200$  m and  $v(0) = -20$  m/s. The first step is to formulate a function that defines the ODEs:

---

Page 570

---

```
def dydt(t,y,cd,m):
    v = y[1]
    dy = np.zeros((2))
    dy[0] = v
    dy[1] = g - cd/m*v*abs(v)
    return dy
```

In order to implement the event, we must create a separate function where the event will be triggered when returned value reaches or crosses zero. The solver will then attempt to adjust the step size so that the zero condition is met precisely.

```
def hit_ground(t,y,cd,m):
    return y[0]
```

Here, we merely indicate `y[0]`, representing the elevation variable  $x$ , because we are interested when the ground is met at  $x = 0$ . Additionally, we set the `terminal` attribute of the event to True with the statement

```
hit_ground.terminal = True
```

This is so that the integration will stop when the condition is met. Finally, we complete the Python script to carry out the simulation and produce a plot.

```
y0 = np.array([-200, -20])
ti = 0 ; tf = math.inf
result = solve_ivp(dydt,(ti,tf),y0,rtol=1.e-8, \
                  events=hit_ground,args=(cd,m))

t = result.t
y = result.y
x = y[0,:]
v = y[1,:]
n = len(t)
print(t[n-1],x[n-1],v[n-1])

pylab.plot(t,-x,c='k',label='height - m')
pylab.plot(t,v,c='k',ls='--',label='velocity - m/s')
pylab.grid()
pylab.xlabel('time -s')
pylab.legend()
```

For completeness, the following `import` commands are located at the top of the script:

```
import numpy as np
import math
from scipy.integrate import solve_ivp
import pylab
```

You will note that we set the final time, `tf`, as  $\infty$  using the `inf` value from the `math` module. We do this because we expect the integration to be stopped by the event. The event function name, `hit_ground`, is included in the arguments to `solve_ivp`. The relative tolerance is set small,  $1 \times 10^{-8}$ , solely to have the solver save more points during the integration for plotting. We mention that setting the `t_eval` argument with time values will interfere with the `events` action, so we leave that out.

The display produced when the script is run is

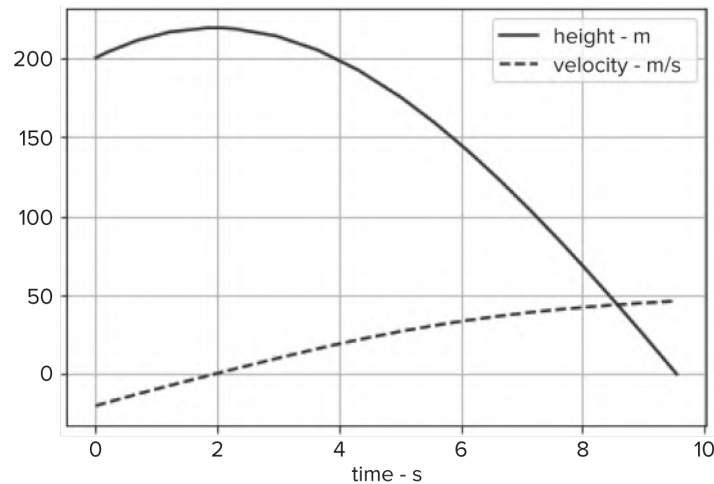
```
9.548027838470393 -2.6645352591003757e-14 46.227508170602206
```

The ground is met at ~9.548 seconds. The second result demonstrates how close the solver achieved zero elevation—very close. And the third result is the velocity at impact, about 46.23 m/s. The plot is shown in [Fig. 23.5](#) and demonstrates how the event controls the termination. Note how the sign of the elevation is reversed for the plot.

---

**FIGURE 23.5** Python plot of the elevation above ground and velocity of the free-falling bungee jumper without a cord.





We also might be interested in finding the apex of the bungee jumper's trajectory, shown as the maximum in the height curve in [Fig. 23.5](#) and the zero crossover of the velocity curve. The latter provides the opportunity to define another event

```
def apex(t,y,cd,m):
    return y[1]
```

and includes that in the `events` argument of the `solve_ivp` function:

```
result = solve_ivp(dydt,(ti,tf),y0,rtol=1.e-8, \
    events=(hit_ground,apex),args=(cd,m))
```

We do not include a terminal condition for the apex event but retain it for the `hit_ground` event. In order to find the event values when they were triggered, we can include the command

```
print(result.y_events)
```

which provides the display

```
[array([[ -2.66453526e-14,  4.62275082e+01]]), array([[ -2.18998493e+02,
  1.11022302e-15]])]
```

The first pair of values replicates our findings for the `hit_ground` event; whereas the second pair shows the elevation of ~219 m above the ground at the apex. The times when the events occurred are not displayed; however, if we include the statement

```
print(result.t_events)
```

the following is added to our output:

```
[array([9.54802784]), array([1.94527347])]
```

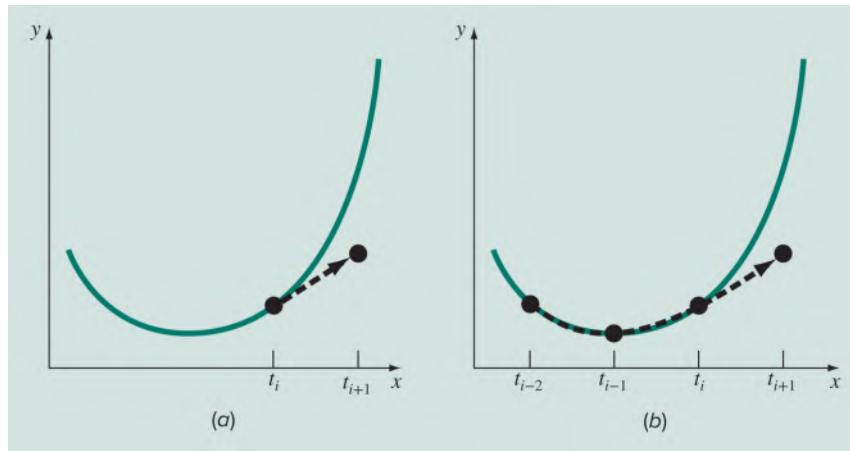
We can here observe that the apex is reached at 1.945 s.

Through creative use of the events functions, a variety of conditions can be noted during a simulation. One issue that arises in solving ODEs is the occurrence of discontinuities, such as a solenoid flow valve being “snapped” open or shut or an electrical heater being turned on or off. An adaptive solver, such as `solve_ivp`, cannot handle such changes well because it will reduce step size to the minimum to attempt to find a continuous solution with minimal error. The inclusion of the events feature will accommodate such changes.

## 23.2 MULTISTEP METHODS

The one-step methods described in the previous sections utilize information at a single point  $t_i$  to predict a value of the dependent variable  $y_{i+1}$  at a future point  $t_{i+1}$  (Fig. 23.6a). Alternative approaches, called *multistep methods* (Fig. 23.6b), are based on the insight that, once the computation has begun, valuable information from previous points is at our command. The curvature of the lines connecting these previous values provides information regarding the trajectory of the solution. Multistep methods exploit this information to solve ODEs. In this section, we will present a simple second-order method that serves to demonstrate the general characteristics of multistep approaches.

**FIGURE 23.6** Graphical depiction of the fundamental difference between (a) one-step and (b) multistep methods for solving ODEs.



### 23.2.1 The Non-Self-Starting Heun Method

Recall that the Heun approach uses Euler's method as a predictor [Eq. (22.15)]:

$$y_{i+1}^0 = y_i + f(t_i, y_i)h \quad (23.4)$$

and the trapezoidal rule as a corrector [Eq. (22.17)]:

$$y_{i+1} = y_i + \frac{f(t_i, y_i) + f(t_{i+1}, y_{i+1}^0)}{2}h \quad (23.5)$$

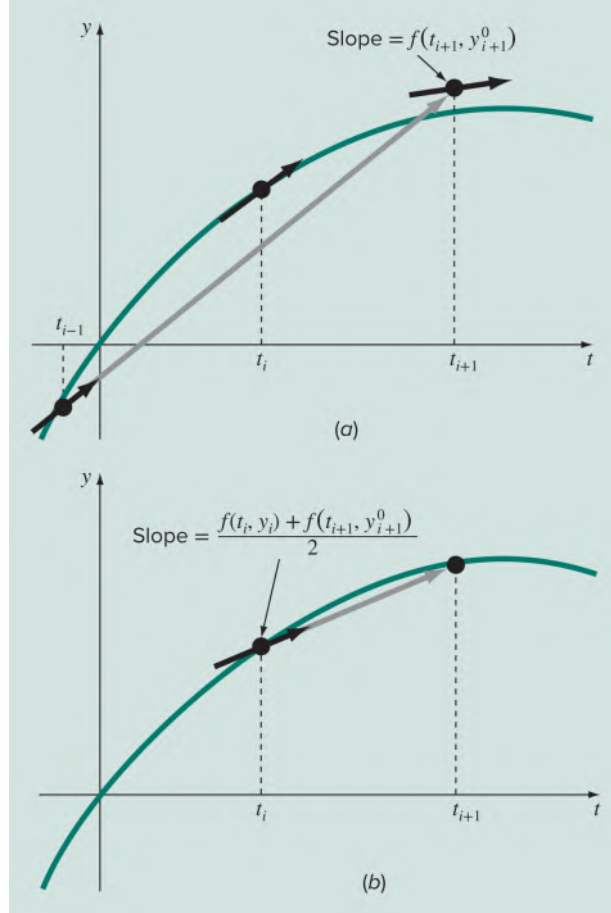
Thus, the predictor and the corrector have local truncation errors of  $\mathcal{O}(h^2)$  and  $\mathcal{O}(h^3)$ , respectively. This suggests that the predictor is the weak link in the method because it has the greatest error. This weakness is significant because the efficiency of the iterative corrector step depends on the accuracy of the initial prediction. Consequently, one way to improve Heun's method is to develop a predictor that has a local error of  $\mathcal{O}(h^3)$ . This can be accomplished by using Euler's method and the slope at  $y_i$ , and extra information from a previous point  $y_{i-1}$ , as in

$$y_{i+1}^0 = y_{i-1} + f(t_i, y_i)2h \quad (23.6)$$

This formula attains  $\mathcal{O}(h^3)$  at the expense of employing a larger step size  $2h$ . In addition, note that the equation is not self-starting because it involves a previous value of the dependent variable  $y_{i-1}$ . Such a value would not be available in a typical initial-value problem. Because of this fact, [Eqs. \(23.5\)](#) and [\(23.6\)](#) are called the *non-self-starting Heun method*. As depicted in [Fig. 23.7](#), the derivative estimate in [Eq. \(23.6\)](#) is now located at the midpoint rather than at the beginning of the interval over which the prediction is made. This centering improves the local error of the predictor to  $\mathcal{O}(h^3)$ .

---

**FIGURE 23.7** A graphical depiction of the non-self-starting Heun method. (a) The midpoint method that is used as a predictor. (b) The trapezoidal rule that is employed as a corrector.



The non-self-starting Heun method can be summarized as

Predictor (  Fig. 23.7a): 
$$y_{i+1}^0 = y_i^m + f(t_i, y_i^m)2h \quad (23.7)$$

Corrector (  Fig. 23.7b): 
$$y_{i+1}^j = y_i^m + \frac{f(t_i, y_i^m) + f(t_{i+1}, y_{i+1}^{j-1})}{2}h \quad (23.8)$$
  
(for  $j = 1, 2, \dots, m$ )

where the superscripts denote that the corrector is applied iteratively from  $j = 1$  to  $m$  to obtain refined solutions. Note that  $y_i^m$  and  $y_{i-1}^m$  are the final results of the corrector iterations at the previous time steps. The iterations are terminated on the basis of an estimate of the approximate error,

$$|\varepsilon_a| = \left| \frac{y_{i+1}^j - y_{i+1}^{j-1}}{y_{i+1}^j} \right| \times 100\% \quad (23.9)$$

When  $|\varepsilon_a|$  is less than a prespecified error tolerance  $\varepsilon_s$ , the iterations are terminated. At this point,  $j = m$ . The use of [Eq. \(23.7\)](#) through [Eq. \(23.9\)](#) to solve an ODE is demonstrated in the following example.

## EXAMPLE 23.3

### Non-Self-Starting Heun's Method

#### Problem Statement

Use the non-self-starting Heun method to perform the same computations as were performed previously in [Example 22.2](#) using Heun's method. That is, integrate  $y' = 4e^{0.8t} - 0.5y$  from  $t = 0$  to 4 with a step size of 1. As with [Example 22.2](#), the initial condition at  $t = 0$  is  $y = 2$ . However, because we are now dealing with a multistep method, we require the additional information that  $y$  is equal to  $-0.3929953$  at  $t = -1$ .

Page 574

#### Solution

The predictor [[Eq. \(23.7\)](#)] is used to extrapolate linearly from  $t = -1$  to 1:

$$y_1^0 = -0.3929953 + [4e^{0.8(0)} - 0.5(2)]2 = 5.607005$$

The corrector [[Eq. \(23.8\)](#)] is then used to compute the value:

$$y_1^1 = 2 + \frac{4e^{0.8(0)} - 0.5(2) + 4e^{0.8(1)} - 0.5(5.607005)}{2}1 = 6.549331$$

which represents a true percent relative error of  $-5.73\%$  (true value = 6.194631). This error is somewhat smaller than the value of  $-8.18\%$  incurred in the self-starting Heun.

Now, [Eq. \(23.8\)](#) can be applied iteratively to improve the solution:

$$y_1^2 = 2 + \frac{3 + 4e^{0.8(1)} - 0.5(6.549331)}{2}1 = 6.313749$$

which represents an error of  $-1.92\%$ . An approximate estimate of the error can be determined using [Eq. \(23.9\)](#):

$$|\varepsilon_a| = \left| \frac{6.313749 - 6.549331}{6.313749} \right| \times 100\% = 3.7\%$$

 **Equation (23.8)** can be applied iteratively until  $\varepsilon_a$  falls below a prespecified value of  $\varepsilon_s$ . As was the case with the Heun method (recall  **Example 22.2**), the iterations converge on a value of 6.36087 ( $\varepsilon_t = -2.68\%$ ). However, because the initial predictor value is more accurate, the multistep method converges at a somewhat faster rate.

For the second step, the predictor is

$$y_2^0 = 2 + \left[ 4e^{0.8(1)} - 0.5(6.36087) \right] 2 = 13.44346 \quad \varepsilon_t = 9.43\%$$

which is superior to the prediction of 12.0826 ( $\varepsilon_t = 18\%$ ) that was computed with the original Heun method. The first corrector yields 15.76693 ( $\varepsilon_t = 6.8\%$ ), and subsequent iterations converge on the same result as was obtained with the self-starting Heun method: 15.30224 ( $\varepsilon_t = -3.09\%$ ). As with the previous step, the rate of convergence of the corrector is somewhat improved because of the better initial prediction.

## 23.2.2 Error Estimates

Aside from providing increased efficiency, the non-self-starting Heun can also be used to estimate the local truncation error. As with the adaptive RK methods in  **Sec. 23.1**, the error estimate then provides a criterion for changing the step size.

The error estimate can be derived by recognizing that the predictor is equivalent to the midpoint rule. Hence, its local truncation error is ( **Table 19.4**)

$$E_p = \frac{1}{3}h^3 y^{(3)}(\xi_p) = \frac{1}{3}h^3 f''(\xi_p) \quad (23.10)$$

where the subscript  $p$  designates that this is the error of the predictor. This error estimate can be combined with the estimate of  $y_{i+1}$  from the predictor step to yield

$$\text{True value} = y_{i+1}^0 + \frac{1}{3}h^3 y^{(3)}(\xi_p) \quad (23.11)$$

By recognizing that the corrector is equivalent to the trapezoidal rule, a similar estimate of the local truncation error for the corrector is ( **Table 19.2**)

$$E_c = -\frac{1}{12}h^3 y^{(3)}(\xi_c) = -\frac{1}{12}h^3 f''(\xi_c) \quad (23.12)$$

This error estimate can be combined with the corrector result  $y_{i+1}$  to give

$$\text{True value} = y_{i+1}^m - \frac{1}{12}h^3y^{(3)}(\xi_c) \quad (23.13)$$

Equation (23.11) can be subtracted from Eq. (23.13) to yield

$$0 = y_{i+1}^m - y_{i+1}^0 - \frac{5}{12}h^3y^{(3)}(\xi) \quad (23.14)$$

where  $\xi$  is now between  $t_{i-1}$  and  $t_i$ . Now, dividing Eq. (23.14) by 5 and rearranging the result give

$$\frac{y_{i+1}^0 - y_{i+1}^m}{5} = -\frac{1}{12}h^3y^{(3)}(\xi) \quad (23.15)$$

Notice that the right-hand sides of Eqs. (23.12) and (23.15) are identical, with the exception of the argument of the third derivative. If the third derivative does not vary appreciably over the interval in question, we can assume that the right-hand sides are equal, and therefore, the left-hand sides should also be equivalent, as in

$$E_c = -\frac{y_{i+1}^0 - y_{i+1}^m}{5} \quad (23.16)$$

Thus, we have arrived at a relationship that can be used to estimate the per-step truncation error on the basis of two quantities that are routine by-products of the computation: the predictor ( $y_{i+1}^0$ ) and the corrector ( $y_{i+1}^m$ ).

## EXAMPLE 23.4

### Estimate of Per-Step Truncation Error

#### Problem Statement

Use Eq. (23.16) to estimate the per-step truncation error of Example 23.3. Note that the true values at  $t = 1$  and  $2$  are  $6.194631$  and  $14.84392$ , respectively.

#### Solution

At  $t_{i+1} = 1$ , the predictor gives 5.607005 and the corrector yields 6.360865. These values can be substituted into  Eq. (23.16) to give

$$E_c = -\frac{6.360865 - 5.607005}{5} = -0.150722$$

which compares well with the exact error,

$$E_t = 6.194631 - 6.360865 = -0.1662341$$

At  $t_{i+1} = 2$ , the predictor gives 13.44346 and the corrector yields 15.30224, which can be used to compute

$$E_c = -\frac{15.30224 - 13.44346}{5} = -0.37176$$

which also compares favorably with the exact error,  $E_t = 14.84392 - 15.30224 = -0.45831$ .

The foregoing has been a brief introduction to multistep methods. Additional information can be found elsewhere (e.g., Chapra and Canale, 2010). Although they still have their place for solving certain types of problems, multistep methods are usually not the method of choice for most problems routinely confronted in engineering and science. That said, they are still used. We have therefore included this section to introduce you to their basic principles.



## 23.3 STIFFNESS

Stiffness is a special problem that can arise in the solution of ordinary differential equations. A *stiff system* is one involving rapidly changing components together with slowly changing ones. In some cases, the rapidly varying components are ephemeral transients that die away quickly, after which the solution becomes dominated by the slowly varying components. Although the transient phenomena exist for only a short part of the integration interval, they can dictate the time step for the entire solution.

Both individual and systems of ODEs can be stiff. An example of a single stiff ODE is

$$\frac{dy}{dt} = -1000y + 3000 - 2000e^{-t} \quad (23.17)$$

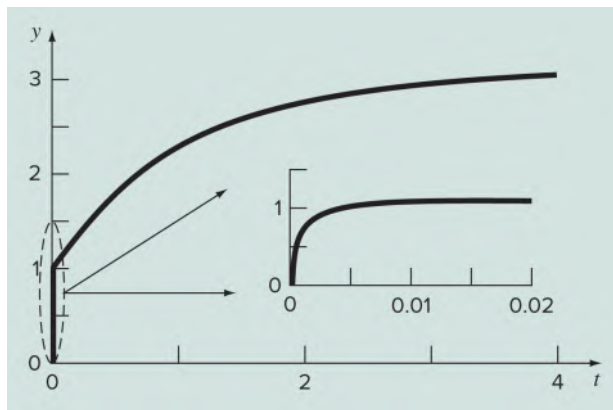
If  $y(0) = 0$ , the analytical solution can be developed as

$$y = 3 - 0.998e^{-1000t} - 2.002e^{-t} \quad (23.18)$$

As in [Fig. 23.8](#), the solution is initially dominated by the fast exponential term ( $e^{-1000t}$ ). After a short period ( $t < 0.005$ ), this transient dies out and the solution becomes governed by the slow exponential ( $e^{-t}$ ).

Page 576

**FIGURE 23.8** Plot of a stiff solution of a single ODE. Although the solution appears to start at 1, there is actually a fast transient from  $y = 0$  to 1 that occurs in less than the 0.005 time unit. This transient is perceptible only when the response is viewed on the finer timescale in the inset.



Insight into the step size required for stability of such a solution can be gained by examining the homogeneous part of [Eq. \(23.17\)](#):

$$\frac{dy}{dt} = -ay \quad (23.19)$$

If  $y(0) = y_0$ , calculus can be used to determine the solution as

$$y = y_0 e^{-at}$$

Thus, the solution starts at  $y_0$  and asymptotically approaches zero.

Euler's method can be used to solve the same problem numerically:

$$y_{i+1} = y_i + \frac{dy_i}{dt} h$$

Substituting  Eq. (23.19) gives

$$y_{i+1} = y_i - ay_i h$$

or

$$y_{i+1} = y_i(1 - ah) \quad (23.20)$$

The stability of this formula clearly depends on the step size  $h$ . That is,  $|1 - ah|$  must be less than 1. Thus, if  $h > 2/a$ ,  $|y_i| \rightarrow \infty$  as  $i \rightarrow \infty$ .

For the fast transient part of  Eq. (23.18), this criterion can be used to show that the step size to maintain stability must be  $< 2/1000 = 0.002$ . In addition, we should note that, whereas this criterion maintains stability (i.e., a bounded solution), an even smaller step size would be required to obtain an accurate solution. Thus, although the transient occurs for only a small fraction of the integration interval, it controls the maximum allowable step size.

Rather than using explicit approaches, implicit methods offer an alternative remedy. Such representations are called *implicit* because the unknown appears on both sides of the equation. An implicit form of Euler's method can be developed by evaluating the derivative at the future time:

$$y_{i+1} = y_i + \frac{dy_{i+1}}{dt} h$$

This is called the *backward*, or *implicit*, Euler's method. Substituting  Eq. (23.19) yields

$$y_{i+1} = y_i - ay_{i+1} h$$

which can be solved for

$$y_{i+1} = \frac{y_i}{1 + ah} \quad (23.21)$$

For this case, regardless of the size of the step,  $|y_i| \rightarrow 0$  as  $i \rightarrow \infty$ . Hence, the approach is called *unconditionally stable*.

## EXAMPLE 23.5

### Explicit and Implicit Euler

#### Problem Statement

Use both the explicit and implicit Euler methods to solve  Eq. (23.17), where  $y(0) = 0$ . **(a)** Use the explicit Euler with step sizes of 0.0005 and 0.0015 to solve for  $y$  between  $t = 0$  and 0.006. **(b)** Use the implicit Euler with a step size of 0.05 to solve for  $y$  between 0 and 0.4.

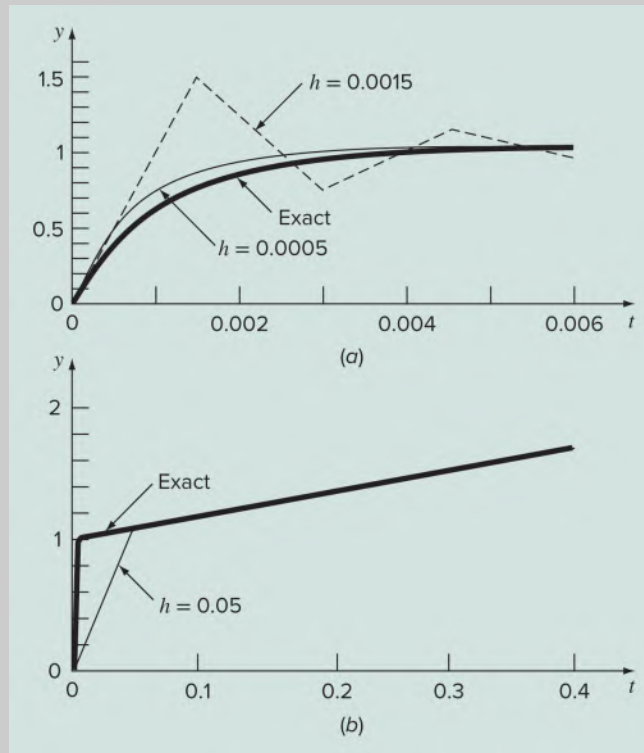
#### Solution

**(a)** For this problem, the explicit Euler's method is

$$y_{i+1} = y_i + (-1000y_i + 3000 - 2000e^{-t_i})h$$

The result for  $h = 0.0005$  is displayed in  Fig. 23.9a along with the analytical solution. Although it exhibits some truncation error, the result captures the general shape of the analytical solution. In contrast, when the step size is increased to a value just below the stability limit ( $h = 0.0015$ ), the solution manifests oscillations. Using  $h > 0.002$  would result in a totally unstable solution—that is, it would go infinite as the solution progressed.

**FIGURE 23.9** Solution of a stiff ODE with (a) the explicit and (b) implicit Euler methods.



(b) The implicit Euler's method is

$$y_{i+1} = y_i + (-1000y_{i+1} + 3000 - 2000e^{-t_{i+1}})h$$

Now because the ODE is linear, we can rearrange this equation so that  $y_{i+1}$  is isolated on the left-hand side:

$$y_{i+1} = \frac{y_i + 3000h - 2000he^{-t_{i+1}}}{1 + 1000h}$$

The result for  $h = 0.05$  is displayed in  Fig. 23.9b along with the analytical solution.

Page 578

Notice that even though we have used a much bigger step size than the one that induced instability for the explicit Euler, the numerical result tracks nicely on the analytical solution.

Systems of ODEs can also be stiff. An example is

$$(23.22a)$$

$$(23.22b)$$

$$\begin{aligned}\frac{dy_1}{dt} &= -5y_1 + 3y_2 \\ \frac{dy_2}{dt} &= 100y_1 - 301y_2\end{aligned}$$

For the initial conditions  $y_1(0) = 52.29$  and  $y_2(0) = 83.82$ , the exact solution is

$$y_1 = 52.96e^{-3.9899t} - 0.67e^{-302.0101t} \quad (23.23a)$$

$$y_2 = 17.83e^{-3.9899t} + 65.99e^{-302.0101t} \quad (23.23b)$$

Note that the exponents are negative and differ by about two orders of magnitude. As with the single equation, it is the large exponents that respond rapidly and are at the heart of the system's stiffness.

An implicit Euler's method for systems can be formulated for the present example as

$$y_{1,i+1} = y_{1,i} + (-5y_{1,i+1} + 3y_{2,i+1})h \quad (23.24a)$$

$$y_{2,i+1} = y_{2,i} + (100y_{1,i+1} - 301y_{2,i+1})h \quad (23.24b)$$

Collecting terms gives

$$(1 + 5h)y_{1,i+1} - 3y_{2,i+1} = y_{1,i} \quad (23.25a)$$

$$-100y_{1,i+1} + (1 + 301h)y_{2,i+1} = y_{2,i} \quad (23.25b)$$

Thus, we can see that the problem consists of solving a set of simultaneous equations for each time step.

For nonlinear ODEs, the solution becomes even more difficult since it involves solving a system of nonlinear simultaneous equations (recall [Sec. 12.2](#)). Thus, although stability is gained through implicit approaches, a price is paid in the form of added solution complexity.

## 23.3.1 Python ODE Solvers for Stiff Systems

Python provides several functions and options via the SciPy `integrate` module that are effective for solving stiff systems of ODEs. Here is a summary:

|                        |                                                                                                                                                                                                                                                             |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>solve_ivp</code> | By specifying "LSODA" for the method argument, an Adams/BDF method with automatic stiffness detection and algorithm switching is implemented. This uses an algorithm sourced from the Fortran solver in ODEPACK (Hindmarsh, Lawrence Livermore Laboratory). |
| LSODA                  | This is a function that provides the LSODA algorithm directly.                                                                                                                                                                                              |

|                     |                                                                                                                                                    |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>odeint</code> | This function is an earlier version with its algorithm adapted from ODEPACK.                                                                       |
| <code>ode</code>    | Several methods are available from the Variable-Order ODE solver (VODE) with original source also from Hindmarsh at Lawrence Livermore Laboratory. |

We would generally suggest using the first two of these because they represent the most current versions.

## EXAMPLE 23.6

### Solving Stiff Systems with `solve_ivp/LSODA`

#### Problem Statement

The *van der Pol equation* is a model of an electronic circuit that arose back in the days of vacuum tubes,

$$\frac{d^2 y_1}{dt^2} - \mu(1 - y_1^2) \frac{dy_1}{dt} + y_1 = 0 \quad (\text{E23.6.1})$$

The solution to this equation becomes progressively stiffer as  $\mu$  becomes large. Given the Page 579 initial conditions,  $y_1(0) = 1$  and  $dy_1/dt = 1$ , use Python to solve the following two cases: **(a)** for  $\mu = 1$ , use the default RK45 method in `solve_ivp` to solve from  $t = 0$  to 20, and **(b)** for  $\mu = 1000$ , use the LSODA method to solve from  $t = 0$  to 6000.

#### Solution

**(a)** The first step is to convert the second-order ODE into a pair of first-order ODEs by defining

$$\frac{dy_1}{dt} = y_2$$

Using this definition of  $y_2$ ,  Eq. (E23.6.1) can be written as

$$\frac{dy_2}{dt} = \mu(1 - y_1^2)y_2 - y_1$$

A Python function can now be created to compute the derivative values based on these two equations:

```
def dydt(t,y,mu):
    dy = np.zeros((2))
    dy[0] = y[1]
```

```
dy[1] = mu*(1-y[0]**2)*y[1]-y[0]
```

Notice how the value of  $\mu$  as a parameter is passed to the function as an argument. We now set the time and initial conditions to invoke `solve_ivp` followed by extracting the solution and creating a plot.

```
mu = 1.
ti = 0.; tf = 20.
tspan = np.linspace(ti,tf,100)
y0 = np.array([1.,1.])
soln = solve_ivp(dydt,(ti,tf),y0,t_eval=tspan,args=(mu,))
# method RK45 is the default

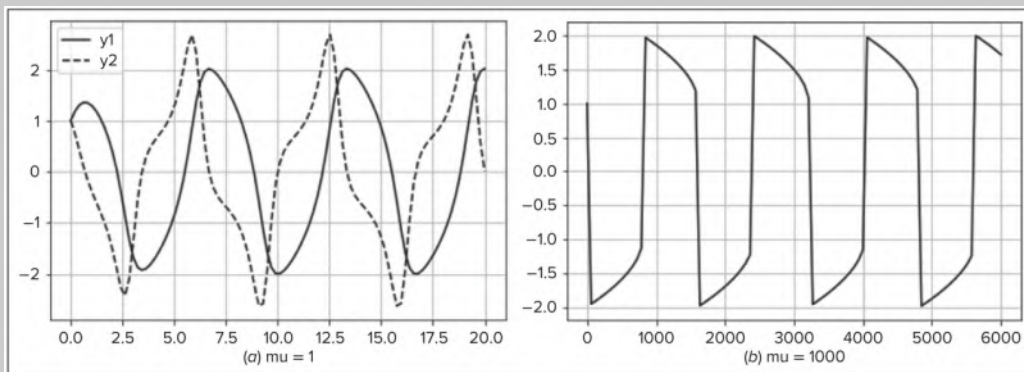
t = soln.t
y = soln.y

pylab.plot(t,y[0,:],c='k',label='y1')
pylab.plot(t,y[1,:],c='k',ls='--',label='y2')
pylab.grid()
pylab.xlabel('a) mu = 1')
pylab.legend()
```

To be fair, we should point out the `args` specification of `(mu,)` for the single extra argument. That is picky, but `(mu)` won't work. Sorry, but don't blame the authors.

The relatively smooth nature of the plot ([Fig. 23.10a](#)) suggests that the van der Pol equation with  $\mu = 1$  is not a stiff system.

**FIGURE 23.10** Solutions for van der Pol's equation. (a) Non-stiff version solved with RK45 method and (b) Stiff version solved with LSODA method.



**(b)** If the default RK45 method is used for the stiff case ( $\mu = 1000$ ), it will fail to complete the solution. The stiffness hangs it up. However, if we specify the LSODA method, an accurate solution is produced rapidly.

```
mu = 1000.
ti = 0.; tf = 6000.
tspan = np.linspace(ti,tf,100)
y0 = np.array([1.,1.])
soln = solve_ivp(dydt,(ti,tf),y0,t_eval=tspan,method='LSODA',args=(mu,))

t = soln.t
y = soln.y

pylab.figure()
```

```
pylab.plot(t,y[0,:],c='k')
pylab.grid()
pylab.xlabel('a mu = 1000')
```

Page 580

We choose to display only the  $y_1$  component here because the  $y_2$  component has a much smaller scale and has dynamics that are much faster than  $y_1$ . It might be displayed on the right axis, but we have not chosen to do that here. Notice how this solution ( [Fig. 23.10b](#)) has much sharper transitions than is the case with the previous case. This characteristic accompanied by the difference in the dynamic response times of the two variables is a manifestation of the “stiffness” of this second case.



## 23.4 PYTHON APPLICATION: BUNGEE JUMPER WITH CORD

In this section, we consider the differential equation model describing the vertical dynamics of a jumper connected to a stationary platform with a bungee cord. As developed at the beginning of [Chap. 22](#), the problem consisted of solving two coupled differential equations for vertical position and velocity. The ODE for position is

$$\frac{dx}{dt} = v \quad (23.26)$$

The differential equation for velocity is different depending on whether the bungee jumper has fallen to a distance where the cord is fully extended, under tension, and begins to stretch. Thus, if the distance fallen is less than the cord length, the jumper is only subject to gravitational and drag forces,

$$\frac{dv}{dt} = g - \text{sign}(v) \frac{c_d}{m} v^2 \quad (23.27a)$$

Once the cord begins to stretch, the spring and dampening forces of the cord must also be included:

$$\frac{dv}{dt} = g - \text{sign}(v) \frac{c_d}{m} v^2 - \frac{k}{m} (x - L) - \frac{\gamma}{m} v \quad (23.27b)$$

The following example shows how Python's `solve_ivp` function can be used to simulate this behavior.

### EXAMPLE 23.7

#### Bungee Jumper with Cord

##### Problem Statement

Determine the position and velocity of a bungee jumper with the following parameters:  $L = 30$  m,  $m = 68.1$  kg,  $c_d = 0.25$  kg/m,  $k = 40$  N/m, and  $\gamma = 8$  N·s/m. Perform the computation from  $t = 0$  to 50 and assume zero initial conditions.

Page 581

##### Solution

The following function can be set up to compute the derivative values based on the three differential equations above:

```
def dydt(t,y,L,cd,m,k,gamma):
    x = y[0] ; v = y[1]
    dy = np.zeros((2))
    dy[0] = v
    if x < L:
        dy[1] = g - np.sign(v)*cd/m*v**2
    else:
        dy[1] = g - np.sign(v)*cd/m*v**2 - k/m*(x-L) - gamma/m*v
    return dy
```

Because these two ODEs are not stiff, we can use the default RK45 method for solution with `solve_ivp` and display the results in a plot:

```
import numpy as np
from scipy.integrate import solve_ivp
import pylab

g = 9.81 # m/s2
L = 30. # m
m = 68.1 # kg
cd = 0.25 # kg/m
k = 40. # N/m
gamma = 8. # N*s/m

ti = 0. ; tf = 50. # s
y0 = np.array([0.,0.])
tspan = np.linspace(ti,tf,100)

res = solve_ivp(dydt,(ti,tf),y0,t_eval=tspan,args=(L,cd,m,k,gamma))

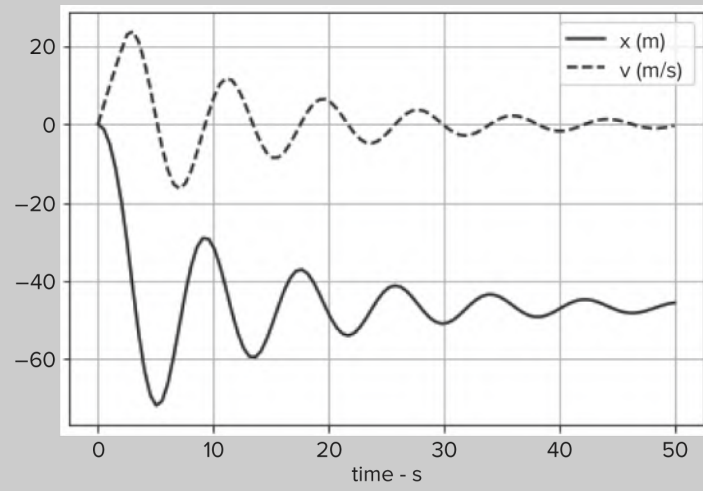
t = res.t
x = res.y[0,:]
v = res.y[1,:]

pylab.plot(t,-x,c='k',label='x (m)')
pylab.plot(t,v,c='k',ls='--',label='v (m/s)')
pylab.grid()
pylab.xlabel('time - s')
pylab.legend()
```

As shown in [Fig. 23.11](#), we have reversed the sign of distance for the plot so that negative distance is in the downward direction. Notice how the simulation captures the jumper's bouncing motion. If we wanted to track the times when there is a transition from slack to tense cord, we could use an event function with  $L-x$  (or  $L - y[0]$ ) as the return.

---

**FIGURE 23.11** Plot of distance and velocity of a bungee jumper with cord.



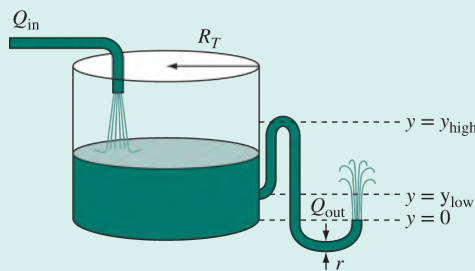
## 23.5 CASE STUDY

### PLINY'S INTERMITTENT FOUNTAIN

#### Background

The Roman natural philosopher, *Pliny the Elder*, purportedly had an intermittent fountain in his garden. As depicted in [Fig. 23.12](#), water enters a cylindrical tank at a constant flow rate,  $Q_{\text{in}}$ , and fills until the water reaches a level,  $y_{\text{high}}$ . At this point, water siphons out of the tank through a discharge pipe, producing a fountain spray at the pipe's exit. The fountain runs until the water level decreases to a level,  $y_{\text{low}}$ , whereupon the siphon fills with air, and the fountain stops. The cycle then repeats as the tank fills until the water once again reaches  $y_{\text{high}}$ , and the fountain flows again.

**FIGURE 23.12** An intermittent fountain.



When the siphon is running, the outflow,  $Q_{\text{out}}$ , is a function of the level as in Torricelli's law:

$$Q_{\text{out}} = C\pi r^2 \sqrt{2gy} \quad (23.28)$$

#### Problem Statement

Neglecting the volume of water in the pipe, compute and plot the level of the water in the tank as a function of time over 100 seconds. Assume an initial condition of an empty tank,  $y(0) = 0$ , and employ the following parameter values for your simulation:

$$\begin{array}{lll} R_T = 0.05 \text{ m} & r = 7 \text{ mm} & y_{\text{low}} = 25 \text{ mm} \\ y_{\text{high}} = 0.1 \text{ m} & C = 0.6 & Q_{\text{in}} = 50 \text{ cm}^3/\text{s} \end{array}$$

We must take care with the units here.

#### Solution

When the fountain is running, the rate of change of the tank's volume,  $V(\text{m}^3/\text{s})$ , is determined by a simple balance involving inflow and outflow:

$$\frac{dV}{dt} = Q_{\text{in}} - Q_{\text{out}} \quad (23.29)$$

Because the tank is a vertical cylinder,  $V = \pi R_T^2 y$ . Substituting this relationship along with [Eq. \(23.28\)](#) into [Eq. \(23.29\)](#) gives

$$\frac{dy}{dt} = \frac{Q_{\text{in}} - C\pi r^2 \sqrt{2gy}}{\pi R_T^2} \quad (23.30)$$

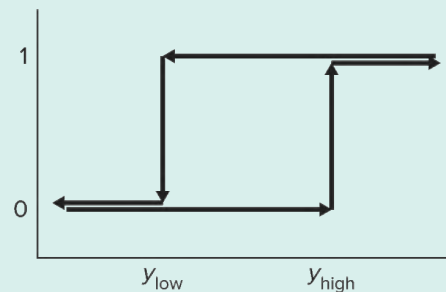
When the fountain is not running, the second term in the numerator goes to zero. We can incorporate this mechanism by introducing a new variable,  $siphon(y)$ , that equals zero when the fountain is not running and one when it is:

$$\frac{dy}{dt} = \frac{Q_{\text{in}} - siphon(y)C\pi r^2 \sqrt{2gy}}{\pi R_T^2} \quad (23.31)$$

Next, we must quantify the relationship that governs the switching of the *siphon* variable, called a *Boolean* or *logical variable*, where zero is equivalent to false and one to true.

The *siphon* variable is set to zero whenever the level is below or falls below  $y_{\text{low}}$ . Conversely, *siphon* is not set to one until the level rises above  $y_{\text{high}}$ . This relationship can be described as *hysteresis* because the change taking place depends on the direction of the level movement. This is illustrated in [Fig. 23.13](#).

**FIGURE 23.13** Siphon switch logic. Switches on as  $y$  increases and crosses  $y_{\text{high}}$ . Stays on until  $y$  decreases and crosses  $y_{\text{low}}$ .



A Python function to determine the *siphon* value can be written as a function of  $y$ .

```
def siphon(t,y,ylow,yhigh):
    if y < ylow:
        siphon.sw = 0
```

```

        elif y > yhigh:
            siphon.sw = 1
siphon.sw = 0

```

The function first checks if the level is below `y_low`. If so, the switch is always zero. If not below `y_low`, it then checks if it is above `y_high`, and sets the switch to one, as per [Fig. 23.13](#). If the level, `y`, is between `y_low` and `y_high`, the function makes no change to the switch. This is where a special consideration enters. If the `siphon` function makes no change to the switch, the previously established value of the switch must be maintained. This is managed by referring to the switch as `siphon.sw` and pre-setting it to zero outside the function.<sup>1</sup> You will note that this function has no return result. This function can then be included in the function which computes the derivative of `y`:

```

def dydt(t,y,ylow,yhigh,Qin):
    if y < 0: y = 0
    siphon(t,y,ylow,yhigh)
    dy = (Qin - siphon.sw * C*np.pi*r**2*np.sqrt(2*g*y))/np.pi/RT**2
    return dy

```

You will note two features here:

- The value of `y` is not allowed to go negative. This prevents a problem with the square root calculation that follows. As it turns out, the `solve_ivp` RK45 method may “experiment” with `y` values that include negative ones.
- The `siphon` function has no return, so it is just executed and not assigned to a result variable.

We now add the following code to carry out the solution with `solve_ivp` and plot the results.

```

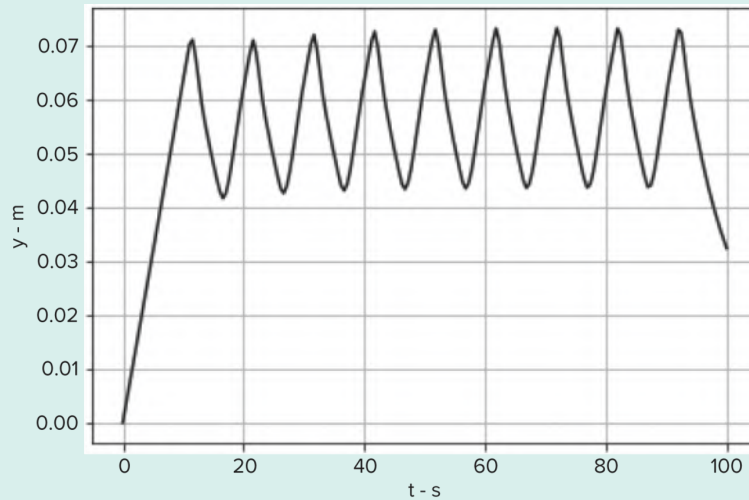
ti = 0. ; tf = 100.
y0 = 0.
tspan = np.linspace(ti,tf,200)
soln = solve_ivp(dydt,(ti,tf),[y0],t_eval=tspan,args=(ylow,yhigh,Qin))
t = soln.t
y = soln.y

pylab.plot(t,y[0,:],c='k')
pylab.grid()
pylab.xlabel('t - s')
pylab.ylabel('y - m')

```

The results are presented in [Fig. 23.14](#). The solution is evidently faulty. The level transitions should take place at 0.025 and 0.1. The `solve_ivp` function apparently has difficulty with the discontinuities that occur when the siphon turns on and off. What to do?

**FIGURE 23.14** The level in Pliny’s fountain versus time as simulated with `solve_ivp`.

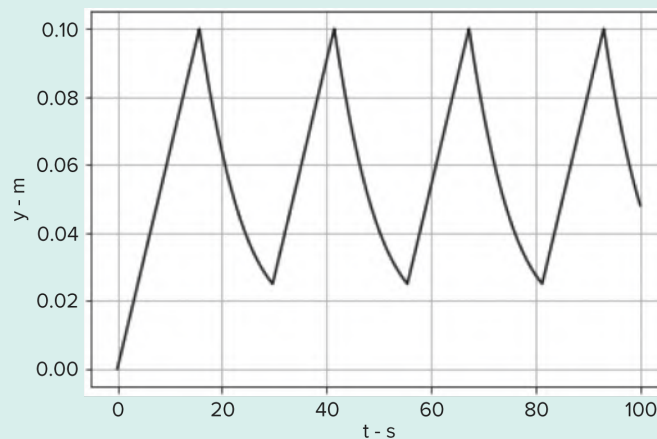


One approach is to give up on adaptive methods that adjust step sizes. For example, we can substitute a fourth-order, fixed-step Runge-Kutta algorithm, studied in [Chap. 22](#). Such a method will not attempt to adjust the step size to accommodate a discontinuity; it will just step over it. If the step size is small, an acceptable solution may be produced. We test this concept with the following code:

```
ti = 0. ; tf = 100.
y0 = 0.
tspan = np.array([ti,tf])
h = 0.01
t,y = rk4(dydt,tspan,y0,h,y_low,y_high,Qin)
```

A plot is produced in a similar way as above and is shown in [Fig. 23.15](#). Here, the results are satisfying. The tank level cycles reliably between  $y_{\text{low}} = 0.025$  m and  $y_{\text{high}} = 0.1$  m. There is no evidence of numerical difficulty with the siphon switching on and off.

**FIGURE 23.15** The level in Pliny’s fountain versus time as simulated using the fourth-order, fixed-step Runge-Kutta method with a small step size.



There are two take-away messages that can be gleaned from this case study. First, although it’s human nature to think the opposite, simpler is sometimes better. After all, to paraphrase Einstein, “Everything should be as

simple as possible, but no simpler.” Second, you should never blindly believe every result generated by a computer. You’ve probably heard the old chestnut, “garbage in equals garbage out” in reference to the impact of data quality on the validity of computer output. Unfortunately, some individuals think that regardless of what went in (the data) and what’s going on inside (the algorithm), it’s always “gospel out.” Situations like the one depicted in  **Fig. 23.14** are particularly dangerous; that is, although the output is incorrect, it’s not obviously wrong. That is, the simulation does not go unstable or yield negative levels. In fact, the solution moves up and down in the manner of an intermittent fountain, albeit incorrectly.

Hopefully, this case study illustrates that even a great piece of software, such as the functions available in the SciPy `integrate` module, is not foolproof. Hence, engineers and scientists always examine numerical output with a healthy skepticism based on their considerable experience and knowledge of the problems they are solving.



## PROBLEMS

---

**23.1** Repeat the simulation carried out in [\[2\] Sec. 23.5](#) for Pliny's fountain but change the `rtol` specification in the `solve_ivp` function from the default of  $10^{-3}$  to  $10^{-5}$  and  $10^{-8}$ . Create a set of stacked subplots of the three solutions. Observe any differences.

**23.2** The following ODEs have been proposed as a model of an epidemic:

$$\begin{aligned}\frac{dS}{dt} &= -aSI \\ \frac{dI}{dt} &= aSI - rI \\ \frac{dR}{dt} &= rI\end{aligned}$$

where  $S$  = susceptible individuals,  $I$  = infected population,  $R$  = recovered group,  $a$  = infection rate, and  $r$  = recovery rate. A city has 10,000 people, all of whom are susceptible.

- (a) If a single infectious individual enters the city at  $t = 0$ , compute the progression of the epidemic until the number of infected individuals falls below 10 (hint: use events). Use the following parameters:  $a = 0.0002/(\text{person}\cdot\text{week})$  and  $r = 0.15/\text{d}$ . Develop a time series plot of  $S$ ,  $I$ , and  $R$ .
- (b) Suppose that, after recovery, there is a loss of immunity that causes recovered individuals once again to become susceptible. This reinfection mechanism can be modeled as  $\rho R$ , where  $\rho$  = the return rate. Modify the model to include this phenomenon and repeat the computations in (a) using  $\rho = 0.03/\text{d}$ .

**23.3** Solve the following initial value problem over the interval  $t = 2$  to 3:

$$\frac{dy}{dt} = -0.5y + e^{-t}$$

Use the non-self-starting Heun method with a step size of 0.5 and initial conditions of  $y(1.5) = 5.222138$  and  $y(2.0) = 4.143883$ . Iterate the corrector to  $\varepsilon_s = 0.1\%$ . Compute the percent relative errors for your results based on the exact solutions obtained analytically:  $y(2.5) = 3.273888$  and  $y(3.0) = 2.577988$ .

**23.4** Solve the following initial-value problem over the interval from  $t = 0$  to 0.5:

$$\frac{dy}{dt} = yt^2 - y$$

Use the fourth-order RK method to predict the first value at  $t = 0.25$ . Then, use the non-self-starting Heun method to make the prediction at  $t = 0.5$ . Note:  $y(0) = 1$ .

**23.5** Given

$$\frac{dy}{dt} = -100,000y + 99,999e^{-t}$$

- (a) Estimate the step size required to maintain stability using the explicit Euler method.
- (b) If  $y(0) = 0$ , use the implicit Euler method to obtain a solution from  $t = 0$  to 2 using a step size of 0.1.

**23.6** Given

$$\frac{dy}{dt} = 30[\sin(t) - y] + 3 \cos(t)$$

If  $y(0) = 0$ , use the implicit Euler method to obtain a solution from  $t = 0$  to 4 using a step size of 0.4.

**23.7** Given

$$\frac{dx_1}{dt} = 999x_1 + 1999x_2$$

$$\frac{dx_2}{dt} = -1000x_1 - 2000x_2$$

If  $x_1(0) = x_2(0) = 1$ , obtain a solution from  $t = 0$  to 0.2 using a step size of 0.5 with the (a) explicit and (b) implicit Euler methods.

**23.8** The following ODE is described by Hornbeck (1975) as having a “parasitic” solution for  $y(0) = 0.08$ :

$$\frac{dy}{dt} = 5(y - t^2)$$

Solve this ODE from  $t = 0$  to 5:

- (a) Analytically,
- (b) Using `solve_ivp` with the default error tolerances, and
- (c) Using `solve_ivp` with `rtol` and `atol` both  $= 1 \times 10^{-12}$ .

Present your results on a single plot. Comment on the results and speculate on the cause of any differences.

**23.9** Recall from [Example 20.5](#) that the “humps” function exhibits both flat and steep regions of change over a relatively short  $x$  domain. The version that was in the example is

$$f(x) = \frac{1}{(x - 0.3)^2 + 0.01} + \frac{1}{(x - 0.9)^2 + 0.04} - 6$$

Estimate the value of the definite integral of  $f(x)$  between  $x = 0$  and 1 using (a) the `quad` function and (b) the `solve_ivp` function, both from the SciPy `integrate` submodule. If the latter differs from the former significantly, try to reduce the `rtol` specification and compare again.

**23.10** The oscillations of a swinging pendulum can be simulated with the following nonlinear ODE model, assuming minimal air drag:

$$\frac{d^2\theta}{dt^2} + \frac{g}{l}\sin(\theta) = 0$$

where  $\theta$  = the angle of displacement from the vertical and  $l$  = length of the pendulum. For small angular displacements,  $\sin(\theta) \cong \theta$ , and the model can be linearized to

$$\frac{d^2\theta}{dt^2} + \frac{g}{l}\theta = 0$$

Use the `solve_ivp` function to solve for  $\theta$  as a function of time for both nonlinear and linear models where  $l = 0.6$  m. First, solve the case where the initial condition is for a small displacement ( $\theta = \pi/8$  and  $d\theta/dt = 0$ ). Then, repeat the calculation for a large displacement ( $\theta = \pi/2$ ). For each case, present the results as a plot of  $\theta$  versus time.

**23.11** Employ the `events` feature of the `solve_ivp` function described in [Sec. 23.1.2](#) to determine the period of a 1-m-long, linear pendulum. Compute the period for the following initial conditions: (a)  $\theta = \pi/8$ , (b)  $\theta = \pi/4$ , and (c)  $\theta = \pi/2$ . For all three cases, set the initial angular velocity to zero. A good way to compute the period is to determine how long it takes for the pendulum to reach the bottom of its arc ( $\theta = 0$ ).

**23.12** Repeat [Prob. 23.11](#) but for the nonlinear pendulum described in [Prob. 23.10](#).

**23.13** The following system is a classic example of stiff ODEs that can occur in the modeling of chemical reaction kinetics.:

$$\begin{aligned}\frac{dc_1}{dt} &= -0.013c_1 - 1000c_1c_3 \\ \frac{dc_2}{dt} &= -2500c_2c_3 \\ \frac{dc_3}{dt} &= 0.013c_1 - 1000c_1c_3 - 2500c_2c_3\end{aligned}$$

Solve these equations from  $t = 0$  to 2 with initial conditions as shown above. Compare the solution, using the `solve_ivp` function, with the RK45 and LSODA methods. For each method, generate plots of  $c_1$  and  $c_2$  and separately  $c_3$ .

**23.14** The following second-order ODE is considered to be stiff:

$$\frac{d^2y}{dt^2} = -1001\frac{dy}{dx} - 1000y$$

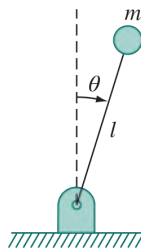
Solve this differential equation (a) analytically and (b) numerically for  $x = 0$  to 5. For (b), use an implicit approach with  $h = 0.5$ . Note that the initial conditions are  $y(0) = 1$  and  $y'(0) = 0$ . Display both results graphically. Finally, use the LSODA method of `solve_ivp` to solve the equation and display its result versus the analytical solution.

**23.15** Consider the thin rod of length  $l$  rotating in the plane as shown in  Fig. P23.15. The rod is fixed with a pin at one end and has a mass at the other. For this example,  $l = 0.5$  m. This system can be modeled as

$$\ddot{\theta} - \frac{g}{L}\theta = 0$$

Let  $\theta(0) = 0$  and  $\dot{\theta}(0) = 0.25$  rad/s. Solve using any method in this chapter. Terminate the solution when the angle reaches  $60^\circ$ . Plot angle (in degrees) versus time. Hint: Decompose the ODE into two first-order ODEs.

**FIGURE P23.15**



**23.16** Given the first-order ODE:

$$\frac{dx}{dt} = -700x - 1000e^{-t} \quad x(0) = 4$$

Solve this equation using a numerical method of your choice over the domain,  $0 \leq t \leq 5$ . Also, solve the equation analytically and create two plots of both solutions, one for the fast transient and another for the slow transient phases of the time scale.

**23.17** Solve the following differential equation from  $t = 0$  to 2:

$$\frac{dy}{dt} = -10y \quad y(0) = 1$$

Use the following techniques to obtain your solutions: (a) analytically, (b) the explicit Euler method, and (c) the implicit Euler method. For (b) and (c), use  $h = 0.1$  and  $0.2$ . Plot your results.

**23.18** The Lotka-Volterra equations described in [Sec. 22.6](#) have been refined to include additional factors that impact predator-prey dynamics. For example, over and above predation, prey population can be limited by other factors such as space. Space limitation can be incorporated into the model as a carrying capacity (recall the logistic model described in [Prob. 22.5](#)) as in

$$\begin{aligned} \frac{dx}{dt} &= a\left(1 - \frac{x}{K}\right)x - bxy \\ \frac{dy}{dt} &= -cy + dxy \end{aligned}$$

where  $K$  = carrying capacity. Use the same parameter values and initial conditions as in [Sec. 22.6](#) to integrate these equations using `solve_ivp`. Present your results as time series and phase-plane plots.

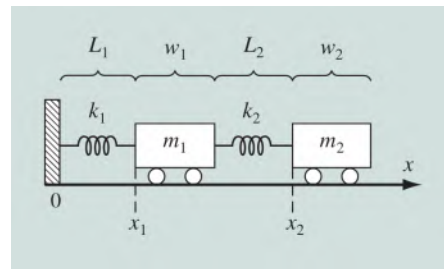
- (a) Employ a large value for  $K$ , for example,  $1 \times 10^{-8}$ , to validate that you obtain the same results as in [Sec. 22.6](#). Solve the equations for  $t = 0$  to 40. Use 200 points for the plots.
- (b) Compare (a) with the more realistic carrying capacity of  $K = 200$ . Solve the equations for  $t = 0$  to 100. Use 1000 points for the plots.

**23.19** Two masses are attached to a wall by linear springs ([Fig. P23.19](#)). Force balances based on Newton's second law can be written as

$$\begin{aligned} \frac{d^2x_1}{dt^2} &= -\frac{k_1}{m_1}(x_1 - L_1) + \frac{k_2}{m_1}(x_2 - x_1 - w_1 - L_2) \\ \frac{d^2x_2}{dt^2} &= -\frac{k_2}{m_2}(x_2 - x_1 - w_1 - L_2) \end{aligned}$$

where the  $k$ 's are the spring constants,  $m$ 's are the masses,  $L$ 's are the relaxed spring lengths, and the  $w$ 's are the widths of the masses. Compute the positions of the masses, the  $x$ 's, as a function of time using the following parameter values:  $k_1 = k_2 = 5$ ,  $m_1 = m_2 = 2$ ,  $w_1 = w_2 = 5$ , and  $L_1 = L_2 = 2$ . Perform the simulation from  $t = 0$  to 20. Set the initial conditions as  $x_1 = L_1$  and  $x_2 = L_1 + w_1 + L_2 + 6$ . The initial velocities are zero. Construct time-series plots of both displacements (same plot) and both velocities (also, same plot). In addition, create a phase plane plot of  $x_2$  versus  $x_1$ .

**FIGURE P23.19**



**23.20** Use the `solve_ivp` function to integrate the equations described in [Prob. 23.19](#). Generate a plot figure with two subplots stacked vertically. The upper subplot should show displacements versus time, and the lower subplot should show velocities versus time. Employ the `fft` function from the `fft` submodule of SciPy to compute the discrete Fourier transform (DFT) of the first mass's displacement. Generate and plot the power spectrum in order to identify the system's resonant frequencies.

**23.21** Perform the same computations as in [Prob. 23.20](#) but based on the first-floor displacement of the structure in [Prob. 22.22](#).

**23.22** As depicted in [Fig. P23.22](#), a double pendulum consists of a pendulum attached to another pendulum. We indicate the upper and lower pendulums by subscripts 1 and 2, respectively, and we place the origin at the pivot point of the upper pendulum with  $y$  increasing downward. We further assume that the system oscillates in a vertical plane subject to gravity, the pendulum rods are of negligible mass, and the pendulum masses are at a point. Under these assumptions, the force balances can be used to derive the following equations of motion:

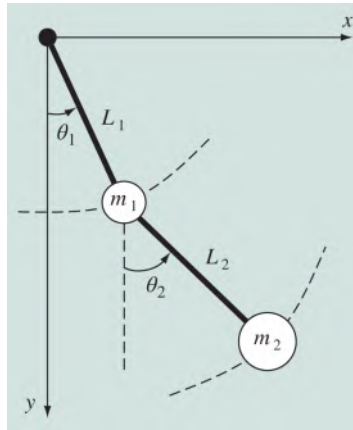
$$\frac{d^2\theta_1}{dt^2} = \frac{-g(2m_1 + m_2)\sin(\theta_1) - m_2 g \sin(\theta_1 - 2\theta_2) - 2 \sin(\theta_1 - \theta_2)m_2(d\theta_2/dt)^2 L_2 + (d\theta_2/dt)^2 L_1(m_1 + m_2 - m_2 \cos(2\theta_1 - \theta_2))}{L_1(2m_1 + m_2 - m_2 \cos(2\theta_1 - \theta_2))}$$

$$\frac{d^2\theta_2}{dt^2} = \frac{2 \sin(\theta_1 - \theta_2) \left( (d\theta_1/dt)^2 L_1(m_1 + m_2) + g(m_1 + m_2)\cos(\theta_1) + (d\theta_2/dt)^2 L_2 m_2 \right)}{L_2(2m_1 + m_2 - m_2 \cos(2\theta_1 - \theta_2))}$$

where the subscripts 1 and 2 designate the top and bottom pendulum, respectively,  $\theta$  = the angle (radians) with 0 = vertical upwards and counterclockwise being positive,  $t$  = time (s),  $m$  = mass (kg), and  $L$  = length (m). Note that the  $x$  and  $y$  coordinates of the masses are related to the angles by

$$\begin{aligned}x_1 &= L_1 \sin(\theta_1) & y_1 &= -L_1 \cos(\theta_1) \\x_2 &= x_1 + L_2 \sin(\theta_2) & y_2 &= y_1 - L_2 \cos(\theta_2)\end{aligned}$$

**FIGURE P23.22**



Use `solve_ivp` to solve for the angles and angular velocities of the two masses as a function of time from  $t = 0$  to 10 s. Create plots of the angles versus time (one plot) and a phase plot of  $\theta_2$  versus  $\theta_1$  (another plot). Use enough points in your solution to get smooth curves on your plots. Test your code for the following two cases:

Case I (small displacement):  $L_1 = L_2 = 1$  m,  $m_1 = m_2 = 0.25$  kg, with initial conditions:  $\theta_1 = 0.05$ , and  $\theta_2$ ,  $d\theta_1/dt$ ,  $d\theta_2/dt$  all equal to zero.

Case II (large displacement):  $L_1 = L_2 = 1$  m,  $m_1 = 0.5$  kg,  $m_2 = 0.25$  kg with initial conditions:  $\theta_1 = 0.2$ , and  $\theta_2$ ,  $d\theta_1/dt$ ,  $d\theta_2/dt$  all equal to zero.

**23.23**  **Figure P23.23** shows the forces exerted on a hot air balloon system. The drag force is formulated as

$$F_D = \frac{1}{2} \rho_a v^2 A C_d$$

where  $\rho_a$  = air density ( $\text{kg}/\text{m}^3$ ),  $v$  = velocity (m/s),  $A$  = projected frontal area ( $\text{m}^2$ ), and  $C_d$  = the dimensionless drag coefficient ( $\cong 0.47$  for a sphere). Note also that the total mass of the balloon consists of two components:

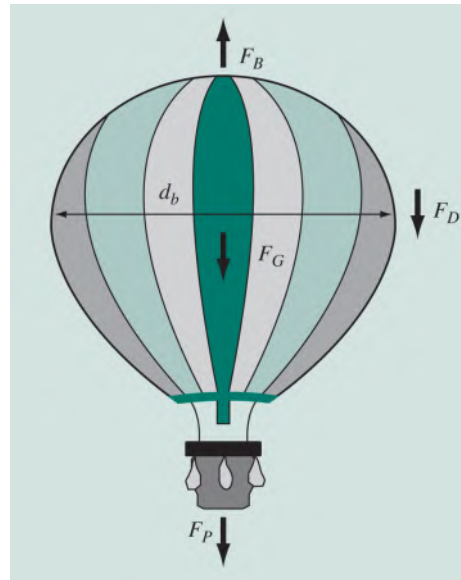
$$m = m_G + m_P$$

where  $m_G$  = the mass of the gas inside the expanded balloon (kg), and  $m_P$  = the mass of the payload (basket, passengers, and the unexpanded balloon = 265 kg). Assume that the ideal gas law holds ( $P = \rho RT/MW$ ), that the balloon is a perfect sphere with a diameter 17.3 m, and that the heated air inside the envelope is at roughly the same pressure as the outside air. Other necessary parameters are the molecular weight of air ( $MW = 28.97 \text{ kg/kmol}$ ), normal atmospheric pressure,  $P = 101,300 \text{ Pa}$ , the gas law constant in appropriate units,  $R = 8314.5 \text{ m}^3 \cdot \text{Pa}/(\text{kmol} \cdot \text{K})$ , average temperature inside the balloon,  $T = 100^\circ \text{C}$ , and the normal ambient density of air,  $\rho_a = 1.2 \text{ kg/m}^3$ .

- Use a force balance to develop the differential equation for  $dv/dt$  as a function of the model's parameters.
- At steady state, calculate the balloon's terminal velocity.
- Use `solve_ivp` to compute the velocity and position of the balloon from  $t = 0$  to 60 s given the previous parameters along with the initial condition:  $v(0) = 0$ . Present plots of your results.

Page 588

FIGURE P23.23



**23.24** Develop a Python script using `solve_ivp` to compute the velocity,  $v$ , and position,  $z$ , of a hot air balloon as described in [Prob. 23.23](#). Perform the calculation from  $t = 0$  to 60 s. At  $z = 200 \text{ m}$ , assume that part of the payload (not a person!) is dropped out of the balloon. Present plots of your results.

**23.25** You are going on a two-week vacation and place your pet goldfish “Freddie” into your bathtub. Note that you dechlorinate the water first! You then place an airtight acrylic cover over the top of the tub in order to protect Freddie from your cat, “Killer.” You mistakenly mix one tablespoon (~15 mL) of sugar into the tub (You were in a hurry and thought it was fish food!). Unfortunately, there are bacteria in



the water (Remember you got rid of the chlorine.), which break down the sugar consuming dissolved oxygen in the process. The oxidation reaction follows first-order kinetics with a reaction rate,  $k_d = 0.029/\text{d}$ . The tub initially has a concentration of sugar (sucrose) of  $20 \text{ mgO}_2/\text{L}$  (oxygen equivalents) and an oxygen concentration of  $8.4 \text{ mgO}_2/\text{L}$ . Mass balances for the sugar and dissolved oxygen can be written as

$$\begin{aligned}\frac{dL}{dt} &= -k_d L \\ \frac{do}{dt} &= -k_d L\end{aligned}$$

where  $L$  = sugar concentration expressed as oxygen equivalents ( $\text{mgO}_2/\text{L}$ ),  $t$  = time (d), and  $o$  = dissolved oxygen concentration ( $\text{mgO}_2/\text{L}$ ). Thus, as the sugar is oxidized, an equivalent amount of oxygen is lost from the tub.

Develop a Python script using `solve_ivp` to compute the two concentrations versus and present your results in a plot. Use the `events` feature to stop the simulation when the oxygen concentration drops to  $2 \text{ mgO}_2/\text{L}$ , indicating the end of Freddie. The results may influence you to change your vacation plans, or not.

**23.26** The growth of bacteria from substrate can be represented by the following pair of ODEs:

$$\begin{aligned}\frac{dX}{dt} &= Y k_{\max} \frac{S}{k_s + S} X \\ \frac{dS}{dt} &= -k_{\max} \frac{S}{k_s + S} X\end{aligned}$$

where  $X$  = bacterial biomass,  $t$  = time (d),  $Y$  = a yield coefficient,  $k_{\max}$  = maximum bacterial growth rate,  $S$  = substrate concentration, and  $k_s$  = half saturation constant. The parameter values are  $Y = 0.75$ ,  $k_{\max} = 0.3$ , and  $k_s = 1 \times 10^{-4}$ , and the initial conditions are  $S(0) = 5$  and  $X(0) = 0.05$ . Note that neither  $X$  nor  $S$  can fall below zero as negative values are impossible.

(a) Use `solve_ivp` to solve for  $X$  and  $S$  from  $t = 0$  to 25.

(b) Repeat the solution with the `rtol` parameter set to  $1 \times 10^{-6}$  (default is 0.001).

**23.27** A number of individuals have made skydives from very high altitudes. Suppose that an 80-kg skydiver launches from an elevation of 36.5 km above the earth's surface. The skydiver has a projected area,  $A = 0.55 \text{ m}^2$ , and a dimensionless drag coefficient,  $C_d = 0.94$ . Note that gravitational acceleration,  $g(\text{m/s}^2)$ , changes with elevation according to

$$g = 9.806412 - 3.039734 \times 10^{-6} z$$

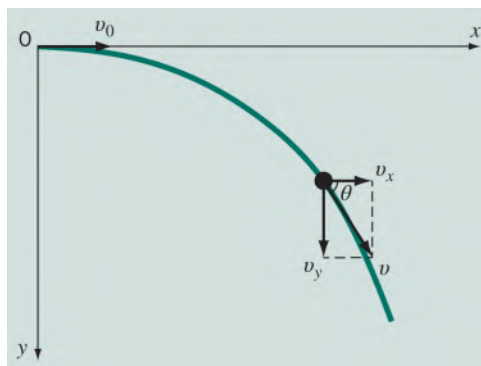
where  $z$  = elevation (m). The density of air is tabulated as

| $z$ (km) | $\rho$ (kg/m <sup>3</sup> ) | $z$ (km) | $\rho$ (kg/m <sup>3</sup> ) | $z$ (km) | $\rho$ (kg/m <sup>3</sup> ) |
|----------|-----------------------------|----------|-----------------------------|----------|-----------------------------|
| -1       | 1.3470                      | 6        | 0.6601                      | 25       | 0.04008                     |
| 0        | 1.2250                      | 7        | 0.5900                      | 30       | 0.01841                     |
| 1        | 1.1120                      | 8        | 0.5258                      | 40       | 0.003996                    |
| 2        | 1.0070                      | 9        | 0.4671                      | 50       | 0.001027                    |
| 3        | 0.9093                      | 10       | 0.4135                      | 60       | 0.0003097                   |
| 4        | 0.8194                      | 15       | 0.1948                      | 70       | $8.283 \times 10^{-5}$      |
| 5        | 0.7364                      | 20       | 0.08891                     | 80       | $1.846 \times 10^{-5}$      |

- Based on a force balance between gravity and drag, derive differential equations for velocity and elevation for the skydiver.
- Use a numerical method to solve for velocity and elevation that terminates when the skydiver deploys the parachute at one kilometer above the earth's surface. Display the conditions (time, elevation, velocity) at the point of deployment. Plot your results.

**23.28** As depicted in [Fig. P23.28](#), a parachutist jumps from an aircraft that is flying in a straight line parallel with the ground. (a) Using force balances, derive four differential equations for the rates of change of the  $x$  and  $y$  components of distance and velocity. [Hint: Recognize that  $\sin(\theta) = v_y/v$  and  $\cos(\theta) = v_x/v$ .] (b) Employ `solve_ivp` with the `events` option to generate a solution from  $t = 0$  until the parachutist would hit the ground if the chute didn't open. (We don't want that to happen!) The drag coefficient is  $0.25 \text{ kg/m}$ , the mass is  $80 \text{ kg}$ , and the ground is  $2000 \text{ m}$  below the initial vertical position of the aircraft. The initial conditions are  $v_x = 135 \text{ m/s}$ , and  $v_y = x = y = 0$ . Present your results in a plot on Cartesian ( $x - y$ ) coordinates.

**FIGURE P23.28**



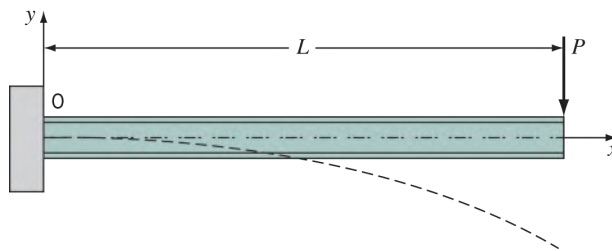
**23.29** The basic differential equation of the elastic curve for a cantilever beam ( Fig. P23.29) is given as

$$EI \frac{d^2 y}{dx^2} = -P(L - x)$$

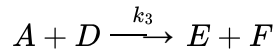
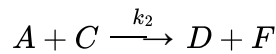
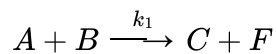
where  $E$  = the modulus of elasticity and  $I$  = the moment of inertia. Solve for the deflection of the beam using `solve_ivp`. The following parameter values apply:  $E = 2 \times 10^{11}$  Pa,  $I = 0.00033$  m<sup>4</sup>,  $P = 4.5$  kN, and  $L = 3$  m. Present your results in a plot along with the analytical solution:

$$y = -\frac{PLx^2}{2EI} + \frac{Px^3}{6EI}$$

**FIGURE P23.29**



**23.30** Three simultaneous chemical reactions have been modeled by the chemical equations:



The letters  $A$ - $F$  represent different chemical species, and, in the ODEs below, also represent their concentrations in mol/L. These represent a real reaction system documented by Svirbely and Blauer (1961), also included in Himmelblau (1970). The reaction rate parameters are the  $k$ 's, and second-order kinetic expressions are used to quantify the system model:

$$\begin{aligned} \frac{dA}{dt} &= -k_1 AB - k_2 AC - k_3 AD & A(0) &= 0.0209 \frac{\text{mol}}{\text{L}} \\ \frac{dB}{dt} &= -k_1 AB & B(0) &= \frac{A(0)}{3} \end{aligned}$$

$$\begin{aligned}\frac{dC}{dt} &= k_1 AB - k_2 AC & C(0) &= 0 \\ \frac{dD}{dt} &= k_2 AC - k_3 AD & D(0) &= 0\end{aligned}$$

The  $E$  and  $F$  components can be computed based on reaction stoichiometry as

$$E = \frac{A(0) - A - C - 2D}{3} \quad \text{and} \quad F = A(0) - A$$

The following values for the rate parameters are proposed here:

$$\begin{aligned}k_1 &= 12 \frac{1}{\text{mol/L}} \times \frac{1}{\text{min}} \\ k_2 &= 3 \frac{1}{\text{mol/L}} \times \frac{1}{\text{min}} \\ k_3 &= 0.5 \frac{1}{\text{mol/L}} \times \frac{1}{\text{min}}\end{aligned}$$

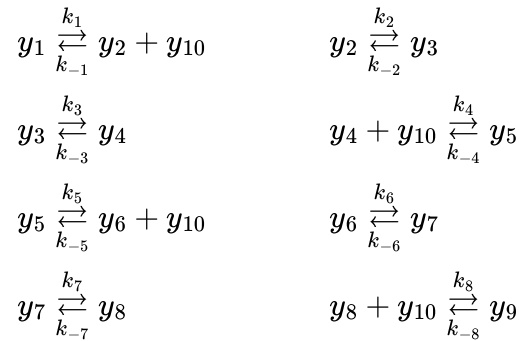
Measurements were taken on a batch reaction apparatus of the concentration of the  $A$  component versus time. These are shown in the table below.

- Use `solve_ivp` to produce a solution of the four ODEs above from  $t = 0$  to 510 min. From your solution, compute the corresponding concentrations of the  $E$  and  $F$  components. Create a plot of all six components using different colors to distinguish the curves. Add the experimental data from the table as markers on your plot.
- Build on your code from (a) to compute the sum of squares of differences between simulated values and experimental values of  $A$ . Note: In order to do this, your simulation will have to produce values at the times given in the table. Employ an optimization function from the SciPy `optimize` submodule to adjust all three  $k$  values and minimize the sum of squares performance criterion. Repeat the plot from (a), but with your new values for the  $k$ 's. Comment on the ability of the model to replicate the data in (a), and in (b).

| Time<br>(min) | $A \times 10^3$ (mol/L) | Time<br>(min) | $A \times 10^3$ (mol/L) |
|---------------|-------------------------|---------------|-------------------------|
| 4.50          | 15.4                    | 76.75         | 8.395                   |
| 8.67          | 14.22                   | 90.00         | 7.891                   |
| 12.67         | 13.35                   | 102.00        | 7.51                    |
| 17.75         | 12.32                   | 108.00        | 7.37                    |
| 22.67         | 11.81                   | 147.92        | 6.646                   |

| Time<br>(min) | $A \times 10^3$ (mol/L) | Time<br>(min) | $A \times 10^3$ (mol/L) |
|---------------|-------------------------|---------------|-------------------------|
| 27.08         | 11.39                   | 198.00        | 5.883                   |
| 32.00         | 10.92                   | 241.75        | 5.322                   |
| 36.00         | 10.54                   | 270.25        | 4.96                    |
| 46.33         | 9.78                    | 326.25        | 4.518                   |
| 57.00         | 9.157                   | 418.00        | 4.075                   |
| 69.00         | 8.594                   | 501.00        | 3.715                   |

**23.31** A complex batch chemical reaction system involving ten species, known as *photochemical isomerization*, is represented by eight reversible chemical reactions:



There are ten ODEs that can be written on the basis of second-order kinetics to track each component:

Page 590

$$\begin{aligned}
 \dot{y}_1 &= -k_1 y_1 + k_{-1} y_2 y_{10} \\
 \dot{y}_2 &= -k_{-1} y_2 y_{10} - k_2 y_2 + k_{-2} y_3 + k_1 y_1 \\
 \dot{y}_3 &= k_2 y_2 - k_{-2} y_3 - k_3 y_3 + k_{-3} y_4 \\
 \dot{y}_4 &= k_3 y_3 - k_{-3} y_4 - k_4 y_4 y_{10} + k_{-4} y_5 \\
 \dot{y}_5 &= k_4 y_4 y_{10} - k_{-4} y_5 - k_5 y_5 + k_{-5} y_6 y_{10} \\
 \dot{y}_6 &= k_5 y_5 - k_{-5} y_6 y_{10} - k_6 y_6 + k_{-6} y_7 \\
 \dot{y}_7 &= k_6 y_6 - k_{-6} y_7 - k_7 y_7 + k_{-7} y_8 \\
 \dot{y}_8 &= k_7 y_7 - k_{-7} y_8 - k_8 y_8 y_{10} + k_{-8} y_9 \\
 \dot{y}_9 &= k_8 y_8 y_{10} - k_{-8} y_9 \\
 \dot{y}_{10} &= k_1 y_1 - k_{-1} y_2 y_{10} - k_4 y_4 y_{10} + k_{-4} y_5 - k_{-5} y_6 y_{10} - k_8 y_8 y_{10} + k_{-8} y_9
 \end{aligned}$$

The rate parameters, the  $k$ 's, for the reactions are given as

$$\begin{array}{ll}
 k_1 = 5 & k_{-1} = 10^{10} \\
 k_2 = 10^6 & k_{-2} = 10^6 \\
 k_3 = 10^5 & k_{-3} = 2 \times 10^5
 \end{array}$$

$$\begin{array}{ll}
k_4 = 10^6 & k_{-4} = 10^{-2} \\
k_5 = 10 & k_{-5} = 10^9 \\
k_6 = 10^6 & k_{-6} = 10^6 \\
k_7 = 5 \times 10^5 & k_{-7} = 10^5 \\
k_8 = 5 \times 10^5 & k_{-8} = 2.5 \times 10^{-4}
\end{array}$$

and the initial conditions are

$$\begin{array}{l}
y_1(0) = 0.1 \\
y_{10}(0) = 1 \\
y_2(0) = y_3(0) = \cdots = y_9(0) = 0
\end{array}$$

The relative speed of the chemical reactions, as evidenced by the  $k$  values, varies over many orders of magnitude. This means that this is a stiff system of equations—in the extreme! Solving this system successfully with traditional RK methods is impossible, and, in general, such equation sets could not be solved until the introduction of adaptive stiff integration methods, such as the pioneering work by Gear (1969) in the 1960s.

Here is your challenge, should you choose to accept it, or more likely have been assigned it:

Solve this system using `solve_ivp` over the time domain  $0 \leq t \leq 10^6$  s. The solution should be returned for 1,000 logarithmically spaced points from  $t = 10^{-14}$  to  $10^6$  (Hint: use `np.logspace`, and, yes, that's 20 orders of magnitude of time.) You must use the LSODA method, which handles stiff systems. Set the `rtol` and `atol` parameters to  $1 \times 10^{-12}$ . Present the results on a log-log plot using combinations of different colors and linestyles. With your results in front of you, stand back, look at them, and be amazed.

# Boundary-Value Problems

## CHAPTER OBJECTIVES

The primary objective of this chapter is to introduce you to boundary-value problems for ordinary differential equations. Specific objectives and topics covered are

- Understanding the difference between initial-value and boundary-value problems.
- Knowing how to express an  $n$ th-order ODE as a system of  $n$  first-order ODEs.
- Knowing how to implement the shooting method for linear ODEs by using linear interpolation to generate accurate “shots.”
- Knowing how to implement the finite-difference method.
- Understanding how derivative boundary conditions are incorporated into the finite-difference method.
- Being able to solve nonlinear ODEs with the finite-difference method by using root-location methods for systems of nonlinear algebraic equations.
- Familiarizing yourself with the Python function `solve_bvp` from the SciPy `integrate` submodule for solving boundary-value ODEs.

## YOU’VE GOT A PROBLEM

To this point, we have been computing the velocity of a free-falling bungee jumper by integrating a single ODE:

$$\frac{dv}{dt} = g - \frac{c_d}{m}v^2 \quad (24.1)$$

Suppose that rather than velocity, you are asked to determine the position of the jumper as a function of time. One way to do this is to recognize that velocity is the first derivative of distance:

$$\frac{dx}{dt} = v \quad (24.2)$$

Thus, by solving the system of two ODEs represented by [Eqs. \(24.1\)](#) and [\(24.2\)](#), we can simultaneously determine both the velocity and the position.

However, because we are now integrating two ODEs, we require two conditions to obtain the solution. We are already familiar with one way to do this for the case where we have values for both position and velocity at the initial time:

$$\begin{aligned}x(t = 0) &= x_i \\v(t = 0) &= v_i\end{aligned}$$

Given such conditions, we can easily integrate the ODEs using the numerical techniques described in [Chaps. 22](#) and [23](#). This is referred to as an *initial-value problem*.

But what if we do not know values for both position and velocity at  $t = 0$ ? Let's say that we know the initial position but rather than having the initial velocity, we want the jumper to be at a specified position at a later time. In other words:

$$\begin{aligned}x(t = 0) &= x_i \\x(t = t_f) &= x_f\end{aligned}$$

Because the two conditions are given at different values of the independent variable, this is called a *boundary-value problem*.

Such problems require special solution techniques. Some of these are related to the methods for initial value problems that were described in the previous two chapters. However, others employ entirely different strategies to obtain solutions. This chapter is designed to introduce you to the more common of these methods.

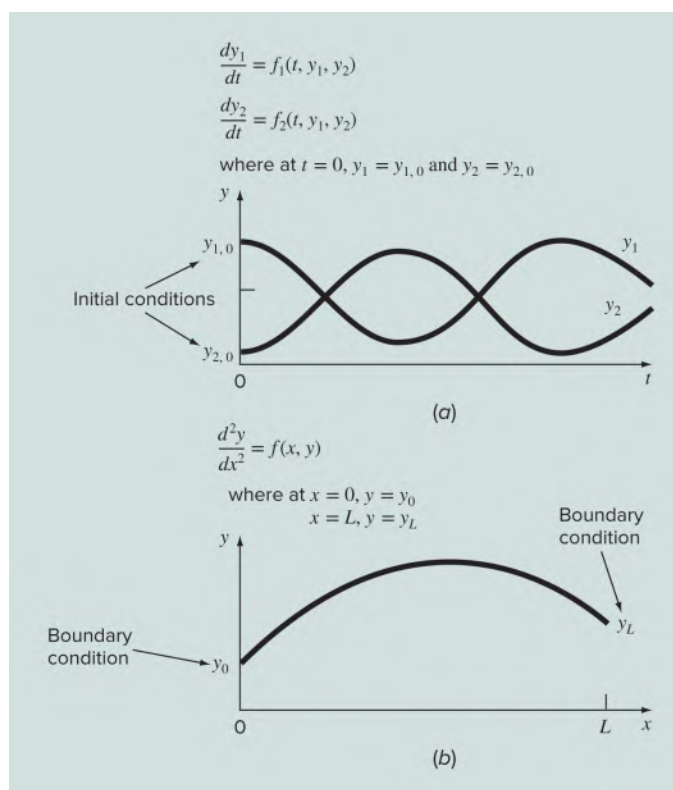


## 24.1 INTRODUCTION AND BACKGROUND

### 24.1.1 What Are Boundary-Value Problems?

An ordinary differential equation is accompanied by auxiliary conditions, which are used to evaluate the constants of integration that result during the solution of the equation. For an  $n$ th-order equation,  $n$  conditions are required. If all the conditions are specified at the same value of the independent variable, then we are dealing with an *initial-value problem* (🔗 Fig. 24.1a). To this point, the material in Part Six (🔗 Chaps. 22 and 🔗 23) has been devoted to this type of problem.

**FIGURE 24.1** Initial-value versus boundary-value problems. (a) An initial-value problem where all the conditions are specified at the same value of the independent variable. (b) A boundary-value problem where the conditions are specified at different values of the independent variable.



In contrast, there are often cases when the conditions are not known at a single point but rather are given at different values of the independent variable. Because these values are often specified at the extreme

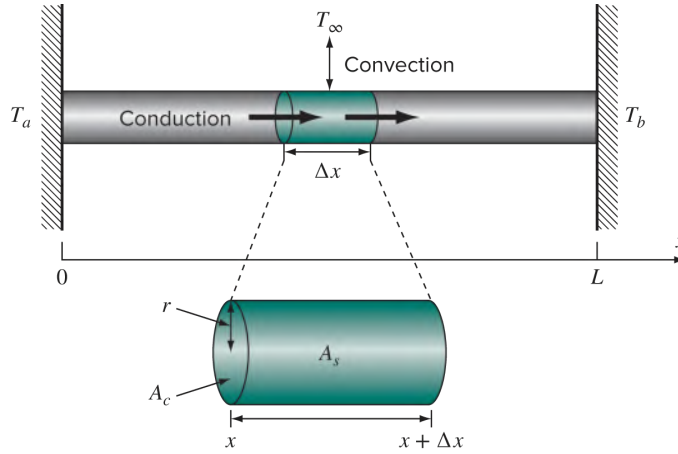
points or boundaries of a system, they are customarily referred to as *boundary-value problems* (Fig. 24.1b). A variety of significant engineering applications fall within this class. In this chapter, we discuss some of the basic approaches for solving such problems.

## 24.1.2 Boundary-Value Problems in Engineering and Science

At the beginning of this chapter, we showed how the determination of the position and velocity of a falling object could be formulated as a boundary-value problem. For that example, a pair of ODEs was integrated in time. Although other time-variable examples can be developed, boundary-value problems arise more naturally when integrating in space. This occurs because auxiliary conditions are often specified at different positions in space.

A case in point is the simulation of the steady-state temperature distribution for a long, thin rod positioned between two constant-temperature walls (Fig. 24.2). The rod's cross-sectional dimensions are small enough so that radial temperature gradients are minimal and, consequently, temperature is a function exclusively of the axial coordinate  $x$ . Heat is transferred along the rod's longitudinal axis by conduction and between the rod and the surrounding gas by convection. For this example, radiation is assumed to be negligible.<sup>1</sup>

**FIGURE 24.2** A heat balance for a differential element of a heated rod subject to conduction and convection.



As depicted in Fig. 24.2, a heat balance can be taken around a differential element of thickness  $\Delta x$  as

$$0 = q(x)A_c - q(x + \Delta x)A_c + hA_s(T_\infty - T) \quad (24.3)$$

where  $q(x)$  = flux into the element due to conduction  $[J/(m^2 \cdot s)]$ ;  $q(x + \Delta x)$  = flux out of the element due to conduction  $[J/(m^2 \cdot s)]$ ;  $A_c$  = cross-sectional area  $[m^2] = \pi r^2$ ,  $r$  = the radius  $[m]$ ;

$h$  = the convection heat transfer coefficient  $[\text{J}/(\text{m}^2 \cdot \text{K} \cdot \text{s})]$ ;  $A_s$  = the element's surface area  $[\text{m}^2] = 2\pi r \Delta x$ ;  $T_\infty$  = the temperature of the surrounding gas  $[\text{K}]$ ; and  $T$  = the rod's temperature  $[\text{K}]$ .

Equation (24.3) can be divided by the element's volume  $(\pi r^2 \Delta x)$  to yield

$$0 = \frac{q(x) - q(x + \Delta x)}{\Delta x} + \frac{2h}{r}(T_\infty - T)$$

Taking the limit  $\Delta x \rightarrow 0$  gives

$$0 = -\frac{dq}{dx} + \frac{2h}{r}(T_\infty - T) \quad (24.4)$$

The flux can be related to the temperature gradient by *Fourier's law*:

$$q = -k \frac{dT}{dx} \quad (24.5)$$

where  $k$  = the coefficient of thermal conductivity  $[\text{J}/(\text{s} \cdot \text{m} \cdot \text{K})]$ . Equation (24.5) can be differentiated with respect to  $x$ , substituted into Eq. (24.4), and the result divided by  $k$  to yield

Page 594

$$0 = \frac{d^2 T}{dx^2} + h'(T_\infty - T) \quad (24.6)$$

where  $h' =$  a bulk heat-transfer parameter reflecting the relative impacts of convection and conduction  $[\text{m}^{-2}] = 2h/(rk)$ .

Equation (24.6) represents a mathematical model that can be used to compute the temperature along the rod's axial dimension. Because it is a second-order ODE, two conditions are required to obtain a solution. As depicted in Fig. 24.2, a common case is where the temperatures at the ends of the rod are held at fixed values. These can be expressed mathematically as

$$\begin{aligned} T(0) &= T_a \\ T(L) &= T_b \end{aligned}$$

The fact that they physically represent the conditions at the rod's "boundaries" is the origin of the terminology: boundary conditions.

Given these conditions, the model represented by [Eq. \(24.6\)](#) can be solved. Because this particular ODE is linear, an analytical solution is possible as illustrated in the following example.

## EXAMPLE 24.1

### Analytical Solution for a Heated Rod

#### Problem Statement

Use calculus to solve [Eq. \(24.6\)](#) for a 10-m rod with  $h' = 0.05 \text{ m}^{-2}$  [ $h = 1 \text{ J}/(\text{m}^2 \cdot \text{K} \cdot \text{s})$ ],  $r = 0.2 \text{ m}$ ,  $k = 200 \text{ J}/(\text{s} \cdot \text{m} \cdot \text{K})$ ,  $T_\infty = 200 \text{ K}$ , and the boundary conditions:

$$T(0) = 300 \text{ K} \quad T(10) = 400 \text{ K}$$

#### Solution

This ODE can be solved in a number of ways. A straightforward approach is to first express the equation as

$$\frac{d^2T}{dx^2} - h'T = -h'T_\infty$$

Because this is a linear ODE with constant coefficients, the general solution can be readily obtained by setting the right-hand side to zero and assuming a solution of the form  $T = e^{\lambda x}$ . Substituting this solution along with its second derivative into the homogeneous form of the ODE yields

$$\lambda^2 e^{\lambda x} - h' e^{\lambda x} = 0$$

which can be solved for  $\lambda = \pm\sqrt{h'}$ . Thus, the general solution is

$$T = Ae^{\lambda x} + Be^{-\lambda x}$$

where  $A$  and  $B$  are constants of integration. Using the method of undetermined coefficients, we can derive the particular solution  $T = T_\infty$ . Therefore, the total solution is

$$T = T_\infty + Ae^{\lambda x} + Be^{-\lambda x}$$

The constants can be evaluated by applying the boundary conditions

$$T_a = T_\infty + A + B$$

$$T_b = T_\infty + Ae^{\lambda L} + Be^{-\lambda L}$$

These two equations can be solved simultaneously for

$$A = \frac{(T_a - T_\infty)e^{-\lambda L} - (T_b - T_\infty)}{e^{-\lambda L} - e^{\lambda L}}$$

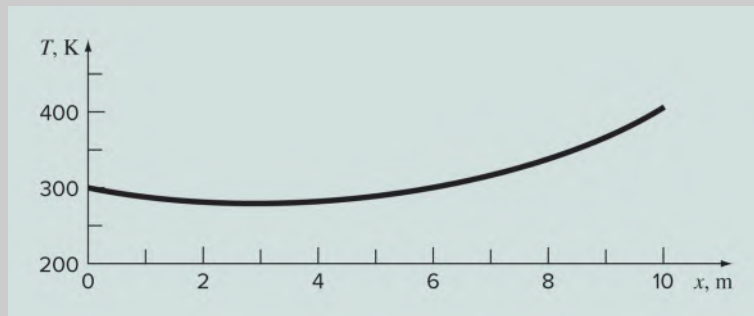
$$B = \frac{(T_b - T_\infty) - (T_a - T_\infty)e^{\lambda L}}{e^{-\lambda L} - e^{\lambda L}}$$

Substituting the parameter values from this problem gives  $A = 20.4671$  and  $B = 79.5329$ . Therefore, the final solution is

$$T = 200 + 20.4671e^{\sqrt{0.05}x} + 79.5329e^{-\sqrt{0.05}x} \quad (24.7)$$

As can be seen in  **Fig. 24.3**, the solution is a smooth curve connecting the two boundary temperatures. The temperature in the middle is depressed due to the convective heat loss to the cooler surrounding gas.

**FIGURE 24.3** Analytical solution for the heated rod.



In the following sections, we will illustrate numerical approaches for solving the same problem we just solved analytically in  **Example 24.1**. The exact analytical solution will be useful in assessing the accuracy of the solutions obtained with the approximate, numerical methods.

## 24.2 THE SHOOTING METHOD

---

The shooting method is based on converting the boundary-value problem into an equivalent initial-value problem. A trial-and-error approach is then implemented to develop a solution for the initial-value version that satisfies the given boundary conditions.

Although the method can be employed for higher-order and nonlinear equations, it is nicely illustrated for a second-order, linear ODE such as the heated rod described in the previous section:

$$0 = \frac{d^2T}{dx^2} + h'(T_\infty - T) \quad (24.8)$$

subject to the boundary conditions

$$\begin{aligned} T(0) &= T_a \\ T(L) &= T_b \end{aligned}$$

We convert this boundary-value problem into an initial-value problem by defining the rate of change of temperature, or *gradient*, as

$$\frac{dT}{dx} = z \quad (24.9)$$

and reexpressing  Eq. (24.8) as

$$\frac{dz}{dx} = -h'(T_\infty - T) \quad (24.10)$$

Thus, we have converted the single second-order equation ( Eq. 24.8) into a pair of first-order ODEs [ Eqs. (24.9) and  (24.10)].

If we had initial conditions for both  $T$  and  $z$ , we could solve these equations as an initial-value problem with the methods described in  Chaps. 22 and  23. However, because we only have an initial value for one of the variables  $T(0) = T_a$ , we simply make a guess for the other  $z(0) = z_{a1}$  and then perform the integration.

After performing the integration, we will have generated a value of  $T$  at the end of the interval, which we will call  $T_{b1}$ . Unless we are incredibly lucky, this result will differ from the desired result  $T_b$ .

Now, let's say that the value of  $T_{b1}$  is too high ( $T_{b1} > T_b$ ), it would make sense that a lower value of the initial slope  $z(0) = z_{a2}$  might result in a better prediction. Using this new guess, we can integrate again to generate a second result at the end of the interval  $T_{b2}$ . We could then continue guessing in a trial-and-error fashion until we arrived at a guess for  $z(0)$  that resulted in the correct value of  $T(L) = T_b$ .

At this point, the origin of the name *shooting method* should be pretty clear. Just as you would adjust the angle of a cannon in order to hit a target, we are adjusting the trajectory of our solution by guessing values of  $z(0)$  until we hit our target  $T(L) = T_b$ .

Although we could certainly keep guessing, a more efficient strategy is possible for linear ODEs. In such cases, the trajectory of the perfect shot  $z_a$  is linearly related to the results of our two erroneous shots  $(z_{a1}, T_{b1})$  and  $(z_{a2}, T_{b2})$ . Consequently, linear interpolation can be employed to arrive at the required trajectory:

$$z_a = z_{a1} + \frac{z_{a2} - z_{a1}}{T_{b2} - T_{b1}}(T_b - T_{b1}) \quad (24.11)$$

The approach can be illustrated by an example.

## EXAMPLE 24.2

### The Shooting Method for a Linear ODE

#### Problem Statement

Use the shooting method to solve [Eq. \(24.6\)](#) for the same conditions as [Example 24.1](#):  $L = 10$  m,  $h' = 0.05$  m<sup>-2</sup>,  $T_\infty = 200$  K,  $T(0) = 300$  K, and  $T(10) = 400$  K.

#### Solution

[Equation \(24.6\)](#) is first expressed as a pair of first-order ODEs:

$$\begin{aligned} \frac{dT}{dx} &= z \\ \frac{dz}{dx} &= -0.05(200 - T) \end{aligned}$$

Along with the initial value for temperature,  $T(0) = 300$  K, we arbitrarily guess a value of  $z_{a1} = -5$  K/m for the initial value  $z(0)$ . The solution is then obtained by integrating the pair of ODEs from  $x = 0$  to 10. We can do this with Python's `solve_ivp` function by first setting up a function to compute the derivatives using the ODEs:

```
def dydx(x,y):
    T = y[0] ; z = y[1]
    dy = np.zeros((2))
    dy[0] = z
```

```
dy[1] = -0.05*(200.-T)
return dy
```

We can then generate the solution as follows:

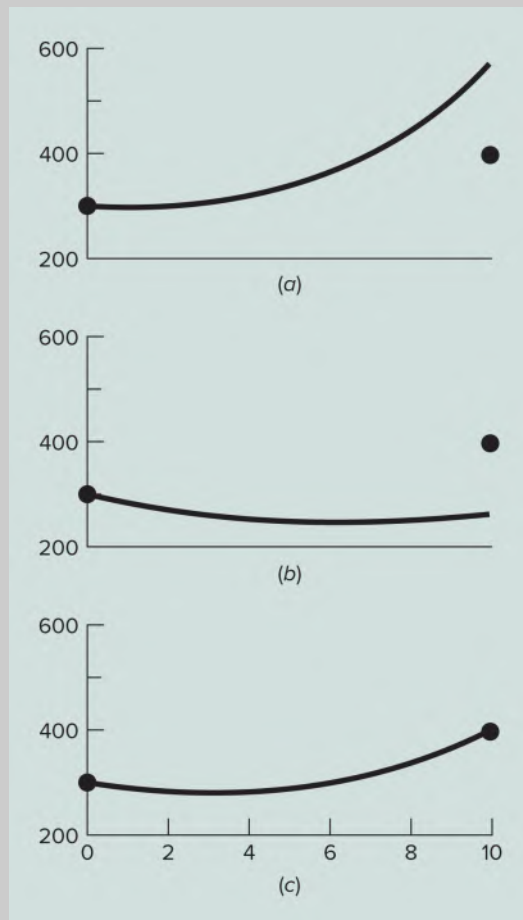
```
import numpy as np
from scipy.integrate import solve_ivp
xi = 0.; xf = 10.
xspan = np.linspace(xi,xf)
y0 = np.array([300.,-5.])
res = solve_ivp(dydx,(xi,xf),y0,t_eval=xspan)
t = res.t
T = res.y[0,:]
n = len(t)
print('Tb1 = {0:7.2f} K'.format(T[n-1]))
```

with the result displayed as

```
Tb1 = 569.77 K
```

Thus, we obtain a value at the end of the interval of  $T_{b1} = 569.77$  K ([Fig. 24.4a](#)), which differs from the desired boundary condition of  $T_b = 400$  K. Therefore, we make another guess,  $z_{a2} = -20$  and perform the computation again.

**FIGURE 24.4** Temperature (K) versus distance (m) computed with the shooting method: (a) the first “shot,” (b) the second “shot,” and (c) the final, exact “hit.”







```
y0 = np.array([300., -20.])
Tb1 = 259.53 K
```

Page 597

This time the result is low of the target ([Fig. 24.4b](#)).

Now, because the ODE is linear, we can use [Eq. \(24.11\)](#) to determine the correct trajectory to yield a perfect shot:

$$z_a = -5 + \frac{-20 - (-5)}{259.53 - 569.77}(400 - 569.77) \cong -13.208$$

This value can then be used in conjunction with `solve_ivp` to generate the correct solution, as depicted in [Fig. 24.4c](#).

Although it is not obvious from the graph, the analytical solution is also plotted on [Fig. 24.4c](#). Thus, the shooting method yields a solution that is virtually indistinguishable from the exact result.

## 24.2.1 Derivative Boundary Conditions

The fixed or Dirichlet boundary condition discussed to this point is but one of several types that are used in engineering and science. A common alternative is the case where the derivative is given as one boundary condition. This is referred to as a *Neumann boundary condition*.

Because it is already set up to compute both the dependent variable and its derivative, incorporating derivative boundary conditions into the shooting method is relatively straightforward.

Just as with the fixed boundary condition case, we first express the second-order ODE as a pair of first-order ODEs. At this point, one of the required initial conditions, whether for the dependent variable or its derivative, will be unknown. Based on guesses for the missing initial condition, we generate solutions to compute the given end condition. As with the initial condition, this end condition can be either for the dependent variable or for its derivative. For linear ODEs, interpolation can then be used to determine the value of the missing initial condition required to generate the final, perfect “shot” that hits the desired end condition.

Page 598

### EXAMPLE 24.3

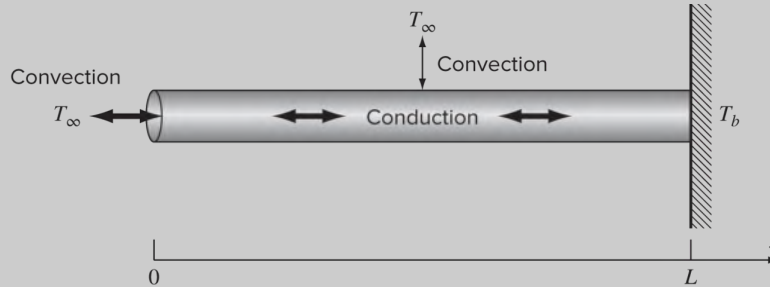
#### The Shooting Method with Derivative Boundary Conditions

##### Problem Statement

Use the shooting method to solve [Eq. \(24.6\)](#) for the rod in [Example 24.1](#):  $L = 10$  m,  $h' = 0.05$  m<sup>-2</sup>,  $h = 1$  W/(m<sup>2</sup>·K),  $r = 0.2$  m,  $k = 200$  W/(m·K),  $T_\infty = 200$  K, and

$T(0) = 400$  K. However, for this case, rather than having a fixed temperature of 300 K, the left end is subject to convective heat transfer as in [Fig. 24.5](#).

**FIGURE 24.5** A rod with a convective boundary condition at one end and a fixed temperature at the other.



For simplicity, we will assume that the convective heat transfer coefficient for the end area of the rod is the same as that for the rod's surface.

## Solution

As in [Example 24.2](#), [Eq. \(24.6\)](#) is first expressed as

$$\begin{aligned}\frac{dT}{dx} &= z \\ \frac{dz}{dx} &= -0.05(200 - T)\end{aligned}$$

Although it might not be obvious, convection at the end of the rod is equivalent to specifying a gradient or derivative boundary condition. In order to see this, we must recognize that, because the system is at steady state, convection must equal conduction at the rod's left boundary ( $x = 0$ ).

Using Fourier's law [[Eq. \(24.5\)](#)] to represent conduction, the energy balance at the end can be formulated as

$$hA_c(T_\infty - T(0)) = -kA_c \frac{dT}{dx}(0) \quad (24.12)$$

If we guess a value for temperature, we can see that this equation specifies the gradient.

The shooting method is implemented by guessing a value for  $T(0)$  arbitrarily. If we choose a value of  $T(0) = T_{a1} = 300$  K, [Eq. \(24.13\)](#) then yields the initial value for the gradient

$$(24.13)$$

$$z_{a1} = \frac{dT}{dx}(0) = \frac{1}{200}(300 - 200) = 0.5$$

The solution is obtained by integrating the pair of ODEs from  $x = 0$  to 10. We can accomplish this with Python's `solve_ivp` function by first setting up a function to compute the derivative values using the ODEs in the same fashion as in [Example 24.2](#). We can then generate the solution as

```
xi = 0.; xf = 10.
xspan = np.linspace(xi,xf)
y0 = np.array([300.,0.5])
res = solve_ivp(dydx,(xi,xf),y0,t_eval=xspan)
t = res.t
T = res.y[0,:]
n = len(t)
print('Tb1 = {0:7.2f} K'.format(T[n-1]))

Tb1 = 683.53 K
```

As expected, the value at the end of the interval of  $T_{b1} = 683.53$  K differs from the desired boundary condition of  $T_b = 400$  K. Therefore, we make another guess  $T_{a2} = 150$  K, which corresponds to  $z_{a2} = -0.25$ , and perform the computation again.

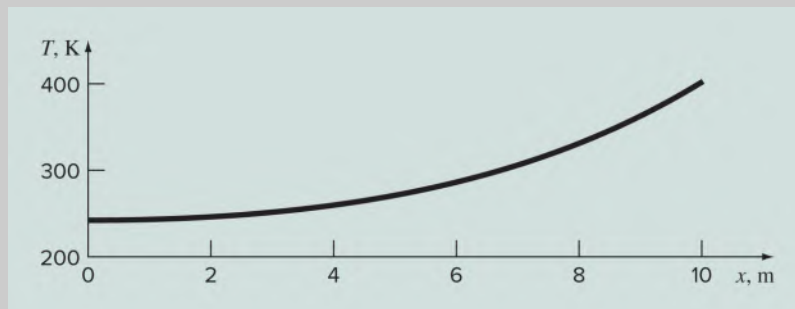
```
y0 = np.array([150.,-0.25])
Tb1 = -41.76 K
```

Linear interpolation can then be employed to compute the correct initial temperature:

$$T_a = 300 + \frac{150 - 300}{-41.76 - 683.53}(400 - 683.53) \cong 241.36 \text{ K}$$

which corresponds to a gradient of  $z_a = 0.207$ . Using these initial conditions, we can use `solve_ivp` to obtain the correct solution, as shown in [Fig. 24.6](#).

**FIGURE 24.6** The solution of a second-order ODE with a convective boundary condition at one end and a fixed temperature at the other.



Note that we can verify that our boundary condition has been satisfied by substituting the initial conditions into [Eq. \(24.12\)](#) to give

$$1 \frac{\text{W}}{\text{m}^2 \cdot \text{K}} \pi (0.2 \text{ m})^2 (200 - 241.36 \text{ K}) \stackrel{?}{=} -200 \frac{\text{W}}{\text{m} \cdot \text{K}} \pi (0.2 \text{ m})^2 0.207 \frac{\text{K}}{\text{m}}$$

which can be evaluated to yield  $-5.20 \text{ W} = -5.20 \text{ W}$ . Thus, the conduction and convection are approximately equal and transfer heat out of the left end of the rod at a rate of 5.2 W.

## 24.2.2 The Shooting Method for Nonlinear ODEs

For nonlinear boundary-value problems, linear interpolation or extrapolation across two solution points will not necessarily result in an accurate estimate of the required boundary condition to attain an exact solution. An alternative is to perform three applications of the shooting method and use a quadratic interpolating polynomial to estimate the proper boundary condition. However, it is unlikely that such an approach would yield the exact answer, and additional iterations would be necessary to home in on the solution.

Another approach for a nonlinear problem involves recasting it as a roots problem. Recall that the general goal of a root-finding task is to find a value of  $x$  that makes the function  $f(x) = 0$ . Now, let us use the heated rod problem to understand how the shooting method can be recast in this form.

First, we recognize that the solution of a pair of differential equations is also a “function” in the sense that we guess a condition at the left end of the rod,  $z_a$ , and the integration yields a prediction of the temperature at the right end,  $T_b$ . Thus, we can think of the integration as representing a function

$$T_b = f(z_a)$$

That is, it represents a process whereby a guess of  $z_a$  yields a prediction of  $T_b$ . Viewed in this way, we can see that what we desire is the value of  $z_a$  that yields a target value of  $T_b$ . If, as in the example, we desire  $T_b = 400 \text{ K}$ , the problem can then be posed as

Page 600

$$400 = f(z_a)$$

By bringing the goal or *target* temperature over to the right-hand side of the equation, we generate a new function  $res(z_a)$  that represents the difference, error, or *residual* between what we have,  $f(z_a)$ , and what we want, 400.

$$res(z_a) = f(z_a) - 400$$

If we drive  $res(z_a)$  to zero, we obtain the solution. The following example illustrates the approach.

## EXAMPLE 24.4

### The Shooting Method for Nonlinear ODEs

#### Problem Statement

Although it served our purposes for illustrating the shooting method, [Eq. \(24.6\)](#) was not a completely realistic model for a heated rod. For one thing, such a rod would lose heat by mechanisms such as radiation that are nonlinear.

Suppose that the following nonlinear ODE is used to simulate the temperature of the heated rod:

$$0 = \frac{d^2T}{dx^2} + h'(T_\infty - T) + \sigma'(T_\infty^4 - T^4)$$

where  $\sigma'$  = a bulk heat transfer parameter reflecting the relative impacts of radiation and conduction ( $2.7 \times 10^{-9} \text{ K}^{-3}\text{m}^{-2}$ ). Note: Absolute temperature must be used in the radiation term. This equation can serve to illustrate how the shooting method is used to solve a two-point boundary-value problem. The remaining problem conditions are as specified in [Example 24.2](#):  $L = 10 \text{ m}$ ,  $h' = 0.05 \text{ m}^{-2}$ ,  $T_\infty = 200 \text{ K}$ ,  $T(0) = 300 \text{ K}$ , and  $T(L) = 400 \text{ K}$ .

#### Solution

Just as with the linear ODE, the nonlinear second-order equation is first decomposed into two first-order ODEs:

$$\begin{aligned}\frac{dT}{dx} &= z \\ \frac{dz}{dx} &= -h'(T_\infty - T) - \sigma'(T_\infty^4 - T^4)\end{aligned}$$

A Python function is written to compute the derivatives based on the differential equations above:

```
def dydx(x,y,hp,sigp,Tinf):
    T = y[0] ; z = y[1]
    dy = np.zeros((2))
    dy[0] = z
    dy[1] = -hp*(Tinf-T) - sigp*(Tinf**4-T**4)
    return dy
```

Next, we can build a function to compute the residual, which we will attempt to drive to zero, as

```
def res(za,hp,sigp,L,Tinf,T0,TL):
    xi = 0. ; xf = L
    y0 = np.array([T0,za])
    result = solve_ivp(dydx,(xi,xf),y0,args=(hp,sigp,Tinf))
    n = len(result.t)
    return result.y[0,n-1]-TL
```

Notice how we use the `solve_ivp` function to solve the two ODEs to generate the temperature at the rod's end, `result.y[n-1]`. We can then find the root with the `brentq` function from the SciPy `optimize` submodule.

```
L = 10 # m
hp = 0.05 # 1/m2
sigp = 2.7e-9 # 1/(K3*m2)
Tinf = 200. # K
TL = 400. # K
T0 = 300. # K

za_soln = brentq(res, -10., -100., args=(hp, sigp, L, Tinf, T0, TL))
print('za = {0:7.2f} K/m'.format(za_soln))
```

The result for  $z_a$  is then

```
za = -41.74 K/m
```

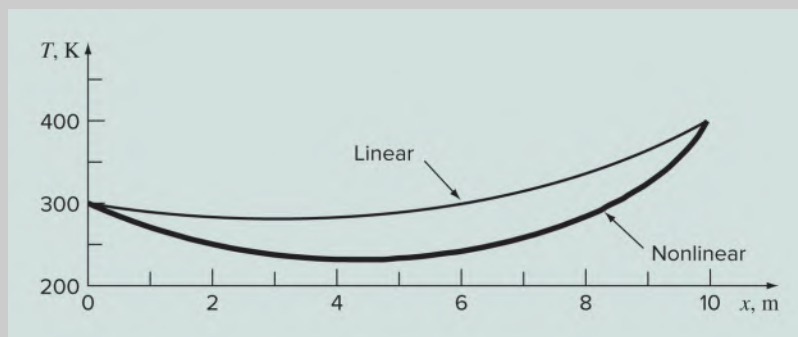
This can be verified by generating the solution with this value for  $z_a$  and plotting temperature versus  $x$ .

```
xi = 0.; xf = L
xspan = np.linspace(xi, xf)
y0 = np.array([T0, za_soln])
sol_out = solve_ivp(dydx, (xi, xf), y0, t_eval=xspan, args=(hp, sigp, Tinf))
x = sol_out.t
T = sol_out.y[0,:]
```

```
pylab.plot(x, T, c='k')
pylab.grid()
pylab.xlabel('distance - m')
pylab.ylabel('temperature - K')
```

The result is shown in a plot in [Fig. 24.7](#) along with the original linear case from [Example 24.2](#). As expected, the nonlinear case is depressed below the linear model due to the additional energy lost to the surroundings by radiation.

**FIGURE 24.7** The result of using the shooting method to solve a nonlinear problem.



## 24.3 FINITE-DIFFERENCE METHODS

The most common alternatives to the shooting method are finite-difference approaches. In these techniques, finite differences (🔗 Chap. 21) are substituted for the derivatives in the original equation. Thus, a linear differential equation is transformed into a set of simultaneous algebraic equations that can be solved using the methods from Part Three.

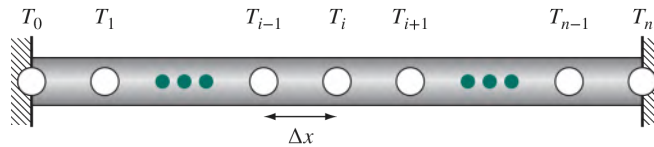
We can illustrate the approach for the heated rod model [🔗 Eq. (24.6)]:

$$0 = \frac{d^2T}{dx^2} + h'(T_\infty - T) \quad (24.14)$$

The solution domain is first divided into a series of nodes (🔗 Fig. 24.8). At each node, finite-difference approximations can be written for the derivatives in the equation. For example, at node  $i$ , the second derivative can be represented by (🔗 Fig. 21.5)

$$\frac{d^2T}{dx^2} = \frac{T_{i-1} - 2T_i + T_{i+1}}{\Delta x^2} \quad (24.15)$$

**FIGURE 24.8** In order to implement the finite-difference approach, the heated rod is divided into a series of nodes.



This approximation can be substituted into 🔗 Eq. (24.14) to give

$$\frac{T_{i-1} - 2T_i + T_{i+1}}{\Delta x^2} + h'(T_\infty - T_i) = 0$$

Thus, the differential equation has been converted into an algebraic equation. Collecting terms gives

$$-T_{i-1} + (2 + h'\Delta x^2)T_i - T_{i+1} = h'\Delta x^2 T_\infty \quad (24.16)$$

This equation can be written for each of the  $n - 1$  interior nodes of the rod. The first and last nodes  $T_0$  and  $T_n$ , respectively, are specified by the boundary conditions. Therefore, the problem reduces to solving  $n - 1$  simultaneous linear algebraic equations for the  $n - 1$  unknowns.

Before providing an example, we should mention two nice features of [Eq. \(24.16\)](#). First, observe that since the nodes are numbered consecutively, and since each equation consists of a node ( $i$ ) and its adjoining neighbors ( $i - 1$  and  $i + 1$ ), the resulting set of linear algebraic equations will be tridiagonal. As such, they can be solved with the efficient algorithms that are available for such systems (recall [Sec. 9.4](#)).

Further, inspection of the coefficients on the left-hand side of [Eq. \(24.16\)](#) indicates that the system of linear equations will also be diagonally dominant. Hence, convergent solutions can also be generated with iterative techniques like the Gauss–Seidel method ([Sec. 12.1](#)).

## EXAMPLE 24.5

### Finite-Difference Approximation of Boundary-Value Problems

#### Problem Statement

Use the finite-difference approach to solve the same problem as in [Examples 24.1](#) and [24.2](#). Use four interior nodes with a segment length of  $\Delta x = 2$  m.

#### Solution

Employing the parameters in [Example 24.1](#) and  $\Delta x = 2$  m, we can write [Eq. \(24.16\)](#) for each of the rod's interior nodes. For example, for node 1:

$$-T_0 + 2.2T_1 - T_2 = 40$$

Substituting the boundary condition  $T_0 = 300$  gives

$$2.2T_1 - T_2 = 340$$

After writing [Eq. \(24.16\)](#) for the other interior nodes, the equations can be assembled in matrix form as

$$\begin{bmatrix} 2.2 & -1 & 0 & 0 \\ -1 & 2.2 & -1 & 0 \\ 0 & -1 & 2.2 & -1 \\ 0 & 0 & -1 & 2.2 \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} = \begin{bmatrix} 340 \\ 40 \\ 40 \\ 440 \end{bmatrix}$$



Notice the matrix is both tridiagonal and diagonally dominant. A Python script to compute the solution is

```
import numpy as np

A = np.array([[2.2, -1., 0., 0.],
              [-1., 2.2, -1., 0.],
              [0., -1., 2.2, -1.],
              [0., 0., -1., 2.2]])
b = np.array([340., 40., 40., 440.])
T = np.linalg.solve(A,b)
for i in range(4):
    print(' {0:7.2f}'.format(T[i]))
```

The resulting temperatures are

```
283.27
283.19
299.74
336.25
```

Page 603

 **Table 24.1** provides a comparison of the analytical solution ( **Eq. 24.7**) and the numerical solutions obtained with the shooting method ( **Example 24.2**) and the finite-difference method ( **Example 24.5**). Note that although there are discrepancies, the numerical approaches agree reasonably well with the analytical solution. Further, the biggest discrepancy occurs for the finite-difference method because of the coarse node spacing we used in  **Example 24.5**. Better agreement would occur if a finer spacing of nodes had been used.

**TABLE 24.1** Comparison of the exact analytical solution for temperature with the results obtained with the shooting and finite-difference methods.

| x  | Analytical Solution | Shooting Method | Finite-Difference |
|----|---------------------|-----------------|-------------------|
| 0  | 300                 | 300             | 300               |
| 2  | 282.8634            | 282.8889        | 283.2660          |
| 4  | 282.5775            | 282.6158        | 283.1853          |
| 6  | 299.0843            | 299.1254        | 299.7416          |
| 8  | 335.7404            | 335.7718        | 336.2462          |
| 10 | 400                 | 400             | 400               |

## 24.3.1 Derivative Boundary Conditions

As mentioned in our discussion of the shooting method, the fixed or *Dirichlet boundary condition* is but one of several types that are used in engineering and science. A common alternative, called the *Neumann boundary condition*, is the case where the derivative is given.

We can use the heated rod introduced earlier in this chapter to demonstrate how a derivative boundary condition can be incorporated into the finite-difference approach:

$$0 = \frac{d^2T}{dx^2} + h'(T_\infty - T)$$

However, in contrast to our previous discussions, we will prescribe a derivative boundary condition at one end of the rod:

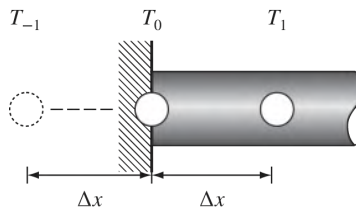
$$\begin{aligned}\frac{dT}{dx}(0) &= T'_a \\ T(L) &= T_b\end{aligned}$$

Thus, we have a derivative boundary condition at one end of the solution domain and a fixed boundary condition at the other.

Just as in the previous section, the rod is divided into a series of nodes and a finite-difference version of the differential equation ([Eq. 24.16](#)) is applied to each interior node. However, because its temperature is not specified, the node at the left end must also be included. [Figure 24.9](#) depicts the node (0) at the left edge of a heated plate for which the derivative boundary condition applies. Writing [Eq. \(24.16\)](#) for this node gives

$$-T_{-1} + (2 + h' \Delta x^2)T_0 - T_1 = h' \Delta x^2 T_\infty \quad (24.17)$$

**FIGURE 24.9** A boundary node at the left end of a heated rod. To approximate the derivative at the boundary, an imaginary node is located a distance  $\Delta x$  to the left of the rod's end.



Notice that an imaginary node ( $-1$ ) lying to the left of the rod's end is required for this equation. Although this exterior point might seem to represent a difficulty, it actually serves as the vehicle for incorporating the derivative boundary condition into the problem. This is done by representing the first derivative in the  $x$  dimension at (0) by the centered difference [[Eq. \(4.25\)](#)]:

$$\frac{dT}{dx} = \frac{T_1 - T_{-1}}{2\Delta x}$$

which can be solved for

$$T_{-1} = T_1 - 2\Delta x \frac{dT}{dx}$$

Now we have a formula for  $T_{-1}$  that actually reflects the impact of the derivative. It can be substituted into [Eq. \(24.17\)](#) to give

$$(2 + h'\Delta x^2)T_0 - 2T_1 = h'\Delta x^2T_\infty - 2\Delta x \frac{dT}{dx} \quad (24.18)$$

Consequently, we have incorporated the derivative into the balance.

A common example of a derivative boundary condition is the situation where the end of the rod is insulated. In this case, the derivative is set to zero. This conclusion follows directly from Fourier's law [[Eq. \(24.5\)](#)], because insulating a boundary means that the heat flux (and consequently the gradient) must be zero. The following example illustrates how the solution is affected by such boundary conditions.

## EXAMPLE 24.6

### Incorporating Derivative Boundary Conditions

#### Problem Statement

Generate the finite-difference solution for a 10-m rod with  $\Delta x = 2$  m,  $h' = 0.05 \text{ m}^{-2}$ ,  $T_\infty = 200$  K, and the boundary conditions:  $T'_a = 0$  and  $T_b = 400$  K. Note that the first condition means that the slope of the solution should approach zero at the rod's left end. Aside from this case, also generate the solution for  $dT/dx = -20$  at  $x = 0$ .

#### Solution

[Equation \(24.18\)](#) can be used to represent node 0 as

$$2.2T_0 - 2T_1 = 40$$

We can write [Eq. \(24.16\)](#) for the interior nodes. For example, for node 1,

$$-T_0 + 2.2T_1 - T_2 = 40$$

A similar approach can be used for the remaining interior nodes. The final system of equations can be assembled in matrix form as

$$\begin{bmatrix} 2.2 & -2 & & & \\ -1 & 2.2 & -1 & & \\ & -1 & 2.2 & -1 & \\ & & -1 & 2.2 & -1 \\ & & & -1 & 2.2 \end{bmatrix} \begin{bmatrix} T_0 \\ T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} = \begin{bmatrix} 40 \\ 40 \\ 40 \\ 40 \\ 440 \end{bmatrix}$$

These equations can be solved for

Page 605

$$T_0 = 243.0278$$

$$T_1 = 247.3306$$

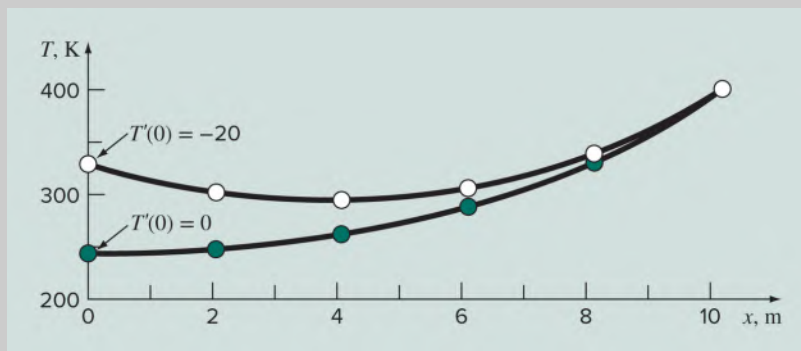
$$T_2 = 261.0994$$

$$T_3 = 287.0882$$

$$T_4 = 330.4946$$

As displayed in [Fig. 24.10](#), the solution is flat at  $x = 0$  due to the zero derivative condition and then curves upward to the fixed condition of  $T = 400$  at  $x = 10$ .

**FIGURE 24.10** The solution of a second-order ODE with a derivative boundary condition at one end and a fixed boundary condition at the other. Two cases are shown reflecting different derivative values at  $x = 0$ .



For the case where the derivative at  $x = 0$  is set to  $-20$ , the simultaneous equations are

$$\begin{bmatrix} 2.2 & -2 & & & \\ -1 & 2.2 & -1 & & \\ & -1 & 2.2 & -1 & \\ & & -1 & 2.2 & -1 \\ & & & -1 & 2.2 \end{bmatrix} \begin{bmatrix} T_0 \\ T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} = \begin{bmatrix} 120 \\ 40 \\ 40 \\ 40 \\ 440 \end{bmatrix}$$

which can be solved for

$$T_0 = 328.2710$$

$$T_1 = 301.0981$$

$$T_2 = 294.1448$$

$$T_3 = 306.0204$$

$$T_4 = 339.1002$$

As in [Fig. 24.10](#), the solution at  $x = 0$  now curves downward due to the negative derivative we imposed at the boundary.

## 24.3.2 Finite-Difference Approaches for Nonlinear ODEs

For nonlinear ODEs, the substitution of finite differences yields a system of nonlinear simultaneous equations. Thus, the most general approach to solving such problems is to use root-location methods for systems of equations such as the Newton-Raphson method described in [Sec. 12.2.2](#). Although this approach is certainly feasible, an adaptation of successive substitution can sometimes provide a simpler alternative.

The heated rod with convection and radiation introduced in [Example 24.4](#) provides a nice vehicle for demonstrating this approach,

Page 606

$$0 = \frac{d^2T}{dx^2} + h'(T_\infty - T) + \sigma''(T_\infty^4 - T^4)$$

We can convert this differential equation into algebraic form by writing it for a node  $i$  and substituting [Eq. \(24.15\)](#) for the second derivative:

$$0 = \frac{T_{i-1} - 2T_i + T_{i+1}}{\Delta x^2} + h'(T_\infty - T_i) + \sigma''(T_\infty^4 - T_i^4)$$

Collecting terms gives

$$-T_{i-1} + (2 + h'\Delta x^2)T_i - T_{i+1} = h'\Delta x^2 T_\infty + \sigma''\Delta x^2(T_\infty^4 - T_i^4)$$

Notice that although there is a nonlinear term on the right-hand side, the left-hand side is expressed in the form of a linear algebraic system that is diagonally dominant. If we assume that the unknown nonlinear term on the right is equal to its value from the previous iteration, the equation can be solved for

$$T_i = \frac{h'\Delta x^2 T_\infty + \sigma''\Delta x^2(T_\infty^4 - T_i^4) + T_{i-1} + T_{i+1}}{2 + h'\Delta x^2} \quad (24.19)$$

As in the Gauss-Seidel method, we can use [Eq. \(24.19\)](#) to successively calculate the temperature of each node and iterate until the process converges to an acceptable tolerance. Although this approach will not work for all cases, it converges for many ODEs derived from physically based systems. Hence, it can sometimes prove useful for solving problems routinely encountered in engineering and science.

## EXAMPLE 24.7

### The Finite-Difference Method for Nonlinear ODEs

#### Problem Statement

Use the finite-difference approach to simulate the temperature of a heated rod subject to both convection and radiation:

$$0 = \frac{d^2 T}{dx^2} + h'(T_\infty - T) + \sigma''(T_\infty^4 - T^4)$$

where  $\sigma' = 2.7 \times 10^{-9} \text{ K}^{-3} \text{ m}^{-2}$ ,  $L = 10 \text{ m}$ ,  $h' = 0.05 \text{ m}^{-2}$ ,  $T_\infty = 200 \text{ K}$ ,  $T(0) = 300 \text{ K}$ , and  $T(10) = 400 \text{ K}$ . Use four interior nodes with a segment length of  $\Delta x = 2 \text{ m}$ . Recall that we solved the same problem with the shooting method in [Example 24.4](#).

#### Solution

Using [Eq. \(24.19\)](#) we can successively solve for the temperatures of the rod's interior nodes. As with the standard Gauss-Seidel technique, the initial values of the interior nodes are zero with the boundary nodes set at the fixed conditions of  $T_0 = 300$  and  $T_5 = 400$ . The results for the first iteration are

$$T_1 = \frac{0.05(2)^2 200 + 2.7 \times 10^{-9}(2)^2(200^4 - 0^4) + 300 + 0}{2 + 0.05(2)^2} = 159.2432$$

$$T_2 = \frac{0.05(2)^2 200 + 2.7 \times 10^{-9}(2)^2(200^4 - 0^4) + 159.2432 + 0}{2 + 0.05(2)^2} = 97.9674$$

$$T_3 = \frac{0.05(2)^2 200 + 2.7 \times 10^{-9}(2)^2(200^4 - 0^4) + 97.9674 + 0}{2 + 0.05(2)^2} = 70.4461$$

$$T_4 = \frac{0.05(2)^2 200 + 2.7 \times 10^{-9}(2)^2(200^4 - 0^4) + 70.4461 + 400}{2 + 0.05(2)^2} = 226.8704$$

The process can be continued until we converge on the final result:

Page 607

$$T_0 = 300$$

$$T_1 = 250.4827$$

$$T_2 = 236.2962$$

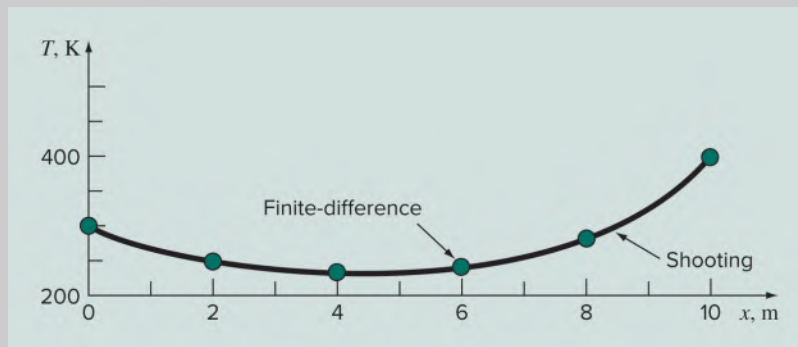
$$T_3 = 245.7596$$

$$T_4 = 286.4921$$

$$T_5 = 400$$

These results are displayed in [Fig. 24.11](#) along with the result generated in [Example 24.4](#) with the shooting method.

**FIGURE 24.11** The filled circles are the result of using the finite-difference method to solve a nonlinear problem. The line generated with the shooting method in [Example 24.4](#) is shown for comparison.



## 24.4 PYTHON FUNCTION: `solve_bvp`

The `solve_bvp` function from the SciPy `integrate` submodule solves ODE boundary-value problems based on a more advanced finite-difference algorithm called *fourth-order collocation* coupled with a *damped* Newton's method. A simplified syntax for `solve_bvp` is

```
from scipy.integrate import solve_bvp
sol = solve_bvp(func, bc, x, y)
```

where `sol` is an object result containing various solution elements, `func` is a function that provides the derivatives based on the ODEs, `bc` is another function that evaluates residuals at the boundary conditions, `x` is an  $m \times 1$  array defining the  $m$  mesh or nodes of the independent variable, and `y` is an  $n \times m$  array of initial guesses for the dependent variable(s) at the node locations.

The format for `func` is

```
def func(x,y):
    .
    .
    return dy
```

where `x` = a value of the independent variable and `y` = an array of  $n$  dependent variable values.

The format for `bc` is

```
def bc(ya,yb):
    .
    .
    return res
```

where `ya` and `yb` are  $n \times 1$  arrays of beginning and end values of the dependent variables, and `res` = an array of  $n$  residuals referencing the `ya` and `yb` values.

Page 608

Note that, whereas your choices for the initial mesh and dependent variable guesses will not be of importance for linear ODEs, they can often be critical for solving nonlinear equations successfully and efficiently.

### EXAMPLE 24.8

#### Solving a Boundary-Value Problem with `solve_bvp`

##### Problem Statement

Use `solve_bvp` to solve the following second-order ODE:

$$\frac{d^2y}{dx^2} + y = 1$$

subject to the boundary conditions



$$y(0) = 1 \quad y(\pi/2) = 0$$

## Solution

First, we express the second-order equation as a pair of first-order ODEs.

$$\begin{aligned} \frac{dy}{dx} &= z \\ \frac{dz}{dx} &= 1 - y \end{aligned}$$

Next, we write a function to compute the derivatives based on the ODEs.

```
def dydx(x,y):
    dy = []
    dy.append(y[1])
    dy.append(1.-y[0])
    return dy
```

We can now develop the function to specify the boundary condition residuals. This is done similarly to a roots problem in that we specify two conditions that should evaluate to zero when the boundary conditions are satisfied. To do this, starting with the `ya` and `yb` arrays of left and right boundary values, the first condition  $y(0) = 1$  is formulated as `ya[0]-1`, and the second condition  $y(\pi/2) = 0$  corresponds to `yb[0]` alone.

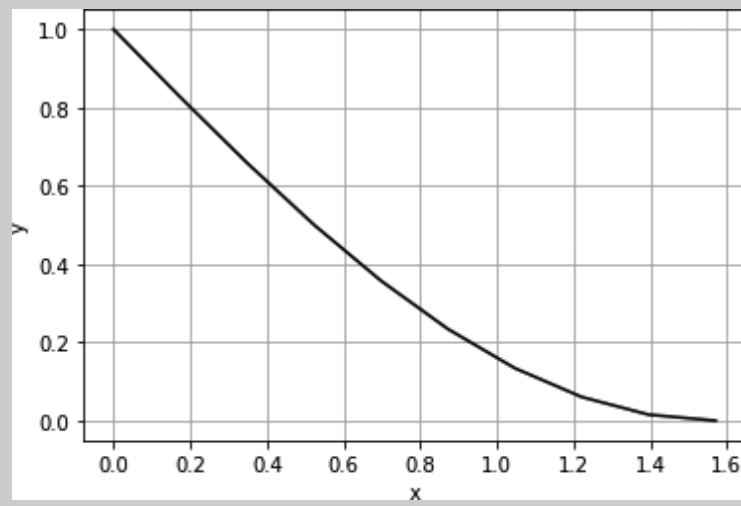
```
def bc(ya,yb):
    return np.array([ya[0]-1,yb[0]])
```

Finally, we can write the script to generate the solution utilizing a mesh of ten equally spaced points and initial guesses of  $y = 1$  and  $z = dy/dx = -1$  throughout the mesh.

```
x = np.linspace(0.,np.pi/2,10)
y = np.zeros((2,10))
y[0,:]= 1 ; y[1,:]= -1
sol = solve_bvp(dydx,bc,x,y)
y = sol.y[0,:]

pylab.plot(x,y,c='k')
pylab.grid()
pylab.xlabel('x')
pylab.ylabel('y')
```

When the script is run, the plot below is generated. Note that the script and functions developed in this example can be applied to other boundary-value problems with minor adjustments. Several problems at the end of the chapter are included to provide the opportunity for you to learn how to do this.



## PROBLEMS

**24.1** A steady-state energy balance for a rod can be represented as

$$\frac{d^2T}{dx^2} - 0.15T = 0$$

Obtain a solution for a  $L = 10$ -m rod with  $T(0) = 240$  and  $T(L) = 150$

- (a) analytically,
- (b) with the shooting method, and
- (c) using the finite-difference approach with  $\Delta x = 1$  m.

**24.2** Repeat [Prob. 24.1](#) but with one end of the rod ( $x = L = 10$  m) insulated and the other end ( $x = 0$ ) at a fixed temperature of 240.

**24.3** Use the shooting method to solve

$$7 \frac{d^2y}{dx^2} - 2 \frac{dy}{dx} - y + x = 0$$

with the boundary conditions  $y(0) = 5$  and  $y(20) = 8$ .

**24.4** Solve [Prob. 24.3](#) using the finite-difference approach with  $\Delta x = 2$ .

**24.5** The following nonlinear differential equation was solved in [Examples 24.4](#) and [24.7](#):

$$0 = \frac{d^2T}{dx^2} + h'(T_\infty - T) + \sigma'(T_\infty^4 - T^4) \quad (\text{P24.5})$$

Such equations are sometimes linearized to obtain an approximate solution. This is done by employing a first-order Taylor series expansion to linearize the quartic term in  $T$  in the equation as

$$T^4 \cong \bar{T}^4 + 4 \cdot \bar{T}^3 \cdot (T - \bar{T})$$

where  $\bar{T}$  is the base temperature about which the term is linearized. Substitute this relationship into [Eq. \(P24.5\)](#), and then solve the resulting linear equation with the finite-difference method. Employ  $\bar{T} = 300$ ,  $\Delta x = 1$  m, and the parameters from [Example 24.4](#) to obtain your solution. Plot your results along with those obtained for the nonlinear version in [Example 24.4](#).

**24.6** Develop a Python function called `linshoot` to implement the shooting method for a linear, second-order ODE with Dirichlet boundary conditions of the form

$$a \frac{d^2 y}{dx^2} + b \frac{dy}{dx} + c y = f(x) \quad L_1 \leq x \leq L_2 \quad y(L_1) = y_1 \quad y(L_2) = y_2$$

Test the program by duplicating  **Prob. 24.1b**.

**24.7** Develop a Python function called `lin_finite` to implement the finite-difference method for a linear, second-order ODE with Dirichlet boundary conditions of the form

$$a \frac{d^2 y}{dx^2} + b \frac{dy}{dx} + c y = f(x) \quad L_1 \leq x \leq L_2 \quad y(L_1) = y_1 \quad y(L_2) = y_2$$

Test the program by duplicating  **Example 24.5**.

**24.8** An insulated, heated rod with a uniform heat source can be modeled with the *Poisson equation*:

$$\frac{d^2 T}{dx^2} = -f(x)$$

Given a constant heat source,  $f(x) = 25 \text{ }^\circ\text{C/m}^2$  and the boundary conditions,  $T(0) = 40 \text{ }^\circ\text{C}$  and  $T(10) = 200 \text{ }^\circ\text{C}$ , solve for the temperature distribution with (a) the shooting method and (b) the finite-difference method with  $\Delta x = 2$ .

**24.9** Repeat  **Problem 24.8**, but do so for the following spatially varying heat source:

$$f(x) = 0.12x^3 - 2.4x^2 + 12x$$

**24.10** The temperature distribution in a tapered conical cooling fin ( **Fig. P24.10**) is described by the following differential equation, which has been made dimensionless:

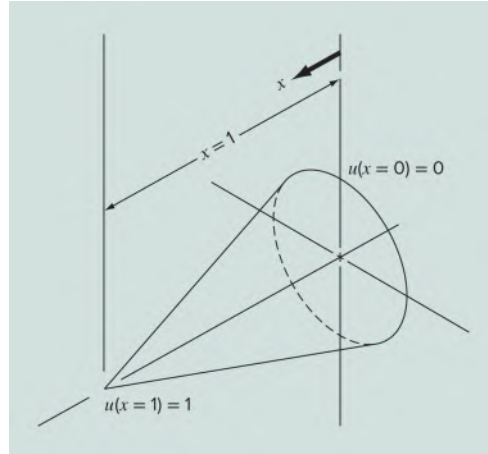
$$\frac{d^2 u}{dx^2} + \left( \frac{2}{x} \right) \left( \frac{du}{dx} - p \right) = 0$$

where  $u$  is the dimensionless temperature ( $0 \leq u \leq 1$ ),  $x$  = the axial distance ( $0 \leq x \leq 1$ ), and  $p$  is a dimensionless parameter that describes the heat transfer geometry:

$$p = \frac{hL}{k} \sqrt{1 + \frac{2}{m^2}}$$

where  $h$  = a heat transfer coefficient,  $k$  = the thermal conductivity of the fin material,  $L$  = the length of the cone, and  $m$  = the slope of the cone wall. The differential equation has boundary conditions:  $u(0) = 1$ ,  $u(1) = 1$ .

**FIGURE P24.10**



Solve this equation for the temperature distribution using finite-difference methods. Use second-order accurate approximations for the derivatives. Write a Python script to obtain the solution and produce a plot of temperature profiles for  $p = 10, 20, 50$ , and  $100$ .

**24.11** Compound  $A$  diffuses through a 4-cm-long tube and reacts as it diffuses. The equation governing diffusion with reaction is

$$D \frac{d^2 A}{dx^2} - kA = 0$$

where  $D$  is the diffusivity ( $\text{cm}^2/\text{s}$ ),  $A$  is the concentration ( $\text{M}$  – molarity), and  $k$  is the rate of reaction ( $\text{s}^{-1}$ ). At one end of the tube ( $x = 0$ ), there is a large source of  $A$  that results in a fixed concentration of  $0.1 \text{ M}$ . At the other end of the tube, there is a material that quickly absorbs and remaining  $A$ , making the concentration there zero. If  $D = 1.5 \times 10^{-6} \text{ cm}^2/\text{s}$  and  $k = 5 \times 10^{-6} \text{ s}^{-1}$ , determine the concentration of  $A$  as a function of distance down the tube.

**24.12** The following differential equation describes the steady-state concentration profile of a substance that reacts with first-order kinetics in tubular reactor with axial dispersion (P24.12):

$$D \frac{d^2 c}{dx^2} - U \frac{dc}{dx} - k c = 0$$

where  $D$  = the dispersion coefficient ( $\text{m}^2/\text{h}$ ),  $c$  = concentration ( $\text{mol/L}$ ),  $x$  = axial distance ( $\text{m}$ ),  $U$  = the velocity ( $\text{m/h}$ ), and  $k$  is the reaction rate ( $\text{h}^{-1}$ ). The boundary conditions can be formulated as

$$U c_{\text{in}} = U c(0) - D \frac{dc}{dx}(0)$$

$$\frac{dc}{dx}(L) = 0$$

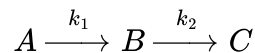
where  $c_{\text{in}}$  = the concentration in the inflow ( $\text{mol/L}$ ), and  $L$  = the length of the reactor tube ( $\text{m}$ ). These are called *Danckwerts boundary conditions*. Use the finite-difference approach to solve for the concentration profile given the following parameter values:  $D = 5000 \text{ m}^2/\text{h}$ ,  $U = 100 \text{ m/h}$ ,  $k = 2 \text{ hr}^{-1}$ ,  $L = 100 \text{ m}$ , and  $c_{\text{in}} = 1 \text{ mol/L}$ . Employ centered-difference approximations with  $\Delta x = 10 \text{ m}$ . Compare your numerical results with the analytical solution:

$$c = \frac{U c_{\text{in}} (\lambda_2 e^{\lambda_2 L} e^{\lambda_1 x} - \lambda_1 \cdot e^{\lambda_1 L} e^{\lambda_2 x})}{(U - D \lambda_1) \lambda_2 e^{\lambda_2 L} - (U - D \lambda_2) \lambda_1 e^{\lambda_1 L}}$$

where the eigenvalues,  $\lambda_1$  and  $\lambda_2$ , are given by

$$\lambda_1, \lambda_2 = \frac{2U}{D} \left( 1 \pm \sqrt{1 + \frac{4kD}{U^2}} \right)$$

**24.13** A series of first-order, liquid-phase reactions creates a desired product,  $B$ , and an undesired by-product,  $C$ :

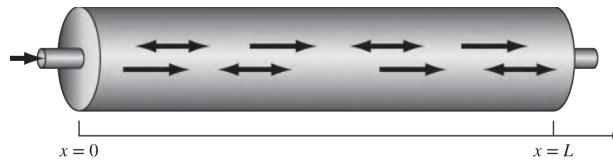


If the reactions take place in a tubular reactor with axial dispersion ( **Fig. P24.12**), steady-state material balances can be used to develop the following set of second-order ODEs:

$$\begin{aligned}
D \frac{d^2 c_a}{dx^2} - U \frac{dc_a}{dx} - k_1 c_a &= 0 \\
D \frac{d^2 c_b}{dx^2} - U \frac{dc_b}{dx} + k_1 c_a - k_2 c_b &= 0 \\
D \frac{d^2 c_c}{dx^2} - U \frac{dc_c}{dx} + k_2 c_b &= 0
\end{aligned}$$

Use the finite-difference approach to solve for the concentration of each substance as a function of distance given the parameter values:  $D = 0.1 \text{ m}^2/\text{min}$ ,  $U = 1 \text{ m/min}$ ,  $k_1 = 3 \text{ min}^{-1}$ ,  $k_2 = 1 \text{ min}^{-1}$ ,  $L = 0.5 \text{ m}$ ,  $c_{a,\text{in}} = 1 \text{ mol/L}$ . Employ centered-difference approximations with  $\Delta x = 0.05 \text{ m}$  to obtain your solutions and assume Danckwerts boundary conditions, as described in [Problem 24.12](#). Also, compute the sum of the substance concentrations as a function of distance. Do your results make sense?

**FIGURE P24.12** Tubular reactor with axial dispersion



**24.14** A biofilm with a thickness  $L_f$  (cm) grows on the surface of a solid ([Fig. P24.14](#)). After traversing a diffusion layer of thickness  $L$  (cm), a chemical compound  $A$  diffuses into the biofilm where it is subject to an irreversible first-order reaction that converts it to product  $B$ . Steady-state material balances can be used to derive the following differential equations for compound  $A$ :

$$\begin{aligned}
D \frac{d^2 c_a}{dx^2} &= 0 & 0 \leq x \leq L \\
D_f \frac{d^2 c_a}{dx^2} - k c_a &= 0 & L \leq x \leq L + L_f
\end{aligned}$$

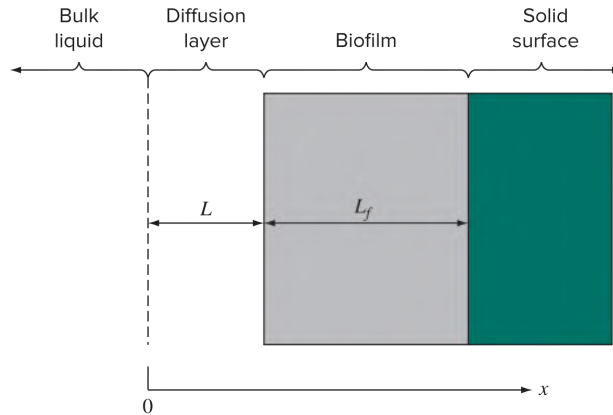
where  $D$  = the diffusion coefficient in the diffusion layer =  $0.08 \text{ cm}^2/\text{d}$ ,  
 $D_f$  = the diffusion coefficient in the biofilm =  $0.04 \text{ cm}^2/\text{d}$ , and  $k$  = the first-order rate parameter for the conversion of  $A$  to  $B$  =  $2000 \text{ d}^{-1}$ . The following boundary conditions hold:

$$\begin{aligned}
c_a &= c_{a0} & \text{at } x = 0 \\
\frac{dc_a}{dx} &= 0 & \text{at } x = L + L_f
\end{aligned}$$

where  $c_{a0}$  is the concentration of A in the bulk liquid = 1 mol/L. Also, note that at the boundary between the diffusion layer and the biofilm, there is continuity in concentration and flux of A. The latter requires that Fick's law is obeyed where

$$D \frac{dc_a}{dx} = D_f \frac{dc'_a}{dx}$$

**FIGURE P24.14** A biofilm growing on a solid surface.



Here,  $c'_a$  represents the concentration in the biofilm. To incorporate the above, you should consider first-order finite-difference approximations. Use the finite-difference approach to compute the steady-state concentration profile of A from  $x = 0$  to  $L + L_f$ , where  $L = 0.008$  cm and  $L_f = 0.004$  cm. Employ center-difference approximations with  $\Delta x = 0.001$  cm.

**24.15** A cable is hanging from two supports, A and B (P24.15). The cable has a distributed load whose magnitude varies with  $x$  as

$$w = w_0 \left[ 1 + \sin\left(\frac{\pi x}{2 l_A}\right) \right]$$

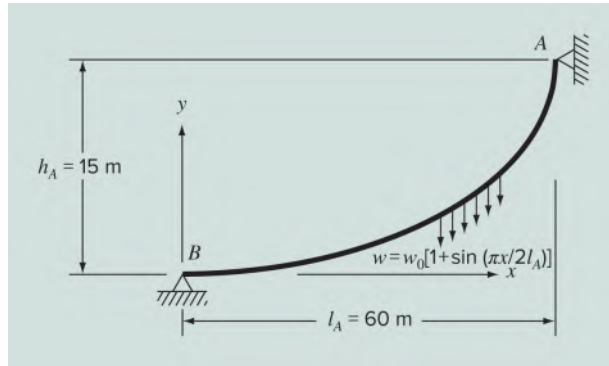
where  $w_0 = 450$  N/m. The slope of the cable ( $dy/dx$ ) = 0 at  $x = 0$ , which is the lowest point for the cable. It is also the point where the tension in the cable is a minimum of  $T_0$ . The differential equation that governs the cable is

$$\frac{d^2 y}{dx^2} = \frac{w_0}{T_0} \left[ 1 + \sin\left(\frac{\pi x}{2 l_A}\right) \right]$$



Solve this equation using a numerical method and plot the shape of the cable ( $y$  vs.  $x$ ). For the numerical solution, the value of  $T_0$  is unknown, so the solution must use an iterative technique, similar to the shooting method, to converge on a correct value of  $h_A$  for various values of  $T_0$ .

**FIGURE P24.15**



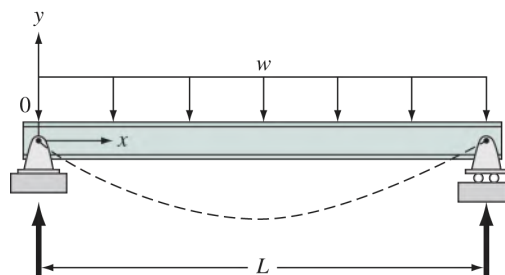
**24.16** The basic differential equation of the elastic curve for a simply supported, uniformly loaded beam (Fig. P24.16) is given as

$$EI \frac{d^2 y}{dx^2} = \frac{w L x}{2} - \frac{w x^2}{2}$$

where  $E$  = the modulus of elasticity and  $I$  = the moment of inertia. The boundary conditions are  $y(0) = y(L) = 0$ . Solve for the deflection of the beam using (a) the finite-difference method ( $\Delta x = 0.6$  m) and (b) the shooting method. The following parameters apply:  $E = 200$  GPa,  $I = 30,000$  cm<sup>4</sup>,  $w = 15$  kN/m, and  $L = 3$  m. Compare your numerical results to the analytical solution:

$$y = \frac{w L x^3}{12 E I} - \frac{w x^4}{24 E I} - \frac{w L^3 x}{24 E I}$$

**FIGURE P24.16**





**24.17** In  **Prob. 24.16**, the basic differential equation of the elastic curve for a uniformly loaded beam was formulated as

$$E I \frac{d^2 y}{dx^2} = \frac{w L x}{2} - \frac{w x^2}{2}$$

Note that the right-hand side represents the moment as a function of  $x$ . An equivalent approach can be expressed as

$$E I \frac{d^4 y}{dx^4} = -w$$

For this formulation, four boundary conditions are required. For the supports shown in  **Fig. P24.16**, the conditions are that the end displacements are zero,  $y(0) = y(L) = 0$ , and that the end moments are also zero,  $y''(0) = y''(L) = 0$ . Solve for the deflection of the beam using the finite-difference method with  $\Delta x = 0.6$  m. The following parameter values apply:  $E = 200$  GPa,  $I = 3 \times 10^{-4}$  m<sup>4</sup>,  $w = 15$  kN/m, and  $L = 3$  m. Compare your results with the analytical solution given in  **Prob. 24.16**.

**24.18** Under several simplifying assumptions, the steady-state height of the water table in a one-dimensional, unconfined, groundwater aquifer ( **Fig. P24.18**) can be modeled with the following second-order ODE:

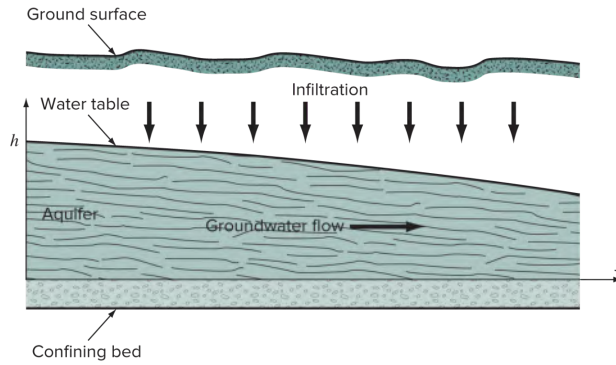
Page 612

$$K \bar{h} \frac{d^2 h}{dx^2} + N = 0$$

where  $x$  = distance (m),  $K$  = hydraulic conductivity (m/d),  $h$  = height of the water table (m),

$\bar{h}$  = average height of the water table (m), and  $N$  = infiltration rate (m/d). Solve for the height of the water table from  $x = 0$  to 1000 m where  $h(0) = 10$  m and  $h(1000) = 5$  m. Use the following parameter values for your computation:  $K = 1$  m/d and  $N = 0.0001$  m/d. Set the average height of the water table as the average of the two boundary conditions. Obtain your solution with (a) the shooting method and (b) the finite-difference method ( $\Delta x = 10$  m).

**FIGURE P24.18** An unconfined or “phreatic” aquifer.



**24.19** In [Prob. 24.18](#), a linearized groundwater model was used to simulate the height of the water table for an unconfined aquifer. A more realistic result can be obtained by using the following nonlinear ODE:

$$\frac{d}{dx} \left( K h \frac{dh}{dx} \right) + N = 0$$

where  $x$  = distance (m),  $K$  = hydraulic conductivity (m/d),  $h$  = height of the water table (m), and  $N$  = infiltration rate (m/d). Solve for the height of the water table for the same case as [Prob. 24.18](#). That is, solve from  $x = 0$  to 1000 m with  $h(0) = 10$  m,  $h(1000) = 5$  m,  $K = 1$  m/d, and  $N = 0.0001$  m/d. Obtain your solution with the shooting method.

**24.20** Just as Fourier's law and the energy balance can be employed to characterize temperature distribution, analogous relationships are available to model field problems in other areas of engineering. For example, electrical engineers use a similar approach when modeling electrostatic fields. Under several simplifying assumptions, an analog of Fourier's law can be represented in one dimension as

$$D = -\epsilon \frac{dV}{dx}$$

where  $D$  is called the electric flux density,  $\epsilon$  = permittivity of the material, and  $V$  = the electrostatic potential. Similarly, a Poisson equation (see [Prob. 24.8](#)) for electrostatic fields can be represented in one dimension as

$$\frac{d^2 V}{dx^2} = -\frac{\rho_v}{\epsilon}$$

where  $\rho_v$  = charge density. Use the finite-difference technique with  $\Delta x = 2$  to determine  $V$  for a wire where  $V(0) = 1000$ ,  $V(20) = 0$ ,  $\epsilon = 2$ ,  $L = 20$ , and  $\rho_v = 30$ .

**24.21** Suppose that the position of a falling object is governed by the following differential equation:

$$\frac{d^2x}{dt^2} + \frac{c}{m} \frac{dx}{dt} - g = 0$$

where  $c$  = a first-order drag coefficient = 12.5 kg/s and  $m$  = mass of the object = 70 kg. Use the shooting method to solve this equation for these boundary conditions:

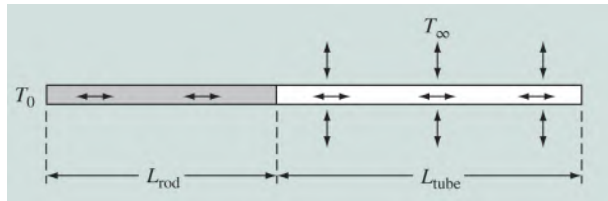
$$x(0) = 0. \quad x(12) = -500$$

**24.22** As shown in  **Fig. P24.22**, an insulated metal rod has a fixed temperature boundary condition ( $T_0$ ) at its left end. On its right end, it is joined to a thin-walled tube filled with water through which heat is conducted. The tube is insulated at its right end and convects heat with the surrounding fixed air temperature ( $T_\infty$ ). The convective heat flux at a location along the tube ( $\text{W}/\text{m}^2$ ) is represented by

$$q = h(T_\infty - T_2(x))$$

where  $h$  = the convective heat transfer coefficient [ $\text{W}/(\text{m}^2 \cdot \text{K})$ ]. Employ the finite-difference method with  $\Delta x = 0.1$  m to compute the temperature profile for the case where both the rod and tube are cylindrical with the same radius,  $r$  (m). Use the following parameter values for your analysis:  $L_{\text{rod}} = 0.6$  m,  $L_{\text{tube}} = 0.8$  m,  $T_0 = 400$  K,  $T_\infty = 300$  K,  $r = 3$  cm,  $k_1 = 80.2$  W/(m·K),  $k_2 = 0.615$  W/(m·K), and  $h = 1$  W/(m<sup>2</sup>·K). The subscripts 1 and 2 designate the rod and the tube, respectively.

**FIGURE P24.22**



**24.23** Perform the same simulation as in  **Prob. 24.22**, but for the case where the tube is also insulated, and the right-hand end of the tube is held at a fixed temperature of 300 K.

**24.24** Solve the following problem using the `solve_bvp` function from the Python SciPy `integrate` submodule. Plot the solution as  $y$  versus  $x$ .

$$\frac{d^2 y}{dx^2} + y = 0 \quad y(0) = 1 \quad \frac{dy}{dx}(1) = 0$$

**24.25**  Figure 24.25a shows a uniform beam subject to a linearly increasing distributed load. The equation for the resulting elastic curve is (see  Fig. 24.25b)

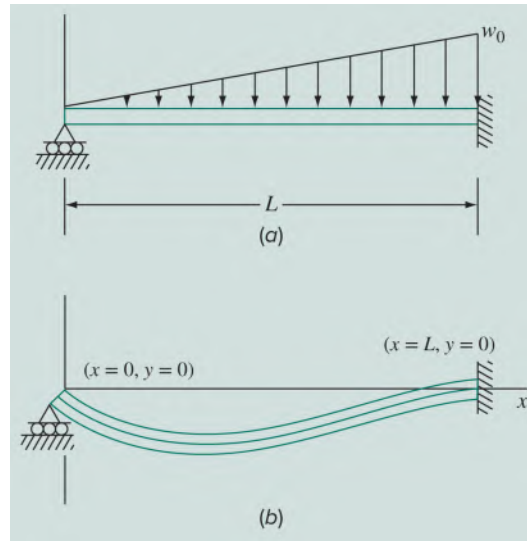
$$E I \frac{d^2 y}{dx^2} - \frac{w_0}{6} \left( 0.6 L x - \frac{x^3}{L} \right) = 0$$

Note that the analytical solution for the elastic curve is

$$y = \frac{w_0}{120 E I L} (-x^5 + 2L^2 x^3 - L^4 x)$$

Use `solve_bvp` to solve the differential equation, and then present the numerical and analytical solutions on the same plot. Use the following parameter values:  $L = 600$  cm,  $E = 50,000$  kN/cm<sup>2</sup>,  $I = 30,000$  cm<sup>4</sup>, and  $w_0 = 2.5$  kN/cm.

**FIGURE P24.25**



**24.26** Use `solve_bvp` to compute the solution of the differential equation

$$\frac{d^2 u}{dx^2} + 6 \frac{du}{dx} - u = 2$$

with the boundary conditions  $u(0) = 10$  and  $u(2) = 1$ . Plot the result as  $u$  versus  $x$ .

**24.27** Use `solve_bvp` to solve the following dimensionless ODE that describes the temperature distribution in a cylindrical rod with internal heat source,  $S$ .

$$\frac{d^2T}{dr^2} + \frac{1}{r} \frac{dT}{dr} + S = 0$$

over the range  $0 \leq r \leq 1$  with the boundary conditions

$$\frac{dT}{dr}(0) = 0 \quad T(1) = 1$$

for the cases,  $S = 1, 10$ , and  $50 \text{ K/m}^2$ . Present the temperature profiles for the three cases on the same plot.

**24.28** A simple unit used in industry to exchange energy between two fluids is the pipe-in-pipe, counter-current heat exchanger described in  **Fig. P24.28**. A mathematical model describing the temperature profiles of the cold and hot fluid is

$$\begin{aligned} \frac{dT_c}{dz} &= \frac{h_i A_i}{w_c C_c} (T_h - T_c) & T_c(0) &= T_{ci} \\ \frac{dT_h}{dz} &= \frac{h_o A_o}{w_h C_h} (T_h - T_c) & T_h(0) &= T_{hi} & h_o &= \frac{h_i D_i}{D_o} \end{aligned}$$

where  $z$  = distance down the heat exchanger from the cold fluid entrance,  $T_c$  and  $T_h$  = the temperatures of the cold and hot fluid, respectively,  $h_i$  = a heat transfer coefficient based on the inside area of the inner pipe per unit length,  $h_o$  = a heat transfer coefficient based on the outside area of the inner pipe, related to  $h_i$  by the inside and outside diameters of the inner pipe,  $D_i$  and  $D_o$ , respectively,  $C_c$  = heat capacity of the cold fluid,  $C_h$  = heat capacity of the hot fluid,  $w_c$  = mass flow rate of the cold fluid,  $w_h$  = mass flow rate of the hot fluid,  $T_{ci}$  = entry temperature of the cold fluid, and  $T_{hi}$  = entry temperature of the hot fluid. These parameter values describe a typical heat exchanger unit:  $L = 5 \text{ m}$ ,  $D_o = 0.0254 \text{ m}$ ,  $D_i = 0.0193 \text{ m}$ ,  $T_{ci} = 10^\circ \text{C}$ ,  $T_{hi} = 50^\circ \text{C}$ ,  $w_c = 0.2921 \text{ kg/s}$ ,  $w_h = 1.050 \text{ kg/s}$ ,  $C_{pc} = C_{ph} = 4.2 \text{ kJ}/(\text{kg} \cdot ^\circ \text{C})$ ,  $h_i = 14.2 \text{ kW}/(\text{m}^2 \cdot ^\circ \text{C})$ .

---

**FIGURE P24.28** A pipe-in-pipe, counter-current heat exchanger.



Find the temperature profiles of the fluids for these conditions and parameter values using the `solve_bvp` function. Plot the profiles on the same figure with a blue line for the cold fluid and a red line for the hot fluid.

# APPENDIX A

## MATPLOTLIB

The *Matplotlib* module provides a wide variety of features for graphics support of Python programs. We have focused on the use of a subset module, `pylab`, in this text because it is less complicated and easier for students to use, especially those who are new to Python. On occasion where necessary, we have included applications of Matplotlib without much explanation, providing examples that students might mimic in homework assignments and other academic projects. The purpose of this appendix is to provide a more thorough coverage of applications of Matplotlib and its module `pyplot` for the creation of plots that are typically needed by engineers and scientists. In that way, this is not intended to be a comprehensive presentation of Matplotlib but rather a practical extension of the material covered in the body of the book.

There are numerous, good resources on Matplotlib. To start with, the Matplotlib site accessed through the Spyder Help menu is helpful and contains tutorials on the many features and capabilities of Matplotlib. As it is far more comprehensive than what we present here, and we recommend it to expand your own knowledge, experience, and capabilities. There are also specific references that you may find useful, such as the books by Devert (2014), Wood (2015), and Hill (2015).

Here is an outline of the topics of this appendix:

-  **A.1.** The basics of creating a simple plot
-  **A.2.** Customizing and adding features to line plots
-  **A.3.** Statistical plots: error bars, histograms, box plots
-  **A.4.** Other plots of interest: stem plots, log-scale plots, polar plots
-  **A.5.** Including multiple plots in a single figure, axes objects
-  **A.6.** Plotting in 3D: contour and surface plots



# A.1 THE BASICS OF CREATING A SIMPLE PLOT

---

The first step is to import the Matplotlib `pyplot` module,

```
import matplotlib.pyplot as plt
```

It is not mandatory to create the shortcut abbreviation `plt`, but this is commonly done in Python applications, so we recommend it here too. This is similar to using `np` for the NumPy module.

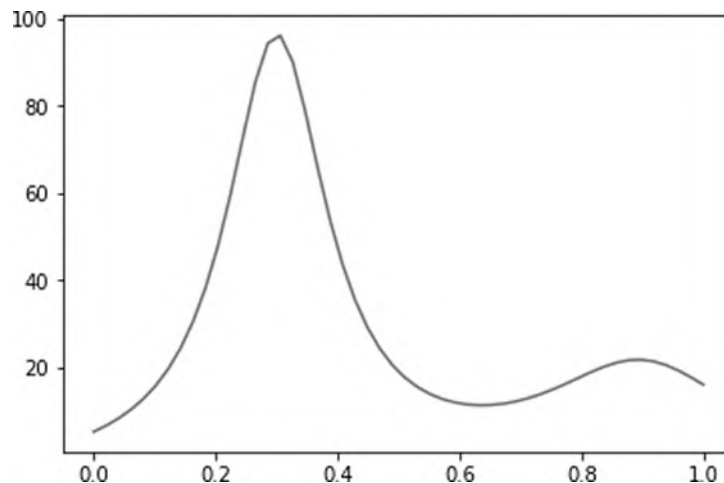
Next, we need a source of data to plot. Generally, data that come from calculations of continuous functions are plotted with interconnecting lines, not markers. Data that represent experimental measurements are plotted as markers, with perhaps interconnecting lines to enhance pattern recognition. When we are plotting hundreds (or even thousands) of sequential data points, such as time series, we generally leave out the markers and just go with lines to avoid the clutter.

As a first example, here a plot is created to represent a continuous function, a version of the *humps function*,

Page 615

$$f(x) = \frac{1}{(x - 0.3)^2 + 0.01} + \frac{1}{(x - 0.9)^2 + 0.04} - 6$$

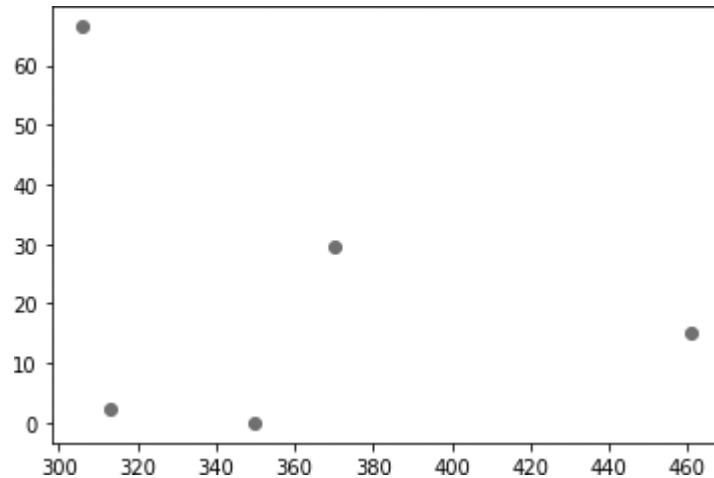
```
import numpy as np
x = np.linspace(0,1)
f = lambda x: 1/((x-0.3)**2+0.01)+1/((x-0.9)**2+0.04)-6
plt.plot(x,f(x))
plt.show()
```



The import of the NumPy module is required to use the `linspace` function to produce the independent variable (in this case, `x`) as an array. The line on the plot is actually blue as it is created. The `plt.show()` command is not required if operating in the Spyder IDE, but may be necessary if the script is executed outside that environment.

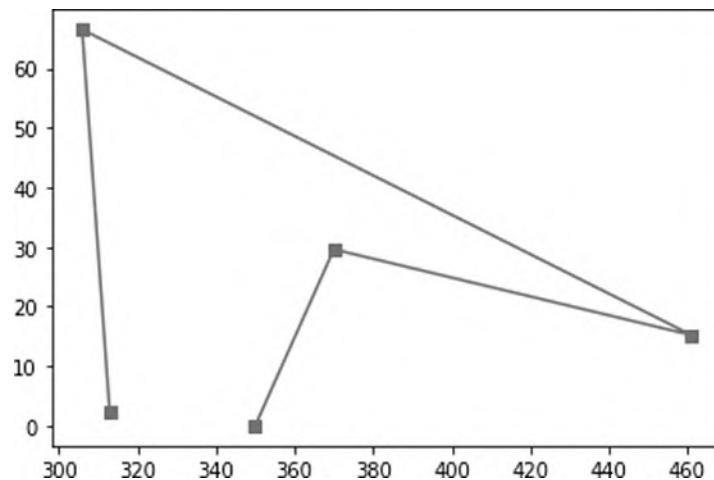
Of course, we see many changes we would like to make to enhance the plot. We will get to those in the next section. But next, let's see how to create a simple plot of a small set of data.

```
x = np.array([350,370,461,306,313])
y = np.array([0.,29.6,15.2,66.5,2.3])
plt.scatter(x,y)
plt.show()
```



If we would like to connect those data markers with lines, we run into a problem.

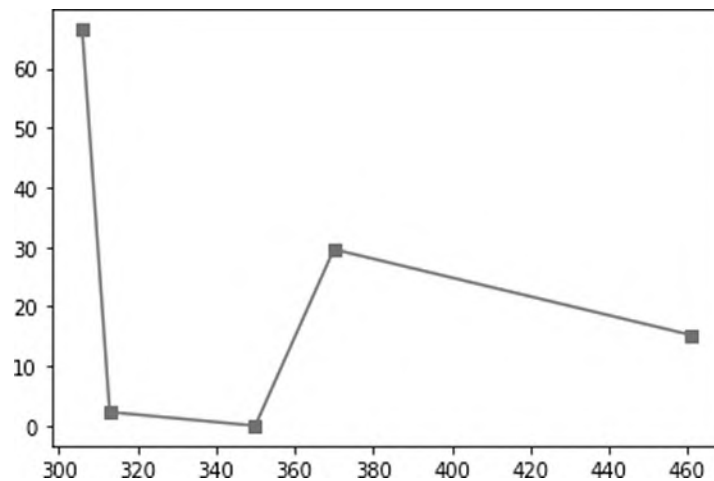
```
x = np.array([350,370,461,306,313])
y = np.array([0.,29.6,15.2,66.5,2.3])
plt.plot(x,y,marker='s')
plt.show()
```



The issue is that we would like the data pairs to be sorted in ascending order of the  $x$  variable. Here, we can do that manually by reordering the vectors according to  $x$ .

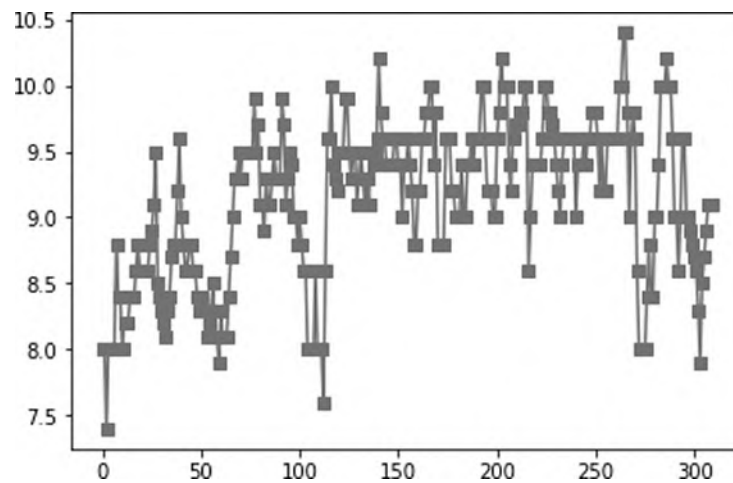
```
x = np.array([306.,313.,350.,370.,461.])
y = np.array([66.5,2.3,0.,29.6,15.2])
plt.plot(x,y,marker='s')
plt.show()
```

For larger data sets, we could include Python code to accomplish the sorting.



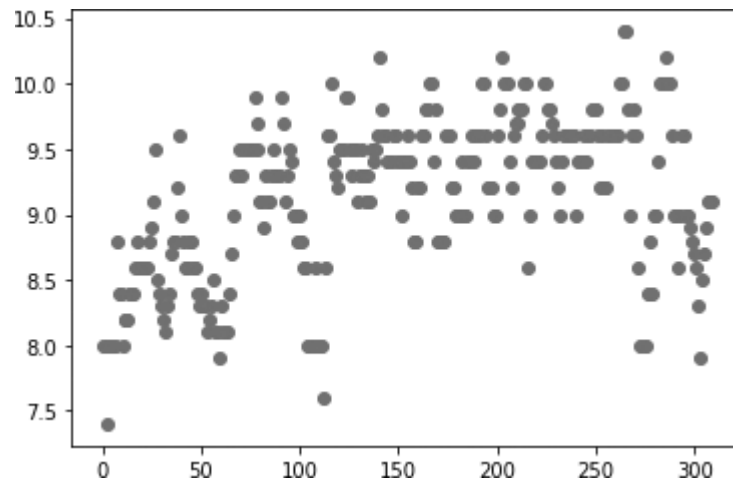
If we have many data, here read in from a text file, markers may provide too much clutter.

```
y = np.loadtxt('visc.txt')
plt.plot(y)
plt.show()
```



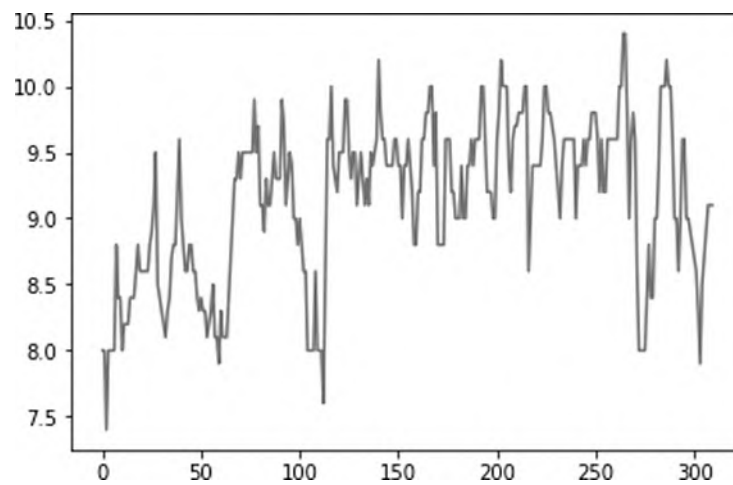
Here, the `plt.plot` command is able to plot the `y` series against the ordinal number of the `y` data. If we wanted to use the `plt.scatter` command without the interconnecting lines, we must provide the `x`-axis sample numbers.

```
y = np.loadtxt('visc.txt')
n = len(y)
x = []
for i in range(n):
    x.append(i)
plt.scatter(x,y)
plt.show()
```



In either case, although we have experimental series data, the markers cause a lot of clutter, so we might prefer a plot with interconnecting lines only.

```
y = np.loadtxt('visc.txt')  
plt.plot(y)  
plt.show()
```



## A.2 CUSTOMIZING AND ADDING FEATURES TO LINE PLOTS

---

Here is a list of common needs for customizing line plots:

- line colors, styles, and widths
- gridlines
- axis labels and plot titles
- axis limits
- multiple lines with legends
- multiple plots in sequence

Line colors and styles are specified in the same way as we introduced in  **Chap. 2** according to these tables,

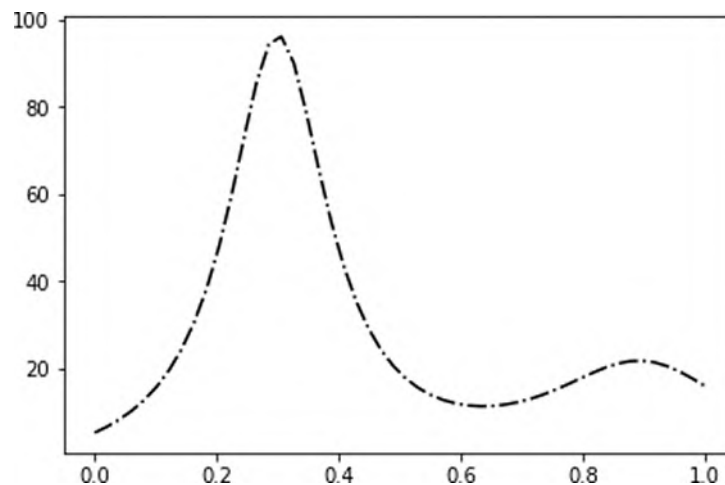
| Character | Color   |
|-----------|---------|
| k         | black   |
| b         | blue    |
| g         | green   |
| r         | red     |
| c         | cyan    |
| m         | magenta |
| y         | yellow  |
| w         | white   |

| Character | Style    |
|-----------|----------|
| —         | solid    |
| --        | dashed   |
| :         | dotted   |
| -.        | dash-dot |

We can modify our plot of the humps function so that the line is black (perhaps hard to see that it was blue, so trust us) with a dash-dot linestyle. Page 619

```
x = np.linspace(0,1)
f = lambda x: 1/((x-0.3)**2+0.01)+1/((x-0.9)**2+0.04)-6
plt.plot(x,f(x),c='k',ls='-.')
```

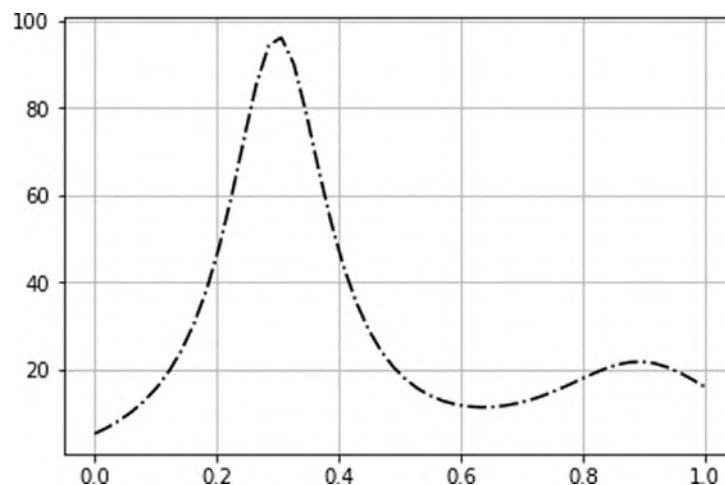
```
plt.show()
```



If we were plotting a single curve, we wouldn't often use a different linestyle like this, but it is here for illustration. If the ultimate destination of the plot is in a black-and-white document, you should be careful about using colored lines. Here, black is better than the default blue for such purposes because it is more distinct. You will note we use accepted abbreviations for *linestyle* (*ls*) and *color* (*c*). We could have used the full words instead.

Generally, engineers and scientists prefer having *gridlines*. (Some would say they like to “drive between the lines.”) In any case, they are commonly included, and we can add them as shown here.

```
x = np.linspace(0,1)
f = lambda x: 1/((x-0.3)**2+0.01)+1/((x-0.9)**2+0.04)-6
plt.plot(x,f(x),c='k',ls='-.')
plt.grid()
plt.show()
```



When you see a command like `plt.grid()`, the parentheses indicate there are optional arguments you could add. You can find out about these via Spyder Help and the Matplotlib selection. In this case,

```
matplotlib.pyplot.grid(b=None, which='major', axis='both', **kwargs)
```

Configure the grid lines.

**Parameters:**

**b**: bool or None, optional

Whether to show the grid lines. If any *kwargs* are supplied, it is assumed you want the grid on and *b* will be set to True.

If *b* is *None* and there are no *kwargs*, this toggles the visibility of the lines.

**which**: ('major', 'minor', 'both'), optional

The grid lines to apply the changes on.

**axis**: ('both', 'x', 'y'), optional

The axis to apply the changes on.

**\*\*kwargs**: Line2D properties

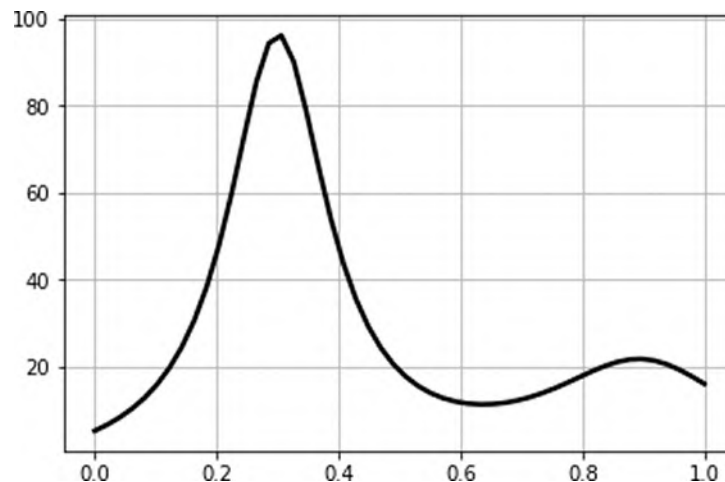
Define the line properties of the grid, e.g.:

```
grid(color='r', linestyle='-', linewidth=2)
```

We see here that we can specify major, minor, or both gridlines, and we can choose which gridline type or axis to apply any changes. Lastly, you can see that we have control over the gridlines' properties.

In certain plots, we may like to change the widths (or “weights”) of the lines displayed. Here is a choice of a width of 2.5 (default is 1.0) for our humps plot with a solid black line.

```
x = np.linspace(0,1)
f = lambda x: 1/((x-0.3)**2+0.01)+1/((x-0.9)**2+0.04)-6
plt.plot(x,f(x),c='k',lw = 2.5)
plt.grid()
plt.show()
```

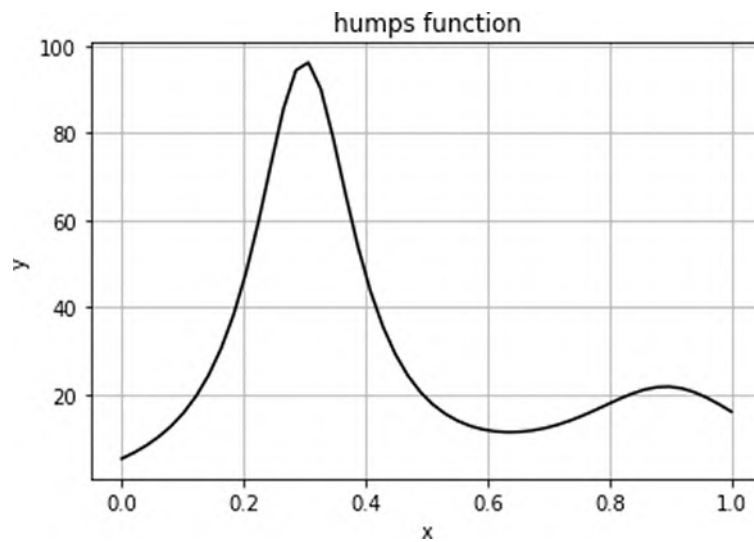


Rarely do we produce a plot for publication without axes labels and a plot title. We can add those to this last plot with additional commands (highlighted). These are like our `pylab` commands from

## Chap. 2.

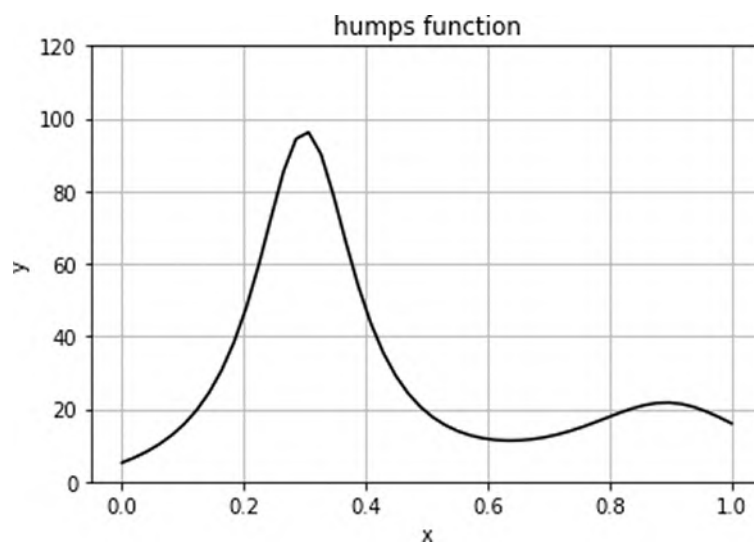
```
x = np.linspace(0,1)
f = lambda x: 1/((x-0.3)**2+0.01)+1/((x-0.9)**2+0.04)-6
plt.plot(x,f(x),c='k')
plt.grid()
plt.xlabel('x')
```

```
plt.ylabel('y')
plt.title('humps function')
plt.show()
```



We encounter plots where the axis limits provided by `plt.plot` by default do not meet our needs. For example, in the above plot, we might want to have *y*-axis limits of 0 and 120. We can accomplish that with

```
x = np.linspace(0,1)
f = lambda x: 1/((x-0.3)**2+0.01)+1/((x-0.9)**2+0.04)-6
plt.plot(x,f(x),c='k')
plt.grid()
plt.ylim(0.,120.)
plt.xlabel('x')
plt.ylabel('y')
plt.title('humps function')
plt.show()
```





Of course, you can adjust the  $x$ -axis limits with `plt.xlim` in a similar fashion. Changing tick intervals and labels comes later.

When we plot multiple quantities, either as lines or as markers (or both), we need to distinguish the lines on the plot. This can be done with different colors, if the end destination of the plots accommodates color, and it can be done with linestyles. There are only four linestyles, so that presents a limit, but we also must consider that a plot with more than four lines may be too dense with information. Beyond four, it is really necessary to employ color (or weights for publication in black and white). Aside from distinguishing the lines on the plot, we need to present a legend to identify the different lines.

An example would be to plot three different functions on the same plot. We will choose here

$$f_1(x) = \sin(x) \text{ , } f_2(x) = \sin(2x) \text{ , and } f_3(x) = \sin(3x)$$

You can see how we identify the plot of each curve with a label and a linestyle. Then we use the `plt.legend()` command to display the legend.

```
x = np.linspace(0,np.pi,100)
f1 = lambda x: np.sin(x)
f2 = lambda x: np.sin(2*x)
f3 = lambda x: np.sin(3*x)
plt.plot(x,f1(x),c='k',label='sin(x)')
plt.plot(x,f2(x),c='k',ls='--',label='sin(2x)')
plt.plot(x,f3(x),c='k',ls=':',label='sin(3x)')
plt.grid()
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('sine functions')
plt.legend()
plt.show()
```

The `plt.legend()` command attempts to find the best location for the legend so that it doesn't interfere with the curves, and it is successful here. We can force a location for the legend by including the `loc = 'legend location'` within the parentheses of the legend command. The location is specified according to this table:

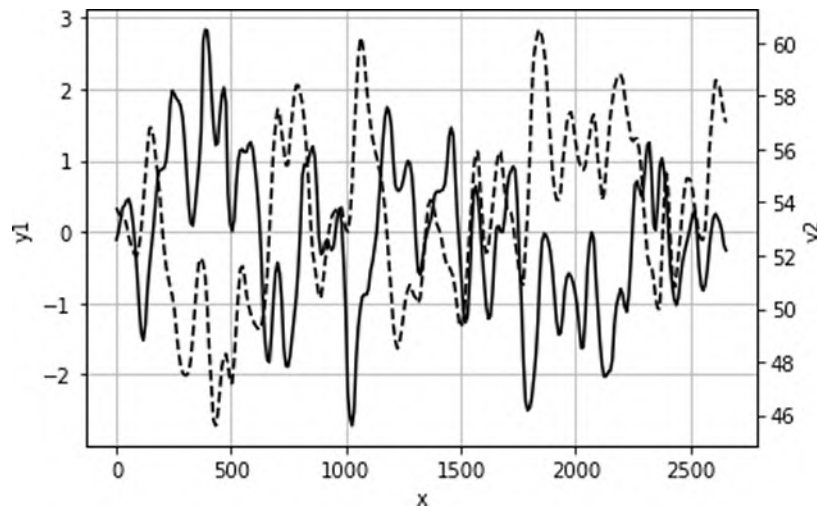
| Location String | Location Code |
|-----------------|---------------|
| 'best'          | 0             |
| 'upper right'   | 1             |
| 'upper left'    | 2             |
| 'lower left'    | 3             |
| 'lower right'   | 4             |
| 'right'         | 5             |
| 'center left'   | 6             |
| 'center right'  | 7             |
| 'lower center'  | 8             |

| Location String | Location Code |
|-----------------|---------------|
| 'upper center'  | 9             |
| 'center'        | 10            |

The default location string 'best' places the legend at the location which minimizes the overlap with other plot elements. Page 623

On occasion, we need to plot more than one data set, and there are wide differences in the scales of the numbers. A good way to handle this when there are two scales that apply is to add a scale on the right side of the plot. In the code segment below, we read in a set of times and two measurements.

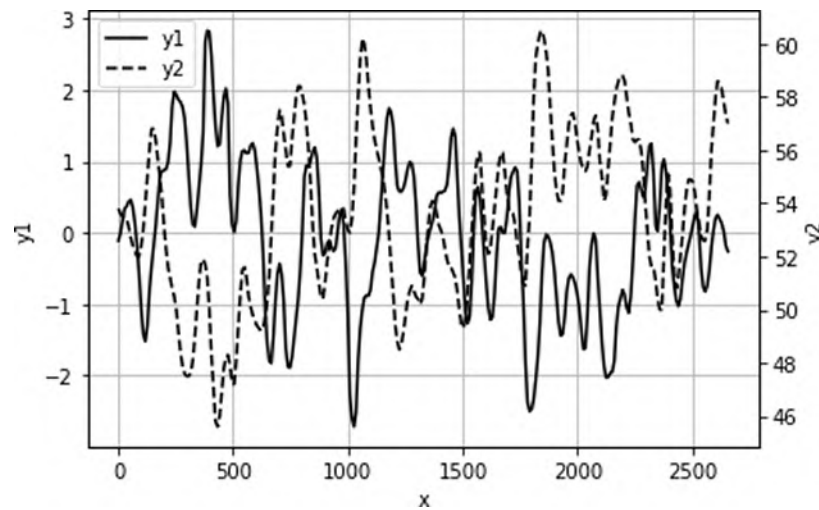
```
t,y1,y2 = np.loadtxt(fname='FurnaceData.txt',unpack=True)
plt.figure()
plt.plot(t,y1,c='k',label='y1')
plt.xlabel('x')
plt.ylabel('y1')
plt.grid()
plt.twinx()
plt.plot(t,y2,c='k',ls='--',label='y2')
plt.ylabel('y2')
```



We have created a plot of two curves on two vertical axes using the `twinx()` command. For this case, we need a legend, but that is not so simple here.

```
t,y1,y2 = np.loadtxt(fname='FurnaceData.txt',unpack=True)
plt.figure()
curve1 = plt.plot(t,y1,c='k',label='y1')
plt.xlabel('x')
plt.ylabel('y1')
plt.grid()
plt.twinx()
curve2 = plt.plot(t,y2,c='k',ls='--',label='y2')
plt.ylabel('y2')
curves = curve1 + curve2
labels = []
for curve in curves:
```

```
labels.append(curve.get_label())
plt.legend(curves,labels)
```



What we have done here is to capture the plotted curves in the variables `curve1` and `curve2`. We have then concatenated these into the `CURVES` entity, combined the labels of the two plotted curves, and issued the legend command for the combined curves and labels. At best, this is awkward. But if you follow this pattern, you will be able to create your plot with two vertical axes and an appropriate legend. This code follows that of Hill (2015).

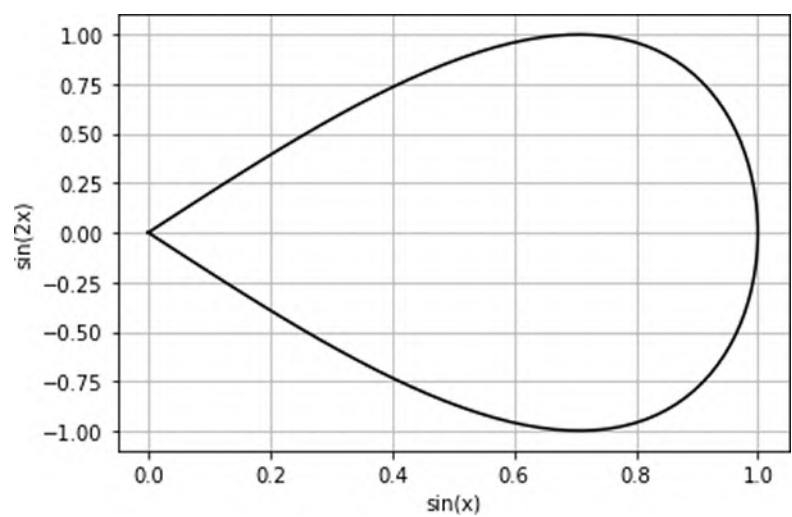
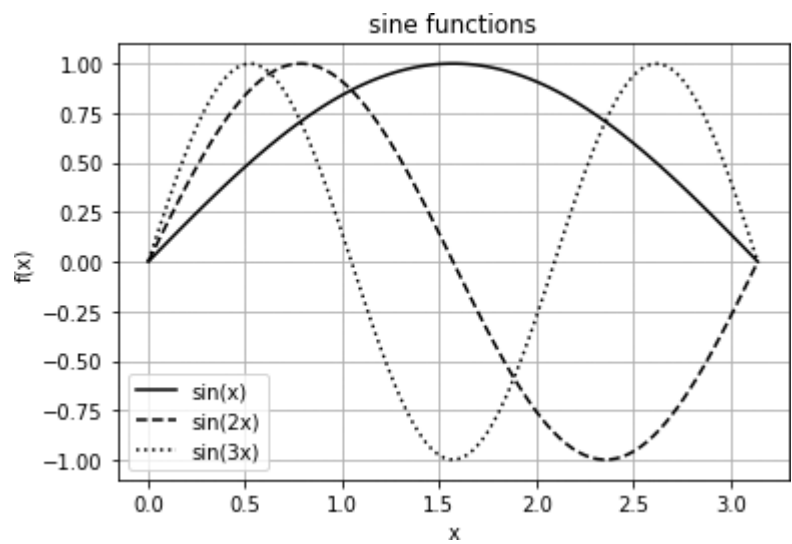
The final topic in this section is to address what we do when our code needs to produce more than one plot. This is illustrated below.

```
x = np.linspace(0,np.pi,100)
f1 = lambda x: np.sin(x)
f2 = lambda x: np.sin(2*x)
f3 = lambda x: np.sin(3*x)
plt.figure()
plt.plot(x,f1(x),c='k',label='sin(x)')
plt.plot(x,f2(x),c='k',ls='--',label='sin(2x)')
plt.plot(x,f3(x),c='k',ls=':',label='sin(3x)')
plt.grid()
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('sine functions')
plt.legend()
plt.show()

plt.figure()
plt.plot(f1(x),f2(x),c='k')
plt.grid()
plt.xlabel('sin(x)')
plt.ylabel('sin(2x)')
plt.show
```

The first plot produced is the same as [Fig. A.1](#), but now we produce a second plot. You will note that we set off the separate plots with the `plt.figure()` command. The first one isn't really needed, but we include it for consistency.

**FIGURE A.1** Plot of three sinusoids



## A.3 STATISTICAL PLOTS

In different areas of endeavor, engineers and scientists use certain plots that differ from the simple line plot. When it comes to applied statistics and signal processing, we often encounter the following plots:

- line/marker plots with error bars
- histograms
- box plots

There are others, but this is a representative set.

Page 625

### A.3.1 Line/Marker Plots with Error Bars

Replication is an important aspect of collecting experimental data. It allows for quantification of the uncertainty of measurements. If we are studying the behavior of a response variable to changes in an input variable or factor, sound experimental design requires that we randomize the order of experimental tests and provide replicates at each level of the input factor. Then, as described in [Chap. 14](#), we summarize the data at each level by statistics that measure central tendency, typically the average, and dispersion, possibly the range, sample standard deviation, or median absolute deviation (MAD). Consequently, we would like our plot to describe both the central value and dispersion quantity for each input factor level. In making such plots, it is important that the dispersion quantity, displayed as error bars above and below the central value, be made clear. The table below shows a data set with three replicates, the sample average, and a range-based estimate of the standard deviation,

$$(\max - \min) / \sqrt{3}.$$

| x   | Measurements |       |       | Average | Std Dev Est |
|-----|--------------|-------|-------|---------|-------------|
| 1   | 2.11         | 1.16  | 1.53  | 1.60    | 0.54        |
| 5   | 5.90         | 5.92  | 6.32  | 6.05    | 0.24        |
| 10  | 7.64         | 9.84  | 9.49  | 8.99    | 1.27        |
| 15  | 11.13        | 11.13 | 11.49 | 11.25   | 0.21        |
| 20  | 12.52        | 12.71 | 12.34 | 12.52   | 0.21        |
| 30  | 12.53        | 13.65 | 14.87 | 13.68   | 1.35        |
| 40  | 12.02        | 13.38 | 13.28 | 12.89   | 0.79        |
| 50  | 12.80        | 12.61 | 13.77 | 13.06   | 0.67        |
| 75  | 12.06        | 12.21 | 12.80 | 12.35   | 0.43        |
| 100 | 11.60        | 11.64 | 13.08 | 12.11   | 0.86        |

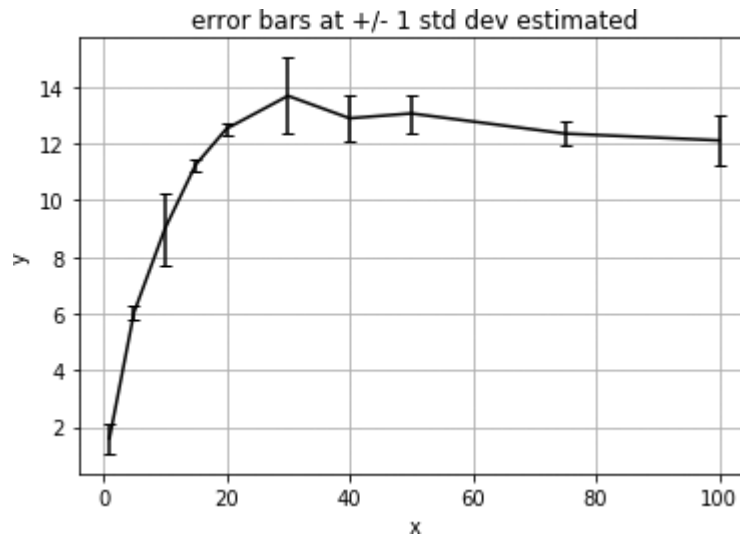
We can use the `errorbar` plot function in `matplotlib` to create the desired plot.

```
x = np.array([1,5,10,15,20,30,40,50,75,100])
```

```

yavg = np.array([1.6,6.05,8.99,11.25,12.52,13.68,12.89,13.06,12.35,12.11])
stdest = np.array([0.54,0.24,1.27,0.21,0.21,1.35,0.79,0.67,0.43,0.86])
plt.errorbar(x,yavg,stdest,c='k',capsize=3)
plt.grid()
plt.xlabel('x')
plt.ylabel('y')
plt.title('error bars at +/- 1 std dev estimated')

```



There are numerous optional arguments for the `errorbar` plot function. We have chosen the `capsize` argument to place horizontal bars at the ends of the error bar vertical lines. One can also have error bars in the  $x$  direction if the  $x$  values vary in the replicate measurements. As usual, more information is readily available in Matplotlib Help on the Spyder Help menu.

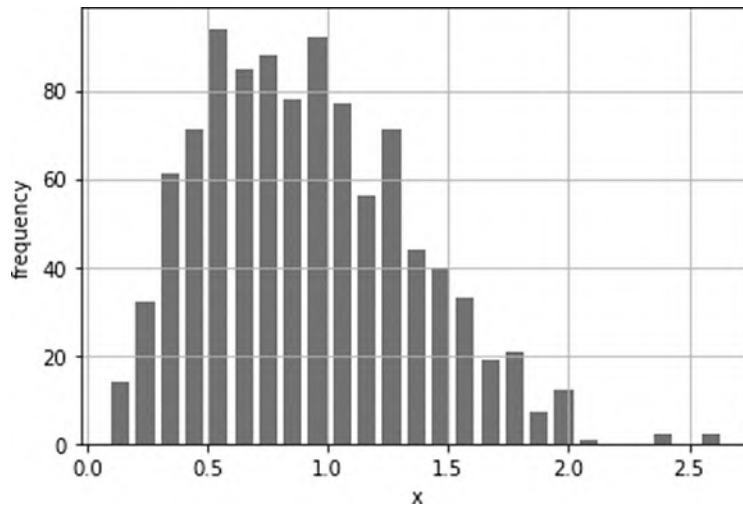
## A.3.2 Histograms

Histograms and their related plots are important in visualizing the distribution characteristics of data with random behavior. Their application is limited when the sample size is small, e.g., less than 25 or so. Matplotlib provides a histogram plot function with flexible capabilities. The `hist` function in `pyplot` provides a convenient way to plot a histogram with a limited amount of formatting capability. Below, we generate 1,000 random numbers drawn from the Weibull distribution and create a histogram with 25 bins. We choose to narrow the bins to 75% width to see gaps between them.

```

x = np.random.weibull(2.3,1000)
plt.hist(x,25,rwidth=0.75)
plt.grid()
plt.xlabel('x')
plt.ylabel('frequency')

```

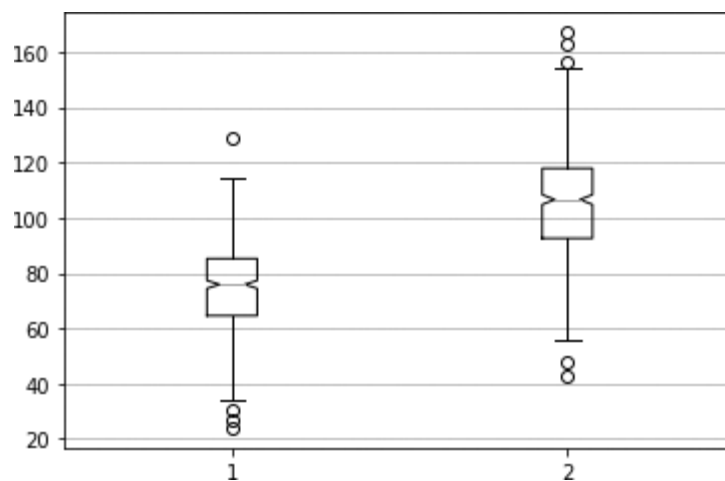


The `bar` function provides much more formatting flexibility and can be combined with the `histogram` function from NumPy to create plots. This was illustrated in [Sec. 14.1.3](#) and need not be repeated here. Bar charts are used widely to present information in various styles. That should be recognized here even though our emphasis is on the analysis of experimental data.

## A.3.3 Box Plots

The boxplot is used to identify outliers in data and compare the range of variation of multiple data sets to judge whether they overlap or are different. The `pyplot` module provides a `boxplot` function which we illustrate below with two data sets acquired from a text file.

```
x1,x2 = np.loadtxt(fname='BoxPlotData.csv',delimiter=',',unpack=True)
plt.boxplot((x1,x2),notch=True)
plt.grid(axis='y')
```



The vertical axis is the  $x$  variable. The boxes represent the 25th to 75th percentile of the data. The height of the box is called the interquartile range (iqr). The “whiskers,” as they are called, extend from the upper and lower edges of the boxes a distance of  $1.5 \times \text{iqr}$ . Any data outside the whiskers are considered to be outliers (circles). In the plot above, we see that the boxes of the two data sets do not overlap, even though the whiskers do. We also note several outliers. The optional notch in the boxes represents an estimate of the confidence interval on the median, which is the central horizontal line. If the median line is displaced significantly from the center of the box, that indicates an asymmetric or skewed distribution.

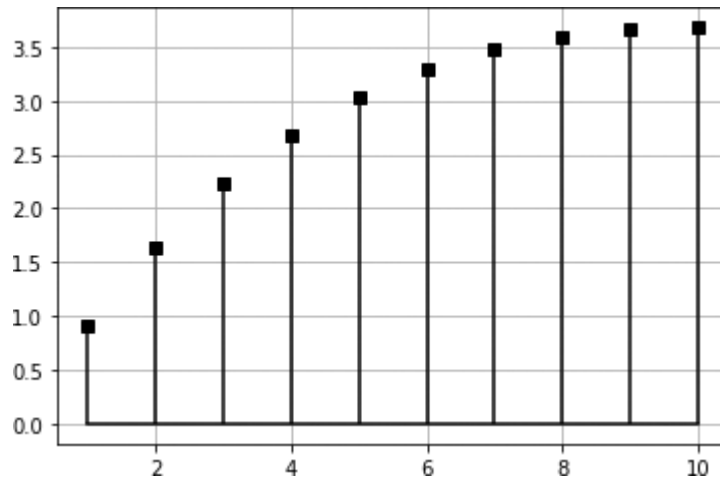
Note that box plots and their statistics are referred to as being *non-parametric* in that they express sample variability without assuming an underlying statistical distribution. They are also *robust* in that they are less sensitive to outliers than classical statistics based on the normal distribution.



## A.4 OTHER PLOTS OF INTEREST

In [Sec. 16.5](#), we created a stem plot as a feature for the Fourier transform. There, we used subplots and axes objects, which we will review in [Sec. A.5](#). *Stem plots* provide vertical lines for  $y$  data at each  $x$  value. The `stem` function can be used directly from the `pyplot` module.

```
x = np.arange(1,11)
y = x*np.exp(-x/10)
plt.stem(x,y,linefmt='k-',markerfmt='ks',basefmt='k-' \
        ,use_line_collection=True)
plt.grid()
```



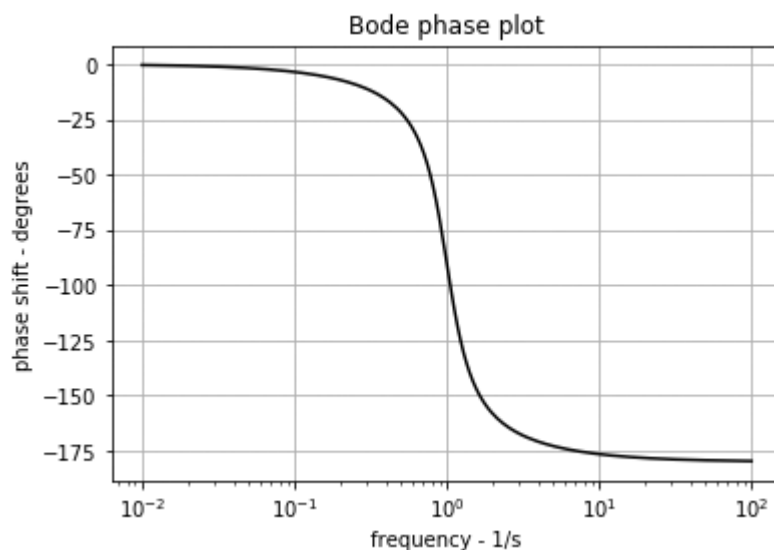
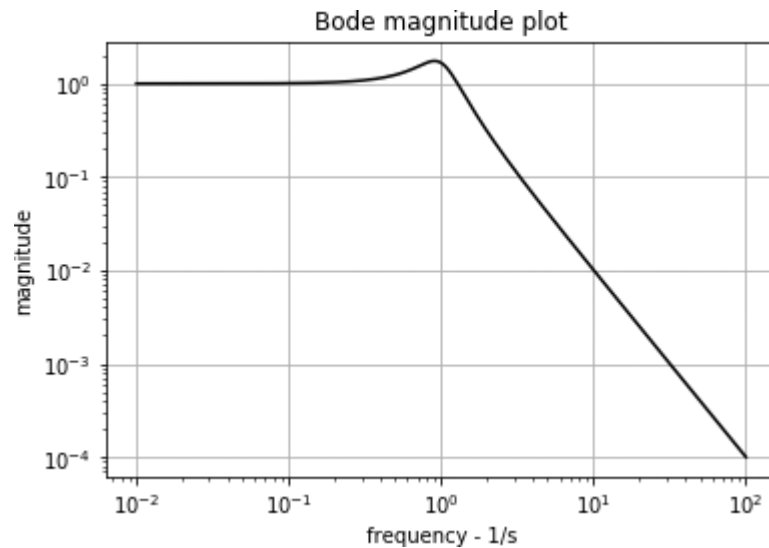
Above, we specified the format of the vertical lines as solid and black, the stem heads as black squares, and the baseline also as solid black. To do this in accordance with the latest version of Matplotlib, we set the `use_line_collection` parameter to `True`.

Scientists and engineers frequently create plots where one or both variables are better viewed in terms of their logarithms. One way to approach this is to compute the log values and plot them on normal scales. It is more typical though to plot the original data using logarithmic scales. Matplotlib provides convenient means to scale the  $x$ - or  $y$ -axis or both. In the following example, we create a pair of *Bode plots* representing the frequency response of a system. The first plot has both scales logarithmic, and the second plot has only the  $x$ -axis as a log scale.

```
w = np.logspace(-2,2,200) # frequency from 0.01 to 100
wc = 0. + w*1j # jw
wn = 1 ; zeta = 0.3
G = 1/(wc**2+2*zeta/wn*wc+1/wn**2) # transfer function G(jw)
M = abs(G) # magnitude
phi = np.arctan2(np.imag(G),np.real(G))*180/np.pi # phase shift

plt.figure()
plt.loglog(w,M,c='k')
plt.grid()
plt.xlabel('frequency - 1/s')
plt.ylabel('magnitude')
plt.title('Bode magnitude plot')
```

```
plt.figure()
plt.semilogx(w,phi,c='k')
plt.grid()
plt.xlabel('frequency - 1/s')
plt.ylabel('phase shift - degrees')
plt.title('Bode phase plot')
```

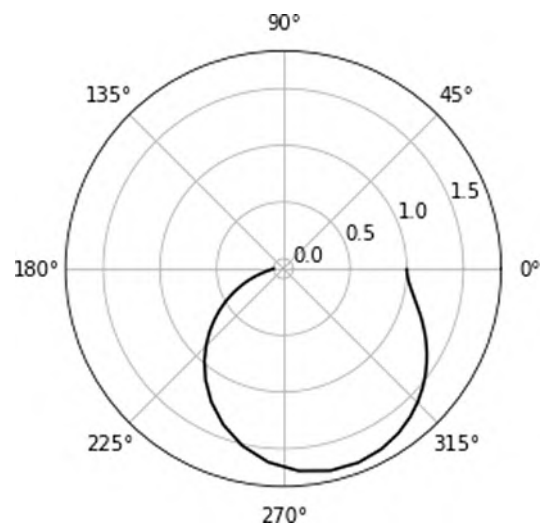


Here, we have generated a log-spaced sequence of frequency values so that we have evenly-spaced points for the plots. Later, we will review how to stack these plots in the same figure frame.

From our last example, what if we wanted to plot the magnitude ( $M$ ) versus the phase angle ( $\phi$ ). A plot with *polar coordinates* would be a natural for that. Let's try that here by adding these lines to the script above.

```
phir = np.arctan2(np.imag(G),np.real(G)) # angles in radians
plt.figure()
```

```
plt.polar(phir,M, 'k-')  
plt.axes(polar=True) # add grid of polar axes  
yt = np.array([0.,0.5,1.0,1.5]) # specify ticks  
plt.yticks(yt)
```



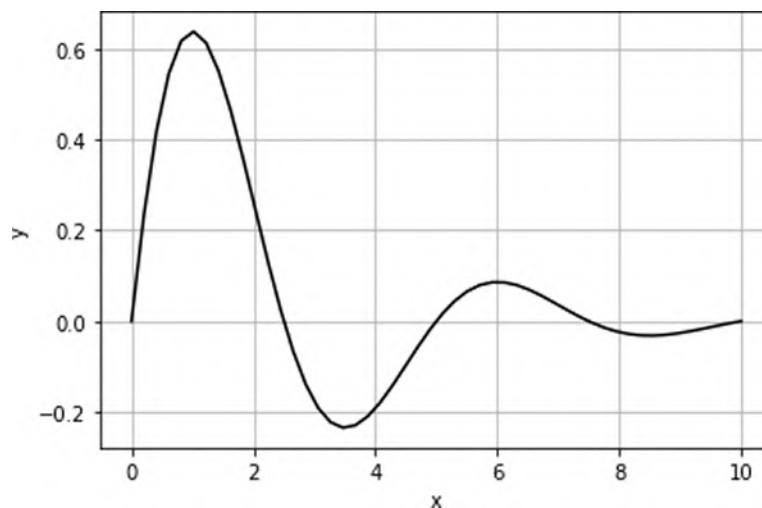
## A.5 INCLUDING MULTIPLE PLOTS IN A SINGLE FIGURE, AXES OBJECTS

To gain more control, options, and flexibility over our plots with Matplotlib, we move to the use of axes objects within a figure frame. The two basic statements that create an axis object are

```
fig = plt.figure() # name a figure object
ax = fig.add_subplot(111) # create axes object for a single plot
```

After creating the axes object, all plotting commands are methods that are attached to that object, such as

```
x = np.linspace(0,10)
y = np.exp(-x/2.5)*np.sin(2*np.pi/5*x)
ax.plot(x,y,c='k')
ax.grid()
ax.set_xlabel('x')
ax.set_ylabel('y')
```

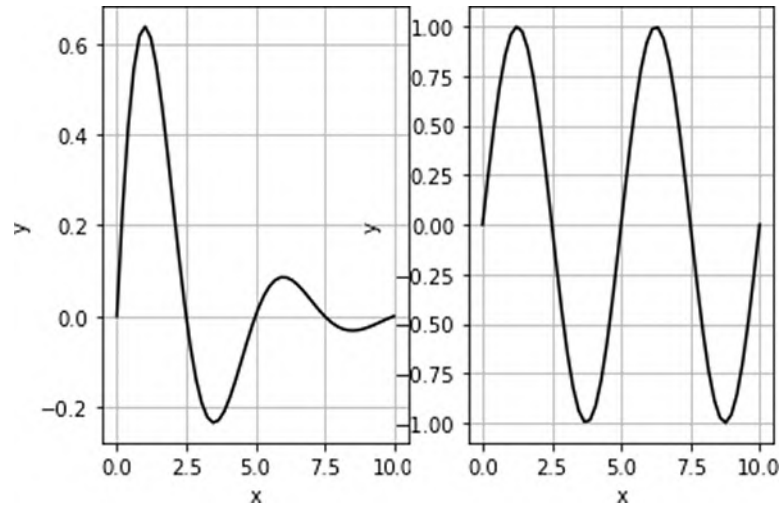


Note that there are certain differences. Before, we used `plt.xlabel('x')`, but now we use the `set_xlabel` method. This approach opens the possibility for placing multiple plots on a single figure. To do that, we need to specify the row-column arrangement of the plots in the `add_subplot` statement. For example,

```
fig = plt.figure() # name a figure object
ax1 = fig.add_subplot(121) # create axes object for one row, two columns
# the last 1 signifies the first plot in the arrangement
x = np.linspace(0,10)
y = np.exp(-x/2.5)*np.sin(2*np.pi/5*x)
ax1.plot(x,y,c='k')
ax1.grid()
ax1.set_xlabel('x')
ax1.set_ylabel('y')

ax2 = fig.add_subplot(122) # for the 2nd plot in the arrangement
y2 = np.sin(2*np.pi/5*x)
```

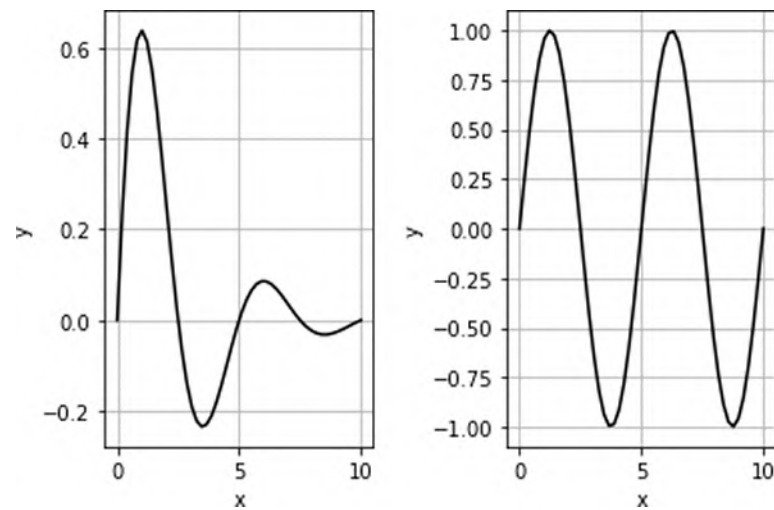
```
ax2.plot(x,y2,c='k')
ax2.grid()
ax2.set_xlabel('x')
ax2.set_ylabel('y')
plt.show()
```



Here, we see spacing problems. We can handle this in a couple of ways. One is to add space between the plots. By including

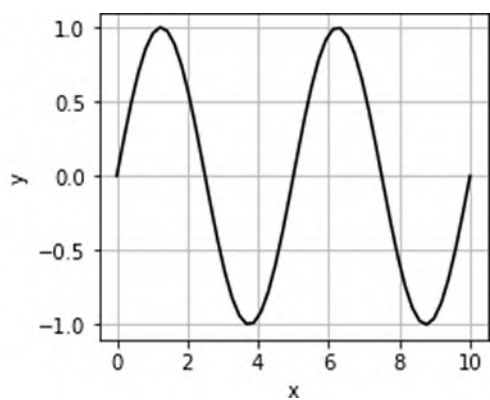
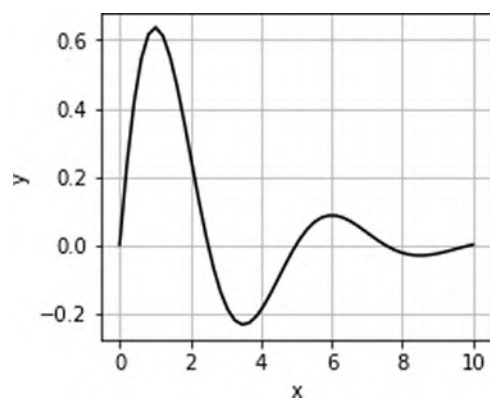
```
fig.subplots_adjust(wspace=0.5))
```

we get



Note: You adjust space between rows of plots with the `hspace` specification. If you don't like the aspect ratio of the plots and want them stretched out horizontally, you can change the size of the figure.

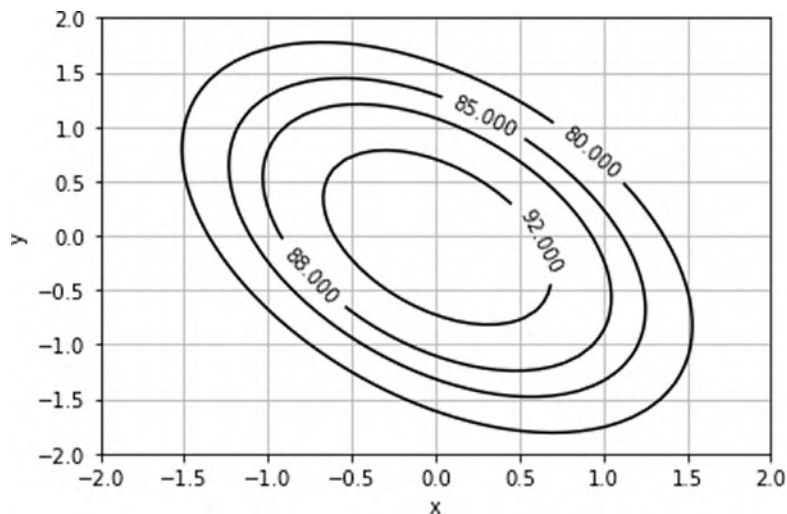
```
fig = plt.figure(figsize=(8,3))
```



## A.6 PLOTTING IN 3D: CONTOUR AND SURFACE PLOTS

As scientists and engineers study phenomena, the need arises to depict graphically one or more response variables in terms of two input variables or factors. There are many ways to do this, but the most common are *contour* and *surface plots*. With each of these, there are shaded-color versions and line versions. For contour plots, the lines are like the *isobars* on a topographic map. For surface plots, lines are interconnected see-through meshes. We will illustrate each of these here.

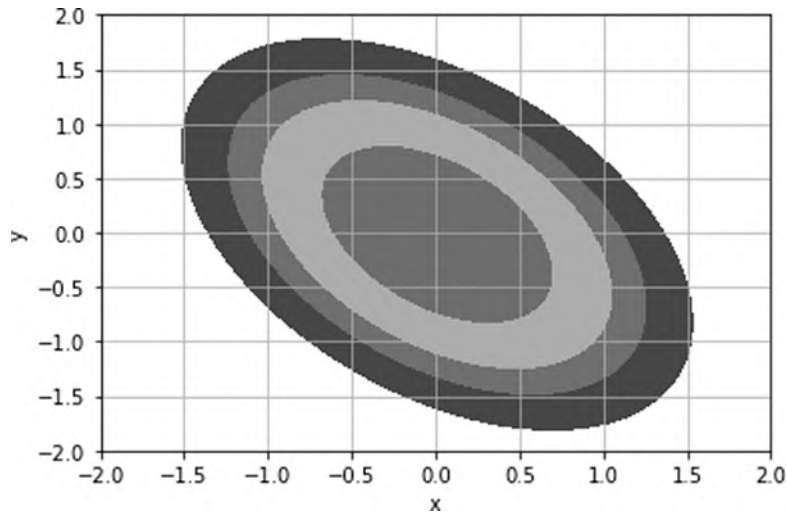
```
def f(x,y):  
    return 95.+0.05*x-0.145*y-8.13*x**2-5.87*y**2-6.25*x*y  
  
x = np.linspace(-2.,2.)  
y = np.linspace(-2.,2.)  
X,Y = np.meshgrid(x,y)  
Z = f(X,Y)  
  
plt.figure()  
contplt = plt.contour(X,Y,Z,[80.,85.,88.,92.,95.],colors='k')  
plt.grid()  
plt.clabel(contplt)  
plt.xlabel('x')  
plt.ylabel('y')
```



The data for the plot are generated by creating a *meshgrid*, which is a series of coordinates of a base mesh of  $x$  and  $y$  values, with all combinations of those specified by the `linspace` commands. The response  $Z$  values are computed using the `f` function. We have included commands for typical features of a contour plot. These include specifying the contour levels `[80., ..., 95.]`, and the colors of the contours, chosen black here. If left out, they will be determined by the `contour` function, and different colors will be used. You will also see the `clabel` command which places labels on the contours. This is a typical requirement for clarity.

An alternative is the filled contour plot using the `contourf` command.

```
plt.figure()
plt.contourf(X,Y,Z,[80.,85.,88.,92.,95.],colors=['b','g','c','m'])
plt.grid()
plt.xlabel('x')
plt.ylabel('y')
```



Here, colors are used, although those may not be visible to you in the plot above (trust us!). The purpose of a filled contour plot is generally qualitative, so we leave out the contour labels. (They don't show very well on the plot anyway.) It is possible to control the colors used in the filled contours, as we have here with blue, green, cyan, and magenta from the outside in. Many other colors are possible via the use of hexadecimal codes, e.g., '#A0A0A0', a gray color, but we don't go that far here.

To take advantage of Matplotlib's 3D surface/mesh plotting capabilities, we must include the following import statement:

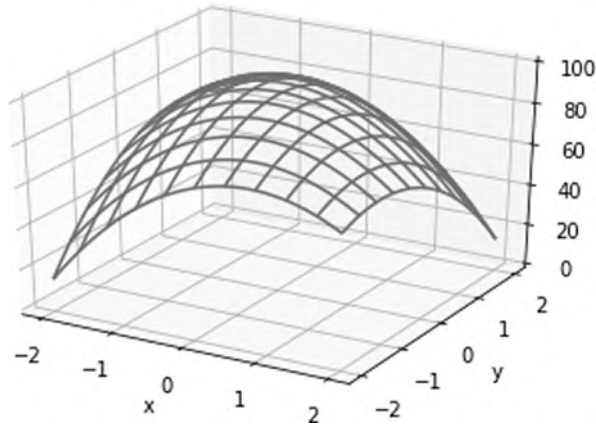
```
from mpl_toolkits.mplot3d import Axes3D
```

Note that The Spyder Editor may provide an "imported but unused" warning. Ignore that. In this case, the Editor lies. Sorry.

Then, we substitute the following in place of the contour plot code:

```
fig = plt.figure()
ax = fig.add_subplot(111,projection='3d')
ax.plot_wireframe(X,Y,Z,color='g',rstride=5,cstride=5)
ax.grid()
ax.set_xticks([-2.,-1.,0.,1.,2.])
ax.set_yticks([-2.,-1.,0.,1.,2.])
ax.set_zlim(0.,100.)
ax.set_zticks([0.,20.,40.,60.,80.,100.])
plt.xlabel('x')
plt.ylabel('y')
plt.zlabel('z')
```

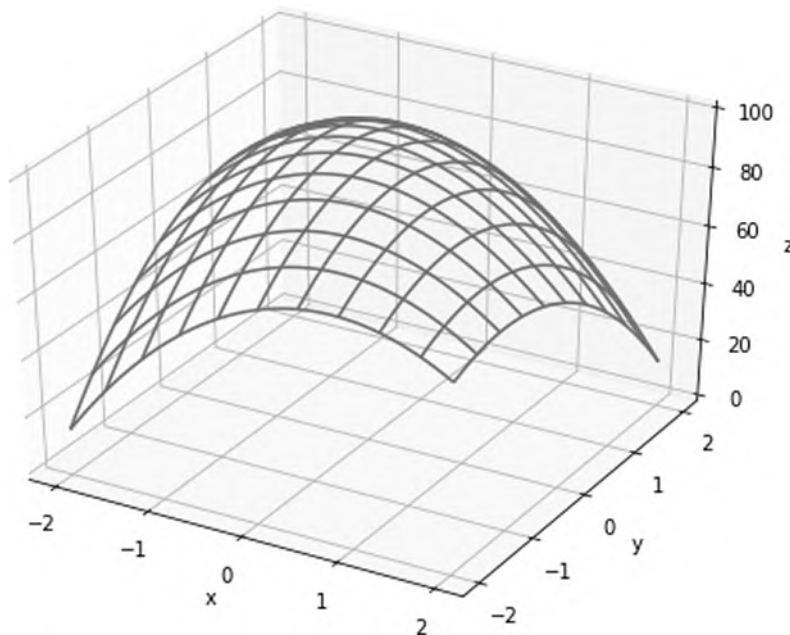




Notice the addition of the `projection='3d'` to the `add_subplot` command. We use the `axes` object, `ax`, here, and the `plot_wireframe` command. The `rstride` and `cstride` specifications allow us to adjust the density of the mesh. You can experiment with this to achieve the appropriate presentation. You can see how we specify the tick marks. That is not required, but typically it is used to take control of the axes' scales.

We can rotate the view of the surface up vertically  $35^\circ$  by adding the following statement:

```
ax.view_init(35)
```



Here, we're looking more at the top of the surface.

Rather than a mesh, or wireframe, `plot`, we can generate a solid surface plot with color shading.

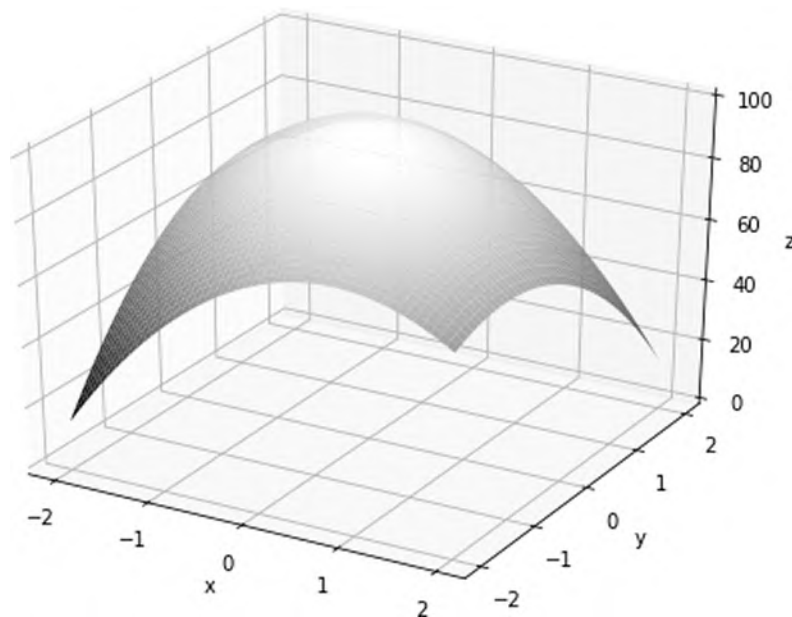
```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
```

```

from matplotlib import cm
def f(x,y):
    return 95.+0.05*x-0.145*y-8.13*x**2-5.87*y**2-6.25*x*y
x = np.linspace(-2.,2.)
y = np.linspace(-2.,2.)
X,Y = np.meshgrid(x,y)
Z = f(X,Y)

fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111,projection='3d')
ax.plot_surface(X,Y,Z,cmap=cm.gray)
ax.grid()
ax.set_xticks([-2.,-1.,0.,1.,2.])
ax.set_yticks([-2.,-1.,0.,1.,2.])
ax.set_zlim(0.,100.)
ax.set_zticks([0.,20.,40.,60.,80.,100.])
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')

```



We have included an import of the colormap (cm), submodule from Matplotlib. The color gray is chosen, although other colors are possible.

# EPILOGUE

---

There are so many possibilities for the creation of plots using Matplotlib that entire books have been written on the topic. Via the Matplotlib help site, many video tutorials are also available. We have only skimmed the surface (no pun intended) here with common plots. Through exploration, we are certain that you will find other types of plots of use to you, and we encourage you to do that.

## CUBIC SPLINE SMOOTHING

In [Sec. 18.7.1](#), we provided a brief description of cubic spline smoothing with enough information to explain the Python function `csp_linesm()` documented and illustrated there. The purpose of this appendix is to explain the derivation of the method in detail. To avoid page-flipping, we repeat here two paragraphs from that section in [Chap. 18](#).

In order to accomplish smoothing with cubic splines, we must relax the requirement that the cubic equation satisfies continuity with the noisy data points,  $\{x_i, y_i, i = 0, \dots, n\}$ . The smoothing spline function,  $s_i(x)$ , is instead chosen to minimize a “smoothing” objective function,

$$L = \lambda \sum_{i=0}^n \left[ \frac{y_i - s_i(x_i)}{\sigma_i} \right]^2 + (1 - \lambda) \sum_{i=0}^{n-1} \int_{x_i}^{x_{i+1}} [s''(x)]^2 dx \quad (18.33)$$

where, recalling

$$s_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$$

The first term of [Eq. \(18.33\)](#) becomes small as the spline function meets the continuity requirement of interpolating cubic splines, and likewise gets larger as the function departs from the data to achieve smoothness. The second term is small when the overall spline function is smooth and becomes large when the segment splines exhibit roughness to meet continuity. The lambda parameter,  $\lambda$ , is for tuning the amount of smoothing. When  $\lambda = 1$ , there is no smoothing, and we have the interpolating cubic spline. As  $\lambda \rightarrow 0$ , the smoothing is extreme. The  $\sigma_i$  values are generally estimates of the standard deviation of each individual element of the  $y_i$  series. For cases where standard deviations for each  $y$  value are unavailable, it is often common to use a single estimate,  $\sigma$ , the standard deviation of the  $y$ 's.

The derivation presented here is adapted from Pollock.<sup>1</sup> The  $s_i''(x)$  term is the second derivative of a cubic spline segment, and, as such, is a linear function that, for segment  $i$ , changes from  $2c_i$  at  $x_i$  to  $2c_{i+1}$  at  $x_{i+1}$ . Given this, we can write

$$\begin{aligned} \int_{x_i}^{x_{i+1}} [s_i''(x)]^2 dx &= 4 \int_0^{h_i} \left[ c_i \left( 1 - \frac{x}{h_i} \right) + c_{i+1} \frac{x}{h_i} \right]^2 dx \\ &= \frac{4h_i}{3} (c_i^2 + c_i c_{i+1} + c_{i+1}^2) \quad h_i \triangleq x_{i+1} - x_i \end{aligned}$$

The objective function to be minimized is then

$$L = \lambda \sum_{i=0}^n \left[ \frac{y_i - a_i}{\sigma_i} \right]^2 + (1 - \lambda) \sum_{i=0}^{n-1} \frac{4h_i}{3} (c_i^2 + c_i c_{i+1} + c_{i+1}^2)$$

From here, we consider the solution for the natural spline case, where  $s''(x_0)2c_0 = 0$  and  $s''(x_n)2c_n = 0$ . An additional issue is to determine the  $a_i$  values, which are no longer the data values,  $y_i$ . The strategy will be to eliminate the parameters  $d_i, b_i, i = 1, \dots, n-1$ , and then determine the parameters  $c_i, a_i, i = 1, \dots, n-1$ .

Consider the conditions on the  $i$ th cubic segment spanning the gap between the knots  $\{x_i, y_i\}$  and  $\{x_{i+1}, y_{i+1}\}$ . We denote the following:

Page 636

$$\begin{array}{lll} \text{at } \{x_i, a_i\} & s_i(x_i) = a_i & s_i''(x_i) = 2c_i \\ \text{at } \{x_{i+1}, a_{i+1}\} & s_i(x_{i+1}) = a_{i+1} & s_i''(x_{i+1}) = 2c_{i+1} \end{array}$$

We can rewrite the first terminal condition as

$$a_i + b_i h_i + c_i h_i^2 + d_i h_i^3 = a_{i+1}$$

and solve that for  $b_i$ ,

$$b_i = \frac{a_{i+1} - a_i}{h_i} - d_i h_i^2 + c_i h_i$$

The second terminal condition is

$$2c_{i+1} = 6d_i h_i + 2c_i \Rightarrow d_i = \frac{c_{i+1} - c_i}{3h_i}$$

The last two results provide  $b_i$  and  $d_i$  in terms of  $c_i$  and  $a_i$ .

The first-derivative continuity requirement is  $s'_{i-1}(x_i) = s'_i(x_i)$ , which can be written as

$$3d_{i-1}h_{i-1}^2 + 2c_{i-1}h_{i-1} + b_{i-1} = b_i$$

We can substitute for  $d_i$  from above to get

$$h_{i-1}c_{i-1} + 2(-h_{i-1} + h_i)c_i + h_ic_{i+1} = \frac{3}{h_i}(a_{i+1} - a_i) - \frac{3}{h_{i-1}}(a_i - a_{i-1})$$

This result can be assembled, for  $i = 1, \dots, n-1$ , into the following linear system of equations:

$$\begin{bmatrix} p_1 & h_1 & 0 & \cdots & 0 & 0 \\ h_1 & p_2 & h_2 & \cdots & 0 & 0 \\ 0 & h_2 & p_3 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & p_{n-2} & h_{n-2} \\ 0 & 0 & 0 & \cdots & h_{n-2} & p_{n-1} \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ \vdots \\ c_{n-2} \\ c_{n-1} \end{bmatrix} = \begin{bmatrix} r_0 & f_1 & r_1 & 0 & \cdots & 0 & 0 \\ 0 & r_1 & f_2 & r_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & r_{n-2} & 0 \\ 0 & 0 & 0 & 0 & \cdots & f_{n-1} & r_{n-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \\ a_n \end{bmatrix}$$

where  $p_i = 2(h_{i-1} - h_i)$ ,  $r_i = 3/h_i$ , and  $f_i = -3(1/h_{i-1} + 1/h_i)$ . With appropriate definitions of matrices, these equations can be written in compact form as

$$\mathbf{R}\mathbf{c} = \mathbf{Q}'\mathbf{a}$$

The objective function can then be written as

$$L = \lambda(\mathbf{y} - \mathbf{a})' \sum^{-1} (\mathbf{y} - \mathbf{a}) + \frac{2}{3}(1 - \lambda)\mathbf{b}'\mathbf{R}\mathbf{b}$$

where  $\sum = \text{diag}[\sigma_0, \sigma_1, \dots, \sigma_n]$ . If  $\sigma$  is constant,  $\sum = \sigma\mathbf{I}$ . From above,  $\mathbf{c} = \mathbf{R}^{-1}\mathbf{Q}'\mathbf{a}$ , so

$$L = \lambda(\mathbf{y} - \mathbf{a})' \sum^{-1} (\mathbf{y} - \mathbf{a}) + \frac{2}{3}(1 - \lambda)\mathbf{a}'\mathbf{Q}\mathbf{R}^{-1}\mathbf{Q}'\mathbf{a}$$

We can now minimize  $L$  by differentiating with respect to  $\mathbf{a}$  and setting the result equal to zero.

$$-2\lambda(\mathbf{y} - \mathbf{a})' \sum^{-1} + \frac{4}{3}(1 - \lambda)\mathbf{a}'\mathbf{Q}\mathbf{R}^{-1}\mathbf{Q}'\mathbf{a} = \mathbf{0}$$

This is rearranged to give

$$\lambda \sum^{-1}(\mathbf{y} - \mathbf{a}) = \frac{2}{3}(1 - \lambda)\mathbf{Q}\mathbf{c}$$

Now, by pre-multiplying by  $\frac{1}{\lambda}\mathbf{Q}'\Sigma$  and using  $\mathbf{R}\mathbf{c} = \mathbf{Q}'\mathbf{a}$ , we arrive at

$$(\mu\mathbf{Q}'\Sigma\mathbf{Q} + \mathbf{R})\mathbf{c} = \mathbf{Q}'\mathbf{y}$$

where

$$\mu = \frac{2(1 - \lambda)}{3\lambda}$$

This set of linear equations can be solved for the  $\mathbf{c}$  values. Given the  $\mathbf{c}$  values, we can then solve for the  $\mathbf{a}$ 's with

$$\mathbf{a} = \mathbf{y} - \mu \sum \mathbf{Q}\mathbf{c}$$

Noting from the natural end conditions that  $c_0 = c_n = 0$ , the polynomials'  $\mathbf{d}$  and  $\mathbf{b}$  coefficients are computed using

$$d_i = \frac{c_{i+1} - c_i}{3h_i} \quad \text{and} \quad b_i = \frac{a_{i+1} - a_i}{h_i} - \frac{1}{3}(c_{i+1} - 2c_i)h_i \quad \text{for } i = 0, \dots, n-1$$

We then have all the coefficients for the polynomials

$$s_i(x) = a_i + b_i \cdot (x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 \quad \text{for } i = 0, \dots, n-1$$

Here then is a step-by-step description of the procedure for applying cubic spline smoothing:

1. Acquire initial data  $\{x_i, y_i, i = 0, \dots, n\}$  and select the smoothing parameter,  $\lambda$ . Compute

$$\mu = \frac{2(1 - \lambda)}{3\lambda}.$$

## 2. Compute

$$\begin{aligned} h_i &= x_{i+1} - x_i & i &= 0, \dots, n-1 \\ p_i &= 2(h_{i-1} - h_i) & i &= 1, \dots, n-1 \\ r_i &= 3/h_i & i &= 0, \dots, n-1 \\ f_i &= -(r_{i+1} + r_i) & i &= 1, \dots, n-1 \end{aligned}$$

## 3. Form $\mathbf{R}$ and $\mathbf{Q}$ .

4. Obtain estimates for  $\sigma_0, \dots, \sigma_n$  or a single estimate  $\sigma$  and form  $\mathbf{\Sigma} = \text{diag}[\sigma_0, \sigma_1, \dots, \sigma_n]$  or  $\mathbf{\Sigma} = \sigma \mathbf{I}$ .

5. Solve for  $\mathbf{c}$  from  $(\mu \mathbf{Q}' \sum \mathbf{Q} + \mathbf{R}) \mathbf{c} = \mathbf{Q}' \mathbf{y}$ .

6. Solve for  $\mathbf{a}$  from  $\mathbf{a} = \mathbf{y} - \mu \sum \mathbf{Q} \mathbf{c}$ .

7. Solve for  $\mathbf{d}$  and  $\mathbf{b}$  from

$$\begin{aligned} d_i &= \frac{c_{i+1} - c_i}{3h_i} \\ b_i &= \frac{a_{i+1} - a_i}{h_i} - \frac{1}{3}(c_{i+1} - 2c_i)h_i \end{aligned} \quad \left| \begin{array}{l} \\ i = 1, \dots, n-1 \end{array} \right.$$

8. Employ the determined cubic spline smoothing equations for interpolation and plotting.

$$s_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 \quad i = 0, \dots, n-1$$

In [Fig. 18.12](#), we tested the above algorithm (as implemented with Python in [Fig. 18.12](#)) with a noisy set of chemical concentration measurements. [Table B.1](#) lists these data for your use in testing your version of the algorithm.

Page 638

**TABLE B.1** A noisy set of chemical concentration measurements,  $\mathbf{c}$  (g/L), developed for testing Python sm algorithms described in this appendix and [Chap. 18](#) (Box et al., 2015).

| Sample Number | $\mathbf{c}$ (g/L) | Sample Number | $\mathbf{c}$ (g/L) | Sample Number | $\mathbf{c}$ (g/L) | Sample Number | $\mathbf{c}$ (g/L) | Sample Number | $\mathbf{c}$ |
|---------------|--------------------|---------------|--------------------|---------------|--------------------|---------------|--------------------|---------------|--------------|
| 1             | 17                 | 41            | 17.6               | 81            | 16.8               | 121           | 16.9               | 161           |              |
| 2             | 16.6               | 42            | 17.5               | 82            | 16.7               | 122           | 17.1               | 162           |              |
| 3             | 16.3               | 43            | 16.5               | 83            | 16.4               | 123           | 16.8               | 163           |              |
| 4             | 16.1               | 44            | 17.8               | 84            | 16.5               | 124           | 17                 | 164           |              |
| 5             | 17.1               | 45            | 17.3               | 85            | 16.4               | 125           | 17.2               | 165           |              |



| Sample<br>Number | c (g/L) | Sample<br>Number | c (g/L) | Sample<br>Number | c (g/L) | Sample<br>Number | c (g/L) | Sample<br>Number | c |
|------------------|---------|------------------|---------|------------------|---------|------------------|---------|------------------|---|
| 6                | 16.9    | 46               | 17.3    | 86               | 16.6    | 126              | 17.3    | 166              |   |
| 7                | 16.8    | 47               | 17.1    | 87               | 16.5    | 127              | 17.2    | 167              |   |
| 8                | 17.4    | 48               | 17.4    | 88               | 16.7    | 128              | 17.3    | 168              |   |
| 9                | 17.1    | 49               | 16.9    | 89               | 16.4    | 129              | 17.2    | 169              |   |
| 10               | 17      | 50               | 17.3    | 90               | 16.4    | 130              | 17.2    | 170              |   |
| 11               | 16.7    | 51               | 17.6    | 91               | 16.2    | 131              | 17.5    | 171              |   |
| 12               | 17.4    | 52               | 16.9    | 92               | 16.4    | 132              | 16.9    | 172              |   |
| 13               | 17.2    | 53               | 16.7    | 93               | 16.3    | 133              | 16.9    | 173              |   |
| 14               | 17.4    | 54               | 16.8    | 94               | 16.4    | 134              | 16.9    | 174              |   |
| 15               | 17.4    | 55               | 16.8    | 95               | 17      | 135              | 17      | 175              |   |
| 16               | 17      | 56               | 17.2    | 96               | 16.9    | 136              | 16.5    | 176              |   |
| 17               | 17.3    | 57               | 16.8    | 97               | 17.1    | 137              | 16.7    | 177              |   |
| 18               | 17.2    | 58               | 17.6    | 98               | 17.1    | 138              | 16.8    | 178              |   |
| 19               | 17.4    | 59               | 17.2    | 99               | 16.7    | 139              | 16.7    | 179              |   |
| 20               | 16.8    | 60               | 16.6    | 100              | 16.9    | 140              | 16.7    | 180              |   |
| 21               | 17.1    | 61               | 17.1    | 101              | 16.5    | 141              | 16.6    | 181              |   |
| 22               | 17.4    | 62               | 16.9    | 102              | 17.2    | 142              | 16.5    | 182              |   |
| 23               | 17.4    | 63               | 16.6    | 103              | 16.4    | 143              | 17      | 183              |   |
| 24               | 17.5    | 64               | 18      | 104              | 17      | 144              | 16.7    | 184              |   |
| 25               | 17.4    | 65               | 17.2    | 105              | 17      | 145              | 16.7    | 185              |   |
| 26               | 17.6    | 66               | 17.3    | 106              | 16.7    | 146              | 16.9    | 186              |   |
| 27               | 17.4    | 67               | 17      | 107              | 16.2    | 147              | 17.4    | 187              |   |
| 28               | 17.3    | 68               | 16.9    | 108              | 16.6    | 148              | 17.1    | 188              |   |
| 29               | 17      | 69               | 17.3    | 109              | 16.9    | 149              | 17      | 189              |   |
| 30               | 17.8    | 70               | 16.8    | 110              | 16.5    | 150              | 16.8    | 190              |   |
| 31               | 17.5    | 71               | 17.3    | 111              | 16.6    | 151              | 17.2    | 191              |   |
| 32               | 18.1    | 72               | 17.4    | 112              | 16.6    | 152              | 17.2    | 192              |   |
| 33               | 17.5    | 73               | 17.7    | 113              | 17      | 153              | 17.4    | 193              |   |
| 34               | 17.4    | 74               | 16.8    | 114              | 17.1    | 154              | 17.2    | 194              |   |
| 35               | 17.4    | 75               | 16.9    | 115              | 17.1    | 155              | 16.9    | 195              |   |
| 36               | 17.1    | 76               | 17      | 116              | 16.7    | 156              | 16.8    | 196              |   |
| 37               | 17.6    | 77               | 16.9    | 117              | 16.8    | 157              | 17      | 197              |   |
| 38               | 17.7    | 78               | 17      | 118              | 16.3    | 158              | 17.4    |                  |   |
| 39               | 17.4    | 79               | 16.6    | 119              | 16.6    | 159              | 17.2    |                  |   |
| 40               | 17.8    | 80               | 16.7    | 120              | 16.8    | 160              | 17.2    |                  |   |



# REFERENCES

---

Pollock, D. S. G. Smoothing with cubic splines. Tech. rep., Queen Mary and Westfeld College, University of London, London, 1993.

Pollock, D. S. G. Smoothing with Cubic Splines, Dept. of Economics, Queen Mary and Westfield College, University of London, London, Paper No. 291, 1994.

# APPENDIX C

Page 639

## PYTHON BUILT-IN KEYWORDS: FUNCTIONS, METHODS, OPERATORS, TYPES

| Keyword     | Module               | Page | Keyword   | Module       |
|-------------|----------------------|------|-----------|--------------|
| abs         | Python               | 32   | format    | Python       |
| acos        | Math                 | 33   | gradient  | NumPy        |
| add_subplot | Matplotlib.pyplot    | 188  | grid      | pylab        |
| and         | Python               | 57   | help      | Python       |
| append      | Python               | 40   | hist      | pylab        |
| arange      | NumPy                | 26   | histogram | NumPy        |
| *args       | Python               | 73   | hstack    | NumPy        |
| array       | NumPy                | 24   | if        | Python       |
| asin        | Math                 | 33   | index     | Python       |
| atan        | Math                 | 33   | inner     | NumPy        |
| atan2       | Math                 | 33   | int       | Python       |
| average     | NumPy                | 42   | interp1d  | SciPy.interp |
| Axes3D      | mpl_toolkits.mplot3d | 188  | interp2d  | SciPy.interp |
| bar         | Matplotlib.pyplot    | 315  | inv       | NumPy.linalg |
| break       | Python               | 65   | isalpha   | Python       |
| brentq      | SciPy.optimize       | 159  | isdigit   | Python       |
| cholesky    | SciPy.linalg         | 250  | **kwargs  | Python       |
| cond        | NumPy.linalg         | 259  | lambda    | Python       |
| contour     | Matplotlib.pyplot    | 188  | legend    | pylab        |
| cos         | Math                 | 33   | len       | Python       |
| cosh        | NumPy                | 70   | linspace  | NumPy        |
| cross       | NumPy                | 31   | load      | NumPy        |
| CubicSpline | SciPy.interpolate    | 433  | loadtxt   | NumPy        |
| dblquad     | SciPy.integrate      | 479  | log       | Math         |
| degrees     | Math                 | 33   | log10     | Math         |
| delete      | NumPy                | 44   | log10     | NumPy        |

| Keyword           | Module            | Page | Keyword                   | Module       |
|-------------------|-------------------|------|---------------------------|--------------|
| det               | NumPy.linalg      | 209  | log2                      | NumPy        |
| diff              | NumPy             | 516  | logspace                  | NumPy        |
| docstring         | Python            | 50   | lower                     | Python       |
| dot               | Python            | 31   | lu                        | SciPy.linalg |
| dstack            | NumPy             | 210  | matrix                    | NumPy        |
| eig               | NumPy.linalg      | 295  | matrix_rank               | NumPy.linalg |
| eigvals           | NumPy.linalg      | 295  | max                       | Python       |
| elif              | Python            | 58   | mean                      | NumPy        |
| else              | Python            | 58   | median                    | NumPy        |
| exp               | Math              | 33   | median_absolute_deviation | SciPy.stats  |
| eye               | NumPy             | 26   | meshgrid                  | NumPy        |
| factorial         | Math              | 89   | min                       | Python       |
| fft               | SciPy.fft         | 388  | minimize                  | SciPy.optimi |
| figure            | Matplotlib.pyplot | 188  | minimize_scalar           | SciPy.optimi |
| finfo             | NumPy             | 95   | mode                      | SciPy.stats  |
| float             | Python            | 28   | norm                      | NumPy.linalg |
| for               | Python            | 61   | norm.pdf                  | SciPy.stats  |
| normal            | NumPy.random      | 319  | set_yticks                | Matplotlib.p |
| normalvariate     | random            | 39   | show                      | Matplotlib.p |
| not               | Python            | 57   | sign                      | NumPy        |
| ones              | NumPy             | 26   | sin                       | Math         |
| ones              | NumPy             | 207  | size                      | NumPy        |
| or                | Python            | 57   | solve                     | NumPy.linalg |
| outer             | NumPy             | 207  | solve_bvp                 | SciPy.integr |
| PChipInterpolator | SciPy.interpolate | 437  | solve_ivp                 | SciPy.integr |
| pi                | NumPy             | 70   | sqrt                      | Math         |
| plot              | pylab             | 35   | sqrt                      | NumPy        |
| plot_wireframe    | Matplotlib.pyplot | 188  | startswith                | Python       |
| poly              | NumPy             | 162  | std                       | NumPy        |
| poly1d            | NumPy             | 162  | stem                      | Matplotlib.p |
| polyfit           | NumPy             | 338  | str                       | Python       |
| polyval           | NumPy             | 162  | subplot                   | Matplotlib.p |
| print             | Python            | 28   | sum                       | NumPy        |
| quad              | SciPy.integrate   | 498  | tan                       | Math         |
| quiver            | Matplotlib.pyplot | 521  | tanh                      | NumPy        |
| radians           | Math              | 33   | title                     | pylab        |

| Keyword          | Module            | Page | Keyword   | Module       |
|------------------|-------------------|------|-----------|--------------|
| range            | Python            | 61   | trace     | NumPy        |
| romberg          | SciPy.integrate   | 491  | transpose | NumPy        |
| root             | SciPy.optimize    | 277  | trapz     | NumPy        |
| roots            | NumPy             | 162  | twinx     | PyLab        |
| round            | Python            | 32   | type      | Python       |
| save             | NumPy             | 56   | uniform   | NumPy.random |
| savetxt          | NumPy             | 55   | upper     | Python       |
| scatter          | pylab             | 37   | var       | NumPy        |
| semilogx         | pylab             | 167  | vstack    | NumPy        |
| set_printoptions | NumPy             | 42   | weibull   | random       |
| set_title        | Matplotlib.pyplot | 188  | while     | Python       |
| set_xlabel       | Matplotlib.pyplot | 188  | xlabel    | pylab        |
| set_xticks       | Matplotlib.pyplot | 188  | ylabel    | pylab        |
| set_ylabel       | Matplotlib.pyplot | 188  | zeros     | NumPy        |

# APPENDIX D

Page 641

## PYTHON FUNCTIONS AND SCRIPTS PRESENTED IN THE TEXT

*Note: These are functions and longer code segments.*

| Function    | Page | Function        | Page |
|-------------|------|-----------------|------|
| bisect      | 130  | newtraph        | 153  |
| bisect1     | 129  | odesimp         | 76   |
| brentsimp   | 159  | odesimp2        | 76   |
| cspline     | 433  | quadadapt       | 498  |
| csplinesm   | 442  | quadroots       | 67   |
| diffex      | 108  | rk4sys          | 551  |
| eulode      | 538  | rkf45           | 564  |
| fixpt       | 147  | romberg         | 490  |
| funcavg     | 71   | strlinregr      | 336  |
| gaussnaive  | 227  | TableLookup     | 424  |
| gausspivot  | 232  | TableLookup2    | 423  |
| GaussSeidel | 269  | TableLookupBin  | 425  |
| goldmin     | 183  | trap            | 467  |
| incsearch   | 123  | trap_cumulative | 474  |
| itermeth    | 89   | trapuneq        | 474  |
| Lagrange    | 407  | tridiag         | 234  |
| loess       | 446  | velocity1       | 74   |
| Newtint     | 405  | wegstein        | 148  |
| newtmult    | 276  |                 |      |

Page 642

| Script          | Page |
|-----------------|------|
| Case Study 15.6 | 369  |
| Case Study 16.7 | 392  |
| Case Study 18.8 | 448  |
| Case Study 22.6 | 553  |

| <b>Script</b> | <b>Page</b> |
|---------------|-------------|
| Example 14.4  | 319         |
| Example 16.3  | 389         |
| Example 16.4  | 391         |
| Example 17.7  | 412         |
| Figure 15.5   | 362         |
| SSE plus code | 366         |



# BIBLIOGRAPHY

- Anscombe, F. J., "Graphs in Statistical Analysis," *Am. Stat.*, 27(1):17–21, 1973.
- Bird, R. B., W. E. Stewart, and E. N. Lightfoot, *Transport Phenomena*, 2nd ed., Wiley, New York, 2007.
- Bogacki, P., and L. F. Shampine, "A 3(2) Pair of Runge-Kutta Formulas," *Appl. Math. Lett.*, 2(1989):1–9, 1989.
- Box, G. E. P., et al., *Time Series Analysis: Forecasting and Control*, 5th ed., Wiley, New York, 2015.
- Brent, R. P., *Algorithms for Minimization Without Derivatives*, Prentice Hall, Englewood Cliffs, NJ, 1973.
- Butcher, J. C., "On Runge-Kutta Processes of Higher Order," *J. Austral. Math. Soc.*, 4:179, 1964.
- Carnahan, B., H. A. Luther, and J. O. Wilkes, *Applied Numerical Methods*, Wiley, New York, 1969.
- Chapra, S. C., and R. P. Canale, *Numerical Methods for Engineers*, 8th ed., McGraw-Hill, New York, 2021.
- Cooley, J. W., and J. W. Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series," *Math. Comput.*, 19:297–301, 1965.
- Dekker, T. J., "Finding a Zero by Means of Successive Linear Interpolation." In B. Dejon and P. Henrici (editors), *Constructive Aspects of the Fundamental Theorem of Algebra*, Wiley-Interscience, New York, 1969, pp. 37–48.
- Devaney, R. L. *Chaos, Fractals, and Dynamics: Computer Experiments in Mathematics*, Addison-Wesley, Menlo Park, CA, 1990.
- Dormand, J. R., and P. J. Prince, "A Family of Embedded Runge-Kutta Formulae," *J. Comp. Appl. Math.*, 6:19–26, 1980.
- Draper, N. R., and H. Smith, *Applied Regression Analysis*, 2nd ed., Wiley, New York, 1981.
- Fadeev, D. K., and V. N. Fadeeva, *Computational Methods of Linear Algebra*, Freeman, San Francisco, 1963.
- Forsythe, G. E., M. A. Malcolm, and C. B. Moler, *Computer Methods for Mathematical Computation*, Prentice Hall, Englewood Cliffs, NJ, 1977.
- Gabel, R. A., and R. A. Roberts, *Signals and Linear Systems*, 3rd ed., Wiley, New York, 1987.
- Gander, W., and W. Gautschi, *Adaptive Quadrature-Revisited*, *BIT Num. Math.*, 40:84–101, 2000.
- Gear, C. W., "The automatic integration of stiff ordinary differential equations," *Information Processing*, 68, A. J. H. Morell, Ed., North Holland, Amsterdam, pp. 187–193, 1969.
- Gerald, C. F., and P. O. Wheatley, *Applied Numerical Analysis*, 6th ed., Addison-Wesley, Reading, MA, 1998.
- Hayt, W. H., and J. E. Kemmerly, *Engineering Circuit Analysis*, McGraw-Hill, New York, 1986.

Heidemman, M. T., D. H. Johnson, and C. S. Burrus, "Gauss and the History of the Fast Fourier Transform," *IEEE ASSP Mag.*, 1(4):14–21, 1984.

Hill, C., *Learning Scientific Programming with Python*, Cambridge University Press, Cambridge, 2015.

Hornbeck, R. W., *Numerical Methods*, Quantum, New York, 1975.

James, M. L., G. M. Smith, and J. C. Wolford, *Applied Numerical Methods for Digital Computations with FORTRAN and CSMP*, 3rd ed., Harper & Row, New York, 1985.

Moler, C. B., *Numerical Computing with MATLAB*, SIAM, Philadelphia, 2004.

Montgomery, D. C., and G. C. Runger, *Applied Statistics and Probability for Engineers*, 6th ed., Wiley, New York, 2014.

Munson, B. R., D. F. Young, T. H. Okiishi, and W. D. Huebsch, *Fundamentals of Fluid Mechanics*, 7th ed., Wiley, Hoboken, NJ, 2013.

Nagar, S., *Introduction to Python for Engineers and Scientists*, Apress-Springer, New York, 2018.

Ortega, J. M., *Numerical Analysis-A Second Course*, Academic Press, New York, 1972.

Press, W. H., S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes in Fortran: The Art of Scientific Computing*, Cambridge University Press, Cambridge, 1992.

Ralston, A., "Runge-Kutta Methods with Minimum Error Bounds," *Math. Comp.*, 16:431, 1962.

Ralston, A., and P. Rabinowitz, *A First Course in Numerical Analysis*, 2nd ed., McGraw-Hill, New York, 1978.

Ramalho, L., *Fluent Python*, O'Reilly, Sebastopol, CA, 2015.

Ramirez, R. W., *The FFT, Fundamentals and Concepts*, Prentice Hall, Englewood Cliffs, NJ, 1985.

Recktenwald, G., *Numerical Methods with MATLAB*, Prentice Hall, Englewood Cliffs, NJ, 2000.

Scarborough, I. B., *Numerical Mathematical Analysis*, 6th ed., The Johns Hopkins University Press, Baltimore, MD, 1966.

Shampine, L. F., *Numerical Solution of Ordinary Differential Equations*, Chapman & Hall, New York, 1994.

Tale, S., *Python - The Ultimate Beginners Guide*, Create Space, 2016.

Tale, S., *Python 3 - The Ultimate Beginners Guide for Python 3 Programming*, Create Space, 2017.

Tukey, J. W. *Exploratory Data Analysis*, Addison-Wesley, Reading, MA, 1971.

VanderPlas, J., *Python Data Science Handbook*, O'Reilly, Sebastopol, CA, 2017.

Van Valkenburg, M. E., *Network Analysis*, Prentice Hall, Englewood Cliffs, NJ, 1974.

White, F. M., *Fluid Mechanics*. 8th ed., McGraw-Hill, New York, 2015.

# INDEX

## A

- Absolute error, 86
- Accuracy, 85
- Adaptive integration, 456
- Adaptive methods
  - adaptive Runge-Kutta methods, 561–571
  - multistep methods, 572–575
  - Pliny's intermittent fountain, 582–585
- Adaptive quadrature, 486, 496–499
- Adaptive Runge-Kutta methods, 561–571
  - adaptive step-size control, 561–562
  - events, 569–571
- Adjusted  $R^2$ , 359
- Allosteric enzymes, 340
- Amplification factor, 537
- Amplitude, 376
- Analytical solution
  - to bungee jumper problem, 5–7
  - defined, 6
  - for heated rod, 594–595
- Angular frequency, 376
- Antoine equation, nonlinear regression of, 364–368
- Areal integral, 460
- Arguments, 60–61
- Arithmetic manipulations of computer numbers, 96–97
  - adding large and small number, 97
  - inner products, 97
  - large computations, 96–97
  - smearing, 97
  - subtractive cancellation, 96, 107
- Arrays, 23–26
  - matrices, 23–26
  - scalars, 22–23
  - subscript/subscript ranges, 26
  - vectors, 23–26
- Associative property, 203–204
- Augmentation, 205
- Avogadro's number, 93

## B

- Backslash ( \ ) character, 27
- Back substitution, 226
- Backward elimination technique, 359
- Backward Euler's method, 576
- Backward finite difference, 104–105, 511
- Banded matrix, 203
- Base-2 number system, 91
- Base-8 number system, 91
- Base-10 number system, 90
- Bias, 85
- Biased format, 94
- Bilinear interpolation, 437–439
- Binary number system, 91
- Binary search, 425
- Bisection method, 125–131
- Bits, 90
- Blasius formula, 164
- Blunders, 111
- Bode plots, 627–628
- Boolean variable, 582
- Boole's rule, 472–473, 497, 547
- Boundary-value problems, 530, 592
  - in engineering and science, 593–595
  - finite-difference methods, 601–607
  - initial-value problem vs., 592–593
  - shooting method, 595–601
  - single ODE, 591
- Box plots, 322, 626–627
- Bracketing methods, 122–125
- Brent, Richard Peirce, 156, 158n2
- Brent's root finding method, 156–160
  - algorithm, 158–160
  - inverse quadratic interpolation, 156–158
- Brent's optimization method, 186–187
- Bungee jumper problem, 3–4
  - analysis, 256
  - analytical solution to, 5–7
  - with bungee cord, 580–581
  - case study, 13–15
  - force balances, 200
  - free-falling, 527, 532–533
  - Newton-Raphson method applied to, 154
  - numerical solution to, 8–9
  - overview, 3–4
  - solving with Python, 211–212

velocity, 74–76  
Butterfly effect, 555  
Byte, 90

## C

Cantilever beam, 193  
Cartesian coordinates, 460  
Case studies  
    chemical reactions, 278–280  
    computing work with numerical integration, 479–482  
    drag force, 13–15  
    eigenvalues and earthquakes, 296–299  
    enzyme kinetics, 339–342  
    equilibrium and minimum potential energy, 189–191  
    fitting experimental data, 368–371  
    greenhouse gases and rainwater, 133–136  
    heated rod model, 235–237  
    heat transfer in lake, 447–450  
    indoor air pollution, 260–263  
    pipe friction, 164–168  
    Pliny’s intermittent fountain, 582–585  
    predator-prey models and chaos, 552–556  
    root-mean-square current, 499–502  
    sunspots, 391–393  
Cauchy’s equation, 357–358  
Centered finite difference, 104–105, 511  
Central limit theorem, 312  
Chaos, 552–556  
Character strings, 27–28  
Characteristic polynomial, 285  
Cholesky factorization, 249–251  
Circuits, currents and voltages in, 212–215  
Clamped end condition, 431–432  
Classical fourth-order Runge-Kutta methods, 545–547  
Closed-form solution, 6  
Closed integration formulas, 456, 462  
cma t h module, 33  
Coefficient matrix, 221  
Coefficient of determination, 327  
Colebrook equation, 140, 164, 172  
Collinearity, 360  
Colon (:), 26, 56, 228  
co l o r argument, 37–38  
Colors, line, 618

- Column, matrix, 201
- Column-sum norm, 258
- Column vectors, 202
- Commutative property, 203–204
- Companion matrix, 162
- Complete pivoting, 230
- Composite Simpson's 1/3 rule, 469–471
- Composite trapezoidal rule, 464–466
- Computer algorithm for iterative calculations, 88–90
- Computer number representation, 90–91
  - floating-point representation, 92–95
  - integer representation, 91–92
  - precision, 95
  - range, 95
- Conditionally stable method, 537
- Conservation laws, 9–10, 11
- Conservation of charge, 11, 212
- Conservation of energy, 11, 213
- Conservation of mass, 11
- Conservation of momentum, 11
- Constant of integration, 529
- Constitutive laws, 508
- Continue command, 66
- Continuity condition, 426
- Continuous Fourier series, 381–383
  - approximation, 381–382
  - Euler's formula, 382–383
- Continuous tables, 422
- contourf command, 632
- Contour plots, 631–634
- Convergence, 141, 268
  - linear, 143
  - quadratic, 150
- Cooley, J. W., 388
- Cooley-Tukey algorithm, 388
- Corrector equation, 539, 540
- Covariance matrix, 356
- Cramer's rule, 219, 221–223, 274
- Critical point, 554
- Cross product, 31
- Cubic splines, 418, 428–432
  - derivation of, 428–430
  - end conditions, 431–432
  - natural, 430–431
- Cubic spline smoothing, 440–443, 635–638

- Current rule, 212
- Currents, in circuits, 212–215
- Curvature, 508
- Curve fitting
  - defined, 303
  - engineering and science, 303–304
  - least-squares fit of sinusoid, 378–380
  - plot of sinusoidal function, 377
  - with sinusoidal functions, 376–380
  - through data points, 304

## D

- Danckwertz boundary condition, 610
- Darcy's law, 509
- Darcy-Weisbach equation, 172
- Data files
  - reading, 54–55
  - writing, 55
- Data smoothing, 439
  - cubic spline smoothing, 440–443
  - LOESS method, 443–447, 443n3
- Data uncertainty, 111–112
- Decelerator, 277
- Decisions, 56
  - arguments, 60–61
  - `if . . . elif . . . else` structure, 58–59
  - `if . . . else` structure, 58
  - `if` structure, 56–57, 59–60
  - logical conditions, 57–59
- Decker, Theodorus, 156
- Definite integrals, 529
- Deflation technique, 161
- Degrees of freedom, 309, 326
- Dependent variable, 4, 527
- Derivative, 455, 507–508
  - boundary conditions, 603
  - for data with errors, 514–515
  - finite-difference methods, 603–605
  - shooting method and, 597–599
  - of unequally spaced data, 513–514
- Derivative mean-value theorem, 101–103
- Descriptive statistics, 307–311
  - location measurement, 309
  - sample statistics, 310–311

- spread measurement, 309–310
- using Python, 314–316
- “Desert island” method, 324–325
- Design problems, fundamental principles used in, 118
- Determinant, 221–222
  - evaluation with Gauss elimination, 232–233
- DFT. *See* Discrete Fourier transform (DFT)
- Diagonal dominance, 268
- Diagonal matrix, 202
- Dictionary, collection type, 24
- Differential equations, 3, 5, 48, 527
  - eigenvalues of system, 288–289
  - first-order, 287
  - ordinary, 12
  - role of eigenvalues and eigenvectors in solution of, 287–289
  - second-order, 287
- Differentiation, 455–456, 507–509
  - in engineering and science, 508–509
- Digits, 90
- Direct method, 188
- Dirichlet boundary condition, 597, 603
- Discrete Fourier transform (DFT), 386–390
  - computation of simple sinusoid with Python, 388–390
  - fast Fourier transform (FFT), 387–388
- Discrete tables, 422
  - extraction of information from, 422–423
- Discretization errors, 535
- Distributed variable problems, 198
- Divided difference table, 403
- Dot notation, 507n1
- Dot product. *See* Inner products
- Double integral, 477
  - to determine average temperature, 478
- Double-precision standard (IEEE 754), 94, 94n3
- Drag coefficient, lumped, 3, 5, 13
- Drag force, 13–15

## E

- Eigenvalues, 199
  - applications of, 286–291
  - basics, 284–286
  - defined, 284
  - and earthquakes (case study), 296–299
  - mass-spring systems, 291–293



- of matrix, 285–286
  - and ordinary differential equations with pure oscillations, 289–291
  - power method, 293–295
  - with Python, 295–296
  - role in solution of differential equations, 287–289
- Eigenvectors, 199, 284
  - applications of, 286–291
  - with Python, 295–296
- Element, matrix, 201
- Elimination of unknowns, 223–224
- Embedded RK methods, 562
- `endswith(suffix)` method, 28
- Enzyme kinetics, 339–342
- Epilimnion layer, 447
- Error(s), 84–85
  - absolute, 86
  - accuracy, 85
  - amplification, 515
  - for bisection, 127–129
  - blunders, 111
  - computer algorithm for iterative calculations, 88–90
  - data uncertainty, 111–112
  - defined, 86–87
  - estimation, 574–575
  - for iterative methods, 87–88
  - for linear least-squares fit, 327–330
  - model, 111
  - numerical, 86
  - overflow, 93
  - percent relative, 86–87
  - precision, 85
  - quantification, in linear regression, 325–330
  - relative fractional, 86
  - roundoff. *See* Roundoff errors
  - total numerical, 107–110
  - truncation. *See* Truncation errors
- Error bars, 625
- Error sum of squares, 325
- Euclidean norm, 257
- Euler-Cauchy method. *See* Euler’s method
- Euler’s formula, 382–383
- Euler’s identity, 289n2
- Euler’s method, 7, 10, 48, 530, 533, 534–543, 576
  - error analysis for, 535–536
  - fundamental source of error in, 539

- Heun's method, 539–542
- implicit, 576–578
- midpoint method, 542–543
- solving systems of ODEs with, 548
- stability of, 536–537

Euler's number, 23

Events, 569–571

Exploratory data analysis, 41–44

Exponential model, 330–331

Extrapolation, 409–411

## F

False position, 131–132, 156

- formula, 131

Fanning friction factor, 140, 164, 172

Fast Fourier transform (FFT), 305, 387–388

Fibonacci sequence, 179n1

Fick's law, 509

Finite-difference approximation, 7, 103

- backward, 104–105, 511
- centered, 104–105, 511
- of derivatives, 105–106
- forward, 103–104, 510
- of higher derivatives, 106–107

Finite-difference methods, 601–607

- of boundary-value problems, 602–603
- derivative (Neumann) boundary conditions, 603–605
- implementation, 603
- for nonlinear ODEs, 605–607

Finite divided differences, 403

First derivative

- backward difference approximation of, 104–105, 511
- centered difference approximation of, 104–105, 511
- forward difference approximation of, 103–104, 510
- eigenvalues of system, 288–289

First-order differential equations, 287

First-order expansion, 98–99

First-order splines, 421–422

Fitting experimental data, 368–371

Fixed-point iteration, 142–149, 268

Floating-point operations (flops), 228

Floating-point representation, 92–95

flops. *See* Floating-point operations (flops)

Forcing functions, 4

- Forward elimination of unknowns, 225–226
- Forward finite difference, 103–104, 510
- Forward selection technique, 359
- Fourier, Jean-Baptiste Joseph, 375n1
- Fourier analysis, 305, 355, 375–376
  - continuous Fourier series, 381–383
  - curve fitting with sinusoidal functions, 376–380
  - discrete Fourier transform (DFT), 386–390
- Fourier integral and transform, 384–386
  - frequency and time domains, 383–384
  - power spectrum, 390–391
  - sunspots, 391–393
- Fourier coefficients, 390
- Fourier integral, 384–386
- Fourier’s law, 509, 514, 593–594, 598
  - of heat conduction, 508
- Fourier transform, 384–386
- Fourth-order RK method, 549–550
- Frequency domain, 383–384
- Frequency plane, 383
- Friction factor, 164
- Frobenius norm, 257
- Function functions, 69–72
- Functions, 50–52, 641
  - built-in, 32–35
  - forcing, 4
  - with function name arguments, 68–74
  - passing parameters, 72–74
  - splines, 418
  - string, 28
  - trigonometric, 33
- Fundamental frequency, 381

## G

- Gauss elimination, 199, 219
  - Cramer’s rule, 221–223
  - determinants, 221–222
  - elimination of unknowns, 223–224
  - graphical method, 220–221
  - as *LU* factorization, 244–249
  - naive. *See* Naive Gauss elimination
  - with Python, 248–249
  - pivoting, 230–233, 247–248
  - solving small numbers of equations, 219–220

- substitution steps, 245–246
- tridiagonal systems, 233–235
- Gaussian distribution, 312. *See also* Normal distribution
- Gauss-Legendre formula
  - three-point, 495
  - two-point, 493–495, 501
  - weighting factors and function arguments used in, 496
- Gauss quadrature, 456, 486, 491–496
  - higher-point formulas, 495–496
  - two-point Gauss-Legendre formula, 493–495
  - undetermined coefficients method, 491–493
- Gauss-Seidel method, 199, 266–271, 606
  - with relaxation, 270–271
- General linear and nonlinear regression, 355–358
  - model building and selection in regression, 359–364
  - multiple linear regression, 353–355
  - polynomial regression, 349–352
  - refractive index of borosilicate glass, 357–358
- General linear least-squares regression, 355–358
- Gigabyte, 128n2
- Global optimum, 178
- Global truncation error, 535
- Golden ratio, 179
- Golden-section search, 179–184
- Gradient, 188, 508–509, 593, 595, 598–599
- Graphical methods, 118–119, 220–221
- Graphics, 35–40, 520–522
- Gridlines, 619–620

## H

- Haaland equation, 165
- Half-saturation constant, 339–340
- Harmonics, 381
- Heat transfer,
  - heated rod, 593–607
  - in lake, 447–450
- Hertz (Hz), 376n2
- Heun's method, 539–542
  - corrector equation, 539–540
  - non-self-starting, 572–574
  - predictor-corrector approach, 540
  - predictor equation, 539
  - without iteration, 544–545
- High-accuracy differentiation formulas, 509–512

- Taylor series, 509–510
- High-accuracy finite-difference formulas, 456
- Higher-order corrections, 488
- Higher-order differential equations, 527–528
- Higher-order polynomial interpolation, dangers of, 411–413
- Higher-point formulas, 495–496
- Hilbert matrix, 258–259
- Histogram, 39, 308, 311–314, 625–626
- Hooke's law, 189, 509, 533
- Humps functions, 499n1, 615
- Hybrid operator, 66n6
- Hyperbolic tangent, 5, 5n1
- Hypolimnion layer, 447
- Hypothesis testing, 303–304
- Hysteresis, 582

## I

- Identity matrix, 202–203
- IEEE 754 double-precision standard, 94, 94n3
- `if...elif...else` structure, 58–59
- `if...else` structure, 58
- `if` structure, 56–57, 59–60
- Ill-conditioned calculations, 90
- Ill-conditioned ODEs, 537
- Ill-conditioned systems, 220–221
- Impedance, 170n8
- Implicit Euler's method, 576–578
- Imprecision, 85
- Inaccuracy, 85
- Incremental search, 122–125
- Increment function, 533, 543
- Indefinite integral, 529
- Indentation, 66–68
- Independent variable, 4, 527
- Infinite loop, 65
- Initial-value problems, 530
  - boundary-value problems vs., 592–593
  - Euler's method, 533, 534–543
  - free-falling bungee jumper problem, 532–533
  - predator-prey models and chaos, 552–556
  - RK methods, 543–547
  - signum function, 532–533, 532n1
  - systems of equations, 547–552
- Inner products, 31, 97

- Integer representation, 91–92
- Integrals, 459–460
  - for data with errors, 514–515
- Integrated Development Environment (IDE), 21
- Integration, 455–456, 458
  - in engineering and science, 459–460
  - with unequal segments, 473–477
- International sunspot numbers, 391n3
- Interpolation, 11, 12, 303, 305, 397
  - linear, 396, 400–401
  - multidimensional. *See* Multidimensional interpolation
  - piecewise. *See* Piecewise interpolation
  - polynomial. *See* Polynomial interpolation
  - polynomial coefficients, determination of, 397–399
  - spline, 305
- Interval of uncertainty, 128
- Inverse, of matrix, 204, 253
  - bungee jumper problem analysis, 256
  - calculating, 253–254
  - error analysis and system condition, 256–257
  - matrix condition number, 258–259
  - norms and condition number with Python, 259–260
  - stimulus-response computations, 255–256
  - vector and matrix norms, 257–258
- Inverse Fourier transform, 386
- Inverse interpolation, 408–409. *See also* Interpolation
- Inverse quadratic interpolation, 156–158
- IPython Console, 2, 21, 22n2
  - features of, 31
- Isobars, 631
- Iterative methods
  - linear systems, 266–271
  - nonlinear systems, 271–278

## J

- Jacobian matrix, 275–276
- Jacobi iteration, 268
- Jacobi method, 199
- Joule’s law, 499–500

## K

- Keyword arguments, 61
- Keywords, built-in, 639–640
- Kilobyte, 128n2

Kirchoff's current rule, 212  
Kirchoff's voltage loop rule, 213  
Knot, 421

## L

Lagging phase angle, 377  
Lagrange function, 407–408  
Lagrange interpolating polynomial, 305, 405–408, 513  
Lagrange polynomial, 158, 513  
 $\lambda$  functions, 69  
Leading phase angle, 377  
Least squares  
    criterion, 322  
    fit of sinusoid, 378–380  
    regression, 303  
Linear algebraic equations, 10  
    distributed variable problems, 198  
    in engineering and science, 197–198  
    free-body diagrams, 200–201  
    Gauss-Seidel method and, 266  
    general form, 197  
    lumped variable problems, 198  
    matrix algebra, 201–211  
    representing in matrix form, 210–211  
    solving with Python, 211–212  
Linear convergence, 143  
Linear function, 99  
Linear interpolation, 396, 400–401  
Linearization of nonlinear relationships, 330–335  
    exponential model, 330–331  
    fitting data with power equation, 332–335  
    power equation, 330–331  
    saturation-growth-rate equation, 330–331  
Linear ODE, shooting method for, 596–597  
Linear regression, 304  
    multiple, 304  
    quantification of error in, 325–330  
    straight-line. *See* Straight-line linear regression  
Linear splines, 420–425. *See also* Cubic splines  
    first-order splines, 421–422  
Linear systems, 266–271  
    convergence and diagonal dominance, 268  
    Gauss-Seidel method, 266–268  
    relaxation, 270–271

- Line colors, 618
- Line/marker plots, 625
- Line plots, customizing and adding features to, 618–624
- Line spectra, 383, 385
- Linestyle, 618–619
- Lists, 24, 25
- Local optimum, 178
- Local truncation error, 535–536
- LOESS method, 443–447, 443n3
- Logical conditions, 57–59
- Logical operators, 57
- Logical variable, 582
- Loop(s), 61–66
  - preallocation of memory, 64
  - rule, 213
  - vectorization, 63–64
- Lorenz equations, 553
- Lotka, Alfred, 552
- Lotka-Volterra equations, 552–553, 555
- Lower triangular matrix, 203
- LSODA function, 578–580
- LU* factorization, 199
  - Cholesky factorization, 249–251
  - Gauss elimination as, 244–249
  - $n$ -dimensional systems, 242
  - overview of, 242–243
  - with pivoting, 247–248
  - substitution steps, 245–246
  - two-step strategy, 243
- Lumped drag coefficient, 3, 5, 13
- Lumped variable problems, 198

## M

- Machine epsilon, 94
- Maclaurin series expansion, 87
- Main diagonal, of matrix, 202
- Mantissa, 92–93
- marker argument, 37
- Marker plots, 625
- Mass-spring systems, 291–293
- Mathematical model, 2
  - analytical solution to bungee jumper problem, 5–7
  - characteristics of, 4
  - defined, 4



- numerical solution to bungee jumper problem, 8–9
- Mathematical operations, 28–32
  - functions of, 33n6, 34n7
  - math module, 23, 33
- Matplotlib, 614
  - Bode plots, 627–628
  - contour plots, 631–634
  - line plots, customizing and adding features to, 618–624
  - multiple plots in single figure/axes objects, 629–631
  - simple plot, creation of, 614–618
  - statistical plots, 624–627
  - stem plots, 627
  - surface plots, 631–634
- Matrix, 23–26
  - addition, 203
  - algebra, 199, 201–211
  - banded, 203
  - coefficient, 221
  - column, 201
  - companion, 162
  - defined, 201
  - diagonal, 202
  - eigenvalues and eigenvectors of, 285–286
  - element, 201
  - form, representing linear algebraic equations in, 210–211
  - Hilbert, 258–259
  - identity, 202–203
  - inverse of. *See* Inverse, of matrix
  - Jacobian, 275–276
  - lower triangular, 203
  - manipulation, 205–210
  - multiplication, 203–204
  - norms, 257–258
  - notation, 201–203
  - operating rules, 203–210
  - permutation, 205
  - principal/main diagonal of, 202
  - row, 201
  - subtraction, 203
  - symmetric, 202
  - transpose of, 31, 204–205
  - transposition, 205
  - tridiagonal, 203
  - upper triangular, 203, 247
- Matrix condition number, 258–260

- Matrix exponential, 287
- Matrix inverse, 199
- Maximum likelihood principle, 326
- maxrow function, 231
- Mean, 309
- Mean value, 376
- Median, 309
- Median absolute deviation (MAD), 310
- Median-Median Line Algorithm, 324
- Megabyte, 128n2
- Meshgrid, 632
- M-estimate, 309
- Michaelis-Menten model, 339–342
- Midpoint method, 542–543, 545, 573
- Midtest loop, 65
- Minimax criterion, 322
- Minors, 221
- Mixed partial derivative, 515
- Mode, 309
- Model adequacy, 325–326
- Model building, in regression, 359–364
- Model errors, 111
- Model realizations, 350
- Modified secant method, 155–156
- Modulus of toughness, 503
- Monte Carlo simulation, 318
- Moody friction factor, 139, 164
- Muller’s method, 161
- Multidimensional interpolation
  - bilinear interpolation, 437–439
  - in Python, 439
- Multidimensional optimization, 187–189
- Multimodal, 178
- Multiple integrals, 477–479
  - double integral, 477–478
- Multiple linear regression, 304
- Multiple root, 121
- multiply function, 31
- Multistep methods, 533, 572–575
  - error estimation, 574–575
  - non-self-starting Heun method, 572–574

## N

- Naive Gauss elimination, 224–230

- back substitution, 226
  - forward elimination of unknowns, 225–226
  - operation counting, 228–230
- Napier's constant, 23n3
- Natural cubic splines, 430–431
- Nearest neighbor interpolation, 436
- Nesting, 66–68
- Neumann (derivative) boundary condition, 597, 603
- Newton-Cotes formulas, 456, 461–462
  - closed integration, 472–473
  - higher-order, 472–473
  - open integration, 477
- Newtonian fluid, 164n4
- Newton interpolating polynomial, 305, 399
  - general form of, 402–405
  - linear interpolation, 400–401
  - quadratic interpolation, 401–402
- Newton linear-interpolation formula, 399
- Newton-Raphson method, 149–154, 273–277
  - applied to bungee jumper problem, 154
  - formula, 149
  - near-zero slope, 151
  - for nonlinear system, 273–277
  - poor convergence, 152
  - slowly converging function with, 151
- Newton's second law, 3, 4, 9, 200, 375, 508
- Newton's viscosity law, 509
- Nongradient method, 188
- Nonlinear ODEs
  - finite-difference approaches for, 605–607
  - shooting method for, 599–601
- Nonlinear regression, 305, 364–368
  - of Antoine equation, 364–368
  - fitting experimental data, 368–371
- Nonlinear relationships, linearization of, 330–335
  - exponential equation, 330–331
  - fitting data with power equation, 332–335
  - power equation, 330–331
  - saturation-growth-rate equation, 330–331
- Nonlinear simultaneous equations, 199
- Nonlinear systems, 271–278
- Newton-Raphson method, 273–277
  - simultaneous nonlinear equations, 271
  - successive substitution, 271–273
- Non-self-starting Heun method, 572–574

## Norm

- column-sum, 258
- defined, 257
- Euclidean, 257
- Frobenius, 257
- matrix, 257–258
- with Python, 259–260
- row-sum, 258
- spectral, 258
- vector, 257–258

## Normal distribution, 311–314

- random numbers from, 319–320

## Normal equations, 323, 350, 379

## Normalization, 92, 94, 225

## Normal probability rule, 312

## “Not-a-knot” end condition, 431–432

## $n$ th-order Taylor series expansion, 100

## Numerical differentiation, 12, 103–107, 456, 507–509

- derivatives and integrals for data with errors, 514–515
- in engineering and science, 508–509
- error analysis of, 108–110
- finite-difference approximation. *See* Finite-difference approximation
- free-falling bungee jumper, 506
- high-accuracy differentiation formulas, 509–512
- partial, 515–516
- Richardson extrapolation, 512–513
- roundoff errors in, 108–110
- truncation errors in, 108–110
- of unequally spaced data, 513–514
- visualizations of vector fields, 520–522

## Numerical errors, 86

- control of, 110

## Numerical integration, 12

- to compute cumulative normal probability, 475–476
- computing work with, 479–482
- free-falling bungee jumper, 457–458
- multiple integrals, 477–479

## Newton-Cotes formulas, 461–462

- adaptive quadrature, 496–499
- Boole’s rule, 472–473, 497, 547
- Gauss quadrature, 491–496
- open methods, 477
- Romberg integration, 487–491
- root-mean-square current, 499–502
- Simpson’s rules, 467–472

- trapezoidal rule, 462–467
- Numerical methods, 115
  - defined, 1, 6
  - iterative methods, 88
  - polynomial roots, 161
  - reasons to study, 1–2
- NumPy module, 24–25, 219
  - to manipulate polynomials, 162–164
- Nyquist frequency, 387

## O

- Occam's/Ockham's Razor, 352n1
- Offset-zero format, 94
- Ohm's law, 212, 214, 500, 509
- One-dimensional array, 24
- One-dimensional optimization, 177–187
  - golden-section search, 179–184
  - parabolic interpolation, 184–186
- One-line `if` style, 57
- One-step methods, 530, 533, 562
- Open integration formulas, 456, 462
  - Newton-Cotes, 477
- Open methods, 122, 141
- Operation counting, 228–230
- Operators
  - logical, 57
  - mathematical, 28–29
  - precedence, 29
  - relational, 57
- Optimization, 10, 115, 175
  - defined, 176
  - elevation as function of time, 176
  - mathematical perspective, 176
  - multidimensional. *See* Multidimensional optimization
  - one-dimensional. *See* One-dimensional optimization
- Ordinary differential equations (ODE), 12, 527
  - boundary-value problem, 530
  - dependent variable, 527
  - finite-difference approaches for, 605–607
  - first-order, 529
  - independent variable, 527
  - initial-value problem, 530
  - linear, shooting method for, 596–597
  - nonlinear, shooting method for, 599–601

- parameters to control integration options, 568–569
- stiff, 531
- Ordinary frequency, 376
- Oscillations, 411–413
- Outliers, 309
- Overdetermined system, 211
- Overfitting concept, 352
- Overflow error, 93
- Overrelaxation, 270
- Oxygen sag, 193

## P

- Parabolic interpolation, 184–186
- Parameters, 4
  - passing, 72–74
- Parsimony concept, 352, 411
- Partial derivatives, 508, 515–516
- Partial pivoting, 199, 230–231
- Partition of the sum of squares, 325
- Pascal (Pa), 164n6
- Percent relative error, 86
- Periodic function, 376
- Permutation matrix, 205
- Per-step truncation error estimation, 575
- Phase angle/shift, 376–377
- Phase-plane plots, 553
- pi ( $\pi$ ), 23
- Piecewise interpolation, 432–437
- Pivot equation, 225
- Pivoting, 230–233
  - complete, 230
  - determinant evaluation with Gauss elimination, 232–233
  - LU* Factorization with, 247–248
  - partial, 230–231
- Place value, 91
- Planck's constant, 23, 93
- Pliny's intermittent fountain, 582–585
- Point-slope method. *See* Euler's method
- Poiseuille equation, 239
- Polar coordinates, 629
- Pollock, D. S. G., 635
- Polynomial(s), 161–164
  - companion matrix, 162
  - Muller's method, 161

- NumPy module to manipulate, 162–164
- regression, 304, 349–352
- of viscosity of water, 360–364
- Polynomial coefficients, determination of, 397–399
- Polynomial interpolation, 397
  - extrapolation, 409–411
  - inverse interpolation, 408–409
  - Lagrange interpolating polynomial, 405–408
  - Newton interpolating polynomial, 399–405
  - oscillations, 411–413
- Population, 309
- Positional arguments, 61
- Positional notation, 91
- Posttest loop, 65
- Potential energy, 189–191
- Power equation, 330–331
  - fitting data with, 332–335
- Power method, 293–295
- Power spectrum, 390–391
  - computation with Python, 390–391
- Preallocation of memory, 64
- Precision, 85, 95
- Predator-prey models, 552–556, 566
- Prediction error sum of squares (PRESS), 359–360
- Predictor-corrector approach, 540
- Predictor equation, 539
- Pretest loop, 65
- Principal diagonal, of matrix, 202
- Principle of mass conservation, 197
- Programming, in Python
  - indentation, 66–68
  - input-output, 53–56
  - nesting, 66–68
  - script files, 49–53
- Propagated truncation error, 535
- Proportionality, 255
- Pure oscillations
  - ordinary differential equations with, eigenvalues and, 289–291
- Python
  - assignment, 22
  - built-in functions, 32–35
  - built-in keywords, 639–640
  - collection types, 23–24
  - compute DFT of simple sinusoid with, 388–390
  - computing power spectrum with, 390–391

- descriptive statistics using, 314–316
- eigenvalues and eigenvectors with, 295–296
- exploratory data analysis, 41–44
- input–output, 53–56
- LU* factorization with, 248–249
- mathematical operations, 28–32
- multidimensional interpolation in, 439
- nonlinear regression of Antoine Equation with, 364–368
- norms and condition number with, 259–260
- numerical differentiation with, 516–520
- ODE solvers for stiff systems, 578–580
- piecewise interpolation in, 432–437
- relational operators in, 57
- resources, 40–41
- scripts, 49–50
- solving linear algebraic equations with, 211–212

Spyder/Python development environment, 2, 21–22

## Q

- Quadratic convergence, 150
- Quadratic interpolation, 401–402
- Quadratic splines, 425–427
- Quadrature, 458

## R

- Ralston’s method, 545
- Random numbers
  - generation, 304
  - from normal distribution, 319–320
  - and simulation, 316–320
  - from uniform distribution, 316–318
- Range, 95, 309
- Range type, 61–62
- Rate equations, 527–528
- Rate of change of velocity, 3, 7
- Rayleigh, Lord, 13
- Refractive index of borosilicate glass, 357–358
- Regression, 11, 12
  - model building and selection in, 359–364
  - multiple linear, 353–355
  - nonlinear, 305, 364–368
  - polynomial, 349–352
- Regression sum of squares, 325
- Relational operators, 57



- Relative fractional error, 86
- Relaxation, 270–271
- Residual, 321
- Reverse-wraparound order, 388
- Reynolds number, 13, 165
- Richardson extrapolation, 456, 486–488, 497, 512–513
  - higher-order corrections, 488
- RKF methods, 562–563. *See also* Runge-Kutta-Fehlberg (RKF)
- RK methods. *See* Runge-Kutta (RK) methods
- Romberg integration, 456, 486, 487–491
- Root location, 10
- Roots, 141
  - bisection method, 125–131
  - bracketing methods, 122–125
  - Brent’s method, 156–160
  - incremental search, 122–125
  - initial guesses, 122–125
  - multiple, 121
  - Newton-Raphson method, 149–154
  - polynomials, 161–164
  - roots function, 162–164, 277–278
  - secant methods, 155–156
  - trial-and-error method, 119–120
  - Wegstein method, 146–149
- Roundoff errors, 2, 90–97, 535. *See also* Truncation errors
  - arithmetic manipulations of computer numbers, 96–97
  - computer number representation, 90–95
  - in numerical differentiation, 108–110
- Row, matrix, 201
- Row-sum norm, 258
- Row vectors, 201
- R-squared, 327
- Runge, Carl David Tolmé, 411n2
- Runge-Kutta-Fehlberg (RKF)
  - 4/5 algorithm, 563–566
- Runge-Kutta (RK) methods, 530, 533, 539, 543–547
  - classical fourth-order, 545–547
  - embedded, 562
  - fifth-order, 546–547
  - increment function, 543
  - second-order, 544–545
  - systems of equations, 549–550
- Runge’s function, 411, 434–435
  - application of PChipInterpolator function to, 436–437
  - comparison of, 412–413

Runge's phenomenon, 411

## S

Sample average, 309

Sample standard deviation, 309

Sample statistics, 309

Sample variance, 309

Saturation-growth-rate equation, 330–331. *See also* Michaelis-Menten model

Scalars, 22–23

SciPy module, 23

Script file, 49–50

Secant methods, 155–156

Second-order differential equations, 287

Second-order equation, 528

Second-order Lagrange interpolating polynomial, 406

Second-order polynomial, 401

Second-order Runge-Kutta methods, 544–545

Semicolons (;), 57

Sequential search, 424

Set, collection type, 24

Shooting method, 595–601

- and derivative boundary conditions, 596–598

- for linear ODE, 596–597

- for nonlinear ODEs, 599–601

- trial-and-error approach, 595

Sigmoidal, 340

Sign bit, 91

Significand, 92

Sigsum function, 532–533, 532n1

Simple plot, creation of, 614–618

Simpson's rules, 467–472, 490

- Simpson's 1/3 rule, 456, 468, 496, 500

- composite, 469–471, 486, 496–497

- single application of, 469

- Simpson's 3/8 rule, 456, 471–472

Simulation, random numbers and, 316–320

Simultaneous equations,

- determining polynomial coefficients with, 398

Singular systems, 220–221

Sinusoid, 376

- curve fitting with sinusoidal functions, 376–380

- least-squares fit of, 378–380

Smearing, 97

Smoothing of data series. *See* Data smoothing

Soave-Redlich-Kwong (SRK) equation of state, 169  
 SOR. *See* Successive overrelaxation (SOR)  
 Spectral norm, 258  
 Spline interpolation, 305  
 Splines cubic, 418, 428–432
 

- drafting technique, 419
- functions, 418
- heat transfer in lake, 447–450
- linear, 420–425
- notation used to derive, 420
- overview of, 418–420
- quadratic, 425–427
- using Python’s built-in functions, 434–435

 Spyder editor, 2  
 Spyder Help menu, 32  
 Spyder/Python development environment, 2, 21–22  
 Square matrices, 202  
 Stability, 576–578  
 Standard deviation, 309  
 Standard error of estimate, 322, 326  
 Static variables, 583n1  
 Statistical plots, 624
 

- box plots, 626–627
- error bars, 625
- histograms, 625–626
- line/marker plots, 625

 Statistics
 

- descriptive, 307–311
- review, 307–314
- sample, 309

 Steady-state calculation, 9–10  
 Steepest descent, 188  
 Stem plots, 627  
 Step halving approach, 562  
 Stepwise regression, 359  
 Stiffness, 575–580
 

- Euler’s method, 576
- explicit and implicit Euler method, 577–578
- Python ODE solvers for, 578–580

 Stiff ordinary differential equations, 531  
 Stiff system, 575  
 Stimulus-response computations, 255–256  
 Stokes drag, 13  
 Stopping criterion, 87  
 Straight-line least squares regression, 320–330

- criteria for “best” fit, 321–322
  - “desert island” method, 324–325
  - estimation of errors for, 327–330
  - least-squares fit of straight line, 322–323
  - quantification of error in linear regression, 325–330
- Straight-line linear regression, 323–324
  - enzyme kinetics, 339–342
  - experimental data for force and velocity, 306–307
  - fluid mechanics, 306–307
  - general comments on, 335
  - linearization of nonlinear relationships, 330–335
  - normal distribution, 311–314
  - statistics, descriptive, 307–311
  - wind tunnel experiment, 306–307
- Strange attractors, 556
- Streeter-Phelps model, 193
- String, 27
  - functions and methods, 28
- Structured programming, 56–66
  - decisions, 56–61
  - loops, 61–66
- Styles, line, 618
- Subscript/subscript ranges, 26
- Subtractive cancellation, 96, 107
- Successive overrelaxation (SOR), 270
- Successive substitution, 142. *See also* Fixed-point iteration
- Sunspots, 391–393
- Superposition, 255
- Surface plots, 631–634
- Symmetric matrix, 202
- Systems of equations, 547
  - Euler’s method, 547–548
  - Runge-Kutta methods, 549–550

## T

- Table lookup, 422–425
- Taylor, Brook, 98n4
- Taylor series, 98–101, 509–510
  - approximation of function with, 100–101
  - to estimate truncation errors, 103
  - expansion, 273, 536
  - first-order expansion, 98–99, 273
  - multivariable, 273
  - $n$ th-order expansion, 100

- remainder for, 101–103
- zero-order expansion, 98–99
- Taylor’s theorem, 98
- Terminal velocity, 6, 20–21
- Thermocline plane, 447
- Third-order polynomial, 428
- Three-point Gauss-Legendre formula, 495
- Time domain, 383–384
- Time series, 375
- Time-variable computation, 9
- Torricelli’s law, 582
- Total corrected sum of squares, 309
- Total numerical error, 107–110
  - control of numerical errors, 110
  - error analysis of numerical differentiation, 108–110
  - tplquad function, 479
- Transfer functions, 171
- Transient computation, 9
- Transpose, of matrix, 31, 204–205
- Transposition matrix, 205
- Trapezoidal rule, 456, 462–467, 490, 492, 500, 542–543, 573
  - composite, 464–466
  - defined, 462
  - error of, 463–464
  - integral evaluation by, 493
  - single application of, 463–464
  - with unequal segments, 473
- Trend analysis, 303
- Trial-and-error method, 115, 119–120, 595
- Tri-cube function, 444
- Tridiagonal matrices, 199
- Tridiagonal matrix, 203
- Tridiagonal systems, 233–235
  - solution of, 233–234
  - splines, 429–430, 440
- Trigonometric functions, 33
- Truncation errors, 2, 97–107, 535. *See also* Roundoff errors
  - defined, 97
  - estimation, 103
  - numerical differentiation, 103–107, 108–110
  - per-step, 575
  - Taylor series. *See* Taylor series
- Tukey, J. W., 388
- Tuple, collection type, 24
- Two-dimensional array, 24

Two-dimensional interpolation, 437–438  
Two-dimensional optimization, 177–178  
Two-point Gauss-Legendre formula, 493–495, 501  
2's complement, 91

## U

Uncertainty, 85  
Unconditionally stable approach, 577  
Underdetermined system, 211  
Underrelaxation, 270  
Undetermined coefficients method, 491–493  
Unequally spaced data, derivatives of, 513–514  
Uniform distribution, random numbers from, 316–318  
Unimodal, 180  
Unit imaginary number, 22  
Upper triangular matrix, 203, 247  
Upper triangular system, 225–226

## V

Vandermonde matrices, 398  
Van der Pol equation, 578–580  
Variables  
    Boolean/logical, 582  
    dependent, 4  
    independent, 4  
    scope, 52–53  
    static, 583n1  
Variance, 309  
Vectorization, 34–35, 63–64  
Vectors, 23–26  
    norms, 257–258  
    vector fields, visualizations of, 520–522  
Visual Basic (VBA), 66n5  
Voltages, in circuits, 212–215  
Volterra, Vito, 552  
Volume integral, 460

## W

Wegstein, J. H., 146n1  
Wegstein method, 146–149  
while . . . break structure, 64–65  
Wölfer Sunspot Numbers, 391n3

## Y

Young's modulus, 192

## Z

Zero-order expansion, 98–99

Zeros, of equation, 11

DIGITAL EDITION



501-1111

# Figure 1.3 Text Alternative (Chapter 1)

---

[!\[\]\(d263118e0bfd47dc6bc704167d936b83\_img.jpg\) Return to Figure 1.3](#)

True slope ( $dv$  over  $dt$ ) increases through  $(t_i, v(t_i))$  and approximate slope increases through  $(t_{i+1}, V(t_{i+1}))$ .

[!\[\]\(3d8c13c92b853674f749aac6fa869926\_img.jpg\) Return to Figure 1.3](#)



# Figure 1.4 Text Alternative (Chapter 1)

---

 [Return to Figure 1.4](#)

The horizontal axis represents time (in seconds) ranging from 0 to 12 in increments of 4, while vertical axis represents velocity (in meter per second) 0 to 60 in increments of 20.

The data is as follows:

Approximate numeral solution curve increases from (0, 0) to (12, 52) (where the level is labeled as terminal velocity) through (2, 20), (4, 38), (6, 48), (8, 50), and (10, 52).

Exact analytical solution curve increases from (0, 0) to (12, 52) drawn slightly under approximate numerical solution curve.

Note: All data is approximate.

 [Return to Figure 1.4](#)

# Figure 1.5 Text Alternative (Chapter 1)

---

[!\[\]\(529949c2c3dadbaa4e538e8c643454bc\_img.jpg\) Return to Figure 1.5](#)

Horizontal segment: Pipe 1 (left) flows in at 100 while pipe 4 (right) flows out at unknown value.

Vertical segment: Pipe 2 (top) flows in at 80 while pipe 3 (bottom) flows out at 120.

[!\[\]\(0f848bbd71cef6b345273b16f905912a\_img.jpg\) Return to Figure 1.5](#)

# Set of Four Cylindrical Reactors Connected Graphic Text Alternative (Chapter 1)

---

 [Return to Set of Four Cylindrical Reactors Connected Graphic](#)

An unlabeled input flows into the first reactor yielding an output that flows into the second reactor below it. The unlabeled output (as an input) flows into the second reactor yielding an output that flows into the third reactor below it. The unlabeled output (as an input) flows into the third reactor yielding an output that flows out. A fourth reactor is placed to the left of the third reactor, where an unlabeled input flows into the fourth reactor yielding an output that flows into the third reactor as an input.

 [Return to Set of Four Cylindrical Reactors Connected Graphic](#)

# Figure 1.6 Text Alternative (Chapter 1)

---

 [Return to Figure 1.6](#)

Roots and optimization:

Roots solve for  $x$  so that  $f$  is zero and optimization problems solve for  $x$  such that the derivative of  $f$  at  $x$  is 0.

Linear algebraic equations:

It is given a set of constant coefficient linear algebra equations, solve for the independent variables.

Curve fitting:

Both using regression to get the best-fit coefficients for a model or using interpolation to connect data points.

Integration and differentiation:

Either finding the area under a curve or the slope along a curve.

Differential equations:

Given an expression for a first-order differential equation as a function of the dependent variable and time, solve the variable for all times.

 [Return to Figure 1.6](#)

# Figure 1.7 Text Alternative (Chapter 1)

---

 [Return to Figure 1.7](#)

A) Velocity:

The horizontal axis represents time (in seconds) ranging from 0 to 12 in increments of 4, while vertical axis represents velocity (in meter per second) negative 40 to 60 in increments of 20.

The data is as follows:

Correct drag:

The curve increases from (0, negative 40) to (12, 50) through (2, negative 9), (4, 11), (6, 31), (8, 43), (10, 49) and (12, 50).

Incorrect drag:

The curve increases from (0, negative 40) to (12, 42) through (2, negative 35), (4, negative 22), (6, 3), (8, 17), (10, 31) and (12, 42).

A) Height:

The horizontal axis represents time (in seconds) ranging from 0 to 12 in increments of 4, while vertical axis represents velocity (in meter per second) negative 200 to 200 in increments of 100.

The data is as follows:

Correct drag:

The curve decreases from (0, 0) to (12, negative 170), through (2, 75), (4, 90), (6, 70), (8, 20), (10, negative 60) and (12, negative 170).

Incorrect drag:

The curve increases from (0, 0) to (12, 100), through (2, 75), (4, 150), (6, 170), (8, 190), and (10, 155).

Note: All data is approximate.

 [Return to Figure 1.7](#)

# Figure P1.9 Text Alternative (Chapter 1)

---

 [Return to Figure P1.9](#)

The top layer of fluid is marked as 0, with vertical axis labeled as  $y$  and a dashed horizontal axis. The flow of fluid is shown coming in from inlet at the base on the left and flowing out from an outlet on the base towards right.

 [Return to Figure P1.9](#)

# Figure P1.11 Text Alternative (Chapter 1)

---

 [Return to Figure P1.11](#)

Liquid flows in from  $Q_{in}$ , and flows out from  $Q_{out}$ . The slope of the side of the cone is given as  $s$ , while the radius of the top of the cone is  $r_{top}$ .

The height till  $r_{top}$  is  $y_{top}$ , height till liquid is filled is  $y$ , and height till  $Q_{out}$  is  $y_{out}$ . The heights  $y_{top}$ ,  $y$ , and  $y_{out}$  arise from 0 (at the base tip of conical tank).

 [Return to Figure P1.11](#)

# Figure P1.13 Text Alternative (Chapter 1)

---

 [Return to Figure P1.13](#)

Food and drink add water from the outside; urine, skin, feces, and sweat lose water to the outside; air can both add from and remove water to the outside; and the metabolism adds water from the inside.

 [Return to Figure P1.13](#)



# Figure P1.16 Text Alternative (Chapter 1)

---

 [Return to Figure P1.16](#)

The network is built as follows:

A pipe section labeled Q 10, a pump, and a section labeled Q 1 are all connected in series with the directions of Q 1 and Q 10 given as the same. There are six different junctions.

The top left junction has Q 1 entering and Q 2 and Q 3 leaving. The top middle junction has Q 3 entering and Q 4 and Q 5 leaving.

The top right junction has Q 5 entering and Q 6 and Q 7 leaving. The bottom right junction has Q 6 and Q 7 entering and Q 8 leaving. The bottom middle junction has Q 4 and Q 8 entering and Q 9 leaving. The bottom right junction has Q 2 and Q 9 entering and Q 10 leaving.

 [Return to Figure P1.16](#)

# Figure P1.23 Text Alternative (Chapter 1)

---

 [Return to Figure P1.23](#)

The beam starts at  $x$  equals 0 and ends at  $x$  equals  $L$ . The distributed load  $w$  is applied across the whole length of the beam. The beam is shown deflected and appears slightly concave-down.

 [Return to Figure P1.23](#)

# Figure P1.25 Text Alternative (Chapter 1)

---

 [Return to Figure P1.25](#)

The total height of the cone is  $H$ , while the height of the cone above the surface is  $h_1$  while the height of the buried section is  $h_2$ . The radius at the base of the cone is  $r_2$  while the cross-sectional radius at the basalt surface is  $r_1$ .

 [Return to Figure P1.25](#)

# Figure P1.26 Text Alternative (Chapter 1)

---

 [Return to Figure P1.26](#)

The voltage drop across the resistor is the current  $i$  times the resistance  $R$ , the voltage drop across the inductor is the inductance  $L$  times the derivative of the current. The voltage across the capacitor is given as the charge on the capacitor  $q$  divided by the capacitance  $C$ .

 [Return to Figure P1.26](#)

# Screengrab of Editor Window Shows Code Graphic Text Alternative (Chapter 2)

---

 [Return to Screengrab of Editor Window Shows Code Graphic](#)

Line 1: # -\*- coding: utf-8 -\*-

Line 2: """

Line 3: Spyder Editor

Line 4:

Line 5: This is a temporary script file.

Line 6: """

Line 7:

Line 8:

 [Return to Screengrab of Editor Window Shows Code Graphic](#)

# Plot of Time (in Seconds) Against Velocity Graphic Text Alternative (Chapter 2)

---

 [Return to Plot of Time \(in Seconds\) Against Velocity Graphic](#)

The horizontal axis represents time (in seconds) ranging from 0.0 to 20.0 in increments of 2.5, while vertical axis represents velocity (in meter per second) 0 to 50 in increments of 10.

The data is as follows:

The curve increases from (0, 0) to (20.0, 52), through (2.5, 23), (5.0, 37), and (10.0, 50).

Note: All data is approximate.

 [Return to Plot of Time \(in Seconds\) Against Velocity Graphic](#)

# Plot of Time (in Seconds) Against Velocity Graphic Text Alternative (Chapter 2)

---

## [Return to Plot of Time \(in Seconds\) Against Velocity Graphic](#)

The horizontal axis represents time (in seconds) ranging from 0.0 to 20.0 in increments of 2.5, while vertical axis represents velocity (in meter per second) 0 to 50 in increments of 10.

The data is as follows:

The dots increase from (0, 0) to (20.0, 52), through (2.4, 19), (3.5, 33), (6.0, 42), (7.7, 47), (10.0, 50), (12.0, 51), (13.5, 52), (16.0, 53), and (18.0, 53).

Note: All data is approximate.

## [Return to Plot of Time \(in Seconds\) Against Velocity Graphic](#)

# Plot of Time (in Seconds) Against Velocity Graphic Text Alternative (Chapter 2)

---

[!\[\]\(d263118e0bfd47dc6bc704167d936b83\_img.jpg\) Return to Plot of Time \(in Seconds\) Against Velocity Graphic](#)

The horizontal axis represents time (in seconds) ranging from 0.0 to 2.0 in increments of 2.5, while vertical axis represents velocity (in meter per second) 0 to 50 in increments of 10.

The data is as follows:

The curve with dots increase from (0, 0) to (20.0, 52), through (2.4, 19), (3.5, 33), (6.0, 42), (7.7, 47), (10.0, 50), (12.0, 51), (13.5, 52), (16.0, 53), and (18.0, 53).

Note: All data is approximate.

[!\[\]\(fa6f3af6bfa46c5d4a2d362681095beb\_img.jpg\) Return to Plot of Time \(in Seconds\) Against Velocity Graphic](#)



# Plot of Time (in Seconds) Against Velocity Graphic Text Alternative (Chapter 2)

---

[!\[\]\(c8d96c8885d3000a912c2582004aed63\_img.jpg\) Return to Plot of Time \(in Seconds\) Against Velocity Graphic](#)

The horizontal axis represents time (in seconds) ranging from 0.0 to 2.0 in increments of 2.5, while vertical axis represents velocity (in meter per second) 0 to 50 in increments of 10.

The data is as follows:

The dashed curve with dots increase from (0, 0) to (20.0, 52), through (2.4, 19), (3.5, 33), (6.0, 42), (7.7, 47), (10.0, 50), (12.0, 51), (13.5, 52), (16.0, 53), and (18.0, 53).

Note: All data is approximate.

[!\[\]\(e3f8612927870f2e0f9f5989e6dd3064\_img.jpg\) Return to Plot of Time \(in Seconds\) Against Velocity Graphic](#)

# Plot of Time (in Seconds) Against Velocity Graphic Text Alternative (Chapter 2)

---

## [Return to Plot of Time \(in Seconds\) Against Velocity Graphic](#)

The horizontal axis represents time (in seconds) ranging from 0.0 to 20.0 in increments of 2.5, while vertical axis represents velocity (in meter per second) 0 to 50 in increments of 10.

The data is as follows:

CD equals 0.25:

The curve with dots increase from (0, 0) to (20.0, 52), through (2.4, 19), (3.5, 33), (6.0, 42), (7.7, 47), (10.0, 50), (12.0, 51), (13.5, 52), (16.0, 53), and (18.0, 53).

CD equals 0.30:

The curve with dots increase from (0, 0) to (20.0, 47), through (2.4, 19), (3.5, 32), (6.0, 42), (7.7, 44), (10.0, 46), (12.0, 46.5), (13.5, 46.6), (16.0, 47), and (18.0, 47).

Note: All data is approximate.

## [Return to Plot of Time \(in Seconds\) Against Velocity Graphic](#)

# Histogram Plot Shows Merged Vertical Bars Graphic Text Alternative (Chapter 2)

---

 [Return to Histogram Plot Shows Merged Vertical Bars Graphic](#)

The horizontal axis ranges from 0 to 160, while vertical axis represents velocity (in meter per second) 0 to 160 in increments of 20.

The data is as follows:

0 to 60: 0, 1, 6.

60 to 80: 5, 18, 23, 58.

80 to 100: 69, 115, 119.

100 to 120: 158, 149, 110, 80, 49.

120 to 140: 38, 17, 13, 3, 0, 1, 0.

Note: All data is approximate.

 [Return to Histogram Plot Shows Merged Vertical Bars Graphic](#)

# Plot of Measured Against Predicted Terminal Velocities Graphic Text Alternative (Chapter 2)

---

 [Return to Plot of Measured Against Predicted Terminal Velocities Graphic](#)

The horizontal axis represents measured terminal velocities ranging from 48 to 56 in increments of 2, while vertical axis represents predicted terminal velocities ranging from 46 to 56 in increments of 2.

The data is as follows:

A dashed line increases from (48, 48) to (56, 56) through (50, 50), (52, 52), and (54, 54).

Multiple circles are marked at (47.75, 47.5), (48.5, 45.5), (50.8, 50), (51, 52.5), (53.5, 53.5), (54, 56.25), and (55.75, 56).

Note: All data is approximate.

 [Return to Plot of Measured Against Predicted Terminal Velocities Graphic](#)

# Plot of Drag Coefficient Against Mass Shows 7 Graphic Text Alternative (Chapter 2)

---

[!\[\]\(8af806fb1314382d09bc5ec5b767526c\_img.jpg\) Return to Plot of Drag Coefficient Against Mass Shows 7 Graphic](#)

The horizontal axis represents measured terminal velocities ranging from 60 to 90 in increments of 5, while vertical axis represents predicted terminal velocities ranging from 0.25 to 0.31 in increments of 0.01.

The data is as follows:

Multiple circles are marked at (60, 0.25), (66, 0.281), (72, 0.273), (81, 0.304), (83.5, 0.288), (92, 0.289), and (97, 0.33)

Note: All data is approximate.

[!\[\]\(0aff635c4179ba9e710b00f4b01d3b20\_img.jpg\) Return to Plot of Drag Coefficient Against Mass Shows 7 Graphic](#)

## Figure P2.11 Text Alternative (Chapter 2)

---

 [Return to Figure P2.11](#)

When the switch is at its left position, a capacitor is connected in parallel to a battery. When the switch is moved to its right position, the capacitor is disconnected from the battery and instead connected to a series combination of a resistor and an inductor

 [Return to Figure P2.11](#)

## Figure P2.26 Text Alternative (Chapter 2)

---

 [Return to Figure P2.26](#)

a) A beam is supported by a roller at its left end; it is attached to a wall at its right. It undergoes a distributed load that linearly increases from 0 at the left end to a maximum value of  $w_0$  at the right end.

b) The displacement graph shows a nonzero slope at the left end, a maximum negative displacement about one-third from the left end, and a 0 displacement and slope at the right end.

 [Return to Figure P2.26](#)

# A Line Graph Shows an Irregularly Concave Down Graphic Text Alternative (Chapter 3)

---

[!\[\]\(d263118e0bfd47dc6bc704167d936b83\_img.jpg\) Return to A Line Graph Shows an Irregularly Concave Down Graphic](#)

The horizontal axis ranges from 0 to 80 in increments of 20, while vertical axis on left ranges from negative 0.6 to 0.4 in increments of 0.2, and vertical axis on right ranges from 52.00 to 53.75 in increments of 0.25.

The data is as follows:

Solid curve: It begins at (0, negative 0.13) and ends at (86.5, 0.6) showing decreasing trend from (60, 0.5).

Dashed curve: It begins at (0, 0.5) and ends at (86.5, 52.00) showing decreasing trend from (20, 53.50).

Note: All data is approximate.

[!\[\]\(17acf1afa8cdf0b67c53d4865a5ed469\_img.jpg\) Return to A Line Graph Shows an Irregularly Concave Down Graphic](#)



# Figure 3.1 Text Alternative (Chapter 3)

---

 [Return to Figure 3.1](#)

a asterisk b greater than 0 and b equals equals 2 and x greater than 7 or not y greater than 'd'. A downward arrow is visible below asterisk, b, x, and y.

Substituting constants result in:

negative 1 asterisk 2 greater than 0 and 2 equals equals 2 and 1 greater than 7 or not 'b' greater than 'd'. A downward arrow is visible below asterisk.

Evaluating arithmetic expressions result in:

negative 2 greater than 0 and 2 equals equals 2 and 1 greater than 7 or not 'b' greater than 'd'. A downward arrow is visible below greater than, equals equals, greater than, and greater than.

Evaluating relational expressions result in:

F and T and F or not F. A downward arrow is visible below combination of not and F.

Evaluating compound logical expressions result in:

F and T and F or T, where combination of F and T shows a downward arrow below. This results in F and F or T, where combination of F and F show a downward arrow below. This leads to F or T leading to T in a box.

 [Return to Figure 3.1](#)

# A Line Graph Shows a Regular Concave Down Curve Graphic Text Alternative (Chapter 3)

---

 [Return to A Line Graph Shows a Regular Concave Down Curve Graphic](#)

The horizontal axis ranges from 0 to 12 in increments of 2, while vertical axis ranges from 0 to 50 in increments of 10.

The data is as follows:

Curve begins at (0, 0) and ends at (12, 51), through (2, 20), (4, 34), and (8, 47).

Note: All data is approximate.

 [Return to A Line Graph Shows a Regular Concave Down Curve Graphic](#)

# A Graph is Visible Across First Quadrant While Graphic Text Alternative (Chapter 3)

---

[!\[\]\(1d3a1175dd4902218e694b9c098adb83\_img.jpg\) Return to A Graph is Visible Across First Quadrant While Graphic](#)

The horizontal axis ranges from 0 to  $x$ , while vertical axis ranges from 0 to  $y$ .

The data is as follows:

Curve  $r$  making angle  $\theta$  with horizontal axis increases from  $(0, 0)$  to  $(x, y)$  in the first quadrant.

[!\[\]\(a03a7eb2f4046e1d3c76772003e549ea\_img.jpg\) Return to A Graph is Visible Across First Quadrant While Graphic](#)

## Figure P3.10 Text Alternative (Chapter 3)

---

 [Return to Figure P3.10](#)

There is a distributed load of 20 kips per foot down that extends from the far left of the beam to the 5 foot point. There is an applied torque of 150 kip feet applied clockwise at the 7 foot point. There is a concentrated load of 15 kips applied upwards at the 8 foot point.

 [Return to Figure P3.10](#)

# Figure 4.1 Text Alternative (Chapter 4)

---

 [Return to Figure 4.1](#)

- A) Four concentric circles, where dots are scattered loosely across the top left over all four circles.
- B) Four concentric circles, where dots are scattered loosely across the center over first three circles.
- C) Four concentric circles, where dots are scattered closely across the top left between second and fourth circle.
- D) Four concentric circles, where dots are scattered closely across the center over first and second circle.

 [Return to Figure 4.1](#)

## Figure 4.3 Text Alternative (Chapter 4)

---

 [Return to Figure 4.3](#)

The top of the circle shows 11111111, 00000000, and 00000001 across it while the bottom of the circle shows 10000001, 1000000, and 01111111 across it.

The data for decimal leading to binary is as follows:

0: 00000000

1: 00000001

2: 00000010

3: 00000011

4: 00000100

127: 01111111

Negative 128: 10000000

Negative 127: 10000001

Negative 2: 11111110

Negative 1: 11111111.

 [Return to Figure 4.3](#)

## Figure 4.4 Text Alternative (Chapter 4)

---

 [Return to Figure 4.4](#)

The overflow with negative 9.9 of 10 raised to power 9 is labeled as maximum, underflow (hole at zero) between negative 1.0 of 10 raised to power negative 9 and 1.0 of 10 raised to power negative 9 is labeled as smallest, and overflow with 9.9 of 10 raised to power 9 is labeled as maximum.

 [Return to Figure 4.4](#)

# Chart Shows the Relative Values for Sign, Exponent Graphic Text Alternative (Chapter 4)

---

 [Return to Chart Shows the Relative Values for Sign, Exponent Graphic](#)

The data for sign is as follows:

0: 0

The data for exponent is as follows:

1: 1

2: 1

3: 1

4: 1

5: 1

6: 1

7: 1

8: 1

9: 1

10: 1

11: 0

The data for mantissa is as follows:

12: 1

13: 1

14: 1

15: 1

16: 1

17: 1

18: 1

19: 1

20: 1

21: 1



22: 1

23: 1

24: 1

25: 1

26: 1

27: 1

28: 1

29: 1

30: 1

31: 1

32: 1

33: 1

34: 1

35: 1

36: 1

37: 1

38: 1

39: 1

40: 1

41: 1

42: 1

43: 1

44: 1

45: 1

46: 1

47: 1

48: 1

49: 1

50: 1

51: 1

52: 1

53: 1

54: 1

55: 1

56: 1

57: 1

58: 1

59: 1

60: 1

61: 1

62: 1

63: 1.

 [Return to Chart Shows the Relative Values for Sign, Exponent Graphic](#)

## Figure 4.7 Text Alternative (Chapter 4)

---

 [Return to Figure 4.7](#)

The horizontal axis represents  $x$  ranging from 0,  $x_1$  equals 0 through  $x_{i+1}$  equals 1, while vertical axis represents  $f$  of  $x$  ranging from 0 to 1.0.

The zero order approximation remains constant from  $f$  of  $x_i$  to  $f$  of  $x_{i+1}$  approximately equal to  $f$  of  $x_i$ . The first order approximation follows the tangent to the curve at the initial value to calculate the next value and ranges from  $f$  of  $x_i$  to  $f$  of  $x_{i+1}$ , approximately equal to  $f$  of  $x_i$  plus  $f'$  of  $x_i$   $h$ . The second order approximation uses both the slope and the acceleration at the initial value to calculate the next value and ranges from  $f$  of  $x_i$  to  $f$  of  $x_{i+1}$  approximately equal to  $f$  of  $x_i$  plus  $f'$  of  $x_i$   $h$  plus  $f''$  of  $x_i$  over 2 integral  $h^2$ . The true slope ranges from  $f$  of  $x_i$  to beyond  $x_{i+1}$  equals 1 through  $f$  of  $x_{i+1}$ .

 [Return to Figure 4.7](#)

## Figure 4.8 Text Alternative (Chapter 4)

---

 [Return to Figure 4.8](#)

The horizontal axis represents  $x$  ranging from 0 through  $x_i$  and  $x_{i+1}$  collectively labeled as  $h$ , while the vertical axis represents  $f$  of  $x$ .

The curve shows two points at bottom left (across  $x_i$ ) and top right (across  $x_{i+1}$ , and height  $R_0$ ) connected together and labeled as exact prediction. The curve shows another two points at bottom left (across  $x_i$ ) and bottom right (across  $x_{i+1}$ , and height  $(f \text{ of } x_i)$ ) connected together and labeled as zero order prediction.

 [Return to Figure 4.8](#)

## Figure 4.9 Text Alternative (Chapter 4)

---

 [Return to Figure 4.9](#)

The horizontal axis represents  $x$  ranging from 0 through  $x_i$  and  $x_i + 1$  collectively labeled as  $h$ , while the vertical axis represents  $f$  of  $x$ .

Along the independent axis, steps  $x_i$  and  $x_i + 1$  are labeled with a distance  $h$  between them. There is an intermediate location of extent of reaction also labeled. The extent of reaction is labeled at such a point that the slope of the function at extent of reaction is the same as the slope from the value of the function at  $x_i$  to the value of the function at  $x_i + 1$ . This slope is also remainder  $R_0$  divided by the step size  $h$ .

 [Return to Figure 4.9](#)

## Figure 4.10 Text Alternative (Chapter 4)

---

 [Return to Figure 4.10](#)

A) The horizontal axis represents  $x$  ranging from 0 through  $x_i$  and  $x_i + 1$  collectively labeled as  $h$ , while the vertical axis represents  $f$  of  $x$ .

The concave down with true derivative curve intersecting at  $x_i$  while approximation curve ranges from  $x_i$  to  $x_i + 1$ .

B) The horizontal axis represents  $x$  ranging from 0 through  $x_i - 1$  and  $x_i$  collectively labeled as  $h$ , while the vertical axis represents  $f$  of  $x$ .

The concave down with true derivative curve intersecting at  $x_i$  while approximation curve ranges from  $x_i - 1$  to  $x_i$ .

C) The horizontal axis represents  $x$  ranging from 0 through  $x_i - 1$  and  $x_i$  collectively labeled as  $2h$ , while the vertical axis represents  $f$  of  $x$ .

The concave down with true derivative curve intersecting at center while approximation curve ranges from  $x_i - 1$  to  $x_i + 1$ .

 [Return to Figure 4.10](#)

## Figure 4.11 Text Alternative (Chapter 4)

---

 [Return to Figure 4.11](#)

The first half of downward sloping curve is labeled as total error, while second half is labeled as round-off error. The first half of upward sloping curve is labeled as truncation error while second half is labeled as point of diminishing returns. The point of diminishing returns follows after total error.

 [Return to Figure 4.11](#)

## Figure 4.12 Text Alternative (Chapter 4)

---

 [Return to Figure 4.12](#)

The horizontal axis represents step size ranging from  $10^{-10}$  to  $10^0$ , while vertical axis represents error ranging from  $10^{-12}$  to  $10^0$ .

The curve increases from  $(10^{-10}, 10^{-7})$  to  $(10^0, 10^{-0.5})$  through  $(10^{-6}, 10^{-11.5})$ .

Note: All data is approximate.

 [Return to Figure 4.12](#)



## Figure PT2.1 Text Alternative (Chapter 5)

---

 [Return to Figure PT2.1](#)

The curve starts slightly above the  $x$  axis; as the independent value increases, the curve dips below the  $x$  axis and reaches a minimum (this point is labeled with the first derivative of the function at this point as zero and the second derivative as greater than zero). The curve continues to be concave up and then crosses the  $X$  axis again (this point is labeled as a root). The curve then begins to be concave down and then reaches a maximum value (this point is labeled with the first derivative equal to zero and the second derivative as less than zero). The curve continues to be concave down and once again passes through the  $x$  axis for another root.

 [Return to Figure PT2.1](#)

# Graph of Mass (in Kilograms) Against Graphic Text Alternative (Chapter 5)

---

[!\[\]\(529949c2c3dadbaa4e538e8c643454bc\_img.jpg\) Return to Graph of Mass \(in Kilograms\) Against Graphic](#)

The horizontal axis represents mass (in kilograms) ranging from 60 through 200 in increments of 20, while vertical axis represents  $f$  of  $m$  (in meters per second) ranging from negative 4 through 1 in increments of 1.

The data is as follows:

The curve increases from (30, negative 2) to (200, 1.2) through (80, negative 2.1), (140, 0) (which is encircled and labeled as root), and (160, 0.4).

Note: All data is approximate.

[!\[\]\(a870788d6ed9b8fd294b7654a8c8526b\_img.jpg\) Return to Graph of Mass \(in Kilograms\) Against Graphic](#)

## Figure 5.1 Text Alternative (Chapter 5)

---

 [Return to Figure 5.1](#)

A) Graph shows that if the sign of the function at the left and right points are the same, there may be no root. This graph shows an upward-facing parabola whose minimum is greater than 0.

B) Graph shows that if the signs of the function at the left and right points are different, there may be one root. This graph shows a gently curved line with a negative value at the left, a positive value at right, and a single root in the middle.

C) Graph shows that if the sign of the function the left and right points are the same, there may be a non-zero even number of roots. The graph shows a curve that is positive at left, dips below the x axis, curves up again and past the x axis, and is positive again at the right point.

D) Graph shows that if the signs of the function at the left and right points are different, there may be any odd number of roots between the points. The graph shows a line that starts positive, then oscillates negative, positive, and negative again, there are thus three roots in the section.

 [Return to Figure 5.1](#)

## Figure 5.2 Text Alternative (Chapter 5)

---

 [Return to Figure 5.2](#)

A) The continuous plot starts positive, then reaches 0 at a local minimum such that the curve is never locally negative. As the curve continues, it bends towards the negative and then does pass through the X axis. The sign of the function on the left is thus positive and the sign of the function on the right is negative, but the segment contains an even number of roots.

B) The discontinuous plot has a segment at left that starts negative, crosses the X axis from below, and then from above. There is a discontinuity in the middle, where the curve starts positive with a negative slope and bends towards the positive without having crossed the X axis. The sign of the function on the left is thus negative and the sign of the function on the right is positive, but the segment also contains an even number of roots.

 [Return to Figure 5.2](#)

## Figure 5.3 Text Alternative (Chapter 5)

---

 [Return to Figure 5.3](#)

The first segment shows the curve starting positive, dipping below the X axis, then going positive. If this segment were used, the sign of the function at both ends would be positive and thus both roots might be missed. The second segment shows the curve as positive the whole duration, with no roots. The third section has the curve go from positive to negative to positive to negative and finally to positive (there are thus four roots in this section but no sign change of the functions on the ends so these roots might all be lost). The fourth section once again shows the function as positive the whole time. The fifth section shows the curve gently curving from a positive value to a negative value (there is thus a sign change of the function and there is one root). The sixth section has the function start off as negative, reach but not cross the X axis, then continue negative. There is thus a "zero touch" here but not a change in signs of the function.

 [Return to Figure 5.3](#)

# A Continuous Curve with Multiple Crests Graphic Text Alternative (Chapter 5)

---

 [Return to A Continuous Curve with Multiple Crests Graphic](#)

The horizontal axis ranges from 3 through 6 in increments of 0.5, while vertical axis ranges from negative 2 through 2 in increments of 1.

The data is as follows:

The curve begins at (3, negative 1.8) to (6, negative 0.4) through multiple crests and troughs. Smaller rectangular blocks are visible at (3.25, 0), (3.26, 0), (4.65, 0), and (5.65, 0).

Note: All data is approximate.

 [Return to A Continuous Curve with Multiple Crests Graphic](#)

# Graph Shows a Parabolic Curve Opening Downwards Graphic Text Alternative (Chapter 5)

---

 [Return to Graph Shows a Parabolic Curve Opening Downwards Graphic](#)

The horizontal axis ranges from 4.20 through 4.30 in increments of 0.02, while vertical axis ranges from 0.00 through 0.10 in increments of 0.02.

The data is as follows:

The curve increases from (4.20, 0.082) to (4.30, 0.112) through (4.22, 0.02), (4.23, 0.00) (which is a root), (4.264, 0.00) (which is a root), and (4.28, 0.04).

Note: All data is approximate.

The horizontal axis ranges from 5.15 through 5.21 in increments of 0.01, while vertical axis ranges from negative 0.03 through negative 0.01 in increments of 0.01.

The data is as follows:

The curve decreases from (5.15, 0.023) to (5.21, 0.032) through (5.161, 0.00) (which is a root), (5.17, 0.01), (5.195, 0.00) (which is a root), and (5.25, negative 0.02).

Note: All data is approximate.

 [Return to Graph Shows a Parabolic Curve Opening Downwards Graphic](#)

## Figure 5.5 Text Alternative (Chapter 5)

---

[!\[\]\(d263118e0bfd47dc6bc704167d936b83\_img.jpg\) Return to Figure 5.5](#)

The curve increases from (50, negative 4.8) to (200, 0.9) intersecting horizontal axis  $m$  at (145, 0) labeled as root.

The bottom of the figure shows iterations leading to each other as:

First iteration:

$x_l$  ranges to  $x_u$  with  $x_r$  in the center.

Second iteration:

$x_l$  ranges to  $x_u$  with  $x_r$  in the center, where the range  $x_l$  to  $x_u$  is half of first iteration.

Third iteration:

$x_l$  ranges to  $x_u$  with  $x_r$  in the center, where the range  $x_l$  to  $x_u$  is half of second iteration.

Fourth iteration:

$x_l$  ranges to  $x_u$  with  $x_r$  in the center, where the range  $x_l$  to  $x_u$  is half of third iteration.

[!\[\]\(4688aadfd656ded00cd6bdfae55089a9\_img.jpg\) Return to Figure 5.5](#)



## Figure 5.6 Text Alternative (Chapter 5)

---

 [Return to Figure 5.6](#)

The horizontal axis represents iterations ranging from 0 through 8 in increments of 2, while vertical axis represents percent relative error ranging from 0.1 through 100.

The data is as follows:

True error:

The curve decreases from (1, 15) to (8, 0.5) through (2, 20), (3, 0.8), (4, 7), (6, 1) and (7, 0.1).

Approximate error:

The curve decreases from (2, 30) to (8, 0.75) through (4, 7.5), (5, 5), and (6, 3).

Note: All data is approximate.

 [Return to Figure 5.6](#)

## Figure 5.9 Text Alternative (Chapter 5)

---

 [Return to Figure 5.9](#)

A straight line is drawn between the value of the function at the left point  $x_l$  and the value of the function at the right point  $x_u$ . The new estimate for the root,  $x_r$ , is where this straight line intercepts the X axis rather than being halfway between  $x_l$  and  $x_u$  as it would be with the bisection method. It intersects the horizontal axis at a point labeled as  $x_r$ .

 [Return to Figure 5.9](#)

## Figure 5.10 Text Alternative (Chapter 5)

---

 [Return to Figure 5.10](#)

The horizontal axis represents years ranging from 1950 through 2030 in increments of 10, while vertical axis represents carbon dioxide concentration ranging from negative 300 through 420 in increments of 20.

The data is as follows:

Polynomial model:

The curve increases from 318 in 1960 to 410 in 2020, through 340 in 1980, 360 in 1995, and 390 in 2010.

Mauna Loa data:

The curve increases from 318 in 1960 to 410 in 2020, through 340 in 1980, 360 in 1995, and 390 in 2010.

Note: All data is approximate.

 [Return to Figure 5.10](#)

# Graph of Years Against Rain Water pH Shows Downward Graphic Text Alternative (Chapter 5)

---

[!\[\]\(21199eb166cc97331a0c54c649195dcc\_img.jpg\) Return to Graph of Years Against Rain Water pH Shows Downward Graphic](#)

The horizontal axis represents years ranging from 1960 through 2020 in increments of 10, while vertical axis represents carbon rainwater pH ranging from 5.42 through 5.48.

The data is as follows:

The curve decreases from 5.48 in 1960 to 5.43 in 2020, through 340 in 1980, 360 in 1995, and 390 in 2010.

Note: All data is approximate.

[!\[\]\(c694a3ff3b077d76910920a6a1593ab4\_img.jpg\) Return to Graph of Years Against Rain Water pH Shows Downward Graphic](#)

## Figure P5.4 Text Alternative (Chapter 5)

---

 [Return to Figure P5.4](#)

The velocity of the fluid leaving the tube is given as  $v$ .

 [Return to Figure P5.4](#)

## Figure P5.11 Text Alternative (Chapter 5)

---

 [Return to Figure P5.11](#)

The fluid inside is labeled as V.

 [Return to Figure P5.11](#)

## Figure P5.12 Text Alternative (Chapter 5)

---

 [Return to Figure P5.12](#)

A) A beam is pinned on its left end & held fixed on its right with an increasing distributed load starting at 0 on the left to  $w_0$  on the right.

B) The deflection of the beam has a negative slope at the pinned end; the deflection gets more negative until about one-third along the beam, then the deflection increases back towards zero at the right end of the beam. The curvature of the deflection changes from concave up over the first two-thirds of the beam to concave down over the last third. The slope of the deflection at the right end is 0.

 [Return to Figure P5.12](#)

## Figure P5.25 Text Alternative (Chapter 5)

---

[!\[\]\(d263118e0bfd47dc6bc704167d936b83\_img.jpg\) Return to Figure P5.25](#)

The distance between the upper base and the surface of the fluid is  $h_1$ .

[!\[\]\(3d8c13c92b853674f749aac6fa869926\_img.jpg\) Return to Figure P5.25](#)



# Figure 6.1 Text Alternative (Chapter 6)

---

[!\[\]\(c8d96c8885d3000a912c2582004aed63\_img.jpg\) Return to Figure 6.1](#)

A) For this demonstration, the function starts negative, continues concave up through a root, and then turns concave down after a local maximum. For the closed method, the root is initially bracketed by points  $x_l$  and  $x_u$ .

B) In the graphs of the open method, there are two different examples given. In the first, the initial guess is at a point in the curve where the tangent to that curve leads the method to a next guess relatively far to the right of the root and to a place where the curve is positive and its slope is negative (this means that the next guess will be even further to the right).

C) The second graph of the open method shows an initial guess just slightly to the right of the previous guess; however, the slope here isn't as large and so the next guess is not thrown as far to the right. The next guess also lands on a part of the curve that is positive with a positive slope, so the second iteration in this graph is much closer to the root.

The process of bisection is shown below the graph demonstrating a closed method, where at each step, either the lower or upper bracket is moved to the point between the two based on the sign of the function at that midpoint. The distance between the brackets is thus halved each time, regardless of the shape of the curve or the relative location of the root within the bracket.

[!\[\]\(e3f8612927870f2e0f9f5989e6dd3064\_img.jpg\) Return to Figure 6.1](#)

## Figure 6.4 Text Alternative (Chapter 6)

---

 [Return to Figure 6.4](#)

A) The function  $e$  to the negative  $x$  minus  $x$  is graphed, and the root is where this function crosses the  $x$  axis at the center.

B) The function  $e$  to the negative  $x$  is graphed in black and  $x$  is graphed in blue here, the root is located at the  $x$  value where the graphs intersect each other.

 [Return to Figure 6.4](#)

## Figure 6.5 Text Alternative (Chapter 6)

---

 [Return to Figure 6.5](#)

A) The horizontal axis represents  $x$  ranging from 0.2 to 0.8 in increments of 0.1, while the vertical axis represents  $\exp$  (negative  $x$ ) ranging from 0.2 to 0.8 in increments of 0.2.

The data is as follows:

$\exp$  (negative  $x$ ):

The curve decreases from (0.24, 0.8) to (0.8, 0.45) through (0.37, 0.675), (0.56, 0.57) (node visible), and (0.7, 0.5).

Diagonal:

The curve increases from (0.2, 0.2) to (0.8, 0.8) through (0.3, 0.3), (0.4, 0.4), (0.5, 0.5), (0.56, 0.56) (node visible), and (0.7, 0.7).

B) The horizontal axis represents  $x$  ranging from 0.40 to 0.80 in increments of 0.05, while the vertical axis represents negative  $\ln(x)$  0.40 to 0.80 in increments of 0.05.

The data is as follows:

Negative  $\ln(x)$ :

The curve decreases from (0.45, 0.80) to (0.67, 0.40) through (0.50, 0.70), (0.55, 0.60), and (0.61, 0.50).

Diagonal:

The curve increases from (0.40, 0.40) to (0.80, 0.80) through (0.45, 0.45), (0.55, 0.55), (0.65, 0.65), (0.70, 0.70), and (0.75, 0.75).

Note: All data is approximate.

 [Return to Figure 6.5](#)

## Figure 6.6 Text Alternative (Chapter 6)

---

 [Return to Figure 6.6](#)

A) The horizontal axis represents  $x$ , while the vertical axis represents  $y$ .

The data is as follows:

The curve  $y_1$  equals  $x$  and  $y_2$  equals  $g$  of  $x$  intersects at bottom left.  $x_0$  shifts to  $y_2$  equals  $g$  of  $x$ , shifts to  $y_1$  equals  $x$  and  $y_1$  shifts back to  $y_2$  equals  $g$  of  $x$ .

B) The horizontal axis represents  $x$ , while the vertical axis represents  $y$ .

The data is as follows:

The curve  $y_2$  equals  $g$  of  $x$  and  $y_1$  equals  $x$  intersects at bottom left.  $x_0$  shifts to  $y_2$  equals  $g$  of  $x$  shifts to  $y_1$  equals  $x$  and  $y_1$  shifts back to  $y_2$  equals  $g$  of  $x$ , finally shifting to  $y_1$  equals  $x$ .

Note: All data is approximate.

 [Return to Figure 6.6](#)

## Figure 6.7 Text Alternative (Chapter 6)

---

[!\[\]\(8af806fb1314382d09bc5ec5b767526c\_img.jpg\) Return to Figure 6.7](#)

The horizontal axis represents  $x$  ( $x_0$ ,  $x_1$ ,  $x_2$ , and  $x_3$ ), while the vertical axis represents  $g$  of  $x$ .

The data is as follows:

The dotted line is an upward sloping curve.

Inverted S shaped curve increases with  $x_0$ ,  $x_1$ ,  $x_3$  (node is encircled), and  $x_2$  leading to it.

[!\[\]\(0aff635c4179ba9e710b00f4b01d3b20\_img.jpg\) Return to Figure 6.7](#)

## Figure 6.8 Text Alternative (Chapter 6)

---

 [Return to Figure 6.8](#)

The function is concave up over the domain shown and crosses from negative values to positive values intersecting the downward sloping curve with arrow head at top right labeled as 'slope equals  $f'$  of  $x$ '. The height between slope equals  $f'$  of  $x$  and  $x_i$  is labeled as  $f$  of  $x_i$  minus 0.

 [Return to Figure 6.8](#)

## Figure 6.9 Text Alternative (Chapter 6)

---

 [Return to Figure 6.9](#)

The initial estimate of 0.5 is on a part of the curve with a tangent line that is nearly horizontal, due to the flat section. The second estimate is shown at about 52; the function here has a value of over 1 of 10 to the seventeenth power. The slope here is also highly positive, so the tangent curve is nearly vertical this time and the estimate moves from 52 to about 46. The process gets to the actual value of 1 only after multiple iterations.

 [Return to Figure 6.9](#)

## Figure 6.10 Text Alternative (Chapter 6)

---

 [Return to Figure 6.10](#)

A) Graph shows a curve that starts with slight positive slope, curves up through the actual root, then curves back down to a slightly positive slope. The initial estimate chosen has a second estimate that is to the left of the root. The tangent at this point sends the next estimate to the right of the root. The tangent here sends the estimate even further to the left of the previous estimate. This loop may continue indefinitely for this curve.

B) Graph shows a curve that has a section that is all positive and concave up. The estimates oscillate to the left and right of the local minimum of this function until finally hitting a part of the curve with a nearly 0 slope, causing the estimate to move far to the right of this section. There was no root here to find, but the method spent several iterations trying to find one.

C) Graph shows a somewhat sinusoidal curve that oscillates above and below the x axis. The initial estimate is chosen on a positive part of the function with a slightly positive slope; this sends the estimate to the left specifically in this example to a point where the curve is negative with a slightly negative slope. This also sends the estimate further to the left. Depending on the shape of the curve, this process could continue indefinitely.

D) Graph shows a single positive concave up section where the first estimate is picked in a place that the second estimate hits a nearly horizontal section of the graph. The next estimate would therefore be very far away.

 [Return to Figure 6.10](#)



## Figure 6.12 Text Alternative (Chapter 6)

---

 [Return to Figure 6.12](#)

A) Graph shows a straight line drawn between estimates at the far left and right of the domain. The straight line intercepts the  $x$  axis relatively far to the left of the actual root since the function is concave up.

B) Graph shows a fit of three points, which yields a curve that is concave left; this fit is generated with  $x$  as a function of  $y$ . This curve has a shape much closer to the actual function and the estimate is much closer to the actual root.

 [Return to Figure 6.12](#)

## Figure 6.13 Text Alternative (Chapter 6)

---

 [Return to Figure 6.13](#)

The horizontal axis represents  $x$  ranging from 0 to 4, while the vertical axis represents  $y$  ranging from 0 to 6 in increments of 2.

The data is as follows:

$y$  equals  $f$  of  $x$ :

The curve begins at (0, 5) and ends at (4, 6) through (0.8, 2) and (1.7, 1).

$x$  equals  $f$  of  $y$ :

The curve begins at (5.5, 5) and ends at (5, negative 0.5) through (4, 0) labeled as root.

Note: All data is approximate.

 [Return to Figure 6.13](#)

# Graph of x Against y Shows a S-Shaped Curve Graphic Text Alternative (Chapter 6)

---

 [Return to Graph of x Against y Shows a S-Shaped Curve Graphic](#)

The horizontal axis represents  $x$  ranging from negative 1 to 3 in increments of 1, while the vertical axis represents  $f(x)$  ranging from negative 20 to 40 in increments of 10.

The data is as follows:

The curve begins at (negative 1.7, negative 22.5), and ends at (3, 42.5) through (1,0), and (2, 0).

Note: All data is approximate.

 [Return to Graph of x Against y Shows a S-Shaped Curve Graphic](#)

# Figure Graph of 1 Over Sqrt of f Against Right Hand Side Text Alternative (Chapter 6)

---

[!\[\]\(21199eb166cc97331a0c54c649195dcc\_img.jpg\) Return to Figure Graph of 1 Over Sqrt of f Against Right Hand Side](#)

The horizontal axis represents 1 over sqrt of f ranging from 5 to 30 in increments of 5, while the vertical axis represents right hand side ranging from 5 to 30 in increments of 5.

The data is as follows:

The dashed curve increases from (2.4, 2.4) to (32.5, 32.5) through (10, 10), (15, 15), (20, 20), and (25, 25).

The solid curve decreases from (2, 14) to (32.5) through (10, 12.5), (20, 11), and (30, 10).

Note: All data is approximate.

[!\[\]\(ec9132f1d27c8919987d92907322654d\_img.jpg\) Return to Figure Graph of 1 Over Sqrt of f Against Right Hand Side](#)

# Graph of Reynold's Number Against Fanning Friction Factor Graphic

## Text Alternative (Chapter 6)

---

[!\[\]\(8af806fb1314382d09bc5ec5b767526c\_img.jpg\) Return to Graph of Reynold's Number Against Fanning Friction Factor Graphic](#)

The horizontal axis represents Reynold's number ranging from 10 raised to power 3 through 10 raised to power 7, while the vertical axis represents fanning friction factor ranging from 0.004 to 0.016 in increments of 0.002.

The data is as follows:

Colebrook:

The curve decreases from (10 raised to power 3, 0.0158) to (10 raised to power 7, 0.0049) through (10 raised to power 4, 0.008), (10 raised to power 3, 0.0053), and (10 raised to power 6, 0.0049).

Haaland:

The curve decreases from (10 raised to power 3, 0.0165) to (10 raised to power 7, 0.0049) through (10 raised to power 4, 0.008), (10 raised to power 3, 0.0053), and (10 raised to power 6, 0.0049).

Note: All data is approximate.

[!\[\]\(8bba887393ca45b761e5cb49e755e762\_img.jpg\) Return to Graph of Reynold's Number Against Fanning Friction Factor Graphic](#)

## Figure P6.17 Text Alternative (Chapter 6)

---

 [Return to Figure P6.17](#)

A) The cable's left end is shown about an inch above the  $x$  axis while the right end is about three inches above. The cable forms a parabola and is shown loaded with a uniform load per unit length  $w$ . The height of the cable's minimum above the  $x$  axis is given as  $y_0$ .

B) On the right of the figure, a small section of cable, which goes from the local minimum to a point about three quarters of the way to the right end and is shown with three forces acting on it, a tension pulling to the left  $T_A$ , a tension pulling up and to the right  $T_B$ , and a weight  $W$  given by the product of the uniform load  $w$  and the cable length  $s$ .  $T_A$  is shown acting directly to the left since it is at the cable's local minimum;  $T_B$  is shown acting up and to the right at an angle  $\theta$ .

 [Return to Figure P6.17](#)

## Figure P6.20 Text Alternative (Chapter 6)

---

 [Return to Figure P6.20](#)

A) Before the ball hits the spring, the distance between the bottom of the ball and the top of the uncompressed spring is given by  $h$ . After the ball drops and the spring is compressed, the distance the top of the spring moves is given by  $d$ .

B) This means that the total distance travelled by the bottom of the ball from its initial position to where the spring is maximally compressed is  $h$  plus  $d$ .

 [Return to Figure P6.20](#)

## Figure P6.21 Text Alternative (Chapter 6)

---

 [Return to Figure P6.21](#)

The parabolic path shows the projectile eventually hitting the ground across a plot of  $x$  against  $y$ .

 [Return to Figure P6.21](#)



## Figure P6.31 Text Alternative (Chapter 6)

---

 [Return to Figure P6.31](#)

The height of the fluid before flowing over the weir is given by  $H$ . The height of the weir, which is smaller than that height, is given by  $H_w$  while the difference in height between the initial fluid and the weir is given by  $H_h$ . The rate of flow across the top of the weir is given as  $Q_w$  and the length of the weir is given as  $L_w$ . The figure shows the fluid height decreasing; starting some distance before the fluid hits the weir and stabilizing about midway across the weir. There are also flow lines drawn in the fluid to show how the water below the level of the weir on the left side needs to rise to get over the weir. There are flow lines to the right of the weir showing that there may be circulation in the fluid to the right of the weir.

 [Return to Figure P6.31](#)

## Figure P.36 Text Alternative (Chapter 6)

---

 [Return to Figure P.36](#)

The angle of attack  $\alpha$  is the angle made by the line connecting the far left and far right points of the airfoil to the horizontal, if the left end is higher, the attack angle is positive. This angle is very small. The angle between the upper leading edge of the airfoil and the horizontal is  $\theta_u$  while the angle between the lower leading edge of the airfoil and the horizontal is  $\theta_l$ . Since this airfoil has a positive attack angle,  $\theta_l$  is larger than  $\theta_u$ . The upper shock angle is measured between the upper shock wave and the upper leading edge of the airfoil; it is denoted  $\beta_u$  and in the figure is about 30 degrees. The lower shock angle is measured between the lower shock wave and the lower leading edge of the airfoil; it is denoted  $\beta_l$  and in the figure is about 20 degrees, which is to say, smaller than  $\beta_u$ .

 [Return to Figure P.36](#)

## Figure P6.38 Text Alternative (Chapter 6)

---

 [Return to Figure P6.38](#)

The cable is connected at point A at left, and goes down and right to a point B where the scoreboard is connected, then goes up and right to point C. The horizontal difference between A and B and also B and C is  $L$ ; the vertical distance between A and B and also C and B is  $d$ . The length of the cable from A to B and also from C to B is given as  $L'$ . Below the figure, there is a free body diagram of point B. The forces are the tension in the rope  $F$  going from B to A, the tension in the rope  $F$  going from B to C, and the weight of the board  $W$ . The angle that the section of the cable going from B to C from the horizontal is  $\theta$ . Good news for Hippos fans, they are up 103 to 85 with about thirteen minutes left in the game.

 [Return to Figure P6.38](#)

## Figure P6.39 Text Alternative (Chapter 6)

---

 [Return to Figure P6.39](#)

The cylindrical tank is mostly filled with fluid. The height of fluid above the horizontal section of pipe is labeled  $h$ . The diameter of both the vertical and horizontal pipe is given by  $d$ , it is significantly smaller than the diameter of the cylindrical tank. The length of the horizontal section of pipe is given by  $L$  and the flow rate of fluid out of the far right end of this pipe is  $Q$ . The valve is shown at the extreme right end of the horizontal pipe.

 [Return to Figure P6.39](#)

# Figure 7.1 Text Alternative (Chapter 7)

---

 [Return to Figure 7.1](#)

The horizontal axis represents  $t, s$  ranging from 0 through 12 in increments of 2, while vertical axis represents  $z, m$  ranging from negative 100 through 200 in increments of 100.

The data is as follows:

The curve begins at (0, 100) and ends at (11.7, negative 10) through (4, 200) which is labeled as maximum elevation.

Note: All data is approximate.

 [Return to Figure 7.1](#)

# Figure 7.2 Text Alternative (Chapter 7)

---

 [Return to Figure 7.2](#)

A sinusoidal signal shows roots where the function is equal to zero, minima are where the derivative of the function is zero and the second derivative is greater than zero, and maxima are where the derivative of the function is zero and the second derivative is less than zero.

 [Return to Figure 7.2](#)

## Figure 7.3 Text Alternative (Chapter 7)

---

 [Return to Figure 7.3](#)

A) Graph shows that the minimum of a function  $f$  of  $x$  is also the maximum of the function negative  $f$  of  $x$ . These points are linked through  $x$  asterisk.

B) Graph shows contours (level curves showing 4 ovals) centered on a point that is labeled as the optimum value of  $f$ . The level curves radiate out from the optimum  $f$  of  $(x \text{ asterisk}, y \text{ asterisk})$  point. The outer oval is labeled as  $f$  of  $(x, y)$ .

 [Return to Figure 7.3](#)

## Figure 7.4 Text Alternative (Chapter 7)

---

 [Return to Figure 7.4](#)

The four extremes are at different distances from the x axis. The extremum with the largest positive value is the global maximum, while the other positive (but not as large) extremum is a local maximum. The extremum with the largest negative value is the global minimum, while the other negative extremum is the local minimum.

 [Return to Figure 7.4](#)



## Figure 7.6 Text Alternative (Chapter 7)

---

 [Return to Figure 7.6](#)

A) Graph shows concave up section of the curve is positive over the whole section; it has a minimum about two-third to the right of the section. The left point of the section is called  $x_l$  and the right point is  $x_u$ . A distance  $d$  is calculated and two subdomains are created, one that starts at the extreme left and is of width  $d$  to a point  $x_1$  and another that starts at the extreme right and comes left a distance  $d$  to a point  $x_2$ . Since these are about 60 percent as large as the original distance between  $x_l$  and  $x_u$ ,  $x_1$  is to the right of  $x_2$ .

B) Graph shows the value of the function at  $x_2$  larger than the value at  $x_1$ , so the section of the curve between  $x_l$  and  $x_2$  is removed from consideration and  $x_2$  now becomes the new  $x_l$ .  $x_u$  remains the same as before. In the next iteration,  $d$  is recalculated (and is smaller since the new search domain is smaller) and new locations for  $x_2$  and  $x_1$  are calculated.

 [Return to Figure 7.6](#)

# Figure 7.8 Text Alternative (Chapter 7)

---

 [Return to Figure 7.8](#)

The horizontal axis represents values for  $x$  ( $x_1$ ,  $x_2$ ,  $x_3$ , and  $x_4$ ) while vertical axis represents  $f$  of  $x$ .

True function:

An interpolating parabola is drawn through four points and the maximum of this interpolation is labeled as true maximum.

Parabolic function:

An interpolating parabola is drawn through four points and the maximum of this interpolation is labeled as parabolic approximation of maximum.

The peak of both parabolas are between  $x_2$  and  $x_3$ .

 [Return to Figure 7.8](#)

# Figure 7.9 Text Alternative (Chapter 7)

---

 [Return to Figure 7.9](#)

Between  $x_1$  and  $x_2$ :

$x_4$  between  $x_2$  and  $x_3$ ?

If N:  $f$  of  $x_4$  lesser than  $f$  of  $x_2$ ?

If Y:  $f$  of  $x_4$  lesser than  $f$  of  $x_2$ ?

$f$  of  $x_4$  lesser than  $f$  of  $x_2$ ?

If N:  $x_4$  leads to  $x_1$

If Y:  $x_2$  leads to  $x_3$  and  $x_4$  leads to  $x_2$ .

$f$  of  $x_4$  lesser than  $f$  of  $x_2$ ?

If N:  $x_4$  leads to  $x_3$

If Y:  $x_2$  leads to  $x_1$  and  $x_4$  leads to  $x_2$ .

 [Return to Figure 7.9](#)

## Figure 7.10 Text Alternative (Chapter 7)

---

 [Return to Figure 7.10](#)

A) The horizontal axis represents  $x_1$  ranging from negative 2.0 through 0.0 in increments of 0.5, another horizontal axis represents  $x_2$  ranging from 0 through 3 in increments of 1, while vertical axis ranges from 1 through 8 in increments of 1.

The data is as follows:

The curve shows a mesh structure starting at (negative 1.75, 7.5) and ends at (2.9, 7.5) between (negative 2.0, 1.9) (highest point) and (negative 0.25, 0.8) (lowest point).

B) The horizontal axis represents  $x_1$  ranging from negative 2.00 through 0.0 in increments of 0.5, while vertical axis represents  $x_2$  ranging from 0.0 through 3.0 in increments of 0.5.

The data is as follows:

The circular curve is visible between (negative 1.50, 2.0), (negative 1.00, 2.0), (negative 0.50, 1.0) and (negative 1.50, 2.0).

The curves at bottom left begin and end at following points: (negative 2.00, 2.0) to (negative 0.50, 0.0); (negative 2.00, 1.35) to (negative 1.00); (negative 2.00, 1.0) to (negative 1.25, 0.0) and (negative 2.00, 0.5) to (1.78, 0.0).

The curves at top right begin and end at following points: (negative 1.50, 3.0) to (0.0, 1.0); (negative 1.00, 3.0) to (0.0, 1.6); (negative 0.50, 3.0) to (0.0, 2.0) and (negative 0.12, 3.0) to (0.0, 3.7).

Note: All data is approximate.

 [Return to Figure 7.10](#)

## Figure 7.12 Text Alternative (Chapter 7)

---

 [Return to Figure 7.12](#)

A) Spring a of length  $L_a$  and spring constant  $k_a$  is fixed to a ceiling and hangs straight down. The bottom is connected to the top of spring b of length  $L_b$  and spring constant  $k_b$ , which is then connected to the floor. Without any external force, the springs are vertical.

B) A horizontal force to the right  $F_1$  and a vertical force up  $F_2$  are applied to the junction between the springs. That junction moves to the right a distance  $x_1$  and up a distance  $x_2$  to remain in equilibrium. Both springs change in length; in the graph, the distance  $x_1$  is large relative to the distance  $x_2$ , and both springs look elongated.

 [Return to Figure 7.12](#)

## Figure P7.15 Text Alternative (Chapter 7)

---

 [Return to Figure P7.15](#)

A) An illustration shows a bicycle with handle, seat, two tires and a pedal.

B) The force from the pedals on to these connectors is given at an angle  $\theta$  from the x axis, with a positive angle,  $F$  directed downward towards right and a positive  $y$  directed downwards. The connectors are of length  $l$  (one goes up and to the left from the pedals and one goes up and to the right). The connectors are symmetric and the top ends are height  $h$  above the pedals and are separated by width  $w$  from each other.

 [Return to Figure P7.15](#)

## Figure P7.16 Text Alternative (Chapter 7)

---

 [Return to Figure P7.16](#)

The outside temperature of the wire is labeled as  $T_{\text{air}}$ .

 [Return to Figure P7.16](#)

## Figure P7.28 Text Alternative (Chapter 7)

---

 [Return to Figure P7.28](#)

The horizontal axis represents function of  $c$  in milligrams per liter ranging from 0 through 10 in increments of 5, while vertical axis represents inverse days ranging from 0 through 0.4 in increments of 0.2.

The data is as follows:

The curve begins at  $(0, 0)$  and ends at  $(10, 0.09)$  showing decreasing trend after  $(1.5, 0.36)$ .

Note: All data is approximate.

 [Return to Figure P7.28](#)



## Figure P7.31 Text Alternative (Chapter 7)

---

[!\[\]\(c8d96c8885d3000a912c2582004aed63\_img.jpg\) Return to Figure P7.31](#)

The horizontal axis represents travel time in days ranging from 0 through 20 in increments of 5, while vertical axis represents milligrams per liter ranging from 0 through 12 in increments of 4.

The data is as follows:

The curve begins at (0, 9.1) and ends at (20, 8.5) showing decreasing trend after (3.2, 3.2) where  $t_c$  and  $O_c$  both are 3.2. A horizontal line from (0.0, 10.15) is labeled as  $O_s$ .

Note: All data is approximate.

[!\[\]\(e3f8612927870f2e0f9f5989e6dd3064\_img.jpg\) Return to Figure P7.31](#)

## Figure P7.34 Text Alternative (Chapter 7)

---

 [Return to Figure P7.34](#)

The horizontal axis represents  $x$  ranging from 0 through 10 in increments of 5, while vertical axis represents  $T$  ranging from 0 through 0.4 in increments of 0.1.

The data is as follows:

The curve begins at (0, 0) and ends at (10, 0.09) showing decreasing trend after (1.0, 0.3).

Note: All data is approximate.

 [Return to Figure P7.34](#)

## Figure P7.35 Text Alternative (Chapter 7)

---

 [Return to Figure P7.35](#)

The horizontal axis represents  $V$  ranging from 0 through 1200 in increments of 400, while vertical axis represents  $D$  ranging from 0 through 20000 in increments of 10000.

The data is as follows:

The curve begins at (2, 20000) and ends at (1400, 10000) showing increasing trend after (470, 4900).

The point at (470, 4900) is labeled as minimum while (690, 4990) is labeled as total.

The thin curve for  $Fric$  increases from (150, 0) to (1400, 10000) and for lift decreases from (2, 20000) to (1190, 10).

Note: All data is approximate.

 [Return to Figure P7.35](#)

## Figure P7.36 Text Alternative (Chapter 7)

---

 [Return to Figure P7.36](#)

The displacement of the location between the cylinders is given by  $x$ , with downward deflections being positive.

 [Return to Figure P7.36](#)

## Figure P7.37 Text Alternative (Chapter 7)

---

 [Return to Figure P7.37](#)

A force  $F$  pulls right on mass 2. Mass 1 has displacement of  $x_1$  and mass 2 has displacement of  $x_2$ .

 [Return to Figure P7.37](#)

## Figure P7.39 Text Alternative (Chapter 7)

---

 [Return to Figure P7.39](#)

As the ladder goes left, it just passes over the fence of height  $h$  and continues to the wall.

 [Return to Figure P7.39](#)

## Figure P7.40 Text Alternative (Chapter 7)

---

 [Return to Figure P7.40](#)

A channel of width  $w_1$  is oriented from left to right; this channel is intersected by a wider channel of width  $w_2$  that comes in from the lower left. The angle between the right part of the  $w_1$  channel and the  $w_2$  channel is an obtuse angle  $\alpha$ . A ladder of length  $L$  is shown trying to make a right turn out of channel  $w_2$  and into the right branch of channel  $w_1$ . The left side of the ladder is hitting the upper wall of the  $w_2$  channel, while the right side of the ladder is hitting the upper wall of the right part of channel  $w_1$ . The ladder is also touching the corner between the lower wall of  $w_2$  and the lower right wall of  $w_1$ . The ladder also makes an angle  $\theta$  from the lower right wall of  $w_1$ .

 [Return to Figure P7.40](#)

## Figure P7.41 Text Alternative (Chapter 7)

---

 [Return to Figure P7.41](#)

The left end can rotate about the pin, whereas the right end can neither move nor change in slope.

 [Return to Figure P7.41](#)



## Figure P7.44 Text Alternative (Chapter 7)

---

[!\[\]\(c8d96c8885d3000a912c2582004aed63\_img.jpg\) Return to Figure P7.44](#)

The base of the nozzle is a height  $h_1$  above ground; the nozzle itself extends from the base a length  $L$  at an angle  $\theta$  from the horizontal. The start of the building is a horizontal distance  $x_1$  away, and the building is of height  $h_2$ . The maximum distance the fire hose can reach on the roof is labeled as  $x_2$ . The idea is that there will be some angle below which the water will hit the side of the building rather than the top.

[!\[\]\(666e09182d4cd268646ea700ea60dcdf\_img.jpg\) Return to Figure P7.44](#)

# Figure PT3.1 Text Alternative (Chapter 8)

---

 [Return to Figure PT3.1](#)

A) A series of interconnected tanks with pipes connecting them. Feed leads to x 1, further leading to x 2 (bottom), x 3 (top), x 4 (bottom) (leading outside), and finally leading to x 5 across bottom (leading outside).

B) A series of chambers with 7 sections are directly connected to each other and labeled as x 1 leading to x n through x blank, i minus 1, x i plus 1 and blank.

 [Return to Figure PT3.1](#)

## Figure 8.1 Text Alternative (Chapter 8)

---

 [Return to Figure 8.1](#)

A) The cords are unstretched at all with their displacements labeled as  $x_1$ ,  $x_2$  and  $x_3$  and is given by 0.

B) The cords have stretched by different amounts.

 [Return to Figure 8.1](#)

## Figure 8.2 Text Alternative (Chapter 8)

---

[!\[\]\(21199eb166cc97331a0c54c649195dcc\_img.jpg\) Return to Figure 8.2](#)

Left panel:

Person has a force up given by the product of the first cord's spring constant,  $k_1$ , and the amount the first cord is stretched,  $x_1$ . There is a force down  $m_1 g$  based on the weight of the person and also a force down based on the second cord's spring constant,  $k_2$ , and the amount the second cord is stretched,  $x_2$  minus  $x_1$ .

Middle panel:

Person has a force up based on the tension in cord 2, which is  $k_2$  times  $x_2$  minus  $x_1$ . The forces pulling down on person 2 are that person's weight  $m_2 g$  and the tension in the third cord,  $k_3$  times  $x_3$  minus  $x_2$ .

Right panel:

Person has a force up based on the tension in cord 3,  $k_3$  times  $x_3$  minus  $x_2$ , and a force down based on weight,  $m_3 g$ .

[!\[\]\(05be7c7a8995decd503647c99211f7c2\_img.jpg\) Return to Figure 8.2](#)

## Figure 8.3 Text Alternative (Chapter 8)

---

 [Return to Figure 8.3](#)

Element a 23, which is in the second row and third column, is shaded.

 [Return to Figure 8.3](#)

## Figure 8.4 Text Alternative (Chapter 8)

---

 [Return to Figure 8.4](#)

A two-by-two matrix: Row-1: 5 and 9, row- 2: 7 and 2 is multiplied with two-by three matrix as: row-1: 2 and 1, row-2: 8 and 6, and row-3: 0 and 4. Elements in row-1 column 1 of two-by two matrix is multiplied with row-1 column 1 of two-by three matrix. It is further added to Row-2 column 1 of two-by two matrix, which is multiplied by row-1 column 2 of two-by three matrix. This leads to matrix with first element calculated as: 3 times 5 plus 1 times 7 equals 22.

 [Return to Figure 8.4](#)

## Figure 8.5 Text Alternative (Chapter 8)

---

 [Return to Figure 8.5](#)

The interior dimensions are equal, multiplication is possible while the exterior dimensions define the dimensions of the result.

 [Return to Figure 8.5](#)

## Figure 8.7 Text Alternative (Chapter 8)

---

[!\[\]\(d263118e0bfd47dc6bc704167d936b83\_img.jpg\) Return to Figure 8.7](#)

- A) A set of four fluid tanks connected by pipes.
- B) A triangular truss section supported at two nodes with an external force on the third.
- C) A two mesh circuit with a source and several resistors.
- D) A car with the tires modeled by springs dampers and a force applied on the front bumper.

[!\[\]\(3d8c13c92b853674f749aac6fa869926\_img.jpg\) Return to Figure 8.7](#)



## Figure 8.8 Text Alternative (Chapter 8)

---

 [Return to Figure 8.8](#)

Node 1 is at top right and is equal to 200 volts. Node 1 is connected to node 2 through a 5 ohm resistor, while node 2 is also connected to node 3 with a 10 ohm resistor and node 5 with a 10 ohm resistor. Node 3 is also connected to node 4 with a 5 ohm resistor. Node 4 is also connected to node 5 with a 15 ohm resistor. Finally, node 5 is also connected to node 6 with a 20 ohm resistor. Node 6 is at 0 volts.

 [Return to Figure 8.8](#)

## Figure 8.9 Text Alternative (Chapter 8)

---

 [Return to Figure 8.9](#)

The current through the left mesh is 1.5835 amperes and the current through the right mesh is 6.1538 amperes (both are measured counter-clockwise). The voltages (in volts) labeled counterclockwise and are given by  $v_1$  equals 200,  $v_2$  equals 169.23,  $v_3$  equals 153.85,  $v_4$  equals 146.15,  $v_5$  equals 123.08, and  $v_6$  equals 0.

 [Return to Figure 8.9](#)

## Figure P8.9 Text Alternative (Chapter 8)

---

 [Return to Figure P8.9](#)

Each pipe's flow rate is labeled as  $Q$  with a subscript indicating the source tank and the target tank.

Each tank is labeled with a concentration  $c$  with a subscript indicating the tank.

The tanks are connected as follows:

Tank  $c_1$  is connected to an external inlet pipe with a flow rate  $Q_{01}$  equals 5 at a concentration  $c_{01}$  equals 10, an outlet pipe  $Q_{12}$  equals 3, an inlet pipe  $Q_{31}$  equals 1, and an outlet pipe  $Q_{15}$  equals 3.

Tank  $c_2$  is connected to an inlet pipe  $Q_{13}$  equals 3, an outlet pipe  $Q_{23}$  equals 1, an outlet pipe  $Q_{24}$  equals 1, and an outlet pipe  $Q_{25}$  equals 1.

Tank  $c_3$  is connected to an external inlet pipe with a flow rate  $Q_{03}$  equals 8 and a concentration  $c_{03}$  equals 20, outlet pipe  $Q_{31}$  equals 1, an inlet pipe  $Q_{23}$  equals 1, and an outlet pipe  $Q_{34}$  equals 8.

Tank  $c_4$  is connected to an external outlet pipe  $Q_{44}$  equals 11, an inlet pipe  $Q_{24}$  equals 1, an inlet pipe  $Q_{34}$  equals 8, and an inlet pipe  $Q_{54}$  equals 2.

Tank  $c_5$  is connected to an external outlet pipe  $Q_{55}$  equals 2, an inlet pipe  $Q_{15}$  equals 3, an inlet pipe  $Q_{25}$  equals 1, and an outlet pipe  $Q_{54}$  equals 2.

 [Return to Figure P8.9](#)

## Figure P8.10 Text Alternative (Chapter 8)

---

 [Return to Figure P8.10](#)

The angles are labeled counter-clockwise as 1, 2, and 3 and measure 90, 30 and 60 degrees respectively.

At 1, 1000 Newton force is applied downwards; while at 2 force  $H_2$  acts from left to right and force  $V_2$  acts upwards; and at 3 force  $V_3$  acts upwards. The side between 1 and 2 is labeled  $F_1$ , between 2 and 3 is labeled  $F_2$ , and between 3 and 1 is labeled  $F_3$ .

 [Return to Figure P8.10](#)

## Figure P8.11 Text Alternative (Chapter 8)

---

 [Return to Figure P8.11](#)

The springs are labeled, from left to right as  $k_1$  through  $k_4$ . The masses between the springs are labeled, from left to right as  $m_1$  through  $m_3$ . The positions of the masses are given as  $x_1$  through  $x_3$ .

 [Return to Figure P8.11](#)

## Figure P8.14 Text Alternative (Chapter 8)

---

 [Return to Figure P8.14](#)

Node 1 is at top right and is equal to 150 volts. Node 1 is connected to node 2 through a 10 ohm resistor. Node 2 is also connected to node 3 with a 20 ohm resistor and node 5 with a 5 ohm resistor. Node 3 is also connected to node 4 with a 2 ohm resistor. Node 4 is also connected to node 5 with a 5 ohm resistor. Finally, node 5 is also connected to node 6 with a 25 ohm resistor. Node 6 is at 0 volts.

 [Return to Figure P8.14](#)

## Figure P8.15 Text Alternative (Chapter 8)

---

 [Return to Figure P8.15](#)

Node 1 is at top right and is equal to 20 volts. Node 1 is connected to node 2 through a 35 ohm resistor. Node 2 is also connected to node 3 with a 30 ohm resistor and node 5 with a 10 ohm resistor. Node 3 is also connected to node 4 with an 8 ohm resistor and node 5 with a 7 ohm resistor. Node 4 is also connected to node 5 with a 15 ohm resistor. Finally, node 5 is also connected to node 6 with a 5 ohm resistor. Node 6 is at 140 volts.

 [Return to Figure P8.15](#)

# Figure 9.1 Text Alternative (Chapter 9)

---

[!\[\]\(d263118e0bfd47dc6bc704167d936b83\_img.jpg\) Return to Figure 9.1](#)

The horizontal axis represents  $x_1$  ranging from 0 through 6 in increments of 2, while vertical axis represents  $x_2$  ranging from 0 through 8 in increments of 2.

The data is as follows:

The curve  $3x_1 + 2x_2 = 18$  decreases from (0, 9) to (5.1, 1) while curve  $-x_1 + 2x_2 = 2$  increases from (0, 1) to (5.9, 3.9) intersecting each other at solution:  $x_1$  equals 4;  $x_2$  equals 3.

Note: All data is approximate.

[!\[\]\(fa6f3af6bfa46c5d4a2d362681095beb\_img.jpg\) Return to Figure 9.1](#)



## Figure 9.2 Text Alternative (Chapter 9)

---

 [Return to Figure 9.2](#)

A) Graph shows two different parallel lines  $-\frac{1}{2}x + 2 = 1$  and  $-\frac{1}{2}x + 1 = \frac{1}{2}$  which never intersect.

B) Graph shows lines,  $-\frac{1}{2}x + 2 = 1$  and  $-\frac{1}{2}x + 2 = 2$ , which always intersect.

C) Graph shows two lines,  $-\frac{23}{5}x + 2 = 1.1$  and  $-\frac{1}{2}x + 1 = 1$ , intersecting multiple times.

 [Return to Figure 9.2](#)

## Figure 9.3 Text Alternative (Chapter 9)

---

 [Return to Figure 9.3](#)

A) The forward elimination phase gets rid of the first  $k-1$  coefficients in the  $k$ th row of the matrix of  $a_{11}$  through  $a_{13}$ ,  $a_{21}$  through  $a_{23}$ , and  $a_{31}$  through  $a_{33}$  and last column for  $b_1$  through  $b_3$ ; it thus forms an upper triangular coefficient matrix of  $a_{11}$  through  $a_{13}$ ,  $a_{22}$  through  $a_{23}$ , and  $a_{33}$  and last column for  $b_1$ ,  $b'_2$ ,  $b''_3$ .

B) The back substitution first solves the equation represented in the last row, which only has a single non-zero coefficient. The back substitution then proceeds up the rows of the matrix since the equation above the equation just solved will now also only have one unknown.

The three lines read:

$x_3$  equals  $b''_3$  over  $a''_{33}$ ;  $x_2$  equals  $(b'_2 \text{ minus } a'_{23} x_3)$  over  $a'_{22}$ ; and  $x_1$  equals  $(b_1 \text{ minus } a_{13} x_2 \text{ minus } a_{12} x_2)$  over  $a_{11}$ .

 [Return to Figure 9.3](#)

## Figure 9.5 Text Alternative (Chapter 9)

---

[!\[\]\(1d3a1175dd4902218e694b9c098adb83\_img.jpg\) Return to Figure 9.5](#)

Typical mathematical representation:

An  $n$  cross  $n$  matrix with elements  $a_{11}$  through  $a_{nn}$  is multiplied with a  $n1$  cross  $n$  matrix with elements  $x_1$  through  $x_n$  equals to  $1$  cross  $n$  matrix with elements  $b_1$  through  $b_n$ .

Equivalent representation with python subscript:

An  $n - 1$  cross  $n - 1$  matrix with elements  $a_{(0,0)}$  through  $a_{(n - 1, n - 1)}$  is multiplied with a  $1$  cross  $n - 1$  matrix with elements  $x_{(0)}$  through  $x_{(n - 1)}$  equals to  $1$  cross  $n - 1$  matrix with elements  $b_{(0)}$  through  $b_{(n - 1)}$ .

[!\[\]\(cbe2492b119e39e02a1dab2af4a4b296\_img.jpg\) Return to Figure 9.5](#)

# Figure 9.8 Text Alternative (Chapter 9)

---

 [Return to Figure 9.8](#)

There are six total nodes as:

A node at the left wall at  $T_0$  equals 40 degrees, four evenly spaced interior nodes, and a node at the right wall at  $T_5$  equals 200 degrees. Heat flow is noted as going from the right wall to the left wall and also from the rod to the air. The temperature at top and bottom is  $T_a$  equals 20. The space between 1 and 2 is labeled as  $\Delta x$ .

 [Return to Figure 9.8](#)

## Figure 9.9 Text Alternative (Chapter 9)

---

 [Return to Figure 9.9](#)

The horizontal axis ranges from 0 through 10 in increments of 2, while vertical axis ranges from 40 through 200 in increments of 20.

The data is as follows:

The curve increases from (0, 40) to (10, 200) through (2, 65), (4, 95), (6, 125), and (8, 160).

Note: All data is approximate.

 [Return to Figure 9.9](#)

## Figure P9.9 Text Alternative (Chapter 9)

---

 [Return to Figure P9.9](#)

Each pipe is labeled with the product of the flow rate in the pipe multiplied by the concentration in the tank from which the fluid flows.

Each pipe's flow rate is labeled as  $Q$  with a subscript indicating the source tank and the target tank.

The concentration is labeled  $c$  with a subscript indicating the tank.

The tanks are connected as follows:

Tank 1 is connected to an external inlet pipe receiving 500 milligrams per second, an outlet pipe  $Q_{12} c_1$ , an inlet pipe  $Q_{21} c_2$ , and an outlet pipe  $Q_{13} c_1$ .

Tank 2 is connected to an inlet pipe  $Q_{12} c_1$ , an outlet pipe  $Q_{21} c_2$ , and an outlet pipe  $Q_{23} c_2$ .

Tank 3 is connected to an inlet pipe  $Q_{13} c_1$ , an inlet pipe  $Q_{23} c_2$ , an external outlet pipe  $Q_{33} c_3$ , and an external inlet pipe receiving 500 milligrams per second.

The chart at bottom right reads as follows:

$Q_{33}$  equals 120,  $Q_{13}$  equals 40,  $Q_{12}$  equals 90,  $Q_{23}$  equals 60, and  $Q_{21}$  equals 30.

 [Return to Figure P9.9](#)

## Figure P9.12 Text Alternative (Chapter 9)

---

 [Return to Figure P9.12](#)

Gear pump with Q 0 leads into the structure with valves through 1, where nodes 2 and 3 contains valve 2 through 4; and nodes 3 and 5 contains valve 3 through 5. This all leads to node 6, further leading to node 7.

 [Return to Figure P9.12](#)

## Figure P9.13 Text Alternative (Chapter 9)

---

 [Return to Figure P9.13](#)

Each stage has a weight fraction of chemical entering from the left and leaving to the right; these are labeled  $y$  with the subscript indicating which tank the chemical leaves from.

$y_{in}$  leads to 1, yielding  $y_1$  where  $y_1$  leads to 2, yielding  $y_2$  where  $y_2$  continues, while  $y_{i-1}$  leads to  $i$  yielding  $y_i$  which continues, and  $y_{n-2}$  leads to  $n-1$  yielding  $y_{n-1}$ ,  $y_{n-1}$  leads to  $n$  yielding  $y_{out}$ .

Each stage also has a weight fraction of solvent entering from the right and leaving to the left; these are labeled  $x$  with the subscript indicating which tank the solvent leaves from.

$x_{in}$  leads to  $n$ , yielding  $x_n$  where  $x_n$  leads to  $n-1$  yielding  $x_{n-1}$  which continues, while  $x_{i+1}$  leads to  $i$  yielding  $x_i$  which continues, and  $x_3$  leads to 2 yielding  $x_2$ ,  $x_2$  leads to 1 yielding  $x_{out}$ .

 [Return to Figure P9.13](#)



## Figure P9.15 Text Alternative (Chapter 9)

---

 [Return to Figure P9.15](#)

There is a 74 kilo-Newton force to the right applied horizontally to the top left of the truss at B and a 24 kilo-Newton force down applied to the top right node of the truss at C. The distance between A and B is 4 meters, A and E is 6 meters where D is the midpoint.

 [Return to Figure P9.15](#)

# Figure 10.1 Text Alternative (Chapter 10)

---

 [Return to Figure 10.1](#)

A) Factorization:

It consists of taking the coefficient matrix  $A$  and breaking it up into the product of an upper triangular matrix  $U$  and a lower triangular matrix  $L$  where  $(x)$  equals  $(b)$ .

B) Forward substitution:

It solves the equation where the lower triangular matrix  $L$  is multiplied by an intermediate vector of unknowns  $d$  to yield the constant vector  $b$ .

C) Back substitution:

The upper triangular matrix  $U$  multiplied by the original vector of unknowns  $x$  should yield  $d$ ; thus finding  $x$ .

 [Return to Figure 10.1](#)

# Figure 11.1 Text Alternative (Chapter 11)

---

 [Return to Figure 11.1](#)

The distance from origin to the point is given by; double mod of F e equals to the square root of the sum of squares of a, b, and c coordinates.

 [Return to Figure 11.1](#)

## Figure 11.2 Text Alternative (Chapter 11)

---

 [Return to Figure 11.2](#)

Flow rates given below are in meters cubed per hour and concentrations are given in milligrams per cubic meter.

The room labeled 1 is the smoking section; it is connected to the half of the elongated room labeled 3. The carbon monoxide level in the room changes as a result of four processes: air coming in from outside at a rate of  $Q_a$  equals 200 at a concentration of 2, carbon monoxide being added at a rate of 1000 milligrams per hour from smokers, air being exchanged between rooms 1 and 3 at a rate of 25 and air leaving room 1 to get to room 3. The room labeled 3 is the bottom half of the elongated room. The carbon monoxide level in the room changes as a result of 5 processes: air coming in from room 1 at an unlabeled rate and a concentration of  $c_1$ , air being exchanged between rooms 1 and 3 at a rate of 25, the faulty grill contributing 2000 milligrams per hour, air being exchanged between rooms 3 and 4 (the top half of the elongated room) at a rate of 50, and air leaving room 3 to enter room 4 at an unlabeled rate that must also equal  $Q_a$  with a concentration of  $c_3$ . The other half of the elongated room, room 4, has its carbon monoxide level changed in five ways: air coming in from room 3 at an unlabeled rate and a concentration of  $c_3$ , air being exchanged between rooms 4 and 3 at a rate of 50, air being exchanged between rooms 4 and the kids section (labeled room 2) at a rate of 25, and the total air leaving the room at a concentration of  $c_4$ . This total amount of air leaving the room includes a rate  $Q_d$  of 100 leaving the building and an unlabeled rate leaving room 4 to get to room 2. Finally, room 2's concentration changes as a result of four processes: air coming in from the outside at a rate of  $Q_b$  equals 50 at a concentration  $c_b$  of 2, air leaving the room at a rate of  $Q_c$  equals 150 and a concentration  $c_2$ , air exchanged between rooms 2 and 4 at a rate of 25, and the air coming in from room 4 with an unlabeled rate that must equal  $Q_a$  minus  $Q_d$  at a concentration of  $c_3$ .

 [Return to Figure 11.2](#)

# Figure P11.12 Text Alternative (Chapter 11)

---

 [Return to Figure P11.12](#)

The lakes are connected as follows:

Lake Powell with concentration  $c_1$ , receives water from the Upper Colorado River, and drains into Lake Mead. Lake Mead receives water from Lake Powell, with concentration  $c_2$ , and drains into Lake Mohave. Lake Mohave receives water from Lake Mead, with concentration  $c_3$ , and drains into Lake Havasu. Lake Havasu receives water from Lake Mohave, with concentration  $c_4$ , and drains to an unlabeled reservoir.

 [Return to Figure P11.12](#)

# Figure P11.15 Text Alternative (Chapter 11)

---

 [Return to Figure P11.15](#)

Volumes given below are in cubic meters, flow rates are in cubic meters per minute, and concentrations are given in grams per cubic meter.

The tanks are connected as follows:

Tank 1 has a volume  $V_1$  of 100. Fluid is flowing into tank 1 from an external reservoir at a rate of  $Q_1$  in equals 100 and a concentration of  $c_1$  in of 10. Fluid is also flowing into tank 1 from tank 2 at a rate of  $Q_{21}$  equals 22 and flowing out to tank 2 at a rate of  $Q_{12}$  equals 5. Fluid is flowing out from tank 1 into tank 3 at a rate of  $Q_{13}$  equals 117.

Tank 2 has a volume  $V_2$  of 50. Fluid is flowing into tank 2 from an external reservoir at a rate of  $Q_2$  in equals 10 and a concentration of  $c_2$  in equals 200. In addition to the previously mentioned connections with tank 1, fluid is flowing into tank 2 from tank 3 at a rate of 7.

Tank 3 has a volume  $V_3$  of 150. In addition to the connections between tank 3 and the other tanks mentioned above, fluid is flowing out of tank 3 at a rate  $Q_3$  out equals 110.

 [Return to Figure P11.15](#)

# Figure P11.18 Text Alternative (Chapter 11)

---

 [Return to Figure P11.18](#)

Flow rates given below are in meters cubed per hour and concentrations are given in milligrams per cubic meter.

The room labeled 1 is connected to rooms 2 and 3. The carbon monoxide level in the room changes as a result of four processes: air coming in from outside at a rate of 150 at a concentration of 1, air leaving room 1 and entering room 2 at a rate of 50, air being exchanged between rooms 1 and 3 at a rate of 50, and air leaving room 1 and entering room 3 at an unlabeled rate.

The room labeled 2 is connected to rooms 1 and 3. The carbon monoxide level in the room changes as a result of four processes: air coming in from the outside at a rate of 50 and a concentration of 40, air entering room 2 from room 1 at a rate of 50, air being exchanged between rooms 2 and 3 at a rate of 50, and air leaving room 2 and entering room 3 at an unlabeled rate.

The room labeled 3 is connected to rooms 1, 2, and 4. The carbon monoxide level changes as a result of seven processes: air leaves room 3 at a rate of 50, air is exchanged with room 1 at a rate of 50, air is exchanged with room 2 at a rate of 50, air is exchanged with room 4 at a rate of 90, air enters room 3 from rooms 1 and 2 at unlabeled rates, and air leaves room 3 for room 4 at an unlabeled rate.

The room labeled 4 is connected to room 3. The carbon monoxide level in room 4 changes as a result of four processes: carbon monoxide enters at 5000 milligrams per hour from the outside, air is exchanged with room 3 at a rate of 90, air enters room 4 from room 3 at an unlabeled rate, and air leaves the building from room 4 at an unlabeled rate.

 [Return to Figure P11.18](#)

# Figure P11.20 Text Alternative (Chapter 11)

---

[!\[\]\(d263118e0bfd47dc6bc704167d936b83\_img.jpg\) Return to Figure P11.20](#)

Joint 2 is pinned at the far left, joint 3 is pinned at the far right, and joint 1 is above. The members at joint 2 make a 30 degree angle, the members at joint 3 make a 60 degree angle, and the members at joint 1 make a 90 degree angle (joint 1 is thus to the right of the center of the truss). A downward external load of 1000 Newton is applied at node 1.

$F_1$  is between joints 1 and 2,  $F_2$  is between joints 2 and 3, and  $F_3$  is between 1 and 3. The reaction at joint 2 is given as a horizontal force to the right  $H_2$  and a vertical force up  $V_2$ . The reaction at joint 3 is given as a vertical force  $V_3$ .

[!\[\]\(6605b201d6f14d9b3bcb8ab5f274d107\_img.jpg\) Return to Figure P11.20](#)



## Figure P11.21 Text Alternative (Chapter 11)

---

 [Return to Figure P11.21](#)

It is a 30 to 45 to 105 triangle, with a member extending from the top node and connecting to the bottom of the triangle to form a 45 degree angle in opposition to the 45 degree angle on the right side of the truss. An isosceles right triangle with the 90 degree angle at the top joint as one part of the truss is visible and, to its left another triangle with far left member forms a 30 degree angle with the base. The load at the top joint is 600 downwards and 1200 rightwards. The third exterior load is given at the joint that forms the bottom left of the isosceles triangle and is 500 downwards.

 [Return to Figure P11.21](#)

# Figure 12.1 Text Alternative (Chapter 12)

---

[!\[\]\(529949c2c3dadbaa4e538e8c643454bc\_img.jpg\) Return to Figure 12.1](#)

A) First iteration:

$x_1$  equals  $(b_1 \text{ minus } a_{12} \text{ times } 2 \text{ minus } a_{13} \text{ times } 3) \text{ over } a_{11}$ , leads to  $x_2$  equals  $(b_2 \text{ minus } a_{21} \text{ times } 1 \text{ minus } a_{23} \text{ times } 3) \text{ over } a_{22}$  finally leading to  $x_3$  equals  $(b_3 \text{ minus } a_{31} \text{ times } 1 \text{ minus } a_{32} \text{ times } 2) \text{ over } a_{33}$ .

Second iteration:

$x_1$  equals  $(b_1 \text{ minus } a_{12} \text{ times } 2 \text{ minus } a_{13} \text{ times } 3) \text{ over } a_{11}$ , further leading to  $x_2$  equals  $(b_2 \text{ minus } a_{21} \text{ times } 1 \text{ minus } a_{23} \text{ times } 3) \text{ over } a_{22}$  finally leading to  $x_3$  equals  $(b_3 \text{ minus } a_{31} \text{ times } 1 \text{ minus } a_{32} \text{ times } 2) \text{ over } a_{33}$ .

B) First iteration:

$x_1$  equals  $(b_1 \text{ minus } a_{12} \text{ times } 2 \text{ minus } a_{13} \text{ times } 3) \text{ over } a_{11}$ , leads to  $x_2$  equals  $(b_2 \text{ minus } a_{21} \text{ times } 1 \text{ minus } a_{23} \text{ times } 3) \text{ over } a_{22}$  finally leading to  $x_3$  equals  $(b_3 \text{ minus } a_{31} \text{ times } 1 \text{ minus } a_{32} \text{ times } 2) \text{ over } a_{33}$ .  $x_1$ ,  $x_2$ , and  $x_3$  is highlighted.

Second iteration:

$x_1$  equals  $(b_1 \text{ minus } a_{12} \text{ times } 2 \text{ minus } a_{13} \text{ times } 3) \text{ over } a_{11}$  (where  $x_1$ ,  $x_2$ ,  $x_3$  leads to it), further leading to  $x_2$  equals  $(b_2 \text{ minus } a_{21} \text{ times } 1 \text{ minus } a_{23} \text{ times } 3) \text{ over } a_{22}$  finally leading to  $x_3$  equals  $(b_3 \text{ minus } a_{31} \text{ times } 1 \text{ minus } a_{32} \text{ times } 2) \text{ over } a_{33}$ .

[!\[\]\(c50c8b7b2cc2cf9ff925edec0ee94c0d\_img.jpg\) Return to Figure 12.1](#)

## Figure 12.3 Text Alternative (Chapter 12)

---

 [Return to Figure 12.3](#)

The horizontal axis represents  $x_1$  ranging from 0 to 3 in increments of 1, while vertical axis represents  $x_2$  ranging from 0 to 8 in increments of 2.

The data is as follows:

The curve  $x_2$  plus 3 times  $x_1$   $x_2$  squared equals 57 decreases from (0.35, 8) to (2.85, 2.5) while curve  $x_1$  squared plus  $x_1$  times  $x_2$  equals 10 intersecting at solution  $x_1$  equals 2,  $x_2$  equals 3.

Note: All data is approximate.

 [Return to Figure 12.3](#)

## Figure P12.10 Text Alternative (Chapter 12)

---

 [Return to Figure P12.10](#)

The gas flows from left to right at a rate of  $Q_G$ ; this rate is consistent as the gas flows from tank to tank and finally exits tank 5 at far right. Gas enters the leftmost reactor at a concentration of  $c_{G0}$  and has concentrations in each tank of  $c_{G1}$  through  $c_{G5}$  respectively.

The fluid flows from right to left at a rate of  $Q_L$ ; this rate is also consistent as the fluid flows from tank to tank and finally exits tank 1 at far left. Fluid enters the rightmost reactor at a concentration of  $c_{L6}$  and has concentrations of  $c_{L5}$  through  $c_{L1}$  going right to left.

The interaction between the fluid and the gas in each reactor is labeled  $D$ , which is the gas-liquid exchange rate.

 [Return to Figure P12.10](#)

## Figure P12.11 Text Alternative (Chapter 12)

---

 [Return to Figure P12.11](#)

The temperature of both labeled top nodes is 25 degrees Celsius, the temperature of both labeled right nodes is 0 degrees Celsius, the temperature of both labeled bottom nodes is 75 degrees Celsius, and the temperature of both labeled left nodes is 100 degrees Celsius. For the unknown temperatures the top left interior node is T 12, the top right interior node is T 22, the bottom left interior node is T 11, and the bottom right interior node is T 21.

 [Return to Figure P12.11](#)

# Figure 13.1 Text Alternative (Chapter 13)

---

[!\[\]\(d0a1791f26d167e866e44ebbf83efebe\_img.jpg\) Return to Figure 13.1](#)

A) Left panel:

The horizontal axis represents time ranging from

0 through 100 in increments of 20, while vertical axis represents position (in meters) ranging from 20 through 100 in increments of 10.

The data is as follows:

x1: The curve begins at (0, 20) and ends at (100, 51) with approximately equal amplitude throughout.

x2: The curve begins at (0, 30) and ends at (100, 62) with approximately equal amplitude throughout.

x3: The curve begins at (0, 60) and ends at (100, 89) with approximately equal amplitude throughout.

B) Right panel:

The horizontal axis represents time ranging from 0 through 100 in increments of 20, while vertical axis represents velocity (in meters per second) ranging from negative 20 through 20 in increments of 5.

The data is as follows:

v1: The curve begins at (0, 0) and ends at (100, 0) with approximately equal amplitude throughout.

v2: The curve begins at (0, 0) and ends at (100, 0) with approximately equal amplitude throughout.

v3: The curve begins at (0, 60) and ends at (100, negative 3) with approximately equal amplitude throughout.

Note: All data is approximate.

[!\[\]\(5abce1a84a655b073239ab33e1199487\_img.jpg\) Return to Figure 13.1](#)

## Figure 13.2 Text Alternative (Chapter 13)

---

 [Return to Figure 13.2](#)

A horizontal line represents real axis. A vertical line represents imaginary axis. First quadrant shows arrow directed from left to right as faster divergent which is unstable, representing a complex plane. The second quadrant shows plane directed from right to left as faster convergent which is stable. Third quadrant shows an arrow pointing at imaginary axis labeled as pure oscillations. Fourth quadrant shows location of eigenvalues with an arrow labeled as no oscillations, pointing at real axis.

 [Return to Figure 13.2](#)

# Graph of T Against Y Graphic Text Alternative (Chapter 13)

---

[!\[\]\(d263118e0bfd47dc6bc704167d936b83\_img.jpg\) Return to Graph of T Against Y Graphic](#)

The horizontal axis represents  $t$  ranging from 0 through 5 in increments of 1, while vertical axis represents  $y$  ranging from negative 1.0 through 1.0 in increments of 0.5.

The data is as follows:

y1: The curve begins at (0, 1.0) and ends at (5, 1.1) with approximately equal amplitude throughout but touching the peak at (2.5, 1.45).

y2: The curve begins at (0, negative 1.0) and ends at (5, 0.6) with approximately equal amplitude throughout but touching the peak at (3.6, 0.85).

Note: All data is approximate.

[!\[\]\(17acf1afa8cdf0b67c53d4865a5ed469\_img.jpg\) Return to Graph of T Against Y Graphic](#)



## Figure 13.3 Text Alternative (Chapter 13)

---

 [Return to Figure 13.3](#)

A) The configuration shows masses  $m_1$  and  $m_2$  at their equilibrium positions within distance of 0 to  $x$ .

B) The configuration shows masses  $m_1$  and  $m_2$  moved to the right, resulting in distances from 0 to  $x_1$  and 0 to  $x_2$  given as the position of each mass relative to its own equilibrium position.

 [Return to Figure 13.3](#)

## Figure 13.4 Text Alternative (Chapter 13)

---

 [Return to Figure 13.4](#)

A) The first mode has the masses oscillating in opposition to each other, with the positions of the masses being displaced in opposite directions. The period of this mode,  $T_r$  is 1.62

B) The second mode has the masses oscillating together in the same direction. The period of this mode,  $T_r$  is 2.81.

 [Return to Figure 13.4](#)

## Figure 13.5 Text Alternative (Chapter 13)

---

 [Return to Figure 13.5](#)

The story at top has a mass  $m_3$  equals 8000 kilograms and is connected to the story below by a spring  $k_3$  equals 1800 kilo-Newton per meter.

The story at middle has a mass  $m_2$  of 10000 kilograms and is connected to the story above by  $k_3$  and the story below by  $k_2$  equals 2400 kilo-Newton per meter.

The story at bottom has a mass  $m_1$  of 12000 kilograms and is connected to the story above by  $k_2$  and to the ground by spring  $k_1$  equals 3000 kilo-Newton per meter.

 [Return to Figure 13.5](#)

## Figure 13.6 Text Alternative (Chapter 13)

---

 [Return to Figure 13.6](#)

The horizontal axis represents time (in seconds) ranging from 0.0 through 1.0 in increments of 0.2, while vertical axis represents relative position ranging from negative 3 through 3 in increments of 1.

The data is as follows:

y1:

The curve begins at (0.0, 1.7) and ends at (1.0, 2.1) with approximately equal amplitude throughout.

y2:

The curve begins at (0.0, negative 2.1) and ends at (1.0, negative 2.7) with approximately equal amplitude throughout.

y3:

The curve begins at (0.0, 1) and ends at (1.0, 1.4) with approximately equal amplitude throughout.

Note: All data is approximate.

 [Return to Figure 13.6](#)

# Figure 13.7 Text Alternative (Chapter 13)

---

 [Return to Figure 13.7](#)

1.20 hertz:

The horizontal axis represents  $t$  ranging from 0.0 through 1.0 in increments of 0.2, while vertical axis represents  $x$  ranging from negative 0.4 through 0.4 in increments of 0.2.

The data is as follows:

The curve begins at (0.0, 0.39) and ends at (1.0, 0.5) with approximately equal amplitude throughout and showing increasing trend after (0.61, negative 0.4).

2.93 hertz:

The horizontal axis represents  $t$  ranging from 0.0 through 1.0 in increments of 0.2, while vertical axis represents  $x$  ranging from negative 1.0 through 1.0 in increments of 0.0.

The data is as follows:

The curve begins at (0.0, negative 0.9) and ends at (1.0, negative 0.51) with approximately equal amplitude throughout and showing increasing trend after (0.73, negative 1.7).

4.21 hertz:

The horizontal axis represents  $t$  ranging from 0.0 through 1.0 in increments of 0.2, while vertical axis represents  $x$  ranging from negative 2 through 2 in increments of 1.

The data is as follows:

The curve begins at (0.0, 1.7) and ends at (1.0, 2.1) with approximately equal amplitude throughout and showing increasing trend after (0.83, negative 2.4).

Note: All data is approximate.

 [Return to Figure 13.7](#)

## Figure P13.5 Text Alternative (Chapter 13)

---

 [Return to Figure P13.5](#)

The springs are labeled, from left to right as  $k_1$  through  $k_4$ . The masses between the springs are labeled from left to right as  $m_1$  through  $m_3$ . The positions of the masses are given as  $x_1$  through  $x_3$  labeled relative to the equilibrium position of each mass.

 [Return to Figure P13.5](#)

## Figure P13.6 Text Alternative (Chapter 13)

---

 [Return to Figure P13.6](#)

The springs are all labeled as  $k$ . The masses are labeled from left to right as  $m_1$  through  $m_3$ . The positions of the masses are given as  $x_1$  through  $x_3$  and are labeled relative to the equilibrium position of each mass.

 [Return to Figure P13.6](#)

## Figure P13.7 Text Alternative (Chapter 13)

---

 [Return to Figure P13.7](#)

First drum with liquid of height  $h_1$  with an inlet  $q_0$ . The outlet of this drum flows into second drum of height  $h_2$  with an inlet  $q_1$ . The outlet of this drum flows into third drum of height  $h_3$  with an inlet  $q_2$ . The outlet of this drum further flows as  $q_3$ .

Height of  $h_1$  is greater than  $h_2$ , which is greater than  $h_3$ .

 [Return to Figure P13.7](#)



## Figure P13.8 Text Alternative (Chapter 13)

---

 [Return to Figure P13.8](#)

The circuit shows five nodes. The top left node is connected to ground via a voltage source. Inductors connect the top nodes from left to right, and capacitors connect the second, third, and fourth nodes to ground. The mesh currents (where currents are directed clockwise) from left to right are labeled as  $i_1$ ,  $i_2$ , and  $i_3$ .

 [Return to Figure P13.8](#)

## Figure P13.12 Text Alternative (Chapter 13)

---

 [Return to Figure P13.12](#)

A) The  $x$ -axis is plotted downwards along the original axis of the rod while the  $y$  axis is plotted to the right. The rod begins at  $(0, 0)$  and ends at  $(L, 0)$  while force  $P$  is applied downwards at the top of the rod and upwards at the bottom of the rod.

B) The top third of the rod is visible. This section shows a force  $P$  applied at the top and bottom, while moment  $M$  counterclockwise is applied to the bottom of this partial section.

 [Return to Figure P13.12](#)

## Figure P13.14 Text Alternative (Chapter 13)

---

 [Return to Figure P13.14](#)

Superior (1) and Michigan (2) flow into Huron (3), while it flows into Erie (4). Erie (4) further flows into Ontario (5), and it flows out of it to an unknown location.

 [Return to Figure P13.14](#)

# Figure PT14.1 Text Alternative (Chapter 14)

---

 [Return to Figure PT14.1](#)

A) Graph shows an upward sloping line passing through three points, two points are bit higher and lower than the regression.

B) Graph shows four different straight line segments creating a pattern of V between two upward sloping segments.

C) Graph shows four different curved line segments creating a pattern of U between two upward sloping segments.

 [Return to Figure PT14.1](#)

## Figure 14.1 Text Alternative (Chapter 14)

---

 [Return to Figure 14.1](#)

The wind blows past the person directed from left to right.

 [Return to Figure 14.1](#)

## Figure 14.2 Text Alternative (Chapter 14)

---

 [Return to Figure 14.2](#)

The horizontal axis represents  $v$  ranging from 0 through 80 in increments of 20, while the vertical axis represents  $F$  ranging from 0 through 1600 in increments of 400.

The dots are plotted at following points:

(10, 0), (20, 100), (30, 400), (40, 550), (50, 600), (60, 1300), (70, 900), and (80, 1400).

Note: All data is approximate.

 [Return to Figure 14.2](#)

# Scatter Plot Graphic Text Alternative (Chapter 14)

---

 [Return to Scatter Plot Graphic](#)

The horizontal axis represents sample number ranging from 0 through 25 in increments of 5, while the vertical axis represents coefficient of expansion ranging from 6.40 through 6.80 in increments of 0.05.

The dots are plotted at following points:

0 to 5: (1, 6.49), (2.4, 6.60), (3, 6.62), and (4, 6.64).

5 to 10: (5, 6.48), (6, 6.56), (7, 6.665), (8, 6.50), and (9, 6.44).

10 to 15: (10, 6.625), (11, 6.72), (12, 6.65), (13, 6.75), and (14, 6.625).

15 to 20: (16, 6.75), (17, 6.65), (17, 6.60), and (18, 6.56).

20 to 25: (22, 6.40), (23, 6.77), and (23.5, 6.68).

Note: All data is approximate.

 [Return to Scatter Plot Graphic](#)

# Histogram of 8 Graphic Text Alternative (Chapter 14)

---

[!\[\]\(d263118e0bfd47dc6bc704167d936b83\_img.jpg\) Return to Histogram of 8 Graphic](#)

The horizontal axis ranges from 6.40 through 6.75 in increments of 0.05, while the vertical axis ranges from 0 through 5 in increments of 1.

The data is as follows:

6.40 to 6.44: 2, 6.441 to 6.49: 1, 6.491 to 6.55: 3, 6.55 to 6.50: 4, 6.59 to 6.64: 5, 6.641 to 6.68: 4, 6.681 to 6.73: 3, and 6.731 to 6.79: 2.

Note: All data is approximate.

[!\[\]\(fa6f3af6bfa46c5d4a2d362681095beb\_img.jpg\) Return to Histogram of 8 Graphic](#)



# Set of 3 Histograms Graphic Text Alternative (Chapter 14)

---

[!\[\]\(c8d96c8885d3000a912c2582004aed63\_img.jpg\) Return to Set of 3 Histograms Graphic](#)

A) The horizontal axis ranges from 6.40 through 6.75 in increments of 0.05, while the vertical axis ranges from 0 through 8 in increments of 2.

The data is as follows:

6.40 to 6.49: 3, 6.491 to 6.57: 7, 6.571 to 6.66: 9, and 6.661 to 6.75: 5.

B) The horizontal axis ranges from 6.40 through 6.75 in increments of 0.05, while the vertical axis ranges from 0 through 8 in increments of 2.

The data is as follows:

6.40 to 6.44: 2, 6.441 to 6.49: 1, 6.491 to 6.54: 3, 6.541 to 6.59: 4, 6.591 to 6.64: 5, 6.641 to 6.65: 4, 6.65 to 6.71: 2, and 6.71 to 6.76: 2.

C) The horizontal axis ranges from 6.40 through 6.75 in increments of 0.05, while the vertical axis ranges from 0.0 through 3.0 in increments of 0.5.

The data is as follows:

6.40 to 6.44: 1.0, 6.46 to 6.52: 2.0, 6.55 to 6.57: 3.0, 6.571 to 6.61: 2.0, 6.61 to 6.65: 3.0, 6.611 to 6.71: 2.0, and 6.75 to 6.77: 20.

Note: All data is approximate.

[!\[\]\(4b7a79268f6ba26c1471d4232fffa85a\_img.jpg\) Return to Set of 3 Histograms Graphic](#)

# Graph of Z Graphic Text Alternative (Chapter 14)

---

[!\[\]\(529949c2c3dadbaa4e538e8c643454bc\_img.jpg\) Return to Graph of Z Graphic](#)

The horizontal axis represents  $z$  ranging from negative 4 through 4 in increments of 1, while the vertical axis represents  $f$  of  $z$  ranging from 0.00 to 0.40 in increments of 0.05.

The data is as follows:

The curve begins at (negative 4, 00) remaining constant till negative 2, which further increases to peak (4, 0.00) through (negative 1, 0.15). It further decreases to (2, 0.00) passing through (1, 0.15), and remaining constant till (4, 0.00)

Note: All data is approximate.

[!\[\]\(a870788d6ed9b8fd294b7654a8c8526b\_img.jpg\) Return to Graph of Z Graphic](#)

## Figure 14.3 Text Alternative (Chapter 14)

---

 [Return to Figure 14.3](#)

The horizontal axis represents coefficient of thermal expansion of structural steel ranging from 6.39 to 6.44 through 6.74 to 6.79, while the vertical axis represents frequency ranging from 0 to 7 in increments of 1.

The data is as follows:

The curve increases from (6.415, 0.9) to (6.76.5, 0.9) through (6.59, 4.8).

6.39 to 6.44: 2, 6.44 to 6.49: 1, 6.49 to 6.54: 3, 6.54 to 6.59 : 4, 6.59 to 6.64: 6, 6.64 to 6.69: 4, 6.69 to 6.74: 2, and 6.74 to 6.79: 2.

Note: All data is approximate.

 [Return to Figure 14.3](#)

# Histogram of 8 Graphic Text Alternative (Chapter 14)

---

[!\[\]\(21199eb166cc97331a0c54c649195dcc\_img.jpg\) Return to Histogram of 8 Graphic](#)

The horizontal axis ranges from 6.40 through 6.80 in increments of 0.05, while the vertical axis ranges from 0 through 6 in increments of 1.

The data is as follows:

6.39 to 6.44: 2, 6.441 to 6.49: 1, 6.491 to 6.54: 3, 6.541 to 6.59: 4, 6.591 to 6.64: 6, 6.64 to 6.69: 4, 6.691 to 6.74: 2, and 6.74 to 6.79: 2.

Note: All data is approximate.

[!\[\]\(c694a3ff3b077d76910920a6a1593ab4\_img.jpg\) Return to Histogram of 8 Graphic](#)

# Histogram of 8 Graphic Text Alternative (Chapter 14)

---

[!\[\]\(8af806fb1314382d09bc5ec5b767526c\_img.jpg\) Return to Histogram of 8 Graphic](#)

The horizontal axis ranges from 6.40 through 6.80 in increments of 0.05, while the vertical axis ranges from 0 through 6 in increments of 1.

The data is as follows:

6.39 to 6.44: 2, 6.441 to 6.49: 1, 6.491 to 6.54: 3, 6.541 to 6.59: 4, 6.591 to 6.64: 6, 6.64 to 6.69: 4, 6.691 to 6.74: 2, and 6.74 to 6.79: 2.

The curve begins at (6.39, 0.5) and ends at (6.79, 0.7) passing through (6.59, 5).

Note: All data is approximate.

[!\[\]\(830769b31eeeaca920791081939ff8ba\_img.jpg\) Return to Histogram of 8 Graphic](#)

# A Shaded Histogram Graphic Text Alternative (Chapter 14)

---

[!\[\]\(d0a1791f26d167e866e44ebbf83efebe\_img.jpg\) Return to A Shaded Histogram Graphic](#)

The horizontal axis ranges from 6.40 through 6.80 in increments of 0.05, while the vertical axis ranges from 0 through 6 in increments of 1.

The data is as follows:

6.39 to 6.44: 2, 6.441 to 6.49: 1, 6.491 to 6.54: 3, 6.541 to 6.59: 4, 6.591 to 6.64: 6, 6.64 to 6.69: 4, 6.691 to 6.74: 2, and 6.74 to 6.79: 2.

The curve begins at (6.39, 0.5) and ends at (6.79, 0.7) passing through (6.59, 5).

Note: All data is approximate.

[!\[\]\(10f8862fc183b400327470ea85afe9ae\_img.jpg\) Return to A Shaded Histogram Graphic](#)

## Figure 14.4 Text Alternative (Chapter 14)

---

 [Return to Figure 14.4](#)

A) Distribution of drag coefficient:

The horizontal axis represents coefficient (kilograms per meter) ranging from 0 through 0.27 in increments of 0.01, while the vertical axis ranges from 0 through 120 in increments of 20.

The data is as follows:

0.225 to 0.23: 101, 0.23 to 0.235: 90, 0.235 to 0.24: 95, 0.24 to 0.245: 92, 0.245 to 0.25: 101, 0.25 to 0.255: 103, 0.255 to 0.26: 99, 0.26 to 0.265: 109, 0.265 to 0.27: 95, and 0.27 to 0.275: 120.

B) Distribution of velocity:

The horizontal axis represents velocity (meters per second) ranging from 33.1 through 33.6 in increments of 0.1, while the vertical axis ranges from 0 through 60 in increments of 20.

The data is as follows:

33.11 to 33.15: 56, 33.15 to 33.21: 48, 33.21 to 33.26: 39, 33.26 to 33.33: 55, 33.33 to 33.36: 38, 33.36 to 33.43: 58, 33.43 to 33.46: 48, 33.46 to 33.53: 40, 33.53 to 33.57: 51, and 33.57 to 33.60: 50.

Note: All data is approximate.

 [Return to Figure 14.4](#)

## Figure 14.5 Text Alternative (Chapter 14)

---

[!\[\]\(d263118e0bfd47dc6bc704167d936b83\_img.jpg\) Return to Figure 14.5](#)

A) Distribution of drag coefficient:

The horizontal axis represents coefficient (kilograms per meter) ranging from 0.20 through 0.30 in increments of 0.02, while the vertical axis ranges from 0 through 250 in increments of 50.

The data is as follows:

0.19 to 0.21: 0, 0.21 to 0.22: 20, 0.22 to 0.23: 65, 0.23 to 0.24: 165, 0.24 to 0.25: 230, 0.25 to 0.26: 260, 0.26 to 0.27: 190, 0.27 to 0.28: 20, and 0.29 to 0.30: 1.

B) Distribution of velocity:

The horizontal axis represents velocity (meters per second) ranging from 32.4 through 34.0 in increments of 0.2, while the vertical axis ranges from 0 through 250 in increments of 50.

The data is as follows:

32.1 to 32.4: 7, 32.4 to 32.6: 25, 32.6 to 32.8: 110, 32.8 to 33.0: 251, 33.0 to 33.2: 280, 33.2 to 33.4: 200, 33.4 to 33.6: 100, 33.6 to 33.8: 25, 33.8 to 34.0: 1, and 34.0 to 34.2: 0.

Note: All data is approximate.

[!\[\]\(d8ab143e904bfa3467271eec5af75a9b\_img.jpg\) Return to Figure 14.5](#)



## Figure 14.6 Text Alternative (Chapter 14)

---

[!\[\]\(c8d96c8885d3000a912c2582004aed63\_img.jpg\) Return to Figure 14.6](#)

Top panel:

A fit is drawn through two points. If merely the sum of the residuals is used, then positive errors could offset negative errors. A second line is shown that is far above the first point and equally far below the last point; this line would also have a zero sum for the residual errors but is clearly wrong.

Middle panel:

Graph shows four points, two fairly close to each other on the left and two fairly close to each other on the right. If the sum of the absolute residuals were used, this graph shows two possible different models that would each have the same sum of the absolutes of the residuals with no way of picking a best one.

Bottom panel:

Graph shows four points on a straight line with an additional point in between the four points but with a y value far below the line connecting the other four points. If a model puts too much emphasis on the outlier, the model might be pulled further away from the collection of four points than desired.

[!\[\]\(17413706fd4997a1a4bdf85c6864eee1\_img.jpg\) Return to Figure 14.6](#)

## Figure 14.7 Text Alternative (Chapter 14)

---

[!\[\]\(529949c2c3dadbaa4e538e8c643454bc\_img.jpg\) Return to Figure 14.7](#)

The horizontal axis represents  $v$  ranging from 0 through 80 in increments of 20, while the vertical axis represents  $F$  ranging from negative 400 through 1600 in increments of 400.

The dots are plotted at following points:

(10, 0), (20, 100), (30, 400), (40, 550), (50, 600), (60, 1300), (70, 900), and (80, 1400).

The curve increases from (0, negative 200) to (80, 1300) through (30, 400), and (40, 500).

Note: All data is approximate.

[!\[\]\(de95854c7ee024cfadc48187bbb781b2\_img.jpg\) Return to Figure 14.7](#)

# Scatter Plot Graphic Text Alternative (Chapter 14)

---

 [Return to Scatter Plot Graphic](#)

The horizontal axis represents velocity ranging from 0 through 90 in increments of 10, while the vertical axis represents force ranging from negative 200 through 1600 in increments of 200.

The dots are plotted at following points:

Sketched line:

The curve increases from (negative 100, 12) to (81, 1450) through (20, 70) and (70, 1210) (encircled).

Linear regression line:

The curve increases from (10, negative 1) to (80, 1330) through (30, 400) and (70, 1150).

The squares are marked at:

(11, 1), (20, 100), (30, 400), (40, 550), (50, 600), (60, 1210), (70, 810), and (80, 1420).

Note: All data is approximate.

 [Return to Scatter Plot Graphic](#)

## Figure 14.9 Text Alternative (Chapter 14)

---

 [Return to Figure 14.9](#)

A) straight line represents the mean of the data (the residuals are large and negative at the beginning and large and positive at the end). The C-shaped curve on right has slightly lesser amplitude.

B) An increasing line shows a positive slope (this fit is collectively much closer to all the points). The C-shaped curve on right has slightly greater amplitude.

Fourth data point is a solid dot.

 [Return to Figure 14.9](#)

# Scatter Plot Graphic Text Alternative (Chapter 14)

---

 [Return to Scatter Plot Graphic](#)

The horizontal axis represents  $x$  ranging from 0 through 80 in increments of 10, while the vertical axis represents  $e$  ranging from negative 300 through 300 in increments of 100.

The data is as follows:

The curve begins at (10, 70) and ends at (80, 120) passing through (20, negative 110), (30, 20), (40, 0), (50, negative 120), (60, 300), and (70, negative 300).

Note: All data is approximate.

 [Return to Scatter Plot Graphic](#)

# Figure 14.10 Text Alternative (Chapter 14)

---

 [Return to Figure 14.10](#)

Top left panel:

The horizontal axis ranges from 0 to 20 in increments of 5, while vertical axis ranges from 0 to 15 in increments of 5.

The data is as follows:

The best fit curve increases from (0, 4) to (20, 14) through (15, 10).

The dots are closer and follow the line with some small random variation.

Top right panel:

The horizontal axis ranges from 0 to 20 in increments of 5, while vertical axis ranges from 0 to 15 in increments of 5.

The data is as follows:

The best fit curve increases from (0, 4) to (20, 14) through (15, 10).

The dots are visible in a concave-down parabola.

Bottom left panel:

The horizontal axis ranges from 0 to 20 in increments of 5, while vertical axis ranges from 0 to 15 in increments of 5.

The data is as follows:

The best fit curve increases from (0, 4) to (20, 14) through (15, 10).

The dots are visible in a straight line across the curve with last two dots slightly higher and lower than the curve.

Bottom right panel:

The horizontal axis ranges from 0 to 20 in increments of 5, while vertical axis ranges from 0 to 15 in increments of 5.

The data is as follows:

The best fit curve increases from (0, 4) to (20, 14) through (15, 10).

The dots are visible with nearly the same independent value but with dependent values that increase very quickly over that small domain - the last point is far to the right and higher.

 [Return to Figure 14.10](#)

## Figure 14.11 Text Alternative (Chapter 14)

---

 [Return to Figure 14.11](#)

The residual error variance is denoted by square of  $S_e$ , while intercept variance is denoted by square of  $S_{a0}$ , and slope variance is denoted by square of  $S_{a1}$ .

 [Return to Figure 14.11](#)

## Figure 14.12 Text Alternative (Chapter 14)

---

 [Return to Figure 14.12](#)

A) The horizontal axis represent  $x$ , while vertical axis represents  $y$ . The curve  $y$  equals  $\alpha_1 e$  raised to power  $\beta_1$  times  $1/x$  is upward sloping concave up in nature.

After linearization:

D) The horizontal axis represent  $x$ , while vertical axis represents  $\ln y$ . The upward sloping curve shows slope as  $\beta_1$  and intercept as  $\ln \alpha_1$ .

B) The horizontal axis represent  $x$ , while vertical axis represents  $y$ . The curve  $y$  equals  $\alpha_2 e$  raised to power  $\beta_2$  times  $2$  is upward sloping concave up in nature.

After linearization:

E) The horizontal axis represent  $\log x$ , while vertical axis represents  $\ln \log y$ . The upward sloping curve shows slope as  $\beta_2$  and intercept as  $\ln \alpha_2$ .

C) The horizontal axis represent  $x$ , while vertical axis represents  $y$ . The curve  $y$  equals  $\alpha_3 e$  raised to power  $x$  over  $\beta_3$  plus  $x$  is upward sloping concave down in nature.

After linearization:

F) The horizontal axis represent  $1/x$ , while vertical axis represents  $\ln 1/y$ . The upward sloping curve shows slope as  $\beta_3$  over  $\alpha_3$  and intercept as  $1/\alpha_3$ .

 [Return to Figure 14.12](#)



## Figure 14.13 Text Alternative (Chapter 14)

---

[!\[\]\(c8d96c8885d3000a912c2582004aed63\_img.jpg\) Return to Figure 14.13](#)

A) The horizontal axis represents  $\log$  of 10 of  $x$  ranging from 1.0 through 2.0 in increments of 0.1, while the vertical axis represents  $\log$  of 10 of  $y$  ranging from 1.0 through 3.5 in increments of 0.5.

The data is as follows:

The best fit curve increases from (1.0, 1.4) to (1.9, 3.2) through (1.5, 2.4).

The data points are at:

(1.0, 1.4), (1.3, 1.35), (1.48, 2.6), (1.6, 2.75), (1.7, 2.355), (1.78, 3.1), (1.84, 2.42), and (1.9, 3.2).

The three dots are visible above the best fit curve, while two dots are visible below it, and one dot is visible on it.

B) The horizontal axis represents  $x$  ranging from 0 through 90 in increments of 10, while the vertical axis represents  $y$  ranging from 0 through 1800 in increments of 200.

The data is as follows:

The best fit curve increases from (10, 250) to (80, 1610) through (50, 610).

The data points are at:

(10, 25), (20, 40), (30, 400), (40, 580), (50, 600), (60, 1200), (70, 800), and (80, 1420).

The three dots are visible above the best fit curve, while two dots are visible below it, and two dots are visible on it.

Note: All data is approximate.

[!\[\]\(4f6bf54ae7e4144a72d78316053e412d\_img.jpg\) Return to Figure 14.13](#)

## Figure 14.14 Text Alternative (Chapter 14)

---

 [Return to Figure 14.14](#)

A) The horizontal axis represents predicted force (in Newton) ranging from 0 through 1800 in increments of 200, while the vertical axis represents residual (in Newton) ranging from negative 500 through 400 in increments of 100.

The data is as follows:

The best fit curve begins at (0, 0) and ends at (1650, negative 200) through (100, negative 40), (210, 150), (400, 150), (650, negative 30), (910, 300), and (150, negative 410).

B) The horizontal axis represents log of 10 of y prediction ranging from 1.0 through 3.5 in increments of 0.5, while the vertical axis represents residual error ranging from negative 0.2 through 0.3 (with double values).

The data is as follows:

The best fit curve begins at (1.negative 0.5, 0) and ends at (3.2, negative 0.1) through (2.1, between negative 0.2), (2.4, 0.22), 92.65, 0.15), (2.35, negative 0.05), (2.9, 0.14), and (3.15, (between negative 0.2).

Note: All data is approximate.

 [Return to Figure 14.14](#)

# Set of 3 Plots Graphic Text Alternative (Chapter 14)

---

 [Return to Set of 3 Plots Graphic](#)

A) The horizontal axis represents  $x$  ranging from 0 through 80 in increments of 20, while the vertical axis represents  $y$  ranging from negative 250 through 1500 in increments of 250.

The data is as follows:

The best fit curve on the scatter plot begins at (0, negative 250) and ends at (89, 1510).

The data points are at:

(10, 0), (20, 10), (30, 270), (40, 510), (50, 610), (60, 1250), (70, 800), and (80, 1450).

The two dots are visible above the best fit curve, while three dots are visible below it, and three dots are visible on it.

B) The horizontal axis represents residual ranging from negative 300 through 300 in increments of 100, while the vertical axis ranges from 0.0 through 4.0.

The data is as follows:

negative 300 to negative 100: 2.0, negative 100 to 100: 4.0, and 100 to 300: 2.0.

C) The horizontal axis represents predicted  $y$  ranging from negative 0 through 1200 in increments of 100, while the vertical axis ranges from negative 300 through 300.

The data is as follows:

The curve begins from (negative 150, 85) and ends at (1250, 120) passing through data points (180, negative 110), (350, 20), (530, 10), (720, negative 120), and (90, 300).

Note: All data is approximate.

 [Return to Set of 3 Plots Graphic](#)

# Scatter Plot Graphic Text Alternative (Chapter 14)

---

 [Return to Scatter Plot Graphic](#)

The horizontal axis represents log of 10 of x ranging from 1.0 through 2.0 in increments of 0.2, while the vertical axis represents log of 10 of y ranging from 1.5 through 3.5 in increments of 0.5.

The data is as follows:

The best fit curve begins at (0.4, 1.15) and ends at (2.0, 3.4).

The data points are at:

(0.9, 1.4), (1.3, 1.8), (1.47, 2.6), (1.6, 2.75), (1.78, 3.1), (1.85, 2.4), and (1.9, 3.2).

The three dots are visible above the best fit curve, while two dots are visible below it, and three dots are visible on it.

Note: All data is approximate.

 [Return to Scatter Plot Graphic](#)

# Graph of X Against Y Graphic Text Alternative (Chapter 14)

---

 [Return to Graph of X Against Y Graphic](#)

The horizontal axis represents  $x$  ranging from 0 through 100 in increments of 20, while the vertical axis represents  $y$  ranging from negative 250 through 1750 in increments of 250.

The data is as follows:

The curve begins at (0, negative 250) and ends at (100, 1745).

through (20, 230), (40, 510), and (80, 1260).

Note: All data is approximate.

 [Return to Graph of X Against Y Graphic](#)

## Figure 14.16 Text Alternative (Chapter 14)

---

 [Return to Figure 14.16](#)

A dashed line from  $V_m$  runs parallel to the horizontal axis. The dashed lines from  $0.5 v_m$  and  $K_s$  intersect each other at  $(K_s, 0.5 V_m)$ .

 [Return to Figure 14.16](#)

## Figure 14.17 Text Alternative (Chapter 14)

---

 [Return to Figure 14.17](#)

A) The horizontal axis represents  $s$  ranging from 0 through 10 in increments of 2, while the vertical axis represents  $v$  ranging from negative 0.1 through 0.5 in increments of 0.1.

The data is as follows:

The best fit curve begins at (1, 0.05) and ends at (10, 0.8) through (6, 0.34).

The data points are at:

(1.2, 0.06), (1.9, 0.13), (3, 0.21), (4.3, 0.27), (6, 0.34), (8, 0.35), and (9, 0.36).

The two dots are visible above the best fit curve, while two dots are visible below it, and three dots are visible on it.

B) The horizontal axis represents fitted value of  $v$  ranging from 0.1 through 0.5 in increments of 0.1, while the vertical axis represents residual ranging from negative 0.125 through 0.050 in increments of 0.125.

The data is as follows:

The curve decreases from (0.08, negative 0.015) to (0.5, negative 0.130) passing through (0.11, 0.025), (0.26, 0.012), (0.39, negative 0.05), and (0.45, negative 0.100).

Note: All data is approximate.

 [Return to Figure 14.17](#)

## Figure 14.18 Text Alternative (Chapter 14)

---

 [Return to Figure 14.18](#)

A) The horizontal axis represents  $s$  ranging from 0 through 10 in increments of 2, while the vertical axis represents  $v$  ranging from negative 0.05 through 0.35 in increments of 0.05.

The data is as follows:

The best fit curve begins at (1, 0.049) and ends at (10, 0.37) through (4, 0.27).

The data points are at:

(1.2, 0.07), (1.9, 0.135), (3, 0.225), (4.3, 0.27), (6, 0.34), (8, 0.35), and (9, 0.36).

There are no dots that are visible above the best fit curve, while three dots are visible below it, and four dots are visible on it.

B) The horizontal axis represents fitted value of  $v$  ranging from 0.10 through 0.35 in increments of 0.05, while the vertical axis represents residual ranging from negative 0.020 through 0.010 in increments of 0.005.

The data is as follows:

The curve decreases from (0.08, negative 0.025) to (0.38, negative 0.125) passing through (0.12, 0.011), (0.26, 0.0025), (0.29, negative 0.018), and (0.37, negative 0.013).

Note: All data is approximate.

 [Return to Figure 14.18](#)



## Figure P14.39 Text Alternative (Chapter 14)

---

 [Return to Figure P14.39](#)

The contour lines have values (from left to right) of 0, negative 10, and negative 20.

 [Return to Figure P14.39](#)

## Figure P14.40 Text Alternative (Chapter 14)

---

 [Return to Figure P14.40](#)

At time 0, the person is at  $x$  equals 0. In the first time step, the person either goes left or right by  $\Delta x$ . In the next time step, the person can go left or right from each of those positions. This is repeated once more, such that there are four possible locations for the person:  $3\Delta x$  to the left or right and  $\Delta x$  to the left or right.

A distribution curve shows that it is more likely for a person to be  $\Delta x$  away than  $3\Delta x$  away.

 [Return to Figure P14.40](#)

## Figure P14.41 Text Alternative (Chapter 14)

---

 [Return to Figure P14.41](#)

At time 0, the person is at the origin. One time step later, the person has moved a distance  $\Delta$  at an angle  $\theta$  from the x axis. The new x location is  $\Delta \cos \theta$  and the new y location is  $\Delta \sin \theta$ . The graph shows the person taking two more steps, each in different directions.

 [Return to Figure P14.41](#)

## Figure 15.1 Text Alternative (Chapter 15)

---

 [Return to Figure 15.1](#)

A) The dots show a concave-down trend, with the straight-line fit is upward sloping. The data points at the left and right are below while line and the points in the middle are above. There are a number of 17 data points across.

B) The dots show a concave-down trend where the parabolic fit is much closer to all the points collectively. There are a number of 16 data points across.

Note: All data is approximate.

 [Return to Figure 15.1](#)

## Figure 15.3 Text Alternative (Chapter 15)

---

 [Return to Figure 15.3](#)

The seven points are shown in the form of a model plane, where four points are above the plane and three are below the plane.

 [Return to Figure 15.3](#)

## Figure 15.4 Text Alternative (Chapter 15)

---

[!\[\]\(d0a1791f26d167e866e44ebbf83efebe\_img.jpg\) Return to Figure 15.4](#)

The horizontal axis represents wavelength (in microns) ranging from 0.35 through 0.70 in increments of 0.05, while the vertical axis on left represents refractive index ranging from 1.506 through 1.528 in increments of 0.002

The data is as follows:

Model:

The best fit curve decreases from (0.39, 0.00004) and ends at (0.665, 0.00001).

Data:

The data points are at:

(0.40, 1.5255), (0.43, 1.5215), (0.48, 1.517), (0.51, 1.5155), (0.53, 1.514), (0.58, 1.511), (0.69, 1.5096), and (0.66, 1.509).

Model:

The best fit curve decreases from (0.40, 1.5255) and ends at (0.66, 1.509).

Residual:

The data points are at:

90.40, 0.000005), (0.44, 0.00003), (0.48, 0.00001), (0.51, 0.00001), (0.53, 0.00001), (0.59, 0.00004), (0.64, 0.000045), and (0.66, 0.000007),

Note: All data is approximate.

[!\[\]\(b538fe54c1f3a7343e37e85cc2d00497\_img.jpg\) Return to Figure 15.4](#)

# Set of 2 Scatter Graphic Text Alternative (Chapter 15)

---

[!\[\]\(35e4f762fc1cfea5610d92e2d225d5b4\_img.jpg\) Return to Set of 2 Scatter Graphic](#)

Left panel:

The horizontal axis represents temperature (in degree Celsius) ranging from 0 through 80 in increments of 20, while the vertical axis represents viscosity (in centipoise) ranging from 0.4 through 1.8 in increments of 0.2.

The data is as follows:

The best fit curve decreases from (0, 1.7) to (90, 0.4).

The data points are at:

(0, 1.8), (5, 1.55), (10, 1.4), (15, 1.15), (20, 1.0), (25, 0.85), (30, 0.8), (35, 0.65), (49, 0.59), (60, 0.45), (70, 0.4), (81, 0.38), and (88, 0.2).

The three dots are visible above the best fit curve, while three dots are visible below it, and six dots are visible on it.

Right panel:

The horizontal axis represents predicted viscosity (in centipoise) ranging from 0.4 through 1.6 in increments of 0.2, while the vertical axis represents residual (in centipoise) ranging from negative 0.075 through 0.125 in increments of 0.025.

The data is as follows:

The curve begins at (0.4, negative 0.075), and passes through (0.2, 0.0020), (0.2, 0.065), (0.4, 0.0075), (0.5, 0.0049), (0.7, 0.000), (0.8, negative 0.025), (0.9, negative 0.050), (1.05, negative 0.060), (1.2, negative 0.060), (1.35, negative 0.051), (1.51, 0.0020), and (1.7, 0.112).

Note: All data is approximate.

[!\[\]\(274fd520e03b61c1b9ffc861754cacdc\_img.jpg\) Return to Set of 2 Scatter Graphic](#)

# Set of 2 Scatter Graphic Text Alternative (Chapter 15)

---

[!\[\]\(d263118e0bfd47dc6bc704167d936b83\_img.jpg\) Return to Set of 2 Scatter Graphic](#)

Left panel:

The horizontal axis represents temperature (in degree Celsius) ranging from 0 through 80 in increments of 20, while the vertical axis represents viscosity (in centipoise) ranging from 0.4 through 1.8 in increments of 0.2.

The data is as follows:

The best fit curve decreases from (0, 1.8) to (90, 0.2) through (5, 1.55), (10, 1.3), (15, 1.1), (20, 1.0), (25, 0.9), (26, 0.8), (29, 0.7), (49, 0.6), (60, 0.45), (70, 0.4), and (81, 0.38).

Right panel:

The horizontal axis represents predicted viscosity (in centipoise) ranging from 0.4 through 1.8 in increments of 0.2, while the vertical axis represents residual (in centipoise) ranging from negative 0.006 through 0.004 in increments of 0.002.

The data is as follows:

The curve begins at (0.2, 0.0001) and ends at (1.8, 0.004), passing through (0.3, negative 0.0029), (0.4, 0.002), (0.45, 0.0025), (0.58, negative 0.0018), (0.7, negative 0.0038), (0.79, negative 0.001), (0.85, 0.0012), (0.9, 0.0035), 91.6, (0.0031), (1.3, negative 0.0025), and (1.55, negative 0.0058).

Note: All data is approximate.

[!\[\]\(d8ab143e904bfa3467271eec5af75a9b\_img.jpg\) Return to Set of 2 Scatter Graphic](#)



# Set of 2 Scatter Graphic Text Alternative (Chapter 15)

---

 [Return to Set of 2 Scatter Graphic](#)

Left panel: The horizontal axis represents temperature (in degree Celsius) ranging from 50 through 300 in increments of 50, while the vertical axis represents vapor pressure ranging from 0 through 14000 in increments of 2000.

The data is as follows:

Model: The dotted curve increases from (25, 0) to (300, 14000) through (100,0), (150, 0), (250, 4000), and (280, 10000).

Data: The dotted curve increases from (25, 0) to (280, 1000) through (100,0), (150, 0), (200, 0) and (250, 0).

Right panel: The horizontal axis represents temperature (in degree Celsius) ranging from 50 through 300 in increments of 50, while the vertical axis represents log of 10 (VP) ranging from negative 3 through 4 in increments of 1.

The data is as follows:

Model: The dotted curve increases from (25, negative 0.5) to (300, 4.1) through (50, 0), (140, 2) and (200, 3).

Data: The dotted curve increases from (25, negative 3) to (280, 3) through (50, negative 2), (150, 0.5), and (250, 2.3).

Note: All data is approximate.

 [Return to Set of 2 Scatter Graphic](#)

# Set of 3 Scatter Graphic Text Alternative (Chapter 15)

---

[!\[\]\(529949c2c3dadbaa4e538e8c643454bc\_img.jpg\) Return to Set of 3 Scatter Graphic](#)

Top left panel: The horizontal axis represents temperature (in degree Celsius) ranging from 50 through 300 in increments of 50, while the vertical axis represents log of 10 (VP) ranging from negative 3 through 3 in increments of 1.

The data is as follows:

The dotted best fit curve increases from (25, negative 3) to (290, 3) through (100, negative 0.5), (150, 0.5), and (250, 2.3).

Top right panel: The horizontal axis represents temperature (in degree Celsius) ranging from 50 through 300 in increments of 50, while the vertical axis represents vapor pressure ranging from 0 through 1000 in increments of 200.

The data is as follows:

The dotted curve increases from (25, negative 3) to (300, 3) through (100,0), (150, 0), (250, 200), and (270, 400).

Bottom panel: The horizontal axis represents predicted log of 10 (VP) ranging from negative 3 through 3 in increments of 1, while the vertical axis represents residual ranging from negative 0.0075 through 0.0100 in increments of 0.0025.

The data is as follows:

The curve begins at (negative 2.8, 0.0000) and ends at (2.8, 0.0112) showing increasing trend after (negative 2.7, negative 0.0055), (negative 0.9, negative 0.0030), and (2.5, negative 0.00625).

Note: All data is approximate.

[!\[\]\(9c2e8d1b5bd77cb5c9f83b7a9cff79fd\_img.jpg\) Return to Set of 3 Scatter Graphic](#)

# Scatter Plot Graphic Text Alternative (Chapter 15)

---

[!\[\]\(1d3a1175dd4902218e694b9c098adb83\_img.jpg\) Return to Scatter Plot Graphic](#)

The horizontal axis represents predicted log KL values ranging from negative 0.0 through 2.0 in increments of 0.5, while the vertical axis represents residual error ranging from negative 0.100 through 0.100 in increments of 0.025.

The data points are at:

(0.05, negative 0.0010), (0.06, 0.090), (0.09, negative 0.050), (0.65, negative 0.40), (0.75, negative 0.080), (0.8, 0.0060), (1.7, 0.030), (1.8, 0.0015), and (2.0, negative 0.025).

Note: All data is approximate.

[!\[\]\(cbe2492b119e39e02a1dab2af4a4b296\_img.jpg\) Return to Scatter Plot Graphic](#)

## Figure 15.6 Text Alternative (Chapter 15)

---

 [Return to Figure 15.6](#)

Left panel:

The horizontal axis represents measured log KL ranging from negative 0.5 through 2.0 in increments of 0.5, while the vertical axis represents predicted log KL ranging from negative 0.5 through 2.0 in increments of 0.5.

The data is as follows:

The dotted best fit curve increases from (negative 0.4, negative 0.4) to (2.2, 2.2) through (1.0, 1.0).

The data sets are at:

(negative 0.3, negative 0.3), (negative 0.4, negative 0.4), (negative 0.35, negative 0.35), (0.6, 0.6), (0.7, 0.7), (0.9, 0.9), (1.75, 1.75), (1.8, 1.8), and (1.99, 1.99).

There are no dots that are visible above the best fit curve, while two dots are visible below it, and seven dots are visible on it.

Right panel:

The horizontal axis represents measured KL ranging from negative 0 through 100 in increments of 20, while the vertical axis represents predicted KL ranging from 0 through 100 in increments of 20.

The data is as follows:

The dotted best fit curve increases from (0, 0) to 100, 100) through (60, 60).

The data sets are at:

(0, 0), (5, 5), (6, 6), (10, 10), (58, 55), (78, 77), and (90, 95).

There is only one dot that is visible above the best fit curve, while two dots are visible below it, and four dots are visible on it.

Note: All data is approximate.

 [Return to Figure 15.6](#)

## Figure 16.1 Text Alternative (Chapter 16)

---

 [Return to Figure 16.1](#)

A) Graph shows a periodic square wave that remains at a positive constant for half a period and at a negative constant for half a period.

B) Graph shows a sinusoid of the same amplitude as the square wave, but with some small-amplitude noise superimposed on it.

 [Return to Figure 16.1](#)

## Figure 16.2 Text Alternative (Chapter 16)

---

 [Return to Figure 16.2](#)

A) The horizontal axis represents  $t$ ,  $s$  ranging from 0 through 2 (and  $\omega t$ , rad, ranging from 0 through  $3\pi$ ), while vertical axis represents  $y$  of  $t$  with distance across as  $\theta$ , ranging from 0 through 2. The sinusoid has an average  $A_0$  above the time axis. The maximum of the sinusoid is  $C_1$  higher than  $A_0$  and the minimum is  $C_1$  below it. The period  $T$  is shown as the time between where the sinusoid crosses through its average value on the way to its maximum to the next time the sinusoid crosses through its average value on the way to its maximum.

B) Graph shows three functions that compose the top graph, a constant at a value of  $A_0$  from  $(0, 1.7)$ , a pure sine at an amplitude of  $B_1$  from  $(0, 4.5)$ , and a frequency of  $\omega_0$ , and a pure cosine at an amplitude of  $A_1$  and a frequency of  $\omega_0$  from  $(0, 0)$ .

 [Return to Figure 16.2](#)

## Figure 16.3 Text Alternative (Chapter 16)

---

 [Return to Figure 16.3](#)

A) Graph shows two functions; a pure cosine and a cosine with a phase angle of negative  $\pi/2$ : the latter's peak is shifted a quarter period to the right.

B) Graph also shows two functions; a pure cosine and a cosine with a phase angle of  $\pi/2$ : the latter's peak is shifted a quarter period to the left.

 [Return to Figure 16.3](#)

## Figure 16.4 Text Alternative (Chapter 16)

---

 [Return to Figure 16.4](#)

A) The first term is a single frequency cosine with an amplitude of  $4/\pi$  and a period the same as the square wave. This representation has a value that is too high in the middle of the positive part of the square wave and too low at the middle of the negative part. It is also too low at the edges of the positive part and too high at the edges of the negative part.

B) The second term is a pure sinusoid at a frequency three times that of the first term and an amplitude of  $4/3\pi$ . The addition of this term improves on the first term by bringing the approximation down at the middle of the positive part and at the edges of the negative parts while also bringing the approximation up at the edges of the positive part and the middle of the negative part.

C) The third term has a frequency of five times the first term and an amplitude of  $4/5\pi$ . The approximation with these three terms even more slowly resembles the square wave.

 [Return to Figure 16.4](#)



## Figure 16.5 Text Alternative (Chapter 16)

---

 [Return to Figure 16.5](#)

A point on the circle,  $e$  raised to power  $I$  times  $\theta$  equals  $\cos \theta + i \sin \theta$ , is labeled at an angle  $\theta$  from the  $x$  (real) axis. The real part is thus  $\cos(\theta)$  while the imaginary part is  $\sin(\theta)$ .

 [Return to Figure 16.5](#)

## Figure 16.6 Text Alternative (Chapter 16)

---

 [Return to Figure 16.6](#)

B) The horizontal axis represents  $t$  while vertical axis represents  $f$  of  $t$ . A sinusoid as a function of time is visible with amplitude  $C = 1$  and time period  $T$ .

A) The horizontal axis represents  $t$  while vertical axis represents  $f$  of  $t$ , while another across the plane axis represents  $f$ . A three-dimensional view of two sinusoidal curves are visible at a distance of  $1$  over  $T$ .

C) Graph shows a discrete amplitude of  $C = 1$  at a frequency of  $1$  over  $T$ .

D) Graph shows a sinusoid as a function of time and a discrete phase of  $\pi$  over  $2$  at a frequency of  $1$  over  $T$ .

 [Return to Figure 16.6](#)

## Figure 16.7 Text Alternative (Chapter 16)

---

 [Return to Figure 16.7](#)

The graph on left shows first peak as a half period to the right of the origin; while the phase is shown as  $\pi$ . The graph on right shows a long vertical curve across the x plane.

The graph on left shows the curve with their peak moving to the right. The graph on right shows a shorter vertical curve across the x plane.

The graph on left shows the curve with their peak moving further to the right. The graph on right shows a dot between the x and negative x plane.

The graph on left shows the curve with their peak moving further to the right. The graph on right shows a shorter vertical curve across the negative x plane.

The graph on left shows the curve with their peak moving to the right reaching a half period to the right of the origin. The graph on right shows a long vertical curve across the negative x plane.

The phase graphs similarly decrease in value from  $\pi$ , to a value of 0 when the peak is at the origin, to a value of negative  $\pi$  when the peak is half a period to the right.

 [Return to Figure 16.7](#)

## Figure 16.8 Text Alternative (Chapter 16)

---

 [Return to Figure 16.8](#)

A) The amplitudes decreases from  $f_0$  at  $4\pi$ , to  $7f_0$  at less than  $2\pi$  through  $3f_0$  at  $2\pi$  over  $\pi$ , and  $5f_0$  at less than  $2\pi$  over  $\pi$ .

B) The amplitudes vary from  $f_0$  at equilibrium to  $7f_0$ , at  $\pi$  through  $3f_0$  at  $\pi$ , and  $5f_0$  at equilibrium.

 [Return to Figure 16.8](#)

## Figure 16.9 Text Alternative (Chapter 16)

---

 [Return to Figure 16.9](#)

The time at the origin is 0 and the value of the curve is  $f_0$ . From left to right, the individual data points are evenly spaced by a distance  $\Delta t$ . The times are labeled  $t_1$  through  $t_{n-1}$  and the function values are labeled  $f_1$  through  $f_{n-1}$ , where  $f_0$  through  $f_3$  are visible across the concave down curve on left and three points ending in  $f_{n-1}$  are visible across the concave up curve on right.

 [Return to Figure 16.9](#)

## Figure 16.10 Text Alternative (Chapter 16)

---

 [Return to Figure 16.10](#)

The horizontal axis represents samples ranging from 0 to 40, while vertical axis represents operations ranging from 0 through 2000 in increments 1000.

The data is as follows:

The curve  $DFT(n^2)$  increases from (0, 0) beyond (40, 2100).

The curve  $FFT(n \log_2 n)$  increases from (0,0) beyond (40, 250).

Note: All the approximate.

 [Return to Figure 16.10](#)

## Figure 16.11 Text Alternative (Chapter 16)

---

[!\[\]\(35e4f762fc1cfea5610d92e2d225d5b4\_img.jpg\) Return to Figure 16.11](#)

A) The horizontal axis represents  $t$  ranging from 0.00 to 0.14, while vertical axis represents  $f$  of  $t$  ranging from 3 through 6 in increments 1000.

The data is as follows:

The curve begins at (0.00, 6) and ends at (0.14, 4.4) through (0.02, 5.6), (0.04, 3), (0.06, 5.7), (0.08, 6), (0.10, 4.2), and (0.12, 5).

B) The horizontal axis represents frequency ranging from 0.0 to 0.4, while vertical axis represents DFT magnitude ranging from 5 through 45 in increments 5

The data is as follows:

The curve begins at (6, 0.0) to (44, 0.0) through (25, 0.0). The power spectrum values are at 0.6 for frequencies of 12.5, 18, 32, and 37.5.

Note: All the approximate.

[!\[\]\(3cb60d42b10e53f9522bb0b392c1c4cd\_img.jpg\) Return to Figure 16.11](#)

## Figure 16.12 Text Alternative (Chapter 16)

---

 [Return to Figure 16.12](#)

The horizontal axis represents frequency (in hertz) ranging from 7.5 through 25.0, while vertical axis ranges from 0.0 through 0.25 in increments of 0.05. A horizontal line runs from approximately 6.0 to 25.0 at the value 0.00. The power spectrum values are at 0.25 for frequencies of 12.25 and 18.75. They are 0.00 elsewhere.

 [Return to Figure 16.12](#)



## Figure 16.13 Text Alternative (Chapter 16)

---

[!\[\]\(c8d96c8885d3000a912c2582004aed63\_img.jpg\) Return to Figure 16.13](#)

The horizontal axis ranges from 1700 through 2000 in increments of 50, while vertical axis ranges from 0 through 250 in increments of 50.

The data is as follows:

The curve begins at (1700, 10) and ends at (2030, 2) showing increasing and decreasing trends across depicted as compressed zig-zag lines having sharp peaks. A curves increases from (1700, 65) to (2030, 90) through (1850, 65).

Note: All the approximate.

[!\[\]\(e3f8612927870f2e0f9f5989e6dd3064\_img.jpg\) Return to Figure 16.13](#)

## Figure 16.14 Text Alternative (Chapter 16)

---

 [Return to Figure 16.14](#)

The horizontal axis represents frequency (cycles per year) ranging from 0.0 through 0.5 in increments of 0.1, while vertical axis ranges from 0 through 5 in increments of 1.

The data is as follows:

The curve begins at (0.0, 0) and ends at (0.5, 0) showing decreasing trend after (0.1, 2.9). A sharp peak is marked between two smaller peaks slightly to the left of 0.1.

Note: All the approximate.

 [Return to Figure 16.14](#)

## Figure 17.2 Text Alternative (Chapter 17)

---

 [Return to Figure 17.2](#)

The horizontal axis represents  $x$ , while vertical axis represents  $f$  of  $x$ .

The data is as follows:

The curve increases from  $(x_1, f \text{ of } x_1)$  to  $(x_2, f \text{ of } x_2)$  through  $(x, f_1 \text{ of } x)$ .

The concave down curve is tangent to the line at  $(x_1, f \text{ of } x_1)$  and  $(x_2, f \text{ of } x_2)$ .

A triangular region is formed between  $(x_1, f \text{ of } x_1)$  and  $(x, f_1 \text{ of } x)$  and a trapezoid region is formed between  $(x, f_1 \text{ of } x)$  and  $(x_2, f \text{ of } x_2)$ .

Note: All data is approximate.

 [Return to Figure 17.2](#)

## Figure 17.3 Text Alternative (Chapter 17)

---

 [Return to Figure 17.3](#)

The horizontal axis represents  $x$  ranging from 0 to 5, while vertical axis represents  $f$  of  $x$  ranging from 0 to 2 in increments of 1.

The data is as follows:

The  $f$  of  $x$  line increases from (1, 0) to (3.5, 1.25) with linear estimate at (2, 0.45) while another  $f$  of  $x$  line increases from (1, 0) to (5.5, 1.65) with linear estimate at (2, 0.3).

The concave down curve is tangent to the lines at (3.5, 1.25), and (6, 1.7), with true value at (2, 0.6).

Note: All data is approximate.

 [Return to Figure 17.3](#)

## Figure 17.4 Text Alternative (Chapter 17)

---

 [Return to Figure 17.4](#)

The horizontal axis represents  $x$  ranging from 0 to 5, while vertical axis represents  $f$  of  $x$  ranging from 0 to 2 in increments of 1.

The data is as follows:

The  $f^2$  of  $x$  line increases from (1, 0) to (7.5, 1.8) through (3.5, 1.4) and (6, 1.8), with quadratic estimate at (2, 0.45); while another dashed line increases from (1, 0) to (3.5, 1.4) with linear estimate at (2, 0.4).

The concave down curve is tangent to  $f^2$  of  $x$  line at (3.5, 1.4) and (6, 1.8), with true value at (2, 0.6).

Note: All data is approximate.

 [Return to Figure 17.4](#)

## Figure 17.5 Text Alternative (Chapter 17)

---

 [Return to Figure 17.5](#)

The first column of the table establishes that the independent points are called  $x_1$  through  $x_4$ .

The second column lists the value of the function at those points,  $f$  of  $x_1$  through  $f$  of  $x_4$ .

The first divided difference takes the difference between each row of the function column and divides it by the domain represented by those functions, that is,  $f$  of  $(x_2, x_1)$  is the difference between  $f$  of  $x_2$  and  $f$  of  $x_1$  divided by  $x_2$  and  $x_1$ ,  $f$  of  $x_3, x_2$  is the difference between  $f$  of  $x_3$  and  $f$  of  $x_2$  divided by  $x_3$  minus  $x_2$ , et cetera. There is thus one fewer entry in the first divided difference column than in the function column.

The second divided difference repeats this process, taking the difference between the first divided differences and dividing by the total domain over which the difference occurs, that is,  $f$  of  $(x_3, x_2, x_1)$  is the difference between  $f$  of  $(x_3, x_2)$  and  $f$  of  $(x_2, x_1)$  divided by  $x_3$  minus  $x_1$ . There are thus two of these entries.

Finally, the third divided difference repeats this process, but using the second divided differences; for four points, there will only be one third divided difference as  $f$  of  $(x_4, x_3, x_2, x_1)$ .

 [Return to Figure 17.5](#)

## Figure 17.6 Text Alternative (Figure 17)

---

 [Return to Figure 17.6](#)

The horizontal axis represents  $x$  ranging from 0 to 5, while vertical axis represents  $f$  of  $x$  ranging from 0 to 2 in increments of 1.

The data is as follows:

The  $f^3$  of  $x$  curve increases from  $(1, 0)$  to  $(7, 2.1)$  through  $(4, 1.4)$ ,  $(5, 1.6)$ , and  $(6, 1.8)$ , with cubic estimate at  $(1, 0.6)$ .

The concave down curve,  $f$  of  $x$  equals  $\ln x$ , is tangent to  $f^3$  of  $x$  at  $(4, 1.4)$ ,  $(5, 1.6)$ , and  $(6, 1.8)$ , with true value at  $(2, 0.7)$ .

Note: All data is approximate.

 [Return to Figure 17.6](#)

## Figure 17.8 Text Alternative (Chapter 17)

---

 [Return to Figure 17.8](#)

The horizontal axis represents  $x$ , while vertical axis represents  $f$  of  $x$ .

The data is as follows:

The  $L_1 f(x)$  curve decreases from  $(x_1, f(x_1))$  to  $(x_2, 0)$ , while  $L_2 f(x)$  curve increases from  $(x_1, f(x_1))$  to  $(x_2, f(x_2))$ .

A curve  $f(x)$  increases from  $(x_1, f(x_1))$  to  $(x_2, f(x_2))$ .

Note: All data is approximate.

 [Return to Figure 17.8](#)



## Figure 17.10 Text Alternative (Chapter 17)

---

 [Return to Figure 17.10](#)

The horizontal axis represents  $x$ , while vertical axis represents  $f$  of  $x$

The data is as follows:

The S-shaped true curve passes through four points across  $x_1$ ,  $x_2$ , and  $x_3$ , while concave down extrapolation of interpolating polynomial curve passes through three points across  $x_1$ ,  $x_2$ , and  $x_3$  before ending beyond  $x_3$  on horizontal axis.

The first three points for both curves fall in interpolation region (with  $x_1$ ,  $x_2$ , and  $x_3$ ), while fourth curve falls in extrapolation region (beyond  $x_3$ ).

Note: All data is approximate.

 [Return to Figure 17.10](#)

## Figure 17.11 Text Alternative (Chapter 17)

---

 [Return to Figure 17.11](#)

The horizontal axis represents years ranging from 1920 to 2020 in increments of 20, while vertical axis represents population (in millions) ranging from negative 400 to 300 in increments of 100.

The data is as follows:

The curve decreases from 100 in 1920 to negative 400 in 2020 showing decreasing trend after 300 in 2010.

The data points are visible at:

100 in 1920, 120 in 1930, 130 in 1940, 150 in 1950, 180 in 1960, 200 in 1970, 220 in 1980, 250 in 1990, 280 in 2000, 300 in 2010, and 350 in 2020.

Note: All data is approximate.

 [Return to Figure 17.11](#)

## Figure 17.12 Text Alternative (Chapter 17)

---

 [Return to Figure 17.12](#)

The horizontal axis represents  $x$  ranging from negative 1.00 to 1.00 in increments of 0.25, while vertical axis represents  $y$  ranging from negative 0.4 to 1.0 in increments of 0.2.

The data is as follows:

Runge:

The parabolic curve begins at (negative 1.00, 0.05) and ends at (1.00, 0.05) through (negative 0.50, 0.15), (0.0, 1.0), and (0.50, 0.15). It exhibits an inverted V-shape.

Fourth order:

The dotted parabolic curve begins at (negative 1.00, 0.05) and ends at (1.00, 0.05) through (negative 0.50, 0.15), (0.0, 1.0), and (0.50, 0.15). It exhibits an inverted U-shape.

Note: All data is approximate.

 [Return to Figure 17.12](#)

## Figure 17.13 Text Alternative (Chapter 17)

---

 [Return to Figure 17.13](#)

The horizontal axis represents  $x$  ranging from negative 1.00 to 1.00 in increments of 0.25, while vertical axis represents  $y$  ranging from 0.0 to 2.0 in increments of 0.5.

The data is as follows:

Tenth order:

The curve begins at (negative 1.00, 0.0) and ends at (1.00, 0.0) through (negative 0.80, 0.15), (negative 0.60, 0.25), and (negative 0.35, 0.9), (negative 0.20, 0.5), (0.20, 0.5), (0.35, 0.9), (0.60, 0.25), and (0.80, 0.15). It exhibits an inverted V-shape with fluctuations and two inverted V-shaped ends.

Runge:

The parabolic curve begins at (negative 1.00, 0.0) and ends at (1.00, 0.0) through (negative 0.80, 0.15), (negative 0.60, 0.25), and (negative 0.35, 0.9), (negative 0.20, 0.5), (0.20, 0.5), (0.35, 0.9), (0.60, 0.25), and (0.80, 0.15). It exhibits an inverted V-shape.

Note: All data is approximate.

 [Return to Figure 17.13](#)

## Figure 17.14 Text Alternative (Chapter 17)

---

 [Return to Figure 17.14](#)

The horizontal axis represents  $x$  ranging from negative 0.3 to 1.0 in increments of 0.1, while vertical axis represents  $y$  ranging from 0.05 to 0.40 in increments of 0.05. The graph shows two curves representing fourth order and range overlapping each other, passing through data points as follows:

The curve begins at (0.25, 0.39) and ends at (1.0, 0.04) through (0.44, 0.175), (0.65, 0.10), and (0.82, 0.06).

Note: All data is approximate.

 [Return to Figure 17.14](#)

## Figure P17.12 Text Alternative (Chapter 17)

---

[!\[\]\(d0a1791f26d167e866e44ebbf83efebe\_img.jpg\) Return to Figure P17.12](#)

The V in with 10 volts is connected to R 1, R 2, and R 3 linked in a rhomboidal fashion with V out flowing across. The cable with lead wires from R 1 and R 3 lead to platinum sensor.

[!\[\]\(eafc244b53721dd1ec133f0772f70fc7\_img.jpg\) Return to Figure P17.12](#)

## Figure P17.16 Text Alternative (Chapter 17)

---

 [Return to Figure P17.16](#)

A stress  $\sigma_z$  is being applied to the top of this surface.

 [Return to Figure P17.16](#)

## Figure P17.18 Text Alternative (Chapter 17)

---

 [Return to Figure P17.18](#)

A) Ideal thermocouple circuit:

An elongated hexagon of material A and B with current directed counter-clockwise shows  $T_h$  labeled at left and  $T_c$  labeled at right.

B) Practical thermocouple circuit:

An elongated rectangle with triangular end on left and bottom, of material A and B with length  $e$  at the right shows  $T_h$  labeled at left and  $T_c$  labeled at bottom.

 [Return to Figure P17.18](#)



# Figure 18.1 Text Alternative (Chapter 18)

---

 [Return to Figure 18.1](#)

A) Plot uses two points before the jump and two after, and then using a cubic to interpolate the points. The interpolating function undershoots the beginning interval and overshoots the end a little and is not very sharp at the jump.

B) Plot uses six points and a fifth order polynomial. The transition is sharper, but there are more pronounced over and undershoots at the beginning and end.

C) Plot uses eight points and a seventh order polynomial, the transition is quite sharp but the oscillations at the start and finish are highly unreasonable.

D) Plot uses eight points but simply connects the dots. The transition is fairly sharp, somewhere between that of the fifth and seventh order polynomial and there is no overshoot.

 [Return to Figure 18.1](#)

## Figure 18.3 Text Alternative (Chapter 18)

---

 [Return to Figure 18.3](#)

The curve shows a continuous function with data points at  $x_1, x_2$ , extending to  $x_n$ . The function values at these points are given by  $f_1, f_2; f_i, f_{i+1}$ ; and  $f_{n-1}, f_n$ .

The interval 'one' is visible between the first two points  $x_1$  and  $x_2$ , while interval 'i' is visible between  $f_i$  and  $f_{i+1}$ , and the interval 'n minus i' is visible between  $f_{n-1}$  and  $f_n$ .

The interpolation functions for each interval are labeled as  $s_1, s_i$  and  $s_{n-1}$  as functions of  $x$ .

 [Return to Figure 18.3](#)

## Figure 18.4 Text Alternative (Chapter 18)

---

[!\[\]\(1d3a1175dd4902218e694b9c098adb83\_img.jpg\) Return to Figure 18.4](#)

A) The horizontal axis represents  $x$  ranging from 2 through 10 in increments of 2, while vertical axis represents  $f$  of  $x$  ranging from 0 through 2.

The data is as follows:

The first order sp line curve begins at (1, 2.5) and ends at (9, 0.5) through (4.5, 1) and (7, 2.5).

B) The horizontal axis represents  $x$ , while vertical axis represents  $f$  of  $x$  ranging from 0 through 2.

The data is as follows:

The second order sp line curve begins at (1, 2.5) and ends at (7, 0.5) through (2.5, 1) and (5, 2.5).

C) The horizontal axis represents  $x$ , while vertical axis represents  $f$  of  $x$  ranging from 0 through 2.

The data is as follows:

The cubic sp line curve begins at (1, 2.5) and ends at (7, 0.5) through (2.5, 1) and (5, 2.5). The interpolating cubic curve begins at (1, 2.5) and ends at (7, 0.5) through (2.5, 1) and (5, 2.5). The interpolating cubic curve shifts slightly right to the cubic sp line curve.

Note: All data is approximate.

[!\[\]\(0d5ec72f61334709c3fc9450209b754f\_img.jpg\) Return to Figure 18.4](#)

# Figure 18.5 Text Alternative (Chapter 18)

---

 [Return to Figure 18.5](#)

The horizontal axis represents  $x$  ranging from 2 through 8 in increments of 2, while vertical axis represents  $f$  of  $x$  ranging from 0 through 3 in increments of 1.

The data is as follows:

The clamped curve begins at  $(3, 2.5)$  and ends at  $(9, 0.5)$  through  $(4.2, 1)$  and  $(6.5, 2.5)$ . It lies slightly inwards.

The natural curve begins at  $(3, 2.5)$  and ends at  $(9, 0.5)$  through  $(4.2, 1)$  and  $(6.5, 2.5)$ . It lies between the clamped and not a knot curve.

The not a knot curve begins at  $(3, 2.5)$  and ends at  $(9, 0.5)$  through  $(4.2, 1)$  and  $(6.5, 2.5)$ . It lies slightly outwards.

Note: All data is approximate.

 [Return to Figure 18.5](#)

# Scatter Plot Graphic Text Alternative (Chapter 18)

---

 [Return to Scatter Plot Graphic](#)

The horizontal axis ranges from 0 through 500 in increments of 100, while vertical axis ranges from 0.4 through 1.4 in increments of 0.2.

The data is as follows:

The curve decreases from (60, 1.55) to (500, 0.47) through data points (0, 1.3), (30, 1.2), (50, 1.1), (100, 0.95), (150, 0.82), (200, 0.75), and (300, 0.6).

Note: All data is approximate.

 [Return to Scatter Plot Graphic](#)

## Figure 18.7 Text Alternative (Chapter 18)

---

 [Return to Figure 18.7](#)

The horizontal axis represents  $x$  ranging from negative 1.00 through 1.00 in increments of 0.25, while vertical axis represents  $f$  of  $x$  ranging from 0.0 through 1.0 in increments of 0.2. The dotted curve for  $\sin x$  and solid curve for Runge begins at (negative 1.00, 0.05) and ends at (1.00, 0.05) through (negative 0.75, 0.09), (negative 0.50, 0.15), (negative 0.25, 0.4), (0.00, 1.0), (0.25, 0.4), (0.50, 0.15), and (0.75, 0.09).

Note: All data is approximate.

 [Return to Figure 18.7](#)

# Figure 18.8 Text Alternative (Chapter 18)

---

[!\[\]\(35e4f762fc1cfea5610d92e2d225d5b4\_img.jpg\) Return to Figure 18.8](#)

The horizontal axis represents  $x$  ranging from negative 1.00 through 1.00 in increments of 0.25, while vertical axis represents  $f$  of  $x$  ranging from 0.0 through 1.0 in increments of 0.2.

The data is as follows:

Natural: The curve begins at (negative 1.00, 0.05) and ends at (1.00, 0.05) through (negative 0.75, 0.09), (negative 0.50, 0.15), (negative 0.25, 0.4), (0.00, 1.0), (0.25, 0.4), (0.50, 0.15), and (0.75, 0.09).

Not a knot: The curve begins at (negative 1.00, 0.05) and ends at (1.00, 0.05) through (negative 0.75, 0.09), (negative 0.50, 0.15), (negative 0.25, 0.4), (0.00, 1.0), (0.25, 0.4), (0.50, 0.15), and (0.75, 0.09).

Damped: The curve begins at (negative 1.00, 0.05) and ends at (1.00, 0.05) through (negative 0.75, 0.09), (negative 0.50, 0.15), (negative 0.25, 0.4), (0.00, 1.0), (0.25, 0.4), (0.50, 0.15), and (0.75, 0.09). This curve shows increasing trends after (negative 1.00, 0.5), (0.75, 0.9), and decreasing trend after (0.50, 0.15) for single increment.

Note: All data is approximate.

[!\[\]\(2b376d1a92330ab09dad2665d2f89bf5\_img.jpg\) Return to Figure 18.8](#)

# Figure 18.9 Text Alternative (Chapter 18)

---

 [Return to Figure 18.9](#)

The horizontal axis represents  $x$  ranging from negative 1.00 through 1.00 in increments of 0.25, while vertical axis represents  $f$  of  $x$  ranging from 0.0 through 1.0 in increments of 0.2. The overlapping curves for cubic sp line, P chip, and Runge begin at (negative 1.00, 0.05) and ends at (1.00, 0.05) through (negative 0.75, 0.09), (negative 0.50, 0.15), (negative 0.25, 0.4), (0.00, 1.0), (0.25, 0.4), (0.50, 0.15), and (0.75, 0.09).

Note: All data is approximate.

 [Return to Figure 18.9](#)



## Figure 18.10 Text Alternative (Chapter 18)

---

 [Return to Figure 18.10](#)

The axis are labeled  $x$ ,  $y$ , and  $f$  of  $(x, y)$ . The points are labeled clockwise as  $f(x_1, y_2)$ ,  $f(x_1, y_1)$ ,  $f(x_2, y_1)$ , and  $f(x_2, y_2)$  resulting in a quadrilateral across the center at the top. The distance between  $f(x_i, y_i)$  and  $(x_i, y_i)$  is labeled as  $y_1$ .

 [Return to Figure 18.10](#)

# Figure 18.11 Text Alternative (Chapter 18)

---

[!\[\]\(529949c2c3dadbaa4e538e8c643454bc\_img.jpg\) Return to Figure 18.11](#)

The external points are labeled clockwise as:

$f(x_1, y_1)$ ,  $f(x_2, y_1)$ ,  $f(x_2, y_2)$ , and  $f(x_1, y_2)$ .

$f(x_1, y_1)$  and  $f(x_2, y_1)$  leads to  $f(x_i, y_1)$  while  $f(x_1, y_2)$  and  $f(x_2, y_2)$  leads to  $f(x_i, y_2)$ .  $f(x_i, y_2)$  and  $f(x_i, y_1)$  both lead to  $f(x_i, y_i)$  across the center.

[!\[\]\(339a16584d5da0f0a3ca4e9ec17bf6a1\_img.jpg\) Return to Figure 18.11](#)

# Scatter Plot Graphic Text Alternative (Chapter 18)

---

 [Return to Scatter Plot Graphic](#)

The horizontal axis represents sample number ranging from 0 through 200 in increments of 25, while vertical axis represents concentration ranging from 16.0 through 18.0 in increments of 0.5.

The data is as follows:

The best fit curve begins at (0, 16.6) and ends at (195, 17.6) through (30, 17.5), (70, 16.9), (125, 17.0), and (175, 17.25).

There are numerous data points across the top and bottom of the curve placed closely together.

Note: All data is approximate.

 [Return to Scatter Plot Graphic](#)

## Figure 18.13 Text Alternative (Chapter 18)

---

 [Return to Figure 18.13](#)

The horizontal axis ranges from 0 through 300 in increments of 50 while vertical axis ranges from 7.0 through 10.5 in increments of 0.5.

The data is as follows:

Regressed model:

The best fit curve increases from (60, 8.3) to (140, 9.4) through (100, 9.27). A data point is marked at (99, 9.4).

Multiple data points are closely visible inside and outside the region across (0, 8.0), (300, 0.8), (0, 10.5), and (300, 10.5).

Note: All data is approximate.

 [Return to Figure 18.13](#)

# Scatter Plot Graphic Text Alternative (Chapter 18)

---

[!\[\]\(8af806fb1314382d09bc5ec5b767526c\_img.jpg\) Return to Scatter Plot Graphic](#)

The horizontal axis represents sample number ranging from 0 through 200 in increments of 25, while vertical axis represents concentration ranging from 16.0 through 18.0 in increments of 0.5.

The data is as follows:

The curve begins at (0, 16.6) and ends at (195, 17.6) through (30, 17.5), (70, 16.9), (125, 17.0), and (175, 17.25).

There are numerous data points across the top and bottom of the curve placed closely together.

Note: All data is approximate.

[!\[\]\(830769b31eeeaca920791081939ff8ba\_img.jpg\) Return to Scatter Plot Graphic](#)

## Figure 18.16 Text Alternative (Chapter 18)

---

 [Return to Figure 18.16](#)

The horizontal axis represents temperature (in degrees Celsius) ranging from 0 through 30 in increments of 10, while vertical axis represents depth (in meters) ranging from 0 through 30 in increments of 10.

The data is as follows:

The region of epilimnion shows four data points as: (0, 21.5), (2, 21.5), (5, 21.5), and (20, 10).

The region of hypolimnion shows four data points as: (12, 10.4), (11, 24), (11, 26.5), and (11, 28).

The dashed line from (0, 10.08) is labeled thermocline.

Note: All data is approximate.

 [Return to Figure 18.16](#)

# Figure 18.17 Text Alternative (Chapter 18)

---

 [Return to Figure 18.17](#)

Left panel:

The horizontal axis represents degrees Celsius ranging from 10 through 20, while vertical axis represents depth,  $z$  (in meters) ranging from 0 through 30 in increments of 10.

The data is as follows:

The best fit curve begins at (23, 0) and ends at (11, 27.5) through data points (23, 2.5), (23, 5), (21, 9), (14, 16.5), (11.5, 22), and (11, 22.8).

Middle panel:

The horizontal axis represents degree Celsius per meter ranging from negative 1 through 0, while vertical axis represents depth,  $z$  (in meters) ranging from 0 through 30 in increments of 10.

The data is as follows:

The curve begins at (0, 0) and ends at (0.3, 27.5) through (0, 5), (negative 0.2, 11.5), (0.2, 17), and (0, 25).

Right panel:

The horizontal axis represents degree Celsius per meter squared ranging from negative 0.250 through 0.25, while vertical axis represents depth,  $z$  (in meters) ranging from 0 through 30 in increments of 10.

The data is as follows:

The curve begins at (0.0, 0) and ends at (0.7, 27.5) through (0.0, 2.5), (negative 2.00, 8), (0.310, 13.5), (0.0, 17.5), and (0.0, 20).

Note: All data is approximate.

 [Return to Figure 18.17](#)

# Figure P18.15 Text Alternative (Chapter 18)

---

 [Return to Figure P18.15](#)

A) The horizontal axis represents temperature (in degrees Celsius) ranging from negative 100 through 500, while vertical axis represents altitude (in kilometers) ranging from negative 10 through 90 in increments of 10.

The data is as follows:

The curve begins at (negative 90, 90) and ends at (20, negative 1) through (negative 90, 80), (negative 20, 60), (0, 50), (negative 50, 28), (negative 60, 20), (negative 60, 11), (negative 25, 6), and (0, 2).

B) The horizontal axis represents pressure (in atmosphere) ranging from 0.0000 through 1.5000, while vertical axis represents altitude (in kilometers) ranging from negative 10 through 90 in increments of 10.

The data is as follows:

The decreasing L-shaped curve begins at (0.0000, 90), (0.0000, 80), (0.0000, 60), (0.0000, 29), (0.1000, 20), (0.2000, 11), (0.5000, 6), and (0.7500, 3).

C) The horizontal axis represents density (in kilogram per meter cube) ranging from 0.0000 through 1.5000, while vertical axis represents altitude (in kilometers) ranging from negative 10 through 90 in increments of 10.

The data is as follows:

The decreasing L-shaped curve begins at (0.0000, 90), (0.0000, 80), (0.0000, 60), (0.0000, 29), (0.1100, 20), (0.4500, 10), (0.5200, 7), and (1.0000, 2).

Note: All data is approximate.

 [Return to Figure P18.15](#)



## Figure P18.18 Text Alternative (Chapter 18)

---

[!\[\]\(c8d96c8885d3000a912c2582004aed63\_img.jpg\) Return to Figure P18.18](#)

The horizontal axis ranges from September 9, 2013 through September 23, 2013, while vertical axis ranges from 0 through 900 in increments of 100.

The data is as follows:

The curve begins between September 9, 2013 and September 11, 2013 at 10 and ends between September 21, 2013 and September 23, 2013 at 90 showing a decreasing trend between September 15, 2013 and September 17, 2013 at 300. The data points between September 11, 2013 and September 13, 2013 through September 15, 2013 and September 17, 2013 are loosely placed. A sharp dotted peak is marked slightly to the right of September 13, 2013 at 850.

Note: All data is approximate.

[!\[\]\(e3f8612927870f2e0f9f5989e6dd3064\_img.jpg\) Return to Figure P18.18](#)

# Figure PT5.1 Text Alternative (Chapter 19)

---

 [Return to Figure PT5.1](#)

A) Top: The horizontal axis represents  $t$  ranging from 0 through 12 in increments of 4. The vertical axis represents  $y$  ranging from 0 through 400 in increments of 200.

The data is as follows:

A concave up curve increases from  $(0, 0)$  to  $(12, 400)$  through  $(5, 110)$ . The slope at this point points to  $(5, 3.1)$  at another curve plotted across the graph at the bottom.

Bottom: The horizontal axis represents  $t$  ranging from 0 through 12 in increments of 4. The vertical axis represents  $v$  ranging from 0 through 4 in increments of 2.

The data is as follows:

A concave down curve increases from  $(0, 0)$  to  $(12, 5)$  through  $(5, 3.1)$ .

B) Top: The horizontal axis represents  $t$  ranging from 0 through 12 in increments of 4. The vertical axis represents  $v$  ranging from 0 through 4 in increments of 2.

The data is as follows:

A concave down curve increases from  $(0, 0)$  to  $(12, 5)$  through  $(5, 3.1)$ . The area under the curve is shaded. It points to  $(5, 110)$  at another curve plotted across the graph at the bottom.

Bottom The horizontal axis represents  $t$  ranging from 0 through 12 in increments of 4. The vertical axis represents  $y$  ranging from 0 through 400 in increments of 200.

The data is as follows:

A concave up curve increases from  $(0, 0)$  to  $(12, 400)$  through  $(5, 110)$ .

Note: All data is approximate.

 [Return to Figure PT5.1](#)

## Figure 19.1 Text Alternative (Chapter 19)

---

 [Return to Figure 19.1](#)

The curve begins at slightly below the vertical axis and to the right of origin, which decreases slightly only to increase as parabolic peak, which further decreases to slightly above the horizontal axis. A point a on the vertical axis extends a line to first depression in the curve. A point b on the horizontal axis extends to declining segment of the curve. The area between the two points is shaded.

 [Return to Figure 19.1](#)

## Figure 19.2 Text Alternative (Chapter 19)

---

 [Return to Figure 19.2](#)

A) A grid bounded by two perpendicular roads and a stream that curves to intersect both, where counting the grid squares are marked under the stream.

B) A cross-section of river bounded by the river floor depicted as w-shaped depression shows a horizontal line on the top. Downward arrows are marked from water surface toward river floor.

C) Horizontal arrows decreasing in length from top to bottom are marked to the left of a building

 [Return to Figure 19.2](#)

## Figure 19.3 Text Alternative (Chapter 19)

---

 [Return to Figure 19.3](#)

A) The horizontal axis represents  $i$  ranging from 0 through 6 in increments of 1, while vertical axis represents  $y$ .

The data shows six discrete vertical bars and a dotted line with a constant value labeled as 'mean'. The bars across 2 and 3 exceed the mean, while the bars across 1, 4, 6, and 5 are visible underneath.

B) The horizontal axis represents  $x$ , while vertical axis represents  $y$  equals  $f$  of  $x$ .

The data shows a continuous curved function and a dotted line with a constant value labeled as 'mean' between two points,  $a$  and  $b$ .

 [Return to Figure 19.3](#)

## Figure 19.4 Text Alternative (Chapter 19)

---

 [Return to Figure 19.4](#)

A) The graph shows a curved function with two points across a and b on the horizontal axis. A dotted line connects the two points under the curve, forming a trapezoidal area under it across a and b, which is shaded.

B) The graph shows a curved function with three points across a and b on the horizontal axis, marked on the curve. The point at the center is the highest, with dot corresponding to point a being second highest, and point c corresponding to be the lowest. A dotted concave down curve connects the three points on the curve, with area under the curve shown shaded.

 [Return to Figure 19.4](#)

## Figure 19.5 Text Alternative (Chapter 19)

---

 [Return to Figure 19.5](#)

The graph shows a curved function with four points across  $a$  and  $b$  on the horizontal axis. The trapezoidal area with an extended semi-circular area under it is shaded. The upper section of shaded area, breaks it up into three different intervals across four points.

 [Return to Figure 19.5](#)

## Figure 19.6 Text Alternative (Chapter 19)

---

 [Return to Figure 19.6](#)

A) The graph shows a curved function with five points over  $a$  and  $b$  on the horizontal axis. The trapezoidal area with an extended semi-circular area under it is shaded.

B) The graph shows a curved function with three points over  $a$  and  $b$  on the horizontal axis. The trapezoidal area with an extended semi-circular area under it is shaded.

 [Return to Figure 19.6](#)



## Figure 19.7 Text Alternative (Chapter 19)

---

 [Return to Figure 19.7](#)

The points  $f$  of  $a$ , and  $f$  of  $b$  connect to form a line, while area under it across  $a$  and  $b$  on the horizontal axis is shaded.

 [Return to Figure 19.7](#)

# Figure 19.8 Text Alternative (Chapter 19)

---

[!\[\]\(529949c2c3dadbaa4e538e8c643454bc\_img.jpg\) Return to Figure 19.8](#)

The horizontal axis represents  $x$  ranging from 0 through 0.8. The vertical axis represents  $f$  of  $x$  ranging from 0 through 2.0.

The data is as follows:

A curve begins at  $(0, 0)$  and ends at  $(0.8, 0)$  shows increasing trend at  $(0.3, 1.25)$ , and decreasing trend at  $(0.6, 3.4)$ . The area over  $(0, 0.25)$  and  $(0.8, 0.25)$  is shaded and labeled as 'error', while the area under  $(0, 0.25)$  and  $(0.8, 0.25)$  is shaded and labeled as 'integral estimate'.

Note: All data is approximate.

[!\[\]\(a870788d6ed9b8fd294b7654a8c8526b\_img.jpg\) Return to Figure 19.8](#)

## Figure 19.9 Text Alternative (Chapter 19)

---

 [Return to Figure 19.9](#)

The curve passes through six points across  $x_0$  through  $x_5$ , where  $x_0$  equals  $a$ , and  $x_n$  equals  $b$ . A vertical line from each point on the horizontal axis as  $x_0$ ,  $x_1$ ,  $x_2$ ,  $x_3$ ,  $x_4$ , and  $x_5$  is directed to the curve, with adjacent values forming a shaded bar under the curve. Each point on the curve is connected by a line. Below the graph, text reads,  $x_0$  equals  $a$  and  $x_n$  equals  $b$ . The horizontal distance between  $x_1$  and  $x_2$  is labeled as  $h$  equals difference of  $b$  and  $a$  divided by  $n$ .

 [Return to Figure 19.9](#)

## Figure 19.11 Text Alternative (Chapter 19)

---

 [Return to Figure 19.11](#)

A) The graph shows an M-shaped function with three points. A vertical line from horizontal axis connects the first and third point on the curve. A dashed parabolic curve is drawn covering the three points, showing a peak at the point at the center.

B) The graph shows an M-shaped function with four points. A vertical line from horizontal axis connects the first and third point on the curve. A dashed curve is drawn covering the four points, depicted as an increasing concave down curve, which decreases from second point to fourth point.

 [Return to Figure 19.11](#)

## Figure 19.12 Text Alternative (Chapter 19)

---

 [Return to Figure 19.12](#)

The curve passes through 11 data points as 1, 4, 2, 4, 2, 4, 2, 4, 2, 4, 2, 4, and 1 resulting in 10 vertical sections across the horizontal axis between a and b. The integrated groups from left to right include:

1, 4, 2 leads to 1, 4, 1; 4 leads to 1, 4, 1; 2, 4, 2 leads to 1, 4, 1; 4 leads to 1, 4, 1; and 2, 4, 1 leads to 1, 4, 1.

 [Return to Figure 19.12](#)

## Figure 19.13 Text Alternative (Chapter 19)

---

 [Return to Figure 19.13](#)

The horizontal axis represents  $x$  ranging from 0 through 0.8. The vertical axis represents  $f$  of  $x$  ranging from 0 through 3 in increments of 1.

The data is as follows:

The curve begins at (0, 2) and ends at (0.75, 0) passing through (0.12, 1.25), (0.25, 1.85), (0.4, 3.2), (0.6, 3.1), and (0.75, 0.25). The area under the curve is divided into 5 vertical sections that are shaded. The two sections from (0, 0) through (0, 0.25) represent one-third rule, while the three sections (0, 0.25) through (0, 0.75) represent three-eighth rule.

Note: All the data is approximate.

 [Return to Figure 19.13](#)

## Figure 19.15 Text Alternative (Chapter 19)

---

 [Return to Figure 19.15](#)

The horizontal axis represents  $x$  ranging from negative 4 through 4, while vertical axis represents cumulative probability (in percentage) ranging from 0.0 through 1.0 in increments of 0.2.

The data is as follows:

The curve begins at (negative 1, 0.0) and ends at (5, 1.0) passing through (negative 4, 0.0), (negative 2, 0.0), (0, 0.2), (2, 1.0), and (4, 1.0).

Note: All the data is approximate.

 [Return to Figure 19.15](#)

## Figure 19.16 Text Alternative (Chapter 19)

---

 [Return to Figure 19.16](#)

The double integral would be the volume between the rectangular patch on the x-y plane and the surface above that patch.

 [Return to Figure 19.16](#)



# Figure 19.17 Text Alternative (Chapter 19)

---

 [Return to Figure 19.17](#)

The average surface area above a strip in the x and y direction is integrated to get the total volume.

The nodes in x y plane have the following values (from top to bottom): 0, 40, 48; 54, 70, 54,; and 72, 64, and 24, where segments across 0, 54, 72 on the y-axis, and 64 and 24 on the x-axis are connected in the shape of 'L' and highlighted in bold. 72 lies at origin.

The calculations on the right reads as follows:

(8 minus 0) with 0 plus 2 times (40) plus 48 whole divided by 4 leads to 256 (highlighted); (8 minus 0) with 54 plus 2 times (70) plus 54 whole divided by 4 leads to 496 (highlighted); and (8 minus 0) with 72 plus 2 times (64) plus 24 whole divided by 4 leads to 448 (highlighted).

256, 496, and 448 lead to (6 minus 0) with 256 plus 2 times (496) plus 448 whole divided by 4, which finally equals to 2544 (highlighted).

 [Return to Figure 19.17](#)

# Figure 19.18 Text Alternative (Chapter 19)

---

 [Return to Figure 19.18](#)

Middle panel:

The horizontal axis represents  $x$ , m ranging from 0 through 30. the vertical axis represents  $F$  of  $x$  (in Newton) ranging from 0 through 10.

The data is as follows:

The curve begins at (0, 0) and ends at (30, 5) through (5, 8), (10, 12.5), (15, 13.5), (20, 10), and (25, 12).

Bottom panel:

The horizontal axis represents  $x$ , m ranging from 0 through 30. The vertical axis represents theta of  $x$  (in radian) ranging from 0 through 2 in increments of 1.

The data is as follows:

The curve begins at (0, 0.5), (5, 1.4), (10, 0.75), (15, 0.8), (20, 11.45), (25, 1.5), and (30, 1.51).

Note: All the data is approximate.

 [Return to Figure 19.18](#)

## Figure 19.19 Text Alternative (Chapter 19)

---

[!\[\]\(1d3a1175dd4902218e694b9c098adb83\_img.jpg\) Return to Figure 19.19](#)

The horizontal axis represents  $x$ , m ranging from 0 through 30. The vertical axis represents  $F$  of  $x$  (theta of  $x$ ).

The data is as follows:

A curve begins at (0, 0) and ends at (0, 30) showing increasing trends after 5 across horizontal axis and decreasing trend after 7 across horizontal axis. Four data points are marked on the declining slope of the curve marked between 15 to 30 on the horizontal axis.

Note: All data is approximate.

[!\[\]\(cbe2492b119e39e02a1dab2af4a4b296\_img.jpg\) Return to Figure 19.19](#)

## Figure 19.20 Text Alternative (Chapter 19)

---

 [Return to Figure 19.20](#)

The horizontal axis represents  $x$ , m ranging from 0 through 30, while vertical axis represents  $F$  of  $x$  (theta of  $x$ ) ranging from 0 through 10.

The data is as follows:

A curve begins at (0, 0) and ends at (30, 0) passing through (5, 2.6), (10, 9.5), (13, 7), (20, 4), and (25, 2). The areas between (2.2, 2.5), (2.5, 2), (3.5, 4.2) and (13, 7), (20, 4), (25, 2), and (30, 0) are labeled as overestimates. The areas between (0, 0), (2, 2.5), (1.5, 3.5) and (4, 4), (5, 9.5), (12, 13) (13, 7) are labeled as underestimates.

Note: All data is approximate.

 [Return to Figure 19.20](#)

## Figure P19.9 Text Alternative (Chapter 19)

---

[!\[\]\(8af806fb1314382d09bc5ec5b767526c\_img.jpg\) Return to Figure P19.9](#)

A) The figure shows pressure linearly increasing with depth below the surface. The values from base to top increases as 0, 25, 50, and 75.

B) The figure shows the dam is widest at the top and narrows towards the bottom. The values from bottom to top increases from 122 to 200 through 130, 135, 160, 175, and 190.

[!\[\]\(e2376d476d06eb31946dc01a69a4403a\_img.jpg\) Return to Figure P19.9](#)

## Figure P19.15 Text Alternative (Chapter 19)

---

 [Return to Figure P19.15](#)

The x values are 0, 4, 8, and 12 while the y values are 0, 2, and 4.

The values (from left to right) for functions in the bottom, middle and top rows are as follows:

With y as 0, the values are negative 2, negative 1, 4, and 10, while with y as 2 the values are negative 4, negative 3, 1.5, and 6.5; and with y as 4 the values are negative 8, negative 8, negative 6, and 4.

The segments across negative 8, negative 4, and negative 2 (along y axis) and negative 2, negative 1, 4, and 10 (along x axis) are connected in an 'L' shape and highlighted in bold.

 [Return to Figure P19.15](#)

## Figure P19.25 Text Alternative (Chapter 19)

---

[!\[\]\(35e4f762fc1cfea5610d92e2d225d5b4\_img.jpg\) Return to Figure P19.25](#)

The horizontal axis represents  $F$  (in kilo-Newton per meter) ranging from 0 through 100 in increments of 50. The vertical axis represents  $z$  (in meters) ranging from 0 through 600 in increments of 100.

The data is as follows:

A curve begins at (0, 0) and ends at (100, 600) passing through (30, 50), (40, 100), (40, 150), (50, 225), (50, 300), (60, 375), and (80, 450). Horizontal arrows from each point on the vertical axis is directed toward right, with length of arrows increasing from bottom to top.

Note: All data is approximate.

[!\[\]\(8d0f0e0fe25b320c33272c52aec1fbca\_img.jpg\) Return to Figure P19.25](#)

## Figure 20.1 Text Alternative (Chapter 20)

---

 [Return to Figure 20.1](#)

A) 0.172800 and 1.068800 [0(h) squared], leading to 1.367467 [0(h) raised to the power of 4], with no values for [0(h) raised to power of 6] and [0(h) raised to power of 8].

B) 0.172800, 1.068800, and 1.484800 [0(h) squared], where .068800, and 1.484800 lead to 1.623467 [0(h) raised to power of 4] below 1.367467, finally leading to 1.640533 [0(h) raised to power of 6] with no values for [0(h) raised to power of 8].

C) 0.172800, 1.068800, 1.484800, and 1.600800 [0(h) squared], where 1.484800, and 1.600800 lead to 1.639467 [0(h) raised to power of 4] below 1.623467, further leading to 1.640533 and 1.640533 [0(h) raised to power of 6], finally leading to 1.640533 [0(h) raised to power of 8].

 [Return to Figure 20.1](#)



## Figure 20.3 Text Alternative (Chapter 20)

---

 [Return to Figure 20.3](#)

A) The graph shows a curve passing through two distant points, which are connected together. These are connected across two distant points on the horizontal axis on left and right resulting in a trapezoidal area under the curve, which is shaded.

B) The graph shows a curve passing through two nearby points, which are connected together. These intersect a trapezoidal area across two distant points on the left and right of horizontal axis, which is shaded.

 [Return to Figure 20.3](#)

## Figure 20.4 Text Alternative (Chapter 20)

---

 [Return to Figure 20.4](#)

A) The graph is plotted across first and second quadrant. The vertical line from the horizontal axis across the second quadrant emerges from negative  $(b \text{ minus } a) \text{ whole divided by } 2$ , while another vertical line from the horizontal axis across the first quadrant emerges from  $(b \text{ minus } a) \text{ whole divided by } 2$ . An extended horizontal line,  $y \text{ equals } 1$ , connects the upper sections of both these lines.

B) The graph is plotted across first and third quadrant. The vertical line from the horizontal axis across the first quadrant emerges from  $(b \text{ minus } a) \text{ whole divided by } 2$ , while another vertical line from the horizontal axis across the third quadrant emerges from negative  $(b \text{ minus } a) \text{ whole divided by } 2$ . An extended diagonal line,  $y \text{ equals } x$  connects the lower and upper sections of both these lines.

 [Return to Figure 20.4](#)

## Figure 20.5 Text Alternative (Chapter 20)

---

 [Return to Figure 20.5](#)

A curve increases third quadrant to the left of negative 1 into the first quadrant as a concave down curve passing from two points as  $(x_0, f \text{ of } x_0)$  and  $(x_1, f \text{ of } x_1)$ . A vertical line from negative 1 to the left of  $x_0$  and 1 from the right of  $x_1$  is drawn, with a diagonal line connecting the two lines and passing through the two coordinates.

 [Return to Figure 20.5](#)

## Figure P20.16 Text Alternative (Chapter 20)

---

 [Return to Figure P20.16](#)

A) A vertical rod is attached to a ceiling with an arrow pointing downward.

B) The horizontal axis represents  $e$ , ranging from 0 through 0.2 in increments of 0.1. The vertical axis represents  $s$ , kilo pound per square inch ranging from 0 through 60 in increments of 20.

The data is as follows:

The curve begins at (0, 40) and ends at (0.28, 55) (labeled as rupture) passing through (0.02, 40), (0.05, 39.5), (0.1, 42), (0.15, 51), and (0.2, 58).

The area under the curve is shaded and labeled as 'modulus of toughness'.

The chart with two columns shows values for  $e$  and  $s$  as:

0.02, 40.0; 0.05, 37.5; 0.10, 43.0; 0.15, 52.0; 0.20, 60.0; and 0.25, 55.0.

Note: All data is approximate.

 [Return to Figure P20.16](#)

## Figure 21.1 Text Alternative (Chapter 21)

---

 [Return to Figure 21.1](#)

A) The curve increases from a point at y-axis to  $(x_i + \Delta x, f(x_i + \Delta x))$  through  $(x_i, f(x_i))$ .

A tangent increases from  $(x_i, f(x_i))$  to  $(x_i + \Delta x, f(x_i + \Delta x))$ .

B) The curve increases from a point at y-axis beyond  $(x_i + \Delta x, f(x_i + \Delta x))$  through  $(x_i, f(x_i))$ .

A tangent increases from  $(x_i, f(x_i))$  beyond  $(x_i + \Delta x, f(x_i + \Delta x))$ .

C) The curve increases from a point at y-axis and passes through a point along  $x_i$ .

A tangent,  $f'(x_i)$  increases along  $x_i$ .

Note: All data is approximate.

 [Return to Figure 21.1](#)

## Figure 21.2 Text Alternative (Chapter 21)

---

 [Return to Figure 21.2](#)

A) The curve decreases with a slope of  $\Delta T$  over  $\Delta i$  lesser than zero, with direction of heat flow from left to right.

B) The curve increases with a slope  $\Delta T$  over  $\Delta i$  greater than zero, with direction of heat flow from right to left.

 [Return to Figure 21.2](#)

## Figure 21.7 Text Alternative (Chapter 21)

---

[Return to Figure 21.7](#)

A) The horizontal axis represents  $t$ . The vertical axis represents  $y$ .

The data is as follows:

The regular concave up curve increases through six points in a constant manner, starting from the origin.

After differentiation:

B) The horizontal axis represents  $t$ . The vertical axis represents  $dy$  over  $dt$ .

The data is as follows:

The regular straight upward sloping curve increases through five points in a constant manner, starting at a distance from the origin.

C) The horizontal axis represents  $t$ . The vertical axis represents  $y$ .

The data is as follows:

The irregular concave up curve increases through six points, starting from the origin showing slight bending at fourth and fifth point.

After differentiation:

D) The horizontal axis represents  $t$ . The vertical axis represents  $dy$  over  $dt$ .

The data is as follows:

The irregular upward sloping curve increases through five points at a distance from the origin, showing bending at third and fourth point.

Note: All data is approximate.

[Return to Figure 21.7](#)

# Graph Shows Graphic Text Alternative (Chapter 21)

---

 [Return to Graph Shows Graphic](#)

The horizontal axis ranges from 0 through 100 in increments of 10. The vertical axis ranges from negative 1.5 through 1.5 in increments of 0.5.

The data is as follows:

X data:

The curve begins at (1, 0.6) and ends at (98, 0.24) showing approximately equal amplitudes at each cycle.

Diff data:

The curve begins at (0, 1.25) and ends at (95, 1.1) showing slightly decreasing amplitudes towards the end.

Note: All data is approximate.

 [Return to Graph Shows Graphic](#)



## Figure 21.8 Text Alternative (Chapter 21)

---

 [Return to Figure 21.8](#)

The horizontal axis ranges from 0.0 through 0.8 in increments of 0.1. The vertical axis ranges from negative 20 through 20 in increments of 10.

The data is as follows:

The curve begins at (0.0, 26.5) and ends at (0.8, negative 24) decreasing through (0.05, 11), (0.15, 0), (0.25, 4), (0.35, 8), (0.45, 9), (0.55, 1.5), (0.65, negative 9), and (0.75, negative 20).

Note: All data is approximate.

 [Return to Figure 21.8](#)

# Figure 21.9 Text Alternative (Chapter 21)

---

 [Return to Figure 21.9](#)

The horizontal axis ranges from 0.0 through 0.8 in increments of 0.1. The vertical axis ranges from negative 20 through 20 in increments of 10.

The data is as follows:

The curve begins at (0.0, 26.5) and ends at (0.8, negative 24) decreasing through (0.05, 11), (0.15, 0), (0.25, 4), (0.35, 8), (0.45, 9), (0.55, 1.5), (0.65, negative 9), and (0.75, negative 20).

The data points are at:

(0.0, 11), (0.1, 5.5), ((0.2, 1.5), (0.3, 7), (0.4, 9), (0.5, 6), (0.6, negative 5), (0.7, 4), and (0.8, negative 21).

There are two dots that are visible above the curve, while two dots are visible below the curve, and five dots are visible on the curve.

Note: All data is approximate.

 [Return to Figure 21.9](#)

## Figure 21.10 Text Alternative (Chapter 21)

---

 [Return to Figure 21.10](#)

The horizontal axis ranges from negative 2.0 through 0.00 in increments of 0.25. The vertical axis ranges from 1.00 through 3.00 in increments of 0.25.

The data from bottom left to top right is as follows:

The dashed line decreases from (negative 2.00, 1.32) to (negative 1.80, 1.00).

The line decreases from (negative 2.00, 2.00) to (negative 1.50, 1.00).

The closed curve begins at (negative 1.00, 1.00) to (negative 0.50, 1.00), through (negative 1.50, 2.00) and (negative 1.25, 2.20).

The line decreases from (negative 1.50, 3.00) to (0.00, 1.00).

The dashed line decreases from (negative 1.00, 3.00) to (0.00, 1.62).

The dashed line decreases from (negative 0.74, 3.00) to (0.00, 2.00).

The dashed line decreases from (negative 0.50, 3.00) to (0.00, 2.30).

The dashed line decreases from (negative 0.30, 3.00) to (0.00, 2.55).

The dashed line decreases from (negative 0.13, 3.00) to (0.00, 2.80).

The arrows at intersection across the range (negative 2.00, 1.00), (negative 2.00, 2.00), (negative 1.50, 1.75), (negative 1.25, 1.50), and (negative 1.25, 1.00) are directed diagonally leftwards.

The arrows at intersection across the range (negative 1.50, 1.00), (negative 1.25, 2.50), (negative 1.00, 2.00), (negative 0.75, 1.75), (negative 0.50, 1.25), and (0.25, 1.00), (negative 0.00, 1.00) and (0.00, 3.00) are directed diagonally rights.

The arrows at intersection of (negative 1.25, 2.00), (negative 1.25, 1.75), (negative 1.00, 1.50), and (0.75, 1.25).

Note: All data is approximate.

 [Return to Figure 21.10](#)

## Figure P21.34 Text Alternative (Chapter 21)

---

 [Return to Figure P21.34](#)

The height of the water between the sediments layer and the lid is labeled as  $H$ . A probe with an ovular end is inserted through the lid into the water. The sediment oxygen demand, or SOD, flows from the water into the sediments layer.

 [Return to Figure P21.34](#)

# Figure PT6.1 Text Alternative (Chapter 22)

---

[!\[\]\(8af806fb1314382d09bc5ec5b767526c\_img.jpg\) Return to Figure PT6.1](#)

Left panel:

Physical law leads to ODE finally leading to solution.

Right panel:

$F = ma$  leads to  $\frac{dv}{dt} = g - \frac{cd}{m}v^2$  which further leads to analytical (calculus) where  $v = \sqrt{\frac{gm}{cd}} \tanh\left(\sqrt{\frac{gd}{m}}t\right)$  and numerical (computer) where  $v_{i+1} = v_i + (g - \frac{cd}{m}v_i^2)\Delta t$ .

[!\[\]\(0aff635c4179ba9e710b00f4b01d3b20\_img.jpg\) Return to Figure PT6.1](#)

## Figure PT6.2 Text Alternative (Chapter 22)

---

[Return to Figure PT6.2](#)

A) The horizontal axis represents  $x$  ranging from 0 through 3. The vertical axis represents  $y$  ranging from 0 through 4.

The data is as follows:

The curve increases from (0, 1) beyond (3, 4) showing crests before (1, 3) and after (3, 4) and trough across (2, 2).

B) The horizontal axis represents  $x$  ranging from 0 through 3. The vertical axis represents  $dy$  over  $dx$  ranging from negative 8 to 8.

The data is as follows:

The curve decreases from (0, 8.1) beyond (3, negative 8) showing trough between (1, 0) and (2, 0) and crest between (2, 0) and beyond (3, 0).

Note: All data is approximate.

[Return to Figure PT6.2](#)

## Figure PT6.3 Text Alternative (Chapter 22)

---

 [Return to Figure PT6.3](#)

The curve for  $c$  equals 1 (third from the top) is highlighted in different color.

 [Return to Figure PT6.3](#)

## Figure 22.1 Text Alternative (Chapter 22)

---

 [Return to Figure 22.1](#)

The true curve increases from origin passing through points along  $t_i$  and  $t_i + 1$  on horizontal axis (the distance between the two points is labeled as  $h$ ).

The predicted tangent touches the true curve along  $t_i$  and increases from there to a point along  $t_i + 1$  on horizontal axis but slightly over the end point of the true curve. The difference between the end points of true curve and predicted tangent is labeled as error.

Note: All data is approximate.

 [Return to Figure 22.1](#)



## Figure 22.2 Text Alternative (Chapter 22)

---

 [Return to Figure 22.2](#)

The horizontal axis represents  $t$  ranging from 0 through 4 in increments of 1. The vertical axis represents  $y$  ranging from 0 through 60 in increments of 20.

The data is as follows:

True solution:

The regular curve increases from (0, 0) to (3.9, 80) showing increasing trend after (2, 16).

Euler solution:

The irregular curve increases from (0, 0) to (4, 55) through (1, 5), (2, 12), and (3, 25).

Note: All data is approximate.

 [Return to Figure 22.2](#)

## Figure 22.4 Text Alternative (Chapter 22)

---

 [Return to Figure 22.4](#)

A) The curve begins from the origin passing through point along  $t_i$  on the horizontal axis. A tangent with slope equals  $f$  of  $(t_i, y_i)$  stretches to a tangent with slope equals  $f$  of  $(t_i + 1, y_{i+1})$  passing through a point along  $t_i + 1$  on the horizontal axis.

B) The curve begins from the origin passing through point along  $t_i$  and  $t_i + 1$  on horizontal axis. A tangent with slope equals  $f$  of  $(t_i, y_i)$  plus  $f$  of  $(t_i + 1, y_{i+1})$  whole divided by 2) passes through a point along  $t_i$  on the horizontal axis, stretching to point  $t_i + 1$  along the horizontal axis.

Note: All data is approximate.

 [Return to Figure 22.4](#)

## Figure 22.5 Text Alternative (Chapter 22)

---

 [Return to Figure 22.5](#)

$Y_{mI} + f(t_i, y_{mi}) + f(t_{I+1}, y_{j-I+1})$  whole divided by 2, whole multiplied with  $h$  leads to  $y_{j-I+1}$ . An arrow from  $y$  from  $j$  to  $I+1$  points to  $Y_{mI} + f(t_i, y_{mi}) + f(t_{I+1}, y_{j-I+1})$  whole divided by 2, and then multiplied with  $h$ .

 [Return to Figure 22.5](#)

## Figure 22.6 Text Alternative (Chapter 22)

---

 [Return to Figure 22.6](#)

A) The curve begins from the origin passing through points along  $t_i$ ,  $t_i + \text{half}$ , and  $t_i + 1$  on horizontal axis. A tangent passes through the first point stretching to another tangent with slope equals  $f$  of  $(t_i + \text{half}, y_i + \text{half})$  passes through point along  $t_i + \text{half}$  on the horizontal axis above the curve.

B) The curve begins from the origin passing through points along  $t_i$  and  $t_i + 1$  on horizontal axis. A tangent with slope equals  $f$  of  $(t_i + \text{half}, y_i + 1 \text{ half})$  passes through a point along  $t_i$  on the horizontal axis stretching to point  $t_i + 1$  along the horizontal axis.

Note: All data is approximate.

 [Return to Figure 22.6](#)

## Figure 22.7 Text Alternative (Chapter 22)

---

 [Return to Figure 22.7](#)

A tangent  $k_1$  begins at point along  $t_i$  on the horizontal axis and ends at point along  $t_i + 1$  on the horizontal axis with slope labeled as  $\theta$ . The tangent  $k_1$  with slope  $k_1$  stretches to tangent  $k_2$  at  $q$  point along  $t_i + 1$  divided by 2 on the horizontal axis (at the top). The tangent  $k_1$  with slope  $k_2$  stretches to tangent  $k_3$  at a point along  $t_i + 1$  divided by 2 on the horizontal axis (at the center). The tangent  $k_1$  with slope  $k_3$  stretches to tangent  $k_4$  at a point along  $t_i + 1$  on horizontal axis (at the bottom-right). The distance between  $t_i$  and  $t_i + 1$  along horizontal axis is labeled as  $h$ .

Note: All data is approximate.

 [Return to Figure 22.7](#)

## Figure 22.9 Text Alternative (Chapter 22)

---

 [Return to Figure 22.9](#)

The horizontal axis represents  $t$  ranging from 0.00 through 2.00 in increments of 0.25. The vertical axis represents  $y$  ranging from 0.00 through 2.00 in increments of 0.25.

The data is as follows:

y1: The curve decreases from (0.00, 0.00) to (2.00, 1.00) through (0.25, 1.25), (0.75, 0.90) and (1.75, 0.99).

y2: The curve increases from (0.00, 0.00) to (2.00, 1.00) through (0.50, 0.45), (1.00, 0.90) and (1.50, 1.00).

Note: All data is approximate.

 [Return to Figure 22.9](#)

## Figure 22.10 Text Alternative (Chapter 22)

---

[!\[\]\(35e4f762fc1cfea5610d92e2d225d5b4\_img.jpg\) Return to Figure 22.10](#)

A) Euler time plot: The horizontal axis ranges from 0 through 40 in increments of 10. The vertical axis ranges from 0 through 10 in increments of 5.

The data is as follows: The solid curve begins from (0, 2.5) and ends at (40, 14) showing uniform intervals and constantly increasing amplitude throughout.

The dashed curve begins from (0, 2) and ends at (40, 0) showing uniform intervals and increasing amplitude throughout.

B) Euler phase plot: The horizontal axis ranges from 0 through 10 in increments of 5. The vertical axis ranges from 0.0 through 7.5 in increments of 2.5.

The data is as follows: The spiral curve begins from (2, 1, 0.125) and ends at (14, 1) through (5, 2.7), (5, 3.2), and (5, 7).

C) RK4 time plot: The horizontal axis ranges from 0 through 40 in increments of 10. The vertical axis ranges from 0.0 through 4 in increments of 2.

The data is as follows: The solid curve begins from (0, 2) and ends at (40, 1.8) showing uniform intervals and approximately equal amplitude throughout, which is greater.

The dashed curve begins from (0, 0.5) and ends at (40, 0.7) showing uniform intervals and approximately equal amplitude throughout, which is lesser.

D) RK4 phase plot: The horizontal axis ranges from 1 through 5 in increments of 1. The vertical axis ranges from 1 through 3 in increments of 1.

The data is as follows: The ovular curve begins and ends at (2, 1) through (3, 1), (4, 1.1), (5.6, 2), (4.5, 3), (3, 3.8), (1.3, 3), and (1, 2).

Note: All data is approximate.

[!\[\]\(0d7ca0919e6c47bbd874bfa0189fe22e\_img.jpg\) Return to Figure 22.10](#)

## Figure 22.11 Text Alternative (Chapter 22)

---

[!\[\]\(d263118e0bfd47dc6bc704167d936b83\_img.jpg\) Return to Figure 22.11](#)

The horizontal axis represents  $t$  ranging from 0.00 through 20.0 in increments of 2.5. The vertical axis represents  $x$  ranging from negative 20 through 15 in increments of 5.

The data is as follows:

$x(0)$  equals  $y(0)$  equals  $z(0)$  equals 5:

The curve begins at (0.0, 5) and ends at (20.0, negative 12.5) showing decreasing fluctuations between (0.1, negative 12) and (6.0, negative 16), (8. 2, negative 12.5) and (12.4, negative 15), (13.5, negative 12.7) and (16.25, negative 16.5); and (18.2, negative 14) and (20.0, negative 12.5). It shows increasing fluctuations between (7, 13) and (8, 18); (12.7, 0) and (13.2, 0); and (16.5, 3.5) and (18.3, 2).

$x(0)$  equals 5.001,  $y(0)$  equals  $z(0)$  equals 5:

The curve begins at (0.0, 5) and ends at (20.0, negative 12.5) showing decreasing fluctuations between (0.1, negative 12) and (6.0, negative 16), (8. 2, negative 12.5) and (12.4, negative 15); and (13.5, negative 13) and (16, negative 18). It shows increasing fluctuations between (7, 13) and (8, 18); (12.7, 0) and (13, 0); and (16.2, 11.5) and (20.0, 4). The curve overlaps with the first curve across first and second decreasing fluctuations and first increasing fluctuation.

Note: All data is approximate.

[!\[\]\(f95dab70c751fda7d824b8b03650f7aa\_img.jpg\) Return to Figure 22.11](#)



# Figure 22.12 Text Alternative (Chapter 22)

---

 [Return to Figure 22.12](#)

Left panel:

The horizontal axis represents  $x$  ranging from negative 10 through 10 in increments of 10. The vertical axis represents  $y$  ranging from negative 20 through 20 in increments of 10.

The data is as follows:

The  $x$   $y$  phase plane looks like a figure 8 turned 45 degrees clockwise where the left side is darker than right. There are several strands where the solution crosses over from one orbit into the other.

Middle panel:

The horizontal axis represents  $x$  ranging from negative 10 through 10 in increments of 10. The vertical axis represents  $y$  ranging from 10 through 40 in increments of 10.

The data is as follows:

The  $z$   $x$  phase plane has one elliptical orbit with its semimajor axis pointed up and two the left and the other one with its semimajor axis pointed up and to the right. The left side is darker than the right side. There are several strands where the solution crosses over from one orbit into the other.

Right panel:

The horizontal axis represents  $y$  ranging from negative 20 through 20 in increments of 20. The vertical axis represents  $z$  ranging from 10 through 40 in increments of 10.

The data is as follows:

The  $z$   $y$  phase plane has two elliptical orbits with their semimajor axes both oriented vertically where the left side is darker than right side. There are several strands where the solution crosses over from one orbit into the other.

Note: All data is approximate.

 [Return to Figure 22.12](#)

## Figure 22.13 Text Alternative (Chapter 22)

---

 [Return to Figure 22.13](#)

The axis representing  $x$  ranges from negative 15 through 15 in increments of 5. The axis representing  $y$  ranges from negative 20 through 20 in increments of 10. The axis representing  $z$  ranges from 5 through 40 in increments of 5.

The data is as follows:

The xyz phase plane has two elliptical orbits with their semimajor axes both oriented vertically where the left side is darker than right side. There are several strands where the solution crosses over from one orbit into the other.

Note: All data is approximate.

 [Return to Figure 22.13](#)

## Figure P22.21 Text Alternative (Chapter 22)

---

 [Return to Figure P22.21](#)

The diameter is centered at a distance  $e$  below the trapezoidal base of the wall. The height of fluid above the trapezoidal base of the wall is given by  $h$ , and the surface area of the pond if the fluid is at a height  $h$  is given by  $A(h)$ .

 [Return to Figure P22.21](#)

## Figure P22.22 Text Alternative (Chapter 22)

---

[!\[\]\(21199eb166cc97331a0c54c649195dcc\_img.jpg\) Return to Figure P22.22](#)

The top story has a mass  $m_3$  of 8000 kilograms and is connected to the story below by a spring  $k_3$  equals 1800 kilo-Newton per meter. The middle story has a mass  $m_2$  of 10000 kilograms and is connected to the story above by  $k_3$  and the story below by  $k_2$  equals 2400 kilo-Newton per meter. The lowest story has a mass  $m_1$  of 12000 kilograms and is connected to the story above by  $k_2$  and to the ground by spring  $k_1$  equals 3000 kilo-Newton per meter.

[!\[\]\(dfbd6b3763a6d1d9afaa974f64e2e4b5\_img.jpg\) Return to Figure P22.22](#)

## Figure P22.25 Text Alternative (Chapter 22)

---

 [Return to Figure P22.25](#)

The bacteria ( $X$ ) (with an arrow directed upwards) leads to substrate ( $S$ ) with  $K_d X$ , while substrate ( $S$ ) leads to bacteria ( $X$ ) through  $Y$  with  $\frac{1}{Y} k_g X$  and  $k_g X$ .  $Y$  leads to  $Y$  minus 1 whole divided by  $Y$  times  $k_g X$ . There are two differently shaded unlabeled ovals across the top, while an unknown output is given out.

 [Return to Figure P22.25](#)

## Figure 23.1 Text Alternative (Chapter 23)

---

 [Return to Figure 23.1](#)

The horizontal axis represents  $t$  ranging from 0 through 3 in increments of 1. The vertical axis represents  $y$  ranging from 0 through 1.

The data is as follows:

The curve begins at  $(0, 0.5)$  and ends beyond  $(3.5, 0.8)$  through  $(1.9, 0.25)$ , and  $(1.9, 1.2)$ .

Note: All data is approximate.

 [Return to Figure 23.1](#)

## Figure 23.3 Text Alternative (Chapter 23)

---

[!\[\]\(35e4f762fc1cfea5610d92e2d225d5b4\_img.jpg\) Return to Figure 23.3](#)

The horizontal axis represents  $t$  ranging from 0.0 through 20.0 in increments of 2.5. The vertical axis represents  $y$  ranging from 1 through 5 in increments of 1.

The data is as follows:

$y_1$ : The curve begins at (0.0, 2) and ends at (20.0, 2) through (2.4, 5), (8.5, 5), and (16.0, 5) showing approximately equal amplitude throughout, which is greater.

$y_2$ : The curve begins at (0.0, 1) and ends at (20.0, 1) through (3, 3.6), (10.0, 3.6), and (16.5, 3.6) showing approximately equal amplitude throughout, which is lesser.

Note: All data is approximate.

The horizontal axis represents  $y_1$  ranging from 1 through 5 in increments of 1. The vertical axis represents  $y_2$  ranging from 1.0 through 3.5 in increments of 0.5.

The data is as follows:

The curve begins and ends at (3, 1.0) passing through (2, 1.0), (2, 3.5), (3.3, 3.5), (4, 3.3), (4.5, 3.0), (5.2, 2.5), (5.5, 2.0), and (5, 1.5).

Note: All data is approximate.

[!\[\]\(51514032c8ca341817228f39f1307b05\_img.jpg\) Return to Figure 23.3](#)

## Figure 23.4 Text Alternative (Chapter 23)

---

[!\[\]\(d263118e0bfd47dc6bc704167d936b83\_img.jpg\) Return to Figure 23.4](#)

A)  $\text{rtol}$  equals 0.001:

The horizontal axis ranges from 0.0 through 4.0 in increments of 0.5. The vertical axis ranges from 0.25 through 1.75 in increments of 0.25.

The data is as follows:

The curve begins at (0.0, 0.50) and ends at (4.0, 0.625) through (1.7, 0.20), and (2.2, 1.80). There are approximately 35 points from beginning of the curve to the end.

B)  $\text{rtol}$  equals 1.e minus 5:

The horizontal axis ranges from 0.0 through 4.0 in increments of 0.5. The vertical axis ranges from 0.25 through 1.75 in increments of 0.25.

The data is as follows:

The curve begins at (0.0, 0.50) and ends at (4.0, 0.625) through (1.7, 0.20), and (2.2, 1.80). There are approximately 60 points from beginning of the curve to the end.

Note: All data is approximate.

[!\[\]\(d8ab143e904bfa3467271eec5af75a9b\_img.jpg\) Return to Figure 23.4](#)



## Figure 23.5 Text Alternative (Chapter 23)

---

 [Return to Figure 23.5](#)

The horizontal axis represents time (in seconds) ranging from 0 through 2 in increments of 1. The vertical axis ranges from 0 through 200 in increments of 50.

The data is as follows:

Height (in meters):

The concave down decreasing curve begins at (0, 200) and ends at (9.5, 0) through (4, 200), (7.2, 100), and (8.5, 50).

Velocity (in meters per second):

The increasing curve begins at (0, negative 12) and ends at (9.4, 49.8) through (2, 0), (4, 24), and (6, 34).

Note: All data is approximate.

 [Return to Figure 23.5](#)

## Figure 23.6 Text Alternative (Chapter 23)

---

 [Return to Figure 23.6](#)

A) The one step method shows taking the tangent to the curve at a single location and following that location for a time step. The dot on curve across  $t_i$  is directed to the right to a dot under the curve across  $t_i + 1$ .

B) The multistep method uses derivatives at multiple locations to come up with an estimated derivative at each point. The dots are visible on the curve across  $t_i - 2$ ,  $t_i - 1$ , and  $t_i$  where dot across  $t_i$  is directed to the right to a dot under the curve across  $t_i + 1$ .

 [Return to Figure 23.6](#)

## Figure 23.7 Text Alternative (Chapter 23)

---

 [Return to Figure 23.7](#)

A) A concave down curve passes along  $t_i - 1$ ,  $t_i$ , and  $t_i + 1$  on horizontal axis. The three tangents pass along three points of  $t_i - 1$ ,  $t_i$ , and  $t_i + 1$  on the horizontal axis, where slope for last tangent equals  $f(t_i + 1, y_{i+1})$ . The first two tangents pass through points on the curve, while the last curve passes through a point above the curve. The first tangent stretches to last tangent. The slope of third tangent on  $t_i + 1$  is labeled as  $f(t_i + 1, y_{i+1})$ .

B) A concave down curve passes along  $t_i$  and  $t_i + 1$  on horizontal axis. The tangent with slope  $f(t_i, y_i) + f(t_i + 1, y_{i+1})$  whole divided by 2 stretches to the point along  $t_i + 1$  on the horizontal axis.

 [Return to Figure 23.7](#)

## Figure 23.8 Text Alternative (Chapter 23)

---

 [Return to Figure 23.8](#)

The horizontal axis ranges from 0 through 4 in increments of 2. The vertical axis ranges from 0 through 3 in increments of 1.

The data is as follows:

The curve begins at (0, 0) and ends at (4, 3) through (0, 1), (2, 2.5), and (2.5, 2.9). A horizontal arrow ranges from (0, 0.75) to (1.5, 0.75), while diagonal arrow ranges from (0, 0.75) to (1.4, 1.8). The range between (0, 0) through (0, 1.5) is encircled.

Graph inside shows the following data:

The horizontal axis ranges from 0 through 0.02 in increments of 0.01. The vertical axis ranges from 0 through 1.

The data is as follows:

The curve begins at (0, 0) and ends at (0.02, 1) through (0.005, 1), (0.01, 1), and (0.02, 1).

Note: All data is approximate.

 [Return to Figure 23.8](#)

## Figure 23.9 Text Alternative (Chapter 23)

---

 [Return to Figure 23.9](#)

A) The horizontal axis represents  $t$  ranging from 0 through 0.006 in increments of 0.002. The vertical axis represents  $y$  ranging from 0 through 1.5 in increments of 0.5.

The data is as follows:

$h$  equals 0.0005:

The curve begins at (0, 0) and ends at (0.006, 1) through (0.001, 0.7).

Exact:

The curve begins at (0, 0) and ends at (0.006, 1) through (0.001, 0.6).

$h$  equals 0.0015:

The curve begins at (0, 0) and ends at (0.006, 0.9) through (0.0015, 1.5) and (0.003, 0.6).

B) The horizontal axis represents  $t$  ranging from 0 through 0.4 in increments of 0.01. The vertical axis represents  $y$  ranging from 0 through 2 in increments of 1.

The data is as follows:

$h$  equals 0.05:

The curve begins at (0, 0) and ends at (0.4, 1.5) through (0.005, 0.5).

Exact:

The curve begins at (0, 0) and ends at (0.4, 1.5) through (0, 0.01) and (0.2, 1.25).

Note: All data is approximate.

 [Return to Figure 23.9](#)

# Figure 23.10 Text Alternative (Chapter 23)

---

[!\[\]\(d0a1791f26d167e866e44ebbf83efebe\_img.jpg\) Return to Figure 23.10](#)

A)  $\mu$  equals 1:

The horizontal axis ranges from 0.0 through 20.0 in increments of 2.5. The vertical axis ranges from negative 2 through 2 in increments of 1.

The data is as follows:

y1:

The curve begins at (0.0, 1) and ends at (20.0, 2) through (7, 2) and (13, 2) showing constant and equal amplitude throughout.

y2:

The curve begins at (0.0, 1) and ends at (20.0, 0) through (6, 2.7), and (12.5, 2.7) showing constant and equal amplitude throughout.

B)  $\mu$  equals 1000:

The horizontal axis ranges from 0 through 6000 in increments of 1000, while vertical axis ranges from negative 2.0 through 2.0 in increments of 0.5.

The data is as follows:

The curve begins at (0, 1.0) and ends at (6000, 1.75) through (850, 2.0), (2400, 2.0), and (4000, 2.0) showing equal amplitude throughout.

Note: All data is approximate.

[!\[\]\(aab88c0d099e5d18d6533a97b13ec28d\_img.jpg\) Return to Figure 23.10](#)

## Figure 23.11 Text Alternative (Chapter 23)

---

 [Return to Figure 23.11](#)

The horizontal axis represents time (in seconds) ranging from 0 through 50 in increments of 10. The vertical axis represents ranges from negative 60 through 20 in increments of 20.

The data is as follows:

X (in meters):

The curve begins at (0, 0) and ends at (50, negative 45) through (9, negative 30) and (25, negative 40) showing fading amplitude.

V (in meters per second):

The curve begins at (0, 0) and ends at (50, 0) through (12, 10) and (30, 0) showing fading amplitude.

Note: All data is approximate.

 [Return to Figure 23.11](#)

## Figure 23.12 Text Alternative (Chapter 23)

---

 [Return to Figure 23.12](#)

A cylinder of fluid with a radius of  $R$  is filled from above at a flow rate of  $Q_{in}$ . A pipe is connected to the cylinder at a height  $y_{low}$  above the bottom of the cylinder first loops up to a height  $y_{high}$ , then down to a height below the bottom of the cylinder, then finally back up to the same height as the bottom of the cylinder at  $y = 0$  (where the water exits). The pipe has radius  $r$  and the flow rate out of the pipe is  $Q_{out}$ .

 [Return to Figure 23.12](#)



## Figure 23.13 Text Alternative (Chapter 23)

---

 [Return to Figure 23.13](#)

The horizontal axis ranges from y low to y high. The vertical axis ranges from 0 through 1.

An arrow begins from 0 to y high through y low, while another arrow begins from y high to 1, and an arrow begins from 1 beyond y high.

Another arrow begins from 1 to y low through y high, while another arrow begins at 1 to y low, and an arrow begins from y low to 0.

Note: All data is approximate.

 [Return to Figure 23.13](#)

## Figure 23.14 Text Alternative (Chapter 23)

---

 [Return to Figure 23.14](#)

The horizontal axis represents  $t$  (in seconds) ranging from 0 through 100 in increments of 20. The vertical axis represents  $y$  (in meters) ranging from 0.0 through 0.07 in increments of 0.01.

The data is as follows:

The curve begins at (0, 0.00) and ends at (100, 0.0325) through (10, 0.07), (30, 0.07), (50, 0.07), (70, 0.07), and (90, 0.07) showing equal amplitude throughout.

Note: All data is approximate.

 [Return to Figure 23.14](#)

## Figure 23.15 Text Alternative (Chapter 23)

---

 [Return to Figure 23.15](#)

The horizontal axis represents  $t$  (in seconds) ranging from 0 through 100 in increments of 20. The vertical axis represents  $y$  (in meters) ranging from 0.0 through 0.10 in increments of 0.02.

The data is as follows:

The curve begins at (0, 0.00) and ends at (100, 0.05) through (15, 0.10), (42, 0.10), (68, 0.10), and (92, 0.10) showing equal amplitude throughout.

Note: All data is approximate.

 [Return to Figure 23.15](#)

## Figure 23.19 Text Alternative (Chapter 23)

---

 [Return to Figure 23.19](#)

Spring  $k_1$  is connected to a wall and has uncompressed length  $L_1$  and is attached to  $m_1$ . Mass  $m_1$  has width  $w_1$  and is connected to spring  $k_2$  which has uncompressed length  $L_2$ . Finally, this spring is connected to mass  $m_2$ , which has width  $w_2$ . The distance  $x_1$  is measured from the wall to the right end of spring 1 and has a value of  $L_1$  when the spring is un-stretched. The distance  $x_2$  is measured at the right end of spring 2 and has a value of  $L_1$  plus  $w_1$  plus  $L_2$  when the springs are unstretched.

 [Return to Figure 23.19](#)

## Figure 23.22 Text Alternative (Chapter 23)

---

 [Return to Figure 23.22](#)

The first pendulum is of length  $L_1$  and mass  $m_1$ , makes an angle of  $\theta_1$  directed counter-clockwise from the y axis. The second pendulum of length  $L_2$  and mass  $m_2$ , makes an angle of  $\theta_2$  directed counter-clockwise from the y axis.

 [Return to Figure 23.22](#)

## Figure 23.23 Text Alternative (Chapter 23)

---

 [Return to Figure 23.23](#)

The force  $F_G$  is directed downwards across the center, and  $F_D$  is directed downwards on the right.

 [Return to Figure 23.23](#)

## Figure 23.28 Text Alternative (Chapter 23)

---

 [Return to Figure 23.28](#)

The curve begins at the origin of this coordinate system with velocity  $v_0$  to the right across the  $x$  axis. At a point across the center, the curve has a horizontal speed to the right as  $v_x$ , a vertical speed down as  $v_y$ , and a directed velocity  $v$  is tangent to the curve. An angle  $\theta$  is formed between  $V_x$  and  $V$ .

 [Return to Figure 23.28](#)

## Figure 23.29 Text Alternative (Chapter 23)

---

 [Return to Figure 23.29](#)

The presumed deflection is a concave down curve that has a deflection of 0 at the left and a maximum at the right. The slope of the deflection is zero at the wall and an arrow shows  $x$  directed to the right across the far right side of the beam.

 [Return to Figure 23.29](#)



## Figure 24.1 Text Alternative (Chapter 24)

---

 [Return to Figure 24.1](#)

The horizontal axis represents  $t$  ranging from 0. The vertical axis represents  $y$  ranging from 0.

A) The curve  $y_1$  starts at initial condition  $y_1, 0$  while curve  $y_2$  starts at initial condition  $y_2, 0$ . These curves are wound together with each other in an opposite overlapping manner depicted as intertwined sine waves.

The equations at the top read:

$\frac{dy_1}{dt}$  equals  $f_1$  of  $(t, y_1, \text{ and } y_2)$  and  $\frac{dy_2}{dt}$  equals  $f_2$  of  $(t, y_1, \text{ and } y_2)$ , where at  $t$  equals 0,  $y_1$  equals  $y_1, 0$ , and  $y_2$  equals  $y_2, 0$ .

B) The concave down curve  $y_L$  starts at boundary condition  $y_0$  and ends at boundary condition  $y_L$ .

The equations at the bottom read:

$\frac{d^2 y}{dx^2}$  equals  $f$  of  $(x, y)$ , where at  $x$  equals 0,  $y$  equals  $y_0$ ; and  $x$  equals  $L$ ,  $y$  equals  $y_L$ .

 [Return to Figure 24.1](#)

## Figure 24.2 Text Alternative (Chapter 24)

---

[!\[\]\(529949c2c3dadbaa4e538e8c643454bc\_img.jpg\) Return to Figure 24.2](#)

The rod shows a cross sectional area  $A_c$ . A small segment of the rod of length  $\Delta x$  near the middle of the rod is highlighted and enlarged. Its left end is at  $x$  while the right end is at  $x + \Delta x$ . The cross sectional area is  $A_c$  and the radius is  $r$ . The cylindrical surface area of the rod is given by  $A_s$ . The temperature of the air surrounding the rod is  $T_\infty$ , the left end of the rod is at  $x = 0$  is at temperature  $T_a$  and the right end of the rod at  $x = L$  is at temperature  $T_b$ .

[!\[\]\(99f58673407353e96a019fbca558fd72\_img.jpg\) Return to Figure 24.2](#)

## Figure 24.3 Text Alternative (Chapter 24)

---

 [Return to Figure 24.3](#)

The horizontal axis represents distance (in meters) ranging from 0 through 10 in increments of 2. The vertical axis represents temperature (in Kelvin) ranging from 200 through 400 in increments of 100.

The data is as follows:

The curve begins at (0, 300) and ends at (10, 410) through (2, 280), (4, 285), and (8, 330).

Note: All data is approximate.

 [Return to Figure 24.3](#)

## Figure 24.4 Text Alternative (Chapter 24)

---

 [Return to Figure 24.4](#)

The horizontal axis represents distance (in meters) ranging from 0 through 10 in increments of 2. The vertical axis represents temperature (in Kelvin) ranging from 200 through 600 in increments of 200.

The data is as follows:

A) The concave up increasing curve begins at (0, 300) and ends at (10, 580), with a data point marked below it at (10, 400).

B) The decreasing curve begins at (0, 300) and ends at (10, 280), with a data point marked above it at (10, 400).

C) The curve begins at (0, 300) and ends at (10, 400) through (4, 280), (5, 285), and (8, 330). The curve connects two data points visible at (0, 300) and (10, 400).

Note: All data is approximate.

 [Return to Figure 24.4](#)

## Figure 24.5 Text Alternative (Chapter 24)

---

 [Return to Figure 24.5](#)

The temperature at the right end is fixed at  $T_b$  (where  $x$  equals  $L$ ), while the surrounding air is at temperature  $T_\infty$  across the center and the left side due to convection (where  $x$  equals  $0$ ). The center of the rod is labeled as conduction between  $x$  equals  $0$  and  $x$  equals  $L$ .

 [Return to Figure 24.5](#)

## Figure 24.6 Text Alternative (Chapter 24)

---

 [Return to Figure 24.6](#)

The horizontal axis represents distance (in meters) ranging from 0 through 10 in increments of 2. The vertical axis represents temperature (in Kelvin) ranging from 200 through 400 in increments of 100.

The data is as follows:

The curve begins at (0, 245) and ends at (10, 410) through (2, 245), (4, 260), (6, 275), and (8, 310).

Note: All data is approximate.

 [Return to Figure 24.6](#)

## Figure 24.7 Text Alternative (Chapter 24)

---

 [Return to Figure 24.7](#)

The horizontal axis represents distance (in meters) ranging from 0 through 10 in increments of 2. The vertical axis represents temperature (in Kelvin) ranging from 200 through 400 in increments of 100.

The data is as follows:

Non-linear:

The curve begins at (0, 300) and ends at (10, 400) through (2, 250), (5, 230), and (8, 280).

Linear:

The curve begins at (0, 300) and ends at (9.8, 400) through (2, 290), (5, 295), and (8, 320).

Note: All data is approximate.

 [Return to Figure 24.7](#)

## Figure 24.8 Text Alternative (Chapter 24)

---

 [Return to Figure 24.8](#)

The nodes from left to right are labeled as  $T_1$ ,  $T_{i-1}$ ,  $T_i$ ,  $T_{i+1}$ , and  $T_{n-1}$ . The distance between the nodes  $T_{i-1}$  and  $T_i$  is labeled as  $\delta x$ .

 [Return to Figure 24.8](#)



## Figure 24.9 Text Alternative (Chapter 24)

---

 [Return to Figure 24.9](#)

The nodes on the right are labeled as  $T_0$  and  $T_1$  while distance between them is labeled as  $\Delta x$ .

 [Return to Figure 24.9](#)

## Figure 24.10 Text Alternative (Chapter 24)

---

 [Return to Figure 24.10](#)

The horizontal axis represents distance (in meters) ranging from 0 through 10 in increments of 2. The vertical axis represents temperature (in Kelvin) ranging from 200 through 400 in increments of 100.

The data is as follows:

The curve begins at (0, 250) (labeled as  $T'(0)$  equals 0) and ends at (10, 400) through (2, 251), (4, 265), (6, 280), and (8.1, 320).

The curve begins at (0, 250) (labeled as  $T'(0)$  equals negative 20) and ends at (10, 400) through (2, 300), (4, 300), (6, 305), and (8.1, 330).

Note: All data is approximate.

 [Return to Figure 24.10](#)

## Figure 24.11 Text Alternative (Chapter 24)

---

 [Return to Figure 24.11](#)

The horizontal axis represents distance,  $x$  (in meters) ranging from 0 through 10 in increments of 2. The vertical axis represents temperature (in Kelvin) ranging from 200 through 400 in increments of 100.

The data is as follows:

The curve begins at (0, 300) and ends at (10, 400) through (2, 250), (4, 230), (6, 240) labeled as finite difference), and (8.1, 280) (labeled as shooting).

Note: All data is approximate.

 [Return to Figure 24.11](#)

## Figure P24.10 Text Alternative (Chapter 24)

---

 [Return to Figure P24.10](#)

The temperature at the tip is given as  $u(x=1) = 1$  and the temperature on the base is given as  $u(x=0) = 0$ .

 [Return to Figure P24.10](#)

## Figure P24.12 Text Alternative (Chapter 24)

---

 [Return to Figure P24.12](#)

Multiple arrows directed to left, right and both directions are visible across the surface of rod.

 [Return to Figure P24.12](#)

## Figure P24.14 Text Alternative (Chapter 24)

---

 [Return to Figure P24.14](#)

There is a diffusion layer of width  $L$  across the left of the biofilm, with a bulk liquid layer across further left. The distance  $x$  is measured from the left edge of the diffusion layer with the positive direction being to the right.

 [Return to Figure P24.14](#)

## Figure P24.15 Text Alternative (Chapter 24)

---

[!\[\]\(35e4f762fc1cfea5610d92e2d225d5b4\_img.jpg\) Return to Figure P24.15](#)

The distance  $l_A$  is given along  $x$  axis while  $h_A$  is given along  $y$  axis. The cable is weighed down by a distributed load given as  $w$  equals  $w_0 (1 + \sin(\pi x / 2l_a))$ .

[!\[\]\(feabb98897b440bc8695a03336a6e2df\_img.jpg\) Return to Figure P24.15](#)

## Figure P24.16 Text Alternative (Chapter 24)

---

 [Return to Figure P24.16](#)

The length of the beam is supported by vertical forces from the pins at each end along  $y$  axis. The presumptive deflection of the beam is visible as concave up with zero deflection at either pin and a local minimum (the largest negative deflection) in the middle along  $x$  axis.

 [Return to Figure P24.16](#)



## Figure P24.18 Text Alternative (Chapter 24)

---

[!\[\]\(c8d96c8885d3000a912c2582004aed63\_img.jpg\) Return to Figure P24.18](#)

The top of the aquifer is labeled as water table, while the bottom is labeled as a confining bed (across  $x$  axis). The aquifer with water table shows a height of  $h$ . The groundwater flows from left to right and the distance between the ground surface and the water table also grows from left to right with the height of aquifer decreasing from left to right. The infiltration is shown going through the ground and into the water table.

[!\[\]\(666e09182d4cd268646ea700ea60dcdf\_img.jpg\) Return to Figure P24.18](#)

## Figure P24.22 Text Alternative (Chapter 24)

---

 [Return to Figure P24.22](#)

The temperature of rod on the left is fixed at  $T_0$  at its left end. There is no heat loss in the rod due to convection, only conduction along the rod. The tube present on the right side of the rod is surrounded by a fluid at a temperature  $T_\infty$  and heat moves through this tube both due to conduction in the tube and convection into the fluid.

 [Return to Figure P24.22](#)

## Figure P24.25 Text Alternative (Chapter 24)

---

 [Return to Figure P24.25](#)

A) A beam of length  $L$  is supported by a pin at the left end and is attached to a wall on its right end. The beam undergoes a distributed load that is zero at the left pinned end and increases linearly to a maximum of  $w_0$  at the right end.

B) The beam is deflecting due to the load. The ends of the beam have zero deflection. At the left end, the beam shows  $x$  equals 0, and  $y$  equals 0; and at the right end the beam shows  $x$  equals  $L$ , and  $y$  equals 0.

 [Return to Figure P24.25](#)

# Graph of X Against Y Graphic Text Alternative (Chapter 24)

---

[!\[\]\(21199eb166cc97331a0c54c649195dcc\_img.jpg\) Return to Graph of X Against Y Graphic](#)

The horizontal axis represents  $x$  ranging from 0.0 through 1.6 in increments of 0.2. The vertical axis represents  $y$  ranging from 0.0 through 1.0 in increments of 0.2.

The data is as follows:

The curve begins at (0.0, 1.0) and ends at (1.58, 0.0) through (0.2, 0.8), (0.4, 0.6), (0.8, 0.3), and (1.2, 0.08).

Note: All data is approximate.

[!\[\]\(c694a3ff3b077d76910920a6a1593ab4\_img.jpg\) Return to Graph of X Against Y Graphic](#)

# Graph Shows Graphic Text Alternative (Appendix A)

---

 [Return to Graph Shows Graphic](#)

The horizontal axis ranges from 0.0 through 1.0 in increments of 0.2, while vertical axis ranges from 0 through 100 in increments of 20.

The data is as follows:

The curve begins at (0.0, 5) and ends at (1.0, 18) showing decreasing trend after (0.31, 95).

Note: All data is approximate.

 [Return to Graph Shows Graphic](#)

# Graph Shows Graphic Text Alternative (Appendix A)

---

 [Return to Graph Shows Graphic](#)

The horizontal axis ranges from 300 through 460 in increments of 20, while vertical axis ranges from 0 through 60 in increments of 10.

The data points are at:

(309, 68), (312, 2), (350, 0), (370, 29), and (460, 15).

Note: All data is approximate.

 [Return to Graph Shows Graphic](#)

# Graph Shows Graphic Text Alternative (Appendix A)

---

[!\[\]\(35e4f762fc1cfea5610d92e2d225d5b4\_img.jpg\) Return to Graph Shows Graphic](#)

The horizontal axis ranges from 300 through 460 in increments of 20, while vertical axis ranges from 0 through 60 in increments of 10.

The data is as follows:

The curve begins at (312, 2) and ends at (350, 0), through (309, 68), (460, 30), and (370, 29).

Note: All data is approximate.

[!\[\]\(8d0f0e0fe25b320c33272c52aec1fbca\_img.jpg\) Return to Graph Shows Graphic](#)

# Graph Shows Graphic Text Alternative (Appendix A)

---

[!\[\]\(d263118e0bfd47dc6bc704167d936b83\_img.jpg\) Return to Graph Shows Graphic](#)

The horizontal axis ranges from 300 through 460 in increments of 20, while vertical axis ranges from 0 through 60 in increments of 10.

The data is as follows:

The curve begins at (309, 68) and ends at (460, 30) through (312, 2), (350, 0), and (370, 29).

Note: All data is approximate.

[!\[\]\(fa6f3af6bfa46c5d4a2d362681095beb\_img.jpg\) Return to Graph Shows Graphic](#)



# Graph Shows Graphic Text Alternative (Appendix A)

---

 [Return to Graph Shows Graphic](#)

The horizontal axis ranges from 0 through 300 in increments of 50, while vertical axis ranges from 7.5 through 10.5 in increments of 0.5.

The data is as follows:

The fluctuated curve begins at (0, 8.0) and ends at (310, 9.1) passing through multiple data points. The curve shows decreasing trend after (100, 10.0), (150, 10.25), (220, 10.0), (275, 10.4), and (290, 9.5).

Note: All data is approximate.

 [Return to Graph Shows Graphic](#)

# Graph Shows Graphic Text Alternative (Appendix A)

---

 [Return to Graph Shows Graphic](#)

The horizontal axis ranges from 0 through 300 in increments of 50, while vertical axis ranges from 7.5 through 10.5 in increments of 0.5.

The data is as follows:

The data points are closely visible across (0, 0), (120, 0), (100, 10.0), and (0, 10.0); (125, 8.5), (125, 10.3), (300, 8.5), and (300, 10.3); and (260, 8.0), (250, 8.49), (300, 8.0), and (300, 8.49).

Note: All data is approximate.

 [Return to Graph Shows Graphic](#)

# Graph Shows Graphic Text Alternative (Appendix A)

---

[!\[\]\(1d3a1175dd4902218e694b9c098adb83\_img.jpg\) Return to Graph Shows Graphic](#)

The horizontal axis ranges from 0 through 300 in increments of 50, while vertical axis ranges from 7.5 through 10.5 in increments of 0.5.

The data is as follows:

The fluctuated curve begins at (0, 8.0) and ends at (310, 9.1) showing decreasing trend after (40, 9.6), (100, 10.0), (150, 10.25), (220, 10.0), (275, 10.4), and (290, 9.5).

Note: All data is approximate.

[!\[\]\(cbe2492b119e39e02a1dab2af4a4b296\_img.jpg\) Return to Graph Shows Graphic](#)

# Graph Shows Graphic Text Alternative (Appendix A)

---

 [Return to Graph Shows Graphic](#)

The horizontal axis ranges from 0.0 through 1.0 in increments of 0.2, while vertical axis ranges from 0 through 100 in increments of 20.

The data is as follows:

The curve begins at (0.0, 5) and ends at (1.0, 18) showing decreasing trend after (0.31, 95).

Note: All data is approximate.

 [Return to Graph Shows Graphic](#)

# Graph with Gridlines Show Graphic Text Alternative (Appendix A)

---

[!\[\]\(8af806fb1314382d09bc5ec5b767526c\_img.jpg\) Return to Graph with Gridlines Show Graphic](#)

The horizontal axis ranges from 0.0 through 1.0 in increments of 0.2, while vertical axis ranges from 0 through 100 in increments of 20.

The data is as follows:

The curve begins at (0.0, 5) and ends at (1.0, 18) showing decreasing trend after (0.31, 95).

Note: All data is approximate.

[!\[\]\(0aff635c4179ba9e710b00f4b01d3b20\_img.jpg\) Return to Graph with Gridlines Show Graphic](#)

# Graph with Gridlines Show Graphic Text Alternative (Appendix A)

---

[!\[\]\(d0a1791f26d167e866e44ebbf83efebe\_img.jpg\) Return to Graph with Gridlines Show Graphic](#)

The horizontal axis ranges from 0.0 through 1.0 in increments of 0.2, while vertical axis ranges from 0 through 100 in increments of 20.

The data is as follows:

The curve begins at (0.0, 5) and ends at (1.0, 18) showing decreasing trend after (0.31, 95).

Note: All data is approximate.

[!\[\]\(5a132f13505a6571904d622757b7a8f0\_img.jpg\) Return to Graph with Gridlines Show Graphic](#)

# Graph of X Against Y Graphic Text Alternative (Appendix A)

---

[!\[\]\(35e4f762fc1cfea5610d92e2d225d5b4\_img.jpg\) Return to Graph of X Against Y Graphic](#)

The horizontal axis represents  $x$  ranging from 0.0 through 1.0 in increments of 0.2, while vertical axis represents  $y$  ranging from 0 through 120 in increments of 20.

The data is as follows:

The curve begins at (0.0, 5) and ends at (1.0, 18) showing decreasing trend after (0.31, 95).

Note: All data is approximate.

[!\[\]\(8d0f0e0fe25b320c33272c52aec1fbca\_img.jpg\) Return to Graph of X Against Y Graphic](#)

# Graph of X Against Y Graphic Text Alternative (Appendix A)

---

[!\[\]\(d263118e0bfd47dc6bc704167d936b83\_img.jpg\) Return to Graph of X Against Y Graphic](#)

The horizontal axis represents  $x$  ranging from 0.0 through 1.0 in increments of 0.2, while vertical axis represents  $y$  ranging from 0 through 120 in increments of 20.

The data is as follows:

The curve begins at (0.0, 5) and ends at (1.0, 18) showing decreasing trend after (0.31, 95).

Note: All data is approximate.

[!\[\]\(fa6f3af6bfa46c5d4a2d362681095beb\_img.jpg\) Return to Graph of X Against Y Graphic](#)



# Graph of X Against Y1 And Y2

## Graphic Text Alternative (Appendix A)

---

[!\[\]\(c8d96c8885d3000a912c2582004aed63\_img.jpg\) Return to Graph of X Against Y1 And Y2 Graphic](#)

The horizontal axis represents  $x$  ranging from 0 through 2500 in increments of 500, while vertical axis at left represents  $y_1$  ranging from negative 2 through 3 in increments of 1 and vertical axis at right represents  $y_2$  ranging from 46 through 60 in increments of 2.

The data is as follows:

$y_1$ :

The solid fluctuated curve begins at (0, 0) and ends at (2750, negative 0.2) showing decreasing trend after (400, 2.85), (990, 0.25), (1490, 1.5), and (2350, 1).

$y_2$ :

The dashed fluctuated curve begins at (0, 54) and ends at (2750, 57) showing decreasing trend after (751, 58), (1100, 60), (1350, 60), (2200, 58.5), and (2700, 58).

Note: All data is approximate.

[!\[\]\(faf942dc3e59ce8eb64b4ac481eca7e0\_img.jpg\) Return to Graph of X Against Y1 And Y2 Graphic](#)

# Graph of X Against Y1 And Y2

## Graphic Text Alternative (Appendix A)

---

[!\[\]\(529949c2c3dadbaa4e538e8c643454bc\_img.jpg\) Return to Graph of X Against Y1 And Y2 Graphic](#)

The horizontal axis represents  $x$  ranging from 0 through 2500 in increments of 500, while vertical axis at left represents  $y_1$  ranging from negative 2 through 3 in increments of 1 and vertical axis at right represents  $y_2$  ranging from 46 through 60 in increments of 2.

The data is as follows:

$y_1$ :

The solid fluctuated curve begins at (0, 0) and ends at (2750, negative 0.2) showing decreasing trend after (400, 2.85), (990, 0.25), (1490, 1.5), and (2350, 1).

$y_2$ :

The dashed fluctuated curve begins at (0, 54) and ends at (2750, 57) showing decreasing trend after (751, 58), (1100, 60), (1350, 60), (2200, 58.5), and (2700, 58).

Note: All data is approximate.

[!\[\]\(6059a5aa8b4ca7bb793408023d6c6e42\_img.jpg\) Return to Graph of X Against Y1 And Y2 Graphic](#)

# Figure A.1 Text Alternative (Appendix A)

---

[!\[\]\(1d3a1175dd4902218e694b9c098adb83\_img.jpg\) Return to Figure A.1](#)

The horizontal axis represents  $x$  ranging from 0.0 through 3.0 in increments of 0.5, while vertical axis represents  $f$  of  $x$ , ranging from negative 1.00 through 1.00 in increments of 0.25.

The data is as follows:

Sin ( $x$ ):

The curve begins at (0.0, 0.00) and ends at (3.4, 0.00) through (1.5, 1.00).

Sin ( $2x$ ):

The curve begins at (0.0, 0.00) and ends at (3.4, 0.00) through (0.75, 1.00), (1.5, 0.125), and (2.25, negative 1.00).

Sin ( $3x$ ):

The curve begins at (0.0, 0.00) and ends at (3.4, 0.00) through (0.5, 1.00), (1.6, negative 1.00), and (2.6, 1.00).

Note: All data is approximate.

[!\[\]\(4fe57c3593bf1b21d272ae7ac8dfaf77\_img.jpg\) Return to Figure A.1](#)

# Graph of Sin (X) Graphic Text Alternative (Appendix A)

---

[!\[\]\(21199eb166cc97331a0c54c649195dcc\_img.jpg\) Return to Graph of Sin \(X\) Graphic](#)

The horizontal axis represents  $\sin(x)$  ranging from 0.0 through 1.0 in increments of 0.2, while vertical axis represents  $\sin(2x)$  ranging from negative 1.00 through 1.00 in increments of 0.25.

The data is as follows:

The curve begins and ends at (0.0, 0.00) passing through (0.4, 0.75), (0.7, 1.00), (1.0, 0.00), (0.7, negative 1.00), and (0.4, negative 0.75).

Note: All data is approximate.

[!\[\]\(c694a3ff3b077d76910920a6a1593ab4\_img.jpg\) Return to Graph of Sin \(X\) Graphic](#)

# Graph of X Against Y Graphic Text Alternative (Appendix A)

---

[!\[\]\(8af806fb1314382d09bc5ec5b767526c\_img.jpg\) Return to Graph of X Against Y Graphic](#)

The horizontal axis represents  $x$  ranging from 0 through 100 in increments of 20, while vertical axis represents  $y$  ranging from 2 through 14 in increments of 2.

The data is as follows:

The curve begins at (0, 2) and ends at (100, 12.1) passing through error bars at (2, 0), (5, 6), (10, 9), (15, 11), (20, 13), (30, 13.5), (40, 13), (50, 13.1), (73, 12.5), and (100, 12.1).

Note: All data is approximate.

[!\[\]\(0aff635c4179ba9e710b00f4b01d3b20\_img.jpg\) Return to Graph of X Against Y Graphic](#)

# Bar Graph of X Graphic Text Alternative (Appendix A)

---

[!\[\]\(d0a1791f26d167e866e44ebbf83efebe\_img.jpg\) Return to Bar Graph of X Graphic](#)

The horizontal axis represents x ranging from 0.0 through 2.5 in increments of 0.5, while vertical axis represents frequency ranging from 0 through 80 in increments of 20.

The data is as follows:

0.1: 15, 0.2: 32, 0.3: 61, 0.4: 72, 0.5: 95, 0.6: 85, 0.7: 92, 0.8: 79, 1.0: 94, 1.1: 78, 1.2: 58, 1.3: 52, 1.4: 45, 1.5: 40, 1.6: 19.5, 1.7: 20.5, 1.8: 6, 2.0: 11, 2.1: 0, 2.2 blank, 2.3: blank, 2.4: 3, 2.5: blank, and 2.6: 3.

Note: All data is approximate.

[!\[\]\(5a132f13505a6571904d622757b7a8f0\_img.jpg\) Return to Bar Graph of X Graphic](#)

# A Box Plot Graphic Text Alternative (Appendix A)

---

[!\[\]\(35e4f762fc1cfea5610d92e2d225d5b4\_img.jpg\) Return to A Box Plot Graphic](#)

First box ranges from 62 to 84 with lower arm at 32 and three outliers at 20 while upper arm at 115 and one outlier at 130.

Second box ranges from 92 to 118 with lower arm at 58 and two outliers at 40 while upper arm at 155 and three outliers at 167.

Note: All data is approximate.

[!\[\]\(83f22ed94ec5517769dd76d702c6bfd8\_img.jpg\) Return to A Box Plot Graphic](#)

# Stem Plot Graphic Text Alternative (Appendix A)

---

 [Return to Stem Plot Graphic](#)

The horizontal axis ranges from 2 through 10 in increments of 2, while vertical axis ranges from 0.0 through 3.5 in increments of 0.5.

The data is as follows:

0.5: 1.0, 2: 1.6, 3: 2.25, 4: 2.7, 5: 3.0, 6: 3.35, 7: 3.5, 8: 3.6, 9: 3.75, and 10: 3.76.

Note: All data is approximate.

 [Return to Stem Plot Graphic](#)



# Graph of Frequency Graphic Text Alternative (Appendix A)

---

 [Return to Graph of Frequency Graphic](#)

The horizontal axis represents frequency (in 1 over second) ranging from 10 raised to power negative 2 through 10 raised to power 2, while vertical axis represents magnitude ranging from 10 raised to power negative 4 through 10 raised to power 0.

The data is as follows:

The curve begins at (10 raised to power negative 2, 10 raised to power 0) and ends at (10 raised to power 2, 10 raised to power negative 4) through (10 raised to power negative 1, 10 raised to power 0), and 10 raised to power 1, 10 raised to power negative 2).

Note: All data is approximate.

 [Return to Graph of Frequency Graphic](#)

# Graph of Frequency Graphic Text Alternative (Appendix A)

---

[!\[\]\(529949c2c3dadbaa4e538e8c643454bc\_img.jpg\) Return to Graph of Frequency Graphic](#)

The horizontal axis represents frequency (in 1 over second) ranging from 10 raised to power negative 2 through 10 raised to power 2, while vertical axis represents phase shift (in degrees) ranging from negative 175 through 0 in increments of 25.

The data is as follows:

The curve begins at (10 raised to power negative 2, 0) and ends at (10 raised to power 2, negative 176) through (10 raised to power negative 1, 1), (10 raised to power 0, negative 100), and (10 raised to power 1, negative 175).

Note: All data is approximate.

[!\[\]\(a870788d6ed9b8fd294b7654a8c8526b\_img.jpg\) Return to Graph of Frequency Graphic](#)

# Plot with Polar Coordinates Graphic Text Alternative (Appendix A)

---

 [Return to Plot with Polar Coordinates Graphic](#)

The values range from 0.0 through 1.5 in increments of 0.5. A curve begins at (0.0) at 180 degrees and ends at (1.0) at 0 degrees.

 [Return to Plot with Polar Coordinates Graphic](#)

# Graph of X Against Y Graphic Text Alternative (Appendix A)

---

[!\[\]\(21199eb166cc97331a0c54c649195dcc\_img.jpg\) Return to Graph of X Against Y Graphic](#)

The horizontal axis represents  $x$  ranging from 0 through 10 in increments of 2, while vertical axis represents  $y$  ranging from negative 0.2 through 0.6 in increments of 0.2.

The data is as follows:

The curve begins at (0, 0.0) and ends at (10, 0.0) through (1, 0.65), (2, 0.2), (4, negative 0.2), (6, 0.09), and (8, negative 0.01).

Note: All data is approximate.

[!\[\]\(c694a3ff3b077d76910920a6a1593ab4\_img.jpg\) Return to Graph of X Against Y Graphic](#)

# Set of 2 Graphs Graphic Text Alternative (Appendix A)

---

 [Return to Set of 2 Graphs Graphic](#)

Left panel:

The horizontal axis represents  $x$  ranging from 0.0 through 10.0 in increments of 2.5, while vertical axis represents  $y$  ranging from negative 0.2 through 0.6 in increments of 0.2.

The data is as follows:

The curve begins at (0, 0.0) and ends at (10, 0.0) through (2.5, 0.0), (5.0, 0.0), and (7.5, 0.0).

Right panel:

The horizontal axis represents  $x$  ranging from 0.0 through 10.0 in increments of 2.5, while vertical axis represents  $y$  ranging from negative 1.00 through 1.00 in increments of 0.25.

The data is as follows:

The curve begins at (0, 0.0) and ends at (10, 0.0) through (2.5, negative 0.25), (5.0, 0.0), and (7.5, 0.0).

Note: All data is approximate.

 [Return to Set of 2 Graphs Graphic](#)

# Set of 2 Graphs Graphic Text Alternative (Appendix A)

---

 [Return to Set of 2 Graphs Graphic](#)

Left panel:

The horizontal axis represents  $x$  ranging from 0.0 through 10.0 in increments of 5, while vertical axis represents  $y$  ranging from negative 0.2 through 0.6 in increments of 0.2.

The data is as follows:

The curve begins at (0, 0.0) and ends at (10, 0.0) through (2.5, 0.0), (5.0, 0.0), and (7.5, 0.0).

Right panel:

The horizontal axis represents  $x$  ranging from 0.0 through 10.0 in increments of 5, while vertical axis represents  $y$  ranging from negative 1.00 through 1.00 in increments of 0.25.

The data is as follows:

The curve begins at (0, 0.0) and ends at (10, 0.0) through (2.5, negative 0.25), (5.0, 0.0), and (7.5, 0.0).

Note: All data is approximate.

 [Return to Set of 2 Graphs Graphic](#)

# Set of 2 Graphs Graphic Text Alternative (Appendix A)

---

 [Return to Set of 2 Graphs Graphic](#)

Left panel:

The horizontal axis represents  $x$  ranging from 0 through 10 in increments of 2, while vertical axis represents  $y$  ranging from negative 0.2 through 0.6 in increments of 0.2.

The data is as follows:

The curve begins at  $(0, 0.0)$  and ends at  $(10, 0.0)$  through  $(2.5, 0.0)$ ,  $(5.0, 0.0)$ , and  $(7.5, 0.0)$ .

Right panel:

The horizontal axis represents  $x$  ranging from 0 through 10 in increments of 2, while vertical axis represents  $y$  ranging from negative 1.0 through 1.0 in increments of 0.5.

The data is as follows:

The curve begins at  $(0, 0.0)$  and ends at  $(10, 0.0)$  through  $(2.5, \text{negative } 0.25)$ ,  $(5.0, 0.0)$ , and  $(7.5, 0.0)$ .

Note: All data is approximate.

 [Return to Set of 2 Graphs Graphic](#)

# Contour Plot Graphic Text Alternative (Appendix A)

---

[!\[\]\(d263118e0bfd47dc6bc704167d936b83\_img.jpg\) Return to Contour Plot Graphic](#)

The horizontal axis represents  $x$  ranging from negative 2.0 through 2.0 in increments of 0.5, while vertical axis represents  $y$  ranging from negative 2.0 through 2.0 in increments of 0.5.

The data is as follows:

92000: It begins and ends at (negative 0.5, negative 0.25) through (0.75, negative 0.5).

88000: It begins and ends at (negative 1.0, 0.5) through (0.5, 0.6) and (1.0, negative 0.5).

85000: It begins and ends at (negative 0.5, 1.5) through (1.0, 0.25) and (0.0, negative 1.0).

80000: It begins and ends at (negative 1.2, 0.5) through (0.0, negative 1.4), and (negative 1.5, 0.5).

Note: All data is approximate.

[!\[\]\(f95dab70c751fda7d824b8b03650f7aa\_img.jpg\) Return to Contour Plot Graphic](#)



# Contour Plot Graphic Text Alternative (Appendix A)

---

[!\[\]\(c8d96c8885d3000a912c2582004aed63\_img.jpg\) Return to Contour Plot Graphic](#)

The horizontal axis represents  $x$  ranging from negative 2.0 through 2.0 in increments of 0.5, while vertical axis represents  $y$  ranging from negative 2.0 through 2.0 in increments of 0.5.

The data is as follows:

First: It begins and ends at (negative 0.5, negative 0.25) through (0.75, negative 0.5).

Second: It begins and ends at (negative 1.0, 0.5) through (0.5, 0.6) and (1.0, negative 0.5).

Third: It begins and ends at (negative 0.5, 1.5) through (1.0, 0.25) and (0.0, negative 1.0).

Fourth: It begins and ends at (negative 1.2, 0.5) through (0.0, negative 1.4), and (negative 1.5, 0.5).

Note: All data is approximate.

[!\[\]\(faf942dc3e59ce8eb64b4ac481eca7e0\_img.jpg\) Return to Contour Plot Graphic](#)

# Graph of X Against Y And Z Graphic Text Alternative (Appendix A)

---

 [Return to Graph of X Against Y And Z Graphic](#)

The x axis ranges from negative 2 through 2 in increments of 1, while y axis ranges from negative 2 through 2 in increments of 1, and z axis ranges from 0 through 100 in increments of 20.

The data is as follows:

The surface is visible between (10, negative 1.5), (0.1, 0), (1.5, 10), and (negative 2, 70).

Note: All data is approximate.

 [Return to Graph of X Against Y And Z Graphic](#)

# Graph of X Against Y And Z Graphic Text Alternative (Appendix A)

---

 [Return to Graph of X Against Y And Z Graphic](#)

The x axis ranges from negative 2 through 2 in increments of 1, while y axis ranges from negative 2 through 2 in increments of 1, and z axis ranges from 0 through 100 in increments of 20.

The data is as follows:

The mesh is visible between (7, negative 1.5), (0.1, 1.8), (1.5, 10), and (negative 2, 67).

Note: All data is approximate.

 [Return to Graph of X Against Y And Z Graphic](#)

# Graph of X Against Y And Z Graphic Text Alternative (Appendix A)

---

 [Return to Graph of X Against Y And Z Graphic](#)

The x axis ranges from negative 2 through 2 in increments of 1, while y axis ranges from negative 2 through 2 in increments of 1, and z axis ranges from 0 through 100 in increments of 20.

The data is as follows:

The surface is visible between (10, negative 1.5), (0.1, 0), (1.5, 10), and (negative 2, 70).

Note: All data is approximate.

 [Return to Graph of X Against Y And Z Graphic](#)

# Glossary

word

Glossary definition for word goes here.

# Remarks

<sup>1</sup> Python allows direct calculation of the *hyperbolic tangent* via the built-in function  $\tanh(x)$ , which is available in its math module.

<sup>1</sup> The best way to “read” this chapter is to “do” this chapter. Complete all the illustrations in the Spyder IDE as you work your way through the chapter.

<sup>2</sup> The IPython Console will automatically number the lines in sequence with the prompt  $\text{In}[n]$ : and produce the result with the matching  $\text{Out}[n]$ :. A blank line will be inserted after the result line. As needed, you can copy results from the Console and paste them into another document, for example, Microsoft Word or PowerPoint.

<sup>3</sup> Also known as Napier’s constant.

<sup>4</sup> Python uses the IEEE standard to represent floating point numbers with eight binary bytes. This provides an approximate range of  $10^{-308}$  to  $10^{308}$  and between 15 and 16 significant decimal digits.

<sup>5</sup> Using the Option Base 1 declaration, VBA will use an origin of 1 for array subscripts.

<sup>6</sup> A complete listing of the math module functions is available at <https://docs.python.org/3/library/math.html>.

<sup>7</sup> The functions of the math module do not support vectorization, whereas the corresponding functions in the NumPy module do.

<sup>8</sup> The SI-base units for the ohm is  $\text{kg}\cdot\text{m}^2\cdot\text{s}^{-3}\cdot\text{A}^{-2}$ , the henry is  $\text{kg}\cdot\text{m}^2\cdot\text{s}^{-2}\cdot\text{A}^{-2}$ , and the farad is  $\text{A}^2\text{s}^4\text{kg}^{-1}\text{m}^{-2}$ . Consequently,  $R/L$  has units  $1/\text{s}$  and  $LC$  has units  $\text{s}^2$ . The symbol A represents current in amperes.

<sup>1</sup> VBA stands for Visual Basic for Applications, the programming language that is available in different versions for the Microsoft Office products, commonly Excel, Word, PowerPoint, and Access.

<sup>2</sup> The Math module already has a factorial function that performs this computation.

<sup>3</sup> This illustrates an important principle. When you have a question or doubt about the way that a statement, type or function in Python works, set up a test to investigate it.

<sup>4</sup> Note: You can also use the numeral 1 for True and 0 for False. We prefer True and False because they are explicit.

<sup>5</sup> Visual Basic (VBA) actually has, in addition to the general *Do...Loop*, a *Do While...Loop*, and *Do...Loop Until* structure. These tend to confuse programmers, and many recommend sticking with the simpler *Do...Loop* with an *If...Exit Do* statement within the loop, equivalent to the *if...break* in Python. MATLAB has a while loop similar to that of Python.

<sup>6</sup> Python allows a hybrid operator for incrementing and decrementing a variable. Instead of  $i=i+1$ , we can write  $i+=1$ . Instead of  $y=y-1$ , the alternate is  $y-=1$ . You can also use the  $*$  and  $/$  operators in this fashion.

<sup>7</sup> This formula can be derived by applying Newton's method to  $f(x) = 1 - x^2$  to find  $f(x) = 0$ . Newton's method is introduced in Chap. 6.

<sup>8</sup> The pascal (Pa) is the SI unit for pressure and is defined as newton/m<sup>2</sup> (N/m<sup>2</sup>)

<sup>1</sup> A definition of the word *digit* is a finger (including the thumb) or toe.

<sup>2</sup> Colloquially, it is said that the last carry goes into the "bit bucket," a garbage can of sorts for binary digits.

<sup>3</sup> According to the IEEE standard, 1111111111 is used to represent "NaN" or "not a number." This might be the result of a numerical calculation leading to infinity. The 0000000000 exponent has special use as a "signed zero." Refer to [https://en.wikipedia.org/wiki/Double-precision\\_floating-point\\_format](https://en.wikipedia.org/wiki/Double-precision_floating-point_format) for more detail on this.

<sup>4</sup> Named after a British mathematician, Brook Taylor, who stated it in 1712.

<sup>5</sup> This is the Greek letter Xi,  $\xi$ , and not zeta,  $\zeta$ . It is pronounced "ksi" in Modern Greek, and is the 14<sup>th</sup> letter of the Greek alphabet.

<sup>1</sup> This function is modeled after a MATLAB function originally presented by Recktenwald (2000).

<sup>2</sup> In computer terms,  $2^{10}$  bytes is a kilobyte,  $2^{20}$  bytes is a megabyte,  $2^{30}$  bytes is a gigabyte, and so on.

<sup>3</sup> Partial pressure is a measure of the concentration of a particular species in mixture of gases. For a given volume (e.g., one cubic meter) of mixed gas, here in the atmosphere, the number of moles of  $\text{CO}_2$ ,  $n_{\text{CO}_2}$ , divided by the total number of moles,  $n_{\text{T}}$ , times the total pressure,  $P_{\text{T}}$ , gives the partial pressure; that is  $p_{\text{CO}_2} = (n_{\text{CO}_2} / n_{\text{T}}) P_{\text{T}}$ .

<sup>4</sup> These data are available in text file format at  
[ftp://aftp.cmdl.noaa.gov/products/trends/co2/co2\\_annmean\\_mlo.txt](ftp://aftp.cmdl.noaa.gov/products/trends/co2/co2_annmean_mlo.txt)

<sup>5</sup> In Part Four, we will learn how to “fit” such polynomials to data.

<sup>6</sup> The *critical depth* is that which produces *critical flow*. *Critical flow* is such that a wave on the surface of the channel moves at the same velocity as the flow velocity. This is characterized by a dimensionless quantity called the Froude number, given by  $Fr = V/\sqrt{gy}$ .  $Fr = 1$  for critical flow. For  $Fr > 1$ , channel flow is *sub-critical*, relatively calm and slow. For  $Fr < 1$ , it is *supercritical*, rapid and turbulent.

<sup>7</sup> The Fanning friction factor is commonly used by chemical engineers. Mechanical engineers usually use the *Moody friction factor*, which is four times the Fanning friction factor.

<sup>1</sup> This method was developed by J. H. Wegstein, **Communications of the ACM**, 1: 9–13, 1958.

<sup>2</sup> Richard Peirce Brent (born 1946, Melbourne) is an Australian mathematician and computer scientist.

<sup>3</sup> <https://pythonhosted.org/pypol/>

<sup>4</sup> For a *Newtonian fluid*, the shearing force per unit area is proportional to the negative of the velocity gradient with the proportionality given by the viscosity. Gases and liquids with molecular weight less than 5,000 are generally Newtonian. For polymeric liquids, solid suspensions, pastes, slurries, and other complex fluids, this proportionality does not hold, and they are termed *non-Newtonian*. See Bird et al. (2007).

<sup>5</sup> In laminar flow, the velocity of the fluid in the conduit varies smoothly from one position to another. In turbulent flow, the velocity at a point fluctuates chaotically. Generally, Newtonian flow with a Reynolds number less than about 2,100 is laminar. Above that, there is a rapid transition to turbulent flow.

<sup>6</sup> Note: The Pascal (Pa) is the SI unit for pressure and corresponds to  $\text{N/m}^2$ . Since a Newton force is equal to  $\text{kg}\cdot\text{m/s}^2$ , another unit description for viscosity is  $\text{kg}/(\text{m}\cdot\text{s})$ . A common English unit for viscosity is the poise or centipoise (cP). One cP is equivalent to  $0.001 \text{ Pa}\cdot\text{s}$ . Water has a viscosity of about one cP.

<sup>7</sup> The worldwide annual production of methanol is about 80 million metrictons. That’s enough methanol to fill 6,600 tanker trucks every day which, lined up bumper to bumper, would stretch for 81 km. Yes, every day!

<sup>8</sup> Impedance is the effective resistance of a circuit subjected to an alternating current.



<sup>2</sup> Air resistance, the spin of the ball, and the dimpled nature of the ball surface are important, but the model incorporating these effects is more complicated and will be considered later on in the book.

<sup>1</sup> Another curious relationship is based on the Fibonacci sequence (0,1,1,2,3,5,8,...) where each number in the sequence is the sum of the previous two numbers. The ratio of a number in the sequence to its predecessor approaches the golden ratio as the length of the sequence increases, reaching the golden ratio in the limit.

<sup>2</sup> There is a highly unusual and pathological case not considered here; that of  $f(x_1) = f(x_2)$ .

<sup>3</sup> It is worth mentioning that this is not quite as efficient as bisection where the values are about one part in 1,000 after ten iterations and one part in a million after 20.

<sup>4</sup> Appendix A provides more details on Matplotlib.

<sup>1</sup> In addition to special brackets, we will use case to distinguish between vectors (lowercase) and matrices (uppercase). Note that later in this text in a few contexts we use bold face fonts to represent vectors and matrices.

<sup>1</sup> If your Python code also imports the NumPy module first, you may not be able to reference the SciPy linalg module in your code because of the conflict with the NumPy linalg module. For this reason, the lu function is imported specifically using the from statement.

<sup>2</sup> It should be noted that, if  $A$  is not square, a factorization into a unitary/orthogonal matrix and an upper triangular matrix is provided by the  $QR$  algorithm. In the SciPy module and linalg submodule, the function is qr.

<sup>1</sup> As mentioned previously in Sec. 8.1, we use bold face fonts rather than braces and brackets to represent vectors and matrices in this chapter in the interest of making the mathematics more concise.

<sup>2</sup> Recall *Euler's identity*:  $e^{\pm ix} \equiv \cos x \pm i \sin x$ .

<sup>1</sup> You might have thought of a simple remedy to outliers—remove them! That is generally considered bad practice in treatment of experimental data, unless one has clear evidence that there was a blunder or failure of measurement.

<sup>1</sup> The sentiment expressed here is commonly referred to as *Occam's* (or *Ockham's*) *Razor*. This is a philosophical heuristic attributed to the 14th-century English friar and logician, William of Ockham (ca. 1285–1349). Although it has been paraphrased in many forms, a nice concise expression is “When you have two competing hypotheses that make exactly the same predictions, the simpler one is best.”

<sup>1</sup> Jean-Baptiste Joseph Fourier, 1768–1830. Apart from his advances in mathematics, Fourier is credited with the discovery of the greenhouse effect.

<sup>2</sup> When the time unit is seconds, the unit for the ordinary frequency is a cycle/s or *Hertz* (Hz).

<sup>3</sup> You may also see these referred to by the German possessive form, *Wölfer Sunspot Numbers*. Note that today, they are more commonly called *international sunspot numbers*.

<sup>1</sup> The value for 2020 is a projection from the US Census Bureau.

<sup>2</sup> Carl David Tolmé Runge (1856–1927) was a German mathematician, physicist, and spectroscopist.

<sup>1</sup> A curious function provided is the CloughTocher2DInterpolator. Truth in advertising, there is no connection to the coauthor of this text!

<sup>2</sup> A detailed derivation is presented in Appendix B for the sake of completeness.

<sup>3</sup> In English, we pronounce LOESS as “low-ess.” The method derives from the German noun, der Löß, a geological bluff in a river valley with a vertical cross section through which a deposit of fine clay or silt runs. The deposit in the Löß appears as a curved stratum, similar to a smoothed curve running through a set of data. The name also cleverly represents an acronym for “locally estimated scatterplot smoothing.”

<sup>1</sup> *Humps functions* are commonly used by numerical analysts to test algorithms for root location, optimization, and integration.

<sup>1</sup> The form  $dy/dx$  was devised by Leibnitz, whereas  $y'$  is attributed to Lagrange. Note that Newton used the so-called dot notation:  $\dot{y}$ . Today, the dot notation is usually used for time derivatives.

<sup>1</sup> Some computer languages represent the signum function as  $\text{sgn}(x)$ . As represented here, we use the nomenclature  $\text{sign}(x)$ . This function is available from Python’s NumPy module.

<sup>1</sup> There are other programming languages that provide for the definition of a *static* variable inside a subprogram. Such *static* variables remember their values from one subprogram execution to the next. Python does not provide for static variables, so we have to find a work-around. This is one. There are others that are more sophisticated.

<sup>1</sup> We incorporate radiation into this problem later in this chapter in Example 24.4.

<sup>1</sup> “Smoothing with Cubic Splines” by D.S.G. Pollock, [https://www.google.com/?gws\\_rd=ssl#q=smoothing+with+cubic+splines+pollock](https://www.google.com/?gws_rd=ssl#q=smoothing+with+cubic+splines+pollock)